

MXVM

1.8.1

Generated by Doxygen 1.16.1

1 MXVM – Virtual Machine, Compiler & Code Generator	1
1.1 Architecture Overview	1
1.2 Building	2
1.3 Running	2
1.3.1 <code>mx</code> Pascal Compiler Arguments	3
1.4 <code>mxvm-html</code> — Syntax Highlighter	3
1.4.1 Usage	3
1.4.1.1 Options	3
1.4.1.2 Auto-naming	4
1.4.2 Examples	4
1.4.3 CSS Customisation	4
1.5 MXVM Bytecode Language Reference	4
1.5.1 Program Structure	4
1.5.1.1 Data Section – Variable Types	5
1.5.1.2 Module Section	5
1.5.1.3 Code Section	5
1.5.2 Instruction Set	5
1.5.2.1 Arithmetic & Logic	5
1.5.2.2 Comparison & Branching	6
1.5.2.3 Memory & Pointers	6
1.5.2.4 Stack Operations	7
1.5.2.5 Control Flow & Calls	7
1.5.2.6 I/O	7
1.5.2.7 Type Conversion	7
1.5.2.8 Calling Convention – <code>invoke</code> / <code>return</code>	7
1.5.3 Object System	7
1.6 MXVM Pascal Frontend	8
1.6.1 Supported Language Features	8
1.6.1.1 Case Insensitivity	8
1.6.1.2 Program Structure	8
1.6.1.3 Uses Clause	8
1.6.1.4 Units (Separately Compiled Modules)	9
1.6.1.5 Data Types	11
1.6.1.6 Constants	11
1.6.1.7 Type Declarations	11
1.6.1.8 Variable Declarations	11
1.6.1.9 Operators	12
1.6.1.10 Control Flow	12
1.6.1.11 Procedures & Functions	12
1.6.1.12 Records	13
1.6.1.13 Enumerated Types	13
1.6.1.14 With Statement	14

1.6.1.15 Goto and Labels	14
1.6.1.16 In Operator (Set Membership)	14
1.6.1.17 Sets	14
1.6.1.18 File I/O	15
1.6.1.19 Doubled-Quote Strings	15
1.6.1.20 Arrays	15
1.6.1.21 Dynamic Arrays	16
1.6.1.22 Pointers	16
1.6.1.23 String Operations	16
1.6.1.24 Built-in Procedures & Functions	17
1.6.2 Limitations & Known Quirks	17
1.7 Runtime Module Reference	18
1.7.1 Module: io	18
1.7.2 Module: std	18
1.7.2.1 Math Functions	18
1.7.2.2 Memory Functions	19
1.7.2.3 Conversion & Classification	19
1.7.2.4 System	19
1.7.3 Module: string	19
1.7.4 Module: sdl	20
1.7.4.1 Core & Window	20
1.7.4.2 Renderer & Drawing	20
1.7.4.3 Textures & Surfaces	20
1.7.4.4 Events & Input	21
1.7.4.5 Timing	21
1.7.4.6 Audio	21
1.7.4.7 Text Rendering (SDL_ttf)	21
1.8 Code Generation & Optimisation	22
1.8.1 Native x86-64 Targets	22
1.8.2 Register Allocation	22
1.8.3 Peephole Optimiser	22
1.9 Examples	22
1.9.1 MXVM Bytecode – Hello World	22
1.9.2 MXVM Bytecode – Fibonacci	22
1.9.3 Pascal – Recursive Factorial	23
1.9.4 Pascal – Records & Pointers	23
1.9.5 Pascal – Dynamic Arrays	23
1.9.6 Pascal – Extended Features Demo	24
1.9.7 Pascal – Set Operations	25
1.9.8 Pascal – File I/O	25
1.9.9 Pascal – Packed Types	26
1.9.10 Pascal – Units (Separate Compilation)	27

1.9.11 Pascal – SDL2 Knight's Tour (Full Game)	27
1.9.12 Pascal – SDL2 Space Shooter (Full Game)	29
1.10 Project Layout	39
2 Directory Hierarchy	41
2.1 Directories	41
3 Namespace Index	45
3.1 Namespace List	45
4 Concept Index	47
4.1 Concepts	47
5 Hierarchical Index	49
5.1 Class Hierarchy	49
6 Class Index	53
6.1 Class List	53
7 File Index	59
7.1 File List	59
8 Directory Documentation	63
8.1 src/frontend Directory Reference	63
8.2 src/html_gen Directory Reference	64
8.3 include Directory Reference	65
8.4 src/frontend/include Directory Reference	65
8.5 src/scanner/include Directory Reference	66
8.6 src/vm/include Directory Reference	66
8.7 modules/io Directory Reference	67
8.8 modules Directory Reference	67
8.9 include/mxvm Directory Reference	68
8.10 src/scanner Directory Reference	69
8.11 src/scanner/include/scanner Directory Reference	69
8.12 modules/sdl Directory Reference	70
8.13 src Directory Reference	71
8.14 modules/std Directory Reference	72
8.15 modules/string Directory Reference	72
8.16 src/vm Directory Reference	73
9 Namespace Documentation	75
9.1 expr_parser Namespace Reference	75
9.1.1 Enumeration Type Documentation	75
9.1.1.1 ExprTokenType	75
9.1.2 Variable Documentation	76

9.1.2.1 vars	76
9.2 mx Namespace Reference	76
9.2.1 Enumeration Type Documentation	77
9.2.1.1 ArgType	77
9.2.2 Function Documentation	78
9.2.2.1 format_hex16()	78
9.2.2.2 format_hex32()	78
9.2.2.3 format_hex64()	78
9.2.2.4 format_hex8()	78
9.2.2.5 format_hex_no_prefix()	78
9.2.2.6 format_signal()	78
9.2.2.7 proc_args()	78
9.2.2.8 terminal_supports_color()	79
9.2.3 Variable Documentation	79
9.2.3.1 color_	79
9.3 mx::Color Namespace Reference	79
9.3.1 Detailed Description	80
9.3.2 Variable Documentation	80
9.3.2.1 BG_GREEN	80
9.3.2.2 BG_RED	80
9.3.2.3 BG_YELLOW	80
9.3.2.4 BLACK	80
9.3.2.5 BLUE	80
9.3.2.6 BOLD	80
9.3.2.7 BRIGHT_BLUE	80
9.3.2.8 BRIGHT_CYAN	80
9.3.2.9 BRIGHT_GREEN	80
9.3.2.10 BRIGHT_MAGENTA	80
9.3.2.11 BRIGHT_RED	81
9.3.2.12 BRIGHT_YELLOW	81
9.3.2.13 CYAN	81
9.3.2.14 DIM	81
9.3.2.15 GREEN	81
9.3.2.16 MAGENTA	81
9.3.2.17 RED	81
9.3.2.18 RESET	81
9.3.2.19 WHITE	81
9.3.2.20 YELLOW	81
9.4 mxvm Namespace Reference	81
9.4.1 Typedef Documentation	84
9.4.1.1 runtime_call	84
9.4.1.2 StackValue	84

9.4.2 Enumeration Type Documentation	84
9.4.2.1 Keywords	84
9.4.2.2 Mode	85
9.4.2.3 OperandType	85
9.4.2.4 OpKind	85
9.4.2.5 Platform	85
9.4.2.6 VArity	86
9.4.2.7 VarType	86
9.4.3 Function Documentation	86
9.4.3.1 buildInstructionSet()	86
9.4.3.2 buildKeywordSet()	86
9.4.3.3 darwin_prefix_calls()	87
9.4.3.4 except_assert()	87
9.4.3.5 has_semicolon()	87
9.4.3.6 is_label()	87
9.4.3.7 is_stdio_name()	87
9.4.3.8 isFunctionReturningOwnedPtr()	87
9.4.3.9 isIdLike()	88
9.4.3.10 isImmediate()	88
9.4.3.11 join_lines()	88
9.4.3.12 opt_core_lines()	88
9.4.3.13 opt_darwin_lines()	88
9.4.3.14 opt_linux_lines()	88
9.4.3.15 opt_x64_windows_lines()	88
9.4.3.16 releaseOwnedPointer()	89
9.4.3.17 stdio_index()	89
9.4.3.18 toStringFromVarType()	89
9.4.3.19 x64_opt_core_lines()	89
9.4.4 Variable Documentation	89
9.4.4.1 debug_mode	89
9.4.4.2 error_label_count [1/2]	89
9.4.4.3 error_label_count [2/2]	89
9.4.4.4 html_mode	89
9.4.4.5 instruct_mode	90
9.4.4.6 keywords	90
9.4.4.7 kInstructions	90
9.4.4.8 kKeywords	90
9.4.4.9 kOpSpecs	90
9.4.4.10 kTypes	90
9.4.4.11 x64_sp_mod16	90
9.4.4.12 xmm_offset	90
9.5 mxx Namespace Reference	90

9.5.1 Enumeration Type Documentation	91
9.5.1.1 Keywords	91
9.5.2 Function Documentation	92
9.5.2.1 ciEquals()	92
9.5.3 Variable Documentation	93
9.5.3.1 key_str	93
9.5.3.2 pascal_keywords	93
9.6 pascal Namespace Reference	93
9.6.1 Enumeration Type Documentation	96
9.6.1.1 VarType	96
9.6.2 Function Documentation	97
9.6.2.1 collect_used()	97
9.6.2.2 copyPropagation()	97
9.6.2.3 foldAndTest()	97
9.6.2.4 foldCmpTest()	97
9.6.2.5 foldMovArithMov()	97
9.6.2.6 foldOrTest()	98
9.6.2.7 invertJump()	98
9.6.2.8 isLabelDef()	98
9.6.2.9 isRegisterName()	98
9.6.2.10 locate_sections()	98
9.6.2.11 mxvmOpt()	98
9.6.2.12 nextNonEmpty()	98
9.6.2.13 parse1()	99
9.6.2.14 parse2()	99
9.6.2.15 parseJump()	99
9.6.2.16 peepholeOpt()	99
9.6.2.17 rtrim_comment()	99
9.6.2.18 sweep_unused_data()	99
9.6.2.19 toLower()	100
9.6.2.20 trim()	100
9.6.3 Variable Documentation	100
9.6.3.1 fileFunctions	100
9.6.3.2 sdlFunctions	100
9.6.3.3 stdFunctions	101
9.7 scan Namespace Reference	101
9.7.1 Typedef Documentation	101
9.7.1.1 StringType	101
9.7.1.2 TMap	101
9.7.1.3 TString	101
9.7.1.4 TToken	102
9.8 scan::buffer Namespace Reference	102

9.9 scan::token Namespace Reference	102
9.9.1 Enumeration Type Documentation	102
9.9.1.1 CharType	102
9.10 types Namespace Reference	102
9.10.1 Enumeration Type Documentation	103
9.10.1.1 CharType	103
9.10.1.2 KeywordType	104
9.10.1.3 OperatorType	104
9.10.1.4 TokenType	105
9.10.2 Function Documentation	106
9.10.2.1 lookUp()	106
9.10.2.2 lookUp_Keyword()	106
9.10.2.3 print_type_CharType()	106
9.10.2.4 print_type_TokenType()	107
9.10.3 Variable Documentation	107
9.10.3.1 kwStr	107
9.10.3.2 opName	107
9.10.3.3 opStrings	107
9.10.3.4 sstrCharType	107
9.10.3.5 strCharType	107
9.10.3.6 strTokenType	108
10 Concept Documentation	109
10.1 mx::StringType Concept Reference	109
10.1.1 Concept definition	109
10.1.2 Detailed Description	109
11 Class Documentation	111
11.1 pascal::AddressOfNode Class Reference	111
11.1.1 Detailed Description	112
11.1.2 Constructor & Destructor Documentation	112
11.1.2.1 AddressOfNode()	112
11.1.3 Member Function Documentation	112
11.1.3.1 accept()	112
11.1.3.2 toString()	112
11.1.4 Member Data Documentation	112
11.1.4.1 operand	112
11.2 mx::ArgException< String > Class Template Reference	112
11.2.1 Detailed Description	113
11.2.2 Constructor & Destructor Documentation	113
11.2.2.1 ArgException() [1/2]	113
11.2.2.2 ArgException() [2/2]	113
11.2.3 Member Function Documentation	113

11.2.3.1 text()	113
11.2.4 Member Data Documentation	113
11.2.4.1 value	113
11.3 Args Struct Reference	113
11.3.1 Detailed Description	114
11.3.2 Member Data Documentation	114
11.3.2.1 action	114
11.3.2.2 argv	114
11.3.2.3 include_path	114
11.3.2.4 Makefile	114
11.3.2.5 module_path	114
11.3.2.6 object_path	114
11.3.2.7 only_test	115
11.3.2.8 output_file	115
11.3.2.9 platform	115
11.3.2.10 platform_argc	115
11.3.2.11 platform_argv	115
11.3.2.12 source_file	115
11.3.2.13 target	115
11.3.2.14 toolchain	115
11.4 mx::Argument< String > Struct Template Reference	115
11.4.1 Detailed Description	116
11.4.2 Constructor & Destructor Documentation	116
11.4.2.1 Argument() [1/2]	116
11.4.2.2 Argument() [2/2]	116
11.4.3 Member Function Documentation	116
11.4.3.1 operator<=>()	116
11.4.3.2 operator=()	117
11.4.4 Member Data Documentation	117
11.4.4.1 arg_letter	117
11.4.4.2 arg_name	117
11.4.4.3 arg_type	117
11.4.4.4 arg_value	117
11.4.4.5 desc	117
11.5 mx::ArgumentData< String > Struct Template Reference	118
11.5.1 Detailed Description	118
11.5.2 Constructor & Destructor Documentation	118
11.5.2.1 ArgumentData() [1/3]	118
11.5.2.2 ArgumentData() [2/3]	118
11.5.2.3 ArgumentData() [3/3]	118
11.5.3 Member Function Documentation	119
11.5.3.1 operator=() [1/2]	119

11.5.3.2 operator=() [2/2]	119
11.5.4 Member Data Documentation	119
11.5.4.1 argc	119
11.5.4.2 args	119
11.6 mx::Arguments Struct Reference	119
11.6.1 Detailed Description	119
11.6.2 Member Data Documentation	120
11.6.2.1 fullscreen	120
11.6.2.2 height	120
11.6.2.3 path	120
11.6.2.4 width	120
11.7 mx::Argz<String> Class Template Reference	120
11.7.1 Detailed Description	121
11.7.2 Constructor & Destructor Documentation	121
11.7.2.1 Argz() [1/4]	121
11.7.2.2 Argz() [2/4]	121
11.7.2.3 Argz() [3/4]	122
11.7.2.4 Argz() [4/4]	122
11.7.3 Member Function Documentation	122
11.7.3.1 addOptionDouble()	122
11.7.3.2 addOptionDoubleValue()	122
11.7.3.3 addOptionSingle()	123
11.7.3.4 addOptionSingleValue()	123
11.7.3.5 help()	123
11.7.3.6 initArgs()	124
11.7.3.7 lookUpCode()	124
11.7.3.8 operator=() [1/2]	125
11.7.3.9 operator=() [2/2]	125
11.7.3.10 proc()	125
11.7.3.11 reset()	125
11.7.4 Member Data Documentation	125
11.7.4.1 arg_data	125
11.7.4.2 arg_info	126
11.7.4.3 cindex	126
11.7.4.4 index	126
11.8 pascal::ArrayAccessNode Class Reference	126
11.8.1 Detailed Description	127
11.8.2 Constructor & Destructor Documentation	127
11.8.2.1 ArrayAccessNode()	127
11.8.3 Member Function Documentation	127
11.8.3.1 accept()	127
11.8.3.2 toString()	127

11.8.4 Member Data Documentation	127
11.8.4.1 base	127
11.8.4.2 index	128
11.9 pascal::ArrayAssignmentNode Class Reference	128
11.9.1 Detailed Description	129
11.9.2 Constructor & Destructor Documentation	129
11.9.2.1 ArrayAssignmentNode()	129
11.9.3 Member Function Documentation	129
11.9.3.1 accept()	129
11.9.3.2 toString()	129
11.9.4 Member Data Documentation	129
11.9.4.1 arrayName	129
11.9.4.2 index	129
11.9.4.3 value	129
11.10 pascal::ArrayDeclarationNode Class Reference	130
11.10.1 Detailed Description	131
11.10.2 Constructor & Destructor Documentation	131
11.10.2.1 ArrayDeclarationNode()	131
11.10.3 Member Function Documentation	131
11.10.3.1 accept()	131
11.10.3.2 toString()	131
11.10.4 Member Data Documentation	131
11.10.4.1 arrayType	131
11.10.4.2 initializers	131
11.10.4.3 name	131
11.11 pascal::ArrayInfo Struct Reference	132
11.11.1 Detailed Description	132
11.11.2 Constructor & Destructor Documentation	132
11.11.2.1 ArrayInfo() [1/3]	132
11.11.2.2 ArrayInfo() [2/3]	132
11.11.2.3 ArrayInfo() [3/3]	132
11.11.3 Member Function Documentation	133
11.11.3.1 operator=() [1/2]	133
11.11.3.2 operator=() [2/2]	133
11.11.4 Member Data Documentation	133
11.11.4.1 elementArray	133
11.11.4.2 elementIsArray	133
11.11.4.3 elementSize	133
11.11.4.4 elementType	133
11.11.4.5 isDynamic	133
11.11.4.6 lowerBound	134
11.11.4.7 size	134

11.11.4.8 upperBound	134
11.12 pascal::ArrayTypeDeclarationNode Class Reference	134
11.12.1 Detailed Description	135
11.12.2 Constructor & Destructor Documentation	135
11.12.2.1 ArrayTypeDeclarationNode()	135
11.12.3 Member Function Documentation	135
11.12.3.1 accept()	135
11.12.3.2 toString()	135
11.12.4 Member Data Documentation	135
11.12.4.1 arrayType	135
11.12.4.2 name	136
11.13 pascal::ArrayTypeNode Class Reference	136
11.13.1 Detailed Description	137
11.13.2 Constructor & Destructor Documentation	137
11.13.2.1 ArrayTypeNode()	137
11.13.3 Member Function Documentation	137
11.13.3.1 accept()	137
11.13.3.2 toString()	137
11.13.4 Member Data Documentation	137
11.13.4.1 elementType	137
11.13.4.2 lowerBound	137
11.13.4.3 upperBound	137
11.14 pascal::AssignmentNode Class Reference	138
11.14.1 Detailed Description	138
11.14.2 Constructor & Destructor Documentation	139
11.14.2.1 AssignmentNode()	139
11.14.3 Member Function Documentation	139
11.14.3.1 accept()	139
11.14.3.2 toString()	139
11.14.4 Member Data Documentation	139
11.14.4.1 expression	139
11.14.4.2 variable	139
11.15 mxvm::ASTNode Class Reference	139
11.15.1 Detailed Description	140
11.15.2 Constructor & Destructor Documentation	140
11.15.2.1 ~ASTNode()	140
11.15.3 Member Function Documentation	140
11.15.3.1 accept()	140
11.15.3.2 toString()	141
11.16 pascal::ASTNode Class Reference	141
11.16.1 Detailed Description	143
11.16.2 Constructor & Destructor Documentation	143

11.16.2.1 ~ASTNode()	143
11.16.3 Member Function Documentation	143
11.16.3.1 accept()	143
11.16.3.2 getLineNumber()	143
11.16.3.3 print()	143
11.16.3.4 setLineNumber()	144
11.16.3.5 toString()	144
11.16.4 Member Data Documentation	144
11.16.4.1 line_number	144
11.17 mxvm::ASTVisitor Class Reference	144
11.17.1 Detailed Description	145
11.17.2 Constructor & Destructor Documentation	145
11.17.2.1 ~ASTVisitor()	145
11.17.3 Member Function Documentation	145
11.17.3.1 visit() [1/9]	145
11.17.3.2 visit() [2/9]	145
11.17.3.3 visit() [3/9]	145
11.17.3.4 visit() [4/9]	145
11.17.3.5 visit() [5/9]	145
11.17.3.6 visit() [6/9]	145
11.17.3.7 visit() [7/9]	145
11.17.3.8 visit() [8/9]	145
11.17.3.9 visit() [9/9]	146
11.18 pascal::ASTVisitor Class Reference	146
11.18.1 Detailed Description	147
11.18.2 Constructor & Destructor Documentation	147
11.18.2.1 ~ASTVisitor()	147
11.18.3 Member Function Documentation	147
11.18.3.1 visit() [1/48]	147
11.18.3.2 visit() [2/48]	147
11.18.3.3 visit() [3/48]	147
11.18.3.4 visit() [4/48]	148
11.18.3.5 visit() [5/48]	148
11.18.3.6 visit() [6/48]	148
11.18.3.7 visit() [7/48]	148
11.18.3.8 visit() [8/48]	148
11.18.3.9 visit() [9/48]	148
11.18.3.10 visit() [10/48]	148
11.18.3.11 visit() [11/48]	148
11.18.3.12 visit() [12/48]	148
11.18.3.13 visit() [13/48]	148
11.18.3.14 visit() [14/48]	149

11.18.3.15 visit() [15/48]	149
11.18.3.16 visit() [16/48]	149
11.18.3.17 visit() [17/48]	149
11.18.3.18 visit() [18/48]	149
11.18.3.19 visit() [19/48]	149
11.18.3.20 visit() [20/48]	149
11.18.3.21 visit() [21/48]	149
11.18.3.22 visit() [22/48]	149
11.18.3.23 visit() [23/48]	149
11.18.3.24 visit() [24/48]	150
11.18.3.25 visit() [25/48]	150
11.18.3.26 visit() [26/48]	150
11.18.3.27 visit() [27/48]	150
11.18.3.28 visit() [28/48]	150
11.18.3.29 visit() [29/48]	150
11.18.3.30 visit() [30/48]	150
11.18.3.31 visit() [31/48]	150
11.18.3.32 visit() [32/48]	150
11.18.3.33 visit() [33/48]	150
11.18.3.34 visit() [34/48]	151
11.18.3.35 visit() [35/48]	151
11.18.3.36 visit() [36/48]	151
11.18.3.37 visit() [37/48]	151
11.18.3.38 visit() [38/48]	151
11.18.3.39 visit() [39/48]	151
11.18.3.40 visit() [40/48]	151
11.18.3.41 visit() [41/48]	152
11.18.3.42 visit() [42/48]	152
11.18.3.43 visit() [43/48]	152
11.18.3.44 visit() [44/48]	152
11.18.3.45 visit() [45/48]	152
11.18.3.46 visit() [46/48]	152
11.18.3.47 visit() [47/48]	152
11.18.3.48 visit() [48/48]	152
11.19 mxvm::Base Class Reference	152
11.19.1 Detailed Description	154
11.19.2 Constructor & Destructor Documentation	154
11.19.2.1 Base() [1/2]	154
11.19.2.2 Base() [2/2]	154
11.19.2.3 ~Base()	154
11.19.3 Member Function Documentation	154
11.19.3.1 add_allocated()	154

11.19.3.2 add_extern()	155
11.19.3.3 add_filename()	155
11.19.3.4 add_global()	155
11.19.3.5 add_instruction()	156
11.19.3.6 add_label()	156
11.19.3.7 add_object()	156
11.19.3.8 add_runtime_extern()	156
11.19.3.9 add_variable()	157
11.19.3.10 operator=()	157
11.19.3.11 setMainBase()	157
11.19.4 Member Data Documentation	157
11.19.4.1 allocated	157
11.19.4.2 assembly_code	158
11.19.4.3 base	158
11.19.4.4 external	158
11.19.4.5 external_functions	158
11.19.4.6 filenames	158
11.19.4.7 inc	158
11.19.4.8 labels	158
11.19.4.9 name	159
11.19.4.10 object_map	159
11.19.4.11 root_name	159
11.19.4.12 vars	159
11.20 pascal::BinaryOpNode Class Reference	159
11.20.1 Detailed Description	161
11.20.2 Member Enumeration Documentation	161
11.20.2.1 OpType	161
11.20.3 Constructor & Destructor Documentation	161
11.20.3.1 BinaryOpNode() [1/2]	161
11.20.3.2 BinaryOpNode() [2/2]	161
11.20.4 Member Function Documentation	162
11.20.4.1 accept()	162
11.20.4.2 opToString()	162
11.20.4.3 toString()	162
11.20.5 Member Data Documentation	162
11.20.5.1 left	162
11.20.5.2 operator_	162
11.20.5.3 right	162
11.21 pascal::BlockNode Class Reference	162
11.21.1 Detailed Description	163
11.21.2 Constructor & Destructor Documentation	163
11.21.2.1 BlockNode()	163

11.21.3 Member Function Documentation	164
11.21.3.1 accept()	164
11.21.3.2 toString()	164
11.21.4 Member Data Documentation	164
11.21.4.1 compoundStatement	164
11.21.4.2 declarations	164
11.22 pascal::BooleanNode Class Reference	164
11.22.1 Detailed Description	165
11.22.2 Constructor & Destructor Documentation	165
11.22.2.1 BooleanNode()	165
11.22.3 Member Function Documentation	166
11.22.3.1 accept()	166
11.22.3.2 toString()	166
11.22.4 Member Data Documentation	166
11.22.4.1 value	166
11.23 pascal::BreakNode Class Reference	166
11.23.1 Detailed Description	167
11.23.2 Constructor & Destructor Documentation	167
11.23.2.1 BreakNode() [1/2]	167
11.23.2.2 BreakNode() [2/2]	167
11.23.3 Member Function Documentation	167
11.23.3.1 accept()	167
11.23.3.2 toString()	167
11.23.4 Member Data Documentation	168
11.23.4.1 expr	168
11.24 pascal::BuiltinFunctionHandler Class Reference	168
11.24.1 Detailed Description	168
11.24.2 Constructor & Destructor Documentation	169
11.24.2.1 ~BuiltinFunctionHandler()	169
11.24.3 Member Function Documentation	169
11.24.3.1 canHandle()	169
11.24.3.2 generate()	169
11.24.3.3 generateWithResult()	169
11.24.3.4 getReturnType()	169
11.25 pascal::BuiltinFunctionRegistry Class Reference	169
11.25.1 Detailed Description	170
11.25.2 Member Function Documentation	170
11.25.2.1 findHandler()	170
11.25.2.2 registerHandler()	170
11.25.3 Member Data Documentation	170
11.25.3.1 handlers	170
11.26 pascal::CaseStmtNode::CaseBranch Struct Reference	170

11.26.1 Detailed Description	171
11.26.2 Constructor & Destructor Documentation	171
11.26.2.1 CaseBranch()	171
11.26.3 Member Data Documentation	171
11.26.3.1 statement	171
11.26.3.2 values	171
11.27 pascal::CaseStmtNode Class Reference	171
11.27.1 Detailed Description	172
11.27.2 Constructor & Destructor Documentation	172
11.27.2.1 CaseStmtNode()	172
11.27.3 Member Function Documentation	172
11.27.3.1 accept()	172
11.27.3.2 toString()	173
11.27.4 Member Data Documentation	173
11.27.4.1 branches	173
11.27.4.2 elseStatement	173
11.27.4.3 expression	173
11.28 pascal::CodeGenVisitor Class Reference	173
11.28.1 Detailed Description	179
11.28.2 Constructor & Destructor Documentation	179
11.28.2.1 CodeGenVisitor()	179
11.28.2.2 ~CodeGenVisitor()	179
11.28.3 Member Function Documentation	179
11.28.3.1 addTempPtr()	179
11.28.3.2 allocateRecordFieldArrays()	180
11.28.3.3 allocFloatReg()	180
11.28.3.4 allocPtrReg()	180
11.28.3.5 allocReg()	180
11.28.3.6 allocTempPtr()	180
11.28.3.7 buildArrayInfoFromNode()	180
11.28.3.8 coerceToIntImmediate()	180
11.28.3.9 convertCharLiteral()	181
11.28.3.10 emit()	181
11.28.3.11 emit1()	181
11.28.3.12 emit2()	181
11.28.3.13 emit3()	181
11.28.3.14 emit4()	181
11.28.3.15 emit_invoke()	182
11.28.3.16 emitArrayBoundsCheck()	182
11.28.3.17 emitDynArrayBoundsCheck()	182
11.28.3.18 emitFree()	183
11.28.3.19 emitLabel()	183

11.28.3.20 emptyString()	183
11.28.3.21 endsWithColon()	183
11.28.3.22 ensureFloatConstSymbol()	183
11.28.3.23 ensurePtrBase()	183
11.28.3.24 escapeStringForMxvm()	184
11.28.3.25 eval()	184
11.28.3.26 evaluateConstantExpression()	184
11.28.3.27 expandFloatRegisters()	184
11.28.3.28 findMangledArrayName()	184
11.28.3.29 findMangledFuncName()	184
11.28.3.30 findMangledName()	184
11.28.3.31 foldNumeric()	185
11.28.3.32 freeFloatReg()	185
11.28.3.33 freePtrReg()	185
11.28.3.34 freeReg()	185
11.28.3.35 freeTempPtr()	185
11.28.3.36 funcLabel()	185
11.28.3.37 generate()	186
11.28.3.38 generateRealConstantName()	186
11.28.3.39 getArrayElementSize()	186
11.28.3.40 getArrayInfoForArrayAccess()	186
11.28.3.41 getArrayNameFromBase()	186
11.28.3.42 getCurrentEndLabel()	187
11.28.3.43 getCurrentScopeName()	187
11.28.3.44 getExpressionType()	187
11.28.3.45 getPointerBaseTypeName()	187
11.28.3.46 getPointerDerefType()	187
11.28.3.47 getPointerElementSize()	187
11.28.3.48 getRecordFieldOffsetAndSize()	187
11.28.3.49 getRecordTypeSize()	188
11.28.3.50 getTypeFromString()	188
11.28.3.51 getTypeSizeByName()	188
11.28.3.52 getTypeString() [1/2]	188
11.28.3.53 getTypeString() [2/2]	188
11.28.3.54 getVarRecordTypeName()	188
11.28.3.55 getVarRecordTypeNameFromExpr()	188
11.28.3.56 getVarType()	188
11.28.3.57 initializeBuiltins()	189
11.28.3.58 initializeFloatRegisters()	189
11.28.3.59 internString()	189
11.28.3.60 isArrayTypeName()	189
11.28.3.61 isFloatLiteral()	189

11.28.3.62 isFloatReg()	189
11.28.3.63 isImportedUnit()	189
11.28.3.64 isIntegerLiteral()	190
11.28.3.65 isIntLiteral()	190
11.28.3.66 isParmReg()	190
11.28.3.67 isPtrLike()	190
11.28.3.68 isPtrReg()	190
11.28.3.69 isRealConstSymbol()	190
11.28.3.70 isRealNumber()	190
11.28.3.71 isRecordTypeName()	191
11.28.3.72 isReg()	191
11.28.3.73 isRegisterSlot()	191
11.28.3.74 isStringVar()	191
11.28.3.75 isTempPtr()	191
11.28.3.76 isTempVar()	191
11.28.3.77 lc()	191
11.28.3.78 mangleVariableName()	192
11.28.3.79 mangleWithScope()	192
11.28.3.80 markAllocatedPtr()	192
11.28.3.81 newLabel()	192
11.28.3.82 newSlotFor()	192
11.28.3.83 newTemp()	192
11.28.3.84 popValue()	192
11.28.3.85 pushCmpResult()	192
11.28.3.86 pushFloatCmpResult()	193
11.28.3.87 pushLogicalAnd()	193
11.28.3.88 pushLogicalOr()	193
11.28.3.89 pushTri()	193
11.28.3.90 pushValue()	193
11.28.3.91 recordLocation()	193
11.28.3.92 registerExternalFunc()	194
11.28.3.93 registerImportedConst()	194
11.28.3.94 registerImportedVar()	194
11.28.3.95 releaseTempPtr()	194
11.28.3.96 resolveTypeName()	194
11.28.3.97 setSlotType()	195
11.28.3.98 setVarType()	195
11.28.3.99 slotVar()	195
11.28.3.100 storageSymbolFor()	195
11.28.3.101 tryGetConstNumeric()	195
11.28.3.102 updateDataSectionInitialValue()	195
11.28.3.103 visit() [1/48]	196

11.28.3.104 visit()	[2 / 48]	196
11.28.3.105 visit()	[3 / 48]	196
11.28.3.106 visit()	[4 / 48]	196
11.28.3.107 visit()	[5 / 48]	196
11.28.3.108 visit()	[6 / 48]	196
11.28.3.109 visit()	[7 / 48]	197
11.28.3.110 visit()	[8 / 48]	197
11.28.3.111 visit()	[9 / 48]	197
11.28.3.112 visit()	[10 / 48]	197
11.28.3.113 visit()	[11 / 48]	197
11.28.3.114 visit()	[12 / 48]	198
11.28.3.115 visit()	[13 / 48]	198
11.28.3.116 visit()	[14 / 48]	198
11.28.3.117 visit()	[15 / 48]	198
11.28.3.118 visit()	[16 / 48]	198
11.28.3.119 visit()	[17 / 48]	198
11.28.3.120 visit()	[18 / 48]	199
11.28.3.121 visit()	[19 / 48]	199
11.28.3.122 visit()	[20 / 48]	199
11.28.3.123 visit()	[21 / 48]	199
11.28.3.124 visit()	[22 / 48]	199
11.28.3.125 visit()	[23 / 48]	199
11.28.3.126 visit()	[24 / 48]	200
11.28.3.127 visit()	[25 / 48]	200
11.28.3.128 visit()	[26 / 48]	200
11.28.3.129 visit()	[27 / 48]	200
11.28.3.130 visit()	[28 / 48]	200
11.28.3.131 visit()	[29 / 48]	200
11.28.3.132 visit()	[30 / 48]	201
11.28.3.133 visit()	[31 / 48]	201
11.28.3.134 visit()	[32 / 48]	201
11.28.3.135 visit()	[33 / 48]	201
11.28.3.136 visit()	[34 / 48]	201
11.28.3.137 visit()	[35 / 48]	201
11.28.3.138 visit()	[36 / 48]	202
11.28.3.139 visit()	[37 / 48]	202
11.28.3.140 visit()	[38 / 48]	202
11.28.3.141 visit()	[39 / 48]	202
11.28.3.142 visit()	[40 / 48]	202
11.28.3.143 visit()	[41 / 48]	202
11.28.3.144 visit()	[42 / 48]	202
11.28.3.145 visit()	[43 / 48]	202

11.28.3.146 visit() [44/48]	203
11.28.3.147 visit() [45/48]	203
11.28.3.148 visit() [46/48]	203
11.28.3.149 visit() [47/48]	203
11.28.3.150 visit() [48/48]	203
11.28.3.151 writeTo()	204
11.28.4 Friends And Related Symbol Documentation	204
11.28.4.1 FileFunctionHandler	204
11.28.4.2 IOFunctionHandler	204
11.28.4.3 MathFunctionHandler	204
11.28.4.4 SDLFunctionHandler	204
11.28.4.5 StdFunctionHandler	204
11.28.4.6 StringFunctionHandler	204
11.28.5 Member Data Documentation	205
11.28.5.1 allocatedPtrs	205
11.28.5.2 allTempPtrs	205
11.28.5.3 arrayInfo	205
11.28.5.4 bufferStringVars	205
11.28.5.5 builtinRegistry	205
11.28.5.6 compileTimeConstants	205
11.28.5.7 constInitialValues	205
11.28.5.8 currentFuncLocalSlots	205
11.28.5.9 currentFunctionName	206
11.28.5.10 currentFunctionReturnSlot	206
11.28.5.11 currentParamLocations	206
11.28.5.12 currentParamTypes	206
11.28.5.13 declaredFuncs	206
11.28.5.14 declaredProcs	206
11.28.5.15 deferredFuncs	206
11.28.5.16 deferredProcs	206
11.28.5.17 dynArrayLenSlot	206
11.28.5.18 enumConstants	207
11.28.5.19 enumTypes	207
11.28.5.20 escapedTempPtrs	207
11.28.5.21 evalStack	207
11.28.5.22 externalFuncs	207
11.28.5.23 fileVarNames	207
11.28.5.24 fileVars	207
11.28.5.25 floatRegInUse	207
11.28.5.26 floatRegisters	207
11.28.5.27 funcSignatures	208
11.28.5.28 functionScopedArrays	208

11.28.5.29 functionSetReturn	208
11.28.5.30 generatingDeferredCode	208
11.28.5.31 globalArrays	208
11.28.5.32 gotoLabels	208
11.28.5.33 importedArrayInfo	208
11.28.5.34 importedUnitNames	208
11.28.5.35 importedVarTypes	208
11.28.5.36 instructions	209
11.28.5.37 isUnit	209
11.28.5.38 labelCounter	209
11.28.5.39 loopContinueLabels	209
11.28.5.40 loopEndLabels	209
11.28.5.41 name	209
11.28.5.42 needsEmptyString	209
11.28.5.43 nextFloatRegIndex	209
11.28.5.44 nextSlot	209
11.28.5.45 nextSpillReg	210
11.28.5.46 nextTemp	210
11.28.5.47 objectDeps	210
11.28.5.48 pointerBaseType	210
11.28.5.49 prolog	210
11.28.5.50 ptrRegInUse	210
11.28.5.51 ptrRegisters	210
11.28.5.52 realConstantCounter	210
11.28.5.53 realConstants	210
11.28.5.54 recordsToFreeInScope	211
11.28.5.55 recordTypes	211
11.28.5.56 regInUse	211
11.28.5.57 registers	211
11.28.5.58 scopeHierarchy	211
11.28.5.59 scopeStack	211
11.28.5.60 setVars	211
11.28.5.61 slotToName	211
11.28.5.62 slotToType	211
11.28.5.63 stringLiterals	212
11.28.5.64 tempPtrByScope	212
11.28.5.65 tempPtrInUse	212
11.28.5.66 typeAliases	212
11.28.5.67 usedModules	212
11.28.5.68 usedRealConstants	212
11.28.5.69 usedStrings	212
11.28.5.70 valueLocations	212

11.28.5.71 varRecordType	212
11.28.5.72 varSlot	213
11.28.5.73 varTypes	213
11.28.5.74 withFieldScopes	213
11.29 mxvm::CommentNode Class Reference	213
11.29.1 Detailed Description	214
11.29.2 Constructor & Destructor Documentation	214
11.29.2.1 CommentNode()	214
11.29.3 Member Function Documentation	214
11.29.3.1 accept()	214
11.29.3.2 toString()	214
11.29.4 Member Data Documentation	214
11.29.4.1 text	214
11.30 pascal::CompoundStmtNode Class Reference	214
11.30.1 Detailed Description	215
11.30.2 Constructor & Destructor Documentation	215
11.30.2.1 CompoundStmtNode()	215
11.30.3 Member Function Documentation	216
11.30.3.1 accept()	216
11.30.3.2 toString()	216
11.30.4 Member Data Documentation	216
11.30.4.1 statements	216
11.31 pascal::ConstDeclNode::ConstAssignment Struct Reference	216
11.31.1 Detailed Description	216
11.31.2 Constructor & Destructor Documentation	216
11.31.2.1 ConstAssignment()	216
11.31.3 Member Data Documentation	217
11.31.3.1 identifier	217
11.31.3.2 value	217
11.32 pascal::ConstDeclNode Class Reference	217
11.32.1 Detailed Description	218
11.32.2 Constructor & Destructor Documentation	218
11.32.2.1 ConstDeclNode()	218
11.32.3 Member Function Documentation	218
11.32.3.1 accept()	218
11.32.3.2 toString()	218
11.32.4 Member Data Documentation	218
11.32.4.1 assignments	218
11.33 pascal::ContinueNode Class Reference	219
11.33.1 Detailed Description	219
11.33.2 Constructor & Destructor Documentation	219
11.33.2.1 ContinueNode() [1/2]	219

11.33.2.2 ContinueNode() [2/2]	220
11.33.3 Member Function Documentation	220
11.33.3.1 accept()	220
11.33.3.2 toString()	220
11.33.4 Member Data Documentation	220
11.33.4.1 expr	220
11.34 pascal::EmptyStmtNode Class Reference	220
11.34.1 Detailed Description	221
11.34.2 Constructor & Destructor Documentation	221
11.34.2.1 EmptyStmtNode()	221
11.34.3 Member Function Documentation	221
11.34.3.1 accept()	221
11.34.3.2 toString()	221
11.35 pascal::EnumTypeDeclNode Class Reference	221
11.35.1 Detailed Description	222
11.35.2 Constructor & Destructor Documentation	222
11.35.2.1 EnumTypeDeclNode()	222
11.35.3 Member Function Documentation	223
11.35.3.1 accept()	223
11.35.3.2 toString()	223
11.35.4 Member Data Documentation	223
11.35.4.1 typeName	223
11.35.4.2 values	223
11.36 mx::Exception Class Reference	223
11.36.1 Detailed Description	224
11.36.2 Constructor & Destructor Documentation	224
11.36.2.1 Exception() [1/2]	224
11.36.2.2 Exception() [2/2]	224
11.36.3 Member Function Documentation	224
11.36.3.1 error()	224
11.37 pascal::ExitNode Class Reference	225
11.37.1 Detailed Description	226
11.37.2 Constructor & Destructor Documentation	226
11.37.2.1 ExitNode() [1/2]	226
11.37.2.2 ExitNode() [2/2]	226
11.37.3 Member Function Documentation	226
11.37.3.1 accept()	226
11.37.3.2 toString()	226
11.37.4 Member Data Documentation	226
11.37.4.1 expr	226
11.38 mxvm::ExpressionNode Class Reference	226
11.38.1 Detailed Description	227

11.38.2 Constructor & Destructor Documentation	227
11.38.2.1 ExpressionNode()	227
11.38.3 Member Function Documentation	227
11.38.3.1 accept()	227
11.38.3.2 toString()	228
11.38.4 Member Data Documentation	228
11.38.4.1 isNumeric	228
11.38.4.2 value	228
11.39 expr_parser::ExprLexer Class Reference	228
11.39.1 Detailed Description	229
11.39.2 Constructor & Destructor Documentation	229
11.39.2.1 ExprLexer()	229
11.39.3 Member Function Documentation	229
11.39.3.1 consume()	229
11.39.3.2 next()	229
11.39.3.3 peek()	229
11.39.4 Member Data Documentation	229
11.39.4.1 current	229
11.39.4.2 pos	230
11.39.4.3 scanner	230
11.39.4.4 variable	230
11.40 expr_parser::ExprParser Class Reference	230
11.40.1 Detailed Description	231
11.40.2 Constructor & Destructor Documentation	231
11.40.2.1 ExprParser()	231
11.40.3 Member Function Documentation	231
11.40.3.1 parse()	231
11.40.3.2 parseAdd()	231
11.40.3.3 parseBitwise()	231
11.40.3.4 parseComparison()	231
11.40.3.5 parseFactor()	232
11.40.3.6 parseLogical()	232
11.40.3.7 parseTerm()	232
11.40.4 Member Data Documentation	232
11.40.4.1 lexer	232
11.41 expr_parser::ExprToken Struct Reference	232
11.41.1 Detailed Description	232
11.41.2 Member Data Documentation	233
11.41.2.1 type	233
11.41.2.2 value	233
11.42 mxvm::ExternalFunction Struct Reference	233
11.42.1 Detailed Description	233

11.42.2 Member Function Documentation	233
11.42.2.1 operator==()	233
11.42.3 Member Data Documentation	234
11.42.3.1 mod	234
11.42.3.2 module	234
11.42.3.3 name	234
11.42.3.4 ret_owned	234
11.42.3.5 ret_type	234
11.43 pascal::FieldAccessNode Class Reference	234
11.43.1 Detailed Description	235
11.43.2 Constructor & Destructor Documentation	235
11.43.2.1 FieldAccessNode()	235
11.43.3 Member Function Documentation	235
11.43.3.1 accept()	235
11.43.3.2 toString()	235
11.43.4 Member Data Documentation	236
11.43.4.1 fieldName	236
11.43.4.2 recordExpr	236
11.44 pascal::FileFunctionHandler Class Reference	236
11.44.1 Detailed Description	237
11.44.2 Member Function Documentation	237
11.44.2.1 canHandle()	237
11.44.2.2 generate()	237
11.44.2.3 generateWithResult()	237
11.44.2.4 getReturnType()	237
11.45 pascal::ForStmtNode Class Reference	238
11.45.1 Detailed Description	238
11.45.2 Constructor & Destructor Documentation	239
11.45.2.1 ForStmtNode()	239
11.45.3 Member Function Documentation	239
11.45.3.1 accept()	239
11.45.3.2 toString()	239
11.45.4 Member Data Documentation	239
11.45.4.1 endValue	239
11.45.4.2 isDownto	239
11.45.4.3 startValue	239
11.45.4.4 statement	239
11.45.4.5 variable	240
11.46 pascal::FuncCallNode Class Reference	240
11.46.1 Detailed Description	241
11.46.2 Constructor & Destructor Documentation	241
11.46.2.1 FuncCallNode()	241

11.46.3 Member Function Documentation	241
11.46.3.1 accept()	241
11.46.3.2 toString()	241
11.46.4 Member Data Documentation	241
11.46.4.1 arguments	241
11.46.4.2 name	241
11.47 pascal::FuncDeclNode Class Reference	241
11.47.1 Detailed Description	242
11.47.2 Constructor & Destructor Documentation	243
11.47.2.1 FuncDeclNode()	243
11.47.3 Member Function Documentation	243
11.47.3.1 accept()	243
11.47.3.2 toString()	243
11.47.4 Member Data Documentation	243
11.47.4.1 block	243
11.47.4.2 name	243
11.47.4.3 parameters	243
11.47.4.4 returnType	243
11.48 pascal::CodeGenVisitor::FuncInfo Struct Reference	244
11.48.1 Detailed Description	244
11.48.2 Member Data Documentation	244
11.48.2.1 paramTypes	244
11.48.2.2 returnType	244
11.49 mxvm::Function< Sig > Class Template Reference	244
11.49.1 Detailed Description	244
11.50 mxvm::Function< R(Args...) > Class Template Reference	244
11.50.1 Detailed Description	245
11.50.2 Member Typedef Documentation	245
11.50.2.1 Fn	245
11.50.3 Constructor & Destructor Documentation	245
11.50.3.1 Function() [1/4]	245
11.50.3.2 Function() [2/4]	246
11.50.3.3 ~Function()	246
11.50.3.4 Function() [3/4]	246
11.50.3.5 Function() [4/4]	246
11.50.4 Member Function Documentation	247
11.50.4.1 operator()()	247
11.50.4.2 operator=() [1/2]	247
11.50.4.3 operator=() [2/2]	247
11.50.5 Member Data Documentation	247
11.50.5.1 fn	247
11.50.5.2 handle	247

11.51	pascal::GotoStmtNode Class Reference	247
11.51.1	Detailed Description	248
11.51.2	Constructor & Destructor Documentation	248
11.51.2.1	GotoStmtNode()	248
11.51.3	Member Function Documentation	249
11.51.3.1	accept()	249
11.51.3.2	toString()	249
11.51.4	Member Data Documentation	249
11.51.4.1	label	249
11.52	mxvm::HTMLGen Class Reference	249
11.52.1	Detailed Description	249
11.52.2	Constructor & Destructor Documentation	249
11.52.2.1	HTMLGen()	249
11.52.3	Member Function Documentation	250
11.52.3.1	output()	250
11.52.4	Member Data Documentation	250
11.52.4.1	source	250
11.53	pascal::IfStmtNode Class Reference	250
11.53.1	Detailed Description	251
11.53.2	Constructor & Destructor Documentation	251
11.53.2.1	IfStmtNode()	251
11.53.3	Member Function Documentation	252
11.53.3.1	accept()	252
11.53.3.2	toString()	252
11.53.4	Member Data Documentation	252
11.53.4.1	condition	252
11.53.4.2	elseStatement	252
11.53.4.3	thenStatement	252
11.54	mxvm::Instruction Struct Reference	252
11.54.1	Detailed Description	253
11.54.2	Member Function Documentation	253
11.54.2.1	toString()	253
11.54.3	Member Data Documentation	253
11.54.3.1	instruction	253
11.54.3.2	label	253
11.54.3.3	op1	253
11.54.3.4	op2	254
11.54.3.5	op3	254
11.54.3.6	vop	255
11.55	mxvm::InstructionNode Class Reference	255
11.55.1	Detailed Description	255
11.55.2	Constructor & Destructor Documentation	256

11.55.2.1 InstructionNode() [1/2]	256
11.55.2.2 InstructionNode() [2/2]	256
11.55.3 Member Function Documentation	256
11.55.3.1 accept()	256
11.55.3.2 addOperand()	256
11.55.3.3 toString()	256
11.55.4 Member Data Documentation	256
11.55.4.1 instruction	256
11.55.4.2 operands	256
11.56 pascal::IOFunctionHandler Class Reference	257
11.56.1 Detailed Description	257
11.56.2 Member Function Documentation	257
11.56.2.1 canHandle()	257
11.56.2.2 generate()	257
11.56.2.3 generateWithResult()	258
11.57 mxvm::LabelNode Class Reference	258
11.57.1 Detailed Description	259
11.57.2 Constructor & Destructor Documentation	259
11.57.2.1 LabelNode()	259
11.57.3 Member Function Documentation	259
11.57.3.1 accept()	259
11.57.3.2 toString()	259
11.57.4 Member Data Documentation	259
11.57.4.1 function	259
11.57.4.2 name	259
11.58 pascal::LabelStmtNode Class Reference	260
11.58.1 Detailed Description	260
11.58.2 Constructor & Destructor Documentation	260
11.58.2.1 LabelStmtNode()	260
11.58.3 Member Function Documentation	261
11.58.3.1 accept()	261
11.58.3.2 toString()	261
11.58.4 Member Data Documentation	261
11.58.4.1 label	261
11.58.4.2 statement	261
11.59 mxvm::ModuleNode Class Reference	261
11.59.1 Detailed Description	262
11.59.2 Constructor & Destructor Documentation	262
11.59.2.1 ModuleNode()	262
11.59.3 Member Function Documentation	262
11.59.3.1 accept()	262
11.59.3.2 toString()	262

11.59.4 Member Data Documentation	263
11.59.4.1 name	263
11.60 mxvm::ModuleParser Class Reference	263
11.60.1 Detailed Description	264
11.60.2 Constructor & Destructor Documentation	264
11.60.2.1 ModuleParser() [1/2]	264
11.60.2.2 ModuleParser() [2/2]	264
11.60.3 Member Function Documentation	264
11.60.3.1 at()	264
11.60.3.2 generateProgramCode()	265
11.60.3.3 match() [1/2]	265
11.60.3.4 match() [2/2]	265
11.60.3.5 next()	265
11.60.3.6 operator=()	265
11.60.3.7 operator[]()	266
11.60.3.8 parse()	266
11.60.3.9 require() [1/2]	266
11.60.3.10 require() [2/2]	266
11.60.3.11 scan()	266
11.60.4 Member Data Documentation	266
11.60.4.1 functions	266
11.60.4.2 index	266
11.60.4.3 mod_name	267
11.60.4.4 parser_mode	267
11.60.4.5 scanner	267
11.60.4.6 token	267
11.61 pascal::NilNode Class Reference	267
11.61.1 Detailed Description	268
11.61.2 Constructor & Destructor Documentation	268
11.61.2.1 NilNode()	268
11.61.3 Member Function Documentation	268
11.61.3.1 accept()	268
11.61.3.2 toString()	268
11.62 pascal::NumberNode Class Reference	268
11.62.1 Detailed Description	269
11.62.2 Constructor & Destructor Documentation	269
11.62.2.1 NumberNode()	269
11.62.3 Member Function Documentation	270
11.62.3.1 accept()	270
11.62.3.2 toString()	270
11.62.4 Member Data Documentation	270
11.62.4.1 isInteger	270

11.62.4.2 isReal	270
11.62.4.3 value	270
11.63 mxvm::ObjectNode Class Reference	270
11.63.1 Detailed Description	271
11.63.2 Constructor & Destructor Documentation	271
11.63.2.1 ObjectNode()	271
11.63.3 Member Function Documentation	271
11.63.3.1 accept()	271
11.63.3.2 toString()	271
11.63.4 Member Data Documentation	272
11.63.4.1 name	272
11.64 mxvm::Operand Struct Reference	272
11.64.1 Detailed Description	272
11.64.2 Member Data Documentation	272
11.64.2.1 label	272
11.64.2.2 object	272
11.64.2.3 op	273
11.64.2.4 op_value	273
11.64.2.5 type	273
11.65 mxvm::OpSpec Struct Reference	274
11.65.1 Detailed Description	274
11.65.2 Member Data Documentation	274
11.65.2.1 fixed	274
11.65.2.2 maxArgs	274
11.65.2.3 minArgs	274
11.65.2.4 name	274
11.65.2.5 varPolicy	274
11.66 pascal::ParameterNode Class Reference	275
11.66.1 Detailed Description	275
11.66.2 Constructor & Destructor Documentation	276
11.66.2.1 ParameterNode()	276
11.66.3 Member Function Documentation	276
11.66.3.1 accept()	276
11.66.3.2 toString()	276
11.66.4 Member Data Documentation	276
11.66.4.1 identifiers	276
11.66.4.2 isVar	276
11.66.4.3 type	276
11.67 mxvm::ParsedOp Struct Reference	276
11.67.1 Detailed Description	277
11.67.2 Member Data Documentation	277
11.67.2.1 at	277

11.67.2.2 kind	277
11.67.2.3 text	277
11.68 pascal::ParseException Class Reference	277
11.68.1 Detailed Description	278
11.68.2 Constructor & Destructor Documentation	278
11.68.2.1 ParseException()	278
11.69 mxvm::Parser Class Reference	278
11.69.1 Detailed Description	279
11.69.2 Constructor & Destructor Documentation	280
11.69.2.1 Parser() [1/2]	280
11.69.2.2 Parser() [2/2]	280
11.69.3 Member Function Documentation	280
11.69.3.1 collectObjectNames()	280
11.69.3.2 generateDebugHTML()	280
11.69.3.3 generateObjectAssemblyFile()	280
11.69.3.4 generateProgramCode()	280
11.69.3.5 operator=()	281
11.69.3.6 operator[]()	281
11.69.3.7 parse()	281
11.69.3.8 parseAST()	281
11.69.3.9 parseCodeInstruction()	281
11.69.3.10 parseComment()	282
11.69.3.11 parseDataVariable()	282
11.69.3.12 parseLabel()	282
11.69.3.13 parseModule()	282
11.69.3.14 parseObject()	282
11.69.3.15 parseProgramOrObject()	282
11.69.3.16 parseSection()	283
11.69.3.17 printObjectHTML()	283
11.69.3.18 processCodeSection()	283
11.69.3.19 processDataSection()	283
11.69.3.20 processModuleFile()	283
11.69.3.21 processModuleSection()	283
11.69.3.22 processObjectFile()	284
11.69.3.23 processObjectSection()	284
11.69.3.24 registerObjectExterns()	284
11.69.3.25 resolveLabelReference()	284
11.69.3.26 scan()	284
11.69.3.27 setDefaultVariableValue()	284
11.69.3.28 setVariableValue()	284
11.69.4 Friends And Related Symbol Documentation	285
11.69.4.1 ModuleParser	285

11.69.5 Member Data Documentation	285
11.69.5.1 include_path	285
11.69.5.2 module_path	285
11.69.5.3 object_mode	285
11.69.5.4 object_name	285
11.69.5.5 object_path	285
11.69.5.6 parser_mode	285
11.69.5.7 platform	285
11.69.5.8 scanner	286
11.69.5.9 source_file	286
11.69.5.10 validator	286
11.70 pascal::PascalParser Class Reference	286
11.70.1 Detailed Description	290
11.70.2 Constructor & Destructor Documentation	290
11.70.2.1 PascalParser() [1/2]	290
11.70.2.2 PascalParser() [2/2]	290
11.70.3 Member Function Documentation	290
11.70.3.1 error()	290
11.70.3.2 expectToken() [1/2]	290
11.70.3.3 expectToken() [2/2]	291
11.70.3.4 getAddOp()	291
11.70.3.5 getArithmeticOperator()	291
11.70.3.6 getComparisonOperator()	291
11.70.3.7 getLogicalOperator()	291
11.70.3.8 getMulOp()	291
11.70.3.9 getRelationalOp()	291
11.70.3.10 getUnaryOp()	291
11.70.3.11 isAddOperator()	291
11.70.3.12 isArrayAccess()	292
11.70.3.13 isBuiltinFunction()	292
11.70.3.14 isBuiltinProcedure()	292
11.70.3.15 isKeyword()	292
11.70.3.16 isMulOperator()	292
11.70.3.17 isRelationalOperator()	292
11.70.3.18 isType()	292
11.70.3.19 isUnitSource()	292
11.70.3.20 match() [1/2]	292
11.70.3.21 match() [2/2]	293
11.70.3.22 parseArgumentList()	293
11.70.3.23 parseArrayDeclaration()	293
11.70.3.24 parseArrayType()	293
11.70.3.25 parseAssignmentOrProcCall()	293

11.70.3.26	parseBlock()	293
11.70.3.27	parseCaseStatement()	293
11.70.3.28	parseCompoundStatement()	294
11.70.3.29	parseConstDeclaration()	294
11.70.3.30	parseDeclarations()	294
11.70.3.31	parseExpression()	294
11.70.3.32	parseFactor()	294
11.70.3.33	parseForStatement()	294
11.70.3.34	parseFunctionCall()	295
11.70.3.35	parseFunctionDeclaration()	295
11.70.3.36	parseFunctionForwardDecl()	295
11.70.3.37	parseGotoStatement()	295
11.70.3.38	parseIfStatement()	295
11.70.3.39	parseInterfaceDeclarations()	295
11.70.3.40	parseLabelDeclaration()	295
11.70.3.41	parseLValue()	296
11.70.3.42	parseParameter()	296
11.70.3.43	parseParameterList()	296
11.70.3.44	parseProcedureCall()	296
11.70.3.45	parseProcedureDeclaration()	296
11.70.3.46	parseProcedureForwardDecl()	296
11.70.3.47	parseProgram()	297
11.70.3.48	parseRecordType()	297
11.70.3.49	parseRepeatStatement()	297
11.70.3.50	parseSimpleExpression()	297
11.70.3.51	parseStatement()	297
11.70.3.52	parseStatementList()	297
11.70.3.53	parseTerm()	297
11.70.3.54	parseTypeDeclaration()	298
11.70.3.55	parseTypeSpec()	298
11.70.3.56	parseUnit()	298
11.70.3.57	parseVarDeclaration()	298
11.70.3.58	parseWhileStatement()	298
11.70.3.59	parseWithStatement()	298
11.70.3.60	removeBraceComments()	299
11.70.3.61	tokenTypeToString()	299
11.70.4	Member Data Documentation	299
11.70.4.1	declaredLabels	299
11.70.4.2	validator	299
11.71	mxvm::PasHTMLGen Class Reference	299
11.71.1	Detailed Description	300
11.71.2	Constructor & Destructor Documentation	300

11.71.2.1 PasHTMLGen()	300
11.71.3 Member Function Documentation	300
11.71.3.1 defaultCss()	300
11.71.3.2 output()	300
11.71.4 Member Data Documentation	300
11.71.4.1 source	300
11.72 pascal::PointerDerefNode Class Reference	301
11.72.1 Detailed Description	301
11.72.2 Constructor & Destructor Documentation	301
11.72.2.1 PointerDerefNode()	301
11.72.3 Member Function Documentation	302
11.72.3.1 accept()	302
11.72.3.2 toString()	302
11.72.4 Member Data Documentation	302
11.72.4.1 pointer	302
11.73 pascal::PointerTypeNode Class Reference	302
11.73.1 Detailed Description	303
11.73.2 Constructor & Destructor Documentation	303
11.73.2.1 PointerTypeNode()	303
11.73.3 Member Function Documentation	303
11.73.3.1 accept()	303
11.73.3.2 toString()	303
11.73.4 Member Data Documentation	303
11.73.4.1 baseTypeName	303
11.74 pascal::ProcCallNode Class Reference	304
11.74.1 Detailed Description	304
11.74.2 Constructor & Destructor Documentation	304
11.74.2.1 ProcCallNode()	304
11.74.3 Member Function Documentation	305
11.74.3.1 accept()	305
11.74.3.2 toString()	305
11.74.4 Member Data Documentation	305
11.74.4.1 arguments	305
11.74.4.2 name	305
11.75 pascal::ProcDeclNode Class Reference	305
11.75.1 Detailed Description	306
11.75.2 Constructor & Destructor Documentation	306
11.75.2.1 ProcDeclNode()	306
11.75.3 Member Function Documentation	307
11.75.3.1 accept()	307
11.75.3.2 toString()	307
11.75.4 Member Data Documentation	307

11.75.4.1 block	307
11.75.4.2 name	307
11.75.4.3 parameters	307
11.76 mxvm::Program Class Reference	307
11.76.1 Detailed Description	316
11.76.2 Member Enumeration Documentation	316
11.76.2.1 LastCmpType	316
11.76.3 Constructor & Destructor Documentation	316
11.76.3.1 Program() [1/2]	316
11.76.3.2 ~Program()	316
11.76.3.3 Program() [2/2]	316
11.76.4 Member Function Documentation	317
11.76.4.1 add_standard()	317
11.76.4.2 addVariables()	317
11.76.4.3 createTempVariable()	317
11.76.4.4 divVariables()	318
11.76.4.5 escapeNewLines()	318
11.76.4.6 exec()	318
11.76.4.7 exec_add()	318
11.76.4.8 exec_alloc()	319
11.76.4.9 exec_and()	319
11.76.4.10 exec_call()	319
11.76.4.11 exec_cmp()	319
11.76.4.12 exec_div()	319
11.76.4.13 exec_done()	319
11.76.4.14 exec_exit()	320
11.76.4.15 exec_fcmp()	320
11.76.4.16 exec_free()	320
11.76.4.17 exec_getline()	320
11.76.4.18 exec_invoke()	320
11.76.4.19 exec_ja()	320
11.76.4.20 exec_jae()	321
11.76.4.21 exec_jb()	321
11.76.4.22 exec_jbe()	321
11.76.4.23 exec_jc()	321
11.76.4.24 exec_je()	321
11.76.4.25 exec_jg()	321
11.76.4.26 exec_jge()	321
11.76.4.27 exec_jl()	321
11.76.4.28 exec_jle()	322
11.76.4.29 exec_jmp()	322
11.76.4.30 exec_jnc()	322

11.76.4.31	exec_jne()	322
11.76.4.32	exec_jno()	322
11.76.4.33	exec_jnp()	322
11.76.4.34	exec_jns()	322
11.76.4.35	exec_jnz()	323
11.76.4.36	exec_jo()	323
11.76.4.37	exec_jp()	323
11.76.4.38	exec_js()	323
11.76.4.39	exec_jz()	323
11.76.4.40	exec_lea()	323
11.76.4.41	exec_load()	323
11.76.4.42	exec_mod()	324
11.76.4.43	exec_mov()	324
11.76.4.44	exec_mul()	324
11.76.4.45	exec_neg()	324
11.76.4.46	exec_not()	324
11.76.4.47	exec_or()	324
11.76.4.48	exec_pop()	325
11.76.4.49	exec_print()	325
11.76.4.50	exec_push()	325
11.76.4.51	exec_realloc()	325
11.76.4.52	exec_ret()	326
11.76.4.53	exec_return()	326
11.76.4.54	exec_stack_load()	326
11.76.4.55	exec_stack_store()	326
11.76.4.56	exec_stack_sub()	326
11.76.4.57	exec_store()	327
11.76.4.58	exec_string_print()	327
11.76.4.59	exec_sub()	327
11.76.4.60	exec_to_float()	327
11.76.4.61	exec_to_int()	328
11.76.4.62	exec_xor()	328
11.76.4.63	flatten()	328
11.76.4.64	flatten_external()	328
11.76.4.65	flatten_inc()	328
11.76.4.66	flatten_label()	329
11.76.4.67	gen_add()	329
11.76.4.68	gen_alloc()	329
11.76.4.69	gen_arth()	329
11.76.4.70	gen_bitop()	330
11.76.4.71	gen_call()	330
11.76.4.72	gen_cmp()	330

11.76.4.73 <code>gen_div()</code>	330
11.76.4.74 <code>gen_done()</code>	330
11.76.4.75 <code>gen_exit()</code>	331
11.76.4.76 <code>gen_fcmp()</code>	331
11.76.4.77 <code>gen_free()</code>	331
11.76.4.78 <code>gen_getline()</code>	331
11.76.4.79 <code>gen_invoke()</code>	331
11.76.4.80 <code>gen_jae()</code>	331
11.76.4.81 <code>gen_jbe()</code>	332
11.76.4.82 <code>gen_jc()</code>	332
11.76.4.83 <code>gen_jmp()</code>	332
11.76.4.84 <code>gen_jnc()</code>	332
11.76.4.85 <code>gen_jno()</code>	332
11.76.4.86 <code>gen_jnp()</code>	332
11.76.4.87 <code>gen_jns()</code>	332
11.76.4.88 <code>gen_jo()</code>	333
11.76.4.89 <code>gen_jp()</code>	333
11.76.4.90 <code>gen_js()</code>	333
11.76.4.91 <code>gen_lea()</code>	333
11.76.4.92 <code>gen_load()</code>	333
11.76.4.93 <code>gen_mod()</code>	333
11.76.4.94 <code>gen_mov()</code>	334
11.76.4.95 <code>gen_mul()</code>	334
11.76.4.96 <code>gen_neg()</code>	334
11.76.4.97 <code>gen_not()</code>	334
11.76.4.98 <code>gen_optimize()</code>	334
11.76.4.99 <code>gen_pop()</code>	335
11.76.4.100 <code>gen_print()</code>	335
11.76.4.101 <code>gen_push()</code>	335
11.76.4.102 <code>gen_realloc()</code>	336
11.76.4.103 <code>gen_ret()</code>	336
11.76.4.104 <code>gen_return()</code>	336
11.76.4.105 <code>gen_stack_load()</code>	336
11.76.4.106 <code>gen_stack_store()</code>	336
11.76.4.107 <code>gen_stack_sub()</code>	337
11.76.4.108 <code>gen_store()</code>	337
11.76.4.109 <code>gen_sub()</code>	337
11.76.4.110 <code>gen_to_float()</code>	337
11.76.4.111 <code>gen_to_int()</code>	337
11.76.4.112 <code>generateCode()</code>	337
11.76.4.113 <code>generateFunctionCall()</code>	338
11.76.4.114 <code>generateInstruction()</code>	338

11.76.4.115 generateInvokeCall()	339
11.76.4.116 generateLoadVar() [1/2]	339
11.76.4.117 generateLoadVar() [2/2]	339
11.76.4.118 getExitCode()	340
11.76.4.119 getMangledName() [1/2]	340
11.76.4.120 getMangledName() [2/2]	340
11.76.4.121 getObjectByName()	341
11.76.4.122 getPlatformSymbolName()	341
11.76.4.123 getRegisterByIndex()	341
11.76.4.124 getVariable()	342
11.76.4.125 isConstant()	343
11.76.4.126 isFunctionValid()	343
11.76.4.127 isVariable()	343
11.76.4.128 memoryDump()	344
11.76.4.129 mulVariables()	345
11.76.4.130 operator=()	345
11.76.4.131 optimize_core()	345
11.76.4.132 optimize_darwin()	345
11.76.4.133 post()	346
11.76.4.134 print()	346
11.76.4.135 printfFormatted()	346
11.76.4.136 setArgs()	346
11.76.4.137 setObject()	347
11.76.4.138 setVariableFromConstant()	347
11.76.4.139 setVariableFromString()	347
11.76.4.140 stop()	347
11.76.4.141 subVariables()	348
11.76.4.142 sysv_analyzeRegAlloc()	348
11.76.4.143 sysv_emitFlushRegs()	348
11.76.4.144 sysv_emitLoadVar()	348
11.76.4.145 sysv_emitReloadRegs()	349
11.76.4.146 sysv_emitRestoreRegs()	349
11.76.4.147 sysv_emitSaveRegs()	349
11.76.4.148 sysv_emitStoreVar()	349
11.76.4.149 sysv_emitStoreVarImm()	349
11.76.4.150 validateNames()	350
11.76.4.151 variableFromOperand()	350
11.76.4.152 x64_analyzeRegAlloc()	350
11.76.4.153 x64_direct_stack_adjust()	351
11.76.4.154 x64_emit_job_func()	351
11.76.4.155 x64_emitFlushRegs()	351
11.76.4.156 x64_emitLoadVar()	351

11.76.4.157 x64_emitReloadRegs()	352
11.76.4.158 x64_emitRestoreRegs()	352
11.76.4.159 x64_emitSaveRegs()	352
11.76.4.160 x64_emitStoreVar()	352
11.76.4.161 x64_emitStoreVarImm()	353
11.76.4.162 x64_gen_alloc()	353
11.76.4.163 x64_gen_arth()	353
11.76.4.164 x64_gen_bitop()	353
11.76.4.165 x64_gen_call()	353
11.76.4.166 x64_gen_cmp()	354
11.76.4.167 x64_gen_div()	354
11.76.4.168 x64_gen_done()	354
11.76.4.169 x64_gen_exit()	354
11.76.4.170 x64_gen_fcmp()	354
11.76.4.171 x64_gen_free()	355
11.76.4.172 x64_gen_getline()	355
11.76.4.173 x64_gen_invoke()	355
11.76.4.174 x64_gen_jae()	355
11.76.4.175 x64_gen_jbe()	355
11.76.4.176 x64_gen_jc()	355
11.76.4.177 x64_gen_jmp()	356
11.76.4.178 x64_gen_jnc()	356
11.76.4.179 x64_gen_jno()	356
11.76.4.180 x64_gen_jnp()	356
11.76.4.181 x64_gen_jns()	356
11.76.4.182 x64_gen_jo()	356
11.76.4.183 x64_gen_jp()	356
11.76.4.184 x64_gen_js()	357
11.76.4.185 x64_gen_lea()	357
11.76.4.186 x64_gen_load()	357
11.76.4.187 x64_gen_mod()	357
11.76.4.188 x64_gen_mov()	357
11.76.4.189 x64_gen_neg()	358
11.76.4.190 x64_gen_not()	358
11.76.4.191 x64_gen_pop()	358
11.76.4.192 x64_gen_print()	358
11.76.4.193 x64_gen_push()	358
11.76.4.194 x64_gen_realloc()	358
11.76.4.195 x64_gen_ret()	359
11.76.4.196 x64_gen_return()	359
11.76.4.197 x64_gen_stack_load()	359
11.76.4.198 x64_gen_stack_store()	359

11.76.4.199 x64_gen_stack_sub()	359
11.76.4.200 x64_gen_store()	359
11.76.4.201 x64_gen_to_float()	360
11.76.4.202 x64_gen_to_int()	360
11.76.4.203 x64_generateCode()	360
11.76.4.204 x64_generateFunctionCall()	360
11.76.4.205 x64_generateInstruction()	361
11.76.4.206 x64_generateInvokeCall()	361
11.76.4.207 x64_generateLoadVar() [1/2]	362
11.76.4.208 x64_generateLoadVar() [2/2]	362
11.76.4.209 x64_getRegisterByIndex()	362
11.76.4.210 x64_release_call_area()	363
11.76.4.211 x64_reserve_call_area()	363
11.76.5 Friends And Related Symbol Documentation	363
11.76.5.1 Parser	363
11.76.6 Member Data Documentation	364
11.76.6.1 args	364
11.76.6.2 carry_flag	364
11.76.6.3 exitCode	364
11.76.6.4 filename	364
11.76.6.5 greater_flag	364
11.76.6.6 last_call_ret_type	364
11.76.6.7 last_call_returns_owned	364
11.76.6.8 last_call_returns_owned_ptr	364
11.76.6.9 last_cmp_type	365
11.76.6.10 less_flag	365
11.76.6.11 main_function	365
11.76.6.12 name	365
11.76.6.13 object	365
11.76.6.14 object_external	365
11.76.6.15 objects	365
11.76.6.16 parent	365
11.76.6.17 pc	365
11.76.6.18 platform	366
11.76.6.19 result	366
11.76.6.20 running	366
11.76.6.21 stack	366
11.76.6.22 sysv_reg_save_order	366
11.76.6.23 sysv_reg_vars	366
11.76.6.24 x64_reg_save_order	366
11.76.6.25 x64_reg_vars	366
11.76.6.26 xmm_offset	367

11.76.6.27 zero_flag	367
11.77 mxvm::ProgramNode Class Reference	367
11.77.1 Detailed Description	368
11.77.2 Member Function Documentation	368
11.77.2.1 accept()	368
11.77.2.2 addInlineObject()	368
11.77.2.3 addSection()	368
11.77.2.4 toString()	368
11.77.3 Member Data Documentation	368
11.77.3.1 inlineObjects	368
11.77.3.2 name	368
11.77.3.3 object	369
11.77.3.4 root_name	369
11.77.3.5 sections	369
11.78 pascal::ProgramNode Class Reference	369
11.78.1 Detailed Description	370
11.78.2 Constructor & Destructor Documentation	370
11.78.2.1 ProgramNode()	370
11.78.3 Member Function Documentation	370
11.78.3.1 accept()	370
11.78.3.2 toString()	370
11.78.4 Member Data Documentation	370
11.78.4.1 block	370
11.78.4.2 name	371
11.78.4.3 uses	371
11.79 pascal::RecordDeclarationNode Class Reference	371
11.79.1 Detailed Description	372
11.79.2 Constructor & Destructor Documentation	372
11.79.2.1 RecordDeclarationNode()	372
11.79.3 Member Function Documentation	372
11.79.3.1 accept()	372
11.79.3.2 toString()	372
11.79.4 Member Data Documentation	372
11.79.4.1 name	372
11.79.4.2 recordType	372
11.80 pascal::CodeGenVisitor::RecordField Struct Reference	373
11.80.1 Detailed Description	373
11.80.2 Constructor & Destructor Documentation	373
11.80.2.1 RecordField() [1/3]	373
11.80.2.2 RecordField() [2/3]	373
11.80.2.3 RecordField() [3/3]	373
11.80.3 Member Function Documentation	373

11.80.3.1 operator=() [1/2]	373
11.80.3.2 operator=() [2/2]	374
11.80.4 Member Data Documentation	374
11.80.4.1 arrayInfo	374
11.80.4.2 isArray	374
11.80.4.3 name	374
11.80.4.4 offset	374
11.80.4.5 size	374
11.80.4.6 typeName	374
11.81 pascal::CodeGenVisitor::RecordTypeInfo Struct Reference	374
11.81.1 Detailed Description	375
11.81.2 Member Data Documentation	375
11.81.2.1 fields	375
11.81.2.2 nameToIndex	375
11.81.2.3 size	375
11.82 pascal::RecordTypeNode Class Reference	375
11.82.1 Detailed Description	376
11.82.2 Constructor & Destructor Documentation	377
11.82.2.1 RecordTypeNode()	377
11.82.3 Member Function Documentation	377
11.82.3.1 accept()	377
11.82.3.2 toString()	377
11.82.4 Member Data Documentation	377
11.82.4.1 fields	377
11.82.4.2 variantArms	377
11.82.4.3 variantTagName	377
11.82.4.4 variantTagType	377
11.83 pascal::RepeatStmtNode Class Reference	378
11.83.1 Detailed Description	378
11.83.2 Constructor & Destructor Documentation	378
11.83.2.1 RepeatStmtNode()	378
11.83.3 Member Function Documentation	379
11.83.3.1 accept()	379
11.83.3.2 toString()	379
11.83.4 Member Data Documentation	379
11.83.4.1 condition	379
11.83.4.2 statements	379
11.84 mxvm::RuntimeFunction Class Reference	379
11.84.1 Detailed Description	380
11.84.2 Constructor & Destructor Documentation	380
11.84.2.1 RuntimeFunction() [1/3]	380
11.84.2.2 RuntimeFunction() [2/3]	380

11.84.2.3 ~RuntimeFunction()	380
11.84.2.4 RuntimeFunction() [3/3]	380
11.84.3 Member Function Documentation	381
11.84.3.1 call()	381
11.84.3.2 operator=()	381
11.84.4 Member Data Documentation	381
11.84.4.1 fname	381
11.84.4.2 func	381
11.84.4.3 handle	381
11.84.4.4 handles	381
11.84.4.5 mod_name	382
11.85 scan::ScanExcept Class Reference	382
11.85.1 Detailed Description	382
11.85.2 Constructor & Destructor Documentation	382
11.85.2.1 ScanExcept() [1/2]	382
11.85.2.2 ScanExcept() [2/2]	382
11.85.3 Member Function Documentation	382
11.85.3.1 why()	382
11.85.4 Member Data Documentation	383
11.85.4.1 msg	383
11.86 scan::Scanner Class Reference	383
11.86.1 Detailed Description	384
11.86.2 Constructor & Destructor Documentation	384
11.86.2.1 Scanner()	384
11.86.3 Member Function Documentation	384
11.86.3.1 getTokens()	384
11.86.3.2 grabDigits()	384
11.86.3.3 grabId()	384
11.86.3.4 grabSingle()	385
11.86.3.5 grabString()	385
11.86.3.6 grabSymbols()	385
11.86.3.7 is_c_sym()	385
11.86.3.8 operator[]()	385
11.86.3.9 parseFileName()	385
11.86.3.10 parseLineNumber()	386
11.86.3.11 removeEOL()	386
11.86.3.12 scan()	386
11.86.3.13 size()	386
11.86.4 Member Data Documentation	386
11.86.4.1 string_buffer	386
11.86.4.2 token_map	386
11.86.4.3 tokens	386

11.87	mxv::Scope Struct Reference	387
11.87.1	Detailed Description	387
11.87.2	Member Data Documentation	387
11.87.2.1	consts	387
11.87.2.2	funcs	387
11.87.2.3	params	387
11.87.2.4	procs	387
11.87.2.5	types	388
11.87.2.6	vars	388
11.88	pascal::CodeGenVisitor::Scope Struct Reference	388
11.88.1	Detailed Description	388
11.88.2	Member Data Documentation	388
11.88.2.1	name	388
11.88.2.2	nestedFuncs	388
11.88.2.3	nestedProcs	388
11.89	pascal::SDLFunctionHandler Class Reference	388
11.89.1	Detailed Description	389
11.89.2	Member Function Documentation	389
11.89.2.1	canHandle()	389
11.89.2.2	generate()	389
11.89.2.3	generateWithResult()	390
11.90	mxvm::SectionNode Class Reference	390
11.90.1	Detailed Description	391
11.90.2	Member Enumeration Documentation	391
11.90.2.1	SectionType	391
11.90.3	Constructor & Destructor Documentation	391
11.90.3.1	SectionNode()	391
11.90.4	Member Function Documentation	391
11.90.4.1	accept()	391
11.90.4.2	addStatement()	391
11.90.4.3	toString()	391
11.90.5	Member Data Documentation	392
11.90.5.1	statements	392
11.90.5.2	type	392
11.91	pascal::SetLiteralNode Class Reference	392
11.91.1	Detailed Description	393
11.91.2	Constructor & Destructor Documentation	393
11.91.2.1	SetLiteralNode()	393
11.91.3	Member Function Documentation	393
11.91.3.1	accept()	393
11.91.3.2	toString()	393
11.91.4	Member Data Documentation	393

11.91.4.1 elements	393
11.92 pascal::SetTypeNode Class Reference	393
11.92.1 Detailed Description	394
11.92.2 Constructor & Destructor Documentation	394
11.92.2.1 SetTypeNode()	394
11.92.3 Member Function Documentation	395
11.92.3.1 accept()	395
11.92.3.2 toString()	395
11.92.4 Member Data Documentation	395
11.92.4.1 baseTypeName	395
11.93 pascal::SimpleTypeNode Class Reference	395
11.93.1 Detailed Description	396
11.93.2 Constructor & Destructor Documentation	396
11.93.2.1 SimpleTypeNode()	396
11.93.3 Member Function Documentation	396
11.93.3.1 accept()	396
11.93.3.2 toString()	396
11.93.4 Member Data Documentation	396
11.93.4.1 typeName	396
11.94 mxvm::Stack Class Reference	397
11.94.1 Detailed Description	397
11.94.2 Constructor & Destructor Documentation	397
11.94.2.1 Stack()	397
11.94.3 Member Function Documentation	397
11.94.3.1 empty()	397
11.94.3.2 operator[]()	397
11.94.3.3 pop()	398
11.94.3.4 push()	398
11.94.3.5 size()	398
11.94.4 Member Data Documentation	398
11.94.4.1 data	398
11.95 pascal::StdFunctionHandler Class Reference	399
11.95.1 Detailed Description	399
11.95.2 Member Function Documentation	399
11.95.2.1 canHandle()	399
11.95.2.2 generate()	399
11.95.2.3 generateWithResult()	400
11.96 scan::buffer::StringBuffer< Ch, String > Class Template Reference	400
11.96.1 Detailed Description	401
11.96.2 Member Typedef Documentation	401
11.96.2.1 ch_type	401
11.96.2.2 string_type	401

11.96.3 Constructor & Destructor Documentation	402
11.96.3.1 StringBuffer()	402
11.96.4 Member Function Documentation	402
11.96.4.1 backward_step()	402
11.96.4.2 cur_file()	402
11.96.4.3 cur_line()	402
11.96.4.4 curch()	402
11.96.4.5 eof()	403
11.96.4.6 forward_step()	403
11.96.4.7 getch()	403
11.96.4.8 peekch()	403
11.96.4.9 prevch()	403
11.96.4.10 process_line_directive()	404
11.96.4.11 reset()	404
11.96.5 Member Data Documentation	404
11.96.5.1 buffer_	404
11.96.5.2 currentColumn	404
11.96.5.3 currentFile	404
11.96.5.4 currentLine	405
11.96.5.5 index	405
11.97 pascal::StringFunctionHandler Class Reference	405
11.97.1 Detailed Description	406
11.97.2 Member Function Documentation	406
11.97.2.1 canHandle()	406
11.97.2.2 generate()	406
11.97.2.3 generateWithResult()	406
11.97.2.4 getReturnType()	406
11.98 pascal::StringNode Class Reference	406
11.98.1 Detailed Description	407
11.98.2 Constructor & Destructor Documentation	407
11.98.2.1 StringNode()	407
11.98.3 Member Function Documentation	408
11.98.3.1 accept()	408
11.98.3.2 toString()	408
11.98.4 Member Data Documentation	408
11.98.4.1 value	408
11.99 scan::token::Token< Ch, String > Class Template Reference	408
11.99.1 Detailed Description	409
11.99.2 Member Typedef Documentation	409
11.99.2.1 ch_type	409
11.99.2.2 string_type	410
11.99.3 Constructor & Destructor Documentation	410

11.99.3.1 Token() [1/4]	410
11.99.3.2 Token() [2/4]	410
11.99.3.3 Token() [3/4]	410
11.99.3.4 Token() [4/4]	410
11.99.4 Member Function Documentation	410
11.99.4.1 get_filename()	410
11.99.4.2 get_pos()	410
11.99.4.3 getCol()	411
11.99.4.4 getLine()	411
11.99.4.5 getTokenType()	411
11.99.4.6 getTokenValue()	411
11.99.4.7 operator=() [1/3]	411
11.99.4.8 operator=() [2/3]	411
11.99.4.9 operator=() [3/3]	411
11.99.4.10 print()	411
11.99.4.11 set_filename()	412
11.99.4.12 set_pos()	412
11.99.4.13 setToken()	412
11.99.5 Member Data Documentation	412
11.99.5.1 col	412
11.99.5.2 filename	412
11.99.5.3 line	413
11.99.5.4 type	413
11.99.5.5 value	413
11.100 scan::token::TokenMap< Ch, MAX_SIZE > Class Template Reference	413
11.100.1 Detailed Description	413
11.100.2 Constructor & Destructor Documentation	414
11.100.2.1 TokenMap()	414
11.100.3 Member Function Documentation	414
11.100.3.1 lookup_int8()	414
11.100.4 Member Data Documentation	414
11.100.4.1 token_map	414
11.101 mxx::TPValidator Class Reference	414
11.101.1 Detailed Description	419
11.101.2 Constructor & Destructor Documentation	419
11.101.2.1 TPValidator()	419
11.101.3 Member Function Documentation	419
11.101.3.1 checkConstOnly()	419
11.101.3.2 checkType()	419
11.101.3.3 checkVar()	419
11.101.3.4 checkVarOrConst()	419
11.101.3.5 currentScope() [1/2]	420

11.101.3.6 currentScope() [2/2]	420
11.101.3.7 currentScopeName()	420
11.101.3.8 declareConst()	420
11.101.3.9 declareFunc()	420
11.101.3.10 declareParam()	420
11.101.3.11 declareProc()	421
11.101.3.12 declareRecordField()	421
11.101.3.13 declareType()	421
11.101.3.14 declareVar()	421
11.101.3.15 fail()	421
11.101.3.16 failAt()	421
11.101.3.17 failHere()	422
11.101.3.18 found()	422
11.101.3.19 inRecordFieldScope()	422
11.101.3.20 isAddOp()	422
11.101.3.21 isBuiltinConst()	422
11.101.3.22 isBuiltinType()	422
11.101.3.23 isBuiltinTypeName()	422
11.101.3.24 isFuncDeclaredHere()	423
11.101.3.25 isKW()	423
11.101.3.26 isMulOp()	423
11.101.3.27 isParamDeclaredHere()	423
11.101.3.28 isPascalKeyword()	423
11.101.3.29 isProcDeclaredHere()	423
11.101.3.30 isRelOp()	423
11.101.3.31 isSetOp()	424
11.101.3.32 isTypeDeclaredHere()	424
11.101.3.33 isVarDeclaredHere()	424
11.101.3.34 isVariableInCurrentScope()	424
11.101.3.35 lower()	424
11.101.3.36 match() [1/2]	424
11.101.3.37 match() [2/2]	425
11.101.3.38 next()	425
11.101.3.39 parseActualParams()	425
11.101.3.40 parseBlock()	425
11.101.3.41 parseCase()	425
11.101.3.42 parseCaseLabel()	425
11.101.3.43 parseCaseLabelList()	426
11.101.3.44 parseCompoundStatement()	426
11.101.3.45 parseConstant()	426
11.101.3.46 parseConstExpr()	426
11.101.3.47 parseConstSection()	426

11.101.3.48	parseConstSimple()	426
11.101.3.49	parseDesignator()	427
11.101.3.50	parseExprStop()	427
11.101.3.51	parseFieldIdentList()	427
11.101.3.52	parseFor()	427
11.101.3.53	parseFormalParams()	427
11.101.3.54	parseGoto()	427
11.101.3.55	parseIdentList()	427
11.101.3.56	parseIf()	428
11.101.3.57	parseInterfaceSection()	428
11.101.3.58	parseLabelSection()	428
11.101.3.59	parseProgram()	428
11.101.3.60	parseRepeat()	428
11.101.3.61	parseSimpleOrCallOrAssign()	428
11.101.3.62	parseStatement()	428
11.101.3.63	parseSubprogram()	429
11.101.3.64	parseSubrange()	429
11.101.3.65	parseType()	429
11.101.3.66	parseTypeName()	429
11.101.3.67	parseTypeSection()	429
11.101.3.68	parseUnit()	429
11.101.3.69	parseUses()	429
11.101.3.70	parseVarSection()	430
11.101.3.71	parseWhile()	430
11.101.3.72	parseWith()	430
11.101.3.73	peekIs()	430
11.101.3.74	popRecordFieldScope()	430
11.101.3.75	popScope()	430
11.101.3.76	predeclareType()	430
11.101.3.77	pushRecordFieldScope()	431
11.101.3.78	pushScope()	431
11.101.3.79	require() [1/2]	431
11.101.3.80	require() [2/2]	431
11.101.3.81	requireKW()	431
11.101.3.82	tokenTypeToString()	431
11.101.3.83	toLower()	432
11.101.3.84	validate()	432
11.101.4	Member Data Documentation	432
11.101.4.1	currentRecordTypeName	432
11.101.4.2	declaredConsts	432
11.101.4.3	declaredFuncs	432
11.101.4.4	declaredProcs	432

11.101.4.5 declaredTypes	432
11.101.4.6 declaredVars	433
11.101.4.7 filename	433
11.101.4.8 importedUnits	433
11.101.4.9 index	433
11.101.4.10 inFunctionDepth	433
11.101.4.11 recordFieldScopesByType	433
11.101.4.12 scanner	433
11.101.4.13 scopeStack	433
11.101.4.14 source	434
11.101.4.15 token	434
11.101.4.16 withDepth	434
11.102 pascal::TypeAliasNode Class Reference	434
11.102.1 Detailed Description	435
11.102.2 Constructor & Destructor Documentation	435
11.102.2.1 TypeAliasNode()	435
11.102.3 Member Function Documentation	435
11.102.3.1 accept()	435
11.102.3.2 toString()	435
11.102.4 Member Data Documentation	435
11.102.4.1 baseType	435
11.102.4.2 typeName	436
11.103 pascal::TypeDeclNode Class Reference	436
11.103.1 Detailed Description	437
11.103.2 Constructor & Destructor Documentation	437
11.103.2.1 TypeDeclNode()	437
11.103.3 Member Function Documentation	437
11.103.3.1 accept()	437
11.103.3.2 toString()	437
11.103.4 Member Data Documentation	437
11.103.4.1 typeDeclarations	437
11.104 pascal::UnaryOpNode Class Reference	437
11.104.1 Detailed Description	438
11.104.2 Member Enumeration Documentation	439
11.104.2.1 Operator	439
11.104.3 Constructor & Destructor Documentation	439
11.104.3.1 UnaryOpNode()	439
11.104.4 Member Function Documentation	439
11.104.4.1 accept()	439
11.104.4.2 opToString()	439
11.104.4.3 toString()	439
11.104.5 Member Data Documentation	439

11.104.5.1 operand	439
11.104.5.2 operator_	440
11.105 pascal::UnitNode Class Reference	440
11.105.1 Detailed Description	441
11.105.2 Constructor & Destructor Documentation	441
11.105.2.1 UnitNode()	441
11.105.3 Member Function Documentation	441
11.105.3.1 accept()	441
11.105.3.2 toString()	441
11.105.4 Member Data Documentation	441
11.105.4.1 implDecls	441
11.105.4.2 interfaceDecls	442
11.105.4.3 name	442
11.105.4.4 uses	442
11.106 mxvm::UseLabel Struct Reference	442
11.106.1 Detailed Description	442
11.106.2 Member Data Documentation	442
11.106.2.1 at	442
11.106.2.2 name	442
11.107 mxvm::UseVar Struct Reference	442
11.107.1 Detailed Description	443
11.107.2 Member Data Documentation	443
11.107.2.1 at	443
11.107.2.2 name	443
11.108 mxvm::Validator Class Reference	443
11.108.1 Detailed Description	444
11.108.2 Constructor & Destructor Documentation	444
11.108.2.1 Validator()	444
11.108.3 Member Function Documentation	445
11.108.3.1 collect_labels()	445
11.108.3.2 collect_objects()	445
11.108.3.3 match() [1/2]	445
11.108.3.4 match() [2/2]	445
11.108.3.5 next()	445
11.108.3.6 parseOperand()	445
11.108.3.7 parseOperandList()	445
11.108.3.8 peekIs() [1/2]	446
11.108.3.9 peekIs() [2/2]	446
11.108.3.10 require() [1/2]	446
11.108.3.11 require() [2/2]	446
11.108.3.12 tokenTypeToString()	446
11.108.3.13 validate()	446

11.108.3.14 validateAgainstSpec()	447
11.108.4 Member Data Documentation	447
11.108.4.1 current_function	447
11.108.4.2 filename	447
11.108.4.3 function_vars	447
11.108.4.4 index	448
11.108.4.5 scanner	448
11.108.4.6 source	448
11.108.4.7 token	448
11.108.4.8 var_names	448
11.109 pascal::CodeGenVisitor::ValueLocation Struct Reference	448
11.109.1 Detailed Description	448
11.109.2 Member Enumeration Documentation	449
11.109.2.1 Type	449
11.109.3 Member Data Documentation	449
11.109.3.1 location	449
11.109.3.2 type	449
11.110 pascal::VarDeclNode Class Reference	449
11.110.1 Detailed Description	450
11.110.2 Constructor & Destructor Documentation	450
11.110.2.1 VarDeclNode()	450
11.110.3 Member Function Documentation	450
11.110.3.1 accept()	450
11.110.3.2 toString()	450
11.110.4 Member Data Documentation	451
11.110.4.1 identifiers	451
11.110.4.2 initializers	451
11.110.4.3 type	451
11.111 mxvm::Variable Struct Reference	451
11.111.1 Detailed Description	452
11.111.2 Constructor & Destructor Documentation	452
11.111.2.1 Variable() [1/3]	452
11.111.2.2 Variable() [2/3]	452
11.111.2.3 Variable() [3/3]	452
11.111.3 Member Function Documentation	452
11.111.3.1 operator=()	452
11.111.3.2 setExtern()	453
11.111.3.3 toString()	453
11.111.4 Member Data Documentation	453
11.111.4.1 is_global	453
11.111.4.2 obj_name	453
11.111.4.3 type	453

11.111.4.4 var_name	454
11.111.4.5 var_value	454
11.112 mxvm::Variable_Value Struct Reference	455
11.112.1 Detailed Description	456
11.112.2 Constructor & Destructor Documentation	456
11.112.2.1 Variable_Value() [1/2]	456
11.112.2.2 Variable_Value() [2/2]	456
11.112.3 Member Function Documentation	456
11.112.3.1 operator=()	456
11.112.4 Member Data Documentation	457
11.112.4.1 [union]	457
11.112.4.2 buffer_size	457
11.112.4.3 float_value	457
11.112.4.4 int_value	457
11.112.4.5 label_value	458
11.112.4.6 owns	458
11.112.4.7 ptr_count	458
11.112.4.8 ptr_size	458
11.112.4.9 ptr_value	458
11.112.4.10 released	459
11.112.4.11 str_value	459
11.112.4.12 type	459
11.113 mxvm::VariableNode Class Reference	459
11.113.1 Detailed Description	460
11.113.2 Constructor & Destructor Documentation	460
11.113.2.1 VariableNode() [1/3]	460
11.113.2.2 VariableNode() [2/3]	461
11.113.2.3 VariableNode() [3/3]	461
11.113.3 Member Function Documentation	461
11.113.3.1 accept()	461
11.113.3.2 toString()	461
11.113.4 Member Data Documentation	461
11.113.4.1 buffer_size	461
11.113.4.2 hasInitializer	461
11.113.4.3 initialValue	461
11.113.4.4 is_global	461
11.113.4.5 name	462
11.113.4.6 object	462
11.113.4.7 type	462
11.114 pascal::VariableNode Class Reference	462
11.114.1 Detailed Description	463
11.114.2 Constructor & Destructor Documentation	463

11.114.2.1 VariableNode()	463
11.114.3 Member Function Documentation	463
11.114.3.1 accept()	463
11.114.3.2 toString()	463
11.114.4 Member Data Documentation	463
11.114.4.1 name	463
11.115 pascal::VariantArm Struct Reference	463
11.115.1 Detailed Description	464
11.115.2 Member Data Documentation	464
11.115.2.1 caseLabels	464
11.115.2.2 fields	464
11.116 mxvm::Validator::VarNamePair Struct Reference	464
11.116.1 Detailed Description	464
11.116.2 Member Data Documentation	464
11.116.2.1 first	464
11.116.2.2 second	465
11.117 pascal::WhileStmtNode Class Reference	465
11.117.1 Detailed Description	466
11.117.2 Constructor & Destructor Documentation	466
11.117.2.1 WhileStmtNode()	466
11.117.3 Member Function Documentation	466
11.117.3.1 accept()	466
11.117.3.2 toString()	466
11.117.4 Member Data Documentation	466
11.117.4.1 condition	466
11.117.4.2 statement	466
11.118 pascal::WithStmtNode Class Reference	466
11.118.1 Detailed Description	467
11.118.2 Constructor & Destructor Documentation	467
11.118.2.1 WithStmtNode()	467
11.118.3 Member Function Documentation	468
11.118.3.1 accept()	468
11.118.3.2 toString()	468
11.118.4 Member Data Documentation	468
11.118.4.1 recordVar	468
11.118.4.2 statement	468
11.119 mx::XParser Class Reference	468
11.119.1 Detailed Description	469
11.119.2 Constructor & Destructor Documentation	469
11.119.2.1 XParser() [1/2]	469
11.119.2.2 XParser() [2/2]	469
11.119.3 Member Function Documentation	470

11.119.3.1 next()	470
11.119.3.2 peekIs() [1/2]	470
11.119.3.3 peekIs() [2/2]	470
11.119.3.4 tokenTypeToString()	470
11.119.4 Member Data Documentation	471
11.119.4.1 filename	471
11.119.4.2 index	471
11.119.4.3 scanner	471
11.119.4.4 token	471
12 File Documentation	473
12.1 include/mxvm/ast.hpp File Reference	473
12.1.1 Detailed Description	474
12.2 ast.hpp	474
12.3 src/frontend/include/ast.hpp File Reference	476
12.3.1 Detailed Description	479
12.4 ast.hpp	480
12.5 include/mxvm/function.hpp File Reference	488
12.5.1 Detailed Description	489
12.6 function.hpp	489
12.7 include/mxvm/html_gen.hpp File Reference	490
12.7.1 Detailed Description	491
12.8 html_gen.hpp	491
12.9 include/mxvm/icode.hpp File Reference	491
12.9.1 Detailed Description	492
12.10 icode.hpp	493
12.11 src/frontend/include/icode.hpp File Reference	499
12.11.1 Detailed Description	500
12.11.2 Macro Definition Documentation	500
12.11.2.1 MXVM_BOUNDS_CHECK	500
12.12 icode.hpp	500
12.13 include/mxvm/instruct.hpp File Reference	526
12.13.1 Detailed Description	528
12.13.2 Enumeration Type Documentation	528
12.13.2.1 Inc	528
12.13.3 Function Documentation	529
12.13.3.1 operator<<() [1/4]	529
12.13.3.2 operator<<() [2/4]	530
12.13.3.3 operator<<() [3/4]	530
12.13.3.4 operator<<() [4/4]	530
12.13.4 Variable Documentation	531
12.13.4.1 IncType	531

12.14 instruct.hpp	531
12.15 include/mxvm/mxvm.hpp File Reference	534
12.15.1 Detailed Description	535
12.16 mxvm.hpp	535
12.17 include/mxvm/parser.hpp File Reference	535
12.17.1 Detailed Description	537
12.18 parser.hpp	537
12.19 src/frontend/include/parser.hpp File Reference	539
12.19.1 Detailed Description	540
12.20 parser.hpp	540
12.21 include/mxvm/valid.hpp File Reference	542
12.21.1 Detailed Description	544
12.22 valid.hpp	544
12.23 include/mxvm/version_info.hpp File Reference	545
12.23.1 Macro Definition Documentation	545
12.23.1.1 VERSION_INFO	545
12.24 version_info.hpp	546
12.25 mainpage.md File Reference	546
12.26 modules/io/io.c File Reference	546
12.26.1 Detailed Description	546
12.26.2 Function Documentation	546
12.26.2.1 fsize()	546
12.26.2.2 mxvm_fgets()	547
12.26.2.3 rand_number()	547
12.26.2.4 seed_random()	547
12.27 io.c	547
12.28 modules/io/io.cpp File Reference	547
12.28.1 Detailed Description	548
12.28.2 Function Documentation	548
12.28.2.1 mxvm_io_fclose()	548
12.28.2.2 mxvm_io_feof()	548
12.28.2.3 mxvm_io_fopen()	549
12.28.2.4 mxvm_io_fprintf()	549
12.28.2.5 mxvm_io_fputs()	549
12.28.2.6 mxvm_io_fread()	549
12.28.2.7 mxvm_io_fseek()	549
12.28.2.8 mxvm_io_fsize()	549
12.28.2.9 mxvm_io_fwrite()	550
12.28.2.10 mxvm_io_mxvm_fgets()	550
12.28.2.11 mxvm_io_rand_number()	550
12.28.2.12 mxvm_io_seed_random()	550
12.29 io.cpp	550

12.30 modules/sdl/mx_sdl.h File Reference	554
12.30.1 Detailed Description	558
12.30.2 Function Documentation	558
12.30.2.1 blit_surface()	558
12.30.2.2 clear()	558
12.30.2.3 clear_queued_audio()	558
12.30.2.4 close_audio()	559
12.30.2.5 create_render_target()	559
12.30.2.6 create_renderer()	559
12.30.2.7 create_rgb_surface()	559
12.30.2.8 create_texture()	560
12.30.2.9 create_window()	560
12.30.2.10 delay()	561
12.30.2.11 destroy_render_target()	561
12.30.2.12 destroy_renderer()	561
12.30.2.13 destroy_texture()	561
12.30.2.14 destroy_window()	561
12.30.2.15 draw_line()	561
12.30.2.16 draw_point()	561
12.30.2.17 draw_rect()	562
12.30.2.18 draw_text()	562
12.30.2.19 fill_rect()	562
12.30.2.20 free_surface()	562
12.30.2.21 free_wav()	563
12.30.2.22 get_clipboard_text()	563
12.30.2.23 get_event_type()	563
12.30.2.24 get_key_code()	563
12.30.2.25 get_keyboard_state()	563
12.30.2.26 get_mouse_button()	563
12.30.2.27 get_mouse_buttons()	564
12.30.2.28 get_mouse_state()	564
12.30.2.29 get_mouse_x()	564
12.30.2.30 get_mouse_y()	564
12.30.2.31 get_num_keys()	564
12.30.2.32 get_queued_audio_size()	564
12.30.2.33 get_relative_mouse_buttons()	565
12.30.2.34 get_relative_mouse_state()	565
12.30.2.35 get_relative_mouse_x()	565
12.30.2.36 get_relative_mouse_y()	565
12.30.2.37 get_renderer_output_size()	565
12.30.2.38 get_ticks()	566
12.30.2.39 get_window_size()	566

12.30.2.40	init()	566
12.30.2.41	init_text()	566
12.30.2.42	is_key_pressed()	566
12.30.2.43	load_font()	567
12.30.2.44	load_texture()	567
12.30.2.45	load_texture_color_key()	568
12.30.2.46	load_texture_color_key_rgb()	568
12.30.2.47	load_wav()	568
12.30.2.48	lock_texture()	569
12.30.2.49	open_audio()	569
12.30.2.50	pause_audio()	570
12.30.2.51	poll_event()	570
12.30.2.52	present()	570
12.30.2.53	present_scaled()	570
12.30.2.54	present_stretched()	570
12.30.2.55	queue_audio()	570
12.30.2.56	quit()	571
12.30.2.57	quit_text()	571
12.30.2.58	render_texture()	571
12.30.2.59	set_clipboard_text()	571
12.30.2.60	set_draw_color()	571
12.30.2.61	set_render_target()	572
12.30.2.62	set_window_fullscreen()	572
12.30.2.63	set_window_icon()	572
12.30.2.64	set_window_position()	572
12.30.2.65	set_window_title()	572
12.30.2.66	show_cursor()	572
12.30.2.67	unlock_texture()	573
12.30.2.68	update_texture()	573
12.31	mx_sdl.h	573
12.32	modules/sdl/sdl.c File Reference	575
12.32.1	Detailed Description	578
12.32.2	Function Documentation	579
12.32.2.1	blit_surface()	579
12.32.2.2	clear()	579
12.32.2.3	clear_queued_audio()	579
12.32.2.4	close_audio()	579
12.32.2.5	create_render_target()	579
12.32.2.6	create_renderer()	579
12.32.2.7	create_rgb_surface()	580
12.32.2.8	create_texture()	580
12.32.2.9	create_window()	581

12.32.2.10 delay()	581
12.32.2.11 destroy_render_target()	581
12.32.2.12 destroy_renderer()	581
12.32.2.13 destroy_texture()	581
12.32.2.14 destroy_window()	582
12.32.2.15 draw_line()	582
12.32.2.16 draw_point()	582
12.32.2.17 draw_rect()	582
12.32.2.18 draw_text()	582
12.32.2.19 fill_rect()	583
12.32.2.20 free_surface()	583
12.32.2.21 free_wav()	583
12.32.2.22 get_clipboard_text()	583
12.32.2.23 get_event_type()	583
12.32.2.24 get_key_code()	583
12.32.2.25 get_keyboard_state()	584
12.32.2.26 get_mouse_button()	584
12.32.2.27 get_mouse_buttons()	584
12.32.2.28 get_mouse_state()	584
12.32.2.29 get_mouse_x()	585
12.32.2.30 get_mouse_y()	585
12.32.2.31 get_num_keys()	585
12.32.2.32 get_queued_audio_size()	585
12.32.2.33 get_relative_mouse_buttons()	585
12.32.2.34 get_relative_mouse_state()	585
12.32.2.35 get_relative_mouse_x()	586
12.32.2.36 get_relative_mouse_y()	586
12.32.2.37 get_renderer_output_size()	586
12.32.2.38 get_ticks()	586
12.32.2.39 get_window_size()	586
12.32.2.40 init()	587
12.32.2.41 init_text()	587
12.32.2.42 is_key_pressed()	587
12.32.2.43 load_font()	587
12.32.2.44 load_texture()	588
12.32.2.45 load_texture_color_key()	588
12.32.2.46 load_texture_color_key_rgb()	588
12.32.2.47 load_wav()	589
12.32.2.48 lock_texture()	589
12.32.2.49 open_audio()	589
12.32.2.50 pause_audio()	590
12.32.2.51 poll_event()	590

12.32.2.52	present()	590
12.32.2.53	present_scaled()	590
12.32.2.54	present_stretched()	591
12.32.2.55	queue_audio()	591
12.32.2.56	quit()	591
12.32.2.57	quit_text()	591
12.32.2.58	render_texture()	591
12.32.2.59	set_clipboard_text()	592
12.32.2.60	set_draw_color()	592
12.32.2.61	set_render_target()	592
12.32.2.62	set_window_fullscreen()	592
12.32.2.63	set_window_icon()	592
12.32.2.64	set_window_position()	592
12.32.2.65	set_window_title()	593
12.32.2.66	show_cursor()	593
12.32.2.67	unlock_texture()	593
12.32.2.68	update_texture()	593
12.32.3	Variable Documentation	593
12.32.3.1	g_event	593
12.32.3.2	g_font_count	593
12.32.3.3	g_fonts	593
12.32.3.4	g_render_target_count	594
12.32.3.5	g_render_targets	594
12.32.3.6	g_renderer_count	594
12.32.3.7	g_renderers	594
12.32.3.8	g_target_heights	594
12.32.3.9	g_target_widths	594
12.32.3.10	g_texture_count	594
12.32.3.11	g_textures	594
12.32.3.12	g_window_count	594
12.32.3.13	g_windows	595
12.33	sdl.c	595
12.34	modules/sdl/sdl.cpp File Reference	603
12.34.1	Detailed Description	605
12.34.2	Function Documentation	605
12.34.2.1	mxvm_sdl_blit_surface()	605
12.34.2.2	mxvm_sdl_clear()	605
12.34.2.3	mxvm_sdl_clear_queued_audio()	605
12.34.2.4	mxvm_sdl_close_audio()	605
12.34.2.5	mxvm_sdl_create_render_target()	605
12.34.2.6	mxvm_sdl_create_renderer()	605
12.34.2.7	mxvm_sdl_create_rgb_surface()	606

12.34.2.8	mxvm_sdl_create_texture()	606
12.34.2.9	mxvm_sdl_create_window()	606
12.34.2.10	mxvm_sdl_delay()	606
12.34.2.11	mxvm_sdl_destroy_render_target()	606
12.34.2.12	mxvm_sdl_destroy_renderer()	606
12.34.2.13	mxvm_sdl_destroy_texture()	607
12.34.2.14	mxvm_sdl_destroy_window()	607
12.34.2.15	mxvm_sdl_draw_line()	607
12.34.2.16	mxvm_sdl_draw_point()	607
12.34.2.17	mxvm_sdl_draw_rect()	607
12.34.2.18	mxvm_sdl_draw_text()	607
12.34.2.19	mxvm_sdl_fill_rect()	607
12.34.2.20	mxvm_sdl_free_surface()	608
12.34.2.21	mxvm_sdl_free_wav()	608
12.34.2.22	mxvm_sdl_get_clipboard_text()	608
12.34.2.23	mxvm_sdl_get_event_type()	608
12.34.2.24	mxvm_sdl_get_key_code()	608
12.34.2.25	mxvm_sdl_get_keyboard_state()	608
12.34.2.26	mxvm_sdl_get_mouse_button()	609
12.34.2.27	mxvm_sdl_get_mouse_buttons()	609
12.34.2.28	mxvm_sdl_get_mouse_state()	609
12.34.2.29	mxvm_sdl_get_mouse_x()	609
12.34.2.30	mxvm_sdl_get_mouse_y()	609
12.34.2.31	mxvm_sdl_get_num_keys()	609
12.34.2.32	mxvm_sdl_get_queued_audio_size()	609
12.34.2.33	mxvm_sdl_get_relative_mouse_buttons()	610
12.34.2.34	mxvm_sdl_get_relative_mouse_state()	610
12.34.2.35	mxvm_sdl_get_relative_mouse_x()	610
12.34.2.36	mxvm_sdl_get_relative_mouse_y()	610
12.34.2.37	mxvm_sdl_get_renderer_output_size()	610
12.34.2.38	mxvm_sdl_get_ticks()	610
12.34.2.39	mxvm_sdl_get_window_size()	610
12.34.2.40	mxvm_sdl_init()	611
12.34.2.41	mxvm_sdl_init_text()	611
12.34.2.42	mxvm_sdl_is_key_pressed()	611
12.34.2.43	mxvm_sdl_load_font()	611
12.34.2.44	mxvm_sdl_load_texture()	611
12.34.2.45	mxvm_sdl_load_texture_color_key()	611
12.34.2.46	mxvm_sdl_load_texture_color_key_rgb()	611
12.34.2.47	mxvm_sdl_load_wav()	612
12.34.2.48	mxvm_sdl_lock_texture()	612
12.34.2.49	mxvm_sdl_open_audio()	612

12.34.2.50 mxvm_sdl_pause_audio()	612
12.34.2.51 mxvm_sdl_poll_event()	612
12.34.2.52 mxvm_sdl_present()	613
12.34.2.53 mxvm_sdl_present_scaled()	613
12.34.2.54 mxvm_sdl_present_stretched()	613
12.34.2.55 mxvm_sdl_queue_audio()	613
12.34.2.56 mxvm_sdl_quit()	613
12.34.2.57 mxvm_sdl_quit_text()	613
12.34.2.58 mxvm_sdl_render_texture()	613
12.34.2.59 mxvm_sdl_set_clipboard_text()	614
12.34.2.60 mxvm_sdl_set_draw_color()	614
12.34.2.61 mxvm_sdl_set_render_target()	614
12.34.2.62 mxvm_sdl_set_window_fullscreen()	614
12.34.2.63 mxvm_sdl_set_window_icon()	614
12.34.2.64 mxvm_sdl_set_window_position()	614
12.34.2.65 mxvm_sdl_set_window_title()	615
12.34.2.66 mxvm_sdl_show_cursor()	615
12.34.2.67 mxvm_sdl_unlock_texture()	615
12.34.2.68 mxvm_sdl_update_texture()	615
12.35 sdl.cpp	615
12.36 modules/std/mx_std.h File Reference	629
12.36.1 Detailed Description	630
12.36.2 Function Documentation	630
12.36.2.1 argc()	630
12.36.2.2 argv()	630
12.36.2.3 free_program_args()	630
12.36.2.4 set_program_args()	630
12.37 mx_std.h	631
12.38 modules/std/std.c File Reference	631
12.38.1 Detailed Description	632
12.38.2 Function Documentation	632
12.38.2.1 argc()	632
12.38.2.2 argv()	632
12.38.2.3 float_to_int()	633
12.38.2.4 free_program_args()	633
12.38.2.5 int_to_float()	633
12.38.2.6 mx_strdup_c()	633
12.38.2.7 release()	633
12.38.2.8 set_program_args()	633
12.38.3 Variable Documentation	633
12.38.3.1 g_argc	633
12.38.3.2 g_args_registered	634

12.38.3.3 g_argv	634
12.39 std.c	634
12.40 modules/std/std.cpp File Reference	635
12.40.1 Detailed Description	636
12.40.2 Function Documentation	636
12.40.2.1 mxvm_std_abs()	636
12.40.2.2 mxvm_std_acos()	637
12.40.2.3 mxvm_std_argc()	637
12.40.2.4 mxvm_std_argv()	637
12.40.2.5 mxvm_std_asin()	637
12.40.2.6 mxvm_std_atan()	637
12.40.2.7 mxvm_std_atan2()	637
12.40.2.8 mxvm_std_atof()	637
12.40.2.9 mxvm_std_atoi()	638
12.40.2.10 mxvm_std_calloc()	638
12.40.2.11 mxvm_std_ceil()	638
12.40.2.12 mxvm_std_cos()	638
12.40.2.13 mxvm_std_cosh()	638
12.40.2.14 mxvm_std_exit()	638
12.40.2.15 mxvm_std_exp()	639
12.40.2.16 mxvm_std_exp2()	639
12.40.2.17 mxvm_std_fabs()	639
12.40.2.18 mxvm_std_float_to_int()	639
12.40.2.19 mxvm_std_floor()	639
12.40.2.20 mxvm_std_fmod()	639
12.40.2.21 mxvm_std_free_program_args()	640
12.40.2.22 mxvm_std_hypot()	640
12.40.2.23 mxvm_std_int_to_float()	640
12.40.2.24 mxvm_std_isalpha()	640
12.40.2.25 mxvm_std_isdigit()	640
12.40.2.26 mxvm_std_isspace()	640
12.40.2.27 mxvm_std_log()	640
12.40.2.28 mxvm_std_log10()	641
12.40.2.29 mxvm_std_log2()	641
12.40.2.30 mxvm_std_malloc()	641
12.40.2.31 mxvm_std_memcmp()	641
12.40.2.32 mxvm_std_memcpy()	641
12.40.2.33 mxvm_std_memmove()	641
12.40.2.34 mxvm_std_memset()	642
12.40.2.35 mxvm_std_pow()	642
12.40.2.36 mxvm_std_rand()	642
12.40.2.37 mxvm_std_release()	642

12.40.2.38 mxvm_std_round()	642
12.40.2.39 mxvm_std_set_program_args()	642
12.40.2.40 mxvm_std_sin()	643
12.40.2.41 mxvm_std_sinh()	643
12.40.2.42 mxvm_std_sqrt()	643
12.40.2.43 mxvm_std_srand()	643
12.40.2.44 mxvm_std_system()	643
12.40.2.45 mxvm_std_tan()	643
12.40.2.46 mxvm_std_tanh()	644
12.40.2.47 mxvm_std_tolower()	644
12.40.2.48 mxvm_std_toupper()	644
12.40.2.49 mxvm_std_trunc()	644
12.41 std.cpp	644
12.42 modules/string/cstring.c File Reference	655
12.42.1 Detailed Description	656
12.42.2 Function Documentation	656
12.42.2.1 copy()	656
12.42.2.2 delete_str()	656
12.42.2.3 insert()	656
12.42.2.4 intostr()	656
12.42.2.5 pos()	656
12.42.2.6 strat()	656
12.42.2.7 strfind()	657
12.42.2.8 strtoint()	657
12.42.2.9 substr()	657
12.43 cstring.c	657
12.44 modules/string/string.cpp File Reference	659
12.44.1 Detailed Description	660
12.44.2 Function Documentation	660
12.44.2.1 asReadPtr()	660
12.44.2.2 asWritePtr()	660
12.44.2.3 getStringFromVar()	660
12.44.2.4 isStringLike()	660
12.44.2.5 mxvm_string_copy()	660
12.44.2.6 mxvm_string_delete()	661
12.44.2.7 mxvm_string_insert()	661
12.44.2.8 mxvm_string_inttostr()	661
12.44.2.9 mxvm_string_pos()	661
12.44.2.10 mxvm_string_snprintf()	661
12.44.2.11 mxvm_string_strat()	661
12.44.2.12 mxvm_string_strcmp()	662
12.44.2.13 mxvm_string_strfind()	662

12.44.2.14 mxvm_string_strlen()	662
12.44.2.15 mxvm_string_strncat()	662
12.44.2.16 mxvm_string_strncpy()	662
12.44.2.17 mxvm_string_strtoint()	662
12.44.2.18 mxvm_string_substr()	663
12.45 string.cpp	663
12.46 src/ast.cpp File Reference	669
12.46.1 Detailed Description	669
12.47 ast.cpp	670
12.48 src/frontend/ast.cpp File Reference	671
12.48.1 Detailed Description	671
12.49 ast.cpp	672
12.50 src/frontend/expr.cpp File Reference	677
12.50.1 Detailed Description	678
12.51 expr.cpp	678
12.52 src/frontend/icode.cpp File Reference	678
12.52.1 Detailed Description	679
12.53 icode.cpp	679
12.54 src/icode.cpp File Reference	687
12.54.1 Detailed Description	688
12.55 icode.cpp	688
12.56 src/frontend/icode_gen.cpp File Reference	693
12.57 icode_gen.cpp	693
12.58 src/icode_gen.cpp File Reference	693
12.58.1 Detailed Description	694
12.59 icode_gen.cpp	694
12.60 src/frontend/icode_imp.cpp File Reference	717
12.60.1 Detailed Description	718
12.61 icode_imp.cpp	718
12.62 src/frontend/icode_visit.cpp File Reference	731
12.62.1 Detailed Description	732
12.63 icode_visit.cpp	732
12.64 src/frontend/include/expr.hpp File Reference	763
12.64.1 Detailed Description	763
12.65 expr.hpp	764
12.66 src/scanner/include/scanner/expr.hpp File Reference	764
12.66.1 Detailed Description	765
12.67 expr.hpp	765
12.68 src/frontend/include/types.hpp File Reference	768
12.68.1 Detailed Description	769
12.69 types.hpp	769
12.70 src/scanner/include/scanner/types.hpp File Reference	770

12.70.1 Detailed Description	772
12.71 types.hpp	772
12.72 src/frontend/include/validator.hpp File Reference	774
12.72.1 Detailed Description	775
12.73 validator.hpp	776
12.74 src/frontend/main.cpp File Reference	777
12.74.1 Detailed Description	778
12.74.2 Function Documentation	778
12.74.2.1 compileFile()	778
12.74.2.2 extractPasName()	778
12.74.2.3 main()	778
12.74.2.4 removeComments()	779
12.75 main.cpp	779
12.76 src/vm/main.cpp File Reference	783
12.76.1 Detailed Description	784
12.76.2 Enumeration Type Documentation	784
12.76.2.1 vm_action	784
12.76.2.2 vm_target	785
12.76.3 Function Documentation	785
12.76.3.1 action_interpret()	785
12.76.3.2 action_translate()	785
12.76.3.3 collectAndRegisterAllExterns()	785
12.76.3.4 createMakefile()	785
12.76.3.5 main()	785
12.76.3.6 operator<<() [1/2]	786
12.76.3.7 operator<<() [2/2]	786
12.76.3.8 print_help()	786
12.76.3.9 proc_args()	786
12.76.3.10 process_arguments()	786
12.76.3.11 signal_action()	786
12.76.3.12 translate_x64()	787
12.76.4 Variable Documentation	787
12.76.4.1 signal_program	787
12.77 main.cpp	787
12.78 src/frontend/parser.cpp File Reference	797
12.78.1 Detailed Description	797
12.79 parser.cpp	797
12.80 src/parser.cpp File Reference	813
12.80.1 Detailed Description	814
12.81 parser.cpp	814
12.82 src/frontend/validator.cpp File Reference	836
12.82.1 Detailed Description	836

12.83 validator.cpp	836
12.84 src/function.cpp File Reference	854
12.84.1 Detailed Description	854
12.85 function.cpp	855
12.86 src/html_gen.cpp File Reference	855
12.86.1 Detailed Description	855
12.87 html_gen.cpp	856
12.88 src/html_gen/html.cpp File Reference	862
12.88.1 Detailed Description	862
12.88.2 Function Documentation	862
12.88.2.1 dirOf()	862
12.88.2.2 endsWith()	863
12.88.2.3 ensureCss()	863
12.88.2.4 extractDeclName()	863
12.88.2.5 fileExists()	863
12.88.2.6 main()	863
12.88.2.7 readFile()	863
12.88.2.8 writeHtml()	863
12.89 html.cpp	864
12.90 src/icode_exec.cpp File Reference	866
12.90.1 Detailed Description	867
12.91 icode_exec.cpp	867
12.92 src/icode_gen_x64.cpp File Reference	893
12.92.1 Detailed Description	894
12.93 icode_gen_x64.cpp	894
12.94 src/icode_opt.cpp File Reference	914
12.94.1 Detailed Description	915
12.95 icode_opt.cpp	915
12.96 src/instruct.cpp File Reference	921
12.96.1 Detailed Description	922
12.96.2 Function Documentation	922
12.96.2.1 operator<<() [1/4]	922
12.96.2.2 operator<<() [2/4]	923
12.96.2.3 operator<<() [3/4]	923
12.96.2.4 operator<<() [4/4]	923
12.97 instruct.cpp	924
12.98 src/mxvm.cpp File Reference	925
12.98.1 Detailed Description	925
12.99 mxvm.cpp	925
12.100 src/scanner/exception.cpp File Reference	926
12.100.1 Detailed Description	926
12.101 exception.cpp	927

12.102 src/scanner/include/scanner/exception.hpp File Reference	928
12.102.1 Detailed Description	930
12.102.2 Function Documentation	930
12.102.2.1 Col()	930
12.103 exception.hpp	930
12.104 src/scanner/include/scanner/scanner.hpp File Reference	931
12.104.1 Detailed Description	933
12.104.2 Function Documentation	933
12.104.2.1 operator<<() [1/2]	933
12.104.2.2 operator<<() [2/2]	933
12.105 scanner.hpp	933
12.106 src/scanner/include/scanner/string_buffer.hpp File Reference	934
12.106.1 Detailed Description	935
12.107 string_buffer.hpp	935
12.108 src/scanner/scanner.cpp File Reference	938
12.108.1 Detailed Description	939
12.109 scanner.cpp	939
12.110 src/scanner/string_buffer.cpp File Reference	945
12.110.1 Detailed Description	945
12.111 string_buffer.cpp	946
12.112 src/scanner/types.cpp File Reference	946
12.112.1 Detailed Description	947
12.113 types.cpp	947
12.114 src/valid.cpp File Reference	947
12.114.1 Detailed Description	948
12.115 valid.cpp	948
12.116 src/vm/include/argz.hpp File Reference	958
12.116.1 Detailed Description	959
12.117 argz.hpp	959

Index	967
--------------	------------

Chapter 1

MXVM – Virtual Machine, Compiler & Code Generator

MXVM is a custom virtual machine and compiler suite written in modern C++20. It provides a RISC-style bytecode language with an interpreter, a Pascal-subset compiler frontend, and native x86-64 code generators targeting Linux (System V ABI), macOS (Darwin), and Windows (Win64).

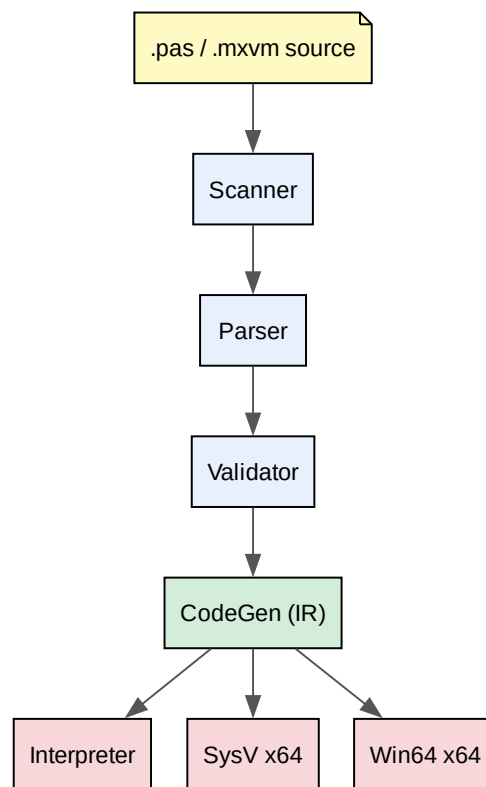
Experimental / educational project – not intended for production use.

[Download PDF Reference Manual](#)

[View Examples Online](#)

1.1 Architecture Overview

Layer	Component	Description
Frontend	Scanner / Lexer	Tokenises <code>.mxvm</code> and <code>.pas</code> source files
Frontend	Parser / AST	Builds a typed abstract syntax tree
Frontend	Validator	Scope-based semantic analysis and type checking
Middle	Code Generator	Emits MXVM bytecode from the AST
Backend	Interpreter	Stack-based execution engine for bytecode
Backend	x86-64 Codegen	Native assembly output (System V, Win64, Darwin)
Backend	Peephole Optimizer	Post-generation assembly cleanup passes
Runtime	Module System	Dynamically loaded C shared libraries (<code>io</code> , <code>std</code> , <code>string</code> , <code>sdl</code>) and Pascal unit linking
Tools	mxvm-html	Converts <code>.mxvm</code> and <code>.pas</code> source files to syntax-highlighted HTML (Borland Delphi colour scheme)



1.2 Building

```

git clone https://github.com/lostjared/MXVM.git
cd MXVM && mkdir build && cd build
cmake .. && make -j$(nproc)

```

Requires: C++20 compiler, CMake >= 3.10.

Optional: SDL2 + SDL2_ttf (for the SDL module), Emscripten (WebAssembly target).

1.3 Running

Note: The `.mxvm` extension is optional — if you omit it, `mxvmc` will append `.mxvm` automatically (e.g. `mxvmc program` is equivalent to `mxvmc program.mxvm`).

Mode	Command
Interpret	<code>mxvmc program.mxvm --path /usr/local/lib</code>
Compile -> Assembly	<code>mxvmc program.mxvm --path /usr/local/lib --action translate</code>
Compile -> Executable	<code>mxvmc program.mxvm --path /usr/local/lib --action compile</code>

1.3.1 mxx Pascal Compiler Arguments

Mode	Command	Description
Legacy	<code>mxx source.pas output.mxvm</code>	Compile a single file to a named output
Explicit I/O	<code>mxx -i source.pas -o output.↵ mxvm</code>	Explicit input/output flags; <code>-o</code> is optional — when omitted, the output filename is derived from the <code>program</code> or <code>unit</code> name declared inside the source file (e.g. <code>program Test; → Test.mxvm</code>)
Batch	<code>mxx -c source1.pas source2.pas ...</code>	Compile multiple files; each output filename is derived from the declared <code>program/unit</code> name inside the file

```
# Legacy positional arguments (explicit output name)
mxx program.pas program.mxvm

# Explicit input/output flags
mxx -i program.pas -o program.mxvm
mxx -i program.pas      # reads 'program MyApp;' inside -> MyApp.mxvm

# Batch compile: output names come from the declared program/unit names
mxx -c unit1.pas unit2.pas main.pas
# e.g. 'unit MathUtils;' -> MathUtils.mxvm, 'program TestMain;' -> TestMain.mxvm
```

1.4 mxvm-html — Syntax Highlighter

`mxvm-html` converts `.mxvm` bytecode and `.pas` Pascal source files to self-contained, syntax-highlighted HTML pages using a Borland Delphi 7 colour scheme. The generated pages embed all CSS inline and work without a web server. An optional external stylesheet can override the defaults.

Installed location: `/usr/local/bin/mxvm-html`

1.4.1 Usage

Mode	Command
Single file	<code>mxvm-html -i <file> [-o <output.html>] [-s <stylesheet.css>]</code>
Batch	<code>mxvm-html -c <file1> [file2 ...]</code>
Help	<code>mxvm-html -h</code>

1.4.1.1 Options

Flag	Argument	Description
<code>-i</code>	<code><file></code>	Input file (<code>.mxvm</code> or <code>.pas</code>)
<code>-o</code>	<code><file.html></code>	Output HTML file. Defaults to auto-named (see below).
<code>-s</code>	<code><stylesheet.css></code>	External CSS href embedded in the <code><link></code> tag. Defaults to <code>mxvm-highlight.css</code> .
<code>-c</code>	<code><file1> [file2 ...]</code>	Batch mode — convert multiple files in one invocation.
<code>-h</code>	—	Print usage and exit.

1.4.1.2 Auto-naming

When `-o` is omitted, the output name is derived from the first top-level declaration in the source file:

Keyword	Example	Output
program	program HelloWorld;	HelloWorld.html
unit	unit MathUtils;	MathUtils.html
object	object Stack;	Stack.html

If no declaration is found the output falls back to `<input>.html`.

1.4.2 Examples

```
# Highlight a single Pascal source file
mxvm-html -i hello.pas -o hello.html

# Auto-name from 'program HelloWorld;' declaration
mxvm-html -i hello.pas           # -> HelloWorld.html

# Full pipeline: compile, then highlight both source and bytecode
mxx -i program.pas -o program.mxvm
mxvm-html -i program.pas -o program.pas.html
mxvm-html -i program.mxvm -o program.mxvm.html

# Batch-convert a directory of Pascal files
mxvm-html -c examples/*.pas

# Use a custom stylesheet
mxvm-html -i hello.pas -o hello.html -s ../../mxvm-highlight.css
```

1.4.3 CSS Customisation

Each output page contains an inline `<style>` block (Borland Delphi defaults) followed by a `<link>` to an external stylesheet. Rules in the external file take precedence via normal CSS cascade. The relevant span classes are:

Class	Applies to
<code>.kw</code>	Reserved keywords / instructions
<code>.typ</code>	Built-in types
<code>.bi</code>	Built-in routines and constants (Pascal only)
<code>.str</code>	String and character literals
<code>.num</code>	Numeric literals
<code>.com</code>	Comments
<code>.lbl</code>	Label declarations (<code>.mxvm</code> only)
<code>.op</code>	Operators / punctuation (<code>.mxvm</code> only)
<code>.id</code>	All other identifiers

See <docs/mxvm-html.html> for the full reference and a dark-theme CSS example.

1.5 MXVM Bytecode Language Reference

1.5.1 Program Structure

Every `.mxvm` file is wrapped in a `program` (or `object`) block containing three sections: **data**, **module**, and **code**.

```
program MyApp {
    section data {
```

```

int    counter = 0
float  pi      = 3.14159
string msg     = "Hello, MXVM!\n"
ptr    buffer  = null
byte   flags   = 0
}

section module { io, string, std }

section code {
start:
  print msg
  add counter, counter, 1
  cmp counter, 10
  jl  start
  done
}
}

```

1.5.1.1 Data Section – Variable Types

Type	Size	Description
int	64-bit	Signed integer
float	64-bit	Double-precision floating point
string	varies	NUL-terminated string with optional max-length: <code>string name, 256</code>
ptr	64-bit	Opaque pointer (for <code>alloc/module</code> use)
byte	8-bit	Unsigned byte
label	–	Code address (for indirect jumps)

- Variables are declared with an optional initialiser: `int x = 42`.
- All string literals must live in the data section – inline string constants are **not** allowed in the code section.
- Hex immediates are supported in code: `mov x, 0xFF`.

1.5.1.2 Module Section

```
section module { io, string, std }
```

Lists the C shared-library modules whose functions may be called via `invoke`. See the [Module Reference](#) below.

1.5.1.3 Code Section

The code section is a flat sequence of labeled instructions.

- **One instruction per line.**
- **Destination-first operand order** (RISC convention): `add dest, src1, src2`.
- Labels end with `:` and may be marked as functions with `function:`

```

my_func: function
; ... body ...
ret

```

1.5.2 Instruction Set

1.5.2.1 Arithmetic & Logic

Instruction	Operands	Effect
<code>mov</code>	<code>dest, src</code>	<code>dest <- src</code>

Instruction	Operands	Effect
add	dest, a, b	dest $\leftarrow$ a + b (2-operand form: dest += a)
sub	dest, a, b	dest $\leftarrow$ a - b
mul	dest, a, b	dest $\leftarrow$ a * b
div	dest, a, b	dest $\leftarrow$ a / b (integer division for int operands)
mod	dest, a, b	dest $\leftarrow$ a mod b
neg	dest, a	dest $\leftarrow$ -a
or	dest, a, b	Bitwise OR
and	dest, a, b	Bitwise AND
xor	dest, a, b	Bitwise XOR
not	dest, a	Bitwise NOT

1.5.2.2 Comparison & Branching

Instruction	Operands	Effect
cmp	a, b	Set flags from a - b
fcmp	a, b	Floating-point compare, set flags
jmp	label	Unconditional jump
je / jne	label	Jump if equal / not equal
jl / jle	label	Jump if less / less-or-equal (signed)
jg / jge	label	Jump if greater / greater-or-equal (signed)
jz / jnz	label	Jump if zero flag set / clear
ja / jb	label	Jump if above / below (unsigned)
jae / jbe	label	Jump if above-or-equal / below-or-equal (unsigned)
jc / jnc	label	Jump if carry / no carry
jp / jnp	label	Jump if parity / no parity
jo / jno	label	Jump if overflow / no overflow
js / jns	label	Jump if sign / no sign

1.5.2.3 Memory & Pointers

Instruction	Operands	Effect
load	dest, ptr, index, size	dest $\leftarrow$ *(ptr + index * size)
store	src, ptr, index, size	*(ptr + index * size) $\leftarrow$ src
lea	dest, base, offset	Load effective address
alloc	ptr, elem_size, count	ptr $\leftarrow$ calloc(count, elem_size)
realloc	ptr, elem_size, count	Resize ptr to hold count elements of elem_size bytes; zero-fills new space
free	ptr	Release allocated memory

1.5.2.4 Stack Operations

Instruction	Operands	Effect
push	src	Push value onto the VM stack
pop	dest	Pop top of stack into dest
stack_load	dest, offset	Read stack slot at offset
stack_store	src, offset	Write to stack slot at offset
stack_sub	amount	Reserve stack space

1.5.2.5 Control Flow & Calls

Instruction	Operands	Effect
call	label	Call an internal function (or <code>Object.Function</code>)
ret	–	Return from function
invoke	func, args...	Call an external module function (C ABI)
return	var	Capture the return value of the last <code>invoke</code>
done	–	Normal program termination
exit	code	Terminate with exit code

1.5.2.6 I/O

Instruction	Operands	Effect
print	fmt, args...	Printf-style formatted output
string_print	ptr	Print raw string at pointer
getline	ptr	Read a line from stdin into buffer

1.5.2.7 Type Conversion

Instruction	Operands	Effect
to_int	dest, ptr	Parse string -> integer
to_float	dest, ptr	Parse string -> float

1.5.2.8 Calling Convention – `invoke` / `return`

External functions loaded from modules are called with `invoke`. The return value is captured into a variable with the `return` pseudo-instruction immediately following the `invoke`:

```
invoke fopen, filename, mode
return file_handle
```

```
invoke strlen, my_string
return len
```

1.5.3 Object System

MXVM supports **objects** – separately compiled units that can be linked into a main program.

```
object Utils {
  section data { ... }
  section code {
    print_line: function
      print fmt
      ret
  }
}
```

```
}
}
```

The main program references object functions with dot notation:

```
call Utils.print_line
```

Objects can be **inline** (defined in the same file) or loaded from separate files via `--object-path`.

1.6 MXVM Pascal Frontend

MXVM includes a **Pascal-subset compiler** that parses Pascal source code, builds a typed AST, validates semantics with scope-based analysis, and emits MXVM bytecode.

1.6.1 Supported Language Features

1.6.1.1 Case Insensitivity

Like standard Pascal, all keywords, built-in routine names, and type names are **case-insensitive**. `BEGIN`, `Begin`, and `begin` are equivalent. User-defined identifiers (variable names, function names, unit names) are also resolved case-insensitively:

```
PROGRAM MixedCase;
VAR x: INTEGER;
BEGIN
  x := 42;
  WriteLn('x = ', x);
  WRITELN('works too');
END.
```

1.6.1.2 Program Structure

```
program HelloWorld;
uses io, std, strlib;

const
  MAX = 100;

type
  PInt = ^integer;
  Point = record
    x: integer;
    y: integer;
  end;
  IntArray = array[0..9] of integer;

var
  i: integer;
  name: string;
  flag: boolean;
  pt: Point;
  nums: IntArray;
  p: PInt;

procedure Greet(msg: string);
begin
  writeln(msg);
end;

function Add(a, b: integer): integer;
begin
  Add := a + b;
end;

begin
  name := 'MXVM Pascal';
  writeln('Hello from ', name);
  writeln('Sum = ', Add(3, 4));
  Greet('Welcome!');
end.
```

1.6.1.3 Uses Clause

The `uses` clause imports runtime modules and/or separately compiled units. It must appear immediately after the `program (or unit) declaration`:

```
program MyApp;
uses std, io, strlib, MathUtils;
```

Built-in runtime modules: `std`, `io`, `strlib`, `sdl`. Any other name is treated as a reference to a separately compiled Pascal **unit** (see [Units \(Separately Compiled Modules\)](#) below). If no `uses` clause is present, modules are auto-detected from builtin function usage.

1.6.1.4 Units (Separately Compiled Modules)

Pascal units allow you to split a project into separately compiled modules. A unit has an **interface** section (exported declarations visible to importers) and an **implementation** section (private code).

1.6.1.4.1 Unit Structure

```
unit MathUtils;

interface

function Add(a, b: integer): integer;
function Multiply(a, b: integer): integer;
procedure PrintResult(x: integer);

implementation

uses io;

function Add(a, b: integer): integer;
begin
    Add := a + b;
end;

function Multiply(a, b: integer): integer;
begin
    Multiply := a * b;
end;

procedure PrintResult(x: integer);
begin
    writeln(x);
end;

end.
```

1.6.1.4.2 Using a Unit

A program imports a unit by listing it in the `uses` clause alongside any runtime modules:

```
program TestMain;
uses io, MathUtils;

var
    result: integer;

begin
    result := MathUtils.Add(3, 4);
    writeln('3 + 4 = ', result);
    result := MathUtils.Multiply(5, 6);
    writeln('5 * 6 = ', result);
    MathUtils.PrintResult(99);
end.
```

1.6.1.4.3 Qualified Names

When calling functions or procedures from another unit, you may use **qualified names** (`UnitName.RoutineName`) to make the origin explicit:

```
result := MathUtils.Add(3, 4);    { qualified function call }
MathUtils.PrintResult(result);   { qualified procedure call }
MathUtils.PrintResult;           { zero-arg qualified call (parens optional) }
```

Unqualified calls are also accepted – the compiler resolves the name from the `uses` list:

```
result := Add(3, 4);              { unqualified -- still valid }
PrintResult(result);             { also valid }
```

Variables and **constants** from another unit **must** always be qualified:

```
writeln(MathUtils.counter);      { variable -- qualification required }
writeln(MathUtils.MAX_VALUE);    { constant -- qualification required }
```

Element	Qualified	Unqualified
Variables	Required	Not allowed
Constants	Required	Not allowed
Functions / Procedures	Allowed	Allowed

1.6.1.4.4 Compiling and Linking

Each unit is compiled to a separate `.mxvm` object file. The main program is compiled separately and references the unit via `section object`. At runtime the VM loads and links unit objects automatically.

```
# 1. Compile the unit
mxx MathUtils.pas MathUtils.mxvm

# 2. Compile the main program (unit .pas must be in the same directory)
mxx TestMain.pas TestMain.mxvm

# 3. Run -- tell the VM where to find object files
mxvmc TestMain.mxvm -x .
```

The `-x` (or `--object-path`) flag tells the VM where to search for `.mxvm` object files referenced by `section object` declarations. The default is `.` (current directory).

1.6.1.4.5 How It Works

Step	Detail
Unit compilation	The Pascal compiler parses the unit and emits an MXVM object block (instead of program). The interface forward declarations register function signatures; the implementation section generates the actual function bodies.
Program compilation	When compiling a program that uses a non-native module name, the compiler locates the unit's <code>.pas</code> file, parses its interface to learn function/procedure signatures, and emits qualified cross-object references (e.g. <code>call MathUtils.FUNC_Add, mov MathUtils.rbx, 3</code>).
VM linking	The VM's <code>section object { MathUtils }</code> triggers loading of <code>MathUtils.mxvm</code> from the object path. Object labels and variables are merged into the execution environment using qualified names (<code>MathUtils.FUNC_Add</code> , <code>MathUtils.rax</code> , etc.).

1.6.1.4.6 Interface Section

The interface section may contain:

- Procedure and function **forward declarations** (signature only, no body)
- Type declarations
- Constant declarations
- Variable declarations

1.6.1.4.7 Implementation Section

The implementation section may contain:

- An optional additional `uses` clause
- Full procedure and function definitions (with bodies)
- Additional type, constant, and variable declarations

1.6.1.5 Data Types

Type	Description
integer	64-bit signed integer
real	64-bit double-precision float
boolean	Stored as integer (0 or 1); <code>true</code> / <code>false</code> literals
char	Stored as integer (character ordinal value)
string	Heap-managed, pointer-based string
<code>^Type</code>	Typed pointer (e.g. <code>^integer</code> , <code>^Point</code>)
<code>array[lo..hi] of T</code>	Fixed-size array with arbitrary integer bounds
<code>array of T</code>	Dynamic array (resized at runtime with <code>SetLength</code>)
<code>record ... end</code>	Named record (struct) with typed fields; variant parts supported (<code>case tag: type of</code>)
<code>(id, id, ...)</code>	Enumerated type – constants mapped to integers starting from 0
<code>set of T</code>	Set of ordinal values (0..255); supports <code>in</code> , <code>+</code> , <code>-</code> , <code>*</code> , <code>include</code> , <code>exclude</code>
<code>file/file of T</code>	File handle for Pascal-style file I/O (<code>assign</code> , <code>reset</code> , <code>rewrite</code> , <code>append</code> , <code>close</code> , <code>eof</code>)

1.6.1.6 Constants

```
const
  MAX      = 100;
  PI       = 3.14159;
  GREETING = 'Hello';
  ENABLED  = true;
```

Built-in constants: `true`, `false`, `maxint`, `nil`.

1.6.1.7 Type Declarations

```
type
  Counter = integer;           { type alias }
  PInt    = ^integer;         { pointer type }
  IntArray = array[1..10] of integer;
  Matrix   = array[0..3] of array[0..3] of real; { multi-dimensional }
  Point = record
    x: integer;
    y: integer;
  end;

  { Enumerated types }
  Color      = (Red, Green, Blue, Yellow);
  Direction  = (North, East, South, West);

  { Variant records }
  Shape = record
    x, y: integer;
    case tag: integer of
      0: (radius: integer);
      1: (width, height: integer);
    end;
```

1.6.1.8 Variable Declarations

```
var
  a, b, c: integer;           { multiple identifiers }
  name: string;
  pi: real;
  items: array[0..99] of integer;
  data: array of real;        { dynamic array }
  pt: Point;
  p: ^integer;
```

Variables may include initialisers: `var x: integer = 42;`

1.6.1.9 Operators

Category	Operators
Arithmetic	+ - * / div mod
Relational	= < > <= > >=
Logical	and or not
String	+ (concatenation)
Pointer	^ (dereference, postfix), @ (address-of, prefix)
Unary	+ - not

1.6.1.10 Control Flow

```

{ If-Then-Else }
if x > 0 then
  writeln('positive')
else
  writeln('non-positive');

{ While }
while i < 10 do
begin
  writeln(i);
  i := i + 1;
end;

{ For }
for i := 1 to 10 do
  writeln(i);

for i := 10 downto 1 do
  writeln(i);

{ Repeat-Until }
repeat
  readln(x);
until x = 0;

{ Case }
case ch of
  1, 2, 3: writeln('small');
  4..6:   writeln('medium');
else
  writeln('other');
end;

{ Flow control }
break;      { exit loop }
continue;   { next iteration }
exit;       { exit current scope }

```

1.6.1.11 Procedures & Functions

```

procedure Swap(var a, b: integer);
var tmp: integer;
begin
  tmp := a;
  a := b;
  b := tmp;
end;

function Factorial(n: integer): integer;
begin
  if n <= 1 then
    Factorial := 1
  else
    Factorial := n * Factorial(n - 1);
end;

```

- **Parameters:** by value (default) or by reference (var keyword).
- **Return values:** assigned by writing to the **function name** (Factorial := ...) or by assigning to the special `result` variable (Delphi/FreePascal style).
- **Nested** procedures and functions are supported.

- **Recursion** is fully supported.

1.6.1.11.1 Result Variable

Inside any function body, the identifier `result` is an implicit alias for the function return value. Both styles may be used interchangeably:

```
function Max(a, b: integer): integer;
begin
    if a > b then
        result := a
    else
        result := b;
end;

function Clamp(val, lo, hi: integer): integer;
begin
    result := val;
    if result < lo then result := lo;
    if result > hi then result := hi;
end;
```

1.6.1.12 Records

```
type
    Point = record
        x: integer;
        y: integer;
    end;

var pt: Point;
begin
    pt.x := 10;
    pt.y := 20;
    writeln(pt.x, ', ', pt.y);
end.
```

Records may contain fields of any type including arrays, strings, pointers, and other record types.

1.6.1.12.1 Variant Records

Records may include a **variant part** after the fixed fields. All variant arms share the same memory region (union semantics), and a tag field selects which arm is active:

```
type
    Shape = record
        name: string;
        x, y: integer;
        case tag: integer of
            0: (radius: integer);           { circle }
            1: (width, height: integer);    { rectangle }
            2: (base, side1, side2: integer); { triangle }
        end;
    end;

var s: Shape;
begin
    s.tag := 1;
    s.width := 8;
    s.height := 6;
    writeln('Area = ', s.width * s.height);
end.
```

1.6.1.13 Enumerated Types

Enumerated types define an ordered set of named constants. Each value is assigned a sequential integer starting from 0:

```
type
    Color = (Red, Green, Blue, Yellow);
    Season = (Spring, Summer, Autumn, Winter);

var c: integer;
begin
    c := Red;           { c = 0 }
    c := Blue;          { c = 2 }
    writeln(ord(Green)); { 1 }
    writeln(succ(Red));  { 1 }
    writeln(pred(Blue)); { 1 }
    inc(c);              { c = 3 }
    dec(c);              { c = 2 }
end.
```

Routine	Description
<code>ord(x)</code>	Return the ordinal (integer) value of an enum constant
<code>succ(x)</code>	Return the next value (ordinal + 1)
<code>pred(x)</code>	Return the previous value (ordinal - 1)
<code>inc(x) / inc(x, n)</code>	Increment variable by 1 or by n
<code>dec(x) / dec(x, n)</code>	Decrement variable by 1 or by n

1.6.1.14 With Statement

The `with` statement provides shorthand access to record fields without repeating the record variable name:

```
type
  Point = record x, y: integer; end;

var pt: Point;
begin
  with pt do
    begin
      x := 10;
      y := 20;
      writeln('(', x, ', ', y, ')');
    end;
end.
```

1.6.1.15 Goto and Labels

Labels must be declared in a `label` section and may be numeric or named. `goto` performs an unconditional jump to the labelled statement:

```
label 100, 200;
var i: integer;
begin
  i := 0;
100:
  i := i + 1;
  if i > 5 then goto 200;
  write(i, ' ');
  goto 100;
200:
  writeln;
end.
```

1.6.1.16 In Operator (Set Membership)

The `in` operator tests whether a value is a member of a set literal or a set variable:

```
if x in [1, 3, 5, 7, 9] then
  writeln('odd')
else
  writeln('even');

var s: set of integer;
s := [2, 3, 5, 7, 11, 13, 17, 19];
for n := 1 to 20 do
  if n in s then
    writeln(n, ' is prime');
```

1.6.1.17 Sets

Set variables hold collections of ordinal values in the range 0..255. Declare with `set of <type>`:

```
type
  Digits = set of integer;

var
  s, t, u: set of integer;
begin
  s := [1, 3, 5, 7, 9];
  t := [2, 4, 6, 8, 10];

  { Set operations }
  u := s + t;      { union }
  u := s * [1, 2, 3]; { intersection }
  u := s - [1, 3];  { difference }

  { Membership test }
```

```

if 3 in s then writeln('yes');

{ Add / remove elements }
include(s, 4);      { add 4 to s }
exclude(s, 3);     { remove 3 from s }
end.

```

Operation	Syntax	Description
Union	$s + t$	Elements in either set
Intersection	$s * t$	Elements in both sets
Difference	$s - t$	Elements in s but not t
Membership	$x \text{ in } s$	Test if x is in set s
Include	<code>include(s, x)</code>	Add element x to set s
Exclude	<code>exclude(s, x)</code>	Remove element x from set s

1.6.1.18 File I/O

Pascal-style file I/O is supported using the `file` type with standard procedures:

```

var
  f: file;
  line: string;
begin
  assign(f, 'output.txt');
  rewrite(f);
  writeln(f, 'Hello, file!');
  close(f);

  assign(f, 'output.txt');
  reset(f);
  while not eof(f) do
  begin
    readln(f, line);
    writeln(line);
  end;
  close(f);
end.

```

Procedure	Description
<code>assign(f, name)</code>	Associate file variable with a filename
<code>reset(f)</code>	Open file for reading
<code>rewrite(f)</code>	Create/truncate file for writing
<code>append(f)</code>	Open file for appending
<code>close(f)</code>	Close the file
<code>eof(f)</code>	Returns <code>true</code> at end of file
<code>writeln(f, ...)</code>	Write to the file (instead of stdout)
<code>readln(f, var)</code>	Read a line from the file

1.6.1.19 Doubled-Quote Strings

To include a single-quote character inside a string literal, double it (standard Pascal convention):

```

writeln('It"s a test');      { prints: It's a test }
writeln('She said "hi"');   { prints: She said 'hi' }

```

1.6.1.20 Arrays

```

var
  nums: array[0..9] of integer;
  grid: array[0..3] of array[0..3] of real;
begin
  nums[0] := 42;
  grid[1][2] := 3.14;
end.

```

- Arbitrary integer bounds (not restricted to 0-based).
- Multi-dimensional arrays via nested `array[...] of array[...]`.
- Runtime bounds checking is enabled by default.

1.6.1.21 Dynamic Arrays

Dynamic arrays have no compile-time bounds. They are declared with `array of <type>` and resized at runtime with `SetLength`:

```
var
  data: array of integer;
begin
  SetLength(data, 10);           { allocate 10 elements (0..9) }
  data[0] := 42;
  writeln(Length(data));         { 10 }
  writeln(High(data));           { 9 }
  writeln(Low(data));            { 0 -- always 0 for dynamic arrays }

  SetLength(data, 20);           { grow to 20; existing data preserved }
end.
```

Routine	Description
<code>SetLength(arr, n)</code>	Resize <code>arr</code> to <code>n</code> elements. Existing data is preserved; new slots are zero-initialised. Passing 0 frees the block.
<code>Length(arr)</code>	Number of elements (runtime for dynamic, compile-time for static).
<code>High(arr)</code>	Highest valid index (<code>Length(arr) - 1</code> for dynamic arrays, upper bound for static).
<code>Low(arr)</code>	Lowest valid index (always 0 for dynamic arrays, lower bound for static).

- Dynamic arrays are always **0-based**.
- Memory is managed via the `realloc` VM instruction; the compiler emits a companion `_dynlen` variable that tracks the runtime length.
- Bounds checking applies at runtime using the companion length variable.

1.6.1.22 Pointers

```
type PInt = ^integer;
var
  p: PInt;
  x: integer;
begin
  new(p);           { allocate }
  p^ := 42;         { dereference and assign }
  x := p^;          { dereference and read }
  dispose(p);       { free }

  p := @x;          { address-of }
end.
```

1.6.1.23 String Operations

```
var
  s: string;
  n: integer;
begin
  s := 'Hello' + ' World';      { concatenation }
  n := length(s);              { length }
  writeln(copy(s, 1, 5));       { substring }
  writeln(pos('World', s));     { search }
  writeln(inttostr(42));        { int -> string }
  writeln(strtoint('123'));     { string -> int }
end.
```

1.6.1.24 Built-in Procedures & Functions

1.6.1.24.1 I/O

Routine	Description
<code>write(args...)</code>	Print values (no trailing newline)
<code>writeln(args...)</code>	Print values with trailing newline
<code>readln(var)</code>	Read a line from stdin; auto-converts to int/float

1.6.1.24.2 Math

`abs`, `sqrt`, `pow`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sinh`, `cosh`, `tanh`, `exp`, `log`, `log10`, `log2`, `exp2`, `fmod`, `ceil`, `floor`, `fabs`, `round`, `trunc`, `hypot`, `atan2`

1.6.1.24.3 Conversion

`float()`, `float_to_int()`, `int_to_float()`, `atoi()`, `atof()`

1.6.1.24.4 Character Classification

`toupper()`, `tolower()`, `isalpha()`, `isdigit()`, `isspace()`

1.6.1.24.5 String Operations

`length()`, `pos()`, `copy()`, `insert()`, `delete()`, `inttostr()`, `strtoint()`

1.6.1.24.6 Memory

`new(ptr)`, `dispose(ptr)`, `malloc()`, `calloc()`, `free()`, `memcpy()`, `memcmp()`, `memmove()`, `memset()`

1.6.1.24.7 Arrays

Routine	Description
<code>SetLength(arr, n)</code>	Resize a dynamic array to <code>n</code> elements
<code>Length(arr)</code>	Number of elements in a static or dynamic array
<code>High(arr)</code>	Highest valid index
<code>Low(arr)</code>	Lowest valid index

1.6.1.24.8 System

`halt(code)`, `system(cmd)`, `rand()`, `srand()`, `seed_random()`, `rand_number(max)`, `argc()`, `argv(index)`

1.6.2 Limitations & Known Quirks

The Pascal frontend implements a practical subset of standard Pascal. The following limitations apply:

Area	Limitation
Limited module support	The <code>uses</code> clause imports runtime modules (<code>io</code> , <code>std</code> , <code>strlib</code> , <code>sdl</code>) and separately compiled Pascal units. Units must be compiled individually and linked via the VM object-path mechanism. There is no automatic dependency resolution or build ordering.

Area	Limitation
packed	Accepted and parsed but has no effect on memory layout.
forward	Parsed and validated but codegen depends on declaration order.
Operator precedence	Follows standard Pascal precedence: <code>not > * / div mod and > + - or ></code> relational.

1.7 Runtime Module Reference

External functions are called from bytecode with `invoke func, args...` and from Pascal with regular function-call syntax. Return values are captured with `return var` (bytecode) or assigned normally (Pascal).

1.7.1 Module: io

File I/O, random numbers.

Function	Params	Returns	Description
<code>fopen</code>	2	POINTER	Open a file (filename, mode). Returns <code>FILE*</code> handle.
<code>fclose</code>	1	INTEGER	Close a file handle.
<code>fread</code>	4	INTEGER	Read from file into buffer (dst, size, count, fh).
<code>fwrite</code>	4	INTEGER	Write buffer to file (src, size, count, fh).
<code>fseek</code>	3	INTEGER	Seek within file (fh, offset, whence).
<code>fsize</code>	1	INTEGER	Get file size in bytes.
<code>fprintf</code>	<code>>= 2</code>	–	Formatted write to file (fh, fmt, ...).
<code>feof</code>	1	INTEGER	Test end-of-file on a file handle. Returns 1 at EOF, 0 otherwise.
<code>fgets</code>	1	STRING	Read one line from file (strips trailing newline).
<code>fputs</code>	2	–	Write a string to a file (str, fh).
<code>rand_number</code>	1	INTEGER	Random integer in [0, max).
<code>seed_random</code>	0	INTEGER	Seed RNG with current time.

1.7.2 Module: std

Standard library – math, memory, string conversion, system calls.

1.7.2.1 Math Functions

Function	Params	Returns
<code>abs</code>	1	INTEGER
<code>fabs</code>	1	FLOAT
<code>sqrt, exp, exp2, log, log10, log2</code>	1	FLOAT
<code>sin, cos, tan, asin, acos, atan</code>	1	FLOAT
<code>sinh, cosh, tanh</code>	1	FLOAT
<code>ceil, floor, round, trunc</code>	1	FLOAT
<code>pow, fmod, atan2, hypot</code>	2	FLOAT

1.7.2.2 Memory Functions

Function	Params	Returns	Description
malloc	1	POINTER	Allocate bytes.
calloc	2	POINTER	Allocate zeroed memory (count, size).
free	1	–	Release pointer.
memcpy	3	POINTER	Copy memory (dest, src, n).
memmove	3	POINTER	Move memory (dest, src, n).
memset	3	POINTER	Fill memory (dest, byte, n).
memcmp	3	INTEGER	Compare memory (a, b, n).

1.7.2.3 Conversion & Classification

Function	Params	Returns	Description
atoi	1	INTEGER	String -> int.
atof	1	FLOAT	String -> float.
toupper, tolower	1	INTEGER	Case conversion.
isalpha, isdigit, isspace	1	INTEGER	Character classification.

1.7.2.4 System

Function	Params	Returns	Description
system	1	INTEGER	Execute shell command.
exit	1	–	Terminate program.
rand	0	INTEGER	Random integer.
srand	1	–	Seed random.
argc	0	INTEGER	Argument count.
argv	1	POINTER	Argument string by index.

1.7.3 Module: string

String manipulation.

Function	Params	Returns	Description
strlen	1	INTEGER	String length.
strcmp	2	INTEGER	Compare two strings. Returns <0, 0, >0.
strncpy	3	INTEGER	Copy up to n characters.
strncat	3	INTEGER	Append up to n characters.
snprintf	>= 4	INTEGER	Printf into buffer (dest, size, fmt, ...).
strfind	3	INTEGER	Find substring (haystack, needle, start). Returns index or -1.
substr	5	INTEGER	Extract substring (dest, maxsize, src, pos, len).
strat	2	INTEGER	Character code at index.

1.7.4 Module: sdl

SDL2 / SDL2_ttf bindings for graphics, events, audio, and text rendering.

1.7.4.1 Core & Window

Function	Description
<code>init</code>	Initialise SDL2 subsystems.
<code>quit</code>	Shut down SDL2.
<code>create_window</code>	Create a window (title, x, y, w, h, flags).
<code>destroy_window</code>	Destroy a window.
<code>set_window_title</code>	Set window title text.
<code>set_window_position</code>	Reposition a window.
<code>get_window_size</code>	Query window dimensions.
<code>set_window_fullscreen</code>	Toggle fullscreen mode.
<code>set_window_icon</code>	Set window icon from image file.

1.7.4.2 Renderer & Drawing

Function	Description
<code>create_renderer</code>	Create a hardware renderer.
<code>destroy_renderer</code>	Destroy a renderer.
<code>set_draw_color</code>	Set RGBA draw colour.
<code>clear</code>	Clear with current draw colour.
<code>present</code>	Flip back buffer to screen.
<code>draw_point</code>	Draw a single pixel.
<code>draw_line</code>	Draw a line.
<code>draw_rect</code>	Draw a rectangle outline.
<code>fill_rect</code>	Draw a filled rectangle.

1.7.4.3 Textures & Surfaces

Function	Description
<code>create_texture</code>	Create a blank texture.
<code>destroy_texture</code>	Destroy a texture.
<code>load_texture</code>	Load texture from image file.
<code>render_texture</code>	Render src rect -> dst rect.
<code>update_texture</code>	Upload pixel data to texture.
<code>lock_texture</code> / <code>unlock_texture</code>	Direct pixel access.
<code>create_rgb_surface</code> / <code>free_surface</code>	Surface management.
<code>blit_surface</code>	Blit surface to surface.
<code>create_render_target</code> / <code>set_render_target</code> / <code>destroy_render_target</code> / <code>↔</code>	Off-screen render targets.

Function	Description
<code>present_scaled / present_stretched</code>	Present render target with scaling.

1.7.4.4 Events & Input

Function	Description
<code>poll_event</code>	Poll for pending event.
<code>get_event_type</code>	Event type of last polled event.
<code>get_key_code</code>	Key code from keyboard event.
<code>get_mouse_x / get_mouse_y</code>	Mouse coordinates.
<code>get_mouse_button</code>	Mouse button from event.
<code>get_mouse_state</code>	Current mouse position + buttons.
<code>get_relative_mouse_state</code>	Relative mouse delta.
<code>is_key_pressed</code>	Check if a scancode is pressed.
<code>get_keyboard_state</code>	Full keyboard state array.
<code>set_clipboard_text / get_clipboard_text</code>	Clipboard access.
<code>show_cursor</code>	Show or hide cursor.

1.7.4.5 Timing

Function	Description
<code>get_ticks</code>	Milliseconds since SDL init.
<code>delay</code>	Sleep for N milliseconds.

1.7.4.6 Audio

Function	Description
<code>open_audio</code>	Open audio device (freq, format, channels, samples).
<code>close_audio</code>	Close audio device.
<code>pause_audio</code>	Pause or resume playback.
<code>load_wav</code>	Load WAV file.
<code>free_wav</code>	Free loaded WAV buffer.
<code>queue_audio</code>	Queue audio data for playback.
<code>get_queued_audio_size</code>	Bytes of queued audio.
<code>clear_queued_audio</code>	Flush audio queue.

1.7.4.7 Text Rendering (SDL_ttf)

Function	Description
<code>init_text</code>	Initialise TTF subsystem.
<code>quit_text</code>	Shut down TTF.
<code>load_font</code>	Load a TrueType font (path, pt_size).

Function	Description
<code>draw_text</code>	Render text (renderer, font, text, x, y, r, g, b, a).

1.8 Code Generation & Optimisation

1.8.1 Native x86-64 Targets

Platform	ABI	Entry Point	Notes
Linux	System V	<code>_start</code> / <code>main</code>	Default target. Uses <code>rdi</code> , <code>rsi</code> , <code>rdx</code> , <code>rcx</code> , <code>r8</code> , <code>r9</code> for args.
macOS	System V (Darwin)	<code>_main</code>	Underscore-prefixed symbols. PIC relocations.
Windows	Win64	<code>main</code>	Uses <code>rcx</code> , <code>rdx</code> , <code>r8</code> , <code>r9</code> + shadow space.

1.8.2 Register Allocation

The code generator performs a register allocation pass that maps frequently-used variables to callee-saved registers:

- **System V:** `r12-r15`, `rbx` (integer); `xmm6-xmm15` (float).
- **Win64:** `r12-r15`, `rbx`, `rdi`, `rsi` (integer); extended XMM set.

Variables not assigned to registers are kept in memory (BSS/data section) and accessed via RIP-relative addressing.

1.8.3 Peephole Optimiser

A post-generation pass (`gen_optimize`) applies pattern-based rewrites:

- **Redundant load elimination:** removes `mov reg, [var]` immediately followed by `mov [var], reg` (and vice versa).
- **Dead store removal:** eliminates stores to variables that are immediately overwritten.
- **Strength reduction:** replaces multiply/divide by powers of 2 with shift instructions.
- **Redundant move elimination:** removes `mov rax, rax` and similar no-op register-to-register moves.

1.9 Examples

1.9.1 MXVM Bytecode – Hello World

```
program HelloWorld {
  section data {
    string msg = "Hello, World!\n"
  }
  section code {
    print msg
    done
  }
}
```

1.9.2 MXVM Bytecode – Fibonacci

```
program Fibonacci {
  section data {
    int a = 0
    int b = 1
  }
}
```

```

    int temp = 0
    int count = 0
    string fmt = "%d\n"
}
section code {
loop:
    print fmt, a
    mov temp, b
    add b, a, b
    mov a, temp
    add count, count, 1
    cmp count, 20
    jl loop
    done
}
}

```

1.9.3 Pascal – Recursive Factorial

```

program FactorialDemo;

function Factorial(n: integer): integer;
begin
    if n <= 1 then
        Factorial := 1
    else
        Factorial := n * Factorial(n - 1);
    end;

var i: integer;
begin
    for i := 1 to 12 do
        writeln(i, '!' = ', Factorial(i));
    end.

```

1.9.4 Pascal – Records & Pointers

```

program RecordDemo;

type
    Point = record
        x: integer;
        y: integer;
    end;

var
    p: ^Point;

begin
    new(p);
    p^.x := 10;
    p^.y := 20;
    writeln('Point: (', p^.x, ', ', p^.y, ')');
    dispose(p);
end.

```

1.9.5 Pascal – Dynamic Arrays

```

program DynArrayDemo;

var
    arr: array of integer;
    i: integer;

begin
    SetLength(arr, 5);
    for i := 0 to High(arr) do
        arr[i] := (i + 1) * 10;

    writeln('Length = ', Length(arr));
    write('Elements:');
    for i := Low(arr) to High(arr) do
        write(' ', arr[i]);
    writeln;

    { Grow the array -- existing data is preserved }
    SetLength(arr, 8);
    arr[5] := 60;
    arr[6] := 70;

```

```

    arr[7] := 80;

    writeln('After grow, Length = ', Length(arr));
end.

```

Output:

```

Length = 5
Elements: 10 20 30 40 50
After grow, Length = 8

```

1.9.6 Pascal – Extended Features Demo

Demonstrates enumerated types, variant records, the result variable, with statement, goto/label, in operator, and doubled-quote strings:

```

program FeaturesDemo;

type
  Season = (Spring, Summer, Autumn, Winter);

  Shape = record
    name: string;
    x, y: integer;
    case tag: integer of
      0: (radius: integer);
      1: (width, height: integer);
    end;
  end;

var
  s: integer;
  shapel: Shape;

label 100, 200;

function SeasonName(s: integer): string;
begin
  case s of
    0: result := 'Spring';
    1: result := 'Summer';
    2: result := 'Autumn';
    3: result := 'Winter';
  else
    result := 'Unknown';
  end;
end;

function Max(a, b: integer): integer;
begin
  if a > b then result := a else result := b;
end;

begin
  { Doubled-quote strings }
  writeln('It"s a beautiful day!');

  { Enumerated types }
  for s := Spring to Winter do
    writeln(SeasonName(s));
  writeln('succ(Spring) = ', succ(Spring));
  s := East;
  inc(s, 2);

  { In operator }
  for s := 1 to 10 do
    if s in [2, 3, 5, 7] then
      writeln(s, ' is prime');

  { With statement }
  with shapel do
  begin
    name := 'Circle';
    x := 10;
    y := 20;
    tag := 0;
    radius := 5;
  end;
  writeln(shapel.name, ' r=', shapel.radius);

  { Variant record: switch to rectangle }
  shapel.tag := 1;
  shapel.width := 8;
  shapel.height := 6;

  { Result variable }

```

```

writeln('Max(42, 17) = ', Max(42, 17));

{ Goto / Label }
s := 0;
100:
  s := s + 1;
  if s > 5 then goto 200;
  write(s, ' ');
  goto 100;
200:
  writeln;
end.

```

Full source

1.9.7 Pascal – Set Operations

Demonstrates set literals, membership testing, union/intersection/difference, and the `include/exclude` procedures:

```

program SetDemo;

uses io;

var
  primes, evens, both: set of integer;
  i: integer;

begin
  primes := [2, 3, 5, 7, 11, 13, 17, 19];
  evens  := [2, 4, 6, 8, 10, 12, 14, 16, 18, 20];

  { Membership test }
  for i := 1 to 20 do
    if i in primes then
      write(i, ' ');
  writeln;

  { Intersection: even primes }
  both := primes * evens;
  for i := 1 to 20 do
    if i in both then
      write(i, ' ');
  writeln;

  { Union }
  both := primes + evens;
  write('Union count: ');
  i := 0;
  for i := 1 to 20 do
    if i in both then
      write(i, ' ');
  writeln;

  { Difference: odd primes }
  both := primes - evens;
  for i := 1 to 20 do
    if i in both then
      write(i, ' ');
  writeln;

  { Include / Exclude }
  include(primes, 23);
  exclude(primes, 2);
  if 23 in primes then writeln('23 is now prime');
  if not (2 in primes) then writeln('2 removed');
end.

```

Output:

```

2 3 5 7 11 13 17 19
2
Union count: 2 3 4 5 6 7 8 10 11 12 13 14 16 17 18 19 20
3 5 7 11 13 17 19
23 is now prime
2 removed

```

1.9.8 Pascal – File I/O

Demonstrates text file creation, writing, reading line-by-line, and appending:

```

program FileDemo;

uses io;

var
  f: file;
  line: string;
  count: integer;

begin
  { Write a file }
  assign(f, 'greeting.txt');
  rewrite(f);
  writeln(f, 'Hello from MXVM!');
  writeln(f, 'Line two');
  writeln(f, 'Line three');
  close(f);

  { Read it back }
  assign(f, 'greeting.txt');
  reset(f);
  count := 0;
  while not eof(f) do
    begin
      readln(f, line);
      count := count + 1;
      writeln([' ', count, ' ', line]);
    end;
  close(f);

  { Append another line }
  assign(f, 'greeting.txt');
  append(f);
  writeln(f, 'Appended line');
  close(f);

  writeln('Done -- wrote ', count, ' lines, then appended one more.');
```

Output:

```

[1] Hello from MXVM!
[2] Line two
[3] Line three
Done -- wrote 3 lines, then appended one more.
```

1.9.9 Pascal – Packed Types

The `packed` keyword is accepted on array and record declarations for source compatibility with standard Pascal. It has no effect on the internal memory layout:

```

program PackedDemo;

uses io;

type
  PackedPoint = packed record
    x, y: integer;
  end;

var
  points: packed array[1..5] of integer;
  p: PackedPoint;
  i: integer;

begin
  for i := 1 to 5 do
    points[i] := i * 10;

  write('Packed array:');
  for i := 1 to 5 do
    write(' ', points[i]);
  writeln;

  p.x := 42;
  p.y := 99;
  writeln('Packed record: x=', p.x, ' y=', p.y);
end.
```

Output:

```

Packed array: 10 20 30 40 50
Packed record: x=42 y=99
```


1.9.10 Pascal – Units (Separate Compilation)

MathUtils.pas (unit):

```
unit MathUtils;

interface

function Add(a, b: integer): integer;
function Multiply(a, b: integer): integer;
procedure PrintResult(x: integer);

implementation

uses io;

function Add(a, b: integer): integer;
begin
    Add := a + b;
end;

function Multiply(a, b: integer): integer;
begin
    Multiply := a * b;
end;

procedure PrintResult(x: integer);
begin
    writeln(x);
end;

end.
```

TestMain.pas (program using the unit):

```
program TestMain;
uses io, MathUtils;

var
    result: integer;

begin
    result := Add(3, 4);
    writeln('3 + 4 = ', result);
    result := Multiply(5, 6);
    writeln('5 * 6 = ', result);
    PrintResult(99);
end.
```

Compile and run:

```
mxx MathUtils.pas MathUtils.mxvm
mxx TestMain.pas TestMain.mxvm
mxvmc TestMain.mxvm -x .
```

Output:

```
3 + 4 = 7
5 * 6 = 30
99
```

1.9.11 Pascal – SDL2 Knight's Tour (Full Game)

A Knight's Tour game demonstrating Warnsdorff's heuristic with backtracking, flat-array board representation, selection sort, SDL2 textures with custom RGB color-key transparency, scancode polling with auto-repeat, and the full compile-to-native pipeline.

The player watches the knight traverse every square on the board exactly once. Press **Space** (tap or hold) to advance the knight, **Return** to generate a new tour from a random starting position, or click **left mouse** / **right mouse** for move / reset.

```
program KnightsTour;

const
    BOARD_SIZE = 8;
    TOTAL_MOVES = 65;
    START_X = 140;
    START_Y = 40;
    CELL_SIZE = 85;
    CELL_DRAW_SIZE = 80;
    KNIGHT_SIZE = 60;

    WIN_W = 960;
    WIN_H = 720;

    SDL_QUIT_EVENT = 256;
    SDL_KEYDOWN = 768;
```

```

SDL_MOUSEBUTTONDOWN = 1025;
SDLK_ESCAPE = 27;
SDLK_RETURN = 13;
SDL_SCANCODE_SPACE = 44;

STATE_INTRO = 0;
STATE_PLAYING = 1;
MAX_SEQ = 64;

var
  window_id, renderer_id: integer;
  font_id, knight_tex, logo_tex: integer;
  running: boolean;
  game_state: integer;

  { board as flat array: board[row * 8 + col] }
  board: array[0..63] of integer;
  seq_row: array[0..63] of integer;
  seq_col: array[0..63] of integer;
  seq_count: integer;

  knight_row, knight_col: integer;
  moves: integer;
  tour_over: boolean;

  { L-shaped move offsets }
  horizontal: array[0..7] of integer;
  vertical: array[0..7] of integer;

  { scratch arrays for Warnsdorff sort }
  sort_deg: array[0..7] of integer;
  sort_row: array[0..7] of integer;
  sort_col: array[0..7] of integer;
  intro_start: integer;

{ ... helper procedures: init_move_offsets, clear_board, get_board,
  set_board, is_valid_move, get_degree, sort_moves ... }

function solve_tour(pos_row, pos_col, move_count: integer): integer;
var
  i, nr, nc, n: integer;
begin
  if move_count = TOTAL_MOVES then
    begin
      solve_tour := 1;
      exit;
    end;

  { collect and sort valid moves by Warnsdorff degree }
  n := 0;
  for i := 0 to 7 do
    begin
      nr := pos_row + vertical[i];
      nc := pos_col + horizontal[i];
      if is_valid_move(nr, nc) = 1 then
        begin
          sort_deg[n] := get_degree(nr, nc);
          sort_row[n] := nr;
          sort_col[n] := nc;
          n := n + 1;
        end;
    end;
  end;
  sort_moves(n);

  { try each move with backtracking }
  i := 0;
  while i < n do
    begin
      set_board(sort_row[i], sort_col[i], move_count);
      seq_row[seq_count] := sort_row[i];
      seq_col[seq_count] := sort_col[i];
      seq_count := seq_count + 1;

      if solve_tour(sort_row[i], sort_col[i], move_count + 1) = 1 then
        begin
          solve_tour := 1;
          exit;
        end
      else
        begin
          set_board(sort_row[i], sort_col[i], 0);
          seq_count := seq_count - 1;
        end;
      i := i + 1;
    end;
  end;
  solve_tour := 0;

```

```

end;

procedure next_move;
var
  nr, nc: integer;
begin
  if moves >= 64 then exit;
  if seq_count <= 0 then exit;
  if tour_over then exit;
  nr := seq_row[moves];
  nc := seq_col[moves];
  set_board(knight_row, knight_col, -1);
  knight_row := nr;
  knight_col := nc;
  moves := moves + 1;
  set_board(knight_row, knight_col, moves);
  if moves >= 64 then
    tour_over := true;
  end;
end;

{ ... draw_board, draw_knight, reset_tour ... }

begin
  sdl_init();
  window_id := sdl_create_window('Knights Tour', 100, 100, WIN_W, WIN_H, 0);
  renderer_id := sdl_create_renderer(window_id, -1, 0);
  sdl_init_text();
  font_id := sdl_load_font('data/font.ttf', 14);
  knight_tex := sdl_load_texture_color_key_rgb(renderer_id,
    'data/knight.bmp', 255, 255, 255);
  logo_tex := sdl_load_texture(renderer_id, 'data/logo.bmp');
  seed_random;
  init_move_offsets;
  game_state := STATE_INTRO;
  running := true;

  while running do
  begin
    { poll SDL events, handle keys/mouse }
    { space bar: first press immediate, hold repeats every 150 ms }
    { render board, knight, and HUD text }
    sdl_present(renderer_id);
    sdl_delay(16 - (sdl_get_ticks() - frame_start));
  end;

  sdl_quit();
end.

```

Building & running:

```

mxx knight.pas knight.mxvm          # compile Pascal -> bytecode
mxvmc knight.mxvm                   # interpret
mxvmc knight.mxvm --action translate # emit x86-64 assembly
LDLAGS="-lSDL2 -lSDL2_ttf" mxvmc knight.mxvm --action compile # compile to native executable

```

[Online Source Code](#)

1.9.12 Pascal – SDL2 Space Shooter (Full Game)

A complete space-shooter game demonstrating records, arrays, SDL2 textures, collision detection, and the full compile-to-native pipeline.

```

program SpaceShooter;
const
  SDL_WINDOW_SHOWN = 4;
  SDL_RENDERER_ACCELERATED = 2;
  SDL_QUIT = 256;
  SDL_KEYDOWN = 768;
  SDL_KEYUP = 769;
  SDLK_ESCAPE = 27;
  SDLK_RETURN = 13;
  SDLK_SPACE = 32;

  WIDTH = 1440;
  HEIGHT = 1080;
  SCALE_W = 1440;
  SCALE_H = 1080;

  EXIT_FAILURE = 1;
  fontfile = 'data/font.ttf';
  logfile = 'data/logo.bmp';
  spacelogofile = 'data/spacelogo.bmp';
  shipfile = 'data/ship.bmp';
  asteroidfile = 'data/asteroid.bmp';
  saucerfile = 'data/saucer.bmp';

```

```

particlefile = 'data/particle.bmp';

SDL_SCANCODE_LEFT = 80;
SDL_SCANCODE_RIGHT = 79;
SDL_SCANCODE_UP = 82;
SDL_SCANCODE_DOWN = 81;
SDL_SCANCODE_SPACE = 44;

PI = 3.14159265358979323846;

MAX_PROJECTILES = 30;
MAX_ENEMIES = 20;
MAX_STARS = 100;
MAX_EXPLOSIONS = 15;
MAX_CIRCULAR = 8;

EXPTYPE_RED = 0;
EXPTYPE_BLUE = 1;

{ game states }
STATE_INTRO = 0;
STATE_PLAYING = 1;
STATE_DIED = 2;
STATE_GAMEOVER = 3;

type
  Star = record
    x, y, speed: integer;
  end;

  Projectile = record
    x, y, w, h: integer;
    active: boolean;
  end;

  Enemy = record
    x, y, w, h: integer;
    active: boolean;
  end;

  CircularEnemy = record
    x, y: integer;
    angle: real;
    direction: integer;
    active: boolean;
  end;

  Explosion = record
    x, y: integer;
    duration: integer;
    cr, cg, cb: integer;
    die: boolean;
    active: boolean;
  end;

var
  running: boolean;
  window_id, renderer, target_id: integer;
  font_id: integer;
  logo_tex, spacelogo_tex: integer;
  ship_tex, asteroid_tex, saucer_tex, particle_tex: integer;
  event_type_result: integer;
  key: integer;

  game_state: integer;

  { player }
  player_x, player_y, player_w, player_h: integer;
  player_die: boolean;

  { game data }
  lives, score: integer;
  regular_enemies_destroyed: integer;
  last_shot_time: integer;
  last_spawn_time: integer;

  { intro }
  intro_alpha: integer;
  intro_fading: boolean;
  intro_timer: integer;

  { arrays - use active flag, no shifting }
  stars: array[0..MAX_STARS - 1] of Star;
  projectiles: array[0..MAX_PROJECTILES - 1] of Projectile;
  enemies: array[0..MAX_ENEMIES - 1] of Enemy;
  circular: array[0..MAX_CIRCULAR - 1] of CircularEnemy;
  circular_cx: array[0..MAX_CIRCULAR - 1] of integer;

```

```

    explosions: array[0..MAX_EXPLOSIONS - 1] of Explosion;

{ ---- collision detection ---- }
function aabb(ax, ay, aw, ah, bx, by, bw, bh: integer): boolean;
begin
    aabb := not ((ax + aw <= bx) or (bx + bw <= ax) or (ay + ah <= by) or (by + bh <= ay));
end;

{ ---- explosion helpers ---- }
procedure add_explosion(ex, ey, etype: integer; d: boolean);
var
    i: integer;
begin
    i := 0;
    while i < MAX_EXPLOSIONS do
    begin
        if explosions[i].active = false then
        begin
            explosions[i].x := ex;
            explosions[i].y := ey;
            explosions[i].duration := 60;
            explosions[i].die := d;
            explosions[i].active := true;
            if etype = EXPTYPE_RED then
            begin
                explosions[i].cr := 255;
                explosions[i].cg := 0;
                explosions[i].cb := 0;
            end
            else
            begin
                explosions[i].cr := 0;
                explosions[i].cg := 0;
                explosions[i].cb := 255;
            end;
            exit;
        end;
        i := i + 1;
    end;
end;

procedure subtract_life;
begin
    lives := lives - 1;
    if lives <= 0 then
        game_state := STATE_GAMEOVER
    else
        game_state := STATE_DIED;
    player_die := false;
end;

procedure update_explosions;
var
    i: integer;
begin
    for i := 0 to MAX_EXPLOSIONS - 1 do
    begin
        if explosions[i].active then
        begin
            explosions[i].duration := explosions[i].duration - 1;
            if explosions[i].duration <= 0 then
            begin
                explosions[i].active := false;
                if explosions[i].die then
                    subtract_life;
            end;
        end;
    end;
end;

procedure draw_explosions;
var
    i, k, dx, dy: integer;
begin
    for i := 0 to MAX_EXPLOSIONS - 1 do
    begin
        if explosions[i].active then
        begin
            if explosions[i].die then
            begin
                { large player death explosion }
                for k := 0 to 39 do
                begin
                    dx := (rand() mod 160) - 80;
                    dy := (rand() mod 160) - 80;
                    sdl_set_draw_color(renderer, 255, 200 + (rand() mod 56), rand() mod 80, 255);
                    sdl_draw_line(renderer, explosions[i].x, explosions[i].y,

```

```

        explosions[i].x + dx, explosions[i].y + dy);
    end;
    { white-hot core }
    sdl_set_draw_color(renderer, 255, 255, 200, 255);
    for k := 0 to 14 do
    begin
        dx := (rand() mod 40) - 20;
        dy := (rand() mod 40) - 20;
        sdl_draw_line(renderer, explosions[i].x + dx, explosions[i].y + dy,
            explosions[i].x + (rand() mod 20) - 10, explosions[i].y + (rand() mod 20) - 10);
    end;
end
else
begin
    { normal enemy explosion }
    sdl_set_draw_color(renderer, explosions[i].cr, explosions[i].cg, explosions[i].cb, 255);
    for k := 0 to 19 do
    begin
        dx := (rand() mod 60) - 30;
        dy := (rand() mod 60) - 30;
        sdl_draw_line(renderer, explosions[i].x, explosions[i].y,
            explosions[i].x + dx, explosions[i].y + dy);
    end;
end;
end;
end;
end;
end;

{ ---- drawing helpers ---- }
procedure draw_gradient_triangle(x1, y1, y3: integer);
var
    i, r, g, height: integer;
begin
    if player_die then exit;
    height := y3 - y1;
    if height <= 0 then exit;
    for i := 0 to height - 1 do
    begin
        r := 255 - (255 * i div height);
        g := 255 * i div height;
        sdl_set_draw_color(renderer, r, g, 255, 255);
        sdl_draw_line(renderer, x1 - i, y1 + i, x1 + i, y1 + i);
    end;
end;

procedure draw_gradient_diamond(cx, cy, hw, hh: integer);
var
    i, r, b, g2: integer;
begin
    for i := 0 to hh - 1 do
    begin
        r := 255 - (255 * i div hh);
        b := 255 * i div hh;
        sdl_set_draw_color(renderer, r, 255, b, 255);
        sdl_draw_line(renderer, cx - i, cy - i, cx + i, cy - i);
    end;
    for i := 0 to hh - 1 do
    begin
        g2 := 255 - (255 * i div hh);
        b := 255 * i div hh;
        sdl_set_draw_color(renderer, 255, g2, b, 255);
        sdl_draw_line(renderer, cx - i, cy + i, cx + i, cy + i);
    end;
end;

procedure draw_gradient_circle(cx, cy, radius: integer);
var
    i, w, color: integer;
begin
    for i := 0 to radius - 1 do
    begin
        w := radius - i;
        color := 255 * w div radius;
        sdl_set_draw_color(renderer, color, 0, color, 255);
        sdl_draw_line(renderer, cx - w, cy - i, cx + w, cy - i);
        sdl_draw_line(renderer, cx - w, cy + i, cx + w, cy + i);
    end;
end;

{ ---- find free slot helpers ---- }
function find_free_projectile: integer;
var
    i: integer;
begin
    find_free_projectile := -1;
    for i := 0 to MAX_PROJECTILES - 1 do
        if projectiles[i].active = false then

```

```

        begin
            find_free_projectile := i;
            exit;
        end;
    end;

function find_free_enemy: integer;
var
    i: integer;
begin
    find_free_enemy := -1;
    for i := 0 to MAX_ENEMIES - 1 do
        if enemies[i].active = false then
            begin
                find_free_enemy := i;
                exit;
            end;
        end;
    end;

function find_free_circular: integer;
var
    i: integer;
begin
    find_free_circular := -1;
    for i := 0 to MAX_CIRCULAR - 1 do
        if circular[i].active = false then
            begin
                find_free_circular := i;
                exit;
            end;
        end;
    end;

{ ---- init / reset ---- }
procedure init_stars;
var
    i: integer;
begin
    for i := 0 to MAX_STARS - 1 do
        begin
            stars[i].x := rand() mod SCALE_W;
            stars[i].y := rand() mod SCALE_H;
            stars[i].speed := 1 + (rand() mod 3);
        end;
    end;

procedure clear_all_entities;
var
    i: integer;
begin
    for i := 0 to MAX_PROJECTILES - 1 do
        projectiles[i].active := false;
    for i := 0 to MAX_ENEMIES - 1 do
        enemies[i].active := false;
    for i := 0 to MAX_CIRCULAR - 1 do
        circular[i].active := false;
    for i := 0 to MAX_EXPLOSIONS - 1 do
        explosions[i].active := false;
    end;

procedure reset_game;
begin
    player_x := (SCALE_W div 2) - 24;
    player_y := SCALE_H - 80;
    player_w := 48;
    player_h := 48;
    player_die := false;
    clear_all_entities;
    lives := 5;
    score := 0;
    regular_enemies_destroyed := 0;
    last_shot_time := 0;
    last_spawn_time := sdl_get_ticks();
    init_stars;
    game_state := STATE_PLAYING;
end;

procedure reset_round;
begin
    clear_all_entities;
    player_die := false;
    player_x := (SCALE_W div 2) - 24;
    player_y := SCALE_H - 80;
    game_state := STATE_PLAYING;
end;

{ ---- game update ---- }
procedure update_game;

```

```

var
  dx, dy: integer;
  ticks: integer;
  i, ei, ci: integer;
  slot: integer;
  crect_x, crect_y: integer;
  new_cx: real;
begin
  if (game_state <> STATE_PLAYING) then exit;

  update_explosions;

  { player movement via keyboard state }
  dx := 0;
  dy := 0;
  if sdl_is_key_pressed(SDL_SCANCODE_LEFT) <> 0 then dx := dx - 5;
  if sdl_is_key_pressed(SDL_SCANCODE_RIGHT) <> 0 then dx := dx + 5;
  if sdl_is_key_pressed(SDL_SCANCODE_UP) <> 0 then dy := dy - 5;
  if sdl_is_key_pressed(SDL_SCANCODE_DOWN) <> 0 then dy := dy + 5;

  player_x := player_x + dx;
  player_y := player_y + dy;

  if player_x < 0 then player_x := 0;
  if player_x > SCALE_W - player_w then player_x := SCALE_W - player_w;
  if player_y < 0 then player_y := 0;
  if player_y > SCALE_H - player_h then player_y := SCALE_H - player_h;

  { shooting }
  ticks := sdl_get_ticks();
  if (sdl_is_key_pressed(SDL_SCANCODE_SPACE) <> 0) and (ticks - last_shot_time > 300) then
  begin
    slot := find_free_projectile();
    if slot <> -1 then
    begin
      projectiles[slot].x := player_x + (player_w div 2) - 8;
      projectiles[slot].y := player_y - 30;
      projectiles[slot].w := 16;
      projectiles[slot].h := 24;
      projectiles[slot].active := true;
      last_shot_time := ticks;
    end;
  end;

  { update projectiles and check collisions }
  for i := 0 to MAX_PROJECTILES - 1 do
  begin
    if projectiles[i].active then
    begin
      projectiles[i].y := projectiles[i].y - 10;

      { off screen }
      if projectiles[i].y < -24 then
      begin
        projectiles[i].active := false;
      end
      else
      begin
        { check against regular enemies }
        for ei := 0 to MAX_ENEMIES - 1 do
        begin
          if enemies[ei].active and projectiles[i].active then
          begin
            if aabb(projectiles[i].x, projectiles[i].y, projectiles[i].w, projectiles[i].h,
              enemies[ei].x, enemies[ei].y, enemies[ei].w, enemies[ei].h) then
            begin
              add_explosion(enemies[ei].x + enemies[ei].w div 2,
                enemies[ei].y + enemies[ei].h div 2, EXPTYPE_RED, false);
              enemies[ei].active := false;
              projectiles[i].active := false;
              score := score + 5;
              regular_enemies_destroyed := regular_enemies_destroyed + 1;
              { spawn circular enemy every 5 regular kills }
              if (regular_enemies_destroyed mod 5) = 0 then
              begin
                slot := find_free_circular();
                if slot <> -1 then
                begin
                  circular_cx[slot] := 100 + (rand() mod (SCALE_W - 200));
                  circular[slot].x := circular_cx[slot];
                  circular[slot].y := -128;
                  circular[slot].angle := 0.0;
                  circular[slot].direction := 1;
                  circular[slot].active := true;
                end;
              end;
            end;
          end;
        end;
      end;
    end;
  end;
end;

```



```

    end;
end;

{ check against circular enemies }
for ci := 0 to MAX_CIRCULAR - 1 do
begin
    if circular[ci].active and projectiles[i].active then
    begin
        crect_x := circular[ci].x - 64;
        crect_y := circular[ci].y - 64;
        if aabb(projectiles[i].x, projectiles[i].y, projectiles[i].w, projectiles[i].h,
            crect_x, crect_y, 128, 128) then
        begin
            add_explosion(circular[ci].x, circular[ci].y, EXPTYPE_BLUE, false);
            circular[ci].active := false;
            projectiles[i].active := false;
            score := score + 10;
        end;
    end;
end;
end;
end;
end;

{ update regular enemies }
for ei := 0 to MAX_ENEMIES - 1 do
begin
    if enemies[ei].active then
    begin
        enemies[ei].y := enemies[ei].y + 2;
        if enemies[ei].y > SCALE_H then
        begin
            add_explosion(enemies[ei].x + enemies[ei].w div 2, SCALE_H - 20, EXPTYPE_RED, true);
            enemies[ei].active := false;
        end
        else if aabb(player_x, player_y, player_w, player_h,
            enemies[ei].x, enemies[ei].y, enemies[ei].w, enemies[ei].h) then
        begin
            add_explosion(player_x + player_w div 2, player_y + player_h div 2, EXPTYPE_RED, true);
            enemies[ei].active := false;
            player_die := true;
        end;
    end;
end;

{ update circular enemies }
for ci := 0 to MAX_CIRCULAR - 1 do
begin
    if circular[ci].active then
    begin
        circular[ci].angle := circular[ci].angle + 0.05;
        new_cx := circular_cx[ci] + 100.0 * cos(circular[ci].angle);
        circular[ci].x := trunc(new_cx);
        circular[ci].y := circular[ci].y + circular[ci].direction * 2;
        if circular[ci].y > SCALE_H then circular[ci].direction := -1;
        if circular[ci].y < 0 then circular[ci].direction := 1;

        crect_x := circular[ci].x - 64;
        crect_y := circular[ci].y - 64;
        if aabb(player_x, player_y, player_w, player_h,
            crect_x, crect_y, 128, 128) then
        begin
            add_explosion(player_x + player_w div 2, player_y + player_h div 2, EXPTYPE_RED, true);
            circular[ci].active := false;
            player_die := true;
        end;
    end;
end;

{ update stars }
for i := 0 to MAX_STARS - 1 do
begin
    stars[i].y := stars[i].y + stars[i].speed;
    if stars[i].y > SCALE_H then
    begin
        stars[i].y := 0;
        stars[i].x := rand() mod SCALE_W;
    end;
end;

{ spawn enemies }
if ticks - last_spawn_time > 1000 then
begin
    slot := find_free_enemy();
    if slot <> -1 then
    begin
        enemies[slot].x := rand() mod (SCALE_W - 64);

```

```

        enemies[slot].y := -64;
        enemies[slot].w := 64;
        enemies[slot].h := 64;
        enemies[slot].active := true;
    end;
    last_spawn_time := ticks;
end;
end;

{ ---- rendering ---- }
procedure render_game;
var
    i: integer;
    brightness: integer;
    r1, g1, b1: integer;
begin
    { draw explosions }
    draw_explosions;

    { draw stars }
    for i := 0 to MAX_STARS - 1 do
    begin
        brightness := 200 + 55 * (stars[i].speed - 1);
        if brightness > 255 then brightness := 255;
        sdl_set_draw_color(renderer, brightness, brightness, brightness, 255);
        sdl_draw_point(renderer, stars[i].x, stars[i].y);
    end;

    if game_state = STATE_PLAYING then
    begin
        { draw player ship }
        if player_die = false then
        begin
            if ship_tex <> -1 then
                sdl_render_texture(renderer, ship_tex, -1, -1, -1, -1,
                                   player_x - 8, player_y - 8, 64, 64)
            else
                draw_gradient_triangle(player_x + player_w div 2, player_y,
                                      player_y + player_h);
            end;

        { draw projectiles }
        for i := 0 to MAX_PROJECTILES - 1 do
        begin
            if projectiles[i].active then
            begin
                if particle_tex <> -1 then
                    sdl_render_texture(renderer, particle_tex, -1, -1, -1, -1,
                                       projectiles[i].x, projectiles[i].y,
                                       projectiles[i].w, projectiles[i].h)
                else
                begin
                    sdl_set_draw_color(renderer, 255, 255, 255, 255);
                    sdl_fill_rect(renderer, projectiles[i].x, projectiles[i].y,
                                 projectiles[i].w, projectiles[i].h);
                end;
            end;

        { draw regular enemies }
        for i := 0 to MAX_ENEMIES - 1 do
        begin
            if enemies[i].active then
            begin
                if asteroid_tex <> -1 then
                    sdl_render_texture(renderer, asteroid_tex, -1, -1, -1, -1,
                                       enemies[i].x, enemies[i].y,
                                       enemies[i].w, enemies[i].h)
                else
                    draw_gradient_diamond(enemies[i].x + enemies[i].w div 2,
                                         enemies[i].y + enemies[i].h div 2,
                                         enemies[i].w div 2, enemies[i].h div 2);
                end;
            end;

        { draw circular enemies }
        for i := 0 to MAX_CIRCULAR - 1 do
        begin
            if circular[i].active then
            begin
                if saucer_tex <> -1 then
                    sdl_render_texture(renderer, saucer_tex, -1, -1, -1, -1,
                                       circular[i].x - 64, circular[i].y - 64, 128, 128)
                else
                    draw_gradient_circle(circular[i].x, circular[i].y, 64);
                end;
            end;

        { draw HUD }
        sdl_draw_text(renderer, font_id, 'Lives: ' + inttostr(lives), 10, 10, 255, 255, 255, 255);
    end;
end;

```

```

sdl_draw_text(renderer, font_id, 'Score: ' + inttostr(score), 10, 30, 255, 255, 255, 255);

{ death message }
if game_state = STATE_DIED then
begin
  if spacelogo_tex <> -1 then
    sdl_render_texture(renderer, spacelogo_tex, -1, -1, -1, -1, 0, 0, SCALE_W, SCALE_H);
    r1 := rand() mod 255;
    g1 := rand() mod 255;
    b1 := rand() mod 255;
    sdl_set_draw_color(renderer, 0, 0, 0, 255);
    sdl_fill_rect(renderer, 470, 530, 500, 40);
    sdl_draw_text(renderer, font_id, 'You Died! Press ENTER to continue', 490, 538, r1, g1, b1, 255);
  end;

  { game over message }
  if game_state = STATE_GAMEOVER then
  begin
    if spacelogo_tex <> -1 then
      sdl_render_texture(renderer, spacelogo_tex, -1, -1, -1, -1, 0, 0, SCALE_W, SCALE_H);
      r1 := rand() mod 255;
      g1 := rand() mod 255;
      b1 := rand() mod 255;
      sdl_set_draw_color(renderer, 0, 0, 0, 255);
      sdl_fill_rect(renderer, 470, 530, 500, 40);
      sdl_draw_text(renderer, font_id, 'Game Over! Press ENTER to restart', 490, 538, r1, g1, b1, 255);
    end;
  end;

  { ---- intro screen ---- }
  procedure render_intro;
  var
    r1, g1, b1: integer;
  begin
    { draw background bitmap }
    if spacelogo_tex <> -1 then
      sdl_render_texture(renderer, spacelogo_tex, -1, -1, -1, -1, 0, 0, 1440, 1080)
    else
      begin
        sdl_set_draw_color(renderer, 0, 0, 0, 255);
        sdl_clear(renderer);
      end;

      { draw logo bitmap }
      if logo_tex <> -1 then
        sdl_render_texture(renderer, logo_tex, -1, -1, -1, -1, 0, 0, SCALE_W, SCALE_H);

      r1 := rand() mod 255;
      g1 := rand() mod 255;
      b1 := rand() mod 255;

      sdl_draw_text(renderer, font_id, 'Press ENTER to Start', 640, 700, r1, g1, b1, 255);
      sdl_draw_text(renderer, font_id, 'Arrow Keys = Move   Space = Shoot', 584, 740, 180, 180, 180, 255);
    end;

  { ---- SDL init / cleanup ---- }
  procedure init_sdl;
  var
    result: integer;
  begin
    result := sdl_init();
    if result <> 0 then
      begin
        writeln('failed to init SDL');
        halt(EXIT_FAILURE);
      end;
    end;

    window_id := sdl_create_window('Space Shooter', 100, 100, WIDTH, HEIGHT, SDL_WINDOW_SHOWN);
    if window_id = -1 then
      begin
        writeln('Could not create window');
        sdl_quit();
        halt(EXIT_FAILURE);
      end;
    end;

    renderer := sdl_create_renderer(window_id, -1, SDL_RENDERER_ACCELERATED);
    if renderer = -1 then
      begin
        writeln('Could not create renderer');
        sdl_destroy_window(window_id);
        sdl_quit();
        halt(EXIT_FAILURE);
      end;
    end;

    target_id := sdl_create_render_target(renderer, SCALE_W, SCALE_H);
    if target_id = -1 then
      begin

```

```

    writeln('Could not create render target');
    sdl_destroy_renderer(renderer);
    sdl_destroy_window(window_id);
    sdl_quit();
    halt(EXIT_FAILURE);
end;

sdl_set_draw_color(renderer, 0, 0, 0, 255);

if sdl_init_text() <> 1 then
begin
    writeln('Font subsystem failed to load');
    halt(EXIT_FAILURE);
end;

font_id := sdl_load_font(fontfile, 14);
if font_id = -1 then
begin
    writeln('Failed to open: ', fontfile);
    halt(EXIT_FAILURE);
end;

{ load background textures without color key }
logo_tex := sdl_load_texture(renderer, logfile);
if logo_tex = -1 then
    writeln('Warning: could not load ', logfile);

spacelogo_tex := sdl_load_texture(renderer, spacelogofile);
if spacelogo_tex = -1 then
    writeln('Warning: could not load ', spacelogofile);

{ load sprite textures with black color key for transparency }
ship_tex := sdl_load_texture_color_key(renderer, shipfile);
if ship_tex = -1 then
    writeln('Warning: could not load ', shipfile);

asteroid_tex := sdl_load_texture_color_key(renderer, asteroidfile);
if asteroid_tex = -1 then
    writeln('Warning: could not load ', asteroidfile);

saucer_tex := sdl_load_texture_color_key(renderer, saucerfile);
if saucer_tex = -1 then
    writeln('Warning: could not load ', saucerfile);

particle_tex := sdl_load_texture_color_key(renderer, particlefile);
if particle_tex = -1 then
    writeln('Warning: could not load ', particlefile);
end;

procedure cleanup;
begin
    if ship_tex <> -1 then sdl_destroy_texture(ship_tex);
    if asteroid_tex <> -1 then sdl_destroy_texture(asteroid_tex);
    if saucer_tex <> -1 then sdl_destroy_texture(saucer_tex);
    if particle_tex <> -1 then sdl_destroy_texture(particle_tex);
    if logo_tex <> -1 then sdl_destroy_texture(logo_tex);
    if spacelogo_tex <> -1 then sdl_destroy_texture(spacelogo_tex);
    sdl_destroy_render_target(target_id);
    sdl_destroy_renderer(renderer);
    sdl_destroy_window(window_id);
    sdl_quit_text();
    sdl_quit();
end;

{ ---- main loop ---- }
var
    prev_time, cur_time: integer;

begin
    init_sdl;
    seed_random;
    game_state := STATE_INTRO;
    lives := 5;
    score := 0;
    clear_all_entities;
    init_stars;

    prev_time := sdl_get_ticks();
    running := true;

    while running = true do
    begin
        { poll events }
        while sdl_poll_event() <> 0 do
            begin

```

```

event_type_result := sdl_get_event_type();

if event_type_result = SDL_QUIT then
    running := false;

if event_type_result = SDL_KEYDOWN then
begin
    key := sdl_get_key_code();
    if key = SDLK_ESCAPE then
        running := false;

    if key = SDLK_RETURN then
    begin
        if game_state = STATE_INTRO then
            reset_game
        else if game_state = STATE_DIED then
            reset_round
        else if game_state = STATE_GAMEOVER then
            reset_game;
        end;
    end;
end;

{ timed update at ~60fps }
cur_time := sdl_get_ticks();
if cur_time - prev_time >= 15 then
begin
    prev_time := cur_time;
    if game_state = STATE_PLAYING then
        update_game;
    end;

    { render }
    sdl_set_render_target(renderer, target_id);
    sdl_set_draw_color(renderer, 0, 0, 0, 255);
    sdl_clear(renderer);

    if game_state = STATE_INTRO then
        render_intro
    else
        render_game;

    sdl_present_scaled(renderer, target_id, WIDTH, HEIGHT);
    sdl_delay(16);
end;

cleanup;
end.

```

Compile and run:

```

mxx space.pas space.mxvm          # Pascal -> MXVM bytecode
mxvmc space.mxvm                  # run in interpreter

```

Compile to native x86-64 binary:

```

LDFLAGS="-lSDL2 -lSDL2_ttf" mxvmc space.mxvm --action compile
./SpaceShooter                  # run native binary

```

[Online Source Code](#)

1.10 Project Layout

```

MXVM/
|---- include/mxvm/          Core VM headers (parser, icode, instruct, etc.)
|---- src/                  Implementation sources
| |---- frontend/          Pascal parser, validator, AST, codegen
| | `---- include/         Frontend-specific headers
| |---- html_gen/          mxvm-html tool (html.cpp, CMakeLists.txt)
| |---- scanner/           Tokeniser / lexer
| | `---- include/scanner/ Scanner headers
| |---- vm/                Interpreter, argument parsing
| | `---- include/         VM headers (argz.hpp)
| `---- webasm/            WebAssembly target (experimental)
|---- modules/             Runtime C modules
| |---- io/                File I/O, random numbers
| |---- std/               Math, memory, conversion, system
| |---- string/            String manipulation
| | `---- sdl/             SDL2 + SDL_ttf bindings
|---- docs/                HTML reference pages (index, standard, sdl, mxvm-html, examples/)
|---- mxvm_src/            Example .mxvm programs
|---- CMakeLists.txt       Top-level build script
`---- Doxyfile             Doxygen configuration

```

[source repository](#) for the latest code.*

Chapter 2

Directory Hierarchy

2.1 Directories

frontend	63
include	65
ast.hpp	476
expr.hpp	763
icode.hpp	499
parser.hpp	539
types.hpp	768
validator.hpp	774
ast.cpp	671
expr.cpp	677
icode.cpp	678
icode_gen.cpp	693
icode_imp.cpp	717
icode_visit.cpp	731
main.cpp	777
parser.cpp	797
validator.cpp	836
html_gen	64
html.cpp	862
include	65
mxvm	68
ast.hpp	473
function.hpp	488
html_gen.hpp	490
icode.hpp	491
instruct.hpp	526
mxvm.hpp	534
parser.hpp	535
valid.hpp	542
version_info.hpp	545
include	65
ast.hpp	476
expr.hpp	763
icode.hpp	499
parser.hpp	539
types.hpp	768
validator.hpp	774
include	66
scanner	69
exception.hpp	928

expr.hpp	764
scanner.hpp	931
string_buffer.hpp	934
types.hpp	770
include	66
argz.hpp	958
io	67
io.c	546
io.cpp	547
modules	67
io	67
io.c	546
io.cpp	547
sdl	70
mx_sdl.h	554
sdl.c	575
sdl.cpp	603
std	72
mx_std.h	629
std.c	631
std.cpp	635
string	72
cstring.c	655
string.cpp	659
mxvm	68
ast.hpp	473
function.hpp	488
html_gen.hpp	490
icode.hpp	491
instruct.hpp	526
mxvm.hpp	534
parser.hpp	535
valid.hpp	542
version_info.hpp	545
scanner	69
include	66
scanner	69
exception.hpp	928
expr.hpp	764
scanner.hpp	931
string_buffer.hpp	934
types.hpp	770
exception.cpp	926
scanner.cpp	938
string_buffer.cpp	945
types.cpp	946
scanner	69
exception.hpp	928
expr.hpp	764
scanner.hpp	931
string_buffer.hpp	934
types.hpp	770
sdl	70
mx_sdl.h	554
sdl.c	575

sdl.cpp	603
src	71
frontend	63
include	65
ast.hpp	476
expr.hpp	763
icode.hpp	499
parser.hpp	539
types.hpp	768
validator.hpp	774
ast.cpp	671
expr.cpp	677
icode.cpp	678
icode_gen.cpp	693
icode_imp.cpp	717
icode_visit.cpp	731
main.cpp	777
parser.cpp	797
validator.cpp	836
html_gen	64
html.cpp	862
scanner	69
include	66
scanner	69
exception.hpp	928
expr.hpp	764
scanner.hpp	931
string_buffer.hpp	934
types.hpp	770
exception.cpp	926
scanner.cpp	938
string_buffer.cpp	945
types.cpp	946
vm	73
include	66
argz.hpp	958
main.cpp	783
ast.cpp	669
function.cpp	854
html_gen.cpp	855
icode.cpp	687
icode_exec.cpp	866
icode_gen.cpp	693
icode_gen_x64.cpp	893
icode_opt.cpp	914
instruct.cpp	921
mxvm.cpp	925
parser.cpp	813
valid.cpp	947
std	72
mx_std.h	629
std.c	631
std.cpp	635
string	72
cstring.c	655
string.cpp	659

vm	73
include	66
argz.hpp	958
main.cpp	783

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all namespaces with brief descriptions:

expr_parser	75
mx	76
mx::Color	
ANSI escape code constants for terminal coloring	79
mxvm	81
mxx	90
pascal	93
scan	101
scan::buffer	102
scan::token	102
types	102

Chapter 4

Concept Index

4.1 Concepts

Here is a list of all concepts with brief descriptions:

[mx::StringType](#)

Concept constraining String template parameters to class types with length(), operator[], operator+=, and operator= (e.g. std::string, std::wstring) 109

Chapter 5

Hierarchical Index

5.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

mx::ArgException< String >	112
Args	113
mx::Argument< String >	115
mx::ArgumentData< String >	118
mx::Arguments	119
mx::Argz< String >	120
pascal::ArrayInfo	132
mxvm::ASTNode	139
mxvm::CommentNode	213
mxvm::ExpressionNode	226
mxvm::InstructionNode	255
mxvm::LabelNode	258
mxvm::ModuleNode	261
mxvm::ObjectNode	270
mxvm::ProgramNode	367
mxvm::SectionNode	390
mxvm::VariableNode	459
pascal::ASTNode	141
pascal::AddressOfNode	111
pascal::ArrayAccessNode	126
pascal::ArrayAssignmentNode	128
pascal::ArrayDeclarationNode	130
pascal::ArrayTypeDeclarationNode	134
pascal::ArrayTypeNode	136
pascal::AssignmentNode	138
pascal::BinaryOpNode	159
pascal::BlockNode	162
pascal::BooleanNode	164
pascal::BreakNode	166
pascal::CaseStmtNode	171
pascal::CompoundStmtNode	214
pascal::ConstDeclNode	217
pascal::ContinueNode	219
pascal::EmptyStmtNode	220
pascal::EnumTypeDeclNode	221
pascal::ExitNode	225
pascal::FieldAccessNode	234
pascal::ForStmtNode	238
pascal::FuncCallNode	240
pascal::FuncDeclNode	241

pascal::GotoStmtNode	247
pascal::IfStmtNode	250
pascal::LabelStmtNode	260
pascal::NilNode	267
pascal::NumberNode	268
pascal::ParameterNode	275
pascal::PointerDerefNode	301
pascal::PointerTypeNode	302
pascal::ProcCallNode	304
pascal::ProcDeclNode	305
pascal::ProgramNode	369
pascal::RecordDeclarationNode	371
pascal::RecordTypeNode	375
pascal::RepeatStmtNode	378
pascal::SetLiteralNode	392
pascal::SetTypeNode	393
pascal::SimpleTypeNode	395
pascal::StringNode	406
pascal::TypeAliasNode	434
pascal::TypeDeclNode	436
pascal::UnaryOpNode	437
pascal::UnitNode	440
pascal::VarDeclNode	449
pascal::VariableNode	462
pascal::WhileStmtNode	465
pascal::WithStmtNode	466
mxvm::ASTVisitor	144
pascal::ASTVisitor	146
pascal::CodeGenVisitor	173
mxvm::Base	152
mxvm::Program	307
pascal::BuiltinFunctionHandler	168
pascal::FileFunctionHandler	236
pascal::IOFunctionHandler	257
pascal::SDLFunctionHandler	388
pascal::StdFunctionHandler	399
pascal::StringFunctionHandler	405
pascal::BuiltinFunctionRegistry	169
pascal::CaseStmtNode::CaseBranch	170
pascal::ConstDeclNode::ConstAssignment	216
expr_parser::ExprLexer	228
expr_parser::ExprParser	230
expr_parser::ExprToken	232
mxvm::ExternalFunction	233
pascal::CodeGenVisitor::FuncInfo	244
mxvm::Function< Sig >	244
mxvm::Function< R(Args...) >	244
mxvm::HTMLGen	249
mxvm::Instruction	252
mxvm::ModuleParser	263
mxvm::Operand	272
mxvm::OpSpec	274
mxvm::ParsedOp	276
mxvm::Parser	278
mxvm::PasHTMLGen	299
pascal::CodeGenVisitor::RecordField	373
pascal::CodeGenVisitor::RecordTypeInfo	374

std::runtime_error	
mx::Exception	223
pascal::ParseException	277
mxvm::RuntimeFunction	379
scan::ScanExcept	382
scan::Scanner	383
mxx::Scope	387
pascal::CodeGenVisitor::Scope	388
mxvm::Stack	397
scan::buffer::StringBuffer< Ch, String >	400
scan::token::Token< Ch, String >	408
scan::token::TokenMap< Ch, MAX_SIZE >	413
mxx::TPValidator	414
mxvm::UseLabel	442
mxvm::UseVar	442
mxvm::Validator	443
pascal::CodeGenVisitor::ValueLocation	448
mxvm::Variable	451
mxvm::Variable_Value	455
pascal::VariantArm	463
mxvm::Validator::VarNamePair	464
mxx::XParser	468
pascal::PascalParser	286

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

pascal::AddressOfNode	AST node for an address-of expression (@operand)	111
mx::ArgException< String >	Exception type thrown for argument parsing errors	112
Args		113
mx::Argument< String >	A parsed or registered command-line argument definition	115
mx::ArgumentData< String >	Raw argument data collected from argc/argv	118
mx::Arguments	Parsed SDL program arguments (resolution, path, fullscreen)	119
mx::Argz< String >	Generic command-line argument parser supporting single/double-dash options	120
pascal::ArrayAccessNode	AST node for an array element access (arr[index])	126
pascal::ArrayAssignmentNode	AST node for assigning to an array element (arr[index] := value)	128
pascal::ArrayDeclarationNode	AST node for a named array variable declaration	130
pascal::ArrayInfo	Metadata describing a Pascal array's bounds, element type, and sizing	132
pascal::ArrayTypeDeclarationNode	AST node for a named array type declaration (type Name = array[...])	134
pascal::ArrayTypeNode	AST node for an array type (array[lower..upper] of elementType)	136
pascal::AssignmentNode	AST node for an assignment statement (variable := expression)	138
mxvm::ASTNode	Abstract base class for all AST nodes	139
pascal::ASTNode	Abstract base class for all AST nodes	141
mxvm::ASTVisitor	Visitor interface for traversing the AST node hierarchy	144
pascal::ASTVisitor	Abstract visitor interface for the AST (double-dispatch)	146
mxvm::Base	Base class for instruction storage, variables, labels, and external function registrations	152
pascal::BinaryOpNode	AST node for a binary operator expression	159

pascal::BlockNode	AST node representing a block (declarations + compound statement)	162
pascal::BooleanNode	AST node for a boolean literal (true / false)	164
pascal::BreakNode	AST node for a break statement	166
pascal::BuiltinFunctionHandler	Abstract handler for a group of built-in functions/procedures	168
pascal::BuiltinFunctionRegistry	Registry of BuiltinFunctionHandler instances	169
pascal::CaseStmtNode::CaseBranch	A single case branch (one or more values mapping to a statement)	170
pascal::CaseStmtNode	AST node for a case statement	171
pascal::CodeGenVisitor	AST visitor that generates MXVM intermediate code	173
mxvm::CommentNode	AST node for a source-level comment	213
pascal::CompoundStmtNode	AST node for a compound statement (begin ... end)	214
pascal::ConstDeclNode::ConstAssignment	A single constant name = value pair	216
pascal::ConstDeclNode	AST node for constant declarations (const id = value)	217
pascal::ContinueNode	AST node for a continue statement	219
pascal::EmptyStmtNode	AST node for an empty statement (no-op)	220
pascal::EnumTypeDeclNode	AST node for an enumerated type declaration (type Color = (Red, Green, Blue))	221
mx::Exception	General-purpose exception with errno-aware factory method	223
pascal::ExitNode	AST node for an exit / halt statement	225
mxvm::ExpressionNode	AST node for a literal expression value	226
expr_parser::ExprLexer	Lexer for constant integer expressions	228
expr_parser::ExprParser	Recursive-descent parser and evaluator for constant integer expressions	230
expr_parser::ExprToken	A single expression token holding its type and optional numeric value	232
mxvm::ExternalFunction	Describes an external function imported from a module or object	233
pascal::FieldAccessNode	AST node for accessing a record field (record.field)	234
pascal::FileFunctionHandler	Built-in handler for Pascal file I/O procedures (assign, reset, rewrite, append, close, eof, eoln)	236
pascal::ForStmtNode	AST node for a for loop (for var := start to/downto end do stmt)	238
pascal::FuncCallNode	AST node for a function call expression	240
pascal::FuncDeclNode	AST node for a function declaration	241
pascal::CodeGenVisitor::FuncInfo	Describes a user-declared function's parameter and return types	244
mxvm::Function< Sig >		244

mxvm::Function< R(Args...) >	
RAII wrapper for dynamically loaded functions via dlopen/dlsym	244
pascal::GotoStmtNode	
AST node for a goto statement	247
mxvm::HTMLGen	
Generates syntax-highlighted HTML output from MXVM source code	249
pascal::IfStmtNode	
AST node for an if-then-else statement	250
mxvm::Instruction	
A complete MXVM instruction with opcode, operands, and optional label	252
mxvm::InstructionNode	
AST node representing a single MXVM instruction with its operands	255
pascal::IOFunctionHandler	
Built-in handler for I/O procedures (write, writeln, read, readln)	257
mxvm::LabelNode	
AST node for a label (or function entry point) in the code section	258
pascal::LabelStmtNode	
AST node for a label definition (label: statement)	260
mxvm::ModuleNode	
AST node for a module import declaration	261
mxvm::ModuleParser	
Parser for MXVM module definition files (.mxvm)	263
pascal::NilNode	
AST node for the nil literal	267
pascal::NumberNode	
AST node for an integer or real numeric literal	268
mxvm::ObjectNode	
AST node for an object import declaration	270
mxvm::Operand	
A single instruction operand (constant value or variable reference)	272
mxvm::OpSpec	
Specification of valid operand patterns for an instruction	274
pascal::ParameterNode	
AST node for a formal parameter list entry	275
mxvm::ParsedOp	
A parsed operand with its kind, text, and source location	276
pascal::ParseException	
Exception thrown on parse errors	277
mxvm::Parser	
Main MXVM parser — tokenizes source, builds AST, and generates intermediate code	278
pascal::PascalParser	
Recursive-descent parser for the Pascal language	286
mxvm::PasHTMLGen	
Generates syntax-highlighted HTML output from Pascal (.pas) source code	299
pascal::PointerDerefNode	
AST node for a pointer dereference (ptr [^])	301
pascal::PointerTypeNode	
AST node for a pointer type declaration (^BaseType)	302
pascal::ProcCallNode	
AST node for a procedure call statement	304
pascal::ProcDeclNode	
AST node for a procedure declaration	305
mxvm::Program	
Complete MXVM program: instruction interpreter and native x64 code generator	307
mxvm::ProgramNode	
Root AST node representing an entire program or object	367
pascal::ProgramNode	
AST node for a complete program (program name + block)	369

pascal::RecordDeclarationNode	
AST node for a named record declaration	371
pascal::CodeGenVisitor::RecordField	
A field within a record type definition	373
pascal::CodeGenVisitor::RecordTypeInfo	
Complete description of a record type	374
pascal::RecordTypeNode	
AST node for a record type (record ... end)	375
pascal::RepeatStmtNode	
AST node for a repeat..until loop	378
mxvm::RuntimeFunction	
Wraps a dynamically loaded native function from a shared library module	379
scan::ScanExcept	
Exception thrown during scanning errors	382
scan::Scanner	
Lexical scanner that tokenizes source text	383
mx::Scope	
A single scope level, tracking declared identifiers by category	387
pascal::CodeGenVisitor::Scope	
	388
pascal::SDLFunctionHandler	
Built-in handler for SDL2 bindings	388
mxvm::SectionNode	
Represents a program section (.data, .code, .module, .object)	390
pascal::SetLiteralNode	
AST node for a set literal [val1, val2, ...]	392
pascal::SetTypeNode	
AST node for a set type declaration (set of <base type>)	393
pascal::SimpleTypeNode	
AST node for a simple (named) type reference	395
mxvm::Stack	
Runtime operand stack for the MXVM interpreter	397
pascal::StdFunctionHandler	
Built-in handler for standard library functions (chr, ord, inc, dec, etc.)	399
scan::buffer::StringBuffer< Ch, String >	
Character-by-character string buffer with line/column tracking	400
pascal::StringFunctionHandler	
Built-in handler for string library functions (length, copy, concat, etc.)	405
pascal::StringNode	
AST node for a string literal	406
scan::token::Token< Ch, String >	
A scanned token with type, value, and source position	408
scan::token::TokenMap< Ch, MAX_SIZE >	
Maps characters to their lexical category (letter, digit, symbol, etc.)	413
mx::TPValidator	
Semantic validator for Pascal programs	414
pascal::TypeAliasNode	
AST node for a type alias (type NewName = ExistingType)	434
pascal::TypeDeclNode	
AST node for a type declaration section	436
pascal::UnaryOpNode	
AST node for a unary operator expression	437
pascal::UnitNode	
AST node for a Pascal unit (separately compiled module)	440
mxvm::UseLabel	
Records a label usage site for validation	442
mxvm::UseVar	
Records a variable usage site for validation	442

mxvm::Validator	Validates MXVM source for correct variable/label usage and instruction operands	443
pascal::CodeGenVisitor::ValueLocation	Describes where a value currently resides (register, memory, or immediate)	448
pascal::VarDeclNode	AST node for variable declarations (var id1, id2 : type)	449
mxvm::Variable	A named variable with type, value, and optional object association	451
mxvm::Variable_Value	Runtime value storage for a variable	455
mxvm::VariableNode	AST node for a variable declaration in a .data section	459
pascal::VariableNode	AST node for a variable reference	462
pascal::VariantArm	A single variant arm inside a variant record	463
mxvm::Validator::VarNamePair	A variable name paired with its declaration token	464
pascal::WhileStmtNode	AST node for a while loop (while cond do stmt)	465
pascal::WithStmtNode	AST node for a with statement (with record do statement)	466
mxx::XParser	Base parser providing token-stream navigation	468

Chapter 7

File Index

7.1 File List

Here is a list of all files with brief descriptions:

include/mxvm/ ast.hpp	473
AST node hierarchy for the MXVM parser using the Visitor pattern	
include/mxvm/ function.hpp	488
Dynamic function loader using dlopen/dlsym for runtime symbol resolution	
include/mxvm/ html_gen.hpp	490
HTML documentation generator from MXVM source code	
include/mxvm/ icode.hpp	491
Intermediate code representation, execution engine, and native x64 code generation (SysV + Win64)	
include/mxvm/ instruct.hpp	526
Instruction set enum, operand/instruction structs, variable types, and Variable/Variable_Value definitions	
include/mxvm/ mxvm.hpp	534
Main MXVM header aggregating all subsystem includes	
include/mxvm/ parser.hpp	535
Parser that tokenizes and parses MXVM source into programs, handles modules/objects, and dispatches code generation	
include/mxvm/ valid.hpp	542
Validator for MXVM programs — checks variable/label usage and instruction operand specifications	
include/mxvm/ version_info.hpp	545
modules/io/ io.c	546
I/O module C implementation — file operations, random numbers, and time functions	
modules/io/ io.cpp	547
I/O module C++ runtime bindings for the MXVM interpreter	
modules/sdl/ mx_sdl.h	554
SDL2 module C API — function declarations for SDL2 bindings exported to MXVM programs	
modules/sdl/ sdl.c	575
SDL2 module C implementation — window, renderer, texture, and event handling	
modules/sdl/ sdl.cpp	603
SDL2 module C++ runtime bindings for the MXVM interpreter	
modules/std/ mx_std.h	629
Standard library module C API — function declarations exported to MXVM programs	
modules/std/ std.c	631
Standard library module C implementation — math, string conversion, and utility functions	
modules/std/ std.cpp	635
Standard library module C++ runtime bindings for the MXVM interpreter	
modules/string/ cstring.c	655
String module C implementation — string manipulation, comparison, and formatting	

modules/string/string.cpp	
String module C++ runtime bindings for the MXVM interpreter	659
src/ast.cpp	
AST node accept() and toString() implementations for the MXVM Visitor pattern	669
src/function.cpp	
RuntimeFunction implementation — dlopen/dlsym dynamic function loading	854
src/html_gen.cpp	
Syntax-highlighted HTML generation from MXVM source code	855
src/icode.cpp	
Base and Program class core methods — instruction storage, variable management, and standard library registration	687
src/icode_exec.cpp	
MXVM interpreter — executes intermediate code instructions at runtime	866
src/icode_gen.cpp	
X86-64 System V ABI native code generator for Linux/macOS	693
src/icode_gen_x64.cpp	
X86-64 Win64 ABI native code generator for Windows	893
src/icode_opt.cpp	
Peephole optimizer for generated x86-64 assembly (SysV and Win64)	914
src/instruct.cpp	
Instruction toString(), keyword table, and opcode stream operator	921
src/mxvm.cpp	
MXVM namespace placeholder (reserved for future top-level utilities)	925
src/parser.cpp	
MXVM parser — tokenization, AST construction, and intermediate code generation	813
src/valid.cpp	
MXVM instruction operand validator and variable/label checker	947
src/frontend/ast.cpp	
Pascal AST node accept(), toString(), and print() implementations	671
src/frontend/expr.cpp	
Expression utility placeholder (reserved)	677
src/frontend/icode.cpp	
Pascal-to-MXVM code generator — CodeGenVisitor visit methods and assembly output	678
src/frontend/icode_gen.cpp	
	693
src/frontend/icode_imp.cpp	
Pascal code generator helper methods and builtin function handler implementations	717
src/frontend/icode_visit.cpp	
Pascal code generator AST visitor method implementations	731
src/frontend/main.cpp	
Pascal-to-MXVM compiler command-line entry point	777
src/frontend/parser.cpp	
Pascal recursive-descent parser implementation	797
src/frontend/validator.cpp	
Pascal semantic validator — scope tracking, type checking, and declaration verification	836
src/frontend/include/ast.hpp	
AST node hierarchy for the Pascal-to-MXVM frontend parser	476
src/frontend/include/expr.hpp	
Placeholder for Pascal expression utilities (reserved)	763
src/frontend/include/icode.hpp	
Pascal-to-MXVM code generator using the Visitor pattern over the Pascal AST	499
src/frontend/include/parser.hpp	
Pascal recursive-descent parser producing a Pascal AST	539
src/frontend/include/types.hpp	
Pascal keyword enumeration and string table	768
src/frontend/include/validator.hpp	
Semantic validator for Pascal programs — scope, type, and declaration checking	774
src/html_gen/html.cpp	
Mxvm-html command-line tool — converts MXVM source to syntax-highlighted HTML	862

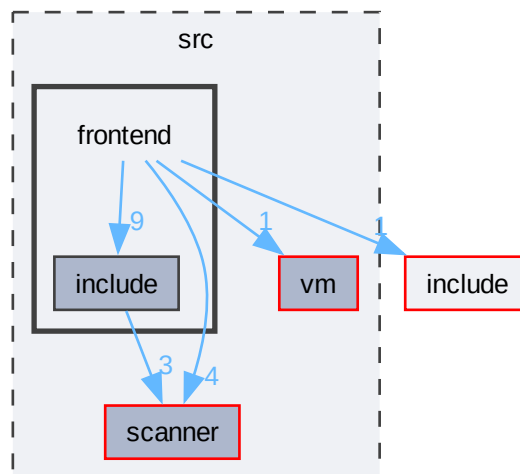
src/scanner/ exception.cpp	
Hex formatting utilities and exception helper implementations	926
src/scanner/ scanner.cpp	
Lexical scanner implementation — tokenizes source text into identifiers, numbers, symbols, and strings	938
src/scanner/ string_buffer.cpp	
StringBuffer line/column tracking and line-directive processing	945
src/scanner/ types.cpp	
Stream operators for TokenType and CharType enumerations	946
src/scanner/include/scanner/ exception.hpp	
Exception class, hex formatting utilities, and terminal color definitions	928
src/scanner/include/scanner/ expr.hpp	
Expression parser and evaluator for constant integer expressions	764
src/scanner/include/scanner/ scanner.hpp	
Lexical scanner that tokenizes source text into typed tokens	931
src/scanner/include/scanner/ string_buffer.hpp	
Character-by-character string buffer with line/column tracking and Token/TokenMap types	934
src/scanner/include/scanner/ types.hpp	
Token, character, and operator type enumerations for the scanner	770
src/vm/ main.cpp	
MXVM command-line entry point — argument parsing, interpretation, and compilation dispatch	783
src/vm/include/ argz.hpp	
Generic command-line argument parser with concept-constrained string types	958

Chapter 8

Directory Documentation

8.1 src/frontend Directory Reference

Directory dependency graph for frontend:



Directories

- directory [include](#)

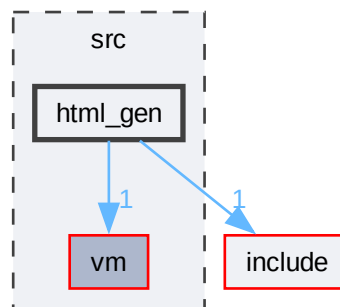
Files

- file [ast.cpp](#)
Pascal AST node accept(), toString(), and print() implementations.
- file [expr.cpp](#)
Expression utility placeholder (reserved).
- file [icode.cpp](#)
Pascal-to-MXVM code generator — CodeGenVisitor visit methods and assembly output.
- file [icode_gen.cpp](#)
- file [icode_imp.cpp](#)

- *Pascal code generator helper methods and builtin function handler implementations.*
file [icode_visit.cpp](#)
- *Pascal code generator AST visitor method implementations.*
file [main.cpp](#)
- *Pascal-to-MXVM compiler command-line entry point.*
file [parser.cpp](#)
- *Pascal recursive-descent parser implementation.*
file [validator.cpp](#)
- *Pascal semantic validator — scope tracking, type checking, and declaration verification.*

8.2 src/html_gen Directory Reference

Directory dependency graph for html_gen:

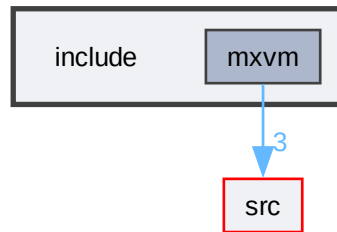


Files

- file [html.cpp](#)
mxvm-html command-line tool — converts MXVM source to syntax-highlighted HTML

8.3 include Directory Reference

Directory dependency graph for include:

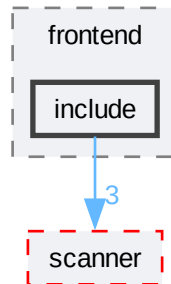


Directories

- directory [mxvm](#)

8.4 src/frontend/include Directory Reference

Directory dependency graph for include:



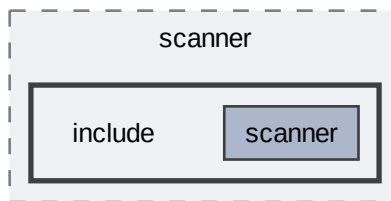
Files

- file [ast.hpp](#)
AST node hierarchy for the Pascal-to-MXVM frontend parser.
- file [expr.hpp](#)
Placeholder for Pascal expression utilities (reserved).
- file [icode.hpp](#)
Pascal-to-MXVM code generator using the Visitor pattern over the Pascal AST.
- file [parser.hpp](#)
Pascal recursive-descent parser producing a Pascal AST.

- file [types.hpp](#)
Pascal keyword enumeration and string table.
- file [validator.hpp](#)
Semantic validator for Pascal programs — scope, type, and declaration checking.

8.5 src/scanner/include Directory Reference

Directory dependency graph for include:

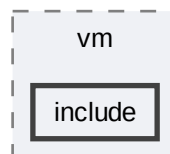


Directories

- directory [scanner](#)

8.6 src/vm/include Directory Reference

Directory dependency graph for include:

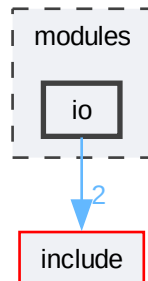


Files

- file [argz.hpp](#)
Generic command-line argument parser with concept-constrained string types.

8.7 modules/io Directory Reference

Directory dependency graph for io:

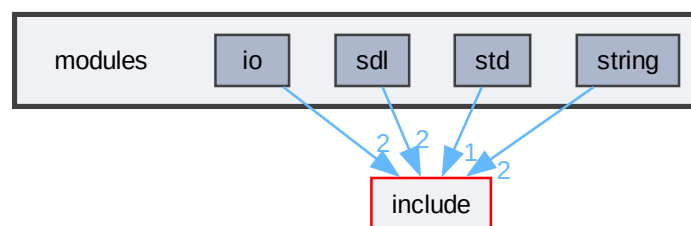


Files

- file [io.c](#)
I/O module C implementation — file operations, random numbers, and time functions.
- file [io.cpp](#)
I/O module C++ runtime bindings for the MXVM interpreter.

8.8 modules Directory Reference

Directory dependency graph for modules:

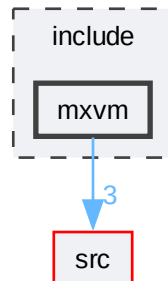


Directories

- directory [io](#)
- directory [sdl](#)
- directory [std](#)
- directory [string](#)

8.9 include/mxvm Directory Reference

Directory dependency graph for mxvm:

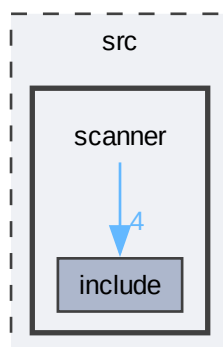


Files

- file [ast.hpp](#)
AST node hierarchy for the MXVM parser using the Visitor pattern.
- file [function.hpp](#)
Dynamic function loader using dlopen/dlsym for runtime symbol resolution.
- file [html_gen.hpp](#)
HTML documentation generator from MXVM source code.
- file [icode.hpp](#)
Intermediate code representation, execution engine, and native x64 code generation (SysV + Win64).
- file [instruct.hpp](#)
Instruction set enum, operand/instruction structs, variable types, and Variable/Variable_Value definitions.
- file [mxvm.hpp](#)
Main MXVM header aggregating all subsystem includes.
- file [parser.hpp](#)
Parser that tokenizes and parses MXVM source into programs, handles modules/objects, and dispatches code generation.
- file [valid.hpp](#)
Validator for MXVM programs — checks variable/label usage and instruction operand specifications.
- file [version_info.hpp](#)

8.10 src/scanner Directory Reference

Directory dependency graph for scanner:



Directories

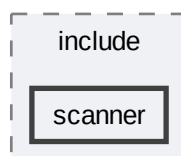
- directory [include](#)

Files

- file [exception.cpp](#)
Hex formatting utilities and exception helper implementations.
- file [scanner.cpp](#)
Lexical scanner implementation — tokenizes source text into identifiers, numbers, symbols, and strings.
- file [string_buffer.cpp](#)
StringBuffer line/column tracking and line-directive processing.
- file [types.cpp](#)
Stream operators for TokenType and CharType enumerations.

8.11 src/scanner/include/scanner Directory Reference

Directory dependency graph for scanner:

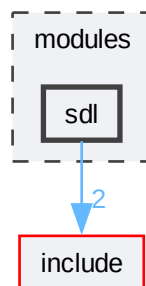


Files

- file [exception.hpp](#)
Exception class, hex formatting utilities, and terminal color definitions.
- file [expr.hpp](#)
Expression parser and evaluator for constant integer expressions.
- file [scanner.hpp](#)
Lexical scanner that tokenizes source text into typed tokens.
- file [string_buffer.hpp](#)
Character-by-character string buffer with line/column tracking and Token/TokenMap types.
- file [types.hpp](#)
Token, character, and operator type enumerations for the scanner.

8.12 modules/sdl Directory Reference

Directory dependency graph for sdl:

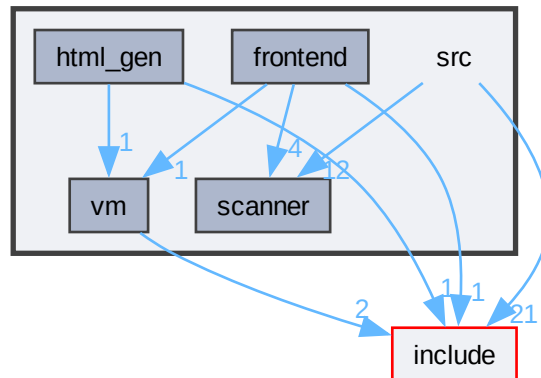


Files

- file [mx_sdl.h](#)
SDL2 module C API — function declarations for SDL2 bindings exported to MXVM programs.
- file [sdl.c](#)
SDL2 module C implementation — window, renderer, texture, and event handling.
- file [sdl.cpp](#)
SDL2 module C++ runtime bindings for the MXVM interpreter.

8.13 src Directory Reference

Directory dependency graph for src:



Directories

- directory [frontend](#)
- directory [html_gen](#)
- directory [scanner](#)
- directory [vm](#)

Files

- file [ast.cpp](#)
AST node accept() and toString() implementations for the MXVM Visitor pattern.
- file [function.cpp](#)
RuntimeFunction implementation — dlopen/dlsym dynamic function loading.
- file [html_gen.cpp](#)
Syntax-highlighted HTML generation from MXVM source code.
- file [icode.cpp](#)
Base and Program class core methods — instruction storage, variable management, and standard library registration.
- file [icode_exec.cpp](#)
MXVM interpreter — executes intermediate code instructions at runtime.
- file [icode_gen.cpp](#)
x86-64 System V ABI native code generator for Linux/macOS
- file [icode_gen_x64.cpp](#)
x86-64 Win64 ABI native code generator for Windows
- file [icode_opt.cpp](#)
Peephole optimizer for generated x86-64 assembly (SysV and Win64).
- file [instruct.cpp](#)
Instruction toString(), keyword table, and opcode stream operator.
- file [mxvm.cpp](#)
MXVM namespace placeholder (reserved for future top-level utilities).
- file [parser.cpp](#)

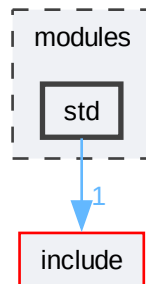
MXVM parser — tokenization, AST construction, and intermediate code generation.

- file [valid.cpp](#)

MXVM instruction operand validator and variable/label checker.

8.14 modules/std Directory Reference

Directory dependency graph for std:



Files

- file [mx_std.h](#)

Standard library module C API — function declarations exported to MXVM programs.

- file [std.c](#)

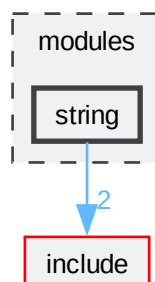
Standard library module C implementation — math, string conversion, and utility functions.

- file [std.cpp](#)

Standard library module C++ runtime bindings for the MXVM interpreter.

8.15 modules/string Directory Reference

Directory dependency graph for string:

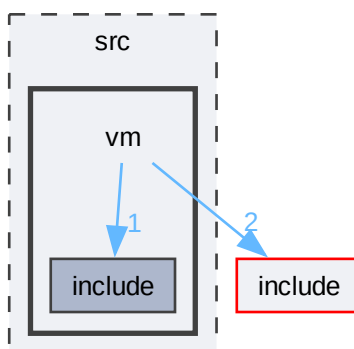


Files

- file [cstring.c](#)
String module C implementation — string manipulation, comparison, and formatting.
- file [string.cpp](#)
String module C++ runtime bindings for the MXVM interpreter.

8.16 src/vm Directory Reference

Directory dependency graph for vm:



Directories

- directory [include](#)

Files

- file [main.cpp](#)
MXVM command-line entry point — argument parsing, interpretation, and compilation dispatch.

Chapter 9

Namespace Documentation

9.1 `expr_parser` Namespace Reference

Classes

- struct [ExprToken](#)
A single expression token holding its type and optional numeric value.
- class [ExprLexer](#)
Lexer for constant integer expressions.
- class [ExprParser](#)
Recursive-descent parser and evaluator for constant integer expressions.

Enumerations

- enum class [ExprTokenType](#) {
 [END](#) , [NUMBER](#) , [PLUS](#) , [MINUS](#) ,
 [MUL](#) , [DIV](#) , [MOD](#) , [LPAREN](#) ,
 [RPAREN](#) , [PLUSEQ](#) , [MINUSEQ](#) , [XOR](#) ,
 [OR](#) , [AND](#) , [LOGICAL_AND](#) , [LOGICAL_OR](#) ,
 [NOT](#) , [EQ](#) , [NEQ](#) , [GT](#) ,
 [GTE](#) , [LT](#) , [LTE](#) , [VARIABLE](#) }
Token types for the constant expression parser.

Variables

- `std::unordered_map< std::string, uint64_t >` [vars](#)
Global variable map for the expression evaluator.

9.1.1 Enumeration Type Documentation

9.1.1.1 `ExprTokenType`

enum class `expr_parser::ExprTokenType` [strong]
Token types for the constant expression parser.

Enumerator

END	end of expression
NUMBER	numeric literal
PLUS	'+'
MINUS	'-'

MUL	'*'
DIV	'/'
MOD	"
LPAREN	'('
RPAREN	)'
PLUSEQ	'+='
MINUSEQ	'-='
XOR	'^'
OR	' '
AND	'&'
LOGICAL_AND	'&&'
LOGICAL_OR	' '
NOT	'!'
EQ	'=='
NEQ	'!='
GT	'>'
GTE	'>='
LT	'<'
LTE	'<='
VARIABLE	named variable reference

Definition at line 16 of file [expr.hpp](#).

9.1.2 Variable Documentation

9.1.2.1 vars

```
std::unordered_map<std::string, uint64_t> expr_parser::vars [inline]
```

Global variable map for the expression evaluator.

Definition at line 50 of file [expr.hpp](#).

Referenced by [expr_parser::ExprParser::parseFactor\(\)](#).

9.2 mx Namespace Reference

Namespaces

- namespace [Color](#)
ANSI escape code constants for terminal coloring.

Classes

- class [Exception](#)
General-purpose exception with errno-aware factory method.
- struct [Argument](#)
A parsed or registered command-line argument definition.
- struct [ArgumentData](#)
Raw argument data collected from argc/argv.
- class [ArgException](#)
Exception type thrown for argument parsing errors.

- class [Argz](#)
Generic command-line argument parser supporting single/double-dash options.
- struct [Arguments](#)
Parsed SDL program arguments (resolution, path, fullscreen).

Concepts

- concept [StringType](#)
Concept constraining String template parameters to class types with `length()`, `operator[]`, `operator+=`, and `operator=` (e.g. `std::string`, `std::wstring`).

Enumerations

- enum class [ArgType](#) {
[ARG_SINGLE](#) , [ARG_SINGLE_VALUE](#) , [ARG_DOUBLE](#) , [ARG_DOUBLE_VALUE](#) ,
[ARG_NONE](#) }
Discriminates argument switch types (single-dash, double-dash, with/without value).

Functions

- `std::string` [format_hex64](#) (uint64_t value)
Format a 64-bit value as a hex string with 0x prefix.
- `std::string` [format_hex32](#) (uint32_t value)
Format a 32-bit value as a hex string with 0x prefix.
- `std::string` [format_hex16](#) (uint16_t value)
Format a 16-bit value as a hex string with 0x prefix.
- `std::string` [format_hex8](#) (uint8_t value)
Format an 8-bit value as a hex string with 0x prefix.
- `std::string` [format_hex_no_prefix](#) (uint64_t value)
Format a 64-bit value as a hex string without 0x prefix.
- `std::string` [format_signal](#) (uint32_t sig)
Format a signal number as a descriptive string.
- `bool` [terminal_supports_color](#) ()
Check whether stdout is a TTY that supports ANSI color.
- [Arguments](#) [proc_args](#) (int &[argc](#), char **[argv](#))
Parse SDL-specific command-line arguments.

Variables

- `bool` [color_](#) = [terminal_supports_color](#)()
cached color support flag

9.2.1 Enumeration Type Documentation

9.2.1.1 ArgType

`enum class mx::ArgType` [strong]

Discriminates argument switch types (single-dash, double-dash, with/without value).

Enumerator

ARG_SINGLE	
ARG_SINGLE_VALUE	

ARG_DOUBLE	
ARG_DOUBLE_VALUE	
ARG_NONE	

Definition at line 39 of file [argz.hpp](#).

9.2.2 Function Documentation

9.2.2.1 format_hex16()

```
std::string mx::format_hex16 (
    uint16_t value)
```

Format a 16-bit value as a hex string with 0x prefix.

Definition at line 26 of file [exception.cpp](#).

9.2.2.2 format_hex32()

```
std::string mx::format_hex32 (
    uint32_t value)
```

Format a 32-bit value as a hex string with 0x prefix.

Definition at line 20 of file [exception.cpp](#).

9.2.2.3 format_hex64()

```
std::string mx::format_hex64 (
    uint64_t value)
```

Format a 64-bit value as a hex string with 0x prefix.

Definition at line 14 of file [exception.cpp](#).

9.2.2.4 format_hex8()

```
std::string mx::format_hex8 (
    uint8_t value)
```

Format an 8-bit value as a hex string with 0x prefix.

Definition at line 32 of file [exception.cpp](#).

9.2.2.5 format_hex_no_prefix()

```
std::string mx::format_hex_no_prefix (
    uint64_t value)
```

Format a 64-bit value as a hex string without 0x prefix.

Definition at line 38 of file [exception.cpp](#).

9.2.2.6 format_signal()

```
std::string mx::format_signal (
    uint32_t sig)
```

Format a signal number as a descriptive string.

Definition at line 44 of file [exception.cpp](#).

9.2.2.7 proc_args()

```
Arguments mx::proc_args (
    int & argc,
    char ** argv) [inline]
```

Parse SDL-specific command-line arguments.

Parameters

<i>argc</i>	Argument count (may be modified)
<i>argv</i>	Argument vector

Returns

Parsed [Arguments](#) struct

Definition at line 496 of file [argz.hpp](#).

References [mx::Argz<String>::addOptionSingle\(\)](#), [mx::Argument<String>::arg_value](#), [argc\(\)](#), [argv\(\)](#), [mx::Arguments::fullscreen](#), [mx::Arguments::height](#), [mx::Argz<String>::help\(\)](#), [mx::Arguments::path](#), [pos\(\)](#), [mx::Argz<String>::proc\(\)](#), [mx::ArgException<String>::text\(\)](#), and [mx::Arguments::width](#).

9.2.2.8 terminal_supports_color()

```
bool mx::terminal_supports_color () [inline]
```

Check whether stdout is a TTY that supports ANSI color.

Definition at line 88 of file [exception.hpp](#).

9.2.3 Variable Documentation**9.2.3.1 color_**

```
bool mx::color_ = terminal_supports_color () [inline]
```

cached color support flag

Definition at line 91 of file [exception.hpp](#).

Referenced by [Col\(\)](#).

9.3 mx::Color Namespace Reference

ANSI escape code constants for terminal coloring.

Variables

- const std::string [RESET](#) = "\033[0m"
- const std::string [BOLD](#) = "\033[1m"
- const std::string [DIM](#) = "\033[2m"
- const std::string [BLACK](#) = "\033[30m"
- const std::string [RED](#) = "\033[31m"
- const std::string [GREEN](#) = "\033[32m"
- const std::string [YELLOW](#) = "\033[33m"
- const std::string [BLUE](#) = "\033[34m"
- const std::string [MAGENTA](#) = "\033[35m"
- const std::string [CYAN](#) = "\033[36m"
- const std::string [WHITE](#) = "\033[37m"
- const std::string [BRIGHT_RED](#) = "\033[91m"
- const std::string [BRIGHT_GREEN](#) = "\033[92m"
- const std::string [BRIGHT_YELLOW](#) = "\033[93m"
- const std::string [BRIGHT_BLUE](#) = "\033[94m"
- const std::string [BRIGHT_MAGENTA](#) = "\033[95m"
- const std::string [BRIGHT_CYAN](#) = "\033[96m"
- const std::string [BG_RED](#) = "\033[41m"
- const std::string [BG_GREEN](#) = "\033[42m"
- const std::string [BG_YELLOW](#) = "\033[43m"

9.3.1 Detailed Description

ANSI escape code constants for terminal coloring.

9.3.2 Variable Documentation

9.3.2.1 BG_GREEN

```
const std::string mx::Color::BG_GREEN = "\033[42m"
```

Definition at line 84 of file [exception.hpp](#).

9.3.2.2 BG_RED

```
const std::string mx::Color::BG_RED = "\033[41m"
```

Definition at line 83 of file [exception.hpp](#).

9.3.2.3 BG_YELLOW

```
const std::string mx::Color::BG_YELLOW = "\033[43m"
```

Definition at line 85 of file [exception.hpp](#).

9.3.2.4 BLACK

```
const std::string mx::Color::BLACK = "\033[30m"
```

Definition at line 69 of file [exception.hpp](#).

9.3.2.5 BLUE

```
const std::string mx::Color::BLUE = "\033[34m"
```

Definition at line 73 of file [exception.hpp](#).

9.3.2.6 BOLD

```
const std::string mx::Color::BOLD = "\033[1m"
```

Definition at line 67 of file [exception.hpp](#).

9.3.2.7 BRIGHT_BLUE

```
const std::string mx::Color::BRIGHT_BLUE = "\033[94m"
```

Definition at line 80 of file [exception.hpp](#).

Referenced by [mxvm::Program::generateCode\(\)](#), [print_help\(\)](#), and [mxvm::Program::x64_generateCode\(\)](#).

9.3.2.8 BRIGHT_CYAN

```
const std::string mx::Color::BRIGHT_CYAN = "\033[96m"
```

Definition at line 82 of file [exception.hpp](#).

Referenced by [mxvm::Program::generateCode\(\)](#), [print_help\(\)](#), and [mxvm::Program::x64_generateCode\(\)](#).

9.3.2.9 BRIGHT_GREEN

```
const std::string mx::Color::BRIGHT_GREEN = "\033[92m"
```

Definition at line 78 of file [exception.hpp](#).

Referenced by [action_interpret\(\)](#), [createMakefile\(\)](#), [mxvm::Parser::generateProgramCode\(\)](#), and [translate_x64\(\)](#).

9.3.2.10 BRIGHT_MAGENTA

```
const std::string mx::Color::BRIGHT_MAGENTA = "\033[95m"
```

Definition at line 81 of file [exception.hpp](#).

9.3.2.11 BRIGHT_RED

```
const std::string mx::Color::BRIGHT_RED = "\033[91m"
```

Definition at line 77 of file [exception.hpp](#).

Referenced by [createMakefile\(\)](#).

9.3.2.12 BRIGHT_YELLOW

```
const std::string mx::Color::BRIGHT_YELLOW = "\033[93m"
```

Definition at line 79 of file [exception.hpp](#).

9.3.2.13 CYAN

```
const std::string mx::Color::CYAN = "\033[36m"
```

Definition at line 75 of file [exception.hpp](#).

9.3.2.14 DIM

```
const std::string mx::Color::DIM = "\033[2m"
```

Definition at line 68 of file [exception.hpp](#).

9.3.2.15 GREEN

```
const std::string mx::Color::GREEN = "\033[32m"
```

Definition at line 71 of file [exception.hpp](#).

9.3.2.16 MAGENTA

```
const std::string mx::Color::MAGENTA = "\033[35m"
```

Definition at line 74 of file [exception.hpp](#).

9.3.2.17 RED

```
const std::string mx::Color::RED = "\033[31m"
```

Definition at line 70 of file [exception.hpp](#).

Referenced by [mxvm::Program::~~Program\(\)](#), [action_interpret\(\)](#), [proc_args\(\)](#), [process_arguments\(\)](#), and [translate_x64\(\)](#).

9.3.2.18 RESET

```
const std::string mx::Color::RESET = "\033[0m"
```

Definition at line 66 of file [exception.hpp](#).

Referenced by [Col\(\)](#).

9.3.2.19 WHITE

```
const std::string mx::Color::WHITE = "\033[37m"
```

Definition at line 76 of file [exception.hpp](#).

9.3.2.20 YELLOW

```
const std::string mx::Color::YELLOW = "\033[33m"
```

Definition at line 72 of file [exception.hpp](#).

9.4 mxvm Namespace Reference

Classes

- class [ASTNode](#)

- Abstract base class for all AST nodes.*
- class [SectionNode](#)
Represents a program section (.data, .code, .module, .object).
- class [ProgramNode](#)
Root AST node representing an entire program or object.
- class [VariableNode](#)
AST node for a variable declaration in a .data section.
- class [ModuleNode](#)
AST node for a module import declaration.
- class [ObjectNode](#)
AST node for an object import declaration.
- class [InstructionNode](#)
AST node representing a single MXVM instruction with its operands.
- class [LabelNode](#)
AST node for a label (or function entry point) in the code section.
- class [ExpressionNode](#)
AST node for a literal expression value.
- class [CommentNode](#)
AST node for a source-level comment.
- class [ASTVisitor](#)
Visitor interface for traversing the AST node hierarchy.
- class [Function](#)
- class [Function< R\(Args...\)>](#)
RAII wrapper for dynamically loaded functions via dlopen/dlsym.
- class [HTMLGen](#)
Generates syntax-highlighted HTML output from MXVM source code.
- class [PasHTMLGen](#)
Generates syntax-highlighted HTML output from Pascal (.pas) source code.
- class [Stack](#)
Runtime operand stack for the MXVM interpreter.
- class [RuntimeFunction](#)
Wraps a dynamically loaded native function from a shared library module.
- class [Base](#)
Base class for instruction storage, variables, labels, and external function registrations.
- class [Program](#)
Complete MXVM program: instruction interpreter and native x64 code generator.
- struct [Operand](#)
A single instruction operand (constant value or variable reference).
- struct [Instruction](#)
A complete MXVM instruction with opcode, operands, and optional label.
- struct [Variable_Value](#)
Runtime value storage for a variable.
- struct [Variable](#)
A named variable with type, value, and optional object association.
- class [Parser](#)
Main MXVM parser — tokenizes source, builds AST, and generates intermediate code.
- struct [ExternalFunction](#)
Describes an external function imported from a module or object.
- class [ModuleParser](#)
Parser for MXVM module definition files (.mxvm).
- struct [ParsedOp](#)

- *A parsed operand with its kind, text, and source location.*
- struct [UseVar](#)
Records a variable usage site for validation.
- struct [UseLabel](#)
Records a label usage site for validation.
- struct [OpSpec](#)
Specification of valid operand patterns for an instruction.
- class [Validator](#)
Validates MXVM source for correct variable/label usage and instruction operands.

Typedefs

- using [StackValue](#) = std::variant<int64_t, void *, double, std::string>
Stack element: integer, pointer, double, or string.
- using [runtime_call](#) = std::function<void([Program](#) *program, std::vector<[Operand](#)> &operands)>
Callback type for runtime-registered native functions.

Enumerations

- enum class [OperandType](#) { [OP_CONSTANT](#) , [OP_VARIABLE](#) }
Discriminates whether an operand is a compile-time constant or a variable reference.
- enum class [VarType](#) {
[VAR_NULL](#) = 0 , [VAR_INTEGER](#) , [VAR_FLOAT](#) , [VAR_STRING](#) ,
[VAR_POINTER](#) , [VAR_LABEL](#) , [VAR_ARRAY](#) , [VAR_EXTERN](#) ,
[VAR_BYTE](#) }
MXVM variable type discriminator.
- enum class [Keywords](#) {
[KEY_PROGRAM](#) , [KEY_OBJECT](#) , [KEY_MODULE](#) , [KEY_DATA](#) ,
[KEY_CODE](#) , [KEY_SECTION](#) }
MXVM language keywords.
- enum class [Mode](#) { [MODE_INTERPRET](#) , [MODE_COMPILE](#) }
Execution mode: interpretation or native compilation.
- enum class [Platform](#) { [LINUX](#) , [DARWIN](#) , [WINX64](#) }
Target platform for native code generation.
- enum class [OpKind](#) {
[Num](#) , [Hex](#) , [Str](#) , [Id](#) ,
[Member](#) , [Label](#) , [Any](#) }
Classification of operand kinds for validation.
- enum class [VArity](#) { [None](#) , [AnyTail](#) , [ArgsTail](#) }
Variadic arity policy for instruction operand specs.

Functions

- void [except_assert](#) (std::string reason, bool value)
Assert a condition, throwing an Exception with the given reason on failure.
- bool [isFunctionReturningOwnedPtr](#) (const std::string &funcName)
Check whether a named function returns an owned (heap-allocated) pointer.
- static std::unordered_set< std::string > [buildInstructionSet](#) ()
- static std::unordered_set< std::string > [buildKeywordSet](#) ()
- static void [releaseOwnedPointer](#) ([Variable](#) &v)
- std::string [toStringFromVarType](#) (const [VarType](#) &v)
- static bool [is_stdio_name](#) (const std::string &s)
- static int [stdio_index](#) (const std::string &s)
- static std::string [join_lines](#) (const std::vector< std::string > &v)

- static bool [is_label](#) (const std::string &line)
- static std::vector< std::string > [opt_core_lines](#) (const std::vector< std::string > &lines)
- static std::vector< std::string > [x64_opt_core_lines](#) (const std::vector< std::string > &lines)
- static std::string [darwin_prefix_calls](#) (const std::string &line, const std::unordered_set< std::string > &macos_functions)
- static std::vector< std::string > [opt_darwin_lines](#) (const std::vector< std::string > &lines)
- static std::vector< std::string > [opt_x64_windows_lines](#) (const std::vector< std::string > &lines)
- static std::vector< std::string > [opt_linux_lines](#) (const std::vector< std::string > &lines)
- static bool [isImmediate](#) (const [OpKind](#) k)
- static bool [has_semicolon](#) (const std::string &s)
- static bool [isIdLike](#) ([OpKind](#) k)

Variables

- std::vector< std::string > [keywords](#) {"program", "object", "module", "data", "code", "section"}
String representations of [Keywords](#) enum values.
- bool [debug_mode](#) = false
enable verbose debug output during parsing
- bool [instruct_mode](#) = false
enable instruction trace mode
- bool [html_mode](#) = false
enable HTML debug output
- static const std::unordered_set< std::string > [kInstructions](#) = [buildInstructionSet](#)()
- static const std::unordered_set< std::string > [kKeywords](#) = [buildKeywordSet](#)()
- static const std::unordered_set< std::string > [kTypes](#)
- static int [error_label_count](#) = 0
- static unsigned [x64_sp_mod16](#) = 0
- size_t [xmm_offset](#)
- static int [error_label_count](#) = 0
- static const std::unordered_map< std::string, [OpSpec](#) > [kOpSpecs](#)

9.4.1 Typedef Documentation

9.4.1.1 runtime_call

using [mxvm::runtime_call](#) = std::function<void([Program](#) *program, std::vector<[Operand](#)> &operands)>

Callback type for runtime-registered native functions.

Definition at line 70 of file [icode.hpp](#).

9.4.1.2 StackValue

using [mxvm::StackValue](#) = std::variant<int64_t, void *, double, std::string>

[Stack](#) element: integer, pointer, double, or string.

Definition at line 20 of file [icode.hpp](#).

9.4.2 Enumeration Type Documentation

9.4.2.1 Keywords

enum class [mxvm::Keywords](#) [strong]

MXVM language keywords.

Enumerator

KEY_PROGRAM	
-------------	--

KEY_OBJECT	
KEY_MODULE	
KEY_DATA	
KEY_CODE	
KEY_SECTION	

Definition at line 192 of file [instruct.hpp](#).

9.4.2.2 Mode

```
enum class mxvm::Mode [strong]
```

Execution mode: interpretation or native compilation.

Enumerator

MODE_INTERPRET	
MODE_COMPILE	

Definition at line 36 of file [parser.hpp](#).

9.4.2.3 OperandType

```
enum class mxvm::OperandType [strong]
```

Discriminates whether an operand is a compile-time constant or a variable reference.

Enumerator

OP_CONSTANT	
OP_VARIABLE	

Definition at line 153 of file [instruct.hpp](#).

9.4.2.4 OpKind

```
enum class mxvm::OpKind [strong]
```

Classification of operand kinds for validation.

Enumerator

Num	
Hex	
Str	
Id	
Member	
Label	
Any	

Definition at line 19 of file [valid.hpp](#).

9.4.2.5 Platform

```
enum class mxvm::Platform [strong]
```

Target platform for native code generation.

Enumerator

LINUX	
DARWIN	
WINX64	

Definition at line 42 of file [parser.hpp](#).

9.4.2.6 VArity

```
enum class mxvm::VArity [strong]
```

Variadic arity policy for instruction operand specs.

Enumerator

None	exact fixed operand count required
AnyTail	fixed operands followed by any number of trailing operands
ArgsTail	fixed operands followed by argument-typed trailing operands

Definition at line 48 of file [valid.hpp](#).

9.4.2.7 VarType

```
enum class mxvm::VarType [strong]
```

MXVM variable type discriminator.

Enumerator

VAR_NULL	
VAR_INTEGER	
VAR_FLOAT	
VAR_STRING	
VAR_POINTER	
VAR_LABEL	
VAR_ARRAY	
VAR_EXTERN	
VAR_BYTE	

Definition at line 180 of file [instruct.hpp](#).

9.4.3 Function Documentation**9.4.3.1 buildInstructionSet()**

```
std::unordered_set< std::string > mxvm::buildInstructionSet () [static]
```

Definition at line 56 of file [html_gen.cpp](#).

References [IncType](#).

9.4.3.2 buildKeywordSet()

```
std::unordered_set< std::string > mxvm::buildKeywordSet () [static]
```

Definition at line 66 of file [html_gen.cpp](#).

References [keywords](#).

9.4.3.3 darwin_prefix_calls()

```
std::string mxvm::darwin_prefix_calls (
    const std::string & line,
    const std::unordered_set< std::string > & macos_functions) [static]
```

Definition at line 329 of file [icode_opt.cpp](#).

References [pos\(\)](#).

Referenced by [opt_darwin_lines\(\)](#).

9.4.3.4 except_assert()

```
void mxvm::except_assert (
    std::string reason,
    bool value)
```

Assert a condition, throwing an Exception with the given reason on failure.

Parameters

<i>reason</i>	Error message
<i>value</i>	Condition to assert (true = pass)

Definition at line 436 of file [icode.cpp](#).

Referenced by [asReadPtr\(\)](#), [asWritePtr\(\)](#), [getStringFromVar\(\)](#), [mxvm_std_memcmp\(\)](#), [mxvm_std_memcpy\(\)](#), [mxvm_std_memmove\(\)](#), [mxvm_std_memset\(\)](#), [mxvm_string_snprintf\(\)](#), [mxvm_string_strcmp\(\)](#), [mxvm_string_strlen\(\)](#), and [mxvm_string_substr\(\)](#).

9.4.3.5 has_semicolon()

```
bool mxvm::has_semicolon (
    const std::string & s) [static]
```

Definition at line 20 of file [valid.cpp](#).

Referenced by [mxvm::Validator::validate\(\)](#).

9.4.3.6 is_label()

```
bool mxvm::is_label (
    const std::string & line) [static]
```

Definition at line 33 of file [icode_opt.cpp](#).

Referenced by [opt_core_lines\(\)](#), and [mxvm::HTMLGen::output\(\)](#).

9.4.3.7 is_stdio_name()

```
bool mxvm::is_stdio_name (
    const std::string & s) [inline], [static]
```

Definition at line 32 of file [icode_gen_x64.cpp](#).

Referenced by [mxvm::Program::x64_analyzeRegAlloc\(\)](#), [mxvm::Program::x64_generateLoadVar\(\)](#), and [mxvm::Program::x64_generate...](#)

9.4.3.8 isFunctionReturningOwnedPtr()

```
bool mxvm::isFunctionReturningOwnedPtr (
    const std::string & funcName)
```

Check whether a named function returns an owned (heap-allocated) pointer.

Parameters

<i>funcName</i>	Function name to check
-----------------	--

Returns

true if the function returns an owned pointer

Definition at line 249 of file [icode_gen_x64.cpp](#).

Referenced by [mxvm::Program::generateInvokeCall\(\)](#), and [mxvm::Program::x64_generateInvokeCall\(\)](#).

9.4.3.9 isIdLike()

```
bool mxvm::isIdLike (
    OpKind k) [static]
```

Definition at line 140 of file [valid.cpp](#).

References [Id](#), and [Member](#).

Referenced by [mxvm::Validator::validateAgainstSpec\(\)](#).

9.4.3.10 isImmediate()

```
bool mxvm::isImmediate (
    const OpKind k) [inline], [static]
```

Definition at line 16 of file [valid.cpp](#).

References [Hex](#), [Num](#), and [Str](#).

Referenced by [mxvm::Validator::validateAgainstSpec\(\)](#).

9.4.3.11 join_lines()

```
std::string mxvm::join_lines (
    const std::vector< std::string > & v) [static]
```

Definition at line 16 of file [icode_opt.cpp](#).

Referenced by [mxvm::Program::gen_optimize\(\)](#).

9.4.3.12 opt_core_lines()

```
std::vector< std::string > mxvm::opt_core_lines (
    const std::vector< std::string > & lines) [static]
```

Definition at line 38 of file [icode_opt.cpp](#).

References [is_label\(\)](#).

Referenced by [mxvm::Program::gen_optimize\(\)](#).

9.4.3.13 opt_darwin_lines()

```
std::vector< std::string > mxvm::opt_darwin_lines (
    const std::vector< std::string > & lines) [static]
```

Definition at line 358 of file [icode_opt.cpp](#).

References [darwin_prefix_calls\(\)](#).

Referenced by [mxvm::Program::gen_optimize\(\)](#).

9.4.3.14 opt_linux_lines()

```
std::vector< std::string > mxvm::opt_linux_lines (
    const std::vector< std::string > & lines) [static]
```

Definition at line 492 of file [icode_opt.cpp](#).

Referenced by [mxvm::Program::gen_optimize\(\)](#).

9.4.3.15 opt_x64_windows_lines()

```
std::vector< std::string > mxvm::opt_x64_windows_lines (
    const std::vector< std::string > & lines) [static]
```

Definition at line 452 of file [icode_opt.cpp](#).

Referenced by [mxvm::Program::gen_optimize\(\)](#).

9.4.3.16 releaseOwnedPointer()

```
void mxvm::releaseOwnedPointer (
    Variable & v) [inline], [static]
```

Definition at line 13 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_count](#), [mxvm::Variable_Value::ptr_size](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [mxvm::Program::exec_alloc\(\)](#), [mxvm::Program::exec_mov\(\)](#), and [mxvm::Program::exec_return\(\)](#).

9.4.3.17 stdio_index()

```
int mxvm::stdio_index (
    const std::string & s) [inline], [static]
```

Definition at line 35 of file [icode_gen_x64.cpp](#).

Referenced by [mxvm::Program::x64_generateLoadVar\(\)](#), and [mxvm::Program::x64_generateLoadVar\(\)](#).

9.4.3.18 toStringFromVarType()

```
std::string mxvm::toStringFromVarType (
    const VarType & v)
```

Definition at line 1852 of file [icode_exec.cpp](#).

References [VAR_BYTE](#), [VAR_EXTERN](#), [VAR_FLOAT](#), [VAR_INTEGER](#), [VAR_POINTER](#), and [VAR_STRING](#).

9.4.3.19 x64_opt_core_lines()

```
std::vector< std::string > mxvm::x64_opt_core_lines (
    const std::vector< std::string > & lines) [static]
```

Definition at line 159 of file [icode_opt.cpp](#).

Referenced by [mxvm::Program::gen_optimize\(\)](#).

9.4.4 Variable Documentation

9.4.4.1 debug_mode

```
bool mxvm::debug_mode = false
enable verbose debug output during parsing
Definition at line 96 of file parser.cpp.
```

Referenced by [mxvm::Program::~~Program\(\)](#), [action_interpret\(\)](#), [proc_args\(\)](#), [signal_action\(\)](#), and [translate_x64\(\)](#).

9.4.4.2 error_label_count [1/2]

```
int mxvm::error_label_count = 0 [static]
Definition at line 12 of file icode\_gen.cpp.
```

Referenced by [mxvm::Program::gen_alloc\(\)](#), [mxvm::Program::gen_realloc\(\)](#), [mxvm::Program::x64_gen_alloc\(\)](#), [mxvm::Program::x64_gen_load\(\)](#), [mxvm::Program::x64_gen_realloc\(\)](#), and [mxvm::Program::x64_gen_store\(\)](#).

9.4.4.3 error_label_count [2/2]

```
int mxvm::error_label_count = 0 [static]
Definition at line 18 of file icode\_gen\_x64.cpp.
```

9.4.4.4 html_mode

```
bool mxvm::html_mode = false
enable HTML debug output
Definition at line 98 of file parser.cpp.
```

Referenced by [action_interpret\(\)](#), [mxvm::Parser::generateProgramCode\(\)](#), [proc_args\(\)](#), and [translate_x64\(\)](#).

9.4.4.5 instruct_mode

```
bool mxvm::instruct_mode = false
```

enable instruction trace mode

Definition at line 97 of file [parser.cpp](#).

Referenced by [mxvm::Program::exec\(\)](#), and [proc_args\(\)](#).

9.4.4.6 keywords

```
std::vector< std::string > mxvm::keywords {"program", "object", "module", "data", "code", "section"}
```

String representations of [Keywords](#) enum values.

Definition at line 10 of file [instruct.cpp](#).

Referenced by [buildKeywordSet\(\)](#), and [operator<<\(\)](#).

9.4.4.7 kInstructions

```
const std::unordered_set<std::string> mxvm::kInstructions = buildInstructionSet() [static]
```

Definition at line 78 of file [html_gen.cpp](#).

Referenced by [mxvm::HTMLGen::output\(\)](#).

9.4.4.8 kKeywords

```
const std::unordered_set<std::string> mxvm::kKeywords = buildKeywordSet() [static]
```

Definition at line 79 of file [html_gen.cpp](#).

Referenced by [mxvm::HTMLGen::output\(\)](#).

9.4.4.9 kOpSpecs

```
const std::unordered_map<std::string, OpSpec> mxvm::kOpSpecs [static]
```

Definition at line 92 of file [valid.cpp](#).

Referenced by [mxvm::Validator::validateAgainstSpec\(\)](#).

9.4.4.10 kTypes

```
const std::unordered_set<std::string> mxvm::kTypes [static]
```

Initial value:

```
    = [] {
    return std::unordered_set<std::string>{"string", "int", "byte", "float", "ptr"};
}()
```

Definition at line 80 of file [html_gen.cpp](#).

Referenced by [mxvm::HTMLGen::output\(\)](#).

9.4.4.11 x64_sp_mod16

```
unsigned mxvm::x64_sp_mod16 = 0 [static]
```

Definition at line 16 of file [icode_gen_x64.cpp](#).

Referenced by [mxvm::Program::x64_emitRestoreRegs\(\)](#), [mxvm::Program::x64_emitSaveRegs\(\)](#), [mxvm::Program::x64_gen_pop\(\)](#), [mxvm::Program::x64_gen_push\(\)](#), [mxvm::Program::x64_gen_stack_sub\(\)](#), [mxvm::Program::x64_generateCode\(\)](#), [mxvm::Program::x64_release_call_area\(\)](#), and [mxvm::Program::x64_reserve_call_area\(\)](#).

9.4.4.12 xmm_offset

```
size_t mxvm::xmm_offset [extern]
```

9.5 mxx Namespace Reference

Classes

- class [XParser](#)

Base parser providing token-stream navigation.

- struct [Scope](#)

A single scope level, tracking declared identifiers by category.

- class [TPValidator](#)

Semantic validator for Pascal programs.

Enumerations

- enum class [Keywords](#) {
[AND](#) , [ARRAY](#) , [BEGIN](#) , [CASE](#) ,
[CONST](#) , [DIV](#) , [DO](#) , [DOWNT0](#) ,
[ELSE](#) , [END](#) , [FILE](#) , [FOR](#) ,
[FUNCTION](#) , [GOTO](#) , [IF](#) , [IN](#) ,
[LABEL](#) , [MOD](#) , [NIL](#) , [NOT](#) ,
[OF](#) , [OR](#) , [PACKED](#) , [PROCEDURE](#) ,
[PROGRAM](#) , [RECORD](#) , [REPEAT](#) , [SET](#) ,
[THEN](#) , [TO](#) , [TYPE](#) , [UNTIL](#) ,
[VAR](#) , [WHILE](#) , [WITH](#) }

Standard Pascal reserved keyword enumeration.

Functions

- static bool [ciEquals](#) (const std::string &a, const std::string &b)

Case-insensitive string equality comparison.

Variables

- static const char * [key_str](#) []

String representations of [Keywords](#) values, indexed by enum ordinal.

- static const std::unordered_set< std::string > [pascal_keywords](#)

9.5.1 Enumeration Type Documentation

9.5.1.1 Keywords

```
enum class mxx::Keywords [strong]
```

Standard Pascal reserved keyword enumeration.

Enumerator

AND	
ARRAY	
BEGIN	
CASE	
CONST	
DIV	
DO	
DOWNT0	
ELSE	
END	
FILE	
FOR	

FUNCTION	
GOTO	
IF	
IN	
LABEL	
MOD	
NIL	
NOT	
OF	
OR	
PACKED	
PROCEDURE	
PROGRAM	
RECORD	
REPEAT	
SET	
THEN	
TO	
TYPE	
UNTIL	
VAR	
WHILE	
WITH	

Definition at line 12 of file [types.hpp](#).

9.5.2 Function Documentation

9.5.2.1 ciEquals()

```
bool mx::ciEquals (
    const std::string & a,
    const std::string & b) [static]
```

Case-insensitive string equality comparison.

Parameters

<i>a</i>	First string
<i>b</i>	Second string

Returns

true if a and b are equal ignoring ASCII case

Definition at line 67 of file [validator.cpp](#).

Referenced by [mxx::TPValidator::match\(\)](#), and [mxx::TPValidator::peekIs\(\)](#).

9.5.3 Variable Documentation

9.5.3.1 key_str

```
const char* mxx::key_str[] [static]
```

String representations of [Keywords](#) values, indexed by enum ordinal.

Definition at line 51 of file [types.hpp](#).

9.5.3.2 pascal_keywords

```
const std::unordered_set<std::string> mxx::pascal_keywords [static]
```

Initial value:

```
= {
"program", "unit", "interface", "implementation",
"uses", "var", "const", "type", "procedure", "function", "begin", "end",
"if", "then", "else", "while", "do", "for", "to", "downto", "repeat", "until",
"case", "of", "with", "goto", "label", "exit", "break", "continue",
"nil", "new", "dispose", "setlength", "high", "low",
"writeln", "write", "readln", "read", "seed_random", "rand_number",

"div", "mod", "and", "or", "not", "in",
"integer", "real", "boolean", "char", "byte", "word", "longint", "shortint",
"smallint", "cardinal", "string", "text", "double", "single", "extended",
"comp", "currency", "ptr", "pointer", "array", "record", "set",
"packed", "file", "text",
"assign", "reset", "rewrite", "append", "close", "eof", "eoln",
"include", "exclude"}
```

Definition at line 1373 of file [validator.cpp](#).

Referenced by [mxx::TPValidator::isPascalKeyword\(\)](#).

9.6 pascal Namespace Reference

Classes

- class [ASTNode](#)
Abstract base class for all AST nodes.
- class [ExitNode](#)
AST node for an exit / halt statement.
- class [ContinueNode](#)
AST node for a continue statement.
- class [BreakNode](#)
AST node for a break statement.
- class [BlockNode](#)
AST node representing a block (declarations + compound statement).
- class [ProgramNode](#)
AST node for a complete program (program name + block).
- class [UnitNode](#)
AST node for a Pascal unit (separately compiled module).
- class [VarDeclNode](#)
AST node for variable declarations (var id1, id2 : type).
- class [TypeDeclNode](#)
AST node for a type declaration section.
- class [TypeAliasNode](#)
AST node for a type alias (type NewName = ExistingType).

- class [ConstDeclNode](#)
AST node for constant declarations (const id = value).
- class [FuncDeclNode](#)
AST node for a function declaration.
- class [ProcDeclNode](#)
AST node for a procedure declaration.
- class [ParameterNode](#)
AST node for a formal parameter list entry.
- class [CompoundStmtNode](#)
AST node for a compound statement (begin ... end).
- class [AssignmentNode](#)
AST node for an assignment statement (variable := expression).
- class [SimpleTypeNode](#)
AST node for a simple (named) type reference.
- class [ArrayTypeNode](#)
AST node for an array type (array[lower..upper] of elementType).
- class [ArrayDeclarationNode](#)
AST node for a named array variable declaration.
- class [ArrayTypeDeclarationNode](#)
AST node for a named array type declaration (type Name = array[...]).
- class [ArrayAccessNode](#)
AST node for an array element access (arr[index]).
- class [ArrayAssignmentNode](#)
AST node for assigning to an array element (arr[index] := value).
- class [IfStmtNode](#)
AST node for an if-then-else statement.
- class [WhileStmtNode](#)
AST node for a while loop (while cond do stmt).
- class [ForStmtNode](#)
AST node for a for loop (for var := start to/downto end do stmt).
- class [CaseStmtNode](#)
AST node for a case statement.
- class [ProcCallNode](#)
AST node for a procedure call statement.
- class [BinaryOpNode](#)
AST node for a binary operator expression.
- class [UnaryOpNode](#)
AST node for a unary operator expression.
- class [FuncCallNode](#)
AST node for a function call expression.
- class [VariableNode](#)
AST node for a variable reference.
- class [NumberNode](#)
AST node for an integer or real numeric literal.
- class [StringNode](#)
AST node for a string literal.
- class [BooleanNode](#)
AST node for a boolean literal (true / false).
- class [EmptyStmtNode](#)
AST node for an empty statement (no-op).
- class [NilNode](#)

- AST node for the nil literal.*
- class [PointerTypeNode](#)
 - AST node for a pointer type declaration ($\wedge$ BaseType).*
- class [PointerDerefNode](#)
 - AST node for a pointer dereference ($\text{ptr}^\wedge$).*
- class [AddressOfNode](#)
 - AST node for an address-of expression (@operand).*
- class [RepeatStmtNode](#)
 - AST node for a repeat..until loop.*
- class [EnumTypeDeclNode](#)
 - AST node for an enumerated type declaration (type Color = (Red, Green, Blue)).*
- struct [VariantArm](#)
 - A single variant arm inside a variant record.*
- class [RecordTypeNode](#)
 - AST node for a record type (record ... end).*
- class [RecordDeclarationNode](#)
 - AST node for a named record declaration.*
- class [WithStmtNode](#)
 - AST node for a with statement (with record do statement).*
- class [GotoStmtNode](#)
 - AST node for a goto statement.*
- class [LabelStmtNode](#)
 - AST node for a label definition (label: statement).*
- class [SetLiteralNode](#)
 - AST node for a set literal [val1, val2, ...].*
- class [SetTypeNode](#)
 - AST node for a set type declaration (set of <base type>).*
- class [FieldAccessNode](#)
 - AST node for accessing a record field (record.field).*
- class [ASTVisitor](#)
 - Abstract visitor interface for the AST (double-dispatch).*
- struct [ArrayInfo](#)
 - Metadata describing a Pascal array's bounds, element type, and sizing.*
- class [BuiltinFunctionHandler](#)
 - Abstract handler for a group of built-in functions/procedures.*
- class [BuiltinFunctionRegistry](#)
 - Registry of [BuiltinFunctionHandler](#) instances.*
- class [IOFunctionHandler](#)
 - Built-in handler for I/O procedures (write, writeln, read, readln).*
- class [StdFunctionHandler](#)
 - Built-in handler for standard library functions (chr, ord, inc, dec, etc.).*
- class [StringFunctionHandler](#)
 - Built-in handler for string library functions (length, copy, concat, etc.).*
- class [SDLFunctionHandler](#)
 - Built-in handler for SDL2 bindings.*
- class [FileFunctionHandler](#)
 - Built-in handler for Pascal file I/O procedures (assign, reset, rewrite, append, close, eof, eoln).*
- class [CodeGenVisitor](#)
 - AST visitor that generates MXVM intermediate code.*
- class [ParseException](#)
 - Exception thrown on parse errors.*
- class [PascalParser](#)
 - Recursive-descent parser for the Pascal language.*

Enumerations

- enum class [VarType](#) {
[INT](#) , [DOUBLE](#) , [STRING](#) , [CHAR](#) ,
[RECORD](#) , [PTR](#) , [BOOL](#) , [UNKNOWN](#) ,
[ARRAY_INT](#) , [ARRAY_DOUBLE](#) , [ARRAY_STRING](#) }

Enumeration of variable / expression value types.

Functions

- static std::string [rtrim_comment](#) (const std::string &line)
- static std::string [trim](#) (const std::string &s)
- static void [locate_sections](#) (const std::vector< std::string > &lines, size_t &dataStart, size_t &dataEnd, size_t &codeStart, size_t &codeEnd)
- static std::unordered_set< std::string > [collect_used](#) (const std::vector< std::string > &lines, size_t codeStart, size_t codeEnd)
- static void [sweep_unused_data](#) (std::vector< std::string > &lines)
- static std::vector< std::pair< size_t, std::string > > [nextNonEmpty](#) (const std::vector< std::string > &code, size_t start, size_t n)
- static std::string [invertJump](#) (const std::string &j)
- static bool [parse2](#) (const std::string &line, const std::string &op, std::string &a, std::string &b)
- static bool [parse1](#) (const std::string &line, const std::string &op, std::string &a)
- static bool [parseJump](#) (const std::string &line, std::string &jop, std::string &label)
- static bool [isLabelDef](#) (const std::string &line, const std::string &label)
- static void [foldCmpTest](#) (std::vector< std::string > &code)
- static void [foldAndTest](#) (std::vector< std::string > &code)
- static void [foldOrTest](#) (std::vector< std::string > &code)
- static bool [isRegisterName](#) (const std::string &s)

Forward declaration — checks whether a name is a hardware register.

- static void [copyPropagation](#) (std::vector< std::string > &code)
- Copy-propagation peephole pass.*
- static void [foldMovArithMov](#) (std::vector< std::string > &code)
- static void [peepholeOpt](#) (std::vector< std::string > &code)
- std::string [mxvmOpt](#) (const std::string &text)
- static std::string [toLower](#) (const std::string &s)

Convert a string to lowercase for case-insensitive comparison.

Variables

- std::unordered_set< std::string > [sdlFunctions](#)
- std::unordered_set< std::string > [stdFunctions](#)
- static std::unordered_set< std::string > [fileFunctions](#)

9.6.1 Enumeration Type Documentation

9.6.1.1 VarType

enum class [pascal::VarType](#) [strong]

Enumeration of variable / expression value types.

Enumerator

INT	integer
DOUBLE	real (floating-point)
STRING	string value

CHAR	character value
RECORD	record (struct)
PTR	generic pointer
BOOL	boolean
UNKNOWN	unresolved type
ARRAY_INT	array of integers
ARRAY_DOUBLE	array of doubles
ARRAY_STRING	array of strings

Definition at line 72 of file [icode.hpp](#).

9.6.2 Function Documentation

9.6.2.1 collect_used()

```
std::unordered_set< std::string > pascal::collect_used (
    const std::vector< std::string > & lines,
    size_t codeStart,
    size_t codeEnd) [static]
```

Definition at line 60 of file [icode.cpp](#).

Referenced by [sweep_unused_data\(\)](#).

9.6.2.2 copyPropagation()

```
void pascal::copyPropagation (
    std::vector< std::string > & code) [static]
```

Copy-propagation peephole pass.

Folds sequences like `mov reg, src ; op reg, rhs` into `op src, rhs` when the intermediate register is not referenced again. The scan respects `ret / done` boundaries so that stores to global variables visible to the caller are never eliminated.

Definition at line 376 of file [icode.cpp](#).

References [isRegisterName\(\)](#), [parse2\(\)](#), [rtrim_comment\(\)](#), and [trim\(\)](#).

Referenced by [peepholeOpt\(\)](#).

9.6.2.3 foldAndTest()

```
void pascal::foldAndTest (
    std::vector< std::string > & code) [static]
```

Definition at line 253 of file [icode.cpp](#).

References [isLabelDef\(\)](#), [nextNonEmpty\(\)](#), [parse1\(\)](#), and [parse2\(\)](#).

Referenced by [peepholeOpt\(\)](#).

9.6.2.4 foldCmpTest()

```
void pascal::foldCmpTest (
    std::vector< std::string > & code) [static]
```

Definition at line 166 of file [icode.cpp](#).

References [invertJump\(\)](#), [isLabelDef\(\)](#), [nextNonEmpty\(\)](#), [parse1\(\)](#), [parse2\(\)](#), [parseJump\(\)](#), [rtrim_comment\(\)](#), and [trim\(\)](#).

Referenced by [peepholeOpt\(\)](#).

9.6.2.5 foldMovArithMov()

```
void pascal::foldMovArithMov (
    std::vector< std::string > & code) [static]
```

Definition at line 501 of file [icode.cpp](#).

References [isRegisterName\(\)](#), [parse2\(\)](#), [rtrim_comment\(\)](#), and [trim\(\)](#).
 Referenced by [peepholeOpt\(\)](#).

9.6.2.6 foldOrTest()

```
void pascal::foldOrTest (
    std::vector< std::string > & code) [static]
```

Definition at line 306 of file [icode.cpp](#).

References [isLabelDef\(\)](#), [nextNonEmpty\(\)](#), [parse1\(\)](#), and [parse2\(\)](#).
 Referenced by [peepholeOpt\(\)](#).

9.6.2.7 invertJump()

```
std::string pascal::invertJump (
    const std::string & j) [static]
```

Definition at line 108 of file [icode.cpp](#).

Referenced by [foldCmpTest\(\)](#).

9.6.2.8 isLabelDef()

```
bool pascal::isLabelDef (
    const std::string & line,
    const std::string & label) [static]
```

Definition at line 162 of file [icode.cpp](#).

Referenced by [foldAndTest\(\)](#), [foldCmpTest\(\)](#), and [foldOrTest\(\)](#).

9.6.2.9 isRegisterName()

```
bool pascal::isRegisterName (
    const std::string & s) [static]
```

Forward declaration — checks whether a name is a hardware register.

Used by the peephole optimizer to distinguish register temporaries from global/local variable names when deciding whether a store is dead.

Definition at line 491 of file [icode.cpp](#).

Referenced by [copyPropagation\(\)](#), [foldMovArithMov\(\)](#), and [mxvmOpt\(\)](#).

9.6.2.10 locate_sections()

```
void pascal::locate_sections (
    const std::vector< std::string > & lines,
    size_t & dataStart,
    size_t & dataEnd,
    size_t & codeStart,
    size_t & codeEnd) [static]
```

Definition at line 33 of file [icode.cpp](#).

Referenced by [mxvmOpt\(\)](#), and [sweep_unused_data\(\)](#).

9.6.2.11 mxvmOpt()

```
std::string pascal::mxvmOpt (
    const std::string & text)
```

Definition at line 589 of file [icode.cpp](#).

References [isRegisterName\(\)](#), [locate_sections\(\)](#), [peepholeOpt\(\)](#), [rtrim_comment\(\)](#), [sweep_unused_data\(\)](#), and [trim\(\)](#).

Referenced by [compileFile\(\)](#).

9.6.2.12 nextNonEmpty()

```
std::vector< std::pair< size_t, std::string > > pascal::nextNonEmpty (
    const std::vector< std::string > & code,
```



```
    size_t start,
    size_t n) [static]
```

Definition at line 97 of file [icode.cpp](#).

References [rtrim_comment\(\)](#), and [trim\(\)](#).

Referenced by [foldAndTest\(\)](#), [foldCmpTest\(\)](#), and [foldOrTest\(\)](#).

9.6.2.13 parse1()

```
bool pascal::parse1 (
    const std::string & line,
    const std::string & op,
    std::string & a) [static]
```

Definition at line 145 of file [icode.cpp](#).

References [trim\(\)](#).

Referenced by [foldAndTest\(\)](#), [foldCmpTest\(\)](#), and [foldOrTest\(\)](#).

9.6.2.14 parse2()

```
bool pascal::parse2 (
    const std::string & line,
    const std::string & op,
    std::string & a,
    std::string & b) [static]
```

Definition at line 132 of file [icode.cpp](#).

References [trim\(\)](#).

Referenced by [copyPropagation\(\)](#), [foldAndTest\(\)](#), [foldCmpTest\(\)](#), [foldMovArithMov\(\)](#), and [foldOrTest\(\)](#).

9.6.2.15 parseJump()

```
bool pascal::parseJump (
    const std::string & line,
    std::string & jop,
    std::string & label) [static]
```

Definition at line 152 of file [icode.cpp](#).

References [trim\(\)](#).

Referenced by [foldCmpTest\(\)](#).

9.6.2.16 peepholeOpt()

```
void pascal::peepholeOpt (
    std::vector< std::string > & code) [static]
```

Definition at line 579 of file [icode.cpp](#).

References [copyPropagation\(\)](#), [foldAndTest\(\)](#), [foldCmpTest\(\)](#), [foldMovArithMov\(\)](#), and [foldOrTest\(\)](#).

Referenced by [mxvmOpt\(\)](#).

9.6.2.17 rtrim_comment()

```
std::string pascal::rtrim_comment (
    const std::string & line) [inline], [static]
```

Definition at line 15 of file [icode.cpp](#).

Referenced by [copyPropagation\(\)](#), [foldCmpTest\(\)](#), [foldMovArithMov\(\)](#), [mxvmOpt\(\)](#), and [nextNonEmpty\(\)](#).

9.6.2.18 sweep_unused_data()

```
void pascal::sweep_unused_data (
    std::vector< std::string > & lines) [static]
```

Definition at line 74 of file [icode.cpp](#).

References [collect_used\(\)](#), and [locate_sections\(\)](#).

Referenced by [mxvmOpt\(\)](#).

9.6.2.19 toLower()

```
std::string pascal::toLower (
    const std::string & s) [static]
```

Convert a string to lowercase for case-insensitive comparison.

Parameters

s	Input string
---	--------------

Returns

Lowercased copy of s

Definition at line 16 of file `icode_imp.cpp`.

Referenced by `pascal::FileFunctionHandler::canHandle()`, `pascal::IOFunctionHandler::canHandle()`, `pascal::SDLFunctionHandler::canHandle()`, `pascal::StdFunctionHandler::canHandle()`, `pascal::StringFunctionHandler::canHandle()`, `pascal::FileFunctionHandler::generate()`, `pascal::IOFunctionHandler::generate()`, `pascal::FileFunctionHandler::generateWithResult()`, `pascal::IOFunctionHandler::generateWithResult()`, `pascal::FileFunctionHandler::getReturnType()`, and `pascal::StringFunctionHandler::getReturnType()`.

9.6.2.20 trim()

```
std::string pascal::trim (
    const std::string & s) [inline], [static]
```

Definition at line 25 of file `icode.cpp`.

Referenced by `copyPropagation()`, `foldCmpTest()`, `foldMovArithMov()`, `mxvmOpt()`, `nextNonEmpty()`, `parse1()`, `parse2()`, and `parseJump()`.

9.6.3 Variable Documentation

9.6.3.1 fileFunctions

```
std::unordered_set<std::string> pascal::fileFunctions [static]
```

Initial value:

```
= {
    "assign", "reset", "rewrite", "append", "close", "eof", "eoln"}
```

Definition at line 985 of file `icode_imp.cpp`.

Referenced by `pascal::FileFunctionHandler::canHandle()`.

9.6.3.2 sdlFunctions

```
std::unordered_set<std::string> pascal::sdlFunctions
```

Initial value:

```
= {
    "sdl_init", "sdl_quit", "sdl_create_window", "sdl_destroy_window", "sdl_set_window_title",
    "sdl_set_window_position", "sdl_get_window_size", "sdl_set_window_fullscreen", "sdl_set_window_icon",
    "sdl_create_renderer", "sdl_destroy_renderer", "sdl_set_draw_color", "sdl_clear", "sdl_present",
    "sdl_get_renderer_output_size", "sdl_draw_point", "sdl_draw_line", "sdl_draw_rect", "sdl_fill_rect",
    "sdl_poll_event", "sdl_get_event_type", "sdl_get_key_code", "sdl_get_mouse_x", "sdl_get_mouse_y",
    "sdl_get_mouse_button", "sdl_get_mouse_buttons", "sdl_get_relative_mouse_x", "sdl_get_relative_mouse_y",
    "sdl_get_relative_mouse_buttons", "sdl_create_rgb_surface", "sdl_free_surface", "sdl_blit_surface",
    "sdl_get_mouse_state", "sdl_get_relative_mouse_state", "sdl_get_keyboard_state", "sdl_is_key_pressed",
    "sdl_get_num_keys", "sdl_set_clipboard_text", "sdl_get_clipboard_text", "sdl_show_cursor",
    "sdl_create_texture", "sdl_destroy_texture", "sdl_load_texture", "sdl_load_texture_color_key",
    "sdl_load_texture_color_key_rgb", "sdl_render_texture",
    "sdl_update_texture", "sdl_lock_texture", "sdl_unlock_texture", "sdl_get_ticks", "sdl_delay",
    "sdl_open_audio", "sdl_close_audio", "sdl_pause_audio", "sdl_load_wav", "sdl_free_wav",
    "sdl_queue_audio", "sdl_get_queued_audio_size", "sdl_clear_queued_audio", "sdl_init_text",
    "sdl_quit_text", "sdl_load_font", "sdl_draw_text", "sdl_create_render_target", "sdl_set_render_target",
    "sdl_destroy_render_target", "sdl_present_scaled", "sdl_present_stretched"}
```

Definition at line 23 of file `icode_imp.cpp`.

Referenced by `pascal::SDLFunctionHandler::canHandle()`.

9.6.3.3 stdFunctions

```
std::unordered_set<std::string> pascal::stdFunctions
```

Initial value:

```
= {
"argc", "argv", "set_program_args", "free_program_args", "abs", "fabs",
"sqrt", "pow", "sin", "cos", "tan", "floor", "ceil", "rand", "srand",
"malloc", "calloc", "free", "toupper", "tolower", "isalpha", "isdigit",
"isspace", "atoi", "atof", "exit", "system", "memcpy", "memcmp",
"memmove", "memset", "exp", "exp2", "log", "log10", "log2", "fmod",
"atan2", "asin", "acos", "atan", "sinh", "cosh", "tanh", "hypot",
"round", "trunc", "float_to_int", "int_to_float", "halt",
"ord", "chr", "succ", "pred", "inc", "dec", "include", "exclude"}
```

Definition at line 40 of file [icode_imp.cpp](#).

Referenced by [pascal::StdFunctionHandler::canHandle\(\)](#).

9.7 scan Namespace Reference

Namespaces

- namespace [buffer](#)
- namespace [token](#)

Classes

- class [Scanner](#)
Lexical scanner that tokenizes source text.
- class [ScanExcept](#)
Exception thrown during scanning errors.

Typedefs

- using [TString](#) = [buffer::StringBuffer](#)<char>
Default character string buffer.
- using [StringType](#) = [buffer::StringBuffer](#)<char>::string_type
Underlying string type.
- using [TToken](#) = [token::Token](#)<char>
Default token type.
- using [TMap](#) = [token::TokenMap](#)<char>
Default character classification map.

9.7.1 Typedef Documentation

9.7.1.1 StringType

```
using scan::StringType = buffer::StringBuffer<char>::string_type
```

Underlying string type.

Definition at line 18 of file [scanner.hpp](#).

9.7.1.2 TMap

```
using scan::TMap = token::TokenMap<char>
```

Default character classification map.

Definition at line 20 of file [scanner.hpp](#).

9.7.1.3 TString

```
using scan::TString = buffer::StringBuffer<char>
```

Default character string buffer.

Definition at line 17 of file [scanner.hpp](#).

9.7.1.4 TToken

using `scan::TToken` = `token::Token<char>`

Default token type.

Definition at line 19 of file `scanner.hpp`.

9.8 scan::buffer Namespace Reference

Classes

- class `StringBuffer`
Character-by-character string buffer with line/column tracking.

9.9 scan::token Namespace Reference

Classes

- class `TokenMap`
Maps characters to their lexical category (letter, digit, symbol, etc.).
- class `Token`
A scanned token with type, value, and source position.

Enumerations

- enum class `CharType`
Classification of individual characters during lexing.

9.9.1 Enumeration Type Documentation

9.9.1.1 CharType

enum class `types::CharType` [strong]

Classification of individual characters during lexing.

Definition at line 26 of file `types.hpp`.

9.10 types Namespace Reference

Enumerations

- enum class `TokenType` {
 `TT_ID`, `TT_ARG`, `TT_SYM`, `TT_STR`,
 `TT_NUM`, `TT_HEX`, `TT_NULL` }
Classification of scanned tokens.
- enum class `CharType` {
 `TT_CHAR`, `TT_DIGIT`, `TT_SYMBOL`, `TT_STRING`,
 `TT_SINGLE`, `TT_SPACE`, `TT_NULL` }
Classification of individual characters during lexing.
- enum class `OperatorType` {
 `OP_INC`, `OP_DEC`, `OP_LSHIFT_ASSIGN`, `OP_RSHIFT_ASSIGN`,
 `OP_PLUS_ASSIGN`, `OP_MINUS_ASSIGN`, `OP_MUL_ASSIGN`, `OP_DIV_ASSIGN`,
 `OP_MOD_ASSIGN`, `OP_AND_ASSIGN`, `OP_OR_ASSIGN`, `OP_XOR_ASSIGN`,
 `OP_LSHIFT`, `OP_RSHIFT`, `OP_EQ`, `OP_NEQ`,
 `OP_LE`, `OP_GE`, `OP_AND_AND`, `OP_OR_OR`,
 `OP_ARROW`, `OP_PLUS`, `OP_MINUS`, `OP_MUL`,
 `OP_DIV`, `OP_MOD`, `OP_ASSIGN`, `OP_AND`,
 `OP_OR`, `OP_XOR`, `OP_NOT`, `OP_LT`,
 ...

```
OP_GT , OP_LPAREN , OP_RPAREN , OP_LBRACKET ,
OP_RBRACKET , OP_LBRACE , OP_RBRACE , OP_COMMA ,
OP_SEMICOLON , OP_COLON , OP_DOT , OP_QUESTION ,
OP_HASH , OP_SCOPE , OP_TILDE , OP_DOLLAR ,
OP_AT }
```

Enumeration of all recognised operator symbols.

- enum class [KeywordType](#) {
[KW_LET](#) , [KW_PROC](#) , [KW_DEFINE](#) , [KW_IF](#) ,
[KW_ELSE](#) , [KW_SWITCH](#) , [KW_WHILE](#) , [KW_FOR](#) ,
[KW_RETURN](#) , [KW_BREAK](#) , [KW_CONTINUE](#) }

Keyword type enumeration.

Functions

- void [print_type_TokenType](#) (std::ostream &out, const [TokenType](#) &tt)
Print a [TokenType](#) name to a stream.
- void [print_type_CharType](#) (std::ostream &out, const [CharType](#) &c)
Print a [CharType](#) name to a stream.
- std::optional< [OperatorType](#) > [lookUp](#) (const std::string &op)
Look up an operator string and return its [OperatorType](#).
- std::optional< [KeywordType](#) > [lookUp_Keyword](#) (const std::string &op)
Look up a keyword string and return its [KeywordType](#).

Variables

- std::vector< std::string > [opName](#)
Human-readable display names indexed by [OperatorType](#).
- std::vector< std::string > [opStrings](#)
Operator string representations indexed by [OperatorType](#).
- std::vector< std::string > [kwStr](#) {"let", "proc", "define", "if", "else", "switch", "while", "for", "return", "break", "continue"}
Keyword string representations indexed by [KeywordType](#).
- std::vector< std::string > [strTokenType](#) {"Identifier", "Argument", "Symbols", "String", "Numeric", "Hex", "NULL"}
string names of [TokenType](#) values
- std::vector< std::string > [sstrCharType](#)
string names of [CharType](#) values
- std::vector< std::string > [strCharType](#) {"Characters", "Digits", "Symbols", "String", "Quote", "Space", "NULL"}

9.10.1 Enumeration Type Documentation

9.10.1.1 CharType

```
enum class types::CharType [strong]
```

Classification of individual characters during lexing.

Enumerator

TT_CHAR	alphabetic character
TT_DIGIT	digit character
TT_SYMBOL	multi-char symbol start
TT_STRING	string delimiter (quote)
TT_SINGLE	single-char operator

TT_SPACE	whitespace
TT_NULL	unmapped / null

Definition at line 26 of file [types.hpp](#).

9.10.1.2 KeywordType

enum class [types::KeywordType](#) [strong]
Keyword type enumeration.

Enumerator

KW_LET	variable declaration
KW_PROC	procedure/function
KW_DEFINE	constant definition
KW_IF	conditional
KW_ELSE	else branch
KW_SWITCH	switch/case
KW_WHILE	while loop
KW_FOR	for loop
KW_RETURN	return statement
KW_BREAK	break statement
KW_CONTINUE	continue statement

Definition at line 196 of file [types.hpp](#).

9.10.1.3 OperatorType

enum class [types::OperatorType](#) [strong]
Enumeration of all recognised operator symbols.

Enumerator

OP_INC	
OP_DEC	
OP_LSHIFT_ASSIGN	
OP_RSHIFT_ASSIGN	
OP_PLUS_ASSIGN	
OP_MINUS_ASSIGN	
OP_MUL_ASSIGN	
OP_DIV_ASSIGN	
OP_MOD_ASSIGN	
OP_AND_ASSIGN	
OP_OR_ASSIGN	
OP_XOR_ASSIGN	
OP_LSHIFT	
OP_RSHIFT	

OP_EQ	
OP_NEQ	
OP_LE	
OP_GE	
OP_AND_AND	
OP_OR_OR	
OP_ARROW	
OP_PLUS	
OP_MINUS	
OP_MUL	
OP_DIV	
OP_MOD	
OP_ASSIGN	
OP_AND	
OP_OR	
OP_XOR	
OP_NOT	
OP_LT	
OP_GT	
OP_LPAREN	
OP_RPAREN	
OP_LBRACKET	
OP_RBRACKET	
OP_LBRACE	
OP_RBRACE	
OP_COMMA	
OP_SEMICOLON	
OP_COLON	
OP_DOT	
OP_QUESTION	
OP_HASH	
OP_SCOPE	
OP_TILDE	
OP_DOLLAR	
OP_AT	

Definition at line 36 of file [types.hpp](#).

9.10.1.4 TokenType

```
enum class types::TokenType [strong]
```

Classification of scanned tokens.

Enumerator

TT_ID	identifier or keyword
TT_ARG	argument token (e.g. \$1)
TT_SYM	symbol / operator token
TT_STR	string literal
TT_NUM	decimal numeric literal
TT_HEX	hexadecimal numeric literal
TT_NULL	sentinel / invalid token

Definition at line 16 of file [types.hpp](#).

9.10.2 Function Documentation**9.10.2.1 lookUp()**

```
std::optional< OperatorType > types::lookUp (
    const std::string & op)
```

Look up an operator string and return its [OperatorType](#).

Parameters

<i>op</i>	Operator string (e.g. "+", "<=")
-----------	----------------------------------

Returns

The matching [OperatorType](#), or nullopt if not found

Definition at line 25 of file [types.cpp](#).

References [opStrings](#).

9.10.2.2 lookUp_Keyword()

```
std::optional< KeywordType > types::lookUp_Keyword (
    const std::string & op)
```

Look up a keyword string and return its [KeywordType](#).

Parameters

<i>op</i>	Keyword string (e.g. "if", "while")
-----------	-------------------------------------

Returns

The matching [KeywordType](#), or nullopt if not found

Definition at line 33 of file [types.cpp](#).

References [kwStr](#).

9.10.2.3 print_type_CharType()

```
void types::print_type_CharType (
    std::ostream & out,
    const CharType & c)
```

Print a [CharType](#) name to a stream.

Parameters

<i>out</i>	Output stream
<i>c</i>	Character type to print

Definition at line 19 of file [types.cpp](#).

References [strCharType](#).

9.10.2.4 print_type_TokenType()

```
void types::print_type_TokenType (
    std::ostream & out,
    const TokenType & tt)
```

Print a [TokenType](#) name to a stream.

Parameters

<i>out</i>	Output stream
<i>tt</i>	Token type to print

Definition at line 13 of file [types.cpp](#).

References [strTokenType](#).

Referenced by [scan::token::Token< Ch, String >::print\(\)](#).

9.10.3 Variable Documentation**9.10.3.1 kwStr**

```
std::vector<std::string> types::kwStr {"let", "proc", "define", "if", "else", "switch", "while",
    "for", "return", "break", "continue"} [inline]
```

Keyword string representations indexed by [KeywordType](#).

Definition at line 210 of file [types.hpp](#).

Referenced by [lookUp_Keyword\(\)](#).

9.10.3.2 opName

```
std::vector<std::string> types::opName [inline]
```

Human-readable display names indexed by [OperatorType](#).

Definition at line 89 of file [types.hpp](#).

9.10.3.3 opStrings

```
std::vector<std::string> types::opStrings [inline]
```

Operator string representations indexed by [OperatorType](#).

Definition at line 142 of file [types.hpp](#).

Referenced by [lookUp\(\)](#).

9.10.3.4 sstrCharType

```
std::vector<std::string> types::sstrCharType [extern]
```

string names of [CharType](#) values

9.10.3.5 strCharType

```
std::vector<std::string> types::strCharType {"Characters", "Digits", "Symbols", "String",
    "Quote", "Space", "NULL"}
```

Definition at line 11 of file [types.cpp](#).

Referenced by [print_type_CharType\(\)](#).

9.10.3.6 strTokenType

```
std::vector< std::string > types::strTokenType {"Identifier", "Argument", "Symbols", "String",  
"Numeric", "Hex", "NULL"}
```

string names of [TokenType](#) values

Definition at line 10 of file [types.cpp](#).

Referenced by [print_type_TokenType\(\)](#).

Chapter 10

Concept Documentation

10.1 mx::StringType Concept Reference

Concept constraining String template parameters to class types with `length()`, `operator[]`, `operator+=`, and `operator=` (e.g. `std::string`, `std::wstring`).

```
#include <argz.hpp>
```

10.1.1 Concept definition

```
template<typename T>
concept StringType = std::is_class_v<T> && requires(T type) {
    type.length();
    type[0];
    type += type;
    type = type;
    typename T::value_type;
    typename T::size_type;
    { type.length() } -> std::same_as<typename T::size_type>;
    { type[0] } -> std::same_as<typename T::value_type &>;
    { type += T{} } -> std::same_as<T &>;
    { type = T{} } -> std::same_as<T &>;
}
```

10.1.2 Detailed Description

Concept constraining String template parameters to class types with `length()`, `operator[]`, `operator+=`, and `operator=` (e.g. `std::string`, `std::wstring`).

Template Parameters

<i>T</i>	String-like type
----------	------------------

Definition at line 25 of file [argz.hpp](#).

Chapter 11

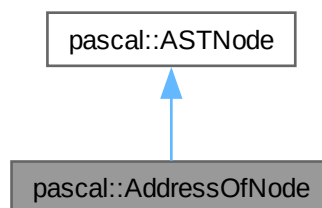
Class Documentation

11.1 pascal::AddressOfNode Class Reference

AST node for an address-of expression (@operand).

```
#include <ast.hpp>
```

Inheritance diagram for pascal::AddressOfNode:



Public Member Functions

- [AddressOfNode](#) (std::unique_ptr< [ASTNode](#) > op)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::unique_ptr< [ASTNode](#) > [operand](#)

operand whose address is taken

11.1.1 Detailed Description

AST node for an address-of expression (@operand).

Definition at line 600 of file [ast.hpp](#).

11.1.2 Constructor & Destructor Documentation

11.1.2.1 AddressOfNode()

```
pascal::AddressOfNode::AddressOfNode (
    std::unique_ptr< ASTNode > op) [inline]
```

Definition at line 604 of file [ast.hpp](#).

References [operand](#).

11.1.3 Member Function Documentation

11.1.3.1 accept()

```
void pascal::AddressOfNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 438 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.1.3.2 toString()

```
std::string pascal::AddressOfNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 607 of file [ast.hpp](#).

11.1.4 Member Data Documentation

11.1.4.1 operand

```
std::unique_ptr<ASTNode> pascal::AddressOfNode::operand
```

operand whose address is taken

Definition at line 602 of file [ast.hpp](#).

Referenced by [AddressOfNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.2 mx::ArgException< String > Class Template Reference

[Exception](#) type thrown for argument parsing errors.

```
#include <argz.hpp>
```

Public Member Functions

- [ArgException](#) ()=default
- [ArgException](#) (const String &s)
Construct with an error message.
- String [text](#) () const
Get the error message.

Private Attributes

- String [value](#)

11.2.1 Detailed Description

```
template<StringType String>
class mx::ArgException< String >
```

[Exception](#) type thrown for argument parsing errors.

Template Parameters

<i>String</i>	A StringType-constrained string class
---------------	---------------------------------------

Definition at line 102 of file [argz.hpp](#).

11.2.2 Constructor & Destructor Documentation**11.2.2.1 ArgException() [1/2]**

```
template<StringType String>
mx::ArgException< String >::ArgException () [default]
```

11.2.2.2 ArgException() [2/2]

```
template<StringType String>
mx::ArgException< String >::ArgException (
    const String & s) [inline]
```

Construct with an error message.

Definition at line 107 of file [argz.hpp](#).

References [value](#).

11.2.3 Member Function Documentation**11.2.3.1 text()**

```
template<StringType String>
String mx::ArgException< String >::text () const [inline]
```

Get the error message.

Definition at line 110 of file [argz.hpp](#).

References [value](#).

Referenced by [main\(\)](#), [mx::proc_args\(\)](#), and [proc_args\(\)](#).

11.2.4 Member Data Documentation**11.2.4.1 value**

```
template<StringType String>
String mx::ArgException< String >::value [private]
```

Definition at line 113 of file [argz.hpp](#).

Referenced by [ArgException\(\)](#), and [text\(\)](#).

The documentation for this class was generated from the following file:

- [src/vm/include/argz.hpp](#)

11.3 Args Struct Reference**Public Attributes**

- std::string [source_file](#)

- `std::string output_file`
- `std::string module_path = "/usr/local/lib"`
- `std::string object_path = "."`
- `std::string include_path = "/usr/local/include/mxvm/modules"`
- `vm_action action = vm_action::null_action`
- `vm_target target = vm_target::x86_64_linux`
- `std::vector< std::string > argv`
- `mxvm::Platform platform = mxvm::Platform::LINUX`
- `int platform_argc = 0`
- `char ** platform_argv = nullptr`
- `bool Makefile = false`
- `bool only_test = false`
- `std::string toolchain`

11.3.1 Detailed Description

Definition at line 72 of file [main.cpp](#).

11.3.2 Member Data Documentation

11.3.2.1 action

`vm_action` `Args::action = vm_action::null_action`

Definition at line 78 of file [main.cpp](#).

Referenced by [proc_args\(\)](#), and [process_arguments\(\)](#).

11.3.2.2 argv

`std::vector<std::string>` `Args::argv`

Definition at line 80 of file [main.cpp](#).

Referenced by [proc_args\(\)](#), and [process_arguments\(\)](#).

11.3.2.3 include_path

`std::string` `Args::include_path = "/usr/local/include/mxvm/modules"`

Definition at line 77 of file [main.cpp](#).

Referenced by [createMakefile\(\)](#), [proc_args\(\)](#), [process_arguments\(\)](#), and [translate_x64\(\)](#).

11.3.2.4 Makefile

`bool` `Args::Makefile = false`

Definition at line 84 of file [main.cpp](#).

Referenced by [proc_args\(\)](#), and [process_arguments\(\)](#).

11.3.2.5 module_path

`std::string` `Args::module_path = "/usr/local/lib"`

Definition at line 75 of file [main.cpp](#).

Referenced by [createMakefile\(\)](#), [proc_args\(\)](#), [process_arguments\(\)](#), and [translate_x64\(\)](#).

11.3.2.6 object_path

`std::string` `Args::object_path = "."`

Definition at line 76 of file [main.cpp](#).

Referenced by [createMakefile\(\)](#), [proc_args\(\)](#), [process_arguments\(\)](#), and [translate_x64\(\)](#).

11.3.2.7 only_test

```
bool Args::only_test = false
```

Definition at line 85 of file [main.cpp](#).

Referenced by [proc_args\(\)](#), and [process_arguments\(\)](#).

11.3.2.8 output_file

```
std::string Args::output_file
```

Definition at line 74 of file [main.cpp](#).

Referenced by [proc_args\(\)](#), and [translate_x64\(\)](#).

11.3.2.9 platform

```
mxvm::Platform Args::platform = mxvm::Platform::LINUX
```

Definition at line 81 of file [main.cpp](#).

Referenced by [proc_args\(\)](#), and [process_arguments\(\)](#).

11.3.2.10 platform_argc

```
int Args::platform_argc = 0
```

Definition at line 82 of file [main.cpp](#).

Referenced by [proc_args\(\)](#).

11.3.2.11 platform_argv

```
char** Args::platform_argv = nullptr
```

Definition at line 83 of file [main.cpp](#).

Referenced by [proc_args\(\)](#).

11.3.2.12 source_file

```
std::string Args::source_file
```

Definition at line 73 of file [main.cpp](#).

Referenced by [createMakefile\(\)](#), [proc_args\(\)](#), [process_arguments\(\)](#), and [translate_x64\(\)](#).

11.3.2.13 target

```
vm_target Args::target = vm_target::x86_64_linux
```

Definition at line 79 of file [main.cpp](#).

Referenced by [createMakefile\(\)](#), and [proc_args\(\)](#).

11.3.2.14 toolchain

```
std::string Args::toolchain
```

Definition at line 86 of file [main.cpp](#).

Referenced by [createMakefile\(\)](#), [proc_args\(\)](#), and [process_arguments\(\)](#).

The documentation for this struct was generated from the following file:

- [src/vm/main.cpp](#)

11.4 mx::Argument< String > Struct Template Reference

A parsed or registered command-line argument definition.

```
#include <argz.hpp>
```

Public Member Functions

- [Argument](#) ()
- [Argument](#) (const [Argument](#) &a)
- [Argument](#) & [operator=](#) (const [Argument](#)< [String](#) > &a)
- auto [operator<=>](#) (const [Argument](#)< [String](#) > &a) const

Public Attributes

- [String](#) [arg_name](#)
long option name (for double-dash)
- [int](#) [arg_letter](#)
short option character code
- [String](#) [arg_value](#)
parsed value (for value-taking options)
- [ArgType](#) [arg_type](#)
switch type
- [String](#) [desc](#)
help description

11.4.1 Detailed Description

```
template<StringType String>
struct mx::Argument< String >
```

A parsed or registered command-line argument definition.

Template Parameters

<i>String</i>	A StringType-constrained string class
---------------	---------------------------------------

Definition at line 52 of file [argz.hpp](#).

11.4.2 Constructor & Destructor Documentation

11.4.2.1 Argument() [1/2]

```
template<StringType String>
mx::Argument< String >::Argument () [inline]
```

Definition at line 58 of file [argz.hpp](#).
References [arg_letter](#), [arg_name](#), [arg_type](#), [arg_value](#), and [desc](#).
Referenced by [Argument\(\)](#), [operator<=>\(\)](#), and [operator=\(\)](#).

11.4.2.2 Argument() [2/2]

```
template<StringType String>
mx::Argument< String >::Argument (
    const Argument< String > & a) [inline]
```

Definition at line 59 of file [argz.hpp](#).
References [Argument\(\)](#), [arg_letter](#), [arg_name](#), [arg_type](#), [arg_value](#), and [desc](#).

11.4.3 Member Function Documentation

11.4.3.1 operator<=>()

```
template<StringType String>
auto mx::Argument< String >::operator<=> (
    const Argument< String > & a) const [inline]
```

Definition at line 68 of file [argz.hpp](#).

References [Argument\(\)](#), and [arg_letter](#).

11.4.3.2 operator=()

```
template<StringType String>
Argument & mx::Argument< String >::operator= (
    const Argument< String > & a) [inline]
```

Definition at line 60 of file [argz.hpp](#).

References [Argument\(\)](#), [arg_letter](#), [arg_name](#), [arg_type](#), [arg_value](#), and [desc](#).

11.4.4 Member Data Documentation

11.4.4.1 arg_letter

```
template<StringType String>
int mx::Argument< String >::arg_letter
short option character code
```

Definition at line 54 of file [argz.hpp](#).

Referenced by [Argument\(\)](#), [Argument\(\)](#), [mx::Argz< String >::addOptionDouble\(\)](#), [mx::Argz< String >::addOptionDoubleValue\(\)](#), [mx::Argz< String >::addOptionSingle\(\)](#), [mx::Argz< String >::addOptionSingleValue\(\)](#), [operator<=>\(\)](#), and [operator=\(\)](#).

11.4.4.2 arg_name

```
template<StringType String>
String mx::Argument< String >::arg_name
long option name (for double-dash)
```

Definition at line 53 of file [argz.hpp](#).

Referenced by [Argument\(\)](#), [Argument\(\)](#), [mx::Argz< String >::addOptionDouble\(\)](#), [mx::Argz< String >::addOptionDoubleValue\(\)](#), [operator=\(\)](#), and [mx::Argz< String >::proc\(\)](#).

11.4.4.3 arg_type

```
template<StringType String>
ArgType mx::Argument< String >::arg_type
switch type
```

Definition at line 56 of file [argz.hpp](#).

Referenced by [Argument\(\)](#), [Argument\(\)](#), [mx::Argz< String >::addOptionDouble\(\)](#), [mx::Argz< String >::addOptionDoubleValue\(\)](#), [mx::Argz< String >::addOptionSingle\(\)](#), [mx::Argz< String >::addOptionSingleValue\(\)](#), [operator=\(\)](#), and [mx::Argz< String >::proc\(\)](#).

11.4.4.4 arg_value

```
template<StringType String>
String mx::Argument< String >::arg_value
parsed value (for value-taking options)
```

Definition at line 55 of file [argz.hpp](#).

Referenced by [Argument\(\)](#), [Argument\(\)](#), [main\(\)](#), [operator=\(\)](#), [mx::Argz< String >::proc\(\)](#), [mx::proc_args\(\)](#), and [proc_args\(\)](#).

11.4.4.5 desc

```
template<StringType String>
String mx::Argument< String >::desc
help description
```

Definition at line 57 of file [argz.hpp](#).

Referenced by [Argument\(\)](#), [Argument\(\)](#), [mx::Argz< String >::addOptionDouble\(\)](#), [mx::Argz< String >::addOptionDoubleValue\(\)](#), [mx::Argz< String >::addOptionSingle\(\)](#), [mx::Argz< String >::addOptionSingleValue\(\)](#), and [operator=\(\)](#).

The documentation for this struct was generated from the following file:

- [src/vm/include/argz.hpp](#)

11.5 mx::ArgumentData< String > Struct Template Reference

Raw argument data collected from argc/argv.

```
#include <argz.hpp>
```

Public Member Functions

- [ArgumentData](#) ()=default
- [ArgumentData](#) (const [ArgumentData](#)< String > &a)
- [ArgumentData](#) & [operator=](#) (const [ArgumentData](#)< String > &a)
- [ArgumentData](#) ([ArgumentData](#)< String > &&a)
- [ArgumentData](#)< String > & [operator=](#) ([ArgumentData](#)< String > &&a)

Public Attributes

- [std::vector](#)< String > [args](#)
parsed argument strings (argv[1..])
- [int](#) [argc](#)
original argument count

11.5.1 Detailed Description

```
template<StringType String>
struct mx::ArgumentData< String >
```

Raw argument data collected from argc/argv.

Template Parameters

<i>String</i>	A StringType-constrained string class
---------------	---------------------------------------

Definition at line 76 of file [argz.hpp](#).

11.5.2 Constructor & Destructor Documentation

11.5.2.1 ArgumentData() [1/3]

```
template<StringType String>
mx::ArgumentData< String >::ArgumentData () [default]
Referenced by ArgumentData\(\), ArgumentData\(\), operator=\(\), and operator=\(\).
```

11.5.2.2 ArgumentData() [2/3]

```
template<StringType String>
mx::ArgumentData< String >::ArgumentData (
    const ArgumentData< String > & a) [inline]
```

Definition at line 80 of file [argz.hpp](#).

References [ArgumentData\(\)](#), [argc](#), and [args](#).

11.5.2.3 ArgumentData() [3/3]

```
template<StringType String>
mx::ArgumentData< String >::ArgumentData (
    ArgumentData< String > && a) [inline]
```

Definition at line 89 of file [argz.hpp](#).

References [ArgumentData\(\)](#), [argc](#), and [args](#).

11.5.3 Member Function Documentation

11.5.3.1 operator=() [1/2]

```
template<StringType String>
ArgumentData< String > & mx::ArgumentData< String >::operator= (
    ArgumentData< String > && a) [inline]
```

Definition at line 90 of file [argz.hpp](#).

References [ArgumentData\(\)](#), [argc](#), and [args](#).

11.5.3.2 operator=() [2/2]

```
template<StringType String>
ArgumentData & mx::ArgumentData< String >::operator= (
    const ArgumentData< String > & a) [inline]
```

Definition at line 81 of file [argz.hpp](#).

References [ArgumentData\(\)](#), [argc](#), and [args](#).

11.5.4 Member Data Documentation

11.5.4.1 argc

```
template<StringType String>
int mx::ArgumentData< String >::argc
```

original argument count

Definition at line 78 of file [argz.hpp](#).

Referenced by [ArgumentData\(\)](#), [ArgumentData\(\)](#), [operator=\(\)](#), and [operator=\(\)](#).

11.5.4.2 args

```
template<StringType String>
std::vector<String> mx::ArgumentData< String >::args
```

parsed argument strings (argv[1..])

Definition at line 77 of file [argz.hpp](#).

Referenced by [ArgumentData\(\)](#), [ArgumentData\(\)](#), [operator=\(\)](#), and [operator=\(\)](#).

The documentation for this struct was generated from the following file:

- [src/vm/include/argz.hpp](#)

11.6 mx::Arguments Struct Reference

Parsed SDL program arguments (resolution, path, fullscreen).

```
#include <argz.hpp>
```

Public Attributes

- int [width](#)
- int [height](#)
window dimensions
- std::string [path](#)
assets path
- bool [fullscreen](#)
fullscreen mode flag

11.6.1 Detailed Description

Parsed SDL program arguments (resolution, path, fullscreen).

Definition at line 485 of file [argz.hpp](#).

11.6.2 Member Data Documentation

11.6.2.1 fullscreen

`bool mx::Arguments::fullscreen`
 fullscreen mode flag
 Definition at line 488 of file [argz.hpp](#).
 Referenced by [mx::proc_args\(\)](#).

11.6.2.2 height

`int mx::Arguments::height`
 window dimensions
 Definition at line 486 of file [argz.hpp](#).
 Referenced by [mx::proc_args\(\)](#).

11.6.2.3 path

`std::string mx::Arguments::path`
 assets path
 Definition at line 487 of file [argz.hpp](#).
 Referenced by [mx::proc_args\(\)](#).

11.6.2.4 width

`int mx::Arguments::width`
 Definition at line 486 of file [argz.hpp](#).
 Referenced by [mx::proc_args\(\)](#).
 The documentation for this struct was generated from the following file:

- [src/vm/include/argz.hpp](#)

11.7 mx::Argz< String > Class Template Reference

Generic command-line argument parser supporting single/double-dash options.
`#include <argz.hpp>`

Public Member Functions

- [Argz](#) ()=default
- [Argz](#) (int [argc](#), char **[argv](#))
Construct and initialize from argc/argv.
- [Argz](#) (const [Argz](#)< String > &a)
Copy constructor.
- [Argz](#)< String > & [operator=](#) (const [Argz](#)< String > &a)
- [Argz](#) ([Argz](#)< String > &&a)
Move constructor.
- [Argz](#)< String > & [operator=](#) ([Argz](#)< String > &&a)
Move-assignment operator.
- [Argz](#)< String > & [initArgs](#) (int [argc](#), char **[argv](#))
Initialize the parser from argc/argv (skips argv[0]).
- void [reset](#) ()
Reset the parser index to the beginning.
- [Argz](#)< String > & [addOptionSingle](#) (const int &c, const String &description)
Register a single-dash flag (no value).
- [Argz](#)< String > & [addOptionSingleValue](#) (const int &c, const String &description)
Register a single-dash option that takes a value.

- [Argz< String > & addOptionDouble](#) (const int &code, const String &value, const String &description)
Register a double-dash flag (no value).
- [Argz< String > & addOptionDoubleValue](#) (const int &code, const String &value, const String &description)
Register a double-dash option that takes a value.
- int [lookUpCode](#) (const String &value)
Look up a registered option's code by its long name.
- int [proc](#) ([Argument< String > &a](#))
Process the next argument from the command line.
- template<typename T>
void [help](#) (T &cout)
Print a formatted help message listing all registered options.

Protected Attributes

- [ArgumentData< String > arg_data](#)
raw argument data from argc/argv
- std::unordered_map< int, [Argument< String > > arg_info](#)
registered option definitions

Private Attributes

- int [index](#) = 0
- int [cindex](#) = 1
current parse position

11.7.1 Detailed Description

template<StringType String>
class mx::Argz< String >

Generic command-line argument parser supporting single/double-dash options.

Supports short options (-x), short options with values (-x value), long options (--name), and long options with values (--name value). Template parameter must satisfy the [StringType](#) concept.

Template Parameters

<i>String</i>	A StringType-constrained string class (std::string or std::wstring)
---------------	---

Definition at line 126 of file [argz.hpp](#).

11.7.2 Constructor & Destructor Documentation

11.7.2.1 Argz() [1/4]

```
template<StringType String>
```

```
mx::Argz< String >::Argz () [default]
```

Referenced by [Argz\(\)](#), [Argz\(\)](#), [addOptionDouble\(\)](#), [addOptionDoubleValue\(\)](#), [addOptionSingle\(\)](#), [addOptionSingleValue\(\)](#), [initArgs\(\)](#), [operator=\(\)](#), and [operator=\(\)](#).

11.7.2.2 Argz() [2/4]

```
template<StringType String>
```

```
mx::Argz< String >::Argz (
```

```
    int argc,
```

```
    char ** argv) [inline]
```

Construct and initialize from argc/argv.

Definition at line 131 of file [argz.hpp](#).

References [argc\(\)](#), [argv\(\)](#), and [initArgs\(\)](#).

11.7.2.3 Argz() [3/4]

```
template<StringType String>
mx::Argz< String >::Argz (
    const Argz< String > & a) [inline]
```

Copy constructor.

Definition at line 134 of file [argz.hpp](#).

References [Argz\(\)](#), [arg_data](#), [arg_info](#), [cindex](#), and [index](#).

11.7.2.4 Argz() [4/4]

```
template<StringType String>
mx::Argz< String >::Argz (
    Argz< String > && a) [inline]
```

Move constructor.

Definition at line 150 of file [argz.hpp](#).

References [Argz\(\)](#), [arg_data](#), [arg_info](#), [cindex](#), and [index](#).

11.7.3 Member Function Documentation

11.7.3.1 addOptionDouble()

```
template<StringType String>
Argz< String > & mx::Argz< String >::addOptionDouble (
    const int & code,
    const String & value,
    const String & description) [inline]
```

Register a double-dash flag (no value).

Parameters

<i>code</i>	Numeric code identifying this option
<i>value</i>	Long option name (e.g. "help")
<i>description</i>	Help text

Returns

Reference to this parser for chaining

Definition at line 232 of file [argz.hpp](#).

References [Argz\(\)](#), [mx::ARG_DOUBLE](#), [arg_info](#), [mx::Argument< String >::arg_letter](#), [mx::Argument< String >::arg_name](#), [mx::Argument< String >::arg_type](#), and [mx::Argument< String >::desc](#).

11.7.3.2 addOptionDoubleValue()

```
template<StringType String>
Argz< String > & mx::Argz< String >::addOptionDoubleValue (
    const int & code,
    const String & value,
    const String & description) [inline]
```

Register a double-dash option that takes a value.

Parameters

<i>code</i>	Numeric code identifying this option
<i>value</i>	Long option name
<i>description</i>	Help text

Returns

Reference to this parser for chaining

Definition at line 248 of file [argz.hpp](#).

References [Argz\(\)](#), [mx::ARG_DOUBLE_VALUE](#), [arg_info](#), [mx::Argument< String >::arg_letter](#), [mx::Argument< String >::arg_name](#), [mx::Argument< String >::arg_type](#), and [mx::Argument< String >::desc](#).

11.7.3.3 addOptionSingle()

```
template<StringType String>
Argz< String > & mx::Argz< String >::addOptionSingle (
    const int & c,
    const String & description) [inline]
```

Register a single-dash flag (no value).

Parameters

<i>c</i>	Option character
<i>description</i>	Help text

Returns

Reference to this parser for chaining

Definition at line 203 of file [argz.hpp](#).

References [Argz\(\)](#), [arg_info](#), [mx::Argument< String >::arg_letter](#), [mx::ARG_SINGLE](#), [mx::Argument< String >::arg_type](#), and [mx::Argument< String >::desc](#).
Referenced by [main\(\)](#), and [mx::proc_args\(\)](#).

11.7.3.4 addOptionSingleValue()

```
template<StringType String>
Argz< String > & mx::Argz< String >::addOptionSingleValue (
    const int & c,
    const String & description) [inline]
```

Register a single-dash option that takes a value.

Parameters

<i>c</i>	Option character
<i>description</i>	Help text

Returns

Reference to this parser for chaining

Definition at line 217 of file [argz.hpp](#).

References [Argz\(\)](#), [arg_info](#), [mx::Argument< String >::arg_letter](#), [mx::ARG_SINGLE_VALUE](#), [mx::Argument< String >::arg_type](#), and [mx::Argument< String >::desc](#).
Referenced by [main\(\)](#), and [proc_args\(\)](#).

11.7.3.5 help()

```
template<StringType String>
template<typename T>
void mx::Argz< String >::help (
    T & cout) [inline]
```

Print a formatted help message listing all registered options.

Template Parameters

<i>T</i>	Output stream type (e.g. <code>std::ostream</code> , <code>std::wostream</code>)
----------	---

Parameters

<i>cout</i>	Output stream
-------------	---------------

Definition at line 427 of file [argz.hpp](#).

References [mx::ARG_DOUBLE](#), [mx::ARG_DOUBLE_VALUE](#), [arg_info](#), [mx::ARG_SINGLE](#), and [mx::ARG_SINGLE_VALUE](#).

Referenced by [main\(\)](#), and [mx::proc_args\(\)](#).

11.7.3.6 initArgs()

```
template<StringType String>
Argz< String > & mx::Argz< String >::initArgs (
    int argc,
    char ** argv) [inline]
```

Initialize the parser from argc/argv (skips argv[0]).

Parameters

<i>argc</i>	Argument count
<i>argv</i>	Argument vector

Returns

Reference to this parser for chaining

Definition at line 166 of file [argz.hpp](#).

References [Argz\(\)](#), [arg_data](#), [argc\(\)](#), [argv\(\)](#), and [reset\(\)](#).

Referenced by [Argz\(\)](#).

11.7.3.7 lookUpCode()

```
template<StringType String>
int mx::Argz< String >::lookUpCode (
    const String & value) [inline]
```

Look up a registered option's code by its long name.

Parameters

<i>value</i>	Long option name to search for
--------------	--------------------------------

Returns

The option code, or -1 if not found

Definition at line 262 of file [argz.hpp](#).

References [arg_info](#).

Referenced by [proc\(\)](#).

11.7.3.8 operator=() [1/2]

```
template<StringType String>
Argz< String > & mx::Argz< String >::operator= (
    Argz< String > && a) [inline]
```

Move-assignment operator.

Definition at line 153 of file [argz.hpp](#).

References [Argz\(\)](#), [arg_data](#), [arg_info](#), [cindex](#), and [index](#).

11.7.3.9 operator=() [2/2]

```
template<StringType String>
Argz< String > & mx::Argz< String >::operator= (
    const Argz< String > & a) [inline]
```

Definition at line 136 of file [argz.hpp](#).

References [Argz\(\)](#), [arg_data](#), [arg_info](#), [cindex](#), and [index](#).

11.7.3.10 proc()

```
template<StringType String>
int mx::Argz< String >::proc (
    Argument< String > & a) [inline]
```

Process the next argument from the command line.

Parameters

out	a	Receives the matched Argument definition and parsed value
-----	---	---

Returns

The option code, '-' for positional arguments, or -1 when done

Exceptions

ArgException	on malformed or unknown arguments
------------------------------	-----------------------------------

Definition at line 276 of file [argz.hpp](#).

References [arg_data](#), [mx::ARG_DOUBLE](#), [arg_info](#), [mx::Argument< String >::arg_name](#), [mx::ARG_NONE](#), [mx::ARG_SINGLE](#), [mx::ARG_SINGLE_VALUE](#), [mx::Argument< String >::arg_type](#), [mx::Argument< String >::arg_value](#), [cindex](#), [index](#), [lookUpCode\(\)](#), and [pos\(\)](#).

Referenced by [main\(\)](#), [mx::proc_args\(\)](#), and [proc_args\(\)](#).

11.7.3.11 reset()

```
template<StringType String>
void mx::Argz< String >::reset () [inline]
```

Reset the parser index to the beginning.

Definition at line 193 of file [argz.hpp](#).

References [cindex](#), and [index](#).

Referenced by [initArgs\(\)](#).

11.7.4 Member Data Documentation**11.7.4.1 arg_data**

```
template<StringType String>
ArgumentData<String> mx::Argz< String >::arg_data [protected]
```

raw argument data from argc/argv

Definition at line 477 of file [argz.hpp](#).

Referenced by [Argz\(\)](#), [Argz\(\)](#), [initArgs\(\)](#), [operator=\(\)](#), [operator=\(\)](#), and [proc\(\)](#).

11.7.4.2 `arg_info`

```
template<StringType String>
```

```
std::unordered_map<int, Argument<String> > mx::Argz< String >::arg_info [protected]
```

registered option definitions

Definition at line 478 of file [argz.hpp](#).

Referenced by [Argz\(\)](#), [Argz\(\)](#), [addOptionDouble\(\)](#), [addOptionDoubleValue\(\)](#), [addOptionSingle\(\)](#), [addOptionSingleValue\(\)](#), [help\(\)](#), [lookUpCode\(\)](#), [operator=\(\)](#), [operator=\(\)](#), and [proc\(\)](#).

11.7.4.3 `cindex`

```
template<StringType String>
```

```
int mx::Argz< String >::cindex = 1 [private]
```

current parse position

Definition at line 481 of file [argz.hpp](#).

Referenced by [Argz\(\)](#), [Argz\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [proc\(\)](#), and [reset\(\)](#).

11.7.4.4 `index`

```
template<StringType String>
```

```
int mx::Argz< String >::index = 0 [private]
```

Definition at line 481 of file [argz.hpp](#).

Referenced by [Argz\(\)](#), [Argz\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [proc\(\)](#), and [reset\(\)](#).

The documentation for this class was generated from the following file:

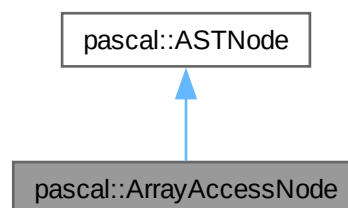
- [src/vm/include/argz.hpp](#)

11.8 `pascal::ArrayAccessNode` Class Reference

AST node for an array element access (`arr[index]`).

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::ArrayAccessNode`:



Public Member Functions

- [ArrayAccessNode](#) (`std::unique_ptr< ASTNode > base`, `std::unique_ptr< ASTNode > index`)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- `std::string` [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from pascal::ASTNode

- virtual `~ASTNode()` = default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::unique_ptr< [ASTNode](#) > `base`
array expression
- std::unique_ptr< [ASTNode](#) > `index`
index expression

11.8.1 Detailed Description

AST node for an array element access (`arr[index]`).
Definition at line 321 of file [ast.hpp](#).

11.8.2 Constructor & Destructor Documentation

11.8.2.1 ArrayAccessNode()

```
pascal::ArrayAccessNode::ArrayAccessNode (
    std::unique_ptr< ASTNode > base,
    std::unique_ptr< ASTNode > index) [inline]
```

Definition at line 326 of file [ast.hpp](#).
References [base](#), and [index](#).

11.8.3 Member Function Documentation

11.8.3.1 accept()

```
void pascal::ArrayAccessNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).
Implements [pascal::ASTNode](#).
Definition at line 334 of file [ast.cpp](#).
References [pascal::ASTVisitor::visit\(\)](#).

11.8.3.2 toString()

```
std::string pascal::ArrayAccessNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.
Implements [pascal::ASTNode](#).
Definition at line 338 of file [ast.cpp](#).

11.8.4 Member Data Documentation

11.8.4.1 base

```
std::unique_ptr<ASTNode> pascal::ArrayAccessNode::base
```

array expression

Definition at line 323 of file [ast.hpp](#).
Referenced by [ArrayAccessNode\(\)](#), [pascal::CodeGenVisitor::getArrayInfoForArrayAccess\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.8.4.2 index

`std::unique_ptr<ASTNode> pascal::ArrayAccessNode::index`

index expression

Definition at line 324 of file [ast.hpp](#).

Referenced by [ArrayAccessNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

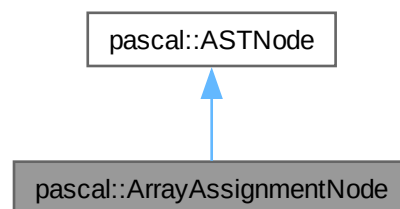
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.9 pascal::ArrayAssignmentNode Class Reference

AST node for assigning to an array element (`arr[index] := value`).

`#include <ast.hpp>`

Inheritance diagram for `pascal::ArrayAssignmentNode`:



Public Member Functions

- [ArrayAssignmentNode](#) (`std::string arrayName`, `std::unique_ptr< ASTNode > index`, `std::unique_ptr< ASTNode > value`)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- `std::string toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual `~ASTNode` ()=default
- virtual void [print](#) (`std::ostream &out`, `int indent=0`) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (`int line`)
Set the originating source line number.
- `int getLineNumber` () const
Get the originating source line number.

Public Attributes

- `std::string arrayName`
array variable name
- `std::unique_ptr< ASTNode > index`
index expression

- `std::unique_ptr< ASTNode > value`
value expression

11.9.1 Detailed Description

AST node for assigning to an array element (`arr[index] := value`).
 Definition at line 334 of file [ast.hpp](#).

11.9.2 Constructor & Destructor Documentation

11.9.2.1 ArrayAssignmentNode()

```
pascal::ArrayAssignmentNode::ArrayAssignmentNode (
    std::string arrayName,
    std::unique_ptr< ASTNode > index,
    std::unique_ptr< ASTNode > value) [inline]
```

Definition at line 340 of file [ast.hpp](#).

References [arrayName](#), [index](#), and [value](#).

11.9.3 Member Function Documentation

11.9.3.1 accept()

```
void pascal::ArrayAssignmentNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 342 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.9.3.2 toString()

```
std::string pascal::ArrayAssignmentNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 346 of file [ast.cpp](#).

References [arrayName](#), [index](#), and [value](#).

11.9.4 Member Data Documentation

11.9.4.1 arrayName

```
std::string pascal::ArrayAssignmentNode::arrayName
```

array variable name

Definition at line 336 of file [ast.hpp](#).

Referenced by [ArrayAssignmentNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.9.4.2 index

```
std::unique_ptr<ASTNode> pascal::ArrayAssignmentNode::index
```

index expression

Definition at line 337 of file [ast.hpp](#).

Referenced by [ArrayAssignmentNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.9.4.3 value

```
std::unique_ptr<ASTNode> pascal::ArrayAssignmentNode::value
```

value expression

Definition at line 338 of file [ast.hpp](#).

Referenced by [ArrayAssignmentNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

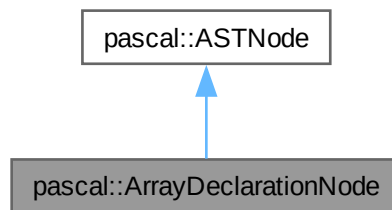
- src/frontend/include/[ast.hpp](#)
- src/frontend/[ast.cpp](#)

11.10 pascal::ArrayDeclarationNode Class Reference

AST node for a named array variable declaration.

```
#include <ast.hpp>
```

Inheritance diagram for pascal::ArrayDeclarationNode:



Public Member Functions

- [ArrayDeclarationNode](#) (std::string [name](#), std::unique_ptr< [ArrayTypeNode](#) > [arrayType](#), std::vector< std::unique_ptr< [ASTNode](#) > > [initializers](#)={})
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [name](#)
array variable name
- std::unique_ptr< [ArrayTypeNode](#) > [arrayType](#)
array type (bounds + element)
- std::vector< std::unique_ptr< [ASTNode](#) > > [initializers](#)
optional initialisers

11.10.1 Detailed Description

AST node for a named array variable declaration.
Definition at line 292 of file [ast.hpp](#).

11.10.2 Constructor & Destructor Documentation

11.10.2.1 ArrayDeclarationNode()

```
pascal::ArrayDeclarationNode::ArrayDeclarationNode (
    std::string name,
    std::unique_ptr< ArrayTypeNode > arrayType,
    std::vector< std::unique_ptr< ASTNode > > initializers = {}) [inline]
```

Definition at line 298 of file [ast.hpp](#).

References [arrayType](#), [initializers](#), and [name](#).

11.10.3 Member Function Documentation

11.10.3.1 accept()

```
void pascal::ArrayDeclarationNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 326 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.10.3.2 toString()

```
std::string pascal::ArrayDeclarationNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 330 of file [ast.cpp](#).

References [arrayType](#), and [name](#).

11.10.4 Member Data Documentation

11.10.4.1 arrayType

```
std::unique_ptr<ArrayTypeName> pascal::ArrayDeclarationNode::arrayType
```

array type (bounds + element)

Definition at line 295 of file [ast.hpp](#).

Referenced by [ArrayDeclarationNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.10.4.2 initializers

```
std::vector<std::unique_ptr<ASTNode> > pascal::ArrayDeclarationNode::initializers
```

optional initialisers

Definition at line 296 of file [ast.hpp](#).

Referenced by [ArrayDeclarationNode\(\)](#).

11.10.4.3 name

```
std::string pascal::ArrayDeclarationNode::name
```

array variable name

Definition at line 294 of file [ast.hpp](#).

Referenced by [ArrayDeclarationNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.11 pascal::ArrayInfo Struct Reference

Metadata describing a Pascal array's bounds, element type, and sizing.

```
#include <icode.hpp>
```

Public Member Functions

- [ArrayInfo](#) ()=default
- [ArrayInfo](#) (const [ArrayInfo](#) &other)
- [ArrayInfo](#) ([ArrayInfo](#) &&other) noexcept=default
- [ArrayInfo](#) & [operator=](#) (const [ArrayInfo](#) &other)
- [ArrayInfo](#) & [operator=](#) ([ArrayInfo](#) &&other) noexcept=default

Public Attributes

- std::string [elementType](#)
element type name (e.g. "integer", "real")
- int [lowerBound](#) = 0
declared lower index bound
- int [upperBound](#) = -1
declared upper index bound
- int [size](#) = 0
number of elements
- int [elementSize](#) = 8
size (bytes) of each element
- bool [elementIsArray](#) = false
true if element type is itself an array
- bool [isDynamic](#) = false
true for dynamic arrays declared as array of <type> (no compile-time bounds)
- std::unique_ptr< [ArrayInfo](#) > [elementArray](#)
nested array descriptor (when elementIsArray)

11.11.1 Detailed Description

Metadata describing a Pascal array's bounds, element type, and sizing.

Definition at line 29 of file [icode.hpp](#).

11.11.2 Constructor & Destructor Documentation

11.11.2.1 [ArrayInfo](#)() [1/3]

```
pascal::ArrayInfo::ArrayInfo () [default]
```

Referenced by [ArrayInfo\(\)](#), [ArrayInfo\(\)](#), [operator=\(\)](#), and [operator=\(\)](#).

11.11.2.2 [ArrayInfo](#)() [2/3]

```
pascal::ArrayInfo::ArrayInfo (
    const ArrayInfo & other) [inline]
```

Definition at line 40 of file [icode.hpp](#).

References [ArrayInfo\(\)](#), [elementArray](#), [elementIsArray](#), [elementSize](#), [elementType](#), [isDynamic](#), [lowerBound](#), [size](#), and [upperBound](#).

11.11.2.3 [ArrayInfo](#)() [3/3]

```
pascal::ArrayInfo::ArrayInfo (
    ArrayInfo && other) [default], [noexcept]
```

References [ArrayInfo\(\)](#).

11.11.3 Member Function Documentation

11.11.3.1 operator=() [1/2]

`ArrayInfo` & `pascal::ArrayInfo::operator=` (
`ArrayInfo` && `other`) [`default`], [`noexcept`]

References [ArrayInfo\(\)](#).

11.11.3.2 operator=() [2/2]

`ArrayInfo` & `pascal::ArrayInfo::operator=` (
`const ArrayInfo` & `other`) [`inline`]

Definition at line 49 of file [icode.hpp](#).

References [ArrayInfo\(\)](#), [elementArray](#), [elementIsArray](#), [elementSize](#), [elementType](#), [isDynamic](#), [lowerBound](#), [size](#), and [upperBound](#).

11.11.4 Member Data Documentation

11.11.4.1 elementArray

`std::unique_ptr<ArrayInfo>` `pascal::ArrayInfo::elementArray`

nested array descriptor (when `elementIsArray`)

Definition at line 37 of file [icode.hpp](#).

Referenced by [ArrayInfo\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), [pascal::CodeGenVisitor::getArrayInfoForArrayAccess\(\)](#), and [operator=\(\)](#).

11.11.4.2 elementIsArray

`bool` `pascal::ArrayInfo::elementIsArray` = `false`

true if element type is itself an array

Definition at line 35 of file [icode.hpp](#).

Referenced by [ArrayInfo\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), [pascal::CodeGenVisitor::getArrayInfoForArrayAccess\(\)](#), [pascal::CodeGenVisitor::getExpressionType\(\)](#), [operator=\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.11.4.3 elementSize

`int` `pascal::ArrayInfo::elementSize` = 8

size (bytes) of each element

Definition at line 34 of file [icode.hpp](#).

Referenced by [ArrayInfo\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), [operator=\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.11.4.4 elementType

`std::string` `pascal::ArrayInfo::elementType`

element type name (e.g. "integer", "real")

Definition at line 30 of file [icode.hpp](#).

Referenced by [ArrayInfo\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), [pascal::CodeGenVisitor::getExpressionType\(\)](#), [pascal::CodeGenVisitor::getVarRecordTypeNameFromExpr\(\)](#), [operator=\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.11.4.5 isDynamic

`bool` `pascal::ArrayInfo::isDynamic` = `false`

true for dynamic arrays declared as `array of <type>` (no compile-time bounds)

Definition at line 36 of file [icode.hpp](#).

Referenced by [ArrayInfo\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), [operator=\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.11.4.6 lowerBound

```
int pascal::ArrayInfo::lowerBound = 0
```

declared lower index bound

Definition at line 31 of file [icode.hpp](#).

Referenced by [ArrayInfo\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), [operator=\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.11.4.7 size

```
int pascal::ArrayInfo::size = 0
```

number of elements

Definition at line 33 of file [icode.hpp](#).

Referenced by [ArrayInfo\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), [operator=\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.11.4.8 upperBound

```
int pascal::ArrayInfo::upperBound = -1
```

declared upper index bound

Definition at line 32 of file [icode.hpp](#).

Referenced by [ArrayInfo\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), [operator=\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this struct was generated from the following file:

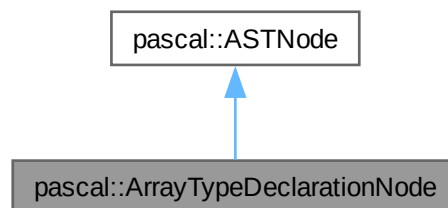
- [src/frontend/include/icode.hpp](#)

11.12 pascal::ArrayTypeDeclarationNode Class Reference

AST node for a named array type declaration (type Name = array[...]).

```
#include <ast.hpp>
```

Inheritance diagram for pascal::ArrayTypeDeclarationNode:



Public Member Functions

- [ArrayTypeDeclarationNode](#) (const std::string &name, std::unique_ptr< [ArrayTypeNode](#) > arrayType)
- void [accept](#) (ASTVisitor &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from pascal::ASTNode

- virtual `~ASTNode()` = default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::string `name`
type name
- std::unique_ptr< [ArrayTypeNode](#) > `arrayType`
the underlying array type

11.12.1 Detailed Description

AST node for a named array type declaration (type Name = array[...]).
Definition at line 308 of file [ast.hpp](#).

11.12.2 Constructor & Destructor Documentation

11.12.2.1 ArrayTypeDeclarationNode()

```
pascal::ArrayTypeDeclarationNode::ArrayTypeDeclarationNode (
    const std::string & name,
    std::unique_ptr< ArrayTypeNode > arrayType) [inline]
```

Definition at line 313 of file [ast.hpp](#).

References [arrayType](#), and [name](#).

11.12.3 Member Function Documentation

11.12.3.1 accept()

```
void pascal::ArrayTypeDeclarationNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 394 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.12.3.2 toString()

```
std::string pascal::ArrayTypeDeclarationNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 398 of file [ast.cpp](#).

References [name](#).

11.12.4 Member Data Documentation

11.12.4.1 arrayType

```
std::unique_ptr<ArrayTypeNode> pascal::ArrayTypeDeclarationNode::arrayType
```

the underlying array type

Definition at line 311 of file [ast.hpp](#).

Referenced by [ArrayTypeDeclarationNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.12.4.2 name

`std::string pascal::ArrayTypeDeclarationNode::name`

type name

Definition at line 310 of file [ast.hpp](#).

Referenced by [ArrayTypeDeclarationNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

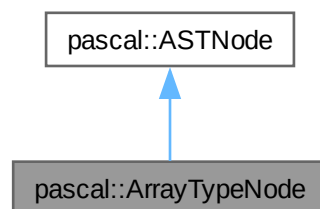
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.13 pascal::ArrayTypeNode Class Reference

AST node for an array type (array[lower..upper] of elementType).

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::ArrayTypeNode`:



Public Member Functions

- [ArrayTypeNode](#) (`std::unique_ptr< ASTNode > elementType`, `std::unique_ptr< ASTNode > lowerBound`, `std::unique_ptr< ASTNode > upperBound`)
- `std::string toString ()` const override
Return a human-readable description of this node.
- `void accept (ASTVisitor &visitor)` override
Accept an [ASTVisitor](#) (double-dispatch).

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual `~ASTNode ()`=default
- virtual `void print (std::ostream &out, int indent=0)` const
Pretty-print this node to a stream with indentation.
- `void setLineNumber (int line)`
Set the originating source line number.
- `int getLineNumber ()` const
Get the originating source line number.

Public Attributes

- `std::unique_ptr< ASTNode > elementType`
element type node
- `std::unique_ptr< ASTNode > lowerBound`
lower index bound expression

- `std::unique_ptr< ASTNode > upperBound`
upper index bound expression

11.13.1 Detailed Description

AST node for an array type (array[lower..upper] of elementType).
 Definition at line 275 of file [ast.hpp](#).

11.13.2 Constructor & Destructor Documentation

11.13.2.1 ArrayTypeNode()

```
pascal::ArrayTypeNode::ArrayTypeNode (
    std::unique_ptr< ASTNode > elementType,
    std::unique_ptr< ASTNode > lowerBound,
    std::unique_ptr< ASTNode > upperBound) [inline]
```

Definition at line 281 of file [ast.hpp](#).

References [elementType](#), [lowerBound](#), and [upperBound](#).

11.13.3 Member Function Documentation

11.13.3.1 accept()

```
void pascal::ArrayTypeNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 301 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.13.3.2 toString()

```
std::string pascal::ArrayTypeNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 305 of file [ast.cpp](#).

References [elementType](#), [lowerBound](#), and [upperBound](#).

11.13.4 Member Data Documentation

11.13.4.1 elementType

```
std::unique_ptr<ASTNode> pascal::ArrayTypeNode::elementType
```

element type node

Definition at line 277 of file [ast.hpp](#).

Referenced by [ArrayTypeNode\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), and [toString\(\)](#).

11.13.4.2 lowerBound

```
std::unique_ptr<ASTNode> pascal::ArrayTypeNode::lowerBound
```

lower index bound expression

Definition at line 278 of file [ast.hpp](#).

Referenced by [ArrayTypeNode\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), and [toString\(\)](#).

11.13.4.3 upperBound

```
std::unique_ptr<ASTNode> pascal::ArrayTypeNode::upperBound
```

upper index bound expression

Definition at line 279 of file [ast.hpp](#).

Referenced by [ArrayTypeNode\(\)](#), [pascal::CodeGenVisitor::buildArrayInfoFromNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

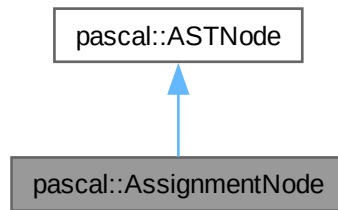
- src/frontend/include/[ast.hpp](#)
- src/frontend/[ast.cpp](#)

11.14 pascal::AssignmentNode Class Reference

AST node for an assignment statement (variable := expression).

```
#include <ast.hpp>
```

Inheritance diagram for pascal::AssignmentNode:



Public Member Functions

- [AssignmentNode](#) (std::unique_ptr< [ASTNode](#) > var, std::unique_ptr< [ASTNode](#) > expr)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::unique_ptr< [ASTNode](#) > [variable](#)
target of the assignment
- std::unique_ptr< [ASTNode](#) > [expression](#)
value expression

11.14.1 Detailed Description

AST node for an assignment statement (variable := expression).

Definition at line [249](#) of file [ast.hpp](#).

11.14.2 Constructor & Destructor Documentation

11.14.2.1 AssignmentNode()

```
pascal::AssignmentNode::AssignmentNode (
    std::unique_ptr< ASTNode > var,
    std::unique_ptr< ASTNode > expr) [inline]
```

Definition at line 254 of file [ast.hpp](#).

References [expression](#), and [variable](#).

11.14.3 Member Function Documentation

11.14.3.1 accept()

```
void pascal::AssignmentNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 117 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.14.3.2 toString()

```
std::string pascal::AssignmentNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 121 of file [ast.cpp](#).

11.14.4 Member Data Documentation

11.14.4.1 expression

```
std::unique_ptr<ASTNode> pascal::AssignmentNode::expression
value expression
```

Definition at line 252 of file [ast.hpp](#).

Referenced by [AssignmentNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.14.4.2 variable

```
std::unique_ptr<ASTNode> pascal::AssignmentNode::variable
target of the assignment
```

Definition at line 251 of file [ast.hpp](#).

Referenced by [AssignmentNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

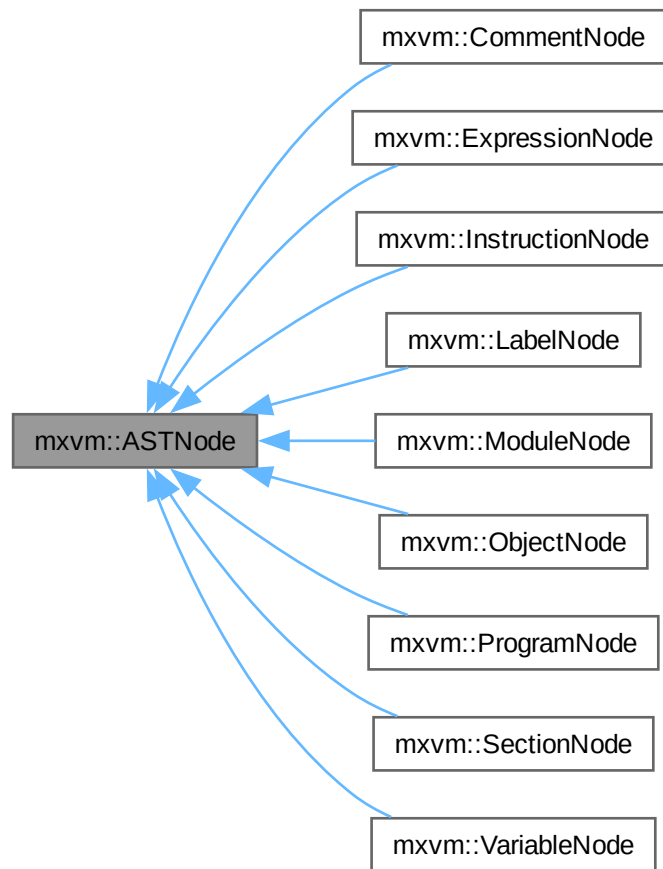
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.15 mxvm::ASTNode Class Reference

Abstract base class for all AST nodes.

```
#include <ast.hpp>
```

Inheritance diagram for mxvm::ASTNode:



Public Member Functions

- virtual [~ASTNode](#) ()=default
- virtual void [accept](#) (ASTVisitor &visitor)=0
- virtual std::string [toString](#) () const =0

11.15.1 Detailed Description

Abstract base class for all AST nodes.
Definition at line 19 of file [ast.hpp](#).

11.15.2 Constructor & Destructor Documentation

11.15.2.1 ~ASTNode()

```
virtual mxvm::ASTNode::~~ASTNode () [virtual], [default]
```

11.15.3 Member Function Documentation

11.15.3.1 accept()

```
virtual void mxvm::ASTNode::accept (
```

[ASTVisitor](#) & *visitor*) [pure virtual]

Implemented in [mxvm::CommentNode](#), [mxvm::ExpressionNode](#), [mxvm::InstructionNode](#), [mxvm::LabelNode](#), [mxvm::ModuleNode](#), [mxvm::ObjectNode](#), [mxvm::ProgramNode](#), [mxvm::SectionNode](#), and [mxvm::VariableNode](#).

11.15.3.2 toString()

virtual std::string mxvm::ASTNode::toString () const [pure virtual]

Implemented in [mxvm::CommentNode](#), [mxvm::ExpressionNode](#), [mxvm::InstructionNode](#), [mxvm::LabelNode](#), [mxvm::ModuleNode](#), [mxvm::ObjectNode](#), [mxvm::ProgramNode](#), [mxvm::SectionNode](#), and [mxvm::VariableNode](#).

The documentation for this class was generated from the following file:

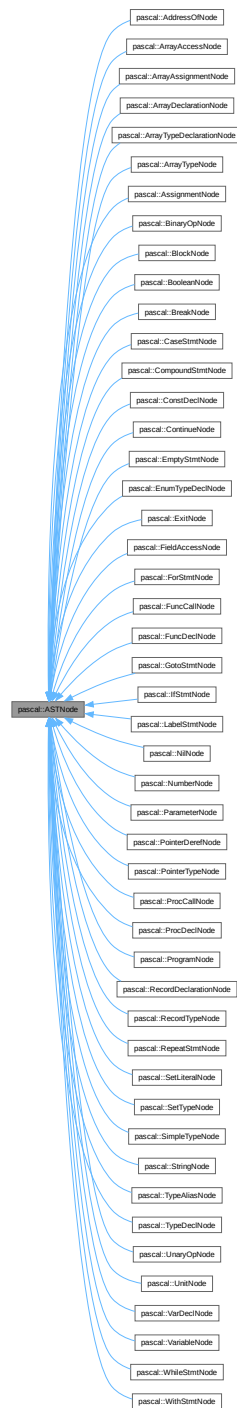
- [include/mxvm/ast.hpp](#)

11.16 pascal::ASTNode Class Reference

Abstract base class for all AST nodes.

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::ASTNode`:



Public Member Functions

- virtual `~ASTNode()` = default
- virtual void `accept (ASTVisitor &visitor)` = 0
Accept an ASTVisitor (double-dispatch).
- virtual std::string `toString ()` const = 0
Return a human-readable description of this node.
- virtual void `print (std::ostream &out, int indent=0)` const

Pretty-print this node to a stream with indentation.

- void [setLineNumber](#) (int line)

Set the originating source line number.

- int [getLineNumber](#) () const

Get the originating source line number.

Private Attributes

- int [line_number](#) = 0

source line number where this node originated

11.16.1 Detailed Description

Abstract base class for all AST nodes.

Every concrete AST node inherits from this class, implementing the visitor pattern via [accept\(\)](#) and providing a debug string via [toString\(\)](#).

Definition at line 26 of file [ast.hpp](#).

11.16.2 Constructor & Destructor Documentation

11.16.2.1 ~ASTNode()

```
virtual pascal::ASTNode::~~ASTNode () [virtual], [default]
```

11.16.3 Member Function Documentation

11.16.3.1 accept()

```
virtual void pascal::ASTNode::accept (
    ASTVisitor & visitor) [pure virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implemented in [pascal::AddressOfNode](#), [pascal::ArrayAccessNode](#), [pascal::ArrayAssignmentNode](#), [pascal::ArrayDeclarationNode](#), [pascal::ArrayTypeDeclarationNode](#), [pascal::ArrayTypeNode](#), [pascal::AssignmentNode](#), [pascal::BinaryOpNode](#), [pascal::BlockNode](#), [pascal::BooleanNode](#), [pascal::BreakNode](#), [pascal::CaseStmtNode](#), [pascal::CompoundStmtNode](#), [pascal::ConstDeclNode](#), [pascal::ContinueNode](#), [pascal::EmptyStmtNode](#), [pascal::EnumTypeDeclNode](#), [pascal::ExitNode](#), [pascal::FieldAccessNode](#), [pascal::ForStmtNode](#), [pascal::FuncCallNode](#), [pascal::FuncDeclNode](#), [pascal::GotoStmtNode](#), [pascal::IfStmtNode](#), [pascal::LabelStmtNode](#), [pascal::NilNode](#), [pascal::NumberNode](#), [pascal::ParameterNode](#), [pascal::PointerDerefNode](#), [pascal::PointerTypeNode](#), [pascal::ProcCallNode](#), [pascal::ProcDeclNode](#), [pascal::ProgramNode](#), [pascal::RecordDeclarationNode](#), [pascal::RecordTypeNode](#), [pascal::RepeatStmtNode](#), [pascal::SetLiteralNode](#), [pascal::SetTypeNode](#), [pascal::SimpleTypeNode](#), [pascal::StringNode](#), [pascal::TypeAliasNode](#), [pascal::TypeDeclNode](#), [pascal::UnaryOpNode](#), [pascal::UnitNode](#), [pascal::VarDeclNode](#), [pascal::VariableNode](#), [pascal::WhileStmtNode](#), and [pascal::WithStmtNode](#).

Referenced by [pascal::CodeGenVisitor::eval\(\)](#), and [pascal::CodeGenVisitor::generate\(\)](#).

11.16.3.2 getLineNumber()

```
int pascal::ASTNode::getLineNumber () const [inline]
```

Get the originating source line number.

Definition at line 40 of file [ast.hpp](#).

References [line_number](#).

11.16.3.3 print()

```
void pascal::ASTNode::print (
    std::ostream & out,
    int indent = 0) const [virtual]
```

Pretty-print this node to a stream with indentation.

Definition at line 17 of file [ast.cpp](#).

References [toString\(\)](#).

11.16.3.4 setLineNumber()

```
void pascal::ASTNode::setLineNumber (
    int line) [inline]
```

Set the originating source line number.

Definition at line 38 of file [ast.hpp](#).

References [line_number](#).

11.16.3.5 toString()

```
virtual std::string pascal::ASTNode::toString () const [pure virtual]
```

Return a human-readable description of this node.

Implemented in [pascal::AddressOfNode](#), [pascal::ArrayAccessNode](#), [pascal::ArrayAssignmentNode](#), [pascal::ArrayDeclarationNode](#), [pascal::ArrayTypeDeclarationNode](#), [pascal::ArrayTypeNode](#), [pascal::AssignmentNode](#), [pascal::BinaryOpNode](#), [pascal::BlockNode](#), [pascal::BooleanNode](#), [pascal::BreakNode](#), [pascal::CaseStmtNode](#), [pascal::CompoundStmtNode](#), [pascal::ConstDeclNode](#), [pascal::ContinueNode](#), [pascal::EmptyStmtNode](#), [pascal::EnumTypeDeclNode](#), [pascal::ExitNode](#), [pascal::FieldAccessNode](#), [pascal::ForStmtNode](#), [pascal::FuncCallNode](#), [pascal::FuncDeclNode](#), [pascal::GotoStmtNode](#), [pascal::IfStmtNode](#), [pascal::LabelStmtNode](#), [pascal::NilNode](#), [pascal::NumberNode](#), [pascal::ParameterNode](#), [pascal::PointerDerefNode](#), [pascal::PointerTypeNode](#), [pascal::ProcCallNode](#), [pascal::ProcDeclNode](#), [pascal::ProgramNode](#), [pascal::RecordDeclarationNode](#), [pascal::RecordTypeNode](#), [pascal::RepeatStmtNode](#), [pascal::SetLiteralNode](#), [pascal::SetTypeNode](#), [pascal::SimpleTypeNode](#), [pascal::StringNode](#), [pascal::TypeAliasNode](#), [pascal::TypeDeclNode](#), [pascal::UnaryOpNode](#), [pascal::UnitNode](#), [pascal::VarDeclNode](#), [pascal::VariableNode](#), [pascal::WhileStmtNode](#), and [pascal::WithStmtNode](#).

Referenced by [print\(\)](#).

11.16.4 Member Data Documentation

11.16.4.1 line_number

```
int pascal::ASTNode::line_number = 0 [private]
```

source line number where this node originated

Definition at line 27 of file [ast.hpp](#).

Referenced by [getLineNumber\(\)](#), and [setLineNumber\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.17 mxvm::ASTVisitor Class Reference

Visitor interface for traversing the AST node hierarchy.

```
#include <ast.hpp>
```

Public Member Functions

- virtual [~ASTVisitor](#) ()=default
- virtual void [visit](#) ([ProgramNode](#) &node)=0
- virtual void [visit](#) ([SectionNode](#) &node)=0
- virtual void [visit](#) ([InstructionNode](#) &node)=0
- virtual void [visit](#) ([LabelNode](#) &node)=0
- virtual void [visit](#) ([VariableNode](#) &node)=0
- virtual void [visit](#) ([ExpressionNode](#) &node)=0
- virtual void [visit](#) ([CommentNode](#) &node)=0
- virtual void [visit](#) ([ModuleNode](#) &node)=0
- virtual void [visit](#) ([ObjectNode](#) &node)=0

11.17.1 Detailed Description

Visitor interface for traversing the AST node hierarchy.
Definition at line 166 of file [ast.hpp](#).

11.17.2 Constructor & Destructor Documentation

11.17.2.1 ~ASTVisitor()

```
virtual mxvm::ASTVisitor::~ASTVisitor () [virtual], [default]
```

11.17.3 Member Function Documentation

11.17.3.1 visit() [1/9]

```
virtual void mxvm::ASTVisitor::visit (
    CommentNode & node) [pure virtual]
```

11.17.3.2 visit() [2/9]

```
virtual void mxvm::ASTVisitor::visit (
    ExpressionNode & node) [pure virtual]
```

11.17.3.3 visit() [3/9]

```
virtual void mxvm::ASTVisitor::visit (
    InstructionNode & node) [pure virtual]
```

11.17.3.4 visit() [4/9]

```
virtual void mxvm::ASTVisitor::visit (
    LabelNode & node) [pure virtual]
```

11.17.3.5 visit() [5/9]

```
virtual void mxvm::ASTVisitor::visit (
    ModuleNode & node) [pure virtual]
```

11.17.3.6 visit() [6/9]

```
virtual void mxvm::ASTVisitor::visit (
    ObjectNode & node) [pure virtual]
```

11.17.3.7 visit() [7/9]

```
virtual void mxvm::ASTVisitor::visit (
    ProgramNode & node) [pure virtual]
```

Referenced by [mxvm::CommentNode::accept\(\)](#), [mxvm::ExpressionNode::accept\(\)](#), [mxvm::InstructionNode::accept\(\)](#), [mxvm::LabelNode::accept\(\)](#), [mxvm::ModuleNode::accept\(\)](#), [mxvm::ObjectNode::accept\(\)](#), [mxvm::ProgramNode::accept\(\)](#), [mxvm::SectionNode::accept\(\)](#), and [mxvm::VariableNode::accept\(\)](#).

11.17.3.8 visit() [8/9]

```
virtual void mxvm::ASTVisitor::visit (
    SectionNode & node) [pure virtual]
```

11.17.3.9 visit() [9/9]

```
virtual void mxvm::ASTVisitor::visit (
    VariableNode & node) [pure virtual]
```

The documentation for this class was generated from the following file:

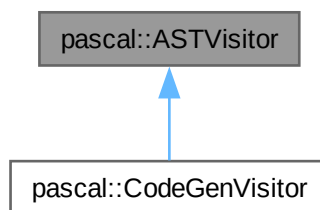
- include/mxvm/[ast.hpp](#)

11.18 pascal::ASTVisitor Class Reference

Abstract visitor interface for the AST (double-dispatch).

```
#include <ast.hpp>
```

Inheritance diagram for pascal::ASTVisitor:



Public Member Functions

- virtual [~ASTVisitor](#) ()=default
- virtual void [visit](#) ([ProgramNode](#) &node)=0
- virtual void [visit](#) ([UnitNode](#) &node)=0
- virtual void [visit](#) ([BlockNode](#) &node)=0
- virtual void [visit](#) ([VarDeclNode](#) &node)=0
- virtual void [visit](#) ([ConstDeclNode](#) &node)=0
- virtual void [visit](#) ([ProcDeclNode](#) &node)=0
- virtual void [visit](#) ([FuncDeclNode](#) &node)=0
- virtual void [visit](#) ([ParameterNode](#) &node)=0
- virtual void [visit](#) ([CompoundStmtNode](#) &node)=0
- virtual void [visit](#) ([AssignmentNode](#) &node)=0
- virtual void [visit](#) ([IfStmtNode](#) &node)=0
- virtual void [visit](#) ([WhileStmtNode](#) &node)=0
- virtual void [visit](#) ([ForStmtNode](#) &node)=0
- virtual void [visit](#) ([ProcCallNode](#) &node)=0
- virtual void [visit](#) ([BinaryOpNode](#) &node)=0
- virtual void [visit](#) ([UnaryOpNode](#) &node)=0
- virtual void [visit](#) ([FuncCallNode](#) &node)=0
- virtual void [visit](#) ([VariableNode](#) &node)=0
- virtual void [visit](#) ([NumberNode](#) &node)=0
- virtual void [visit](#) ([StringNode](#) &node)=0
- virtual void [visit](#) ([BooleanNode](#) &node)=0
- virtual void [visit](#) ([EmptyStmtNode](#) &node)=0
- virtual void [visit](#) ([RepeatStmtNode](#) &node)=0
- virtual void [visit](#) ([CaseStmtNode](#) &node)=0
- virtual void [visit](#) ([ArrayTypeNode](#) &node)=0

- virtual void [visit](#) ([ArrayDeclarationNode](#) &node)=0
- virtual void [visit](#) ([ArrayAccessNode](#) &node)=0
- virtual void [visit](#) ([ArrayAssignmentNode](#) &node)=0
- virtual void [visit](#) ([RecordTypeNode](#) &node)=0
- virtual void [visit](#) ([RecordDeclarationNode](#) &node)=0
- virtual void [visit](#) ([FieldAccessNode](#) &node)=0
- virtual void [visit](#) ([TypeDeclNode](#) &node)=0
- virtual void [visit](#) ([TypeAliasNode](#) &node)=0
- virtual void [visit](#) ([ExitNode](#) &node)=0
- virtual void [visit](#) ([SimpleTypeNode](#) &node)=0
- virtual void [visit](#) ([ArrayTypeDeclarationNode](#) &node)=0
- virtual void [visit](#) ([ContinueNode](#) &node)=0
- virtual void [visit](#) ([BreakNode](#) &node)=0
- virtual void [visit](#) ([NilNode](#) &node)=0
- virtual void [visit](#) ([PointerTypeNode](#) &node)=0
- virtual void [visit](#) ([PointerDerefNode](#) &node)=0
- virtual void [visit](#) ([AddressOfNode](#) &node)=0
- virtual void [visit](#) ([WithStmtNode](#) &node)=0
- virtual void [visit](#) ([GotoStmtNode](#) &node)=0
- virtual void [visit](#) ([LabelStmtNode](#) &node)=0
- virtual void [visit](#) ([SetLiteralNode](#) &node)=0
- virtual void [visit](#) ([SetTypeNode](#) &node)=0
- virtual void [visit](#) ([EnumTypeDeclNode](#) &node)=0

11.18.1 Detailed Description

Abstract visitor interface for the AST (double-dispatch).

Derived visitors implement [visit\(\)](#) overloads for every concrete AST node type to perform code generation, validation, or printing.

Definition at line 741 of file [ast.hpp](#).

11.18.2 Constructor & Destructor Documentation

11.18.2.1 ~ASTVisitor()

```
virtual pascal::ASTVisitor::~ASTVisitor () [virtual], [default]
```

11.18.3 Member Function Documentation

11.18.3.1 visit() [1/48]

```
virtual void pascal::ASTVisitor::visit (
    AddressOfNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.2 visit() [2/48]

```
virtual void pascal::ASTVisitor::visit (
    ArrayAccessNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.3 visit() [3/48]

```
virtual void pascal::ASTVisitor::visit (
    ArrayAssignmentNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.4 visit() [4/48]

```
virtual void pascal::ASTVisitor::visit (  
    ArrayDeclarationNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.5 visit() [5/48]

```
virtual void pascal::ASTVisitor::visit (  
    ArrayTypeDeclarationNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.6 visit() [6/48]

```
virtual void pascal::ASTVisitor::visit (  
    ArrayTypeNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.7 visit() [7/48]

```
virtual void pascal::ASTVisitor::visit (  
    AssignmentNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.8 visit() [8/48]

```
virtual void pascal::ASTVisitor::visit (  
    BinaryOpNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.9 visit() [9/48]

```
virtual void pascal::ASTVisitor::visit (  
    BlockNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.10 visit() [10/48]

```
virtual void pascal::ASTVisitor::visit (  
    BooleanNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.11 visit() [11/48]

```
virtual void pascal::ASTVisitor::visit (  
    BreakNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.12 visit() [12/48]

```
virtual void pascal::ASTVisitor::visit (  
    CaseStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.13 visit() [13/48]

```
virtual void pascal::ASTVisitor::visit (  
    CompoundStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.14 visit() [14/48]

```
virtual void pascal::ASTVisitor::visit (  
    ConstDeclNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.15 visit() [15/48]

```
virtual void pascal::ASTVisitor::visit (  
    ContinueNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.16 visit() [16/48]

```
virtual void pascal::ASTVisitor::visit (  
    EmptyStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.17 visit() [17/48]

```
virtual void pascal::ASTVisitor::visit (  
    EnumTypeDeclNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.18 visit() [18/48]

```
virtual void pascal::ASTVisitor::visit (  
    ExitNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.19 visit() [19/48]

```
virtual void pascal::ASTVisitor::visit (  
    FieldAccessNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.20 visit() [20/48]

```
virtual void pascal::ASTVisitor::visit (  
    ForStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.21 visit() [21/48]

```
virtual void pascal::ASTVisitor::visit (  
    FuncCallNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.22 visit() [22/48]

```
virtual void pascal::ASTVisitor::visit (  
    FuncDeclNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.23 visit() [23/48]

```
virtual void pascal::ASTVisitor::visit (  
    GotoStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.24 visit() [24/48]

```
virtual void pascal::ASTVisitor::visit (  
    IfStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.25 visit() [25/48]

```
virtual void pascal::ASTVisitor::visit (  
    LabelStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.26 visit() [26/48]

```
virtual void pascal::ASTVisitor::visit (  
    NilNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.27 visit() [27/48]

```
virtual void pascal::ASTVisitor::visit (  
    NumberNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.28 visit() [28/48]

```
virtual void pascal::ASTVisitor::visit (  
    ParameterNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.29 visit() [29/48]

```
virtual void pascal::ASTVisitor::visit (  
    PointerDerefNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.30 visit() [30/48]

```
virtual void pascal::ASTVisitor::visit (  
    PointerTypeNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.31 visit() [31/48]

```
virtual void pascal::ASTVisitor::visit (  
    ProcCallNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.32 visit() [32/48]

```
virtual void pascal::ASTVisitor::visit (  
    ProcDeclNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.33 visit() [33/48]

```
virtual void pascal::ASTVisitor::visit (  
    ProgramNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

Referenced by [pascal::AddressOfNode::accept\(\)](#), [pascal::ArrayAccessNode::accept\(\)](#), [pascal::ArrayAssignmentNode::accept\(\)](#), [pascal::ArrayDeclarationNode::accept\(\)](#), [pascal::ArrayTypeDeclarationNode::accept\(\)](#), [pascal::ArrayTypeNode::accept\(\)](#), [pascal::AssignmentNode::accept\(\)](#), [pascal::BinaryOpNode::accept\(\)](#), [pascal::BlockNode::accept\(\)](#), [pascal::BooleanNode::accept\(\)](#), [pascal::BreakNode::accept\(\)](#), [pascal::CaseStmtNode::accept\(\)](#), [pascal::CompoundStmtNode::accept\(\)](#), [pascal::ConstDeclNode::accept\(\)](#), [pascal::ContinueNode::accept\(\)](#), [pascal::EmptyStmtNode::accept\(\)](#), [pascal::EnumTypeDeclNode::accept\(\)](#), [pascal::ExitNode::accept\(\)](#), [pascal::FieldAccessNode::accept\(\)](#), [pascal::ForStmtNode::accept\(\)](#), [pascal::FuncCallNode::accept\(\)](#), [pascal::FuncDeclNode::accept\(\)](#), [pascal::GotoStmtNode::accept\(\)](#), [pascal::IfStmtNode::accept\(\)](#), [pascal::LabelStmtNode::accept\(\)](#), [pascal::NilNode::accept\(\)](#), [pascal::NumberNode::accept\(\)](#), [pascal::ParameterNode::accept\(\)](#), [pascal::PointerDerefNode::accept\(\)](#), [pascal::PointerTypeNode::accept\(\)](#), [pascal::ProcCallNode::accept\(\)](#), [pascal::ProcDeclNode::accept\(\)](#), [pascal::ProgramNode::accept\(\)](#), [pascal::RecordDeclarationNode::accept\(\)](#), [pascal::RecordTypeNode::accept\(\)](#), [pascal::RepeatStmtNode::accept\(\)](#), [pascal::SetLiteralNode::accept\(\)](#), [pascal::SetTypeNode::accept\(\)](#), [pascal::SimpleTypeNode::accept\(\)](#), [pascal::StringNode::accept\(\)](#), [pascal::TypeAliasNode::accept\(\)](#), [pascal::TypeDeclNode::accept\(\)](#), [pascal::UnaryOpNode::accept\(\)](#), [pascal::UnitNode::accept\(\)](#), [pascal::VarDeclNode::accept\(\)](#), [pascal::VariableNode::accept\(\)](#), [pascal::WhileStmtNode::accept\(\)](#), and [pascal::WithStmtNode::accept\(\)](#).

11.18.3.34 visit() [34/48]

```
virtual void pascal::ASTVisitor::visit (
    RecordDeclarationNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.35 visit() [35/48]

```
virtual void pascal::ASTVisitor::visit (
    RecordTypeNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.36 visit() [36/48]

```
virtual void pascal::ASTVisitor::visit (
    RepeatStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.37 visit() [37/48]

```
virtual void pascal::ASTVisitor::visit (
    SetLiteralNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.38 visit() [38/48]

```
virtual void pascal::ASTVisitor::visit (
    SetTypeNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.39 visit() [39/48]

```
virtual void pascal::ASTVisitor::visit (
    SimpleTypeNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.40 visit() [40/48]

```
virtual void pascal::ASTVisitor::visit (
    StringNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.41 visit() [41/48]

```
virtual void pascal::ASTVisitor::visit (
    TypeAliasNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.42 visit() [42/48]

```
virtual void pascal::ASTVisitor::visit (
    TypeDeclNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.43 visit() [43/48]

```
virtual void pascal::ASTVisitor::visit (
    UnaryOpNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.44 visit() [44/48]

```
virtual void pascal::ASTVisitor::visit (
    UnitNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.45 visit() [45/48]

```
virtual void pascal::ASTVisitor::visit (
    VarDeclNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.46 visit() [46/48]

```
virtual void pascal::ASTVisitor::visit (
    VariableNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.47 visit() [47/48]

```
virtual void pascal::ASTVisitor::visit (
    WhileStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

11.18.3.48 visit() [48/48]

```
virtual void pascal::ASTVisitor::visit (
    WithStmtNode & node) [pure virtual]
```

Implemented in [pascal::CodeGenVisitor](#).

The documentation for this class was generated from the following file:

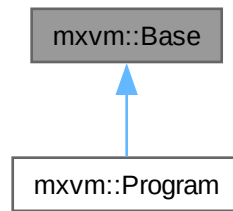
- [src/frontend/include/ast.hpp](#)

11.19 mxvm::Base Class Reference

[Base](#) class for instruction storage, variables, labels, and external function registrations.

```
#include <icode.hpp>
```

Inheritance diagram for mxvm::Base:



Public Member Functions

- `Base()`=default
- `Base(Base &&other)` noexcept
Move constructor.
- `~Base()`=default
- `Base & operator= (Base &&other)` noexcept
Move-assignment operator.
- void `setMainBase (Base *b)`
Set the root program base pointer (used for global variable lookup).
- void `add_instruction` (const `Instruction` &i)
Append an instruction to the instruction stream.
- void `add_label` (const std::string &name, uint64_t address, bool f)
Register a label at the given instruction address.
- void `add_variable` (const std::string &name, const `Variable` &v)
Declare a local variable in the symbol table.
- void `add_global` (const std::string &objname, const std::string &name, const `Variable` &v)
Declare a global variable scoped to an object.
- void `add_extern` (const std::string &mod, const std::string &name, bool module)
Register an external function import.
- void `add_runtime_extern` (const std::string &mod_name, const std::string &mod, const std::string &func_name, const std::string &name)
Register and resolve a runtime-loaded external function.
- void `add_filename` (const std::string &fname)
Record a source filename for debug/error reporting.
- void `add_allocated` (const std::string &name, `Variable` &v)
Track a heap-allocated variable for cleanup.
- void `add_object` (const std::string &name, `Program` *prog)
Register a child object program.

Public Attributes

- std::string `name`
- std::vector< `Instruction` > `inc`
instruction stream
- std::unordered_map< std::string, `Variable` > `vars`
variable symbol table

- `std::unordered_map< std::string, std::pair< uint64_t, bool > > labels`
label -> (address, is_function)
- `std::vector< ExternalFunction > external`
declared external function imports
- `std::unordered_map< std::string, RuntimeFunction > external_functions`
resolved runtime function bindings
- `std::string assembly_code`
generated assembly output for objects

Static Public Attributes

- static `Base * base = nullptr`
pointer to the main program base
- static `std::string root_name`
- static `std::vector< std::string > filenames`
- static `std::unordered_map< std::string, Variable > allocated`
heap-allocated variables
- static `std::unordered_map< std::string, Program * > object_map`
object name -> Program mapping

11.19.1 Detailed Description

`Base` class for instruction storage, variables, labels, and external function registrations.

Provides the shared data structures used by both interpreted and compiled programs: instruction list, variable table, label map, external function bindings, and object references.

Definition at line 121 of file `icode.hpp`.

11.19.2 Constructor & Destructor Documentation

11.19.2.1 `Base()` [1/2]

```
mxvm::Base::Base () [default]
```

References `Base()`.

Referenced by `Base()`, `Base()`, `~Base()`, `operator=()`, and `setMainBase()`.

11.19.2.2 `Base()` [2/2]

```
mxvm::Base::Base (
    Base && other) [noexcept]
```

Move constructor.

Definition at line 72 of file `icode.cpp`.

References `Base()`, `external`, `external_functions`, `inc`, `labels`, `name`, and `vars`.

11.19.2.3 `~Base()`

```
mxvm::Base::~Base () [default]
```

References `Base()`.

11.19.3 Member Function Documentation

11.19.3.1 `add_allocated()`

```
void mxvm::Base::add_allocated (
    const std::string & name,
    Variable & v)
```

Track a heap-allocated variable for cleanup.

Parameters

<i>name</i>	Variable name
<i>v</i>	Variable (with allocation metadata)

Definition at line 90 of file [icode.cpp](#).

References [allocated](#), and [name](#).

11.19.3.2 add_extern()

```
void mxvm::Base::add_extern (
    const std::string & mod,
    const std::string & name,
    bool module)
```

Register an external function import.

Parameters

<i>mod</i>	Module name
<i>name</i>	Function symbol name
<i>module</i>	true if the import is a module-level extern

Definition at line 266 of file [icode.cpp](#).

References [external](#), and [name](#).

Referenced by [mxvm::Program::Program\(\)](#).

11.19.3.3 add_filename()

```
void mxvm::Base::add_filename (
    const std::string & fname)
```

Record a source filename for debug/error reporting.

Parameters

<i>fname</i>	Filename to add
--------------	-----------------

Definition at line 257 of file [icode.cpp](#).

References [base](#).

11.19.3.4 add_global()

```
void mxvm::Base::add_global (
    const std::string & objname,
    const std::string & name,
    const Variable & v)
```

Declare a global variable scoped to an object.

Parameters

<i>objname</i>	Object that owns the variable
<i>name</i>	Variable name
<i>v</i>	Variable definition

Definition at line 254 of file [icode.cpp](#).

References [name](#).

11.19.3.5 add_instruction()

```
void mxvm::Base::add_instruction (
    const Instruction & i)
```

Append an instruction to the instruction stream.

Parameters

<i>i</i>	Instruction to add
----------	------------------------------------

Definition at line 237 of file [icode.cpp](#).

References [inc](#).

Referenced by [mxvm::Program::flatten_inc\(\)](#).

11.19.3.6 add_label()

```
void mxvm::Base::add_label (
    const std::string & name,
    uint64_t address,
    bool f)
```

Register a label at the given instruction address.

Parameters

<i>name</i>	Label identifier
<i>address</i>	Instruction index the label points to
<i>f</i>	true if the label marks a function entry

Definition at line 241 of file [icode.cpp](#).

References [labels](#), and [name](#).

Referenced by [mxvm::Program::flatten_label\(\)](#).

11.19.3.7 add_object()

```
void mxvm::Base::add_object (
    const std::string & name,
    Program * prog)
```

Register a child object program.

Parameters

<i>name</i>	Object identifier
<i>prog</i>	Pointer to the object's Program

Definition at line 136 of file [icode.cpp](#).

References [base](#), and [name](#).

11.19.3.8 add_runtime_extern()

```
void mxvm::Base::add_runtime_extern (
    const std::string & mod_name,
    const std::string & mod,
    const std::string & func_name,
    const std::string & name)
```

Register and resolve a runtime-loaded external function.

Parameters

<i>mod_name</i>	Module display name
<i>mod</i>	Shared library path
<i>func_name</i>	Native symbol name
<i>name</i>	MXVM-visible function name

Definition at line 273 of file [icode.cpp](#).

References [base](#), and [name](#).

11.19.3.9 add_variable()

```
void mxvm::Base::add_variable (
    const std::string & name,
    const Variable & v)
```

Declare a local variable in the symbol table.

Parameters

<i>name</i>	Variable name
<i>v</i>	Variable definition

Definition at line 246 of file [icode.cpp](#).

References [name](#), and [vars](#).

Referenced by [mxvm::Program::add_standard\(\)](#).

11.19.3.10 operator=()

```
Base & mxvm::Base::operator= (
    Base && other) [noexcept]
```

Move-assignment operator.

Definition at line 76 of file [icode.cpp](#).

References [Base\(\)](#), [external](#), [external_functions](#), [inc](#), [labels](#), [name](#), and [vars](#).

Referenced by [mxvm::Program::operator=\(\)](#).

11.19.3.11 setMainBase()

```
void mxvm::Base::setMainBase (
    Base * b) [inline]
```

Set the root program base pointer (used for global variable lookup).

Parameters

<i>b</i>	Pointer to the main program's Base
----------	--

Definition at line 133 of file [icode.hpp](#).

References [Base\(\)](#), and [base](#).

11.19.4 Member Data Documentation**11.19.4.1 allocated**

```
std::unordered_map< std::string, Variable > mxvm::Base::allocated [static]
```

heap-allocated variables

Definition at line 200 of file [icode.hpp](#).

Referenced by [add_allocated\(\)](#).

11.19.4.2 assembly_code

`std::string mxvm::Base::assembly_code`
 generated assembly output for objects
 Definition at line 202 of file [icode.hpp](#).

11.19.4.3 base

`Base * mxvm::Base::base = nullptr [static]`

pointer to the main program base

Definition at line 197 of file [icode.hpp](#).

Referenced by [action_interpret\(\)](#), [add_filename\(\)](#), [add_runtime_extern\(\)](#), [mxvm::Program::addVariables\(\)](#), [collectAndRegisterAllExterns\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_store\(\)](#), [mxvm::Program::generateCode\(\)](#), [mxvm::Parser::generateProgramCode\(\)](#), [mxvm::Program::getMangledName\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [process_arguments\(\)](#), [mxvm::Parser::processObjectFile\(\)](#), [setMainBase\(\)](#), [mxvm::Program::subVariable\(\)](#), [translate_x64\(\)](#), and [mxvm::Program::x64_generateCode\(\)](#).

11.19.4.4 external

`std::vector<ExternalFunction> mxvm::Base::external`

declared external function imports

Definition at line 195 of file [icode.hpp](#).

Referenced by [Base\(\)](#), [add_extern\(\)](#), [mxvm::Program::gen_done\(\)](#), [mxvm::Program::gen_exit\(\)](#), [mxvm::Program::generateCode\(\)](#), [operator=\(\)](#), [mxvm::Program::x64_gen_done\(\)](#), [mxvm::Program::x64_gen_exit\(\)](#), and [mxvm::Program::x64_generateCode\(\)](#).

11.19.4.5 external_functions

`std::unordered_map<std::string, RuntimeFunction> mxvm::Base::external_functions`

resolved runtime function bindings

Definition at line 196 of file [icode.hpp](#).

Referenced by [Base\(\)](#), [mxvm::Program::exec_invoke\(\)](#), [mxvm::Program::flatten\(\)](#), [mxvm::Program::flatten_external\(\)](#), and [operator=\(\)](#).

11.19.4.6 filenames

`std::vector< std::string > mxvm::Base::filenames [static]`

Definition at line 199 of file [icode.hpp](#).

11.19.4.7 inc

`std::vector<Instruction> mxvm::Base::inc`

instruction stream

Definition at line 192 of file [icode.hpp](#).

Referenced by [Base\(\)](#), [add_instruction\(\)](#), [mxvm::Program::exec\(\)](#), [mxvm::Program::flatten\(\)](#), [mxvm::Program::generateCode\(\)](#), [operator=\(\)](#), [mxvm::Program::print\(\)](#), [mxvm::Program::sysv_analyzeRegAlloc\(\)](#), [mxvm::Program::x64_analyzeRegAlloc\(\)](#), and [mxvm::Program::x64_generateCode\(\)](#).

11.19.4.8 labels

`std::unordered_map<std::string, std::pair<uint64_t, bool> > mxvm::Base::labels`

label -> (address, is_function)

Definition at line 194 of file [icode.hpp](#).

Referenced by [Base\(\)](#), [add_label\(\)](#), [mxvm::Program::exec_call\(\)](#), [mxvm::Program::exec_jump\(\)](#), [mxvm::Program::flatten\(\)](#), [mxvm::Program::flatten_inc\(\)](#), [mxvm::Program::gen_jump\(\)](#), [mxvm::Program::generateCode\(\)](#), [mxvm::Program::getMangledName\(\)](#), [mxvm::Program::isFunctionValid\(\)](#), [mxvm::Program::memoryDump\(\)](#), [operator=\(\)](#), [mxvm::Program::x64_gen_jump\(\)](#), and [mxvm::Program::x64_generateCode\(\)](#).

11.19.4.9 name

std::string mxvm::Base::name

Definition at line 191 of file [icode.hpp](#).

Referenced by [Base\(\)](#), [add_allocated\(\)](#), [add_extern\(\)](#), [add_global\(\)](#), [add_label\(\)](#), [add_object\(\)](#), [add_runtime_extern\(\)](#), [add_variable\(\)](#), and [operator=\(\)](#).

11.19.4.10 object_map

std::unordered_map< std::string, [Program](#) * > mxvm::Base::object_map [static]

object name -> [Program](#) mapping

Definition at line 201 of file [icode.hpp](#).

Referenced by [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isFunctionValid\(\)](#), and [mxvm::Program::isVariable\(\)](#).

11.19.4.11 root_name

std::string mxvm::Base::root_name [static]

Definition at line 198 of file [icode.hpp](#).

Referenced by [mxvm::Program::generateCode\(\)](#), and [mxvm::Program::x64_generateCode\(\)](#).

11.19.4.12 vars

std::unordered_map<std::string, [Variable](#)> mxvm::Base::vars

variable symbol table

Definition at line 193 of file [icode.hpp](#).

Referenced by [Base\(\)](#), [mxvm::Program::Program\(\)](#), [mxvm::Program::~~Program\(\)](#), [add_variable\(\)](#), [mxvm::Program::exec_free\(\)](#), [mxvm::Program::exec_mov\(\)](#), [mxvm::Program::flatten_inc\(\)](#), [mxvm::Program::generateCode\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Program::memoryDump\(\)](#), [mxvm_io_fclose\(\)](#), [mxvm_io_feof\(\)](#), [mxvm_io_fopen\(\)](#), [mxvm_io_fputs\(\)](#), [mxvm_io_fread\(\)](#), [mxvm_io_fseek\(\)](#), [mxvm_io_fsize\(\)](#), [mxvm_io_fwrite\(\)](#), [mxvm_io_mxvm_fgets\(\)](#), [mxvm_io_rand_number\(\)](#), [mxvm_io_seed_random\(\)](#), [mxvm_sdl_blit_surface\(\)](#), [mxvm_sdl_create_render_target\(\)](#), [mxvm_sdl_create_renderer\(\)](#), [mxvm_sdl_create_rgb_surface\(\)](#), [mxvm_sdl_create_texture\(\)](#), [mxvm_sdl_create_window\(\)](#), [mxvm_sdl_get_clipboard_text\(\)](#), [mxvm_sdl_get_event_type\(\)](#), [mxvm_sdl_get_key_code\(\)](#), [mxvm_sdl_get_keyboard_state\(\)](#), [mxvm_sdl_get_mouse_button\(\)](#), [mxvm_sdl_get_mouse_buttons\(\)](#), [mxvm_sdl_get_mouse_state\(\)](#), [mxvm_sdl_get_mouse_x\(\)](#), [mxvm_sdl_get_mouse_y\(\)](#), [mxvm_sdl_get_num_keys\(\)](#), [mxvm_sdl_get_queued_audio_size\(\)](#), [mxvm_sdl_get_relative_mouse_button\(\)](#), [mxvm_sdl_get_relative_mouse_state\(\)](#), [mxvm_sdl_get_relative_mouse_x\(\)](#), [mxvm_sdl_get_relative_mouse_y\(\)](#), [mxvm_sdl_get_ticks\(\)](#), [mxvm_sdl_init\(\)](#), [mxvm_sdl_init_text\(\)](#), [mxvm_sdl_is_key_pressed\(\)](#), [mxvm_sdl_load_font\(\)](#), [mxvm_sdl_load_texture\(\)](#), [mxvm_sdl_load_texture_color_key\(\)](#), [mxvm_sdl_load_texture_color_key_rgb\(\)](#), [mxvm_sdl_load_wav\(\)](#), [mxvm_sdl_lock_texture\(\)](#), [mxvm_sdl_open_audio\(\)](#), [mxvm_sdl_poll_event\(\)](#), [mxvm_sdl_queue_audio\(\)](#), [mxvm_sdl_update_texture\(\)](#), [mxvm_std_abs\(\)](#), [mxvm_std_acos\(\)](#), [mxvm_std_argc\(\)](#), [mxvm_std_argv\(\)](#), [mxvm_std_asin\(\)](#), [mxvm_std_atan\(\)](#), [mxvm_std_atan2\(\)](#), [mxvm_std_atof\(\)](#), [mxvm_std_atoi\(\)](#), [mxvm_std_calloc\(\)](#), [mxvm_std_ceil\(\)](#), [mxvm_std_cos\(\)](#), [mxvm_std_cosh\(\)](#), [mxvm_std_exp\(\)](#), [mxvm_std_exp2\(\)](#), [mxvm_std_fabs\(\)](#), [mxvm_std_float_to_int\(\)](#), [mxvm_std_floor\(\)](#), [mxvm_std_fmod\(\)](#), [mxvm_std_hypot\(\)](#), [mxvm_std_int_to_float\(\)](#), [mxvm_std_isalpha\(\)](#), [mxvm_std_isdigit\(\)](#), [mxvm_std_isspace\(\)](#), [mxvm_std_log\(\)](#), [mxvm_std_log10\(\)](#), [mxvm_std_log2\(\)](#), [mxvm_std_malloc\(\)](#), [mxvm_std_memcmp\(\)](#), [mxvm_std_memcpy\(\)](#), [mxvm_std_memmove\(\)](#), [mxvm_std_memset\(\)](#), [mxvm_std_pow\(\)](#), [mxvm_std_rand\(\)](#), [mxvm_std_round\(\)](#), [mxvm_std_sin\(\)](#), [mxvm_std_sinh\(\)](#), [mxvm_std_sqrt\(\)](#), [mxvm_std_system\(\)](#), [mxvm_std_tan\(\)](#), [mxvm_std_tanh\(\)](#), [mxvm_std_tolower\(\)](#), [mxvm_std_toupper\(\)](#), [mxvm_std_trunc\(\)](#), [mxvm_string_copy\(\)](#), [mxvm_string_delete\(\)](#), [mxvm_string_insert\(\)](#), [mxvm_string_inttostr\(\)](#), [mxvm_string_pos\(\)](#), [mxvm_string_sprintf\(\)](#), [mxvm_string_strat\(\)](#), [mxvm_string_strcmp\(\)](#), [mxvm_string_strfind\(\)](#), [mxvm_string_strlen\(\)](#), [mxvm_string_strncat\(\)](#), [mxvm_string_strncpy\(\)](#), [mxvm_string_strtoint\(\)](#), [mxvm_string_substr\(\)](#), [operator=\(\)](#), [mxvm::Program::operator=\(\)](#), [mxvm::Program::validateNames\(\)](#), and [mxvm::Program::x64_generateCode\(\)](#).

The documentation for this class was generated from the following files:

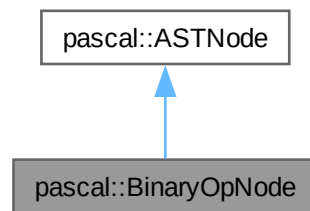
- [include/mxvm/icode.hpp](#)
- [src/icode.cpp](#)

11.20 pascal::BinaryOpNode Class Reference

AST node for a binary operator expression.

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::BinaryOpNode`:



Public Types

- enum `OpType` {
`PLUS` , `MINUS` , `MULTIPLY` , `DIVIDE` ,
`DIV` , `MOD` , `EQUAL` , `NOT_EQUAL` ,
`LESS` , `LESS_EQUAL` , `GREATER` , `GREATER_EQUAL` ,
`AND` , `OR` , `IN` }

Binary operator kinds.

Public Member Functions

- `BinaryOpNode` (`std::unique_ptr< ASTNode > l`, `OpType op`, `std::unique_ptr< ASTNode > r`)
- `BinaryOpNode` (`OpType op`, `std::unique_ptr< ASTNode > l`, `std::unique_ptr< ASTNode > r`)
- void `accept` (`ASTVisitor &visitor`) override
Accept an `ASTVisitor` (double-dispatch).
- `std::string toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from `pascal::ASTNode`

- virtual `~ASTNode` ()=default
- virtual void `print` (`std::ostream &out`, `int indent=0`) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (`int line`)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Static Public Member Functions

- static `std::string opToString` (`OpType op`)
Convert an `OpType` to its string representation.

Public Attributes

- `std::unique_ptr< ASTNode > left`
left operand
- `OpType operator_`
operator kind

- `std::unique_ptr< ASTNode > right`
right operand

11.20.1 Detailed Description

AST node for a binary operator expression.
Definition at line [438](#) of file [ast.hpp](#).

11.20.2 Member Enumeration Documentation

11.20.2.1 OpType

enum [pascal::BinaryOpNode::OpType](#)
Binary operator kinds.

Enumerator

PLUS	
MINUS	
MULTIPLY	
DIVIDE	
DIV	
MOD	
EQUAL	
NOT_EQUAL	
LESS	
LESS_EQUAL	
GREATER	
GREATER_EQUAL	
AND	
OR	
IN	

Definition at line [441](#) of file [ast.hpp](#).

11.20.3 Constructor & Destructor Documentation

11.20.3.1 BinaryOpNode() [1/2]

```
pascal::BinaryOpNode::BinaryOpNode (
    std::unique_ptr< ASTNode > l,
    OpType op,
    std::unique_ptr< ASTNode > r) [inline]
```

Definition at line [463](#) of file [ast.hpp](#).
References [left](#), [operator_](#), and [right](#).

11.20.3.2 BinaryOpNode() [2/2]

```
pascal::BinaryOpNode::BinaryOpNode (
    OpType op,
    std::unique_ptr< ASTNode > l,
    std::unique_ptr< ASTNode > r) [inline]
```

Definition at line [466](#) of file [ast.hpp](#).
References [left](#), [operator_](#), and [right](#).

11.20.4 Member Function Documentation

11.20.4.1 `accept()`

```
void pascal::BinaryOpNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 165 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.20.4.2 `opToString()`

```
std::string pascal::BinaryOpNode::opToString (
    OpType op) [static]
```

Convert an [OpType](#) to its string representation.

Definition at line 173 of file [ast.cpp](#).

References [AND](#), [DIV](#), [DIVIDE](#), [EQUAL](#), [GREATER](#), [GREATER_EQUAL](#), [IN](#), [LESS](#), [LESS_EQUAL](#), [MINUS](#), [MOD](#), [MULTIPLY](#), [NOT_EQUAL](#), [OR](#), and [PLUS](#).

Referenced by [toString\(\)](#).

11.20.4.3 `toString()`

```
std::string pascal::BinaryOpNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 169 of file [ast.cpp](#).

References [operator_](#), and [opToString\(\)](#).

11.20.5 Member Data Documentation

11.20.5.1 `left`

```
std::unique_ptr<ASTNode> pascal::BinaryOpNode::left
```

left operand

Definition at line 459 of file [ast.hpp](#).

Referenced by [BinaryOpNode\(\)](#), [BinaryOpNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.20.5.2 `operator_`

```
OpType pascal::BinaryOpNode::operator_
```

operator kind

Definition at line 460 of file [ast.hpp](#).

Referenced by [BinaryOpNode\(\)](#), [BinaryOpNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.20.5.3 `right`

```
std::unique_ptr<ASTNode> pascal::BinaryOpNode::right
```

right operand

Definition at line 461 of file [ast.hpp](#).

Referenced by [BinaryOpNode\(\)](#), [BinaryOpNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

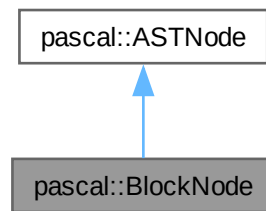
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.21 `pascal::BlockNode` Class Reference

AST node representing a block (declarations + compound statement).

```
#include <ast.hpp>
```


Inheritance diagram for pascal::BlockNode:



Public Member Functions

- `BlockNode` (`std::vector< std::unique_ptr< ASTNode > > decls, std::unique_ptr< CompoundStmtNode > stmt`)
Construct a block node.
- `void accept (ASTVisitor &visitor)` override
Accept an ASTVisitor (double-dispatch).
- `std::string toString ()` const override
Return a human-readable description of this node.

Public Member Functions inherited from pascal::ASTNode

- `virtual ~ASTNode ()`=default
- `virtual void print (std::ostream &out, int indent=0)` const
Pretty-print this node to a stream with indentation.
- `void setLineNumber (int line)`
Set the originating source line number.
- `int getLineNumber ()` const
Get the originating source line number.

Public Attributes

- `std::vector< std::unique_ptr< ASTNode > > declarations`
variable / type / const / procedure / function declarations
- `std::unique_ptr< CompoundStmtNode > compoundStatement`
the begin..end compound statement

11.21.1 Detailed Description

AST node representing a block (declarations + compound statement).
Definition at line 79 of file [ast.hpp](#).

11.21.2 Constructor & Destructor Documentation

11.21.2.1 BlockNode()

```

pascal::BlockNode::BlockNode (
    std::vector< std::unique_ptr< ASTNode > > decls,
    std::unique_ptr< CompoundStmtNode > stmt)

```

Construct a block node.

Parameters

<i>decls</i>	List of declaration nodes
<i>stmt</i>	The compound statement body

Definition at line 11 of file [ast.cpp](#).

References [compoundStatement](#), and [declarations](#).

11.21.3 Member Function Documentation**11.21.3.1 accept()**

```
void pascal::BlockNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 39 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.21.3.2 toString()

```
std::string pascal::BlockNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 43 of file [ast.cpp](#).

References [declarations](#).

11.21.4 Member Data Documentation**11.21.4.1 compoundStatement**

```
std::unique_ptr<CompoundStmtNode> pascal::BlockNode::compoundStatement
```

the begin..end compound statement

Definition at line 82 of file [ast.hpp](#).

Referenced by [BlockNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.21.4.2 declarations

```
std::vector<std::unique_ptr<ASTNode> > pascal::BlockNode::declarations
variable / type / const / procedure / function declarations
```

Definition at line 81 of file [ast.hpp](#).

Referenced by [BlockNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

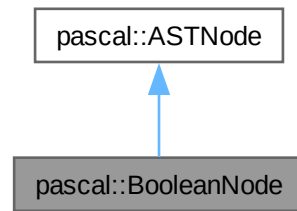
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.22 pascal::BooleanNode Class Reference

AST node for a boolean literal (true / false).

```
#include <ast.hpp>
```

Inheritance diagram for pascal::BooleanNode:



Public Member Functions

- [BooleanNode](#) (bool val)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- bool [value](#)
the boolean value

11.22.1 Detailed Description

AST node for a boolean literal (true / false).
Definition at line 549 of file [ast.hpp](#).

11.22.2 Constructor & Destructor Documentation

11.22.2.1 BooleanNode()

```
pascal::BooleanNode::BooleanNode (
    bool val) [inline]
```

Definition at line 553 of file [ast.hpp](#).

References [value](#).

11.22.3 Member Function Documentation

11.22.3.1 `accept()`

```
void pascal::BooleanNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 263 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.22.3.2 `toString()`

```
std::string pascal::BooleanNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 267 of file [ast.cpp](#).

References [value](#).

11.22.4 Member Data Documentation

11.22.4.1 `value`

```
bool pascal::BooleanNode::value
```

the boolean value

Definition at line 551 of file [ast.hpp](#).

Referenced by [BooleanNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

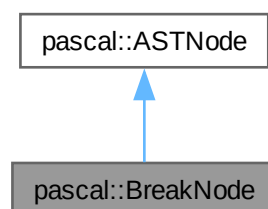
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.23 `pascal::BreakNode` Class Reference

AST node for a break statement.

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::BreakNode`:



Public Member Functions

- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

- [BreakNode](#) ()=default
- [BreakNode](#) (std::unique_ptr< [ASTNode](#) > e)

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::unique_ptr< [ASTNode](#) > [expr](#)
reserved expression (unused)

11.23.1 Detailed Description

AST node for a break statement.
Definition at line 65 of file [ast.hpp](#).

11.23.2 Constructor & Destructor Documentation

11.23.2.1 BreakNode() [1/2]

```
pascal::BreakNode::BreakNode () [default]
```

11.23.2.2 BreakNode() [2/2]

```
pascal::BreakNode::BreakNode (
    std::unique_ptr< ASTNode > e) [inline], [explicit]
```

Definition at line 71 of file [ast.hpp](#).
References [expr](#).

11.23.3 Member Function Documentation

11.23.3.1 accept()

```
void pascal::BreakNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).
Implements [pascal::ASTNode](#).
Definition at line 418 of file [ast.cpp](#).
References [pascal::ASTVisitor::visit\(\)](#).

11.23.3.2 toString()

```
std::string pascal::BreakNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.
Implements [pascal::ASTNode](#).
Definition at line 422 of file [ast.cpp](#).

11.23.4 Member Data Documentation

11.23.4.1 expr

`std::unique_ptr<ASTNode> pascal::BreakNode::expr`

reserved expression (unused)

Definition at line 69 of file [ast.hpp](#).

Referenced by [BreakNode\(\)](#).

The documentation for this class was generated from the following files:

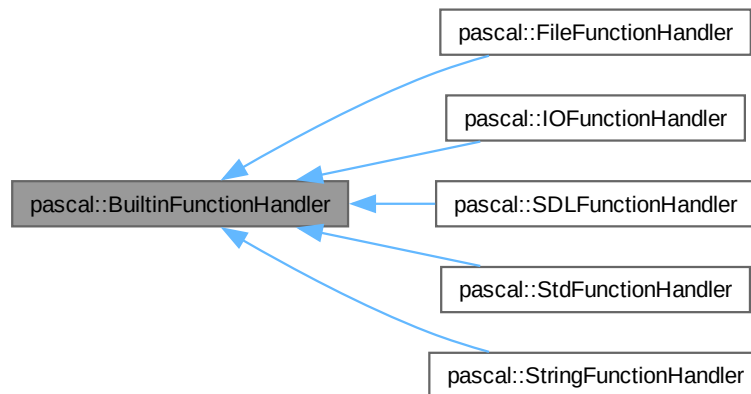
- `src/frontend/include/ast.hpp`
- `src/frontend/ast.cpp`

11.24 pascal::BuiltinFunctionHandler Class Reference

Abstract handler for a group of built-in functions/procedures.

`#include <icode.hpp>`

Inheritance diagram for `pascal::BuiltinFunctionHandler`:



Public Member Functions

- virtual `~BuiltinFunctionHandler()`=default
- virtual bool `canHandle` (const std::string &funcName) const =0
Return true if this handler implements funcName.
- virtual void `generate` (CodeGenVisitor &visitor, const std::string &funcName, const std::vector< std::unique_ptr< ASTNode > > &arguments)=0
Emit code for a procedure-style call (no return value used).
- virtual bool `generateWithResult` (CodeGenVisitor &visitor, const std::string &funcName, const std::vector< std::unique_ptr< ASTNode > > &arguments)
Emit code for a function-style call (result pushed on eval stack); return true if handled.
- virtual `VarType` `getReturnType` (const std::string &funcName) const
Return the VarType produced by funcName.

11.24.1 Detailed Description

Abstract handler for a group of built-in functions/procedures.

Derived classes ([IOFunctionHandler](#), [StdFunctionHandler](#), etc.) implement code generation for built-in calls such as `writeln`, `readln`, `chr`, `ord`, etc.

Definition at line 92 of file [icode.hpp](#).

11.24.2 Constructor & Destructor Documentation

11.24.2.1 ~BuiltinFunctionHandler()

```
virtual pascal::BuiltinFunctionHandler::~~BuiltinFunctionHandler () [virtual], [default]
```

11.24.3 Member Function Documentation

11.24.3.1 canHandle()

```
virtual bool pascal::BuiltinFunctionHandler::canHandle (
    const std::string & funcName) const [pure virtual]
```

Return true if this handler implements funcName.

Implemented in [pascal::FileFunctionHandler](#), [pascal::IOFunctionHandler](#), [pascal::SDLFunctionHandler](#), [pascal::StdFunctionHandler](#), and [pascal::StringFunctionHandler](#).

11.24.3.2 generate()

```
virtual void pascal::BuiltinFunctionHandler::generate (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [pure virtual]
```

Emit code for a procedure-style call (no return value used).

Implemented in [pascal::FileFunctionHandler](#), [pascal::IOFunctionHandler](#), [pascal::SDLFunctionHandler](#), [pascal::StdFunctionHandler](#), and [pascal::StringFunctionHandler](#).

11.24.3.3 generateWithResult()

```
virtual bool pascal::BuiltinFunctionHandler::generateWithResult (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [inline], [virtual]
```

Emit code for a function-style call (result pushed on eval stack); return true if handled.

Reimplemented in [pascal::FileFunctionHandler](#), [pascal::IOFunctionHandler](#), [pascal::SDLFunctionHandler](#), [pascal::StdFunctionHandler](#), and [pascal::StringFunctionHandler](#).

Definition at line 100 of file [icode.hpp](#).

11.24.3.4 getReturnType()

```
virtual VarType pascal::BuiltinFunctionHandler::getReturnType (
    const std::string & funcName) const [inline], [virtual]
```

Return the [VarType](#) produced by funcName.

Reimplemented in [pascal::FileFunctionHandler](#), and [pascal::StringFunctionHandler](#).

Definition at line 102 of file [icode.hpp](#).

References [pascal::UNKNOWN](#).

The documentation for this class was generated from the following file:

- [src/frontend/include/icode.hpp](#)

11.25 pascal::BuiltinFunctionRegistry Class Reference

Registry of [BuiltinFunctionHandler](#) instances.

```
#include <icode.hpp>
```

Public Member Functions

- void [registerHandler](#) (std::unique_ptr< [BuiltinFunctionHandler](#) > handler)
Register a new handler.
- [BuiltinFunctionHandler](#) * [findHandler](#) (const std::string & funcName)
Find the handler for a given function name.

Private Attributes

- `std::vector< std::unique_ptr< BuiltinFunctionHandler > > handlers`
registered handlers

11.25.1 Detailed Description

Registry of [BuiltinFunctionHandler](#) instances.

Holds all built-in function group handlers and dispatches to the first handler that can handle a given function name.

Definition at line 111 of file [icode.hpp](#).

11.25.2 Member Function Documentation

11.25.2.1 findHandler()

```
BuiltinFunctionHandler * pascal::BuiltinFunctionRegistry::findHandler (
    const std::string & funcName) [inline]
```

Find the handler for a given function name.

Returns

Pointer to the matching handler, or nullptr

Definition at line 122 of file [icode.hpp](#).

References [handlers](#).

11.25.2.2 registerHandler()

```
void pascal::BuiltinFunctionRegistry::registerHandler (
    std::unique_ptr< BuiltinFunctionHandler > handler) [inline]
```

Register a new handler.

Definition at line 117 of file [icode.hpp](#).

References [handlers](#).

11.25.3 Member Data Documentation

11.25.3.1 handlers

```
std::vector<std::unique_ptr<BuiltinFunctionHandler> > pascal::BuiltinFunctionRegistry::↵
handlers [private]
```

registered handlers

Definition at line 113 of file [icode.hpp](#).

Referenced by [findHandler\(\)](#), and [registerHandler\(\)](#).

The documentation for this class was generated from the following file:

- `src/frontend/include/icode.hpp`

11.26 pascal::CaseStmtNode::CaseBranch Struct Reference

A single case branch (one or more values mapping to a statement).

```
#include <ast.hpp>
```

Public Member Functions

- [CaseBranch](#) (`std::vector< std::unique_ptr< ASTNode > > vals`, `std::unique_ptr< ASTNode > stmt`)

Public Attributes

- `std::vector< std::unique_ptr< ASTNode > > values`
match values

- `std::unique_ptr< ASTNode > statement`
branch body

11.26.1 Detailed Description

A single case branch (one or more values mapping to a statement).
Definition at line 403 of file [ast.hpp](#).

11.26.2 Constructor & Destructor Documentation

11.26.2.1 CaseBranch()

```
pascal::CaseStmtNode::CaseBranch::CaseBranch (
    std::vector< std::unique_ptr< ASTNode > > vals,
    std::unique_ptr< ASTNode > stmt) [inline]
```

Definition at line 407 of file [ast.hpp](#).
References [statement](#), and [values](#).

11.26.3 Member Data Documentation

11.26.3.1 statement

```
std::unique_ptr<ASTNode> pascal::CaseStmtNode::CaseBranch::statement
```

branch body
Definition at line 405 of file [ast.hpp](#).
Referenced by [CaseBranch\(\)](#).

11.26.3.2 values

```
std::vector<std::unique_ptr<ASTNode> > pascal::CaseStmtNode::CaseBranch::values
```

match values
Definition at line 404 of file [ast.hpp](#).
Referenced by [CaseBranch\(\)](#).

The documentation for this struct was generated from the following file:

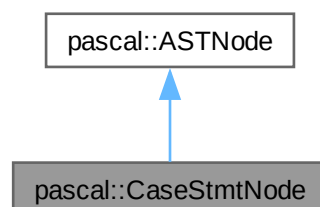
- `src/frontend/include/ast.hpp`

11.27 pascal::CaseStmtNode Class Reference

AST node for a case statement.

```
#include <ast.hpp>
```

Inheritance diagram for pascal::CaseStmtNode:



Classes

- struct [CaseBranch](#)
A single case branch (one or more values mapping to a statement).

Public Member Functions

- [CaseStmtNode](#) (std::unique_ptr< [ASTNode](#) > expr, std::vector< std::unique_ptr< [CaseBranch](#) > > branches, std::unique_ptr< [ASTNode](#) > elseStmt=nullptr)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::unique_ptr< [ASTNode](#) > [expression](#)
selector expression
- std::vector< std::unique_ptr< [CaseBranch](#) > > [branches](#)
case branches
- std::unique_ptr< [ASTNode](#) > [elseStatement](#)
optional else/otherwise branch

11.27.1 Detailed Description

AST node for a case statement.
Definition at line 400 of file [ast.hpp](#).

11.27.2 Constructor & Destructor Documentation

11.27.2.1 CaseStmtNode()

```
pascal::CaseStmtNode::CaseStmtNode (
    std::unique_ptr< ASTNode > expr,
    std::vector< std::unique_ptr< CaseBranch > > branches,
    std::unique_ptr< ASTNode > elseStmt = nullptr) [inline]
```

Definition at line 415 of file [ast.hpp](#).
References [branches](#), [elseStatement](#), and [expression](#).

11.27.3 Member Function Documentation

11.27.3.1 accept()

```
void pascal::CaseStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).
Implements [pascal::ASTNode](#).
Definition at line 293 of file [ast.cpp](#).
References [pascal::ASTVisitor::visit\(\)](#).

11.27.3.2 toString()

```
std::string pascal::CaseStmtNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 297 of file [ast.cpp](#).

References [branches](#).

11.27.4 Member Data Documentation

11.27.4.1 branches

```
std::vector<std::unique_ptr<CaseBranch> > pascal::CaseStmtNode::branches
```

case branches

Definition at line 412 of file [ast.hpp](#).

Referenced by [CaseStmtNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.27.4.2 elseStatement

```
std::unique_ptr<ASTNode> pascal::CaseStmtNode::elseStatement
```

optional else/otherwise branch

Definition at line 413 of file [ast.hpp](#).

Referenced by [CaseStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.27.4.3 expression

```
std::unique_ptr<ASTNode> pascal::CaseStmtNode::expression
```

selector expression

Definition at line 411 of file [ast.hpp](#).

Referenced by [CaseStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

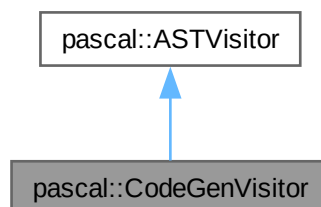
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.28 pascal::CodeGenVisitor Class Reference

AST visitor that generates MXVM intermediate code.

```
#include <icode.hpp>
```

Inheritance diagram for pascal::CodeGenVisitor:



Classes

- struct [Scope](#)

- struct [FuncInfo](#)
Describes a user-declared function's parameter and return types.
- struct [ValueLocation](#)
Describes where a value currently resides (register, memory, or immediate).
- struct [RecordField](#)
A field within a record type definition.
- struct [RecordTypeInfo](#)
Complete description of a record type.

Public Member Functions

- void [registerExternalFunc](#) (const std::string &funcName, const std::string &unitName)
Register a function or procedure as originating from an external unit.
- bool [isImportedUnit](#) (const std::string &name) const
Check whether a name refers to an imported unit.
- void [registerImportedConst](#) (const std::string &unitName, [ConstDeclNode](#) &node)
Register imported constant values from a dependency unit.
- void [registerImportedVar](#) (const std::string &unitName, [VarDeclNode](#) &node)
Register imported variable metadata from a dependency unit (no alloc emitted).
- [CodeGenVisitor](#) ()
Construct and initialise registers and float register pool.
- virtual [~CodeGenVisitor](#) ()=default
- void [initializeBuiltins](#) ()
Register all built-in function handlers (IO, Std, SDL, String).
- void [emitFree](#) (const std::string &s)
Emit a free instruction for an allocated pointer.
- void [generate](#) ([ASTNode](#) *root)
Generate MXVM code for the entire AST.
- void [markAllocatedPtr](#) (const std::string &p)
Mark a pointer name as allocated (will be freed at scope end).
- void [emit_invoke](#) (const std::string &funcName, const std::vector< std::string > ¶ms)
Emit an invoke instruction calling a named function with parameters.
- bool [isIntLiteral](#) (const std::string &s) const
- void [writeTo](#) (std::ostream &out) const
Write the complete MXVM program to a stream.
- std::string [getTypeString](#) (const [VarDeclNode](#) &node)
- std::string [storageSymbolFor](#) (const std::string &mangled)
- bool [isRecordTypeName](#) (const std::string &t)
- int [getTypeSizeByName](#) (const std::string &t)
- [VarType](#) [getTypeFromString](#) (const std::string &typeStr)
- void [emitArrayBoundsCheck](#) (const std::string &idxReg, int lower, int upper)
Emit bounds-check code for a static array access.
- void [emitDynArrayBoundsCheck](#) (const std::string &idxReg, const std::string &lenSym)
Emit bounds-check code for a dynamic array access.
- [VarType](#) [getExpressionType](#) ([ASTNode](#) *node)
- void [updateDataSectionInitialValue](#) (const std::string &varName, const std::string &type, const std::string &value)

AST Visitor Overrides

Code generation for each Pascal AST node type.

- void [visit](#) ([ProgramNode](#) &node) override
- void [visit](#) ([UnitNode](#) &node) override

Generate MXVM object code from a Pascal unit.

- void [visit](#) ([BlockNode](#) &node) override
- void [visit](#) ([VarDeclNode](#) &node) override
- void [visit](#) ([ProcCallNode](#) &node) override
- void [visit](#) ([FuncCallNode](#) &node) override
- void [visit](#) ([FuncDeclNode](#) &node) override
- void [visit](#) ([ProcDeclNode](#) &node) override
- void [visit](#) ([TypeAliasNode](#) &node) override
- void [visit](#) ([ParameterNode](#) &node) override
- void [visit](#) ([CompoundStmtNode](#) &node) override

Visit a compound statement block, freeing temporary pointers after each statement.

- void [visit](#) ([IfStmtNode](#) &node) override
- void [visit](#) ([WhileStmtNode](#) &node) override
- void [visit](#) ([ForStmtNode](#) &node) override
- void [visit](#) ([BinaryOpNode](#) &node) override
- void [visit](#) ([UnaryOpNode](#) &node) override
- void [visit](#) ([VariableNode](#) &node) override
- void [visit](#) ([NumberNode](#) &node) override
- void [visit](#) ([StringNode](#) &node) override
- void [visit](#) ([BooleanNode](#) &node) override
- void [visit](#) ([EmptyStmtNode](#) &node) override
- void [visit](#) ([ConstDeclNode](#) &node) override
- void [visit](#) ([RepeatStmtNode](#) &node) override
- void [visit](#) ([CaseStmtNode](#) &node) override
- void [visit](#) ([ArrayTypeNode](#) &node) override
- void [visit](#) ([ArrayDeclarationNode](#) &node) override
- void [visit](#) ([FieldAccessNode](#) &node) override
- void [visit](#) ([AssignmentNode](#) &node) override
- void [visit](#) ([ArrayAssignmentNode](#) &node) override
- void [visit](#) ([ArrayAccessNode](#) &node) override
- void [visit](#) ([RecordTypeNode](#) &node) override
- void [visit](#) ([RecordDeclarationNode](#) &node) override
- void [visit](#) ([TypeDeclNode](#) &node) override
- void [visit](#) ([SimpleTypeNode](#) &node) override
- void [visit](#) ([ArrayTypeDeclarationNode](#) &node) override
- void [visit](#) ([ExitNode](#) &node) override
- void [visit](#) ([BreakNode](#) &node) override
- void [visit](#) ([ContinueNode](#) &node) override
- void [visit](#) ([NilNode](#) &node) override
- void [visit](#) ([PointerTypeNode](#) &node) override
- void [visit](#) ([PointerDerefNode](#) &node) override
- void [visit](#) ([AddressOfNode](#) &node) override
- void [visit](#) ([WithStmtNode](#) &node) override
- void [visit](#) ([GotoStmtNode](#) &node) override
- void [visit](#) ([LabelStmtNode](#) &node) override
- void [visit](#) ([SetLiteralNode](#) &node) override
- void [visit](#) ([SetTypeNode](#) &node) override
- void [visit](#) ([EnumTypeDeclNode](#) &node) override

Public Member Functions inherited from [pascal::ASTVisitor](#)

- virtual [~ASTVisitor](#) ()=default

Public Attributes

- [BuiltinFunctionRegistry](#) [builtinRegistry](#)
registry of built-in function handlers
- std::string [name](#) = "App"
program name emitted in the output header
- std::unordered_map< std::string, int > [enumConstants](#)
Enum value → ordinal mapping (lowercased name → integer).
- std::unordered_map< std::string, std::vector< std::string > > [enumTypes](#)

Enum type name → ordered list of value names.

- `std::unordered_set< std::string >` [setVars](#)

Set of variable names that are set types (allocated as 256-byte bitsets).

- `std::unordered_set< std::string >` [fileVars](#)

Set of variable names that are file types (pointer to FILE).

- `std::unordered_map< std::string, std::string >` [fileVarNames](#)

Maps file variable name to companion filename variable name.

Private Member Functions

- `std::string` [generateRealConstantName](#) ()
- `bool` [isRealNumber](#) (const `std::string` &s) const
- `std::string` [mangleWithScope](#) (const `std::string` &baseName, const `std::vector< std::string >` &scopePath) const
- `std::string` [mangleVariableName](#) (const `std::string` &varName) const
- `std::string` [getCurrentScopeName](#) () const
- `std::string` [funcLabel](#) (const char *prefix, const `std::string` &name)
- `VarType` [getVarType](#) (const `std::string` &name) const
- `bool` [isRealConstSymbol](#) (const `std::string` &s) const
- `std::string` [coerceToIntImmediate](#) (const `std::string` &v)
- `void` [setVarType](#) (const `std::string` &name, `VarType` t)
- `void` [setSlotType](#) (int slot, `VarType` t)
- `bool` [isStringVar](#) (const `std::string` &name) const
- `std::string` [emptyString](#) ()
- `void` [initializeFloatRegisters](#) ()
- `void` [addTempPtr](#) (const `std::string` &v)
- `bool` [isRegisterSlot](#) (int slot) const
- `bool` [isTempVar](#) (const `std::string` &name) const
- `std::string` [slotVar](#) (int slot) const
- `std::string` [allocReg](#) ()
- `std::string` [allocPtrReg](#) ()
- `std::string` [allocFloatReg](#) ()
- `void` [expandFloatRegisters](#) ()
- `void` [freePtrReg](#) (const `std::string` ®)
- `void` [freeReg](#) (const `std::string` ®)
- `void` [freeFloatReg](#) (const `std::string` ®)
- `bool` [isFloatReg](#) (const `std::string` &name) const
- `bool` [isPtrReg](#) (const `std::string` &name) const
- `bool` [isReg](#) (const `std::string` &name) const
- `int` [newSlotFor](#) (const `std::string` &name)
- `std::string` [newTemp](#) ()
- `std::string` [newLabel](#) (const `std::string` &prefix="L")
- `void` [freeTempPtr](#) (const `std::string` &ptrName)
- `void` [releaseTempPtr](#) (const `std::string` &ptrName)
- `bool` [isTempPtr](#) (const `std::string` &name) const
- `std::string` [allocTempPtr](#) (const `std::string` &forScope="")
- `void` [emit](#) (const `std::string` &s)
- `void` [emitLabel](#) (const `std::string` &label)
- `void` [emit1](#) (const `std::string` &op, const `std::string` &a)
- `void` [emit2](#) (const `std::string` &op, const `std::string` &a, const `std::string` &b)
- `void` [emit3](#) (const `std::string` &op, const `std::string` &a, const `std::string` &b, const `std::string` &c)
- `void` [emit4](#) (const `std::string` &op, const `std::string` &a, const `std::string` &b, const `std::string` &c, const `std::string` &d)
- `std::string` [escapeStringForMxvm](#) (const `std::string` &raw) const
- `std::string` [ensureFloatConstSymbol](#) (const `std::string` &value)

- std::string [internString](#) (const std::string &val)
- std::string [convertCharLiteral](#) (const std::string &c)
- void [pushValue](#) (const std::string &v)
- std::string [popValue](#) ()
- std::string [eval](#) (ASTNode *n)
- void [recordLocation](#) (const std::string &var, ValueLocation loc)
- void [pushTri](#) (const char *op, const std::string &a, const std::string &b)
- void [pushCmpResult](#) (const std::string &a, const std::string &b, const char *jop)
- void [pushFloatCmpResult](#) (const std::string &aIn, const std::string &bIn, const char *jop)
- void [pushLogicalAnd](#) (const std::string &a, const std::string &b)
- void [pushLogicalOr](#) (const std::string &a, const std::string &b)
- bool [isParmReg](#) (const std::string &name) const
- void [allocateRecordFieldArrays](#) (const std::string &recordVarName, const std::string &recordTypeName)
- ArrayInfo [buildArrayInfoFromNode](#) (ArrayTypeNode *atn)
- int [getArrayElementSize](#) (const std::string &tIn)
- std::string [evaluateConstantExpression](#) (ASTNode *node)
- std::string [getTypeString](#) (ASTNode *typeNode)
- std::string [resolveTypeName](#) (std::string t) const
- int [getRecordTypeSize](#) (const std::string &typeName)
- std::string [findMangledFuncName](#) (const std::string &name, bool isProc) const
- std::string [findMangledName](#) (const std::string &name) const
- std::string [findMangledArrayName](#) (const std::string &name) const
- bool [isIntegerLiteral](#) (const std::string &s) const
- bool [isFloatLiteral](#) (const std::string &s) const
- bool [tryGetConstNumeric](#) (const std::string &name, std::string &out)
- std::string [foldNumeric](#) (ASTNode *n)
- std::pair< int, int > [getRecordFieldOffsetAndSize](#) (const std::string &recType, const std::string &field)
- std::string [getVarRecordTypeNameFromExpr](#) (ASTNode *expr)
- std::string [getVarRecordTypeName](#) (const std::string &varName)
- bool [isArrayTypeName](#) (const std::string &t) const
- std::string [getPointerBaseTypeName](#) (const std::string &varName) const
- int [getPointerElementSize](#) (const std::string &varName)
- VarType [getPointerDerefType](#) (const std::string &varName)
- std::string [getCurrentEndLabel](#) () const
- std::string [ensurePtrBase](#) (const std::string &v)
- std::string [getArrayNameFromBase](#) (ASTNode *base)
- ArrayInfo * [getArrayInfoForArrayAccess](#) (ArrayAccessNode *arr)

Static Private Member Functions

- static bool [endsWithColon](#) (const std::string &s)
- static std::string [lc](#) (std::string s)
- static bool [isPtrLike](#) (VarType t)

Private Attributes

- std::unordered_map< std::string, std::string > [compileTimeConstants](#)
- std::unordered_set< std::string > [usedRealConstants](#)
- int [realConstantCounter](#) = 0
- std::unordered_map< std::string, std::string > [currentParamTypes](#)
- std::set< std::string > [usedModules](#)
- std::vector< std::string > [objectDeps](#)
unit/object dependencies for section object
- bool [isUnit](#) = false
true when generating code for a unit (object output)

- `std::unordered_map< std::string, std::string >` [externalFuncs](#)
maps function/proc name to source unit name
- `std::unordered_set< std::string >` [importedUnitNames](#)
names of units imported via uses clause
- `std::unordered_map< std::string, ArrayInfo >` [importedArrayInfo](#)
array metadata for imported unit vars
- `std::unordered_map< std::string, VarType >` [importedVarTypes](#)
var types for imported unit vars
- `std::vector< std::string >` [globalArrays](#)
- `std::unordered_map< std::string, std::vector< std::string > >` [functionScopedArrays](#)
- `std::vector< std::string >` [scopeHierarchy](#)
- `std::vector< Scope >` [scopeStack](#)
- `std::unordered_map< std::string, std::string >` [typeAliases](#)
- `std::map< std::string, std::pair< std::string, std::string > >` [constInitialValues](#)
- `std::map< std::string, size_t >` [bufferStringVars](#)
- `std::map< std::string, std::string >` [currentParamLocations](#)
- `std::vector< std::pair< ProcDeclNode *, std::vector< std::string > > >` [deferredProcs](#)
- `std::vector< std::string >` [allTempPtrs](#)
- `std::vector< bool >` [tempPtrInUse](#)
- `std::unordered_map< std::string, FuncInfo >` [funcSignatures](#)
- `std::unordered_map< std::string, VarType >` [varTypes](#)
- `std::unordered_map< int, VarType >` [slotToType](#)
- `bool` [needsEmptyString](#) = false
- `int` [nextSpillReg](#) = 0
- `std::vector< std::string >` [registers](#) = {"rax", "rbx", "rcx", "rdx", "rsi", "rdi", "r8", "r9", "r10", "r11", "r12", "r13", "r14", "r15"}
- `const std::vector< std::string >` [ptrRegisters](#)
- `std::vector< std::string >` [floatRegisters](#)
- `std::vector< bool >` [floatRegInUse](#)
- `int` [nextFloatRegIndex](#) = 0
- `std::vector< bool >` [regInUse](#)
- `std::vector< bool >` [ptrRegInUse](#)
- `std::vector< std::pair< FuncDeclNode *, std::vector< std::string > > >` [deferredFuncs](#)
- `std::string` [currentFunctionName](#)
- `std::string` [currentFunctionReturnSlot](#)
slot variable that holds the return value until FUNC_END
- `std::vector< std::string >` [currentFuncLocalSlots](#)
- `bool` [functionSetReturn](#) = false
- `int` [nextSlot](#) = 0
- `int` [nextTemp](#) = 0
- `int` [labelCounter](#) = 0
- `std::unordered_map< std::string, int >` [varSlot](#)
- `std::vector< std::string >` [prolog](#)
- `std::vector< std::string >` [instructions](#)
- `std::vector< std::string >` [evalStack](#)
- `std::vector< std::pair< std::string, std::string > >` [stringLiterals](#)
- `std::unordered_set< std::string >` [usedStrings](#)
- `std::unordered_map< int, std::string >` [slotToName](#)
- `std::unordered_map< std::string, std::vector< std::string > >` [recordsToFreeInScope](#)
- `std::unordered_map< std::string, std::vector< std::string > >` [tempPtrByScope](#)
- `std::unordered_map< std::string, bool >` [declaredProcs](#)
- `std::unordered_map< std::string, bool >` [declaredFuncs](#)
- `bool` [generatingDeferredCode](#) = false
- `std::unordered_map< std::string, ValueLocation >` [valueLocations](#)

- `std::unordered_set< std::string >` [allocatedPtrs](#)
- `std::unordered_set< std::string >` [escapedTempPtrs](#)
- `std::unordered_map< std::string, std::string >` [realConstants](#)
- `std::unordered_map< std::string, ArrayInfo >` [arrayInfo](#)
- `std::unordered_map< std::string, int >` [dynArrayLenSlot](#)
mangled array name → slot of companion length variable
- `std::unordered_map< std::string, RecordTypeInfo >` [recordTypes](#)
- `std::unordered_map< std::string, std::string >` [varRecordType](#)
- `std::unordered_map< std::string, std::string >` [pointerBaseType](#)
- `std::vector< std::string >` [loopEndLabels](#)
- `std::vector< std::string >` [loopContinueLabels](#)
- `std::vector< std::unordered_map< std::string, std::string > >` [withFieldScopes](#)
Stack of active `with` scopes: each entry maps unqualified field name -> record variable name.
- `std::unordered_map< std::string, std::string >` [gotoLabels](#)
Maps user-declared `goto` label numbers to generated MXVM labels.

Friends

- class [IOFunctionHandler](#)
- class [MathFunctionHandler](#)
- class [StdFunctionHandler](#)
- class [SDLFunctionHandler](#)
- class [StringFunctionHandler](#)
- class [FileFunctionHandler](#)

11.28.1 Detailed Description

AST visitor that generates MXVM intermediate code.

Traverses the Pascal AST, emitting register-based intermediate instructions into an instruction list. The final output is written via [writeTo\(\)](#) in MXVM program format.

Definition at line 178 of file [icode.hpp](#).

11.28.2 Constructor & Destructor Documentation

11.28.2.1 CodeGenVisitor()

```
pascal::CodeGenVisitor::CodeGenVisitor ()
```

Construct and initialise registers and float register pool.

Definition at line 10 of file [icode_visit.cpp](#).

References [initializeBuiltins\(\)](#), [initializeFloatRegisters\(\)](#), [ptrRegInUse](#), [ptrRegisters](#), [regInUse](#), and [registers](#).

11.28.2.2 ~CodeGenVisitor()

```
virtual pascal::CodeGenVisitor::~~CodeGenVisitor () [virtual], [default]
```

11.28.3 Member Function Documentation

11.28.3.1 addTempPtr()

```
void pascal::CodeGenVisitor::addTempPtr (
    const std::string & v) [inline], [private]
```

Definition at line 353 of file [icode.hpp](#).

References [getCurrentScopeName\(\)](#), and [tempPtrByScope](#).

11.28.3.2 allocateRecordFieldArrays()

```
void pascal::CodeGenVisitor::allocateRecordFieldArrays (
    const std::string & recordVarName,
    const std::string & recordTypeName) [inline], [private]
```

Definition at line 785 of file [icode.hpp](#).

References [emit3\(\)](#), [emit4\(\)](#), [ensurePtrBase\(\)](#), [functionScopedArrays](#), [getCurrentScopeName\(\)](#), [globalArrays](#), [newSlotFor\(\)](#), [pascal::PTR](#), [recordTypes](#), [setSlotType\(\)](#), [slotVar\(\)](#), and [updateDataSectionInitialValue\(\)](#).

Referenced by [visit\(\)](#).

11.28.3.3 allocFloatReg()

```
std::string pascal::CodeGenVisitor::allocFloatReg () [inline], [private]
```

Definition at line 412 of file [icode.hpp](#).

References [expandFloatRegisters\(\)](#), [floatRegInUse](#), and [floatRegisters](#).

Referenced by [emit_invoke\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pushTri\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.4 allocPtrReg()

```
std::string pascal::CodeGenVisitor::allocPtrReg () [inline], [private]
```

Definition at line 401 of file [icode.hpp](#).

References [ptrRegInUse](#), and [ptrRegisters](#).

11.28.3.5 allocReg()

```
std::string pascal::CodeGenVisitor::allocReg () [inline], [private]
```

Definition at line 389 of file [icode.hpp](#).

References [name](#), [nextSpillReg](#), [regInUse](#), and [registers](#).

Referenced by [emit_invoke\(\)](#), [emitDynArrayBoundsCheck\(\)](#), [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generateWithResult\(\)](#), [pascal::StdFunctionHandler::generate\(\)](#), [pascal::FileFunctionHandler::generateWithResult\(\)](#), [pascal::IOFunctionHandler::generateWithResult\(\)](#), [pascal::SDLFunctionHandler::generateWithResult\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pascal::StringFunctionHandler::generateWithResult\(\)](#), [pushCmpResult\(\)](#), [pushFloatCmpResult\(\)](#), [pushLogicalAnd\(\)](#), [pushLogicalOr\(\)](#), [pushTri\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.6 allocTempPtr()

```
std::string pascal::CodeGenVisitor::allocTempPtr (
    const std::string & forScope = "") [inline], [private]
```

Definition at line 502 of file [icode.hpp](#).

References [allTempPtrs](#), [getCurrentScopeName\(\)](#), [nextTemp](#), [tempPtrByScope](#), and [tempPtrInUse](#).

Referenced by [ensurePtrBase\(\)](#), [pascal::StringFunctionHandler::generateWithResult\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.7 buildArrayInfoFromNode()

```
ArrayInfo pascal::CodeGenVisitor::buildArrayInfoFromNode (
    ArrayTypeNode * atn) [inline], [private]
```

Definition at line 817 of file [icode.hpp](#).

References [buildArrayInfoFromNode\(\)](#), [pascal::ArrayInfo::elementArray](#), [pascal::ArrayInfo::elementIsArray](#), [pascal::ArrayInfo::elementSize](#), [pascal::ArrayInfo::elementType](#), [pascal::ArrayTypeNode::elementType](#), [evaluateConstantExpression\(\)](#), [getArrayElementSize\(\)](#), [pascal::ArrayInfo::isDynamic](#), [lc\(\)](#), [pascal::ArrayInfo::lowerBound](#), [pascal::ArrayTypeNode::lowerBound](#), [resolveTypeName\(\)](#), [pascal::ArrayInfo::size](#), [typeAliases](#), [pascal::ArrayInfo::upperBound](#), and [pascal::ArrayTypeNode::upperBound](#).
Referenced by [buildArrayInfoFromNode\(\)](#), [registerImportedVar\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.8 coerceToIntImmediate()

```
std::string pascal::CodeGenVisitor::coerceToIntImmediate (
    const std::string & v) [inline], [private]
```

Definition at line 290 of file [icode.hpp](#).

References [isFloatLiteral\(\)](#), [isRealConstSymbol\(\)](#), and [realConstants](#).
 Referenced by [visit\(\)](#).

11.28.3.9 convertCharLiteral()

```
std::string pascal::CodeGenVisitor::convertCharLiteral (
    const std::string & c) [inline], [private]
```

Definition at line 587 of file [icode.hpp](#).

11.28.3.10 emit()

```
void pascal::CodeGenVisitor::emit (
    const std::string & s) [inline], [private]
```

Definition at line 525 of file [icode.hpp](#).

References [instructions](#).

Referenced by [emit1\(\)](#), [emit2\(\)](#), [emit4\(\)](#), [emit_invoke\(\)](#), [emitFree\(\)](#), [generate\(\)](#), [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [pascal::StdFunctionHandler::generate\(\)](#), [pascal::FileFunctionHandler::generateWithResult\(\)](#), [pascal::IOFunctionHandler::generateWithResult\(\)](#), [pascal::SDLFunctionHandler::generateWithResult\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pascal::StringFunctionHandler::generateWithResult\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.11 emit1()

```
void pascal::CodeGenVisitor::emit1 (
    const std::string & op,
    const std::string & a) [inline], [private]
```

Definition at line 529 of file [icode.hpp](#).

References [emit\(\)](#).

Referenced by [emitArrayBoundsCheck\(\)](#), [emitDynArrayBoundsCheck\(\)](#), [freeTempPtr\(\)](#), [generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [pushCmpResult\(\)](#), [pushFloatCmpResult\(\)](#), [pushLogicalAnd\(\)](#), [pushLogicalOr\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.12 emit2()

```
void pascal::CodeGenVisitor::emit2 (
    const std::string & op,
    const std::string & a,
    const std::string & b) [inline], [private]
```

Definition at line 530 of file [icode.hpp](#).

References [emit\(\)](#), [ensureFloatConstSymbol\(\)](#), and [isRealNumber\(\)](#).

Referenced by [emit_invoke\(\)](#), [emitArrayBoundsCheck\(\)](#), [emitDynArrayBoundsCheck\(\)](#), [ensurePtrBase\(\)](#), [generate\(\)](#), [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [pascal::StdFunctionHandler::generate\(\)](#), [pascal::FileFunctionHandler::generateWithResult\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pushCmpResult\(\)](#), [pushFloatCmpResult\(\)](#), [pushLogicalAnd\(\)](#), [pushLogicalOr\(\)](#), [pushTri\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.13 emit3()

```
void pascal::CodeGenVisitor::emit3 (
    const std::string & op,
    const std::string & a,
    const std::string & b,
    const std::string & c) [inline], [private]
```

Definition at line 537 of file [icode.hpp](#).

References [allocatedPtrs](#), and [instructions](#).

Referenced by [allocateRecordFieldArrays\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.14 emit4()

```
void pascal::CodeGenVisitor::emit4 (
    const std::string & op,
```

```

    const std::string & a,
    const std::string & b,
    const std::string & c,
    const std::string & d) [inline], [private]

```

Definition at line 543 of file [icode.hpp](#).

References [emit\(\)](#).

Referenced by [allocateRecordFieldArrays\(\)](#), [pascal::StdFunctionHandler::generate\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.15 emit_invoke()

```

void pascal::CodeGenVisitor::emit_invoke (
    const std::string & funcName,
    const std::vector< std::string > & params) [inline]

```

Emit an invoke instruction calling a named function with parameters.

Parameters

<i>funcName</i>	Function/procedure label
<i>params</i>	Values to pass (literals are first moved into registers)

Definition at line 1287 of file [icode.hpp](#).

References [allocFloatReg\(\)](#), [allocReg\(\)](#), [emit\(\)](#), [emit2\(\)](#), [freeReg\(\)](#), [isIntLiteral\(\)](#), and [isRealNumber\(\)](#).

Referenced by [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [pascal::SDLFunctionHandler::generate\(\)](#), [pascal::StdFunctionHandler::generate\(\)](#), [pascal::FileFunctionHandler::generateWithResult\(\)](#), [pascal::IOFunctionHandler::generateWithResult\(\)](#), [pascal::SDLFunctionHandler::generateWithResult\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pascal::StringFunctionHandler::generate\(\)](#), and [visit\(\)](#).

11.28.3.16 emitArrayBoundsCheck()

```

void pascal::CodeGenVisitor::emitArrayBoundsCheck (
    const std::string & idxReg,
    int lower,
    int upper) [inline]

```

Emit bounds-check code for a static array access.

Parameters

<i>idxReg</i>	Register or variable holding the index
<i>lower</i>	Compile-time lower bound
<i>upper</i>	Compile-time upper bound

Generates code that exits with code 1 when the index is out of the static [lower..upper] range. Gated by `MXVM_BOUNDS_CHECK`.

Definition at line 1572 of file [icode.hpp](#).

References [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), and [newLabel\(\)](#).

Referenced by [visit\(\)](#), and [visit\(\)](#).

11.28.3.17 emitDynArrayBoundsCheck()

```

void pascal::CodeGenVisitor::emitDynArrayBoundsCheck (
    const std::string & idxReg,
    const std::string & lenSym) [inline]

```

Emit bounds-check code for a dynamic array access.

Parameters

<i>idxReg</i>	Register or variable holding the index
<i>lenSym</i>	Variable holding the runtime array length

Generates code that exits with code 1 when the index is negative or $\geq$ the runtime length stored in `lenSym`. Gated by `MXVM_BOUNDS_CHECK`.

Definition at line 1596 of file `icode.hpp`.

References `allocReg()`, `emit1()`, `emit2()`, `emitLabel()`, `freeReg()`, and `newLabel()`.

Referenced by `visit()`, and `visit()`.

11.28.3.18 emitFree()

```
void pascal::CodeGenVisitor::emitFree (
    const std::string & s) [inline]
```

Emit a free instruction for an allocated pointer.

Definition at line 980 of file `icode.hpp`.

References `allocatedPtrs`, and `emit()`.

Referenced by `generate()`.

11.28.3.19 emitLabel()

```
void pascal::CodeGenVisitor::emitLabel (
    const std::string & label) [inline], [private]
```

Definition at line 528 of file `icode.hpp`.

References `instructions`.

Referenced by `emitArrayBoundsCheck()`, `emitDynArrayBoundsCheck()`, `generate()`, `pushCmpResult()`, `pushFloatCmpResult()`, `pushLogicalAnd()`, `pushLogicalOr()`, `visit()`, `visit()`, `visit()`, `visit()`, `visit()`, `visit()`, `visit()`, `visit()`, `visit()`, `visit()`, `visit()`, and `visit()`.

11.28.3.20 emptyString()

```
std::string pascal::CodeGenVisitor::emptyString () [inline], [private]
```

Definition at line 311 of file `icode.hpp`.

References `needsEmptyString`.

Referenced by `generate()`.

11.28.3.21 endsWithColon()

```
bool pascal::CodeGenVisitor::endsWithColon (
    const std::string & s) [inline], [static], [private]
```

Definition at line 368 of file `icode.hpp`.

Referenced by `writeTo()`.

11.28.3.22 ensureFloatConstSymbol()

```
std::string pascal::CodeGenVisitor::ensureFloatConstSymbol (
    const std::string & value) [inline], [private]
```

Definition at line 564 of file `icode.hpp`.

References `generateRealConstantName()`, `isRealNumber()`, `realConstants`, and `usedRealConstants`.

Referenced by `emit2()`, `registerImportedConst()`, `visit()`, `visit()`, and `visit()`.

11.28.3.23 ensurePtrBase()

```
std::string pascal::CodeGenVisitor::ensurePtrBase (
    const std::string & v) [inline], [private]
```

Definition at line 2171 of file `icode.hpp`.

References `allocTempPtr()`, `emit2()`, `isPtrReg()`, `pascal::PTR`, `pascal::RECORD`, `slotToType`, `pascal::STRING`, and `varSlot`.

Referenced by [allocateRecordFieldArrays\(\)](#), [pascal::StdFunctionHandler::generate\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.24 escapeStringForMxvm()

```
std::string pascal::CodeGenVisitor::escapeStringForMxvm (
    const std::string & raw) const [inline], [private]
```

Definition at line 545 of file [icode.hpp](#).

Referenced by [writeTo\(\)](#).

11.28.3.25 eval()

```
std::string pascal::CodeGenVisitor::eval (
    ASTNode * n) [inline], [private]
```

Definition at line 634 of file [icode.hpp](#).

References [pascal::ASTNode::accept\(\)](#), [evalStack](#), [isRealNumber\(\)](#), and [popValue\(\)](#).

Referenced by [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [pascal::SDLFunctionHandler::generate\(\)](#), [pascal::StdFunctionHandler::generate\(\)](#), [pascal::FileFunctionHandler::generateWithResult\(\)](#), [pascal::IOFunctionHandler::generateWithResult\(\)](#), [pascal::SDLFunctionHandler::generateWithResult\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pascal::StringFunctionHandler::generateWithResult\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.26 evaluateConstantExpression()

```
std::string pascal::CodeGenVisitor::evaluateConstantExpression (
    ASTNode * node) [inline], [private]
```

Definition at line 1696 of file [icode.hpp](#).

References [compileTimeConstants](#), [pascal::BinaryOpNode::DIV](#), [pascal::BinaryOpNode::DIVIDE](#), [evaluateConstantExpression\(\)](#), [findMangledName\(\)](#), [isFloatLiteral\(\)](#), [pascal::BinaryOpNode::MINUS](#), [pascal::UnaryOpNode::MINUS](#), [pascal::BinaryOpNode::MOD](#), [pascal::BinaryOpNode::MULTIPLY](#), [pascal::BinaryOpNode::PLUS](#), and [pascal::UnaryOpNode::PLUS](#).

Referenced by [buildArrayInfoFromNode\(\)](#), [evaluateConstantExpression\(\)](#), [registerImportedConst\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.27 expandFloatRegisters()

```
void pascal::CodeGenVisitor::expandFloatRegisters () [inline], [private]
```

Definition at line 425 of file [icode.hpp](#).

References [floatRegInUse](#), and [floatRegisters](#).

Referenced by [allocFloatReg\(\)](#).

11.28.3.28 findMangledArrayName()

```
std::string pascal::CodeGenVisitor::findMangledArrayName (
    const std::string & name) const [inline], [private]
```

Definition at line 1877 of file [icode.hpp](#).

References [name](#), [scopeHierarchy](#), and [varSlot](#).

Referenced by [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.29 findMangledFuncName()

```
std::string pascal::CodeGenVisitor::findMangledFuncName (
    const std::string & name,
    bool isProc) const [inline], [private]
```

Definition at line 1843 of file [icode.hpp](#).

References [declaredFuncs](#), [declaredProcs](#), [mangleWithScope\(\)](#), [name](#), and [scopeHierarchy](#).

Referenced by [getCurrentEndLabel\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.30 findMangledName()

```
std::string pascal::CodeGenVisitor::findMangledName (
    const std::string & name) const [inline], [private]
```

Definition at line 1855 of file [icode.hpp](#).

References [name](#), [scopeHierarchy](#), and [varSlot](#).

Referenced by [evaluateConstantExpression\(\)](#), [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [getArrayInfoForArrayAccess\(\)](#), [getPointerBaseTypeName\(\)](#), [getVarRecordTypeName\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [getVarType\(\)](#), [tryGetConstNumeric\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.31 foldNumeric()

```
std::string pascal::CodeGenVisitor::foldNumeric (
    ASTNode * n) [inline], [private]
```

Definition at line 1928 of file [icode.hpp](#).

References [pascal::BinaryOpNode::DIV](#), [pascal::BinaryOpNode::DIVIDE](#), [foldNumeric\(\)](#), [isFloatLiteral\(\)](#), [pascal::BinaryOpNode::MINUS](#), [pascal::BinaryOpNode::MOD](#), [pascal::BinaryOpNode::MULTIPLY](#), [pascal::BinaryOpNode::PLUS](#), and [tryGetConstNumeric\(\)](#).

Referenced by [foldNumeric\(\)](#), and [visit\(\)](#).

11.28.3.32 freeFloatReg()

```
void pascal::CodeGenVisitor::freeFloatReg (
    const std::string & reg) [inline], [private]
```

Definition at line 448 of file [icode.hpp](#).

References [floatRegInUse](#), and [floatRegisters](#).

Referenced by [pascal::StdFunctionHandler::generateWithResult\(\)](#), and [pushFloatCmpResult\(\)](#).

11.28.3.33 freePtrReg()

```
void pascal::CodeGenVisitor::freePtrReg (
    const std::string & reg) [inline], [private]
```

Definition at line 431 of file [icode.hpp](#).

References [ptrRegInUse](#), and [ptrRegisters](#).

Referenced by [freeReg\(\)](#).

11.28.3.34 freeReg()

```
void pascal::CodeGenVisitor::freeReg (
    const std::string & reg) [inline], [private]
```

Definition at line 439 of file [icode.hpp](#).

References [freePtrReg\(\)](#), [regInUse](#), [registers](#), and [releaseTempPtr\(\)](#).

Referenced by [emit_invoke\(\)](#), [emitDynArrayBoundsCheck\(\)](#), [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [pascal::SDLFunctionHandler::generate\(\)](#), [pascal::StdFunctionHandler::generate\(\)](#), [pascal::FileFunctionHandler::generateWithResult\(\)](#), [pascal::IOFunctionHandler::generateWithResult\(\)](#), [pascal::SDLFunctionHandler::generateWithResult\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pascal::StringFunctionHandler::generateWithResult\(\)](#), [pushCmpResult\(\)](#), [pushLogicalAnd\(\)](#), [pushLogicalOr\(\)](#), [pushTri\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.35 freeTempPtr()

```
void pascal::CodeGenVisitor::freeTempPtr (
    const std::string & ptrName) [inline], [private]
```

Definition at line 477 of file [icode.hpp](#).

References [allTempPtrs](#), [emit1\(\)](#), and [tempPtrInUse](#).

11.28.3.36 funcLabel()

```
std::string pascal::CodeGenVisitor::funcLabel (
    const char * prefix,
    const std::string & name) [inline], [private]
```

Definition at line 274 of file [icode.hpp](#).

References [name](#).

Referenced by [generate\(\)](#).

11.28.3.37 generate()

```
void pascal::CodeGenVisitor::generate (
    ASTNode * root) [inline]
```

Generate MXVM code for the entire AST.

Parameters

<i>root</i>	Root AST node (typically a ProgramNode)
-------------	--

Definition at line 990 of file [icode.hpp](#).

References [pascal::ASTNode::accept\(\)](#), [allocatedPtrs](#), [arrayInfo](#), [compileTimeConstants](#), [currentFuncLocalSlots](#), [currentFunctionName](#), [currentFunctionReturnSlot](#), [currentParamLocations](#), [currentParamTypes](#), [declaredFuncs](#), [deferredFuncs](#), [deferredProcs](#), [pascal::DOUBLE](#), [emit\(\)](#), [emit1\(\)](#), [emit2\(\)](#), [emitFree\(\)](#), [emitLabel\(\)](#), [emptyString\(\)](#), [escapedTempPtrs](#), [floatRegInUse](#), [floatRegisters](#), [funcLabel\(\)](#), [functionScopedArrays](#), [functionSetReturn](#), [generateRealConstantName\(\)](#), [generatingDeferredCode](#), [getTypeFromString\(\)](#), [getVarType\(\)](#), [globalArrays](#), [importedArrayInfo](#), [importedVarTypes](#), [instructions](#), [isUnit](#), [lc\(\)](#), [mangleVariableName\(\)](#), [mangleWithScope\(\)](#), [name](#), [newSlotFor\(\)](#), [objectDeps](#), [pointerBaseType](#), [prolog](#), [pascal::PTR](#), [ptrRegInUse](#), [ptrRegisters](#), [realConstants](#), [pascal::RECORD](#), [recordsToFreeInScope](#), [recordTypes](#), [regInUse](#), [registers](#), [resolveTypeName\(\)](#), [scopeHierarchy](#), [scopeStack](#), [setSlotType\(\)](#), [setVarType\(\)](#), [slotVar\(\)](#), [pascal::STRING](#), [tempPtrByScope](#), [typeAliases](#), [usedRealConstants](#), [usedStrings](#), [valueLocations](#), [varRecordType](#), and [varTypes](#).

Referenced by [compileFile\(\)](#).

11.28.3.38 generateRealConstantName()

```
std::string pascal::CodeGenVisitor::generateRealConstantName () [inline], [private]
```

Definition at line 208 of file [icode.hpp](#).

References [realConstantCounter](#).

Referenced by [ensureFloatConstSymbol\(\)](#), [generate\(\)](#), and [pushFloatCmpResult\(\)](#).

11.28.3.39 getArrayElementSize()

```
int pascal::CodeGenVisitor::getArrayElementSize (
    const std::string & tIn) [inline], [private]
```

Definition at line 865 of file [icode.hpp](#).

References [lc\(\)](#), [recordTypes](#), and [resolveTypeName\(\)](#).

Referenced by [buildArrayInfoFromNode\(\)](#), and [visit\(\)](#).

11.28.3.40 getArrayInfoForArrayAccess()

```
ArrayInfo * pascal::CodeGenVisitor::getArrayInfoForArrayAccess (
    ArrayAccessNode * arr) [inline], [private]
```

Definition at line 2208 of file [icode.hpp](#).

References [arrayInfo](#), [pascal::ArrayAccessNode::base](#), [pascal::ArrayInfo::elementArray](#), [pascal::ArrayInfo::elementIsArray](#), [findMangledName\(\)](#), [getArrayInfoForArrayAccess\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [isImportedUnit\(\)](#), [lc\(\)](#), [recordTypes](#), and [resolveTypeName\(\)](#).

Referenced by [getArrayInfoForArrayAccess\(\)](#), [getExpressionType\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.41 getArrayNameFromBase()

```
std::string pascal::CodeGenVisitor::getArrayNameFromBase (
    ASTNode * base) [inline], [private]
```

Definition at line 2194 of file [icode.hpp](#).

References [getArrayNameFromBase\(\)](#).

Referenced by [getArrayNameFromBase\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.42 getCurrentEndLabel()

```
std::string pascal::CodeGenVisitor::getCurrentEndLabel () const [inline], [private]
```

Definition at line 2150 of file [icode.hpp](#).

References [currentFunctionName](#), [declaredFuncs](#), [declaredProcs](#), and [findMangledFuncName\(\)](#).

Referenced by [visit\(\)](#).

11.28.3.43 getCurrentScopeName()

```
std::string pascal::CodeGenVisitor::getCurrentScopeName () const [inline], [private]
```

Definition at line 245 of file [icode.hpp](#).

References [deferredFuncs](#), [generatingDeferredCode](#), [mangleWithScope\(\)](#), and [scopeHierarchy](#).

Referenced by [addTempPtr\(\)](#), [allocateRecordFieldArrays\(\)](#), [allocTempPtr\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.44 getExpressionType()

```
VarType pascal::CodeGenVisitor::getExpressionType (
    ASTNode * node) [inline]
```

Definition at line 1614 of file [icode.hpp](#).

References [pascal::BOOL](#), [builtinRegistry](#), [pascal::CHAR](#), [pascal::DOUBLE](#), [pascal::ArrayInfo::elementsArray](#), [pascal::ArrayInfo::elementType](#), [funcSignatures](#), [getArrayInfoForArrayAccess\(\)](#), [getExpressionType\(\)](#), [getPointerDerefType\(\)](#), [getTypeFromString\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [getVarType\(\)](#), [pascal::INT](#), [pascal::NumberNode::isReal](#), [isRecordTypeName\(\)](#), [lc\(\)](#), [pascal::BinaryOpNode::PLUS](#), [pascal::PTR](#), [pascal::RECORD](#), [recordTypes](#), [resolveTypeName\(\)](#), and [pascal::STRING](#).

Referenced by [pascal::IOFunctionHandler::generate\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [getExpressionType\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.45 getPointerBaseTypeName()

```
std::string pascal::CodeGenVisitor::getPointerBaseTypeName (
    const std::string & varName) const [inline], [private]
```

Definition at line 2115 of file [icode.hpp](#).

References [findMangledName\(\)](#), and [pointerBaseType](#).

Referenced by [getPointerDerefType\(\)](#), [getPointerElementSize\(\)](#), and [visit\(\)](#).

11.28.3.46 getPointerDerefType()

```
VarType pascal::CodeGenVisitor::getPointerDerefType (
    const std::string & varName) [inline], [private]
```

Definition at line 2134 of file [icode.hpp](#).

References [getPointerBaseTypeName\(\)](#), [getTypeFromString\(\)](#), and [pascal::INT](#).

Referenced by [getExpressionType\(\)](#), and [visit\(\)](#).

11.28.3.47 getPointerElementSize()

```
int pascal::CodeGenVisitor::getPointerElementSize (
    const std::string & varName) [inline], [private]
```

Definition at line 2126 of file [icode.hpp](#).

References [getPointerBaseTypeName\(\)](#), [getTypeSizeByName\(\)](#), [lc\(\)](#), and [resolveTypeName\(\)](#).

Referenced by [visit\(\)](#).

11.28.3.48 getRecordFieldOffsetAndSize()

```
std::pair< int, int > pascal::CodeGenVisitor::getRecordFieldOffsetAndSize (
    const std::string & recType,
    const std::string & field) [inline], [private]
```

Definition at line 2005 of file [icode.hpp](#).

References [lc\(\)](#), [recordTypes](#), and [resolveTypeName\(\)](#).

Referenced by [visit\(\)](#), and [visit\(\)](#).

11.28.3.49 getRecordTypeSize()

```
int pascal::CodeGenVisitor::getRecordTypeSize (
    const std::string & typeName) [inline], [private]
```

Definition at line 1835 of file [icode.hpp](#).

References [recordTypes](#).

11.28.3.50 getTypeFromString()

```
VarType pascal::CodeGenVisitor::getTypeFromString (
    const std::string & typeStr) [inline]
```

Definition at line 1538 of file [icode.hpp](#).

References [arrayInfo](#), [pascal::CHAR](#), [pascal::DOUBLE](#), [pascal::INT](#), [isRecordTypeName\(\)](#), [lc\(\)](#), [pascal::PTR](#), [pascal::RECORD](#), [resolveTypeName\(\)](#), and [pascal::UNKNOWN](#).

Referenced by [generate\(\)](#), [getExpressionType\(\)](#), [getPointerDerefType\(\)](#), [registerImportedVar\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.51 getTypeSizeByName()

```
int pascal::CodeGenVisitor::getTypeSizeByName (
    const std::string & t) [inline]
```

Definition at line 1522 of file [icode.hpp](#).

References [lc\(\)](#), [recordTypes](#), and [resolveTypeName\(\)](#).

Referenced by [getPointerElementSize\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.52 getTypeString() [1/2]

```
std::string pascal::CodeGenVisitor::getTypeString (
    ASTNode * typeNode) [inline], [private]
```

Definition at line 1802 of file [icode.hpp](#).

11.28.3.53 getTypeString() [2/2]

```
std::string pascal::CodeGenVisitor::getTypeString (
    const VarDeclNode & node) [inline]
```

Definition at line 1502 of file [icode.hpp](#).

References [pascal::VarDeclNode::type](#).

11.28.3.54 getVarRecordTypeName()

```
std::string pascal::CodeGenVisitor::getVarRecordTypeName (
    const std::string & varName) [inline], [private]
```

Definition at line 2089 of file [icode.hpp](#).

References [currentParamTypes](#), [findMangledName\(\)](#), [resolveTypeName\(\)](#), and [varRecordType](#).

Referenced by [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.55 getVarRecordTypeNameFromExpr()

```
std::string pascal::CodeGenVisitor::getVarRecordTypeNameFromExpr (
    ASTNode * expr) [inline], [private]
```

Definition at line 2021 of file [icode.hpp](#).

References [currentParamTypes](#), [pascal::ArrayInfo::elementType](#), [findMangledName\(\)](#), [getArrayInfoForArrayAccess\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [lc\(\)](#), [recordTypes](#), [resolveTypeName\(\)](#), [typeAliases](#), and [varRecordType](#).

Referenced by [getArrayInfoForArrayAccess\(\)](#), [getExpressionType\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.56 getVarType()

```
VarType pascal::CodeGenVisitor::getVarType (
    const std::string & name) const [inline], [private]
```

Definition at line 276 of file [icode.hpp](#).

References [findMangledName\(\)](#), [name](#), [slotToType](#), [pascal::UNKNOWN](#), [varSlot](#), and [varTypes](#).

Referenced by [generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [getExpressionType\(\)](#), [isStringVar\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.57 initializeBuiltins()

```
void pascal::CodeGenVisitor::initializeBuiltins () [inline]
```

Register all built-in function handlers (IO, Std, SDL, String).

Definition at line 969 of file [icode.hpp](#).

References [builtinRegistry](#).

Referenced by [CodeGenVisitor\(\)](#).

11.28.3.58 initializeFloatRegisters()

```
void pascal::CodeGenVisitor::initializeFloatRegisters () [inline], [private]
```

Definition at line 323 of file [icode.hpp](#).

References [floatRegInUse](#), and [floatRegisters](#).

Referenced by [CodeGenVisitor\(\)](#).

11.28.3.59 internString()

```
std::string pascal::CodeGenVisitor::internString (
    const std::string & val) [inline], [private]
```

Definition at line 578 of file [icode.hpp](#).

References [stringLiterals](#).

Referenced by [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [registerImportedConst\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.60 isArrayTypeName()

```
bool pascal::CodeGenVisitor::isArrayTypeName (
    const std::string & t) const [inline], [private]
```

Definition at line 2110 of file [icode.hpp](#).

References [arrayInfo](#), [lc\(\)](#), and [resolveTypeName\(\)](#).

11.28.3.61 isFloatLiteral()

```
bool pascal::CodeGenVisitor::isFloatLiteral (
    const std::string & s) const [inline], [private]
```

Definition at line 1906 of file [icode.hpp](#).

Referenced by [coerceToIntImmediate\(\)](#), [evaluateConstantExpression\(\)](#), [foldNumeric\(\)](#), [tryGetConstNumeric\(\)](#), and [visit\(\)](#).

11.28.3.62 isFloatReg()

```
bool pascal::CodeGenVisitor::isFloatReg (
    const std::string & name) const [inline], [private]
```

Definition at line 455 of file [icode.hpp](#).

References [floatRegisters](#), and [name](#).

Referenced by [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pushFloatCmpResult\(\)](#), [pushTri\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.63 isImportedUnit()

```
bool pascal::CodeGenVisitor::isImportedUnit (
    const std::string & name) const [inline], [nodiscard]
```

Check whether a name refers to an imported unit.

Definition at line 909 of file [icode.hpp](#).

11.28.3.71 isRecordTypeName()

```
bool pascal::CodeGenVisitor::isRecordTypeName (
    const std::string & t) [inline]
```

Definition at line 1517 of file icode.hpp.

References `lc()`, `recordTypes`, and `resolveTypeName()`.

Referenced by [getExpressionType\(\)](#), [getTypeFromString\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.72 isReg()

```
bool pascal::CodeGenVisitor::isReg (
    const std::string & name) const [inline], [private]
```

Definition at line 457 of file icode.hpp.

References `floatRegisters`, `isPtrReg()`, `isTempPtr()`, `name`, and `registers`.

[illegible]

11.28.3.73 isRegisterSlot()

```
bool pascal::CodeGenVisitor::isRegisterSlot (
    int slot) const [inline], [private]
```

Definition at line 369 of file icode.hpp.

Referenced by `writeTo()`.

11.28.3.74 isStringVar()

```
bool pascal::CodeGenVisitor::isStringVar (
    const std::string & name) const [inline], [private]
```

Definition at line 307 of file icode.hpp.

References [getVarType\(\)](#), [name](#), [pascal::PTR](#), and [pascal::STRING](#).

11.28.3.75 isTempPtr()

```
bool pascal::CodeGenVisitor::isTempPtr (
    const std::string & name) const [inline], [private]
```

Definition at line 498 of file icode.hpp.

References `allTempPtrs`, and `name`.

Referenced by `isReg()`, and `visit()`.

11.28.3.76 isTempVar()

```
bool pascal::CodeGenVisitor::isTempVar (
    const std::string & name) const [inline], [private]
```

Definition at line 370 of file icode.hpp.

References [name](#).

Referenced by `writeTo()`.

11.28.3.77 lc()

```
std::string pascal::CodeGenVisitor::lc (
    std::string s) [inline], [static], [private]
```

Definition at line 1813 of file icode.hpp.

[illegible]

11.28.3.78 mangleVariableName()

```
std::string pascal::CodeGenVisitor::mangleVariableName (
    const std::string & varName) const [inline], [private]
```

Definition at line 241 of file [icode.hpp](#).

References [mangleWithScope\(\)](#), and [scopeHierarchy](#).

Referenced by [generate\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.79 mangleWithScope()

```
std::string pascal::CodeGenVisitor::mangleWithScope (
    const std::string & baseName,
    const std::vector< std::string > & scopePath) const [inline], [private]
```

Definition at line 227 of file [icode.hpp](#).

Referenced by [findMangledFuncName\(\)](#), [generate\(\)](#), [getCurrentScopeName\(\)](#), and [mangleVariableName\(\)](#).

11.28.3.80 markAllocatedPtr()

```
void pascal::CodeGenVisitor::markAllocatedPtr (
    const std::string & p) [inline]
```

Mark a pointer name as allocated (will be freed at scope end).

Definition at line 1280 of file [icode.hpp](#).

References [allocatedPtrs](#).

Referenced by [pascal::StringFunctionHandler::generateWithResult\(\)](#).

11.28.3.81 newLabel()

```
std::string pascal::CodeGenVisitor::newLabel (
    const std::string & prefix = "L") [inline], [private]
```

Definition at line 473 of file [icode.hpp](#).

References [labelCounter](#).

Referenced by [emitArrayBoundsCheck\(\)](#), [emitDynArrayBoundsCheck\(\)](#), [pushCmpResult\(\)](#), [pushFloatCmpResult\(\)](#), [pushLogicalAnd\(\)](#), [pushLogicalOr\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.82 newSlotFor()

```
int pascal::CodeGenVisitor::newSlotFor (
    const std::string & name) [inline], [private]
```

Definition at line 461 of file [icode.hpp](#).

References [name](#), [nextSlot](#), [slotToName](#), and [varSlot](#).

Referenced by [allocateRecordFieldArrays\(\)](#), [generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [newTemp\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.83 newTemp()

```
std::string pascal::CodeGenVisitor::newTemp () [inline], [private]
```

Definition at line 472 of file [icode.hpp](#).

References [newSlotFor\(\)](#), [nextTemp](#), and [slotVar\(\)](#).

11.28.3.84 popValue()

```
std::string pascal::CodeGenVisitor::popValue () [inline], [private]
```

Definition at line 626 of file [icode.hpp](#).

References [evalStack](#).

Referenced by [eval\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.85 pushCmpResult()

```
void pascal::CodeGenVisitor::pushCmpResult (
    const std::string & a,
```

```
const std::string & b,
const char * jop) [inline], [private]
```

Definition at line 681 of file [icode.hpp](#).

References [allocReg\(\)](#), [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), [freeReg\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [newLabel\(\)](#), and [pushValue\(\)](#).

Referenced by [visit\(\)](#).

11.28.3.86 pushFloatCmpResult()

```
void pascal::CodeGenVisitor::pushFloatCmpResult (
    const std::string & aIn,
    const std::string & bIn,
    const char * jop) [inline], [private]
```

Definition at line 704 of file [icode.hpp](#).

References [allocReg\(\)](#), [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), [freeFloatReg\(\)](#), [generateRealConstantName\(\)](#), [isFloatReg\(\)](#), [isParmReg\(\)](#), [isRealNumber\(\)](#), [newLabel\(\)](#), [pushValue\(\)](#), [realConstants](#), and [usedRealConstants](#).

11.28.3.87 pushLogicalAnd()

```
void pascal::CodeGenVisitor::pushLogicalAnd (
    const std::string & a,
    const std::string & b) [inline], [private]
```

Definition at line 735 of file [icode.hpp](#).

References [allocReg\(\)](#), [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), [freeReg\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [newLabel\(\)](#), and [pushValue\(\)](#).

Referenced by [visit\(\)](#).

11.28.3.88 pushLogicalOr()

```
void pascal::CodeGenVisitor::pushLogicalOr (
    const std::string & a,
    const std::string & b) [inline], [private]
```

Definition at line 755 of file [icode.hpp](#).

References [allocReg\(\)](#), [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), [freeReg\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [newLabel\(\)](#), and [pushValue\(\)](#).

Referenced by [visit\(\)](#).

11.28.3.89 pushTri()

```
void pascal::CodeGenVisitor::pushTri (
    const char * op,
    const std::string & a,
    const std::string & b) [inline], [private]
```

Definition at line 649 of file [icode.hpp](#).

References [allocFloatReg\(\)](#), [allocReg\(\)](#), [emit2\(\)](#), [freeReg\(\)](#), [isFloatReg\(\)](#), [isParmReg\(\)](#), [isRealNumber\(\)](#), [isReg\(\)](#), and [pushValue\(\)](#).

11.28.3.90 pushValue()

```
void pascal::CodeGenVisitor::pushValue (
    const std::string & v) [inline], [private]
```

Definition at line 621 of file [icode.hpp](#).

References [evalStack](#), [isReg\(\)](#), [recordLocation\(\)](#), and [pascal::CodeGenVisitor::ValueLocation::REGISTER](#).

Referenced by [pascal::FileFunctionHandler::generateWithResult\(\)](#), [pascal::IOFunctionHandler::generateWithResult\(\)](#), [pascal::SDLFunctionHandler::generateWithResult\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pascal::StringFunctionHandler::generateWithResult\(\)](#), [pushCmpResult\(\)](#), [pushFloatCmpResult\(\)](#), [pushLogicalAnd\(\)](#), [pushLogicalOr\(\)](#), [pushTri\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.91 recordLocation()

```
void pascal::CodeGenVisitor::recordLocation (
    const std::string & var,
    ValueLocation loc) [inline], [private]
```

Definition at line 647 of file [icode.hpp](#).

References [valueLocations](#).

Referenced by [pascal::IOFunctionHandler::generate\(\)](#), [pushValue\(\)](#), and [visit\(\)](#).

11.28.3.92 registerExternalFunc()

```
void pascal::CodeGenVisitor::registerExternalFunc (
    const std::string & funcName,
    const std::string & unitName) [inline]
```

Register a function or procedure as originating from an external unit.

When generating code for a program that imports units, the compiler needs to emit qualified MXVM object references (e.g. `MathUtils.FUNC_Add`) for cross-object calls. This method records the mapping so that the code generator can prefix call targets and parameter/return registers with the correct object name.

Parameters

<i>funcName</i>	Unqualified function or procedure name
<i>unitName</i>	Name of the unit that defines the function

Definition at line 903 of file [icode.hpp](#).

References [externalFuncs](#), and [importedUnitNames](#).

Referenced by [compileFile\(\)](#).

11.28.3.93 registerImportedConst()

```
void pascal::CodeGenVisitor::registerImportedConst (
    const std::string & unitName,
    ConstDeclNode & node) [inline]
```

Register imported constant values from a dependency unit.

Definition at line 914 of file [icode.hpp](#).

References [pascal::ConstDeclNode::assignments](#), [compileTimeConstants](#), [ensureFloatConstSymbol\(\)](#), [evaluateConstantExpression\(\)](#), [importedUnitNames](#), [internString\(\)](#), [isRealNumber\(\)](#), and [realConstants](#).

Referenced by [compileFile\(\)](#).

11.28.3.94 registerImportedVar()

```
void pascal::CodeGenVisitor::registerImportedVar (
    const std::string & unitName,
    VarDeclNode & node) [inline]
```

Register imported variable metadata from a dependency unit (no alloc emitted).

Definition at line 940 of file [icode.hpp](#).

References [arrayInfo](#), [buildArrayInfoFromNode\(\)](#), [getTypeFromString\(\)](#), [pascal::VarDeclNode::identifiers](#), [importedArrayInfo](#), [importedUnitNames](#), [importedVarTypes](#), [pascal::PTR](#), [setVarType\(\)](#), and [pascal::VarDeclNode::type](#).

Referenced by [compileFile\(\)](#).

11.28.3.95 releaseTempPtr()

```
void pascal::CodeGenVisitor::releaseTempPtr (
    const std::string & ptrName) [inline], [private]
```

Definition at line 489 of file [icode.hpp](#).

References [allTempPtrs](#), and [tempPtrInUse](#).

Referenced by [freeReg\(\)](#).

11.28.3.96 resolveTypeName()

```
std::string pascal::CodeGenVisitor::resolveTypeName (
    std::string t) const [inline], [private]
```

Definition at line 1819 of file [icode.hpp](#).

References [lc\(\)](#), and [typeAliases](#).

Referenced by [buildArrayInfoFromNode\(\)](#), [generate\(\)](#), [getArrayElementSize\(\)](#), [getArrayInfoForArrayAccess\(\)](#), [getExpressionType\(\)](#), [getPointerElementSize\(\)](#), [getRecordFieldOffsetAndSize\(\)](#), [getTypeFromString\(\)](#), [getTypeSizeByName\(\)](#), [getVarRecordTypeName\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [isArrayTypeName\(\)](#), [isRecordTypeName\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.97 setSlotType()

```
void pascal::CodeGenVisitor::setSlotType (
    int slot,
    VarType t) [inline], [private]
```

Definition at line 306 of file [icode.hpp](#).

References [slotToType](#).

Referenced by [allocateRecordFieldArrays\(\)](#), [generate\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.98 setVarType()

```
void pascal::CodeGenVisitor::setVarType (
    const std::string & name,
    VarType t) [inline], [private]
```

Definition at line 305 of file [icode.hpp](#).

References [name](#), and [varTypes](#).

Referenced by [generate\(\)](#), [registerImportedVar\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.99 slotVar()

```
std::string pascal::CodeGenVisitor::slotVar (
    int slot) const [inline], [private]
```

Definition at line 372 of file [icode.hpp](#).

References [floatRegisters](#), [registers](#), and [slotToName](#).

Referenced by [allocateRecordFieldArrays\(\)](#), [generate\(\)](#), [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [newTemp\(\)](#), [storageSymbolFor\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.3.100 storageSymbolFor()

```
std::string pascal::CodeGenVisitor::storageSymbolFor (
    const std::string & mangled) [inline]
```

Definition at line 1511 of file [icode.hpp](#).

References [slotVar\(\)](#), and [varSlot](#).

Referenced by [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.101 tryGetConstNumeric()

```
bool pascal::CodeGenVisitor::tryGetConstNumeric (
    const std::string & name,
    std::string & out) [inline], [private]
```

Definition at line 1914 of file [icode.hpp](#).

References [compileTimeConstants](#), [findMangledName\(\)](#), [isFloatLiteral\(\)](#), [isIntegerLiteral\(\)](#), and [name](#).

Referenced by [foldNumeric\(\)](#), and [visit\(\)](#).

11.28.3.102 updateDataSectionInitialValue()

```
void pascal::CodeGenVisitor::updateDataSectionInitialValue (
    const std::string & varName,
    const std::string & type,
    const std::string & value) [inline]
```

Definition at line 1791 of file [icode.hpp](#).

References [constInitialValues](#).

Referenced by [allocateRecordFieldArrays\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.103 visit() [1/48]

```
void pascal::CodeGenVisitor::visit (
    AddressOfNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2566 of file [icode_visit.cpp](#).

References [allocTempPtr\(\)](#), [emit2\(\)](#), [findMangledName\(\)](#), [pascal::AddressOfNode::operand](#), and [pushValue\(\)](#).

11.28.3.104 visit() [2/48]

```
void pascal::CodeGenVisitor::visit (
    ArrayAccessNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2228 of file [icode_visit.cpp](#).

References [allocFloatReg\(\)](#), [allocReg\(\)](#), [allocTempPtr\(\)](#), [pascal::ArrayAccessNode::base](#), [pascal::DOUBLE](#), [pascal::ArrayInfo::elementsArray](#), [pascal::ArrayInfo::elementSize](#), [pascal::ArrayInfo::elementType](#), [emit2\(\)](#), [emit4\(\)](#), [ensurePtrBase\(\)](#), [eval\(\)](#), [findMangledArrayName\(\)](#), [freeReg\(\)](#), [getArrayInfoForArrayAccess\(\)](#), [getArrayNameFromBase\(\)](#), [getExpressionType\(\)](#), [getRecordFieldOffsetAndSize\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [pascal::ArrayAccessNode::index](#), [isImportedUnit\(\)](#), [isParmReg\(\)](#), [isRecordTypeName\(\)](#), [isReg\(\)](#), [pascal::ArrayInfo::lowerBound](#), [popValue\(\)](#), [pascal::PTR](#), [pushValue\(\)](#), [storageSymbolFor\(\)](#), and [pascal::STRING](#).

11.28.3.105 visit() [3/48]

```
void pascal::CodeGenVisitor::visit (
    ArrayAssignmentNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2154 of file [icode_visit.cpp](#).

References [allocFloatReg\(\)](#), [allocReg\(\)](#), [arrayInfo](#), [pascal::ArrayAssignmentNode::arrayName](#), [pascal::DOUBLE](#), [dynArrayLenSlot](#), [pascal::ArrayInfo::elementSize](#), [pascal::ArrayInfo::elementType](#), [emit2\(\)](#), [emit4\(\)](#), [emitArrayBoundsCheck\(\)](#), [emitDynArrayBoundsCheck\(\)](#), [ensurePtrBase\(\)](#), [eval\(\)](#), [freeReg\(\)](#), [getExpressionType\(\)](#), [pascal::ArrayAssignmentNode::index](#), [pascal::INT](#), [pascal::ArrayInfo::isDynamic](#), [isFloatReg\(\)](#), [isParmReg\(\)](#), [isRecordTypeName\(\)](#), [isReg\(\)](#), [pascal::ArrayInfo::lowerBound](#), [pascal::PTR](#), [pascal::RECORD](#), [slotVar\(\)](#), [storageSymbolFor\(\)](#), [pascal::ArrayInfo::upperBound](#), and [pascal::ArrayAssignmentNode::va](#).

11.28.3.106 visit() [4/48]

```
void pascal::CodeGenVisitor::visit (
    ArrayDeclarationNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1677 of file [icode_visit.cpp](#).

References [arrayInfo](#), [pascal::ArrayDeclarationNode::arrayType](#), [pascal::ArrayInfo::elementSize](#), [pascal::ArrayInfo::elementType](#), [emit3\(\)](#), [evaluateConstantExpression\(\)](#), [findMangledName\(\)](#), [functionScopedArrays](#), [getArrayElementSize\(\)](#), [getCurrentScopeName\(\)](#), [globalArrays](#), [isRecordTypeName\(\)](#), [pascal::ArrayInfo::lowerBound](#), [pascal::ArrayDeclarationNode::name](#), [newSlotFor\(\)](#), [pascal::PTR](#), [recordTypes](#), [setSlotType\(\)](#), [setVarType\(\)](#), [pascal::ArrayInfo::size](#), [slotVar\(\)](#), [updateDataSectionInitialValue\(\)](#), [pascal::ArrayInfo::upperBound](#), and [varSlot](#).

11.28.3.107 visit() [5/48]

```
void pascal::CodeGenVisitor::visit (
    ArrayTypeDeclarationNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2482 of file [icode_visit.cpp](#).

References [arrayInfo](#), [pascal::ArrayTypeDeclarationNode::arrayType](#), [buildArrayInfoFromNode\(\)](#), [lc\(\)](#), [pascal::ArrayTypeDeclarationNode](#) and [resolveTypeName\(\)](#).

11.28.3.108 visit() [6/48]

```
void pascal::CodeGenVisitor::visit (
    ArrayTypeNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1673 of file [icode_visit.cpp](#).

11.28.3.109 visit() [7/48]

```
void pascal::CodeGenVisitor::visit (
    AssignmentNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1831 of file [icode_visit.cpp](#).

References [allocatedPtrs](#), [allocFloatReg\(\)](#), [allocReg\(\)](#), [allocTempPtr\(\)](#), [pascal::CHAR](#), [currentFunctionName](#), [currentFunctionReturnSlot](#), [currentParamLocations](#), [pascal::DOUBLE](#), [dynArrayLenSlot](#), [pascal::ArrayInfo::elementSize](#), [pascal::ArrayInfo::elementType](#), [emit1\(\)](#), [emit2\(\)](#), [emit4\(\)](#), [emitArrayBoundsCheck\(\)](#), [emitDynArrayBoundsCheck\(\)](#), [emitLabel\(\)](#), [ensurePtrBase\(\)](#), [escapedTempPtrs](#), [eval\(\)](#), [pascal::AssignmentNode::expression](#), [findMangledArrayName\(\)](#), [findMangledName\(\)](#), [freeReg\(\)](#), [functionSetReturn](#), [getArrayInfoForArrayAccess\(\)](#), [getArrayNameFromBase\(\)](#), [getExpressionType\(\)](#), [getRecordFieldOffsetAndSize\(\)](#), [getTypeFromString\(\)](#), [getVarRecordTypeName\(\)](#), [getVarRecordTypeNameFrom](#), [getVarType\(\)](#), [pascal::INT](#), [pascal::ArrayInfo::isDynamic](#), [isFloatReg\(\)](#), [isImportedUnit\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [isTempPtr\(\)](#), [lc\(\)](#), [pascal::ArrayInfo::lowerBound](#), [pascal::CodeGenVisitor::ValueLocation::MEMORY](#), [pascal::VariableNode::name](#), [newLabel\(\)](#), [newSlotFor\(\)](#), [popValue\(\)](#), [pascal::PTR](#), [pascal::RECORD](#), [recordLocation\(\)](#), [recordTypes](#), [resolveTypeName\(\)](#), [setSlotType\(\)](#), [setVars](#), [slotVar\(\)](#), [storageSymbolFor\(\)](#), [pascal::STRING](#), [pascal::ArrayInfo::upperBound](#), [pascal::AssignmentNode::variable](#), [varSlot](#), [visit\(\)](#), and [withFieldScopes](#).

11.28.3.110 visit() [8/48]

```
void pascal::CodeGenVisitor::visit (
    BinaryOpNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1141 of file [icode_visit.cpp](#).

References [allocFloatReg\(\)](#), [allocReg\(\)](#), [allocTempPtr\(\)](#), [pascal::BinaryOpNode::AND](#), [pascal::BinaryOpNode::DIV](#), [pascal::BinaryOpNode::DIVIDE](#), [pascal::DOUBLE](#), [emit\(\)](#), [emit1\(\)](#), [emit2\(\)](#), [emit3\(\)](#), [emit4\(\)](#), [emit_invoke\(\)](#), [emitLabel\(\)](#), [ensureFloatConstSymbol\(\)](#), [pascal::BinaryOpNode::EQUAL](#), [eval\(\)](#), [findMangledName\(\)](#), [foldNumeric\(\)](#), [freeReg\(\)](#), [getExpressionType\(\)](#), [pascal::BinaryOpNode::GREATER](#), [pascal::BinaryOpNode::GREATER_EQUAL](#), [pascal::BinaryOpNode::IN](#), [isFloatLiteral\(\)](#), [isFloatReg\(\)](#), [isIntegerLiteral\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [pascal::BinaryOpNode::left](#), [pascal::BinaryOpNode::LESS](#), [pascal::BinaryOpNode::LESS_EQUAL](#), [pascal::BinaryOpNode::MINUS](#), [pascal::BinaryOpNode::MOD](#), [pascal::BinaryOpNode::MULTIPLY](#), [newLabel\(\)](#), [pascal::BinaryOpNode::NOT_EQUAL](#), [pascal::BinaryOpNode::operator_](#), [pascal::BinaryOpNode::OR](#), [pascal::BinaryOpNode::PLUS](#), [pascal::PTR](#), [pushCmpResult\(\)](#), [pushLogicalAnd\(\)](#), [pushLogicalOr\(\)](#), [pushValue\(\)](#), [pascal::BinaryOpNode::right](#), [setVars](#), [pascal::STRING](#), [tryGetConstNumeric\(\)](#), and [usedModules](#).

11.28.3.111 visit() [9/48]

```
void pascal::CodeGenVisitor::visit (
    BlockNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 90 of file [icode_visit.cpp](#).

References [pascal::BlockNode::compoundStatement](#), [pascal::BlockNode::declarations](#), and [generatingDeferredCode](#).

11.28.3.112 visit() [10/48]

```
void pascal::CodeGenVisitor::visit (
    BooleanNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1544 of file [icode_visit.cpp](#).

References [pushValue\(\)](#), and [pascal::BooleanNode::value](#).

11.28.3.113 visit() [11/48]

```
void pascal::CodeGenVisitor::visit (
    BreakNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2516 of file [icode_visit.cpp](#).

References [emit1\(\)](#), and [loopEndLabels](#).

11.28.3.114 visit() [12/48]

```
void pascal::CodeGenVisitor::visit (
    CaseStmtNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1625 of file [icode_visit.cpp](#).

References [pascal::CaseStmtNode::branches](#), [pascal::CHAR](#), [pascal::CaseStmtNode::elseStatement](#), [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), [eval\(\)](#), [pascal::CaseStmtNode::expression](#), [freeReg\(\)](#), [getExpressionType\(\)](#), [pascal::INT](#), [isParmReg\(\)](#), [isReg\(\)](#), and [newLabel\(\)](#).

11.28.3.115 visit() [13/48]

```
void pascal::CodeGenVisitor::visit (
    CompoundStmtNode & node) [override], [virtual]
```

Visit a compound statement block, freeing temporary pointers after each statement.

Snapshots the set of allocated temporary pointers before each statement and emits `free` instructions for any new temporaries that were not "escaped" (assigned to a named variable). This prevents memory leaks from intermediate string concatenation results and other temporary heap allocations.

Implements [pascal::ASTVisitor](#).

Definition at line 1010 of file [icode_visit.cpp](#).

References [allocatedPtrs](#), [declaredProcs](#), [emit\(\)](#), [emit1\(\)](#), [escapedTempPtrs](#), [externalFuncs](#), [findMangledFuncName\(\)](#), [funcSignatures](#), and [pascal::CompoundStmtNode::statements](#).

11.28.3.116 visit() [14/48]

```
void pascal::CodeGenVisitor::visit (
    ConstDeclNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1552 of file [icode_visit.cpp](#).

References [pascal::ConstDeclNode::assignments](#), [compileTimeConstants](#), [pascal::DOUBLE](#), [emit2\(\)](#), [ensureFloatConstSymbol\(\)](#), [eval\(\)](#), [evaluateConstantExpression\(\)](#), [freeReg\(\)](#), [getExpressionType\(\)](#), [pascal::INT](#), [internString\(\)](#), [isParmReg\(\)](#), [isRealNumber\(\)](#), [isReg\(\)](#), [mangleVariableName\(\)](#), [newSlotFor\(\)](#), [prolog](#), [pascal::PTR](#), [realConstants](#), [setSlotType\(\)](#), [setVarType\(\)](#), [slotVar\(\)](#), and [updateDataSectionInitialValue\(\)](#).

11.28.3.117 visit() [15/48]

```
void pascal::CodeGenVisitor::visit (
    ContinueNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2522 of file [icode_visit.cpp](#).

References [emit1\(\)](#), and [loopContinueLabels](#).

11.28.3.118 visit() [16/48]

```
void pascal::CodeGenVisitor::visit (
    EmptyStmtNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1548 of file [icode_visit.cpp](#).

11.28.3.119 visit() [17/48]

```
void pascal::CodeGenVisitor::visit (
    EnumTypeDeclNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2656 of file [icode_visit.cpp](#).

References [compileTimeConstants](#), [enumConstants](#), [enumTypes](#), [lc\(\)](#), [typeAliases](#), [pascal::EnumTypeDeclNode::typeName](#), and [pascal::EnumTypeDeclNode::values](#).

11.28.3.120 visit() [18/48]

```
void pascal::CodeGenVisitor::visit (
    ExitNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2488 of file [icode_visit.cpp](#).

References [currentFunctionName](#), [pascal::DOUBLE](#), [emit\(\)](#), [emit1\(\)](#), [emit2\(\)](#), [eval\(\)](#), [pascal::ExitNode::expr](#), [freeReg\(\)](#), [functionSetReturn](#), [getCurrentEndLabel\(\)](#), [getVarType\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [pascal::PTR](#), [pascal::RECORD](#), [pascal::STRING](#), and [pascal::UNKNOWN](#).

11.28.3.121 visit() [19/48]

```
void pascal::CodeGenVisitor::visit (
    FieldAccessNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1733 of file [icode_visit.cpp](#).

References [allocFloatReg\(\)](#), [allocReg\(\)](#), [allocTempPtr\(\)](#), [compileTimeConstants](#), [pascal::DOUBLE](#), [emit2\(\)](#), [emit4\(\)](#), [ensurePtrBase\(\)](#), [externalFuncs](#), [pascal::FieldAccessNode::fieldName](#), [findMangledName\(\)](#), [getTypeFromString\(\)](#), [getVarRecordTypeName\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [getVarType\(\)](#), [isImportedUnit\(\)](#), [isRecordTypeName\(\)](#), [lc\(\)](#), [popValue\(\)](#), [pascal::PTR](#), [pushValue\(\)](#), [pascal::FieldAccessNode::recordExpr](#), [recordTypes](#), [resolveTypeName\(\)](#), [pascal::STRING](#), and [visit\(\)](#).

11.28.3.122 visit() [20/48]

```
void pascal::CodeGenVisitor::visit (
    ForStmtNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1094 of file [icode_visit.cpp](#).

References [coerceToIntImmediate\(\)](#), [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), [pascal::ForStmtNode::endValue](#), [eval\(\)](#), [freeReg\(\)](#), [pascal::ForStmtNode::isDownto](#), [isParmReg\(\)](#), [isReg\(\)](#), [loopContinueLabels](#), [loopEndLabels](#), [mangleVariableName\(\)](#), [newLabel\(\)](#), [newSlotFor\(\)](#), [slotVar\(\)](#), [pascal::ForStmtNode::startValue](#), [pascal::ForStmtNode::statement](#), and [pascal::ForStmtNode::variable](#).

11.28.3.123 visit() [21/48]

```
void pascal::CodeGenVisitor::visit (
    FuncCallNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 727 of file [icode_visit.cpp](#).

References [allocFloatReg\(\)](#), [allocReg\(\)](#), [allocTempPtr\(\)](#), [pascal::FuncCallNode::arguments](#), [arrayInfo](#), [builtinRegistry](#), [currentFuncLocalSlots](#), [pascal::DOUBLE](#), [dynArrayLenSlot](#), [emit1\(\)](#), [emit2\(\)](#), [eval\(\)](#), [externalFuncs](#), [findMangledArrayName\(\)](#), [findMangledFuncName\(\)](#), [floatRegisters](#), [freeReg\(\)](#), [funcSignatures](#), [getExpressionType\(\)](#), [pascal::INT](#), [isParmReg\(\)](#), [isReg\(\)](#), [pascal::FuncCallNode::name](#), [pascal::PTR](#), [ptrRegisters](#), [pushValue\(\)](#), [pascal::RECORD](#), [regInUse](#), [registers](#), [slotVar\(\)](#), and [pascal::STRING](#).

11.28.3.124 visit() [22/48]

```
void pascal::CodeGenVisitor::visit (
    FuncDeclNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 899 of file [icode_visit.cpp](#).

References [pascal::FuncDeclNode::block](#), [declaredFuncs](#), [deferredFuncs](#), [funcSignatures](#), [getTypeFromString\(\)](#), [mangleVariableName\(\)](#), [pascal::FuncDeclNode::name](#), [pascal::FuncDeclNode::parameters](#), [pascal::CodeGenVisitor::FuncInfo::param](#), [pascal::CodeGenVisitor::FuncInfo::returnType](#), [pascal::FuncDeclNode::returnType](#), [scopeHierarchy](#), and [setVarType\(\)](#).

11.28.3.125 visit() [23/48]

```
void pascal::CodeGenVisitor::visit (
    GotoStmtNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2597 of file [icode_visit.cpp](#).

References [emit1\(\)](#), [gotoLabels](#), [pascal::GotoStmtNode::label](#), and [newLabel\(\)](#).

11.28.3.126 visit() [24/48]

```
void pascal::CodeGenVisitor::visit (
    IfStmtNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1050 of file [icode_visit.cpp](#).

References [pascal::IfStmtNode::condition](#), [pascal::IfStmtNode::elseStatement](#), [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), [eval\(\)](#), [freeReg\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [newLabel\(\)](#), and [pascal::IfStmtNode::thenStatement](#).

11.28.3.127 visit() [25/48]

```
void pascal::CodeGenVisitor::visit (
    LabelStmtNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2609 of file [icode_visit.cpp](#).

References [emitLabel\(\)](#), [gotoLabels](#), [pascal::LabelStmtNode::label](#), [newLabel\(\)](#), and [pascal::LabelStmtNode::statement](#).

11.28.3.128 visit() [26/48]

```
void pascal::CodeGenVisitor::visit (
    NilNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2528 of file [icode_visit.cpp](#).

References [pushValue\(\)](#).

11.28.3.129 visit() [27/48]

```
void pascal::CodeGenVisitor::visit (
    NumberNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1529 of file [icode_visit.cpp](#).

References [allocFloatReg\(\)](#), [emit2\(\)](#), [ensureFloatConstSymbol\(\)](#), [pascal::NumberNode::isReal](#), [isRealNumber\(\)](#), [pushValue\(\)](#), and [pascal::NumberNode::value](#).

11.28.3.130 visit() [28/48]

```
void pascal::CodeGenVisitor::visit (
    ParameterNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 967 of file [icode_visit.cpp](#).

References [currentParamTypes](#), [pascal::DOUBLE](#), [pascal::ParameterNode::identifiers](#), [pascal::INT](#), [lc\(\)](#), [newSlotFor\(\)](#), [pascal::PTR](#), [pascal::RECORD](#), [recordTypes](#), [resolveTypeName\(\)](#), [setSlotType\(\)](#), [setVarType\(\)](#), [pascal::STRING](#), [pascal::ParameterNode::type](#), and [varRecordType](#).

11.28.3.131 visit() [29/48]

```
void pascal::CodeGenVisitor::visit (
    PointerDerefNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2536 of file [icode_visit.cpp](#).

References [allocFloatReg\(\)](#), [allocReg\(\)](#), [allocTempPtr\(\)](#), [pascal::DOUBLE](#), [emit4\(\)](#), [ensurePtrBase\(\)](#), [eval\(\)](#), [freeReg\(\)](#), [getPointerDerefType\(\)](#), [pascal::INT](#), [isParmReg\(\)](#), [isReg\(\)](#), [pascal::PointerDerefNode::pointer](#), [pascal::PTR](#), [pushValue\(\)](#), and [pascal::STRING](#).

11.28.3.132 visit() [30/48]

```
void pascal::CodeGenVisitor::visit (
    PointerTypeNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2532 of file [icode_visit.cpp](#).

11.28.3.133 visit() [31/48]

```
void pascal::CodeGenVisitor::visit (
    ProcCallNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 346 of file [icode_visit.cpp](#).

References [allocReg\(\)](#), [allocTempPtr\(\)](#), [pascal::ProcCallNode::arguments](#), [arrayInfo](#), [builtinRegistry](#), [currentFuncLocalSlots](#), [pascal::DOUBLE](#), [dynArrayLenSlot](#), [pascal::ArrayInfo::elementSize](#), [pascal::ArrayInfo::elementType](#), [emit1\(\)](#), [emit2\(\)](#), [emit3\(\)](#), [emit4\(\)](#), [ensurePtrBase\(\)](#), [eval\(\)](#), [externalFuncs](#), [findMangledArrayName\(\)](#), [findMangledFuncName\(\)](#), [findMangledName\(\)](#), [floatRegisters](#), [freeReg\(\)](#), [funcSignatures](#), [getArrayInfoForArrayAccess\(\)](#), [getArrayNameFromBase\(\)](#), [getExpressionType\(\)](#), [getPointerBaseTypeName\(\)](#), [getPointerElementSize\(\)](#), [getTypeSizeByName\(\)](#), [getVarRecordTypeName\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [isParmReg\(\)](#), [isRecordTypeName\(\)](#), [isReg\(\)](#), [lc\(\)](#), [pascal::ArrayInfo::lowerBound](#), [pascal::ProcCallNode::name](#), [popValue\(\)](#), [pascal::PTR](#), [ptrRegisters](#), [pascal::RECORD](#), [recordTypes](#), [regInUse](#), [registers](#), [resolveTypeName\(\)](#), [slotVar\(\)](#), [storageSymbolFor\(\)](#), and [pascal::STRING](#).

11.28.3.134 visit() [32/48]

```
void pascal::CodeGenVisitor::visit (
    ProcDeclNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 936 of file [icode_visit.cpp](#).

References [pascal::ProcDeclNode::block](#), [declaredProcs](#), [deferredProcs](#), [mangleVariableName\(\)](#), [pascal::ProcDeclNode::name](#), and [scopeHierarchy](#).

11.28.3.135 visit() [33/48]

```
void pascal::CodeGenVisitor::visit (
    ProgramNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 17 of file [icode_visit.cpp](#).

References [pascal::ProgramNode::block](#), [name](#), [pascal::ProgramNode::name](#), [objectDeps](#), [usedModules](#), and [pascal::ProgramNode::uses](#).

Referenced by [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.3.136 visit() [34/48]

```
void pascal::CodeGenVisitor::visit (
    RecordDeclarationNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2323 of file [icode_visit.cpp](#).

References [pascal::CodeGenVisitor::RecordField::arrayInfo](#), [buildArrayInfoFromNode\(\)](#), [pascal::ArrayInfo::elementSize](#), [pascal::CodeGenVisitor::RecordTypeInfo::fields](#), [getTypeSizeByName\(\)](#), [pascal::CodeGenVisitor::RecordField::isArray](#), [isRecordTypeName\(\)](#), [lc\(\)](#), [pascal::CodeGenVisitor::RecordField::name](#), [pascal::RecordDeclarationNode::name](#), [pascal::CodeGenVisitor::RecordTypeInfo::nameToIndex](#), [pascal::CodeGenVisitor::RecordField::offset](#), [pascal::RecordDeclarationNode::recordTypes](#), [resolveTypeName\(\)](#), [pascal::ArrayInfo::size](#), [pascal::CodeGenVisitor::RecordField::size](#), [pascal::CodeGenVisitor::RecordField::typeName](#), [typeAliases](#), and [pascal::CodeGenVisitor::RecordField::typeName](#).

11.28.3.137 visit() [35/48]

```
void pascal::CodeGenVisitor::visit (
    RecordTypeNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2319 of file [icode_visit.cpp](#).

11.28.3.138 visit() [36/48]

```
void pascal::CodeGenVisitor::visit (
    RepeatStmtNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1602 of file [icode_visit.cpp](#).

References [pascal::RepeatStmtNode::condition](#), [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), [eval\(\)](#), [freeReg\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [loopContinueLabels](#), [loopEndLabels](#), [newLabel\(\)](#), and [pascal::RepeatStmtNode::statements](#).

11.28.3.139 visit() [37/48]

```
void pascal::CodeGenVisitor::visit (
    SetLiteralNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2623 of file [icode_visit.cpp](#).

References [allocReg\(\)](#), [allocTempPtr\(\)](#), [pascal::SetLiteralNode::elements](#), [emit1\(\)](#), [emit2\(\)](#), [emit3\(\)](#), [emit4\(\)](#), [emitLabel\(\)](#), [eval\(\)](#), [freeReg\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [newLabel\(\)](#), and [pushValue\(\)](#).

11.28.3.140 visit() [38/48]

```
void pascal::CodeGenVisitor::visit (
    SetTypeNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2652 of file [icode_visit.cpp](#).

11.28.3.141 visit() [39/48]

```
void pascal::CodeGenVisitor::visit (
    SimpleTypeNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2478 of file [icode_visit.cpp](#).

11.28.3.142 visit() [40/48]

```
void pascal::CodeGenVisitor::visit (
    StringNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1539 of file [icode_visit.cpp](#).

References [internString\(\)](#), [pushValue\(\)](#), and [pascal::StringNode::value](#).

11.28.3.143 visit() [41/48]

```
void pascal::CodeGenVisitor::visit (
    TypeAliasNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 961 of file [icode_visit.cpp](#).

References [pascal::TypeAliasNode::baseType](#), [lc\(\)](#), [typeAliases](#), and [pascal::TypeAliasNode::typeName](#).

11.28.3.144 visit() [42/48]

```
void pascal::CodeGenVisitor::visit (
    TypeDeclNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2472 of file [icode_visit.cpp](#).

References [pascal::TypeDeclNode::typeDeclarations](#).

11.28.3.145 visit() [43/48]

```
void pascal::CodeGenVisitor::visit (
    UnaryOpNode & node) [override], [virtual]
```


Implements [pascal::ASTVisitor](#).

Definition at line 1439 of file [icode_visit.cpp](#).

References [allocReg\(\)](#), [emit\(\)](#), [emit2\(\)](#), [eval\(\)](#), [pascal::UnaryOpNode::MINUS](#), [pascal::UnaryOpNode::NOT](#), [pascal::UnaryOpNode::operand](#), [pascal::UnaryOpNode::operator_](#), [pascal::UnaryOpNode::PLUS](#), and [pushValue\(\)](#).

11.28.3.146 visit() [44/48]

```
void pascal::CodeGenVisitor::visit (
    UnitNode & node) [override], [virtual]
```

Generate MXVM object code from a Pascal unit.

Implements [pascal::ASTVisitor](#).

Definition at line 34 of file [icode_visit.cpp](#).

References [emit\(\)](#), [emitLabel\(\)](#), [funcSignatures](#), [getTypeFromString\(\)](#), [pascal::UnitNode::implDecls](#), [pascal::UnitNode::interfaceDecls](#), [isUnit](#), [name](#), [pascal::UnitNode::name](#), [objectDeps](#), [pascal::CodeGenVisitor::FuncInfo::paramTypes](#), [pascal::CodeGenVisitor::FuncInfo::setVarType\(\)](#), [usedModules](#), and [pascal::UnitNode::uses](#).

11.28.3.147 visit() [45/48]

```
void pascal::CodeGenVisitor::visit (
    VarDeclNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 119 of file [icode_visit.cpp](#).

References [allocateRecordFieldArrays\(\)](#), [allocReg\(\)](#), [arrayInfo](#), [buildArrayInfoFromNode\(\)](#), [currentFuncLocalSlots](#), [currentFunctionName](#), [pascal::DOUBLE](#), [dynArrayLenSlot](#), [emit1\(\)](#), [emit2\(\)](#), [emit3\(\)](#), [emit4\(\)](#), [emitLabel\(\)](#), [fileVarNames](#), [fileVars](#), [freeReg\(\)](#), [functionScopedArrays](#), [getCurrentScopeName\(\)](#), [getTypeFromString\(\)](#), [globalArrays](#), [pascal::VarDeclNode::identifiers](#), [pascal::INT](#), [lc\(\)](#), [mangleVariableName\(\)](#), [newLabel\(\)](#), [newSlotFor\(\)](#), [pointerBaseType](#), [pascal::PTR](#), [pascal::RECORD](#), [recordsToFreeInScope](#), [recordTypes](#), [resolveTypeName\(\)](#), [setSlotType\(\)](#), [setVars](#), [setVarType\(\)](#), [slotVar\(\)](#), [pascal::VarDeclNode::type](#), [updateDataSectionInitialValue\(\)](#), [varRecordType](#), and [varSlot](#).

11.28.3.148 visit() [46/48]

```
void pascal::CodeGenVisitor::visit (
    VariableNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1461 of file [icode_visit.cpp](#).

References [compileTimeConstants](#), [currentFunctionName](#), [currentFunctionReturnSlot](#), [currentParamLocations](#), [pascal::DOUBLE](#), [externalFuncs](#), [findMangledName\(\)](#), [funcSignatures](#), [getVarType\(\)](#), [pascal::INT](#), [lc\(\)](#), [pascal::VariableNode::name](#), [newSlotFor\(\)](#), [pascal::PTR](#), [pushValue\(\)](#), [pascal::RECORD](#), [setSlotType\(\)](#), [slotVar\(\)](#), [pascal::STRING](#), [varSlot](#), [visit\(\)](#), and [withFieldScopes](#).

11.28.3.149 visit() [47/48]

```
void pascal::CodeGenVisitor::visit (
    WhileStmtNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 1075 of file [icode_visit.cpp](#).

References [pascal::WhileStmtNode::condition](#), [emit1\(\)](#), [emit2\(\)](#), [emitLabel\(\)](#), [eval\(\)](#), [freeReg\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [loopContinueLabels](#), [loopEndLabels](#), [newLabel\(\)](#), and [pascal::WhileStmtNode::statement](#).

11.28.3.150 visit() [48/48]

```
void pascal::CodeGenVisitor::visit (
    WithStmtNode & node) [override], [virtual]
```

Implements [pascal::ASTVisitor](#).

Definition at line 2577 of file [icode_visit.cpp](#).

References [getVarRecordTypeName\(\)](#), [lc\(\)](#), [recordTypes](#), [pascal::WithStmtNode::recordVar](#), [resolveTypeName\(\)](#), [pascal::WithStmtNode::statement](#), and [withFieldScopes](#).

11.28.3.151 writeTo()

```
void pascal::CodeGenVisitor::writeTo (
    std::ostream & out) const [inline]
```

Write the complete MXVM program to a stream.

Parameters

<i>out</i>	Output stream
------------	---------------

Emits the program header, module section, data section (registers, variables, string literals, real constants), and code section.

Definition at line 1324 of file [icode.hpp](#).

References [allTempPtrs](#), [bufferStringVars](#), [pascal::CHAR](#), [constInitialValues](#), [pascal::DOUBLE](#), [endsWithColon\(\)](#), [escapeStringForMxvm\(\)](#), [floatRegisters](#), [instructions](#), [isPtrReg\(\)](#), [isRegisterSlot\(\)](#), [isTempVar\(\)](#), [isUnit](#), [name](#), [needsEmptyString](#), [nextSlot](#), [objectDeps](#), [prolog](#), [pascal::PTR](#), [ptrRegisters](#), [realConstants](#), [registers](#), [slotToType](#), [slotVar\(\)](#), [pascal::STRING](#), [stringLiterals](#), [tempPtrByScope](#), [usedModules](#), and [usedStrings](#).

Referenced by [compileFile\(\)](#).

11.28.4 Friends And Related Symbol Documentation

11.28.4.1 FileFunctionHandler

```
friend class FileFunctionHandler [friend]
```

Definition at line 887 of file [icode.hpp](#).

References [FileFunctionHandler](#).

Referenced by [FileFunctionHandler](#).

11.28.4.2 IOFunctionHandler

```
friend class IOFunctionHandler [friend]
```

Definition at line 882 of file [icode.hpp](#).

References [IOFunctionHandler](#).

Referenced by [IOFunctionHandler](#).

11.28.4.3 MathFunctionHandler

```
friend class MathFunctionHandler [friend]
```

Definition at line 883 of file [icode.hpp](#).

References [MathFunctionHandler](#).

Referenced by [MathFunctionHandler](#).

11.28.4.4 SDLFunctionHandler

```
friend class SDLFunctionHandler [friend]
```

Definition at line 885 of file [icode.hpp](#).

References [SDLFunctionHandler](#).

Referenced by [SDLFunctionHandler](#).

11.28.4.5 StdFunctionHandler

```
friend class StdFunctionHandler [friend]
```

Definition at line 884 of file [icode.hpp](#).

References [StdFunctionHandler](#).

Referenced by [StdFunctionHandler](#).

11.28.4.6 StringFunctionHandler

```
friend class StringFunctionHandler [friend]
```

Definition at line 886 of file [icode.hpp](#).
 References [StringFunctionHandler](#).
 Referenced by [StringFunctionHandler](#).

11.28.5 Member Data Documentation

11.28.5.1 allocatedPtrs

`std::unordered_set<std::string> pascal::CodeGenVisitor::allocatedPtrs [private]`
 Definition at line 1794 of file [icode.hpp](#).
 Referenced by [emit3\(\)](#), [emitFree\(\)](#), [generate\(\)](#), [markAllocatedPtr\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.2 allTempPtrs

`std::vector<std::string> pascal::CodeGenVisitor::allTempPtrs [private]`
 Definition at line 205 of file [icode.hpp](#).
 Referenced by [allocTempPtr\(\)](#), [freeTempPtr\(\)](#), [isTempPtr\(\)](#), [releaseTempPtr\(\)](#), and [writeTo\(\)](#).

11.28.5.3 arrayInfo

`std::unordered_map<std::string, ArrayInfo> pascal::CodeGenVisitor::arrayInfo [private]`
 Definition at line 1799 of file [icode.hpp](#).
 Referenced by [generate\(\)](#), [getArrayInfoForArrayAccess\(\)](#), [getTypeFromString\(\)](#), [isArrayTypeName\(\)](#), [registerImportedVar\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.4 bufferStringVars

`std::map<std::string, size_t> pascal::CodeGenVisitor::bufferStringVars [private]`
 Definition at line 202 of file [icode.hpp](#).
 Referenced by [writeTo\(\)](#).

11.28.5.5 builtinRegistry

[BuiltinFunctionRegistry](#) `pascal::CodeGenVisitor::builtinRegistry`
 registry of built-in function handlers
 Definition at line 889 of file [icode.hpp](#).
 Referenced by [getExpressionType\(\)](#), [initializeBuiltins\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.6 compileTimeConstants

`std::unordered_map<std::string, std::string> pascal::CodeGenVisitor::compileTimeConstants [private]`
 Definition at line 180 of file [icode.hpp](#).
 Referenced by [evaluateConstantExpression\(\)](#), [generate\(\)](#), [registerImportedConst\(\)](#), [tryGetConstNumeric\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.7 constInitialValues

`std::map<std::string, std::pair<std::string, std::string> > pascal::CodeGenVisitor::constInitialValues [private]`
 Definition at line 201 of file [icode.hpp](#).
 Referenced by [updateDataSectionInitialValue\(\)](#), and [writeTo\(\)](#).

11.28.5.8 currentFuncLocalSlots

`std::vector<std::string> pascal::CodeGenVisitor::currentFuncLocalSlots [private]`
 Definition at line 334 of file [icode.hpp](#).
 Referenced by [generate\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.9 currentFunctionName

`std::string pascal::CodeGenVisitor::currentFunctionName [private]`

Definition at line 332 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [getCurrentEndLabel\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.10 currentFunctionReturnSlot

`std::string pascal::CodeGenVisitor::currentFunctionReturnSlot [private]`

slot variable that holds the return value until FUNC_END

Definition at line 333 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.11 currentParamLocations

`std::map<std::string, std::string> pascal::CodeGenVisitor::currentParamLocations [private]`

Definition at line 203 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.12 currentParamTypes

`std::unordered_map<std::string, std::string> pascal::CodeGenVisitor::currentParamTypes [private]`

Definition at line 183 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [getVarRecordTypeName\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), and [visit\(\)](#).

11.28.5.13 declaredFuncs

`std::unordered_map<std::string, bool> pascal::CodeGenVisitor::declaredFuncs [private]`

Definition at line 350 of file [icode.hpp](#).

Referenced by [findMangledFuncName\(\)](#), [generate\(\)](#), [getCurrentEndLabel\(\)](#), and [visit\(\)](#).

11.28.5.14 declaredProcs

`std::unordered_map<std::string, bool> pascal::CodeGenVisitor::declaredProcs [private]`

Definition at line 349 of file [icode.hpp](#).

Referenced by [findMangledFuncName\(\)](#), [getCurrentEndLabel\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.15 deferredFuncs

`std::vector<std::pair<FuncDeclNode *, std::vector<std::string> > > pascal::CodeGenVisitor::↵
deferredFuncs [private]`

Definition at line 331 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [getCurrentScopeName\(\)](#), and [visit\(\)](#).

11.28.5.16 deferredProcs

`std::vector<std::pair<ProcDeclNode *, std::vector<std::string> > > pascal::CodeGenVisitor::↵
deferredProcs [private]`

Definition at line 204 of file [icode.hpp](#).

Referenced by [generate\(\)](#), and [visit\(\)](#).

11.28.5.17 dynArrayLenSlot

`std::unordered_map<std::string, int> pascal::CodeGenVisitor::dynArrayLenSlot [private]`

mangled array name → slot of companion length variable

Definition at line 1800 of file [icode.hpp](#).

Referenced by [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.18 enumConstants

`std::unordered_map<std::string, int> pascal::CodeGenVisitor::enumConstants`

Enum value → ordinal mapping (lowercased name → integer).

Definition at line 1492 of file [icode.hpp](#).

Referenced by [visit\(\)](#).

11.28.5.19 enumTypes

`std::unordered_map<std::string, std::vector<std::string> > pascal::CodeGenVisitor::enumTypes`

Enum type name → ordered list of value names.

Definition at line 1494 of file [icode.hpp](#).

Referenced by [visit\(\)](#).

11.28.5.20 escapedTempPtrs

`std::unordered_set<std::string> pascal::CodeGenVisitor::escapedTempPtrs [private]`

Definition at line 1795 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.21 evalStack

`std::vector<std::string> pascal::CodeGenVisitor::evalStack [private]`

Definition at line 343 of file [icode.hpp](#).

Referenced by [eval\(\)](#), [popValue\(\)](#), and [pushValue\(\)](#).

11.28.5.22 externalFuncs

`std::unordered_map<std::string, std::string> pascal::CodeGenVisitor::externalFuncs [private]`

maps function/proc name to source unit name

Definition at line 187 of file [icode.hpp](#).

Referenced by [registerExternalFunc\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.23 fileVarNames

`std::unordered_map<std::string, std::string> pascal::CodeGenVisitor::fileVarNames`

Maps file variable name to companion filename variable name.

Definition at line 1500 of file [icode.hpp](#).

Referenced by [pascal::FileFunctionHandler::generate\(\)](#), and [visit\(\)](#).

11.28.5.24 fileVars

`std::unordered_set<std::string> pascal::CodeGenVisitor::fileVars`

Set of variable names that are file types (pointer to FILE).

Definition at line 1498 of file [icode.hpp](#).

Referenced by [pascal::IOFunctionHandler::generate\(\)](#), and [visit\(\)](#).

11.28.5.25 floatRegInUse

`std::vector<bool> pascal::CodeGenVisitor::floatRegInUse [private]`

Definition at line 320 of file [icode.hpp](#).

Referenced by [allocFloatReg\(\)](#), [expandFloatRegisters\(\)](#), [freeFloatReg\(\)](#), [generate\(\)](#), and [initializeFloatRegisters\(\)](#).

11.28.5.26 floatRegisters

`std::vector<std::string> pascal::CodeGenVisitor::floatRegisters [private]`

Definition at line 319 of file [icode.hpp](#).

Referenced by [allocFloatReg\(\)](#), [expandFloatRegisters\(\)](#), [freeFloatReg\(\)](#), [generate\(\)](#), [initializeFloatRegisters\(\)](#), [isFloatReg\(\)](#), [isReg\(\)](#), [slotVar\(\)](#), [visit\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.5.27 funcSignatures

`std::unordered_map<std::string, FuncInfo> pascal::CodeGenVisitor::funcSignatures [private]`

Definition at line 269 of file [icode.hpp](#).

Referenced by [getExpressionType\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.28 functionScopedArrays

`std::unordered_map<std::string, std::vector<std::string> > pascal::CodeGenVisitor::functionScopedArrays [private]`

Definition at line 192 of file [icode.hpp](#).

Referenced by [allocateRecordFieldArrays\(\)](#), [generate\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.29 functionSetReturn

`bool pascal::CodeGenVisitor::functionSetReturn = false [private]`

Definition at line 335 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.30 generatingDeferredCode

`bool pascal::CodeGenVisitor::generatingDeferredCode = false [private]`

Definition at line 351 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [getCurrentScopeName\(\)](#), and [visit\(\)](#).

11.28.5.31 globalArrays

`std::vector<std::string> pascal::CodeGenVisitor::globalArrays [private]`

Definition at line 191 of file [icode.hpp](#).

Referenced by [allocateRecordFieldArrays\(\)](#), [generate\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.32 gotoLabels

`std::unordered_map<std::string, std::string> pascal::CodeGenVisitor::gotoLabels [private]`

Maps user-declared goto label numbers to generated MXVM labels.

Definition at line 2148 of file [icode.hpp](#).

Referenced by [visit\(\)](#), and [visit\(\)](#).

11.28.5.33 importedArrayInfo

`std::unordered_map<std::string, ArrayInfo> pascal::CodeGenVisitor::importedArrayInfo [private]`

array metadata for imported unit vars

Definition at line 189 of file [icode.hpp](#).

Referenced by [generate\(\)](#), and [registerImportedVar\(\)](#).

11.28.5.34 importedUnitNames

`std::unordered_set<std::string> pascal::CodeGenVisitor::importedUnitNames [private]`

names of units imported via uses clause

Definition at line 188 of file [icode.hpp](#).

Referenced by [isImportedUnit\(\)](#), [registerExternalFunc\(\)](#), [registerImportedConst\(\)](#), and [registerImportedVar\(\)](#).

11.28.5.35 importedVarTypes

`std::unordered_map<std::string, VarType> pascal::CodeGenVisitor::importedVarTypes [private]`

var types for imported unit vars

Definition at line 190 of file [icode.hpp](#).

Referenced by [generate\(\)](#), and [registerImportedVar\(\)](#).

11.28.5.36 instructions

```
std::vector<std::string> pascal::CodeGenVisitor::instructions [private]
```

Definition at line 342 of file [icode.hpp](#).

Referenced by [emit\(\)](#), [emit3\(\)](#), [emitLabel\(\)](#), [generate\(\)](#), and [writeTo\(\)](#).

11.28.5.37 isUnit

```
bool pascal::CodeGenVisitor::isUnit = false [private]
```

true when generating code for a unit (object output)

Definition at line 186 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.5.38 labelCounter

```
int pascal::CodeGenVisitor::labelCounter = 0 [private]
```

Definition at line 338 of file [icode.hpp](#).

Referenced by [newLabel\(\)](#).

11.28.5.39 loopContinueLabels

```
std::vector<std::string> pascal::CodeGenVisitor::loopContinueLabels [private]
```

Definition at line 2143 of file [icode.hpp](#).

Referenced by [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.40 loopEndLabels

```
std::vector<std::string> pascal::CodeGenVisitor::loopEndLabels [private]
```

Definition at line 2142 of file [icode.hpp](#).

Referenced by [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.41 name

```
std::string pascal::CodeGenVisitor::name = "App"
```

program name emitted in the output header

Definition at line 977 of file [icode.hpp](#).

Referenced by [allocReg\(\)](#), [findMangledArrayName\(\)](#), [findMangledFuncName\(\)](#), [findMangledName\(\)](#), [funcLabel\(\)](#), [generate\(\)](#), [getVarType\(\)](#), [isFloatReg\(\)](#), [isImportedUnit\(\)](#), [isParmReg\(\)](#), [isPtrReg\(\)](#), [isReg\(\)](#), [isStringVar\(\)](#), [isTempPtr\(\)](#), [isTempVar\(\)](#), [newSlotFor\(\)](#), [setVarType\(\)](#), [tryGetConstNumeric\(\)](#), [visit\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.5.42 needsEmptyString

```
bool pascal::CodeGenVisitor::needsEmptyString = false [private]
```

Definition at line 273 of file [icode.hpp](#).

Referenced by [emptyString\(\)](#), and [writeTo\(\)](#).

11.28.5.43 nextFloatRegIndex

```
int pascal::CodeGenVisitor::nextFloatRegIndex = 0 [private]
```

Definition at line 321 of file [icode.hpp](#).

11.28.5.44 nextSlot

```
int pascal::CodeGenVisitor::nextSlot = 0 [private]
```

Definition at line 336 of file [icode.hpp](#).

Referenced by [newSlotFor\(\)](#), and [writeTo\(\)](#).

11.28.5.45 nextSpillReg

```
int pascal::CodeGenVisitor::nextSpillReg = 0 [private]
```

Definition at line 315 of file [icode.hpp](#).

Referenced by [allocReg\(\)](#).

11.28.5.46 nextTemp

```
int pascal::CodeGenVisitor::nextTemp = 0 [private]
```

Definition at line 337 of file [icode.hpp](#).

Referenced by [allocTempPtr\(\)](#), and [newTemp\(\)](#).

11.28.5.47 objectDeps

```
std::vector<std::string> pascal::CodeGenVisitor::objectDeps [private]
```

unit/object dependencies for section object

Definition at line 185 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [visit\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.5.48 pointerBaseType

```
std::unordered_map<std::string, std::string> pascal::CodeGenVisitor::pointerBaseType [private]
```

Definition at line 2003 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [getPointerBaseTypeName\(\)](#), and [visit\(\)](#).

11.28.5.49 prolog

```
std::vector<std::string> pascal::CodeGenVisitor::prolog [private]
```

Definition at line 341 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.5.50 ptrRegInUse

```
std::vector<bool> pascal::CodeGenVisitor::ptrRegInUse [private]
```

Definition at line 329 of file [icode.hpp](#).

Referenced by [CodeGenVisitor\(\)](#), [allocPtrReg\(\)](#), [freePtrReg\(\)](#), and [generate\(\)](#).

11.28.5.51 ptrRegisters

```
const std::vector<std::string> pascal::CodeGenVisitor::ptrRegisters [private]
```

Initial value:

```
= {
"arg0", "arg1", "arg2", "arg3", "arg4", "arg5", "arg6", "arg7", "arg8", "arg9"}
```

Definition at line 317 of file [icode.hpp](#).

Referenced by [CodeGenVisitor\(\)](#), [allocPtrReg\(\)](#), [freePtrReg\(\)](#), [generate\(\)](#), [isParmReg\(\)](#), [isPtrReg\(\)](#), [visit\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.5.52 realConstantCounter

```
int pascal::CodeGenVisitor::realConstantCounter = 0 [private]
```

Definition at line 182 of file [icode.hpp](#).

Referenced by [generateRealConstantName\(\)](#).

11.28.5.53 realConstants

```
std::unordered_map<std::string, std::string> pascal::CodeGenVisitor::realConstants [private]
```

Definition at line 1797 of file [icode.hpp](#).

Referenced by [coerceToIntImmediate\(\)](#), [ensureFloatConstSymbol\(\)](#), [generate\(\)](#), [pushFloatCmpResult\(\)](#), [registerImportedConst\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.5.54 recordsToFreeInScope

```
std::unordered_map<std::string, std::vector<std::string> > pascal::CodeGenVisitor::recordsToFreeInScope [private]
```

Definition at line 347 of file [icode.hpp](#).

Referenced by [generate\(\)](#), and [visit\(\)](#).

11.28.5.55 recordTypes

```
std::unordered_map<std::string, RecordTypeInfo> pascal::CodeGenVisitor::recordTypes [private]
```

Definition at line 2001 of file [icode.hpp](#).

Referenced by [allocateRecordFieldArrays\(\)](#), [generate\(\)](#), [getArrayElementSize\(\)](#), [getArrayInfoForArrayAccess\(\)](#), [getExpressionType\(\)](#), [getRecordFieldOffsetAndSize\(\)](#), [getRecordTypeSize\(\)](#), [getTypeSizeByName\(\)](#), [getVarRecordTypeNameFromExpression\(\)](#), [isRecordTypeName\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.56 regInUse

```
std::vector<bool> pascal::CodeGenVisitor::regInUse [private]
```

Definition at line 328 of file [icode.hpp](#).

Referenced by [CodeGenVisitor\(\)](#), [allocReg\(\)](#), [freeReg\(\)](#), [generate\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.57 registers

```
std::vector<std::string> pascal::CodeGenVisitor::registers = {"rax", "rbx", "rcx", "rdx", "rsi", "rdi", "r8", "r9", "r10", "r11", "r12", "r13", "r14", "r15"} [private]
```

Definition at line 316 of file [icode.hpp](#).

Referenced by [CodeGenVisitor\(\)](#), [allocReg\(\)](#), [freeReg\(\)](#), [generate\(\)](#), [isParmReg\(\)](#), [isReg\(\)](#), [slotVar\(\)](#), [visit\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.5.58 scopeHierarchy

```
std::vector<std::string> pascal::CodeGenVisitor::scopeHierarchy [private]
```

Definition at line 193 of file [icode.hpp](#).

Referenced by [findMangledArrayName\(\)](#), [findMangledFuncName\(\)](#), [findMangledName\(\)](#), [generate\(\)](#), [getCurrentScopeName\(\)](#), [mangleVariableName\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.59 scopeStack

```
std::vector<Scope> pascal::CodeGenVisitor::scopeStack [private]
```

Definition at line 199 of file [icode.hpp](#).

Referenced by [generate\(\)](#).

11.28.5.60 setVars

```
std::unordered_set<std::string> pascal::CodeGenVisitor::setVars
```

Set of variable names that are set types (allocated as 256-byte bitsets).

Definition at line 1496 of file [icode.hpp](#).

Referenced by [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.61 slotToName

```
std::unordered_map<int, std::string> pascal::CodeGenVisitor::slotToName [private]
```

Definition at line 346 of file [icode.hpp](#).

Referenced by [newSlotFor\(\)](#), and [slotVar\(\)](#).

11.28.5.62 slotToType

```
std::unordered_map<int, VarType> pascal::CodeGenVisitor::slotToType [private]
```

Definition at line 271 of file [icode.hpp](#).

Referenced by [ensurePtrBase\(\)](#), [getVarType\(\)](#), [setSlotType\(\)](#), and [writeTo\(\)](#).

11.28.5.63 stringLiterals

```
std::vector<std::pair<std::string, std::string> > pascal::CodeGenVisitor::stringLiterals
[private]
```

Definition at line 344 of file [icode.hpp](#).

Referenced by [internString\(\)](#), and [writeTo\(\)](#).

11.28.5.64 tempPtrByScope

```
std::unordered_map<std::string, std::vector<std::string> > pascal::CodeGenVisitor::tempPtrByScope
[private]
```

Definition at line 348 of file [icode.hpp](#).

Referenced by [addTempPtr\(\)](#), [allocTempPtr\(\)](#), [generate\(\)](#), and [writeTo\(\)](#).

11.28.5.65 tempPtrInUse

```
std::vector<bool> pascal::CodeGenVisitor::tempPtrInUse [private]
```

Definition at line 206 of file [icode.hpp](#).

Referenced by [allocTempPtr\(\)](#), [freeTempPtr\(\)](#), and [releaseTempPtr\(\)](#).

11.28.5.66 typeAliases

```
std::unordered_map<std::string, std::string> pascal::CodeGenVisitor::typeAliases [private]
```

Definition at line 200 of file [icode.hpp](#).

Referenced by [buildArrayInfoFromNode\(\)](#), [generate\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [resolveTypeName\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.67 usedModules

```
std::set<std::string> pascal::CodeGenVisitor::usedModules [private]
```

Definition at line 184 of file [icode.hpp](#).

Referenced by [pascal::FileFunctionHandler::generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), [pascal::SDLFunctionHandler::generate\(\)](#), [pascal::StdFunctionHandler::generate\(\)](#), [pascal::FileFunctionHandler::generateWithResult\(\)](#), [pascal::IOFunctionHandler::generateWithResult\(\)](#), [pascal::SDLFunctionHandler::generateWithResult\(\)](#), [pascal::StdFunctionHandler::generateWithResult\(\)](#), [pascal::StringFunctionHandler::generateWithResult\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [writeTo\(\)](#).

11.28.5.68 usedRealConstants

```
std::unordered_set<std::string> pascal::CodeGenVisitor::usedRealConstants [private]
```

Definition at line 181 of file [icode.hpp](#).

Referenced by [ensureFloatConstSymbol\(\)](#), [generate\(\)](#), and [pushFloatCmpResult\(\)](#).

11.28.5.69 usedStrings

```
std::unordered_set<std::string> pascal::CodeGenVisitor::usedStrings [private]
```

Definition at line 345 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [pascal::IOFunctionHandler::generate\(\)](#), and [writeTo\(\)](#).

11.28.5.70 valueLocations

```
std::unordered_map<std::string, ValueLocation> pascal::CodeGenVisitor::valueLocations [private]
```

Definition at line 366 of file [icode.hpp](#).

Referenced by [generate\(\)](#), and [recordLocation\(\)](#).

11.28.5.71 varRecordType

```
std::unordered_map<std::string, std::string> pascal::CodeGenVisitor::varRecordType [private]
```

Definition at line 2002 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [getVarRecordTypeName\(\)](#), [getVarRecordTypeNameFromExpr\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.72 varSlot

`std::unordered_map<std::string, int> pascal::CodeGenVisitor::varSlot [private]`

Definition at line 340 of file [icode.hpp](#).

Referenced by [ensurePtrBase\(\)](#), [findMangledArrayName\(\)](#), [findMangledName\(\)](#), [pascal::FileFunctionHandler::generate\(\)](#), [getVarType\(\)](#), [newSlotFor\(\)](#), [storageSymbolFor\(\)](#), [visit\(\)](#), [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

11.28.5.73 varTypes

`std::unordered_map<std::string, VarType> pascal::CodeGenVisitor::varTypes [private]`

Definition at line 270 of file [icode.hpp](#).

Referenced by [generate\(\)](#), [getVarType\(\)](#), and [setVarType\(\)](#).

11.28.5.74 withFieldScopes

`std::vector<std::unordered_map<std::string, std::string> > pascal::CodeGenVisitor::withFieldScopes [private]`

Stack of active `with` scopes: each entry maps unqualified field name -> record variable name.

Definition at line 2146 of file [icode.hpp](#).

Referenced by [visit\(\)](#), [visit\(\)](#), and [visit\(\)](#).

The documentation for this class was generated from the following files:

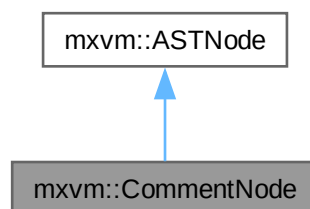
- [src/frontend/include/icode.hpp](#)
- [src/frontend/icode_visit.cpp](#)

11.29 mxvm::CommentNode Class Reference

AST node for a source-level comment.

`#include <ast.hpp>`

Inheritance diagram for `mxvm::CommentNode`:



Public Member Functions

- [CommentNode](#) (const std::string &commentText)
- void [accept](#) (ASTVisitor &visitor) override
- std::string [toString](#) () const override

Public Member Functions inherited from [mxvm::ASTNode](#)

- virtual [~ASTNode](#) ()=default

Public Attributes

- std::string [text](#)

11.29.1 Detailed Description

AST node for a source-level comment.
Definition at line 156 of file [ast.hpp](#).

11.29.2 Constructor & Destructor Documentation

11.29.2.1 CommentNode()

```
mxvm::CommentNode::CommentNode (
    const std::string & commentText) [inline]
```

Definition at line 159 of file [ast.hpp](#).

References [text](#).

11.29.3 Member Function Documentation

11.29.3.1 accept()

```
void mxvm::CommentNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 103 of file [ast.cpp](#).

References [mxvm::ASTVisitor::visit\(\)](#).

11.29.3.2 toString()

```
std::string mxvm::CommentNode::toString () const [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 107 of file [ast.cpp](#).

References [text](#).

11.29.4 Member Data Documentation

11.29.4.1 text

```
std::string mxvm::CommentNode::text
```

Definition at line 158 of file [ast.hpp](#).

Referenced by [CommentNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

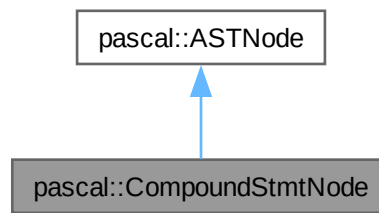
- [include/mxvm/ast.hpp](#)
- [src/ast.cpp](#)

11.30 pascal::CompoundStmtNode Class Reference

AST node for a compound statement (begin ... end).

```
#include <ast.hpp>
```

Inheritance diagram for pascal::CompoundStmtNode:



Public Member Functions

- `CompoundStmtNode` (`std::vector< std::unique_ptr< ASTNode > > stmts`)
- void `accept` (`ASTVisitor &visitor`) override
Accept an `ASTVisitor` (double-dispatch).
- `std::string toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from pascal::ASTNode

- virtual `~ASTNode` ()=default
- virtual void `print` (`std::ostream &out`, `int indent=0`) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (`int line`)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- `std::vector< std::unique_ptr< ASTNode > > statements`
ordered list of statements

11.30.1 Detailed Description

AST node for a compound statement (begin ... end).
Definition at line 237 of file `ast.hpp`.

11.30.2 Constructor & Destructor Documentation

11.30.2.1 CompoundStmtNode()

```
pascal::CompoundStmtNode::CompoundStmtNode (
    std::vector< std::unique_ptr< ASTNode > > stmts) [inline]
```

Definition at line 241 of file `ast.hpp`.

References `statements`.

11.30.3 Member Function Documentation

11.30.3.1 accept()

```
void pascal::CompoundStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 109 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.30.3.2 toString()

```
std::string pascal::CompoundStmtNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 113 of file [ast.cpp](#).

References [statements](#).

11.30.4 Member Data Documentation

11.30.4.1 statements

```
std::vector<std::unique_ptr<ASTNode> > pascal::CompoundStmtNode::statements
```

ordered list of statements

Definition at line 239 of file [ast.hpp](#).

Referenced by [CompoundStmtNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.31 pascal::ConstDeclNode::ConstAssignment Struct Reference

A single constant name = value pair.

```
#include <ast.hpp>
```

Public Member Functions

- [ConstAssignment](#) (const std::string &id, std::unique_ptr< [ASTNode](#) > val)

Public Attributes

- std::string [identifier](#)
constant name
- std::unique_ptr< [ASTNode](#) > [value](#)
constant value expression

11.31.1 Detailed Description

A single constant name = value pair.

Definition at line 177 of file [ast.hpp](#).

11.31.2 Constructor & Destructor Documentation

11.31.2.1 ConstAssignment()

```
pascal::ConstDeclNode::ConstAssignment::ConstAssignment (
    const std::string & id,
    std::unique_ptr< ASTNode > val) [inline]
```

Definition at line 181 of file [ast.hpp](#).
References [identifier](#), and [value](#).

11.31.3 Member Data Documentation

11.31.3.1 identifier

`std::string pascal::ConstDeclNode::ConstAssignment::identifier`
constant name
Definition at line 178 of file [ast.hpp](#).
Referenced by [ConstAssignment\(\)](#).

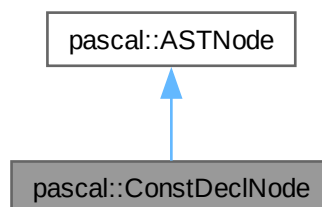
11.31.3.2 value

`std::unique_ptr<ASTNode> pascal::ConstDeclNode::ConstAssignment::value`
constant value expression
Definition at line 179 of file [ast.hpp](#).
Referenced by [ConstAssignment\(\)](#).
The documentation for this struct was generated from the following file:

- [src/frontend/include/ast.hpp](#)

11.32 pascal::ConstDeclNode Class Reference

AST node for constant declarations (const id = value).
`#include <ast.hpp>`
Inheritance diagram for `pascal::ConstDeclNode`:



Classes

- struct [ConstAssignment](#)
A single constant name = value pair.

Public Member Functions

- [ConstDeclNode](#) (`std::vector< std::unique_ptr< ConstAssignment > >` assigns)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- `std::string` [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::vector< std::unique_ptr< [ConstAssignment](#) > > [assignments](#)
list of constant assignments

11.32.1 Detailed Description

AST node for constant declarations (const id = value).
Definition at line 174 of file [ast.hpp](#).

11.32.2 Constructor & Destructor Documentation

11.32.2.1 ConstDeclNode()

```
pascal::ConstDeclNode::ConstDeclNode (
    std::vector< std::unique_ptr< ConstAssignment > > assigns) [inline]
```

Definition at line 186 of file [ast.hpp](#).

References [assignments](#).

11.32.3 Member Function Documentation

11.32.3.1 accept()

```
void pascal::ConstDeclNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 285 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.32.3.2 toString()

```
std::string pascal::ConstDeclNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 289 of file [ast.cpp](#).

11.32.4 Member Data Documentation

11.32.4.1 assignments

```
std::vector<std::unique_ptr<ConstAssignment> > pascal::ConstDeclNode::assignments
```

list of constant assignments

Definition at line 185 of file [ast.hpp](#).

Referenced by [ConstDeclNode\(\)](#), [pascal::CodeGenVisitor::registerImportedConst\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

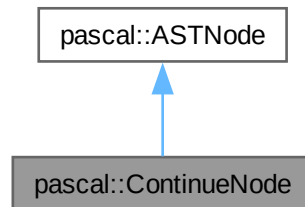
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.33 pascal::ContinueNode Class Reference

AST node for a continue statement.

```
#include <ast.hpp>
```

Inheritance diagram for pascal::ContinueNode:



Public Member Functions

- void `accept` (`ASTVisitor` &visitor) override
Accept an `ASTVisitor` (double-dispatch).
- `std::string toString` () const override
Return a human-readable description of this node.
- `ContinueNode` ()=default
- `ContinueNode` (`std::unique_ptr`< `ASTNode` > e)

Public Member Functions inherited from pascal::ASTNode

- virtual `~ASTNode` ()=default
- virtual void `print` (`std::ostream` &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- `std::unique_ptr`< `ASTNode` > `expr`
reserved expression (unused)

11.33.1 Detailed Description

AST node for a continue statement.

Definition at line 55 of file `ast.hpp`.

11.33.2 Constructor & Destructor Documentation

11.33.2.1 ContinueNode() [1/2]

```
pascal::ContinueNode::ContinueNode () [default]
```

11.33.2.2 ContinueNode() [2/2]

```
pascal::ContinueNode::ContinueNode (  
    std::unique_ptr< ASTNode > e) [inline], [explicit]
```

Definition at line 61 of file [ast.hpp](#).

References [expr](#).

11.33.3 Member Function Documentation

11.33.3.1 accept()

```
void pascal::ContinueNode::accept (  
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 410 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.33.3.2 toString()

```
std::string pascal::ContinueNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 414 of file [ast.cpp](#).

11.33.4 Member Data Documentation

11.33.4.1 expr

```
std::unique_ptr<ASTNode> pascal::ContinueNode::expr
```

reserved expression (unused)

Definition at line 59 of file [ast.hpp](#).

Referenced by [ContinueNode\(\)](#).

The documentation for this class was generated from the following files:

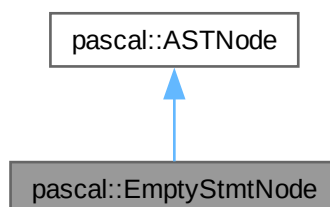
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.34 pascal::EmptyStmtNode Class Reference

AST node for an empty statement (no-op).

```
#include <ast.hpp>
```

Inheritance diagram for [pascal::EmptyStmtNode](#):



Public Member Functions

- [EmptyStmtNode](#) ()
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

11.34.1 Detailed Description

AST node for an empty statement (no-op).
Definition at line 560 of file [ast.hpp](#).

11.34.2 Constructor & Destructor Documentation**11.34.2.1 EmptyStmtNode()**

```
pascal::EmptyStmtNode::EmptyStmtNode () [inline]
```

Definition at line 562 of file [ast.hpp](#).

11.34.3 Member Function Documentation**11.34.3.1 accept()**

```
void pascal::EmptyStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 277 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.34.3.2 toString()

```
std::string pascal::EmptyStmtNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 281 of file [ast.cpp](#).

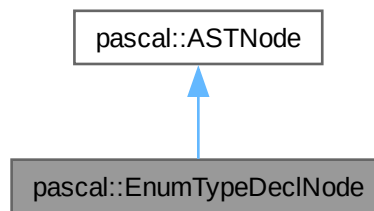
The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.35 pascal::EnumTypeDeclNode Class Reference

AST node for an enumerated type declaration (type Color = (Red, Green, Blue)).
`#include <ast.hpp>`

Inheritance diagram for `pascal::EnumTypeDeclNode`:



Public Member Functions

- [EnumTypeDeclNode](#) (const std::string &name, std::vector< std::string > vals)
- void [accept](#) (ASTVisitor &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [typeName](#)
the enum type name
- std::vector< std::string > [values](#)
ordered list of enum value identifiers

11.35.1 Detailed Description

AST node for an enumerated type declaration (type `Color = (Red, Green, Blue)`).
Definition at line [627](#) of file [ast.hpp](#).

11.35.2 Constructor & Destructor Documentation

11.35.2.1 EnumTypeDeclNode()

```

pascal::EnumTypeDeclNode::EnumTypeDeclNode (
    const std::string & name,
    std::vector< std::string > vals) [inline]
  
```

Definition at line [632](#) of file [ast.hpp](#).
References [typeName](#), and [values](#).

11.35.3 Member Function Documentation

11.35.3.1 accept()

```
void pascal::EnumTypeDeclNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 462 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.35.3.2 toString()

```
std::string pascal::EnumTypeDeclNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 636 of file [ast.hpp](#).

References [typeName](#).

11.35.4 Member Data Documentation

11.35.4.1 typeName

```
std::string pascal::EnumTypeDeclNode::typeName
```

the enum type name

Definition at line 629 of file [ast.hpp](#).

Referenced by [EnumTypeDeclNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.35.4.2 values

```
std::vector<std::string> pascal::EnumTypeDeclNode::values
```

ordered list of enum value identifiers

Definition at line 630 of file [ast.hpp](#).

Referenced by [EnumTypeDeclNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

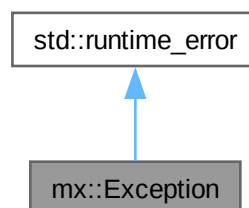
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.36 mx::Exception Class Reference

General-purpose exception with errno-aware factory method.

```
#include <exception.hpp>
```

Inheritance diagram for `mx::Exception`:



Public Member Functions

- [Exception](#) (const std::string &message)
Construct an exception from a string message.
- [Exception](#) (const char *message)
Construct an exception from a C string message.

Static Public Member Functions

- static [Exception error](#) (const std::string &context)
Create an exception with errno appended.

11.36.1 Detailed Description

General-purpose exception with errno-aware factory method.

Extends std::runtime_error. The static [error\(\)](#) method appends strerror(errno) to the context string.

Definition at line 38 of file [exception.hpp](#).

11.36.2 Constructor & Destructor Documentation

11.36.2.1 Exception() [1/2]

```
mx::Exception::Exception (
    const std::string & message) [inline], [explicit]
```

Construct an exception from a string message.

Parameters

<i>message</i>	Human-readable error description
----------------	----------------------------------

Definition at line 44 of file [exception.hpp](#).

Referenced by [error\(\)](#).

11.36.2.2 Exception() [2/2]

```
mx::Exception::Exception (
    const char * message) [inline], [explicit]
```

Construct an exception from a C string message.

Parameters

<i>message</i>	Human-readable error description
----------------	----------------------------------

Definition at line 51 of file [exception.hpp](#).

11.36.3 Member Function Documentation

11.36.3.1 error()

```
Exception mx::Exception::error (
    const std::string & context) [inline], [static]
```

Create an exception with errno appended.

Parameters

<i>context</i>	Descriptive context prefix
----------------	----------------------------

Returns

[Exception](#) with "context: strerror(errno)" message

Definition at line 59 of file [exception.hpp](#).

References [Exception\(\)](#).

The documentation for this class was generated from the following file:

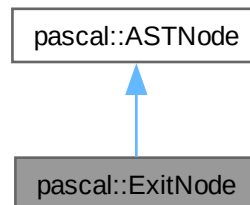
- [src/scanner/include/scanner/exception.hpp](#)

11.37 pascal::ExitNode Class Reference

AST node for an exit / halt statement.

```
#include <ast.hpp>
```

Inheritance diagram for pascal::ExitNode:



Public Member Functions

- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.
- [ExitNode](#) ()=default
- [ExitNode](#) (std::unique_ptr< [ASTNode](#) > e)
Construct with an optional exit-code expression.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::unique_ptr< [ASTNode](#) > [expr](#)
optional exit-code expression

11.37.1 Detailed Description

AST node for an exit / halt statement.

Definition at line 44 of file [ast.hpp](#).

11.37.2 Constructor & Destructor Documentation

11.37.2.1 ExitNode() [1/2]

```
pascal::ExitNode::ExitNode () [default]
```

11.37.2.2 ExitNode() [2/2]

```
pascal::ExitNode::ExitNode (
    std::unique_ptr< ASTNode > e) [inline], [explicit]
```

Construct with an optional exit-code expression.

Definition at line 51 of file [ast.hpp](#).

References [expr](#).

11.37.3 Member Function Documentation

11.37.3.1 accept()

```
void pascal::ExitNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 402 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.37.3.2 toString()

```
std::string pascal::ExitNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 324 of file [ast.cpp](#).

11.37.4 Member Data Documentation

11.37.4.1 expr

```
std::unique_ptr<ASTNode> pascal::ExitNode::expr
```

optional exit-code expression

Definition at line 48 of file [ast.hpp](#).

Referenced by [ExitNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

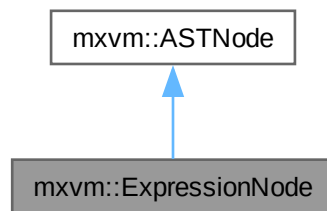
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.38 mxvm::ExpressionNode Class Reference

AST node for a literal expression value.

```
#include <ast.hpp>
```


Inheritance diagram for mxvm::ExpressionNode:



Public Member Functions

- [ExpressionNode](#) (const std::string &val, bool numeric=false)
- void [accept](#) (ASTVisitor &visitor) override
- std::string [toString](#) () const override

Public Member Functions inherited from [mxvm::ASTNode](#)

- virtual [~ASTNode](#) ()=default

Public Attributes

- std::string [value](#)
- bool [isNumeric](#)

11.38.1 Detailed Description

AST node for a literal expression value.

Definition at line 143 of file [ast.hpp](#).

11.38.2 Constructor & Destructor Documentation

11.38.2.1 ExpressionNode()

```
mxvm::ExpressionNode::ExpressionNode (
    const std::string & val,
    bool numeric = false) [inline]
```

Definition at line 148 of file [ast.hpp](#).

References [isNumeric](#), and [value](#).

11.38.3 Member Function Documentation

11.38.3.1 accept()

```
void mxvm::ExpressionNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 95 of file [ast.cpp](#).

References [mxvm::ASTVisitor::visit\(\)](#).

11.38.3.2 toString()

std::string mxvm::ExpressionNode::toString () const [override], [virtual]

Implements [mxvm::ASTNode](#).

Definition at line 99 of file [ast.cpp](#).

References [isNumeric](#), and [value](#).

11.38.4 Member Data Documentation

11.38.4.1 isNumeric

bool mxvm::ExpressionNode::isNumeric

Definition at line 146 of file [ast.hpp](#).

Referenced by [ExpressionNode\(\)](#), and [toString\(\)](#).

11.38.4.2 value

std::string mxvm::ExpressionNode::value

Definition at line 145 of file [ast.hpp](#).

Referenced by [ExpressionNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

- include/mxvm/[ast.hpp](#)
- src/[ast.cpp](#)

11.39 expr_parser::ExprLexer Class Reference

Lexer for constant integer expressions.

```
#include <expr.hpp>
```

Public Member Functions

- [ExprLexer](#) (const std::string &input)
Construct a lexer and scan the input string.
- void [next](#) ()
Advance to the next token and update current.
- [ExprToken peek](#) () const
Return the current token without consuming it.
- void [consume](#) ()
Consume the current token and advance to the next.

Public Attributes

- std::string [variable](#)
last scanned variable name

Private Attributes

- [scan::Scanner scanner](#)
underlying scanner for tokenization
- size_t [pos](#)
current position in the token stream
- [ExprToken current](#)
most recently read token

11.39.1 Detailed Description

Lexer for constant integer expressions.

Tokenizes an expression string into [ExprToken](#) values using the underlying Scanner. Supports numbers, operators, and variable names.

Definition at line 58 of file [expr.hpp](#).

11.39.2 Constructor & Destructor Documentation

11.39.2.1 `ExprLexer()`

```
expr_parser::ExprLexer::ExprLexer (
    const std::string & input) [inline]
```

Construct a lexer and scan the input string.

Parameters

<i>input</i>	Expression string to tokenize
--------------	-------------------------------

Definition at line 65 of file [expr.hpp](#).

References [next\(\)](#), [pos](#), and [scanner](#).

11.39.3 Member Function Documentation

11.39.3.1 `consume()`

```
void expr_parser::ExprLexer::consume () [inline]
```

Consume the current token and advance to the next.

Definition at line 135 of file [expr.hpp](#).

References [next\(\)](#).

11.39.3.2 `next()`

```
void expr_parser::ExprLexer::next () [inline]
```

Advance to the next token and update current.

Definition at line 71 of file [expr.hpp](#).

References [expr_parser::AND](#), [current](#), [expr_parser::DIV](#), [expr_parser::END](#), [expr_parser::EQ](#), [expr_parser::GT](#), [expr_parser::GTE](#), [expr_parser::LOGICAL_AND](#), [expr_parser::LOGICAL_OR](#), [expr_parser::LPAREN](#), [expr_parser::LT](#), [expr_parser::LTE](#), [expr_parser::MINUS](#), [expr_parser::MINUSEQ](#), [expr_parser::MOD](#), [expr_parser::MUL](#), [expr_parser::NEQ](#), [expr_parser::NOT](#), [expr_parser::NUMBER](#), [expr_parser::OR](#), [expr_parser::PLUS](#), [expr_parser::PLUSEQ](#), [pos](#), [expr_parser::RPAREN](#), [scanner](#), [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), [expr_parser::VARIABLE](#), [variable](#), and [expr_parser::XOR](#).

Referenced by [ExprLexer\(\)](#), and [consume\(\)](#).

11.39.3.3 `peek()`

```
ExprToken expr_parser::ExprLexer::peek () const [inline]
```

Return the current token without consuming it.

Definition at line 133 of file [expr.hpp](#).

References [current](#).

11.39.4 Member Data Documentation

11.39.4.1 `current`

```
ExprToken expr_parser::ExprLexer::current [private]
```

most recently read token

Definition at line 140 of file [expr.hpp](#).

Referenced by [next\(\)](#), and [peek\(\)](#).

11.39.4.2 pos

`size_t expr_parser::ExprLexer::pos` [private]

current position in the token stream

Definition at line 139 of file [expr.hpp](#).

Referenced by [ExprLexer\(\)](#), and [next\(\)](#).

11.39.4.3 scanner

`scan::Scanner expr_parser::ExprLexer::scanner` [private]

underlying scanner for tokenization

Definition at line 138 of file [expr.hpp](#).

Referenced by [ExprLexer\(\)](#), and [next\(\)](#).

11.39.4.4 variable

`std::string expr_parser::ExprLexer::variable`

last scanned variable name

Definition at line 60 of file [expr.hpp](#).

Referenced by [next\(\)](#).

The documentation for this class was generated from the following file:

- [src/scanner/include/scanner/expr.hpp](#)

11.40 expr_parser::ExprParser Class Reference

Recursive-descent parser and evaluator for constant integer expressions.

`#include <expr.hpp>`

Public Member Functions

- [ExprParser \(ExprLexer &lexer\)](#)
Construct a parser bound to a lexer.
- `uint64_t` [parse \(\)](#)
Parse and evaluate the expression.

Private Member Functions

- `uint64_t` [parseAdd \(\)](#)
Parse additive expressions (+, -).
- `uint64_t` [parseBitwise \(\)](#)
Parse bitwise expressions (^, |, &).
- `uint64_t` [parseComparison \(\)](#)
Parse comparison expressions (==, !=, >, >=, <, <=).
- `uint64_t` [parseLogical \(\)](#)
Parse logical expressions (&&, ||) — lowest precedence.
- `uint64_t` [parseTerm \(\)](#)
Parse multiplicative expressions (, /, %).*
- `uint64_t` [parseFactor \(\)](#)
Parse primary expressions: numbers, variables, unary ops, and parenthesised sub-expressions.

Private Attributes

- [ExprLexer &lexer](#)
reference to the token source

11.40.1 Detailed Description

Recursive-descent parser and evaluator for constant integer expressions.

Evaluates expressions with C-like operator precedence: logical $\rightarrow$ comparison $\rightarrow$ bitwise $\rightarrow$ additive $\rightarrow$ term (mul/div/mod) $\rightarrow$ factor. Supports variable references resolved via the global `vars` map.

Definition at line 150 of file [expr.hpp](#).

11.40.2 Constructor & Destructor Documentation

11.40.2.1 `ExprParser()`

```
expr_parser::ExprParser::ExprParser (
    ExprLexer & lexer) [inline]
```

Construct a parser bound to a lexer.

Parameters

lexer	Reference to an initialised ExprLexer
-----------------------	---

Definition at line 156 of file [expr.hpp](#).

References [lexer](#).

11.40.3 Member Function Documentation

11.40.3.1 `parse()`

```
uint64_t expr_parser::ExprParser::parse () [inline]
```

Parse and evaluate the expression.

Returns

Computed `uint64_t` result

Definition at line 161 of file [expr.hpp](#).

References [parseLogical\(\)](#).

11.40.3.2 `parseAdd()`

```
uint64_t expr_parser::ExprParser::parseAdd () [inline], [private]
```

Parse additive expressions (+, -).

Definition at line 167 of file [expr.hpp](#).

References [lexer](#), [expr_parser::MINUS](#), [parseTerm\(\)](#), and [expr_parser::PLUS](#).

Referenced by [parseBitwise\(\)](#).

11.40.3.3 `parseBitwise()`

```
uint64_t expr_parser::ExprParser::parseBitwise () [inline], [private]
```

Parse bitwise expressions (^, |, &).

Definition at line 183 of file [expr.hpp](#).

References [expr_parser::AND](#), [lexer](#), [expr_parser::OR](#), [parseAdd\(\)](#), and [expr_parser::XOR](#).

Referenced by [parseComparison\(\)](#).

11.40.3.4 `parseComparison()`

```
uint64_t expr_parser::ExprParser::parseComparison () [inline], [private]
```

Parse comparison expressions (==, !=, >, >=, <, <=).

Definition at line 203 of file [expr.hpp](#).

References [expr_parser::EQ](#), [expr_parser::GT](#), [expr_parser::GTE](#), [lexer](#), [expr_parser::LT](#), [expr_parser::LTE](#), [expr_parser::NEQ](#), and [parseBitwise\(\)](#).

Referenced by [parseLogical\(\)](#).

11.40.3.5 parseFactor()

```
uint64_t expr_parser::ExprParser::parseFactor () [inline], [private]
```

Parse primary expressions: numbers, variables, unary ops, and parenthesised sub-expressions.

Definition at line 277 of file [expr.hpp](#).

References [lexer](#), [expr_parser::LPAREN](#), [expr_parser::MINUS](#), [expr_parser::NOT](#), [expr_parser::NUMBER](#), [parseFactor\(\)](#), [parseLogical\(\)](#), [expr_parser::RPAREN](#), [expr_parser::VARIABLE](#), and [expr_parser::vars](#).

Referenced by [parseFactor\(\)](#), and [parseTerm\(\)](#).

11.40.3.6 parseLogical()

```
uint64_t expr_parser::ExprParser::parseLogical () [inline], [private]
```

Parse logical expressions (&&, ||) — lowest precedence.

Definition at line 235 of file [expr.hpp](#).

References [lexer](#), [expr_parser::LOGICAL_AND](#), [expr_parser::LOGICAL_OR](#), and [parseComparison\(\)](#).

Referenced by [parse\(\)](#), and [parseFactor\(\)](#).

11.40.3.7 parseTerm()

```
uint64_t expr_parser::ExprParser::parseTerm () [inline], [private]
```

Parse multiplicative expressions (*, /, %).

Definition at line 251 of file [expr.hpp](#).

References [expr_parser::DIV](#), [lexer](#), [expr_parser::MOD](#), [expr_parser::MUL](#), and [parseFactor\(\)](#).

Referenced by [parseAdd\(\)](#).

11.40.4 Member Data Documentation

11.40.4.1 lexer

```
ExprLexer& expr_parser::ExprParser::lexer [private]
```

reference to the token source

Definition at line 164 of file [expr.hpp](#).

Referenced by [ExprParser\(\)](#), [parseAdd\(\)](#), [parseBitwise\(\)](#), [parseComparison\(\)](#), [parseFactor\(\)](#), [parseLogical\(\)](#), and [parseTerm\(\)](#).

The documentation for this class was generated from the following file:

- [src/scanner/include/scanner/expr.hpp](#)

11.41 expr_parser::ExprToken Struct Reference

A single expression token holding its type and optional numeric value.

```
#include <expr.hpp>
```

Public Attributes

- [ExprTokenType](#) type
token classification
- [uint64_t](#) value
numeric value (valid when type == NUMBER)

11.41.1 Detailed Description

A single expression token holding its type and optional numeric value.

Definition at line 44 of file [expr.hpp](#).

11.41.2 Member Data Documentation

11.41.2.1 type

`ExprTokenType` `expr_parser::ExprToken::type`
 token classification
 Definition at line 45 of file [expr.hpp](#).

11.41.2.2 value

`uint64_t` `expr_parser::ExprToken::value`
 numeric value (valid when type == NUMBER)
 Definition at line 46 of file [expr.hpp](#).
 The documentation for this struct was generated from the following file:

- [src/scanner/include/scanner/expr.hpp](#)

11.42 mxvm::ExternalFunction Struct Reference

Describes an external function imported from a module or object.
`#include <parser.hpp>`

Public Member Functions

- `bool` `operator==` (const [ExternalFunction](#) &f)
Equality comparison by name and module.

Public Attributes

- `std::string` `name`
function symbol name
- `std::string` `mod`
module or object name
- `bool` `module`
true if from a module, false if from an object
- `VarType` `ret_type` = `VarType::VAR_INTEGER`
return type of the external function
- `bool` `ret_owned` = false
true if the return value is an owned pointer

11.42.1 Detailed Description

Describes an external function imported from a module or object.
 Definition at line 162 of file [parser.hpp](#).

11.42.2 Member Function Documentation

11.42.2.1 operator==()

```
bool mxvm::ExternalFunction::operator== (
    const ExternalFunction & f) [inline]
```

Equality comparison by name and module.
 Definition at line 167 of file [parser.hpp](#).
 References [mod](#), and [name](#).

11.42.3 Member Data Documentation

11.42.3.1 mod

`std::string mxvm::ExternalFunction::mod`
 module or object name
 Definition at line 164 of file [parser.hpp](#).
 Referenced by [operator==\(\)](#).

11.42.3.2 module

`bool mxvm::ExternalFunction::module`
 true if from a module, false if from an object
 Definition at line 165 of file [parser.hpp](#).

11.42.3.3 name

`std::string mxvm::ExternalFunction::name`
 function symbol name
 Definition at line 163 of file [parser.hpp](#).
 Referenced by [operator==\(\)](#).

11.42.3.4 ret_owned

`bool mxvm::ExternalFunction::ret_owned = false`
 true if the return value is an owned pointer
 Definition at line 172 of file [parser.hpp](#).

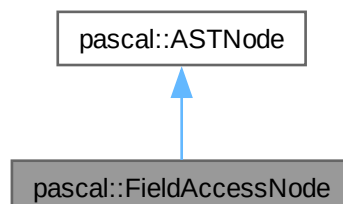
11.42.3.5 ret_type

`VarType mxvm::ExternalFunction::ret_type = VarType::VAR_INTEGER`
 return type of the external function
 Definition at line 171 of file [parser.hpp](#).
 The documentation for this struct was generated from the following file:

- [include/mxvm/parser.hpp](#)

11.43 pascal::FieldAccessNode Class Reference

AST node for accessing a record field (record.field).
`#include <ast.hpp>`
 Inheritance diagram for `pascal::FieldAccessNode`:



Public Member Functions

- [FieldAccessNode](#) (std::unique_ptr< [ASTNode](#) > [recordExpr](#), const std::string &[fieldName](#))
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::unique_ptr< [ASTNode](#) > [recordExpr](#)
expression yielding a record
- std::string [fieldName](#)
name of the accessed field

11.43.1 Detailed Description

AST node for accessing a record field (record.field).

Definition at line 726 of file [ast.hpp](#).

11.43.2 Constructor & Destructor Documentation

11.43.2.1 FieldAccessNode()

```
pascal::FieldAccessNode::FieldAccessNode (
    std::unique_ptr< ASTNode > recordExpr,
    const std::string & fieldName)
```

Definition at line 364 of file [ast.cpp](#).

References [fieldName](#), and [recordExpr](#).

11.43.3 Member Function Documentation

11.43.3.1 accept()

```
void pascal::FieldAccessNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 367 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.43.3.2 toString()

```
std::string pascal::FieldAccessNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 369 of file [ast.cpp](#).

References [fieldName](#).

11.43.4 Member Data Documentation

11.43.4.1 fieldName

`std::string pascal::FieldAccessNode::fieldName`

name of the accessed field

Definition at line 729 of file [ast.hpp](#).

Referenced by [FieldAccessNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.43.4.2 recordExpr

`std::unique_ptr<ASTNode> pascal::FieldAccessNode::recordExpr`

expression yielding a record

Definition at line 728 of file [ast.hpp](#).

Referenced by [FieldAccessNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

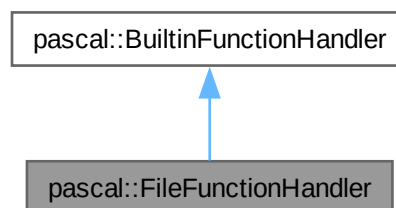
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.44 pascal::FileFunctionHandler Class Reference

Built-in handler for Pascal file I/O procedures (assign, reset, rewrite, append, close, eof, eoln).

`#include <icode.hpp>`

Inheritance diagram for `pascal::FileFunctionHandler`:



Public Member Functions

- bool [canHandle](#) (const std::string &funcName) const override
Return true if this handler implements funcName.
- void [generate](#) ([CodeGenVisitor](#) &visitor, const std::string &funcName, const std::vector< std::unique_ptr< [ASTNode](#) > > &arguments) override
Emit code for a procedure-style call (no return value used).
- bool [generateWithResult](#) ([CodeGenVisitor](#) &visitor, const std::string &funcName, const std::vector< std::↵
unique_ptr< [ASTNode](#) > > &arguments) override
Emit code for a function-style call (result pushed on eval stack); return true if handled.
- [VarType](#) [getReturnType](#) (const std::string &funcName) const override
Return the VarType produced by funcName.

Public Member Functions inherited from [pascal::BuiltinFunctionHandler](#)

- virtual [~BuiltinFunctionHandler](#) ()=default

11.44.1 Detailed Description

Built-in handler for Pascal file I/O procedures (assign, reset, rewrite, append, close, eof, eoln).
Definition at line 163 of file [icode.hpp](#).

11.44.2 Member Function Documentation

11.44.2.1 canHandle()

```
bool pascal::FileFunctionHandler::canHandle (
    const std::string & funcName) const [override], [virtual]
```

Return true if this handler implements funcName.

Implements [pascal::BuiltinFunctionHandler](#).

Definition at line 988 of file [icode_imp.cpp](#).

References [pascal::fileFunctions](#), and [pascal::toLower\(\)](#).

11.44.2.2 generate()

```
void pascal::FileFunctionHandler::generate (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [virtual]
```

Emit code for a procedure-style call (no return value used).

Implements [pascal::BuiltinFunctionHandler](#).

Definition at line 992 of file [icode_imp.cpp](#).

References [pascal::CodeGenVisitor::allocReg\(\)](#), [pascal::CodeGenVisitor::emit\(\)](#), [pascal::CodeGenVisitor::emit2\(\)](#), [pascal::CodeGenVisitor::emit_invoke\(\)](#), [pascal::CodeGenVisitor::eval\(\)](#), [pascal::CodeGenVisitor::fileVarNames](#), [pascal::CodeGenVisitor::findMangledName\(\)](#), [pascal::CodeGenVisitor::freeReg\(\)](#), [pascal::CodeGenVisitor::internString\(\)](#), [pascal::CodeGenVisitor::isParmReg\(\)](#), [pascal::CodeGenVisitor::isReg\(\)](#), [pascal::CodeGenVisitor::slotVar\(\)](#), [pascal::toLower\(\)](#), [pascal::CodeGenVisitor::usedModules](#), and [pascal::CodeGenVisitor::varSlot](#).

11.44.2.3 generateWithResult()

```
bool pascal::FileFunctionHandler::generateWithResult (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [virtual]
```

Emit code for a function-style call (result pushed on eval stack); return true if handled.

Reimplemented from [pascal::BuiltinFunctionHandler](#).

Definition at line 1069 of file [icode_imp.cpp](#).

References [pascal::CodeGenVisitor::allocReg\(\)](#), [pascal::CodeGenVisitor::emit\(\)](#), [pascal::CodeGenVisitor::emit2\(\)](#), [pascal::CodeGenVisitor::emit_invoke\(\)](#), [pascal::CodeGenVisitor::eval\(\)](#), [pascal::CodeGenVisitor::freeReg\(\)](#), [pascal::CodeGenVisitor::isParmReg\(\)](#), [pascal::CodeGenVisitor::isReg\(\)](#), [pascal::CodeGenVisitor::pushValue\(\)](#), [pascal::toLower\(\)](#), and [pascal::CodeGenVisitor::usedModules](#).

11.44.2.4 getReturnType()

```
VarType pascal::FileFunctionHandler::getReturnType (
    const std::string & funcName) const [override], [virtual]
```

Return the [VarType](#) produced by funcName.

Reimplemented from [pascal::BuiltinFunctionHandler](#).

Definition at line 1097 of file [icode_imp.cpp](#).

References [pascal::INT](#), [pascal::toLower\(\)](#), and [pascal::UNKNOWN](#).

The documentation for this class was generated from the following files:

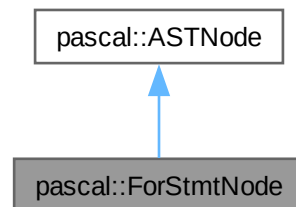
- [src/frontend/include/icode.hpp](#)
- [src/frontend/icode_imp.cpp](#)

11.45 pascal::ForStmtNode Class Reference

AST node for a for loop (for var := start to/downto end do stmt).

```
#include <ast.hpp>
```

Inheritance diagram for pascal::ForStmtNode:



Public Member Functions

- `ForStmtNode` (const std::string &var, std::unique_ptr< [ASTNode](#) > start, std::unique_ptr< [ASTNode](#) > end, bool downto, std::unique_ptr< [ASTNode](#) > stmt)
- void `accept` ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string `toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual `~ASTNode` ()=default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::string `variable`
loop variable name
- std::unique_ptr< [ASTNode](#) > `startValue`
initial value expression
- std::unique_ptr< [ASTNode](#) > `endValue`
final value expression
- bool `isDownto`
true if downto, false if to
- std::unique_ptr< [ASTNode](#) > `statement`
loop body

11.45.1 Detailed Description

AST node for a for loop (for var := start to/downto end do stmt).

Definition at line 379 of file [ast.hpp](#).

11.45.2 Constructor & Destructor Documentation

11.45.2.1 ForStmtNode()

```
pascal::ForStmtNode::ForStmtNode (
    const std::string & var,
    std::unique_ptr< ASTNode > start,
    std::unique_ptr< ASTNode > end,
    bool downto,
    std::unique_ptr< ASTNode > stmt) [inline]
```

Definition at line 387 of file [ast.hpp](#).

References [endValue](#), [isDownto](#), [startValue](#), [statement](#), and [variable](#).

11.45.3 Member Function Documentation

11.45.3.1 accept()

```
void pascal::ForStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 145 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.45.3.2 toString()

```
std::string pascal::ForStmtNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 149 of file [ast.cpp](#).

References [isDownto](#), and [variable](#).

11.45.4 Member Data Documentation

11.45.4.1 endValue

```
std::unique_ptr<ASTNode> pascal::ForStmtNode::endValue
```

final value expression

Definition at line 383 of file [ast.hpp](#).

Referenced by [ForStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.45.4.2 isDownto

```
bool pascal::ForStmtNode::isDownto
```

true if downto, false if to

Definition at line 384 of file [ast.hpp](#).

Referenced by [ForStmtNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.45.4.3 startValue

```
std::unique_ptr<ASTNode> pascal::ForStmtNode::startValue
```

initial value expression

Definition at line 382 of file [ast.hpp](#).

Referenced by [ForStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.45.4.4 statement

```
std::unique_ptr<ASTNode> pascal::ForStmtNode::statement
```

loop body

Definition at line 385 of file [ast.hpp](#).

Referenced by [ForStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.45.4.5 variable

`std::string pascal::ForStmtNode::variable`

loop variable name

Definition at line 381 of file [ast.hpp](#).

Referenced by [ForStmtNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

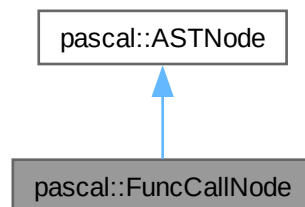
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.46 pascal::FuncCallNode Class Reference

AST node for a function call expression.

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::FuncCallNode`:



Public Member Functions

- [FuncCallNode](#) (const `std::string` &funcName, `std::vector`< `std::unique_ptr`< [ASTNode](#) > > args)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- `std::string` [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (`std::ostream` &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- `std::string` [name](#)
function name
- `std::vector`< `std::unique_ptr`< [ASTNode](#) > > [arguments](#)
actual arguments

11.46.1 Detailed Description

AST node for a function call expression.
Definition at line 500 of file [ast.hpp](#).

11.46.2 Constructor & Destructor Documentation

11.46.2.1 FuncCallNode()

```
pascal::FuncCallNode::FuncCallNode (
    const std::string & funcName,
    std::vector< std::unique_ptr< ASTNode > > args) [inline]
```

Definition at line 505 of file [ast.hpp](#).
References [arguments](#), and [name](#).

11.46.3 Member Function Documentation

11.46.3.1 accept()

```
void pascal::FuncCallNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).
Implements [pascal::ASTNode](#).
Definition at line 231 of file [ast.cpp](#).
References [pascal::ASTVisitor::visit\(\)](#).

11.46.3.2 toString()

```
std::string pascal::FuncCallNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.
Implements [pascal::ASTNode](#).
Definition at line 235 of file [ast.cpp](#).
References [arguments](#), and [name](#).

11.46.4 Member Data Documentation

11.46.4.1 arguments

```
std::vector<std::unique_ptr<ASTNode> > pascal::FuncCallNode::arguments
```

actual arguments
Definition at line 503 of file [ast.hpp](#).
Referenced by [FuncCallNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.46.4.2 name

```
std::string pascal::FuncCallNode::name
```

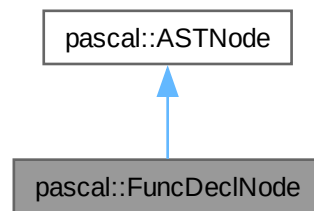
function name
Definition at line 502 of file [ast.hpp](#).
Referenced by [FuncCallNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).
The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.47 pascal::FuncDeclNode Class Reference

AST node for a function declaration.
`#include <ast.hpp>`

Inheritance diagram for `pascal::FuncDeclNode`:



Public Member Functions

- `FuncDeclNode` (const std::string &n, std::vector< std::unique_ptr< [ASTNode](#) > > p, const std::string &rt, std::unique_ptr< [ASTNode](#) > b)
- void `accept` ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string `toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual `~ASTNode` ()=default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::string `name`
function name
- std::vector< std::unique_ptr< [ASTNode](#) > > `parameters`
formal parameters
- std::string `returnType`
return type name
- std::shared_ptr< [ASTNode](#) > `block`
function body block

11.47.1 Detailed Description

AST node for a function declaration.
Definition at line [194](#) of file [ast.hpp](#).

11.47.2 Constructor & Destructor Documentation

11.47.2.1 FuncDeclNode()

```
pascal::FuncDeclNode::FuncDeclNode (
    const std::string & n,
    std::vector< std::unique_ptr< ASTNode > > p,
    const std::string & rt,
    std::unique_ptr< ASTNode > b) [inline]
```

Definition at line 201 of file [ast.hpp](#).

References [block](#), [name](#), [parameters](#), and [returnType](#).

11.47.3 Member Function Documentation

11.47.3.1 accept()

```
void pascal::FuncDeclNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 83 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.47.3.2 toString()

```
std::string pascal::FuncDeclNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 87 of file [ast.cpp](#).

References [name](#), and [returnType](#).

11.47.4 Member Data Documentation

11.47.4.1 block

```
std::shared_ptr<ASTNode> pascal::FuncDeclNode::block
```

function body block

Definition at line 199 of file [ast.hpp](#).

Referenced by [FuncDeclNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.47.4.2 name

```
std::string pascal::FuncDeclNode::name
```

function name

Definition at line 196 of file [ast.hpp](#).

Referenced by [FuncDeclNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.47.4.3 parameters

```
std::vector<std::unique_ptr<ASTNode> > pascal::FuncDeclNode::parameters
```

formal parameters

Definition at line 197 of file [ast.hpp](#).

Referenced by [FuncDeclNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.47.4.4 returnType

```
std::string pascal::FuncDeclNode::returnType
```

return type name

Definition at line 198 of file [ast.hpp](#).

Referenced by [FuncDeclNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.48 pascal::CodeGenVisitor::FuncInfo Struct Reference

Describes a user-declared function's parameter and return types.

Public Attributes

- `std::vector< VarType > paramTypes`
parameter types in order
- `VarType returnType = VarType::UNKNOWN`
return type

11.48.1 Detailed Description

Describes a user-declared function's parameter and return types.

Definition at line 264 of file [icode.hpp](#).

11.48.2 Member Data Documentation

11.48.2.1 paramTypes

```
std::vector<VarType> pascal::CodeGenVisitor::FuncInfo::paramTypes
```

parameter types in order

Definition at line 265 of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.48.2.2 returnType

```
VarType pascal::CodeGenVisitor::FuncInfo::returnType = VarType::UNKNOWN
```

return type

Definition at line 266 of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this struct was generated from the following file:

- [src/frontend/include/icode.hpp](#)

11.49 mxvm::Function< Sig > Class Template Reference

11.49.1 Detailed Description

```
template<typename Sig>
```

```
class mxvm::Function< Sig >
```

Definition at line 15 of file [function.hpp](#).

The documentation for this class was generated from the following file:

- [include/mxvm/function.hpp](#)

11.50 mxvm::Function< R(Args...) > Class Template Reference

RAII wrapper for dynamically loaded functions via dlopen/dlsym.

```
#include <function.hpp>
```

Public Types

- using [Fn](#) = R (*)(Args...)

Public Member Functions

- [Function](#) (char const *lib, char const *name)
Load a function from a named shared library.
- [Function](#) (char const *name)
Load a function from the main program's symbol table.
- [~Function](#) ()
- [Function](#) (const Function &)=delete
- [Function](#) & [operator=](#) (const [Function](#) &)=delete
- [Function](#) (Function &&other) noexcept
- [Function](#) & [operator=](#) ([Function](#) &&other) noexcept
- [R operator\(\)](#) (Args... args) const
Invoke the loaded function.

Private Attributes

- void * [handle](#) = nullptr
dlopen library handle
- [Fn fn](#) = nullptr
resolved function pointer

11.50.1 Detailed Description

template<typename R, typename... Args>

class mxvm::Function< R(Args...)>

RAII wrapper for dynamically loaded functions via dlopen/dlsym.

Template Parameters

<i>R</i>	Return type of the loaded function
<i>Args</i>	Parameter types of the loaded function

Loads a shared library and resolves a function symbol at construction time. The library handle is automatically closed on destruction. Move-only.

Definition at line 25 of file [function.hpp](#).

11.50.2 Member Typedef Documentation

11.50.2.1 Fn

```
template<typename R, typename... Args>
using mxvm::Function< R(Args...)>::Fn = R (*) (Args...)
```

Definition at line 27 of file [function.hpp](#).

11.50.3 Constructor & Destructor Documentation

11.50.3.1 Function() [1/4]

```
template<typename R, typename... Args>
mxvm::Function< R(Args...)>::Function (
    char const * lib,
    char const * name) [inline]
```

Load a function from a named shared library.

Parameters

<i>lib</i>	Path to the shared library
<i>name</i>	Symbol name to resolve

Exceptions

<i>std::runtime_error</i>	if dlopen or dlsym fails
---------------------------	--------------------------

Definition at line 35 of file [function.hpp](#).

References [fn](#), and [handle](#).

Referenced by [Function\(\)](#), [Function\(\)](#), [operator=\(\)](#), and [operator=\(\)](#).

11.50.3.2 Function() [2/4]

```
template<typename R, typename... Args>
mxvm::Function< R(Args...)>::Function (
    char const * name) [inline]
```

Load a function from the main program's symbol table.

Parameters

<i>name</i>	Symbol name to resolve (via dlopen(nullptr, ...))
-------------	---

Exceptions

<i>std::runtime_error</i>	if dlsym fails
---------------------------	----------------

Definition at line 52 of file [function.hpp](#).

References [fn](#), and [handle](#).

11.50.3.3 ~Function()

```
template<typename R, typename... Args>
mxvm::Function< R(Args...)>::~~Function () [inline]
```

Definition at line 64 of file [function.hpp](#).

References [handle](#).

11.50.3.4 Function() [3/4]

```
template<typename R, typename... Args>
mxvm::Function< R(Args...)>::Function (
    const Function< R(Args...)> & ) [delete]
```

References [Function\(\)](#).

11.50.3.5 Function() [4/4]

```
template<typename R, typename... Args>
mxvm::Function< R(Args...)>::Function (
    Function< R(Args...)> && other) [inline], [noexcept]
```

Definition at line 72 of file [function.hpp](#).

References [Function\(\)](#), [fn](#), and [handle](#).

11.50.4 Member Function Documentation

11.50.4.1 operator>()

```
template<typename R, typename... Args>
R mxvm::Function< R(Args...)>::operator() (
    Args... args) const [inline]
```

Invoke the loaded function.

Definition at line 89 of file [function.hpp](#).

References [fn](#).

11.50.4.2 operator=() [1/2]

```
template<typename R, typename... Args>
Function & mxvm::Function< R(Args...)>::operator= (
    const Function< R(Args...)> & ) [delete]
```

References [Function\(\)](#).

11.50.4.3 operator=() [2/2]

```
template<typename R, typename... Args>
Function & mxvm::Function< R(Args...)>::operator= (
    Function< R(Args...)> && other) [inline], [noexcept]
```

Definition at line 76 of file [function.hpp](#).

References [Function\(\)](#), [fn](#), and [handle](#).

11.50.5 Member Data Documentation

11.50.5.1 fn

```
template<typename R, typename... Args>
Fn mxvm::Function< R(Args...)>::fn = nullptr [private]
resolved function pointer
```

Definition at line 97 of file [function.hpp](#).

Referenced by [Function\(\)](#), [Function\(\)](#), [Function\(\)](#), [operator>\(\)](#), and [operator=\(\)](#).

11.50.5.2 handle

```
template<typename R, typename... Args>
void* mxvm::Function< R(Args...)>::handle = nullptr [private]
dlopen library handle
```

Definition at line 96 of file [function.hpp](#).

Referenced by [Function\(\)](#), [Function\(\)](#), [Function\(\)](#), [~Function\(\)](#), and [operator=\(\)](#).

The documentation for this class was generated from the following file:

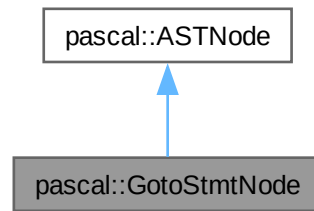
- [include/mxvm/function.hpp](#)

11.51 pascal::GotoStmtNode Class Reference

AST node for a goto statement.

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::GotoStmtNode`:



Public Member Functions

- `GotoStmtNode` (const std::string &lbl)
- void `accept` (ASTVisitor &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string `toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual `~ASTNode` ()=default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::string `label`
target label (numeric string)

11.51.1 Detailed Description

AST node for a goto statement.
Definition at line [681](#) of file [ast.hpp](#).

11.51.2 Constructor & Destructor Documentation

11.51.2.1 GotoStmtNode()

```
pascal::GotoStmtNode::GotoStmtNode (
    const std::string & lbl) [inline]
```

Definition at line [685](#) of file [ast.hpp](#).
References [label](#).

11.51.3 Member Function Documentation

11.51.3.1 accept()

```
void pascal::GotoStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 446 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.51.3.2 toString()

```
std::string pascal::GotoStmtNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 688 of file [ast.hpp](#).

References [label](#).

11.51.4 Member Data Documentation

11.51.4.1 label

```
std::string pascal::GotoStmtNode::label
```

target label (numeric string)

Definition at line 683 of file [ast.hpp](#).

Referenced by [GotoStmtNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.52 mxvm::HTMLGen Class Reference

Generates syntax-highlighted HTML output from MXVM source code.

```
#include <html_gen.hpp>
```

Public Member Functions

- [HTMLGen](#) (const std::string &src)
Construct an HTML generator from MXVM source text.
- void [output](#) (std::ostream &out, std::string filename)
Write the generated HTML to an output stream.

Private Attributes

- std::string [source](#)
the MXVM source text to convert

11.52.1 Detailed Description

Generates syntax-highlighted HTML output from MXVM source code.

Definition at line 15 of file [html_gen.hpp](#).

11.52.2 Constructor & Destructor Documentation

11.52.2.1 HTMLGen()

```
mxvm::HTMLGen::HTMLGen (
    const std::string & src)
```

Construct an HTML generator from MXVM source text.

Parameters

<i>src</i>	MXVM source code string
------------	-------------------------

Definition at line 84 of file [html_gen.cpp](#).

References [source](#).

11.52.3 Member Function Documentation

11.52.3.1 output()

```
void mxvm::HTMLGen::output (
    std::ostream & out,
    std::string filename)
```

Write the generated HTML to an output stream.

Parameters

<i>out</i>	Output stream to write HTML to
<i>filename</i>	Source filename used in the HTML title

Definition at line 86 of file [html_gen.cpp](#).

References [mxvm::is_label\(\)](#), [mxvm::kInstructions](#), [mxvm::kKeywords](#), [mxvm::kTypes](#), [mxvm::None](#), and [source](#).

Referenced by [writeHtml\(\)](#).

11.52.4 Member Data Documentation

11.52.4.1 source

```
std::string mxvm::HTMLGen::source [private]
```

the MXVM source text to convert

Definition at line 31 of file [html_gen.hpp](#).

Referenced by [HTMLGen\(\)](#), and [output\(\)](#).

The documentation for this class was generated from the following files:

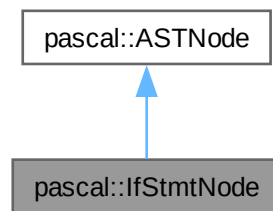
- [include/mxvm/html_gen.hpp](#)
- [src/html_gen.cpp](#)

11.53 pascal::IfStmtNode Class Reference

AST node for an if-then-else statement.

```
#include <ast.hpp>
```


Inheritance diagram for pascal::IfStmtNode:



Public Member Functions

- `IfStmtNode` (`std::unique_ptr< ASTNode > cond`, `std::unique_ptr< ASTNode > thenStmt`, `std::unique_ptr< ASTNode > elseStmt=nullptr`)
- `void accept (ASTVisitor &visitor)` override
Accept an `ASTVisitor` (double-dispatch).
- `std::string toString ()` const override
Return a human-readable description of this node.

Public Member Functions inherited from pascal::ASTNode

- `virtual ~ASTNode ()`=default
- `virtual void print (std::ostream &out, int indent=0)` const
Pretty-print this node to a stream with indentation.
- `void setLineNumber (int line)`
Set the originating source line number.
- `int getLineNumber ()` const
Get the originating source line number.

Public Attributes

- `std::unique_ptr< ASTNode > condition`
boolean condition
- `std::unique_ptr< ASTNode > thenStatement`
then branch
- `std::unique_ptr< ASTNode > elseStatement`
optional else branch

11.53.1 Detailed Description

AST node for an if-then-else statement.
Definition at line 350 of file `ast.hpp`.

11.53.2 Constructor & Destructor Documentation

11.53.2.1 IfStmtNode()

```

pascal::IfStmtNode::IfStmtNode (
    std::unique_ptr< ASTNode > cond,

```

```
std::unique_ptr< ASTNode > thenStmt,
std::unique_ptr< ASTNode > elseStmt = nullptr) [inline]
```

Definition at line 356 of file [ast.hpp](#).

References [condition](#), [elseStatement](#), and [thenStatement](#).

11.53.3 Member Function Documentation

11.53.3.1 accept()

```
void pascal::IfStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 125 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.53.3.2 toString()

```
std::string pascal::IfStmtNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 129 of file [ast.cpp](#).

References [elseStatement](#).

11.53.4 Member Data Documentation

11.53.4.1 condition

```
std::unique_ptr<ASTNode> pascal::IfStmtNode::condition
```

boolean condition

Definition at line 352 of file [ast.hpp](#).

Referenced by [IfStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.53.4.2 elseStatement

```
std::unique_ptr<ASTNode> pascal::IfStmtNode::elseStatement
```

optional else branch

Definition at line 354 of file [ast.hpp](#).

Referenced by [IfStmtNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.53.4.3 thenStatement

```
std::unique_ptr<ASTNode> pascal::IfStmtNode::thenStatement
```

then branch

Definition at line 353 of file [ast.hpp](#).

Referenced by [IfStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.54 mxvm::Instruction Struct Reference

A complete MXVM instruction with opcode, operands, and optional label.

```
#include <instruct.hpp>
```

Public Member Functions

- `std::string toString () const`
Convert this instruction to its assembly text representation.

Public Attributes

- [Inc instruction](#)
the opcode
- [Operand op1](#)
- [Operand op2](#)
- [Operand op3](#)
up to three fixed operands
- `std::vector< Operand > vop`
variable-length operand list (for print, invoke, etc.)
- `std::string label = ""`
label at this instruction address

11.54.1 Detailed Description

A complete MXVM instruction with opcode, operands, and optional label.
Definition at line 168 of file [instruct.hpp](#).

11.54.2 Member Function Documentation

11.54.2.1 toString()

```
std::string mxvm::Instruction::toString () const
```

Convert this instruction to its assembly text representation.

Returns

Formatted string with opcode, operands, and label

Definition at line 84 of file [instruct.cpp](#).

11.54.3 Member Data Documentation

11.54.3.1 instruction

```
Inc mxvm::Instruction::instruction
```

the opcode

Definition at line 169 of file [instruct.hpp](#).

Referenced by [mxvm::Program::exec\(\)](#), [mxvm::Program::flatten_inc\(\)](#), [mxvm::Program::gen_jump\(\)](#), [mxvm::Program::generateCode\(\)](#), [mxvm::Program::generateInstruction\(\)](#), [operator<<\(\)](#), [mxvm::Program::x64_gen_jump\(\)](#), [mxvm::Program::x64_generateCode\(\)](#), and [mxvm::Program::x64_generateInstruction\(\)](#).

11.54.3.2 label

```
std::string mxvm::Instruction::label = ""
```

label at this instruction address

Definition at line 172 of file [instruct.hpp](#).

11.54.3.3 op1

```
Operand mxvm::Instruction::op1
```

Definition at line 170 of file [instruct.hpp](#).

Referenced by [mxvm::Program::exec_add\(\)](#), [mxvm::Program::exec_alloc\(\)](#), [mxvm::Program::exec_and\(\)](#), [mxvm::Program::exec_call\(\)](#), [mxvm::Program::exec_cmp\(\)](#), [mxvm::Program::exec_div\(\)](#), [mxvm::Program::exec_exit\(\)](#), [mxvm::Program::exec_fcmp\(\)](#), [mxvm::Program::exec_free\(\)](#), [mxvm::Program::exec_getline\(\)](#), [mxvm::Program::exec_invoke\(\)](#), [mxvm::Program::exec_jump\(\)](#), [mxvm::Program::exec_lea\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_mod\(\)](#), [mxvm::Program::exec_mov\(\)](#), [mxvm::Program::exec_mul\(\)](#), [mxvm::Program::exec_neg\(\)](#), [mxvm::Program::exec_not\(\)](#), [mxvm::Program::exec_or\(\)](#), [mxvm::Program::exec_pop\(\)](#), [mxvm::Program::exec_print\(\)](#), [mxvm::Program::exec_push\(\)](#), [mxvm::Program::exec_realloc\(\)](#), [mxvm::Program::exec_return\(\)](#), [mxvm::Program::exec_stack_load\(\)](#), [mxvm::Program::exec_stack_store\(\)](#), [mxvm::Program::exec_stack_sub\(\)](#), [mxvm::Program::exec_store\(\)](#), [mxvm::Program::exec_string_print\(\)](#), [mxvm::Program::exec_sub\(\)](#),

mxvm::Program::exec_to_float(), mxvm::Program::exec_to_int(), mxvm::Program::exec_xor(), mxvm::Program::flatten_inc(), mxvm::Program::gen_alloc(), mxvm::Program::gen_arth(), mxvm::Program::gen_bitop(), mxvm::Program::gen_call(), mxvm::Program::gen_cmp(), mxvm::Program::gen_div(), mxvm::Program::gen_exit(), mxvm::Program::gen_fcmp(), mxvm::Program::gen_free(), mxvm::Program::gen_getline(), mxvm::Program::gen_invoke(), mxvm::Program::gen_jae(), mxvm::Program::gen_jbe(), mxvm::Program::gen_jc(), mxvm::Program::gen_jmp(), mxvm::Program::gen_jnc(), mxvm::Program::gen_jno(), mxvm::Program::gen_jnp(), mxvm::Program::gen_jns(), mxvm::Program::gen_jo(), mxvm::Program::gen_jp(), mxvm::Program::gen_js(), mxvm::Program::gen_lea(), mxvm::Program::gen_load(), mxvm::Program::gen_mod(), mxvm::Program::gen_mov(), mxvm::Program::gen_neg(), mxvm::Program::gen_not(), mxvm::Program::gen_pop(), mxvm::Program::gen_print(), mxvm::Program::gen_push(), mxvm::Program::gen_realloc(), mxvm::Program::gen_return(), mxvm::Program::gen_stack_load(), mxvm::Program::gen_stack_store(), mxvm::Program::gen_stack_sub(), mxvm::Program::gen_store(), mxvm::Program::gen_to_float(), mxvm::Program::gen_to_int(), operator<<(), mxvm::Program::x64_gen_alloc(), mxvm::Program::x64_gen_arth(), mxvm::Program::x64_gen_bitop(), mxvm::Program::x64_gen_call(), mxvm::Program::x64_gen_cmp(), mxvm::Program::x64_gen_div(), mxvm::Program::x64_gen_exit(), mxvm::Program::x64_gen_fcmp(), mxvm::Program::x64_gen_free(), mxvm::Program::x64_gen_getline(), mxvm::Program::x64_gen_invoke(), mxvm::Program::x64_gen_jae(), mxvm::Program::x64_gen_jbe(), mxvm::Program::x64_gen_jc(), mxvm::Program::x64_gen_jmp(), mxvm::Program::x64_gen_jnc(), mxvm::Program::x64_gen_jno(), mxvm::Program::x64_gen_jnp(), mxvm::Program::x64_gen_jns(), mxvm::Program::x64_gen_jo(), mxvm::Program::x64_gen_jp(), mxvm::Program::x64_gen_js(), mxvm::Program::x64_gen_lea(), mxvm::Program::x64_gen_load(), mxvm::Program::x64_gen_mod(), mxvm::Program::x64_gen_mov(), mxvm::Program::x64_gen_neg(), mxvm::Program::x64_gen_not(), mxvm::Program::x64_gen_pop(), mxvm::Program::x64_gen_print(), mxvm::Program::x64_gen_push(), mxvm::Program::x64_gen_realloc(), mxvm::Program::x64_gen_return(), mxvm::Program::x64_gen_stack_load(), mxvm::Program::x64_gen_stack_store(), mxvm::Program::x64_gen_stack_sub(), mxvm::Program::x64_gen_store(), mxvm::Program::x64_gen_to_float(), and mxvm::Program::x64_gen_to_int().

11.54.3.4 op2

Operand mxvm::Instruction::op2

Definition at line 170 of file `instruct.hpp`.

Referenced by mxvm::Program::exec_add(), mxvm::Program::exec_alloc(), mxvm::Program::exec_and(), mxvm::Program::exec_cmp(), mxvm::Program::exec_div(), mxvm::Program::exec_fcmp(), mxvm::Program::exec_invoke(), mxvm::Program::exec_lea(), mxvm::Program::exec_load(), mxvm::Program::exec_mod(), mxvm::Program::exec_mov(), mxvm::Program::exec_mul(), mxvm::Program::exec_or(), mxvm::Program::exec_print(), mxvm::Program::exec_realloc(), mxvm::Program::exec_stack_load(), mxvm::Program::exec_stack_store(), mxvm::Program::exec_store(), mxvm::Program::exec_string_print(), mxvm::Program::exec_sub(), mxvm::Program::exec_to_float(), mxvm::Program::exec_to_int(), mxvm::Program::exec_xor(), mxvm::Program::flatten_inc(), mxvm::Program::gen_alloc(), mxvm::Program::gen_arth(), mxvm::Program::gen_bitop(), mxvm::Program::gen_cmp(), mxvm::Program::gen_div(), mxvm::Program::gen_fcmp(), mxvm::Program::gen_invoke(), mxvm::Program::gen_lea(), mxvm::Program::gen_load(), mxvm::Program::gen_mod(), mxvm::Program::gen_mov(), mxvm::Program::gen_print(), mxvm::Program::gen_realloc(), mxvm::Program::gen_stack_load(), mxvm::Program::gen_stack_store(), mxvm::Program::gen_store(), mxvm::Program::gen_to_float(), mxvm::Program::gen_to_int(), operator<<(), mxvm::Program::x64_gen_alloc(), mxvm::Program::x64_gen_arth(), mxvm::Program::x64_gen_bitop(), mxvm::Program::x64_gen_call(), mxvm::Program::x64_gen_cmp(), mxvm::Program::x64_gen_div(), mxvm::Program::x64_gen_fcmp(), mxvm::Program::x64_gen_invoke(), mxvm::Program::x64_gen_lea(), mxvm::Program::x64_gen_load(), mxvm::Program::x64_gen_mod(), mxvm::Program::x64_gen_mov(), mxvm::Program::x64_gen_print(), mxvm::Program::x64_gen_realloc(), mxvm::Program::x64_gen_stack_load(), mxvm::Program::x64_gen_stack_store(), mxvm::Program::x64_gen_store(), mxvm::Program::x64_gen_to_float(), and mxvm::Program::x64_gen_to_int().

11.54.3.5 op3

Operand mxvm::Instruction::op3

up to three fixed operands

Definition at line 170 of file `instruct.hpp`.

Referenced by mxvm::Program::exec_add(), mxvm::Program::exec_alloc(), mxvm::Program::exec_and(), mxvm::Program::exec_div(), mxvm::Program::exec_invoke(), mxvm::Program::exec_load(), mxvm::Program::exec_mod(), mxvm::Program::exec_mul(), mxvm::Program::exec_or(), mxvm::Program::exec_print(), mxvm::Program::exec_realloc(), mxvm::Program::exec_store(), mxvm::Program::exec_string_print(), mxvm::Program::exec_sub(), mxvm::Program::exec_xor(), mxvm::Program::flatten_inc(), mxvm::Program::gen_alloc(), mxvm::Program::gen_arth(), mxvm::Program::gen_bitop(), mxvm::Program::gen_div(), mxvm::Program::gen_invoke(), mxvm::Program::gen_load(), mxvm::Program::gen_mod(), mxvm::Program::gen_print(), mxvm::Program::gen_realloc(), mxvm::Program::gen_store(), operator<<(), mxvm::Program::x64_gen_alloc(), mxvm::Program::x64_gen_arth(), mxvm::Program::x64_gen_bitop(), mxvm::Program::x64_gen_div(), mxvm::Program::x64_gen_invoke(), mxvm::Program::x64_gen_load(), mxvm::Program::x64_gen_mod(), mxvm::Program::x64_gen_print(), mxvm::Program::x64_gen_realloc(), and mxvm::Program::x64_gen_store().

11.54.3.6 vop

`std::vector<Operand> mxvm::Instruction::vop`

variable-length operand list (for print, invoke, etc.)

Definition at line 171 of file [instruct.hpp](#).

Referenced by [mxvm::Program::exec_invoke\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_print\(\)](#), [mxvm::Program::exec_store\(\)](#), [mxvm::Program::exec_string_print\(\)](#), [mxvm::Program::flatten_inc\(\)](#), [mxvm::Program::gen_invoke\(\)](#), [mxvm::Program::gen_load\(\)](#), [mxvm::Program::gen_print\(\)](#), [mxvm::Program::gen_store\(\)](#), [operator<<\(\)](#), [mxvm::Program::x64_gen_invoke\(\)](#), [mxvm::Program::x64_gen_load\(\)](#), [mxvm::Program::x64_gen_print\(\)](#), and [mxvm::Program::x64_gen_store\(\)](#).

The documentation for this struct was generated from the following files:

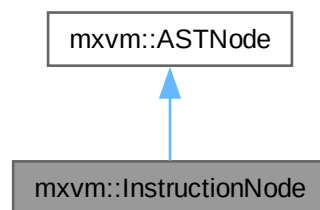
- [include/mxvm/instruct.hpp](#)
- [src/instruct.cpp](#)

11.55 mxvm::InstructionNode Class Reference

AST node representing a single MXVM instruction with its operands.

`#include <ast.hpp>`

Inheritance diagram for `mxvm::InstructionNode`:



Public Member Functions

- [InstructionNode](#) ([Inc](#) inst)
- [InstructionNode](#) ([Inc](#) inst, const `std::vector< Operand >` &ops)
- void [addOperand](#) (const [Operand](#) &operand)
- void [accept](#) ([ASTVisitor](#) &visitor) override
- `std::string` [toString](#) () const override

Public Member Functions inherited from [mxvm::ASTNode](#)

- virtual [~ASTNode](#) ()=default

Public Attributes

- [Inc](#) [instruction](#)
instruction opcode
- `std::vector< Operand >` [operands](#)

11.55.1 Detailed Description

AST node representing a single MXVM instruction with its operands.

Definition at line 113 of file [ast.hpp](#).

11.55.2 Constructor & Destructor Documentation

11.55.2.1 InstructionNode() [1/2]

```
mxvm::InstructionNode::InstructionNode (
    Inc inst) [inline]
```

Definition at line 118 of file [ast.hpp](#).

References [instruction](#).

11.55.2.2 InstructionNode() [2/2]

```
mxvm::InstructionNode::InstructionNode (
    Inc inst,
    const std::vector< Operand > & ops) [inline]
```

Definition at line 120 of file [ast.hpp](#).

References [instruction](#), and [operands](#).

11.55.3 Member Function Documentation

11.55.3.1 accept()

```
void mxvm::InstructionNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 41 of file [ast.cpp](#).

References [mxvm::ASTVisitor::visit\(\)](#).

11.55.3.2 addOperand()

```
void mxvm::InstructionNode::addOperand (
    const Operand & operand) [inline]
```

Definition at line 123 of file [ast.hpp](#).

References [operands](#).

11.55.3.3 toString()

```
std::string mxvm::InstructionNode::toString () const [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 45 of file [ast.cpp](#).

References [instruction](#), and [operands](#).

11.55.4 Member Data Documentation

11.55.4.1 instruction

```
Inc mxvm::InstructionNode::instruction
```

instruction opcode

Definition at line 115 of file [ast.hpp](#).

Referenced by [InstructionNode\(\)](#), [InstructionNode\(\)](#), and [toString\(\)](#).

11.55.4.2 operands

```
std::vector<Operand> mxvm::InstructionNode::operands
```

Definition at line 116 of file [ast.hpp](#).

Referenced by [InstructionNode\(\)](#), [addOperand\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

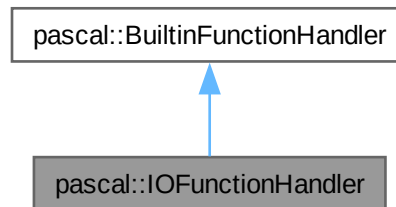
- [include/mxvm/ast.hpp](#)
- [src/ast.cpp](#)

11.56 pascal::IOFunctionHandler Class Reference

Built-in handler for I/O procedures (write, writeln, read, readln).

#include <icode.hpp>

Inheritance diagram for pascal::IOFunctionHandler:



Public Member Functions

- bool `canHandle` (const std::string &funcName) const override
Return true if this handler implements funcName.
- void `generate` (CodeGenVisitor &visitor, const std::string &funcName, const std::vector< std::unique_ptr< ASTNode > > &arguments) override
Emit code for a procedure-style call (no return value used).
- bool `generateWithResult` (CodeGenVisitor &visitor, const std::string &funcName, const std::vector< std::unique_ptr< ASTNode > > &arguments) override
Emit code for a function-style call (result pushed on eval stack); return true if handled.

Public Member Functions inherited from pascal::BuiltinFunctionHandler

- virtual `~BuiltinFunctionHandler` ()=default
- virtual `VarType getReturnType` (const std::string &funcName) const
Return the VarType produced by funcName.

11.56.1 Detailed Description

Built-in handler for I/O procedures (write, writeln, read, readln).

Definition at line 131 of file `icode.hpp`.

11.56.2 Member Function Documentation

11.56.2.1 canHandle()

```
bool pascal::IOFunctionHandler::canHandle (
    const std::string & funcName) const [override], [virtual]
```

Return true if this handler implements funcName.

Implements `pascal::BuiltinFunctionHandler`.

Definition at line 50 of file `icode_imp.cpp`.

References `pascal::toLowerCase()`.

11.56.2.2 generate()

```
void pascal::IOFunctionHandler::generate (
    CodeGenVisitor & visitor,
```

```
const std::string & funcName,
const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [virtual]
```

Emit code for a procedure-style call (no return value used).

Implements [pascal::BuiltinFunctionHandler](#).

Definition at line 56 of file [icode_imp.cpp](#).

References [pascal::CodeGenVisitor::allocReg\(\)](#), [pascal::CHAR](#), [pascal::DOUBLE](#), [pascal::CodeGenVisitor::emit\(\)](#), [pascal::CodeGenVisitor::emit1\(\)](#), [pascal::CodeGenVisitor::emit2\(\)](#), [pascal::CodeGenVisitor::emit_invoke\(\)](#), [pascal::CodeGenVisitor::eval\(\)](#), [pascal::CodeGenVisitor::fileVars](#), [pascal::CodeGenVisitor::findMangledName\(\)](#), [pascal::CodeGenVisitor::freeReg\(\)](#), [pascal::CodeGenVisitor::getExpressionType\(\)](#), [pascal::CodeGenVisitor::getVarType\(\)](#), [pascal::INT](#), [pascal::CodeGenVisitor::internString\(\)](#), [pascal::CodeGenVisitor::isParmReg\(\)](#), [pascal::CodeGenVisitor::isReg\(\)](#), [pascal::CodeGenVisitor::ValueLocation::MEMORY](#), [pascal::CodeGenVisitor::newSlotFor\(\)](#), [pascal::PTR](#), [pascal::CodeGenVisitor::rec](#), [pascal::CodeGenVisitor::slotVar\(\)](#), [pascal::STRING](#), [pascal::toLowerCase\(\)](#), [pascal::CodeGenVisitor::usedModules](#), and [pascal::CodeGenVisitor::usedStrings](#).

11.56.2.3 generateWithResult()

```
bool pascal::IOFunctionHandler::generateWithResult (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [virtual]
```

Emit code for a function-style call (result pushed on eval stack); return true if handled.

Reimplemented from [pascal::BuiltinFunctionHandler](#).

Definition at line 196 of file [icode_imp.cpp](#).

References [pascal::CodeGenVisitor::allocReg\(\)](#), [pascal::CodeGenVisitor::emit\(\)](#), [pascal::CodeGenVisitor::emit_invoke\(\)](#), [pascal::CodeGenVisitor::eval\(\)](#), [pascal::CodeGenVisitor::freeReg\(\)](#), [pascal::CodeGenVisitor::isParmReg\(\)](#), [pascal::CodeGenVisitor::isReg\(\)](#), [pascal::CodeGenVisitor::pushValue\(\)](#), [pascal::toLowerCase\(\)](#), and [pascal::CodeGenVisitor::usedModules](#).

The documentation for this class was generated from the following files:

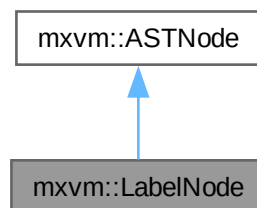
- [src/frontend/include/icode.hpp](#)
- [src/frontend/icode_imp.cpp](#)

11.57 mxvm::LabelNode Class Reference

AST node for a label (or function entry point) in the code section.

`#include <ast.hpp>`

Inheritance diagram for `mxvm::LabelNode`:



Public Member Functions

- [LabelNode](#) (const std::string &labelName)
- void [accept](#) (ASTVisitor &visitor) override
- std::string [toString](#) () const override

Public Member Functions inherited from [mxvm::ASTNode](#)

- virtual [~ASTNode](#) ()=default

Public Attributes

- std::string [name](#)
- bool [function](#) = false
true if this label marks a callable function

11.57.1 Detailed Description

AST node for a label (or function entry point) in the code section.
Definition at line [132](#) of file [ast.hpp](#).

11.57.2 Constructor & Destructor Documentation

11.57.2.1 LabelNode()

```
mxvm::LabelNode::LabelNode (
    const std::string & labelName) [inline]
```

Definition at line [136](#) of file [ast.hpp](#).

References [name](#).

11.57.3 Member Function Documentation

11.57.3.1 accept()

```
void mxvm::LabelNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line [87](#) of file [ast.cpp](#).

References [mxvm::ASTVisitor::visit\(\)](#).

11.57.3.2 toString()

```
std::string mxvm::LabelNode::toString () const [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line [91](#) of file [ast.cpp](#).

References [name](#).

11.57.4 Member Data Documentation

11.57.4.1 function

```
bool mxvm::LabelNode::function = false
```

true if this label marks a callable function

Definition at line [135](#) of file [ast.hpp](#).

11.57.4.2 name

```
std::string mxvm::LabelNode::name
```

Definition at line [134](#) of file [ast.hpp](#).

Referenced by [LabelNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

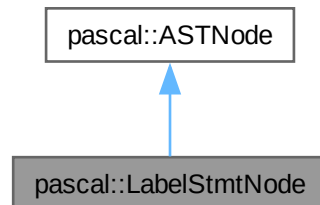
- include/mxvm/[ast.hpp](#)
- src/[ast.cpp](#)

11.58 pascal::LabelStmtNode Class Reference

AST node for a label definition (label: statement).

#include <ast.hpp>

Inheritance diagram for pascal::LabelStmtNode:



Public Member Functions

- [LabelStmtNode](#) (const std::string &lbl, std::unique_ptr< [ASTNode](#) > stmt)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [label](#)
the label (numeric string)
- std::unique_ptr< [ASTNode](#) > [statement](#)
labeled statement

11.58.1 Detailed Description

AST node for a label definition (label: statement).

Definition at line 692 of file [ast.hpp](#).

11.58.2 Constructor & Destructor Documentation

11.58.2.1 LabelStmtNode()

```

pascal::LabelStmtNode::LabelStmtNode (
    const std::string & lbl,
    std::unique_ptr< ASTNode > stmt) [inline]
  
```

Definition at line 697 of file [ast.hpp](#).
References [label](#), and [statement](#).

11.58.3 Member Function Documentation

11.58.3.1 accept()

```
void pascal::LabelStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 450 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.58.3.2 toString()

```
std::string pascal::LabelStmtNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 701 of file [ast.hpp](#).

References [label](#).

11.58.4 Member Data Documentation

11.58.4.1 label

```
std::string pascal::LabelStmtNode::label
```

the label (numeric string)

Definition at line 694 of file [ast.hpp](#).

Referenced by [LabelStmtNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.58.4.2 statement

```
std::unique_ptr<ASTNode> pascal::LabelStmtNode::statement
```

labeled statement

Definition at line 695 of file [ast.hpp](#).

Referenced by [LabelStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

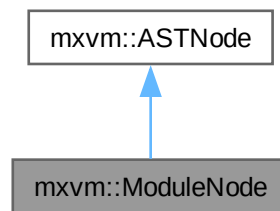
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.59 mxvm::ModuleNode Class Reference

AST node for a module import declaration.

```
#include <ast.hpp>
```

Inheritance diagram for `mxvm::ModuleNode`:



Public Member Functions

- `ModuleNode` (`const std::string &n`)
- `void accept (ASTVisitor &visitor)` override
- `std::string toString ()` const override

Public Member Functions inherited from `mxvm::ASTNode`

- `virtual ~ASTNode ()`=default

Public Attributes

- `std::string name`

11.59.1 Detailed Description

AST node for a module import declaration.
Definition at line 95 of file [ast.hpp](#).

11.59.2 Constructor & Destructor Documentation

11.59.2.1 ModuleNode()

```
mxvm::ModuleNode::ModuleNode (
    const std::string & n) [inline]
```

Definition at line 98 of file [ast.hpp](#).

References [name](#).

11.59.3 Member Function Documentation

11.59.3.1 accept()

```
void mxvm::ModuleNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 67 of file [ast.cpp](#).

References [mxvm::ASTVisitor::visit\(\)](#).

11.59.3.2 toString()

```
std::string mxvm::ModuleNode::toString () const [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 71 of file [ast.cpp](#).
References [name](#).

11.59.4 Member Data Documentation

11.59.4.1 name

std::string mxvm::ModuleNode::name

Definition at line 97 of file [ast.hpp](#).

Referenced by [ModuleNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

- include/mxvm/[ast.hpp](#)
- src/[ast.cpp](#)

11.60 mxvm::ModuleParser Class Reference

[Parser](#) for MXVM module definition files (.mxvm).

```
#include <parser.hpp>
```

Public Member Functions

- [ModuleParser](#) (const [Mode](#) &m, const std::string &mod_name, const std::string &source)
Construct a module parser.
- [ModuleParser](#) (const [ModuleParser](#) &other)
Copy constructor.
- [ModuleParser](#) & operator= (const [ModuleParser](#) &other)
Copy assignment.
- uint64_t [scan](#) ()
Tokenize the module source.
- bool [parse](#) ()
Parse the module declarations.
- [scan::Token](#) operator[] (size_t pos)
Access a token by position.
- [scan::Token](#) at (size_t pos)
Access a token by position (bounds-checked).
- bool [generateProgramCode](#) (const [Mode](#) &m, const std::string &mod_id, const std::string &mod_name, std::unique_ptr< [Program](#) > &program)
Register the module's external functions in a [Program](#).
- bool [next](#) ()
Advance to the next token.
- bool [match](#) (const std::string &s)
Check if the current token value matches and advance.
- bool [match](#) (const [types::TokenType](#) &t)
Check if the current token type matches and advance.
- void [require](#) (const std::string &s)
Require the current token value to match, or throw.
- void [require](#) (const [types::TokenType](#) &t)
Require the current token type to match, or throw.

Protected Attributes

- `std::vector< ExternalFunction > functions`
parsed external function declarations
- `std::string mod\_name`
module name
- `scan::Scanner scanner`
underlying scanner
- `uint64_t index = 0`
current token index
- `scan::TToken * token = nullptr`
current token pointer
- `Mode parser\_mode`
interpret or compile

11.60.1 Detailed Description

[Parser](#) for MXVM module definition files (.mxvm).

Reads a module's function declarations and registers them as external functions in the calling program.

Definition at line 181 of file [parser.hpp](#).

11.60.2 Constructor & Destructor Documentation

11.60.2.1 [ModuleParser\(\)](#) [1/2]

```
mxvm::ModuleParser::ModuleParser (
    const Mode & m,
    const std::string & mod\_name,
    const std::string & source)
```

Construct a module parser.

Parameters

<i>m</i>	Execution mode
<i>mod_name</i>	Module name
<i>source</i>	Module source text

Definition at line 100 of file [parser.cpp](#).

References [mod_name](#), [parser_mode](#), and [scanner](#).

Referenced by [ModuleParser\(\)](#), and [operator=\(\)](#).

11.60.2.2 [ModuleParser\(\)](#) [2/2]

```
mxvm::ModuleParser::ModuleParser (
    const ModuleParser & other) [inline]
```

Copy constructor.

Definition at line 192 of file [parser.hpp](#).

References [ModuleParser\(\)](#), [functions](#), [index](#), [mod_name](#), [scanner](#), and [token](#).

11.60.3 Member Function Documentation

11.60.3.1 [at\(\)](#)

```
scan::TToken mxvm::ModuleParser::at (
    size_t pos)
```

Access a token by position (bounds-checked).

Definition at line 133 of file [parser.cpp](#).

References [pos\(\)](#), and [scanner](#).
 Referenced by [operator\[\]\(\)](#).

11.60.3.2 generateProgramCode()

```
bool mxvm::ModuleParser::generateProgramCode (
    const Mode & m,
    const std::string & mod_id,
    const std::string & mod_name,
    std::unique_ptr< Program > & program)
```

Register the module's external functions in a [Program](#).

Parameters

<i>m</i>	Execution mode
<i>mod_id</i>	Module identifier
<i>mod_name</i>	Module display name
<i>program</i>	Target program

Returns

true on success

Definition at line 141 of file [parser.cpp](#).
 References [functions](#), [mod_name](#), and [mxvm::MODE_INTERPRET](#).

11.60.3.3 match() [1/2]

```
bool mxvm::ModuleParser::match (
    const std::string & s)
```

Check if the current token value matches and advance.

Definition at line 162 of file [parser.cpp](#).

References [token](#).

Referenced by [parse\(\)](#), [require\(\)](#), and [require\(\)](#).

11.60.3.4 match() [2/2]

```
bool mxvm::ModuleParser::match (
    const types::TokenType & t)
```

Check if the current token type matches and advance.

Definition at line 166 of file [parser.cpp](#).

References [token](#).

11.60.3.5 next()

```
bool mxvm::ModuleParser::next ()
```

Advance to the next token.

Definition at line 151 of file [parser.cpp](#).

References [index](#), [scanner](#), [token](#), and [types::TT_STR](#).

Referenced by [parse\(\)](#).

11.60.3.6 operator=()

```
ModuleParser & mxvm::ModuleParser::operator= (
    const ModuleParser & other) [inline]
```

Copy assignment.

Definition at line 200 of file [parser.hpp](#).

References [ModuleParser\(\)](#), [functions](#), [index](#), [mod_name](#), [scanner](#), and [token](#).

11.60.3.7 operator[]()

```
scan::TToken mxvm::ModuleParser::operator[] (
    size_t pos)
```

Access a token by position.

Definition at line 129 of file [parser.cpp](#).

References [at\(\)](#), and [pos\(\)](#).

11.60.3.8 parse()

```
bool mxvm::ModuleParser::parse ()
```

Parse the module declarations.

Definition at line 107 of file [parser.cpp](#).

References [functions](#), [match\(\)](#), [mod_name](#), [next\(\)](#), [require\(\)](#), [token](#), and [types::TT_ID](#).

11.60.3.9 require() [1/2]

```
void mxvm::ModuleParser::require (
    const std::string & s)
```

Require the current token value to match, or throw.

Definition at line 170 of file [parser.cpp](#).

References [match\(\)](#), and [token](#).

Referenced by [parse\(\)](#).

11.60.3.10 require() [2/2]

```
void mxvm::ModuleParser::require (
    const types::TokenType & t)
```

Require the current token type to match, or throw.

Definition at line 175 of file [parser.cpp](#).

References [match\(\)](#), and [token](#).

11.60.3.11 scan()

```
uint64_t mxvm::ModuleParser::scan ()
```

Tokenize the module source.

Definition at line 102 of file [parser.cpp](#).

References [scanner](#).

11.60.4 Member Data Documentation

11.60.4.1 functions

```
std::vector<ExternalFunction> mxvm::ModuleParser::functions [protected]
```

parsed external function declarations

Definition at line 242 of file [parser.hpp](#).

Referenced by [ModuleParser\(\)](#), [generateProgramCode\(\)](#), [operator=\(\)](#), and [parse\(\)](#).

11.60.4.2 index

```
uint64_t mxvm::ModuleParser::index = 0 [protected]
```

current token index

Definition at line 245 of file [parser.hpp](#).

Referenced by [ModuleParser\(\)](#), [next\(\)](#), and [operator=\(\)](#).

11.60.4.3 mod_name

`std::string mxvm::ModuleParser::mod_name` [protected]

module name

Definition at line 243 of file [parser.hpp](#).

Referenced by [ModuleParser\(\)](#), [ModuleParser\(\)](#), [generateProgramCode\(\)](#), [operator=\(\)](#), and [parse\(\)](#).

11.60.4.4 parser_mode

`Mode mxvm::ModuleParser::parser_mode` [protected]

interpret or compile

Definition at line 247 of file [parser.hpp](#).

Referenced by [ModuleParser\(\)](#).

11.60.4.5 scanner

`scan::Scanner mxvm::ModuleParser::scanner` [protected]

underlying scanner

Definition at line 244 of file [parser.hpp](#).

Referenced by [ModuleParser\(\)](#), [ModuleParser\(\)](#), [at\(\)](#), [next\(\)](#), [operator=\(\)](#), and [scan\(\)](#).

11.60.4.6 token

`scan::TToken* mxvm::ModuleParser::token = nullptr` [protected]

current token pointer

Definition at line 246 of file [parser.hpp](#).

Referenced by [ModuleParser\(\)](#), [match\(\)](#), [match\(\)](#), [next\(\)](#), [operator=\(\)](#), [parse\(\)](#), [require\(\)](#), and [require\(\)](#).

The documentation for this class was generated from the following files:

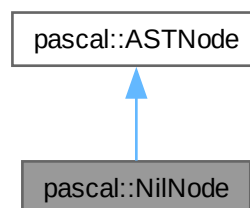
- [include/mxvm/parser.hpp](#)
- [src/parser.cpp](#)

11.61 pascal::NilNode Class Reference

AST node for the nil literal.

```
#include <ast.hpp>
```

Inheritance diagram for pascal::NilNode:



Public Member Functions

- [NilNode](#) ()=default
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).

- `std::string toString ()` const override
Return a human-readable description of this node.

Public Member Functions inherited from `pascal::ASTNode`

- virtual `~ASTNode ()`=default
- virtual void `print (std::ostream &out, int indent=0)` const
Pretty-print this node to a stream with indentation.
- void `setLineNumber (int line)`
Set the originating source line number.
- int `getLineNumber ()` const
Get the originating source line number.

11.61.1 Detailed Description

AST node for the nil literal.
Definition at line 569 of file [ast.hpp](#).

11.61.2 Constructor & Destructor Documentation

11.61.2.1 NilNode()

```
pascal::NilNode::NilNode () [default]
```

11.61.3 Member Function Documentation

11.61.3.1 accept()

```
void pascal::NilNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 426 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.61.3.2 toString()

```
std::string pascal::NilNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 574 of file [ast.hpp](#).

The documentation for this class was generated from the following files:

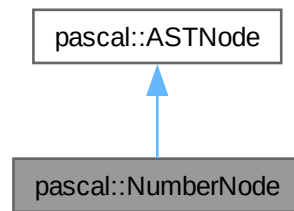
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.62 pascal::NumberNode Class Reference

AST node for an integer or real numeric literal.

```
#include <ast.hpp>
```

Inheritance diagram for pascal::NumberNode:



Public Member Functions

- `NumberNode` (const std::string &val, bool integer=true, bool real=false)
- void `accept` (ASTVisitor &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string `toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual `~ASTNode` ()=default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::string `value`
textual representation of the number
- bool `isInteger`
true if integer literal
- bool `isReal`
true if real (floating-point) literal

11.62.1 Detailed Description

AST node for an integer or real numeric literal.
Definition at line [524](#) of file [ast.hpp](#).

11.62.2 Constructor & Destructor Documentation

11.62.2.1 NumberNode()

```

pascal::NumberNode::NumberNode (
    const std::string & val,
    bool integer = true,
    bool real = false) [inline]
  
```

Definition at line 530 of file [ast.hpp](#).
References [isInteger](#), [isReal](#), and [value](#).

11.62.3 Member Function Documentation

11.62.3.1 `accept()`

```
void pascal::NumberNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 247 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.62.3.2 `toString()`

```
std::string pascal::NumberNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 251 of file [ast.cpp](#).

References [isInteger](#), and [value](#).

11.62.4 Member Data Documentation

11.62.4.1 `isInteger`

```
bool pascal::NumberNode::isInteger
```

true if integer literal

Definition at line 527 of file [ast.hpp](#).

Referenced by [NumberNode\(\)](#), and [toString\(\)](#).

11.62.4.2 `isReal`

```
bool pascal::NumberNode::isReal
```

true if real (floating-point) literal

Definition at line 528 of file [ast.hpp](#).

Referenced by [NumberNode\(\)](#), [pascal::CodeGenVisitor::getExpressionType\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.62.4.3 `value`

```
std::string pascal::NumberNode::value
```

textual representation of the number

Definition at line 526 of file [ast.hpp](#).

Referenced by [NumberNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

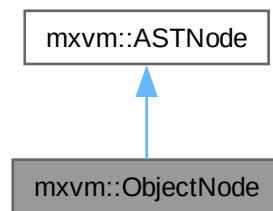
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.63 `mxvm::ObjectNode` Class Reference

AST node for an object import declaration.

```
#include <ast.hpp>
```

Inheritance diagram for mxvm::ObjectNode:



Public Member Functions

- [ObjectNode](#) (const std::string &o)
- void [accept](#) (ASTVisitor &visitor) override
- std::string [toString](#) () const override

Public Member Functions inherited from [mxvm::ASTNode](#)

- virtual [~ASTNode](#) ()=default

Public Attributes

- std::string [name](#)

11.63.1 Detailed Description

AST node for an object import declaration.
Definition at line 104 of file [ast.hpp](#).

11.63.2 Constructor & Destructor Documentation

11.63.2.1 ObjectNode()

```
mxvm::ObjectNode::ObjectNode (
    const std::string & o) [inline]
```

Definition at line 107 of file [ast.hpp](#).

References [name](#).

11.63.3 Member Function Documentation

11.63.3.1 accept()

```
void mxvm::ObjectNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 77 of file [ast.cpp](#).

References [mxvm::ASTVisitor::visit\(\)](#).

11.63.3.2 toString()

```
std::string mxvm::ObjectNode::toString () const [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 81 of file [ast.cpp](#).
References [name](#).

11.63.4 Member Data Documentation

11.63.4.1 name

`std::string mxvm::ObjectNode::name`

Definition at line 106 of file [ast.hpp](#).

Referenced by [ObjectNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

- [include/mxvm/ast.hpp](#)
- [src/ast.cpp](#)

11.64 mxvm::Operand Struct Reference

A single instruction operand (constant value or variable reference).

`#include <instruct.hpp>`

Public Attributes

- `std::string label`
label target for jump/call operands
- `std::string op`
textual operand value
- `int op_value = 0`
numeric operand value
- `OperandType type`
- `std::string object`
owning object name for member access

11.64.1 Detailed Description

A single instruction operand (constant value or variable reference).

Definition at line 159 of file [instruct.hpp](#).

11.64.2 Member Data Documentation

11.64.2.1 label

`std::string mxvm::Operand::label`

label target for jump/call operands

Definition at line 160 of file [instruct.hpp](#).

Referenced by [mxvm::Program::getMangledName\(\)](#), and [mxvm::Parser::parseCodeInstruction\(\)](#).

11.64.2.2 object

`std::string mxvm::Operand::object`

owning object name for member access

Definition at line 164 of file [instruct.hpp](#).

Referenced by [mxvm::Program::getMangledName\(\)](#), and [mxvm::Parser::parseCodeInstruction\(\)](#).

11.64.2.3 op

`std::string mxvm::Operand::op`

textual operand value

Definition at line 161 of file `instruct.hpp`.

Referenced by `mxvm::Program::exec_add()`, `mxvm::Program::exec_alloc()`, `mxvm::Program::exec_and()`, `mxvm::Program::exec_call()`, `mxvm::Program::exec_cmp()`, `mxvm::Program::exec_div()`, `mxvm::Program::exec_exit()`, `mxvm::Program::exec_fcmp()`, `mxvm::Program::exec_free()`, `mxvm::Program::exec_getline()`, `mxvm::Program::exec_invoke()`, `mxvm::Program::exec_jump()`, `mxvm::Program::exec_lea()`, `mxvm::Program::exec_load()`, `mxvm::Program::exec_mod()`, `mxvm::Program::exec_mov()`, `mxvm::Program::exec_mul()`, `mxvm::Program::exec_neg()`, `mxvm::Program::exec_not()`, `mxvm::Program::exec_or()`, `mxvm::Program::exec_pop()`, `mxvm::Program::exec_print()`, `mxvm::Program::exec_push()`, `mxvm::Program::exec_realloc()`, `mxvm::Program::exec_return()`, `mxvm::Program::exec_stack_load()`, `mxvm::Program::exec_stack_store()`, `mxvm::Program::exec_stack_sub()`, `mxvm::Program::exec_store()`, `mxvm::Program::exec_string_print()`, `mxvm::Program::exec_sub()`, `mxvm::Program::exec_to_float()`, `mxvm::Program::exec_to_int()`, `mxvm::Program::exec_xor()`, `mxvm::Program::gen_alloc()`, `mxvm::Program::gen_arth()`, `mxvm::Program::gen_bitop()`, `mxvm::Program::gen_call()`, `mxvm::Program::gen_cmp()`, `mxvm::Program::gen_div()`, `mxvm::Program::gen_exit()`, `mxvm::Program::gen_fcmp()`, `mxvm::Program::gen_free()`, `mxvm::Program::gen_getline()`, `mxvm::Program::gen_invoke()`, `mxvm::Program::gen_jae()`, `mxvm::Program::gen_jbe()`, `mxvm::Program::gen_jc()`, `mxvm::Program::gen_jump()`, `mxvm::Program::gen_jnc()`, `mxvm::Program::gen_jno()`, `mxvm::Program::gen_jnp()`, `mxvm::Program::gen_jns()`, `mxvm::Program::gen_jo()`, `mxvm::Program::gen_jp()`, `mxvm::Program::gen_js()`, `mxvm::Program::gen_lea()`, `mxvm::Program::gen_load()`, `mxvm::Program::gen_mod()`, `mxvm::Program::gen_mov()`, `mxvm::Program::gen_neg()`, `mxvm::Program::gen_not()`, `mxvm::Program::gen_pop()`, `mxvm::Program::gen_print()`, `mxvm::Program::gen_push()`, `mxvm::Program::gen_realloc()`, `mxvm::Program::gen_ret()`, `mxvm::Program::gen_return()`, `mxvm::Program::gen_stack_load()`, `mxvm::Program::gen_stack_store()`, `mxvm::Program::gen_stack_sub()`, `mxvm::Program::gen_store()`, `mxvm::Program::gen_to_float()`, `mxvm::Program::gen_to_int()`, `mxvm::Program::generateCode()`, `mxvm::Program::generateLoadVar()`, `mxvm::Program::generateLoadVar()`, `mxvm::Program::getMangledName()`, `operator<<()`, `mxvm::Parser::parseCodeInstruction()`, `mxvm::Program::sysv_emitLoadVar()`, `mxvm::Program::sysv_emitStoreVar()`, `mxvm::Program::sysv_emitStoreVarImm()`, `mxvm::Program::variableFromOperand()`, `mxvm::Program::x64_emitLoadVar()`, `mxvm::Program::x64_emitStoreVar()`, `mxvm::Program::x64_emitStoreVarImm()`, `mxvm::Program::x64_gen_alloc()`, `mxvm::Program::x64_gen_arth()`, `mxvm::Program::x64_gen_bitop()`, `mxvm::Program::x64_gen_call()`, `mxvm::Program::x64_gen_cmp()`, `mxvm::Program::x64_gen_div()`, `mxvm::Program::x64_gen_exit()`, `mxvm::Program::x64_gen_fcmp()`, `mxvm::Program::x64_gen_free()`, `mxvm::Program::x64_gen_getline()`, `mxvm::Program::x64_gen_invoke()`, `mxvm::Program::x64_gen_jae()`, `mxvm::Program::x64_gen_jbe()`, `mxvm::Program::x64_gen_jc()`, `mxvm::Program::x64_gen_jump()`, `mxvm::Program::x64_gen_jnc()`, `mxvm::Program::x64_gen_jno()`, `mxvm::Program::x64_gen_jnp()`, `mxvm::Program::x64_gen_jns()`, `mxvm::Program::x64_gen_jo()`, `mxvm::Program::x64_gen_jp()`, `mxvm::Program::x64_gen_js()`, `mxvm::Program::x64_gen_lea()`, `mxvm::Program::x64_gen_load()`, `mxvm::Program::x64_gen_mod()`, `mxvm::Program::x64_gen_mov()`, `mxvm::Program::x64_gen_neg()`, `mxvm::Program::x64_gen_not()`, `mxvm::Program::x64_gen_pop()`, `mxvm::Program::x64_gen_print()`, `mxvm::Program::x64_gen_push()`, `mxvm::Program::x64_gen_realloc()`, `mxvm::Program::x64_gen_return()`, `mxvm::Program::x64_gen_stack_load()`, `mxvm::Program::x64_gen_stack_store()`, `mxvm::Program::x64_gen_stack_sub()`, `mxvm::Program::x64_gen_store()`, `mxvm::Program::x64_gen_to_float()`, `mxvm::Program::x64_generateCode()`, `mxvm::Program::x64_generateLoadVar()`, and `mxvm::Program::x64_generateLoadVar()`.

11.64.2.4 op_value

`int mxvm::Operand::op_value = 0`

numeric operand value

Definition at line 162 of file `instruct.hpp`.

Referenced by `mxvm::Parser::parseCodeInstruction()`.

11.64.2.5 type

`OperandType mxvm::Operand::type`

Definition at line 163 of file `instruct.hpp`.

Referenced by `mxvm::Program::exec_print()`, `mxvm::Program::exec_store()`, `mxvm::Program::exec_string_print()`, `mxvm::Program::gen_alloc()`, `mxvm::Program::gen_cmp()`, `mxvm::Program::gen_mov()`, `mxvm::Program::gen_push()`, `mxvm::Program::gen_to_float()`, `mxvm::Program::gen_to_int()`, `mxvm::Program::generateLoadVar()`, `mxvm::Program::generateLoadVar()`, `mxvm::Parser::parseCodeInstruction()`, `mxvm::Program::variableFromOperand()`, `mxvm::Program::x64_gen_cmp()`, `mxvm::Program::x64_gen_push()`, `mxvm::Program::x64_generateLoadVar()`, and `mxvm::Program::x64_generateLoadVar()`.

The documentation for this struct was generated from the following file:

- `include/mxvm/instruct.hpp`

11.65 mxvm::OpSpec Struct Reference

Specification of valid operand patterns for an instruction.

#include <valid.hpp>

Public Attributes

- std::string [name](#)
instruction mnemonic
- std::vector< [OpKind](#) > [fixed](#)
required fixed operand kinds
- [VArity](#) [varPolicy](#) = [VArity::None](#)
- int [minArgs](#) = -1
- int [maxArgs](#) = -1

11.65.1 Detailed Description

Specification of valid operand patterns for an instruction.

Definition at line 55 of file [valid.hpp](#).

11.65.2 Member Data Documentation

11.65.2.1 fixed

std::vector<[OpKind](#)> mxvm::OpSpec::fixed

required fixed operand kinds

Definition at line 57 of file [valid.hpp](#).

Referenced by [mxvm::Validator::validateAgainstSpec\(\)](#).

11.65.2.2 maxArgs

int mxvm::OpSpec::maxArgs = -1

Definition at line 60 of file [valid.hpp](#).

Referenced by [mxvm::Validator::validateAgainstSpec\(\)](#).

11.65.2.3 minArgs

int mxvm::OpSpec::minArgs = -1

Definition at line 59 of file [valid.hpp](#).

Referenced by [mxvm::Validator::validateAgainstSpec\(\)](#).

11.65.2.4 name

std::string mxvm::OpSpec::name

instruction mnemonic

Definition at line 56 of file [valid.hpp](#).

11.65.2.5 varPolicy

[VArity](#) mxvm::OpSpec::varPolicy = [VArity::None](#)

Definition at line 58 of file [valid.hpp](#).

Referenced by [mxvm::Validator::validateAgainstSpec\(\)](#).

The documentation for this struct was generated from the following file:

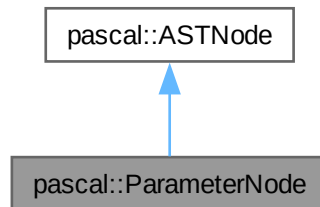
- include/mxvm/[valid.hpp](#)

11.66 pascal::ParameterNode Class Reference

AST node for a formal parameter list entry.

#include <ast.hpp>

Inheritance diagram for pascal::ParameterNode:



Public Member Functions

- [ParameterNode](#) (std::vector< std::string > ids, const std::string ¶mType, bool var=false)
- void [accept](#) (ASTVisitor &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from pascal::ASTNode

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::vector< std::string > [identifiers](#)
parameter names
- std::string [type](#)
parameter type name
- bool [isVar](#)
true if passed by reference (var parameter)

11.66.1 Detailed Description

AST node for a formal parameter list entry.

Definition at line 223 of file [ast.hpp](#).

11.66.2 Constructor & Destructor Documentation

11.66.2.1 ParameterNode()

```
pascal::ParameterNode::ParameterNode (
    std::vector< std::string > ids,
    const std::string & paramType,
    bool var = false) [inline]
```

Definition at line 229 of file [ast.hpp](#).

References [identifiers](#), [isVar](#), and [type](#).

11.66.3 Member Function Documentation

11.66.3.1 accept()

```
void pascal::ParameterNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 91 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.66.3.2 toString()

```
std::string pascal::ParameterNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 95 of file [ast.cpp](#).

References [identifiers](#), [isVar](#), and [type](#).

11.66.4 Member Data Documentation

11.66.4.1 identifiers

```
std::vector<std::string> pascal::ParameterNode::identifiers
```

parameter names

Definition at line 225 of file [ast.hpp](#).

Referenced by [ParameterNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.66.4.2 isVar

```
bool pascal::ParameterNode::isVar
```

true if passed by reference (var parameter)

Definition at line 227 of file [ast.hpp](#).

Referenced by [ParameterNode\(\)](#), and [toString\(\)](#).

11.66.4.3 type

```
std::string pascal::ParameterNode::type
```

parameter type name

Definition at line 226 of file [ast.hpp](#).

Referenced by [ParameterNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.67 mxvm::ParsedOp Struct Reference

A parsed operand with its kind, text, and source location.

```
#include <valid.hpp>
```

Public Attributes

- [OpKind](#) `kind`
- `std::string` `text`
- `const scan::TToken *` `at`
token at the operand's source position

11.67.1 Detailed Description

A parsed operand with its kind, text, and source location.
 Definition at line 30 of file [valid.hpp](#).

11.67.2 Member Data Documentation**11.67.2.1 at**

`const scan::TToken*` `mxvm::ParsedOp::at`
 token at the operand's source position
 Definition at line 33 of file [valid.hpp](#).
 Referenced by [mxvm::Validator::parseOperand\(\)](#).

11.67.2.2 kind

[OpKind](#) `mxvm::ParsedOp::kind`
 Definition at line 31 of file [valid.hpp](#).
 Referenced by [mxvm::Validator::parseOperand\(\)](#).

11.67.2.3 text

`std::string` `mxvm::ParsedOp::text`
 Definition at line 32 of file [valid.hpp](#).
 Referenced by [mxvm::Validator::parseOperand\(\)](#).
 The documentation for this struct was generated from the following file:

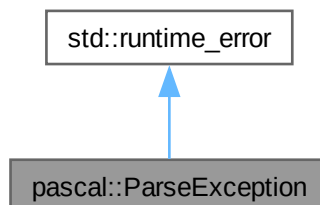
- [include/mxvm/valid.hpp](#)

11.68 pascal::ParseException Class Reference

Exception thrown on parse errors.

```
#include <parser.hpp>
```

Inheritance diagram for `pascal::ParseException`:



Public Member Functions

- [ParseException](#) (const std::string &msg)
Construct with an error message.

11.68.1 Detailed Description

Exception thrown on parse errors.

Definition at line 18 of file [parser.hpp](#).

11.68.2 Constructor & Destructor Documentation

11.68.2.1 ParseException()

```
pascal::ParseException::ParseException (
    const std::string & msg) [inline], [explicit]
```

Construct with an error message.

Definition at line 21 of file [parser.hpp](#).

The documentation for this class was generated from the following file:

- src/frontend/include/[parser.hpp](#)

11.69 mxvm::Parser Class Reference

Main MXVM parser — tokenizes source, builds AST, and generates intermediate code.

```
#include <parser.hpp>
```

Public Member Functions

- [Parser](#) (const std::string &source)
Construct a parser from MXVM source text.
- [Parser](#) (const Parser &other)
Copy constructor.
- [Parser & operator=](#) (const [Parser](#) &other)
Copy assignment.
- uint64_t [scan](#) ()
Tokenize the source and return the token count.
- void [parse](#) ()
Parse all tokens into the internal AST.
- auto [operator\[\]](#) (size_t pos)
Access a token by position.
- std::unique_ptr< [ProgramNode](#) > [parseProgramOrObject](#) (uint64_t &index)
Parse a top-level program or object node.
- std::unique_ptr< [ProgramNode](#) > [parseAST](#) ()
Build the full AST from tokens.
- bool [generateProgramCode](#) (const [Mode](#) &m, std::unique_ptr< [Program](#) > &program)
Walk the AST and populate a [Program](#) with instructions and variables.
- bool [generateDebugHTML](#) (std::ostream &out, std::unique_ptr< [Program](#) > &program)
Generate debug HTML output for the program.
- void [generateObjectAssemblyFile](#) (std::unique_ptr< [Program](#) > &objProgram)
Generate a native assembly file for an object.
- void [registerObjectExterns](#) (std::unique_ptr< [Program](#) > &mainProgram, const std::unique_ptr< [Program](#) > &objProgram)
Register external functions from an object into the main program.

Public Attributes

- `std::string` [source_file](#)
- `scan::Scanner` [scanner](#)
- `Validator` [validator](#)
- `Mode` [parser_mode](#) = `Mode::MODE_INTERPRET`
- `std::string` [module_path](#) = `"."`
search path for .mxvm module files
- `std::string` [object_path](#) = `"."`
search path for .mxvm object files
- `std::string` [include_path](#) = `"/usr/local/include/mxvm/modules"`
system module include path
- `bool` [object_mode](#) = `false`
true when parsing an object definition
- `std::string` [object_name](#)
- `Platform` [platform](#)

Private Member Functions

- `std::unique_ptr< SectionNode >` [parseSection](#) (`uint64_t` &index)
- `std::unique_ptr< VariableNode >` [parseDataVariable](#) (`uint64_t` &index)
- `std::unique_ptr< InstructionNode >` [parseCodeInstruction](#) (`uint64_t` &index)
- `std::unique_ptr< CommentNode >` [parseComment](#) (`uint64_t` &index)
- `std::unique_ptr< LabelNode >` [parseLabel](#) (`uint64_t` &index)
- `std::unique_ptr< ModuleNode >` [parseModule](#) (`uint64_t` &index)
- `std::unique_ptr< ObjectNode >` [parseObject](#) (`uint64_t` &index)
- `void` [processDataSection](#) (`SectionNode` *sectionNode, `std::unique_ptr< Program >` &program)
- `void` [processCodeSection](#) (`SectionNode` *sectionNode, `std::unique_ptr< Program >` &program)
- `void` [processModuleSection](#) (`SectionNode` *sectionNode, `std::unique_ptr< Program >` &program)
- `void` [processObjectSection](#) (`SectionNode` *sectionNode, `std::unique_ptr< Program >` &program)
- `void` [processModuleFile](#) (`const std::string` &src, `std::unique_ptr< Program >` &program)
- `void` [processObjectFile](#) (`const std::string` &src, `std::unique_ptr< Program >` &program)
- `void` [setVariableValue](#) (`Variable` &var, `VarType` type, `const std::string` &value, `size_t` buf_size=0)
- `void` [setDefaultVariableValue](#) (`Variable` &var, `VarType` type)
- `void` [resolveLabelReference](#) (`Operand` &operand, `const std::unordered_map< std::string, size_t >` &labelMap)
- `void` [collectObjectNames](#) (`std::vector< std::pair< std::string, std::string > >` &names, `const std::unique_ptr< Program >` &program)
- `void` [printObjectHTML](#) (`std::ostream` &out, `const std::unique_ptr< Program >` &objPtr)

Friends

- `class` [ModuleParser](#)

11.69.1 Detailed Description

Main MXVM parser — tokenizes source, builds AST, and generates intermediate code.

Handles the full pipeline from source text to a populated [Program](#): scanning tokens, parsing sections (data/code/↵ module/object), resolving modules and objects, and dispatching to the code generator.

Definition at line 57 of file [parser.hpp](#).

11.69.2 Constructor & Destructor Documentation

11.69.2.1 Parser() [1/2]

```
mxvm::Parser::Parser (
    const std::string & source) [explicit]
```

Construct a parser from MXVM source text.

Parameters

<i>source</i>	The MXVM source code to parse
---------------	-------------------------------

Definition at line 181 of file [parser.cpp](#).

References [scanner](#), [source_file](#), and [validator](#).

Referenced by [Parser\(\)](#), [operator=\(\)](#), and [processObjectFile\(\)](#).

11.69.2.2 Parser() [2/2]

```
mxvm::Parser::Parser (
    const Parser & other) [inline]
```

Copy constructor.

Definition at line 67 of file [parser.hpp](#).

References [Parser\(\)](#), [include_path](#), [module_path](#), [object_mode](#), [object_name](#), [object_path](#), [parser_mode](#), [platform](#), [scanner](#), [source_file](#), and [validator](#).

11.69.3 Member Function Documentation

11.69.3.1 collectObjectNames()

```
void mxvm::Parser::collectObjectNames (
    std::vector< std::pair< std::string, std::string > > & names,
    const std::unique_ptr< Program > & program) [private]
```

Definition at line 842 of file [parser.cpp](#).

References [collectObjectNames\(\)](#).

Referenced by [collectObjectNames\(\)](#).

11.69.3.2 generateDebugHTML()

```
bool mxvm::Parser::generateDebugHTML (
    std::ostream & out,
    std::unique_ptr< Program > & program)
```

Generate debug HTML output for the program.

Definition at line 987 of file [parser.cpp](#).

Referenced by [action_interpret\(\)](#), [generateProgramCode\(\)](#), and [translate_x64\(\)](#).

11.69.3.3 generateObjectAssemblyFile()

```
void mxvm::Parser::generateObjectAssemblyFile (
    std::unique_ptr< Program > & objProgram)
```

Generate a native assembly file for an object.

Definition at line 1394 of file [parser.cpp](#).

Referenced by [generateProgramCode\(\)](#).

11.69.3.4 generateProgramCode()

```
bool mxvm::Parser::generateProgramCode (
    const Mode & m,
    std::unique_ptr< Program > & program)
```

Walk the AST and populate a [Program](#) with instructions and variables.

Parameters

<i>m</i>	Execution mode (interpret or compile)
<i>program</i>	Output program to populate

Returns

true on success

Definition at line 737 of file [parser.cpp](#).

References [mxvm::Base::base](#), [mx::Color::BRIGHT_GREEN](#), [mxvm::SectionNode::CODE](#), [Col\(\)](#), [mxvm::SectionNode::DATA](#), [generateDebugHTML\(\)](#), [generateObjectAssemblyFile\(\)](#), [mxvm::html_mode](#), [mxvm::MODE_COMPILE](#), [mxvm::SectionNode::MODULE](#), [mxvm::SectionNode::OBJECT](#), [object_name](#), [parseAST\(\)](#), [parser_mode](#), [platform](#), [processCodeSection\(\)](#), [processDataSection\(\)](#), [processModuleSection\(\)](#), [processObjectSection\(\)](#), [registerObjectExterns\(\)](#), and [validator](#).

Referenced by [action_interpret\(\)](#), and [translate_x64\(\)](#).

11.69.3.5 operator=()

```
Parser & mxvm::Parser::operator= (
    const Parser & other) [inline]
```

Copy assignment.

Definition at line 80 of file [parser.hpp](#).

References [Parser\(\)](#), [include_path](#), [module_path](#), [object_mode](#), [object_name](#), [object_path](#), [parser_mode](#), [platform](#), [scanner](#), [source_file](#), and [validator](#).

11.69.3.6 operator[]()

```
auto mxvm::Parser::operator[] (
    size_t pos)
```

Access a token by position.

Definition at line 193 of file [parser.cpp](#).

References [pos\(\)](#), and [scanner](#).

Referenced by [parseAST\(\)](#), [parseCodeInstruction\(\)](#), [parseComment\(\)](#), [parseDataVariable\(\)](#), [parseLabel\(\)](#), [parseModule\(\)](#), [parseObject\(\)](#), and [parseSection\(\)](#).

11.69.3.7 parse()

```
void mxvm::Parser::parse ()
```

Parse all tokens into the internal AST.

Definition at line 730 of file [parser.cpp](#).

References [parseAST\(\)](#).

11.69.3.8 parseAST()

```
std::unique_ptr< ProgramNode > mxvm::Parser::parseAST ()
```

Build the full AST from tokens.

Returns

Root [ProgramNode](#) of the AST

Definition at line 201 of file [parser.cpp](#).

References [object_name](#), [operator\[\]\(\)](#), [parseProgramOrObject\(\)](#), and [scanner](#).

Referenced by [generateProgramCode\(\)](#), and [parse\(\)](#).

11.69.3.9 parseCodeInstruction()

```
std::unique_ptr< InstructionNode > mxvm::Parser::parseCodeInstruction (
    uint64_t & index) [private]
```

Definition at line 579 of file [parser.cpp](#).

References [ADD](#), [ALLOC](#), [AND](#), [CALL](#), [CMP](#), [DIV](#), [DONE](#), [EXIT](#), [FCMP](#), [FREE](#), [GETLINE](#), [INVOKE](#), [JA](#), [JAE](#), [JB](#), [JBE](#), [JC](#), [JE](#), [JG](#), [JGE](#), [JL](#), [JLE](#), [JMP](#), [JNC](#), [JNE](#), [JNO](#), [JNP](#), [JNS](#), [JNZ](#), [JO](#), [JP](#), [JS](#), [JZ](#), [mxvm::Operand::label](#), [LEA](#), [LOAD](#), [MOD](#), [MOV](#), [MUL](#), [NEG](#), [NOT](#), [mxvm::Operand::object](#), [mxvm::Operand::op](#), [mxvm::OP_CONSTANT](#), [mxvm::Operand::op_value](#), [mxvm::OP_VARIABLE](#), [operator\[\]\(\)](#), [OR](#), [POP](#), [PRINT](#), [PUSH](#), [REALLOC](#), [RET](#), [RETURN](#), [scanner](#), [STACK_LOAD](#), [STACK_STORE](#), [STACK_SUB](#), [STORE](#), [STRING_PRINT](#), [SUB](#), [TO_FLOAT](#), [TO_INT](#), [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), [types::TT_STR](#), [types::TT_SYM](#), [mxvm::Operand::type](#), and [XOR](#).

Referenced by [parseSection\(\)](#).

11.69.3.10 parseComment()

```
std::unique_ptr< CommentNode > mxvm::Parser::parseComment (
    uint64_t & index) [private]
```

Definition at line 688 of file [parser.cpp](#).

References [operator\[\]\(\)](#), and [scanner](#).

Referenced by [parseSection\(\)](#).

11.69.3.11 parseDataVariable()

```
std::unique_ptr< VariableNode > mxvm::Parser::parseDataVariable (
    uint64_t & index) [private]
```

Definition at line 461 of file [parser.cpp](#).

References [operator\[\]\(\)](#), [scanner](#), [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), [types::TT_STR](#), [types::TT_SYM](#), [mxvm::VAR_ARRAY](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

Referenced by [parseSection\(\)](#).

11.69.3.12 parseLabel()

```
std::unique_ptr< LabelNode > mxvm::Parser::parseLabel (
    uint64_t & index) [private]
```

Definition at line 709 of file [parser.cpp](#).

References [operator\[\]\(\)](#), [scanner](#), and [types::TT_ID](#).

Referenced by [parseSection\(\)](#).

11.69.3.13 parseModule()

```
std::unique_ptr< ModuleNode > mxvm::Parser::parseModule (
    uint64_t & index) [private]
```

Definition at line 449 of file [parser.cpp](#).

References [operator\[\]\(\)](#), and [scanner](#).

11.69.3.14 parseObject()

```
std::unique_ptr< ObjectNode > mxvm::Parser::parseObject (
    uint64_t & index) [private]
```

Definition at line 245 of file [parser.cpp](#).

References [operator\[\]\(\)](#), and [scanner](#).

Referenced by [parseSection\(\)](#).

11.69.3.15 parseProgramOrObject()

```
std::unique_ptr< ProgramNode > mxvm::Parser::parseProgramOrObject (
    uint64_t & index)
```

Parse a top-level program or object node.

Parameters

<i>index</i>	Token index (advanced past consumed tokens)
--------------	---

Returns

Parsed [ProgramNode](#)

Definition at line 1353 of file [parser.cpp](#).

References [mxvm::VAR_ARRAY](#), [mxvm::VAR_BYTE](#), and [mxvm::VAR_EXTERN](#).

Referenced by [parseAST\(\)](#).

11.69.3.16 parseSection()

```
std::unique_ptr< SectionNode > mxvm::Parser::parseSection (
    uint64_t & index) [private]
```

Definition at line 347 of file [parser.cpp](#).

References [mxvm::SectionNode::CODE](#), [mxvm::SectionNode::DATA](#), [mxvm::SectionNode::MODULE](#), [mxvm::SectionNode::OBJECT](#), [operator\[\]\(\)](#), [parseCodeInstruction\(\)](#), [parseComment\(\)](#), [parseDataVariable\(\)](#), [parseLabel\(\)](#), [parseObject\(\)](#), [scanner](#), and [types::TT_ID](#).

11.69.3.17 printObjectHTML()

```
void mxvm::Parser::printObjectHTML (
    std::ostream & out,
    const std::unique_ptr< Program > & objPtr) [private]
```

Definition at line 854 of file [parser.cpp](#).

References [mxvm::Program::escapeNewLines\(\)](#), [mxvm::VAR_ARRAY](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_LABEL](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

11.69.3.18 processCodeSection()

```
void mxvm::Parser::processCodeSection (
    SectionNode * sectionNode,
    std::unique_ptr< Program > & program) [private]
```

Definition at line 1228 of file [parser.cpp](#).

Referenced by [generateProgramCode\(\)](#).

11.69.3.19 processDataSection()

```
void mxvm::Parser::processDataSection (
    SectionNode * sectionNode,
    std::unique_ptr< Program > & program) [private]
```

Definition at line 1140 of file [parser.cpp](#).

Referenced by [generateProgramCode\(\)](#).

11.69.3.20 processModuleFile()

```
void mxvm::Parser::processModuleFile (
    const std::string & src,
    std::unique_ptr< Program > & program) [private]
```

Definition at line 1170 of file [parser.cpp](#).

11.69.3.21 processModuleSection()

```
void mxvm::Parser::processModuleSection (
    SectionNode * sectionNode,
    std::unique_ptr< Program > & program) [private]
```

Definition at line 1160 of file [parser.cpp](#).

Referenced by [generateProgramCode\(\)](#).

11.69.3.22 processObjectFile()

```
void mxvm::Parser::processObjectFile (
    const std::string & src,
    std::unique_ptr< Program > & program) [private]
```

Definition at line 267 of file [parser.cpp](#).

References [Parser\(\)](#), [mxvm::Base::base](#), [mxvm::SectionNode::CODE](#), [mxvm::SectionNode::DATA](#), [mxvm::SectionNode::MODULE](#), [module_path](#), [mxvm::SectionNode::OBJECT](#), [object_path](#), [parser_mode](#), [platform](#), [registerObjectExterns\(\)](#), and [mxvm::Validator::validate\(\)](#).

Referenced by [processObjectSection\(\)](#).

11.69.3.23 processObjectSection()

```
void mxvm::Parser::processObjectSection (
    SectionNode * sectionNode,
    std::unique_ptr< Program > & program) [private]
```

Definition at line 257 of file [parser.cpp](#).

References [processObjectFile\(\)](#), and [mxvm::SectionNode::statements](#).

Referenced by [generateProgramCode\(\)](#).

11.69.3.24 registerObjectExterns()

```
void mxvm::Parser::registerObjectExterns (
    std::unique_ptr< Program > & mainProgram,
    const std::unique_ptr< Program > & objProgram)
```

Register external functions from an object into the main program.

Definition at line 1415 of file [parser.cpp](#).

Referenced by [generateProgramCode\(\)](#), and [processObjectFile\(\)](#).

11.69.3.25 resolveLabelReference()

```
void mxvm::Parser::resolveLabelReference (
    Operand & operand,
    const std::unordered_map< std::string, size_t > & labelMap) [private]
```

Definition at line 1344 of file [parser.cpp](#).

References [mxvm::VAR_LABEL](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

11.69.3.26 scan()

```
uint64_t mxvm::Parser::scan ()
```

Tokenize the source and return the token count.

Definition at line 184 of file [parser.cpp](#).

References [scanner](#), and [scan::ScanExcept::why\(\)](#).

Referenced by [action_interpret\(\)](#), and [translate_x64\(\)](#).

11.69.3.27 setDefaultVariableValue()

```
void mxvm::Parser::setDefaultVariableValue (
    Variable & var,
    VarType type) [private]
```

Definition at line 1311 of file [parser.cpp](#).

References [mxvm::VAR_FLOAT](#), and [mxvm::VAR_INTEGER](#).

11.69.3.28 setVariableValue()

```
void mxvm::Parser::setVariableValue (
    Variable & var,
    VarType type,
    const std::string & value,
    size_t buf_size = 0) [private]
```

Definition at line 1269 of file [parser.cpp](#).

11.69.4 Friends And Related Symbol Documentation

11.69.4.1 ModuleParser

```
friend class ModuleParser [friend]
```

Definition at line 60 of file [parser.hpp](#).

References [ModuleParser](#).

Referenced by [ModuleParser](#).

11.69.5 Member Data Documentation

11.69.5.1 include_path

```
std::string mxvm::Parser::include_path = "/usr/local/include/mxvm/modules"
```

system module include path

Definition at line 134 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), [action_interpret\(\)](#), [operator=\(\)](#), and [translate_x64\(\)](#).

11.69.5.2 module_path

```
std::string mxvm::Parser::module_path = "."
```

search path for .mxvm module files

Definition at line 132 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), [action_interpret\(\)](#), [operator=\(\)](#), [processObjectFile\(\)](#), and [translate_x64\(\)](#).

11.69.5.3 object_mode

```
bool mxvm::Parser::object_mode = false
```

true when parsing an object definition

Definition at line 135 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), and [operator=\(\)](#).

11.69.5.4 object_name

```
std::string mxvm::Parser::object_name
```

Definition at line 136 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), [generateProgramCode\(\)](#), [operator=\(\)](#), and [parseAST\(\)](#).

11.69.5.5 object_path

```
std::string mxvm::Parser::object_path = "."
```

search path for .mxvm object files

Definition at line 133 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), [action_interpret\(\)](#), [operator=\(\)](#), [processObjectFile\(\)](#), and [translate_x64\(\)](#).

11.69.5.6 parser_mode

```
Mode mxvm::Parser::parser_mode = Mode::MODE_INTERPRET
```

Definition at line 131 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), [generateProgramCode\(\)](#), [operator=\(\)](#), and [processObjectFile\(\)](#).

11.69.5.7 platform

```
Platform mxvm::Parser::platform
```

Definition at line 137 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), [generateProgramCode\(\)](#), [operator=\(\)](#), [processObjectFile\(\)](#), and [translate_x64\(\)](#).

11.69.5.8 scanner

`scan::Scanner` `mxvm::Parser::scanner`

Definition at line 129 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), [Parser\(\)](#), [operator=\(\)](#), [operator\[\]\(\)](#), [parseAST\(\)](#), [parseCodeInstruction\(\)](#), [parseComment\(\)](#), [parseDataVariable\(\)](#), [parseLabel\(\)](#), [parseModule\(\)](#), [parseObject\(\)](#), [parseSection\(\)](#), and [scan\(\)](#).

11.69.5.9 source_file

`std::string` `mxvm::Parser::source_file`

Definition at line 128 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), [Parser\(\)](#), and [operator=\(\)](#).

11.69.5.10 validator

`Validator` `mxvm::Parser::validator`

Definition at line 130 of file [parser.hpp](#).

Referenced by [Parser\(\)](#), [Parser\(\)](#), [generateProgramCode\(\)](#), and [operator=\(\)](#).

The documentation for this class was generated from the following files:

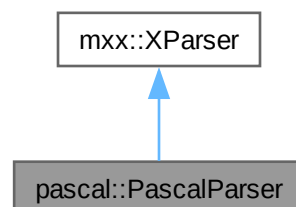
- [include/mxvm/parser.hpp](#)
- [src/parser.cpp](#)

11.70 pascal::PascalParser Class Reference

Recursive-descent parser for the Pascal language.

```
#include <parser.hpp>
```

Inheritance diagram for `pascal::PascalParser`:



Public Member Functions

- [PascalParser](#) ()=delete
- [PascalParser](#) (const std::string &source)
Construct and prepare the parser.
- std::unique_ptr< [ProgramNode](#) > [parseProgram](#) ()
Parse a complete Pascal program and return the root AST node.
- std::unique_ptr< [UnitNode](#) > [parseUnit](#) ()
Parse a Pascal unit and return the root AST node.
- bool [isUnitSource](#) () const
Check if the source starts with a 'unit' keyword.

Public Member Functions inherited from [mx::XParser](#)

- [XParser](#) ()=delete
- [XParser](#) (const std::string &source)
Construct and scan the source text.
- bool [next](#) ()
Advance to the next token; returns false at end of stream.
- bool [peekIs](#) (const std::string &s)
Check if the current token's value matches a string (case-insensitive).
- bool [peekIs](#) (const [types::TokenType](#) &t)
Check if the current token's type matches.

Public Attributes

- [mx::TPValidator](#) validator
semantic validator

Public Attributes inherited from [mx::XParser](#)

- std::string [filename](#)
source filename for error messages

Private Member Functions

- void [removeBraceComments](#) ()
Remove {brace} comments from the token stream.
- void [error](#) (const std::string &message)
Report a parse error with the given message.
- void [expectToken](#) (const std::string &expected)
Consume a token matching the expected string, or throw.
- void [expectToken](#) ([types::TokenType](#) expected)
Consume a token matching the expected type, or throw.
- bool [match](#) (const std::string &s)
Try to match and consume a token by value.
- bool [match](#) ([types::TokenType](#) t)
Try to match and consume a token by type.
- std::unique_ptr< [ASTNode](#) > [parseLValue](#) ()
Parse an lvalue (variable, array access, pointer deref, field access).
- std::unique_ptr< [BlockNode](#) > [parseBlock](#) ()
Parse a block (declarations + compound statement).
- std::unique_ptr< [CompoundStmtNode](#) > [parseCompoundStatement](#) ()
Parse a compound statement (begin..end).
- std::vector< std::unique_ptr< [ASTNode](#) > > [parseDeclarations](#) ()
Parse the declarations section.
- std::unique_ptr< [ASTNode](#) > [parseVarDeclaration](#) ()
Parse a var declaration section.
- std::unique_ptr< [ASTNode](#) > [parseConstDeclaration](#) ()
Parse a const declaration section.
- std::unique_ptr< [ASTNode](#) > [parseProcedureDeclaration](#) ()
Parse a procedure declaration.
- std::unique_ptr< [ASTNode](#) > [parseFunctionDeclaration](#) ()
Parse a function declaration.
- std::vector< std::unique_ptr< [ASTNode](#) > > [parseInterfaceDeclarations](#) ()

- Parse interface forward declarations (procedure/function signatures only).*
 - `std::unique_ptr< ASTNode > parseProcedureForwardDecl ()`
- Parse a procedure forward declaration (signature only, no body).*
 - `std::unique_ptr< ASTNode > parseFunctionForwardDecl ()`
- Parse a function forward declaration (signature only, no body).*
 - `std::vector< std::unique_ptr< ASTNode > > parseParameterList ()`
- Parse a formal parameter list.*
 - `std::vector< std::unique_ptr< ASTNode > > parseStatementList ()`
- Parse a list of statements (separated by semicolons).*
 - `std::unique_ptr< ASTNode > parseParameter ()`
- Parse a single formal parameter.*
 - `std::unique_ptr< ASTNode > parseRecordType ()`
- Parse a record type definition.*
 - `std::unique_ptr< ASTNode > parseTypeDeclaration ()`
- Parse a type declaration section.*
 - `std::unique_ptr< ASTNode > parseStatement ()`
- Parse a single statement.*
 - `std::unique_ptr< ASTNode > parseAssignmentOrProcCall ()`
- Parse an assignment or procedure call starting with an identifier.*
 - `std::unique_ptr< ASTNode > parseSelfStatement ()`
- Parse an if-then-else statement.*
 - `std::unique_ptr< ASTNode > parseWhileStatement ()`
- Parse a while-do statement.*
 - `std::unique_ptr< ASTNode > parseForStatement ()`
- Parse a for-to/downto-do statement.*
 - `std::unique_ptr< ASTNode > parseRepeatStatement ()`
- Parse a repeat-until statement.*
 - `std::unique_ptr< ASTNode > parseCaseStatement ()`
- Parse a case statement.*
 - `std::unique_ptr< ASTNode > parseWithStatement ()`
- Parse a with statement.*
 - `std::unique_ptr< ASTNode > parseGotoStatement ()`
- Parse a goto statement.*
 - `void parseLabelDeclaration ()`
- Parse a label declaration section (label 100, 200;).*
 - `std::unique_ptr< ASTNode > parseExpression ()`
- Parse a full expression (simple expression with optional relational op).*
 - `std::unique_ptr< ASTNode > parseSimpleExpression ()`
- Parse a simple expression (terms combined by +, -, or).*
 - `std::unique_ptr< ASTNode > parseTerm ()`
- Parse a term (factors combined by *, /, div, mod, and).*
 - `std::unique_ptr< ASTNode > parseFactor ()`
- Parse a factor (literal, variable, function call, sub-expression).*
 - `std::unique_ptr< ASTNode > parseProcedureCall (const std::string &name)`
- Parse a procedure call given its already-parsed name.*
 - `std::unique_ptr< ASTNode > parseFunctionCall (const std::string &name)`
- Parse a function call given its already-parsed name.*
 - `std::vector< std::unique_ptr< ASTNode > > parseArgumentList ()`
- Parse a comma-separated argument list.*
 - `bool isRelationalOperator ()`
- Check if current token is a relational operator.*

- bool [isAddOperator](#) ()
Check if current token is an additive operator.
- bool [isMulOperator](#) ()
Check if current token is a multiplicative operator.
- std::unique_ptr< [ASTNode](#) > [parseTypeSpec](#) ()
Parse a type specifier.
- std::string [getRelationalOp](#) ()
Consume and return a relational operator string.
- std::string [getAddOp](#) ()
Consume and return an additive operator string.
- std::string [getMulOp](#) ()
Consume and return a multiplicative operator string.
- std::string [getUnaryOp](#) ()
Consume and return a unary operator string.
- [BinaryOpNode::OpType](#) [getComparisonOperator](#) (const std::string &op)
Map a comparison operator string to [BinaryOpNode::OpType](#).
- [BinaryOpNode::OpType](#) [getLogicalOperator](#) (const std::string &op)
Map a logical operator string to [BinaryOpNode::OpType](#).
- [BinaryOpNode::OpType](#) [getArithmeticOperator](#) (const std::string &op)
Map an arithmetic operator string to [BinaryOpNode::OpType](#).
- std::string [tokenTypeToString](#) ([types::TokenType](#) type)
Convert a [TokenType](#) to its display string.
- bool [isType](#) (const std::string &token)
Check if a string is a known type name.
- bool [isBuiltinProcedure](#) (const std::string &name)
Check if a name is a built-in procedure.
- bool [isBuiltinFunction](#) (const std::string &name)
Check if a name is a built-in function.
- bool [isKeyword](#) (const std::string &s)
Check if a string is a reserved keyword.
- std::unique_ptr< [ASTNode](#) > [parseArrayType](#) ()
Parse an array type specification.
- std::unique_ptr< [ASTNode](#) > [parseArrayDeclaration](#) (const std::string &varName)
Parse an array variable declaration.
- bool [isArrayAccess](#) ()
Check if the current context is an array access.

Private Attributes

- std::unordered_set< std::string > [declaredLabels](#)
Set of user-declared goto labels.

Additional Inherited Members

Static Public Member Functions inherited from [mxx::XParser](#)

- static std::string [tokenTypeToString](#) ([types::TokenType](#) t)
Convert a [TokenType](#) to a human-readable string.

Protected Attributes inherited from [mxx::XParser](#)

- [scan::Scanner](#) `scanner`
underlying scanner
- [scan::TToken](#) * `token` = nullptr
current token pointer
- `size_t` `index` = 0
current token index

11.70.1 Detailed Description

Recursive-descent parser for the Pascal language.

Parses Pascal source code into an AST rooted at a [ProgramNode](#). Includes a TPValidator for semantic checks during parsing.

Definition at line 112 of file [parser.hpp](#).

11.70.2 Constructor & Destructor Documentation

11.70.2.1 PascalParser() [1/2]

```
pascal::PascalParser::PascalParser () [delete]
```

11.70.2.2 PascalParser() [2/2]

```
pascal::PascalParser::PascalParser (
    const std::string & source) [inline], [explicit]
```

Construct and prepare the parser.

Parameters

<i>source</i>	Pascal source code string
---------------	---------------------------

Definition at line 119 of file [parser.hpp](#).

References [mxx::XParser::XParser\(\)](#), [mxx::XParser::index](#), [removeBraceComments\(\)](#), [mxx::XParser::scanner](#), [mxx::XParser::token](#), and [validator](#).

11.70.3 Member Function Documentation

11.70.3.1 error()

```
void pascal::PascalParser::error (
    const std::string & message) [private]
```

Report a parse error with the given message.

Definition at line 33 of file [parser.cpp](#).

References [mxx::XParser::token](#).

Referenced by [expectToken\(\)](#), [expectToken\(\)](#), [getRelationalOp\(\)](#), [parseFactor\(\)](#), [parseForStatement\(\)](#), [parseRepeatStatement\(\)](#), and [parseStatement\(\)](#).

11.70.3.2 expectToken() [1/2]

```
void pascal::PascalParser::expectToken (
    const std::string & expected) [private]
```

Consume a token matching the expected string, or throw.

Definition at line 37 of file [parser.cpp](#).

References [error\(\)](#), [mxx::XParser::peekIs\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseArrayType\(\)](#), [parseAssignmentOrProcCall\(\)](#), [parseCaseStatement\(\)](#), [parseCompoundStatement\(\)](#), [parseConstDeclaration\(\)](#), [parseFactor\(\)](#), [parseForStatement\(\)](#), [parseFunctionCall\(\)](#), [parseFunctionDeclaration\(\)](#), [parseFunctionForwardDecl\(\)](#), [parseGotoStatement\(\)](#), [parseIfStatement\(\)](#), [parseLabelDeclaration\(\)](#), [parseLValue\(\)](#),

[parseParameter\(\)](#), [parseProcedureCall\(\)](#), [parseProcedureDeclaration\(\)](#), [parseProcedureForwardDecl\(\)](#), [parseProgram\(\)](#), [parseRecordType\(\)](#), [parseRepeatStatement\(\)](#), [parseStatement\(\)](#), [parseTypeDeclaration\(\)](#), [parseTypeSpec\(\)](#), [parseUnit\(\)](#), [parseVarDeclaration\(\)](#), [parseWhileStatement\(\)](#), and [parseWithStatement\(\)](#).

11.70.3.3 expectToken() [2/2]

```
void pascal::PascalParser::expectToken (
    types::TokenType expected) [private]
```

Consume a token matching the expected type, or throw.

Definition at line 42 of file [parser.cpp](#).

References [error\(\)](#), [mxx::XParser::peekIs\(\)](#), and [tokenTypeToString\(\)](#).

11.70.3.4 getAddOp()

```
std::string pascal::PascalParser::getAddOp () [private]
```

Consume and return an additive operator string.

11.70.3.5 getArithmeticOperator()

```
BinaryOpNode::OpType pascal::PascalParser::getArithmeticOperator (
    const std::string & op) [private]
```

Map an arithmetic operator string to [BinaryOpNode::OpType](#).

11.70.3.6 getComparisonOperator()

```
BinaryOpNode::OpType pascal::PascalParser::getComparisonOperator (
    const std::string & op) [private]
```

Map a comparison operator string to [BinaryOpNode::OpType](#).

11.70.3.7 getLogicalOperator()

```
BinaryOpNode::OpType pascal::PascalParser::getLogicalOperator (
    const std::string & op) [private]
```

Map a logical operator string to [BinaryOpNode::OpType](#).

11.70.3.8 getMulOp()

```
std::string pascal::PascalParser::getMulOp () [private]
```

Consume and return a multiplicative operator string.

11.70.3.9 getRelationalOp()

```
std::string pascal::PascalParser::getRelationalOp () [private]
```

Consume and return a relational operator string.

Definition at line 965 of file [parser.cpp](#).

References [error\(\)](#), and [mxx::XParser::peekIs\(\)](#).

Referenced by [parseExpression\(\)](#).

11.70.3.10 getUnaryOp()

```
std::string pascal::PascalParser::getUnaryOp () [private]
```

Consume and return a unary operator string.

11.70.3.11 isAddOperator()

```
bool pascal::PascalParser::isAddOperator () [private]
```

Check if current token is an additive operator.

11.70.3.12 isArrayAccess()

```
bool pascal::PascalParser::isArrayAccess () [private]
```

Check if the current context is an array access.

11.70.3.13 isBuiltinFunction()

```
bool pascal::PascalParser::isBuiltinFunction (
    const std::string & name) [private]
```

Check if a name is a built-in function.

11.70.3.14 isBuiltinProcedure()

```
bool pascal::PascalParser::isBuiltinProcedure (
    const std::string & name) [private]
```

Check if a name is a built-in procedure.

11.70.3.15 isKeyword()

```
bool pascal::PascalParser::isKeyword (
    const std::string & s) [private]
```

Check if a string is a reserved keyword.

Definition at line 261 of file [parser.cpp](#).

Referenced by [parseConstDeclaration\(\)](#), [parseDeclarations\(\)](#), [parseInterfaceDeclarations\(\)](#), and [parseTypeDeclaration\(\)](#).

11.70.3.16 isMulOperator()

```
bool pascal::PascalParser::isMulOperator () [private]
```

Check if current token is a multiplicative operator.

Definition at line 951 of file [parser.cpp](#).

References [mxx::XParser::peekIs\(\)](#), [types::TT_ID](#), and [types::TT_SYM](#).

Referenced by [parseTerm\(\)](#).

11.70.3.17 isRelationalOperator()

```
bool pascal::PascalParser::isRelationalOperator () [private]
```

Check if current token is a relational operator.

Definition at line 957 of file [parser.cpp](#).

References [mxx::XParser::peekIs\(\)](#), and [types::TT_SYM](#).

Referenced by [parseExpression\(\)](#).

11.70.3.18 isType()

```
bool pascal::PascalParser::isType (
    const std::string & token) [private]
```

Check if a string is a known type name.

References [mxx::XParser::token](#).

11.70.3.19 isUnitSource()

```
bool pascal::PascalParser::isUnitSource () const
```

Check if the source starts with a 'unit' keyword.

Definition at line 87 of file [parser.cpp](#).

References [mxx::XParser::token](#).

Referenced by [compileFile\(\)](#).

11.70.3.20 match() [1/2]

```
bool pascal::PascalParser::match (
    const std::string & s) [private]
```

Try to match and consume a token by value.

Definition at line 79 of file [parser.cpp](#).

References [mxx::XParser::next\(\)](#), and [mxx::XParser::peekIs\(\)](#).

Referenced by [parseDeclarations\(\)](#), and [parseInterfaceDeclarations\(\)](#).

11.70.3.21 match() [2/2]

```
bool pascal::PascalParser::match (
    types::TokenType t) [private]
```

Try to match and consume a token by type.

Definition at line 241 of file [parser.cpp](#).

References [mxx::XParser::next\(\)](#), and [mxx::XParser::peekIs\(\)](#).

11.70.3.22 parseArgumentList()

```
std::vector< std::unique_ptr< ASTNode > > pascal::PascalParser::parseArgumentList () [private]
```

Parse a comma-separated argument list.

Definition at line 939 of file [parser.cpp](#).

References [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), and [mxx::XParser::peekIs\(\)](#).

Referenced by [parseFunctionCall\(\)](#), and [parseProcedureCall\(\)](#).

11.70.3.23 parseArrayDeclaration()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseArrayDeclaration (
    const std::string & varName) [private]
```

Parse an array variable declaration.

11.70.3.24 parseArrayType()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseArrayType () [private]
```

Parse an array type specification.

Definition at line 1230 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseTypeSpec\(\)](#), and [mxx::XParser::peekIs\(\)](#).

Referenced by [parseTypeDeclaration\(\)](#), and [parseTypeSpec\(\)](#).

11.70.3.25 parseAssignmentOrProcCall()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseAssignmentOrProcCall () [private]
```

Parse an assignment or procedure call starting with an identifier.

Definition at line 517 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseProcedureCall\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

11.70.3.26 parseBlock()

```
std::unique_ptr< BlockNode > pascal::PascalParser::parseBlock () [private]
```

Parse a block (declarations + compound statement).

Definition at line 249 of file [parser.cpp](#).

References [parseCompoundStatement\(\)](#), [parseDeclarations\(\)](#), [parseLabelDeclaration\(\)](#), [mxx::XParser::peekIs\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseFunctionDeclaration\(\)](#), [parseProcedureDeclaration\(\)](#), and [parseProgram\(\)](#).

11.70.3.27 parseCaseStatement()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseCaseStatement () [private]
```

Parse a case statement.

Definition at line 842 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseStatement\(\)](#), [mxx::XParser::peekIs\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseStatement\(\)](#).

11.70.3.28 parseCompoundStatement()

```
std::unique_ptr< CompoundStmtNode > pascal::PascalParser::parseCompoundStatement () [private]
```

Parse a compound statement (begin..end).

Definition at line 400 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseStatementList\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseBlock\(\)](#), and [parseStatement\(\)](#).

11.70.3.29 parseConstDeclaration()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseConstDeclaration () [private]
```

Parse a const declaration section.

Definition at line 878 of file [parser.cpp](#).

References [expectToken\(\)](#), [isKeyword\(\)](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseDeclarations\(\)](#), and [parseInterfaceDeclarations\(\)](#).

11.70.3.30 parseDeclarations()

```
std::vector< std::unique_ptr< ASTNode > > pascal::PascalParser::parseDeclarations () [private]
```

Parse the declarations section.

Definition at line 899 of file [parser.cpp](#).

References [isKeyword\(\)](#), [match\(\)](#), [parseConstDeclaration\(\)](#), [parseFunctionDeclaration\(\)](#), [parseLabelDeclaration\(\)](#), [parseProcedureDeclaration\(\)](#), [parseTypeDeclaration\(\)](#), [parseVarDeclaration\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseBlock\(\)](#), and [parseUnit\(\)](#).

11.70.3.31 parseExpression()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseExpression () [private]
```

Parse a full expression (simple expression with optional relational op).

Definition at line 615 of file [parser.cpp](#).

References [pascal::BinaryOpNode::AND](#), [pascal::BinaryOpNode::EQUAL](#), [getRelationalOp\(\)](#), [pascal::BinaryOpNode::GREATER](#), [pascal::BinaryOpNode::GREATER_EQUAL](#), [pascal::BinaryOpNode::IN](#), [isRelationalOperator\(\)](#), [pascal::BinaryOpNode::LESS](#), [pascal::BinaryOpNode::LESS_EQUAL](#), [mxx::XParser::next\(\)](#), [pascal::BinaryOpNode::NOT_EQUAL](#), [pascal::BinaryOpNode::OR](#), [parseSimpleExpression\(\)](#), [mxx::XParser::peekIs\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseArgumentList\(\)](#), [parseArrayType\(\)](#), [parseAssignmentOrProcCall\(\)](#), [parseCaseStatement\(\)](#), [parseConstDeclaration\(\)](#), [parseFactor\(\)](#), [parseForStatement\(\)](#), [parseIfStatement\(\)](#), [parseLValue\(\)](#), [parseRecordType\(\)](#), [parseRepeatStatement\(\)](#), [parseStatement\(\)](#), [parseVarDeclaration\(\)](#), and [parseWhileStatement\(\)](#).

11.70.3.32 parseFactor()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseFactor () [private]
```

Parse a factor (literal, variable, function call, sub-expression).

Definition at line 715 of file [parser.cpp](#).

References [error\(\)](#), [expectToken\(\)](#), [mxx::XParser::index](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseFactor\(\)](#), [parseFunctionCall\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), [types::TT_ID](#), [types::TT_NUM](#), and [types::TT_STR](#).

Referenced by [parseFactor\(\)](#), and [parseTerm\(\)](#).

11.70.3.33 parseForStatement()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseForStatement () [private]
```

Parse a for-to/downto-do statement.

Definition at line 566 of file [parser.cpp](#).

References [error\(\)](#), [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseStatement\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseStatement\(\)](#).

11.70.3.34 parseFunctionCall()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseFunctionCall (
    const std::string & name) [private]
```

Parse a function call given its already-parsed name.

Definition at line 829 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseArgumentList\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseFactor\(\)](#).

11.70.3.35 parseFunctionDeclaration()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseFunctionDeclaration () [private]
```

Parse a function declaration.

Definition at line 303 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseBlock\(\)](#), [parseParameterList\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseDeclarations\(\)](#).

11.70.3.36 parseFunctionForwardDecl()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseFunctionForwardDecl () [private]
```

Parse a function forward declaration (signature only, no body).

Definition at line 207 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseParameterList\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseInterfaceDeclarations\(\)](#).

11.70.3.37 parseGotoStatement()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseGotoStatement () [private]
```

Parse a goto statement.

Definition at line 1325 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [mxx::XParser::token](#), and [types::TT_NUM](#).

Referenced by [parseStatement\(\)](#).

11.70.3.38 parseIfStatement()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseIfStatement () [private]
```

Parse an if-then-else statement.

Definition at line 535 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseStatement\(\)](#), [mxx::XParser::peekIs\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseStatement\(\)](#).

11.70.3.39 parseInterfaceDeclarations()

```
std::vector< std::unique_ptr< ASTNode > > pascal::PascalParser::parseInterfaceDeclarations ()
[private]
```

Parse interface forward declarations (procedure/function signatures only).

Definition at line 163 of file [parser.cpp](#).

References [isKeyword\(\)](#), [match\(\)](#), [parseConstDeclaration\(\)](#), [parseFunctionForwardDecl\(\)](#), [parseProcedureForwardDecl\(\)](#), [parseTypeDeclaration\(\)](#), [parseVarDeclaration\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseUnit\(\)](#).

11.70.3.40 parseLabelDeclaration()

```
void pascal::PascalParser::parseLabelDeclaration () [private]
```

Parse a label declaration section (label 100, 200;).

Definition at line 1337 of file [parser.cpp](#).

References [declaredLabels](#), [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_NUM](#).

Referenced by [parseBlock\(\)](#), and [parseDeclarations\(\)](#).

11.70.3.41 [parseLValue\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseLValue () [private]
```

Parse an lvalue (variable, array access, pointer deref, field access).

Definition at line 1275 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::index](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseStatement\(\)](#).

11.70.3.42 [parseParameter\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseParameter () [private]
```

Parse a single formal parameter.

Definition at line 360 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseParameterList\(\)](#).

11.70.3.43 [parseParameterList\(\)](#)

```
std::vector< std::unique_ptr< ASTNode > > pascal::PascalParser::parseParameterList () [private]
```

Parse a formal parameter list.

Definition at line 348 of file [parser.cpp](#).

References [mxx::XParser::next\(\)](#), [parseParameter\(\)](#), and [mxx::XParser::peekIs\(\)](#).

Referenced by [parseFunctionDeclaration\(\)](#), [parseFunctionForwardDecl\(\)](#), [parseProcedureDeclaration\(\)](#), and [parseProcedureForwardDecl\(\)](#).

11.70.3.44 [parseProcedureCall\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseProcedureCall (
    const std::string & name) [private]
```

Parse a procedure call given its already-parsed name.

Definition at line 815 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseArgumentList\(\)](#), [mxx::XParser::peekIs\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseAssignmentOrProcCall\(\)](#), and [parseStatement\(\)](#).

11.70.3.45 [parseProcedureDeclaration\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseProcedureDeclaration () [private]
```

Parse a procedure declaration.

Definition at line 279 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseBlock\(\)](#), [parseParameterList\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseDeclarations\(\)](#).

11.70.3.46 [parseProcedureForwardDecl\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseProcedureForwardDecl () [private]
```

Parse a procedure forward declaration (signature only, no body).

Definition at line 186 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseParameterList\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseInterfaceDeclarations\(\)](#).

11.70.3.47 parseProgram()

```
std::unique_ptr< ProgramNode > pascal::PascalParser::parseProgram ()
```

Parse a complete Pascal program and return the root AST node.

Definition at line 47 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseBlock\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [compileFile\(\)](#).

11.70.3.48 parseRecordType()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseRecordType () [private]
```

Parse a record type definition.

Definition at line 1070 of file [parser.cpp](#).

References [pascal::VariantArm::caseLabels](#), [expectToken\(\)](#), [pascal::VariantArm::fields](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseTypeSpec\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseTypeDeclaration\(\)](#), and [parseTypeSpec\(\)](#).

11.70.3.49 parseRepeatStatement()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseRepeatStatement () [private]
```

Parse a repeat-until statement.

Definition at line 594 of file [parser.cpp](#).

References [error\(\)](#), [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseStatement\(\)](#), [mxx::XParser::peekIs\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseStatement\(\)](#).

11.70.3.50 parseSimpleExpression()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseSimpleExpression () [private]
```

Parse a simple expression (terms combined by +, -, or).

Definition at line 662 of file [parser.cpp](#).

References [pascal::BinaryOpNode::MINUS](#), [pascal::UnaryOpNode::MINUS](#), [mxx::XParser::next\(\)](#), [pascal::UnaryOpNode::NOT](#), [parseTerm\(\)](#), [mxx::XParser::peekIs\(\)](#), [pascal::BinaryOpNode::PLUS](#), [pascal::UnaryOpNode::PLUS](#), [mxx::XParser::token](#), and [types::TT_SYM](#).

Referenced by [parseExpression\(\)](#).

11.70.3.51 parseStatement()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseStatement () [private]
```

Parse a single statement.

Definition at line 412 of file [parser.cpp](#).

References [error\(\)](#), [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseCaseStatement\(\)](#), [parseCompoundStatement\(\)](#), [parseExpression\(\)](#), [parseForStatement\(\)](#), [parseGotoStatement\(\)](#), [parseIfStatement\(\)](#), [parseLValue\(\)](#), [parseProcedureCall\(\)](#), [parseRepeatStatement\(\)](#), [parseStatement\(\)](#), [parseWhileStatement\(\)](#), [parseWithStatement\(\)](#), [mxx::XParser::peekIs\(\)](#), [mxx::XParser::token](#), [types::TT_ID](#), and [types::TT_NUM](#).

Referenced by [parseCaseStatement\(\)](#), [parseForStatement\(\)](#), [parseIfStatement\(\)](#), [parseRepeatStatement\(\)](#), [parseStatement\(\)](#), [parseStatementList\(\)](#), [parseWhileStatement\(\)](#), and [parseWithStatement\(\)](#).

11.70.3.52 parseStatementList()

```
std::vector< std::unique_ptr< ASTNode > > pascal::PascalParser::parseStatementList () [private]
```

Parse a list of statements (separated by semicolons).

Definition at line 924 of file [parser.cpp](#).

References [mxx::XParser::next\(\)](#), [parseStatement\(\)](#), and [mxx::XParser::peekIs\(\)](#).

Referenced by [parseCompoundStatement\(\)](#).

11.70.3.53 parseTerm()

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseTerm () [private]
```

Parse a term (factors combined by *, /, div, mod, and).

Definition at line 694 of file [parser.cpp](#).

References [pascal::BinaryOpNode::DIV](#), [pascal::BinaryOpNode::DIVIDE](#), [isMulOperator\(\)](#), [pascal::BinaryOpNode::MOD](#), [pascal::BinaryOpNode::MULTIPLY](#), [mxx::XParser::next\(\)](#), [parseFactor\(\)](#), [mxx::XParser::peekls\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseSimpleExpression\(\)](#).

11.70.3.54 [parseTypeDeclaration\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseTypeDeclaration () [private]
```

Parse a type declaration section.

Definition at line 984 of file [parser.cpp](#).

References [expectToken\(\)](#), [isKeyword\(\)](#), [mxx::XParser::next\(\)](#), [parseArrayType\(\)](#), [parseRecordType\(\)](#), [mxx::XParser::peekls\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseDeclarations\(\)](#), and [parseInterfaceDeclarations\(\)](#).

11.70.3.55 [parseTypeSpec\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseTypeSpec () [private]
```

Parse a type specifier.

Definition at line 1192 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseArrayType\(\)](#), [parseRecordType\(\)](#), [mxx::XParser::peekls\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseArrayType\(\)](#), [parseRecordType\(\)](#), and [parseVarDeclaration\(\)](#).

11.70.3.56 [parseUnit\(\)](#)

```
std::unique_ptr< UnitNode > pascal::PascalParser::parseUnit ()
```

Parse a Pascal unit and return the root AST node.

Definition at line 95 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseDeclarations\(\)](#), [parseInterfaceDeclarations\(\)](#), [mxx::XParser::peekls\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [compileFile\(\)](#).

11.70.3.57 [parseVarDeclaration\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseVarDeclaration () [private]
```

Parse a var declaration section.

Definition at line 1159 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseTypeSpec\(\)](#), [mxx::XParser::peekls\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseDeclarations\(\)](#), and [parseInterfaceDeclarations\(\)](#).

11.70.3.58 [parseWhileStatement\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseWhileStatement () [private]
```

Parse a while-do statement.

Definition at line 553 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseExpression\(\)](#), [parseStatement\(\)](#), and [mxx::XParser::token](#).

Referenced by [parseStatement\(\)](#).

11.70.3.59 [parseWithStatement\(\)](#)

```
std::unique_ptr< ASTNode > pascal::PascalParser::parseWithStatement () [private]
```

Parse a with statement.

Definition at line 1310 of file [parser.cpp](#).

References [expectToken\(\)](#), [mxx::XParser::next\(\)](#), [parseStatement\(\)](#), [mxx::XParser::token](#), and [types::TT_ID](#).

Referenced by [parseStatement\(\)](#).

11.70.3.60 removeBraceComments()

```
void pascal::PascalParser::removeBraceComments () [private]
```

Remove {brace} comments from the token stream.

Definition at line 12 of file [parser.cpp](#).

References [mxv::XParser::scanner](#).

Referenced by [PascalParser\(\)](#).

11.70.3.61 tokenTypeToString()

```
std::string pascal::PascalParser::tokenTypeToString (
    types::TokenType type) [private]
```

Convert a TokenType to its display string.

Definition at line 1260 of file [parser.cpp](#).

References [types::TT_ID](#), [types::TT_NUM](#), [types::TT_STR](#), and [types::TT_SYM](#).

Referenced by [expectToken\(\)](#).

11.70.4 Member Data Documentation

11.70.4.1 declaredLabels

```
std::unordered_set<std::string> pascal::PascalParser::declaredLabels [private]
```

Set of user-declared goto labels.

Definition at line 203 of file [parser.hpp](#).

Referenced by [parseLabelDeclaration\(\)](#).

11.70.4.2 validator

```
mxv::TPValidator pascal::PascalParser::validator
```

semantic validator

Definition at line 135 of file [parser.hpp](#).

Referenced by [PascalParser\(\)](#), and [compileFile\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/parser.hpp](#)
- [src/frontend/parser.cpp](#)

11.71 mxvm::PasHTMLGen Class Reference

Generates syntax-highlighted HTML output from Pascal (.pas) source code.

```
#include <html_gen.hpp>
```

Public Member Functions

- [PasHTMLGen](#) (const std::string &src)
Construct from Pascal source text.
- void [output](#) (std::ostream &out, const std::string &filename, const std::string &cssHref="mxvm-highlight.css")
Write generated HTML to an output stream.

Static Public Member Functions

- static std::string [defaultCss](#) ()
Return the default stylesheet text.

Private Attributes

- std::string [source](#)

11.71.1 Detailed Description

Generates syntax-highlighted HTML output from Pascal (.pas) source code.

Supports all keywords, types, and built-ins recognised by the MXVM Pascal compiler (from both the parser and validator keyword sets), plus Pascal comment styles (`{ }`, `(* *)`, `//`), single-quoted string literals with `' '` escape, hex literals (`$FF`), and char literals (`#65 / # $41`).

The generated HTML links to an external stylesheet instead of embedding styles inline, so colours and fonts can be customised by editing that file. Call `defaultCss()` to obtain the bundled default stylesheet text.

Definition at line 46 of file [html_gen.hpp](#).

11.71.2 Constructor & Destructor Documentation

11.71.2.1 PasHTMLGen()

```
mxvm::PasHTMLGen::PasHTMLGen (
    const std::string & src) [explicit]
```

Construct from Pascal source text.

Definition at line 324 of file [html_gen.cpp](#).

References [source](#).

11.71.3 Member Function Documentation

11.71.3.1 defaultCss()

```
std::string mxvm::PasHTMLGen::defaultCss () [static]
```

Return the default stylesheet text.

Covers all span classes used by both [HTMLGen](#) (.mxvm) and [PasHTMLGen](#) (.pas) output. Write this string to `mxvm-highlight.css` alongside the HTML files and edit it freely to change the colour scheme.

Definition at line 326 of file [html_gen.cpp](#).

Referenced by [ensureCss\(\)](#), and [output\(\)](#).

11.71.3.2 output()

```
void mxvm::PasHTMLGen::output (
    std::ostream & out,
    const std::string & filename,
    const std::string & cssHref = "mxvm-highlight.css")
```

Write generated HTML to an output stream.

Parameters

<i>out</i>	Output stream
<i>filename</i>	Source filename (used in the HTML <code><title></code>)
<i>cssHref</i>	href for the linked stylesheet (default: <code>"mxvm-highlight.css"</code>)

Definition at line 378 of file [html_gen.cpp](#).

References [defaultCss\(\)](#), and [source](#).

Referenced by [writeHtml\(\)](#).

11.71.4 Member Data Documentation

11.71.4.1 source

```
std::string mxvm::PasHTMLGen::source [private]
```

Definition at line 70 of file [html_gen.hpp](#).

Referenced by [PasHTMLGen\(\)](#), and [output\(\)](#).

The documentation for this class was generated from the following files:

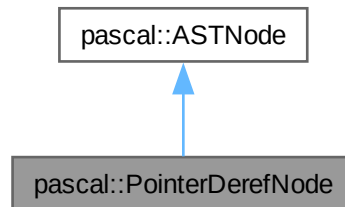
- [include/mxvm/html_gen.hpp](#)
- [src/html_gen.cpp](#)

11.72 pascal::PointerDerefNode Class Reference

AST node for a pointer dereference (ptr[^]).

```
#include <ast.hpp>
```

Inheritance diagram for pascal::PointerDerefNode:



Public Member Functions

- `PointerDerefNode` (std::unique_ptr< [ASTNode](#) > ptr)
- void `accept` (ASTVisitor &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string `toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual `~ASTNode` ()=default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::unique_ptr< [ASTNode](#) > `pointer`
expression yielding a pointer

11.72.1 Detailed Description

AST node for a pointer dereference (ptr[^]).

Definition at line [589](#) of file [ast.hpp](#).

11.72.2 Constructor & Destructor Documentation

11.72.2.1 PointerDerefNode()

```
pascal::PointerDerefNode::PointerDerefNode (
    std::unique_ptr< ASTNode > ptr) [inline]
```

Definition at line [593](#) of file [ast.hpp](#).

References [pointer](#).

11.72.3 Member Function Documentation

11.72.3.1 `accept()`

```
void pascal::PointerDerefNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 434 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.72.3.2 `toString()`

```
std::string pascal::PointerDerefNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 596 of file [ast.hpp](#).

11.72.4 Member Data Documentation

11.72.4.1 `pointer`

```
std::unique_ptr<ASTNode> pascal::PointerDerefNode::pointer
```

expression yielding a pointer

Definition at line 591 of file [ast.hpp](#).

Referenced by [PointerDerefNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

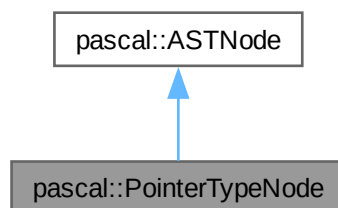
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.73 `pascal::PointerTypeNode` Class Reference

AST node for a pointer type declaration ($\wedge$ BaseType).

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::PointerTypeNode`:



Public Member Functions

- [PointerTypeNode](#) (const std::string &base)
- void [accept](#) (ASTVisitor &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from pascal::ASTNode

- virtual `~ASTNode()` = default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::string `baseTypeName`
the type being pointed to

11.73.1 Detailed Description

AST node for a pointer type declaration ([^]BaseType).
Definition at line 578 of file [ast.hpp](#).

11.73.2 Constructor & Destructor Documentation

11.73.2.1 PointerTypeNode()

```
pascal::PointerTypeNode::PointerTypeNode (
    const std::string & base) [inline]
```

Definition at line 582 of file [ast.hpp](#).

References [baseTypeName](#).

11.73.3 Member Function Documentation

11.73.3.1 accept()

```
void pascal::PointerTypeNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 430 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.73.3.2 toString()

```
std::string pascal::PointerTypeNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 585 of file [ast.hpp](#).

References [baseTypeName](#).

11.73.4 Member Data Documentation

11.73.4.1 baseTypeName

```
std::string pascal::PointerTypeNode::baseTypeName
```

the type being pointed to

Definition at line 580 of file [ast.hpp](#).

Referenced by [PointerTypeNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

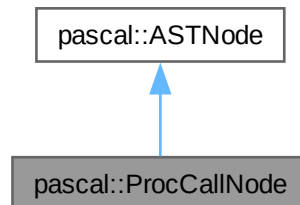
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.74 pascal::ProcCallNode Class Reference

AST node for a procedure call statement.

```
#include <ast.hpp>
```

Inheritance diagram for pascal::ProcCallNode:



Public Member Functions

- [ProcCallNode](#) (const std::string &procName, std::vector< std::unique_ptr< [ASTNode](#) > > args)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [name](#)
procedure name
- std::vector< std::unique_ptr< [ASTNode](#) > > [arguments](#)
actual arguments

11.74.1 Detailed Description

AST node for a procedure call statement.

Definition at line 425 of file [ast.hpp](#).

11.74.2 Constructor & Destructor Documentation

11.74.2.1 ProcCallNode()

```

pascal::ProcCallNode::ProcCallNode (
    const std::string & procName,
    std::vector< std::unique_ptr< ASTNode > > args) [inline]
  
```

Definition at line 430 of file [ast.hpp](#).
References [arguments](#), and [name](#).

11.74.3 Member Function Documentation

11.74.3.1 accept()

```
void pascal::ProcCallNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 157 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.74.3.2 toString()

```
std::string pascal::ProcCallNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 161 of file [ast.cpp](#).

References [arguments](#), and [name](#).

11.74.4 Member Data Documentation

11.74.4.1 arguments

```
std::vector<std::unique_ptr<ASTNode> > pascal::ProcCallNode::arguments
```

actual arguments

Definition at line 428 of file [ast.hpp](#).

Referenced by [ProcCallNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.74.4.2 name

```
std::string pascal::ProcCallNode::name
```

procedure name

Definition at line 427 of file [ast.hpp](#).

Referenced by [ProcCallNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

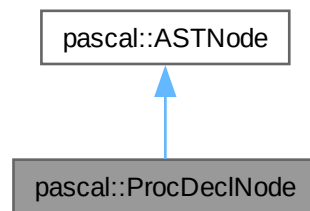
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.75 pascal::ProcDeclNode Class Reference

AST node for a procedure declaration.

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::ProcDeclNode`:



Public Member Functions

- `ProcDeclNode` (const std::string &n, std::vector< std::unique_ptr< [ASTNode](#) > > p, std::unique_ptr< [ASTNode](#) > b)
- void `accept` ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string `toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual `~ASTNode` ()=default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::string `name`
procedure name
- std::vector< std::unique_ptr< [ASTNode](#) > > `parameters`
formal parameters
- std::shared_ptr< [ASTNode](#) > `block`
procedure body block

11.75.1 Detailed Description

AST node for a procedure declaration.
Definition at line 209 of file [ast.hpp](#).

11.75.2 Constructor & Destructor Documentation

11.75.2.1 ProcDeclNode()

```

pascal::ProcDeclNode::ProcDeclNode (
    const std::string & n,

```



```
std::vector< std::unique_ptr< ASTNode > > p,
std::unique_ptr< ASTNode > b) [inline]
```

Definition at line 215 of file [ast.hpp](#).

References [block](#), [name](#), and [parameters](#).

11.75.3 Member Function Documentation

11.75.3.1 accept()

```
void pascal::ProcDeclNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 75 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.75.3.2 toString()

```
std::string pascal::ProcDeclNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 79 of file [ast.cpp](#).

References [name](#), and [parameters](#).

11.75.4 Member Data Documentation

11.75.4.1 block

```
std::shared_ptr<ASTNode> pascal::ProcDeclNode::block
```

procedure body block

Definition at line 213 of file [ast.hpp](#).

Referenced by [ProcDeclNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.75.4.2 name

```
std::string pascal::ProcDeclNode::name
```

procedure name

Definition at line 211 of file [ast.hpp](#).

Referenced by [ProcDeclNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.75.4.3 parameters

```
std::vector<std::unique_ptr<ASTNode> > pascal::ProcDeclNode::parameters
```

formal parameters

Definition at line 212 of file [ast.hpp](#).

Referenced by [ProcDeclNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

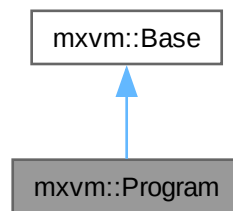
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.76 mxvm::Program Class Reference

Complete MXVM program: instruction interpreter and native x64 code generator.

```
#include <icode.hpp>
```

Inheritance diagram for mxvm::Program:



Public Member Functions

- [Program](#) ()
Default constructor — initializes execution state (pc, flags, etc.).
- [~Program](#) ()
Destructor — frees any heap-allocated variables.
- [Program](#) (Program &&other) noexcept
Move constructor — transfers all execution state and child objects.
- [Program](#) & [operator=](#) (Program &&other) noexcept
Move-assignment operator.
- void [add_standard](#) ()
Register standard library runtime functions.
- void [stop](#) ()
Halt the interpreter loop.
- int [exec](#) ()
Execute the program via the interpreter.
- void [print](#) (std::ostream &out)
Print all instructions and data to an output stream.
- void [post](#) (std::ostream &out)
Generate native data section (variable declarations) into the output stream.
- int [getExitCode](#) () const
Get the program's exit code after execution.
- std::string [getPlatformSymbolName](#) (const std::string &name)
Mangle a symbol name with the platform prefix (e.g. `_name` for Darwin).
- void [generateCode](#) (const Platform &platform, bool obj, std::ostream &out)
Generate native x64 assembly for the entire program.
- void [memoryDump](#) (std::ostream &out)
Dump all variable values to a stream for debugging.
- void [setArgs](#) (const std::vector< std::string > &argv)
Set the command-line arguments available to the program.
- void [setObject](#) (bool obj)
Mark this program as an object (no main entry point).
- bool [validateNames](#) (Validator &v)
Validate all variable and label names against the [Validator](#).
- void [flatten](#) (Program *program)
Flatten child object programs into the root instruction stream.

- void `flatten_inc` (`Program *root`, `Instruction &i`)
Rewrite object-qualified operand references in a single instruction.
- void `flatten_label` (`Program *root`, `int64_t offset`, `const std::string &label`, `bool func`)
Copy a label from a child object into the root label map with an address offset.
- void `flatten_external` (`Program *root`, `const std::string &e`, `RuntimeFunction &r`)
Copy an external function binding from a child object into the root.
- `std::string getMangledName` (`const std::string &var`)
Mangle a variable name with its owning object prefix.
- `std::string getMangledName` (`const Operand &op`)
Mangle an operand's variable reference with its object prefix.
- `Program * getObjectByName` (`const std::string &name`)
Look up a child object program by name.
- `bool isFunctionValid` (`const std::string &label`)
Check whether a label exists and is marked as a function entry.
- `size_t x64_reserve_call_area` (`std::ostream &out`, `size_t spill_bytes`)
Reserve stack space for a Win64 call frame including spill area.
- void `x64_release_call_area` (`std::ostream &out`, `size_t frame`)
Release a Win64 call frame previously reserved.
- void `x64_direct_stack_adjust` (`std::ostream &out`, `int64_t bytes`)
Emit a direct stack pointer adjustment.

System V ABI (Linux/macOS) x86-64 code generation

- void `generateFunctionCall` (`std::ostream &out`, `const std::string &name`, `std::vector< Operand > &op`)
Generate a System V ABI function call with operands as arguments.
- void `generateInvokeCall` (`std::ostream &out`, `std::vector< Operand > &op`)
Generate a dynamic invoke call (runtime-resolved external function).
- void `generateInstruction` (`std::ostream &out`, `const Instruction &i`)
Dispatch a single instruction to its System V code generator.
- int `generateLoadVar` (`std::ostream &out`, `int reg`, `const Operand &op`)
Load a variable or constant into a numbered register.
- int `generateLoadVar` (`std::ostream &out`, `VarType type`, `std::string reg`, `const Operand &op`)
Load a variable or constant into a named register.
- `std::string getRegisterByIndex` (`int index`, `VarType type`)
Get the register name for a given index and variable type.
- void `gen_print` (`std::ostream &out`, `const Instruction &i`)
- void `gen_arth` (`std::ostream &out`, `std::string arth`, `const Instruction &i`)
- void `gen_exit` (`std::ostream &out`, `const Instruction &i`)
- void `gen_mov` (`std::ostream &out`, `const Instruction &i`)
- void `gen_jmp` (`std::ostream &out`, `const Instruction &i`)
- void `gen_cmp` (`std::ostream &out`, `const Instruction &i`)
- void `gen_alloc` (`std::ostream &out`, `const Instruction &i`)
- void `gen_realloc` (`std::ostream &out`, `const Instruction &i`)
Generate System V x86-64 assembly for the REALLOC instruction (dynamic array resize).
- void `gen_free` (`std::ostream &out`, `const Instruction &i`)
- void `gen_load` (`std::ostream &out`, `const Instruction &i`)
- void `gen_store` (`std::ostream &out`, `const Instruction &i`)
- void `gen_ret` (`std::ostream &out`, `const Instruction &i`)
- void `gen_call` (`std::ostream &out`, `const Instruction &i`)
- void `gen_done` (`std::ostream &out`, `const Instruction &i`)
- void `gen_bitop` (`std::ostream &out`, `const std::string &opc`, `const Instruction &i`)
- void `gen_not` (`std::ostream &out`, `const Instruction &i`)
- void `gen_push` (`std::ostream &out`, `const Instruction &i`)
Emit x86-64 PUSH for a variable or constant.
- void `gen_pop` (`std::ostream &out`, `const Instruction &i`)
Emit x86-64 POP into a variable.
- void `gen_stack_load` (`std::ostream &out`, `const Instruction &i`)
- void `gen_stack_store` (`std::ostream &out`, `const Instruction &i`)

- void [gen_stack_sub](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_getline](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_to_int](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_to_float](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_invoke](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_return](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_neg](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_lea](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_fcmp](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_jae](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_jbe](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_jc](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_jnc](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_jp](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_jnp](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_jo](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_jno](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_js](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_jns](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_add](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_sub](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_mul](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_div](#) (std::ostream &out, const [Instruction](#) &i)
 - void [gen_mod](#) (std::ostream &out, const [Instruction](#) &i)
 - std::string [gen_optimize](#) (const std::string &code, const [Platform](#) &platform)
- Peephole optimizer for generated assembly.*
- std::string [optimize_darwin](#) (const std::string &code)
- Darwin-specific assembly optimizations.*
- std::string [optimize_core](#) (const std::string &code)
- Platform-independent peephole optimizations.*

Interpreter execution methods

- void [exec_mov](#) (const [Instruction](#) &instr)
 - void [exec_add](#) (const [Instruction](#) &instr)
 - void [exec_sub](#) (const [Instruction](#) &instr)
 - void [exec_mul](#) (const [Instruction](#) &instr)
 - void [exec_div](#) (const [Instruction](#) &instr)
 - void [exec_cmp](#) (const [Instruction](#) &instr)
 - void [exec_jmp](#) (const [Instruction](#) &instr)
 - void [exec_load](#) (const [Instruction](#) &instr)
 - void [exec_store](#) (const [Instruction](#) &instr)
- Execute a STORE instruction — write a value into a pointer at a given offset.*
- void [exec_or](#) (const [Instruction](#) &instr)
 - void [exec_and](#) (const [Instruction](#) &instr)
 - void [exec_xor](#) (const [Instruction](#) &instr)
 - void [exec_not](#) (const [Instruction](#) &instr)
 - void [exec_mod](#) (const [Instruction](#) &instr)
 - void [exec_je](#) (const [Instruction](#) &instr)
 - void [exec_jne](#) (const [Instruction](#) &instr)
 - void [exec_jl](#) (const [Instruction](#) &instr)
 - void [exec_jle](#) (const [Instruction](#) &instr)
 - void [exec_jg](#) (const [Instruction](#) &instr)
 - void [exec_jge](#) (const [Instruction](#) &instr)
 - void [exec_jz](#) (const [Instruction](#) &instr)
 - void [exec_jnz](#) (const [Instruction](#) &instr)
 - void [exec_ja](#) (const [Instruction](#) &instr)
 - void [exec_jb](#) (const [Instruction](#) &instr)
 - void [exec_print](#) (const [Instruction](#) &instr)
 - void [exec_string_print](#) (const [Instruction](#) &instr)
 - void [exec_exit](#) (const [Instruction](#) &instr)
 - void [exec_alloc](#) (const [Instruction](#) &instr)
 - void [exec_free](#) (const [Instruction](#) &instr)
 - void [exec_getline](#) (const [Instruction](#) &instr)

- void `exec_push` (const `Instruction` &instr)
Execute a PUSH instruction — save a variable or constant onto the VM stack.
- void `exec_pop` (const `Instruction` &instr)
Execute a POP instruction — restore a value from the VM stack into a variable.
- void `exec_stack_load` (const `Instruction` &instr)
- void `exec_stack_store` (const `Instruction` &instr)
- void `exec_stack_sub` (const `Instruction` &instr)
- void `exec_call` (const `Instruction` &instr)
- void `exec_ret` (const `Instruction` &instr)
- void `exec_done` (const `Instruction` &instr)
- void `exec_to_int` (const `Instruction` &instr)
- void `exec_to_float` (const `Instruction` &instr)
- void `exec_invoke` (const `Instruction` &instr)
- void `exec_return` (const `Instruction` &instr)
- void `exec_lea` (const `Instruction` &instr)
- void `exec_neg` (const `Instruction` &instr)
- void `exec_realloc` (const `Instruction` &instr)
Execute the REALLOC instruction in the interpreter.
- void `exec_fcmp` (const `Instruction` &instr)
- void `exec_jae` (const `Instruction` &instr)
- void `exec_jbe` (const `Instruction` &instr)
- void `exec_jc` (const `Instruction` &instr)
- void `exec_jnc` (const `Instruction` &instr)
- void `exec_jp` (const `Instruction` &instr)
- void `exec_jnp` (const `Instruction` &instr)
- void `exec_jo` (const `Instruction` &instr)
- void `exec_jno` (const `Instruction` &instr)
- void `exec_js` (const `Instruction` &instr)
- void `exec_jns` (const `Instruction` &instr)

Interpreter helper methods

- `Variable` & `getVariable` (const std::string &name)
Look up a variable by name, searching local scope then global.
- bool `isVariable` (const std::string &name)
Check whether a variable exists in local or global scope.
- void `setVariableFromString` (`Variable` &var, const std::string &value)
Set a variable's value from a string representation, coercing types.
- void `addVariables` (`Variable` &dest, `Variable` &src1, `Variable` &src2)
Add two variables and store the result in dest.
- void `subVariables` (`Variable` &dest, `Variable` &src1, `Variable` &src2)
Subtract src2 from src1 and store the result in dest.
- void `mulVariables` (`Variable` &dest, `Variable` &src1, `Variable` &src2)
Multiply two variables and store the result in dest.
- void `divVariables` (`Variable` &dest, `Variable` &src1, `Variable` &src2)
Divide src1 by src2 and store the result in dest.
- std::string `printFormatted` (const std::string &format, const std::vector< `Variable` * > &args, bool output=true)
Format and optionally print a printf-style format string with variable arguments.
- void `setVariableFromConstant` (`Variable` &var, const std::string &value)
Set a variable's value from a constant string (integer, float, or string literal).
- `Variable` `createTempVariable` (`VarType` type, const std::string &value)
Create a temporary unnamed variable with the given type and value.
- bool `isConstant` (const std::string &value)
Check whether a string represents a compile-time constant (number or quoted string).
- `Variable` `variableFromOperand` (const `Operand` &op)
Resolve an `Operand` to its corresponding `Variable`.

Public Member Functions inherited from `mxvm::Base`

- `Base()`=default
- `Base(Base &&other)` noexcept
Move constructor.
- `~Base()`=default
- `Base & operator= (Base &&other)` noexcept
Move-assignment operator.
- void `setMainBase (Base *b)`
Set the root program base pointer (used for global variable lookup).
- void `add_instruction (const Instruction &i)`
Append an instruction to the instruction stream.
- void `add_label (const std::string &name, uint64_t address, bool f)`
Register a label at the given instruction address.
- void `add_variable (const std::string &name, const Variable &v)`
Declare a local variable in the symbol table.
- void `add_global (const std::string &objname, const std::string &name, const Variable &v)`
Declare a global variable scoped to an object.
- void `add_extern (const std::string &mod, const std::string &name, bool module)`
Register an external function import.
- void `add_runtime_extern (const std::string &mod_name, const std::string &mod, const std::string &func_name, const std::string &name)`
Register and resolve a runtime-loaded external function.
- void `add_filename (const std::string &fname)`
Record a source filename for debug/error reporting.
- void `add_allocated (const std::string &name, Variable &v)`
Track a heap-allocated variable for cleanup.
- void `add_object (const std::string &name, Program *prog)`
Register a child object program.

Static Public Member Functions

- static std::string `escapeNewLines (const std::string &text)`
Escape newline characters in a string literal for assembly output.

Public Attributes

- std::string `name`
- std::string `filename`
- std::vector< std::unique_ptr< Program > > `objects`
- bool `object` = false
- `Stack` `stack`
- `Operand` `result`
return value operand from last invoke
- `Program *` `parent` = nullptr
parent program (for object programs)
- `Platform` `platform`
- bool `last_call_returns_owned` = false
- `VarType` `last_call_ret_type` = `VarType::VAR_INTEGER`

Public Attributes inherited from [mxvm::Base](#)

- `std::string` [name](#)
- `std::vector< Instruction >` [inc](#)
instruction stream
- `std::unordered_map< std::string, Variable >` [vars](#)
variable symbol table
- `std::unordered_map< std::string, std::pair< uint64_t, bool > >` [labels](#)
label -> (address, is_function)
- `std::vector< ExternalFunction >` [external](#)
declared external function imports
- `std::unordered_map< std::string, RuntimeFunction >` [external_functions](#)
resolved runtime function bindings
- `std::string` [assembly_code](#)
generated assembly output for objects

Private Types

- enum [LastCmpType](#) { [CMP_NONE](#) , [CMP_INTEGER](#) , [CMP_FLOAT](#) }
Tracks type of last comparison for conditional jumps.

Private Attributes

- `size_t` [pc](#)
program counter
- `bool` [running](#)
interpreter running flag
- `int` [exitCode](#) = 0
- `bool` [zero_flag](#) = false
comparison zero flag
- `bool` [less_flag](#) = false
comparison less-than flag
- `bool` [greater_flag](#) = false
comparison greater-than flag
- `bool` [carry_flag](#) = false
comparison carry flag
- `int` [xmm_offset](#) = 0
current XMM register allocation offset
- `std::vector< std::string >` [args](#)
- `bool` [main_function](#)
true if this program has a main entry point
- `bool` [object_external](#) = false
- [LastCmpType](#) [last_cmp_type](#) = [CMP_NONE](#)
- `bool` [last_call_returns_owned_ptr](#) = false
tracks ownership of last call's return value

Friends

- class [Parser](#)

Win64 ABI x86-64 code generation

- `std::unordered_map< std::string, std::string > x64_reg_vars`
Win64 variable-to-register mapping.
- `std::vector< std::string > x64_reg_save_order`
- `void x64_emit_job_func (std::ostream &out, int index, const std::string &dstReg)`
Emit an integer-or-byte function argument load for Win64 ABI.
- `std::string x64_getRegisterByIndex (int index, VarType type)`
Get the Win64 register name for a given index and variable type.
- `int x64_generateLoadVar (std::ostream &out, int r, const Operand &op)`
Load a variable or constant into a numbered Win64 register.
- `int x64_generateLoadVar (std::ostream &out, VarType type, std::string reg, const Operand &op)`
Load a variable or constant into a named Win64 register.
- `void x64_generateFunctionCall (std::ostream &out, const std::string &name, std::vector< Operand > &args)`
Generate a Win64 ABI function call with operands as arguments.
- `void x64_generateInvokeCall (std::ostream &out, std::vector< Operand > &op)`
Generate a Win64 dynamic invoke call.
- `void x64_generateCode (const Platform &platform, bool obj, std::ostream &out)`
Top-level Win64 code generation entry point.
- `void x64_generateInstruction (std::ostream &out, const Instruction &i)`
Dispatch a single instruction to its Win64 code generator.
- `void x64_gen_done (std::ostream &out, const Instruction &i)`
- `void x64_gen_ret (std::ostream &out, const Instruction &i)`
- `void x64_gen_return (std::ostream &out, const Instruction &i)`
- `void x64_gen_neg (std::ostream &out, const Instruction &i)`
- `void x64_gen_invoke (std::ostream &out, const Instruction &i)`
- `void x64_gen_call (std::ostream &out, const Instruction &i)`
- `void x64_gen_alloc (std::ostream &out, const Instruction &i)`
- `void x64_gen_realloc (std::ostream &out, const Instruction &i)`
Generate Win64 x86-64 assembly for the REALLOC instruction (dynamic array resize).
- `void x64_gen_free (std::ostream &out, const Instruction &i)`
- `void x64_gen_load (std::ostream &out, const Instruction &i)`
- `void x64_gen_store (std::ostream &out, const Instruction &i)`
- `void x64_gen_to_int (std::ostream &out, const Instruction &i)`
- `void x64_gen_to_float (std::ostream &out, const Instruction &i)`
- `void x64_gen_div (std::ostream &out, const Instruction &i)`
- `void x64_gen_mod (std::ostream &out, const Instruction &i)`
- `void x64_gen_getline (std::ostream &out, const Instruction &i)`
- `void x64_gen_bitop (std::ostream &out, const std::string &opc, const Instruction &i)`
- `void x64_gen_not (std::ostream &out, const Instruction &i)`
- `void x64_gen_push (std::ostream &out, const Instruction &i)`
- `void x64_gen_pop (std::ostream &out, const Instruction &i)`
- `void x64_gen_stack_load (std::ostream &out, const Instruction &i)`
- `void x64_gen_stack_store (std::ostream &out, const Instruction &i)`
- `void x64_gen_stack_sub (std::ostream &out, const Instruction &i)`
- `void x64_gen_print (std::ostream &out, const Instruction &i)`
- `void x64_gen_jump (std::ostream &out, const Instruction &i)`
- `void x64_gen_cmp (std::ostream &out, const Instruction &i)`
- `void x64_gen_mov (std::ostream &out, const Instruction &i)`
- `void x64_gen_arth (std::ostream &out, std::string arth, const Instruction &i)`
- `void x64_gen_exit (std::ostream &out, const Instruction &i)`
- `void x64_gen_lea (std::ostream &out, const Instruction &i)`
- `void x64_gen_fcmp (std::ostream &out, const Instruction &i)`

- void [x64_gen_jae](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_gen_jbe](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_gen_jc](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_gen_jnc](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_gen_jp](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_gen_jnp](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_gen_jo](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_gen_jno](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_gen_js](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_gen_jns](#) (std::ostream &out, const [Instruction](#) &i)
- void [x64_analyzeRegAlloc](#) (bool uses_std_module)
 - Analyze variable usage and assign Win64 callee-saved registers.*
- void [x64_emitFlushRegs](#) (std::ostream &out)
 - Write all register-allocated variables back to memory.*
- void [x64_emitReloadRegs](#) (std::ostream &out)
 - Reload register-allocated variables from memory.*
- void [x64_emitSaveRegs](#) (std::ostream &out)
 - Save callee-saved registers at function prologue.*
- void [x64_emitRestoreRegs](#) (std::ostream &out)
 - Restore callee-saved registers at function epilogue.*
- void [x64_emitStoreVar](#) (std::ostream &out, const std::string &srcReg, const [Operand](#) &dest)
 - Store a register value into a variable (register-allocated or memory).*
- void [x64_emitStoreVarImm](#) (std::ostream &out, const std::string &imm, const [Operand](#) &dest)
 - Store an immediate value into a variable.*
- void [x64_emitLoadVar](#) (std::ostream &out, const std::string &dstReg, const [Operand](#) &src)
 - Load a variable into a destination register.*

System V ABI register allocation

- std::unordered_map< std::string, std::string > [sysv_reg_vars](#)
 - SysV variable-to-register mapping.*
- std::vector< std::string > [sysv_reg_save_order](#)
- void [sysv_analyzeRegAlloc](#) (bool uses_std_module)
 - Analyze variable usage and assign System V callee-saved registers.*
- void [sysv_emitFlushRegs](#) (std::ostream &out)
 - Write all register-allocated variables back to memory.*
- void [sysv_emitReloadRegs](#) (std::ostream &out)
 - Reload register-allocated variables from memory.*
- void [sysv_emitSaveRegs](#) (std::ostream &out)
 - Save callee-saved registers at function prologue.*
- void [sysv_emitRestoreRegs](#) (std::ostream &out)
 - Restore callee-saved registers at function epilogue.*
- void [sysv_emitStoreVar](#) (std::ostream &out, const std::string &srcReg, const [Operand](#) &dest)
 - Store a register value into a variable (register-allocated or memory).*
- void [sysv_emitStoreVarImm](#) (std::ostream &out, const std::string &imm, const [Operand](#) &dest)
 - Store an immediate value into a variable.*
- void [sysv_emitLoadVar](#) (std::ostream &out, const std::string &dstReg, const [Operand](#) &src)
 - Load a variable into a destination register.*

Additional Inherited Members

Static Public Attributes inherited from `mxvm::Base`

- static `Base * base` = nullptr
pointer to the main program base
- static `std::string root_name`
- static `std::vector< std::string > filenames`
- static `std::unordered_map< std::string, Variable > allocated`
heap-allocated variables
- static `std::unordered_map< std::string, Program * > object_map`
object name -> Program mapping

11.76.1 Detailed Description

Complete MXVM program: instruction interpreter and native x64 code generator.

Extends `Base` with execution state (program counter, flags, stack) for interpretation, and a full set of code generation methods targeting x86-64 assembly (System V ABI for Linux/macOS, Win64 ABI for Windows).

Definition at line 212 of file `icode.hpp`.

11.76.2 Member Enumeration Documentation

11.76.2.1 LastCmpType

```
enum mxvm::Program::LastCmpType [private]
```

Tracks type of last comparison for conditional jumps.

Enumerator

CMP_NONE	
CMP_INTEGER	
CMP_FLOAT	

Definition at line 412 of file `icode.hpp`.

11.76.3 Constructor & Destructor Documentation

11.76.3.1 Program() [1/2]

```
mxvm::Program::Program ()
```

Default constructor — initializes execution state (pc, flags, etc.).

Definition at line 96 of file `icode.cpp`.

References `mxvm::Base::add_extern()`, `pc`, and `running`.

Referenced by `Program()`, `flatten()`, `flatten_external()`, `flatten_inc()`, `flatten_label()`, `getObjectByName()`, `isFunctionValid()`, `memoryDump()`, and `operator=()`.

11.76.3.2 ~Program()

```
mxvm::Program::~~Program ()
```

Destructor — frees any heap-allocated variables.

Definition at line 144 of file `icode.cpp`.

References `Col()`, `mxvm::debug_mode`, `mxvm::RuntimeFunction::handles`, `name`, `mx::Color::RED`, and `mxvm::Base::vars`.

11.76.3.3 Program() [2/2]

```
mxvm::Program::Program (
    Program && other) [inline], [noexcept]
```

Move constructor — transfers all execution state and child objects.

Definition at line 221 of file [icode.hpp](#).

References [Program\(\)](#), [name](#), [pc](#), and [mxvm::Base::vars](#).

11.76.4 Member Function Documentation

11.76.4.1 add_standard()

```
void mxvm::Program::add_standard ()
```

Register standard library runtime functions.

Definition at line 107 of file [icode.cpp](#).

References [mxvm::Base::add_variable\(\)](#), [name](#), and [mxvm::Variable::setExtern\(\)](#).

Referenced by [exec\(\)](#), [generateCode\(\)](#), and [x64_generateCode\(\)](#).

11.76.4.2 addVariables()

```
void mxvm::Program::addVariables (
    Variable & dest,
    Variable & src1,
    Variable & src2)
```

Add two variables and store the result in dest.

Parameters

<i>dest</i>	Destination variable
<i>src1</i>	First operand
<i>src2</i>	Second operand

Definition at line 787 of file [icode_exec.cpp](#).

References [mxvm::Base::base](#), [mxvm::Variable_Value::float_value](#), [mxvm::Variable_Value::int_value](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec_add\(\)](#).

11.76.4.3 createTempVariable()

```
Variable mxvm::Program::createTempVariable (
    VarType type,
    const std::string & value)
```

Create a temporary unnamed variable with the given type and value.

Parameters

<i>type</i>	Variable type
<i>value</i>	Initial value as string

Returns

A new [Variable](#)

Definition at line 1972 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Variable_Value::int_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_name](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

Referenced by [exec_add\(\)](#), [exec_cmp\(\)](#), [exec_div\(\)](#), [exec_fcmp\(\)](#), [exec_mod\(\)](#), [exec_mul\(\)](#), [exec_print\(\)](#), [exec_string_print\(\)](#), [exec_sub\(\)](#), [mxvm_io_fprintf\(\)](#), and [variableFromOperand\(\)](#).

11.76.4.4 divVariables()

```
void mxvm::Program::divVariables (
    Variable & dest,
    Variable & src1,
    Variable & src2)
```

Divide src1 by src2 and store the result in dest.

Definition at line 841 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Variable_Value::int_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec_div\(\)](#).

11.76.4.5 escapeNewLines()

```
std::string mxvm::Program::escapeNewLines (
    const std::string & text) [static]
```

Escape newline characters in a string literal for assembly output.

Parameters

<i>text</i>	Raw string
-------------	------------

Returns

Escaped string

Definition at line 296 of file [icode.cpp](#).

References [result](#).

Referenced by [generateCode\(\)](#), [memoryDump\(\)](#), [mxvm::Parser::printObjectHTML\(\)](#), and [x64_generateCode\(\)](#).

11.76.4.6 exec()

```
int mxvm::Program::exec ()
```

Execute the program via the interpreter.

Returns

Exit code

Definition at line 122 of file [icode_exec.cpp](#).

References [ADD](#), [add_standard\(\)](#), [ALLOC](#), [AND](#), [CALL](#), [CMP](#), [DIV](#), [DONE](#), [exec_add\(\)](#), [exec_alloc\(\)](#), [exec_and\(\)](#), [exec_call\(\)](#), [exec_cmp\(\)](#), [exec_div\(\)](#), [exec_done\(\)](#), [exec_exit\(\)](#), [exec_fcmp\(\)](#), [exec_free\(\)](#), [exec_getline\(\)](#), [exec_invoke\(\)](#), [exec_ja\(\)](#), [exec_jae\(\)](#), [exec_jb\(\)](#), [exec_jbe\(\)](#), [exec_jc\(\)](#), [exec_je\(\)](#), [exec_jg\(\)](#), [exec_jge\(\)](#), [exec_jl\(\)](#), [exec_jle\(\)](#), [exec_jmp\(\)](#), [exec_jnc\(\)](#), [exec_jne\(\)](#), [exec_jno\(\)](#), [exec_jnp\(\)](#), [exec_jns\(\)](#), [exec_jnz\(\)](#), [exec_jo\(\)](#), [exec_jp\(\)](#), [exec_js\(\)](#), [exec_jz\(\)](#), [exec_lea\(\)](#), [exec_load\(\)](#), [exec_mod\(\)](#), [exec_mov\(\)](#), [exec_mul\(\)](#), [exec_neg\(\)](#), [exec_not\(\)](#), [exec_or\(\)](#), [exec_pop\(\)](#), [exec_print\(\)](#), [exec_push\(\)](#), [exec_realloc\(\)](#), [exec_ret\(\)](#), [exec_return\(\)](#), [exec_stack_load\(\)](#), [exec_stack_store\(\)](#), [exec_stack_sub\(\)](#), [exec_store\(\)](#), [exec_string_print\(\)](#), [exec_sub\(\)](#), [exec_to_float\(\)](#), [exec_to_int\(\)](#), [exec_xor\(\)](#), [EXIT](#), [FCMP](#), [FREE](#), [getExitCode\(\)](#), [GETLINE](#), [mxvm::Base::inc](#), [mxvm::instruct_mode](#), [mxvm::Instruction::instruction](#), [INVOKE](#), [JA](#), [JAE](#), [JB](#), [JBE](#), [JC](#), [JE](#), [JG](#), [JGE](#), [JL](#), [JLE](#), [JMP](#), [JNC](#), [JNE](#), [JNO](#), [JNP](#), [JNS](#), [JNZ](#), [JO](#), [JP](#), [JS](#), [JZ](#), [LEA](#), [LOAD](#), [MOD](#), [MOV](#), [MUL](#), [NEG](#), [NOT](#), [OR](#), [pc](#), [POP](#), [PRINT](#), [PUSH](#), [REALLOC](#), [RET](#), [RETURN](#), [running](#), [STACK_LOAD](#), [STACK_STORE](#), [STACK_SUB](#), [STORE](#), [STRING_PRINT](#), [SUB](#), [TO_FLOAT](#), [TO_INT](#), and [XOR](#).

11.76.4.7 exec_add()

```
void mxvm::Program::exec_add (
    const Instruction & instr)
```

Definition at line 383 of file [icode_exec.cpp](#).

References [addVariables\(\)](#), [createTempVariable\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), and [mxvm::VAR_POINTER](#).

Referenced by [exec\(\)](#).

11.76.4.8 exec_alloc()

```
void mxvm::Program::exec_alloc (
    const Instruction & instr)
```

Definition at line 1170 of file [icode_exec.cpp](#).

References [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_count](#), [mxvm::Variable_Value::ptr_size](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::releaseOwnedPointer\(\)](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.9 exec_and()

```
void mxvm::Program::exec_and (
    const Instruction & instr)
```

Definition at line 1119 of file [icode_exec.cpp](#).

References [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_INTEGER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.10 exec_call()

```
void mxvm::Program::exec_call (
    const Instruction & instr)
```

Definition at line 1759 of file [icode_exec.cpp](#).

References [mxvm::Base::labels](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [pc](#), and [stack](#).

Referenced by [exec\(\)](#).

11.76.4.11 exec_cmp()

```
void mxvm::Program::exec_cmp (
    const Instruction & instr)
```

Definition at line 532 of file [icode_exec.cpp](#).

References [createTempVariable\(\)](#), [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [greater_flag](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [less_flag](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::toString\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [zero_flag](#).

Referenced by [exec\(\)](#).

11.76.4.12 exec_div()

```
void mxvm::Program::exec_div (
    const Instruction & instr)
```

Definition at line 496 of file [icode_exec.cpp](#).

References [createTempVariable\(\)](#), [divVariables\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), and [mxvm::Variable::type](#).

Referenced by [exec\(\)](#).

11.76.4.13 exec_done()

```
void mxvm::Program::exec_done (
    const Instruction & instr)
```

Definition at line 1783 of file [icode_exec.cpp](#).

References [exitCode](#), and [stop\(\)](#).

Referenced by [exec\(\)](#).

11.76.4.14 exec_exit()

```
void mxvm::Program::exec_exit (
    const Instruction & instr)
```

Definition at line 668 of file [icode_exec.cpp](#).

References [exitCode](#), [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [stop\(\)](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.15 exec_fcmp()

```
void mxvm::Program::exec_fcmp (
    const Instruction & instr)
```

Definition at line 2132 of file [icode_exec.cpp](#).

References [createTempVariable\(\)](#), [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [greater_flag](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [less_flag](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [zero_flag](#).

Referenced by [exec\(\)](#).

11.76.4.16 exec_free()

```
void mxvm::Program::exec_free (
    const Instruction & instr)
```

Definition at line 1207 of file [icode_exec.cpp](#).

References [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_count](#), [mxvm::Variable_Value::ptr_size](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

Referenced by [exec\(\)](#).

11.76.4.17 exec_getline()

```
void mxvm::Program::exec_getline (
    const Instruction & instr)
```

Definition at line 1561 of file [icode_exec.cpp](#).

References [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.18 exec_invoke()

```
void mxvm::Program::exec_invoke (
    const Instruction & instr)
```

Definition at line 2056 of file [icode_exec.cpp](#).

References [args](#), [mxvm::Base::external_functions](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::OP_CONSTANT](#), [mxvm::Variable_Value::ptr_value](#), [result](#), [mxvm::Variable::type](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Instruction::vop](#).

Referenced by [exec\(\)](#).

11.76.4.19 exec_ja()

```
void mxvm::Program::exec_ja (
    const Instruction & instr)
```

Definition at line 1384 of file [icode_exec.cpp](#).

References [exec_jmp\(\)](#), [greater_flag](#), and [pc](#).

Referenced by [exec\(\)](#).

11.76.4.20 exec_jae()

```
void mxvm::Program::exec_jae (  
    const Instruction & instr)
```

Definition at line 2180 of file [icode_exec.cpp](#).

References [exec_jmp\(\)](#), [greater_flag](#), [pc](#), and [zero_flag](#).

Referenced by [exec\(\)](#).

11.76.4.21 exec_jb()

```
void mxvm::Program::exec_jb (  
    const Instruction & instr)
```

Definition at line 1391 of file [icode_exec.cpp](#).

References [exec_jmp\(\)](#), [less_flag](#), and [pc](#).

Referenced by [exec\(\)](#).

11.76.4.22 exec_jbe()

```
void mxvm::Program::exec_jbe (  
    const Instruction & instr)
```

Definition at line 2187 of file [icode_exec.cpp](#).

References [exec_jmp\(\)](#), [less_flag](#), [pc](#), and [zero_flag](#).

Referenced by [exec\(\)](#).

11.76.4.23 exec_jc()

```
void mxvm::Program::exec_jc (  
    const Instruction & instr)
```

Definition at line 2194 of file [icode_exec.cpp](#).

References [carry_flag](#), [exec_jmp\(\)](#), and [pc](#).

Referenced by [exec\(\)](#).

11.76.4.24 exec_je()

```
void mxvm::Program::exec_je (  
    const Instruction & instr)
```

Definition at line 1328 of file [icode_exec.cpp](#).

References [exec_jmp\(\)](#), [pc](#), and [zero_flag](#).

Referenced by [exec\(\)](#).

11.76.4.25 exec_jg()

```
void mxvm::Program::exec_jg (  
    const Instruction & instr)
```

Definition at line 1356 of file [icode_exec.cpp](#).

References [exec_jmp\(\)](#), [greater_flag](#), and [pc](#).

Referenced by [exec\(\)](#).

11.76.4.26 exec_jge()

```
void mxvm::Program::exec_jge (  
    const Instruction & instr)
```

Definition at line 1363 of file [icode_exec.cpp](#).

References [exec_jmp\(\)](#), [greater_flag](#), [pc](#), and [zero_flag](#).

Referenced by [exec\(\)](#).

11.76.4.27 exec_jl()

```
void mxvm::Program::exec_jl (  
    const Instruction & instr)
```

Definition at line 1342 of file `icode_exec.cpp`.
 References `exec_jump()`, `less_flag`, and `pc`.
 Referenced by `exec()`.

11.76.4.28 `exec_jle()`

```
void mxvm::Program::exec_jle (
    const Instruction & instr)
```

Definition at line 1349 of file `icode_exec.cpp`.
 References `exec_jump()`, `less_flag`, `pc`, and `zero_flag`.
 Referenced by `exec()`.

11.76.4.29 `exec_jump()`

```
void mxvm::Program::exec_jump (
    const Instruction & instr)
```

Definition at line 622 of file `icode_exec.cpp`.
 References `mxvm::Base::labels`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, and `pc`.
 Referenced by `exec()`, `exec_ja()`, `exec_jae()`, `exec_jb()`, `exec_jbe()`, `exec_jc()`, `exec_je()`, `exec_jg()`, `exec_jge()`, `exec_jl()`, `exec_jle()`, `exec_jnc()`, `exec_jne()`, `exec_jno()`, `exec_jnp()`, `exec_jns()`, `exec_jnz()`, and `exec_jz()`.

11.76.4.30 `exec_jnc()`

```
void mxvm::Program::exec_jnc (
    const Instruction & instr)
```

Definition at line 2201 of file `icode_exec.cpp`.
 References `carry_flag`, `exec_jump()`, and `pc`.
 Referenced by `exec()`.

11.76.4.31 `exec_jne()`

```
void mxvm::Program::exec_jne (
    const Instruction & instr)
```

Definition at line 1335 of file `icode_exec.cpp`.
 References `exec_jump()`, `pc`, and `zero_flag`.
 Referenced by `exec()`.

11.76.4.32 `exec_jno()`

```
void mxvm::Program::exec_jno (
    const Instruction & instr)
```

Definition at line 2220 of file `icode_exec.cpp`.
 References `exec_jump()`.
 Referenced by `exec()`.

11.76.4.33 `exec_jnp()`

```
void mxvm::Program::exec_jnp (
    const Instruction & instr)
```

Definition at line 2212 of file `icode_exec.cpp`.
 References `exec_jump()`.
 Referenced by `exec()`.

11.76.4.34 `exec_jns()`

```
void mxvm::Program::exec_jns (
    const Instruction & instr)
```

Definition at line 2228 of file `icode_exec.cpp`.
 References `exec_jump()`.
 Referenced by `exec()`.

11.76.4.35 exec_jnz()

```
void mxvm::Program::exec_jnz (
    const Instruction & instr)
```

Definition at line 1377 of file [icode_exec.cpp](#).

References [exec_jmp\(\)](#), [pc](#), and [zero_flag](#).

Referenced by [exec\(\)](#).

11.76.4.36 exec_jo()

```
void mxvm::Program::exec_jo (
    const Instruction & instr)
```

Definition at line 2216 of file [icode_exec.cpp](#).

References [pc](#).

Referenced by [exec\(\)](#).

11.76.4.37 exec_jp()

```
void mxvm::Program::exec_jp (
    const Instruction & instr)
```

Definition at line 2208 of file [icode_exec.cpp](#).

References [pc](#).

Referenced by [exec\(\)](#).

11.76.4.38 exec_js()

```
void mxvm::Program::exec_js (
    const Instruction & instr)
```

Definition at line 2224 of file [icode_exec.cpp](#).

References [pc](#).

Referenced by [exec\(\)](#).

11.76.4.39 exec_jz()

```
void mxvm::Program::exec_jz (
    const Instruction & instr)
```

Definition at line 1370 of file [icode_exec.cpp](#).

References [exec_jmp\(\)](#), [pc](#), and [zero_flag](#).

Referenced by [exec\(\)](#).

11.76.4.40 exec_lea()

```
void mxvm::Program::exec_lea (
    const Instruction & instr)
```

Definition at line 1234 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_count](#), [mxvm::Variable_Value::ptr_size](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.41 exec_load()

```
void mxvm::Program::exec_load (
    const Instruction & instr)
```

Definition at line 864 of file [icode_exec.cpp](#).

References [mxvm::Base::base](#), [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_count](#), [mxvm::Variable_Value::ptr_size](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_BYTE](#),

[mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Instruction::vop](#).
Referenced by [exec\(\)](#).

11.76.4.42 `exec_mod()`

```
void mxvm::Program::exec_mod (
    const Instruction & instr)
```

Definition at line 1281 of file [icode_exec.cpp](#).

References [createTempVariable\(\)](#), [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.43 `exec_mov()`

```
void mxvm::Program::exec_mov (
    const Instruction & instr)
```

Definition at line 316 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_count](#), [mxvm::Variable_Value::ptr_size](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::releaseOwnedPointer\(\)](#), [setVariableFromConstant\(\)](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

Referenced by [exec\(\)](#).

11.76.4.44 `exec_mul()`

```
void mxvm::Program::exec_mul (
    const Instruction & instr)
```

Definition at line 460 of file [icode_exec.cpp](#).

References [createTempVariable\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mulVariables\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), and [mxvm::Variable::type](#).

Referenced by [exec\(\)](#).

11.76.4.45 `exec_neg()`

```
void mxvm::Program::exec_neg (
    const Instruction & instr)
```

Definition at line 1885 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.46 `exec_not()`

```
void mxvm::Program::exec_not (
    const Instruction & instr)
```

Definition at line 1269 of file [icode_exec.cpp](#).

References [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_INTEGER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.47 `exec_or()`

```
void mxvm::Program::exec_or (
    const Instruction & instr)
```

Definition at line 1136 of file `icode_exec.cpp`.

References `getVariable()`, `mxvm::Variable_Value::int_value`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `mxvm::Variable_Value::type`, `mxvm::VAR_INTEGER`, and `mxvm::Variable::var_value`.

Referenced by `exec()`.

11.76.4.48 `exec_pop()`

```
void mxvm::Program::exec_pop (
    const Instruction & instr)
```

Execute a POP instruction — restore a value from the VM stack into a variable.

Matches the pushed type via `std::holds_alternative` and writes back into the correct union member. Float variables are restored from `std::get<double>` to preserve IEEE 754 representation.

Parameters

<i>instr</i>	<code>Instruction</code> whose op1 is the destination variable.
--------------	---

Definition at line 1621 of file `icode_exec.cpp`.

References `mxvm::Variable_Value::float_value`, `getVariable()`, `mxvm::Variable_Value::int_value`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Variable_Value::ptr_value`, `stack`, `mxvm::Variable_Value::str_value`, `mxvm::Variable::type`, `mxvm::Variable_Value::type`, `mxvm::VAR_BYTE`, `mxvm::VAR_EXTERN`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, and `mxvm::Variable::var_value`.

Referenced by `exec()`.

11.76.4.49 `exec_print()`

```
void mxvm::Program::exec_print (
    const Instruction & instr)
```

Definition at line 631 of file `icode_exec.cpp`.

References `args`, `createTempVariable()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `mxvm::OP_VARIABLE`, `printFormatted()`, `mxvm::Variable_Value::str_value`, `mxvm::Operand::type`, `mxvm::Variable::type`, `mxvm::VAR_INTEGER`, `mxvm::VAR_STRING`, `mxvm::Variable::var_value`, and `mxvm::Instruction::vop`.

Referenced by `exec()`.

11.76.4.50 `exec_push()`

```
void mxvm::Program::exec_push (
    const Instruction & instr)
```

Execute a PUSH instruction — save a variable or constant onto the VM stack.

Handles integer/byte, pointer/extern, float, and string variable types. Float variables are pushed as native doubles so that POP can restore them correctly using `std::get<double>`.

Parameters

<i>instr</i>	<code>Instruction</code> whose op1 is the value to push.
--------------	--

Definition at line 1588 of file `icode_exec.cpp`.

References `mxvm::Variable_Value::float_value`, `getVariable()`, `mxvm::Variable_Value::int_value`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Variable_Value::ptr_value`, `stack`, `mxvm::Variable_Value::str_value`, `mxvm::Variable_Value::type`, `mxvm::VAR_BYTE`, `mxvm::VAR_EXTERN`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, and `mxvm::Variable::var_value`.

Referenced by `exec()`.

11.76.4.51 `exec_realloc()`

```
void mxvm::Program::exec_realloc (
```

```
const Instruction & instr)
```

Execute the REALLOC instruction in the interpreter.

Execute the REALLOC instruction.

Resizes the memory block pointed to by the destination variable. Operands: dest (pointer var), elemSize, count. Zero-initialises any newly allocated bytes beyond the old allocation. A count of zero frees the block.

Resizes the heap block of a pointer variable using C realloc(). Operands: op1 = destination ptr variable, op2 = element size, op3 = new element count. Zero-fills newly allocated bytes. A count of zero frees the block.

Definition at line 1910 of file [icode_exec.cpp](#).

References [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_count](#), [mxvm::Variable_Value::ptr_size](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.52 [exec_ret\(\)](#)

```
void mxvm::Program::exec_ret (
    const Instruction & instr)
```

Definition at line 1772 of file [icode_exec.cpp](#).

References [pc](#), and [stack](#).

Referenced by [exec\(\)](#).

11.76.4.53 [exec_return\(\)](#)

```
void mxvm::Program::exec_return (
    const Instruction & instr)
```

Definition at line 1872 of file [icode_exec.cpp](#).

References [getVariable\(\)](#), [isVariable\(\)](#), [name](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable_Value::owns](#), [mxvm::releaseOwnedPointer\(\)](#), [result](#), [mxvm::Variable::type](#), [mxvm::Variable::var_name](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.54 [exec_stack_load\(\)](#)

```
void mxvm::Program::exec_stack_load (
    const Instruction & instr)
```

Definition at line 1661 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Variable_Value::ptr_value](#), [stack](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.55 [exec_stack_store\(\)](#)

```
void mxvm::Program::exec_stack_store (
    const Instruction & instr)
```

Definition at line 1710 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Variable_Value::ptr_value](#), [stack](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.56 [exec_stack_sub\(\)](#)

```
void mxvm::Program::exec_stack_sub (
    const Instruction & instr)
```

Definition at line 1742 of file `icode_exec.cpp`.

References `getVariable()`, `mxvm::Variable_Value::int_value`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `stack`, and `mxvm::Variable::var_value`.

Referenced by `exec()`.

11.76.4.57 `exec_store()`

```
void mxvm::Program::exec_store (
    const Instruction & instr)
```

Execute a STORE instruction — write a value into a pointer at a given offset.

Supports integer, float, pointer, byte, and string source types. For constants the full 8-byte integer representation is always stored, regardless of the stride operand, to prevent partial-write corruption when storing values larger than one byte.

Parameters

<i>instr</i>	<code>Instruction</code> with op1 = source, op2 = destination pointer, op3 = index, vop[0] = stride.
--------------	--

Definition at line 1008 of file `icode_exec.cpp`.

References `mxvm::Base::base`, `mxvm::Variable_Value::float_value`, `getVariable()`, `mxvm::Variable_Value::int_value`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `mxvm::OP_CONSTANT`, `mxvm::Variable_Value::owns`, `mxvm::Variable_Value::ptr_count`, `mxvm::Variable_Value::ptr_size`, `mxvm::Variable_Value::ptr_value`, `mxvm::Variable_Value::str_value`, `mxvm::Operand::type`, `mxvm::Variable::type`, `mxvm::VAR_BYTE`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::Variable::var_name`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, `mxvm::Variable::var_value`, and `mxvm::Instruction::vop`.

Referenced by `exec()`.

11.76.4.58 `exec_string_print()`

```
void mxvm::Program::exec_string_print (
    const Instruction & instr)
```

Definition at line 1398 of file `icode_exec.cpp`.

References `args`, `createTempVariable()`, `mxvm::Variable_Value::float_value`, `getVariable()`, `mxvm::Variable_Value::int_value`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `mxvm::OP_VARIABLE`, `mxvm::Variable_Value::ptr_value`, `mxvm::Variable_Value::str_value`, `mxvm::Operand::type`, `mxvm::Variable::type`, `mxvm::Variable_Value::type`, `mxvm::VAR_BYTE`, `mxvm::VAR_EXTERN`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, `mxvm::Variable::var_value`, and `mxvm::Instruction::vop`.

Referenced by `exec()`.

11.76.4.59 `exec_sub()`

```
void mxvm::Program::exec_sub (
    const Instruction & instr)
```

Definition at line 421 of file `icode_exec.cpp`.

References `createTempVariable()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `subVariables()`, `mxvm::Variable::type`, `mxvm::VAR_INTEGER`, and `mxvm::VAR_POINTER`.

Referenced by `exec()`.

11.76.4.60 `exec_to_float()`

```
void mxvm::Program::exec_to_float (
    const Instruction & instr)
```

Definition at line 1814 of file `icode_exec.cpp`.

References `mxvm::Variable_Value::float_value`, `getVariable()`, `mxvm::Variable_Value::int_value`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Variable_Value::ptr_value`, `mxvm::Variable_Value::str_value`, `mxvm::Variable::type`, `mxvm::Variable_Value::type`, `mxvm::VAR_BYTE`, `mxvm::VAR_EXTERN`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, and `mxvm::Variable::var_value`.

Referenced by `exec()`.

11.76.4.61 exec_to_int()

```
void mxvm::Program::exec_to_int (
    const Instruction & instr)
```

Definition at line 1788 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.62 exec_xor()

```
void mxvm::Program::exec_xor (
    const Instruction & instr)
```

Definition at line 1153 of file [icode_exec.cpp](#).

References [getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_INTEGER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec\(\)](#).

11.76.4.63 flatten()

```
void mxvm::Program::flatten (
    Program * program)
```

Flatten child object programs into the root instruction stream.

Parameters

<i>program</i>	Root program to merge into
----------------	----------------------------

Definition at line 102 of file [icode_exec.cpp](#).

References [Program\(\)](#), [mxvm::Base::external_functions](#), [flatten_external\(\)](#), [flatten_inc\(\)](#), [flatten_label\(\)](#), [mxvm::Base::inc](#), [mxvm::Base::labels](#), and [objects](#).

11.76.4.64 flatten_external()

```
void mxvm::Program::flatten_external (
    Program * root,
    const std::string & e,
    RuntimeFunction & r)
```

Copy an external function binding from a child object into the root.

Parameters

<i>root</i>	Root program
<i>e</i>	External function name
<i>r</i>	Resolved RuntimeFunction binding

Definition at line 96 of file [icode_exec.cpp](#).

References [Program\(\)](#), and [mxvm::Base::external_functions](#).

Referenced by [flatten\(\)](#).

11.76.4.65 flatten_inc()

```
void mxvm::Program::flatten_inc (
    Program * root,
    Instruction & i)
```

Rewrite object-qualified operand references in a single instruction.

Parameters

<i>root</i>	Root program owning the merged instruction stream
<i>i</i>	Instruction to rewrite (modified in place)

Definition at line 27 of file [icode_exec.cpp](#).

References [Program\(\)](#), [mxvm::Base::add_instruction\(\)](#), [CALL](#), [mxvm::Instruction::instruction](#), [JA](#), [JB](#), [JE](#), [JG](#), [JGE](#), [JL](#), [JLE](#), [JMP](#), [JNE](#), [JNZ](#), [JZ](#), [mxvm::Base::labels](#), [name](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::OP_VARIABLE](#), [mxvm::Base::vars](#), and [mxvm::Instruction::vop](#).

Referenced by [flatten\(\)](#).

11.76.4.66 flatten_label()

```
void mxvm::Program::flatten_label (
    Program * root,
    int64_t offset,
    const std::string & label,
    bool func)
```

Copy a label from a child object into the root label map with an address offset.

Parameters

<i>root</i>	Root program
<i>offset</i>	Instruction index offset
<i>label</i>	Label name
<i>func</i>	true if the label is a function entry

Definition at line 87 of file [icode_exec.cpp](#).

References [Program\(\)](#), [mxvm::Base::add_label\(\)](#), and [name](#).

Referenced by [flatten\(\)](#).

11.76.4.67 gen_add()

```
void mxvm::Program::gen_add (
    std::ostream & out,
    const Instruction & i)
```

11.76.4.68 gen_alloc()

```
void mxvm::Program::gen_alloc (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 661 of file [icode_gen.cpp](#).

References [mxvm::error_label_count](#), [getMangledName\(\)](#), [getPlatformSymbolName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::OP_CONSTANT](#), [mxvm::Variable_Value::owns](#), [sysv_emitLoadVar\(\)](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.69 gen_arth()

```
void mxvm::Program::gen_arth (
    std::ostream & out,
    std::string arth,
    const Instruction & i)
```

Definition at line 1944 of file [icode_gen.cpp](#).

References [generateLoadVar\(\)](#), [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::OP_CONSTANT](#), [sysv_emitStoreVar\(\)](#), [sysv_reg_vars](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), and [mxvm::VAR_POINTER](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.70 gen_bitop()

```
void mxvm::Program::gen_bitop (
    std::ostream & out,
    const std::string & opc,
    const Instruction & i)
```

Definition at line 1302 of file [icode_gen.cpp](#).

References [generateLoadVar\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [sysv_emitStoreVar\(\)](#), [mxvm::Variable::type](#), and [mxvm::VAR_INTEGER](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.71 gen_call()

```
void mxvm::Program::gen_call (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 638 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [getPlatformSymbolName\(\)](#), [getVariable\(\)](#), [isFunctionValid\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [sysv_emitFlushRegs\(\)](#), [sysv_emitLoadVar\(\)](#), [sysv_emitReloadRegs\(\)](#), and [mxvm::VAR_INTEGER](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.72 gen_cmp()

```
void mxvm::Program::gen_cmp (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1749 of file [icode_gen.cpp](#).

References [CMP_FLOAT](#), [CMP_INTEGER](#), [generateLoadVar\(\)](#), [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [last_cmp_type](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::OP_CONSTANT](#), [sysv_reg_vars](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), and [mxvm::VAR_POINTER](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.73 gen_div()

```
void mxvm::Program::gen_div (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1155 of file [icode_gen.cpp](#).

References [generateLoadVar\(\)](#), [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [sysv_emitStoreVar\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), and [mxvm::VAR_INTEGER](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.74 gen_done()

```
void mxvm::Program::gen_done (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 475 of file [icode_gen.cpp](#).

References [mxvm::Base::external](#), [getPlatformSymbolName\(\)](#), [sysv_emitFlushRegs\(\)](#), and [sysv_emitRestoreRegs\(\)](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.75 gen_exit()

```
void mxvm::Program::gen_exit (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 2002 of file `icode_gen.cpp`.

References `mxvm::Base::external`, `generateFunctionCall()`, `getPlatformSymbolName()`, `mxvm::Operand::op`, and `mxvm::Instruction::op1`.

Referenced by `generateInstruction()`.

11.76.4.76 gen_fcmp()

```
void mxvm::Program::gen_fcmp (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1793 of file `icode_gen.cpp`.

References `CMP_FLOAT`, `generateLoadVar()`, `last_cmp_type`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, and `mxvm::VAR_FLOAT`.

Referenced by `generateInstruction()`.

11.76.4.77 gen_free()

```
void mxvm::Program::gen_free (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1013 of file `icode_gen.cpp`.

References `getMangledName()`, `getPlatformSymbolName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Variable_Value::owns`, `mxvm::Variable::type`, `mxvm::VAR_POINTER`, and `mxvm::Variable::var_value`.

Referenced by `generateInstruction()`.

11.76.4.78 gen_getline()

```
void mxvm::Program::gen_getline (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1271 of file `icode_gen.cpp`.

References `mxvm::Variable_Value::buffer_size`, `getMangledName()`, `getPlatformSymbolName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Variable::type`, `mxvm::VAR_STRING`, and `mxvm::Variable::var_value`.

Referenced by `generateInstruction()`.

11.76.4.79 gen_invoke()

```
void mxvm::Program::gen_invoke (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 574 of file `icode_gen.cpp`.

References `generateInvokeCall()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, and `mxvm::Instruction::vop`.

Referenced by `generateInstruction()`.

11.76.4.80 gen_jae()

```
void mxvm::Program::gen_jae (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1804 of file `icode_gen.cpp`.

References `mxvm::Operand::op`, and `mxvm::Instruction::op1`.

Referenced by `generateInstruction()`.

11.76.4.81 gen_jbe()

```
void mxvm::Program::gen_jbe (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1808 of file [icode_gen.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.82 gen_jc()

```
void mxvm::Program::gen_jc (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1812 of file [icode_gen.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.83 gen_jmp()

```
void mxvm::Program::gen_jmp (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1698 of file [icode_gen.cpp](#).

References [CMP_FLOAT](#), [mxvm::Instruction::instruction](#), [JA](#), [JB](#), [JE](#), [JG](#), [JGE](#), [JL](#), [JLE](#), [JMP](#), [JNE](#), [JNZ](#), [JZ](#), [mxvm::Base::labels](#), [last_cmp_type](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), and [pos\(\)](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.84 gen_jnc()

```
void mxvm::Program::gen_jnc (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1816 of file [icode_gen.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.85 gen_jno()

```
void mxvm::Program::gen_jno (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1832 of file [icode_gen.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.86 gen_jnp()

```
void mxvm::Program::gen_jnp (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1824 of file [icode_gen.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.87 gen_jns()

```
void mxvm::Program::gen_jns (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1840 of file `icode_gen.cpp`.

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.88 gen_jo()

```
void mxvm::Program::gen_jo (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1828 of file `icode_gen.cpp`.

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.89 gen_jp()

```
void mxvm::Program::gen_jp (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1820 of file `icode_gen.cpp`.

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.90 gen_js()

```
void mxvm::Program::gen_js (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1836 of file `icode_gen.cpp`.

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.91 gen_lea()

```
void mxvm::Program::gen_lea (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 557 of file `icode_gen.cpp`.

References [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Variable_Value::owns](#), [sysv_reg_vars](#), and [mxvm::Variable::var_value](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.92 gen_load()

```
void mxvm::Program::gen_load (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 768 of file `icode_gen.cpp`.

References [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [sysv_emitFlushRegs\(\)](#), [sysv_emitLoadVar\(\)](#), [sysv_emitStoreVar\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Instruction::vop](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.93 gen_mod()

```
void mxvm::Program::gen_mod (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1225 of file `icode_gen.cpp`.

References [generateLoadVar\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [sysv_emitStoreVar\(\)](#), [mxvm::Variable::type](#), and [mxvm::VAR_INTEGER](#).
Referenced by [generateInstruction\(\)](#).

11.76.4.94 gen_mov()

```
void mxvm::Program::gen_mov (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1844 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [getPlatformSymbolName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::OP_CONSTANT](#), [sysv_emitLoadVar\(\)](#), [sysv_emitStoreVar\(\)](#), [sysv_emitStoreVarImm\(\)](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.95 gen_mul()

```
void mxvm::Program::gen_mul (
    std::ostream & out,
    const Instruction & i)
```

References [platform](#).

11.76.4.96 gen_neg()

```
void mxvm::Program::gen_neg (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 537 of file [icode_gen.cpp](#).

References [generateLoadVar\(\)](#), [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [sysv_emitStoreVar\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), and [mxvm::VAR_INTEGER](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.97 gen_not()

```
void mxvm::Program::gen_not (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1334 of file [icode_gen.cpp](#).

References [generateLoadVar\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [sysv_emitStoreVar\(\)](#), [mxvm::Variable::type](#), and [mxvm::VAR_INTEGER](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.98 gen_optimize()

```
std::string mxvm::Program::gen_optimize (
    const std::string & code,
    const Platform & platform)
```

Peephole optimizer for generated assembly.

Apply peephole optimizations to generated assembly

Parameters

<i>code</i>	Raw assembly text
<i>platform</i>	Target platform (affects optimization rules)

Returns

Optimized assembly text

Definition at line 496 of file `icode_opt.cpp`.

References `mxvm::DARWIN`, `mxvm::join_lines()`, `mxvm::opt_core_lines()`, `mxvm::opt_darwin_lines()`, `mxvm::opt_linux_lines()`, `mxvm::opt_x64_windows_lines()`, `mxvm::WINX64`, and `mxvm::x64_opt_core_lines()`.

11.76.4.99 gen_pop()

```
void mxvm::Program::gen_pop (
    std::ostream & out,
    const Instruction & i)
```

Emit x86-64 POP into a variable.

Handles integer (popq + store), pointer/extern (popq + movq), and float variables. Floating-point values are popped via `movsd (rsp), xmm0; addq $8, rsp; movsd xmm0, var(rip)` mirroring the SSE push sequence in `gen_push()`.

Parameters

<i>out</i>	Assembly output stream.
<i>i</i>	<code>Instruction</code> whose op1 is the destination variable.

Definition at line 1400 of file `icode_gen.cpp`.

References `getMangledName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `sysv_emitStoreVar()`, `mxvm::Variable::type`, `mxvm::VAR_EXTERN`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, and `mxvm::VAR_POINTER`.

Referenced by `generateInstruction()`.

11.76.4.100 gen_print()

```
void mxvm::Program::gen_print (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1644 of file `icode_gen.cpp`.

References `args`, `generateLoadVar()`, `getMangledName()`, `getPlatformSymbolName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `mxvm::Variable::type`, `mxvm::VAR_FLOAT`, `mxvm::Instruction::vop`, and `xmm_offset`.

Referenced by `generateInstruction()`.

11.76.4.101 gen_push()

```
void mxvm::Program::gen_push (
    std::ostream & out,
    const Instruction & i)
```

Emit x86-64 PUSH for a variable or constant.

Handles integer/pointer (pushq), string (lea+pushq), and float variables. Floating-point values are pushed via `movsd var(rip), xmm0; subq $8, rsp; movsd xmm0, (rsp)` since x86 PUSH cannot operate directly on XMM registers.

Parameters

<i>out</i>	Assembly output stream.
<i>i</i>	<code>Instruction</code> whose op1 is the value to push.

Definition at line 1365 of file `icode_gen.cpp`.

References `getMangledName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::OP_CONSTANT`, `sysv_emitLoadVar()`, `mxvm::Operand::type`, `mxvm::Variable::type`, `mxvm::VAR_EXTERN`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, and `mxvm::VAR_STRING`.

Referenced by [generateInstruction\(\)](#).

11.76.4.102 gen_realloc()

```
void mxvm::Program::gen_realloc (
    std::ostream & out,
    const Instruction & i)
```

Generate System V x86-64 assembly for the REALLOC instruction (dynamic array resize).

Generate System V x86-64 native code for REALLOC.

Computes total = count * elemSize, calls C realloc() with the existing pointer and new size, and stores the result back. [Stack](#) is 16-byte aligned around the call. Registers are flushed/reloaded via sysv_emitFlushRegs / sysv_↵emitReloadRegs.

Definition at line 715 of file [icode_gen.cpp](#).

References [mxvm::error_label_count](#), [getMangledName\(\)](#), [getPlatformSymbolName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable_Value::owns](#), [sysv_emitFlushRegs\(\)](#), [sysv_emitLoadVar\(\)](#), [sysv_emitReloadRegs\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.103 gen_ret()

```
void mxvm::Program::gen_ret (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 505 of file [icode_gen.cpp](#).

References [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [sysv_emitFlushRegs\(\)](#), [sysv_emitLoadVar\(\)](#), [sysv_emitRestoreRegs\(\)](#), and [mxvm::VAR_INTEGER](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.104 gen_return()

```
void mxvm::Program::gen_return (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 518 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [last_call_returns_owned_ptr](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable_Value::owns](#), [sysv_emitStoreVar\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), and [mxvm::Variable::var_value](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.105 gen_stack_load()

```
void mxvm::Program::gen_stack_load (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1420 of file [icode_gen.cpp](#).

References [generateLoadVar\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [sysv_emitStoreVar\(\)](#), and [mxvm::Variable::type](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.106 gen_stack_store()

```
void mxvm::Program::gen_stack_store (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1433 of file [icode_gen.cpp](#).

References [generateLoadVar\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [sysv_emitLoadVar\(\)](#), and [mxvm::Variable::type](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.107 gen_stack_sub()

```
void mxvm::Program::gen_stack_sub (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1446 of file [icode_gen.cpp](#).

References [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), and [sysv_emitLoadVar\(\)](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.108 gen_store()

```
void mxvm::Program::gen_store (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 891 of file [icode_gen.cpp](#).

References [mxvm::Variable::Value::buffer_size](#), [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [sysv_emitFlushRegs\(\)](#), [sysv_emitLoadVar\(\)](#), [sysv_emitReloadRegs\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Instruction::vop](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.109 gen_sub()

```
void mxvm::Program::gen_sub (
    std::ostream & out,
    const Instruction & i)
```

11.76.4.110 gen_to_float()

```
void mxvm::Program::gen_to_float (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1093 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [getPlatformSymbolName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::OP_CONSTANT](#), [sysv_reg_vars](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.111 gen_to_int()

```
void mxvm::Program::gen_to_int (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1034 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [getPlatformSymbolName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::OP_CONSTANT](#), [sysv_emitStoreVar\(\)](#), [sysv_reg_vars](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

Referenced by [generateInstruction\(\)](#).

11.76.4.112 generateCode()

```
void mxvm::Program::generateCode (
    const Platform & platform,
    bool obj,
    std::ostream & out)
```

Generate native x64 assembly for the entire program.

Parameters

<i>platform</i>	Target platform (Linux, Darwin, WinX64)
<i>obj</i>	true if generating as an object (no <code>_start/main</code>)
<i>out</i>	Output stream for the assembly

Definition at line 120 of file `icode_gen.cpp`.

References `add_standard()`, `mxvm::Base::base`, `mx::Color::BRIGHT_BLUE`, `mx::Color::BRIGHT_CYAN`, `Col()`, `mxvm::DARWIN`, `DONE`, `escapeNewLines()`, `mxvm::Base::external`, `generateInstruction()`, `getMangledName()`, `getPlatformSymbolName()`, `getVariable()`, `mxvm::Base::inc`, `mxvm::Instruction::instruction`, `isVariable()`, `mxvm::Base::labels`, `mxvm::LINUX`, `name`, `mxvm::Operand::op`, `platform`, `mxvm::Base::root_name`, `sysv_analyzeRegAlloc()`, `sysv_emitReloadRegs()`, `sysv_emitSaveRegs()`, `sysv_emitStoreVar()`, `mxvm::VAR_BYTE`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, `mxvm::Base::vars`, `mxvm::WINX64`, and `x64_generateCode()`.

11.76.4.113 generateFunctionCall()

```
void mxvm::Program::generateFunctionCall (
    std::ostream & out,
    const std::string & name,
    std::vector< Operand > & op)
```

Generate a System V ABI function call with operands as arguments.

Parameters

<i>out</i>	Assembly output stream
<i>name</i>	Callee function name
<i>op</i>	Operand list to pass as arguments

Definition at line 610 of file `icode_gen.cpp`.

References `args`, `generateLoadVar()`, `getPlatformSymbolName()`, `name`, `mxvm::VAR_INTEGER`, and `xmm_offset`.
Referenced by `gen_exit()`, and `generateInvokeCall()`.

11.76.4.114 generateInstruction()

```
void mxvm::Program::generateInstruction (
    std::ostream & out,
    const Instruction & i)
```

Dispatch a single instruction to its System V code generator.

Parameters

<i>out</i>	Assembly output stream
<i>i</i>	Instruction to generate code for

Definition at line 321 of file `icode_gen.cpp`.

References `ADD`, `ALLOC`, `AND`, `CALL`, `CMP`, `CMP_NONE`, `DIV`, `DONE`, `EXIT`, `FCMP`, `FREE`, `gen_alloc()`, `gen_arth()`, `gen_bitop()`, `gen_call()`, `gen_cmp()`, `gen_div()`, `gen_done()`, `gen_exit()`, `gen_fcmp()`, `gen_free()`, `gen_getline()`, `gen_invoke()`, `gen_jae()`, `gen_jbe()`, `gen_jc()`, `gen_jmp()`, `gen_jnc()`, `gen_jno()`, `gen_jnp()`, `gen_jns()`, `gen_jo()`, `gen_jp()`, `gen_js()`, `gen_lea()`, `gen_load()`, `gen_mod()`, `gen_mov()`, `gen_neg()`, `gen_not()`, `gen_pop()`, `gen_print()`, `gen_push()`, `gen_realloc()`, `gen_ret()`, `gen_return()`, `gen_stack_load()`, `gen_stack_store()`, `gen_stack_sub()`, `gen_store()`, `gen_to_float()`, `gen_to_int()`, `GETLINE`, `mxvm::Instruction::instruction`, `INVOKE`, `JA`, `JAE`, `JB`, `JBE`, `JC`, `JE`, `JG`, `JGE`, `JL`, `JLE`, `JMP`, `JNC`, `JNE`, `JNO`, `JNP`, `JNS`, `JNZ`, `JO`, `JP`, `JS`, `JZ`, `last_cmp_type`, `LEA`, `LOAD`, `MOD`, `MOV`, `MUL`, `NEG`, `NOT`, `OR`, `POP`, `PRINT`, `PUSH`, `REALLOC`, `RET`, `RETURN`, `STACK_LOAD`, `STACK_STORE`, `STACK_SUB`, `STORE`, `SUB`, `TO_FLOAT`, `TO_INT`, and `XOR`.
Referenced by `generateCode()`.

11.76.4.115 generateInvokeCall()

```
void mxvm::Program::generateInvokeCall (
    std::ostream & out,
    std::vector< Operand > & op)
```

Generate a dynamic invoke call (runtime-resolved external function).

Parameters

<i>out</i>	Assembly output stream
<i>op</i>	Operand list: first is the function name, rest are arguments

Definition at line 591 of file [icode_gen.cpp](#).

References [args](#), [generateFunctionCall\(\)](#), [mxvm::isFunctionReturningOwnedPtr\(\)](#), [last_call_returns_owned_ptr](#), and [name](#).

Referenced by [gen_invoke\(\)](#).

11.76.4.116 generateLoadVar() [1/2]

```
int mxvm::Program::generateLoadVar (
    std::ostream & out,
    int reg,
    const Operand & op)
```

Load a variable or constant into a numbered register.

Parameters

<i>out</i>	Assembly output stream
<i>reg</i>	Register index (0-based)
<i>op</i>	Operand to load

Returns

1 if the operand was a constant, 0 if variable

Definition at line 1460 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [getPlatformSymbolName\(\)](#), [getRegisterByIndex\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::OP_CONSTANT](#), [mxvm::OP_VARIABLE](#), [sysv_reg_vars](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

Referenced by [gen_arth\(\)](#), [gen_bitop\(\)](#), [gen_cmp\(\)](#), [gen_div\(\)](#), [gen_fcmp\(\)](#), [gen_mod\(\)](#), [gen_neg\(\)](#), [gen_not\(\)](#), [gen_print\(\)](#), [gen_stack_load\(\)](#), [gen_stack_store\(\)](#), and [generateFunctionCall\(\)](#).

11.76.4.117 generateLoadVar() [2/2]

```
int mxvm::Program::generateLoadVar (
    std::ostream & out,
    VarType type,
    std::string reg,
    const Operand & op)
```

Load a variable or constant into a named register.

Parameters

<i>out</i>	Assembly output stream
<i>type</i>	Expected variable type

<i>reg</i>	Target register name (e.g. "rax")
<i>op</i>	Operand to load

Returns

1 if the operand was a constant, 0 if variable

Definition at line 1507 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [getPlatformSymbolName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::OP_CONSTANT](#), [mxvm::OP_VARIABLE](#), [sysv_reg_vars](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

11.76.4.118 getExitCode()

```
int mxvm::Program::getExitCode () const [inline]
```

Get the program's exit code after execution.

Returns

Exit code (0 on success)

Definition at line 305 of file [icode.hpp](#).

References [exitCode](#).

Referenced by [exec\(\)](#).

11.76.4.119 getMangledName() [1/2]

```
std::string mxvm::Program::getMangledName (
    const Operand & op)
```

Mangle an operand's variable reference with its object prefix.

Parameters

<i>op</i>	Operand containing the variable reference
-----------	---

Returns

Object-qualified name

Definition at line 55 of file [icode.cpp](#).

References [mxvm::Base::base](#), [mxvm::DARWIN](#), [getMangledName\(\)](#), [mxvm::Operand::label](#), [mxvm::Base::labels](#), [name](#), [mxvm::Operand::object](#), [mxvm::Operand::op](#), and [platform](#).

11.76.4.120 getMangledName() [2/2]

```
std::string mxvm::Program::getMangledName (
    const std::string & var)
```

Mangle a variable name with its owning object prefix.

Parameters

<i>var</i>	Raw variable name
------------	-------------------

Returns

Object-qualified name (e.g. "Object.var")

Definition at line 22 of file [icode.cpp](#).

References [mxvm::DARWIN](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Variable::obj_name](#), and [platform](#).

Referenced by [gen_alloc\(\)](#), [gen_arth\(\)](#), [gen_call\(\)](#), [gen_cmp\(\)](#), [gen_div\(\)](#), [gen_free\(\)](#), [gen_getline\(\)](#), [gen_lea\(\)](#), [gen_load\(\)](#), [gen_mov\(\)](#), [gen_neg\(\)](#), [gen_pop\(\)](#), [gen_print\(\)](#), [gen_push\(\)](#), [gen_realloc\(\)](#), [gen_return\(\)](#), [gen_store\(\)](#), [gen_to_float\(\)](#), [gen_to_int\(\)](#), [generateCode\(\)](#), [generateLoadVar\(\)](#), [generateLoadVar\(\)](#), [getMangledName\(\)](#), [sysv_emitFlushRegs\(\)](#), [sysv_emitLoadVar\(\)](#), [sysv_emitReloadRegs\(\)](#), [sysv_emitStoreVar\(\)](#), [sysv_emitStoreVarImm\(\)](#), [x64_emitFlushRegs\(\)](#), [x64_emitLoadVar\(\)](#), [x64_emitReloadRegs\(\)](#), [x64_emitStoreVar\(\)](#), [x64_emitStoreVarImm\(\)](#), [x64_gen_alloc\(\)](#), [x64_gen_arth\(\)](#), [x64_gen_call\(\)](#), [x64_gen_cmp\(\)](#), [x64_gen_div\(\)](#), [x64_gen_free\(\)](#), [x64_gen_getline\(\)](#), [x64_gen_lea\(\)](#), [x64_gen_load\(\)](#), [x64_gen_mov\(\)](#), [x64_gen_neg\(\)](#), [x64_gen_push\(\)](#), [x64_gen_realloc\(\)](#), [x64_gen_store\(\)](#), [x64_gen_to_float\(\)](#), [x64_gen_to_int\(\)](#), [x64_generateCode\(\)](#), [x64_generateLoadVar\(\)](#), and [x64_generateLoadVar\(\)](#).

11.76.4.121 getObjectByName()

```
Program * mxvm::Program::getObjectByName (
    const std::string & name)
```

Look up a child object program by name.

Parameters

name	Object identifier
----------------------	-------------------

Returns

Pointer to the [Program](#), or nullptr if not found

Definition at line 285 of file [icode.cpp](#).

References [Program\(\)](#), [name](#), and [objects](#).

11.76.4.122 getPlatformSymbolName()

```
std::string mxvm::Program::getPlatformSymbolName (
    const std::string & name)
```

Mangle a symbol name with the platform prefix (e.g. _name for Darwin).

Parameters

name	Original symbol name
----------------------	----------------------

Returns

Platform-adjusted symbol name

Definition at line 14 of file [icode_gen.cpp](#).

References [mxvm::DARWIN](#), [name](#), and [platform](#).

Referenced by [gen_alloc\(\)](#), [gen_call\(\)](#), [gen_done\(\)](#), [gen_exit\(\)](#), [gen_free\(\)](#), [gen_getline\(\)](#), [gen_mov\(\)](#), [gen_print\(\)](#), [gen_realloc\(\)](#), [gen_to_float\(\)](#), [gen_to_int\(\)](#), [generateCode\(\)](#), [generateFunctionCall\(\)](#), [generateLoadVar\(\)](#), and [generateLoadVar\(\)](#).

11.76.4.123 getRegisterByIndex()

```
std::string mxvm::Program::getRegisterByIndex (
    int index,
    VarType type)
```

Get the register name for a given index and variable type.

Parameters

<i>index</i>	Register index
<i>type</i>	Variable type (determines integer vs. XMM register)

Returns

Register name string

Definition at line 1634 of file `icode_gen.cpp`.

References [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [xmm_offset](#).

Referenced by [generateLoadVar\(\)](#).

11.76.4.124 `getVariable()`

```
Variable & mxvm::Program::getVariable (
    const std::string & name)
```

Look up a variable by name, searching local scope then global.

Parameters

<i>name</i>	Variable identifier
-------------	-------------------------------------

Returns

Reference to the [Variable](#)

Exceptions

mx::Exception	if the variable is not found
-------------------------------	------------------------------

Definition at line 681 of file `icode_exec.cpp`.

References [mxvm::Base::base](#), [name](#), [mxvm::Base::object_map](#), [pos\(\)](#), and [mxvm::Base::vars](#).

Referenced by [exec_add\(\)](#), [exec_alloc\(\)](#), [exec_and\(\)](#), [exec_cmp\(\)](#), [exec_div\(\)](#), [exec_exit\(\)](#), [exec_fcmp\(\)](#), [exec_free\(\)](#), [exec_getline\(\)](#), [exec_invoke\(\)](#), [exec_lea\(\)](#), [exec_load\(\)](#), [exec_mod\(\)](#), [exec_mov\(\)](#), [exec_mul\(\)](#), [exec_neg\(\)](#), [exec_not\(\)](#), [exec_or\(\)](#), [exec_pop\(\)](#), [exec_print\(\)](#), [exec_push\(\)](#), [exec_realloc\(\)](#), [exec_return\(\)](#), [exec_stack_load\(\)](#), [exec_stack_store\(\)](#), [exec_stack_sub\(\)](#), [exec_store\(\)](#), [exec_string_print\(\)](#), [exec_sub\(\)](#), [exec_to_float\(\)](#), [exec_to_int\(\)](#), [exec_xor\(\)](#), [gen_alloc\(\)](#), [gen_arth\(\)](#), [gen_bitop\(\)](#), [gen_call\(\)](#), [gen_cmp\(\)](#), [gen_div\(\)](#), [gen_free\(\)](#), [gen_getline\(\)](#), [gen_lea\(\)](#), [gen_load\(\)](#), [gen_mod\(\)](#), [gen_mov\(\)](#), [gen_neg\(\)](#), [gen_not\(\)](#), [gen_pop\(\)](#), [gen_print\(\)](#), [gen_push\(\)](#), [gen_realloc\(\)](#), [gen_ret\(\)](#), [gen_return\(\)](#), [gen_stack_load\(\)](#), [gen_stack_store\(\)](#), [gen_store\(\)](#), [gen_to_float\(\)](#), [gen_to_int\(\)](#), [generateCode\(\)](#), [generateLoadVar\(\)](#), [generateLoadVar\(\)](#), [getMangledName\(\)](#), [getStringFromVar\(\)](#), [mxvm_io_fclose\(\)](#), [mxvm_io_eof\(\)](#), [mxvm_io_fopen\(\)](#), [mxvm_io_fprintf\(\)](#), [mxvm_io_fputs\(\)](#), [mxvm_io_fread\(\)](#), [mxvm_io_fseek\(\)](#), [mxvm_io_fsize\(\)](#), [mxvm_io_fwrite\(\)](#), [mxvm_io_mxvm_fgets\(\)](#), [mxvm_io_rand_number\(\)](#), [mxvm_sdl_blit_surface\(\)](#), [mxvm_sdl_clear\(\)](#), [mxvm_sdl_create_render_target\(\)](#), [mxvm_sdl_create_renderer\(\)](#), [mxvm_sdl_create_rgb_surface\(\)](#), [mxvm_sdl_create_texture\(\)](#), [mxvm_sdl_create_window\(\)](#), [mxvm_sdl_delay\(\)](#), [mxvm_sdl_destroy_render_target\(\)](#), [mxvm_sdl_destroy_renderer\(\)](#), [mxvm_sdl_destroy_texture\(\)](#), [mxvm_sdl_destroy_window\(\)](#), [mxvm_sdl_draw_line\(\)](#), [mxvm_sdl_draw_point\(\)](#), [mxvm_sdl_draw_rect\(\)](#), [mxvm_sdl_draw_text\(\)](#), [mxvm_sdl_fill_rect\(\)](#), [mxvm_sdl_free_surface\(\)](#), [mxvm_sdl_free_wav\(\)](#), [mxvm_sdl_get_keyboard_state\(\)](#), [mxvm_sdl_get_mouse_state\(\)](#), [mxvm_sdl_get_relative_mouse_state\(\)](#), [mxvm_sdl_get_renderer_output_size\(\)](#), [mxvm_sdl_get_window_size\(\)](#), [mxvm_sdl_is_key_pressed\(\)](#), [mxvm_sdl_load_font\(\)](#), [mxvm_sdl_load_texture\(\)](#), [mxvm_sdl_load_texture_color_key\(\)](#), [mxvm_sdl_load_texture_color_key_rgb\(\)](#), [mxvm_sdl_load_wav\(\)](#), [mxvm_sdl_lock_texture\(\)](#), [mxvm_sdl_open_audio\(\)](#), [mxvm_sdl_pause_audio\(\)](#), [mxvm_sdl_present\(\)](#), [mxvm_sdl_present_scaled\(\)](#), [mxvm_sdl_present_stretched\(\)](#), [mxvm_sdl_queue_audio\(\)](#), [mxvm_sdl_render_texture\(\)](#), [mxvm_sdl_set_clipboard_text\(\)](#), [mxvm_sdl_set_draw_color\(\)](#), [mxvm_sdl_set_render_target\(\)](#), [mxvm_sdl_set_window_fullscreen\(\)](#), [mxvm_sdl_set_window_icon\(\)](#), [mxvm_sdl_set_window_position\(\)](#), [mxvm_sdl_set_window_title\(\)](#), [mxvm_sdl_show_cursor\(\)](#), [mxvm_sdl_unlock_texture\(\)](#), [mxvm_sdl_update_texture\(\)](#),

mxvm_std_abs(), mxvm_std_acos(), mxvm_std_argv(), mxvm_std_asin(), mxvm_std_atan(), mxvm_std_atan2(), mxvm_std_atof(), mxvm_std_atoi(), mxvm_std_calloc(), mxvm_std_ceil(), mxvm_std_cos(), mxvm_std_cosh(), mxvm_std_exit(), mxvm_std_exp(), mxvm_std_exp2(), mxvm_std_fabs(), mxvm_std_float_to_int(), mxvm_std_floor(), mxvm_std_fmod(), mxvm_std_hypot(), mxvm_std_int_to_float(), mxvm_std_isalpha(), mxvm_std_isdigit(), mxvm_std_isspace(), mxvm_std_log(), mxvm_std_log10(), mxvm_std_log2(), mxvm_std_malloc(), mxvm_std_memcmp(), mxvm_std_memcpy(), mxvm_std_memmove(), mxvm_std_memset(), mxvm_std_pow(), mxvm_std_release(), mxvm_std_round(), mxvm_std_set_program_args(), mxvm_std_sin(), mxvm_std_sinh(), mxvm_std_sqrt(), mxvm_std_srand(), mxvm_std_system(), mxvm_std_tan(), mxvm_std_tanh(), mxvm_std_tolower(), mxvm_std_toupper(), mxvm_std_trunc(), mxvm_string_copy(), mxvm_string_delete(), mxvm_string_insert(), mxvm_string_inttostr(), mxvm_string_snprintf(), mxvm_string_strat(), mxvm_string_strcmp(), mxvm_string_strfind(), mxvm_string_strlen(), mxvm_string_strncat(), mxvm_string_strncpy(), mxvm_string_substr(), sysv_analyzeRegAlloc(), variableFromOperand(), x64_analyzeRegAlloc(), x64_gen_alloc(), x64_gen_arth(), x64_gen_bitop(), x64_gen_call(), x64_gen_cmp(), x64_gen_div(), x64_gen_free(), x64_gen_getline(), x64_gen_lea(), x64_gen_load(), x64_gen_mod(), x64_gen_mov(), x64_gen_neg(), x64_gen_not(), x64_gen_pop(), x64_gen_push(), x64_gen_realloc(), x64_gen_ret(), x64_gen_return(), x64_gen_store(), x64_generateCode(), x64_generateFunctionCall(), x64_generateLoadVar(), and x64_generateLoadVar().

11.76.4.125 isConstant()

```
bool mxvm::Program::isConstant (
    const std::string & value)
```

Check whether a string represents a compile-time constant (number or quoted string).

Parameters

<i>value</i>	String to test
--------------	----------------

Returns

true if it is a constant

11.76.4.126 isFunctionValid()

```
bool mxvm::Program::isFunctionValid (
    const std::string & label)
```

Check whether a label exists and is marked as a function entry.

Parameters

<i>label</i>	Label name to check
--------------	---------------------

Returns

true if the label is a valid function

Definition at line 175 of file [icode.cpp](#).

References [Program\(\)](#), [mxvm::Base::labels](#), and [mxvm::Base::object_map](#).

Referenced by [gen_call\(\)](#), and [x64_gen_call\(\)](#).

11.76.4.127 isVariable()

```
bool mxvm::Program::isVariable (
    const std::string & name)
```

Check whether a variable exists in local or global scope.

Parameters

<i>name</i>	Variable identifier
-------------	---------------------

Returns

true if the variable is declared

Definition at line 724 of file `icode_exec.cpp`.

References `mxvm::Base::base`, `name`, `mxvm::Base::object_map`, and `mxvm::Base::vars`.

Referenced by `exec_add()`, `exec_alloc()`, `exec_and()`, `exec_cmp()`, `exec_div()`, `exec_exit()`, `exec_fcmp()`, `exec_free()`, `exec_getline()`, `exec_invoke()`, `exec_lea()`, `exec_load()`, `exec_mod()`, `exec_mov()`, `exec_mul()`, `exec_neg()`, `exec_not()`, `exec_or()`, `exec_pop()`, `exec_print()`, `exec_push()`, `exec_realloc()`, `exec_return()`, `exec_stack_load()`, `exec_stack_store()`, `exec_stack_sub()`, `exec_store()`, `exec_string_print()`, `exec_sub()`, `exec_to_float()`, `exec_to_int()`, `exec_xor()`, `gen_alloc()`, `gen_arth()`, `gen_bitop()`, `gen_call()`, `gen_cmp()`, `gen_div()`, `gen_free()`, `gen_getline()`, `gen_lea()`, `gen_load()`, `gen_mod()`, `gen_mov()`, `gen_neg()`, `gen_not()`, `gen_pop()`, `gen_print()`, `gen_push()`, `gen_realloc()`, `gen_ret()`, `gen_return()`, `gen_stack_load()`, `gen_stack_store()`, `gen_stack_sub()`, `gen_store()`, `gen_to_float()`, `gen_to_int()`, `generateCode()`, `generateLoadVar()`, `generateLoadVar()`, `getMangledName()`, `getStringFromVar()`, `mxvm_io_fclose()`, `mxvm_io_feof()`, `mxvm_io_fopen()`, `mxvm_io_fprintf()`, `mxvm_io_fputs()`, `mxvm_io_fread()`, `mxvm_io_fseek()`, `mxvm_io_fsize()`, `mxvm_io_fwrite()`, `mxvm_io_mxvm_fgets()`, `mxvm_io_rand_number()`, `mxvm_sdl_blit_surface()`, `mxvm_sdl_clear()`, `mxvm_sdl_create_render_target()`, `mxvm_sdl_create_renderer()`, `mxvm_sdl_create_rgb_surface()`, `mxvm_sdl_create_texture()`, `mxvm_sdl_create_window()`, `mxvm_sdl_delay()`, `mxvm_sdl_destroy_render_target()`, `mxvm_sdl_destroy_renderer()`, `mxvm_sdl_destroy_texture()`, `mxvm_sdl_destroy_window()`, `mxvm_sdl_draw_line()`, `mxvm_sdl_draw_point()`, `mxvm_sdl_draw_rect()`, `mxvm_sdl_draw_text()`, `mxvm_sdl_fill_rect()`, `mxvm_sdl_free_surface()`, `mxvm_sdl_free_wav()`, `mxvm_sdl_get_keyboard_state()`, `mxvm_sdl_get_mouse_state()`, `mxvm_sdl_get_relative_mouse_state()`, `mxvm_sdl_get_renderer_output_size()`, `mxvm_sdl_get_window_size()`, `mxvm_sdl_is_key_pressed()`, `mxvm_sdl_load_font()`, `mxvm_sdl_load_texture()`, `mxvm_sdl_load_texture_color_key()`, `mxvm_sdl_load_texture_color_key_rgb()`, `mxvm_sdl_load_wav()`, `mxvm_sdl_lock_texture()`, `mxvm_sdl_open_audio()`, `mxvm_sdl_pause_audio()`, `mxvm_sdl_present()`, `mxvm_sdl_present_scaled()`, `mxvm_sdl_present_stretched()`, `mxvm_sdl_queue_audio()`, `mxvm_sdl_render_texture()`, `mxvm_sdl_set_clipboard_text()`, `mxvm_sdl_set_draw_color()`, `mxvm_sdl_set_render_target()`, `mxvm_sdl_set_window_fullscreen()`, `mxvm_sdl_set_window_icon()`, `mxvm_sdl_set_window_position()`, `mxvm_sdl_set_window_title()`, `mxvm_sdl_show_cursor()`, `mxvm_sdl_unlock_texture()`, `mxvm_sdl_update_texture()`, `mxvm_std_abs()`, `mxvm_std_acos()`, `mxvm_std_argv()`, `mxvm_std_asin()`, `mxvm_std_atan()`, `mxvm_std_atan2()`, `mxvm_std_atof()`, `mxvm_std_atoi()`, `mxvm_std_calloc()`, `mxvm_std_ceil()`, `mxvm_std_cos()`, `mxvm_std_cosh()`, `mxvm_std_exit()`, `mxvm_std_exp()`, `mxvm_std_exp2()`, `mxvm_std_fabs()`, `mxvm_std_float_to_int()`, `mxvm_std_floor()`, `mxvm_std_fmod()`, `mxvm_std_hypot()`, `mxvm_std_int_to_float()`, `mxvm_std_isalpha()`, `mxvm_std_isdigit()`, `mxvm_std_isspace()`, `mxvm_std_log()`, `mxvm_std_log10()`, `mxvm_std_log2()`, `mxvm_std_malloc()`, `mxvm_std_memcmp()`, `mxvm_std_memcpy()`, `mxvm_std_memmove()`, `mxvm_std_memset()`, `mxvm_std_pow()`, `mxvm_std_release()`, `mxvm_std_round()`, `mxvm_std_set_program_args()`, `mxvm_std_sin()`, `mxvm_std_sinh()`, `mxvm_std_sqrt()`, `mxvm_std_srand()`, `mxvm_std_system()`, `mxvm_std_tan()`, `mxvm_std_tanh()`, `mxvm_std_tolower()`, `mxvm_std_toupper()`, `mxvm_std_trunc()`, `mxvm_string_snprintf()`, `mxvm_string_strat()`, `mxvm_string_strcmp()`, `mxvm_string_strfind()`, `mxvm_string_strlen()`, `mxvm_string_strncat()`, `mxvm_string_strncpy()`, `mxvm_string_substr()`, `sysv_analyzeRegAlloc()`, `variableFromOperand()`, `x64_analyzeRegAlloc()`, `x64_gen_alloc()`, `x64_gen_arth()`, `x64_gen_bitop()`, `x64_gen_call()`, `x64_gen_cmp()`, `x64_gen_div()`, `x64_gen_free()`, `x64_gen_getline()`, `x64_gen_lea()`, `x64_gen_load()`, `x64_gen_mod()`, `x64_gen_mov()`, `x64_gen_neg()`, `x64_gen_not()`, `x64_gen_pop()`, `x64_gen_push()`, `x64_gen_realloc()`, `x64_gen_ret()`, `x64_gen_return()`, `x64_gen_stack_load()`, `x64_gen_stack_store()`, `x64_gen_stack_sub()`, `x64_gen_store()`, `x64_gen_to_float()`, `x64_generateCode()`, `x64_generateFunctionCall()`, `x64_generateLoadVar()`, and `x64_generateLoadVar()`.

11.76.4.128 memoryDump()

```
void mxvm::Program::memoryDump (
    std::ostream & out)
```

Dump all variable values to a stream for debugging.

Parameters

<i>out</i>	Destination stream
------------	--------------------

Definition at line 316 of file `icode.cpp`.

References [Program\(\)](#), [escapeNewLines\(\)](#), [mxvm::Base::labels](#), [name](#), [objects](#), [mxvm::VAR_ARRAY](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_LABEL](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Base::vars](#).

11.76.4.129 mulVariables()

```
void mxvm::Program::mulVariables (
    Variable & dest,
    Variable & src1,
    Variable & src2)
```

Multiply two variables and store the result in dest.

Definition at line 827 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Variable_Value::int_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec_mul\(\)](#).

11.76.4.130 operator=()

```
Program & mxvm::Program::operator= (
    Program && other) [inline], [noexcept]
```

Move-assignment operator.

Definition at line 250 of file [icode.hpp](#).

References [Program\(\)](#), [args](#), [carry_flag](#), [exitCode](#), [filename](#), [greater_flag](#), [less_flag](#), [main_function](#), [name](#), [object_external](#), [objects](#), [mxvm::Base::operator=\(\)](#), [parent](#), [pc](#), [platform](#), [result](#), [running](#), [stack](#), [mxvm::Base::vars](#), [xmm_offset](#), and [zero_flag](#).

11.76.4.131 optimize_core()

```
std::string mxvm::Program::optimize_core (
    const std::string & code)
```

Platform-independent peephole optimizations.

Parameters

<i>code</i>	Assembly text
-------------	---------------

Returns

Optimized assembly text

References [args](#), [name](#), and [platform](#).

11.76.4.132 optimize_darwin()

```
std::string mxvm::Program::optimize_darwin (
    const std::string & code)
```

Darwin-specific assembly optimizations.

Parameters

<i>code</i>	Assembly text
-------------	---------------

Returns

Optimized assembly text

11.76.4.133 post()

```
void mxvm::Program::post (
    std::ostream & out)
```

Generate native data section (variable declarations) into the output stream.

Parameters

<i>out</i>	Destination stream
------------	--------------------

Definition at line 2093 of file [icode_exec.cpp](#).

References [stack](#).

11.76.4.134 print()

```
void mxvm::Program::print (
    std::ostream & out)
```

Print all instructions and data to an output stream.

Parameters

<i>out</i>	Destination stream
------------	--------------------

Definition at line 2013 of file [icode_exec.cpp](#).

References [mxvm::Base::inc](#), and [IncType](#).

11.76.4.135 printFormatted()

```
std::string mxvm::Program::printFormatted (
    const std::string & format,
    const std::vector< Variable * > & args,
    bool output = true)
```

Format and optionally print a printf-style format string with variable arguments.

Parameters

<i>format</i>	Format string with % specifiers
<i>args</i>	Variables corresponding to format specifiers
<i>output</i>	true to print, false to only return the string

Returns

The formatted string

Definition at line 1501 of file [icode_exec.cpp](#).

References [args](#), [mxvm::Variable_Value::float_value](#), [mxvm::Variable_Value::int_value](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

Referenced by [exec_print\(\)](#), and [mxvm_io_fprintf\(\)](#).

11.76.4.136 setArgs()

```
void mxvm::Program::setArgs (
    const std::vector< std::string > & argv)
```

Set the command-line arguments available to the program.

Parameters

<i>argv</i>	Argument vector
-------------	-----------------

Definition at line 128 of file [icode.cpp](#).

References [args](#), and [argv\(\)](#).

11.76.4.137 setObject()

```
void mxvm::Program::setObject (
    bool obj)
```

Mark this program as an object (no main entry point).

Parameters

<i>obj</i>	true for object mode, false for stand-alone program
------------	---

Definition at line 132 of file [icode.cpp](#).

11.76.4.138 setVariableFromConstant()

```
void mxvm::Program::setVariableFromConstant (
    Variable & var,
    const std::string & value)
```

Set a variable's value from a constant string (integer, float, or string literal).

Parameters

<i>var</i>	Variable to modify
<i>value</i>	Constant string to parse

Definition at line 1991 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Variable_Value::int_value](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).
Referenced by [exec_mov\(\)](#).

11.76.4.139 setVariableFromString()

```
void mxvm::Program::setVariableFromString (
    Variable & var,
    const std::string & value)
```

Set a variable's value from a string representation, coercing types.

Parameters

<i>var</i>	Variable to modify
<i>value</i>	String to parse

Definition at line 771 of file [icode_exec.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Variable_Value::int_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

11.76.4.140 stop()

```
void mxvm::Program::stop ()
```

Halt the interpreter loop.

Definition at line 23 of file [icode_exec.cpp](#).

References [running](#).

Referenced by [exec_done\(\)](#), and [exec_exit\(\)](#).

11.76.4.141 subVariables()

```
void mxvm::Program::subVariables (
    Variable & dest,
    Variable & src1,
    Variable & src2)
```

Subtract src2 from src1 and store the result in dest.

Definition at line 807 of file [icode_exec.cpp](#).

References [mxvm::Base::base](#), [mxvm::Variable_Value::float_value](#), [mxvm::Variable_Value::int_value](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [exec_sub\(\)](#).

11.76.4.142 sysv_analyzeRegAlloc()

```
void mxvm::Program::sysv_analyzeRegAlloc (
    bool uses_std_module)
```

Analyze variable usage and assign System V callee-saved registers.

Parameters

<i>uses_std_module</i>	true if the program uses the std module
------------------------	---

Definition at line 24 of file [icode_gen.cpp](#).

References [getVariable\(\)](#), [mxvm::Base::inc](#), [mxvm::Variable::is_global](#), [isVariable\(\)](#), [sysv_reg_save_order](#), [sysv_reg_vars](#), [mxvm::Variable::type](#), and [mxvm::VAR_INTEGER](#).

Referenced by [generateCode\(\)](#).

11.76.4.143 sysv_emitFlushRegs()

```
void mxvm::Program::sysv_emitFlushRegs (
    std::ostream & out)
```

Write all register-allocated variables back to memory.

Definition at line 67 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), and [sysv_reg_vars](#).

Referenced by [gen_call\(\)](#), [gen_done\(\)](#), [gen_load\(\)](#), [gen_realloc\(\)](#), [gen_ret\(\)](#), and [gen_store\(\)](#).

11.76.4.144 sysv_emitLoadVar()

```
void mxvm::Program::sysv_emitLoadVar (
    std::ostream & out,
    const std::string & dstReg,
    const Operand & src)
```

Load a variable into a destination register.

Parameters

<i>out</i>	Assembly output stream
<i>dstReg</i>	Destination register name
<i>src</i>	Source operand

Definition at line 110 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [mxvm::Operand::op](#), and [sysv_reg_vars](#).

Referenced by [gen_alloc\(\)](#), [gen_call\(\)](#), [gen_load\(\)](#), [gen_mov\(\)](#), [gen_push\(\)](#), [gen_realloc\(\)](#), [gen_ret\(\)](#), [gen_stack_store\(\)](#), [gen_stack_sub\(\)](#), and [gen_store\(\)](#).

11.76.4.145 sysv_emitReloadRegs()

```
void mxvm::Program::sysv_emitReloadRegs (
    std::ostream & out)
```

Reload register-allocated variables from memory.

Definition at line 73 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), and [sysv_reg_vars](#).

Referenced by [gen_call\(\)](#), [gen_realloc\(\)](#), [gen_store\(\)](#), and [generateCode\(\)](#).

11.76.4.146 sysv_emitRestoreRegs()

```
void mxvm::Program::sysv_emitRestoreRegs (
    std::ostream & out)
```

Restore callee-saved registers at function epilogue.

Definition at line 85 of file [icode_gen.cpp](#).

References [sysv_reg_save_order](#).

Referenced by [gen_done\(\)](#), and [gen_ret\(\)](#).

11.76.4.147 sysv_emitSaveRegs()

```
void mxvm::Program::sysv_emitSaveRegs (
    std::ostream & out)
```

Save callee-saved registers at function prologue.

Definition at line 79 of file [icode_gen.cpp](#).

References [sysv_reg_save_order](#).

Referenced by [generateCode\(\)](#).

11.76.4.148 sysv_emitStoreVar()

```
void mxvm::Program::sysv_emitStoreVar (
    std::ostream & out,
    const std::string & srcReg,
    const Operand & dest)
```

Store a register value into a variable (register-allocated or memory).

Parameters

<i>out</i>	Assembly output stream
<i>srcReg</i>	Source register name
<i>dest</i>	Destination operand

Definition at line 91 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [mxvm::Operand::op](#), and [sysv_reg_vars](#).

Referenced by [gen_arth\(\)](#), [gen_bitop\(\)](#), [gen_div\(\)](#), [gen_load\(\)](#), [gen_mod\(\)](#), [gen_mov\(\)](#), [gen_neg\(\)](#), [gen_not\(\)](#), [gen_pop\(\)](#), [gen_return\(\)](#), [gen_stack_load\(\)](#), [gen_to_int\(\)](#), and [generateCode\(\)](#).

11.76.4.149 sysv_emitStoreVarImm()

```
void mxvm::Program::sysv_emitStoreVarImm (
    std::ostream & out,
    const std::string & imm,
    const Operand & dest)
```

Store an immediate value into a variable.

Parameters

<i>out</i>	Assembly output stream
<i>imm</i>	Immediate value string
<i>dest</i>	Destination operand

Definition at line 101 of file [icode_gen.cpp](#).

References [getMangledName\(\)](#), [mxvm::Operand::op](#), and [sysv_reg_vars](#).

Referenced by [gen_mov\(\)](#).

11.76.4.150 validateNames()

```
bool mxvm::Program::validateNames (
    Validator & v)
```

Validate all variable and label names against the [Validator](#).

Parameters

<i>v</i>	Validator reference
----------	-------------------------------------

Returns

true if all names pass validation

Definition at line 754 of file [icode_exec.cpp](#).

References [objects](#), [mxvm::Validator::var_names](#), and [mxvm::Base::vars](#).

11.76.4.151 variableFromOperand()

```
Variable mxvm::Program::variableFromOperand (
    const Operand & op)
```

Resolve an [Operand](#) to its corresponding [Variable](#).

Parameters

<i>op</i>	Operand to resolve
-----------	------------------------------------

Returns

The [Variable](#) (looked up or created as a temporary for constants)

Definition at line 2123 of file [icode_exec.cpp](#).

References [createTempVariable\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::OP_CONSTANT](#), [mxvm::Operand::type](#), and [mxvm::VAR_INTEGER](#).

Referenced by [mxvm_io_fread\(\)](#), [mxvm_io_fseek\(\)](#), and [mxvm_io_fwrite\(\)](#).

11.76.4.152 x64_analyzeRegAlloc()

```
void mxvm::Program::x64_analyzeRegAlloc (
    bool uses_std_module)
```

Analyze variable usage and assign Win64 callee-saved registers.

Parameters

<i>uses_std_module</i>	true if the program uses the std module (affects register choices)
------------------------	--

Definition at line 39 of file [icode_gen_x64.cpp](#).

References [getVariable\(\)](#), [mxvm::Base::inc](#), [mxvm::Variable::is_global](#), [mxvm::is_stdio_name\(\)](#), [isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [x64_reg_save_order](#), and [x64_reg_vars](#).

Referenced by [x64_generateCode\(\)](#).

11.76.4.153 x64_direct_stack_adjust()

```
void mxvm::Program::x64_direct_stack_adjust (
    std::ostream & out,
    int64_t bytes)
```

Emit a direct stack pointer adjustment.

Parameters

<i>out</i>	Assembly output stream
<i>bytes</i>	Positive = allocate, negative = deallocate

11.76.4.154 x64_emit_iob_func()

```
void mxvm::Program::x64_emit_iob_func (
    std::ostream & out,
    int index,
    const std::string & dstReg)
```

Emit an integer-or-byte function argument load for Win64 ABI.

Parameters

<i>out</i>	Assembly output stream
<i>index</i>	Argument index (determines register)
<i>dstReg</i>	Destination register name

Definition at line 151 of file [icode_gen_x64.cpp](#).

References [x64_release_call_area\(\)](#), and [x64_reserve_call_area\(\)](#).

Referenced by [x64_generateLoadVar\(\)](#), and [x64_generateLoadVar\(\)](#).

11.76.4.155 x64_emitFlushRegs()

```
void mxvm::Program::x64_emitFlushRegs (
    std::ostream & out)
```

Write all register-allocated variables back to memory.

Definition at line 80 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), and [x64_reg_vars](#).

Referenced by [x64_gen_call\(\)](#), [x64_gen_done\(\)](#), [x64_gen_load\(\)](#), [x64_gen_realloc\(\)](#), [x64_gen_ret\(\)](#), and [x64_gen_store\(\)](#).

11.76.4.156 x64_emitLoadVar()

```
void mxvm::Program::x64_emitLoadVar (
    std::ostream & out,
    const std::string & dstReg,
    const Operand & src)
```

Load a variable into a destination register.

Parameters

<i>out</i>	Assembly output stream
------------	------------------------

<i>dstReg</i>	Destination register name
<i>src</i>	Source operand

Definition at line 125 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), [mxvm::Operand::op](#), and [x64_reg_vars](#).

Referenced by [x64_gen_alloc\(\)](#), [x64_gen_call\(\)](#), [x64_gen_load\(\)](#), [x64_gen_mov\(\)](#), [x64_gen_push\(\)](#), [x64_gen_realloc\(\)](#), [x64_gen_ret\(\)](#), [x64_gen_stack_store\(\)](#), [x64_gen_stack_sub\(\)](#), and [x64_gen_store\(\)](#).

11.76.4.157 x64_emitReloadRegs()

```
void mxvm::Program::x64_emitReloadRegs (
    std::ostream & out)
```

Reload register-allocated variables from memory.

Definition at line 86 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), and [x64_reg_vars](#).

Referenced by [x64_gen_call\(\)](#), [x64_gen_realloc\(\)](#), [x64_gen_store\(\)](#), and [x64_generateCode\(\)](#).

11.76.4.158 x64_emitRestoreRegs()

```
void mxvm::Program::x64_emitRestoreRegs (
    std::ostream & out)
```

Restore callee-saved registers at function epilogue.

Definition at line 99 of file [icode_gen_x64.cpp](#).

References [x64_reg_save_order](#), and [mxvm::x64_sp_mod16](#).

Referenced by [x64_gen_done\(\)](#), and [x64_gen_ret\(\)](#).

11.76.4.159 x64_emitSaveRegs()

```
void mxvm::Program::x64_emitSaveRegs (
    std::ostream & out)
```

Save callee-saved registers at function prologue.

Definition at line 92 of file [icode_gen_x64.cpp](#).

References [x64_reg_save_order](#), and [mxvm::x64_sp_mod16](#).

Referenced by [x64_generateCode\(\)](#).

11.76.4.160 x64_emitStoreVar()

```
void mxvm::Program::x64_emitStoreVar (
    std::ostream & out,
    const std::string & srcReg,
    const Operand & dest)
```

Store a register value into a variable (register-allocated or memory).

Parameters

<i>out</i>	Assembly output stream
<i>srcReg</i>	Source register name
<i>dest</i>	Destination operand

Definition at line 106 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), [mxvm::Operand::op](#), and [x64_reg_vars](#).

Referenced by [x64_gen_arth\(\)](#), [x64_gen_bitop\(\)](#), [x64_gen_div\(\)](#), [x64_gen_load\(\)](#), [x64_gen_mod\(\)](#), [x64_gen_mov\(\)](#), [x64_gen_neg\(\)](#), [x64_gen_not\(\)](#), [x64_gen_pop\(\)](#), [x64_gen_return\(\)](#), [x64_gen_stack_load\(\)](#), [x64_gen_to_int\(\)](#), and [x64_generateCode\(\)](#).

11.76.4.161 x64_emitStoreVarImm()

```
void mxvm::Program::x64_emitStoreVarImm (
    std::ostream & out,
    const std::string & imm,
    const Operand & dest)
```

Store an immediate value into a variable.

Parameters

<i>out</i>	Assembly output stream
<i>imm</i>	Immediate value string
<i>dest</i>	Destination operand

Definition at line 116 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), [mxvm::Operand::op](#), and [x64_reg_vars](#).

Referenced by [x64_gen_mov\(\)](#).

11.76.4.162 x64_gen_alloc()

```
void mxvm::Program::x64_gen_alloc (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 751 of file [icode_gen_x64.cpp](#).

References [mxvm::error_label_count](#), [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), [x64_emitLoadVar\(\)](#), [x64_release_call_area\(\)](#), and [x64_reserve_call_area\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.163 x64_gen_arth()

```
void mxvm::Program::x64_gen_arth (
    std::ostream & out,
    std::string arth,
    const Instruction & i)
```

Definition at line 1611 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::OP_CONSTANT](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [x64_emitStoreVar\(\)](#), [x64_generateLoadVar\(\)](#), and [x64_reg_vars](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.164 x64_gen_bitop()

```
void mxvm::Program::x64_gen_bitop (
    std::ostream & out,
    const std::string & opc,
    const Instruction & i)
```

Definition at line 1380 of file [icode_gen_x64.cpp](#).

References [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [x64_emitStoreVar\(\)](#), and [x64_generateLoadVar\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.165 x64_gen_call()

```
void mxvm::Program::x64_gen_call (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 732 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), [getVariable\(\)](#), [isFunctionValid\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::VAR_INTEGER](#), [x64_emitFlushRegs\(\)](#), [x64_emitLoadVar\(\)](#), [x64_emitReloadRegs\(\)](#), [x64_release_call_area\(\)](#), and [x64_reserve_call_area\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.166 x64_gen_cmp()

```
void mxvm::Program::x64_gen_cmp (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1562 of file [icode_gen_x64.cpp](#).

References [CMP_FLOAT](#), [CMP_INTEGER](#), [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [last_cmp_type](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::OP_CONSTANT](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [x64_generateLoadVar\(\)](#), and [x64_reg_vars](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.167 x64_gen_div()

```
void mxvm::Program::x64_gen_div (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1235 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [x64_emitStoreVar\(\)](#), and [x64_generateLoadVar\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.168 x64_gen_done()

```
void mxvm::Program::x64_gen_done (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 476 of file [icode_gen_x64.cpp](#).

References [mxvm::Base::external](#), [x64_emitFlushRegs\(\)](#), [x64_emitRestoreRegs\(\)](#), [x64_release_call_area\(\)](#), and [x64_reserve_call_area\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.169 x64_gen_exit()

```
void mxvm::Program::x64_gen_exit (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1663 of file [icode_gen_x64.cpp](#).

References [mxvm::Base::external](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [x64_generateFunctionCall\(\)](#), [x64_release_call_area\(\)](#), and [x64_reserve_call_area\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.170 x64_gen_fcmp()

```
void mxvm::Program::x64_gen_fcmp (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1688 of file [icode_gen_x64.cpp](#).

References [CMP_FLOAT](#), [last_cmp_type](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::VAR_FLOAT](#), and [x64_generateLoadVar\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.171 x64_gen_free()

```
void mxvm::Program::x64_gen_free (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 856 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), [x64_release_call_area\(\)](#), and [x64_reserve_call_area\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.172 x64_gen_getline()

```
void mxvm::Program::x64_gen_getline (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1343 of file [icode_gen_x64.cpp](#).

References [mxvm::Variable_Value::buffer_size](#), [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable::type](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), [x64_release_call_area\(\)](#), and [x64_reserve_call_area\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.173 x64_gen_invoke()

```
void mxvm::Program::x64_gen_invoke (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 717 of file [icode_gen_x64.cpp](#).

References [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Instruction::vop](#), and [x64_generateInvokeCall\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.174 x64_gen_jae()

```
void mxvm::Program::x64_gen_jae (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1699 of file [icode_gen_x64.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.175 x64_gen_jbe()

```
void mxvm::Program::x64_gen_jbe (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1703 of file [icode_gen_x64.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.176 x64_gen_jc()

```
void mxvm::Program::x64_gen_jc (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1707 of file [icode_gen_x64.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.177 x64_gen_jump()

```
void mxvm::Program::x64_gen_jump (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1514 of file [icode_gen_x64.cpp](#).

References [CMP_FLOAT](#), [mxvm::Instruction::instruction](#), [JA](#), [JB](#), [JE](#), [JG](#), [JGE](#), [JL](#), [JLE](#), [JMP](#), [JNE](#), [JNZ](#), [JZ](#), [mxvm::Base::labels](#), [last_cmp_type](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), and [pos\(\)](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.178 x64_gen_jnc()

```
void mxvm::Program::x64_gen_jnc (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1711 of file [icode_gen_x64.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.179 x64_gen_jno()

```
void mxvm::Program::x64_gen_jno (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1727 of file [icode_gen_x64.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.180 x64_gen_jnp()

```
void mxvm::Program::x64_gen_jnp (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1719 of file [icode_gen_x64.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.181 x64_gen_jns()

```
void mxvm::Program::x64_gen_jns (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1735 of file [icode_gen_x64.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.182 x64_gen_jo()

```
void mxvm::Program::x64_gen_jo (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1723 of file [icode_gen_x64.cpp](#).

References [mxvm::Operand::op](#), and [mxvm::Instruction::op1](#).

Referenced by [x64_generateInstruction\(\)](#).

11.76.4.183 x64_gen_jp()

```
void mxvm::Program::x64_gen_jp (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1715 of file `icode_gen_x64.cpp`.

References `mxvm::Operand::op`, and `mxvm::Instruction::op1`.

Referenced by `x64_generateInstruction()`.

11.76.4.184 x64_gen_js()

```
void mxvm::Program::x64_gen_js (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1731 of file `icode_gen_x64.cpp`.

References `mxvm::Operand::op`, and `mxvm::Instruction::op1`.

Referenced by `x64_generateInstruction()`.

11.76.4.185 x64_gen_lea()

```
void mxvm::Program::x64_gen_lea (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 544 of file `icode_gen_x64.cpp`.

References `getMangledName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Variable_Value::owns`, `mxvm::Variable::var_value`, and `x64_reg_vars`.

Referenced by `x64_generateInstruction()`.

11.76.4.186 x64_gen_load()

```
void mxvm::Program::x64_gen_load (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 952 of file `icode_gen_x64.cpp`.

References `mxvm::Variable_Value::buffer_size`, `mxvm::error_label_count`, `getMangledName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `mxvm::Variable::type`, `mxvm::VAR_BYTE`, `mxvm::VAR_EXTERN`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, `mxvm::Variable::var_value`, `mxvm::Instruction::vop`, `x64_emitFlushRegs()`, `x64_emitLoadVar()`, and `x64_emitStoreVar()`.

Referenced by `x64_generateInstruction()`.

11.76.4.187 x64_gen_mod()

```
void mxvm::Program::x64_gen_mod (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1301 of file `icode_gen_x64.cpp`.

References `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `mxvm::Variable::type`, `mxvm::VAR_INTEGER`, `x64_emitStoreVar()`, and `x64_generateLoadVar()`.

Referenced by `x64_generateInstruction()`.

11.76.4.188 x64_gen_mov()

```
void mxvm::Program::x64_gen_mov (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 874 of file `icode_gen_x64.cpp`.

References `getMangledName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Variable_Value::owns`, `mxvm::Variable::type`, `mxvm::VAR_BYTE`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, `mxvm::Variable::var_value`, `x64_emitLoadVar()`, `x64_emitStoreVar()`, `x64_emitStoreVarImm()`, and `x64_generateLoadVar()`.

Referenced by `x64_generateInstruction()`.

11.76.4.189 x64_gen_neg()

```
void mxvm::Program::x64_gen_neg (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 526 of file `icode_gen_x64.cpp`.

References [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [x64_emitStoreVar\(\)](#), and [x64_generateLoadVar\(\)](#).
Referenced by [x64_generateInstruction\(\)](#).

11.76.4.190 x64_gen_not()

```
void mxvm::Program::x64_gen_not (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1408 of file `icode_gen_x64.cpp`.

References [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [x64_emitStoreVar\(\)](#), and [x64_generateLoadVar\(\)](#).
Referenced by [x64_generateInstruction\(\)](#).

11.76.4.191 x64_gen_pop()

```
void mxvm::Program::x64_gen_pop (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1448 of file `icode_gen_x64.cpp`.

References [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Variable::type](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [x64_emitStoreVar\(\)](#), and [mxvm::x64_sp_mod16](#).
Referenced by [x64_generateInstruction\(\)](#).

11.76.4.192 x64_gen_print()

```
void mxvm::Program::x64_gen_print (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1500 of file `icode_gen_x64.cpp`.

References [args](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), [mxvm::Instruction::vop](#), [x64_generateFunctionCall\(\)](#), and [xmm_offset](#).
Referenced by [x64_generateInstruction\(\)](#).

11.76.4.193 x64_gen_push()

```
void mxvm::Program::x64_gen_push (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1425 of file `icode_gen_x64.cpp`.

References [getMangledName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::OP_CONSTANT](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [x64_emitLoadVar\(\)](#), and [mxvm::x64_sp_mod16](#).
Referenced by [x64_generateInstruction\(\)](#).

11.76.4.194 x64_gen_realloc()

```
void mxvm::Program::x64_gen_realloc (
    std::ostream & out,
    const Instruction & i)
```

Generate Win64 x86-64 assembly for the REALLOC instruction (dynamic array resize).

Generate Win64 x86-64 native code for REALLOC.

Uses the Win64 calling convention (rcx = ptr, rdx = size) for the C realloc() call. Shadow space is reserved. Registers are flushed/reloaded around the call.

Definition at line 804 of file `icode_gen_x64.cpp`.

References `mxvm::error_label_count`, `getMangledName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `mxvm::Variable_Value::owns`, `mxvm::Variable::type`, `mxvm::VAR_POINTER`, `mxvm::Variable::var_value`, `x64_emitFlushRegs()`, `x64_emitLoadVar()`, `x64_emitReloadRegs()`, `x64_release_call_area()`, and `x64_reserve_call_area()`.

Referenced by `x64_generateInstruction()`.

11.76.4.195 x64_gen_ret()

```
void mxvm::Program::x64_gen_ret (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 502 of file `icode_gen_x64.cpp`.

References `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::VAR_INTEGER`, `x64_emitFlushRegs()`, `x64_emitLoadVar()`, and `x64_emitRestoreRegs()`.

Referenced by `x64_generateInstruction()`.

11.76.4.196 x64_gen_return()

```
void mxvm::Program::x64_gen_return (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 514 of file `icode_gen_x64.cpp`.

References `getVariable()`, `isVariable()`, `last_call_returns_owned_ptr`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Variable_Value::owns`, `mxvm::Variable::var_value`, and `x64_emitStoreVar()`.

Referenced by `x64_generateInstruction()`.

11.76.4.197 x64_gen_stack_load()

```
void mxvm::Program::x64_gen_stack_load (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1480 of file `icode_gen_x64.cpp`.

References `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::VAR_INTEGER`, `x64_emitStoreVar()`, and `x64_generateLoadVar()`.

Referenced by `x64_generateInstruction()`.

11.76.4.198 x64_gen_stack_store()

```
void mxvm::Program::x64_gen_stack_store (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1490 of file `icode_gen_x64.cpp`.

References `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::VAR_INTEGER`, `x64_emitLoadVar()`, and `x64_generateLoadVar()`.

Referenced by `x64_generateInstruction()`.

11.76.4.199 x64_gen_stack_sub()

```
void mxvm::Program::x64_gen_stack_sub (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1461 of file `icode_gen_x64.cpp`.

References `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `x64_emitLoadVar()`, and `mxvm::x64_sp_mod16`.

Referenced by `x64_generateInstruction()`.

11.76.4.200 x64_gen_store()

```
void mxvm::Program::x64_gen_store (
```

```
std::ostream & out,
const Instruction & i)
```

Definition at line 1087 of file `icode_gen_x64.cpp`.

References `mxvm::Variable_Value::buffer_size`, `mxvm::error_label_count`, `getMangledName()`, `getVariable()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::Instruction::op3`, `mxvm::Variable::type`, `mxvm::VAR_BYTE`, `mxvm::VAR_EXTERN`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, `mxvm::Variable::var_value`, `mxvm::Instruction::vop`, `x64_emitFlushRegs()`, `x64_emitLoadVar()`, and `x64_emitReloadRegs()`.

Referenced by `x64_generateInstruction()`.

11.76.4.201 x64_gen_to_float()

```
void mxvm::Program::x64_gen_to_float (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1226 of file `icode_gen_x64.cpp`.

References `getMangledName()`, `isVariable()`, `mxvm::Operand::op`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `mxvm::VAR_INTEGER`, and `x64_generateLoadVar()`.

Referenced by `x64_generateInstruction()`.

11.76.4.202 x64_gen_to_int()

```
void mxvm::Program::x64_gen_to_int (
    std::ostream & out,
    const Instruction & i)
```

Definition at line 1218 of file `icode_gen_x64.cpp`.

References `getMangledName()`, `mxvm::Instruction::op1`, `mxvm::Instruction::op2`, `x64_emitStoreVar()`, `x64_release_call_area()`, and `x64_reserve_call_area()`.

Referenced by `x64_generateInstruction()`.

11.76.4.203 x64_generateCode()

```
void mxvm::Program::x64_generateCode (
    const Platform & platform,
    bool obj,
    std::ostream & out)
```

Top-level Win64 code generation entry point.

Parameters

<i>platform</i>	Target platform
<i>obj</i>	true if generating as an object
<i>out</i>	Assembly output stream

Definition at line 314 of file `icode_gen_x64.cpp`.

References `add_standard()`, `mxvm::Base::base`, `mx::Color::BRIGHT_BLUE`, `mx::Color::BRIGHT_CYAN`, `Col()`, `DONE`, `escapeNewLines()`, `mxvm::Base::external`, `getMangledName()`, `getVariable()`, `mxvm::Base::inc`, `mxvm::Instruction::instruction`, `isVariable()`, `mxvm::Base::labels`, `name`, `mxvm::Operand::op`, `mxvm::Base::root_name`, `mxvm::VAR_BYTE`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, `mxvm::Base::vars`, `x64_analyzeRegAlloc()`, `x64_emitReloadRegs()`, `x64_emitSaveRegs()`, `x64_emitStoreVar()`, `x64_generateInstruction()`, `x64_release_call_area()`, `x64_reserve_call_area()`, and `mxvm::x64_sp_mod16`.

Referenced by `generateCode()`.

11.76.4.204 x64_generateFunctionCall()

```
void mxvm::Program::x64_generateFunctionCall (
    std::ostream & out,
```

```
const std::string & name,
std::vector< Operand > & args)
```

Generate a Win64 ABI function call with operands as arguments.

Parameters

<i>out</i>	Assembly output stream
<i>name</i>	Callee function name
<i>args</i>	Argument operand list

Definition at line 257 of file [icode_gen_x64.cpp](#).

References [args](#), [getVariable\(\)](#), [isVariable\(\)](#), [name](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [x64_generateLoadVar\(\)](#), [x64_release_call_area\(\)](#), [x64_reserve_call_area\(\)](#), and [xmm_offset](#).

Referenced by [x64_gen_exit\(\)](#), [x64_gen_print\(\)](#), and [x64_generateInvokeCall\(\)](#).

11.76.4.205 x64_generateInstruction()

```
void mxvm::Program::x64_generateInstruction (
    std::ostream & out,
    const Instruction & i)
```

Dispatch a single instruction to its Win64 code generator.

Parameters

<i>out</i>	Assembly output stream
<i>i</i>	Instruction to generate code for

Definition at line 561 of file [icode_gen_x64.cpp](#).

References [ADD](#), [ALLOC](#), [AND](#), [CALL](#), [CMP](#), [CMP_NONE](#), [DIV](#), [DONE](#), [EXIT](#), [FCMP](#), [FREE](#), [GETLINE](#), [mxvm::Instruction::instruction](#), [INVOKE](#), [JA](#), [JAE](#), [JB](#), [JBE](#), [JC](#), [JE](#), [JG](#), [JGE](#), [JL](#), [JLE](#), [JMP](#), [JNC](#), [JNE](#), [JNO](#), [JNP](#), [JNS](#), [JNZ](#), [JO](#), [JP](#), [JS](#), [JZ](#), [last_cmp_type](#), [LEA](#), [LOAD](#), [MOD](#), [MOV](#), [MUL](#), [NEG](#), [NOT](#), [OR](#), [POP](#), [PRINT](#), [PUSH](#), [REALLOC](#), [RET](#), [RETURN](#), [STACK_LOAD](#), [STACK_STORE](#), [STACK_SUB](#), [STORE](#), [SUB](#), [TO_FLOAT](#), [TO_INT](#), [x64_gen_alloc\(\)](#), [x64_gen_arth\(\)](#), [x64_gen_bitop\(\)](#), [x64_gen_call\(\)](#), [x64_gen_cmp\(\)](#), [x64_gen_div\(\)](#), [x64_gen_done\(\)](#), [x64_gen_exit\(\)](#), [x64_gen_fcmp\(\)](#), [x64_gen_free\(\)](#), [x64_gen_getline\(\)](#), [x64_gen_invoke\(\)](#), [x64_gen_jae\(\)](#), [x64_gen_jbe\(\)](#), [x64_gen_jc\(\)](#), [x64_gen_jmp\(\)](#), [x64_gen_jnc\(\)](#), [x64_gen_jno\(\)](#), [x64_gen_jnp\(\)](#), [x64_gen_jns\(\)](#), [x64_gen_jo\(\)](#), [x64_gen_jp\(\)](#), [x64_gen_js\(\)](#), [x64_gen_lea\(\)](#), [x64_gen_load\(\)](#), [x64_gen_mod\(\)](#), [x64_gen_mov\(\)](#), [x64_gen_neg\(\)](#), [x64_gen_not\(\)](#), [x64_gen_pop\(\)](#), [x64_gen_print\(\)](#), [x64_gen_push\(\)](#), [x64_gen_realloc\(\)](#), [x64_gen_ret\(\)](#), [x64_gen_return\(\)](#), [x64_gen_stack_load\(\)](#), [x64_gen_stack_store\(\)](#), [x64_gen_stack_sub\(\)](#), [x64_gen_store\(\)](#), [x64_gen_to_float\(\)](#), [x64_gen_to_int\(\)](#), and [XOR](#).

Referenced by [x64_generateCode\(\)](#).

11.76.4.206 x64_generateInvokeCall()

```
void mxvm::Program::x64_generateInvokeCall (
    std::ostream & out,
    std::vector< Operand > & op)
```

Generate a Win64 dynamic invoke call.

Parameters

<i>out</i>	Assembly output stream
<i>op</i>	Operand list

Definition at line 296 of file [icode_gen_x64.cpp](#).

References [mxvm::isFunctionReturningOwnedPtr\(\)](#), [last_call_returns_owned_ptr](#), and [x64_generateFunctionCall\(\)](#).

Referenced by [x64_gen_invoke\(\)](#).

11.76.4.207 x64_generateLoadVar() [1/2]

```
int mxvm::Program::x64_generateLoadVar (
    std::ostream & out,
    int r,
    const Operand & op)
```

Load a variable or constant into a numbered Win64 register.

Parameters

<i>out</i>	Assembly output stream
<i>r</i>	Register index
<i>op</i>	Operand to load

Returns

1 if the operand was a constant, 0 if variable

Definition at line 205 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), [getVariable\(\)](#), [mxvm::is_stdio_name\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::OP_CONSTANT](#), [mxvm::stdio_index\(\)](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [x64_emit_iob_func\(\)](#), [x64_getRegisterByIndex\(\)](#), and [x64_reg_vars](#).

Referenced by [x64_gen_arth\(\)](#), [x64_gen_bitop\(\)](#), [x64_gen_cmp\(\)](#), [x64_gen_div\(\)](#), [x64_gen_fcmp\(\)](#), [x64_gen_mod\(\)](#), [x64_gen_mov\(\)](#), [x64_gen_neg\(\)](#), [x64_gen_not\(\)](#), [x64_gen_stack_load\(\)](#), [x64_gen_stack_store\(\)](#), [x64_gen_to_float\(\)](#), and [x64_generateFunctionCall\(\)](#).

11.76.4.208 x64_generateLoadVar() [2/2]

```
int mxvm::Program::x64_generateLoadVar (
    std::ostream & out,
    VarType type,
    std::string reg,
    const Operand & op)
```

Load a variable or constant into a named Win64 register.

Parameters

<i>out</i>	Assembly output stream
<i>type</i>	Expected variable type
<i>reg</i>	Target register name
<i>op</i>	Operand to load

Returns

1 if the operand was a constant, 0 if variable

Definition at line 160 of file [icode_gen_x64.cpp](#).

References [getMangledName\(\)](#), [getVariable\(\)](#), [mxvm::is_stdio_name\(\)](#), [isVariable\(\)](#), [mxvm::Operand::op](#), [mxvm::OP_CONSTANT](#), [mxvm::stdio_index\(\)](#), [mxvm::Operand::type](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [x64_emit_iob_func\(\)](#), and [x64_reg_vars](#).

11.76.4.209 x64_getRegisterByIndex()

```
std::string mxvm::Program::x64_getRegisterByIndex (
    int index,
    VarType type)
```


Get the Win64 register name for a given index and variable type.

Parameters

<i>index</i>	Register index
<i>type</i>	Variable type (determines integer vs. XMM register)

Returns

Register name string

Definition at line 20 of file [icode_gen_x64.cpp](#).

References [mxvm::VAR_FLOAT](#).

Referenced by [x64_generateLoadVar\(\)](#).

11.76.4.210 x64_release_call_area()

```
void mxvm::Program::x64_release_call_area (
    std::ostream & out,
    size_t frame)
```

Release a Win64 call frame previously reserved.

Parameters

<i>out</i>	Assembly output stream
<i>frame</i>	Frame size to release

Definition at line 147 of file [icode_gen_x64.cpp](#).

References [mxvm::x64_sp_mod16](#).

Referenced by [x64_emit_iob_func\(\)](#), [x64_gen_alloc\(\)](#), [x64_gen_call\(\)](#), [x64_gen_done\(\)](#), [x64_gen_exit\(\)](#), [x64_gen_free\(\)](#), [x64_gen_getline\(\)](#), [x64_gen_realloc\(\)](#), [x64_gen_to_int\(\)](#), [x64_generateCode\(\)](#), and [x64_generateFunctionCall\(\)](#).

11.76.4.211 x64_reserve_call_area()

```
size_t mxvm::Program::x64_reserve_call_area (
    std::ostream & out,
    size_t spill_bytes)
```

Reserve stack space for a Win64 call frame including spill area.

Parameters

<i>out</i>	Assembly output stream
<i>spill_bytes</i>	Additional spill bytes needed

Returns

Total frame size allocated (for matching release)

Definition at line 135 of file [icode_gen_x64.cpp](#).

References [mxvm::x64_sp_mod16](#).

Referenced by [x64_emit_iob_func\(\)](#), [x64_gen_alloc\(\)](#), [x64_gen_call\(\)](#), [x64_gen_done\(\)](#), [x64_gen_exit\(\)](#), [x64_gen_free\(\)](#), [x64_gen_getline\(\)](#), [x64_gen_realloc\(\)](#), [x64_gen_to_int\(\)](#), [x64_generateCode\(\)](#), and [x64_generateFunctionCall\(\)](#).

11.76.5 Friends And Related Symbol Documentation

11.76.5.1 Parser

```
friend class Parser [friend]
```

Definition at line 214 of file [icode.hpp](#).

References [Parser](#).

Referenced by [Parser](#).

11.76.6 Member Data Documentation

11.76.6.1 args

```
std::vector<std::string> mxvm::Program::args [private]
```

Definition at line 407 of file [icode.hpp](#).

Referenced by [exec_invoke\(\)](#), [exec_print\(\)](#), [exec_string_print\(\)](#), [gen_print\(\)](#), [generateFunctionCall\(\)](#), [generateInvokeCall\(\)](#), [operator=\(\)](#), [optimize_core\(\)](#), [printFormatted\(\)](#), [setArgs\(\)](#), [x64_gen_print\(\)](#), and [x64_generateFunctionCall\(\)](#).

11.76.6.2 carry_flag

```
bool mxvm::Program::carry_flag = false [private]
```

comparison carry flag

Definition at line 405 of file [icode.hpp](#).

Referenced by [exec_jc\(\)](#), [exec_jnc\(\)](#), and [operator=\(\)](#).

11.76.6.3 exitCode

```
int mxvm::Program::exitCode = 0 [private]
```

Definition at line 401 of file [icode.hpp](#).

Referenced by [exec_done\(\)](#), [exec_exit\(\)](#), [getExitCode\(\)](#), and [operator=\(\)](#).

11.76.6.4 filename

```
std::string mxvm::Program::filename
```

Definition at line 307 of file [icode.hpp](#).

Referenced by [operator=\(\)](#).

11.76.6.5 greater_flag

```
bool mxvm::Program::greater_flag = false [private]
```

comparison greater-than flag

Definition at line 404 of file [icode.hpp](#).

Referenced by [exec_cmp\(\)](#), [exec_fcmp\(\)](#), [exec_ja\(\)](#), [exec_jae\(\)](#), [exec_jg\(\)](#), [exec_jge\(\)](#), and [operator=\(\)](#).

11.76.6.6 last_call_ret_type

```
VarType mxvm::Program::last_call_ret_type = VarType::VAR_INTEGER
```

Definition at line 881 of file [icode.hpp](#).

11.76.6.7 last_call_returns_owned

```
bool mxvm::Program::last_call_returns_owned = false
```

Definition at line 880 of file [icode.hpp](#).

11.76.6.8 last_call_returns_owned_ptr

```
bool mxvm::Program::last_call_returns_owned_ptr = false [private]
```

tracks ownership of last call's return value

Definition at line 418 of file [icode.hpp](#).

Referenced by [gen_return\(\)](#), [generateInvokeCall\(\)](#), [x64_gen_return\(\)](#), and [x64_generateInvokeCall\(\)](#).

11.76.6.9 last_cmp_type

`LastCmpType` mxvm::Program::last_cmp_type = `CMP_NONE` [private]

Definition at line 417 of file [icode.hpp](#).

Referenced by [gen_cmp\(\)](#), [gen_fcmp\(\)](#), [gen_jump\(\)](#), [generateInstruction\(\)](#), [x64_gen_cmp\(\)](#), [x64_gen_fcmp\(\)](#), [x64_gen_jump\(\)](#), and [x64_generateInstruction\(\)](#).

11.76.6.10 less_flag

`bool` mxvm::Program::less_flag = `false` [private]

comparison less-than flag

Definition at line 403 of file [icode.hpp](#).

Referenced by [exec_cmp\(\)](#), [exec_fcmp\(\)](#), [exec_jb\(\)](#), [exec_jbe\(\)](#), [exec_jl\(\)](#), [exec_jle\(\)](#), and [operator=\(\)](#).

11.76.6.11 main_function

`bool` mxvm::Program::main_function [private]

true if this program has a main entry point

Definition at line 408 of file [icode.hpp](#).

Referenced by [operator=\(\)](#).

11.76.6.12 name

`std::string` mxvm::Program::name

Definition at line 306 of file [icode.hpp](#).

Referenced by [Program\(\)](#), [~Program\(\)](#), [add_standard\(\)](#), [exec_return\(\)](#), [flatten_inc\(\)](#), [flatten_label\(\)](#), [generateCode\(\)](#), [generateFunctionCall\(\)](#), [generateInvokeCall\(\)](#), [getMangledName\(\)](#), [getObjectByName\(\)](#), [getPlatformSymbolName\(\)](#), [getVariable\(\)](#), [isVariable\(\)](#), [memoryDump\(\)](#), [operator=\(\)](#), [optimize_core\(\)](#), [x64_generateCode\(\)](#), and [x64_generateFunctionCall\(\)](#).

11.76.6.13 object

`bool` mxvm::Program::object = `false`

Definition at line 341 of file [icode.hpp](#).

11.76.6.14 object_external

`bool` mxvm::Program::object_external = `false` [private]

Definition at line 409 of file [icode.hpp](#).

Referenced by [operator=\(\)](#).

11.76.6.15 objects

`std::vector<std::unique_ptr<Program> >` mxvm::Program::objects

Definition at line 340 of file [icode.hpp](#).

Referenced by [flatten\(\)](#), [getObjectByName\(\)](#), [memoryDump\(\)](#), [operator=\(\)](#), and [validateNames\(\)](#).

11.76.6.16 parent

`Program*` mxvm::Program::parent = `nullptr`

parent program (for object programs)

Definition at line 860 of file [icode.hpp](#).

Referenced by [operator=\(\)](#).

11.76.6.17 pc

`size_t` mxvm::Program::pc [private]

program counter

Definition at line 399 of file [icode.hpp](#).

Referenced by [Program\(\)](#), [Program\(\)](#), [exec\(\)](#), [exec_call\(\)](#), [exec_ja\(\)](#), [exec_jae\(\)](#), [exec_jb\(\)](#), [exec_jbe\(\)](#), [exec_jc\(\)](#), [exec_je\(\)](#), [exec_jg\(\)](#), [exec_jge\(\)](#), [exec_jl\(\)](#), [exec_jle\(\)](#), [exec_jump\(\)](#), [exec_jnc\(\)](#), [exec_jne\(\)](#), [exec_jnz\(\)](#), [exec_jo\(\)](#), [exec_jp\(\)](#), [exec_js\(\)](#), [exec_jz\(\)](#), [exec_ret\(\)](#), and [operator=\(\)](#).

11.76.6.18 platform

[Platform](#) `mxvm::Program::platform`

Definition at line 861 of file [icode.hpp](#).

Referenced by [gen_mul\(\)](#), [generateCode\(\)](#), [getMangledName\(\)](#), [getMangledName\(\)](#), [getPlatformSymbolName\(\)](#), [operator=\(\)](#), and [optimize_core\(\)](#).

11.76.6.19 result

[Operand](#) `mxvm::Program::result`

return value operand from last invoke

Definition at line 859 of file [icode.hpp](#).

Referenced by [escapeNewLines\(\)](#), [exec_invoke\(\)](#), [exec_return\(\)](#), and [operator=\(\)](#).

11.76.6.20 running

`bool mxvm::Program::running [private]`

interpreter running flag

Definition at line 400 of file [icode.hpp](#).

Referenced by [Program\(\)](#), [exec\(\)](#), [operator=\(\)](#), and [stop\(\)](#).

11.76.6.21 stack

[Stack](#) `mxvm::Program::stack`

Definition at line 858 of file [icode.hpp](#).

Referenced by [exec_call\(\)](#), [exec_pop\(\)](#), [exec_push\(\)](#), [exec_ret\(\)](#), [exec_stack_load\(\)](#), [exec_stack_store\(\)](#), [exec_stack_sub\(\)](#), [operator=\(\)](#), and [post\(\)](#).

11.76.6.22 sysv_reg_save_order

`std::vector<std::string> mxvm::Program::sysv_reg_save_order`

Definition at line 678 of file [icode.hpp](#).

Referenced by [sysv_analyzeRegAlloc\(\)](#), [sysv_emitRestoreRegs\(\)](#), and [sysv_emitSaveRegs\(\)](#).

11.76.6.23 sysv_reg_vars

`std::unordered_map<std::string, std::string> mxvm::Program::sysv_reg_vars`

SysV variable-to-register mapping.

Definition at line 677 of file [icode.hpp](#).

Referenced by [gen_arth\(\)](#), [gen_cmp\(\)](#), [gen_lea\(\)](#), [gen_to_float\(\)](#), [gen_to_int\(\)](#), [generateLoadVar\(\)](#), [generateLoadVar\(\)](#), [sysv_analyzeRegAlloc\(\)](#), [sysv_emitFlushRegs\(\)](#), [sysv_emitLoadVar\(\)](#), [sysv_emitReloadRegs\(\)](#), [sysv_emitStoreVar\(\)](#), and [sysv_emitStoreVarImm\(\)](#).

11.76.6.24 x64_reg_save_order

`std::vector<std::string> mxvm::Program::x64_reg_save_order`

Definition at line 634 of file [icode.hpp](#).

Referenced by [x64_analyzeRegAlloc\(\)](#), [x64_emitRestoreRegs\(\)](#), and [x64_emitSaveRegs\(\)](#).

11.76.6.25 x64_reg_vars

`std::unordered_map<std::string, std::string> mxvm::Program::x64_reg_vars`

Win64 variable-to-register mapping.

Definition at line 633 of file [icode.hpp](#).

Referenced by [x64_analyzeRegAlloc\(\)](#), [x64_emitFlushRegs\(\)](#), [x64_emitLoadVar\(\)](#), [x64_emitReloadRegs\(\)](#), [x64_emitStoreVar\(\)](#), [x64_emitStoreVarImm\(\)](#), [x64_gen_arth\(\)](#), [x64_gen_cmp\(\)](#), [x64_gen_lea\(\)](#), [x64_generateLoadVar\(\)](#), and [x64_generateLoadVar\(\)](#).

11.76.6.26 xmm_offset

```
int mxvm::Program::xmm_offset = 0 [private]
```

current XMM register allocation offset

Definition at line 406 of file [icode.hpp](#).

Referenced by [gen_print\(\)](#), [generateFunctionCall\(\)](#), [getRegisterByIndex\(\)](#), [operator=\(\)](#), [x64_gen_print\(\)](#), and [x64_generateFunctionCall\(\)](#).

11.76.6.27 zero_flag

```
bool mxvm::Program::zero_flag = false [private]
```

comparison zero flag

Definition at line 402 of file [icode.hpp](#).

Referenced by [exec_cmp\(\)](#), [exec_fcmp\(\)](#), [exec_jae\(\)](#), [exec_jbe\(\)](#), [exec_je\(\)](#), [exec_jge\(\)](#), [exec_jle\(\)](#), [exec_jne\(\)](#), [exec_jnz\(\)](#), [exec_jz\(\)](#), and [operator=\(\)](#).

The documentation for this class was generated from the following files:

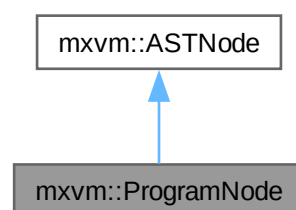
- [include/mxvm/icode.hpp](#)
- [src/icode.cpp](#)
- [src/icode_exec.cpp](#)
- [src/icode_gen.cpp](#)
- [src/icode_gen_x64.cpp](#)
- [src/icode_opt.cpp](#)

11.77 mxvm::ProgramNode Class Reference

Root AST node representing an entire program or object.

```
#include <ast.hpp>
```

Inheritance diagram for `mxvm::ProgramNode`:



Public Member Functions

- void [addSection](#) (std::unique_ptr< [SectionNode](#) > section)
- void [addInlineObject](#) (std::unique_ptr< [ProgramNode](#) > obj)
- virtual void [accept](#) ([ASTVisitor](#) &visitor) override
- virtual std::string [toString](#) () const override

Public Member Functions inherited from [mxvm::ASTNode](#)

- virtual [~ASTNode](#) ()=default

Public Attributes

- `std::string name`
- `bool object = false`
true if this node represents an object rather than a program
- `std::string root_name`
- `std::vector< std::unique_ptr< SectionNode > > sections`
- `std::vector< std::unique_ptr< ProgramNode > > inlineObjects`

11.77.1 Detailed Description

Root AST node representing an entire program or object.
 Definition at line 50 of file [ast.hpp](#).

11.77.2 Member Function Documentation

11.77.2.1 accept()

```
void mxvm::ProgramNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 12 of file [ast.cpp](#).

References [mxvm::ASTVisitor::visit\(\)](#).

11.77.2.2 addInlineObject()

```
void mxvm::ProgramNode::addInlineObject (
    std::unique_ptr< ProgramNode > obj) [inline]
```

Definition at line 62 of file [ast.hpp](#).

References [inlineObjects](#).

11.77.2.3 addSection()

```
void mxvm::ProgramNode::addSection (
    std::unique_ptr< SectionNode > section) [inline]
```

Definition at line 58 of file [ast.hpp](#).

References [sections](#).

11.77.2.4 toString()

```
std::string mxvm::ProgramNode::toString () const [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 16 of file [ast.cpp](#).

References [sections](#).

11.77.3 Member Data Documentation

11.77.3.1 inlineObjects

```
std::vector<std::unique_ptr<ProgramNode> > mxvm::ProgramNode::inlineObjects
```

Definition at line 56 of file [ast.hpp](#).

Referenced by [addInlineObject\(\)](#).

11.77.3.2 name

```
std::string mxvm::ProgramNode::name
```

Definition at line 52 of file [ast.hpp](#).

11.77.3.3 object

`bool mxvm::ProgramNode::object = false`
 true if this node represents an object rather than a program
 Definition at line 53 of file [ast.hpp](#).

11.77.3.4 root_name

`std::string mxvm::ProgramNode::root_name`
 Definition at line 54 of file [ast.hpp](#).

11.77.3.5 sections

`std::vector<std::unique_ptr<SectionNode> > mxvm::ProgramNode::sections`

Definition at line 55 of file [ast.hpp](#).

Referenced by [addSection\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

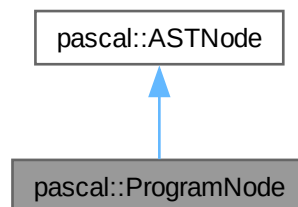
- [include/mxvm/ast.hpp](#)
- [src/ast.cpp](#)

11.78 pascal::ProgramNode Class Reference

AST node for a complete program (program name + block).

`#include <ast.hpp>`

Inheritance diagram for `pascal::ProgramNode`:



Public Member Functions

- [ProgramNode](#) (const std::string &name, std::unique_ptr< [BlockNode](#) > blk)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.

- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [name](#)
program identifier
- std::unique_ptr< [BlockNode](#) > [block](#)
the program body
- std::vector< std::string > [uses](#)
modules imported via 'uses' clause

11.78.1 Detailed Description

AST node for a complete program (program name + block).
Definition at line 98 of file [ast.hpp](#).

11.78.2 Constructor & Destructor Documentation

11.78.2.1 ProgramNode()

```
pascal::ProgramNode::ProgramNode (
    const std::string & name,
    std::unique_ptr< BlockNode > blk)
```

Definition at line 14 of file [ast.cpp](#).
References [block](#), and [name](#).

11.78.3 Member Function Documentation

11.78.3.1 accept()

```
void pascal::ProgramNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).
Implements [pascal::ASTNode](#).
Definition at line 23 of file [ast.cpp](#).
References [pascal::ASTVisitor::visit\(\)](#).

11.78.3.2 toString()

```
std::string pascal::ProgramNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.
Implements [pascal::ASTNode](#).
Definition at line 27 of file [ast.cpp](#).
References [name](#).

11.78.4 Member Data Documentation

11.78.4.1 block

```
std::unique_ptr<BlockNode> pascal::ProgramNode::block
```

the program body
Definition at line 101 of file [ast.hpp](#).
Referenced by [ProgramNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.78.4.2 name

`std::string pascal::ProgramNode::name`

program identifier

Definition at line 100 of file [ast.hpp](#).

Referenced by [ProgramNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.78.4.3 uses

`std::vector<std::string> pascal::ProgramNode::uses`

modules imported via 'uses' clause

Definition at line 102 of file [ast.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

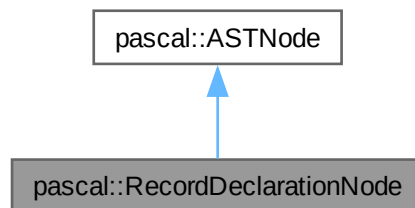
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.79 pascal::RecordDeclarationNode Class Reference

AST node for a named record declaration.

`#include <ast.hpp>`

Inheritance diagram for `pascal::RecordDeclarationNode`:



Public Member Functions

- [RecordDeclarationNode](#) (const `std::string` &name, `std::unique_ptr`< [RecordTypeNode](#) > recordType)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- `std::string` [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (`std::ostream` &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- `std::string name`
record type name
- `std::unique_ptr< RecordTypeNode > recordType`
the record body

11.79.1 Detailed Description

AST node for a named record declaration.
Definition at line 658 of file [ast.hpp](#).

11.79.2 Constructor & Destructor Documentation

11.79.2.1 RecordDeclarationNode()

```
pascal::RecordDeclarationNode::RecordDeclarationNode (
    const std::string & name,
    std::unique_ptr< RecordTypeNode > recordType)
```

Definition at line 357 of file [ast.cpp](#).
References [name](#), and [recordType](#).

11.79.3 Member Function Documentation

11.79.3.1 accept()

```
void pascal::RecordDeclarationNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).
Implements [pascal::ASTNode](#).
Definition at line 360 of file [ast.cpp](#).
References [pascal::ASTVisitor::visit\(\)](#).

11.79.3.2 toString()

```
std::string pascal::RecordDeclarationNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.
Implements [pascal::ASTNode](#).
Definition at line 362 of file [ast.cpp](#).
References [name](#).

11.79.4 Member Data Documentation

11.79.4.1 name

```
std::string pascal::RecordDeclarationNode::name
```

record type name
Definition at line 660 of file [ast.hpp](#).
Referenced by [RecordDeclarationNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.79.4.2 recordType

```
std::unique_ptr<RecordTypeNode> pascal::RecordDeclarationNode::recordType
```

the record body
Definition at line 661 of file [ast.hpp](#).
Referenced by [RecordDeclarationNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).
The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.80 pascal::CodeGenVisitor::RecordField Struct Reference

A field within a record type definition.

Public Member Functions

- [RecordField](#) ()=default
- [RecordField](#) (const [RecordField](#) &other)=default
- [RecordField](#) ([RecordField](#) &&other) noexcept=default
- [RecordField](#) & [operator=](#) (const [RecordField](#) &other)=default
- [RecordField](#) & [operator=](#) ([RecordField](#) &&other) noexcept=default

Public Attributes

- std::string [name](#)
field name
- std::string [typeName](#)
field type name
- int [offset](#)
byte offset within the record
- int [size](#)
field size in bytes
- bool [isArray](#) = false
true if the field is an array
- [ArrayInfo](#) [arrayInfo](#)
array descriptor (when isArray)

11.80.1 Detailed Description

A field within a record type definition.
Definition at line [1981](#) of file [icode.hpp](#).

11.80.2 Constructor & Destructor Documentation

11.80.2.1 RecordField() [1/3]

`pascal::CodeGenVisitor::RecordField::RecordField () [default]`
Referenced by [RecordField\(\)](#), [RecordField\(\)](#), [operator=\(\)](#), and [operator=\(\)](#).

11.80.2.2 RecordField() [2/3]

`pascal::CodeGenVisitor::RecordField::RecordField (const RecordField & other) [default]`
References [RecordField\(\)](#).

11.80.2.3 RecordField() [3/3]

`pascal::CodeGenVisitor::RecordField::RecordField (RecordField && other) [default], [noexcept]`
References [RecordField\(\)](#).

11.80.3 Member Function Documentation

11.80.3.1 operator=() [1/2]

`RecordField & pascal::CodeGenVisitor::RecordField::operator= (const RecordField & other) [default]`
References [RecordField\(\)](#).

11.80.3.2 operator=() [2/2]

`RecordField` & `pascal::CodeGenVisitor::RecordField::operator=` (
`RecordField` && `other`) [default], [noexcept]

References [RecordField\(\)](#).

11.80.4 Member Data Documentation

11.80.4.1 arrayInfo

`ArrayInfo` `pascal::CodeGenVisitor::RecordField::arrayInfo`
 array descriptor (when isArray)

Definition at line 1987 of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

11.80.4.2 isArray

`bool` `pascal::CodeGenVisitor::RecordField::isArray` = false
 true if the field is an array

Definition at line 1986 of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

11.80.4.3 name

`std::string` `pascal::CodeGenVisitor::RecordField::name`
 field name

Definition at line 1982 of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

11.80.4.4 offset

`int` `pascal::CodeGenVisitor::RecordField::offset`
 byte offset within the record

Definition at line 1984 of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

11.80.4.5 size

`int` `pascal::CodeGenVisitor::RecordField::size`
 field size in bytes

Definition at line 1985 of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

11.80.4.6 typeName

`std::string` `pascal::CodeGenVisitor::RecordField::typeName`
 field type name

Definition at line 1983 of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this struct was generated from the following file:

- [src/frontend/include/icode.hpp](#)

11.81 pascal::CodeGenVisitor::RecordTypeInfo Struct Reference

Complete description of a record type.

Public Attributes

- int [size](#) = 0
total record size in bytes
- std::vector< [RecordField](#) > [fields](#)
ordered field list
- std::unordered_map< std::string, int > [nameToIndex](#)
field name -> index into fields

11.81.1 Detailed Description

Complete description of a record type.

Definition at line [1996](#) of file [icode.hpp](#).

11.81.2 Member Data Documentation

11.81.2.1 fields

```
std::vector<RecordField> pascal::CodeGenVisitor::RecordTypeInfo::fields
```

ordered field list

Definition at line [1998](#) of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

11.81.2.2 nameToIndex

```
std::unordered_map<std::string, int> pascal::CodeGenVisitor::RecordTypeInfo::nameToIndex
```

field name -> index into fields

Definition at line [1999](#) of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

11.81.2.3 size

```
int pascal::CodeGenVisitor::RecordTypeInfo::size = 0
```

total record size in bytes

Definition at line [1997](#) of file [icode.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this struct was generated from the following file:

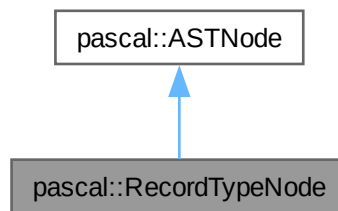
- [src/frontend/include/icode.hpp](#)

11.82 pascal::RecordTypeNode Class Reference

AST node for a record type (record ... end).

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::RecordTypeNode`:



Public Member Functions

- `RecordTypeNode` (`std::vector< std::unique_ptr< ASTNode > > fields`)
- void `accept` (`ASTVisitor &visitor`) override
Accept an `ASTVisitor` (double-dispatch).
- `std::string toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from `pascal::ASTNode`

- virtual `~ASTNode` ()=default
- virtual void `print` (`std::ostream &out`, `int indent=0`) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (`int line`)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- `std::vector< std::unique_ptr< ASTNode > > fields`
fixed field declarations
- `std::string variantTagName`
tag field name (empty if no variant part)
- `std::string variantTagType`
tag field type name
- `std::vector< VariantArm > variantArms`
variant arms (empty if no variant part)

11.82.1 Detailed Description

AST node for a record type (record ... end).
Definition at line 646 of file `ast.hpp`.

11.82.2 Constructor & Destructor Documentation

11.82.2.1 RecordTypeNode()

```
pascal::RecordTypeNode::RecordTypeNode (
    std::vector< std::unique_ptr< ASTNode > > fields)
```

Definition at line 350 of file [ast.cpp](#).

References [fields](#).

11.82.3 Member Function Documentation

11.82.3.1 accept()

```
void pascal::RecordTypeNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 353 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.82.3.2 toString()

```
std::string pascal::RecordTypeNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 355 of file [ast.cpp](#).

11.82.4 Member Data Documentation

11.82.4.1 fields

```
std::vector<std::unique_ptr<ASTNode> > pascal::RecordTypeNode::fields
```

fixed field declarations

Definition at line 648 of file [ast.hpp](#).

Referenced by [RecordTypeNode\(\)](#).

11.82.4.2 variantArms

```
std::vector<VariantArm> pascal::RecordTypeNode::variantArms
```

variant arms (empty if no variant part)

Definition at line 651 of file [ast.hpp](#).

11.82.4.3 variantTagName

```
std::string pascal::RecordTypeNode::variantTagName
```

tag field name (empty if no variant part)

Definition at line 649 of file [ast.hpp](#).

11.82.4.4 variantTagType

```
std::string pascal::RecordTypeNode::variantTagType
```

tag field type name

Definition at line 650 of file [ast.hpp](#).

The documentation for this class was generated from the following files:

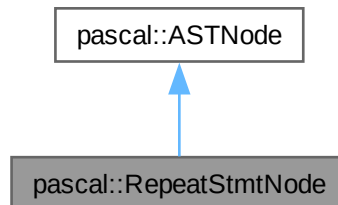
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.83 pascal::RepeatStmtNode Class Reference

AST node for a repeat..until loop.

#include <ast.hpp>

Inheritance diagram for pascal::RepeatStmtNode:



Public Member Functions

- [RepeatStmtNode](#) (std::vector< std::unique_ptr< [ASTNode](#) > > stmts, std::unique_ptr< [ASTNode](#) > cond)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::vector< std::unique_ptr< [ASTNode](#) > > [statements](#)
loop body statements
- std::unique_ptr< [ASTNode](#) > [condition](#)
termination condition

11.83.1 Detailed Description

AST node for a repeat..until loop.

Definition at line 611 of file [ast.hpp](#).

11.83.2 Constructor & Destructor Documentation

11.83.2.1 RepeatStmtNode()

```

pascal::RepeatStmtNode::RepeatStmtNode (
    std::vector< std::unique_ptr< ASTNode > > stmts,
    std::unique_ptr< ASTNode > cond) [inline]
  
```


Definition at line 616 of file [ast.hpp](#).
References [condition](#), and [statements](#).

11.83.3 Member Function Documentation

11.83.3.1 accept()

```
void pascal::RepeatStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 153 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.83.3.2 toString()

```
std::string pascal::RepeatStmtNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 621 of file [ast.hpp](#).

11.83.4 Member Data Documentation

11.83.4.1 condition

```
std::unique_ptr<ASTNode> pascal::RepeatStmtNode::condition
```

termination condition

Definition at line 614 of file [ast.hpp](#).

Referenced by [RepeatStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.83.4.2 statements

```
std::vector<std::unique_ptr<ASTNode> > pascal::RepeatStmtNode::statements
```

loop body statements

Definition at line 613 of file [ast.hpp](#).

Referenced by [RepeatStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.84 mxvm::RuntimeFunction Class Reference

Wraps a dynamically loaded native function from a shared library module.

```
#include <icode.hpp>
```

Public Member Functions

- [RuntimeFunction](#) ()
Default constructor — null function pointer and handle.
- [RuntimeFunction](#) (const [RuntimeFunction](#) &r)
Copy constructor.
- [RuntimeFunction](#) & operator= (const [RuntimeFunction](#) &r)
Copy-assignment operator.
- [~RuntimeFunction](#) ()=default
- [RuntimeFunction](#) (const std::string &mod, const std::string &name)
Load a function symbol from a shared library.
- void [call](#) ([Program](#) *program, std::vector< [Operand](#) > &operands)
Invoke the loaded function.

Public Attributes

- void * [func](#) = nullptr
resolved function pointer
- void * [handle](#) = nullptr
dlopen handle
- std::string [mod_name](#)
module name
- std::string [fname](#)
function symbol name

Static Public Attributes

- static std::unordered_map< std::string, void * > [handles](#)
cached library handles

11.84.1 Detailed Description

Wraps a dynamically loaded native function from a shared library module.

Manages dlopen handles and function pointers for external module calls. Shared library handles are cached in a static map.

Definition at line 78 of file [icode.hpp](#).

11.84.2 Constructor & Destructor Documentation

11.84.2.1 RuntimeFunction() [1/3]

```
mxvm::RuntimeFunction::RuntimeFunction () [inline]
```

Default constructor — null function pointer and handle.

Definition at line 81 of file [icode.hpp](#).

References [func](#), and [handle](#).

Referenced by [RuntimeFunction\(\)](#), and [operator=\(\)](#).

11.84.2.2 RuntimeFunction() [2/3]

```
mxvm::RuntimeFunction::RuntimeFunction (
    const RuntimeFunction & r) [inline]
```

Copy constructor.

Definition at line 84 of file [icode.hpp](#).

References [RuntimeFunction\(\)](#), [fname](#), [func](#), [handle](#), and [mod_name](#).

11.84.2.3 ~RuntimeFunction()

```
mxvm::RuntimeFunction::~~RuntimeFunction () [default]
```

11.84.2.4 RuntimeFunction() [3/3]

```
mxvm::RuntimeFunction::RuntimeFunction (
    const std::string & mod,
    const std::string & name)
```

Load a function symbol from a shared library.

Parameters

<i>mod</i>	Module/library name (without extension)
<i>name</i>	Function symbol name to resolve

Definition at line 201 of file [icode.cpp](#).

References [fname](#), [func](#), [handle](#), and [handles](#).

11.84.3 Member Function Documentation

11.84.3.1 call()

```
void mxvm::RuntimeFunction::call (
    Program * program,
    std::vector< Operand > & operands)
```

Invoke the loaded function.

Parameters

<i>program</i>	Pointer to the running Program for variable access
<i>operands</i>	Instruction operands passed to the native function

Definition at line 224 of file [icode.cpp](#).

References [fname](#), [func](#), [handle](#), and [mod_name](#).

11.84.3.2 operator=()

```
RuntimeFunction & mxvm::RuntimeFunction::operator= (
    const RuntimeFunction & r) [inline]
```

Copy-assignment operator.

Definition at line 87 of file [icode.hpp](#).

References [RuntimeFunction\(\)](#), [fname](#), [func](#), [handle](#), and [mod_name](#).

11.84.4 Member Data Documentation

11.84.4.1 fname

```
std::string mxvm::RuntimeFunction::fname
```

function symbol name

Definition at line 110 of file [icode.hpp](#).

Referenced by [RuntimeFunction\(\)](#), [RuntimeFunction\(\)](#), [call\(\)](#), and [operator=\(\)](#).

11.84.4.2 func

```
void* mxvm::RuntimeFunction::func = nullptr
```

resolved function pointer

Definition at line 107 of file [icode.hpp](#).

Referenced by [RuntimeFunction\(\)](#), [RuntimeFunction\(\)](#), [RuntimeFunction\(\)](#), [call\(\)](#), and [operator=\(\)](#).

11.84.4.3 handle

```
void* mxvm::RuntimeFunction::handle = nullptr
```

dlopen handle

Definition at line 108 of file [icode.hpp](#).

Referenced by [RuntimeFunction\(\)](#), [RuntimeFunction\(\)](#), [RuntimeFunction\(\)](#), [call\(\)](#), and [operator=\(\)](#).

11.84.4.4 handles

```
std::unordered_map< std::string, void * > mxvm::RuntimeFunction::handles [static]
```

cached library handles

Definition at line 111 of file [icode.hpp](#).

Referenced by [RuntimeFunction\(\)](#), and [mxvm::Program::~~Program\(\)](#).

11.84.4.5 mod_name

std::string mxvm::RuntimeFunction::mod_name

module name

Definition at line 109 of file [icode.hpp](#).

Referenced by [RuntimeFunction\(\)](#), [call\(\)](#), and [operator=\(\)](#).

The documentation for this class was generated from the following files:

- [include/mxvm/icode.hpp](#)
- [src/icode.cpp](#)

11.85 scan::ScanExcept Class Reference

Exception thrown during scanning errors.

#include <scanner.hpp>

Public Member Functions

- [ScanExcept](#) ()=default
- [ScanExcept](#) (const [StringType](#) &why)
Construct a scan exception with a reason string.
- [StringType](#) why () const
Get the error description.

Private Attributes

- [StringType](#) msg

11.85.1 Detailed Description

Exception thrown during scanning errors.

Definition at line 80 of file [scanner.hpp](#).

11.85.2 Constructor & Destructor Documentation

11.85.2.1 ScanExcept() [1/2]

scan::ScanExcept::ScanExcept () [default]

11.85.2.2 ScanExcept() [2/2]

scan::ScanExcept::ScanExcept (
const [StringType](#) & why) [inline]

Construct a scan exception with a reason string.

Parameters

why	Description of the scanning error
---------------------	-----------------------------------

Definition at line 87 of file [scanner.hpp](#).

References [msg](#), and [why\(\)](#).

11.85.3 Member Function Documentation

11.85.3.1 why()

[StringType](#) scan::ScanExcept::why () const [inline]

Get the error description.

Definition at line 89 of file [scanner.hpp](#).

References [msg](#).

Referenced by [ScanExcept\(\)](#), and [mxvm::Parser::scan\(\)](#).

11.85.4 Member Data Documentation

11.85.4.1 msg

[StringType](#) scan::ScanExcept::msg [private]

Definition at line 92 of file [scanner.hpp](#).

Referenced by [ScanExcept\(\)](#), and [why\(\)](#).

The documentation for this class was generated from the following file:

- [src/scanner/include/scanner/scanner.hpp](#)

11.86 scan::Scanner Class Reference

Lexical scanner that tokenizes source text.

```
#include <scanner.hpp>
```

Public Member Functions

- [Scanner](#) (const [TString](#) &b)
Construct a scanner from a string buffer.
- [uint64_t scan](#) ()
Tokenize the entire source buffer.
- [std::optional< TToken > grabId](#) ()
Try to grab an identifier or keyword token.
- [std::optional< TToken > grabDigits](#) ()
Try to grab a numeric literal token.
- [std::optional< TToken > grabSymbols](#) ()
Try to grab one or more symbol characters as a token.
- [std::optional< TToken > grabString](#) ()
Try to grab a double-quoted string literal token.
- [std::optional< TToken > grabSingle](#) ()
Try to grab a single-quoted character literal token.
- [TToken & operator\[\]](#) (size_t index)
Access a token by index.
- [size_t size](#) () const
Return the total number of scanned tokens.
- [void removeEOL](#) ()
Remove end-of-line tokens from the token list.
- [std::vector< TToken > & getTokens](#) ()
Get the underlying token vector.

Private Member Functions

- [bool is_c_sym](#) (const [StringType](#) &str)
Check if a string is a recognized C-style symbol.
- [uint64_t parseLineNumber](#) ()
Parse a #line directive line number.
- [std::string parseFileName](#) ()
Parse a #line directive filename.

Private Attributes

- [TString string_buffer](#)
source text buffer
- [TMap token_map](#)
character classification map
- `std::vector< TToken >` [tokens](#)
scanned token list

11.86.1 Detailed Description

Lexical scanner that tokenizes source text.

Scans a StringBuffer character-by-character and produces a list of typed tokens (identifiers, numbers, strings, symbols, hex literals).

Definition at line 28 of file [scanner.hpp](#).

11.86.2 Constructor & Destructor Documentation

11.86.2.1 Scanner()

```
scan::Scanner::Scanner (
    const TString & b)
```

Construct a scanner from a string buffer.

Parameters

<i>b</i>	Source text buffer to scan
----------	----------------------------

Definition at line 12 of file [scanner.cpp](#).

References [string_buffer](#).

11.86.3 Member Function Documentation

11.86.3.1 getTokens()

```
std::vector< TToken > & scan::Scanner::getTokens () [inline]
```

Get the underlying token vector.

Definition at line 62 of file [scanner.hpp](#).

References [tokens](#).

Referenced by [mxx::TPValidator::parseUses\(\)](#).

11.86.3.2 grabDigits()

```
std::optional< TToken > scan::Scanner::grabDigits ()
```

Try to grab a numeric literal token.

Definition at line 211 of file [scanner.cpp](#).

References [pos\(\)](#), [string_buffer](#), [types::TT_HEX](#), and [types::TT_NUM](#).

Referenced by [scan\(\)](#).

11.86.3.3 grabId()

```
std::optional< TToken > scan::Scanner::grabId ()
```

Try to grab an identifier or keyword token.

Definition at line 185 of file [scanner.cpp](#).

References [scan::token::Token< Ch, String >::set_filename\(\)](#), [scan::token::Token< Ch, String >::set_pos\(\)](#), [scan::token::Token< Ch, String >::setToken\(\)](#), [string_buffer](#), and [types::TT_ID](#).

Referenced by [scan\(\)](#).

11.86.3.4 grabSingle()

```
std::optional< TToken > scan::Scanner::grabSingle ()
```

Try to grab a single-quoted character literal token.

Definition at line 420 of file [scanner.cpp](#).

References [pos\(\)](#), [string_buffer](#), and [types::TT_STR](#).

Referenced by [scan\(\)](#).

11.86.3.5 grabString()

```
std::optional< TToken > scan::Scanner::grabString ()
```

Try to grab a double-quoted string literal token.

Definition at line 367 of file [scanner.cpp](#).

References [pos\(\)](#), [string_buffer](#), and [types::TT_STR](#).

Referenced by [scan\(\)](#).

11.86.3.6 grabSymbols()

```
std::optional< TToken > scan::Scanner::grabSymbols ()
```

Try to grab one or more symbol characters as a token.

Definition at line 292 of file [scanner.cpp](#).

References [is_c_sym\(\)](#), [pos\(\)](#), [string_buffer](#), [token_map](#), [types::TT_ARG](#), [types::TT_SYM](#), and [types::TT_SYMBOL](#).

Referenced by [scan\(\)](#).

11.86.3.7 is_c_sym()

```
bool scan::Scanner::is_c_sym (
    const StringType & str) [private]
```

Check if a string is a recognized C-style symbol.

Definition at line 283 of file [scanner.cpp](#).

Referenced by [grabSymbols\(\)](#).

11.86.3.8 operator[]()

```
TToken & scan::Scanner::operator[] (
    size_t index)
```

Access a token by index.

Parameters

<i>index</i>	Zero-based token index
--------------	------------------------

Returns

Reference to the token

Definition at line 480 of file [scanner.cpp](#).

References [size\(\)](#), and [tokens](#).

11.86.3.9 parseFileName()

```
std::string scan::Scanner::parseFileName () [private]
```

Parse a #line directive filename.

Definition at line 44 of file [scanner.cpp](#).

References [string_buffer](#).

11.86.3.10 parseLineNumber()

```
uint64_t scan::Scanner::parseLineNumber () [private]
```

Parse a #line directive line number.

Definition at line 22 of file [scanner.cpp](#).

References [string_buffer](#).

11.86.3.11 removeEOL()

```
void scan::Scanner::removeEOL ()
```

Remove end-of-line tokens from the token list.

Definition at line 14 of file [scanner.cpp](#).

References [tokens](#), and [types::TT_SYM](#).

11.86.3.12 scan()

```
uint64_t scan::Scanner::scan ()
```

Tokenize the entire source buffer.

Returns

Number of tokens produced

Definition at line 62 of file [scanner.cpp](#).

References [grabDigits\(\)](#), [grabId\(\)](#), [grabSingle\(\)](#), [grabString\(\)](#), [grabSymbols\(\)](#), [pos\(\)](#), [string_buffer](#), [token_map](#), [tokens](#), [types::TT_CHAR](#), [types::TT_DIGIT](#), [types::TT_NULL](#), [types::TT_SINGLE](#), [types::TT_SPACE](#), [types::TT_STRING](#), [types::TT_SYM](#), and [types::TT_SYMBOL](#).

Referenced by [mxx::TPValidator::parseUses\(\)](#).

11.86.3.13 size()

```
size_t scan::Scanner::size () const
```

Return the total number of scanned tokens.

Definition at line 487 of file [scanner.cpp](#).

References [tokens](#).

Referenced by [operator\[\]\(\)](#).

11.86.4 Member Data Documentation**11.86.4.1 string_buffer**

```
TString scan::Scanner::string_buffer [private]
```

source text buffer

Definition at line 65 of file [scanner.hpp](#).

Referenced by [Scanner\(\)](#), [grabDigits\(\)](#), [grabId\(\)](#), [grabSingle\(\)](#), [grabString\(\)](#), [grabSymbols\(\)](#), [parseFileName\(\)](#), [parseLineNumber\(\)](#), and [scan\(\)](#).

11.86.4.2 token_map

```
TMap scan::Scanner::token_map [private]
```

character classification map

Definition at line 66 of file [scanner.hpp](#).

Referenced by [grabSymbols\(\)](#), and [scan\(\)](#).

11.86.4.3 tokens

```
std::vector<TToken> scan::Scanner::tokens [private]
```

scanned token list

Definition at line 67 of file [scanner.hpp](#).

Referenced by [getTokens\(\)](#), [operator\[\]\(\)](#), [removeEOL\(\)](#), [scan\(\)](#), and [size\(\)](#).

The documentation for this class was generated from the following files:

- [src/scanner/include/scanner/scanner.hpp](#)
- [src/scanner/scanner.cpp](#)

11.87 mxx::Scope Struct Reference

A single scope level, tracking declared identifiers by category.

`#include <validator.hpp>`

Public Attributes

- `std::unordered_set< std::string > vars`
variable names
- `std::unordered_set< std::string > consts`
constant names
- `std::unordered_set< std::string > types`
type names
- `std::unordered_set< std::string > funcs`
function names
- `std::unordered_set< std::string > procs`
procedure names
- `std::unordered_set< std::string > params`
parameter names

11.87.1 Detailed Description

A single scope level, tracking declared identifiers by category.

Definition at line 18 of file [validator.hpp](#).

11.87.2 Member Data Documentation

11.87.2.1 consts

`std::unordered_set<std::string> mxx::Scope::consts`

constant names

Definition at line 20 of file [validator.hpp](#).

11.87.2.2 funcs

`std::unordered_set<std::string> mxx::Scope::funcs`

function names

Definition at line 22 of file [validator.hpp](#).

Referenced by [mxx::TPValidator::declareFunc\(\)](#), and [mxx::TPValidator::isFunctionDeclaredHere\(\)](#).

11.87.2.3 params

`std::unordered_set<std::string> mxx::Scope::params`

parameter names

Definition at line 24 of file [validator.hpp](#).

Referenced by [mxx::TPValidator::declareParam\(\)](#), and [mxx::TPValidator::isParamDeclaredHere\(\)](#).

11.87.2.4 procs

`std::unordered_set<std::string> mxx::Scope::procs`

procedure names

Definition at line 23 of file [validator.hpp](#).

Referenced by [mxx::TPValidator::declareProc\(\)](#), and [mxx::TPValidator::isProcDeclaredHere\(\)](#).

11.87.2.5 types

```
std::unordered_set<std::string> mxx::Scope::types
```

type names

Definition at line 21 of file [validator.hpp](#).

Referenced by [mxx::TPValidator::declareType\(\)](#), and [mxx::TPValidator::isTypeDeclaredHere\(\)](#).

11.87.2.6 vars

```
std::unordered_set<std::string> mxx::Scope::vars
```

variable names

Definition at line 19 of file [validator.hpp](#).

Referenced by [mxx::TPValidator::declareVar\(\)](#), and [mxx::TPValidator::isVarDeclaredHere\(\)](#).

The documentation for this struct was generated from the following file:

- [src/frontend/include/validator.hpp](#)

11.88 pascal::CodeGenVisitor::Scope Struct Reference

Public Attributes

- `std::string` [name](#)
- `std::vector< ProcDeclNode * >` [nestedProcs](#)
- `std::vector< FuncDeclNode * >` [nestedFuncs](#)

11.88.1 Detailed Description

Definition at line 194 of file [icode.hpp](#).

11.88.2 Member Data Documentation

11.88.2.1 name

```
std::string pascal::CodeGenVisitor::Scope::name
```

Definition at line 195 of file [icode.hpp](#).

11.88.2.2 nestedFuncs

```
std::vector<FuncDeclNode *> pascal::CodeGenVisitor::Scope::nestedFuncs
```

Definition at line 197 of file [icode.hpp](#).

11.88.2.3 nestedProcs

```
std::vector<ProcDeclNode *> pascal::CodeGenVisitor::Scope::nestedProcs
```

Definition at line 196 of file [icode.hpp](#).

The documentation for this struct was generated from the following file:

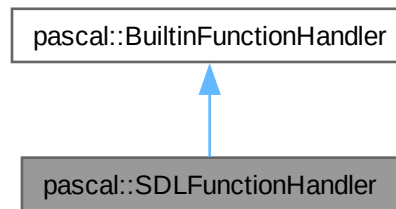
- [src/frontend/include/icode.hpp](#)

11.89 pascal::SDLFunctionHandler Class Reference

Built-in handler for SDL2 bindings.

```
#include <icode.hpp>
```

Inheritance diagram for pascal::SDLFunctionHandler:



Public Member Functions

- bool `canHandle` (const std::string &funcName) const override
Return true if this handler implements funcName.
- void `generate` (CodeGenVisitor &visitor, const std::string &funcName, const std::vector< std::unique_ptr< ASTNode > > &arguments) override
Emit code for a procedure-style call (no return value used).
- bool `generateWithResult` (CodeGenVisitor &visitor, const std::string &funcName, const std::vector< std::unique_ptr< ASTNode > > &arguments) override
Emit code for a function-style call (result pushed on eval stack); return true if handled.

Public Member Functions inherited from pascal::BuiltinFunctionHandler

- virtual `~BuiltinFunctionHandler` ()=default
- virtual `VarType getReturnType` (const std::string &funcName) const
Return the VarType produced by funcName.

11.89.1 Detailed Description

Built-in handler for SDL2 bindings.

Definition at line 155 of file `icode.hpp`.

11.89.2 Member Function Documentation

11.89.2.1 canHandle()

```
bool pascal::SDLFunctionHandler::canHandle (
    const std::string & funcName) const [override], [virtual]
```

Return true if this handler implements funcName.

Implements `pascal::BuiltinFunctionHandler`.

Definition at line 428 of file `icode_imp.cpp`.

References `pascal::sdlFunctions`, and `pascal::toLower()`.

11.89.2.2 generate()

```
void pascal::SDLFunctionHandler::generate (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [virtual]
```

Emit code for a procedure-style call (no return value used).

Implements `pascal::BuiltinFunctionHandler`.

Definition at line 432 of file `icode_imp.cpp`.

References `pascal::CodeGenVisitor::emit_invoke()`, `pascal::CodeGenVisitor::eval()`, `pascal::CodeGenVisitor::freeReg()`, `pascal::CodeGenVisitor::isReg()`, and `pascal::CodeGenVisitor::usedModules`.

11.89.2.3 generateWithResult()

```
bool pascal::SDLFunctionHandler::generateWithResult (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [virtual]
```

Emit code for a function-style call (result pushed on eval stack); return true if handled.

Reimplemented from `pascal::BuiltinFunctionHandler`.

Definition at line 653 of file `icode_imp.cpp`.

References `pascal::CodeGenVisitor::allocReg()`, `pascal::CodeGenVisitor::emit()`, `pascal::CodeGenVisitor::emit_invoke()`, `pascal::CodeGenVisitor::eval()`, `pascal::CodeGenVisitor::freeReg()`, `pascal::CodeGenVisitor::isReg()`, `pascal::CodeGenVisitor::pushV` and `pascal::CodeGenVisitor::usedModules`.

The documentation for this class was generated from the following files:

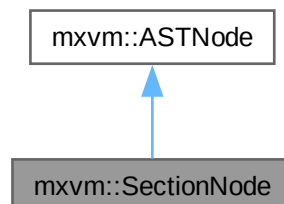
- `src/frontend/include/icode.hpp`
- `src/frontend/icode_imp.cpp`

11.90 mxvm::SectionNode Class Reference

Represents a program section (.data, .code, .module, .object).

`#include <ast.hpp>`

Inheritance diagram for `mxvm::SectionNode`:



Public Types

- enum `SectionType` { `DATA` , `CODE` , `MODULE` , `OBJECT` }
- Section type discriminator.*

Public Member Functions

- `SectionNode` (`SectionType` sectionType)
- void `addStatement` (`std::unique_ptr< ASTNode >` stmt)
- void `accept` (`ASTVisitor` &visitor) override
- `std::string toString` () const override

Public Member Functions inherited from `mxvm::ASTNode`

- virtual `~ASTNode` ()=default

Public Attributes

- [SectionType](#) *type*
- `std::vector< std::unique_ptr< ASTNode > > statements`

11.90.1 Detailed Description

Represents a program section (.data, .code, .module, .object).

Definition at line 27 of file [ast.hpp](#).

11.90.2 Member Enumeration Documentation

11.90.2.1 SectionType

```
enum mxvm::SectionNode::SectionType
```

Section type discriminator.

Enumerator

DATA	
CODE	
MODULE	
OBJECT	

Definition at line 30 of file [ast.hpp](#).

11.90.3 Constructor & Destructor Documentation

11.90.3.1 SectionNode()

```
mxvm::SectionNode::SectionNode (  
    SectionType sectionType) [inline]
```

Definition at line 39 of file [ast.hpp](#).

References [type](#).

11.90.4 Member Function Documentation

11.90.4.1 accept()

```
void mxvm::SectionNode::accept (  
    ASTVisitor & visitor) [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 27 of file [ast.cpp](#).

References [mxvm::ASTVisitor::visit\(\)](#).

11.90.4.2 addStatement()

```
void mxvm::SectionNode::addStatement (  
    std::unique_ptr< ASTNode > stmt) [inline]
```

Definition at line 41 of file [ast.hpp](#).

References [statements](#).

11.90.4.3 toString()

```
std::string mxvm::SectionNode::toString () const [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 31 of file [ast.cpp](#).

References [DATA](#), [statements](#), and [type](#).

11.90.5 Member Data Documentation

11.90.5.1 statements

`std::vector<std::unique_ptr<ASTNode> > mxvm::SectionNode::statements`

Definition at line 37 of file [ast.hpp](#).

Referenced by [addStatement\(\)](#), [mxvm::Parser::processObjectSection\(\)](#), and [toString\(\)](#).

11.90.5.2 type

`SectionType mxvm::SectionNode::type`

Definition at line 36 of file [ast.hpp](#).

Referenced by [SectionNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

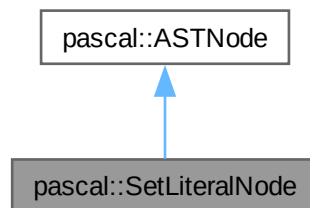
- [include/mxvm/ast.hpp](#)
- [src/ast.cpp](#)

11.91 pascal::SetLiteralNode Class Reference

AST node for a set literal [val1, val2, ...].

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::SetLiteralNode`:



Public Member Functions

- [SetLiteralNode](#) (`std::vector< std::unique_ptr< ASTNode > > elems`)
- void [accept](#) (`ASTVisitor &visitor`) override
Accept an [ASTVisitor](#) (double-dispatch).
- `std::string` [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual `~ASTNode` ()=default
- virtual void [print](#) (`std::ostream &out`, `int indent=0`) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (`int line`)
Set the originating source line number.
- `int` [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- `std::vector< std::unique_ptr< ASTNode > > elements`
set elements

11.91.1 Detailed Description

AST node for a set literal [val1, val2, ...].

Definition at line 705 of file [ast.hpp](#).

11.91.2 Constructor & Destructor Documentation

11.91.2.1 SetLiteralNode()

```
pascal::SetLiteralNode::SetLiteralNode (
    std::vector< std::unique_ptr< ASTNode > > elems) [inline]
```

Definition at line 709 of file [ast.hpp](#).

References [elements](#).

11.91.3 Member Function Documentation

11.91.3.1 accept()

```
void pascal::SetLiteralNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 454 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.91.3.2 toString()

```
std::string pascal::SetLiteralNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 713 of file [ast.hpp](#).

11.91.4 Member Data Documentation

11.91.4.1 elements

```
std::vector<std::unique_ptr<ASTNode> > pascal::SetLiteralNode::elements
set elements
```

Definition at line 707 of file [ast.hpp](#).

Referenced by [SetLiteralNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

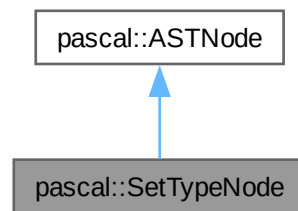
- `src/frontend/include/ast.hpp`
- `src/frontend/ast.cpp`

11.92 pascal::SetTypeNode Class Reference

AST node for a set type declaration (set of <base type>).

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::SetTypeNode`:



Public Member Functions

- [SetTypeNode](#) (const std::string &base)
- void [accept](#) (ASTVisitor &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [baseTypeName](#)
the ordinal base type of the set

11.92.1 Detailed Description

AST node for a set type declaration (set of <base type>).
Definition at line 716 of file [ast.hpp](#).

11.92.2 Constructor & Destructor Documentation

11.92.2.1 SetTypeNode()

```
pascal::SetTypeNode::SetTypeNode (
    const std::string & base) [inline]
```

Definition at line 720 of file [ast.hpp](#).

References [baseTypeName](#).

11.92.3 Member Function Documentation

11.92.3.1 accept()

```
void pascal::SetTypeNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 458 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.92.3.2 toString()

```
std::string pascal::SetTypeNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 723 of file [ast.hpp](#).

References [baseTypeName](#).

11.92.4 Member Data Documentation

11.92.4.1 baseTypeName

```
std::string pascal::SetTypeNode::baseTypeName
```

the ordinal base type of the set

Definition at line 718 of file [ast.hpp](#).

Referenced by [SetTypeNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

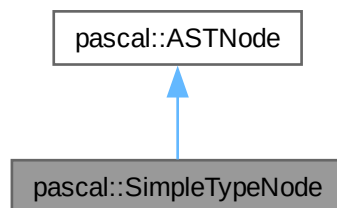
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.93 pascal::SimpleTypeNode Class Reference

AST node for a simple (named) type reference.

```
#include <ast.hpp>
```

Inheritance diagram for pascal::SimpleTypeNode:



Public Member Functions

- [SimpleTypeNode](#) (const std::string &[typeName](#))
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [typeName](#)
the referenced type name

11.93.1 Detailed Description

AST node for a simple (named) type reference.
Definition at line 264 of file [ast.hpp](#).

11.93.2 Constructor & Destructor Documentation

11.93.2.1 SimpleTypeNode()

```
pascal::SimpleTypeNode::SimpleTypeNode (  
    const std::string & typeName) [inline]
```

Definition at line 268 of file [ast.hpp](#).

References [typeName](#).

11.93.3 Member Function Documentation

11.93.3.1 accept()

```
void pascal::SimpleTypeNode::accept (  
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 406 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.93.3.2 toString()

```
std::string pascal::SimpleTypeNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 271 of file [ast.hpp](#).

References [typeName](#).

11.93.4 Member Data Documentation

11.93.4.1 typeName

```
std::string pascal::SimpleTypeNode::typeName
```

the referenced type name

Definition at line 266 of file [ast.hpp](#).

Referenced by [SimpleTypeNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.94 mxvm::Stack Class Reference

Runtime operand stack for the MXVM interpreter.

#include <icode.hpp>

Public Member Functions

- [Stack](#) ()=default
- void [push](#) (const [StackValue](#) &value)
Push a value onto the stack.
- [StackValue pop](#) ()
Pop and return the top value.
- bool [empty](#) () const
Check whether the stack is empty.
- size_t [size](#) () const
Return the number of elements on the stack.
- [StackValue & operator\[\]](#) (size_t index)
Random access into the stack.

Private Attributes

- std::vector< [StackValue](#) > [data](#)

11.94.1 Detailed Description

Runtime operand stack for the MXVM interpreter.

Definition at line 23 of file [icode.hpp](#).

11.94.2 Constructor & Destructor Documentation

11.94.2.1 Stack()

```
mxvm::Stack::Stack () [default]
```

11.94.3 Member Function Documentation

11.94.3.1 empty()

```
bool mxvm::Stack::empty () const [inline], [nodiscard]
```

Check whether the stack is empty.

Definition at line 47 of file [icode.hpp](#).

References [data](#).

11.94.3.2 operator[]()

```
StackValue & mxvm::Stack::operator[] (
    size_t index) [inline]
```

Random access into the stack.

Parameters

<i>index</i>	Zero-based index from the bottom
--------------	----------------------------------

Returns

Reference to the stack element

Exceptions

<i>mx::Exception</i>	if index is out of bounds
--------------------------------------	---------------------------

Definition at line 57 of file [icode.hpp](#).
References [data](#).

11.94.3.3 pop()

[*StackValue*](#) mxvm::Stack::pop () [inline], [nodiscard]
Pop and return the top value.

Returns

The top stack value

Exceptions

<i>mx::Exception</i>	on stack underflow
--------------------------------------	--------------------

Definition at line 38 of file [icode.hpp](#).
References [data](#).

11.94.3.4 push()

```
void mxvm::Stack::push (
    const StackValue & value) [inline]
```

Push a value onto the stack.

Parameters

<i>value</i>	Integer or pointer value to push
--------------	----------------------------------

Definition at line 30 of file [icode.hpp](#).
References [data](#).

11.94.3.5 size()

```
size_t mxvm::Stack::size () const [inline], [nodiscard]
```

Return the number of elements on the stack.
Definition at line 50 of file [icode.hpp](#).
References [data](#).

11.94.4 Member Data Documentation**11.94.4.1 data**

```
std::vector<StackValue> mxvm::Stack::data [private]
```

Definition at line 64 of file [icode.hpp](#).
Referenced by [empty\(\)](#), [operator\[\]\(\)](#), [pop\(\)](#), [push\(\)](#), and [size\(\)](#).
The documentation for this class was generated from the following file:

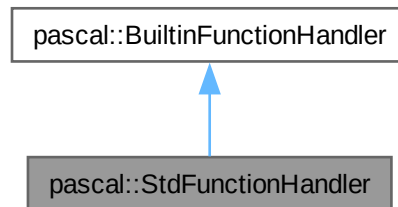
- [include/mxvm/icode.hpp](#)

11.95 pascal::StdFunctionHandler Class Reference

Built-in handler for standard library functions (chr, ord, inc, dec, etc.).

#include <icode.hpp>

Inheritance diagram for pascal::StdFunctionHandler:



Public Member Functions

- bool `canHandle` (const std::string &funcName) const override
Return true if this handler implements funcName.
- void `generate` (CodeGenVisitor &visitor, const std::string &funcName, const std::vector< std::unique_ptr< ASTNode > > &arguments) override
Emit code for a procedure-style call (no return value used).
- bool `generateWithResult` (CodeGenVisitor &visitor, const std::string &funcName, const std::vector< std::unique_ptr< ASTNode > > &arguments) override
Emit code for a function-style call (result pushed on eval stack); return true if handled.

Public Member Functions inherited from pascal::BuiltinFunctionHandler

- virtual `~BuiltinFunctionHandler` ()=default
- virtual `VarType getReturnType` (const std::string &funcName) const
Return the VarType produced by funcName.

11.95.1 Detailed Description

Built-in handler for standard library functions (chr, ord, inc, dec, etc.).

Definition at line 139 of file `icode.hpp`.

11.95.2 Member Function Documentation

11.95.2.1 canHandle()

```
bool pascal::StdFunctionHandler::canHandle (
    const std::string & funcName) const [override], [virtual]
```

Return true if this handler implements funcName.

Implements `pascal::BuiltinFunctionHandler`.

Definition at line 224 of file `icode_imp.cpp`.

References `pascal::stdFunctions`, and `pascal::toLower()`.

11.95.2.2 generate()

```
void pascal::StdFunctionHandler::generate (
    CodeGenVisitor & visitor,
```

```
const std::string & funcName,
const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [virtual]
```

Emit code for a procedure-style call (no return value used).

Implements [pascal::BuiltinFunctionHandler](#).

Definition at line 228 of file [icode_imp.cpp](#).

References [pascal::CodeGenVisitor::allocReg\(\)](#), [pascal::CodeGenVisitor::emit\(\)](#), [pascal::CodeGenVisitor::emit2\(\)](#), [pascal::CodeGenVisitor::emit4\(\)](#), [pascal::CodeGenVisitor::emit_invoke\(\)](#), [pascal::CodeGenVisitor::ensurePtrBase\(\)](#), [pascal::CodeGenVisitor::eval\(\)](#), [pascal::CodeGenVisitor::freeReg\(\)](#), [pascal::CodeGenVisitor::isReg\(\)](#), and [pascal::CodeGenVisitor::usedModules](#).

11.95.2.3 generateWithResult()

```
bool pascal::StdFunctionHandler::generateWithResult (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [virtual]
```

Emit code for a function-style call (result pushed on eval stack); return true if handled.

Reimplemented from [pascal::BuiltinFunctionHandler](#).

Definition at line 292 of file [icode_imp.cpp](#).

References [pascal::CodeGenVisitor::allocFloatReg\(\)](#), [pascal::CodeGenVisitor::allocReg\(\)](#), [pascal::DOUBLE](#), [pascal::CodeGenVisitor::emit\(\)](#), [pascal::CodeGenVisitor::emit2\(\)](#), [pascal::CodeGenVisitor::emit_invoke\(\)](#), [pascal::CodeGenVisitor::eval\(\)](#), [pascal::CodeGenVisitor::freeFloatReg\(\)](#), [pascal::CodeGenVisitor::freeReg\(\)](#), [pascal::CodeGenVisitor::getExpressionType\(\)](#), [pascal::CodeGenVisitor::isFloatReg\(\)](#), [pascal::CodeGenVisitor::isParmReg\(\)](#), [pascal::CodeGenVisitor::isReg\(\)](#), [pascal::CodeGenVisitor::pushValue\(\)](#), and [pascal::CodeGenVisitor::usedModules](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/icode.hpp](#)
- [src/frontend/icode_imp.cpp](#)

11.96 scan::buffer::StringBuffer< Ch, String > Class Template Reference

Character-by-character string buffer with line/column tracking.

```
#include <string_buffer.hpp>
```

Public Types

- using [ch_type](#) = Ch
character type alias
- using [string_type](#) = String
string type alias

Public Member Functions

- [StringBuffer](#) (const [string_type](#) &buf, const std::string &filename="unknown")
Construct a string buffer.
- std::optional< [ch_type](#) > [getch](#) ()
Consume and return the next character, advancing the position.
- std::optional< [ch_type](#) > [curch](#) ()
Return the current character without advancing.
- std::optional< [ch_type](#) > [peekch](#) (int num=1)
Peek ahead by num positions without advancing.
- std::optional< [ch_type](#) > [prevch](#) ()
Return the previously consumed character.
- std::optional< [ch_type](#) > [forward_step](#) (int num=1)
Advance the position by num characters.

- `std::optional< ch\_type > backward\_step` (int num=1)
Move the position backward by num characters.
- `std::pair< uint64_t, uint64_t > cur\_line` ()
Get the current (line, column) position.
- `std::string cur\_file` () const
Get the current source filename.
- `void process\_line\_directive` (uint64_t newLine, const std::string &newFile)
Process a #line directive to update position tracking.
- `bool eof` (uint64_t [pos](#)=0)
Check if end-of-buffer has been reached.
- `void reset` (uint64_t [pos](#)=0)
Reset the buffer position.

Private Attributes

- `string_type buffer\_`
the source text with appended sentinel
- `uint64_t index`
current read position
- `uint64_t currentLine`
current line number (1-based)
- `uint64_t currentColumn`
current column number (1-based)
- `std::string currentFile`
current source filename

11.96.1 Detailed Description

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
class scan::buffer::StringBuffer< Ch, String >
```

Character-by-character string buffer with line/column tracking.

Template Parameters

<i>Ch</i>	Character type (default: char)
<i>String</i>	String type (default: std::basic_string<Ch>)

Wraps a source string and provides sequential character access with automatic line and column tracking for error reporting. Supports #line directive processing.

Definition at line 27 of file [string_buffer.hpp](#).

11.96.2 Member Typedef Documentation

11.96.2.1 ch_type

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
using scan::buffer::StringBuffer< Ch, String >::ch_type = Ch
```

character type alias

Definition at line 29 of file [string_buffer.hpp](#).

11.96.2.2 string_type

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
using scan::buffer::StringBuffer< Ch, String >::string_type = String
```

string type alias

Definition at line 30 of file [string_buffer.hpp](#).

11.96.3 Constructor & Destructor Documentation

11.96.3.1 StringBuffer()

```
template<typename Ch, typename String>
scan::buffer::StringBuffer< Ch, String >::StringBuffer (
    const string_type & buf,
    const std::string & filename = "unknown")
```

Construct a string buffer.

Parameters

<i>buf</i>	Source text
<i>filename</i>	Source filename for error messages

Definition at line 88 of file [string_buffer.hpp](#).

References [buffer_](#), [currentColumn](#), [currentFile](#), [currentLine](#), and [index](#).

11.96.4 Member Function Documentation

11.96.4.1 backward_step()

```
template<typename Ch, typename String>
std::optional< Ch > scan::buffer::StringBuffer< Ch, String >::backward_step (
    int num = 1)
```

Move the position backward by num characters.

Parameters

<i>num</i>	Number of characters to step back (default: 1)
------------	--

Definition at line 140 of file [string_buffer.hpp](#).

References [buffer_](#), [currentColumn](#), [currentLine](#), and [index](#).

11.96.4.2 cur_file()

```
template<typename Ch, typename String>
std::string scan::buffer::StringBuffer< Ch, String >::cur_file () const
```

Get the current source filename.

Definition at line 160 of file [string_buffer.hpp](#).

References [currentFile](#).

11.96.4.3 cur_line()

```
template<typename Ch, typename String>
std::pair< uint64_t, uint64_t > scan::buffer::StringBuffer< Ch, String >::cur_line ()
```

Get the current (line, column) position.

Definition at line 153 of file [string_buffer.hpp](#).

References [currentColumn](#), [currentLine](#), and [index](#).

11.96.4.4 curch()

```
template<typename Ch, typename String>
std::optional< Ch > scan::buffer::StringBuffer< Ch, String >::curch ()
```

Return the current character without advancing.

Definition at line 110 of file [string_buffer.hpp](#).

References [buffer_](#), and [index](#).

11.96.4.5 eof()

```
template<typename Ch, typename String>
bool scan::buffer::StringBuffer< Ch, String >::eof (
    uint64_t pos = 0)
```

Check if end-of-buffer has been reached.

Parameters

<i>pos</i>	Additional offset from current position
------------	---

Definition at line 173 of file [string_buffer.hpp](#).

References [buffer_](#), [index](#), and [pos\(\)](#).

11.96.4.6 forward_step()

```
template<typename Ch, typename String>
std::optional< Ch > scan::buffer::StringBuffer< Ch, String >::forward_step (
    int num = 1)
```

Advance the position by num characters.

Parameters

<i>num</i>	Number of characters to skip (default: 1)
------------	---

Definition at line 131 of file [string_buffer.hpp](#).

References [buffer_](#), and [index](#).

11.96.4.7 getch()

```
template<typename Ch, typename String>
std::optional< Ch > scan::buffer::StringBuffer< Ch, String >::getch ()
```

Consume and return the next character, advancing the position.

Definition at line 95 of file [string_buffer.hpp](#).

References [buffer_](#), [currentColumn](#), [currentLine](#), and [index](#).

11.96.4.8 peekch()

```
template<typename Ch, typename String>
std::optional< Ch > scan::buffer::StringBuffer< Ch, String >::peekch (
    int num = 1)
```

Peek ahead by num positions without advancing.

Parameters

<i>num</i>	Number of positions to look ahead (default: 1)
------------	--

Definition at line 117 of file [string_buffer.hpp](#).

References [buffer_](#), and [index](#).

11.96.4.9 prevch()

```
template<typename Ch, typename String>
std::optional< Ch > scan::buffer::StringBuffer< Ch, String >::prevch ()
```

Return the previously consumed character.

Definition at line 124 of file [string_buffer.hpp](#).

References [buffer_](#), and [index](#).

11.96.4.10 process_line_directive()

```
template<typename Ch, typename String>
void scan::buffer::StringBuffer< Ch, String >::process_line_directive (
    uint64_t newLine,
    const std::string & newFile)
```

Process a #line directive to update position tracking.

Parameters

<i>newLine</i>	New line number
<i>newFile</i>	New filename

Definition at line 165 of file [string_buffer.hpp](#).

References [currentColumn](#), [currentFile](#), and [currentLine](#).

11.96.4.11 reset()

```
template<typename Ch, typename String>
void scan::buffer::StringBuffer< Ch, String >::reset (
    uint64_t pos = 0)
```

Reset the buffer position.

Parameters

<i>pos</i>	New position (default: 0)
------------	---------------------------

Definition at line 178 of file [string_buffer.hpp](#).

References [currentColumn](#), [currentLine](#), [index](#), and [pos\(\)](#).

11.96.5 Member Data Documentation

11.96.5.1 buffer_

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
string_type scan::buffer::StringBuffer< Ch, String >::buffer_ [private]
```

the source text with appended sentinel

Definition at line 80 of file [string_buffer.hpp](#).

Referenced by [StringBuffer\(\)](#), [backward_step\(\)](#), [curch\(\)](#), [eof\(\)](#), [forward_step\(\)](#), [getch\(\)](#), [peekch\(\)](#), and [prevch\(\)](#).

11.96.5.2 currentColumn

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
uint64_t scan::buffer::StringBuffer< Ch, String >::currentColumn [private]
```

current column number (1-based)

Definition at line 83 of file [string_buffer.hpp](#).

Referenced by [StringBuffer\(\)](#), [backward_step\(\)](#), [cur_line\(\)](#), [getch\(\)](#), [process_line_directive\(\)](#), and [reset\(\)](#).

11.96.5.3 currentFile

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
std::string scan::buffer::StringBuffer< Ch, String >::currentFile [private]
```

current source filename

Definition at line 84 of file [string_buffer.hpp](#).

Referenced by [StringBuffer\(\)](#), [cur_file\(\)](#), and [process_line_directive\(\)](#).

11.96.5.4 currentLine

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
uint64_t scan::buffer::StringBuffer< Ch, String >::currentLine [private]
```

current line number (1-based)

Definition at line 82 of file [string_buffer.hpp](#).

Referenced by [StringBuffer\(\)](#), [backward_step\(\)](#), [cur_line\(\)](#), [getch\(\)](#), [process_line_directive\(\)](#), and [reset\(\)](#).

11.96.5.5 index

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
uint64_t scan::buffer::StringBuffer< Ch, String >::index [private]
```

current read position

Definition at line 81 of file [string_buffer.hpp](#).

Referenced by [StringBuffer\(\)](#), [backward_step\(\)](#), [cur_line\(\)](#), [curch\(\)](#), [eof\(\)](#), [forward_step\(\)](#), [getch\(\)](#), [peekch\(\)](#), [prevch\(\)](#), and [reset\(\)](#).

The documentation for this class was generated from the following file:

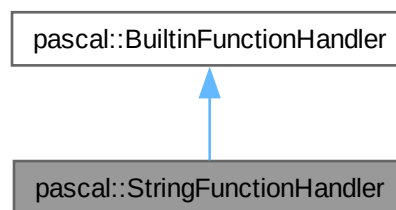
- [src/scanner/include/scanner/string_buffer.hpp](#)

11.97 pascal::StringFunctionHandler Class Reference

Built-in handler for string library functions (length, copy, concat, etc.).

```
#include <icode.hpp>
```

Inheritance diagram for pascal::StringFunctionHandler:



Private Member Functions

- bool [canHandle](#) (const std::string &funcName) const override
Return true if this handler implements funcName.
- void [generate](#) ([CodeGenVisitor](#) &visitor, const std::string &funcName, const std::vector< std::unique_ptr< [ASTNode](#) > > &arguments) override
Emit code for a procedure-style call (no return value used).
- bool [generateWithResult](#) ([CodeGenVisitor](#) &visitor, const std::string &funcName, const std::vector< std::unique_ptr< [ASTNode](#) > > &arguments) override
Emit code for a function-style call (result pushed on eval stack); return true if handled.
- [VarType](#) [getReturnType](#) (const std::string &funcName) const override
Return the VarType produced by funcName.

Additional Inherited Members

Public Member Functions inherited from pascal::BuiltinFunctionHandler

- virtual [~BuiltinFunctionHandler](#) ()=default

11.97.1 Detailed Description

Built-in handler for string library functions (length, copy, concat, etc.).
Definition at line 147 of file [icode.hpp](#).

11.97.2 Member Function Documentation

11.97.2.1 canHandle()

```
bool pascal::StringFunctionHandler::canHandle (
    const std::string & funcName) const [override], [private], [virtual]
```

Return true if this handler implements funcName.

Implements [pascal::BuiltinFunctionHandler](#).

Definition at line 897 of file [icode_imp.cpp](#).

References [pascal::toLowerCase\(\)](#).

11.97.2.2 generate()

```
void pascal::StringFunctionHandler::generate (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [private],
[virtual]
```

Emit code for a procedure-style call (no return value used).

Implements [pascal::BuiltinFunctionHandler](#).

Definition at line 912 of file [icode_imp.cpp](#).

References [generateWithResult\(\)](#).

11.97.2.3 generateWithResult()

```
bool pascal::StringFunctionHandler::generateWithResult (
    CodeGenVisitor & visitor,
    const std::string & funcName,
    const std::vector< std::unique_ptr< ASTNode > > & arguments) [override], [private],
[virtual]
```

Emit code for a function-style call (result pushed on eval stack); return true if handled.

Reimplemented from [pascal::BuiltinFunctionHandler](#).

Definition at line 918 of file [icode_imp.cpp](#).

References [pascal::CodeGenVisitor::allocReg\(\)](#), [pascal::CodeGenVisitor::allocTempPtr\(\)](#), [pascal::CodeGenVisitor::emit\(\)](#), [pascal::CodeGenVisitor::emit_invoke\(\)](#), [pascal::CodeGenVisitor::eval\(\)](#), [pascal::CodeGenVisitor::freeReg\(\)](#), [pascal::CodeGenVisitor::isParmReg\(\)](#), [pascal::CodeGenVisitor::isReg\(\)](#), [pascal::CodeGenVisitor::markAllocatedPtr\(\)](#), [pascal::CodeGenVisitor::pushValue\(\)](#), and [pascal::CodeGenVisitor::usedModules](#).

Referenced by [generate\(\)](#).

11.97.2.4 getReturnType()

```
VarType pascal::StringFunctionHandler::getReturnType (
    const std::string & funcName) const [override], [private], [virtual]
```

Return the [VarType](#) produced by funcName.

Reimplemented from [pascal::BuiltinFunctionHandler](#).

Definition at line 903 of file [icode_imp.cpp](#).

References [pascal::INT](#), [pascal::PTR](#), [pascal::toLowerCase\(\)](#), and [pascal::UNKNOWN](#).

The documentation for this class was generated from the following files:

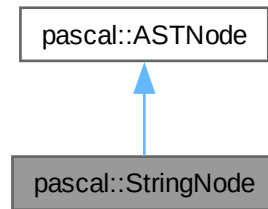
- [src/frontend/include/icode.hpp](#)
- [src/frontend/icode_imp.cpp](#)

11.98 pascal::StringNode Class Reference

AST node for a string literal.

```
#include <ast.hpp>
```

Inheritance diagram for pascal::StringNode:



Public Member Functions

- [StringNode](#) (const std::string &val)
- void [accept](#) (ASTVisitor &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [value](#)
the string contents (without quotes)

11.98.1 Detailed Description

AST node for a string literal.

Definition at line 538 of file [ast.hpp](#).

11.98.2 Constructor & Destructor Documentation

11.98.2.1 StringNode()

```
pascal::StringNode::StringNode (
    const std::string & val) [inline]
```

Definition at line 542 of file [ast.hpp](#).

References [value](#).

11.98.3 Member Function Documentation

11.98.3.1 `accept()`

```
void pascal::StringNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 255 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.98.3.2 `toString()`

```
std::string pascal::StringNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 259 of file [ast.cpp](#).

References [value](#).

11.98.4 Member Data Documentation

11.98.4.1 `value`

```
std::string pascal::StringNode::value
```

the string contents (without quotes)

Definition at line 540 of file [ast.hpp](#).

Referenced by [StringNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.99 `scan::token::Token< Ch, String >` Class Template Reference

A scanned token with type, value, and source position.

```
#include <string_buffer.hpp>
```

Public Types

- using [ch_type](#) = Ch
character type alias
- using [string_type](#) = String
string type alias

Public Member Functions

- [Token](#) ()
Default constructor — creates a null token at line 1, col 1.
- [Token](#) (const [types::TokenType](#) &t)
Construct a token with a given type.
- [Token](#) (const Token< Ch, String > &t)
Copy constructor.
- [Token](#) (Token< Ch, String > &&t)
Move constructor.
- [Token](#)< Ch, String > & [operator=](#) (const [types::TokenType](#) &type)
Assign a new token type, keeping the current value.
- [Token](#)< Ch, String > & [operator=](#) (const [Token](#)< Ch, String > &t)

- Copy assignment.*
- [Token](#)< Ch, String > & [operator=](#) ([Token](#)< Ch, String > &&t)
- Move assignment.*
- void [setToken](#) (const [types::TokenType](#) &type, const String &value)
- Set the token's type and value.*
- [string_type](#) [getTokenValue](#) () const
- Get the token's text value.*
- [types::TokenType](#) [getTokenType](#) () const
- Get the token's type classification.*
- void [print](#) (std::ostream &out)
- Print the token value and type to a stream.*
- const std::pair< uint64_t, uint64_t > [get_pos](#) () const
- Get the (line, column) source position.*
- void [set_pos](#) (const std::pair< uint64_t, uint64_t > &p)
- Set the source position from a (line, column) pair.*
- void [set_filename](#) (const std::string &filename)
- Set the source filename for this token.*
- std::string [get_filename](#) () const
- Get the source filename for this token.*
- uint64_t [getLine](#) () const
- Get the source line number.*
- uint64_t [getCol](#) () const
- Get the source column number.*

Private Attributes

- [types::TokenType](#) type
- [string_type](#) value
- uint64_t [line](#) = 1
- uint64_t [col](#) = 1
- std::string [filename](#)

11.99.1 Detailed Description

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
class scan::token::Token< Ch, String >
```

A scanned token with type, value, and source position.

Template Parameters

<i>Ch</i>	Character type
<i>String</i>	String type for token values

Definition at line 275 of file [string_buffer.hpp](#).

11.99.2 Member Typedef Documentation

11.99.2.1 ch_type

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
using scan::token::Token< Ch, String >::ch_type = Ch
```

character type alias

Definition at line 277 of file [string_buffer.hpp](#).

11.99.2.2 string_type

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
using scan::token::Token< Ch, String >::string_type = String
```

string type alias

Definition at line 278 of file [string_buffer.hpp](#).

11.99.3 Constructor & Destructor Documentation

11.99.3.1 Token() [1/4]

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
scan::token::Token< Ch, String >::Token () [inline]
```

Default constructor — creates a null token at line 1, col 1.

Definition at line 280 of file [string_buffer.hpp](#).

Referenced by [operator=\(\)](#), [operator=\(\)](#), and [operator=\(\)](#).

11.99.3.2 Token() [2/4]

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
scan::token::Token< Ch, String >::Token (
    const types::TokenType & t) [inline]
```

Construct a token with a given type.

Definition at line 282 of file [string_buffer.hpp](#).

11.99.3.3 Token() [3/4]

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
scan::token::Token< Ch, String >::Token (
    const Token< Ch, String > & t) [inline]
```

Copy constructor.

Definition at line 284 of file [string_buffer.hpp](#).

11.99.3.4 Token() [4/4]

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
scan::token::Token< Ch, String >::Token (
    Token< Ch, String > && t) [inline]
```

Move constructor.

Definition at line 286 of file [string_buffer.hpp](#).

11.99.4 Member Function Documentation

11.99.4.1 get_filename()

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
std::string scan::token::Token< Ch, String >::get_filename () const [inline]
```

Get the source filename for this token.

Definition at line 312 of file [string_buffer.hpp](#).

11.99.4.2 get_pos()

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
const std::pair< uint64_t, uint64_t > scan::token::Token< Ch, String >::get_pos () const
[inline]
```

Get the (line, column) source position.

Definition at line 306 of file [string_buffer.hpp](#).

11.99.4.3 getCol()

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
uint64_t scan::token::Token< Ch, String >::getCol () const [inline]
```

Get the source column number.

Definition at line 316 of file [string_buffer.hpp](#).

11.99.4.4 getLine()

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
uint64_t scan::token::Token< Ch, String >::getLine () const [inline]
```

Get the source line number.

Definition at line 314 of file [string_buffer.hpp](#).

Referenced by [mxs::TPValidator::failAt\(\)](#).

11.99.4.5 getTokenType()

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
types::TokenType scan::token::Token< Ch, String >::getTokenType () const [inline]
```

Get the token's type classification.

Definition at line 302 of file [string_buffer.hpp](#).

11.99.4.6 getTokenValue()

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
string_type scan::token::Token< Ch, String >::getTokenValue () const [inline]
```

Get the token's text value.

Definition at line 300 of file [string_buffer.hpp](#).

11.99.4.7 operator=() [1/3]

```
template<typename Ch, typename String>
Token< Ch, String > & scan::token::Token< Ch, String >::operator= (
    const Token< Ch, String > & t)
```

Copy assignment.

Definition at line 337 of file [string_buffer.hpp](#).

References [Token\(\)](#), [filename](#), [setToken\(\)](#), [type](#), and [value](#).

11.99.4.8 operator=() [2/3]

```
template<typename Ch, typename String>
Token< Ch, String > & scan::token::Token< Ch, String >::operator= (
    const types::TokenType & type)
```

Assign a new token type, keeping the current value.

Definition at line 332 of file [string_buffer.hpp](#).

References [Token\(\)](#), [setToken\(\)](#), [type](#), and [value](#).

11.99.4.9 operator=() [3/3]

```
template<typename Ch, typename String>
Token< Ch, String > & scan::token::Token< Ch, String >::operator= (
    Token< Ch, String > && t)
```

Move assignment.

Definition at line 343 of file [string_buffer.hpp](#).

References [Token\(\)](#), and [setToken\(\)](#).

11.99.4.10 print()

```
template<typename Ch, typename String>
void scan::token::Token< Ch, String >::print (
```

```
std::ostream & out)
```

Print the token value and type to a stream.

Definition at line 349 of file [string_buffer.hpp](#).

References [types::print_type_TokenType\(\)](#), [type](#), and [value](#).

11.99.4.11 set_filename()

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
void scan::token::Token< Ch, String >::set_filename (
    const std::string & filename) [inline]
```

Set the source filename for this token.

Definition at line 310 of file [string_buffer.hpp](#).

Referenced by [scan::Scanner::grabld\(\)](#).

11.99.4.12 set_pos()

```
template<typename Ch, typename String>
void scan::token::Token< Ch, String >::set_pos (
    const std::pair< uint64_t, uint64_t > & p)
```

Set the source position from a (line, column) pair.

Definition at line 355 of file [string_buffer.hpp](#).

References [col](#), and [line](#).

Referenced by [scan::Scanner::grabld\(\)](#).

11.99.4.13 setToken()

```
template<typename Ch, typename String>
void scan::token::Token< Ch, String >::setToken (
    const types::TokenType & type,
    const String & value)
```

Set the token's type and value.

Parameters

type	New token type
value	New token value string

Definition at line 326 of file [string_buffer.hpp](#).

References [type](#), and [value](#).

Referenced by [scan::Scanner::grabld\(\)](#), [operator=\(\)](#), [operator=\(\)](#), and [operator=\(\)](#).

11.99.5 Member Data Documentation

11.99.5.1 col

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
uint64_t scan::token::Token< Ch, String >::col = 1 [private]
```

Definition at line 321 of file [string_buffer.hpp](#).

Referenced by [set_pos\(\)](#).

11.99.5.2 filename

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
std::string scan::token::Token< Ch, String >::filename [private]
```

Definition at line 322 of file [string_buffer.hpp](#).

Referenced by [operator=\(\)](#).

11.99.5.3 line

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
uint64_t scan::token::Token< Ch, String >::line = 1 [private]
```

Definition at line 321 of file [string_buffer.hpp](#).

Referenced by [set_pos\(\)](#).

11.99.5.4 type

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
types::TokenType scan::token::Token< Ch, String >::type [private]
```

Definition at line 319 of file [string_buffer.hpp](#).

Referenced by [operator=\(\)](#), [operator=\(\)](#), [print\(\)](#), and [setToken\(\)](#).

11.99.5.5 value

```
template<typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>>
string_type scan::token::Token< Ch, String >::value [private]
```

Definition at line 320 of file [string_buffer.hpp](#).

Referenced by [operator=\(\)](#), [operator=\(\)](#), [print\(\)](#), and [setToken\(\)](#).

The documentation for this class was generated from the following file:

- [src/scanner/include/scanner/string_buffer.hpp](#)

11.100 scan::token::TokenMap< Ch, MAX_SIZE > Class Template Reference

Maps characters to their lexical category (letter, digit, symbol, etc.).

```
#include <string_buffer.hpp>
```

Public Member Functions

- [TokenMap \(\)](#)
Construct the map with default ASCII character classifications.
- [std::optional< types::CharType > lookup_int8 \(int8_t c\)](#)
Look up the character type for an 8-bit character.

Private Attributes

- [std::unordered_map< Ch, types::CharType > token_map](#)

11.100.1 Detailed Description

```
template<typename Ch, int MAX_SIZE = 256>
class scan::token::TokenMap< Ch, MAX_SIZE >
```

Maps characters to their lexical category (letter, digit, symbol, etc.).

Template Parameters

<i>Ch</i>	Character type
<i>MAX_SIZE</i>	Maximum character code value to classify

Definition at line 195 of file [string_buffer.hpp](#).

11.100.2 Constructor & Destructor Documentation

11.100.2.1 TokenMap()

```
template<typename Ch, int MAX_CHARS>
scan::token::TokenMap< Ch, MAX_CHARS >::TokenMap ()
```

Construct the map with default ASCII character classifications.
Definition at line 211 of file [string_buffer.hpp](#).
References [token_map](#).

11.100.3 Member Function Documentation

11.100.3.1 lookup_int8()

```
template<typename Ch, int MAX_CHARS>
std::optional< types::CharType > scan::token::TokenMap< Ch, MAX_CHARS >::lookup_int8 (
    int8_t c)
```

Look up the character type for an 8-bit character.

Parameters

<code>c</code>	Character to classify
----------------	-----------------------

Returns

The [CharType](#), or nullopt if not mapped

Definition at line 260 of file [string_buffer.hpp](#).
References [token_map](#).

11.100.4 Member Data Documentation

11.100.4.1 token_map

```
template<typename Ch, int MAX_SIZE = 256>
std::unordered_map<Ch, types::CharType> scan::token::TokenMap< Ch, MAX_SIZE >::token_map
[private]
```

Definition at line 207 of file [string_buffer.hpp](#).
Referenced by [TokenMap\(\)](#), and [lookup_int8\(\)](#).
The documentation for this class was generated from the following file:

- [src/scanner/include/scanner/string_buffer.hpp](#)

11.101 mxx::TPValidator Class Reference

Semantic validator for Pascal programs.
`#include <validator.hpp>`

Public Member Functions

- [TPValidator](#) (const std::string &source_)
Construct a validator for the given source.
- bool [validate](#) (const std::string &name)
Run semantic validation.

Private Member Functions

- bool [isPascalKeyword](#) (const std::string &s) const
Check if a string is a Pascal reserved keyword.

- void `pushScope` ()
Push a new scope onto the scope stack.
- void `popScope` ()
Pop the current scope from the stack.
- `Scope * currentScope` ()
Get the current (top) scope.
- const `Scope * currentScope` () const
Get the current scope (const).
- bool `isVariableInCurrentScope` (const std::string &name) const
Check if a variable is declared in the current scope.
- std::string `currentScopeName` () const
Return a display name for the current scope.
- bool `isVarDeclaredHere` (const std::string &name) const
Check if a variable is declared in the innermost scope.
- bool `isTypeDeclaredHere` (const std::string &name) const
Check if a type is declared in the innermost scope.
- bool `isFuncDeclaredHere` (const std::string &name) const
Check if a function is declared in the innermost scope.
- bool `isProcDeclaredHere` (const std::string &name) const
Check if a procedure is declared in the innermost scope.
- bool `isParamDeclaredHere` (const std::string &name) const
Check if a parameter is declared in the innermost scope.
- void `declareVar` (const std::string &name, const `scan::TToken` *at)
Declare a variable in the current scope.
- void `declareConst` (const std::string &name, const `scan::TToken` *at)
Declare a constant in the current scope.
- void `declareFunc` (const std::string &name, const `scan::TToken` *at)
Declare a function in the current scope.
- void `declareProc` (const std::string &name, const `scan::TToken` *at)
Declare a procedure in the current scope.
- void `declareParam` (const std::string &name, const `scan::TToken` *at)
Declare a parameter in the current scope.
- void `checkVar` (const std::string &name, const `scan::TToken` *at)
Assert a variable is declared and accessible in scope.
- void `checkVarOrConst` (const std::string &name, const `scan::TToken` *at)
Assert a name is a variable or constant in scope.
- void `checkConstOnly` (const std::string &name, const `scan::TToken` *at)
Assert a name is a constant (used in const expressions).
- void `parseConstExpr` (const std::unordered_set< std::string > &stops, bool constOnly)
Parse and validate a constant expression.
- bool `isBuiltinType` () const
Check if the current token is a built-in type name.
- void `declareType` (const std::string &name, const `scan::TToken` *at)
Declare a user-defined type in the current scope.
- void `checkType` (const std::string &name, const `scan::TToken` *at)
Assert a type name is declared and accessible.
- bool `isBuiltinConst` (const std::string &name) const
Check if a name is a built-in constant (true, false, maxint).
- bool `next` ()
Advance to the next token.
- bool `match` (const std::string &s) const

- Check if the current token value matches a string.*

 - bool `match` (`types::TokenType` t) const

Check if the current token type matches.
- bool `peekIs` (const std::string &s)

Check if the current token value matches (and optionally advance).
- void `require` (const std::string &s)

Require the current token to match a string, or fail.
- void `require` (`types::TokenType` t)

Require the current token to match a type, or fail.
- void `requireKW` (const std::string &k)

Require the current token to be a keyword, or fail.
- void `fail` (const std::string &msg)

Report a validation error with a generic message.
- void `failHere` (const std::string &msg)

Report a validation error at the current token.
- void `failAt` (const `scan::TToken` *at, const std::string &msg)

Report a validation error at a specific token.
- std::string `found` () const

Return a display string for the current token value.
- bool `isKW` (const std::string &k) const

Check if current token value (lowercased) matches a keyword.
- void `parseProgram` ()

Validate a complete Pascal program.
- void `parseUnit` ()

Validate a Pascal unit.
- void `parseInterfaceSection` ()

Validate interface section declarations.
- void `parseUses` ()

Validate a uses clause.
- void `parseBlock` ()

Validate a block (declarations + compound statement).
- void `parseLabelSection` ()

Validate a label section.
- void `parseConstSection` ()

Validate a const section.
- void `parseTypeSection` ()

Validate a type section.
- void `parseVarSection` ()

Validate a var section.
- void `parseSubprogram` ()

Validate a procedure or function declaration.
- void `parseFormalParams` ()

Validate formal parameter declarations.
- void `parseIdentList` ()

Validate a comma-separated identifier list.
- void `parseType` (const std::string &)

Validate a type specification.
- void `parseTypeName` ()

Validate a type name reference.
- void `parseSubrange` ()

Validate a subrange type.

- void [parseConstant](#) ()
Validate a constant value.
- void [parseConstSimple](#) ()
Validate a simple constant (number, string, or name).
- void [parseCompoundStatement](#) ()
Validate a compound statement (begin..end).
- void [parseStatement](#) ()
Validate a single statement.
- void [parseIf](#) ()
Validate an if statement.
- void [parseWhile](#) ()
Validate a while statement.
- void [parseRepeat](#) ()
Validate a repeat..until statement.
- void [parseFor](#) ()
Validate a for statement.
- void [parseCase](#) ()
Validate a case statement.
- void [parseCaseLabelList](#) ()
Validate a case label list.
- void [parseCaseLabel](#) ()
Validate a single case label.
- void [parseWith](#) ()
Validate a with statement.
- void [parseGoto](#) ()
Validate a goto statement.
- void [parseSimpleOrCallOrAssign](#) ()
Validate a simple statement, procedure call, or assignment.
- void [parseDesignator](#) ()
Validate a designator (variable with selectors).
- void [parseActualParams](#) ()
Validate actual parameter list.
- void [parseExprStop](#) (const std::unordered_set< std::string > &stops)
Validate an expression up to a set of stop tokens.
- bool [isRelOp](#) () const
Check if current token is a relational operator.
- bool [isAddOp](#) () const
Check if current token is an additive operator.
- bool [isMulOp](#) () const
Check if current token is a multiplicative operator.
- bool [isSetOp](#) () const
Check if current token is a set operator.
- void [predeclareType](#) (const std::string &name)
Pre-declare a built-in or forward-declared type.
- bool [isBuiltinTypeName](#) (const std::string &key) const
Check if a lowercase key is a built-in type name.
- void [pushRecordFieldScope](#) (const std::string &recordTypeName)
Push a record field scope for the given type.
- void [popRecordFieldScope](#) ()
Pop the current record field scope.
- bool [inRecordFieldScope](#) () const

Check if currently inside a record field scope.

- void `declareRecordField` (const std::string &name, const `scan::TToken` *at)

Declare a record field in the current record scope.

- void `parseFieldIdentList` ()

Parse a field identifier list within a record definition.

Static Private Member Functions

- static std::string `tokenTypeToString` (`types::TokenType` t)

Convert a TokenType to a display string.

- static std::string `lower` (std::string s)

Convert a string to lowercase.

- static std::string `toLower` (const std::string &s)

Convert a string to lowercase (static helper).

Private Attributes

- std::unordered_map< std::string, std::unordered_set< std::string > > `recordFieldScopesByType`

field names per record type

- std::string `currentRecordTypeName`

name of record type currently being parsed

- std::vector< `Scope` > `scopeStack`

stack of lexical scopes

- `scan::Scanner` `scanner`

underlying scanner

- const `scan::TToken` * `token` = nullptr

current token pointer

- std::string `source`

original source text

- std::string `filename`

source filename for error messages

- size_t `index` = 0

current token index

- int `inFunctionDepth` = 0

- std::unordered_set< std::string > `declaredVars`

globally declared variables

- std::unordered_set< std::string > `declaredConsts`

globally declared constants

- std::unordered_set< std::string > `declaredFuncs`

globally declared functions

- std::unordered_set< std::string > `declaredProcs`

globally declared procedures

- std::unordered_set< std::string > `importedUnits`

unit names from uses clause

- std::unordered_set< std::string > `declaredTypes`

globally declared type names

- int `withDepth` = 0

nesting depth of with statements (suppress undeclared-id errors)

11.101.1 Detailed Description

Semantic validator for Pascal programs.

Performs a second pass over the token stream to check that all variables, constants, types, functions, and procedures are properly declared before use, enforcing scope rules.

Definition at line 34 of file [validator.hpp](#).

11.101.2 Constructor & Destructor Documentation

11.101.2.1 TPValidator()

```
mx::TPValidator::TPValidator (
    const std::string & source_) [explicit]
```

Construct a validator for the given source.

Parameters

<i>source</i> ↔	Pascal source code string
—	

Definition at line 16 of file [validator.cpp](#).

References [scanner](#), and [source](#).

11.101.3 Member Function Documentation

11.101.3.1 checkConstOnly()

```
void mx::TPValidator::checkConstOnly (
    const std::string & name,
    const scan::TToken * at) [private]
```

Assert a name is a constant (used in const expressions).

Definition at line 238 of file [validator.cpp](#).

References [failAt\(\)](#), [isBuiltinConst\(\)](#), [lower\(\)](#), and [scopeStack](#).

Referenced by [parseConstExpr\(\)](#), and [parseConstSimple\(\)](#).

11.101.3.2 checkType()

```
void mx::TPValidator::checkType (
    const std::string & name,
    const scan::TToken * at) [private]
```

Assert a type name is declared and accessible.

Definition at line 1350 of file [validator.cpp](#).

References [failAt\(\)](#), [isBuiltinType\(\)](#), [lower\(\)](#), and [scopeStack](#).

Referenced by [parseTypeName\(\)](#).

11.101.3.3 checkVar()

```
void mx::TPValidator::checkVar (
    const std::string & name,
    const scan::TToken * at) [private]
```

Assert a variable is declared and accessible in scope.

Definition at line 200 of file [validator.cpp](#).

References [declaredFuncs](#), [declaredProcs](#), [failAt\(\)](#), [importedUnits](#), [inFunctionDepth](#), [isBuiltinConst\(\)](#), [isPascalKeyword\(\)](#), [lower\(\)](#), [scopeStack](#), and [withDepth](#).

Referenced by [parseDesignator\(\)](#), and [parseFor\(\)](#).

11.101.3.4 checkVarOrConst()

```
void mx::TPValidator::checkVarOrConst (
    const std::string & name,
```

```
const scan::Token * at) [private]
```

Assert a name is a variable or constant in scope.

Definition at line 219 of file [validator.cpp](#).

References [declaredFuncs](#), [declaredProcs](#), [failAt\(\)](#), [importedUnits](#), [inFunctionDepth](#), [isBuiltinConst\(\)](#), [isPascalKeyword\(\)](#), [lower\(\)](#), [scopeStack](#), and [withDepth](#).

Referenced by [parseExprStop\(\)](#).

11.101.3.5 currentScope() [1/2]

```
mx::Scope * mx::TPValidator::currentScope () [private]
```

Get the current (top) scope.

Definition at line 1361 of file [validator.cpp](#).

References [scopeStack](#).

Referenced by [declareFunc\(\)](#), [declareParam\(\)](#), [declareProc\(\)](#), [declareType\(\)](#), [declareVar\(\)](#), [isFuncDeclaredHere\(\)](#), [isParamDeclaredHere\(\)](#), [isProcDeclaredHere\(\)](#), [isTypeDeclaredHere\(\)](#), and [isVarDeclaredHere\(\)](#).

11.101.3.6 currentScope() [2/2]

```
const mx::Scope * mx::TPValidator::currentScope () const [private]
```

Get the current scope (const).

Definition at line 1367 of file [validator.cpp](#).

References [scopeStack](#).

11.101.3.7 currentScopeName()

```
std::string mx::TPValidator::currentScopeName () const [private]
```

Return a display name for the current scope.

11.101.3.8 declareConst()

```
void mx::TPValidator::declareConst (
    const std::string & name,
    const scan::Token * at) [private]
```

Declare a constant in the current scope.

Definition at line 187 of file [validator.cpp](#).

References [failAt\(\)](#), [lower\(\)](#), and [scopeStack](#).

Referenced by [parseConstSection\(\)](#), and [parseType\(\)](#).

11.101.3.9 declareFunc()

```
void mx::TPValidator::declareFunc (
    const std::string & name,
    const scan::Token * at) [private]
```

Declare a function in the current scope.

Definition at line 1418 of file [validator.cpp](#).

References [currentScope\(\)](#), [failAt\(\)](#), [mx::Scope::funcs](#), [isFuncDeclaredHere\(\)](#), [isParamDeclaredHere\(\)](#), [isProcDeclaredHere\(\)](#), [isTypeDeclaredHere\(\)](#), [isVarDeclaredHere\(\)](#), and [lower\(\)](#).

11.101.3.10 declareParam()

```
void mx::TPValidator::declareParam (
    const std::string & name,
    const scan::Token * at) [private]
```

Declare a parameter in the current scope.

Definition at line 1436 of file [validator.cpp](#).

References [currentScope\(\)](#), [failAt\(\)](#), [isFuncDeclaredHere\(\)](#), [isParamDeclaredHere\(\)](#), [isProcDeclaredHere\(\)](#), [isTypeDeclaredHere\(\)](#), [isVarDeclaredHere\(\)](#), [lower\(\)](#), and [mx::Scope::params](#).

11.101.3.11 declareProc()

```
void mxs::TPValidator::declareProc (
    const std::string & name,
    const scan::Token * at) [private]
```

Declare a procedure in the current scope.

Definition at line 1427 of file [validator.cpp](#).

References [currentScope\(\)](#), [failAt\(\)](#), [isFunctionDeclaredHere\(\)](#), [isParamDeclaredHere\(\)](#), [isProcDeclaredHere\(\)](#), [isTypeDeclaredHere\(\)](#), [isVarDeclaredHere\(\)](#), [lower\(\)](#), and [mxs::Scope::procs](#).

11.101.3.12 declareRecordField()

```
void mxs::TPValidator::declareRecordField (
    const std::string & name,
    const scan::Token * at) [private]
```

Declare a record field in the current record scope.

Definition at line 1470 of file [validator.cpp](#).

References [currentRecordTypeName](#), [declareVar\(\)](#), [failAt\(\)](#), [inRecordFieldScope\(\)](#), [lower\(\)](#), and [recordFieldScopesByType](#).
Referenced by [parseFieldIdentList\(\)](#).

11.101.3.13 declareType()

```
void mxs::TPValidator::declareType (
    const std::string & name,
    const scan::Token * at) [private]
```

Declare a user-defined type in the current scope.

Definition at line 1445 of file [validator.cpp](#).

References [currentScope\(\)](#), [failAt\(\)](#), [isFunctionDeclaredHere\(\)](#), [isParamDeclaredHere\(\)](#), [isProcDeclaredHere\(\)](#), [isTypeDeclaredHere\(\)](#), [isVarDeclaredHere\(\)](#), [lower\(\)](#), and [mxs::Scope::types](#).
Referenced by [parseTypeSection\(\)](#).

11.101.3.14 declareVar()

```
void mxs::TPValidator::declareVar (
    const std::string & name,
    const scan::Token * at) [private]
```

Declare a variable in the current scope.

Definition at line 1409 of file [validator.cpp](#).

References [currentScope\(\)](#), [failAt\(\)](#), [isFunctionDeclaredHere\(\)](#), [isParamDeclaredHere\(\)](#), [isProcDeclaredHere\(\)](#), [isTypeDeclaredHere\(\)](#), [isVarDeclaredHere\(\)](#), [lower\(\)](#), and [mxs::Scope::vars](#).
Referenced by [declareRecordField\(\)](#), [parseIdentList\(\)](#), and [parseVarSection\(\)](#).

11.101.3.15 fail()

```
void mxs::TPValidator::fail (
    const std::string & msg) [private]
```

Report a validation error with a generic message.

Definition at line 126 of file [validator.cpp](#).

References [filename](#).

Referenced by [parseCompoundStatement\(\)](#), [require\(\)](#), [require\(\)](#), and [requireKW\(\)](#).

11.101.3.16 failAt()

```
void mxs::TPValidator::failAt (
    const scan::Token * at,
    const std::string & msg) [private]
```

Report a validation error at a specific token.

Definition at line 136 of file [validator.cpp](#).

References [filename](#), and [scan::token::Token< Ch, String >::getLine\(\)](#).

Referenced by [checkConstOnly\(\)](#), [checkType\(\)](#), [checkVar\(\)](#), [checkVarOrConst\(\)](#), [declareConst\(\)](#), [declareFunc\(\)](#), [declareParam\(\)](#), [declareProc\(\)](#), [declareRecordField\(\)](#), [declareType\(\)](#), [declareVar\(\)](#), [parseCaseLabel\(\)](#), [parseConstExpr\(\)](#), [require\(\)](#), [require\(\)](#), and [requireKW\(\)](#).

11.101.3.17 failHere()

```
void mx::TPValidator::failHere (
    const std::string & msg) [private]
```

Report a validation error at the current token.

Definition at line 130 of file [validator.cpp](#).

References [filename](#), and [token](#).

Referenced by [parseCaseLabel\(\)](#), [parseCompoundStatement\(\)](#), [parseConstExpr\(\)](#), [parseExprStop\(\)](#), [parseRepeat\(\)](#), [parseStatement\(\)](#), and [validate\(\)](#).

11.101.3.18 found()

```
std::string mx::TPValidator::found () const [private]
```

Return a display string for the current token value.

Definition at line 165 of file [validator.cpp](#).

References [token](#), and [tokenTypeToString\(\)](#).

Referenced by [require\(\)](#), [require\(\)](#), and [requireKW\(\)](#).

11.101.3.19 inRecordFieldScope()

```
bool mx::TPValidator::inRecordFieldScope () const [private]
```

Check if currently inside a record field scope.

Definition at line 1466 of file [validator.cpp](#).

References [currentRecordTypeName](#).

Referenced by [declareRecordField\(\)](#).

11.101.3.20 isAddOp()

```
bool mx::TPValidator::isAddOp () const [private]
```

Check if current token is an additive operator.

Definition at line 1339 of file [validator.cpp](#).

References [isKW\(\)](#), and [match\(\)](#).

Referenced by [parseExprStop\(\)](#).

11.101.3.21 isBuiltinConst()

```
bool mx::TPValidator::isBuiltinConst (
    const std::string & name) const [private]
```

Check if a name is a built-in constant (true, false, maxint).

Definition at line 195 of file [validator.cpp](#).

References [lower\(\)](#).

Referenced by [checkConstOnly\(\)](#), [checkVar\(\)](#), and [checkVarOrConst\(\)](#).

11.101.3.22 isBuiltinType()

```
bool mx::TPValidator::isBuiltinType () const [private]
```

Check if the current token is a built-in type name.

Definition at line 1343 of file [validator.cpp](#).

References [isKW\(\)](#).

Referenced by [checkType\(\)](#), and [parseType\(\)](#).

11.101.3.23 isBuiltinTypeName()

```
bool mx::TPValidator::isBuiltinTypeName (
    const std::string & key) const [private]
```

Check if a lowercase key is a built-in type name.

11.101.3.24 isFuncDeclaredHere()

```
bool mxx::TPValidator::isFuncDeclaredHere (
    const std::string & name) const [private]
```

Check if a function is declared in the innermost scope.

Definition at line 1399 of file [validator.cpp](#).

References [currentScope\(\)](#), [mxx::Scope::funcs](#), and [lower\(\)](#).

Referenced by [declareFunc\(\)](#), [declareParam\(\)](#), [declareProc\(\)](#), [declareType\(\)](#), and [declareVar\(\)](#).

11.101.3.25 isKW()

```
bool mxx::TPValidator::isKW (
    const std::string & k) const [private]
```

Check if current token value (lowercased) matches a keyword.

Definition at line 171 of file [validator.cpp](#).

References [isPascalKeyword\(\)](#), [lower\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [isAddOp\(\)](#), [isBuiltinType\(\)](#), [isMulOp\(\)](#), [isRelOp\(\)](#), [isSetOp\(\)](#), [parseActualParams\(\)](#), [parseBlock\(\)](#), [parseCase\(\)](#), [parseCompoundStatement\(\)](#), [parseConstExpr\(\)](#), [parseConstSection\(\)](#), [parseConstSimple\(\)](#), [parseExprStop\(\)](#), [parseFor\(\)](#), [parseFormalParams\(\)](#), [parseIf\(\)](#), [parseInterfaceSection\(\)](#), [parseProgram\(\)](#), [parseRepeat\(\)](#), [parseStatement\(\)](#), [parseSubprogram\(\)](#), [parseType\(\)](#), [parseTypeSection\(\)](#), [parseUnit\(\)](#), [parseVarSection\(\)](#), [requireKW\(\)](#), and [validate\(\)](#).

11.101.3.26 isMulOp()

```
bool mxx::TPValidator::isMulOp () const [private]
```

Check if current token is a multiplicative operator.

Definition at line 1340 of file [validator.cpp](#).

References [isKW\(\)](#), and [match\(\)](#).

Referenced by [parseExprStop\(\)](#).

11.101.3.27 isParamDeclaredHere()

```
bool mxx::TPValidator::isParamDeclaredHere (
    const std::string & name) const [private]
```

Check if a parameter is declared in the innermost scope.

Definition at line 1405 of file [validator.cpp](#).

References [currentScope\(\)](#), [lower\(\)](#), and [mxx::Scope::params](#).

Referenced by [declareFunc\(\)](#), [declareParam\(\)](#), [declareProc\(\)](#), [declareType\(\)](#), and [declareVar\(\)](#).

11.101.3.28 isPascalKeyword()

```
bool mxx::TPValidator::isPascalKeyword (
    const std::string & s) const [private]
```

Check if a string is a Pascal reserved keyword.

Definition at line 1389 of file [validator.cpp](#).

References [lower\(\)](#), and [mxx::pascal_keywords](#).

Referenced by [checkVar\(\)](#), [checkVarOrConst\(\)](#), and [isKW\(\)](#).

11.101.3.29 isProcDeclaredHere()

```
bool mxx::TPValidator::isProcDeclaredHere (
    const std::string & name) const [private]
```

Check if a procedure is declared in the innermost scope.

Definition at line 1402 of file [validator.cpp](#).

References [currentScope\(\)](#), [lower\(\)](#), and [mxx::Scope::procs](#).

Referenced by [declareFunc\(\)](#), [declareParam\(\)](#), [declareProc\(\)](#), [declareType\(\)](#), and [declareVar\(\)](#).

11.101.3.30 isRelOp()

```
bool mxx::TPValidator::isRelOp () const [private]
```

Check if current token is a relational operator.
 Definition at line 1338 of file [validator.cpp](#).
 References [isKW\(\)](#), and [match\(\)](#).
 Referenced by [parseExprStop\(\)](#).

11.101.3.31 isSetOp()

```
bool mxx::TPValidator::isSetOp () const [private]
```

Check if current token is a set operator.
 Definition at line 1341 of file [validator.cpp](#).
 References [isKW\(\)](#).
 Referenced by [parseExprStop\(\)](#).

11.101.3.32 isTypeDeclaredHere()

```
bool mxx::TPValidator::isTypeDeclaredHere (
    const std::string & name) const [private]
```

Check if a type is declared in the innermost scope.
 Definition at line 1396 of file [validator.cpp](#).
 References [currentScope\(\)](#), [lower\(\)](#), and [mxx::Scope::types](#).
 Referenced by [declareFunc\(\)](#), [declareParam\(\)](#), [declareProc\(\)](#), [declareType\(\)](#), and [declareVar\(\)](#).

11.101.3.33 isVarDeclaredHere()

```
bool mxx::TPValidator::isVarDeclaredHere (
    const std::string & name) const [private]
```

Check if a variable is declared in the innermost scope.
 Definition at line 1393 of file [validator.cpp](#).
 References [currentScope\(\)](#), [lower\(\)](#), and [mxx::Scope::vars](#).
 Referenced by [declareFunc\(\)](#), [declareParam\(\)](#), [declareProc\(\)](#), [declareType\(\)](#), and [declareVar\(\)](#).

11.101.3.34 isVariableInCurrentScope()

```
bool mxx::TPValidator::isVariableInCurrentScope (
    const std::string & name) const [private]
```

Check if a variable is declared in the current scope.

11.101.3.35 lower()

```
std::string mxx::TPValidator::lower (
    std::string s) [static], [private]
```

Convert a string to lowercase.
 Definition at line 159 of file [validator.cpp](#).
 Referenced by [checkConstOnly\(\)](#), [checkType\(\)](#), [checkVar\(\)](#), [checkVarOrConst\(\)](#), [declareConst\(\)](#), [declareFunc\(\)](#), [declareParam\(\)](#), [declareProc\(\)](#), [declareRecordField\(\)](#), [declareType\(\)](#), [declareVar\(\)](#), [isBuiltinConst\(\)](#), [isFuncDeclaredHere\(\)](#), [isKW\(\)](#), [isParamDeclaredHere\(\)](#), [isPascalKeyword\(\)](#), [isProcDeclaredHere\(\)](#), [isTypeDeclaredHere\(\)](#), [isVarDeclaredHere\(\)](#), [parseConstExpr\(\)](#), [parseExprStop\(\)](#), [parseInterfaceSection\(\)](#), [parseSubprogram\(\)](#), and [parseUses\(\)](#).

11.101.3.36 match() [1/2]

```
bool mxx::TPValidator::match (
    const std::string & s) const [private]
```

Check if the current token value matches a string.
 Definition at line 76 of file [validator.cpp](#).
 References [mxx::ciEquals\(\)](#), and [token](#).
 Referenced by [isAddOp\(\)](#), [isMulOp\(\)](#), [isRelOp\(\)](#), [parseActualParams\(\)](#), [parseCase\(\)](#), [parseCaseLabel\(\)](#), [parseCaseLabelList\(\)](#), [parseCompoundStatement\(\)](#), [parseConstant\(\)](#), [parseConstExpr\(\)](#), [parseConstSection\(\)](#), [parseConstSimple\(\)](#), [parseDesignator\(\)](#), [parseExprStop\(\)](#), [parseFieldIdentList\(\)](#), [parseFormalParams\(\)](#), [parseIdentList\(\)](#), [parseInterfaceSection\(\)](#), [parseLabelSection\(\)](#), [parseRepeat\(\)](#), [parseSimpleOrCallOrAssign\(\)](#), [parseStatement\(\)](#),

[parseSubprogram\(\)](#), [parseSubrange\(\)](#), [parseType\(\)](#), [parseTypeSection\(\)](#), [parseUses\(\)](#), [parseVarSection\(\)](#), [parseWith\(\)](#), [require\(\)](#), and [require\(\)](#).

11.101.3.37 match() [2/2]

```
bool mxx::TPValidator::match (
    types::TokenType t) const [private]
```

Check if the current token type matches.

Definition at line 80 of file [validator.cpp](#).

References [token](#).

11.101.3.38 next()

```
bool mxx::TPValidator::next () [private]
```

Advance to the next token.

Definition at line 84 of file [validator.cpp](#).

References [index](#), [scanner](#), [token](#), and [types::TT_SYM](#).

Referenced by [parseActualParams\(\)](#), [parseCase\(\)](#), [parseCaseLabel\(\)](#), [parseCaseLabelList\(\)](#), [parseCompoundStatement\(\)](#), [parseConstant\(\)](#), [parseConstExpr\(\)](#), [parseConstSection\(\)](#), [parseConstSimple\(\)](#), [parseDesignator\(\)](#), [parseExprStop\(\)](#), [parseFieldIdentList\(\)](#), [parseFor\(\)](#), [parseFormalParams\(\)](#), [parseGoto\(\)](#), [parseIdentList\(\)](#), [parseIf\(\)](#), [parseInterfaceSection\(\)](#), [parseLabelSection\(\)](#), [parseProgram\(\)](#), [parseRepeat\(\)](#), [parseSimpleOrCallOrAssign\(\)](#), [parseStatement\(\)](#), [parseSubprogram\(\)](#), [parseSubrange\(\)](#), [parseType\(\)](#), [parseTypeName\(\)](#), [parseTypeSection\(\)](#), [parseUnit\(\)](#), [parseUses\(\)](#), [parseVarSection\(\)](#), [parseWhile\(\)](#), [parseWith\(\)](#), and [validate\(\)](#).

11.101.3.39 parseActualParams()

```
void mxx::TPValidator::parseActualParams () [private]
```

Validate actual parameter list.

Definition at line 1147 of file [validator.cpp](#).

References [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseExprStop\(\)](#), and [require\(\)](#).

Referenced by [parseExprStop\(\)](#), and [parseSimpleOrCallOrAssign\(\)](#).

11.101.3.40 parseBlock()

```
void mxx::TPValidator::parseBlock () [private]
```

Validate a block (declarations + compound statement).

Definition at line 501 of file [validator.cpp](#).

References [isKW\(\)](#), [parseCompoundStatement\(\)](#), [parseConstSection\(\)](#), [parseLabelSection\(\)](#), [parseSubprogram\(\)](#), [parseTypeSection\(\)](#), and [parseVarSection\(\)](#).

Referenced by [parseProgram\(\)](#), and [parseSubprogram\(\)](#).

11.101.3.41 parseCase()

```
void mxx::TPValidator::parseCase () [private]
```

Validate a case statement.

Definition at line 1021 of file [validator.cpp](#).

References [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseCaseLabelList\(\)](#), [parseExprStop\(\)](#), [parseStatement\(\)](#), [require\(\)](#), and [requireKW\(\)](#).

Referenced by [parseStatement\(\)](#).

11.101.3.42 parseCaseLabel()

```
void mxx::TPValidator::parseCaseLabel () [private]
```

Validate a single case label.

Definition at line 1056 of file [validator.cpp](#).

References [failAt\(\)](#), [failHere\(\)](#), [match\(\)](#), [next\(\)](#), [token](#), [types::TT_ID](#), [types::TT_NUM](#), and [types::TT_STR](#).

Referenced by [parseCaseLabelList\(\)](#).

11.101.3.43 parseCaseLabelList()

```
void mx::TPValidator::parseCaseLabelList () [private]
```

Validate a case label list.

Definition at line 1048 of file [validator.cpp](#).

References [match\(\)](#), [next\(\)](#), and [parseCaseLabel\(\)](#).

Referenced by [parseCase\(\)](#).

11.101.3.44 parseCompoundStatement()

```
void mx::TPValidator::parseCompoundStatement () [private]
```

Validate a compound statement (begin..end).

Definition at line 862 of file [validator.cpp](#).

References [fail\(\)](#), [failHere\(\)](#), [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseStatement\(\)](#), [requireKW\(\)](#), and [token](#).

Referenced by [parseBlock\(\)](#), and [parseStatement\(\)](#).

11.101.3.45 parseConstant()

```
void mx::TPValidator::parseConstant () [private]
```

Validate a constant value.

Definition at line 839 of file [validator.cpp](#).

References [match\(\)](#), [next\(\)](#), and [parseConstSimple\(\)](#).

11.101.3.46 parseConstExpr()

```
void mx::TPValidator::parseConstExpr (
    const std::unordered_set< std::string > & stops,
    bool constOnly) [private]
```

Parse and validate a constant expression.

Parameters

<i>stops</i>	Set of token values that terminate the expression
<i>constOnly</i>	If true, reject non-constant identifiers

Definition at line 1280 of file [validator.cpp](#).

References [checkConstOnly\(\)](#), [failAt\(\)](#), [failHere\(\)](#), [isKW\(\)](#), [lower\(\)](#), [match\(\)](#), [next\(\)](#), [token](#), [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), [types::TT_STR](#), and [types::TT_SYM](#).

Referenced by [parseConstSection\(\)](#), [parseSubrange\(\)](#), [parseType\(\)](#), and [parseVarSection\(\)](#).

11.101.3.47 parseConstSection()

```
void mx::TPValidator::parseConstSection () [private]
```

Validate a const section.

Definition at line 533 of file [validator.cpp](#).

References [declareConst\(\)](#), [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseConstExpr\(\)](#), [require\(\)](#), [requireKW\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseBlock\(\)](#), [parseInterfaceSection\(\)](#), and [parseUnit\(\)](#).

11.101.3.48 parseConstSimple()

```
void mx::TPValidator::parseConstSimple () [private]
```

Validate a simple constant (number, string, or name).

Definition at line 848 of file [validator.cpp](#).

References [checkConstOnly\(\)](#), [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [require\(\)](#), [token](#), [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), and [types::TT_STR](#).

Referenced by [parseConstant\(\)](#).

11.101.3.49 parseDesignator()

```
void mxs::TPValidator::parseDesignator () [private]
```

Validate a designator (variable with selectors).

Definition at line 1117 of file [validator.cpp](#).

References [checkVar\(\)](#), [match\(\)](#), [next\(\)](#), [parseExprStop\(\)](#), [require\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseSimpleOrCallOrAssign\(\)](#), and [parseWith\(\)](#).

11.101.3.50 parseExprStop()

```
void mxs::TPValidator::parseExprStop (
    const std::unordered_set< std::string > & stops) [private]
```

Validate an expression up to a set of stop tokens.

Definition at line 1168 of file [validator.cpp](#).

References [checkVarOrConst\(\)](#), [failHere\(\)](#), [isAddOp\(\)](#), [isKW\(\)](#), [isMulOp\(\)](#), [isRelOp\(\)](#), [isSetOp\(\)](#), [lower\(\)](#), [match\(\)](#), [next\(\)](#), [parseActualParams\(\)](#), [parseExprStop\(\)](#), [require\(\)](#), [token](#), [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), [types::TT_STR](#), and [types::TT_SYM](#).

Referenced by [parseActualParams\(\)](#), [parseCase\(\)](#), [parseDesignator\(\)](#), [parseExprStop\(\)](#), [parseFor\(\)](#), [parseIf\(\)](#), [parseRepeat\(\)](#), [parseSimpleOrCallOrAssign\(\)](#), and [parseWhile\(\)](#).

11.101.3.51 parseFieldIdentList()

```
void mxs::TPValidator::parseFieldIdentList () [private]
```

Parse a field identifier list within a record definition.

Definition at line 1486 of file [validator.cpp](#).

References [declareRecordField\(\)](#), [match\(\)](#), [next\(\)](#), [require\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseType\(\)](#).

11.101.3.52 parseFor()

```
void mxs::TPValidator::parseFor () [private]
```

Validate a for statement.

Definition at line 1000 of file [validator.cpp](#).

References [checkVar\(\)](#), [isKW\(\)](#), [next\(\)](#), [parseExprStop\(\)](#), [parseStatement\(\)](#), [require\(\)](#), [requireKW\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseStatement\(\)](#).

11.101.3.53 parseFormalParams()

```
void mxs::TPValidator::parseFormalParams () [private]
```

Validate formal parameter declarations.

Definition at line 656 of file [validator.cpp](#).

References [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseIdentList\(\)](#), [parseType\(\)](#), and [require\(\)](#).

Referenced by [parseInterfaceSection\(\)](#), and [parseSubprogram\(\)](#).

11.101.3.54 parseGoto()

```
void mxs::TPValidator::parseGoto () [private]
```

Validate a goto statement.

Definition at line 1092 of file [validator.cpp](#).

References [next\(\)](#), [require\(\)](#), [requireKW\(\)](#), and [types::TT_NUM](#).

Referenced by [parseStatement\(\)](#).

11.101.3.55 parseIdentList()

```
void mxs::TPValidator::parseIdentList () [private]
```

Validate a comma-separated identifier list.

Definition at line 681 of file [validator.cpp](#).

References [declareVar\(\)](#), [match\(\)](#), [next\(\)](#), [require\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseFormalParams\(\)](#).

11.101.3.56 parseIf()

```
void mx::TPValidator::parseIf () [private]
```

Validate an if statement.

Definition at line 954 of file [validator.cpp](#).

References [isKW\(\)](#), [next\(\)](#), [parseExprStop\(\)](#), [parseStatement\(\)](#), and [requireKW\(\)](#).

Referenced by [parseStatement\(\)](#).

11.101.3.57 parseInterfaceSection()

```
void mx::TPValidator::parseInterfaceSection () [private]
```

Validate interface section declarations.

Definition at line 302 of file [validator.cpp](#).

References [declaredFuncs](#), [declaredProcs](#), [isKW\(\)](#), [lower\(\)](#), [match\(\)](#), [next\(\)](#), [parseConstSection\(\)](#), [parseFormalParams\(\)](#), [parseType\(\)](#), [parseTypeSection\(\)](#), [parseVarSection\(\)](#), [popScope\(\)](#), [pushScope\(\)](#), [require\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseUnit\(\)](#).

11.101.3.58 parseLabelSection()

```
void mx::TPValidator::parseLabelSection () [private]
```

Validate a label section.

Definition at line 519 of file [validator.cpp](#).

References [match\(\)](#), [next\(\)](#), [require\(\)](#), [requireKW\(\)](#), and [types::TT_NUM](#).

Referenced by [parseBlock\(\)](#).

11.101.3.59 parseProgram()

```
void mx::TPValidator::parseProgram () [private]
```

Validate a complete Pascal program.

Definition at line 249 of file [validator.cpp](#).

References [isKW\(\)](#), [next\(\)](#), [parseBlock\(\)](#), [parseUses\(\)](#), [require\(\)](#), [requireKW\(\)](#), and [types::TT_ID](#).

Referenced by [validate\(\)](#).

11.101.3.60 parseRepeat()

```
void mx::TPValidator::parseRepeat () [private]
```

Validate a repeat..until statement.

Definition at line 976 of file [validator.cpp](#).

References [failHere\(\)](#), [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseExprStop\(\)](#), [parseStatement\(\)](#), and [requireKW\(\)](#).

Referenced by [parseStatement\(\)](#).

11.101.3.61 parseSimpleOrCallOrAssign()

```
void mx::TPValidator::parseSimpleOrCallOrAssign () [private]
```

Validate a simple statement, procedure call, or assignment.

Definition at line 1099 of file [validator.cpp](#).

References [match\(\)](#), [next\(\)](#), [parseActualParams\(\)](#), [parseDesignator\(\)](#), [parseExprStop\(\)](#), [peekIs\(\)](#), and [types::TT_ID](#).

Referenced by [parseStatement\(\)](#).

11.101.3.62 parseStatement()

```
void mx::TPValidator::parseStatement () [private]
```

Validate a single statement.

Definition at line 898 of file [validator.cpp](#).

References [failHere\(\)](#), [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseCase\(\)](#), [parseCompoundStatement\(\)](#), [parseFor\(\)](#), [parseGoto\(\)](#), [parseIf\(\)](#), [parseRepeat\(\)](#), [parseSimpleOrCallOrAssign\(\)](#), [parseStatement\(\)](#), [parseWhile\(\)](#), [parseWith\(\)](#), [token](#), [types::TT_ID](#), and [types::TT_NUM](#).

Referenced by [parseCase\(\)](#), [parseCompoundStatement\(\)](#), [parseFor\(\)](#), [parseIf\(\)](#), [parseRepeat\(\)](#), [parseStatement\(\)](#), [parseWhile\(\)](#), and [parseWith\(\)](#).

11.101.3.63 parseSubprogram()

```
void mx::TPValidator::parseSubprogram () [private]
```

Validate a procedure or function declaration.

Definition at line 606 of file [validator.cpp](#).

References [declaredFuncs](#), [declaredProcs](#), [inFunctionDepth](#), [isKW\(\)](#), [lower\(\)](#), [match\(\)](#), [next\(\)](#), [parseBlock\(\)](#), [parseFormalParams\(\)](#), [parseType\(\)](#), [popScope\(\)](#), [pushScope\(\)](#), [require\(\)](#), [requireKW\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseBlock\(\)](#), and [parseUnit\(\)](#).

11.101.3.64 parseSubrange()

```
void mx::TPValidator::parseSubrange () [private]
```

Validate a subrange type.

Definition at line 831 of file [validator.cpp](#).

References [match\(\)](#), [next\(\)](#), and [parseConstExpr\(\)](#).

Referenced by [parseType\(\)](#).

11.101.3.65 parseType()

```
void mx::TPValidator::parseType (
    const std::string & typeName) [private]
```

Validate a type specification.

Definition at line 693 of file [validator.cpp](#).

References [declareConst\(\)](#), [isBuiltinType\(\)](#), [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseConstExpr\(\)](#), [parseFieldIdentList\(\)](#), [parseSubrange\(\)](#), [parseType\(\)](#), [parseTypeName\(\)](#), [popRecordFieldScope\(\)](#), [pushRecordFieldScope\(\)](#), [require\(\)](#), [requireKW\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseFormalParams\(\)](#), [parseInterfaceSection\(\)](#), [parseSubprogram\(\)](#), [parseType\(\)](#), [parseTypeSection\(\)](#), and [parseVarSection\(\)](#).

11.101.3.66 parseTypeName()

```
void mx::TPValidator::parseTypeName () [private]
```

Validate a type name reference.

Definition at line 824 of file [validator.cpp](#).

References [checkType\(\)](#), [next\(\)](#), [require\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseType\(\)](#).

11.101.3.67 parseTypeSection()

```
void mx::TPValidator::parseTypeSection () [private]
```

Validate a type section.

Definition at line 554 of file [validator.cpp](#).

References [declareType\(\)](#), [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseType\(\)](#), [require\(\)](#), [requireKW\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseBlock\(\)](#), [parseInterfaceSection\(\)](#), and [parseUnit\(\)](#).

11.101.3.68 parseUnit()

```
void mx::TPValidator::parseUnit () [private]
```

Validate a Pascal unit.

Definition at line 263 of file [validator.cpp](#).

References [isKW\(\)](#), [next\(\)](#), [parseConstSection\(\)](#), [parseInterfaceSection\(\)](#), [parseSubprogram\(\)](#), [parseTypeSection\(\)](#), [parseUses\(\)](#), [parseVarSection\(\)](#), [require\(\)](#), [requireKW\(\)](#), and [types::TT_ID](#).

Referenced by [validate\(\)](#).

11.101.3.69 parseUses()

```
void mx::TPValidator::parseUses () [private]
```

Validate a uses clause.

Definition at line 339 of file [validator.cpp](#).

References [declaredFuncs](#), [declaredProcs](#), [filename](#), [scan::Scanner::getTokens\(\)](#), [importedUnits](#), [lower\(\)](#), [match\(\)](#), [next\(\)](#), [pos\(\)](#), [require\(\)](#), [requireKW\(\)](#), [scan::Scanner::scan\(\)](#), [scopeStack](#), [token](#), [types::TT_ID](#), and [types::TT_SYM](#).
 Referenced by [parseProgram\(\)](#), and [parseUnit\(\)](#).

11.101.3.70 [parseVarSection\(\)](#)

```
void mx::TPValidator::parseVarSection () [private]
```

Validate a var section.

Definition at line 578 of file [validator.cpp](#).

References [declareVar\(\)](#), [isKW\(\)](#), [match\(\)](#), [next\(\)](#), [parseConstExpr\(\)](#), [parseType\(\)](#), [require\(\)](#), [requireKW\(\)](#), [token](#), and [types::TT_ID](#).

Referenced by [parseBlock\(\)](#), [parseInterfaceSection\(\)](#), and [parseUnit\(\)](#).

11.101.3.71 [parseWhile\(\)](#)

```
void mx::TPValidator::parseWhile () [private]
```

Validate a while statement.

Definition at line 967 of file [validator.cpp](#).

References [next\(\)](#), [parseExprStop\(\)](#), [parseStatement\(\)](#), and [requireKW\(\)](#).

Referenced by [parseStatement\(\)](#).

11.101.3.72 [parseWith\(\)](#)

```
void mx::TPValidator::parseWith () [private]
```

Validate a with statement.

Definition at line 1077 of file [validator.cpp](#).

References [match\(\)](#), [next\(\)](#), [parseDesignator\(\)](#), [parseStatement\(\)](#), [requireKW\(\)](#), and [withDepth](#).

Referenced by [parseStatement\(\)](#).

11.101.3.73 [peekIs\(\)](#)

```
bool mx::TPValidator::peekIs (
    const std::string & s) [private]
```

Check if the current token value matches (and optionally advance).

Definition at line 98 of file [validator.cpp](#).

References [mx::ciEquals\(\)](#), [index](#), and [scanner](#).

Referenced by [parseSimpleOrCallOrAssign\(\)](#).

11.101.3.74 [popRecordFieldScope\(\)](#)

```
void mx::TPValidator::popRecordFieldScope () [private]
```

Pop the current record field scope.

Definition at line 1462 of file [validator.cpp](#).

References [currentRecordTypeName](#).

Referenced by [parseType\(\)](#).

11.101.3.75 [popScope\(\)](#)

```
void mx::TPValidator::popScope () [private]
```

Pop the current scope from the stack.

Definition at line 181 of file [validator.cpp](#).

References [scopeStack](#).

Referenced by [parseInterfaceSection\(\)](#), [parseSubprogram\(\)](#), and [validate\(\)](#).

11.101.3.76 [predeclareType\(\)](#)

```
void mx::TPValidator::predeclareType (
    const std::string & name) [private]
```

Pre-declare a built-in or forward-declared type.

11.101.3.77 pushRecordFieldScope()

```
void mxs::TPValidator::pushRecordFieldScope (
    const std::string & recordTypeName) [private]
```

Push a record field scope for the given type.

Definition at line 1454 of file [validator.cpp](#).

References [currentRecordTypeName](#), and [recordFieldScopesByType](#).

Referenced by [parseType\(\)](#).

11.101.3.78 pushScope()

```
void mxs::TPValidator::pushScope () [private]
```

Push a new scope onto the scope stack.

Definition at line 177 of file [validator.cpp](#).

References [scopeStack](#).

Referenced by [parseInterfaceSection\(\)](#), [parseSubprogram\(\)](#), and [validate\(\)](#).

11.101.3.79 require() [1/2]

```
void mxs::TPValidator::require (
    const std::string & s) [private]
```

Require the current token to match a string, or fail.

Definition at line 102 of file [validator.cpp](#).

References [fail\(\)](#), [failAt\(\)](#), [found\(\)](#), [match\(\)](#), and [token](#).

Referenced by [parseActualParams\(\)](#), [parseCase\(\)](#), [parseConstSection\(\)](#), [parseConstSimple\(\)](#), [parseDesignator\(\)](#), [parseExprStop\(\)](#), [parseFieldIdentList\(\)](#), [parseFor\(\)](#), [parseFormalParams\(\)](#), [parseGoto\(\)](#), [parseIdentList\(\)](#), [parseInterfaceSection\(\)](#), [parseLabelSection\(\)](#), [parseProgram\(\)](#), [parseSubprogram\(\)](#), [parseType\(\)](#), [parseTypeName\(\)](#), [parseTypeSection\(\)](#), [parseUnit\(\)](#), [parseUses\(\)](#), and [parseVarSection\(\)](#).

11.101.3.80 require() [2/2]

```
void mxs::TPValidator::require (
    types::TokenType t) [private]
```

Require the current token to match a type, or fail.

Definition at line 110 of file [validator.cpp](#).

References [fail\(\)](#), [failAt\(\)](#), [found\(\)](#), [match\(\)](#), [token](#), and [tokenTypeToString\(\)](#).

11.101.3.81 requireKW()

```
void mxs::TPValidator::requireKW (
    const std::string & k) [private]
```

Require the current token to be a keyword, or fail.

Definition at line 118 of file [validator.cpp](#).

References [fail\(\)](#), [failAt\(\)](#), [found\(\)](#), [isKW\(\)](#), and [token](#).

Referenced by [parseCase\(\)](#), [parseCompoundStatement\(\)](#), [parseConstSection\(\)](#), [parseFor\(\)](#), [parseGoto\(\)](#), [parseIf\(\)](#), [parseLabelSection\(\)](#), [parseProgram\(\)](#), [parseRepeat\(\)](#), [parseSubprogram\(\)](#), [parseType\(\)](#), [parseTypeSection\(\)](#), [parseUnit\(\)](#), [parseUses\(\)](#), [parseVarSection\(\)](#), [parseWhile\(\)](#), and [parseWith\(\)](#).

11.101.3.82 tokenTypeToString()

```
std::string mxs::TPValidator::tokenTypeToString (
    types::TokenType t) [static], [private]
```

Convert a TokenType to a display string.

Definition at line 142 of file [validator.cpp](#).

References [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), [types::TT_STR](#), and [types::TT_SYM](#).

Referenced by [found\(\)](#), and [require\(\)](#).

11.101.3.83 toLower()

```
std::string mxx::TPValidator::toLower (
    const std::string & s) [static], [private]
```

Convert a string to lowercase (static helper).

11.101.3.84 validate()

```
bool mxx::TPValidator::validate (
    const std::string & name)
```

Run semantic validation.

Parameters

<i>name</i>	Program name (used in error messages)
-------------	---------------------------------------

Returns

true if the program is semantically valid

Definition at line 18 of file [validator.cpp](#).

References [declaredFuncs](#), [declaredProcs](#), [failHere\(\)](#), [filename](#), [importedUnits](#), [index](#), [isKW\(\)](#), [next\(\)](#), [parseProgram\(\)](#), [parseUnit\(\)](#), [popScope\(\)](#), [pushScope\(\)](#), [scanner](#), [scopeStack](#), and [token](#).

Referenced by [compileFile\(\)](#).

11.101.4 Member Data Documentation**11.101.4.1 currentRecordTypeName**

```
std::string mxx::TPValidator::currentRecordTypeName [private]
```

name of record type currently being parsed

Definition at line 51 of file [validator.hpp](#).

Referenced by [declareRecordField\(\)](#), [inRecordFieldScope\(\)](#), [popRecordFieldScope\(\)](#), and [pushRecordFieldScope\(\)](#).

11.101.4.2 declaredConsts

```
std::unordered_set<std::string> mxx::TPValidator::declaredConsts [private]
```

globally declared constants

Definition at line 64 of file [validator.hpp](#).

11.101.4.3 declaredFuncs

```
std::unordered_set<std::string> mxx::TPValidator::declaredFuncs [private]
```

globally declared functions

Definition at line 65 of file [validator.hpp](#).

Referenced by [checkVar\(\)](#), [checkVarOrConst\(\)](#), [parseInterfaceSection\(\)](#), [parseSubprogram\(\)](#), [parseUses\(\)](#), and [validate\(\)](#).

11.101.4.4 declaredProcs

```
std::unordered_set<std::string> mxx::TPValidator::declaredProcs [private]
```

globally declared procedures

Definition at line 66 of file [validator.hpp](#).

Referenced by [checkVar\(\)](#), [checkVarOrConst\(\)](#), [parseInterfaceSection\(\)](#), [parseSubprogram\(\)](#), [parseUses\(\)](#), and [validate\(\)](#).

11.101.4.5 declaredTypes

```
std::unordered_set<std::string> mxx::TPValidator::declaredTypes [private]
```

globally declared type names

Definition at line 117 of file [validator.hpp](#).

11.101.4.6 declaredVars

```
std::unordered_set<std::string> mxx::TPValidator::declaredVars [private]
```

globally declared variables

Definition at line 63 of file [validator.hpp](#).

11.101.4.7 filename

```
std::string mxx::TPValidator::filename [private]
```

source filename for error messages

Definition at line 56 of file [validator.hpp](#).

Referenced by [fail\(\)](#), [failAt\(\)](#), [failHere\(\)](#), [parseUses\(\)](#), and [validate\(\)](#).

11.101.4.8 importedUnits

```
std::unordered_set<std::string> mxx::TPValidator::importedUnits [private]
```

unit names from uses clause

Definition at line 67 of file [validator.hpp](#).

Referenced by [checkVar\(\)](#), [checkVarOrConst\(\)](#), [parseUses\(\)](#), and [validate\(\)](#).

11.101.4.9 index

```
size_t mxx::TPValidator::index = 0 [private]
```

current token index

Definition at line 57 of file [validator.hpp](#).

Referenced by [next\(\)](#), [peekIs\(\)](#), and [validate\(\)](#).

11.101.4.10 inFunctionDepth

```
int mxx::TPValidator::inFunctionDepth = 0 [private]
```

0 when inside a function body

Definition at line 58 of file [validator.hpp](#).

Referenced by [checkVar\(\)](#), [checkVarOrConst\(\)](#), and [parseSubprogram\(\)](#).

11.101.4.11 recordFieldScopesByType

```
std::unordered_map<std::string, std::unordered_set<std::string> > mxx::TPValidator::record↵  
FieldScopesByType [private]
```

field names per record type

Definition at line 50 of file [validator.hpp](#).

Referenced by [declareRecordField\(\)](#), and [pushRecordFieldScope\(\)](#).

11.101.4.12 scanner

```
scan::Scanner mxx::TPValidator::scanner [private]
```

underlying scanner

Definition at line 53 of file [validator.hpp](#).

Referenced by [TPValidator\(\)](#), [next\(\)](#), [peekIs\(\)](#), and [validate\(\)](#).

11.101.4.13 scopeStack

```
std::vector<Scope> mxx::TPValidator::scopeStack [private]
```

stack of lexical scopes

Definition at line 52 of file [validator.hpp](#).

Referenced by [checkConstOnly\(\)](#), [checkType\(\)](#), [checkVar\(\)](#), [checkVarOrConst\(\)](#), [currentScope\(\)](#), [currentScope\(\)](#), [declareConst\(\)](#), [parseUses\(\)](#), [popScope\(\)](#), [pushScope\(\)](#), and [validate\(\)](#).

11.101.4.14 source

```
std::string mxx::TPValidator::source [private]
```

original source text

Definition at line 55 of file [validator.hpp](#).

Referenced by [TPValidator\(\)](#).

11.101.4.15 token

```
const scan::TToken* mxx::TPValidator::token = nullptr [private]
```

current token pointer

Definition at line 54 of file [validator.hpp](#).

Referenced by [failHere\(\)](#), [found\(\)](#), [isKW\(\)](#), [match\(\)](#), [match\(\)](#), [next\(\)](#), [parseCaseLabel\(\)](#), [parseCompoundStatement\(\)](#), [parseConstExpr\(\)](#), [parseConstSection\(\)](#), [parseConstSimple\(\)](#), [parseDesignator\(\)](#), [parseExprStop\(\)](#), [parseFieldIdentList\(\)](#), [parseFor\(\)](#), [parseIdentList\(\)](#), [parseInterfaceSection\(\)](#), [parseStatement\(\)](#), [parseSubprogram\(\)](#), [parseType\(\)](#), [parseTypeName\(\)](#), [parseTypeSection\(\)](#), [parseUses\(\)](#), [parseVarSection\(\)](#), [require\(\)](#), [require\(\)](#), [requireKW\(\)](#), and [validate\(\)](#).

11.101.4.16 withDepth

```
int mxx::TPValidator::withDepth = 0 [private]
```

nesting depth of with statements (suppress undeclared-id errors)

Definition at line 252 of file [validator.hpp](#).

Referenced by [checkVar\(\)](#), [checkVarOrConst\(\)](#), and [parseWith\(\)](#).

The documentation for this class was generated from the following files:

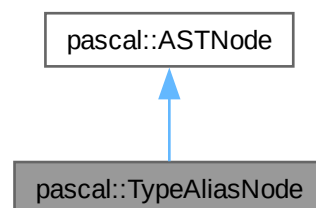
- [src/frontend/include/validator.hpp](#)
- [src/frontend/validator.cpp](#)

11.102 pascal::TypeAliasNode Class Reference

AST node for a type alias (type NewName = ExistingType).

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::TypeAliasNode`:

**Public Member Functions**

- [TypeAliasNode](#) (const std::string &name, const std::string &base)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from pascal::ASTNode

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [typeName](#)
the new type name
- std::string [baseType](#)
the existing type being aliased

11.102.1 Detailed Description

AST node for a type alias (type NewName = ExistingType).
Definition at line 161 of file [ast.hpp](#).

11.102.2 Constructor & Destructor Documentation

11.102.2.1 TypeAliasNode()

```
pascal::TypeAliasNode::TypeAliasNode (
    const std::string & name,
    const std::string & base) [inline]
```

Definition at line 166 of file [ast.hpp](#).
References [baseType](#), and [typeName](#).

11.102.3 Member Function Documentation

11.102.3.1 accept()

```
void pascal::TypeAliasNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).
Implements [pascal::ASTNode](#).
Definition at line 375 of file [ast.cpp](#).
References [pascal::ASTVisitor::visit\(\)](#).

11.102.3.2 toString()

```
std::string pascal::TypeAliasNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.
Implements [pascal::ASTNode](#).
Definition at line 390 of file [ast.cpp](#).
References [baseType](#), and [typeName](#).

11.102.4 Member Data Documentation

11.102.4.1 baseType

```
std::string pascal::TypeAliasNode::baseType
```

the existing type being aliased
Definition at line 164 of file [ast.hpp](#).
Referenced by [TypeAliasNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.102.4.2 typeName

`std::string pascal::TypeAliasNode::typeName`

the new type name

Definition at line 163 of file [ast.hpp](#).

Referenced by [TypeAliasNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

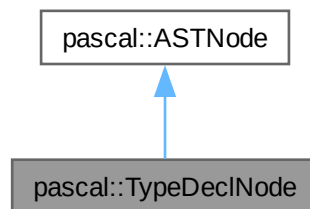
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.103 pascal::TypeDeclNode Class Reference

AST node for a type declaration section.

`#include <ast.hpp>`

Inheritance diagram for `pascal::TypeDeclNode`:



Public Member Functions

- [TypeDeclNode](#) (`std::vector< std::unique_ptr< ASTNode > >` declarations)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- `std::string` [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (`std::ostream` &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- `std::vector< std::unique_ptr< ASTNode > >` [typeDeclarations](#)
list of individual type declarations

11.103.1 Detailed Description

AST node for a type declaration section.
Definition at line 149 of file [ast.hpp](#).

11.103.2 Constructor & Destructor Documentation

11.103.2.1 TypeDeclNode()

```
pascal::TypeDeclNode::TypeDeclNode (
    std::vector< std::unique_ptr< ASTNode > > declarations) [inline]
```

Definition at line 153 of file [ast.hpp](#).

References [typeDeclarations](#).

11.103.3 Member Function Documentation

11.103.3.1 accept()

```
void pascal::TypeDeclNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 371 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.103.3.2 toString()

```
std::string pascal::TypeDeclNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 379 of file [ast.cpp](#).

References [typeDeclarations](#).

11.103.4 Member Data Documentation

11.103.4.1 typeDeclarations

```
std::vector<std::unique_ptr<ASTNode> > pascal::TypeDeclNode::typeDeclarations
```

list of individual type declarations

Definition at line 151 of file [ast.hpp](#).

Referenced by [TypeDeclNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

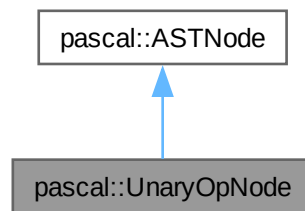
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.104 pascal::UnaryOpNode Class Reference

AST node for a unary operator expression.

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::UnaryOpNode`:



Public Types

- enum `Operator` { `PLUS` , `MINUS` , `NOT` }
Unary operator kinds.

Public Member Functions

- `UnaryOpNode` (`Operator` op, `std::unique_ptr< ASTNode >` operandNode)
- void `accept` (`ASTVisitor` &visitor) override
Accept an `ASTVisitor` (double-dispatch).
- `std::string toString` () const override
Return a human-readable description of this node.

Public Member Functions inherited from `pascal::ASTNode`

- virtual `~ASTNode` ()=default
- virtual void `print` (`std::ostream` &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Static Public Member Functions

- static `std::string opToString` (`Operator` op)
Convert a unary `Operator` to its string representation.

Public Attributes

- `Operator operator_`
the unary operator
- `std::unique_ptr< ASTNode >` operand
the operand expression

11.104.1 Detailed Description

AST node for a unary operator expression.
Definition at line 477 of file `ast.hpp`.

11.104.2 Member Enumeration Documentation

11.104.2.1 Operator

enum `pascal::UnaryOpNode::Operator`

Unary operator kinds.

Enumerator

PLUS	unary +
MINUS	unary -
NOT	logical not

Definition at line 480 of file `ast.hpp`.

11.104.3 Constructor & Destructor Documentation

11.104.3.1 UnaryOpNode()

```
pascal::UnaryOpNode::UnaryOpNode (
    Operator op,
    std::unique_ptr< ASTNode > operandNode) [inline]
```

Definition at line 489 of file `ast.hpp`.

References `operand`, and `operator_`.

11.104.4 Member Function Documentation

11.104.4.1 accept()

```
void pascal::UnaryOpNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an `ASTVisitor` (double-dispatch).

Implements `pascal::ASTNode`.

Definition at line 210 of file `ast.cpp`.

References `pascal::ASTVisitor::visit()`.

11.104.4.2 opToString()

```
std::string pascal::UnaryOpNode::opToString (
    Operator op) [static]
```

Convert a unary `Operator` to its string representation.

Definition at line 218 of file `ast.cpp`.

References `MINUS`, `NOT`, and `PLUS`.

Referenced by `toString()`.

11.104.4.3 toString()

```
std::string pascal::UnaryOpNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements `pascal::ASTNode`.

Definition at line 214 of file `ast.cpp`.

References `operator_`, and `opToString()`.

11.104.5 Member Data Documentation

11.104.5.1 operand

```
std::unique_ptr<ASTNode> pascal::UnaryOpNode::operand
```

the operand expression

Definition at line 487 of file [ast.hpp](#).

Referenced by [UnaryOpNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.104.5.2 operator_

[Operator](#) [pascal::UnaryOpNode::operator_](#)

the unary operator

Definition at line 486 of file [ast.hpp](#).

Referenced by [UnaryOpNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

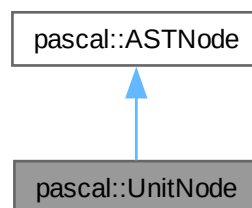
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.105 pascal::UnitNode Class Reference

AST node for a Pascal unit (separately compiled module).

```
#include <ast.hpp>
```

Inheritance diagram for [pascal::UnitNode](#):



Public Member Functions

- [UnitNode](#) (const std::string &name)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- `std::string` [name](#)
unit identifier
- `std::vector< std::string >` [uses](#)
modules/units imported via 'uses' clause
- `std::vector< std::unique_ptr< ASTNode > >` [interfaceDecls](#)
exported procedure/function forward declarations
- `std::vector< std::unique_ptr< ASTNode > >` [implDecls](#)
implementation declarations (proc/func/var/const/type)

11.105.1 Detailed Description

AST node for a Pascal unit (separately compiled module).

A unit has an interface section (exported forward declarations) and an implementation section (actual code). Compiles to an MXVM object.

Definition at line 116 of file [ast.hpp](#).

11.105.2 Constructor & Destructor Documentation

11.105.2.1 UnitNode()

```
pascal::UnitNode::UnitNode (
    const std::string & name) [inline]
```

Definition at line 123 of file [ast.hpp](#).

References [name](#).

11.105.3 Member Function Documentation

11.105.3.1 accept()

```
void pascal::UnitNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 31 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.105.3.2 toString()

```
std::string pascal::UnitNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 35 of file [ast.cpp](#).

References [name](#).

11.105.4 Member Data Documentation

11.105.4.1 implDecls

```
std::vector<std::unique_ptr<ASTNode> > pascal::UnitNode::implDecls
```

implementation declarations (proc/func/var/const/type)

Definition at line 121 of file [ast.hpp](#).

Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

11.105.4.2 interfaceDecls

`std::vector<std::unique_ptr<ASTNode> > pascal::UnitNode::interfaceDecls`
 exported procedure/function forward declarations
 Definition at line 120 of file [ast.hpp](#).
 Referenced by [pascal::CodeGenVisitor::visit\(\)](#).

11.105.4.3 name

`std::string pascal::UnitNode::name`
 unit identifier
 Definition at line 118 of file [ast.hpp](#).
 Referenced by [UnitNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.105.4.4 uses

`std::vector<std::string> pascal::UnitNode::uses`
 modules/units imported via 'uses' clause
 Definition at line 119 of file [ast.hpp](#).
 Referenced by [pascal::CodeGenVisitor::visit\(\)](#).
 The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.106 mxvm::UseLabel Struct Reference

Records a label usage site for validation.
`#include <valid.hpp>`

Public Attributes

- `std::string` [name](#)
- `const scan::TToken *` [at](#)

11.106.1 Detailed Description

Records a label usage site for validation.
 Definition at line 42 of file [valid.hpp](#).

11.106.2 Member Data Documentation

11.106.2.1 at

`const scan::TToken* mxvm::UseLabel::at`
 Definition at line 44 of file [valid.hpp](#).

11.106.2.2 name

`std::string mxvm::UseLabel::name`
 Definition at line 43 of file [valid.hpp](#).
 The documentation for this struct was generated from the following file:

- [include/mxvm/valid.hpp](#)

11.107 mxvm::UseVar Struct Reference

Records a variable usage site for validation.
`#include <valid.hpp>`

Public Attributes

- std::string [name](#)
- const [scan::TToken](#) * [at](#)

11.107.1 Detailed Description

Records a variable usage site for validation.

Definition at line 37 of file [valid.hpp](#).

11.107.2 Member Data Documentation**11.107.2.1 at**

```
const scan::TToken* mxvm::UseVar::at
```

Definition at line 39 of file [valid.hpp](#).

11.107.2.2 name

```
std::string mxvm::UseVar::name
```

Definition at line 38 of file [valid.hpp](#).

The documentation for this struct was generated from the following file:

- include/mxvm/[valid.hpp](#)

11.108 mxvm::Validator Class Reference

Validates MXVM source for correct variable/label usage and instruction operands.

```
#include <valid.hpp>
```

Classes

- struct [VarNamePair](#)
A variable name paired with its declaration token.

Public Member Functions

- [Validator](#) (const std::string &source)
Construct a validator from MXVM source code.
- void [validateAgainstSpec](#) (const std::string &op, const std::vector< [ParsedOp](#) > &ops, const std::unordered_map< std::string, [Variable](#) > &vars, const std::unordered_map< std::string, std::string > &labels, const std::unordered_set< std::string > &objects, std::vector< [UseVar](#) > &usedVarsRef, std::vector< [UseLabel](#) > &usedLabelsRef)
Validate an instruction's operands against the spec table.
- bool [validate](#) (const std::string &name)
Run full validation on the source program.
- bool [match](#) (const std::string &m)
Check if the current token value matches and advance.
- void [require](#) (const std::string &r)
Require the current token value to match, or report an error.
- bool [match](#) (const [types::TokenType](#) &t)
Check if the current token type matches and advance.
- void [require](#) (const [types::TokenType](#) &t)
Require the current token type to match, or report an error.
- bool [next](#) ()
Advance to the next token; returns false at end.

- bool [peekIs](#) (const std::string &s)
Check if the current token value matches without consuming.
- bool [peekIs](#) (const [types::TokenType](#) &t)
Check if the current token type matches without consuming.
- std::string [tokenTypeToString](#) ([types::TokenType](#) t)
Convert a TokenType to its display string.
- void [collect_labels](#) (std::unordered_map< std::string, std::string > &labels)
Pre-collect all label declarations and map them to function names.
- void [collect_objects](#) (std::unordered_set< std::string > &objects, size_t start_index, size_t end_index)
Collect all object names declared between token indices.
- [ParsedOp](#) [parseOperand](#) ()
Parse a single instruction operand from the token stream.
- std::vector< [ParsedOp](#) > [parseOperandList](#) ()
Parse a comma-separated list of operands.

Public Attributes

- std::vector< [VarNamePair](#) > [var_names](#)

Private Attributes

- std::string [filename](#)
- [scan::Scanner](#) [scanner](#)
- std::string [source](#)
- size_t [index](#) = 0
- const [scan::TToken](#) * [token](#) = nullptr
- std::string [current_function](#)
- std::unordered_map< std::string, std::unordered_set< std::string > > [function_vars](#)

11.108.1 Detailed Description

Validates MXVM source for correct variable/label usage and instruction operands.

Scans the source token stream and checks that all instructions use valid operand patterns, all referenced variables are declared, and all jump targets exist.

Definition at line 69 of file [valid.hpp](#).

11.108.2 Constructor & Destructor Documentation

11.108.2.1 Validator()

```
mxvm::Validator::Validator (
    const std::string & source)
```

Construct a validator from MXVM source code.

Parameters

source	The MXVM source text to validate
------------------------	----------------------------------

Definition at line 796 of file [valid.cpp](#).

References [scanner](#), and [source](#).

11.108.3 Member Function Documentation

11.108.3.1 collect_labels()

```
void mxvm::Validator::collect_labels (
    std::unordered_map< std::string, std::string > & labels)
```

Pre-collect all label declarations and map them to function names.

Definition at line 219 of file [valid.cpp](#).

References [scanner](#), and [types::TT_ID](#).

Referenced by [validate\(\)](#).

11.108.3.2 collect_objects()

```
void mxvm::Validator::collect_objects (
    std::unordered_set< std::string > & objects,
    size_t start_index,
    size_t end_index)
```

Collect all object names declared between token indices.

Definition at line 53 of file [valid.cpp](#).

References [filename](#), [scanner](#), and [types::TT_ID](#).

Referenced by [validate\(\)](#).

11.108.3.3 match() [1/2]

```
bool mxvm::Validator::match (
    const std::string & m)
```

Check if the current token value matches and advance.

Definition at line 718 of file [valid.cpp](#).

References [token](#).

Referenced by [parseOperand\(\)](#), [parseOperandList\(\)](#), and [validate\(\)](#).

11.108.3.4 match() [2/2]

```
bool mxvm::Validator::match (
    const types::TokenType & t)
```

Check if the current token type matches and advance.

Definition at line 737 of file [valid.cpp](#).

References [index](#), [scanner](#), and [token](#).

11.108.3.5 next()

```
bool mxvm::Validator::next ()
```

Advance to the next token; returns false at end.

Definition at line 757 of file [valid.cpp](#).

References [index](#), [scanner](#), [token](#), and [types::TT_SYM](#).

Referenced by [parseOperand\(\)](#), [parseOperandList\(\)](#), and [validate\(\)](#).

11.108.3.6 parseOperand()

```
ParsedOp mxvm::Validator::parseOperand ()
```

Parse a single instruction operand from the token stream.

Definition at line 142 of file [valid.cpp](#).

References [mxvm::ParsedOp::at](#), [filename](#), [mxvm::Hex](#), [mxvm::Id](#), [mxvm::ParsedOp::kind](#), [match\(\)](#), [mxvm::Member](#), [next\(\)](#), [mxvm::Num](#), [peekIs\(\)](#), [require\(\)](#), [mxvm::Str](#), [mxvm::ParsedOp::text](#), [token](#), [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), and [types::TT_STR](#).

Referenced by [parseOperandList\(\)](#).

11.108.3.7 parseOperandList()

```
std::vector< ParsedOp > mxvm::Validator::parseOperandList ()
```

Parse a comma-separated list of operands.

Definition at line 196 of file [valid.cpp](#).

References [filename](#), [match\(\)](#), [next\(\)](#), [parseOperand\(\)](#), and [token](#).

Referenced by [validate\(\)](#).

11.108.3.8 peekIs() [1/2]

```
bool mxvm::Validator::peekIs (
    const std::string & s)
```

Check if the current token value matches without consuming.

Definition at line 771 of file [valid.cpp](#).

References [index](#), and [scanner](#).

Referenced by [parseOperand\(\)](#), and [validate\(\)](#).

11.108.3.9 peekIs() [2/2]

```
bool mxvm::Validator::peekIs (
    const types::TokenType & t)
```

Check if the current token type matches without consuming.

Definition at line 775 of file [valid.cpp](#).

References [index](#), and [scanner](#).

11.108.3.10 require() [1/2]

```
void mxvm::Validator::require (
    const std::string & r)
```

Require the current token value to match, or report an error.

Definition at line 726 of file [valid.cpp](#).

References [filename](#), and [token](#).

Referenced by [parseOperand\(\)](#), and [validate\(\)](#).

11.108.3.11 require() [2/2]

```
void mxvm::Validator::require (
    const types::TokenType & t)
```

Require the current token type to match, or report an error.

Definition at line 745 of file [valid.cpp](#).

References [filename](#), [index](#), [scanner](#), [token](#), and [tokenTypeToString\(\)](#).

11.108.3.12 tokenTypeToString()

```
std::string mxvm::Validator::tokenTypeToString (
    types::TokenType t)
```

Convert a TokenType to its display string.

Definition at line 779 of file [valid.cpp](#).

References [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), [types::TT_STR](#), and [types::TT_SYM](#).

Referenced by [require\(\)](#).

11.108.3.13 validate()

```
bool mxvm::Validator::validate (
    const std::string & name)
```

Run full validation on the source program.

Parameters

<i>name</i>	Program name (for error messages)
-------------	-----------------------------------

Returns

true if the program passes all checks

Definition at line 368 of file [valid.cpp](#).

References [collect_labels\(\)](#), [collect_objects\(\)](#), [filename](#), [mxvm::has_semicolon\(\)](#), [IncType](#), [index](#), [match\(\)](#), [next\(\)](#), [parseOperandList\(\)](#), [peekIs\(\)](#), [require\(\)](#), [scanner](#), [source](#), [token](#), [types::TT_HEX](#), [types::TT_ID](#), [types::TT_NUM](#), [types::TT_STR](#), [types::TT_SYM](#), and [validateAgainstSpec\(\)](#).

Referenced by [mxvm::Parser::processObjectFile\(\)](#).

11.108.3.14 validateAgainstSpec()

```
void mxvm::Validator::validateAgainstSpec (
    const std::string & op,
    const std::vector< ParsedOp > & ops,
    const std::unordered_map< std::string, Variable > & vars,
    const std::unordered_map< std::string, std::string > & labels,
    const std::unordered_set< std::string > & objects,
    std::vector< UseVar > & usedVarsRef,
    std::vector< UseLabel > & usedLabelsRef)
```

Validate an instruction's operands against the spec table.

Parameters

<i>op</i>	Instruction mnemonic
<i>ops</i>	Parsed operand list
<i>vars</i>	Declared variables map
<i>labels</i>	Declared labels map
<i>objects</i>	Declared object names
<i>usedVarsRef</i>	Output: variable usage records
<i>usedLabelsRef</i>	Output: label usage records

Definition at line 230 of file [valid.cpp](#).

References [mxvm::Any](#), [mxvm::ArgsTail](#), [filename](#), [mxvm::OpSpec::fixed](#), [mxvm::Id](#), [mxvm::isIdLike\(\)](#), [mxvm::isImmediate\(\)](#), [mxvm::kOpSpecs](#), [mxvm::Label](#), [mxvm::OpSpec::maxArgs](#), [mxvm::Member](#), [mxvm::OpSpec::minArgs](#), [mxvm::None](#), and [mxvm::OpSpec::varPolicy](#).

Referenced by [validate\(\)](#).

11.108.4 Member Data Documentation**11.108.4.1 current_function**

```
std::string mxvm::Validator::current_function [private]
```

Definition at line 76 of file [valid.hpp](#).

11.108.4.2 filename

```
std::string mxvm::Validator::filename [private]
```

Definition at line 71 of file [valid.hpp](#).

Referenced by [collect_objects\(\)](#), [parseOperand\(\)](#), [parseOperandList\(\)](#), [require\(\)](#), [require\(\)](#), [validate\(\)](#), and [validateAgainstSpec\(\)](#).

11.108.4.3 function_vars

```
std::unordered_map<std::string, std::unordered_set<std::string> > mxvm::Validator::function↵
_vars [private]
```

Definition at line 77 of file [valid.hpp](#).

11.108.4.4 index

`size_t mxvm::Validator::index = 0 [private]`

Definition at line 74 of file [valid.hpp](#).

Referenced by [match\(\)](#), [next\(\)](#), [peekls\(\)](#), [peekls\(\)](#), [require\(\)](#), and [validate\(\)](#).

11.108.4.5 scanner

`scan::Scanner mxvm::Validator::scanner [private]`

Definition at line 72 of file [valid.hpp](#).

Referenced by [Validator\(\)](#), [collect_labels\(\)](#), [collect_objects\(\)](#), [match\(\)](#), [next\(\)](#), [peekls\(\)](#), [peekls\(\)](#), [require\(\)](#), and [validate\(\)](#).

11.108.4.6 source

`std::string mxvm::Validator::source [private]`

Definition at line 73 of file [valid.hpp](#).

Referenced by [Validator\(\)](#), and [validate\(\)](#).

11.108.4.7 token

`const scan::Token* mxvm::Validator::token = nullptr [private]`

Definition at line 75 of file [valid.hpp](#).

Referenced by [match\(\)](#), [match\(\)](#), [next\(\)](#), [parseOperand\(\)](#), [parseOperandList\(\)](#), [require\(\)](#), [require\(\)](#), and [validate\(\)](#).

11.108.4.8 var_names

`std::vector<VarNamePair> mxvm::Validator::var_names`

Definition at line 90 of file [valid.hpp](#).

Referenced by [mxvm::Program::validateNames\(\)](#).

The documentation for this class was generated from the following files:

- [include/mxvm/valid.hpp](#)
- [src/valid.cpp](#)

11.109 pascal::CodeGenVisitor::ValueLocation Struct Reference

Describes where a value currently resides (register, memory, or immediate).

Public Types

- enum [Type](#) { [REGISTER](#) , [MEMORY](#) , [IMMEDIATE](#) }

Public Attributes

- enum [pascal::CodeGenVisitor::ValueLocation::Type](#) type
- `std::string` [location](#)

register name, slot name, or literal value

11.109.1 Detailed Description

Describes where a value currently resides (register, memory, or immediate).

Definition at line 358 of file [icode.hpp](#).

11.109.2 Member Enumeration Documentation

11.109.2.1 Type

enum [pascal::CodeGenVisitor::ValueLocation::Type](#)

Enumerator

REGISTER	value is in a named register
MEMORY	value is in a memory slot
IMMEDIATE	value is a compile-time constant

Definition at line 359 of file [icode.hpp](#).

11.109.3 Member Data Documentation

11.109.3.1 location

std::string [pascal::CodeGenVisitor::ValueLocation::location](#)
register name, slot name, or literal value

Definition at line 364 of file [icode.hpp](#).

11.109.3.2 type

enum [pascal::CodeGenVisitor::ValueLocation::Type](#) [pascal::CodeGenVisitor::ValueLocation::type](#)

The documentation for this struct was generated from the following file:

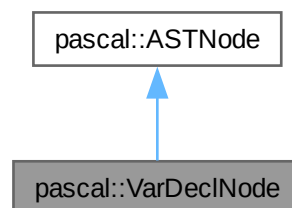
- [src/frontend/include/icode.hpp](#)

11.110 pascal::VarDeclNode Class Reference

AST node for variable declarations (var id1, id2 : type).

#include <ast.hpp>

Inheritance diagram for pascal::VarDeclNode:



Public Member Functions

- [VarDeclNode](#) (std::vector< std::string > [identifiers](#), std::variant< std::string, std::unique_ptr< [ASTNode](#) > > [type](#), std::vector< std::unique_ptr< [ASTNode](#) > > [initializers](#))
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from `pascal::ASTNode`

- virtual `~ASTNode()` = default
- virtual void `print` (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void `setLineNumber` (int line)
Set the originating source line number.
- int `getLineNumber` () const
Get the originating source line number.

Public Attributes

- std::vector< std::string > `identifiers`
declared variable names
- std::variant< std::string, std::unique_ptr< `ASTNode` > > `type`
type (simple name or complex type node)
- std::vector< std::unique_ptr< `ASTNode` > > `initializers`
optional initialisers

11.110.1 Detailed Description

AST node for variable declarations (var id1, id2 : type).
Definition at line 132 of file `ast.hpp`.

11.110.2 Constructor & Destructor Documentation

11.110.2.1 VarDeclNode()

```
pascal::VarDeclNode::VarDeclNode (
    std::vector< std::string > identifiers,
    std::variant< std::string, std::unique_ptr< ASTNode > > type,
    std::vector< std::unique_ptr< ASTNode > > initializers) [inline]
```

Definition at line 138 of file `ast.hpp`.
References `identifiers`, `initializers`, and `type`.

11.110.3 Member Function Documentation

11.110.3.1 accept()

```
void pascal::VarDeclNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an `ASTVisitor` (double-dispatch).
Implements `pascal::ASTNode`.
Definition at line 47 of file `ast.cpp`.
References `pascal::ASTVisitor::visit()`.

11.110.3.2 toString()

```
std::string pascal::VarDeclNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.
Implements `pascal::ASTNode`.
Definition at line 51 of file `ast.cpp`.
References `identifiers`, and `type`.

11.110.4 Member Data Documentation

11.110.4.1 identifiers

`std::vector<std::string> pascal::VarDeclNode::identifiers`

declared variable names

Definition at line 134 of file [ast.hpp](#).

Referenced by [VarDeclNode\(\)](#), [pascal::CodeGenVisitor::registerImportedVar\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.110.4.2 initializers

`std::vector<std::unique_ptr<ASTNode> > pascal::VarDeclNode::initializers`

optional initialisers

Definition at line 136 of file [ast.hpp](#).

Referenced by [VarDeclNode\(\)](#).

11.110.4.3 type

`std::variant<std::string, std::unique_ptr<ASTNode> > pascal::VarDeclNode::type`

type (simple name or complex type node)

Definition at line 135 of file [ast.hpp](#).

Referenced by [VarDeclNode\(\)](#), [pascal::CodeGenVisitor::getTypeString\(\)](#), [pascal::CodeGenVisitor::registerImportedVar\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.111 mxvm::Variable Struct Reference

A named variable with type, value, and optional object association.

`#include <instruct.hpp>`

Public Member Functions

- [Variable](#) ()
Default constructor — VAR_NULL type with empty name and value.
- [Variable](#) (const std::string &name, const [VarType](#) &vtype, const [Variable_Value](#) &value)
Construct a variable with the given name, type, and initial value.
- [Variable](#) (const [Variable](#) &v)
Copy constructor.
- void [setExtern](#) (std::string name, void *value)
Configure this variable as an external pointer reference.
- [Variable](#) & [operator=](#) (const [Variable](#) &v)
Copy-assignment operator.
- std::string [toString](#) () const
Convert this variable to a human-readable string.

Public Attributes

- [VarType](#) type
- std::string [var_name](#)
- [Variable_Value](#) [var_value](#)
- bool [is_global](#) = false
- std::string [obj_name](#)

11.111.1 Detailed Description

A named variable with type, value, and optional object association.
Definition at line 292 of file [instruct.hpp](#).

11.111.2 Constructor & Destructor Documentation

11.111.2.1 Variable() [1/3]

```
mxvm::Variable::Variable () [inline]
```

Default constructor — VAR_NULL type with empty name and value.

Definition at line 299 of file [instruct.hpp](#).

References [type](#), [var_name](#), and [var_value](#).

Referenced by [Variable\(\)](#), and [operator=\(\)](#).

11.111.2.2 Variable() [2/3]

```
mxvm::Variable::Variable (
    const std::string & name,
    const VarType & vtype,
    const Variable_Value & value) [inline]
```

Construct a variable with the given name, type, and initial value.

Parameters

<i>name</i>	Variable identifier
<i>vtype</i>	Variable type discriminator
<i>value</i>	Initial runtime value

Definition at line 306 of file [instruct.hpp](#).

References [is_global](#), [type](#), [var_name](#), and [var_value](#).

11.111.2.3 Variable() [3/3]

```
mxvm::Variable::Variable (
    const Variable & v) [inline]
```

Copy constructor.

Parameters

<i>v</i>	Source variable to copy
----------	-------------------------

Definition at line 312 of file [instruct.hpp](#).

References [Variable\(\)](#), [is_global](#), [obj_name](#), [type](#), [var_name](#), and [var_value](#).

11.111.3 Member Function Documentation

11.111.3.1 operator=()

```
Variable & mxvm::Variable::operator= (
    const Variable & v) [inline]
```

Copy-assignment operator.

Parameters

<i>v</i>	Source variable to copy
----------	-------------------------

Returns

Reference to this

Definition at line 333 of file [instruct.hpp](#).

References [Variable\(\)](#), [is_global](#), [obj_name](#), [type](#), [var_name](#), and [var_value](#).

11.111.3.2 setExtern()

```
void mxvm::Variable::setExtern (
    std::string name,
    void * value) [inline]
```

Configure this variable as an external pointer reference.

Parameters

<i>name</i>	External variable name
<i>value</i>	Pointer to the external data

Definition at line 319 of file [instruct.hpp](#).

References [type](#), [mxvm::VAR_EXTERN](#), [var_name](#), and [var_value](#).

Referenced by [mxvm::Program::add_standard\(\)](#).

11.111.3.3 toString()

```
std::string mxvm::Variable::toString () const
```

Convert this variable to a human-readable string.

Returns

Formatted string describing the variable name, type, and value

Definition at line 90 of file [instruct.cpp](#).

References [type](#).

Referenced by [mxvm::Program::exec_cmp\(\)](#).

11.111.4 Member Data Documentation**11.111.4.1 is_global**

```
bool mxvm::Variable::is_global = false
```

Definition at line 296 of file [instruct.hpp](#).

Referenced by [Variable\(\)](#), [Variable\(\)](#), [operator=\(\)](#), [mxvm::Program::sysv_analyzeRegAlloc\(\)](#), and [mxvm::Program::x64_analyzeRegAlloc\(\)](#).

11.111.4.2 obj_name

```
std::string mxvm::Variable::obj_name
```

Definition at line 297 of file [instruct.hpp](#).

Referenced by [Variable\(\)](#), [mxvm::Program::getMangledName\(\)](#), and [operator=\(\)](#).

11.111.4.3 type

```
VarType mxvm::Variable::type
```

Definition at line 293 of file [instruct.hpp](#).

Referenced by [Variable\(\)](#), [Variable\(\)](#), [Variable\(\)](#), [mxvm::Program::addVariables\(\)](#), [asReadPtr\(\)](#), [asWritePtr\(\)](#), [mxvm::Program::createTempVariable\(\)](#), [mxvm::Program::divVariables\(\)](#), [mxvm::Program::exec_add\(\)](#), [mxvm::Program::exec_alloc\(\)](#), [mxvm::Program::exec_cmp\(\)](#), [mxvm::Program::exec_div\(\)](#), [mxvm::Program::exec_fcmp\(\)](#), [mxvm::Program::exec_free\(\)](#), [mxvm::Program::exec_getline\(\)](#), [mxvm::Program::exec_invoke\(\)](#), [mxvm::Program::exec_lea\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_mod\(\)](#), [mxvm::Program::exec_mov\(\)](#), [mxvm::Program::exec_mul\(\)](#), [mxvm::Program::exec_neg\(\)](#), [mxvm::Program::exec_pop\(\)](#), [mxvm::Program::exec_print\(\)](#), [mxvm::Program::exec_realloc\(\)](#), [mxvm::Program::exec_return\(\)](#), [mxvm::Program::exec_stack_load\(\)](#), [mxvm::Program::exec_stack_store\(\)](#), [mxvm::Program::exec_store\(\)](#), [mxvm::Program::exec_string\(\)](#).

mxvm::Program::exec_sub(), mxvm::Program::exec_to_float(), mxvm::Program::exec_to_int(), mxvm::Program::gen_alloc(), mxvm::Program::gen_arth(), mxvm::Program::gen_bitop(), mxvm::Program::gen_cmp(), mxvm::Program::gen_div(), mxvm::Program::gen_free(), mxvm::Program::gen_getline(), mxvm::Program::gen_load(), mxvm::Program::gen_mod(), mxvm::Program::gen_mov(), mxvm::Program::gen_neg(), mxvm::Program::gen_not(), mxvm::Program::gen_pop(), mxvm::Program::gen_print(), mxvm::Program::gen_push(), mxvm::Program::gen_realloc(), mxvm::Program::gen_return(), mxvm::Program::gen_stack_load(), mxvm::Program::gen_stack_store(), mxvm::Program::gen_store(), mxvm::Program::gen_to_float(), mxvm::Program::gen_to_int(), mxvm::Program::generateLoadVar(), mxvm::Program::generateLoadVar(), getStringFromVar(), isStringLike(), mxvm::Program::mulVariables(), mxvm_io_fclose(), mxvm_io_feof(), mxvm_io_fopen(), mxvm_io_fprintf(), mxvm_io_fputs(), mxvm_io_fread(), mxvm_io_fseek(), mxvm_io_fsize(), mxvm_io_fwrite(), mxvm_io_mxvm_fgets(), mxvm_io_rand_number(), mxvm_sdl_create_window(), mxvm_sdl_draw_text(), mxvm_sdl_get_keyboard_state(), mxvm_sdl_get_mouse_state(), mxvm_sdl_get_relative_mouse_state(), mxvm_sdl_get_renderer_output_size(), mxvm_sdl_get_window_size(), mxvm_sdl_load_font(), mxvm_sdl_load_texture(), mxvm_sdl_load_texture_color_key(), mxvm_sdl_load_texture_color_key_rgb(), mxvm_sdl_load_wav(), mxvm_sdl_lock_texture(), mxvm_sdl_queue_audio(), mxvm_sdl_set_clipboard_text(), mxvm_sdl_set_window_icon(), mxvm_sdl_set_window_title(), mxvm_sdl_update_texture(), mxvm_std_argv(), mxvm_std_atof(), mxvm_std_atoi(), mxvm_std_ceil(), mxvm_std_cos(), mxvm_std_fabs(), mxvm_std_float_to_int(), mxvm_std_floor(), mxvm_std_int_to_float(), mxvm_std_memcmp(), mxvm_std_memcpy(), mxvm_std_memmove(), mxvm_std_memset(), mxvm_std_pow(), mxvm_std_release(), mxvm_std_set_program_args(), mxvm_std_sin(), mxvm_std_sqrt(), mxvm_std_system(), mxvm_std_tan(), mxvm_string_snprintf(), mxvm_string_strcmp(), mxvm_string_strlen(), mxvm_string_strncat(), mxvm_string_strncpy(), mxvm_string_substr(), operator=(), mxvm::Program::printfFormatted(), mxvm::releaseOwnedPointer(), setExtern(), mxvm::Program::setVariableFromConstant(), mxvm::Program::setVariableFromString(), mxvm::Program::subVariables(), mxvm::Program::sysv_analyzeRegAlloc(), toString(), mxvm::Program::x64_analyzeRegAlloc(), mxvm::Program::x64_gen_alloc(), mxvm::Program::x64_gen_arth(), mxvm::Program::x64_gen_bitop(), mxvm::Program::x64_gen_cmp(), mxvm::Program::x64_gen_div(), mxvm::Program::x64_gen_free(), mxvm::Program::x64_gen_getline(), mxvm::Program::x64_gen_load(), mxvm::Program::x64_gen_mod(), mxvm::Program::x64_gen_r, mxvm::Program::x64_gen_neg(), mxvm::Program::x64_gen_not(), mxvm::Program::x64_gen_pop(), mxvm::Program::x64_gen_push, mxvm::Program::x64_gen_realloc(), mxvm::Program::x64_gen_store(), mxvm::Program::x64_generateFunctionCall(), mxvm::Program::x64_generateLoadVar(), and mxvm::Program::x64_generateLoadVar().

11.111.4.4 var_name

std::string mxvm::Variable::var_name

Definition at line 294 of file `instruct.hpp`.

Referenced by `Variable()`, `Variable()`, `Variable()`, `mxvm::Program::createTempVariable()`, `mxvm::Program::exec_return()`, `mxvm::Program::exec_store()`, `mxvm_string_strcmp()`, `mxvm_string_strlen()`, `operator=()`, and `setExtern()`.

11.111.4.5 var_value

Variable_Value mxvm::Variable::var_value

Definition at line 295 of file `instruct.hpp`.

Referenced by `Variable()`, `Variable()`, `Variable()`, `mxvm::Program::addVariables()`, `asReadPtr()`, `asWritePtr()`, `mxvm::Program::createTempVariable()`, `mxvm::Program::divVariables()`, `mxvm::Program::exec_alloc()`, `mxvm::Program::exec_and()`, `mxvm::Program::exec_cmp()`, `mxvm::Program::exec_exit()`, `mxvm::Program::exec_fcmp()`, `mxvm::Program::exec_free()`, `mxvm::Program::exec_getline()`, `mxvm::Program::exec_invoke()`, `mxvm::Program::exec_lea()`, `mxvm::Program::exec_load()`, `mxvm::Program::exec_mod()`, `mxvm::Program::exec_mov()`, `mxvm::Program::exec_neg()`, `mxvm::Program::exec_not()`, `mxvm::Program::exec_or()`, `mxvm::Program::exec_pop()`, `mxvm::Program::exec_print()`, `mxvm::Program::exec_push()`, `mxvm::Program::exec_realloc()`, `mxvm::Program::exec_return()`, `mxvm::Program::exec_stack_load()`, `mxvm::Program::exec_stack_store()`, `mxvm::Program::exec_stack_sub()`, `mxvm::Program::exec_store()`, `mxvm::Program::exec_string_print()`, `mxvm::Program::exec_to_float()`, `mxvm::Program::exec_to_int()`, `mxvm::Program::exec_xor()`, `mxvm::Program::gen_alloc()`, `mxvm::Program::gen_free()`, `mxvm::Program::gen_getline()`, `mxvm::Program::gen_lea()`, `mxvm::Program::gen_realloc()`, `mxvm::Program::gen_return()`, `mxvm::Program::gen_store()`, `getStringFromVar()`, `mxvm::Program::mulVariables()`, `mxvm_io_fclose()`, `mxvm_io_feof()`, `mxvm_io_fopen()`, `mxvm_io_fprintf()`, `mxvm_io_fputs()`, `mxvm_io_fread()`, `mxvm_io_fseek()`, `mxvm_io_fsize()`, `mxvm_io_fwrite()`, `mxvm_io_mxvm_fgets()`, `mxvm_io_rand_number()`, `mxvm_sdl_blit_surface()`, `mxvm_sdl_clear()`, `mxvm_sdl_create_render_target()`, `mxvm_sdl_create_renderer()`, `mxvm_sdl_create_rgb_surface()`, `mxvm_sdl_create_texture()`, `mxvm_sdl_create_window()`, `mxvm_sdl_delay()`, `mxvm_sdl_destroy_render_target()`, `mxvm_sdl_destroy_renderer()`, `mxvm_sdl_destroy_texture()`, `mxvm_sdl_destroy_window()`, `mxvm_sdl_draw_line()`, `mxvm_sdl_draw_point()`, `mxvm_sdl_draw_rect()`, `mxvm_sdl_draw_text()`, `mxvm_sdl_fill_rect()`, `mxvm_sdl_free_surface()`, `mxvm_sdl_free_wav()`, `mxvm_sdl_get_keyboard_state()`, `mxvm_sdl_get_mouse_state()`, `mxvm_sdl_get_relative_mouse_state()`, `mxvm_sdl_get_renderer_output_size()`, `mxvm_sdl_get_window_size()`, `mxvm_sdl_is_key_pressed()`, `mxvm_sdl_load_font()`, `mxvm_sdl_load_texture()`, `mxvm_sdl_load_texture_color_key()`, `mxvm_sdl_load_texture_color_key_rgb()`, `mxvm_sdl_load_wav()`, `mxvm_sdl_lock_texture()`,

mxvm_sdl_open_audio(), mxvm_sdl_pause_audio(), mxvm_sdl_present(), mxvm_sdl_present_scaled(), mxvm_sdl_present_stretch(), mxvm_sdl_queue_audio(), mxvm_sdl_render_texture(), mxvm_sdl_set_clipboard_text(), mxvm_sdl_set_draw_color(), mxvm_sdl_set_render_target(), mxvm_sdl_set_window_fullscreen(), mxvm_sdl_set_window_icon(), mxvm_sdl_set_window_position(), mxvm_sdl_set_window_title(), mxvm_sdl_show_cursor(), mxvm_sdl_unlock_texture(), mxvm_sdl_update_texture(), mxvm_std_abs(), mxvm_std_acos(), mxvm_std_argv(), mxvm_std_asin(), mxvm_std_atan(), mxvm_std_atan2(), mxvm_std_atof(), mxvm_std_atoi(), mxvm_std_calloc(), mxvm_std_ceil(), mxvm_std_cos(), mxvm_std_cosh(), mxvm_std_exit(), mxvm_std_exp(), mxvm_std_exp2(), mxvm_std_fabs(), mxvm_std_float_to_int(), mxvm_std_floor(), mxvm_std_fmod(), mxvm_std_hypot(), mxvm_std_int_to_float(), mxvm_std_isalpha(), mxvm_std_isdigit(), mxvm_std_isspace(), mxvm_std_log(), mxvm_std_log10(), mxvm_std_log2(), mxvm_std_malloc(), mxvm_std_memcmp(), mxvm_std_memcpy(), mxvm_std_memmove(), mxvm_std_memset(), mxvm_std_pow(), mxvm_std_release(), mxvm_std_round(), mxvm_std_set_program_args(), mxvm_std_sin(), mxvm_std_sinh(), mxvm_std_sqrt(), mxvm_std_srand(), mxvm_std_system(), mxvm_std_tan(), mxvm_std_tanh(), mxvm_std_tolower(), mxvm_std_toupper(), mxvm_std_trunc(), mxvm_string_copy(), mxvm_string_delete(), mxvm_string_insert(), mxvm_string_inttostr(), mxvm_string_snprintf(), mxvm_string_strat(), mxvm_string_strcmp(), mxvm_string_strfind(), mxvm_string_strlen(), mxvm_string_strncat(), mxvm_string_strncpy(), mxvm_string_substr(), operator=(), mxvm::Program::printFormatted(), mxvm::releaseOwnedPointer(), setExtern(), mxvm::Program::setVariableFromConstant(), mxvm::Program::setVariableFromString(), mxvm::Program::subVariables(), mxvm::Program::x64_gen_alloc(), mxvm::Program::x64_gen_free(), mxvm::Program::x64_gen_getli(), mxvm::Program::x64_gen_lea(), mxvm::Program::x64_gen_load(), mxvm::Program::x64_gen_mov(), mxvm::Program::x64_gen_reali(), mxvm::Program::x64_gen_return(), and mxvm::Program::x64_gen_store().

The documentation for this struct was generated from the following files:

- include/mxvm/instruct.hpp
- src/instruct.cpp

11.112 mxvm::Variable_Value Struct Reference

Runtime value storage for a variable.

```
#include <instruct.hpp>
```

Public Member Functions

- [Variable_Value](#) (const [Variable_Value](#) &other)
Copy constructor — deep-copies all fields, dispatching on type for the union member.
- [Variable_Value](#) & [operator=](#) (const [Variable_Value](#) &other)
Copy-assignment operator — self-check then deep copy, dispatching on type.
- [Variable_Value](#) ()
Default constructor — initialises to VAR_NULL with zeroed fields.

Public Attributes

- std::string [str_value](#)
- std::string [label_value](#)
- union {
 int64_t [int_value](#)
 double [float_value](#)
 void * [ptr_value](#)
};
- uint64_t [ptr_size](#) = 0
size of each element in allocated memory
- uint64_t [ptr_count](#) = 0
number of elements allocated
- [VarType](#) [type](#)
- uint64_t [buffer_size](#)
declared buffer size
- bool [owns](#) = false

true if this value owns the allocated memory

- bool `released` = false

true if the owned memory has been freed

11.112.1 Detailed Description

Runtime value storage for a variable.

Uses a union for int/float/pointer storage. Tracks pointer ownership and allocation state for memory management.

Definition at line 210 of file `instruct.hpp`.

11.112.2 Constructor & Destructor Documentation

11.112.2.1 Variable_Value() [1/2]

```
mxvm::Variable_Value::Variable_Value (
    const Variable_Value & other) [inline]
```

Copy constructor — deep-copies all fields, dispatching on type for the union member.

Parameters

<i>other</i>	Source value to copy from
--------------	---------------------------

Definition at line 228 of file `instruct.hpp`.

References `Variable_Value()`, `buffer_size`, `float_value`, `int_value`, `label_value`, `owns`, `ptr_count`, `ptr_size`, `ptr_value`, `released`, `str_value`, `type`, `mxvm::VAR_BYTE`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, and `mxvm::VAR_POINTER`.

Referenced by `Variable_Value()`, and `operator=()`.

11.112.2.2 Variable_Value() [2/2]

```
mxvm::Variable_Value::Variable_Value () [inline]
```

Default constructor — initialises to VAR_NULL with zeroed fields.

Definition at line 288 of file `instruct.hpp`.

References `buffer_size`, `int_value`, `owns`, `ptr_count`, `ptr_size`, `released`, and `type`.

11.112.3 Member Function Documentation

11.112.3.1 operator=()

```
Variable_Value & mxvm::Variable_Value::operator= (
    const Variable_Value & other) [inline]
```

Copy-assignment operator — self-check then deep copy, dispatching on type.

Parameters

<i>other</i>	Source value to copy from
--------------	---------------------------

Returns

Reference to this

Definition at line 258 of file `instruct.hpp`.

References `Variable_Value()`, `buffer_size`, `float_value`, `int_value`, `label_value`, `owns`, `ptr_count`, `ptr_size`, `ptr_value`, `released`, `str_value`, `type`, `mxvm::VAR_BYTE`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, and `mxvm::VAR_POINTER`.

11.112.4 Member Data Documentation

11.112.4.1 [union]

```
union { ... } mxvm::Variable_Value
```

11.112.4.2 buffer_size

```
uint64_t mxvm::Variable_Value::buffer_size
```

declared buffer size

Definition at line 221 of file `instruct.hpp`.

Referenced by `Variable_Value()`, `Variable_Value()`, `mxvm::Program::gen_getline()`, `mxvm::Program::gen_store()`, `mxvm_string_strncat()`, `mxvm_string_strncpy()`, `mxvm_string_substr()`, `operator=()`, `mxvm::Program::x64_gen_getline()`, `mxvm::Program::x64_gen_load()`, and `mxvm::Program::x64_gen_store()`.

11.112.4.3 float_value

```
double mxvm::Variable_Value::float_value
```

Definition at line 215 of file `instruct.hpp`.

Referenced by `Variable_Value()`, `mxvm::Program::addVariables()`, `mxvm::Program::createTempVariable()`, `mxvm::Program::divVariables()`, `mxvm::Program::exec_cmp()`, `mxvm::Program::exec_fcmp()`, `mxvm::Program::exec_lea()`, `mxvm::Program::exec_load()`, `mxvm::Program::exec_mod()`, `mxvm::Program::exec_mov()`, `mxvm::Program::exec_neg()`, `mxvm::Program::exec_pop()`, `mxvm::Program::exec_push()`, `mxvm::Program::exec_stack_load()`, `mxvm::Program::exec_stack_store()`, `mxvm::Program::exec_store()`, `mxvm::Program::exec_string_print()`, `mxvm::Program::exec_to_float()`, `mxvm::Program::exec_to_int()`, `mxvm::Program::mulVariables()`, `mxvm_std_acos()`, `mxvm_std_asin()`, `mxvm_std_atan()`, `mxvm_std_atan2()`, `mxvm_std_ceil()`, `mxvm_std_cos()`, `mxvm_std_cosh()`, `mxvm_std_exp()`, `mxvm_std_exp2()`, `mxvm_std_fabs()`, `mxvm_std_float_to_int()`, `mxvm_std_floor()`, `mxvm_std_fmod()`, `mxvm_std_hypot()`, `mxvm_std_int_to_float()`, `mxvm_std_log()`, `mxvm_std_log10()`, `mxvm_std_log2()`, `mxvm_std_pow()`, `mxvm_std_round()`, `mxvm_std_sin()`, `mxvm_std_sinh()`, `mxvm_std_sqrt()`, `mxvm_std_tan()`, `mxvm_std_tanh()`, `mxvm_std_trunc()`, `mxvm_string_snprintf()`, `operator=()`, `mxvm::Program::printFormatted()`, `mxvm::Program::setVariableFromConstant()`, `mxvm::Program::setVariableFromString()`, and `mxvm::Program::subVariables()`.

11.112.4.4 int_value

```
int64_t mxvm::Variable_Value::int_value
```

Definition at line 214 of file `instruct.hpp`.

Referenced by `Variable_Value()`, `Variable_Value()`, `mxvm::Program::addVariables()`, `mxvm::Program::createTempVariable()`, `mxvm::Program::divVariables()`, `mxvm::Program::exec_alloc()`, `mxvm::Program::exec_and()`, `mxvm::Program::exec_cmp()`, `mxvm::Program::exec_exit()`, `mxvm::Program::exec_fcmp()`, `mxvm::Program::exec_lea()`, `mxvm::Program::exec_load()`, `mxvm::Program::exec_mod()`, `mxvm::Program::exec_mov()`, `mxvm::Program::exec_neg()`, `mxvm::Program::exec_not()`, `mxvm::Program::exec_or()`, `mxvm::Program::exec_pop()`, `mxvm::Program::exec_push()`, `mxvm::Program::exec_realloc()`, `mxvm::Program::exec_stack_load()`, `mxvm::Program::exec_stack_store()`, `mxvm::Program::exec_stack_sub()`, `mxvm::Program::exec_store()`, `mxvm::Program::exec_string_print()`, `mxvm::Program::exec_to_float()`, `mxvm::Program::exec_to_int()`, `mxvm::Program::exec_xor()`, `mxvm::Program::mulVariables()`, `mxvm_io_fread()`, `mxvm_io_fseek()`, `mxvm_io_fwrite()`, `mxvm_io_rand_number()`, `mxvm_sdl_blit_surface()`, `mxvm_sdl_clear()`, `mxvm_sdl_create_render_target()`, `mxvm_sdl_create_renderer()`, `mxvm_sdl_create_rgb_surface()`, `mxvm_sdl_create_texture()`, `mxvm_sdl_create_window()`, `mxvm_sdl_delay()`, `mxvm_sdl_destroy_render_target()`, `mxvm_sdl_destroy_renderer()`, `mxvm_sdl_destroy_texture()`, `mxvm_sdl_destroy_window()`, `mxvm_sdl_draw_line()`, `mxvm_sdl_draw_point()`, `mxvm_sdl_draw_rect()`, `mxvm_sdl_draw_text()`, `mxvm_sdl_fill_rect()`, `mxvm_sdl_free_surface()`, `mxvm_sdl_free_wav()`, `mxvm_sdl_get_keyboard_state()`, `mxvm_sdl_get_mouse_state()`, `mxvm_sdl_get_relative_mouse_state()`, `mxvm_sdl_get_renderer_output_size()`, `mxvm_sdl_get_window_size()`, `mxvm_sdl_is_key_pressed()`, `mxvm_sdl_load_font()`, `mxvm_sdl_load_texture()`, `mxvm_sdl_load_texture_color_key()`, `mxvm_sdl_load_texture_color_key_rgb()`, `mxvm_sdl_load_wav()`, `mxvm_sdl_lock_texture()`, `mxvm_sdl_open_audio()`, `mxvm_sdl_pause_audio()`, `mxvm_sdl_present()`, `mxvm_sdl_present_scaled()`, `mxvm_sdl_present_stretch()`, `mxvm_sdl_queue_audio()`, `mxvm_sdl_render_texture()`, `mxvm_sdl_set_draw_color()`, `mxvm_sdl_set_render_target()`, `mxvm_sdl_set_window_fullscreen()`, `mxvm_sdl_set_window_icon()`, `mxvm_sdl_set_window_position()`, `mxvm_sdl_set_window_title()`, `mxvm_sdl_show_cursor()`, `mxvm_sdl_unlock_texture()`, `mxvm_sdl_update_texture()`, `mxvm_std_abs()`, `mxvm_std_acos()`, `mxvm_std_argv()`, `mxvm_std_asin()`, `mxvm_std_atan()`, `mxvm_std_atan2()`, `mxvm_std_calloc()`, `mxvm_std_ceil()`, `mxvm_std_cos()`, `mxvm_std_cosh()`, `mxvm_std_exit()`, `mxvm_std_exp()`, `mxvm_std_exp2()`, `mxvm_std_fabs()`, `mxvm_std_float_to_int()`, `mxvm_std_floor()`, `mxvm_std_fmod()`, `mxvm_std_hypot()`, `mxvm_std_int_to_float()`, `mxvm_std_isalpha()`, `mxvm_std_isdigit()`, `mxvm_std_isspace()`, `mxvm_std_log()`, `mxvm_std_log10()`, `mxvm_std_log2()`,

[mxvm_std_malloc\(\)](#), [mxvm_std_memcmp\(\)](#), [mxvm_std_memcpy\(\)](#), [mxvm_std_memmove\(\)](#), [mxvm_std_memset\(\)](#), [mxvm_std_pow\(\)](#), [mxvm_std_round\(\)](#), [mxvm_std_set_program_args\(\)](#), [mxvm_std_sin\(\)](#), [mxvm_std_sinh\(\)](#), [mxvm_std_sqrt\(\)](#), [mxvm_std_srand\(\)](#), [mxvm_std_tan\(\)](#), [mxvm_std_tanh\(\)](#), [mxvm_std_tolower\(\)](#), [mxvm_std_toupper\(\)](#), [mxvm_std_trunc\(\)](#), [mxvm_string_copy\(\)](#), [mxvm_string_delete\(\)](#), [mxvm_string_insert\(\)](#), [mxvm_string_inttostr\(\)](#), [mxvm_string_snprintf\(\)](#), [mxvm_string_strat\(\)](#), [mxvm_string_strfind\(\)](#), [mxvm_string_strncat\(\)](#), [mxvm_string_strncpy\(\)](#), [mxvm_string_substr\(\)](#), [operator=\(\)](#), [mxvm::Program::printFormatted\(\)](#), [mxvm::Program::setVariableFromConstant\(\)](#), [mxvm::Program::setVariableFromString\(\)](#), and [mxvm::Program::subVariables\(\)](#).

11.112.4.5 label_value

`std::string mxvm::Variable_Value::label_value`

Definition at line 212 of file [instruct.hpp](#).

Referenced by [Variable_Value\(\)](#), and [operator=\(\)](#).

11.112.4.6 owns

`bool mxvm::Variable_Value::owns = false`

true if this value owns the allocated memory

Definition at line 222 of file [instruct.hpp](#).

Referenced by [Variable_Value\(\)](#), [Variable_Value\(\)](#), [mxvm::Program::addVariables\(\)](#), [mxvm::Program::exec_alloc\(\)](#), [mxvm::Program::exec_free\(\)](#), [mxvm::Program::exec_lea\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_mov\(\)](#), [mxvm::Program::exec_realloc\(\)](#), [mxvm::Program::exec_return\(\)](#), [mxvm::Program::exec_store\(\)](#), [mxvm::Program::gen_alloc\(\)](#), [mxvm::Program::gen_free\(\)](#), [mxvm::Program::gen_lea\(\)](#), [mxvm::Program::gen_realloc\(\)](#), [mxvm::Program::gen_return\(\)](#), [mxvm_std_release\(\)](#), [operator=\(\)](#), [mxvm::releaseOwnedPointer\(\)](#), [mxvm::Program::setVariableFromConstant\(\)](#), [mxvm::Program::subVariables\(\)](#), [mxvm::Program::x64_gen_alloc\(\)](#), [mxvm::Program::x64_gen_free\(\)](#), [mxvm::Program::x64_gen_lea\(\)](#), [mxvm::Program::x64_gen_mov\(\)](#), [mxvm::Program::x64_gen_realloc\(\)](#), and [mxvm::Program::x64_gen_return\(\)](#).

11.112.4.7 ptr_count

`uint64_t mxvm::Variable_Value::ptr_count = 0`

number of elements allocated

Definition at line 219 of file [instruct.hpp](#).

Referenced by [Variable_Value\(\)](#), [Variable_Value\(\)](#), [mxvm::Program::exec_alloc\(\)](#), [mxvm::Program::exec_free\(\)](#), [mxvm::Program::exec_lea\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_mov\(\)](#), [mxvm::Program::exec_realloc\(\)](#), [mxvm::Program::exec_store\(\)](#), [mxvm_std_release\(\)](#), [operator=\(\)](#), and [mxvm::releaseOwnedPointer\(\)](#).

11.112.4.8 ptr_size

`uint64_t mxvm::Variable_Value::ptr_size = 0`

size of each element in allocated memory

Definition at line 218 of file [instruct.hpp](#).

Referenced by [Variable_Value\(\)](#), [Variable_Value\(\)](#), [mxvm::Program::exec_alloc\(\)](#), [mxvm::Program::exec_free\(\)](#), [mxvm::Program::exec_lea\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_mov\(\)](#), [mxvm::Program::exec_realloc\(\)](#), [mxvm::Program::exec_store\(\)](#), [mxvm_std_release\(\)](#), [operator=\(\)](#), and [mxvm::releaseOwnedPointer\(\)](#).

11.112.4.9 ptr_value

`void* mxvm::Variable_Value::ptr_value`

Definition at line 216 of file [instruct.hpp](#).

Referenced by [Variable_Value\(\)](#), [mxvm::Program::addVariables\(\)](#), [asReadPtr\(\)](#), [asWritePtr\(\)](#), [mxvm::Program::exec_alloc\(\)](#), [mxvm::Program::exec_cmp\(\)](#), [mxvm::Program::exec_free\(\)](#), [mxvm::Program::exec_invoke\(\)](#), [mxvm::Program::exec_lea\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_mov\(\)](#), [mxvm::Program::exec_pop\(\)](#), [mxvm::Program::exec_push\(\)](#), [mxvm::Program::exec_realloc\(\)](#), [mxvm::Program::exec_stack_load\(\)](#), [mxvm::Program::exec_stack_store\(\)](#), [mxvm::Program::exec_store\(\)](#), [mxvm::Program::exec_string_print\(\)](#), [mxvm::Program::exec_to_float\(\)](#), [getStringFromVar\(\)](#), [mxvm_io_fclose\(\)](#), [mxvm_io_feof\(\)](#), [mxvm_io_fprintf\(\)](#), [mxvm_io_fputs\(\)](#), [mxvm_io_fread\(\)](#), [mxvm_io_fseek\(\)](#), [mxvm_io_fsize\(\)](#), [mxvm_io_fwrite\(\)](#), [mxvm_io_mxvm_fgets\(\)](#), [mxvm_sdl_create_window\(\)](#), [mxvm_sdl_draw_text\(\)](#), [mxvm_sdl_load_font\(\)](#), [mxvm_sdl_load_texture\(\)](#), [mxvm_sdl_load_texture_color_key\(\)](#), [mxvm_sdl_load_texture_color_key_rgb\(\)](#), [mxvm_sdl_load_wav\(\)](#), [mxvm_sdl_lock_texture\(\)](#), [mxvm_sdl_queue_audio\(\)](#), [mxvm_sdl_set_clipboard_text\(\)](#), [mxvm_sdl_set_window_icon\(\)](#), [mxvm_sdl_set_window_title\(\)](#), [mxvm_sdl_update_texture\(\)](#), [mxvm_std_atof\(\)](#),

[mxvm_std_atoi\(\)](#), [mxvm_std_memcmp\(\)](#), [mxvm_std_memcpy\(\)](#), [mxvm_std_memmove\(\)](#), [mxvm_std_memset\(\)](#), [mxvm_std_release\(\)](#), [mxvm_std_set_program_args\(\)](#), [mxvm_std_system\(\)](#), [mxvm_string_snprintf\(\)](#), [mxvm_string_strcmp\(\)](#), [mxvm_string_strlen\(\)](#), [mxvm_string_strncat\(\)](#), [mxvm_string_substr\(\)](#), [operator=\(\)](#), [mxvm::Program::printFormatted\(\)](#), [mxvm::releaseOwnedPointer\(\)](#), [mxvm::Program::setVariableFromConstant\(\)](#), and [mxvm::Program::subVariables\(\)](#).

11.112.4.10 released

`bool mxvm::Variable_Value::released = false`

true if the owned memory has been freed

Definition at line 223 of file [instruct.hpp](#).

Referenced by [Variable_Value\(\)](#), [Variable_Value\(\)](#), and [operator=\(\)](#).

11.112.4.11 str_value

`std::string mxvm::Variable_Value::str_value`

Definition at line 211 of file [instruct.hpp](#).

Referenced by [Variable_Value\(\)](#), [asReadPtr\(\)](#), [mxvm::Program::createTempVariable\(\)](#), [mxvm::Program::exec_getline\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_pop\(\)](#), [mxvm::Program::exec_print\(\)](#), [mxvm::Program::exec_push\(\)](#), [mxvm::Program::exec_stack_load\(\)](#), [mxvm::Program::exec_stack_store\(\)](#), [mxvm::Program::exec_store\(\)](#), [mxvm::Program::exec_string_print\(\)](#), [mxvm::Program::exec_to_float\(\)](#), [mxvm::Program::exec_to_int\(\)](#), [getStringFromVar\(\)](#), [mxvm_io_fopen\(\)](#), [mxvm_io_fprintf\(\)](#), [mxvm_io_fputs\(\)](#), [mxvm_sdl_create_window\(\)](#), [mxvm_sdl_draw_text\(\)](#), [mxvm_sdl_load_font\(\)](#), [mxvm_sdl_load_texture\(\)](#), [mxvm_sdl_load_texture_color_key\(\)](#), [mxvm_sdl_load_texture_color_key_rgb\(\)](#), [mxvm_sdl_load_wav\(\)](#), [mxvm_sdl_set_clipboard_text\(\)](#), [mxvm_sdl_set_window_icon\(\)](#), [mxvm_sdl_set_window_title\(\)](#), [mxvm_std_atof\(\)](#), [mxvm_std_atoi\(\)](#), [mxvm_std_system\(\)](#), [mxvm_string_snprintf\(\)](#), [mxvm_string_strat\(\)](#), [mxvm_string_strcmp\(\)](#), [mxvm_string_strlen\(\)](#), [mxvm_string_strncat\(\)](#), [mxvm_string_strncpy\(\)](#), [mxvm_string_substr\(\)](#), [operator=\(\)](#), [mxvm::Program::printFormatted\(\)](#), [mxvm::Program::setVariableFromConstant\(\)](#), and [mxvm::Program::setVariableFromString\(\)](#).

11.112.4.12 type

`VarType mxvm::Variable_Value::type`

Definition at line 220 of file [instruct.hpp](#).

Referenced by [Variable_Value\(\)](#), [Variable_Value\(\)](#), [mxvm::Program::addVariables\(\)](#), [mxvm::Program::createTempVariable\(\)](#), [mxvm::Program::divVariables\(\)](#), [mxvm::Program::exec_alloc\(\)](#), [mxvm::Program::exec_and\(\)](#), [mxvm::Program::exec_getline\(\)](#), [mxvm::Program::exec_lea\(\)](#), [mxvm::Program::exec_load\(\)](#), [mxvm::Program::exec_mod\(\)](#), [mxvm::Program::exec_mov\(\)](#), [mxvm::Program::exec_not\(\)](#), [mxvm::Program::exec_or\(\)](#), [mxvm::Program::exec_pop\(\)](#), [mxvm::Program::exec_push\(\)](#), [mxvm::Program::exec_realloc\(\)](#), [mxvm::Program::exec_stack_load\(\)](#), [mxvm::Program::exec_string_print\(\)](#), [mxvm::Program::exec_to_float\(\)](#), [mxvm::Program::exec_xor\(\)](#), [mxvm::Program::mulVariables\(\)](#), [mxvm_sdl_get_keyboard_state\(\)](#), [mxvm_sdl_get_mouse_state\(\)](#), [mxvm_sdl_get_relative_mouse_state\(\)](#), [mxvm_sdl_get_renderer_output_size\(\)](#), [mxvm_sdl_get_window_size\(\)](#), [mxvm_sdl_load_wav\(\)](#), [mxvm_sdl_lock_texture\(\)](#), [operator=\(\)](#), [mxvm::Program::printFormatted\(\)](#), [mxvm::Program::setVariableFromConstant\(\)](#), [mxvm::Program::setVariableFromString\(\)](#), and [mxvm::Program::subVariables\(\)](#).

The documentation for this struct was generated from the following file:

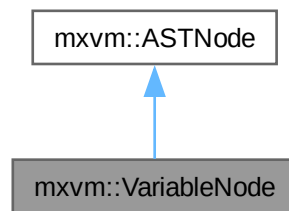
- [include/mxvm/instruct.hpp](#)

11.113 mxvm::VariableNode Class Reference

AST node for a variable declaration in a .data section.

```
#include <ast.hpp>
```

Inheritance diagram for `mxvm::VariableNode`:



Public Member Functions

- `VariableNode` (`VarType` varType, const std::string &varName)
- `VariableNode` (`VarType` varType, const std::string &varName, size_t buf_size)
- `VariableNode` (`VarType` varType, const std::string &varName, const std::string &value)
- void `accept` (`ASTVisitor` &visitor) override
- std::string `toString` () const override

Public Member Functions inherited from `mxvm::ASTNode`

- virtual `~ASTNode` ()=default

Public Attributes

- std::string `name`
- `VarType` `type`
- std::string `initialValue`
- bool `hasInitializer`
- size_t `buffer_size` = 0
buffer allocation size (for pointer/array types)
- bool `is_global` = false
- std::string `object`
owning object name, if any

11.113.1 Detailed Description

AST node for a variable declaration in a .data section.

Definition at line 71 of file [ast.hpp](#).

11.113.2 Constructor & Destructor Documentation

11.113.2.1 `VariableNode()` [1/3]

```

mxvm::VariableNode::VariableNode (
    VarType varType,
    const std::string & varName) [inline]

```

Definition at line 81 of file [ast.hpp](#).

References [buffer_size](#), [hasInitializer](#), [name](#), and [type](#).

11.113.2.2 VariableNode() [2/3]

```
mxvm::VariableNode::VariableNode (
    VarType varType,
    const std::string & varName,
    size_t buf_size) [inline]
```

Definition at line 84 of file [ast.hpp](#).

References [buffer_size](#), [hasInitializer](#), [name](#), and [type](#).

11.113.2.3 VariableNode() [3/3]

```
mxvm::VariableNode::VariableNode (
    VarType varType,
    const std::string & varName,
    const std::string & value) [inline]
```

Definition at line 87 of file [ast.hpp](#).

References [buffer_size](#), [hasInitializer](#), [initialValue](#), [name](#), and [type](#).

11.113.3 Member Function Documentation**11.113.3.1 accept()**

```
void mxvm::VariableNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 54 of file [ast.cpp](#).

References [mxvm::ASTVisitor::visit\(\)](#).

11.113.3.2 toString()

```
std::string mxvm::VariableNode::toString () const [override], [virtual]
```

Implements [mxvm::ASTNode](#).

Definition at line 58 of file [ast.cpp](#).

References [hasInitializer](#), [initialValue](#), [name](#), and [type](#).

11.113.4 Member Data Documentation**11.113.4.1 buffer_size**

```
size_t mxvm::VariableNode::buffer_size = 0
```

buffer allocation size (for pointer/array types)

Definition at line 77 of file [ast.hpp](#).

Referenced by [VariableNode\(\)](#), [VariableNode\(\)](#), and [VariableNode\(\)](#).

11.113.4.2 hasInitializer

```
bool mxvm::VariableNode::hasInitializer
```

Definition at line 76 of file [ast.hpp](#).

Referenced by [VariableNode\(\)](#), [VariableNode\(\)](#), [VariableNode\(\)](#), and [toString\(\)](#).

11.113.4.3 initialValue

```
std::string mxvm::VariableNode::initialValue
```

Definition at line 75 of file [ast.hpp](#).

Referenced by [VariableNode\(\)](#), and [toString\(\)](#).

11.113.4.4 is_global

```
bool mxvm::VariableNode::is_global = false
```

Definition at line 78 of file [ast.hpp](#).

11.113.4.5 name

`std::string mxvm::VariableNode::name`

Definition at line 73 of file [ast.hpp](#).

Referenced by [VariableNode\(\)](#), [VariableNode\(\)](#), [VariableNode\(\)](#), and [toString\(\)](#).

11.113.4.6 object

`std::string mxvm::VariableNode::object`

owning object name, if any

Definition at line 79 of file [ast.hpp](#).

11.113.4.7 type

[VarType](#) `mxvm::VariableNode::type`

Definition at line 74 of file [ast.hpp](#).

Referenced by [VariableNode\(\)](#), [VariableNode\(\)](#), [VariableNode\(\)](#), and [toString\(\)](#).

The documentation for this class was generated from the following files:

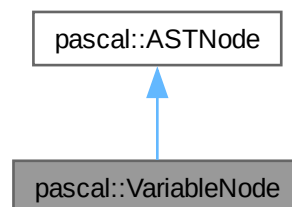
- [include/mxvm/ast.hpp](#)
- [src/ast.cpp](#)

11.114 pascal::VariableNode Class Reference

AST node for a variable reference.

```
#include <ast.hpp>
```

Inheritance diagram for `pascal::VariableNode`:



Public Member Functions

- [VariableNode](#) (const std::string &varName)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)

Set the originating source line number.

- `int getLineNumber () const`

Get the originating source line number.

Public Attributes

- `std::string name`
variable name

11.114.1 Detailed Description

AST node for a variable reference.

Definition at line 513 of file [ast.hpp](#).

11.114.2 Constructor & Destructor Documentation

11.114.2.1 VariableNode()

```
pascal::VariableNode::VariableNode (
    const std::string & varName) [inline]
```

Definition at line 517 of file [ast.hpp](#).

References [name](#).

11.114.3 Member Function Documentation

11.114.3.1 accept()

```
void pascal::VariableNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 239 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.114.3.2 toString()

```
std::string pascal::VariableNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 243 of file [ast.cpp](#).

References [name](#).

11.114.4 Member Data Documentation

11.114.4.1 name

```
std::string pascal::VariableNode::name
variable name
```

Definition at line 515 of file [ast.hpp](#).

Referenced by [VariableNode\(\)](#), [toString\(\)](#), [pascal::CodeGenVisitor::visit\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

- `src/frontend/include/ast.hpp`
- `src/frontend/ast.cpp`

11.115 pascal::VariantArm Struct Reference

A single variant arm inside a variant record.

```
#include <ast.hpp>
```

Public Attributes

- `std::vector< std::unique_ptr< ASTNode > > caseLabels`
case constant(s) for this arm
- `std::vector< std::unique_ptr< ASTNode > > fields`
field declarations in this arm

11.115.1 Detailed Description

A single variant arm inside a variant record.
 Definition at line [640](#) of file [ast.hpp](#).

11.115.2 Member Data Documentation

11.115.2.1 [caseLabels](#)

`std::vector<std::unique_ptr<ASTNode> > pascal::VariantArm::caseLabels`
 case constant(s) for this arm
 Definition at line [641](#) of file [ast.hpp](#).
 Referenced by [pascal::PascalParser::parseRecordType\(\)](#).

11.115.2.2 [fields](#)

`std::vector<std::unique_ptr<ASTNode> > pascal::VariantArm::fields`
 field declarations in this arm
 Definition at line [642](#) of file [ast.hpp](#).
 Referenced by [pascal::PascalParser::parseRecordType\(\)](#).
 The documentation for this struct was generated from the following file:

- `src/frontend/include/ast.hpp`

11.116 [mxvm::Validator::VarNamePair](#) Struct Reference

A variable name paired with its declaration token.
`#include <valid.hpp>`

Public Attributes

- `std::string first`
variable name
- `const scan::TToken * second`
token at the declaration

11.116.1 Detailed Description

A variable name paired with its declaration token.
 Definition at line [86](#) of file [valid.hpp](#).

11.116.2 Member Data Documentation

11.116.2.1 [first](#)

`std::string mxvm::Validator::VarNamePair::first`
 variable name
 Definition at line [87](#) of file [valid.hpp](#).

11.116.2.2 second

const [scan::TToken*](#) mxvm::Validator::VarNamePair::second

token at the declaration

Definition at line 88 of file [valid.hpp](#).

The documentation for this struct was generated from the following file:

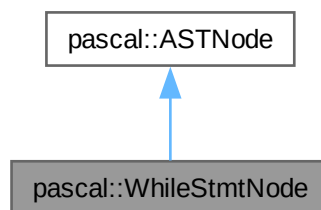
- [include/mxvm/valid.hpp](#)

11.117 pascal::WhileStmtNode Class Reference

AST node for a while loop (while cond do stmt).

#include <ast.hpp>

Inheritance diagram for pascal::WhileStmtNode:



Public Member Functions

- [WhileStmtNode](#) (std::unique_ptr< [ASTNode](#) > cond, std::unique_ptr< [ASTNode](#) > stmt)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::unique_ptr< [ASTNode](#) > [condition](#)
loop condition
- std::unique_ptr< [ASTNode](#) > [statement](#)
loop body

11.117.1 Detailed Description

AST node for a while loop (while cond do stmt).
Definition at line 366 of file [ast.hpp](#).

11.117.2 Constructor & Destructor Documentation

11.117.2.1 WhileStmtNode()

```
pascal::WhileStmtNode::WhileStmtNode (
    std::unique_ptr< ASTNode > cond,
    std::unique_ptr< ASTNode > stmt) [inline]
```

Definition at line 371 of file [ast.hpp](#).
References [condition](#), and [statement](#).

11.117.3 Member Function Documentation

11.117.3.1 accept()

```
void pascal::WhileStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).
Implements [pascal::ASTNode](#).
Definition at line 137 of file [ast.cpp](#).
References [pascal::ASTVisitor::visit\(\)](#).

11.117.3.2 toString()

```
std::string pascal::WhileStmtNode::toString () const [override], [virtual]
```

Return a human-readable description of this node.
Implements [pascal::ASTNode](#).
Definition at line 141 of file [ast.cpp](#).

11.117.4 Member Data Documentation

11.117.4.1 condition

```
std::unique_ptr<ASTNode> pascal::WhileStmtNode::condition
```

loop condition
Definition at line 368 of file [ast.hpp](#).
Referenced by [WhileStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.117.4.2 statement

```
std::unique_ptr<ASTNode> pascal::WhileStmtNode::statement
```

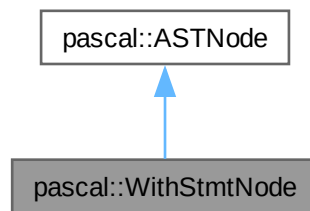
loop body
Definition at line 369 of file [ast.hpp](#).
Referenced by [WhileStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).
The documentation for this class was generated from the following files:

- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.118 pascal::WithStmtNode Class Reference

AST node for a with statement (with record do statement).
`#include <ast.hpp>`

Inheritance diagram for pascal::WithStmtNode:



Public Member Functions

- [WithStmtNode](#) (const std::string &[recordVar](#), std::unique_ptr< [ASTNode](#) > stmt)
- void [accept](#) ([ASTVisitor](#) &visitor) override
Accept an [ASTVisitor](#) (double-dispatch).
- std::string [toString](#) () const override
Return a human-readable description of this node.

Public Member Functions inherited from [pascal::ASTNode](#)

- virtual [~ASTNode](#) ()=default
- virtual void [print](#) (std::ostream &out, int indent=0) const
Pretty-print this node to a stream with indentation.
- void [setLineNumber](#) (int line)
Set the originating source line number.
- int [getLineNumber](#) () const
Get the originating source line number.

Public Attributes

- std::string [recordVar](#)
record variable name
- std::unique_ptr< [ASTNode](#) > [statement](#)
body statement

11.118.1 Detailed Description

AST node for a with statement (with record do statement).
Definition at line 668 of file [ast.hpp](#).

11.118.2 Constructor & Destructor Documentation

11.118.2.1 WithStmtNode()

```
pascal::WithStmtNode::WithStmtNode (
    const std::string & recordVar,
    std::unique_ptr< ASTNode > stmt) [inline]
```

Definition at line 673 of file [ast.hpp](#).

References [recordVar](#), and [statement](#).

11.118.3 Member Function Documentation

11.118.3.1 accept()

```
void pascal::WithStmtNode::accept (
    ASTVisitor & visitor) [override], [virtual]
```

Accept an [ASTVisitor](#) (double-dispatch).

Implements [pascal::ASTNode](#).

Definition at line 442 of file [ast.cpp](#).

References [pascal::ASTVisitor::visit\(\)](#).

11.118.3.2 toString()

```
std::string pascal::WithStmtNode::toString () const [inline], [override], [virtual]
```

Return a human-readable description of this node.

Implements [pascal::ASTNode](#).

Definition at line 677 of file [ast.hpp](#).

References [recordVar](#).

11.118.4 Member Data Documentation

11.118.4.1 recordVar

```
std::string pascal::WithStmtNode::recordVar
```

record variable name

Definition at line 670 of file [ast.hpp](#).

Referenced by [WithStmtNode\(\)](#), [toString\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

11.118.4.2 statement

```
std::unique_ptr<ASTNode> pascal::WithStmtNode::statement
```

body statement

Definition at line 671 of file [ast.hpp](#).

Referenced by [WithStmtNode\(\)](#), and [pascal::CodeGenVisitor::visit\(\)](#).

The documentation for this class was generated from the following files:

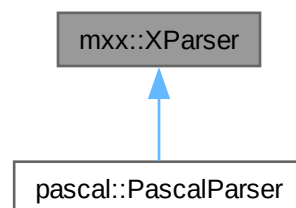
- [src/frontend/include/ast.hpp](#)
- [src/frontend/ast.cpp](#)

11.119 mxx::XParser Class Reference

Base parser providing token-stream navigation.

```
#include <parser.hpp>
```

Inheritance diagram for `mxx::XParser`:



Public Member Functions

- [XParser](#) ()=delete
- [XParser](#) (const std::string &source)
Construct and scan the source text.
- bool [next](#) ()
Advance to the next token; returns false at end of stream.
- bool [peekls](#) (const std::string &s)
Check if the current token's value matches a string (case-insensitive).
- bool [peekls](#) (const [types::TokenType](#) &t)
Check if the current token's type matches.

Static Public Member Functions

- static std::string [tokenTypeToString](#) ([types::TokenType](#) t)
Convert a TokenType to a human-readable string.

Public Attributes

- std::string [filename](#)
source filename for error messages

Protected Attributes

- [scan::Scanner](#) [scanner](#)
underlying scanner
- [scan::Token](#) * [token](#) = nullptr
current token pointer
- size_t [index](#) = 0
current token index

11.119.1 Detailed Description

Base parser providing token-stream navigation.

Wraps a Scanner to provide [next\(\)](#), [peekls\(\)](#), and token-type-to-string conversion for derived parsers.

Definition at line 32 of file [parser.hpp](#).

11.119.2 Constructor & Destructor Documentation

11.119.2.1 XParser() [1/2]

```
mx::XParser::XParser () [delete]
```

Referenced by [pascal::PascalParser::PascalParser\(\)](#).

11.119.2.2 XParser() [2/2]

```
mx::XParser::XParser (
    const std::string & source) [inline], [explicit]
```

Construct and scan the source text.

Parameters

<i>source</i>	Source code string
---------------	--------------------

Definition at line 39 of file [parser.hpp](#).

References [index](#), [scanner](#), and [token](#).

11.119.3 Member Function Documentation

11.119.3.1 next()

```
bool mx::XParser::next () [inline]
```

Advance to the next token; returns false at end of stream.

Definition at line 50 of file [parser.hpp](#).

References [index](#), [scanner](#), and [token](#).

Referenced by [pascal::PascalParser::match\(\)](#), [pascal::PascalParser::match\(\)](#), [pascal::PascalParser::parseArgumentList\(\)](#), [pascal::PascalParser::parseArrayType\(\)](#), [pascal::PascalParser::parseAssignmentOrProcCall\(\)](#), [pascal::PascalParser::parseCaseStatement\(\)](#), [pascal::PascalParser::parseCompoundStatement\(\)](#), [pascal::PascalParser::parseConstDeclaration\(\)](#), [pascal::PascalParser::parseExpression\(\)](#), [pascal::PascalParser::parseFactor\(\)](#), [pascal::PascalParser::parseForStatement\(\)](#), [pascal::PascalParser::parseFunctionCall\(\)](#), [pascal::PascalParser::parseFunctionDeclaration\(\)](#), [pascal::PascalParser::parseFunctionForwardDecl\(\)](#), [pascal::PascalParser::parseGlobalStatement\(\)](#), [pascal::PascalParser::parseLabelDeclaration\(\)](#), [pascal::PascalParser::parseLValue\(\)](#), [pascal::PascalParser::parseParameter\(\)](#), [pascal::PascalParser::parseParameterList\(\)](#), [pascal::PascalParser::parseProcedureCall\(\)](#), [pascal::PascalParser::parseProcedureDeclaration\(\)](#), [pascal::PascalParser::parseProcedureForwardDecl\(\)](#), [pascal::PascalParser::parseProgram\(\)](#), [pascal::PascalParser::parseRecordType\(\)](#), [pascal::PascalParser::parseRepeatStatement\(\)](#), [pascal::PascalParser::parseSimpleExpression\(\)](#), [pascal::PascalParser::parseStatement\(\)](#), [pascal::PascalParser::parseStatementList\(\)](#), [pascal::PascalParser::parseTerm\(\)](#), [pascal::PascalParser::parseTypeDeclaration\(\)](#), [pascal::PascalParser::parseTypeSpec\(\)](#), [pascal::PascalParser::parseUnit\(\)](#), [pascal::PascalParser::parseVarDeclaration\(\)](#), [pascal::PascalParser::parseWhileStatement\(\)](#), and [pascal::PascalParser::parseWithStatement\(\)](#).

11.119.3.2 peekIs() [1/2]

```
bool mx::XParser::peekIs (
    const std::string & s) [inline]
```

Check if the current token's value matches a string (case-insensitive).

Definition at line 61 of file [parser.hpp](#).

References [token](#).

Referenced by [pascal::PascalParser::expectToken\(\)](#), [pascal::PascalParser::expectToken\(\)](#), [pascal::PascalParser::getRelationalOp\(\)](#), [pascal::PascalParser::isMulOperator\(\)](#), [pascal::PascalParser::isRelationalOperator\(\)](#), [pascal::PascalParser::match\(\)](#), [pascal::PascalParser::match\(\)](#), [pascal::PascalParser::parseArgumentList\(\)](#), [pascal::PascalParser::parseArrayType\(\)](#), [pascal::PascalParser::parseAssignmentOrProcCall\(\)](#), [pascal::PascalParser::parseBlock\(\)](#), [pascal::PascalParser::parseCaseStatement\(\)](#), [pascal::PascalParser::parseConstDeclaration\(\)](#), [pascal::PascalParser::parseDeclarations\(\)](#), [pascal::PascalParser::parseExpression\(\)](#), [pascal::PascalParser::parseFactor\(\)](#), [pascal::PascalParser::parseForStatement\(\)](#), [pascal::PascalParser::parseFunctionDeclaration\(\)](#), [pascal::PascalParser::parseFunctionForwardDecl\(\)](#), [pascal::PascalParser::parseGlobalStatement\(\)](#), [pascal::PascalParser::parseLabelDeclaration\(\)](#), [pascal::PascalParser::parseLValue\(\)](#), [pascal::PascalParser::parseParameter\(\)](#), [pascal::PascalParser::parseParameterList\(\)](#), [pascal::PascalParser::parseProcedureCall\(\)](#), [pascal::PascalParser::parseProcedureDeclaration\(\)](#), [pascal::PascalParser::parseProcedureForwardDecl\(\)](#), [pascal::PascalParser::parseProgram\(\)](#), [pascal::PascalParser::parseRecordType\(\)](#), [pascal::PascalParser::parseRepeatStatement\(\)](#), [pascal::PascalParser::parseSimpleExpression\(\)](#), [pascal::PascalParser::parseStatement\(\)](#), [pascal::PascalParser::parseStatementList\(\)](#), [pascal::PascalParser::parseTerm\(\)](#), [pascal::PascalParser::parseTypeDeclaration\(\)](#), [pascal::PascalParser::parseTypeSpec\(\)](#), [pascal::PascalParser::parseUnit\(\)](#), and [pascal::PascalParser::parseVarDeclaration\(\)](#).

11.119.3.3 peekIs() [2/2]

```
bool mx::XParser::peekIs (
    const types::TokenType & t) [inline]
```

Check if the current token's type matches.

Definition at line 72 of file [parser.hpp](#).

References [token](#).

11.119.3.4 tokenTypeToString()

```
std::string mx::XParser::tokenTypeToString (
    types::TokenType t) [inline], [static]
```

Convert a TokenType to a human-readable string.

Parameters

<i>t</i>	Token type
----------	------------

Returns

Display name

Definition at line 81 of file [parser.hpp](#).

References [types::TT_ID](#), [types::TT_NUM](#), [types::TT_STR](#), and [types::TT_SYM](#).

11.119.4 Member Data Documentation**11.119.4.1 filename**

```
std::string mxx::XParser::filename
```

source filename for error messages

Definition at line 96 of file [parser.hpp](#).

11.119.4.2 index

```
size_t mxx::XParser::index = 0 [protected]
```

current token index

Definition at line 101 of file [parser.hpp](#).

Referenced by [pascal::PascalParser::PascalParser\(\)](#), [XParser\(\)](#), [next\(\)](#), [pascal::PascalParser::parseFactor\(\)](#), and [pascal::PascalParser::parseLValue\(\)](#).

11.119.4.3 scanner

```
scan::Scanner mxx::XParser::scanner [protected]
```

underlying scanner

Definition at line 99 of file [parser.hpp](#).

Referenced by [pascal::PascalParser::PascalParser\(\)](#), [XParser\(\)](#), [next\(\)](#), and [pascal::PascalParser::removeBraceComments\(\)](#).

11.119.4.4 token

```
scan::TToken* mxx::XParser::token = nullptr [protected]
```

current token pointer

Definition at line 100 of file [parser.hpp](#).

Referenced by [pascal::PascalParser::PascalParser\(\)](#), [XParser\(\)](#), [pascal::PascalParser::error\(\)](#), [pascal::PascalParser::expectToken\(\)](#), [pascal::PascalParser::isType\(\)](#), [pascal::PascalParser::isUnitSource\(\)](#), [next\(\)](#), [pascal::PascalParser::parseAssignmentOrProcCall\(\)](#), [pascal::PascalParser::parseBlock\(\)](#), [pascal::PascalParser::parseCaseStatement\(\)](#), [pascal::PascalParser::parseCompoundStatement\(\)](#), [pascal::PascalParser::parseConstDeclaration\(\)](#), [pascal::PascalParser::parseDeclarations\(\)](#), [pascal::PascalParser::parseExpression\(\)](#), [pascal::PascalParser::parseFactor\(\)](#), [pascal::PascalParser::parseForStatement\(\)](#), [pascal::PascalParser::parseFunctionCall\(\)](#), [pascal::PascalParser::parseFunctionDeclaration\(\)](#), [pascal::PascalParser::parseFunctionForwardDecl\(\)](#), [pascal::PascalParser::parseGlobalDeclaration\(\)](#), [pascal::PascalParser::parseInterfaceDeclarations\(\)](#), [pascal::PascalParser::parseLabelDeclaration\(\)](#), [pascal::PascalParser::parseLValue\(\)](#), [pascal::PascalParser::parseParameter\(\)](#), [pascal::PascalParser::parseProcedureCall\(\)](#), [pascal::PascalParser::parseProcedureDeclaration\(\)](#), [pascal::PascalParser::parseProcedureForwardDecl\(\)](#), [pascal::PascalParser::parseRecordType\(\)](#), [pascal::PascalParser::parseRepeatStatement\(\)](#), [pascal::PascalParser::parseSimpleExpression\(\)](#), [pascal::PascalParser::parseStatement\(\)](#), [pascal::PascalParser::parseTerm\(\)](#), [pascal::PascalParser::parseTypeDeclaration\(\)](#), [pascal::PascalParser::parseTypeSpec\(\)](#), [pascal::PascalParser::parseUnit\(\)](#), [pascal::PascalParser::parseVarDeclaration\(\)](#), [pascal::PascalParser::parseWhileStatement\(\)](#), [pascal::PascalParser::parseWithStatement\(\)](#), [peekIs\(\)](#), and [peekIs\(\)](#).

The documentation for this class was generated from the following file:

- [src/frontend/include/parser.hpp](#)

Chapter 12

File Documentation

12.1 include/mxvm/ast.hpp File Reference

AST node hierarchy for the MXVM parser using the Visitor pattern.

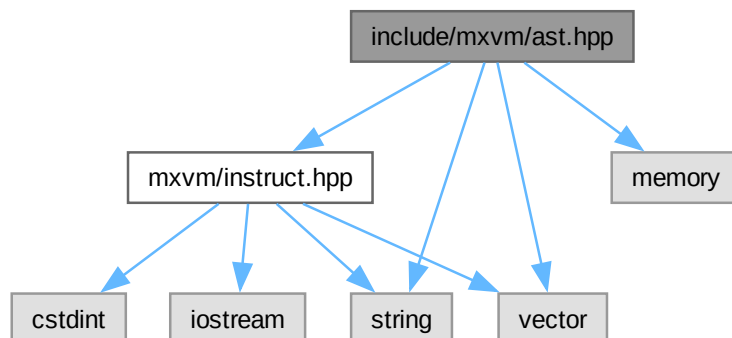
```
#include "mxvm/instruct.hpp"
```

```
#include <memory>
```

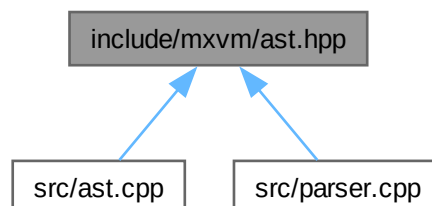
```
#include <string>
```

```
#include <vector>
```

Include dependency graph for ast.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [mxvm::ASTNode](#)
Abstract base class for all AST nodes.
- class [mxvm::SectionNode](#)
Represents a program section (.data, .code, .module, .object).
- class [mxvm::ProgramNode](#)
Root AST node representing an entire program or object.
- class [mxvm::VariableNode](#)
AST node for a variable declaration in a .data section.
- class [mxvm::ModuleNode](#)
AST node for a module import declaration.
- class [mxvm::ObjectNode](#)
AST node for an object import declaration.
- class [mxvm::InstructionNode](#)
AST node representing a single MXVM instruction with its operands.
- class [mxvm::LabelNode](#)
AST node for a label (or function entry point) in the code section.
- class [mxvm::ExpressionNode](#)
AST node for a literal expression value.
- class [mxvm::CommentNode](#)
AST node for a source-level comment.
- class [mxvm::ASTVisitor](#)
Visitor interface for traversing the AST node hierarchy.

Namespaces

- namespace [mxvm](#)

12.1.1 Detailed Description

AST node hierarchy for the MXVM parser using the Visitor pattern.

Author

Jared Bruni

Definition in file [ast.hpp](#).

12.2 ast.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef __AST_HPP_X_
00007 #define __AST_HPP_X_
00008
00009 #include "mxvm/instruct.hpp"
00010 #include <memory>
00011 #include <string>
00012 #include <vector>
00013
00014 namespace mxvm {
00015
00016     class ASTVisitor;
00017
00019     class ASTNode {
00020     public:
00021         virtual ~ASTNode() = default;
00022         virtual void accept(ASTVisitor &visitor) = 0;
00023         virtual std::string toString() const = 0;
00024     };
00025
00027     class SectionNode : public ASTNode {

```



```

00028     public:
00030         enum SectionType {
00031             DATA,
00032             CODE,
00033             MODULE,
00034             OBJECT
00035         };
00036         SectionType type;
00037         std::vector<std::unique_ptr<ASTNode> statements;
00038
00039         SectionNode(SectionType sectionType) : type(sectionType) {}
00040
00041         void addStatement(std::unique_ptr<ASTNode> stmt) {
00042             statements.push_back(std::move(stmt));
00043         }
00044
00045         void accept(ASTVisitor &visitor) override;
00046         std::string toString() const override;
00047     };
00048
00050     class ProgramNode : public ASTNode {
00051     public:
00052         std::string name;
00053         bool object = false;
00054         std::string root_name;
00055         std::vector<std::unique_ptr<SectionNode> sections;
00056         std::vector<std::unique_ptr<ProgramNode> inlineObjects;
00057
00058         void addSection(std::unique_ptr<SectionNode> section) {
00059             sections.push_back(std::move(section));
00060         }
00061
00062         void addInlineObject(std::unique_ptr<ProgramNode> obj) {
00063             inlineObjects.push_back(std::move(obj));
00064         }
00065
00066         virtual void accept(ASTVisitor &visitor) override;
00067         virtual std::string toString() const override;
00068     };
00069
00071     class VariableNode : public ASTNode {
00072     public:
00073         std::string name;
00074         VarType type;
00075         std::string initialValue;
00076         bool hasInitializer;
00077         size_t buffer_size = 0;
00078         bool is_global = false;
00079         std::string object;
00080
00081         VariableNode(VarType varType, const std::string &varName)
00082             : name(varName), type(varType), hasInitializer(false), buffer_size(0) {}
00083
00084         VariableNode(VarType varType, const std::string &varName, size_t buf_size)
00085             : name(varName), type(varType), hasInitializer(false), buffer_size(buf_size) {}
00086
00087         VariableNode(VarType varType, const std::string &varName, const std::string &value)
00088             : name(varName), type(varType), initialValue(value), hasInitializer(true), buffer_size(0)
00089         {}
00090
00091         void accept(ASTVisitor &visitor) override;
00092         std::string toString() const override;
00093     };
00095     class ModuleNode : public ASTNode {
00096     public:
00097         std::string name;
00098         ModuleNode(const std::string &n) : name(n) {}
00099         void accept(ASTVisitor &visitor) override;
00100         std::string toString() const override;
00101     };
00102
00104     class ObjectNode : public ASTNode {
00105     public:
00106         std::string name;
00107         ObjectNode(const std::string &o) : name(o) {}
00108         void accept(ASTVisitor &visitor) override;
00109         std::string toString() const override;
00110     };
00111
00113     class InstructionNode : public ASTNode {
00114     public:
00115         Inc instruction;
00116         std::vector<Operand> operands;
00117
00118         InstructionNode(Inc inst) : instruction(inst) {}
00119

```

```

00120         InstructionNode(Inc inst, const std::vector<Operand> &ops)
00121             : instruction(inst), operands(ops) {}
00122
00123         void addOperand(const Operand &operand) {
00124             operands.push_back(operand);
00125         }
00126
00127         void accept(ASTVisitor &visitor) override;
00128         std::string toString() const override;
00129     };
00130
00132     class LabelNode : public ASTNode {
00133     public:
00134         std::string name;
00135         bool function = false;
00136         LabelNode(const std::string &labelName) : name(labelName) {}
00137
00138         void accept(ASTVisitor &visitor) override;
00139         std::string toString() const override;
00140     };
00141
00143     class ExpressionNode : public ASTNode {
00144     public:
00145         std::string value;
00146         bool isNumeric;
00147
00148         ExpressionNode(const std::string &val, bool numeric = false)
00149             : value(val), isNumeric(numeric) {}
00150
00151         void accept(ASTVisitor &visitor) override;
00152         std::string toString() const override;
00153     };
00154
00156     class CommentNode : public ASTNode {
00157     public:
00158         std::string text;
00159         CommentNode(const std::string &commentText) : text(commentText) {}
00160
00161         void accept(ASTVisitor &visitor) override;
00162         std::string toString() const override;
00163     };
00164
00166     class ASTVisitor {
00167     public:
00168         virtual ~ASTVisitor() = default;
00169         virtual void visit(ProgramNode &node) = 0;
00170         virtual void visit(SectionNode &node) = 0;
00171         virtual void visit(InstructionNode &node) = 0;
00172         virtual void visit(LabelNode &node) = 0;
00173         virtual void visit(VariableNode &node) = 0;
00174         virtual void visit(ExpressionNode &node) = 0;
00175         virtual void visit(CommentNode &node) = 0;
00176         virtual void visit(ModuleNode &node) = 0;
00177         virtual void visit(ObjectNode &node) = 0;
00178     };
00179 } // namespace mxvm
00180
00181 #endif

```

12.3 src/frontend/include/ast.hpp File Reference

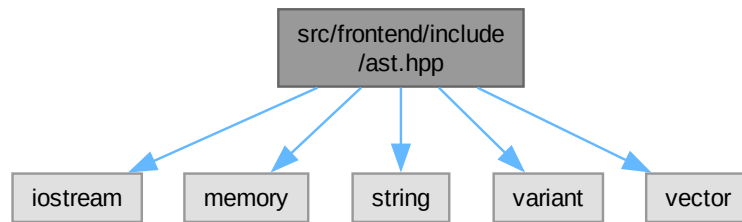
AST node hierarchy for the Pascal-to-MXVM frontend parser.

```

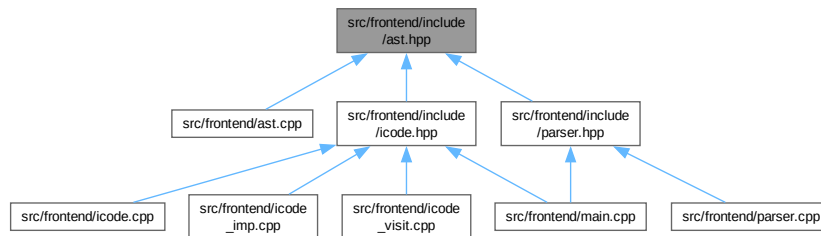
#include <iostream>
#include <memory>
#include <string>
#include <variant>
#include <vector>

```

Include dependency graph for ast.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [pascal::ASTNode](#)
Abstract base class for all AST nodes.
- class [pascal::ExitNode](#)
AST node for an exit / halt statement.
- class [pascal::ContinueNode](#)
AST node for a continue statement.
- class [pascal::BreakNode](#)
AST node for a break statement.
- class [pascal::BlockNode](#)
AST node representing a block (declarations + compound statement).
- class [pascal::ProgramNode](#)
AST node for a complete program (program name + block).
- class [pascal::UnitNode](#)
AST node for a Pascal unit (separately compiled module).
- class [pascal::VarDeclNode](#)
AST node for variable declarations (var id1, id2 : type).
- class [pascal::TypeDeclNode](#)
AST node for a type declaration section.
- class [pascal::TypeAliasNode](#)
AST node for a type alias (type NewName = ExistingType).
- class [pascal::ConstDeclNode](#)
AST node for constant declarations (const id = value).

- struct `pascal::ConstDeclNode::ConstAssignment`
A single constant name = value pair.
- class `pascal::FuncDeclNode`
AST node for a function declaration.
- class `pascal::ProcDeclNode`
AST node for a procedure declaration.
- class `pascal::ParameterNode`
AST node for a formal parameter list entry.
- class `pascal::CompoundStmtNode`
AST node for a compound statement (begin ... end).
- class `pascal::AssignmentNode`
AST node for an assignment statement (variable := expression).
- class `pascal::SimpleTypeNode`
AST node for a simple (named) type reference.
- class `pascal::ArrayTypeNode`
AST node for an array type (array[lower..upper] of elementType).
- class `pascal::ArrayDeclarationNode`
AST node for a named array variable declaration.
- class `pascal::ArrayTypeDeclarationNode`
AST node for a named array type declaration (type Name = array[...]).
- class `pascal::ArrayAccessNode`
AST node for an array element access (arr[index]).
- class `pascal::ArrayAssignmentNode`
AST node for assigning to an array element (arr[index] := value).
- class `pascal::IfStmtNode`
AST node for an if-then-else statement.
- class `pascal::WhileStmtNode`
AST node for a while loop (while cond do stmt).
- class `pascal::ForStmtNode`
AST node for a for loop (for var := start to/downto end do stmt).
- class `pascal::CaseStmtNode`
AST node for a case statement.
- struct `pascal::CaseStmtNode::CaseBranch`
A single case branch (one or more values mapping to a statement).
- class `pascal::ProcCallNode`
AST node for a procedure call statement.
- class `pascal::BinaryOpNode`
AST node for a binary operator expression.
- class `pascal::UnaryOpNode`
AST node for a unary operator expression.
- class `pascal::FuncCallNode`
AST node for a function call expression.
- class `pascal::VariableNode`
AST node for a variable reference.
- class `pascal::NumberNode`
AST node for an integer or real numeric literal.
- class `pascal::StringNode`
AST node for a string literal.
- class `pascal::BooleanNode`
AST node for a boolean literal (true / false).
- class `pascal::EmptyStmtNode`

- AST node for an empty statement (no-op).*

 - class [pascal::NilNode](#)

AST node for the nil literal.
 - class [pascal::PointerTypeNode](#)

AST node for a pointer type declaration ($\wedge$ Base Type).
 - class [pascal::PointerDerefNode](#)

AST node for a pointer dereference ($\text{ptr}^\wedge$).
 - class [pascal::AddressOfNode](#)

AST node for an address-of expression (@operand).
 - class [pascal::RepeatStmtNode](#)

AST node for a repeat..until loop.
 - class [pascal::EnumTypeDeclNode](#)

AST node for an enumerated type declaration (type Color = (Red, Green, Blue)).
 - struct [pascal::VariantArm](#)

A single variant arm inside a variant record.
 - class [pascal::RecordTypeNode](#)

AST node for a record type (record ... end).
 - class [pascal::RecordDeclarationNode](#)

AST node for a named record declaration.
 - class [pascal::WithStmtNode](#)

AST node for a with statement (with record do statement).
 - class [pascal::GotoStmtNode](#)

AST node for a goto statement.
 - class [pascal::LabelStmtNode](#)

AST node for a label definition (label: statement).
 - class [pascal::SetLiteralNode](#)

AST node for a set literal [val1, val2, ...].
 - class [pascal::SetTypeNode](#)

AST node for a set type declaration (set of <base type>).
 - class [pascal::FieldAccessNode](#)

AST node for accessing a record field (record.field).
 - class [pascal::ASTVisitor](#)

Abstract visitor interface for the AST (double-dispatch).

Namespaces

- namespace [pascal](#)

12.3.1 Detailed Description

AST node hierarchy for the Pascal-to-MXVM frontend parser.

Author

Jared Bruni

Definition in file [ast.hpp](#).

12.4 ast.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef _PASCAL_AST_H_
00007 #define _PASCAL_AST_H_
00008
00009 #include <iostream>
00010 #include <memory>
00011 #include <string>
00012 #include <variant>
00013 #include <vector>
00014
00015 namespace pascal {
00016
00017     class ASTVisitor;
00018
00026     class ASTNode {
00027         int line_number = 0;
00028
00029     public:
00030         virtual ~ASTNode() = default;
00032         virtual void accept(ASTVisitor &visitor) = 0;
00034         virtual std::string toString() const = 0;
00036         virtual void print(std::ostream &out, int indent = 0) const;
00038         void setLineNumber(int line) { line_number = line; }
00040         int getLineNumber() const { return line_number; }
00041     };
00042
00044     class ExitNode : public ASTNode {
00045     public:
00046         void accept(ASTVisitor &visitor) override;
00047         std::string toString() const override;
00048         std::unique_ptr<ASTNode> expr;
00049         ExitNode() = default;
00051         explicit ExitNode(std::unique_ptr<ASTNode> e) : expr(std::move(e)) {}
00052     };
00053
00055     class ContinueNode : public ASTNode {
00056     public:
00057         void accept(ASTVisitor &visitor) override;
00058         std::string toString() const override;
00059         std::unique_ptr<ASTNode> expr;
00060         ContinueNode() = default;
00061         explicit ContinueNode(std::unique_ptr<ASTNode> e) : expr(std::move(e)) {}
00062     };
00063
00065     class BreakNode : public ASTNode {
00066     public:
00067         void accept(ASTVisitor &visitor) override;
00068         std::string toString() const override;
00069         std::unique_ptr<ASTNode> expr;
00070         BreakNode() = default;
00071         explicit BreakNode(std::unique_ptr<ASTNode> e) : expr(std::move(e)) {}
00072     };
00073
00074     class CompoundStmtNode;
00075
00079     class BlockNode : public ASTNode {
00080     public:
00081         std::vector<std::unique_ptr<ASTNode>> declarations;
00082         std::unique_ptr<CompoundStmtNode> compoundStatement;
00083
00084         BlockNode(std::vector<std::unique_ptr<ASTNode>> decls, std::unique_ptr<CompoundStmtNode> stmt);
00089
00091         void accept(ASTVisitor &visitor) override;
00092         std::string toString() const override;
00093     };
00094
00098     class ProgramNode : public ASTNode {
00099     public:
00100         std::string name;
00101         std::unique_ptr<BlockNode> block;
00102         std::vector<std::string> uses;
00103
00104         ProgramNode(const std::string &name, std::unique_ptr<BlockNode> blk);
00105
00106         void accept(ASTVisitor &visitor) override;
00107         std::string toString() const override;
00108     };
00109
00116     class UnitNode : public ASTNode {
00117     public:
00118         std::string name;
00119         std::vector<std::string> uses;
00120         std::vector<std::unique_ptr<ASTNode>> interfaceDecls;

```

```

00121         std::vector<std::unique_ptr<ASTNode>> implDecls;
00122
00123         UnitNode(const std::string &name) : name(name) {}
00124
00125         void accept(ASTVisitor &visitor) override;
00126         std::string toString() const override;
00127     };
00128
00132     class VarDeclNode : public ASTNode {
00133     public:
00134         std::vector<std::string> identifiers;
00135         std::variant<std::string, std::unique_ptr<ASTNode>> type;
00136         std::vector<std::unique_ptr<ASTNode>> initializers;
00137
00138         VarDeclNode(std::vector<std::string> identifiers,
00139                     std::variant<std::string, std::unique_ptr<ASTNode>> type,
00140                     std::vector<std::unique_ptr<ASTNode>> initializers)
00141             : identifiers(std::move(identifiers)), type(std::move(type)),
00142               initializers(std::move(initializers)) {}
00143
00144         void accept(ASTVisitor &visitor) override;
00145         std::string toString() const override;
00146     };
00147
00149     class TypeDeclNode : public ASTNode {
00150     public:
00151         std::vector<std::unique_ptr<ASTNode>> typeDeclarations;
00152
00153         TypeDeclNode(std::vector<std::unique_ptr<ASTNode>> declarations)
00154             : typeDeclarations(std::move(declarations)) {}
00155
00156         void accept(ASTVisitor &visitor) override;
00157         std::string toString() const override;
00158     };
00159
00161     class TypeAliasNode : public ASTNode {
00162     public:
00163         std::string typeName;
00164         std::string baseType;
00165
00166         TypeAliasNode(const std::string &name, const std::string &base)
00167             : typeName(name), baseType(base) {}
00168
00169         void accept(ASTVisitor &visitor) override;
00170         std::string toString() const override;
00171     };
00172
00174     class ConstDeclNode : public ASTNode {
00175     public:
00176         struct ConstAssignment {
00177             std::string identifier;
00178             std::unique_ptr<ASTNode> value;
00179
00180             ConstAssignment(const std::string &id, std::unique_ptr<ASTNode> val)
00181                 : identifier(id), value(std::move(val)) {}
00182         };
00183
00184         std::vector<std::unique_ptr<ConstAssignment>> assignments;
00185         ConstDeclNode(std::vector<std::unique_ptr<ConstAssignment>> assigns)
00186             : assignments(std::move(assigns)) {}
00187
00188         void accept(ASTVisitor &visitor) override;
00189         std::string toString() const override;
00190     };
00191
00192     class FuncDeclNode : public ASTNode {
00193     public:
00194         std::string name;
00195         std::vector<std::unique_ptr<ASTNode>> parameters;
00196         std::string returnType;
00197         std::shared_ptr<ASTNode> block;
00198
00199         FuncDeclNode(const std::string &n, std::vector<std::unique_ptr<ASTNode>> p, const std::string
00200 &rt, std::unique_ptr<ASTNode> b)
00201             : name(n), parameters(std::move(p)), returnType(rt), block(std::move(b)) {}
00202
00203         void accept(ASTVisitor &visitor) override;
00204         std::string toString() const override;
00205     };
00206
00207     class ProcDeclNode : public ASTNode {
00208     public:
00209         std::string name;
00210         std::vector<std::unique_ptr<ASTNode>> parameters;
00211         std::shared_ptr<ASTNode> block;
00212
00213         ProcDeclNode(const std::string &n, std::vector<std::unique_ptr<ASTNode>> p,

```

```

std::unique_ptr<ASTNode> b)
00216         : name(n), parameters(std::move(p)), block(std::move(b)) {}
00217
00218     void accept(ASTVisitor &visitor) override;
00219     std::string toString() const override;
00220 };
00221
00222 class ParameterNode : public ASTNode {
00223 public:
00224     std::vector<std::string> identifiers;
00225     std::string type;
00226     bool isVar;
00227
00228     ParameterNode(std::vector<std::string> ids, const std::string &paramType, bool var = false)
00229         : identifiers(std::move(ids)), type(paramType), isVar(var) {}
00230
00231     void accept(ASTVisitor &visitor) override;
00232     std::string toString() const override;
00233 };
00234
00235 class CompoundStmtNode : public ASTNode {
00236 public:
00237     std::vector<std::unique_ptr<ASTNode>> statements;
00238
00239     CompoundStmtNode(std::vector<std::unique_ptr<ASTNode>> stmts)
00240         : statements(std::move(stmts)) {}
00241
00242     void accept(ASTVisitor &visitor) override;
00243     std::string toString() const override;
00244 };
00245
00246 class AssignmentNode : public ASTNode {
00247 public:
00248     std::unique_ptr<ASTNode> variable;
00249     std::unique_ptr<ASTNode> expression;
00250
00251     AssignmentNode(std::unique_ptr<ASTNode> var, std::unique_ptr<ASTNode> expr)
00252         : variable(std::move(var)), expression(std::move(expr)) {}
00253
00254     void accept(ASTVisitor &visitor) override;
00255     std::string toString() const override;
00256 };
00257
00258 enum class VarType;
00259
00260 class SimpleTypeNode : public ASTNode {
00261 public:
00262     std::string typeName;
00263
00264     SimpleTypeNode(const std::string &typeName) : typeName(typeName) {}
00265
00266     void accept(ASTVisitor &visitor) override;
00267     std::string toString() const override { return "SimpleType: " + typeName; }
00268 };
00269
00270 class ArrayTypeNode : public ASTNode {
00271 public:
00272     std::unique_ptr<ASTNode> elementType;
00273     std::unique_ptr<ASTNode> lowerBound;
00274     std::unique_ptr<ASTNode> upperBound;
00275
00276     ArrayTypeNode(std::unique_ptr<ASTNode> elementType,
00277                 std::unique_ptr<ASTNode> lowerBound,
00278                 std::unique_ptr<ASTNode> upperBound)
00279         : elementType(std::move(elementType)),
00280           lowerBound(std::move(lowerBound)),
00281           upperBound(std::move(upperBound)) {}
00282
00283     std::string toString() const override;
00284     void accept(ASTVisitor &visitor) override;
00285 };
00286
00287 class ArrayDeclarationNode : public ASTNode {
00288 public:
00289     std::string name;
00290     std::unique_ptr<ArrayTypeNode> arrayType;
00291     std::vector<std::unique_ptr<ASTNode>> initializers;
00292
00293     ArrayDeclarationNode(std::string name, std::unique_ptr<ArrayTypeNode> arrayType,
00294                        std::vector<std::unique_ptr<ASTNode>> initializers = {})
00295         : name(std::move(name)), arrayType(std::move(arrayType)),
00296           initializers(std::move(initializers)) {}
00297
00298     void accept(ASTVisitor &visitor) override;
00299     std::string toString() const override;
00300 };
00301
00302 class ArrayTypeDeclarationNode : public ASTNode {

```



```

00309     public:
00310         std::string name;
00311         std::unique_ptr<ArrayTypeNode> arrayType;
00312
00313         ArrayTypeDeclarationNode(const std::string &name, std::unique_ptr<ArrayTypeNode> arrayType)
00314             : name(name), arrayType(std::move(arrayType)) {}
00315
00316         void accept(ASTVisitor &visitor) override;
00317         std::string toString() const override;
00318     };
00319
00321     class ArrayAccessNode : public ASTNode {
00322     public:
00323         std::unique_ptr<ASTNode> base;
00324         std::unique_ptr<ASTNode> index;
00325
00326         ArrayAccessNode(std::unique_ptr<ASTNode> base, std::unique_ptr<ASTNode> index)
00327             : base(std::move(base)), index(std::move(index)) {}
00328
00329         void accept(ASTVisitor &visitor) override;
00330         std::string toString() const override;
00331     };
00332
00334     class ArrayAssignmentNode : public ASTNode {
00335     public:
00336         std::string arrayName;
00337         std::unique_ptr<ASTNode> index;
00338         std::unique_ptr<ASTNode> value;
00339
00340         ArrayAssignmentNode(std::string arrayName, std::unique_ptr<ASTNode> index,
00341                             std::unique_ptr<ASTNode> value)
00342             : arrayName(std::move(arrayName)), index(std::move(index)),
00343               value(std::move(value)) {}
00344
00345         void accept(ASTVisitor &visitor) override;
00346         std::string toString() const override;
00347     };
00348
00350     class IfStmtNode : public ASTNode {
00351     public:
00352         std::unique_ptr<ASTNode> condition;
00353         std::unique_ptr<ASTNode> thenStatement;
00354         std::unique_ptr<ASTNode> elseStatement;
00355
00356         IfStmtNode(std::unique_ptr<ASTNode> cond,
00357                   std::unique_ptr<ASTNode> thenStmt,
00358                   std::unique_ptr<ASTNode> elseStmt = nullptr)
00359             : condition(std::move(cond)), thenStatement(std::move(thenStmt)),
00360               elseStatement(std::move(elseStmt)) {}
00361
00362         void accept(ASTVisitor &visitor) override;
00363         std::string toString() const override;
00364     };
00366     class WhileStmtNode : public ASTNode {
00367     public:
00368         std::unique_ptr<ASTNode> condition;
00369         std::unique_ptr<ASTNode> statement;
00370
00371         WhileStmtNode(std::unique_ptr<ASTNode> cond, std::unique_ptr<ASTNode> stmt)
00372             : condition(std::move(cond)), statement(std::move(stmt)) {}
00373
00374         void accept(ASTVisitor &visitor) override;
00375         std::string toString() const override;
00376     };
00377
00379     class ForStmtNode : public ASTNode {
00380     public:
00381         std::string variable;
00382         std::unique_ptr<ASTNode> startValue;
00383         std::unique_ptr<ASTNode> endValue;
00384         bool isDownto;
00385         std::unique_ptr<ASTNode> statement;
00386
00387         ForStmtNode(const std::string &var,
00388                   std::unique_ptr<ASTNode> start,
00389                   std::unique_ptr<ASTNode> end,
00390                   bool downto,
00391                   std::unique_ptr<ASTNode> stmt)
00392             : variable(var), startValue(std::move(start)), endValue(std::move(end)),
00393               isDownto(downto), statement(std::move(stmt)) {}
00394
00395         void accept(ASTVisitor &visitor) override;
00396         std::string toString() const override;
00397     };
00398
00400     class CaseStmtNode : public ASTNode {

```

```

00401     public:
00402     struct CaseBranch {
00403         std::vector<std::unique_ptr<ASTNode>> values;
00404         std::unique_ptr<ASTNode> statement;
00405
00406         CaseBranch(std::vector<std::unique_ptr<ASTNode>> vals, std::unique_ptr<ASTNode> stmt)
00407             : values(std::move(vals)), statement(std::move(stmt)) {}
00408     };
00409
00410     std::unique_ptr<ASTNode> expression;
00411     std::vector<std::unique_ptr<CaseBranch>> branches;
00412     std::unique_ptr<ASTNode> elseStatement;
00413
00414     CaseStmtNode(std::unique_ptr<ASTNode> expr,
00415         std::vector<std::unique_ptr<CaseBranch>> branches,
00416         std::unique_ptr<ASTNode> elseStmt = nullptr)
00417         : expression(std::move(expr)), branches(std::move(branches)),
00418         elseStatement(std::move(elseStmt)) {}
00419
00420     void accept(ASTVisitor &visitor) override;
00421     std::string toString() const override;
00422 };
00423
00424 class ProcCallNode : public ASTNode {
00425     public:
00426         std::string name;
00427         std::vector<std::unique_ptr<ASTNode>> arguments;
00428
00429         ProcCallNode(const std::string &procName, std::vector<std::unique_ptr<ASTNode>> args)
00430             : name(procName), arguments(std::move(args)) {}
00431
00432         void accept(ASTVisitor &visitor) override;
00433         std::string toString() const override;
00434 };
00435
00436 class BinaryOpNode : public ASTNode {
00437     public:
00438         enum OpType {
00439             PLUS,
00440             MINUS,
00441             MULTIPLY,
00442             DIVIDE,
00443             DIV,
00444             MOD,
00445             EQUAL,
00446             NOT_EQUAL,
00447             LESS,
00448             LESS_EQUAL,
00449             GREATER,
00450             GREATER_EQUAL,
00451             AND,
00452             OR,
00453             IN
00454         };
00455
00456         std::unique_ptr<ASTNode> left;
00457         OpType operator_;
00458         std::unique_ptr<ASTNode> right;
00459
00460         BinaryOpNode(std::unique_ptr<ASTNode> l, OpType op, std::unique_ptr<ASTNode> r)
00461             : left(std::move(l)), operator_(op), right(std::move(r)) {}
00462
00463         BinaryOpNode(OpType op, std::unique_ptr<ASTNode> l, std::unique_ptr<ASTNode> r)
00464             : left(std::move(l)), operator_(op), right(std::move(r)) {}
00465
00466         void accept(ASTVisitor &visitor) override;
00467         std::string toString() const override;
00468
00469         static std::string opToString(OpType op);
00470 };
00471
00472 class UnaryOpNode : public ASTNode {
00473     public:
00474         enum Operator {
00475             PLUS,
00476             MINUS,
00477             NOT
00478         };
00479
00480         Operator operator_;
00481         std::unique_ptr<ASTNode> operand;
00482
00483         UnaryOpNode(Operator op, std::unique_ptr<ASTNode> operandNode)
00484             : operator_(op), operand(std::move(operandNode)) {}
00485
00486         void accept(ASTVisitor &visitor) override;
00487         std::string toString() const override;
00488

```

```

00494
00496     static std::string opToString(Operator op);
00497 };
00498
00500 class FuncCallNode : public ASTNode {
00501 public:
00502     std::string name;
00503     std::vector<std::unique_ptr<ASTNode>> arguments;
00504
00505     FuncCallNode(const std::string &funcName, std::vector<std::unique_ptr<ASTNode>> args)
00506         : name(funcName), arguments(std::move(args)) {}
00507
00508     void accept(ASTVisitor &visitor) override;
00509     std::string toString() const override;
00510 };
00511
00513 class VariableNode : public ASTNode {
00514 public:
00515     std::string name;
00516
00517     VariableNode(const std::string &varName) : name(varName) {}
00518
00519     void accept(ASTVisitor &visitor) override;
00520     std::string toString() const override;
00521 };
00522
00524 class NumberNode : public ASTNode {
00525 public:
00526     std::string value;
00527     bool isInteger;
00528     bool isReal;
00529
00530     NumberNode(const std::string &val, bool integer = true, bool real = false)
00531         : value(val), isInteger(integer), isReal(real) {}
00532
00533     void accept(ASTVisitor &visitor) override;
00534     std::string toString() const override;
00535 };
00536
00538 class StringNode : public ASTNode {
00539 public:
00540     std::string value;
00541
00542     StringNode(const std::string &val) : value(val) {}
00543
00544     void accept(ASTVisitor &visitor) override;
00545     std::string toString() const override;
00546 };
00547
00549 class BooleanNode : public ASTNode {
00550 public:
00551     bool value;
00552
00553     BooleanNode(bool val) : value(val) {}
00554
00555     void accept(ASTVisitor &visitor) override;
00556     std::string toString() const override;
00557 };
00558
00560 class EmptyStmtNode : public ASTNode {
00561 public:
00562     EmptyStmtNode() {}
00563
00564     void accept(ASTVisitor &visitor) override;
00565     std::string toString() const override;
00566 };
00567
00569 class NilNode : public ASTNode {
00570 public:
00571     NilNode() = default;
00572
00573     void accept(ASTVisitor &visitor) override;
00574     std::string toString() const override { return "nil"; }
00575 };
00576
00578 class PointerTypeNode : public ASTNode {
00579 public:
00580     std::string baseTypeName;
00581
00582     PointerTypeNode(const std::string &base) : baseTypeName(base) {}
00583
00584     void accept(ASTVisitor &visitor) override;
00585     std::string toString() const override { return "PointerType: ^" + baseTypeName; }
00586 };
00587
00589 class PointerDerefNode : public ASTNode {
00590 public:

```

```

00591         std::unique_ptr<ASTNode> pointer;
00592
00593         PointerDerefNode(std::unique_ptr<ASTNode> ptr) : pointer(std::move(ptr)) {}
00594
00595         void accept(ASTVisitor &visitor) override;
00596         std::string toString() const override { return "PointerDeref"; }
00597     };
00598
00600     class AddressOfNode : public ASTNode {
00601     public:
00602         std::unique_ptr<ASTNode> operand;
00603
00604         AddressOfNode(std::unique_ptr<ASTNode> op) : operand(std::move(op)) {}
00605
00606         void accept(ASTVisitor &visitor) override;
00607         std::string toString() const override { return "AddressOf"; }
00608     };
00609
00611     class RepeatStmtNode : public ASTNode {
00612     public:
00613         std::vector<std::unique_ptr<ASTNode>> statements;
00614         std::unique_ptr<ASTNode> condition;
00615
00616         RepeatStmtNode(std::vector<std::unique_ptr<ASTNode>> stmts, std::unique_ptr<ASTNode> cond)
00617             : statements(std::move(stmts)), condition(std::move(cond)) {}
00618
00619         void accept(ASTVisitor &visitor) override;
00620
00621         std::string toString() const override {
00622             return "RepeatStmt";
00623         }
00624     };
00625
00627     class EnumTypeDeclNode : public ASTNode {
00628     public:
00629         std::string typeName;
00630         std::vector<std::string> values;
00631
00632         EnumTypeDeclNode(const std::string &name, std::vector<std::string> vals)
00633             : typeName(name), values(std::move(vals)) {}
00634
00635         void accept(ASTVisitor &visitor) override;
00636         std::string toString() const override { return "EnumTypeDecl: " + typeName; }
00637     };
00638
00640     struct VariantArm {
00641         std::vector<std::unique_ptr<ASTNode>> caseLabels;
00642         std::vector<std::unique_ptr<ASTNode>> fields;
00643     };
00644
00646     class RecordTypeNode : public ASTNode {
00647     public:
00648         std::vector<std::unique_ptr<ASTNode>> fields;
00649         std::string variantTagName;
00650         std::string variantTagType;
00651         std::vector<VariantArm> variantArms;
00652         RecordTypeNode(std::vector<std::unique_ptr<ASTNode>> fields);
00653         void accept(ASTVisitor &visitor) override;
00654         std::string toString() const override;
00655     };
00656
00658     class RecordDeclarationNode : public ASTNode {
00659     public:
00660         std::string name;
00661         std::unique_ptr<RecordTypeNode> recordType;
00662         RecordDeclarationNode(const std::string &name, std::unique_ptr<RecordTypeNode> recordType);
00663         void accept(ASTVisitor &visitor) override;
00664         std::string toString() const override;
00665     };
00666
00668     class WithStmtNode : public ASTNode {
00669     public:
00670         std::string recordVar;
00671         std::unique_ptr<ASTNode> statement;
00672
00673         WithStmtNode(const std::string &recordVar, std::unique_ptr<ASTNode> stmt)
00674             : recordVar(recordVar), statement(std::move(stmt)) {}
00675
00676         void accept(ASTVisitor &visitor) override;
00677         std::string toString() const override { return "WithStmt: " + recordVar; }
00678     };
00679
00681     class GotoStmtNode : public ASTNode {
00682     public:
00683         std::string label;
00684
00685         GotoStmtNode(const std::string &lbl) : label(lbl) {}

```

```

00686
00687     void accept(ASTVisitor &visitor) override;
00688     std::string toString() const override { return "Goto: " + label; }
00689 };
00690
00692 class LabelStmtNode : public ASTNode {
00693 public:
00694     std::string label;
00695     std::unique_ptr<ASTNode> statement;
00696
00697     LabelStmtNode(const std::string &lbl, std::unique_ptr<ASTNode> stmt)
00698         : label(lbl), statement(std::move(stmt)) {}
00699
00700     void accept(ASTVisitor &visitor) override;
00701     std::string toString() const override { return "Label: " + label; }
00702 };
00703
00705 class SetLiteralNode : public ASTNode {
00706 public:
00707     std::vector<std::unique_ptr<ASTNode>> elements;
00708
00709     SetLiteralNode(std::vector<std::unique_ptr<ASTNode>> elems)
00710         : elements(std::move(elems)) {}
00711
00712     void accept(ASTVisitor &visitor) override;
00713     std::string toString() const override { return "SetLiteral"; }
00714 };
00716 class SetTypeNode : public ASTNode {
00717 public:
00718     std::string baseTypeName;
00719
00720     SetTypeNode(const std::string &base) : baseTypeName(base) {}
00721
00722     void accept(ASTVisitor &visitor) override;
00723     std::string toString() const override { return "SetType: set of " + baseTypeName; }
00724 };
00726 class FieldAccessNode : public ASTNode {
00727 public:
00728     std::unique_ptr<ASTNode> recordExpr;
00729     std::string fieldName;
00730     FieldAccessNode(std::unique_ptr<ASTNode> recordExpr, const std::string &fieldName);
00731     void accept(ASTVisitor &visitor) override;
00732     std::string toString() const override;
00733 };
00734
00741 class ASTVisitor {
00742 public:
00743     virtual ~ASTVisitor() = default;
00744     virtual void visit(ProgramNode &node) = 0;
00745     virtual void visit(UnitNode &node) = 0;
00746     virtual void visit(BlockNode &node) = 0;
00747     virtual void visit(VarDeclNode &node) = 0;
00748     virtual void visit(ConstDeclNode &node) = 0;
00749     virtual void visit(ProcDeclNode &node) = 0;
00750     virtual void visit(FuncDeclNode &node) = 0;
00751     virtual void visit(ParameterNode &node) = 0;
00752     virtual void visit(CompoundStmtNode &node) = 0;
00753     virtual void visit(AssignmentNode &node) = 0;
00754     virtual void visit(IfStmtNode &node) = 0;
00755     virtual void visit(WhileStmtNode &node) = 0;
00756     virtual void visit(ForStmtNode &node) = 0;
00757     virtual void visit(ProcCallNode &node) = 0;
00758     virtual void visit(BinaryOpNode &node) = 0;
00759     virtual void visit(UnaryOpNode &node) = 0;
00760     virtual void visit(FuncCallNode &node) = 0;
00761     virtual void visit(VariableNode &node) = 0;
00762     virtual void visit(NumberNode &node) = 0;
00763     virtual void visit(StringNode &node) = 0;
00764     virtual void visit(BooleanNode &node) = 0;
00765     virtual void visit(EmptyStmtNode &node) = 0;
00766     virtual void visit(RepeatStmtNode &node) = 0;
00767     virtual void visit(CaseStmtNode &node) = 0;
00768     virtual void visit(ArrayTypeNode &node) = 0;
00769     virtual void visit(ArrayDeclarationNode &node) = 0;
00770     virtual void visit(ArrayAccessNode &node) = 0;
00771     virtual void visit(ArrayAssignmentNode &node) = 0;
00772     virtual void visit(RecordTypeNode &node) = 0;
00773     virtual void visit(RecordDeclarationNode &node) = 0;
00774     virtual void visit(FieldAccessNode &node) = 0;
00775     virtual void visit(TypeDeclNode &node) = 0;
00776     virtual void visit(TypeAliasNode &node) = 0;
00777     virtual void visit(ExitNode &node) = 0;
00778     virtual void visit(SimpleTypeNode &node) = 0;
00779     virtual void visit(ArrayTypeDeclarationNode &node) = 0;
00780     virtual void visit(ContinueNode &node) = 0;
00781     virtual void visit(BreakNode &node) = 0;
00782     virtual void visit(NilNode &node) = 0;

```

```

00783     virtual void visit(PointerTypeNode &node) = 0;
00784     virtual void visit(PointerDerefNode &node) = 0;
00785     virtual void visit(AddressOfNode &node) = 0;
00786     virtual void visit(WithStmtNode &node) = 0;
00787     virtual void visit(GotoStmtNode &node) = 0;
00788     virtual void visit(LabelStmtNode &node) = 0;
00789     virtual void visit(SetLiteralNode &node) = 0;
00790     virtual void visit(SetTypeNode &node) = 0;
00791     virtual void visit(EnumTypeDeclNode &node) = 0;
00792 };
00793 } // namespace pascal
00794
00795 #endif

```

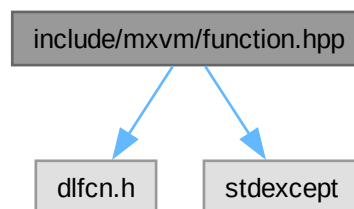
12.5 include/mxvm/function.hpp File Reference

Dynamic function loader using dlopen/dlsym for runtime symbol resolution.

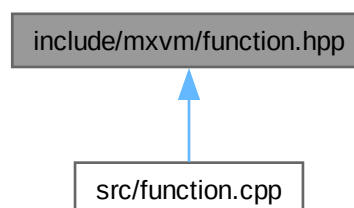
```
#include <dlfcn.h>
```

```
#include <stdexcept>
```

Include dependency graph for function.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `mxvm::Function< R(Args...)>`
RAII wrapper for dynamically loaded functions via dlopen/dlsym.

Namespaces

- namespace `mxvm`

12.5.1 Detailed Description

Dynamic function loader using dlopen/dlsym for runtime symbol resolution.

Author

Jared Bruni

Definition in file [function.hpp](#).

12.6 function.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef __FUNCTION_H__
00007 #define __FUNCTION_H__
00008
00009 #include <dlfcn.h>
00010 #include <stdexcept>
00011
00012 namespace mxvm {
00013
00014     template <typename Sig>
00015     class Function;
00016
00017     template <typename R, typename... Args>
00018     class Function<R(Args...)> {
00019     public:
00020         using Fn = R (*)(Args...);
00021
00022         Function(char const *lib, char const *name) {
00023             handle = dlopen(lib, RTLD_LAZY);
00024             if (!handle)
00025                 throw std::runtime_error(dlerror());
00026             void *sym = dlsym(handle, name);
00027             if (!sym) {
00028                 dlclose(handle);
00029                 throw std::runtime_error(dlerror());
00030             }
00031             fn = reinterpret_cast<Fn>(sym);
00032         }
00033
00034         Function(char const *name) {
00035             handle = dlopen(nullptr, RTLD_LAZY);
00036             if (!handle)
00037                 throw std::runtime_error(dlerror());
00038             void *sym = dlsym(handle, name);
00039             if (!sym) {
00040                 dlclose(handle);
00041                 throw std::runtime_error(dlerror());
00042             }
00043             fn = reinterpret_cast<Fn>(sym);
00044         }
00045
00046         ~Function() {
00047             if (handle)
00048                 dlclose(handle);
00049         }
00050
00051         Function(const Function &) = delete;
00052         Function &operator=(const Function &) = delete;
00053
00054         Function(Function &&other) noexcept : handle(other.handle), fn(other.fn) {
00055             other.handle = nullptr;
00056             other.fn = nullptr;
00057         }
00058
00059         Function &operator=(Function &&other) noexcept {
00060             if (this != &other) {
00061                 if (handle)
00062                     dlclose(handle);
00063                 handle = other.handle;
00064                 fn = other.fn;
00065                 other.handle = nullptr;
00066                 other.fn = nullptr;
00067             }
00068             return *this;
00069         }
00070
00071         R operator()(Args... args) const {
00072             if (!fn)
00073                 throw std::runtime_error("Null function pointer");
00074             return fn(std::forward<Args>(args)...);
00075         }
00076     };

```

```

00094
00095     private:
00096         void *handle = nullptr;
00097         Fn fn = nullptr;
00098     };
00099
00100 } // namespace mxvm
00101
00102 #endif

```

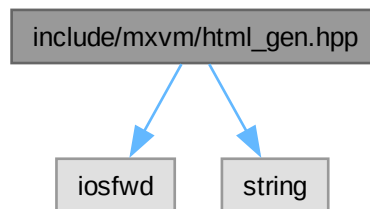
12.7 include/mxvm/html_gen.hpp File Reference

HTML documentation generator from MXVM source code.

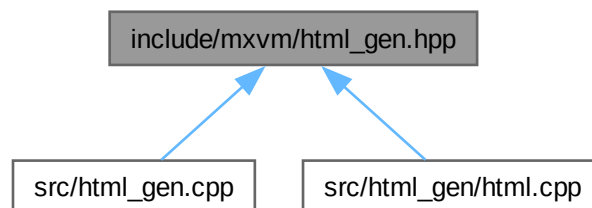
```
#include <iosfwd>
```

```
#include <string>
```

Include dependency graph for html_gen.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [mxvm::HTMLGen](#)
Generates syntax-highlighted HTML output from MXVM source code.
- class [mxvm::PasHTMLGen](#)
Generates syntax-highlighted HTML output from Pascal (.pas) source code.

Namespaces

- namespace [mxvm](#)

12.7.1 Detailed Description

HTML documentation generator from MXVM source code.

Author

Jared Bruni

Definition in file [html_gen.hpp](#).

12.8 html_gen.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef __HTML_GEN__H_
00007 #define __HTML_GEN__H_
00008
00009 #include <iosfwd>
00010 #include <string>
00011
00012 namespace mxvm {
00013
00015     class HTMLGen {
00016     public:
00021         HTMLGen(const std::string &src);
00022
00028         void output(std::ostream &out, std::string filename);
00029
00030     private:
00031         std::string source;
00032     };
00033
00046     class PasHTMLGen {
00047     public:
00049         explicit PasHTMLGen(const std::string &src);
00050
00057         void output(std::ostream &out, const std::string &filename,
00058                     const std::string &cssHref = "mxvm-highlight.css");
00059
00067         static std::string defaultCss();
00068
00069     private:
00070         std::string source;
00071     };
00072 } // namespace mxvm
00073
00074 #endif

```

12.9 include/mxvm/icode.hpp File Reference

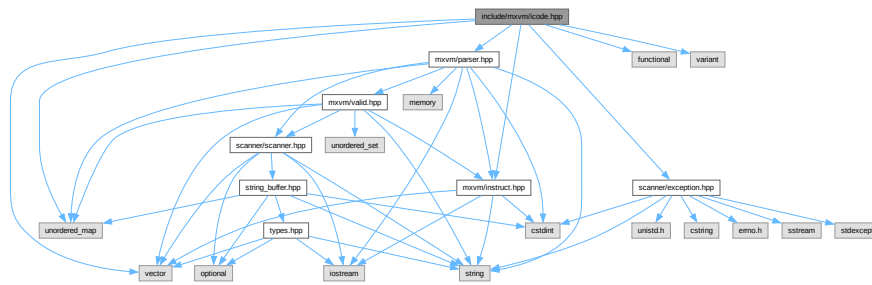
Intermediate code representation, execution engine, and native x64 code generation (SysV + Win64).

```

#include "mxvm/instruct.hpp"
#include "mxvm/parser.hpp"
#include "scanner/exception.hpp"
#include <functional>
#include <unordered_map>
#include <variant>
#include <vector>

```

Include dependency graph for icode.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [mxvm::Stack](#)
Runtime operand stack for the MXVM interpreter.
- class [mxvm::RuntimeFunction](#)
Wraps a dynamically loaded native function from a shared library module.
- class [mxvm::Base](#)
Base class for instruction storage, variables, labels, and external function registrations.
- class [mxvm::Program](#)
Complete MXVM program: instruction interpreter and native x64 code generator.

Namespaces

- namespace [mxvm](#)

Typedefs

- using [mxvm::StackValue](#) = `std::variant<int64_t, void *, double, std::string>`
Stack element: integer, pointer, double, or string.
- using [mxvm::runtime_call](#) = `std::function<void(Program *program, std::vector<Operand> &operands)>`
Callback type for runtime-registered native functions.

Functions

- void [mxvm::except_assert](#) (std::string reason, bool value)
Assert a condition, throwing an Exception with the given reason on failure.
- bool [mxvm::isFunctionReturningOwnedPtr](#) (const std::string &funcName)
Check whether a named function returns an owned (heap-allocated) pointer.

12.9.1 Detailed Description

Intermediate code representation, execution engine, and native x64 code generation (SysV + Win64).

Author

Jared Bruni

Definition in file [icode.hpp](#).

12.10 icode.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef ICODE_HPP_
00007 #define ICODE_HPP_
00008
00009 #include "mxvm/instruct.hpp"
00010 #include "mxvm/parser.hpp"
00011 #include "scanner/exception.hpp"
00012 #include <functional>
00013 #include <unordered_map>
00014 #include <variant>
00015 #include <vector>
00016
00017 namespace mxvm {
00018
00020     using StackValue = std::variant<int64_t, void *, double, std::string>;
00021
00023     class Stack {
00024     public:
00025         Stack() = default;
00026
00030         void push(const StackValue &value) {
00031             data.push_back(value);
00032         }
00033
00038         [[nodiscard]] StackValue pop() {
00039             if (data.empty())
00040                 throw mx::Exception("Stack underflow");
00041             StackValue val = data.back();
00042             data.pop_back();
00043             return val;
00044         }
00045
00047         [[nodiscard]] bool empty() const { return data.empty(); }
00048
00050         [[nodiscard]] size_t size() const { return data.size(); }
00051
00057         StackValue &operator[](size_t index) {
00058             if (index >= data.size())
00059                 throw mx::Exception("Stack index out of bounds: " + std::to_string(index));
00060             return data[index];
00061         }
00062
00063     private:
00064         std::vector<StackValue> data;
00065     };
00066
00067     class Program;
00068
00070     using runtime_call = std::function<void(Program *program, std::vector<Operand> &operands)>;
00071
00078     class RuntimeFunction {
00079     public:
00081         RuntimeFunction() : func(nullptr), handle(nullptr) {}
00082
00084         RuntimeFunction(const RuntimeFunction &r) : func(r.func), handle(r.handle),
mod_name(r.mod_name), fname(r.fname) {}
00085
00087         RuntimeFunction &operator=(const RuntimeFunction &r) {
00088             func = r.func;
00089             handle = r.handle;
00090             mod_name = r.mod_name;
00091             fname = r.fname;
00092             return *this;
00093         }
00094         ~RuntimeFunction() = default;
00095
00100         RuntimeFunction(const std::string &mod, const std::string &name);
00101
00106         void call(Program *program, std::vector<Operand> &operands);
00107         void *func = nullptr;
00108         void *handle = nullptr;
00109         std::string mod_name;
00110         std::string fname;
00111         static std::unordered_map<std::string, void *> handles;

```

```

00112     };
00113
00121     class Base {
00122     public:
00123         Base() = default;
00125         Base(Base &&other) noexcept;
00126         ~Base() = default;
00128         Base &operator=(Base &&other) noexcept;
00129
00133         void setMainBase(Base *b) { Base::base = b; }
00134
00138         void add_instruction(const Instruction &i);
00139
00145         void add_label(const std::string &name, uint64_t address, bool f);
00146
00151         void add_variable(const std::string &name, const Variable &v);
00152
00158         void add_global(const std::string &objname, const std::string &name, const Variable &v);
00159
00165         void add_extern(const std::string &mod, const std::string &name, bool module);
00166
00173         void add_runtime_extern(const std::string &mod_name, const std::string &mod, const std::string
&func_name, const std::string &name);
00174
00178         void add_filename(const std::string &fname);
00179
00184         void add_allocated(const std::string &name, Variable &v);
00185
00190         void add_object(const std::string &name, Program *prog);
00191         std::string name;
00192         std::vector<Instruction> inc;
00193         std::unordered_map<std::string, Variable> vars;
00194         std::unordered_map<std::string, std::pair<uint64_t, bool> labels;
00195         std::vector<ExternalFunction> external;
00196         std::unordered_map<std::string, RuntimeFunction> external_functions;
00197         static Base *base;
00198         static std::string root_name;
00199         static std::vector<std::string> filenames;
00200         static std::unordered_map<std::string, Variable> allocated;
00201         static std::unordered_map<std::string, Program *> object_map;
00202         std::string assembly_code;
00203     };
00204
00212     class Program : public Base {
00213     public:
00214         friend class Parser;
00216         Program();
00218         ~Program();
00219
00221         Program(Program &&other) noexcept
00222             : Base(std::move(other)),
00223               filename(std::move(other.filename)),
00224               objects(std::move(other.objects)),
00225               object(other.object),
00226               pc(other.pc),
00227               running(other.running),
00228               exitCode(other.exitCode),
00229               zero_flag(other.zero_flag),
00230               less_flag(other.less_flag),
00231               greater_flag(other.greater_flag),
00232               carry_flag(other.carry_flag),
00233               xmm_offset(other.xmm_offset),
00234               args(std::move(other.args)),
00235               main_function(other.main_function),
00236               object_external(other.object_external),
00237               stack(std::move(other.stack)),
00238               result(std::move(other.result)),
00239               parent(other.parent),
00240               platform(other.platform) {
00241             other.pc = 0;
00242             other.running = false;
00243             other.exitCode = 0;
00244             for (auto &var_pair : vars) {
00245                 var_pair.second.obj_name = name;
00246             }
00247         }
00248
00250         Program &operator=(Program &&other) noexcept {
00251             if (this != &other) {
00252                 Base::operator=(std::move(other));
00253                 filename = std::move(other.filename);
00254                 objects = std::move(other.objects);
00255                 object = other.object;
00256                 pc = other.pc;
00257                 running = other.running;
00258                 exitCode = other.exitCode;
00259                 zero_flag = other.zero_flag;

```

```

00260         less_flag = other.less_flag;
00261         greater_flag = other.greater_flag;
00262         carry_flag = other.carry_flag;
00263         xmm_offset = other.xmm_offset;
00264         args = std::move(other.args);
00265         main_function = other.main_function;
00266         object_external = other.object_external;
00267         stack = std::move(other.stack);
00268         result = std::move(other.result);
00269         parent = other.parent;
00270         other.pc = 0;
00271         other.running = false;
00272         other.exitCode = 0;
00273         for (auto &var_pair : vars) {
00274             var_pair.second.obj_name = name;
00275         }
00276         platform = other.platform;
00277     }
00278     return *this;
00279 }
00280
00282 void add_standard();
00283
00285 void stop();
00286
00291 int exec();
00295 void print(std::ostream &out);
00296
00300 void post(std::ostream &out);
00301
00305 int getExitCode() const { return exitCode; }
00306 std::string name;
00307 std::string filename;
00312 std::string getPlatformSymbolName(const std::string &name);
00319 void generateCode(const Platform &platform, bool obj, std::ostream &out);
00324 static std::string escapeNewLines(const std::string &text);
00325
00329 void memoryDump(std::ostream &out);
00330
00334 void setArgs(const std::vector<std::string> &argv);
00335
00339 void setObject(bool obj);
00340 std::vector<std::unique_ptr<Program>> objects;
00341 bool object = false;
00346 bool validateNames(Validator &v);
00347
00351 void flatten(Program *program);
00352
00357 void flatten_inc(Program *root, Instruction &i);
00358
00365 void flatten_label(Program *root, int64_t offset, const std::string &label, bool func);
00366
00372 void flatten_external(Program *root, const std::string &e, RuntimeFunction &r);
00373
00378 std::string getMangledName(const std::string &var);
00379
00384 std::string getMangledName(const Operand &op);
00385
00390 Program *getObjectByName(const std::string &name);
00391
00396 bool isFunctionValid(const std::string &label);
00397
00398 private:
00399     size_t pc;
00400     bool running;
00401     int exitCode = 0;
00402     bool zero_flag = false;
00403     bool less_flag = false;
00404     bool greater_flag = false;
00405     bool carry_flag = false;
00406     int xmm_offset = 0;
00407     std::vector<std::string> args;
00408     bool main_function;
00409     bool object_external = false;
00410
00412     enum LastCmpType {
00413         CMP_NONE,
00414         CMP_INTEGER,
00415         CMP_FLOAT
00416     };
00417     LastCmpType last_cmp_type = CMP_NONE;
00418     bool last_call_returns_owned_ptr = false;
00419
00420 public:
00428     void generateFunctionCall(std::ostream &out, const std::string &name, std::vector<Operand>
&op);
00429

```

```

00434 void generateInvokeCall(std::ostream &out, std::vector<Operand> &op);
00435
00440 void generateInstruction(std::ostream &out, const Instruction &i);
00441
00448 int generateLoadVar(std::ostream &out, int reg, const Operand &op);
00449
00457 int generateLoadVar(std::ostream &out, VarType type, std::string reg, const Operand &op);
00458
00464 std::string getRegisterByIndex(int index, VarType type);
00465 void gen_print(std::ostream &out, const Instruction &i);
00466 void gen_arth(std::ostream &out, std::string arth, const Instruction &i);
00467 void gen_exit(std::ostream &out, const Instruction &i);
00468 void gen_mov(std::ostream &out, const Instruction &i);
00469 void gen_jmp(std::ostream &out, const Instruction &i);
00470 void gen_cmp(std::ostream &out, const Instruction &i);
00471 void gen_alloc(std::ostream &out, const Instruction &i);
00473 void gen_realloc(std::ostream &out, const Instruction &i);
00474 void gen_free(std::ostream &out, const Instruction &i);
00475 void gen_load(std::ostream &out, const Instruction &i);
00476 void gen_store(std::ostream &out, const Instruction &i);
00477 void gen_ret(std::ostream &out, const Instruction &i);
00478 void gen_call(std::ostream &out, const Instruction &i);
00479 void gen_done(std::ostream &out, const Instruction &i);
00480 void gen_bitop(std::ostream &out, const std::string &opc, const Instruction &i);
00481 void gen_not(std::ostream &out, const Instruction &i);
00482 void gen_push(std::ostream &out, const Instruction &i);
00483 void gen_pop(std::ostream &out, const Instruction &i);
00484 void gen_stack_load(std::ostream &out, const Instruction &i);
00485 void gen_stack_store(std::ostream &out, const Instruction &i);
00486 void gen_stack_sub(std::ostream &out, const Instruction &i);
00487 void gen_getline(std::ostream &out, const Instruction &i);
00488 void gen_to_int(std::ostream &out, const Instruction &i);
00489 void gen_to_float(std::ostream &out, const Instruction &i);
00490 void gen_invoke(std::ostream &out, const Instruction &i);
00491 void gen_return(std::ostream &out, const Instruction &i);
00492 void gen_neg(std::ostream &out, const Instruction &i);
00493 void gen_lea(std::ostream &out, const Instruction &i);
00494 void gen_fcmp(std::ostream &out, const Instruction &i);
00495 void gen_jae(std::ostream &out, const Instruction &i);
00496 void gen_jbe(std::ostream &out, const Instruction &i);
00497 void gen_jc(std::ostream &out, const Instruction &i);
00498 void gen_jnc(std::ostream &out, const Instruction &i);
00499 void gen_jp(std::ostream &out, const Instruction &i);
00500 void gen_jnp(std::ostream &out, const Instruction &i);
00501 void gen_jo(std::ostream &out, const Instruction &i);
00502 void gen_jno(std::ostream &out, const Instruction &i);
00503 void gen_js(std::ostream &out, const Instruction &i);
00504 void gen_jns(std::ostream &out, const Instruction &i);
00505 void gen_add(std::ostream &out, const Instruction &i);
00506 void gen_sub(std::ostream &out, const Instruction &i);
00507 void gen_mul(std::ostream &out, const Instruction &i);
00508 void gen_div(std::ostream &out, const Instruction &i);
00509 void gen_mod(std::ostream &out, const Instruction &i);
00510
00517 std::string gen_optimize(const std::string &code, const Platform &platform);
00518
00523 std::string optimize_darwin(const std::string &code);
00524
00529 std::string optimize_core(const std::string &code);
00531
00539 void x64_emit_iob_func(std::ostream &out, int index, const std::string &dstReg);
00540
00546 std::string x64_getRegisterByIndex(int index, VarType type);
00547
00554 int x64_generateLoadVar(std::ostream &out, int r, const Operand &op);
00555
00563 int x64_generateLoadVar(std::ostream &out, VarType type, std::string reg, const Operand &op);
00564
00570 void x64_generateFunctionCall(std::ostream &out, const std::string &name, std::vector<Operand>
&args);
00571
00576 void x64_generateInvokeCall(std::ostream &out, std::vector<Operand> &op);
00577
00583 void x64_generateCode(const Platform &platform, bool obj, std::ostream &out);
00584
00589 void x64_generateInstruction(std::ostream &out, const Instruction &i);
00590 void x64_gen_done(std::ostream &out, const Instruction &i);
00591 void x64_gen_ret(std::ostream &out, const Instruction &i);
00592 void x64_gen_return(std::ostream &out, const Instruction &i);
00593 void x64_gen_neg(std::ostream &out, const Instruction &i);
00594 void x64_gen_invoke(std::ostream &out, const Instruction &i);
00595 void x64_gen_call(std::ostream &out, const Instruction &i);
00596 void x64_gen_alloc(std::ostream &out, const Instruction &i);
00598 void x64_gen_realloc(std::ostream &out, const Instruction &i);
00599 void x64_gen_free(std::ostream &out, const Instruction &i);
00600 void x64_gen_load(std::ostream &out, const Instruction &i);
00601 void x64_gen_store(std::ostream &out, const Instruction &i);

```

```

00602 void x64_gen_to_int(std::ostream &out, const Instruction &i);
00603 void x64_gen_to_float(std::ostream &out, const Instruction &i);
00604 void x64_gen_div(std::ostream &out, const Instruction &i);
00605 void x64_gen_mod(std::ostream &out, const Instruction &i);
00606 void x64_gen_getline(std::ostream &out, const Instruction &i);
00607 void x64_gen_bitop(std::ostream &out, const std::string &opc, const Instruction &i);
00608 void x64_gen_not(std::ostream &out, const Instruction &i);
00609 void x64_gen_push(std::ostream &out, const Instruction &i);
00610 void x64_gen_pop(std::ostream &out, const Instruction &i);
00611 void x64_gen_stack_load(std::ostream &out, const Instruction &i);
00612 void x64_gen_stack_store(std::ostream &out, const Instruction &i);
00613 void x64_gen_stack_sub(std::ostream &out, const Instruction &i);
00614 void x64_gen_print(std::ostream &out, const Instruction &i);
00615 void x64_gen_jmp(std::ostream &out, const Instruction &i);
00616 void x64_gen_cmp(std::ostream &out, const Instruction &i);
00617 void x64_gen_mov(std::ostream &out, const Instruction &i);
00618 void x64_gen_arth(std::ostream &out, std::string arth, const Instruction &i);
00619 void x64_gen_exit(std::ostream &out, const Instruction &i);
00620 void x64_gen_lea(std::ostream &out, const Instruction &i);
00621 void x64_gen_fcmp(std::ostream &out, const Instruction &i);
00622 void x64_gen_jae(std::ostream &out, const Instruction &i);
00623 void x64_gen_jbe(std::ostream &out, const Instruction &i);
00624 void x64_gen_jc(std::ostream &out, const Instruction &i);
00625 void x64_gen_jnc(std::ostream &out, const Instruction &i);
00626 void x64_gen_jp(std::ostream &out, const Instruction &i);
00627 void x64_gen_jnp(std::ostream &out, const Instruction &i);
00628 void x64_gen_jo(std::ostream &out, const Instruction &i);
00629 void x64_gen_jno(std::ostream &out, const Instruction &i);
00630 void x64_gen_js(std::ostream &out, const Instruction &i);
00631 void x64_gen_jns(std::ostream &out, const Instruction &i);
00632
00633 std::unordered_map<std::string, std::string> x64_reg_vars;
00634 std::vector<std::string> x64_reg_save_order;
00635
00639 void x64_analyzeRegAlloc(bool uses_std_module);
00640
00642 void x64_emitFlushRegs(std::ostream &out);
00643
00645 void x64_emitReloadRegs(std::ostream &out);
00646
00648 void x64_emitSaveRegs(std::ostream &out);
00649
00651 void x64_emitRestoreRegs(std::ostream &out);
00652
00658 void x64_emitStoreVar(std::ostream &out, const std::string &srcReg, const Operand &dest);
00659
00665 void x64_emitStoreVarImm(std::ostream &out, const std::string &imm, const Operand &dest);
00666
00672 void x64_emitLoadVar(std::ostream &out, const std::string &dstReg, const Operand &src);
00674
00677 std::unordered_map<std::string, std::string> sysv_reg_vars;
00678 std::vector<std::string> sysv_reg_save_order;
00679
00683 void sysv_analyzeRegAlloc(bool uses_std_module);
00684
00686 void sysv_emitFlushRegs(std::ostream &out);
00687
00689 void sysv_emitReloadRegs(std::ostream &out);
00690
00692 void sysv_emitSaveRegs(std::ostream &out);
00693
00695 void sysv_emitRestoreRegs(std::ostream &out);
00696
00702 void sysv_emitStoreVar(std::ostream &out, const std::string &srcReg, const Operand &dest);
00703
00709 void sysv_emitStoreVarImm(std::ostream &out, const std::string &imm, const Operand &dest);
00710
00716 void sysv_emitLoadVar(std::ostream &out, const std::string &dstReg, const Operand &src);
00718
00721 void exec_mov(const Instruction &instr);
00722 void exec_add(const Instruction &instr);
00723 void exec_sub(const Instruction &instr);
00724 void exec_mul(const Instruction &instr);
00725 void exec_div(const Instruction &instr);
00726 void exec_cmp(const Instruction &instr);
00727 void exec_jmp(const Instruction &instr);
00728 void exec_load(const Instruction &instr);
00729 void exec_store(const Instruction &instr);
00730 void exec_or(const Instruction &instr);
00731 void exec_and(const Instruction &instr);
00732 void exec_xor(const Instruction &instr);
00733 void exec_not(const Instruction &instr);
00734 void exec_mod(const Instruction &instr);
00735 void exec_je(const Instruction &instr);
00736 void exec_jne(const Instruction &instr);
00737 void exec_jl(const Instruction &instr);
00738 void exec_jle(const Instruction &instr);

```

```

00739     void exec_jg(const Instruction &instr);
00740     void exec_jge(const Instruction &instr);
00741     void exec_jz(const Instruction &instr);
00742     void exec_jnz(const Instruction &instr);
00743     void exec_ja(const Instruction &instr);
00744     void exec_jb(const Instruction &instr);
00745     void exec_print(const Instruction &instr);
00746     void exec_string_print(const Instruction &instr);
00747     void exec_exit(const Instruction &instr);
00748     void exec_alloc(const Instruction &instr);
00749     void exec_free(const Instruction &instr);
00750     void exec_getline(const Instruction &instr);
00751     void exec_push(const Instruction &instr);
00752     void exec_pop(const Instruction &instr);
00753     void exec_stack_load(const Instruction &instr);
00754     void exec_stack_store(const Instruction &instr);
00755     void exec_stack_sub(const Instruction &instr);
00756     void exec_call(const Instruction &instr);
00757     void exec_ret(const Instruction &instr);
00758     void exec_done(const Instruction &instr);
00759     void exec_to_int(const Instruction &instr);
00760     void exec_to_float(const Instruction &instr);
00761     void exec_invoke(const Instruction &instr);
00762     void exec_return(const Instruction &instr);
00763     void exec_lea(const Instruction &instr);
00764     void exec_neg(const Instruction &instr);
00773     void exec_realloc(const Instruction &instr);
00774     void exec_fcmp(const Instruction &instr);
00775     void exec_jae(const Instruction &instr);
00776     void exec_jbe(const Instruction &instr);
00777     void exec_jc(const Instruction &instr);
00778     void exec_jnc(const Instruction &instr);
00779     void exec_jp(const Instruction &instr);
00780     void exec_jnp(const Instruction &instr);
00781     void exec_jo(const Instruction &instr);
00782     void exec_jno(const Instruction &instr);
00783     void exec_js(const Instruction &instr);
00784     void exec_jns(const Instruction &instr);
00786
00794     Variable &getVariable(const std::string &name);
00795
00800     bool isVariable(const std::string &name);
00801
00806     void setVariableFromString(Variable &var, const std::string &value);
00807
00813     void addVariables(Variable &dest, Variable &src1, Variable &src2);
00814
00816     void subVariables(Variable &dest, Variable &src1, Variable &src2);
00817
00819     void mulVariables(Variable &dest, Variable &src1, Variable &src2);
00820
00822     void divVariables(Variable &dest, Variable &src1, Variable &src2);
00823
00830     std::string printFormatted(const std::string &format, const std::vector<Variable *> &args,
bool output = true);
00831
00836     void setVariableFromConstant(Variable &var, const std::string &value);
00837
00843     Variable createTempVariable(VarType type, const std::string &value);
00844
00849     bool isConstant(const std::string &value);
00850
00855     Variable variableFromOperand(const Operand &op);
00857
00858     Stack stack;
00859     Operand result;
00860     Program *parent = nullptr;
00861     Platform platform;
00867     size_t x64_reserve_call_area(std::ostream &out, size_t spill_bytes);
00868
00873     void x64_release_call_area(std::ostream &out, size_t frame);
00874
00879     void x64_direct_stack_adjust(std::ostream &out, int64_t bytes);
00880     bool last_call_returns_owned = false;
00881     VarType last_call_ret_type = VarType::VAR_INTEGER;
00882 };
00883
00888     void except_assert(std::string reason, bool value);
00889
00894     bool isFunctionReturningOwnedPtr(const std::string &funcName);
00895
00896 } // namespace mxvm
00897
00898 #endif

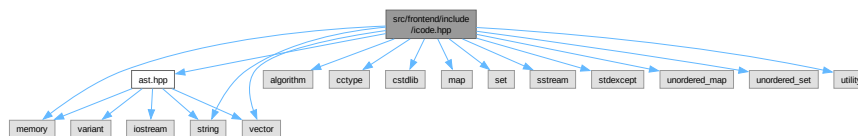
```


12.11 src/frontend/include/icode.hpp File Reference

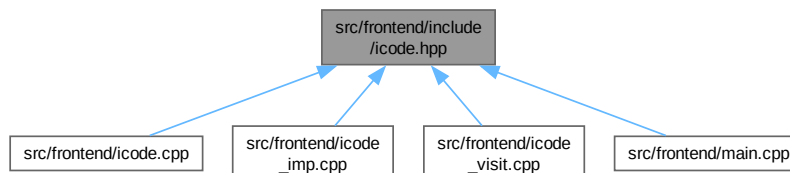
Pascal-to-MXVM code generator using the Visitor pattern over the Pascal AST.

```
#include "ast.hpp"
#include <algorithm>
#include <cctype>
#include <cstdlib>
#include <map>
#include <memory>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <utility>
#include <vector>
```

Include dependency graph for icode.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [pascal::ArrayInfo](#)
Metadata describing a Pascal array's bounds, element type, and sizing.
- class [pascal::BuiltinFunctionHandler](#)
Abstract handler for a group of built-in functions/procedures.
- class [pascal::BuiltinFunctionRegistry](#)
Registry of [BuiltinFunctionHandler](#) instances.
- class [pascal::IOFunctionHandler](#)
Built-in handler for I/O procedures (write, writeln, read, readln).
- class [pascal::StdFunctionHandler](#)
Built-in handler for standard library functions (chr, ord, inc, dec, etc.).
- class [pascal::StringFunctionHandler](#)
Built-in handler for string library functions (length, copy, concat, etc.).
- class [pascal::SDLFunctionHandler](#)
Built-in handler for SDL2 bindings.

- class [pascal::FileFunctionHandler](#)
Built-in handler for Pascal file I/O procedures (assign, reset, rewrite, append, close, eof, eoln).
- class [pascal::CodeGenVisitor](#)
AST visitor that generates MXVM intermediate code.
- struct [pascal::CodeGenVisitor::Scope](#)
- struct [pascal::CodeGenVisitor::FuncInfo](#)
Describes a user-declared function's parameter and return types.
- struct [pascal::CodeGenVisitor::ValueLocation](#)
Describes where a value currently resides (register, memory, or immediate).
- struct [pascal::CodeGenVisitor::RecordField](#)
A field within a record type definition.
- struct [pascal::CodeGenVisitor::RecordTypeInfo](#)
Complete description of a record type.

Namespaces

- namespace [pascal](#)

Macros

- `#define` [MXVM_BOUNDS_CHECK](#)

Enumerations

- enum class [pascal::VarType](#) {
[pascal::INT](#) , [pascal::DOUBLE](#) , [pascal::STRING](#) , [pascal::CHAR](#) ,
[pascal::RECORD](#) , [pascal::PTR](#) , [pascal::BOOL](#) , [pascal::UNKNOWN](#) ,
[pascal::ARRAY_INT](#) , [pascal::ARRAY_DOUBLE](#) , [pascal::ARRAY_STRING](#) }
Enumeration of variable / expression value types.

Functions

- `std::string` [pascal::mxvmOpt](#) (const `std::string` &text)

12.11.1 Detailed Description

Pascal-to-MXVM code generator using the Visitor pattern over the Pascal AST.

Author

Jared Bruni

Definition in file [icode.hpp](#).

12.11.2 Macro Definition Documentation

12.11.2.1 MXVM_BOUNDS_CHECK

```
#define MXVM_BOUNDS_CHECK
```

Definition at line 22 of file [icode.hpp](#).

12.12 icode.hpp

[Go to the documentation of this file.](#)

```
00001
00006 #ifndef __CODEGEN_H_
00007 #define __CODEGEN_H_
00008 #include "ast.hpp"
00009 #include <algorithm>
```

```

00010 #include <cctype>
00011 #include <cstdlib>
00012 #include <map>
00013 #include <memory>
00014 #include <set>
00015 #include <sstream>
00016 #include <stdexcept>
00017 #include <string>
00018 #include <unordered_map>
00019 #include <unordered_set>
00020 #include <utility>
00021 #include <vector>
00022 #define MXVM_BOUNDS_CHECK
00023
00024 namespace pascal {
00025
00026     struct ArrayInfo {
00027         std::string elementType;
00028         int lowerBound = 0;
00029         int upperBound = -1;
00030         int size = 0;
00031         int elementSize = 8;
00032         bool elementIsArray = false;
00033         bool isDynamic = false;
00034         std::unique_ptr<ArrayInfo> elementArray;
00035
00036         ArrayInfo() = default;
00037         ArrayInfo(const ArrayInfo &other)
00038             : elementType(other.elementType), lowerBound(other.lowerBound),
00039               upperBound(other.upperBound), size(other.size), elementSize(other.elementSize),
00040               elementIsArray(other.elementIsArray), isDynamic(other.isDynamic) {
00041             if (other.elementArray) {
00042                 elementArray = std::make_unique<ArrayInfo>(*other.elementArray);
00043             }
00044         }
00045         ArrayInfo(ArrayInfo &&other) noexcept = default;
00046         ArrayInfo &operator=(const ArrayInfo &other) {
00047             if (this != &other) {
00048                 elementType = other.elementType;
00049                 lowerBound = other.lowerBound;
00050                 upperBound = other.upperBound;
00051                 size = other.size;
00052                 elementSize = other.elementSize;
00053                 elementIsArray = other.elementIsArray;
00054                 isDynamic = other.isDynamic;
00055                 if (other.elementArray) {
00056                     elementArray = std::make_unique<ArrayInfo>(*other.elementArray);
00057                 } else {
00058                     elementArray.reset();
00059                 }
00060             }
00061             return *this;
00062         }
00063         ArrayInfo &operator=(ArrayInfo &&other) noexcept = default;
00064     };
00065
00066     class CodeGenVisitor;
00067
00068     enum class VarType {
00069         INT,
00070         DOUBLE,
00071         STRING,
00072         CHAR,
00073         RECORD,
00074         PTR,
00075         BOOL,
00076         UNKNOWN,
00077         ARRAY_INT,
00078         ARRAY_DOUBLE,
00079         ARRAY_STRING
00080     };
00081
00082     class BuiltinFunctionHandler {
00083     public:
00084         virtual ~BuiltinFunctionHandler() = default;
00085         virtual bool canHandle(const std::string &funcName) const = 0;
00086         virtual void generate(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) = 0;
00087         virtual bool generateWithResult(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) { return false; }
00088         virtual VarType getReturnType(const std::string &funcName) const { return VarType::UNKNOWN; }
00089     };
00090
00091     class BuiltinFunctionRegistry {
00092     private:
00093         std::vector<std::unique_ptr<BuiltinFunctionHandler> > handlers;
00094     };

```

```

00115     public:
00117         void registerHandler(std::unique_ptr<BuiltinFunctionHandler> handler) {
00118             handlers.push_back(std::move(handler)); }
00122         BuiltinFunctionHandler *findHandler(const std::string &funcName) {
00123             for (auto &h : handlers)
00124                 if (h->canHandle(funcName))
00125                     return h.get();
00126             return nullptr;
00127         }
00128     };
00129
00131     class IOFunctionHandler : public BuiltinFunctionHandler {
00132     public:
00133         bool canHandle(const std::string &funcName) const override;
00134         void generate(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00135         bool generateWithResult(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00136     };
00137
00139     class StdFunctionHandler : public BuiltinFunctionHandler {
00140     public:
00141         bool canHandle(const std::string &funcName) const override;
00142         void generate(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00143         bool generateWithResult(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00144     };
00145
00147     class StringFunctionHandler : public BuiltinFunctionHandler {
00148     public:
00149         bool canHandle(const std::string &funcName) const override;
00150         void generate(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00151         bool generateWithResult(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00152         VarType getReturnType(const std::string &funcName) const override;
00153     };
00155     class SDLFunctionHandler : public BuiltinFunctionHandler {
00156     public:
00157         bool canHandle(const std::string &funcName) const override;
00158         void generate(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00159         bool generateWithResult(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00160     };
00161
00163     class FileFunctionHandler : public BuiltinFunctionHandler {
00164     public:
00165         bool canHandle(const std::string &funcName) const override;
00166         void generate(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00167         bool generateWithResult(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) override;
00168         VarType getReturnType(const std::string &funcName) const override;
00169     };
00170
00178     class CoGenVisitor : public ASTVisitor {
00179     private:
00180         std::unordered_map<std::string, std::string> compileTimeConstants;
00181         std::unordered_set<std::string> usedRealConstants;
00182         int realConstantCounter = 0;
00183         std::unordered_map<std::string, std::string> currentParamTypes;
00184         std::set<std::string> usedModules;
00185         std::vector<std::string> objectDeps;
00186         bool isUnit = false;
00187         std::unordered_map<std::string, std::string> externalFuncs;
00188         std::unordered_set<std::string> importedUnitNames;
00189         std::unordered_map<std::string, ArrayInfo> importedArrayInfo;
00190         std::unordered_map<std::string, VarType> importedVarTypes;
00191         std::vector<std::string> globalArrays;
00192         std::unordered_map<std::string, std::vector<std::string> > functionScopedArrays;
00193         std::vector<std::string> scopeHierarchy;
00194         struct Scope {
00195             std::string name;
00196             std::vector<ProcDeclNode * > nestedProcs;
00197             std::vector<FuncDeclNode * > nestedFuncs;
00198         };
00199         std::vector<Scope> scopeStack;
00200         std::unordered_map<std::string, std::string> typeAliases;
00201         std::map<std::string, std::pair<std::string, std::string> > constInitialValues;
00202         std::map<std::string, size_t> bufferStringVars;
00203         std::map<std::string, std::string> currentParamLocations;
00204         std::vector<std::pair<ProcDeclNode *, std::vector<std::string> > > deferredProcs;
00205         std::vector<std::string> allTempPtrs;
00206         std::vector<bool> tempPtrInUse;
00207

```

```

00208     std::string generateRealConstantName() { return "real_const_" +
std::to_string(realConstantCounter++); }
00209
00210     bool isRealNumber(const std::string &s) const {
00211         if (s.empty())
00212             return false;
00213         bool hasDot = false, hasExp = false;
00214         for (char c : s) {
00215             if (c == '.')
00216                 hasDot = true;
00217             if (c == 'e' || c == 'E')
00218                 hasExp = true;
00219         }
00220         if (!hasDot && !hasExp)
00221             return false;
00222         char *end = nullptr;
00223         std::strtod(s.c_str(), &end);
00224         return end && *end == '\0';
00225     }
00226
00227     std::string mangleWithScope(const std::string &baseName, const std::vector<std::string>
&scopePath) const {
00228         if (scopePath.empty() || scopePath.size() <= 1) {
00229             return baseName;
00230         }
00231         std::string m;
00232         for (size_t i = 1; i < scopePath.size(); ++i) {
00233             if (i > 1)
00234                 m += "_";
00235             m += scopePath[i];
00236         }
00237         m += "_" + baseName;
00238         return m;
00239     }
00240
00241     std::string mangleVariableName(const std::string &varName) const {
00242         return mangleWithScope(varName, scopeHierarchy);
00243     }
00244
00245     std::string getCurrentScopeName() const {
00246         if (scopeHierarchy.empty() || scopeHierarchy.back() == "__global__")
00247             return "";
00248
00249         std::string currentScope = scopeHierarchy.back();
00250         auto &funcs = generatingDeferredCode ? deferredFuncs : std::vector<std::pair<FuncDeclNode
*, std::vector<std::string>>>();
00251         bool isFunc = false;
00252         for (const auto &df : deferredFuncs) {
00253             if (df.first->name == currentScope) {
00254                 isFunc = true;
00255                 break;
00256             }
00257         }
00258
00259         std::vector<std::string> parentScope(scopeHierarchy.begin(), scopeHierarchy.end() - 1);
00260         return (isFunc ? "FUNC_" : "PROC_") + mangleWithScope(currentScope, parentScope);
00261     }
00262
00263     struct FuncInfo {
00264         std::vector<VarType> paramTypes;
00265         VarType returnType = VarType::UNKNOWN;
00266     };
00267
00268     std::unordered_map<std::string, FuncInfo> funcSignatures;
00269     std::unordered_map<std::string, VarType> varTypes;
00270     std::unordered_map<int, VarType> slotToType;
00271
00272     bool needsEmptyString = false;
00273     std::string funcLabel(const char *prefix, const std::string &name) { return
std::string(prefix) + name; }
00274
00275     VarType getVarType(const std::string &name) const {
00276         std::string mangledName = findMangledName(name);
00277         auto slotIt = varSlot.find(mangledName);
00278         if (slotIt != varSlot.end()) {
00279             auto typeIt = slotToType.find(slotIt->second);
00280             if (typeIt != slotToType.end())
00281                 return typeIt->second;
00282         }
00283         auto it = varTypes.find(name);
00284         return it != varTypes.end() ? it->second : VarType::UNKNOWN;
00285     }
00286
00287     bool isRealConstSymbol(const std::string &s) const { return s.rfind("real_const_", 0) == 0; }
00288
00289     std::string coerceToIntImmediate(const std::string &v) {
00290         if (isFloatLiteral(v)) {

```

```

00292         long long x = static_cast<long long>(std::stod(v));
00293         return std::to_string(x);
00294     }
00295     if (isRealConstSymbol(v)) {
00296         auto it = realConstants.find(v);
00297         if (it != realConstants.end()) {
00298             long long x = static_cast<long long>(std::stod(it->second));
00299             return std::to_string(x);
00300         }
00301     }
00302     return v;
00303 }
00304
00305 void setVarType(const std::string &name, VarType t) { varTypes[name] = t; }
00306 void setSlotType(int slot, VarType t) { slotToType[slot] = t; }
00307 bool isStringVar(const std::string &name) const {
00308     auto t = getVarType(name);
00309     return t == VarType::STRING || t == VarType::PTR;
00310 }
00311 std::string emptyString() {
00312     needsEmptyString = true;
00313     return "empty_str";
00314 }
00315 int nextSpillReg = 0;
00316 std::vector<std::string> registers = {"rax", "rbx", "rcx", "rdx", "rsi", "rdi", "r8", "r9",
"r10", "r11", "r12", "r13", "r14", "r15"};
00317 const std::vector<std::string> ptrRegisters = {
00318     "arg0", "arg1", "arg2", "arg3", "arg4", "arg5", "arg6", "arg7", "arg8", "arg9"};
00319 std::vector<std::string> floatRegisters;
00320 std::vector<bool> floatRegInUse;
00321 int nextFloatRegIndex = 0;
00322
00323 void initializeFloatRegisters() {
00324     floatRegisters = {"xmm0", "xmm1", "xmm2", "xmm3", "xmm4", "xmm5", "xmm6", "xmm7"};
00325     floatRegInUse.resize(floatRegisters.size(), false);
00326 }
00327
00328 std::vector<bool> regInUse;
00329 std::vector<bool> ptrRegInUse;
00330
00331 std::vector<std::pair<FuncDeclNode *, std::vector<std::string>>> deferredFuncs;
00332 std::string currentFunctionName;
00333 std::string currentFunctionReturnSlot;
00334 std::vector<std::string> currentFuncLocalSlots;
00335 bool functionSetReturn = false;
00336 int nextSlot = 0;
00337 int nextTemp = 0;
00338 int labelCounter = 0;
00339
00340 std::unordered_map<std::string, int> varSlot;
00341 std::vector<std::string> prolog;
00342 std::vector<std::string> instructions;
00343 std::vector<std::string> evalStack;
00344 std::vector<std::pair<std::string, std::string>> stringLiterals;
00345 std::unordered_set<std::string> usedStrings;
00346 std::unordered_map<int, std::string> slotToName;
00347 std::unordered_map<std::string, std::vector<std::string>> recordsToFreeInScope;
00348 std::unordered_map<std::string, std::vector<std::string>> tempPtrByScope;
00349 std::unordered_map<std::string, bool> declaredProcs;
00350 std::unordered_map<std::string, bool> declaredFuncs;
00351 bool generatingDeferredCode = false;
00352
00353 void addTempPtr(const std::string &v) {
00354     tempPtrByScope[getCurrentScopeName()].push_back(v);
00355 }
00356
00357 struct ValueLocation {
00358     enum Type {
00359         REGISTER,
00360         MEMORY,
00361         IMMEDIATE
00362     } type;
00363     std::string location;
00364 };
00365 std::unordered_map<std::string, ValueLocation> valueLocations;
00366
00367 static bool endsWithColon(const std::string &s) { return !s.empty() && s.back() == ':'; }
00368 bool isRegisterSlot(int slot) const { return slot < -1; }
00369 bool isTempVar(const std::string &name) const { return name.find("__t") != std::string::npos; }
00370 }
00371
00372 std::string slotVar(int slot) const {
00373     auto it = slotToName.find(slot);
00374     if (it != slotToName.end())
00375         return it->second;
00376     if (slot < -1 && slot >= -(int)registers.size()) {
00377         int regIndex = -2 - slot;

```

```

00378         if (regIndex >= 0 && regIndex < (int)registers.size())
00379             return registers[regIndex];
00380     }
00381     if (slot <= -100) {
00382         int regIndex = -100 - slot;
00383         if (regIndex >= 0 && regIndex < (int)floatRegisters.size())
00384             return floatRegisters[regIndex];
00385     }
00386     return "v" + std::to_string(slot);
00387 }
00388
00389 std::string allocReg() {
00390     for (size_t i = 1; i < regInUse.size(); ++i)
00391         if (!regInUse[i]) {
00392             regInUse[i] = true;
00393             return registers[i];
00394         }
00395     std::string name = "_ir" + std::to_string(nextSpillReg++);
00396     registers.push_back(name);
00397     regInUse.push_back(true);
00398     return name;
00399 }
00400
00401 std::string allocPtrReg() {
00402     for (size_t i = 0; i < ptrRegInUse.size(); ++i)
00403         if (!ptrRegInUse[i]) {
00404             ptrRegInUse[i] = true;
00405             return ptrRegisters[i];
00406         }
00407
00408     size_t i = ptrRegisters.size() - 1;
00409     ptrRegInUse[i] = true;
00410     return ptrRegisters[i];
00411 }
00412
00413 std::string allocFloatReg() {
00414     for (size_t i = 0; i < floatRegInUse.size(); ++i) {
00415         if (!floatRegInUse[i]) {
00416             floatRegInUse[i] = true;
00417             return floatRegisters[i];
00418         }
00419     }
00420     expandFloatRegisters();
00421     size_t newIndex = floatRegisters.size() - 1;
00422     floatRegInUse[newIndex] = true;
00423     return floatRegisters[newIndex];
00424 }
00425
00426 void expandFloatRegisters() {
00427     int currentSize = floatRegisters.size();
00428     floatRegisters.push_back("xmm" + std::to_string(currentSize));
00429     floatRegInUse.push_back(false);
00430 }
00431
00432 void freePtrReg(const std::string &reg) {
00433     for (size_t i = 0; i < ptrRegisters.size(); ++i)
00434         if (ptrRegisters[i] == reg) {
00435             ptrRegInUse[i] = false;
00436             return;
00437         }
00438 }
00439
00440 void freeReg(const std::string &reg) {
00441     for (size_t i = 0; i < registers.size(); ++i)
00442         if (registers[i] == reg) {
00443             regInUse[i] = false;
00444             return;
00445         }
00446     freePtrReg(reg);
00447     releaseTempPtr(reg);
00448 }
00449
00450 void freeFloatReg(const std::string &reg) {
00451     for (size_t i = 0; i < floatRegisters.size(); ++i)
00452         if (floatRegisters[i] == reg) {
00453             floatRegInUse[i] = false;
00454             return;
00455         }
00456 }
00457
00458 bool isFloatReg(const std::string &name) const { return std::find(floatRegisters.begin(),
floatRegisters.end(), name) != floatRegisters.end(); }
00459
00460 bool isPtrReg(const std::string &name) const { return std::find(ptrRegisters.begin(),
ptrRegisters.end(), name) != ptrRegisters.end(); }
00461
00462 bool isReg(const std::string &name) const {
00463     return std::find(registers.begin(), registers.end(), name) != registers.end() ||
std::find(floatRegisters.begin(), floatRegisters.end(), name) != floatRegisters.end() ||
isPtrReg(name) || isTempPtr(name);
00464 }
00465
00466

```

```

00461     int newSlotFor(const std::string &name) {
00462         auto it = varSlot.find(name);
00463         if (it != varSlot.end())
00464             return it->second;
00465         int slot = nextSlot++;
00466         varSlot[name] = slot;
00467         if (!name.empty() && name.find("__t") == std::string::npos)
00468             slotToName[slot] = name;
00469         return slot;
00470     }
00471
00472     std::string newTemp() { return slotVar(newSlotFor("__t" + std::to_string(nextTemp++))); }
00473     std::string newLabel(const std::string &prefix = "L") {
00474         return prefix + "_" + std::to_string(labelCounter++);
00475     }
00476
00477     void freeTempPtr(const std::string &ptrName) {
00478         for (size_t i = 0; i < allTempPtrs.size(); ++i) {
00479             if (allTempPtrs[i] == ptrName) {
00480                 if (tempPtrInUse[i]) {
00481                     emit1("free", ptrName);
00482                     tempPtrInUse[i] = false;
00483                 }
00484                 return;
00485             }
00486         }
00487     }
00488
00489     void releaseTempPtr(const std::string &ptrName) {
00490         for (size_t i = 0; i < allTempPtrs.size(); ++i) {
00491             if (allTempPtrs[i] == ptrName) {
00492                 tempPtrInUse[i] = false;
00493                 return;
00494             }
00495         }
00496     }
00497
00498     bool isTempPtr(const std::string &name) const {
00499         return std::find(allTempPtrs.begin(), allTempPtrs.end(), name) != allTempPtrs.end();
00500     }
00501
00502     std::string allocTempPtr(const std::string &forScope = "") {
00503         std::string scope = forScope.empty() ? getCurrentScopeName() : forScope;
00504         if (scope.empty()) {
00505             scope = "";
00506         }
00507
00508         for (size_t i = 0; i < allTempPtrs.size(); ++i) {
00509             if (!tempPtrInUse[i]) {
00510                 tempPtrInUse[i] = true;
00511                 std::string tempName = allTempPtrs[i];
00512                 tempPtrByScope[scope].push_back(tempName);
00513                 return tempName;
00514             }
00515         }
00516
00517         std::string tempName = "_tmpptr" + std::to_string(nextTemp++);
00518         allTempPtrs.push_back(tempName);
00519         tempPtrInUse.push_back(true);
00520
00521         tempPtrByScope[scope].push_back(tempName);
00522         return tempName;
00523     }
00524
00525     void emit(const std::string &s) {
00526         instructions.push_back(s);
00527     }
00528     void emitLabel(const std::string &label) { instructions.push_back(label + ":"); }
00529     void emit1(const std::string &op, const std::string &a) { emit(op + " " + a); }
00530     void emit2(const std::string &op, const std::string &a, const std::string &b) {
00531         std::string A = a;
00532         std::string B = b;
00533         if (isRealNumber(B) && B.rfind("real_const_", 0) != 0)
00534             B = ensureFloatConstSymbol(B);
00535         emit(op + " " + A + ", " + B);
00536     }
00537     void emit3(const std::string &op, const std::string &a, const std::string &b, const
std::string &c) {
00538         instructions.push_back(op + " " + a + ", " + b + ", " + c);
00539         if (op == "alloc") {
00540             allocatedPtrs.insert(a);
00541         }
00542     }
00543     void emit4(const std::string &op, const std::string &a, const std::string &b, const
std::string &c, const std::string &d) { emit(op + " " + a + ", " + b + ", " + c + ", " + d); }
00544
00545     std::string escapeStringForMxvm(const std::string &raw) const {

```



```

00546         std::ostringstream o;
00547         o << "\"";
00548         for (char c : raw) {
00549             if (c == '\\')
00550                 o << "\\\\";
00551             else if (c == '\"')
00552                 o << "\\\"";
00553             else if (c == '\n')
00554                 o << "\\n";
00555             else if (c == '\t')
00556                 o << "\\t";
00557             else
00558                 o << c;
00559         }
00560         o << "\"";
00561         return o.str();
00562     }
00563
00564     std::string ensureFloatConstSymbol(const std::string &value) {
00565         if (!isRealNumber(value))
00566             return value;
00567         if (value.rfind("real_const_", 0) == 0)
00568             return value;
00569         for (const auto &p : realConstants)
00570             if (p.second == value)
00571                 return p.first;
00572         std::string n = generateRealConstantName();
00573         realConstants[n] = value;
00574         usedRealConstants.insert(n);
00575         return n;
00576     }
00577
00578     std::string internString(const std::string &val) {
00579         for (const auto &p : stringLiterals)
00580             if (p.first == val)
00581                 return p.second;
00582         std::string key = "str_" + std::to_string(stringLiterals.size());
00583         stringLiterals.push_back({val, key});
00584         return key;
00585     }
00586
00587     std::string convertCharLiteral(const std::string &c) {
00588         if (c.length() == 3 && c[0] == '\\' && c[2] == '\\')
00589             return std::to_string((int)c[1]);
00590         if (c.length() == 4 && c[0] == '\\' && c[1] == '\\\' && c[3] == '\\') {
00591             char e = c[2];
00592             int v = 0;
00593             switch (e) {
00594                 case 'n':
00595                     v = 10;
00596                     break;
00597                 case 't':
00598                     v = 9;
00599                     break;
00600                 case 'r':
00601                     v = 13;
00602                     break;
00603                 case '\\':
00604                     v = 92;
00605                     break;
00606                 case '\"':
00607                     v = 39;
00608                     break;
00609                 case '0':
00610                     v = 0;
00611                     break;
00612                 default:
00613                     v = (int)e;
00614                     break;
00615             }
00616             return std::to_string(v);
00617         }
00618         return c;
00619     }
00620
00621     void pushValue(const std::string &v) {
00622         evalStack.push_back(v);
00623         if (isReg(v))
00624             recordLocation(v, {ValueLocation::REGISTER, v});
00625     }
00626
00627     std::string popValue() {
00628         if (evalStack.empty())
00629             throw std::runtime_error("Evaluation stack underflow");
00630         std::string v = evalStack.back();
00631         evalStack.pop_back();
00632         return v;
00633     }

```

```

00633
00634     std::string eval(ASTNode *n) {
00635         if (!n)
00636             return "0";
00637         if (auto num = dynamic_cast<NumberNode *>(n)) {
00638             if (!num->isReal && !isRealNumber(num->value))
00639                 return num->value;
00640         }
00641         n->accept(*this);
00642         if (evalStack.empty())
00643             throw std::runtime_error("Expression produced no value");
00644         return popValue();
00645     }
00646
00647 void recordLocation(const std::string &var, ValueLocation loc) { valueLocations[var] = loc; }
00648
00649 void pushTri(const char *op, const std::string &a, const std::string &b) {
00650     bool isFloatOp =
00651         isFloatReg(a) || isFloatReg(b) ||
00652         isRealNumber(a) || isRealNumber(b) ||
00653         a.find("real_const_") != std::string::npos ||
00654         b.find("real_const_") != std::string::npos;
00655
00656     std::string leftOp = a;
00657     std::string rightOp = b;
00658
00659     bool mustCopy = isParmReg(leftOp);
00660     if (isReg(leftOp) && !mustCopy) {
00661         emit2(op, leftOp, rightOp);
00662         pushValue(leftOp);
00663         if (isReg(b) && !isParmReg(b))
00664             freeReg(b);
00665         if (rightOp != b)
00666             freeReg(rightOp);
00667     } else {
00668         std::string result = isFloatOp ? allocFloatReg() : allocReg();
00669         emit2("mov", result, leftOp);
00670         emit2(op, result, rightOp);
00671         pushValue(result);
00672         if (isReg(b) && !isParmReg(b))
00673             freeReg(b);
00674         if (leftOp != a)
00675             freeReg(leftOp);
00676         if (rightOp != b)
00677             freeReg(rightOp);
00678     }
00679 }
00680
00681 void pushCmpResult(const std::string &a, const std::string &b, const char *jop) {
00682     std::string t = allocReg();
00683     std::string L1 = newLabel("CMP_TRUE");
00684     std::string L2 = newLabel("CMP_END");
00685     emit2("cmp", a, b);
00686     emit1(jop, L1);
00687     emit2("mov", t, "0");
00688     emit1("jmp", L2);
00689     emitLabel(L1);
00690     emit2("mov", t, "1");
00691     emitLabel(L2);
00692     pushValue(t);
00693     if (a == b) {
00694         if (isReg(a) && !isParmReg(a))
00695             freeReg(a);
00696     } else {
00697         if (isReg(a) && !isParmReg(a))
00698             freeReg(a);
00699         if (isReg(b) && !isParmReg(b))
00700             freeReg(b);
00701     }
00702 }
00703
00704 void pushFloatCmpResult(const std::string &aIn, const std::string &bIn, const char *jop) {
00705     std::string a = aIn, b = bIn;
00706     if (isRealNumber(a) && a.find("real_const_") != 0) {
00707         std::string n = generateRealConstantName();
00708         realConstants[n] = a;
00709         usedRealConstants.insert(n);
00710         a = n;
00711     }
00712     if (isRealNumber(b) && b.find("real_const_") != 0) {
00713         std::string n = generateRealConstantName();
00714         realConstants[n] = b;
00715         usedRealConstants.insert(n);
00716         b = n;
00717     }
00718     std::string t = allocReg();
00719     std::string L1 = newLabel("CMP_TRUE");

```

```

00720         std::string L2 = newLabel("CMP_END");
00721         emit2("fcmp", a, b);
00722         emit1("jop", L1);
00723         emit2("mov", t, "0");
00724         emit1("jmp", L2);
00725         emitLabel(L1);
00726         emit2("mov", t, "1");
00727         emitLabel(L2);
00728         pushValue(t);
00729         if (isFloatReg(a) && a != aIn && !isParmReg(aIn))
00730             freeFloatReg(a);
00731         if (isFloatReg(b) && b != bIn && !isParmReg(bIn))
00732             freeFloatReg(b);
00733     }
00734
00735 void pushLogicalAnd(const std::string &a, const std::string &b) {
00736     std::string t = allocReg();
00737     std::string L0 = newLabel("AND_ZERO");
00738     std::string L1 = newLabel("AND_END");
00739     emit2("cmp", a, "0");
00740     emit1("je", L0);
00741     emit2("cmp", b, "0");
00742     emit1("je", L0);
00743     emit2("mov", t, "1");
00744     emit1("jmp", L1);
00745     emitLabel(L0);
00746     emit2("mov", t, "0");
00747     emitLabel(L1);
00748     pushValue(t);
00749     if (isReg(a) && !isParmReg(a))
00750         freeReg(a);
00751     if (isReg(b) && !isParmReg(b))
00752         freeReg(b);
00753 }
00754
00755 void pushLogicalOr(const std::string &a, const std::string &b) {
00756     std::string t = allocReg();
00757     std::string L1 = newLabel("OR_ONE");
00758     std::string L2 = newLabel("OR_END");
00759     emit2("cmp", a, "0");
00760     emit1("jne", L1);
00761     emit2("cmp", b, "0");
00762     emit1("jne", L1);
00763     emit2("mov", t, "0");
00764     emit1("jmp", L2);
00765     emitLabel(L1);
00766     emit2("mov", t, "1");
00767     emitLabel(L2);
00768     pushValue(t);
00769     if (isReg(a) && !isParmReg(a))
00770         freeReg(a);
00771     if (isReg(b) && !isParmReg(b))
00772         freeReg(b);
00773 }
00774
00775 bool isParmReg(const std::string &name) const {
00776     for (size_t i = 1; i <= 6 && i < registers.size(); ++i)
00777         if (registers[i] == name)
00778             return true;
00779     for (size_t i = 0; i < ptrRegisters.size(); ++i)
00780         if (ptrRegisters[i] == name)
00781             return true;
00782     return false;
00783 }
00784
00785 void allocateRecordFieldArrays(const std::string &recordVarName, const std::string
&recordTypeName) {
00786     auto recTypeIt = recordTypes.find(recordTypeName);
00787     if (recTypeIt == recordTypes.end())
00788         return;
00789     auto &recInfo = recTypeIt->second;
00790     std::string currentScope = getCurrentScopeName();
00791
00792     for (const auto &field : recInfo.fields) {
00793         if (!field.isArray)
00794             continue;
00795
00796         std::string fieldArrayName = recordVarName + "_" + field.name;
00797
00798         int slot = newSlotFor(fieldArrayName);
00799         setSlotType(slot, VarType::PTR);
00800         updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00801
00802         emit3("alloc",
00803             slotVar(slot),
00804             std::to_string(field.arrayInfo.elementSize),
00805             std::to_string(field.arrayInfo.size));

```

```

00806
00807         std::string basePtr = ensurePtrBase(recordVarName);
00808         emit4("store", slotVar(slot), basePtr, std::to_string(field.offset), "1");
00809
00810         if (currentScope.empty())
00811             globalArrays.push_back(slotVar(slot));
00812         else
00813             functionScopedArrays[currentScope].push_back(slotVar(slot));
00814     }
00815 }
00816
00817 ArrayInfo buildArrayInfoFromNode(ArrayTypeNode *atn) {
00818     auto lc = [](std::string s) { std::transform(s.begin(), s.end(), s.begin(),
00819         [](unsigned char c){ return std::tolower(c); }); return s; };
00820     auto resolveTypeName = [&](std::string t) {
00821         t = lc(t);
00822         std::unordered_set<std::string> seen;
00823         while (typeAliases.count(t) && !seen.count(t)) {
00824             seen.insert(t);
00825             t = lc(typeAliases.at(t));
00826         }
00827         return t;
00828     };
00829
00830     ArrayInfo info{};
00831
00832     // Dynamic array: no bounds specified
00833     if (!atn->lowerBound && !atn->upperBound) {
00834         info.isDynamic = true;
00835         info.lowerBound = 0;
00836         info.upperBound = -1;
00837         info.size = 0;
00838     } else {
00839         info.lowerBound = std::stoi(evaluateConstantExpression(atn->lowerBound.get()));
00840         info.upperBound = std::stoi(evaluateConstantExpression(atn->upperBound.get()));
00841         info.size = info.upperBound - info.lowerBound + 1;
00842     }
00843
00844     if (auto *childArr = dynamic_cast<ArrayTypeNode *>(atn->elementType.get())) {
00845         info.elementType = "array";
00846         info.elementIsArray = true;
00847         info.elementArray = std::make_unique<ArrayInfo>(buildArrayInfoFromNode(childArr));
00848         info.elementSize = info.elementArray->size * info.elementArray->elementSize;
00849     } else if (auto *childSimple = dynamic_cast<SimpleTypeNode *>(atn->elementType.get())) {
00850         info.elementType = resolveTypeName(childSimple->typeName);
00851         info.elementSize = getArrayElementSize(info.elementType);
00852     } else if (dynamic_cast<RecordTypeNode *>(atn->elementType.get())) {
00853         info.elementType = "record";
00854         info.elementSize = 8;
00855     } else if (auto *pt = dynamic_cast<PointerTypeNode *>(atn->elementType.get())) {
00856         info.elementType = "^" + pt->baseTypeName;
00857         info.elementSize = 8;
00858     } else {
00859         info.elementType = "integer";
00860         info.elementSize = 8;
00861     }
00862     return info;
00863 }
00864
00865 int getArrayElementSize(const std::string &tIn) {
00866     auto t = resolveTypeName(lc(tIn));
00867     if (t == "integer" || t == "boolean")
00868         return 8;
00869     if (t == "real")
00870         return 8;
00871     if (t == "char")
00872         return 8;
00873     if (t == "string" || t == "ptr")
00874         return 8;
00875     auto it = recordTypes.find(t);
00876     if (it != recordTypes.end())
00877         return it->second.size;
00878     return 8;
00879 }
00880
00881 public:
00882     friend class IOFunctionHandler;
00883     friend class MathFunctionHandler;
00884     friend class StdFunctionHandler;
00885     friend class SDLFunctionHandler;
00886     friend class StringFunctionHandler;
00887     friend class FileFunctionHandler;
00888
00889     BuiltinFunctionRegistry builtinRegistry;
00890
00891     void registerExternalFunc(const std::string &funcName, const std::string &unitName) {
00892         externalFuncs[funcName] = unitName;
00893     }

```

```

00905         importedUnitNames.insert(unitName);
00906     }
00907
00909     [[nodiscard]] bool isImportedUnit(const std::string &name) const {
00910         return importedUnitNames.count(name) > 0;
00911     }
00912
00914     void registerImportedConst(const std::string &unitName, ConstDeclNode &node) {
00915         importedUnitNames.insert(unitName);
00916         for (const auto &assignment : node.assignments) {
00917             if (auto *strNode = dynamic_cast<StringNode *>(assignment->value.get())) {
00918                 std::string sym = internString(strNode->value);
00919                 std::string qualifiedName = unitName + "." + assignment->identifier;
00920                 compileTimeConstants[qualifiedName] = sym;
00921                 compileTimeConstants[assignment->identifier] = sym;
00922                 continue;
00923             }
00924             try {
00925                 std::string literalValue = evaluateConstantExpression(assignment->value.get());
00926                 bool isFloat = isRealNumber(literalValue);
00927                 if (isFloat)
00928                     literalValue = ensureFloatConstSymbol(literalValue);
00929                 std::string qualifiedName = unitName + "." + assignment->identifier;
00930                 std::string val = isFloat ? realConstants[literalValue] : literalValue;
00931                 compileTimeConstants[qualifiedName] = val;
00932                 compileTimeConstants[assignment->identifier] = val;
00933             } catch (...) {
00934                 // Non-constant expression; skip
00935             }
00936         }
00937     }
00938
00940     void registerImportedVar(const std::string &unitName, VarDeclNode &node) {
00941         importedUnitNames.insert(unitName);
00942         for (const auto &varName : node.identifiers) {
00943             std::string qualifiedName = unitName + "." + varName;
00944             if (std::holds_alternative<std::unique_ptr<ASTNode>(node.type)) {
00945                 auto &typeNode = std::get<std::unique_ptr<ASTNode>(node.type);
00946                 if (auto *arrayTypeNode = dynamic_cast<ArrayTypeNode *>(typeNode.get())) {
00947                     ArrayInfo info = buildArrayInfoFromNode(arrayTypeNode);
00948                     importedArrayInfo[qualifiedName] = info;
00949                     importedVarTypes[qualifiedName] = VarType::PTR;
00950                     arrayInfo[qualifiedName] = std::move(info);
00951                     setVarType(qualifiedName, VarType::PTR);
00952                     continue;
00953                 }
00954             }
00955             std::string typeName;
00956             if (std::holds_alternative<std::string>(node.type))
00957                 typeName = std::get<std::string>(node.type);
00958             VarType vt = getTypeFromString(typeName);
00959             importedVarTypes[qualifiedName] = vt;
00960             setVarType(qualifiedName, vt);
00961         }
00962     }
00963
00965     CodeGenVisitor();
00966     virtual ~CodeGenVisitor() = default;
00967
00969     void initializeBuiltinRegistry() {
00970         builtinRegistry.registerHandler(std::make_unique<IOFunctionHandler>());
00971         builtinRegistry.registerHandler(std::make_unique<StdFunctionHandler>());
00972         builtinRegistry.registerHandler(std::make_unique<SDLFunctionHandler>());
00973         builtinRegistry.registerHandler(std::make_unique<StringFunctionHandler>());
00974         builtinRegistry.registerHandler(std::make_unique<FileFunctionHandler>());
00975     }
00976
00977     std::string name = "App";
00978
00980     void emitFree(const std::string &s) {
00981         if (allocatedPtrs.erase(s)) {
00982             emit("free " + s);
00983         }
00984     }
00985
00990     void generate(ASTNode *root) {
00991         if (!root)
00992             return;
00993
00994         instructions.clear();
00995         usedStrings.clear();
00996         prolog.clear();
00997         deferredProcs.clear();
00998         deferredFuncs.clear();
00999         valueLocations.clear();
01000         objectDeps.clear();
01001         isUnit = false;

```

```

01002         varTypes.clear();
01003         globalArrays.clear();
01004         auto savedConstants = compileTimeConstants;
01005         compileTimeConstants.clear();
01006         functionScopedArrays.clear();
01007         scopeStack.clear();
01008         scopeHierarchy.clear();
01009         scopeStack.push_back({"__global__"});
01010         scopeHierarchy.push_back("__global__");
01011         generatingDeferredCode = false;
01012         functionSetReturn = false;
01013         recordTypes.clear();
01014         varRecordType.clear();
01015         pointerBaseType.clear();
01016         typeAliases.clear();
01017         arrayInfo.clear();
01018
01019         // Restore imported unit variable metadata (registered before generate())
01020         arrayInfo.insert(importedArrayInfo.begin(), importedArrayInfo.end());
01021         varTypes.insert(importedVarTypes.begin(), importedVarTypes.end());
01022         compileTimeConstants.insert(savedConstants.begin(), savedConstants.end());
01023
01024         for (size_t i = 0; i < regInUse.size(); ++i)
01025             regInUse[i] = false;
01026         for (size_t i = 0; i < ptrRegInUse.size(); ++i)
01027             ptrRegInUse[i] = false;
01028         for (size_t i = 0; i < floatRegInUse.size(); ++i)
01029             floatRegInUse[i] = false;
01030
01031         if (auto prog = dynamic_cast<ProgramNode *>(root))
01032             name = prog->name;
01033         if (auto unit = dynamic_cast<UnitNode *>(root))
01034             name = unit->name;
01035
01036         root->accept(*this);
01037
01038         if (!isUnit) {
01039             for (const auto &arrayName : globalArrays)
01040                 emit1("free", arrayName);
01041
01042             if (recordsToFreeInScope.count("")) {
01043                 for (const auto &recordVar : recordsToFreeInScope[""]) {
01044                     emit1("free", recordVar);
01045                 }
01046             }
01047
01048             emit("done");
01049         }
01050
01051         generatingDeferredCode = true;
01052
01053         for (size_t i = 0; i < deferredProcs.size(); ++i) {
01054             auto pn = deferredProcs[i].first;
01055             auto oldHierarchy = scopeHierarchy;
01056             scopeHierarchy = deferredProcs[i].second;
01057
01058             std::vector<std::string> parentScope(scopeHierarchy.begin(), scopeHierarchy.end() -
01059 1);
01060             std::string mangledName = mangleWithScope(pn->name, parentScope);
01061
01062             std::string scopeName = "PROC_" + mangledName;
01063
01064             emitLabel(funcLabel("function PROC_", mangledName));
01065             currentFuncLocalSlots.clear();
01066             currentParamLocations.clear();
01067             currentParamTypes.clear();
01068
01069             if (!pn->parameters.empty()) {
01070                 size_t intParamIndex = 1;
01071                 size_t ptrParamIndex = 0;
01072                 size_t floatParamIndex = 0;
01073
01074                 for (auto &p_node : pn->parameters) {
01075                     if (auto *param = dynamic_cast<ParameterNode *>(p_node.get())) {
01076                         const std::string normType = resolveTypeName(lc(param->type));
01077                         const bool isArrayParam = (arrayInfo.find(normType) != arrayInfo.end());
01078
01079                         VarType declType = getTypeFromString(param->type);
01080                         std::unordered_map<std::string, bool> declaredFuncs;
01081
01082                         for (const auto &id : param->identifiers) {
01083                             VarType useType = declType;
01084                             std::string incomingReg;

```

```

01088         if (declType == VarType::DOUBLE) {
01089             if (floatParamIndex < floatRegisters.size())
01090                 incomingReg = floatRegisters[floatParamIndex++];
01091         } else if (declType == VarType::PTR || declType == VarType::STRING ||
01092                 declType == VarType::RECORD || isArrayParam) {
01093             useType = VarType::PTR;
01094             if (ptrParamIndex < ptrRegisters.size())
01095                 incomingReg = ptrRegisters[ptrParamIndex++];
01096         } else {
01097             if (intParamIndex < registers.size())
01098                 incomingReg = registers[intParamIndex++];
01099         }
01100
01101         std::string mangledId = mangleVariableName(id);
01102         int localSlot = newSlotFor(mangledId);
01103         setSlotType(localSlot, useType);
01104         setVarType(id, useType);
01105
01106         if (!incomingReg.empty())
01107             emit2("mov", slotVar(localSlot), incomingReg);
01108         currentFuncLocalSlots.push_back(slotVar(localSlot));
01109
01110         if (isArrayParam) {
01111             arrayInfo[mangledId] = arrayInfo.at(normType);
01112         }
01113
01114         currentParamTypes[id] = normType;
01115     }
01116 }
01117 }
01118 }
01119
01120 for (size_t r = 0; r < regInUse.size(); ++r)
01121     regInUse[r] = false;
01122 for (size_t r = 0; r < ptrRegInUse.size(); ++r)
01123     ptrRegInUse[r] = false;
01124 for (size_t r = 0; r < floatRegInUse.size(); ++r)
01125     floatRegInUse[r] = false;
01126 for (const auto &pair : currentParamLocations) {
01127     for (size_t reg_idx = 0; reg_idx < registers.size(); ++reg_idx) {
01128         if (registers[reg_idx] == pair.second) {
01129             regInUse[reg_idx] = true;
01130             break;
01131         }
01132     }
01133 }
01134
01135 if (pn->block)
01136     pn->block->accept(*this);
01137
01138 emitLabel("PROC_END_" + mangledName);
01139
01140 for (const auto &recVar : recordsToFreeInScope[scopeName]) {
01141     emit1("free", recVar);
01142 }
01143 auto temp1 = tempPtrByScope[scopeName];
01144 for (const auto &tp : temp1) {
01145     if (allocatedPtrs.count(tp) && !escapedTempPtrs.count(tp))
01146         emitFree(tp);
01147 }
01148 escapedTempPtrs.clear();
01149
01150 emit("ret");
01151 scopeHierarchy = oldHierarchy;
01152 }
01153
01154 for (size_t i = 0; i < deferredFuncs.size(); ++i) {
01155     auto fn = deferredFuncs[i].first;
01156     auto oldHierarchy = scopeHierarchy;
01157     scopeHierarchy = deferredFuncs[i].second;
01158
01159     std::vector<std::string> parentScope(scopeHierarchy.begin(), scopeHierarchy.end() -
01160 1);
01161
01162     std::string mangledName = mangleWithScope(fn->name, parentScope);
01163
01164     std::string scopeName = "FUNC_" + mangledName;
01165
01166     emitLabel(funcLabel("function FUNC_", mangledName));
01167     currentFunctionName = fn->name;
01168     currentFunctionReturnSlot.clear();
01169     currentFuncLocalSlots.clear();
01170     currentParamLocations.clear();
01171     currentParamTypes.clear();
01172     functionSetReturn = false;
01173
01174     if (!fn->parameters.empty()) {
01175         int intParamIndex = 1;

```

```

01174         int ptrParamIndex = 0;
01175         int floatParamIndex = 0;
01176         functionSetReturn = false;
01177
01178         for (auto &p_node : fn->parameters) {
01179             if (auto *param = dynamic_cast<ParameterNode *>(p_node.get())) {
01180                 const std::string normType = resolveTypeName(lc(param->type));
01181                 const bool isArrayParam = (arrayInfo.find(normType) != arrayInfo.end());
01182
01183                 for (const auto &id : param->identifiers) {
01184                     VarType paramType = getTypeFromString(param->type);
01185                     std::string incomingReg;
01186
01187                     if (paramType == VarType::STRING || paramType == VarType::RECORD ||
isArrayParam) {
01188                         if (static_cast<size_t>(ptrParamIndex) < ptrRegisters.size())
01189                             incomingReg = ptrRegisters[ptrParamIndex++];
01190                         paramType = VarType::PTR;
01191                     } else if (paramType == VarType::DOUBLE) {
01192                         if (static_cast<size_t>(floatParamIndex) < floatRegisters.size())
01193                             incomingReg = floatRegisters[floatParamIndex++];
01194                     } else {
01195                         if (static_cast<size_t>(intParamIndex) < registers.size())
01196                             incomingReg = registers[intParamIndex++];
01197                     }
01198
01199                     if (!incomingReg.empty()) {
01200                         std::string mangledId = mangleVariableName(id);
01201                         int localSlot = newSlotFor(mangledId);
01202                         setSlotType(localSlot, paramType);
01203                         setVarType(id, paramType);
01204                         emit2("mov", slotVar(localSlot), incomingReg);
01205                         currentFuncLocalSlots.push_back(slotVar(localSlot));
01206
01207                         if (isArrayParam) {
01208                             arrayInfo[mangledId] = arrayInfo.at(normType);
01209                         }
01210                     }
01211                     currentParamTypes[id] = normType;
01212                 }
01213             }
01214         }
01215     }
01216
01217     for (size_t r = 0; r < regInUse.size(); ++r)
01218         regInUse[r] = false;
01219     for (size_t r = 0; r < ptrRegInUse.size(); ++r)
01220         ptrRegInUse[r] = false;
01221     for (size_t r = 0; r < floatRegInUse.size(); ++r)
01222         floatRegInUse[r] = false;
01223     for (const auto &pair : currentParamLocations) {
01224         for (size_t reg_idx = 0; reg_idx < registers.size(); ++reg_idx) {
01225             if (registers[reg_idx] == pair.second) {
01226                 regInUse[reg_idx] = true;
01227                 break;
01228             }
01229         }
01230     }
01231
01232     if (fn->block)
01233         fn->block->accept(*this);
01234
01235     emitLabel("FUNC_END_" + mangledName);
01236
01237     if (functionSetReturn && !currentFunctionReturnSlot.empty()) {
01238         VarType rt = getVarType(fn->name);
01239         if (rt == VarType::STRING || rt == VarType::PTR || rt == VarType::RECORD)
01240             emit2("mov", "arg0", currentFunctionReturnSlot);
01241         else if (rt == VarType::DOUBLE)
01242             emit2("mov", "xmm0", currentFunctionReturnSlot);
01243         else
01244             emit2("mov", "rax", currentFunctionReturnSlot);
01245     } else if (!functionSetReturn) {
01246         VarType rt = getVarType(fn->name);
01247         if (rt == VarType::STRING || rt == VarType::PTR || rt == VarType::RECORD) {
01248             emit2("mov", "arg0", emptyString());
01249         } else if (rt == VarType::DOUBLE) {
01250             std::string zc = generateRealConstantName();
01251             realConstants[zc] = "0.0";
01252             usedRealConstants.insert(zc);
01253             emit2("mov", "xmm0", zc);
01254         } else {
01255             emit2("mov", "rax", "0");
01256         }
01257     }
01258
01259     for (const auto &recVar : recordsToFreeInScope[scopeName]) {

```



```

01260         emit1("free", recVar);
01261     }
01262
01263     auto templ = tempPtrByScope[scopeName];
01264     for (const auto &tp : templ) {
01265         if (allocatedPtrs.count(tp) && !escapedTempPtrs.count(tp))
01266             emitFree(tp);
01267     }
01268     escapedTempPtrs.clear();
01269
01270     emit("ret");
01271     scopeHierarchy = oldHierarchy;
01272 }
01273
01274 generatingDeferredCode = false;
01275 currentFunctionName.clear();
01276 currentParamLocations.clear();
01277 }
01278
01280 void markAllocatedPtr(const std::string &p) { allocatedPtrs.insert(p); }
01281
01287 void emit_invoke(const std::string &funcName, const std::vector<std::string> &params) {
01288     std::string instruction = "invoke " + funcName;
01289     std::vector<std::string> tempRegs;
01290     for (const auto &p : params) {
01291         if (isIntLiteral(p) || isRealNumber(p)) {
01292             std::string r = isRealNumber(p) ? allocFloatReg() : allocReg();
01293             emit2("mov", r, p);
01294             instruction += ", " + r;
01295             tempRegs.push_back(r);
01296         } else {
01297             instruction += ", " + p;
01298         }
01299     }
01300     emit(instruction);
01301     for (const auto &r : tempRegs)
01302         freeReg(r);
01303 }
01304
01305 bool isIntLiteral(const std::string &s) const {
01306     if (s.empty())
01307         return false;
01308     size_t start = (s[0] == '-') ? 1 : 0;
01309     if (start >= s.size())
01310         return false;
01311     for (size_t i = start; i < s.size(); ++i)
01312         if (!std::isdigit(static_cast<unsigned char>(s[i])))
01313             return false;
01314     return true;
01315 }
01316
01324 void writeTo(std::ostream &out) const {
01325     out << (isUnit ? "object " : "program ") << name << " {\n";
01326
01327     // section object (unit/object dependencies)
01328     if (!objectDeps.empty()) {
01329         out << "\tsection object {\n\t\t";
01330         bool first = true;
01331         for (const auto &dep : objectDeps) {
01332             if (!first)
01333                 out << ", ";
01334             out << dep;
01335             first = false;
01336         }
01337         out << "\n\t}\n";
01338     }
01339
01340     out << "\tsection module {\n ";
01341     bool first = true;
01342     for (const auto &mod : usedModules) {
01343         if (!first)
01344             out << ",\n";
01345         out << "\t\t" << mod;
01346         first = false;
01347     }
01348     out << "\n\t}\n";
01349     out << "\tsection data {\n";
01350
01351     for (const auto &reg : registers)
01352         out << "\t\tint " << reg << " = 0\n";
01353
01354     for (size_t i = 0; i < floatRegisters.size(); ++i) {
01355         out << "\t\tfloat " << floatRegisters[i] << " = 0.0\n";
01356     }
01357
01358     {
01359         std::set<std::string> temps;

```

```

01360         for (const auto &p : ptrRegisters)
01361             temps.insert(p);
01362         for (const auto &t : allTempPtrs)
01363             temps.insert(t);
01364         for (const auto &kv : tempPtrByScope)
01365             for (const auto &t : kv.second)
01366                 temps.insert(t);
01367         for (const auto &tempPtr : temps)
01368             out << "\t\tptr " << tempPtr << " = null\n";
01369     }
01370
01371     for (const auto &constant : realConstants)
01372         out << "\t\tfloat " << constant.first << " = " << constant.second << "\n";
01373     if (needsEmptyString)
01374         out << "\t\tstring empty_str = \"\" \n";
01375     for (const auto &pair : stringLiterals)
01376         out << "\t\tstring " << pair.second << " = " << escapeStringForMxvm(pair.first) << "\n";
01377     if (usedStrings.count("fmt_int"))
01378         out << "\t\tstring fmt_int = \"%lld\" \n";
01379     if (usedStrings.count("fmt_str"))
01380         out << "\t\tstring fmt_str = \"%s\" \n";
01381     if (usedStrings.count("fmt_chr"))
01382         out << "\t\tstring fmt_chr = \"%c\" \n";
01383     if (usedStrings.count("fmt_float"))
01384         out << "\t\tstring fmt_float = \"%f\" \n";
01385     if (usedStrings.count("newline"))
01386         out << "\t\tstring newline = \"\\n\" \n";
01387     out << "\t\tstring input_buffer, 256 \n";
01388     for (int i = 0; i < nextSlot; ++i) {
01389         if (!isRegisterSlot(i) && !isTempVar(slotVar(i)) && !isPtrReg(slotVar(i))) {
01390             auto it = slotToType.find(i);
01391             if (it != slotToType.end()) {
01392                 std::string varName = slotVar(i);
01393                 auto bufIt = bufferStringVars.find(varName);
01394                 if (bufIt != bufferStringVars.end())
01395                     out << "\t\tstring " << varName << ", " << bufIt->second << "\n";
01396                 else {
01397                     auto constIt = constInitialValues.find(varName);
01398                     if (constIt != constInitialValues.end())
01399                         out << "\t\t" << constIt->second.first << " " << varName << " = " <<
constIt->second.second << "\n";
01400                     else {
01401                         if (it->second == VarType::STRING)
01402                             out << "\t\tstring " << varName << " = \"\" \n";
01403                         else if (it->second == VarType::PTR)
01404                             out << "\t\tptr " << varName << " = null\n";
01405                         else if (it->second == VarType::CHAR)
01406                             out << "\t\tint " << varName << " = 0\n";
01407                         else if (it->second == VarType::DOUBLE)
01408                             out << "\t\tfloat " << varName << " = 0.0\n";
01409                         else
01410                             out << "\t\tint " << varName << " = 0\n";
01411                     }
01412                 }
01413             } else
01414                 out << "\t\tint " << slotVar(i) << " = 0\n";
01415         }
01416     }
01417     out << "\t}\n";
01418     out << "\tsection code {\n";
01419     if (!isUnit) {
01420         out << "\tstart:\n";
01421         // Call each dependency unit's UNIT_INIT to allocate their arrays
01422         for (const auto &dep : objectDeps) {
01423             out << "\t\tcall " << dep << ".PROC_UNIT_INIT\n";
01424         }
01425         for (auto &s : prolog)
01426             out << "\t\t" << s << "\n";
01427     }
01428     for (auto &s : instructions) {
01429         if (endsWithColon(s))
01430             out << "\t" << s << "\n";
01431         else
01432             out << "\t\t" << s << "\n";
01433     }
01434     out << "\t}\n";
01435     out << "}\n";
01436 }
01437
01441 void visit(ProgramNode &node) override;
01442 void visit(UnitNode &node) override;
01443 void visit(BlockNode &node) override;
01444 void visit(VarDeclNode &node) override;
01445 void visit(ProcCallNode &node) override;
01446 void visit(FuncCallNode &node) override;
01447 void visit(FuncDeclNode &node) override;
01448 void visit(ProcDeclNode &node) override;

```

```

01449     void visit(TypeAliasNode &node) override;
01450     void visit(ParameterNode &node) override;
01451     void visit(CompoundStmtNode &node) override;
01452     void visit(IfStmtNode &node) override;
01453     void visit(WhileStmtNode &node) override;
01454     void visit(ForStmtNode &node) override;
01455     void visit(BinaryOpNode &node) override;
01456     void visit(UnaryOpNode &node) override;
01457     void visit(VariableNode &node) override;
01458     void visit(NumberNode &node) override;
01459     void visit(StringNode &node) override;
01460     void visit(BooleanNode &node) override;
01461     void visit(EmptyStmtNode &node) override;
01462     void visit(ConstDeclNode &node) override;
01463     void visit(RepeatStmtNode &node) override;
01464     void visit(CaseStmtNode &node) override;
01465     void visit(ArrayTypeNode &node) override;
01466     void visit(ArrayDeclarationNode &node) override;
01467     void visit(FieldAccessNode &node) override;
01468     void visit(AssignmentNode &node) override;
01469     void visit(ArrayAssignmentNode &node) override;
01470     void visit(ArrayAccessNode &node) override;
01471     void visit(RecordTypeNode &node) override;
01472     void visit(RecordDeclarationNode &node) override;
01473     void visit(TypeDeclNode &node) override;
01474     void visit(SimpleTypeNode &node) override;
01475     void visit(ArrayTypeDeclarationNode &node) override;
01476     void visit(ExitNode &node) override;
01477     void visit(BreakNode &node) override;
01478     void visit(ContinueNode &node) override;
01479     void visit(NilNode &node) override;
01480     void visit(PointerTypeNode &node) override;
01481     void visit(PointerDerefNode &node) override;
01482     void visit(AddressOfNode &node) override;
01483     void visit(WithStmtNode &node) override;
01484     void visit(GotoStmtNode &node) override;
01485     void visit(LabelStmtNode &node) override;
01486     void visit(SetLiteralNode &node) override;
01487     void visit(SetTypeNode &node) override;
01488     void visit(EnumTypeDeclNode &node) override;
01490
01492     std::unordered_map<std::string, int> enumConstants;
01494     std::unordered_map<std::string, std::vector<std::string> > enumTypes;
01496     std::unordered_set<std::string> setVars;
01498     std::unordered_set<std::string> fileVars;
01500     std::unordered_map<std::string, std::string> fileVarNames;
01501
01502     std::string getTypeString(const VarDeclNode &node) {
01503         if (std::holds_alternative<std::string>(node.type))
01504             return std::get<std::string>(node.type);
01505         if (std::holds_alternative<std::unique_ptr<ASTNode>(node.type)) {
01506             return "record";
01507         }
01508         return "";
01509     }
01510
01511     std::string storageSymbolFor(const std::string &mangled) {
01512         auto it = varSlot.find(mangled);
01513         if (it != varSlot.end())
01514             return slotVar(it->second);
01515         return mangled;
01516     }
01517     bool isRecordTypeName(const std::string &t) {
01518         auto base = resolveTypeName(lc(t));
01519         return recordTypes.find(base) != recordTypes.end();
01520     }
01521
01522     int getSizeByName(const std::string &t) {
01523         auto base = resolveTypeName(lc(t));
01524         if (base == "integer" || base == "boolean")
01525             return 8;
01526         if (base == "real")
01527             return 8;
01528         if (base == "char")
01529             return 8;
01530         if (base == "string" || base == "ptr")
01531             return 8;
01532         auto it = recordTypes.find(base);
01533         if (it != recordTypes.end())
01534             return it->second.size;
01535         return 8;
01536     }
01537
01538     VarType getTypeFromString(const std::string &typeStr) {
01539         auto base = resolveTypeName(lc(typeStr));
01540         if (base == "integer" || base == "boolean")
01541             return VarType::INT;

```

```

01542         if (base == "real")
01543             return VarType::DOUBLE;
01544         if (base == "string")
01545             return VarType::PTR;
01546         if (base == "char")
01547             return VarType::CHAR;
01548         if (base == "pointer")
01549             return VarType::PTR;
01550         if (base == "file" || base == "text")
01551             return VarType::PTR;
01552         if (base.rfind("set of ", 0) == 0)
01553             return VarType::PTR;
01554         if (!base.empty() && base[0] == '^')
01555             return VarType::PTR;
01556         if (isRecordTypeName(base))
01557             return VarType::RECORD;
01558         if (arrayInfo.find(base) != arrayInfo.end())
01559             return VarType::PTR;
01560         return VarType::UNKNOWN;
01561     }
01562
01572     void emitArrayBoundsCheck(const std::string &idxReg, int lower, int upper) {
01573 #ifdef MXVM_BOUNDS_CHECK
01574         std::string L_ok = newLabel("IDX_OK");
01575         std::string L_fail = newLabel("IDX_OOB");
01576         emit2("cmp", idxReg, std::to_string(lower));
01577         emit1("jl", L_fail);
01578         emit2("cmp", idxReg, std::to_string(upper));
01579         emit1("jg", L_fail);
01580         emit1("jmp", L_ok);
01581         emitLabel(L_fail);
01582         emit1("exit", "1");
01583         emitLabel(L_ok);
01584 #endif
01585     }
01586
01596     void emitDynArrayBoundsCheck(const std::string &idxReg, const std::string &lenSym) {
01597 #ifdef MXVM_BOUNDS_CHECK
01598         std::string L_ok = newLabel("IDX_OK");
01599         std::string L_fail = newLabel("IDX_OOB");
01600         emit2("cmp", idxReg, "0");
01601         emit1("jl", L_fail);
01602         std::string tmp = allocReg();
01603         emit2("mov", tmp, lenSym);
01604         emit2("cmp", idxReg, tmp);
01605         freeReg(tmp);
01606         emit1("jge", L_fail);
01607         emit1("jmp", L_ok);
01608         emitLabel(L_fail);
01609         emit1("exit", "1");
01610         emitLabel(L_ok);
01611 #endif
01612     }
01613
01614     VarType getExpressionType(ASTNode *node) {
01615         if (dynamic_cast<NilNode *>(node))
01616             return VarType::PTR;
01617         if (dynamic_cast<AddressOfNode *>(node))
01618             return VarType::PTR;
01619         if (dynamic_cast<NumberNode *>(node)) {
01620             auto n = static_cast<NumberNode *>(node);
01621             return n->isReal ? VarType::DOUBLE : VarType::INT;
01622         }
01623         if (dynamic_cast<StringNode *>(node)) {
01624             return VarType::STRING;
01625         }
01626         if (dynamic_cast<BooleanNode *>(node))
01627             return VarType::BOOL;
01628         if (auto derefNode = dynamic_cast<PointerDerefNode *>(node)) {
01629             if (auto varNode = dynamic_cast<VariableNode *>(derefNode->pointer.get())) {
01630                 return getPointerDerefType(varNode->name);
01631             }
01632             return VarType::INT;
01633         }
01634         if (auto varNode = dynamic_cast<VariableNode *>(node))
01635             return getVarType(varNode->name);
01636         if (auto funcCall = dynamic_cast<FuncCallNode *>(node)) {
01637             std::string fnLower = lc(funcCall->name);
01638             auto handler = builtinRegistry.findHandler(fnLower);
01639             if (handler)
01640                 return handler->getReturnType(fnLower);
01641             auto it = funcSignatures.find(funcCall->name);
01642             if (it != funcSignatures.end())
01643                 return it->second.returnType;
01644         }
01645         if (auto fieldNode = dynamic_cast<FieldAccessNode *>(node)) {
01646             std::string recType = getVarRecordTypeNameFromExpr(fieldNode->recordExpr.get());

```

```

01647         recType = resolveTypeName(lc(recType));
01648         auto it = recordTypes.find(recType);
01649         if (it != recordTypes.end()) {
01650             auto jt = it->second.nameToIndex.find(lc(fieldNode->fieldName));
01651             if (jt != it->second.nameToIndex.end()) {
01652                 const auto &f = it->second.fields[jt->second];
01653                 if (f.isArray) {
01654                     return VarType::PTR;
01655                 }
01656                 return getTypeFromString(f.typeName);
01657             }
01658         }
01659     }
01660     if (auto a = dynamic_cast<ArrayAccessNode *>(node)) {
01661         ArrayInfo *info = getArrayInfoForArrayAccess(a);
01662         if (info) {
01663             if (info->elementIsArray)
01664                 return VarType::PTR;
01665             const std::string &t = info->elementType;
01666             if (t == "integer" || t == "boolean")
01667                 return VarType::INT;
01668             if (t == "real")
01669                 return VarType::DOUBLE;
01670             if (t == "char")
01671                 return VarType::CHAR;
01672             if (t == "string" || t == "ptr" ||
01673                 t == "pointer" ||
01674                 (!t.empty() && t[0] == '^'))
01675                 return VarType::PTR;
01676             if (isRecordTypeName(t))
01677                 return VarType::RECORD;
01678         }
01679     }
01680     if (auto b = dynamic_cast<BinaryOpNode *>(node)) {
01681         VarType lt = getExpressionType(b->left.get());
01682         VarType rt = getExpressionType(b->right.get());
01683         if (b->operator_ == BinaryOpNode::PLUS) {
01684             auto isStrLike = [](VarType v) { return v == VarType::STRING || v == VarType::PTR; };
01685             if (isStrLike(lt) || isStrLike(rt))
01686                 return VarType::PTR;
01687             if (lt == VarType::DOUBLE || rt == VarType::DOUBLE)
01688                 return VarType::DOUBLE;
01689             return lt;
01690         }
01691         return VarType::INT;
01692     }
01693 }
01694
01695 private:
01696 std::string evaluateConstantExpression(ASTNode *node) {
01697     if (!node)
01698         return "";
01699     if (auto numNode = dynamic_cast<NumberNode *>(node)) {
01700         return numNode->value;
01701     }
01702     if (auto varNode = dynamic_cast<VariableNode *>(node)) {
01703         auto it = compileTimeConstants.find(findMangledName(varNode->name));
01704         if (it != compileTimeConstants.end())
01705             return it->second;
01706         it = compileTimeConstants.find(varNode->name);
01707         if (it != compileTimeConstants.end())
01708             return it->second;
01709         throw std::runtime_error("Cannot use non-constant variable '" + varNode->name + "' in
a constant expression.");
01710     }
01711     if (auto binOp = dynamic_cast<BinaryOpNode *>(node)) {
01712         std::string L = evaluateConstantExpression(binOp->left.get());
01713         std::string R = evaluateConstantExpression(binOp->right.get());
01714         if (L.empty() || R.empty())
01715             throw std::runtime_error("Unsupported node type in constant expression.");
01716         bool isFloatOp = isFloatLiteral(L) || isFloatLiteral(R) || binOp->operator_ ==
BinaryOpNode::DIVIDE;
01717         auto toD = [&](const std::string &s) { return std::stod(s); };
01718         auto toI = [&](const std::string &s) {
01719             return isFloatLiteral(s) ? static_cast<long long>(std::stod(s)) : std::stoll(s);
01720         };
01721         if (isFloatOp) {
01722             double result;
01723             switch (binOp->operator_) {
01724             case BinaryOpNode::PLUS:
01725                 result = toD(L) + toD(R);
01726                 break;
01727             case BinaryOpNode::MINUS:

```

```

01731         result = toD(L) - toD(R);
01732         break;
01733     case BinaryOpNode::MULTIPLY:
01734         result = toD(L) * toD(R);
01735         break;
01736     case BinaryOpNode::DIVIDE:
01737         if (toD(R) == 0.0)
01738             throw std::runtime_error("division by zero in constant expression");
01739         result = toD(L) / toD(R);
01740         break;
01741     default:
01742         throw std::runtime_error("Unsupported operator for floats in constant
expression.");
01743     }
01744     return std::to_string(result);
01745 } else {
01746     long long result;
01747     switch (binOp->operator_) {
01748     case BinaryOpNode::PLUS:
01749         result = toI(L) + toI(R);
01750         break;
01751     case BinaryOpNode::MINUS:
01752         result = toI(L) - toI(R);
01753         break;
01754     case BinaryOpNode::MULTIPLY:
01755         result = toI(L) * toI(R);
01756         break;
01757     case BinaryOpNode::DIV:
01758         if (toI(R) == 0)
01759             throw std::runtime_error("division by zero in constant expression");
01760         result = toI(L) / toI(R);
01761         break;
01762     case BinaryOpNode::MOD:
01763         if (toI(R) == 0)
01764             throw std::runtime_error("mod by zero in constant expression");
01765         result = toI(L) % toI(R);
01766         break;
01767     default:
01768         throw std::runtime_error("Unsupported operator for integers in constant
expression.");
01769     }
01770     return std::to_string(result);
01771 }
01772 }
01773 if (auto un = dynamic_cast<UnaryOpNode *>(node)) {
01774     std::string v = evaluateConstantExpression(un->operand.get());
01775     if (v.empty())
01776         throw std::runtime_error("Unsupported node type in constant expression.");
01777     double dv = std::stod(v);
01778     switch (un->operator_) {
01779     case UnaryOpNode::MINUS:
01780         return std::to_string(-dv);
01781     case UnaryOpNode::PLUS:
01782         return v;
01783     default:
01784         throw std::runtime_error("Unsupported unary op in constant expression.");
01785     }
01786 }
01787 throw std::runtime_error("Unsupported node type in constant expression.");
01788 }
01789
01790 public:
01791 void updateDataSectionInitialValue(const std::string &varName, const std::string &type, const
std::string &value) { constInitialValues[varName] = {type, value}; }
01792
01793 private:
01794 std::unordered_set<std::string> allocatedPtrs;
01795 std::unordered_set<std::string> escapedTempPtrs;
01796
01797 std::unordered_map<std::string, std::string> realConstants;
01798
01799 std::unordered_map<std::string, ArrayInfo> arrayInfo;
01800 std::unordered_map<std::string, int> dynArrayLenSlot;
01801
01802 std::string getTypeString(ASTNode *typeNode) {
01803     if (auto simpleType = dynamic_cast<SimpleTypeNode *>(typeNode)) {
01804         return simpleType->typeName;
01805     } else if (dynamic_cast<ArrayTypeNode *>(typeNode)) {
01806         return "array";
01807     } else if (dynamic_cast<RecordTypeNode *>(typeNode)) {
01808         return "record";
01809     }
01810     return "unknown";
01811 }
01812
01813 static inline std::string lc(std::string s) {
01814     std::transform(s.begin(), s.end(), s.begin(),

```

```

01815         [] (unsigned char c) { return std::tolower(c); });
01816     return s;
01817 }
01818
01819 std::string resolveTypeName(std::string t) const {
01820     if (t.empty())
01821         return t;
01822     t = lc(t);
01823     std::unordered_set<std::string> seen;
01824     while (true) {
01825         if (!typeAliases.count(t))
01826             break;
01827         if (seen.count(t))
01828             break;
01829         seen.insert(t);
01830         t = lc(typeAliases.at(t));
01831     }
01832     return t;
01833 }
01834
01835 int getRecordTypeSize(const std::string &typeName) {
01836     auto it = recordTypes.find(typeName);
01837     if (it != recordTypes.end()) {
01838         return it->second.size;
01839     }
01840     return 8;
01841 }
01842
01843 std::string findMangledFuncName(const std::string &name, bool isProc) const {
01844     auto &lookupMap = isProc ? declaredProcs : declaredFuncs;
01845     for (int depth = (int)scopeHierarchy.size() - 1; depth >= 0; --depth) {
01846         std::vector<std::string> tempScope(scopeHierarchy.begin(), scopeHierarchy.begin() +
depth + 1);
01847         std::string candidate = mangleWithScope(name, tempScope);
01848         if (lookupMap.count(candidate)) {
01849             return candidate;
01850         }
01851     }
01852     return name;
01853 }
01854
01855 std::string findMangledName(const std::string &name) const {
01856     if (scopeHierarchy.empty())
01857         return name;
01858     for (int depth = (int)scopeHierarchy.size() - 1; depth >= 0; --depth) {
01859         std::string candidate;
01860         if (depth == 0)
01861             candidate = name;
01862         else {
01863             for (int i = 1; i <= depth; ++i) {
01864                 if (i > 1)
01865                     candidate += "_";
01866                 candidate += scopeHierarchy[i];
01867             }
01868             candidate += "_" + name;
01869         }
01870         auto it = varSlot.find(candidate);
01871         if (it != varSlot.end())
01872             return candidate;
01873     }
01874     return name;
01875 }
01876
01877 std::string findMangledArrayName(const std::string &name) const {
01878     if (scopeHierarchy.empty())
01879         return name;
01880     for (int depth = (int)scopeHierarchy.size() - 1; depth >= 1; --depth) {
01881         std::string candidate;
01882         for (int i = 1; i <= depth; ++i) {
01883             if (i > 1)
01884                 candidate += "_";
01885             candidate += scopeHierarchy[i];
01886         }
01887         candidate += "_" + name;
01888         if (varSlot.count(candidate))
01889             return candidate;
01890     }
01891     return name;
01892 }
01893
01894 bool isIntegerLiteral(const std::string &s) const {
01895     if (s.empty())
01896         return false;
01897     size_t i = (s[0] == '+' || s[0] == '-') ? 1 : 0;
01898     if (i >= s.size())
01899         return false;
01900     for (; i < s.size(); ++i)

```

```

01901         if (!std::isdigit(static_cast<unsigned char>(s[i])))
01902             return false;
01903     return true;
01904 }
01905
01906 bool isFloatLiteral(const std::string &s) const {
01907     if (s.find_first_of(".eE") == std::string::npos)
01908         return false;
01909     char *end = nullptr;
01910     std::strtod(s.c_str(), &end);
01911     return end && *end == '\0';
01912 }
01913
01914 bool tryGetConstNumeric(const std::string &name, std::string &out) {
01915     auto itM = compileTimeConstants.find(findMangledName(name));
01916     if (itM != compileTimeConstants.end() && (isIntegerLiteral(itM->second) ||
isFloatLiteral(itM->second))) {
01917         out = itM->second;
01918         return true;
01919     }
01920     auto it = compileTimeConstants.find(name);
01921     if (it != compileTimeConstants.end() && (isIntegerLiteral(it->second) ||
isFloatLiteral(it->second))) {
01922         out = it->second;
01923         return true;
01924     }
01925     return false;
01926 }
01927
01928 std::string foldNumeric(ASTNode *n) {
01929     if (!n)
01930         return "";
01931     if (auto num = dynamic_cast<NumberNode *>(n))
01932         return num->value;
01933     if (auto var = dynamic_cast<VariableNode *>(n)) {
01934         std::string v;
01935         if (tryGetConstNumeric(var->name, v))
01936             return v;
01937         return "";
01938     }
01939     if (auto bin = dynamic_cast<BinaryOpNode *>(n)) {
01940         std::string L = foldNumeric(bin->left.get());
01941         std::string R = foldNumeric(bin->right.get());
01942         if (L.empty() || R.empty())
01943             return "";
01944         auto toD = [&](const std::string &s) -> double { return std::stod(s); };
01945         auto toI = [&](const std::string &s) -> long long {
01946             return isFloatLiteral(s) ? static_cast<long long>(std::stod(s)) : std::stoll(s);
01947         };
01948         switch (bin->operator_) {
01949             case BinaryOpNode::PLUS:
01950                 return (isFloatLiteral(L) || isFloatLiteral(R)) ? std::to_string(toD(L) + toD(R))
: std::to_string(toI(L) + toI(R));
01951             case BinaryOpNode::MINUS:
01952                 return (isFloatLiteral(L) || isFloatLiteral(R)) ? std::to_string(toD(L) - toD(R))
: std::to_string(toI(L) - toI(R));
01953             case BinaryOpNode::MULTIPLY:
01954                 return (isFloatLiteral(L) || isFloatLiteral(R)) ? std::to_string(toD(L) * toD(R))
: std::to_string(toI(L) * toI(R));
01955             case BinaryOpNode::DIVIDE: {
01956                 double rv = toD(R);
01957                 if (rv == 0.0)
01958                     throw std::runtime_error("division by zero");
01959                 return std::to_string(toD(L) / rv);
01960             }
01961             case BinaryOpNode::DIV: {
01962                 long long ri = toI(R);
01963                 if (ri == 0)
01964                     throw std::runtime_error("division by zero");
01965                 return std::to_string(toI(L) / ri);
01966             }
01967             case BinaryOpNode::MOD: {
01968                 long long ri = toI(R);
01969                 if (ri == 0)
01970                     throw std::runtime_error("mod by zero");
01971                 return std::to_string(toI(L) % ri);
01972             }
01973             default:
01974                 return "";
01975         }
01976     }
01977     return "";
01978 }
01979
01980 struct RecordField {
01981     std::string name;
01982     std::string typeName;
01983 }

```



```

01984         int offset;
01985         int size;
01986         bool isArray = false;
01987         ArrayInfo arrayInfo;
01988
01989         RecordField() = default;
01990         RecordField(const RecordField &other) = default;
01991         RecordField(RecordField &&other) noexcept = default;
01992         RecordField &operator=(const RecordField &other) = default;
01993         RecordField &operator=(RecordField &&other) noexcept = default;
01994     };
01995     struct RecordTypeInfo {
01996         int size = 0;
01997         std::vector<RecordField> fields;
01998         std::unordered_map<std::string, int> nameToIndex;
01999     };
02000
02001     std::unordered_map<std::string, RecordTypeInfo> recordTypes;
02002     std::unordered_map<std::string, std::string> varRecordType;
02003     std::unordered_map<std::string, std::string> pointerBaseType;
02004
02005     std::pair<int, int> getRecordFieldOffsetAndSize(const std::string &recType, const std::string
&field) {
02006         std::string normalizedRecType = resolveTypeName(lc(recType));
02007         std::string normalizedField = lc(field);
02008
02009         auto it = recordTypes.find(normalizedRecType);
02010         if (it == recordTypes.end())
02011             return {0, 8};
02012
02013         auto jt = it->second.nameToIndex.find(normalizedField);
02014         if (jt == it->second.nameToIndex.end())
02015             return {0, 8};
02016
02017         const auto &f = it->second.fields[jt->second];
02018         return {f.offset, f.size};
02019     }
02020
02021     std::string getVarRecordTypeNameFromExpr(ASTNode *expr) {
02022         auto lc = [] (std::string s) {
02023             std::transform(s.begin(), s.end(), s.begin(),
02024                 [](unsigned char c) { return std::tolower(c); });
02025             return s;
02026         };
02027         auto resolveAlias = [&] (std::string t) {
02028             t = lc(t);
02029             std::unordered_set<std::string> seen;
02030             while (typeAliases.count(t) && !seen.count(t)) {
02031                 seen.insert(t);
02032                 t = lc(typeAliases.at(t));
02033             }
02034             return t;
02035         };
02036         if (auto *v = dynamic_cast<VariableNode *>(expr)) {
02037             if (auto it = currentParamTypes.find(v->name); it != currentParamTypes.end()) {
02038                 auto resolved = resolveAlias(it->second);
02039                 if (recordTypes.count(resolved))
02040                     return resolved;
02041             }
02042             std::string mangled = findMangledName(v->name);
02043             if (auto it2 = varRecordType.find(mangled); it2 != varRecordType.end()) {
02044                 auto resolved = resolveAlias(it2->second);
02045                 if (recordTypes.count(resolved))
02046                     return resolved;
02047             }
02048             if (auto it3 = varRecordType.find(v->name); it3 != varRecordType.end()) {
02049                 auto resolved = resolveAlias(it3->second);
02050                 if (recordTypes.count(resolved))
02051                     return resolved;
02052             }
02053             return "";
02054         }
02055         if (auto *arrAccess = dynamic_cast<ArrayAccessNode *>(expr)) {
02056             ArrayInfo *info = getArrayInfoForArrayAccess(arrAccess);
02057             if (info) {
02058                 auto elem = resolveAlias(info->elementType);
02059                 if (recordTypes.count(elem))
02060                     return elem;
02061             }
02062             return "";
02063         }
02064
02065         if (auto *fa = dynamic_cast<FieldAccessNode *>(expr)) {
02066             std::string owner = getVarRecordTypeNameFromExpr(fa->recordExpr.get());
02067             owner = resolveTypeName(lc(owner));
02068             if (owner.empty())
02069                 return "";
02070         }

```

```

02071         auto rit = recordTypes.find(owner);
02072         if (rit == recordTypes.end())
02073             return "";
02074         auto idxIt = rit->second.nameToIndex.find(lc(fa->fieldName));
02075         if (idxIt == rit->second.nameToIndex.end())
02076             return "";
02077
02078         const auto &f = rit->second.fields[idxIt->second];
02079         std::string fieldType = resolveTypeName(lc(f.typeName));
02080         if (recordTypes.count(fieldType)) {
02081             return fieldType;
02082         }
02083         return "";
02084     }
02085
02086     return "";
02087 }
02088
02089 std::string getVarRecordTypeName(const std::string &varName) {
02090
02091     if (auto it = currentParamTypes.find(varName); it != currentParamTypes.end()) {
02092         std::string resolved = resolveTypeName(it->second);
02093         return resolved;
02094     }
02095
02096     std::string mangled = findMangledName(varName);
02097     if (auto it2 = varRecordType.find(mangled); it2 != varRecordType.end()) {
02098         std::string resolved = resolveTypeName(it2->second);
02099         return resolved;
02100     }
02101
02102     if (auto it3 = varRecordType.find(varName); it3 != varRecordType.end()) {
02103         std::string resolved = resolveTypeName(it3->second);
02104         return resolved;
02105     }
02106
02107     return "";
02108 }
02109
02110 bool isArrayTypeName(const std::string &t) const {
02111     auto base = resolveTypeName(lc(t));
02112     return arrayInfo.find(base) != arrayInfo.end();
02113 }
02114
02115 std::string getPointerBaseTypeName(const std::string &varName) const {
02116     std::string mangled = findMangledName(varName);
02117     auto it = pointerBaseType.find(mangled);
02118     if (it != pointerBaseType.end())
02119         return it->second;
02120     it = pointerBaseType.find(varName);
02121     if (it != pointerBaseType.end())
02122         return it->second;
02123     return "";
02124 }
02125
02126 int getPointerElementSize(const std::string &varName) {
02127     std::string base = getPointerBaseTypeName(varName);
02128     if (base.empty())
02129         return 8;
02130     base = resolveTypeName(lc(base));
02131     return getTypeSizeByName(base);
02132 }
02133
02134 VarType getPointerDerefType(const std::string &varName) {
02135     std::string base = getPointerBaseTypeName(varName);
02136     if (base.empty())
02137         return VarType::INT;
02138     return getTypeFromString(base);
02139 }
02140
02141 private:
02142     std::vector<std::string> loopEndLabels;
02143     std::vector<std::string> loopContinueLabels;
02144
02145     std::vector<std::unordered_map<std::string, std::string> > withFieldScopes;
02146     std::unordered_map<std::string, std::string> gotoLabels;
02147
02148     std::string getCurrentEndLabel() const {
02149         if (currentFunctionName.empty())
02150             return "";
02151
02152         std::string mangledName = findMangledFuncName(currentFunctionName, false);
02153         if (declaredFuncs.count(mangledName)) {
02154             return "FUNC_END_" + mangledName;
02155         }
02156     }
02157
02158     mangledName = findMangledFuncName(currentFunctionName, true);

```

```

02160         if (declaredProcs.count(mangledName)) {
02161             return "PROC_END_" + mangledName;
02162         }
02163     }
02164     return "";
02165 }
02166
02167 static inline bool isPtrLike(VarType t) {
02168     return t == VarType::PTR || t == VarType::RECORD || t == VarType::STRING;
02169 }
02170
02171 std::string ensurePtrBase(const std::string &v) {
02172     auto isNamedPtr = [&](const std::string &s) -> bool {
02173         if (isPtrReg(s))
02174             return true;
02175         if (s.rfind("_tmpptr", 0) == 0)
02176             return true;
02177         auto it = varSlot.find(s);
02178         if (it != varSlot.end()) {
02179             auto jt = slotToType.find(it->second);
02180             if (jt != slotToType.end()) {
02181                 auto t = jt->second;
02182                 return t == VarType::PTR || t == VarType::RECORD || t == VarType::STRING;
02183             }
02184         }
02185         return false;
02186     };
02187     if (isNamedPtr(v))
02188         return v;
02189     std::string p = allocTempPtr();
02190     emit2("mov", p, v);
02191     return p;
02192 }
02193
02194 std::string getArrayNameFromBase(ASTNode *base) {
02195     if (auto var = dynamic_cast<VariableNode *>(base)) {
02196         return var->name;
02197     }
02198     if (auto field = dynamic_cast<FieldAccessNode *>(base)) {
02199         return getArrayNameFromBase(field->recordExpr.get()) + "." + field->fieldName;
02200     }
02201     if (auto arr = dynamic_cast<ArrayAccessNode *>(base)) {
02202         return getArrayNameFromBase(arr->base.get());
02203     }
02204 }
02205
02206 throw std::runtime_error("Array base is not a simple variable or field");
02207
02208 ArrayInfo *getArrayInfoForArrayAccess(ArrayAccessNode *arr) {
02209     if (auto var = dynamic_cast<VariableNode *>(arr->base.get())) {
02210         std::string mangled = findMangledName(var->name);
02211         auto it = arrayInfo.find(mangled);
02212         if (it != arrayInfo.end())
02213             return &it->second;
02214         it = arrayInfo.find(var->name);
02215         if (it != arrayInfo.end())
02216             return &it->second;
02217         return nullptr;
02218     }
02219     if (auto field = dynamic_cast<FieldAccessNode *>(arr->base.get())) {
02220         // Cross-unit variable reference: UnitName.varName
02221         if (auto *baseVar = dynamic_cast<VariableNode *>(field->recordExpr.get())) {
02222             if (isImportedUnit(baseVar->name)) {
02223                 std::string qualifiedName = baseVar->name + "." + field->fieldName;
02224                 auto it = arrayInfo.find(qualifiedName);
02225                 if (it != arrayInfo.end())
02226                     return &it->second;
02227                 return nullptr;
02228             }
02229         }
02230         // Record field array access
02231         std::string recTypeName = getVarRecordTypeNameFromExpr(field->recordExpr.get());
02232         recTypeName = resolveTypeName(lc(recTypeName));
02233         auto recTypeInfoIt = recordTypes.find(recTypeName);
02234         if (recTypeInfoIt != recordTypes.end()) {
02235             auto &recInfo = recTypeInfoIt->second;
02236             auto fieldIndexIt = recInfo.nameToIndex.find(lc(field->fieldName));
02237             if (fieldIndexIt != recInfo.nameToIndex.end()) {
02238                 auto &f = recInfo.fields[fieldIndexIt->second];
02239                 if (f.isArray)
02240                     return const_cast<ArrayInfo *>(&f.arrayInfo);
02241             }
02242         }
02243     }
02244     if (auto inner = dynamic_cast<ArrayAccessNode *>(arr->base.get())) {
02245         ArrayInfo *baseInfo = getArrayInfoForArrayAccess(inner);
02246         if (baseInfo && baseInfo->elementIsArray) {

```


Namespaces

- namespace `mxvm`

Enumerations

- enum `Inc` {
`NULL_INC` = 0 , `MOV` , `LOAD` , `STORE` ,
`ADD` , `SUB` , `MUL` , `DIV` ,
`OR` , `AND` , `XOR` , `NOT` ,
`MOD` , `CMP` , `JMP` , `JE` ,
`JNE` , `JL` , `JLE` , `JG` ,
`JGE` , `JZ` , `JNZ` , `JA` ,
`JB` , `PRINT` , `EXIT` , `ALLOC` ,
`FREE` , `GETLINE` , `PUSH` , `POP` ,
`STACK_LOAD` , `STACK_STORE` , `STACK_SUB` , `CALL` ,
`RET` , `STRING_PRINT` , `DONE` , `TO_INT` ,
`TO_FLOAT` , `INVOKE` , `RETURN` , `NEG` ,
`FCMP` , `JAE` , `JBE` , `JC` ,
`JNC` , `JP` , `JNP` , `JO` ,
`JNO` , `JS` , `JNS` , `LEA` ,
`REALLOC` }
- MXVM instruction set opcodes.*
- enum class `mxvm::OperandType` { `mxvm::OP_CONSTANT` , `mxvm::OP_VARIABLE` }
Discriminates whether an operand is a compile-time constant or a variable reference.
- enum class `mxvm::VarType` {
`mxvm::VAR_NULL` = 0 , `mxvm::VAR_INTEGER` , `mxvm::VAR_FLOAT` , `mxvm::VAR_STRING` ,
`mxvm::VAR_POINTER` , `mxvm::VAR_LABEL` , `mxvm::VAR_ARRAY` , `mxvm::VAR_EXTERN` ,
`mxvm::VAR_BYTE` }
- MXVM variable type discriminator.*
- enum class `mxvm::Keywords` {
`mxvm::KEY_PROGRAM` , `mxvm::KEY_OBJECT` , `mxvm::KEY_MODULE` , `mxvm::KEY_DATA` ,
`mxvm::KEY_CODE` , `mxvm::KEY_SECTION` }
- MXVM language keywords.*

Functions

- `std::ostream & operator<< (std::ostream &out, const enum Inc &i)`
Stream insertion operator for `Inc` opcodes.
- `std::ostream & operator<< (std::ostream &out, const mxvm::Instruction &inc)`
Stream insertion operator for `Instruction`.
- `std::ostream & operator<< (std::ostream &out, const mxvm::VarType &type)`
Stream insertion operator for `VarType`.
- `std::ostream & operator<< (std::ostream &out, const mxvm::Keywords &key)`
Stream insertion operator for `Keywords`.

Variables

- `std::vector< std::string > IncType`
String representations of `Inc` opcodes, indexed by enum value.
- `std::vector< std::string > mxvm::keywords {"program", "object", "module", "data", "code", "section"}`
String representations of `Keywords` enum values.

12.13.1 Detailed Description

Instruction set enum, operand/instruction structs, variable types, and Variable/Variable_Value definitions.

Author

Jared Bruni

Definition in file [instruct.hpp](#).

12.13.2 Enumeration Type Documentation

12.13.2.1 Inc

enum [Inc](#)

MXVM instruction set opcodes.

Enumerates all supported virtual machine instructions including data movement, arithmetic, logic, control flow, stack operations, memory management, I/O, and type conversion.

Enumerator

NULL_INC	
MOV	
LOAD	
STORE	
ADD	
SUB	
MUL	
DIV	
OR	
AND	
XOR	
NOT	
MOD	
CMP	
JMP	
JE	
JNE	
JL	
JLE	
JG	
JGE	
JZ	
JNZ	
JA	
JB	
PRINT	
EXIT	
ALLOC	

FREE	
GETLINE	
PUSH	
POP	
STACK_LOAD	
STACK_STORE	
STACK_SUB	
CALL	
RET	
STRING_PRINT	
DONE	
TO_INT	
TO_FLOAT	
INVOKE	
RETURN	
NEG	
FCMP	
JAE	
JBE	
JC	
JNC	
JP	
JNP	
JO	
JNO	
JS	
JNS	
LEA	
REALLOC	Reallocate a dynamic memory block: realloc dest, elemSize, count.

Definition at line 21 of file [instruct.hpp](#).

12.13.3 Function Documentation

12.13.3.1 operator<<() [1/4]

```
std::ostream & operator<< (
    std::ostream & out,
    const enum Inc & i)
```

Stream insertion operator for [Inc](#) opcodes.

Parameters

<i>out</i>	Output stream
<i>i</i>	Opcode to print

Returns

Reference to the output stream

Definition at line 23 of file [instruct.cpp](#).

References [IncType](#), and [pos\(\)](#).

12.13.3.2 operator<<() [2/4]

```
std::ostream & operator<< (
    std::ostream & out,
    const mxvm::Instruction & inc)
```

Stream insertion operator for Instruction.

Parameters

<i>out</i>	Output stream
<i>inc</i>	Instruction to format and print

Returns

Reference to the output stream

Definition at line 32 of file [instruct.cpp](#).

References [mxvm::Instruction::instruction](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), and [mxvm::Instruction::vop](#).

12.13.3.3 operator<<() [3/4]

```
std::ostream & operator<< (
    std::ostream & out,
    const mxvm::Keywords & key)
```

Stream insertion operator for Keywords.

Parameters

<i>out</i>	Output stream
<i>key</i>	Keyword to print

Returns

Reference to the output stream

Definition at line 14 of file [instruct.cpp](#).

References [mxvm::keywords](#).

12.13.3.4 operator<<() [4/4]

```
std::ostream & operator<< (
    std::ostream & out,
    const mxvm::VarType & type)
```

Stream insertion operator for VarType.

Parameters

<i>out</i>	Output stream
<i>type</i>	Variable type to print

Returns

Reference to the output stream

Definition at line 54 of file [instruct.cpp](#).

References [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

12.13.4 Variable Documentation**12.13.4.1 IncType**

```
std::vector<std::string> IncType [inline]
```

String representations of [Inc](#) opcodes, indexed by enum value.

Definition at line 82 of file [instruct.hpp](#).

Referenced by [mxvm::buildInstructionSet\(\)](#), [operator<<\(\)](#), [mxvm::Program::print\(\)](#), and [mxvm::Validator::validate\(\)](#).

12.14 instruct.hpp

[Go to the documentation of this file.](#)

```
00001
00006 #ifndef __INSTRUCT_HPP_X_
00007 #define __INSTRUCT_HPP_X_
00008
00009 #include <stdint>
00010 #include <iostream>
00011 #include <string>
00012 #include <vector>
00013
00021 enum Inc {
00022     NULL_INC = 0,
00023     MOV,
00024     LOAD,
00025     STORE,
00026     ADD,
00027     SUB,
00028     MUL,
00029     DIV,
00030     OR,
00031     AND,
00032     XOR,
00033     NOT,
00034     MOD,
00035     CMP,
00036     JMP,
00037     JE,
00038     JNE,
00039     JL,
00040     JLE,
00041     JG,
00042     JGE,
00043     JZ,
00044     JNZ,
00045     JA,
00046     JB,
00047     PRINT,
00048     EXIT,
00049     ALLOC,
00050     FREE,
00051     GETLINE,
00052     PUSH,
00053     POP,
00054     STACK_LOAD,
00055     STACK_STORE,
00056     STACK_SUB,
00057     CALL,
00058     RET,
00059     STRING_PRINT,
00060     DONE,
00061     TO_INT,
00062     TO_FLOAT,
00063     INVOKE,
00064     RETURN,
00065     NEG,
00066     FCMP,
00067     JAE,
00068     JBE,
00069     JC,
```

```

00070     JNC,
00071     JP,
00072     JNP,
00073     JO,
00074     JNO,
00075     JS,
00076     JNS,
00077     LEA,
00078     REALLOC
00079 };
00080
00082 inline std::vector<std::string> IncType{
00083     "NULL",          // NULL_INC = 0
00084     "mov",           // MOV = 1
00085     "load",          // LOAD = 2
00086     "store",         // STORE = 3
00087     "add",           // ADD = 4
00088     "sub",           // SUB = 5
00089     "mul",           // MUL = 6
00090     "div",           // DIV = 7
00091     "or",            // OR = 8
00092     "and",           // AND = 9
00093     "xor",           // XOR = 10
00094     "not",           // NOT = 11
00095     "mod",           // MOD = 12
00096     "cmp",           // CMP = 13
00097     "jmp",           // JMP = 14
00098     "je",            // JE = 15
00099     "jne",           // JNE = 16
00100     "jl",            // JL = 17
00101     "jle",           // JLE = 18
00102     "jg",            // JG = 19
00103     "jge",           // JGE = 20
00104     "jz",            // JZ = 21
00105     "jnz",           // JNZ = 22
00106     "ja",            // JA = 23
00107     "jb",            // JB = 24
00108     "print",         // PRINT = 25
00109     "exit",          // EXIT = 26
00110     "alloc",         // ALLOC = 27
00111     "free",          // FREE = 28
00112     "getline",       // GETLINE = 29
00113     "push",          // PUSH = 30
00114     "pop",           // POP = 31
00115     "stack_load",    // STACK_LOAD = 32
00116     "stack_store",   // STACK_STORE = 33
00117     "stack_sub",     // STACK_SUB = 34
00118     "call",          // CALL = 35
00119     "ret",           // RET = 36
00120     "string_print",  // STRING_PRINT = 37
00121     "done",          // DONE = 38
00122     "to_int",        // TO_INT = 39
00123     "to_float",      // TO_FLOAT = 40
00124     "invoke",        // INVOKE = 41
00125     "return",        // RETURN = 42
00126     "neg",           // NEG = 43
00127     "fcmp",          // FCMP = 44 (floating-point compare)
00128     "jae",           // JAE = 45 (jump if above or equal, unsigned)
00129     "jbe",           // JBE = 46 (jump if below or equal, unsigned)
00130     "jc",            // JC = 47 (jump if carry)
00131     "jnc",           // JNC = 48 (jump if no carry)
00132     "jp",            // JP = 49 (jump if parity)
00133     "jnp",           // JNP = 50 (jump if no parity)
00134     "jo",            // JO = 51 (jump if overflow)
00135     "jno",           // JNO = 52 (jump if no overflow)
00136     "js",            // JS = 53 (jump if sign)
00137     "jns",           // JNS = 54 (jump if no sign)
00138     "lea",           // LEA = 55 (load effective address)
00139     "realloc"        // REALLOC = 56 (reallocate memory)
00140 };
00141
00148 std::ostream &operator<<(std::ostream &out, const enum Inc &i);
00149
00150 namespace mxvm {
00151
00153     enum class OperandType {
00154         OP_CONSTANT,
00155         OP_VARIABLE
00156     };
00157
00159     struct Operand {
00160         std::string label;
00161         std::string op;
00162         int op_value = 0;
00163         OperandType type;
00164         std::string object;
00165     };

```

```

00166
00167 struct Instruction {
00168     Inc instruction;
00169     Operand opl, op2, op3;
00170     std::vector<Operand> vop;
00171     std::string label = "";
00172     std::string toString() const;
00173 };
00174
00175 enum class VarType {
00176     VAR_NULL = 0,
00177     VAR_INTEGER,
00178     VAR_FLOAT,
00179     VAR_STRING,
00180     VAR_POINTER,
00181     VAR_LABEL,
00182     VAR_ARRAY,
00183     VAR_EXTERN,
00184     VAR_BYTE
00185 };
00186
00187 enum class Keywords {
00188     KEY_PROGRAM,
00189     KEY_OBJECT,
00190     KEY_MODULE,
00191     KEY_DATA,
00192     KEY_CODE,
00193     KEY_SECTION
00194 };
00195
00196 extern std::vector<std::string> keywords;
00197 // const char *keywords[] = {"program", "object", "module", "data", "code", "section" };
00198
00199 struct Variable_Value {
00200     std::string str_value;
00201     std::string label_value;
00202     union {
00203         int64_t int_value;
00204         double float_value;
00205         void *ptr_value;
00206     };
00207     uint64_t ptr_size = 0;
00208     uint64_t ptr_count = 0;
00209     VarType type;
00210     uint64_t buffer_size;
00211     bool owns = false;
00212     bool released = false;
00213
00214     Variable_Value(const Variable_Value &other)
00215     : str_value(other.str_value),
00216       label_value(other.label_value),
00217       ptr_size(other.ptr_size),
00218       ptr_count(other.ptr_count),
00219       type(other.type),
00220       buffer_size(other.buffer_size),
00221       owns(other.owns),
00222       released(other.released) {
00223         switch (type) {
00224             case VarType::VAR_INTEGER:
00225                 int_value = other.int_value;
00226                 break;
00227             case VarType::VAR_FLOAT:
00228                 float_value = other.float_value;
00229                 break;
00230             case VarType::VAR_POINTER:
00231                 ptr_value = other.ptr_value;
00232                 break;
00233             default:
00234                 int_value = other.int_value;
00235                 break;
00236         }
00237     }
00238
00239     Variable_Value &operator=(const Variable_Value &other) {
00240         if (this != &other) {
00241             str_value = other.str_value;
00242             label_value = other.label_value;
00243             ptr_size = other.ptr_size;
00244             ptr_count = other.ptr_count;
00245             type = other.type;
00246             buffer_size = other.buffer_size;
00247             owns = other.owns;
00248             released = other.released;
00249             switch (type) {
00250                 case VarType::VAR_INTEGER:
00251                 case VarType::VAR_BYTE:
00252                     int_value = other.int_value;
00253                     break;

```

```

00273         case VarType::VAR_FLOAT:
00274             float_value = other.float_value;
00275             break;
00276         case VarType::VAR_POINTER:
00277             ptr_value = other.ptr_value;
00278             break;
00279         default:
00280             int_value = other.int_value;
00281             break;
00282     }
00283 }
00284 return *this;
00285 }
00286
00288     Variable_Value() : int_value(0), ptr_size(0), ptr_count(0), type(VarType::VAR_NULL),
buffer_size(0), owns(false), released(false) {}
00289 };
00290
00292     struct Variable {
00293         VarType type;
00294         std::string var_name;
00295         Variable_Value var_value;
00296         bool is_global = false;
00297         std::string obj_name;
00299         Variable() : type(VarType::VAR_NULL), var_name(), var_value() {}
00300
00306         Variable(const std::string &name, const VarType &vtype, const Variable_Value &value)
00307             : type(vtype), var_name(name), var_value(value), is_global(false) {}
00308
00312         Variable(const Variable &v)
00313             : type(v.type), var_name(v.var_name), var_value(v.var_value), is_global(v.is_global),
obj_name(v.obj_name) {}
00314
00319         void setExtern(std::string name, void *value) {
00320             type = VarType::VAR_EXTERN;
00321             var_value.type = VarType::VAR_EXTERN;
00322             var_value.ptr_value = value;
00323             var_value.ptr_count = 0;
00324             var_value.ptr_size = 0;
00325             var_value.owns = false;
00326             var_name = name;
00327         }
00328
00333         Variable &operator=(const Variable &v) {
00334             if (this != &v) {
00335                 type = v.type;
00336                 var_name = v.var_name;
00337                 var_value = v.var_value;
00338                 is_global = v.is_global;
00339                 obj_name = v.obj_name;
00340             }
00341             return *this;
00342         }
00346         std::string toString() const;
00347     };
00348 } // namespace mxvm
00349
00356 std::ostream &operator<<(std::ostream &out, const mxvm::Instruction &inc);
00357
00364 std::ostream &operator<<(std::ostream &out, const mxvm::VarType &type);
00365
00372 std::ostream &operator<<(std::ostream &out, const mxvm::Keywords &key);
00373 #endif

```

12.15 include/mxvm/mxvm.hpp File Reference

Main MXVM header aggregating all subsystem includes.

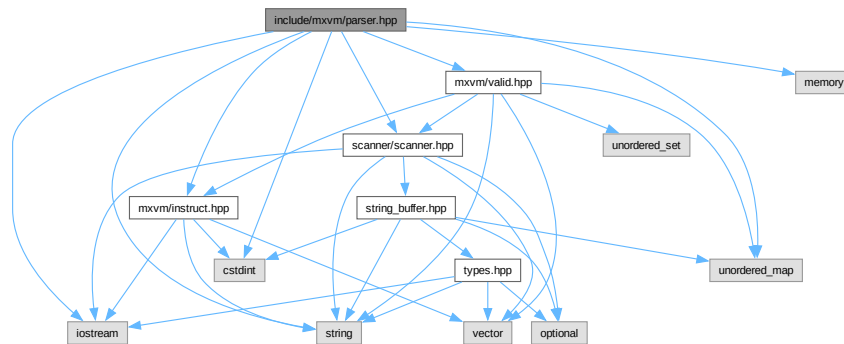
```

#include "icode.hpp"
#include "version_info.hpp"

```



```
#include <cstdint>
#include <iostream>
#include <memory>
#include <string>
#include <unordered_map>
Include dependency graph for parser.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [mxvm::Parser](#)
Main MXVM parser — tokenizes source, builds AST, and generates intermediate code.
- struct [mxvm::ExternalFunction](#)
Describes an external function imported from a module or object.
- class [mxvm::ModuleParser](#)
Parser for MXVM module definition files (.mxvm).

Namespaces

- namespace [mxvm](#)

Enumerations

- enum class [mxvm::Mode](#) { [mxvm::MODE_INTERPRET](#) , [mxvm::MODE_COMPILE](#) }
Execution mode: interpretation or native compilation.
- enum class [mxvm::Platform](#) { [mxvm::LINUX](#) , [mxvm::DARWIN](#) , [mxvm::WINX64](#) }
Target platform for native code generation.

Variables

- bool [mxvm::debug_mode](#) = false
enable verbose debug output during parsing
- bool [mxvm::instruct_mode](#) = false

- *enable instruction trace mode*
- `bool mxvm::html_mode = false`
- *enable HTML debug output*

12.17.1 Detailed Description

Parser that tokenizes and parses MXVM source into programs, handles modules/objects, and dispatches code generation.

Author

Jared Bruni

Definition in file [parser.hpp](#).

12.18 parser.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef _PARSER_H_X
00007 #define _PARSER_H_X
00008
00009 #include "mxvm/instruct.hpp"
00010 #include "mxvm/valid.hpp"
00011 #include "scanner/scanner.hpp"
00012 #include <cstdlib>
00013 #include <iostream>
00014 #include <memory>
00015 #include <string>
00016 #include <unordered_map>
00017
00018 namespace mxvm {
00019     class ProgramNode;
00020     class SectionNode;
00021     class InstructionNode;
00022     class VariableNode;
00023     class CommentNode;
00024     class LabelNode;
00025     class Program;
00026     class ModuleNode;
00027     class ObjectNode;
00028     struct Variable;
00029     struct Operand;
00030
00031     extern bool debug_mode;
00032     extern bool instruct_mode;
00033     extern bool html_mode;
00034
00036     enum class Mode {
00037         MODE_INTERPRET,
00038         MODE_COMPILE
00039     };
00040
00042     enum class Platform {
00043         LINUX,
00044         DARWIN,
00045         WINX64
00046     };
00047
00048     class ModuleParser;
00049
00057     class Parser {
00058     public:
00059         friend class ModuleParser;
00060         explicit Parser(const std::string &source);
00067         Parser(const Parser &other)
00068             : source_file(other.source_file),
00069               scanner(other.scanner),
00070               validator(other.validator),
00071               parser_mode(other.parser_mode),
00072               module_path(other.module_path),
00073               object_path(other.object_path),
00074               include_path(other.include_path),
00075               object_mode(other.object_mode),
00076               object_name(other.object_name),
00077               platform{other.platform} {}
00078

```

```

00080     Parser &operator=(const Parser &other) {
00081         if (this != &other) {
00082             source_file = other.source_file;
00083             scanner = other.scanner;
00084             validator = other.validator;
00085             parser_mode = other.parser_mode;
00086             module_path = other.module_path;
00087             object_path = other.object_path;
00088             include_path = other.include_path;
00089             object_mode = other.object_mode;
00090             object_name = other.object_name;
00091             platform = other.platform;
00092         }
00093         return *this;
00094     }
00095
00096     uint64_t scan();
00097
00098     void parse();
00099     auto operator[] (size_t pos);
00100
00101     std::unique_ptr<ProgramNode> parseProgramOrObject (uint64_t &index);
00102     std::unique_ptr<ProgramNode> parseAST();
00103     bool generateProgramCode(const Mode &m, std::unique_ptr<Program> &program);
00104     bool generateDebugHTML(std::ostream &out, std::unique_ptr<Program> &program);
00105     void generateObjectAssemblyFile(std::unique_ptr<Program> &objProgram);
00106     void registerObjectExterns(std::unique_ptr<Program> &mainProgram, const
00107     std::unique_ptr<Program> &objProgram);
00108     std::string source_file;
00109     Scanner scanner;
00110     Validator validator;
00111     Mode parser_mode = Mode::MODE_INTERPRET;
00112     std::string module_path = ".";
00113     std::string object_path = ".";
00114     std::string include_path = "/usr/local/include/mxvm/modules";
00115     bool object_mode = false;
00116     std::string object_name;
00117     Platform platform;
00118
00119 private:
00120     std::unique_ptr<SectionNode> parseSection(uint64_t &index);
00121     std::unique_ptr<VariableNode> parseDataVariable(uint64_t &index);
00122     std::unique_ptr<InstructionNode> parseCodeInstruction(uint64_t &index);
00123     std::unique_ptr<CommentNode> parseComment(uint64_t &index);
00124     std::unique_ptr<LabelNode> parseLabel(uint64_t &index);
00125     std::unique_ptr<ModuleNode> parseModule(uint64_t &index);
00126     std::unique_ptr<ObjectNode> parseObject(uint64_t &index);
00127
00128     void processDataSection(SectionNode *sectionNode, std::unique_ptr<Program> &program);
00129     void processCodeSection(SectionNode *sectionNode, std::unique_ptr<Program> &program);
00130     void processModuleSection(SectionNode *sectionNode, std::unique_ptr<Program> &program);
00131     void processObjectSection(SectionNode *sectionNode, std::unique_ptr<Program> &program);
00132     void processModuleFile(const std::string &src, std::unique_ptr<Program> &program);
00133     void processObjectFile(const std::string &src, std::unique_ptr<Program> &program);
00134     void setVariableValue(Variable &var, VarType type, const std::string &value, size_t buf_size =
00135     0);
00136     void setDefaultVariableValue(Variable &var, VarType type);
00137     void resolveLabelReference(Operand &operand, const std::unordered_map<std::string, size_t>
00138     &labelMap);
00139     void collectObjectNames(std::vector<std::pair<std::string, std::string> &names, const
00140     std::unique_ptr<Program> &program);
00141     void printObjectHTML(std::ostream &out, const std::unique_ptr<Program> &objPtr);
00142 };
00143
00144 struct ExternalFunction {
00145     std::string name;
00146     std::string mod;
00147     bool module;
00148     bool operator==(const ExternalFunction &f) {
00149         return (name == f.name && mod == f.mod);
00150     }
00151
00152     VarType ret_type = VarType::VAR_INTEGER;
00153     bool ret_owned = false;
00154 };
00155
00156 class ModuleParser {
00157 public:
00158     ModuleParser(const Mode &m, const std::string &mod_name, const std::string &source);
00159
00160     ModuleParser(const ModuleParser &other)
00161         : functions(other.functions),
00162           mod_name(other.mod_name),
00163           scanner(other.scanner),
00164           index(other.index),
00165           token(other.token ? new scan::Token(*other.token) : nullptr) {}

```



```

00200     ModuleParser &operator=(const ModuleParser &other) {
00201         if (this != &other) {
00202             functions = other.functions;
00203             mod_name = other.mod_name;
00204             scanner = other.scanner;
00205             index = other.index;
00206             if (token)
00207                 delete token;
00208             token = other.token ? new scan::TToken(*other.token) : nullptr;
00209         }
00210         return *this;
00211     }
00212
00214     uint64_t scan();
00216     bool parse();
00218     scan::TToken operator[](size_t pos);
00220     scan::TToken at(size_t pos);
00222     bool generateProgramCode(const Mode &m, const std::string &mod_id, const std::string
&mod_name, std::unique_ptr<Program> &program);
00231     bool next();
00233     bool match(const std::string &s);
00235     bool match(const types::TokenType &t);
00237     void require(const std::string &s);
00239     void require(const types::TokenType &t);
00240
00241     protected:
00242     std::vector<ExternalFunction> functions;
00243     std::string mod_name;
00244     scan::Scanner scanner;
00245     uint64_t index = 0;
00246     scan::TToken *token = nullptr;
00247     Mode parser_mode;
00248 };
00249 } // namespace mxvm
00250
00251 #endif

```

12.19 src/frontend/include/parser.hpp File Reference

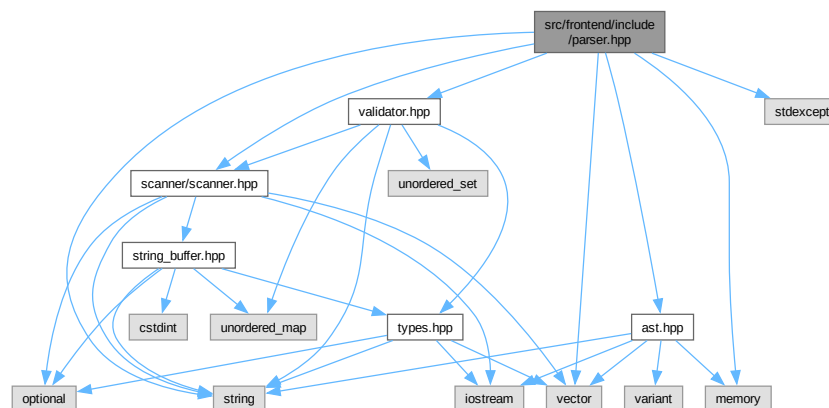
Pascal recursive-descent parser producing a Pascal AST.

```

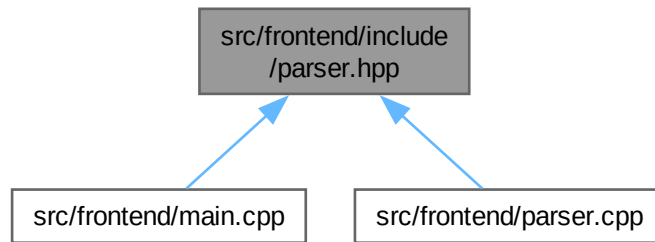
#include "ast.hpp"
#include "scanner/scanner.hpp"
#include "validator.hpp"
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>

```

Include dependency graph for parser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [pascal::ParseException](#)
Exception thrown on parse errors.
- class [mxx::XParser](#)
Base parser providing token-stream navigation.
- class [pascal::PascalParser](#)
Recursive-descent parser for the Pascal language.

Namespaces

- namespace [pascal](#)
- namespace [mxx](#)

12.19.1 Detailed Description

Pascal recursive-descent parser producing a Pascal AST.

Author

Jared Bruni

Definition in file [parser.hpp](#).

12.20 parser.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef _PARSER_H__H
00007 #define _PARSER_H__H
00008 #include "ast.hpp"
00009 #include "scanner/scanner.hpp"
00010 #include "validator.hpp"
00011 #include <memory>
00012 #include <stdexcept>
00013 #include <string>
00014 #include <vector>
00015
00016 namespace pascal {
00018     class ParseException : public std::runtime_error {
00019     public:
00021         explicit ParseException(const std::string &msg) : std::runtime_error(msg) {}
00022     };
00023 } // namespace pascal
00024
00025 namespace mxx {

```

```

00032     class XParser {
00033     public:
00034         XParser() = delete;
00039         explicit XParser(const std::string &source)
00040             : scanner(source), token(nullptr), index(0) {
00041             scanner.scan();
00042             scanner.removeEOL();
00043             if (scanner.size() > 0) {
00044                 token = &scanner[0];
00045                 index = 0;
00046             }
00047         }
00048
00050         bool next() {
00051             if (index + 1 < scanner.size()) {
00052                 index++;
00053                 token = &scanner[index];
00054                 return true;
00055             }
00056             token = nullptr;
00057             return false;
00058         }
00059
00061         bool peekIs(const std::string &s) {
00062             if (!token) return false;
00063             const auto &v = token->getTokenValue();
00064             if (v.size() != s.size()) return false;
00065             for (size_t i = 0; i < v.size(); ++i)
00066                 if (std::tolower(static_cast<unsigned char>(v[i])) !=
00067                     std::tolower(static_cast<unsigned char>(s[i])))
00068                     return false;
00069             return true;
00070         }
00072         bool peekIs(const types::TokenType &t) {
00073             return token && token->getTokenType() == t;
00074         }
00075
00081         static std::string tokenTypeToString(types::TokenType t) {
00082             switch (t) {
00083             case types::TokenType::TT_ID:
00084                 return "IDENTIFIER";
00085             case types::TokenType::TT_NUM:
00086                 return "NUMBER";
00087             case types::TokenType::TT_STR:
00088                 return "STRING";
00089             case types::TokenType::TT_SYM:
00090                 return "SYMBOL";
00091             default:
00092                 return "UNKNOWN";
00093             }
00094         }
00095
00096         std::string filename;
00097
00098     protected:
00099         scan::Scanner scanner;
00100         scan::Token *token = nullptr;
00101         size_t index = 0;
00102     };
00103 } // namespace mxx
00104
00105 namespace pascal {
00112     class PascalParser : public mxx::XParser {
00113     public:
00114         PascalParser() = delete;
00119         explicit PascalParser(const std::string &source) : mxx::XParser(source), validator(source) {
00120             removeBraceComments();
00121             if (scanner.size() > 0) {
00122                 token = &scanner[0];
00123                 index = 0;
00124             } else {
00125                 token = nullptr;
00126             }
00127         }
00128
00130         std::unique_ptr<ProgramNode> parseProgram();
00132         std::unique_ptr<UnitNode> parseUnit();
00134         bool isUnitSource() const;
00135         mxx::TPValidator validator;
00136
00137     private:
00139         void removeBraceComments();
00141         void error(const std::string &message);
00143         void expectToken(const std::string &expected);
00145         void expectToken(types::TokenType expected);
00147         bool match(const std::string &s);
00149         bool match(types::TokenType t);

```

```

00151     std::unique_ptr<ASTNode> parseLValue();
00153     std::unique_ptr<BlockNode> parseBlock();
00155     std::unique_ptr<CompoundStmtNode> parseCompoundStatement();
00157     std::vector<std::unique_ptr<ASTNode>> parseDeclarations();
00159     std::unique_ptr<ASTNode> parseVarDeclaration();
00161     std::unique_ptr<ASTNode> parseConstDeclaration();
00163     std::unique_ptr<ASTNode> parseProcedureDeclaration();
00165     std::unique_ptr<ASTNode> parseFunctionDeclaration();
00167     std::vector<std::unique_ptr<ASTNode>> parseInterfaceDeclarations();
00169     std::unique_ptr<ASTNode> parseProcedureForwardDecl();
00171     std::unique_ptr<ASTNode> parseFunctionForwardDecl();
00173     std::vector<std::unique_ptr<ASTNode>> parseParameterList();
00175     std::vector<std::unique_ptr<ASTNode>> parseStatementList();
00177     std::unique_ptr<ASTNode> parseParameter();
00179     std::unique_ptr<ASTNode> parseRecordType();
00181     std::unique_ptr<ASTNode> parseTypeDeclaration();
00183     std::unique_ptr<ASTNode> parseStatement();
00185     std::unique_ptr<ASTNode> parseAssignmentOrProcCall();
00187     std::unique_ptr<ASTNode> parseIfStatement();
00189     std::unique_ptr<ASTNode> parseWhileStatement();
00191     std::unique_ptr<ASTNode> parseForStatement();
00193     std::unique_ptr<ASTNode> parseRepeatStatement();
00195     std::unique_ptr<ASTNode> parseCaseStatement();
00197     std::unique_ptr<ASTNode> parseWithStatement();
00199     std::unique_ptr<ASTNode> parseGotoStatement();
00201     void parseLabelDeclaration();
00203     std::unordered_set<std::string> declaredLabels;
00205     std::unique_ptr<ASTNode> parseExpression();
00207     std::unique_ptr<ASTNode> parseSimpleExpression();
00209     std::unique_ptr<ASTNode> parseTerm();
00211     std::unique_ptr<ASTNode> parseFactor();
00213     std::unique_ptr<ASTNode> parseProcedureCall(const std::string &name);
00215     std::unique_ptr<ASTNode> parseFunctionCall(const std::string &name);
00217     std::vector<std::unique_ptr<ASTNode>> parseArgumentList();
00218
00220     bool isRelationalOperator();
00222     bool isAddOperator();
00224     bool isMulOperator();
00225
00227     std::unique_ptr<ASTNode> parseTypeSpec();
00228
00230     std::string getRelationalOp();
00232     std::string getAddOp();
00234     std::string getMulOp();
00236     std::string getUnaryOp();
00238     BinaryOpNode::OpType getComparisonOperator(const std::string &op);
00240     BinaryOpNode::OpType getLogicalOperator(const std::string &op);
00242     BinaryOpNode::OpType getArithmeticOperator(const std::string &op);
00244     std::string tokenTypeToString(types::TokenType type);
00246     bool isType(const std::string &token);
00248     bool isBuiltinProcedure(const std::string &name);
00250     bool isBuiltinFunction(const std::string &name);
00252     bool isKeyword(const std::string &s);
00254     std::unique_ptr<ASTNode> parseArrayType();
00256     std::unique_ptr<ASTNode> parseArrayDeclaration(const std::string &varName);
00258     bool isArrayAccess();
00259 };
00260 } // namespace pascal
00261
00262 #endif

```

12.21 include/mxvm/valid.hpp File Reference

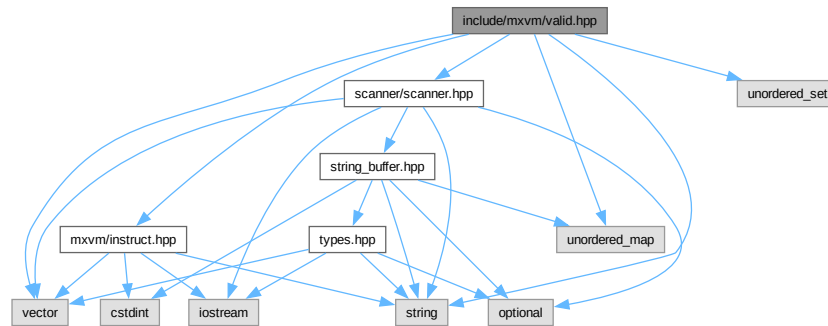
Validator for MXVM programs — checks variable/label usage and instruction operand specifications.

```

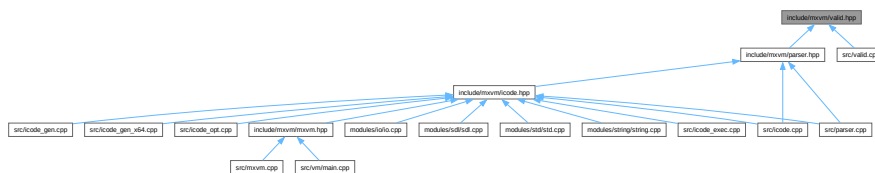
#include "mxvm/instruct.hpp"
#include "scanner/scanner.hpp"
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>

```

Include dependency graph for valid.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [mxvm::ParsedOp](#)
A parsed operand with its kind, text, and source location.
- struct [mxvm::UseVar](#)
Records a variable usage site for validation.
- struct [mxvm::UseLabel](#)
Records a label usage site for validation.
- struct [mxvm::OpSpec](#)
Specification of valid operand patterns for an instruction.
- class [mxvm::Validator](#)
Validates MXVM source for correct variable/label usage and instruction operands.
- struct [mxvm::Validator::VarNamePair](#)
A variable name paired with its declaration token.

Namespaces

- namespace [mxvm](#)

Enumerations

- enum class [mxvm::OpKind](#) {
[mxvm::Num](#) , [mxvm::Hex](#) , [mxvm::Str](#) , [mxvm::Id](#) ,
[mxvm::Member](#) , [mxvm::Label](#) , [mxvm::Any](#) }
Classification of operand kinds for validation.
- enum class [mxvm::VArity](#) { [mxvm::None](#) , [mxvm::AnyTail](#) , [mxvm::ArgsTail](#) }
Variadic arity policy for instruction operand specs.

12.21.1 Detailed Description

Validator for MXVM programs — checks variable/label usage and instruction operand specifications.

Author

Jared Bruni

Definition in file [valid.hpp](#).

12.22 valid.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef _VALID_H_X
00007 #define _VALID_H_X
00008
00009 #include "mxvm/instruct.hpp"
00010 #include "scanner/scanner.hpp"
00011 #include <string>
00012 #include <unordered_map>
00013 #include <unordered_set>
00014 #include <vector>
00015
00016 namespace mxvm {
00017
00019     enum class OpKind {
00020         Num,
00021         Hex,
00022         Str,
00023         Id,
00024         Member,
00025         Label,
00026         Any
00027     };
00028
00030     struct ParsedOp {
00031         OpKind kind;
00032         std::string text;
00033         const scan::TToken *at;
00034     };
00035
00037     struct UseVar {
00038         std::string name;
00039         const scan::TToken *at;
00040     };
00042     struct UseLabel {
00043         std::string name;
00044         const scan::TToken *at;
00045     };
00046
00048     enum class VArity {
00049         None,
00050         AnyTail,
00051         ArgsTail
00052     };
00053
00055     struct OpSpec {
00056         std::string name;
00057         std::vector<OpKind> fixed;
00058         VArity varPolicy = VArity::None;
00059         int minArgs = -1;
00060         int maxArgs = -1;
00061     };
00062
00069     class Validator {
00070     private:
00071         std::string filename;
00072         scan::Scanner scanner;
00073         std::string source;
00074         size_t index = 0;
00075         const scan::TToken *token = nullptr;
00076         std::string current_function;
00077         std::unordered_map<std::string, std::unordered_set<std::string> function_vars;
00078
00079     public:
00084         Validator(const std::string &source);
00086         struct VarNamePair {
00087             std::string first;
00088             const scan::TToken *second;
00089         };
00090         std::vector<VarNamePair> var_names;

```

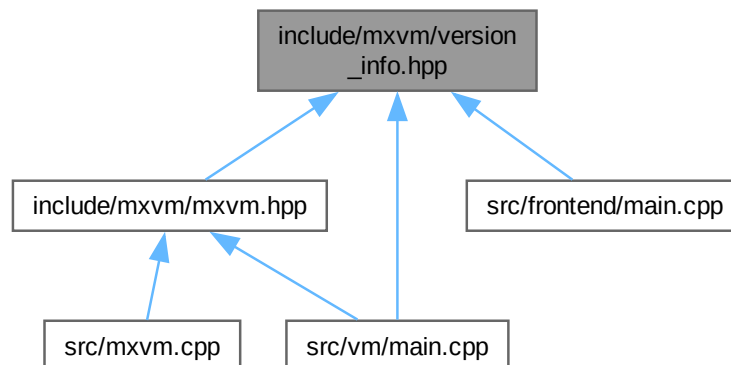
```

00101         void validateAgainstSpec(
00102             const std::string &op,
00103             const std::vector<ParsedOp> &ops,
00104             const std::unordered_map<std::string, Variable> &vars,
00105             const std::unordered_map<std::string, std::string> &labels,
00106             const std::unordered_set<std::string> &objects,
00107             std::vector<UseVar> &usedVarsRef,
00108             std::vector<UseLabel> &usedLabelsRef);
00109
00115         bool validate(const std::string &name);
00117         bool match(const std::string &m);
00119         void require(const std::string &r);
00121         bool match(const types::TokenType &t);
00123         void require(const types::TokenType &t);
00125         bool next();
00127         bool peekIs(const std::string &s);
00129         bool peekIs(const types::TokenType &t);
00131         std::string tokenTypeToString(types::TokenType t);
00133         void collect_labels(std::unordered_map<std::string, std::string> &labels);
00135         void collect_objects(std::unordered_set<std::string> &objects, size_t start_index, size_t
end_index);
00137         ParsedOp parseOperand();
00139         std::vector<ParsedOp> parseOperandList();
00140     };
00141 } // namespace mxvm
00142
00143 #endif

```

12.23 include/mxvm/version_info.hpp File Reference

This graph shows which files directly or indirectly include this file:



Macros

- `#define VERSION_INFO "1.8.1"`
MXVM version string.

12.23.1 Macro Definition Documentation

12.23.1.1 VERSION_INFO

```
#define VERSION_INFO "1.8.1"
```

MXVM version string.

Definition at line 3 of file [version_info.hpp](#).

Referenced by [main\(\)](#), and [print_help\(\)](#).

12.24 version_info.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef __VERSION_INFO_MX
00002 #define __VERSION_INFO_MX
00003 #define VERSION_INFO "1.8.1"
00004 #endif
```

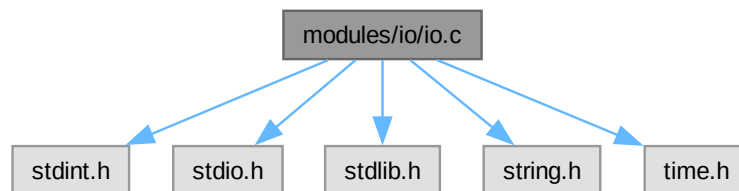
12.25 mainpage.md File Reference

12.26 modules/io/io.c File Reference

I/O module C implementation — file operations, random numbers, and time functions.

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
```

Include dependency graph for io.c:



Functions

- `int64_t fsize` (`FILE *fptr`)
- `int64_t seed_random` ()
- `int64_t rand_number` (`int64_t size`)
- `char * mxvm_fgets` (`FILE *fp`)

12.26.1 Detailed Description

I/O module C implementation — file operations, random numbers, and time functions.

Author

Jared Bruni

Definition in file [io.c](#).

12.26.2 Function Documentation

12.26.2.1 fsize()

```
int64_t fsize (
    FILE * fptr)
```

Definition at line 12 of file [io.c](#).

12.26.2.2 mxvm_fgets()

```
char * mxvm_fgets (
    FILE * fp)
```

Definition at line 29 of file [io.c](#).

12.26.2.3 rand_number()

```
int64_t rand_number (
    int64_t size)
```

Definition at line 25 of file [io.c](#).

12.26.2.4 seed_random()

```
int64_t seed_random ()
```

Definition at line 19 of file [io.c](#).

12.27 io.c

[Go to the documentation of this file.](#)

```
00001
00006 #include <stdint.h>
00007 #include <stdio.h>
00008 #include <stdlib.h>
00009 #include <string.h>
00010 #include <time.h>
00011
00012 int64_t fsize(FILE *fp) {
00013     fseek(fp, 0, SEEK_END);
00014     int64_t total = ftell(fp);
00015     fseek(fp, 0, SEEK_SET);
00016     return total;
00017 }
00018
00019 int64_t seed_random() {
00020     unsigned int seed = (unsigned int)time(NULL) ^ (uintptr_t)&seed;
00021     srand(seed);
00022     return 0;
00023 }
00024
00025 int64_t rand_number(int64_t size) {
00026     return rand() % size;
00027 }
00028
00029 char *mxvm_fgets(FILE *fp) {
00030     static char buf[4096];
00031     char *res = fgets(buf, sizeof(buf), fp);
00032     if (res) {
00033         size_t len = strlen(buf);
00034         if (len > 0 && buf[len - 1] == '\n')
00035             buf[len - 1] = '\0';
00036         return strdup(buf);
00037     }
00038     return strdup("");
00039 }
```

12.28 modules/io/io.cpp File Reference

I/O module C++ runtime bindings for the MXVM interpreter.

```
#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <ctime>
#include <mxvm/icode.hpp>
#include <mxvm/instruct.hpp>
#include <string>
#include <vector>
```


12.28.2.3 mxvm_io_fopen()

```
void mxvm_io_fopen (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 15 of file [io.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.28.2.4 mxvm_io_fprintf()

```
void mxvm_io_fprintf (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 45 of file [io.cpp](#).

References [mxvm::Program::createTempVariable\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::OP_CONSTANT](#), [mxvm::Program::printFormatted\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

12.28.2.5 mxvm_io_fputs()

```
void mxvm_io_fputs (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 309 of file [io.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.28.2.6 mxvm_io_fread()

```
void mxvm_io_fread (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 124 of file [io.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), [mxvm::Program::variableFromOperand\(\)](#), and [mxvm::Base::vars](#).

12.28.2.7 mxvm_io_fseek()

```
void mxvm_io_fseek (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 186 of file [io.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), [mxvm::Program::variableFromOperand\(\)](#), and [mxvm::Base::vars](#).

12.28.2.8 mxvm_io_fsize()

```
void mxvm_io_fsize (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 100 of file [io.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.28.2.9 mxvm_io_fwrite()

```
void mxvm_io_fwrite (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 155 of file [io.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), [mxvm::Program::variableFromOperand\(\)](#), and [mxvm::Base::vars](#).

12.28.2.10 mxvm_io_mxvm_fgets()

```
void mxvm_io_mxvm_fgets (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 279 of file [io.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.28.2.11 mxvm_io_rand_number()

```
void mxvm_io_rand_number (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 217 of file [io.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::OP_CONSTANT](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.28.2.12 mxvm_io_seed_random()

```
void mxvm_io_seed_random (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 244 of file [io.cpp](#).

References [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.29 io.cpp

[Go to the documentation of this file.](#)

```
00001
00006 #include <chrono>
00007 #include <cstdint>
00008 #include <cstdlib>
00009 #include <ctime>
00010 #include <mxvm/icode.hpp>
00011 #include <mxvm/instruct.hpp>
00012 #include <string>
00013 #include <vector>
00014
00015 extern "C" void mxvm_io_fopen(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00016     if (operand.size() == 2) {
00017         std::string &filename = operand[0].op;
00018         std::string &mode = operand[1].op;
00019         if (program->isVariable(filename)) {
00020             mxvm::Variable &v = program->getVariable(filename);
00021             if (v.type != mxvm::VarType::VAR_STRING) {
00022                 throw mx::Exception("Requires string variable type for fopen.\n");
00023             }
00024             filename = v.var_value.str_value;
00025         } else {
00026             throw mx::Exception("Variable required for fopen.");
00027         }
00028         if (program->isVariable(mode)) {
00029             mxvm::Variable &v = program->getVariable(mode);
00030             if (v.type != mxvm::VarType::VAR_STRING) {
00031                 throw mx::Exception("Requires string variable type for fopen.\n");
00032             }
00033             mode = v.var_value.str_value;
```

```

00034         } else {
00035             throw mx::Exception("Variable required for fopen.");
00036         }
00037         program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00038         program->vars["%rax"].var_name = "%rax";
00039         program->vars["%rax"].var_value.ptr_value = (void *)fopen(filename.c_str(), mode.c_str());
00040     } else {
00041         throw mx::Exception("Error invalid types for fopen.\n");
00042     }
00043 }
00044
00045 extern "C" void mxvm_io_fprintf(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00046     if (operand.size() < 2) {
00047         throw mx::Exception("Requires at least two arguments for fprintf.");
00048     }
00049     if (program->isVariable(operand[0].op)) {
00050         std::vector<mxvm::Variable *> v;
00051         for (size_t i = 2; i < operand.size(); ++i) {
00052             if (program->isVariable(operand[i].op)) {
00053                 mxvm::Variable &varval = program->getVariable(operand[i].op);
00054                 v.push_back(&varval);
00055             } else {
00056                 if (operand[i].type == mxvm::OperandType::OP_CONSTANT) {
00057                     mxvm::Variable val = program->createTempVariable(mxvm::VarType::VAR_INTEGER,
operand[i].op);
00058                     v.push_back(&val);
00059                 }
00060             }
00061         }
00062         if (program->isVariable(operand[1].op)) {
00063             mxvm::Variable &fmtv = program->getVariable(operand[1].op);
00064             if (fmtv.type != mxvm::VarType::VAR_STRING) {
00065                 throw mx::Exception("fprintf requires format to be a string.");
00066             }
00067             std::string value = program->printFormatted(fmtv.var_value.str_value, v, false);
00068             mxvm::Variable &vx = program->getVariable(operand[0].op);
00069             if (vx.type == mxvm::VarType::VAR_POINTER || vx.type == mxvm::VarType::VAR_EXTERN) {
00070                 FILE *fptr = reinterpret_cast<FILE *>(vx.var_value.ptr_value);
00071                 fprintf(fptr, "%s", value.c_str());
00072             }
00073         } else {
00074             throw mx::Exception("fprintf requires format to be a variable");
00075         }
00076     } else {
00077         throw mx::Exception("fprintf requires first argument to be valid variable found: " +
operand[0].op);
00078     }
00079 }
00080
00081 extern "C" void mxvm_io_fclose(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00082     if (operand.size() != 1) {
00083         throw mx::Exception("fclose requires a single file pointer argument.");
00084     }
00085     std::string &file_var = operand[0].op;
00086     if (!program->isVariable(file_var)) {
00087         throw mx::Exception("fclose argument must be a variable (file pointer).");
00088     }
00089     mxvm::Variable &file_v = program->getVariable(file_var);
00090     if (file_v.type != mxvm::VarType::VAR_POINTER) {
00091         throw mx::Exception("fclose argument must be a pointer variable.");
00092     }
00093     FILE *fp = reinterpret_cast<FILE *>(file_v.var_value.ptr_value);
00094     int result = fclose(fp);
00095     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00096     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00097     program->vars["%rax"].var_value.int_value = result;
00098 }
00099
00100 extern "C" void mxvm_io_fsize(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00101     if (operand.size() != 1) {
00102         throw mx::Exception("fsize requires a single file pointer argument.");
00103     }
00104     std::string &file_var = operand[0].op;
00105     if (!program->isVariable(file_var)) {
00106         throw mx::Exception("fsize argument must be a variable pointer.");
00107     }
00108     mxvm::Variable &file_v = program->getVariable(file_var);
00109     if (file_v.type != mxvm::VarType::VAR_POINTER) {
00110         throw mx::Exception("fsize argument must be a pointer variable.");
00111     }
00112     FILE *fptr = reinterpret_cast<FILE *>(file_v.var_value.ptr_value);
00113     if (fptr == NULL) {
00114         throw mx::Exception("Error pointer handle must not be null");
00115     }
00116     fseek(fptr, 0, SEEK_END);
00117     size_t result = ftell(fptr);
00118     fseek(fptr, 0, SEEK_SET);

```

```

00119     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00120     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00121     program->vars["%rax"].var_value.int_value = result;
00122 }
00123
00124 extern "C" void mxvm_io_fread(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00125     if (operand.size() != 4) {
00126         throw mx::Exception("fread requires four arguments.");
00127     }
00128     std::string &buf_var = operand[0].op;
00129     if (!program->isVariable(buf_var)) {
00130         throw mx::Exception("fread argument must be a variable pointer.");
00131     }
00132     mxvm::Variable &buf_v = program->getVariable(buf_var);
00133     if (buf_v.type != mxvm::VarType::VAR_POINTER) {
00134         throw mx::Exception("fread argument must be a pointer variable.");
00135     }
00136
00137     mxvm::Variable vsize, vcount, file_ptr;
00138     vsize = program->variableFromOperand(operand[1]);
00139     vcount = program->variableFromOperand(operand[2]);
00140     file_ptr = program->variableFromOperand(operand[3]);
00141
00142     if (vsize.type != mxvm::VarType::VAR_INTEGER || vcount.type != mxvm::VarType::VAR_INTEGER) {
00143         throw mx::Exception("Argument type mismatch expected integer for fread");
00144     }
00145     if (file_ptr.type != mxvm::VarType::VAR_POINTER && file_ptr.type != mxvm::VarType::VAR_EXTERN) {
00146         throw mx::Exception("Final argument for fread requires pointer");
00147     }
00148     FILE *fptr = reinterpret_cast<FILE *>(file_ptr.var_value.ptr_value);
00149     size_t result = fread(buf_v.var_value.ptr_value, vsize.var_value.int_value,
00150         vcount.var_value.int_value, fptr);
00151     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00152     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00153     program->vars["%rax"].var_value.int_value = result;
00154 }
00155 extern "C" void mxvm_io_fwrite(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00156     if (operand.size() != 4) {
00157         throw mx::Exception("fwrite requires four arguments.");
00158     }
00159     std::string &buf_var = operand[0].op;
00160     if (!program->isVariable(buf_var)) {
00161         throw mx::Exception("fwrite argument must be a variable pointer.");
00162     }
00163     mxvm::Variable &buf_v = program->getVariable(buf_var);
00164     if (buf_v.type != mxvm::VarType::VAR_POINTER) {
00165         throw mx::Exception("fwrite argument must be a pointer variable.");
00166     }
00167
00168     mxvm::Variable vsize, vcount, file_ptr;
00169     vsize = program->variableFromOperand(operand[1]);
00170     vcount = program->variableFromOperand(operand[2]);
00171     file_ptr = program->variableFromOperand(operand[3]);
00172
00173     if (vsize.type != mxvm::VarType::VAR_INTEGER || vcount.type != mxvm::VarType::VAR_INTEGER) {
00174         throw mx::Exception("Argument type mismatch expected integer for fwrite");
00175     }
00176     if (file_ptr.type != mxvm::VarType::VAR_POINTER && file_ptr.type != mxvm::VarType::VAR_EXTERN) {
00177         throw mx::Exception("Final argument for fwrite requires pointer");
00178     }
00179     FILE *fptr = reinterpret_cast<FILE *>(file_ptr.var_value.ptr_value);
00180     size_t result = fwrite(buf_v.var_value.ptr_value, vsize.var_value.int_value,
00181         vcount.var_value.int_value, fptr);
00182     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00183     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00184     program->vars["%rax"].var_value.int_value = result;
00185 }
00186 extern "C" void mxvm_io_fseek(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00187     if (operand.size() != 3) {
00188         throw mx::Exception("fseek requires three arguments got: " + std::to_string(operand.size()));
00189     }
00190     std::string &buf_var = operand[0].op;
00191     if (!program->isVariable(buf_var)) {
00192         throw mx::Exception("fseek argument must be a variable found: " + buf_var);
00193     }
00194     mxvm::Variable &buf_v = program->getVariable(buf_var);
00195     if (buf_v.type != mxvm::VarType::VAR_POINTER) {
00196         throw mx::Exception("fseek argument must be a pointer variable.");
00197     }
00198
00199     mxvm::Variable vsize, vcount, file_ptr;
00200     file_ptr = program->variableFromOperand(operand[0]);
00201     vsize = program->variableFromOperand(operand[1]);
00202     vcount = program->variableFromOperand(operand[2]);
00203

```

```

00204     if (vsize.type != mxvm::VarType::VAR_INTEGER || vcount.type != mxvm::VarType::VAR_INTEGER) {
00205         throw mx::Exception("Argument type mismatch expected integer for fread");
00206     }
00207     if (file_ptr.type != mxvm::VarType::VAR_POINTER && file_ptr.type != mxvm::VarType::VAR_EXTERN) {
00208         throw mx::Exception("Final argument for fread requires pointer");
00209     }
00210     FILE *fptr = reinterpret_cast<FILE *>(file_ptr.var_value.ptr_value);
00211     int result = fseek(fptr, vsize.var_value.int_value, vcount.var_value.int_value);
00212     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00213     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00214     program->vars["%rax"].var_value.int_value = result;
00215 }
00216
00217 extern "C" void mxvm_io_rand_number(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00218     if (operand.size() != 1) {
00219         throw mx::Exception("rand_number requires one argument (size).");
00220     }
00221     int64_t rsize = 1;
00222     if (program->isVariable(operand[0].op)) {
00223         mxvm::Variable &vsize = program->getVariable(operand[0].op);
00224         if (vsize.type != mxvm::VarType::VAR_INTEGER) {
00225             throw mx::Exception("rand_number argument must be integer (size).");
00226         }
00227         rsize = vsize.var_value.int_value;
00228     } else {
00229         if (operand[0].type == mxvm::OperandType::OP_CONSTANT)
00230             rsize = std::stoll(operand[0].op, nullptr, 0);
00231         else
00232             throw mx::Exception("rand_number expected variable or constant: " + operand[0].op);
00233     }
00234     if (rsize <= 0) {
00235         throw mx::Exception("rand_number: size must be positive, got " + std::to_string(rsize));
00236     }
00237     int result = std::rand() % rsize;
00238     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00239     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00240     program->vars["%rax"].var_value.int_value = result;
00241 }
00242
00243
00244 extern "C" void mxvm_io_seed_random(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00245     auto now = std::chrono::high_resolution_clock::now().time_since_epoch().count();
00246     uint64_t mixed = static_cast<uint64_t>(now);
00247     mixed ^= static_cast<uint64_t>(reinterpret_cast<std::uintptr_t>(program));
00248     mixed ^= (mixed >> 33);
00249     mixed *= 0xff51afd7ed558ccdULL;
00250     mixed ^= (mixed >> 33);
00251     mixed *= 0xc4ceb9f1e1a85ec53ULL;
00252     mixed ^= (mixed >> 33);
00253     std::srand(static_cast<unsigned int>(mixed));
00254     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00255     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00256     program->vars["%rax"].var_value.int_value = 0;
00257 }
00258
00259
00260 extern "C" void mxvm_io_feof(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00261     if (operand.size() != 1) {
00262         throw mx::Exception("feof requires one file pointer argument.");
00263     }
00264     std::string &file_var = operand[0].op;
00265     if (!program->isVariable(file_var)) {
00266         throw mx::Exception("feof argument must be a variable (file pointer).");
00267     }
00268     mxvm::Variable &file_v = program->getVariable(file_var);
00269     if (file_v.type != mxvm::VarType::VAR_POINTER) {
00270         throw mx::Exception("feof argument must be a pointer variable.");
00271     }
00272     FILE *fp = reinterpret_cast<FILE *>(file_v.var_value.ptr_value);
00273     int result = feof(fp) ? 1 : 0;
00274     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00275     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00276     program->vars["%rax"].var_value.int_value = result;
00277 }
00278
00279 extern "C" void mxvm_io_mxvm_fgets(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00280     if (operand.size() != 1) {
00281         throw mx::Exception("mxvm_fgets requires one file pointer argument.");
00282     }
00283     std::string &file_var = operand[0].op;
00284     if (!program->isVariable(file_var)) {
00285         throw mx::Exception("fgets argument must be a variable (file pointer).");
00286     }
00287     mxvm::Variable &file_v = program->getVariable(file_var);
00288     if (file_v.type != mxvm::VarType::VAR_POINTER) {
00289         throw mx::Exception("fgets argument must be a pointer variable.");
00290     }

```

```

00291     FILE *fp = reinterpret_cast<FILE *>(file_v.var_value.ptr_value);
00292     char buf[4096];
00293     char *res = fgets(buf, sizeof(buf), fp);
00294     if (res) {
00295         size_t len = strlen(buf);
00296         if (len > 0 && buf[len - 1] == '\n')
00297             buf[len - 1] = '\0';
00298         char *str = strdup(buf);
00299         program->vars["%rax"].type = mxvm::VarType::VAR_STRING;
00300         program->vars["%rax"].var_value.type = mxvm::VarType::VAR_STRING;
00301         program->vars["%rax"].var_value.str_value = str;
00302     } else {
00303         program->vars["%rax"].type = mxvm::VarType::VAR_STRING;
00304         program->vars["%rax"].var_value.type = mxvm::VarType::VAR_STRING;
00305         program->vars["%rax"].var_value.str_value = strdup("");
00306     }
00307 }
00308
00309 extern "C" void mxvm_io_fputs(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00310     if (operand.size() != 2) {
00311         throw mx::Exception("fputs requires two arguments (string, file).");
00312     }
00313     std::string str_val;
00314     if (program->isVariable(operand[0].op)) {
00315         mxvm::Variable &v = program->getVariable(operand[0].op);
00316         if (v.type == mxvm::VarType::VAR_STRING) {
00317             str_val = v.var_value.str_value;
00318         } else {
00319             throw mx::Exception("fputs first argument must be a string variable.");
00320         }
00321     } else {
00322         throw mx::Exception("fputs first argument must be a variable.");
00323     }
00324     std::string &file_var = operand[1].op;
00325     if (!program->isVariable(file_var)) {
00326         throw mx::Exception("fputs second argument must be a variable (file pointer).");
00327     }
00328     mxvm::Variable &file_v = program->getVariable(file_var);
00329     if (file_v.type != mxvm::VarType::VAR_POINTER) {
00330         throw mx::Exception("fputs second argument must be a pointer variable.");
00331     }
00332     FILE *fp = reinterpret_cast<FILE *>(file_v.var_value.ptr_value);
00333     int result = fputs(str_val.c_str(), fp);
00334     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00335     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00336     program->vars["%rax"].var_value.int_value = result;
00337 }

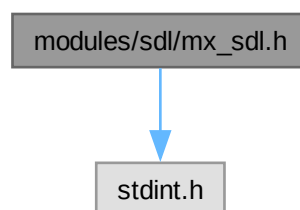
```

12.30 modules/sdl/mx_sdl.h File Reference

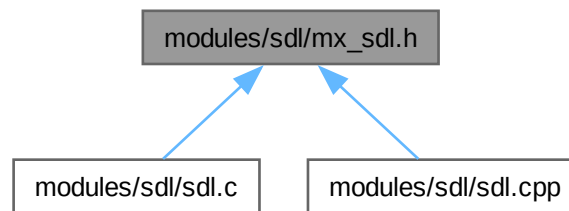
SDL2 module C API — function declarations for SDL2 bindings exported to MXVM programs.

#include <stdint.h>

Include dependency graph for mx_sdl.h:



This graph shows which files directly or indirectly include this file:



Functions

Core SDL functions

- `int64_t init` (void)
Initialize SDL2 subsystems.
- `void quit` (void)
Shut down SDL2 and release all resources.

Window management

- `int64_t create_window` (const char *title, int64_t x, int64_t y, int64_t w, int64_t h, int64_t flags)
Create an SDL window.
- `void destroy_window` (int64_t window_id)
Destroy a window by ID.
- `void set_window_title` (int64_t window_id, const char *title)
Set the title of a window.
- `void set_window_position` (int64_t window_id, int64_t x, int64_t y)
Set the position of a window.
- `void get_window_size` (int64_t window_id, int64_t *w, int64_t *h)
Get the size of a window.
- `void set_window_fullscreen` (int64_t window_id, int64_t fullscreen)
Set or clear fullscreen mode for a window.
- `void set_window_icon` (uint64_t window_id, const char *path)
Set the window icon from an image file.

Renderer management

- `int64_t create_renderer` (int64_t window_id, int64_t index, int64_t flags)
Create a hardware-accelerated renderer for a window.
- `void destroy_renderer` (int64_t renderer_id)
Destroy a renderer by ID.
- `void set_draw_color` (int64_t renderer_id, int64_t r, int64_t g, int64_t b, int64_t a)
Set the current draw color (RGBA).
- `void clear` (int64_t renderer_id)
Clear the renderer with the current draw color.
- `void present` (int64_t renderer_id)
Present the renderer's back buffer to the screen.
- `void get_renderer_output_size` (int64_t renderer_id, int64_t *w, int64_t *h)
Get the renderer output size.

Drawing functions

- void [draw_point](#) (int64_t renderer_id, int64_t x, int64_t y)
Draw a single pixel.
- void [draw_line](#) (int64_t renderer_id, int64_t x1, int64_t y1, int64_t x2, int64_t y2)
Draw a line between two points.
- void [draw_rect](#) (int64_t renderer_id, int64_t x, int64_t y, int64_t w, int64_t h)
Draw a rectangle outline.
- void [fill_rect](#) (int64_t renderer_id, int64_t x, int64_t y, int64_t w, int64_t h)
Draw a filled rectangle.

Event handling

- int64_t [poll_event](#) (void)
Poll for a pending SDL event.
- int64_t [get_event_type](#) (void)
Get the type of the last polled event.
- int64_t [get_key_code](#) (void)
Get the key code from the last keyboard event.
- int64_t [get_mouse_x](#) (void)
Get the mouse x coordinate from the last mouse event.
- int64_t [get_mouse_y](#) (void)
Get the mouse y coordinate from the last mouse event.
- int64_t [get_mouse_button](#) (void)
Get the mouse button from the last mouse event.

Surface functions

- int64_t [create_rgb_surface](#) (int64_t width, int64_t height, int64_t depth)
Create an RGB surface.
- void [free_surface](#) (int64_t surf_ptr)
Free a surface.
- int64_t [blit_surface](#) (int64_t src_ptr, int64_t dst_ptr, int64_t x, int64_t y)
Blit a source surface onto a destination surface at (x, y).

Mouse state functions

- int64_t [get_mouse_state](#) (int64_t *x, int64_t *y)
Get the current mouse state with pointer coordinates.
- int64_t [get_relative_mouse_state](#) (int64_t *x, int64_t *y)
Get the relative mouse state.
- int64_t [get_mouse_buttons](#) (void)
Get the current mouse button bitmask (pointerless variant).
- int64_t [get_relative_mouse_x](#) (void)
Get the relative mouse x delta (pointerless variant).
- int64_t [get_relative_mouse_y](#) (void)
Get the relative mouse y delta (pointerless variant).
- int64_t [get_relative_mouse_buttons](#) (void)
Get the relative mouse button bitmask (pointerless variant).

Keyboard state functions

- int64_t [get_keyboard_state](#) (int64_t *numkeys)
Get the keyboard state array.
- int64_t [is_key_pressed](#) (int64_t scancode)
Check whether a specific key is currently pressed.
- int64_t [get_num_keys](#) (void)
Get the total number of keyboard keys.

Clipboard functions

- void [set_clipboard_text](#) (const char *text)
Set the system clipboard text.

- const char * [get_clipboard_text](#) (void)
Get the system clipboard text.

Cursor functions

- void [show_cursor](#) (int64_t show)
Show or hide the mouse cursor.

Texture functions

- int64_t [create_texture](#) (int64_t renderer_id, int64_t format, int64_t access, int64_t w, int64_t h)
Create a blank texture.
- void [destroy_texture](#) (int64_t texture_id)
Destroy a texture.
- int64_t [load_texture](#) (int64_t renderer_id, const char *file_path)
Load a texture from an image file.
- int64_t [load_texture_color_key](#) (int64_t renderer_id, const char *file_path)
Load a BMP file as a texture with black color-key transparency.
- int64_t [load_texture_color_key_rgb](#) (int64_t renderer_id, const char *file_path, int64_t r, int64_t g, int64_t b)
Load a BMP file as a texture with a custom RGB color-key transparency.
- void [render_texture](#) (int64_t renderer_id, int64_t texture_id, int64_t src_x, int64_t src_y, int64_t src_w, int64_t src_h, int64_t dst_x, int64_t dst_y, int64_t dst_w, int64_t dst_h)
Render a portion of a texture to a destination rectangle.

Timing functions

- int64_t [get_ticks](#) (void)
Get the number of milliseconds since SDL initialization.
- void [delay](#) (int64_t ms)
Pause execution for the given number of milliseconds.

Audio functions

- int64_t [open_audio](#) (int64_t freq, int64_t format, int64_t channels, int64_t samples)
Open the audio device.
- void [close_audio](#) (void)
Close the audio device.
- void [pause_audio](#) (int64_t pause_on)
Pause or unpause audio playback.
- int64_t [load_wav](#) (const char *file_path, int64_t *audio_buf, int64_t *audio_len, int64_t *audio_spec)
Load a WAV file.
- void [free_wav](#) (int64_t audio_buf)
Free a loaded WAV buffer.
- int64_t [queue_audio](#) (const void *data, int64_t len)
Queue audio data for playback.
- int64_t [get_queued_audio_size](#) (void)
Get the number of bytes of queued audio.
- void [clear_queued_audio](#) (void)
Clear all queued audio data.

Texture pixel access

- int64_t [update_texture](#) (int64_t texture_id, const void *pixels, int64_t pitch)
Update a texture with new pixel data.
- int64_t [lock_texture](#) (int64_t texture_id, void **pixels, int64_t *pitch)
Lock a texture for direct pixel access.
- void [unlock_texture](#) (int64_t texture_id)
Unlock a previously locked texture.

Text rendering (SDL_ttf)

- `int64_t init_text` (void)
Initialize the SDL_ttf text rendering subsystem.
- `void quit_text` (void)
Shut down the SDL_ttf subsystem.
- `int64_t load_font` (const char *file, int64_t ptsize)
Load a TrueType font.
- `void draw_text` (int64_t renderer_id, int64_t font_id, const char *text, int64_t x, int64_t y, int64_t r, int64_t g, int64_t b, int64_t a)
Draw text with a font at the given position and color.

Render targets

- `int64_t create_render_target` (int64_t renderer_id, int64_t width, int64_t height)
Create an off-screen render target texture.
- `void set_render_target` (int64_t renderer_id, int64_t target_id)
Set the current render target (0 to render to the window).
- `void destroy_render_target` (int64_t target_id)
Destroy a render target texture.
- `void present_scaled` (int64_t renderer_id, int64_t target_id, int64_t scale_width, int64_t scale_height)
Present a render target scaled to the given dimensions.
- `void present_stretched` (int64_t renderer_id, int64_t target_id, int64_t dst_width, int64_t dst_height)
Present a render target stretched to the given dimensions.

12.30.1 Detailed Description

SDL2 module C API — function declarations for SDL2 bindings exported to MXVM programs.

Author

Jared Bruni

Definition in file [mx_sdl.h](#).

12.30.2 Function Documentation

12.30.2.1 blit_surface()

```
int64_t blit_surface (
    int64_t src_ptr,
    int64_t dst_ptr,
    int64_t x,
    int64_t y)
```

Blit a source surface onto a destination surface at (x, y).

Definition at line 684 of file [sdl.c](#).

Referenced by [mxvm_sdl_blit_surface\(\)](#).

12.30.2.2 clear()

```
void clear (
    int64_t renderer_id)
```

Clear the renderer with the current draw color.

Definition at line 212 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_clear\(\)](#).

12.30.2.3 clear_queued_audio()

```
void clear_queued_audio (
    void )
```

Clear all queued audio data.

Definition at line 510 of file [sdl.c](#).

Referenced by [mxvm_sdl_clear_queued_audio\(\)](#).

12.30.2.4 close_audio()

```
void close_audio (
    void )
```

Close the audio device.

Definition at line 470 of file [sdl.c](#).

Referenced by [mxvm_sdl_close_audio\(\)](#).

12.30.2.5 create_render_target()

```
int64_t create_render_target (
    int64_t renderer_id,
    int64_t width,
    int64_t height)
```

Create an off-screen render target texture.

Definition at line 146 of file [sdl.c](#).

References [g_render_target_count](#), [g_renderer_count](#), [g_renderers](#), [g_target_heights](#), and [g_target_widths](#).

Referenced by [mxvm_sdl_create_render_target\(\)](#).

12.30.2.6 create_renderer()

```
int64_t create_renderer (
    int64_t window_id,
    int64_t index,
    int64_t flags)
```

Create a hardware-accelerated renderer for a window.

Parameters

<i>window_id</i>	Target window
<i>index</i>	Rendering driver index (-1 for first available)
<i>flags</i>	SDL renderer flags

Returns

Renderer ID handle

Definition at line 121 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_create_renderer\(\)](#).

12.30.2.7 create_rgb_surface()

```
int64_t create_rgb_surface (
    int64_t width,
    int64_t height,
    int64_t depth)
```

Create an RGB surface.

Returns

Surface pointer cast to int64_t

Definition at line 663 of file [sdl.c](#).

Referenced by [mxvm_sdl_create_rgb_surface\(\)](#).

12.30.2.8 create_texture()

```
int64_t create_texture (
    int64_t renderer_id,
    int64_t format,
    int64_t access,
    int64_t w,
    int64_t h)
```

Create a blank texture.

Parameters

<i>renderer_id</i>	Renderer handle
<i>format</i>	Pixel format
<i>access</i>	Texture access pattern (static, streaming, target)
<i>w</i>	Width
<i>h</i>	Height

Returns

Texture ID handle

Definition at line 268 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_create_texture\(\)](#).

12.30.2.9 create_window()

```
int64_t create_window (
    const char * title,
    int64_t x,
    int64_t y,
    int64_t w,
    int64_t h,
    int64_t flags)
```

Create an SDL window.

Parameters

<i>title</i>	Window title
<i>x</i>	Initial x position
<i>y</i>	Initial y position
<i>w</i>	Width in pixels
<i>h</i>	Height in pixels
<i>flags</i>	SDL window flags

Returns

Window ID handle

Definition at line 86 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_create_window\(\)](#).

12.30.2.10 delay()

```
void delay (  
    int64_t ms)
```

Pause execution for the given number of milliseconds.

Definition at line 454 of file [sdl.c](#).

Referenced by [mxvm_sdl_delay\(\)](#).

12.30.2.11 destroy_render_target()

```
void destroy_render_target (  
    int64_t target_id)
```

Destroy a render target texture.

Definition at line 178 of file [sdl.c](#).

Referenced by [mxvm_sdl_destroy_render_target\(\)](#).

12.30.2.12 destroy_renderer()

```
void destroy_renderer (  
    int64_t renderer_id)
```

Destroy a renderer by ID.

Definition at line 139 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_destroy_renderer\(\)](#).

12.30.2.13 destroy_texture()

```
void destroy_texture (  
    int64_t texture_id)
```

Destroy a texture.

Definition at line 286 of file [sdl.c](#).

References [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_destroy_texture\(\)](#).

12.30.2.14 destroy_window()

```
void destroy_window (  
    int64_t window_id)
```

Destroy a window by ID.

Definition at line 106 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_destroy_window\(\)](#).

12.30.2.15 draw_line()

```
void draw_line (  
    int64_t renderer_id,  
    int64_t x1,  
    int64_t y1,  
    int64_t x2,  
    int64_t y2)
```

Draw a line between two points.

Definition at line 248 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_draw_line\(\)](#).

12.30.2.16 draw_point()

```
void draw_point (  
    int64_t renderer_id,
```

```
    int64_t x,  
    int64_t y)
```

Draw a single pixel.

Definition at line 242 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_draw_point\(\)](#).

12.30.2.17 draw_rect()

```
void draw_rect (  
    int64_t renderer_id,  
    int64_t x,  
    int64_t y,  
    int64_t w,  
    int64_t h)
```

Draw a rectangle outline.

Definition at line 254 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_draw_rect\(\)](#).

12.30.2.18 draw_text()

```
void draw_text (  
    int64_t renderer_id,  
    int64_t font_id,  
    const char * text,  
    int64_t x,  
    int64_t y,  
    int64_t r,  
    int64_t g,  
    int64_t b,  
    int64_t a)
```

Draw text with a font at the given position and color.

Definition at line 640 of file [sdl.c](#).

References [g_font_count](#), [g_fonts](#), [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_draw_text\(\)](#).

12.30.2.19 fill_rect()

```
void fill_rect (  
    int64_t renderer_id,  
    int64_t x,  
    int64_t y,  
    int64_t w,  
    int64_t h)
```

Draw a filled rectangle.

Definition at line 261 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_fill_rect\(\)](#).

12.30.2.20 free_surface()

```
void free_surface (  
    int64_t surf_ptr)
```

Free a surface.

Definition at line 679 of file [sdl.c](#).

Referenced by [mxvm_sdl_free_surface\(\)](#).

12.30.2.21 free_wav()

```
void free_wav (
    int64_t audio_buf)
```

Free a loaded WAV buffer.

Definition at line 496 of file [sdl.c](#).

Referenced by [mxvm_sdl_free_wav\(\)](#).

12.30.2.22 get_clipboard_text()

```
const char * get_clipboard_text (
    void )
```

Get the system clipboard text.

Definition at line 561 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_clipboard_text\(\)](#).

12.30.2.23 get_event_type()

```
int64_t get_event_type (
    void )
```

Get the type of the last polled event.

Definition at line 186 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_get_event_type\(\)](#).

12.30.2.24 get_key_code()

```
int64_t get_key_code (
    void )
```

Get the key code from the last keyboard event.

Definition at line 190 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_get_key_code\(\)](#).

12.30.2.25 get_keyboard_state()

```
int64_t get_keyboard_state (
    int64_t * numkeys)
```

Get the keyboard state array.

Parameters

out	<i>numkeys</i>	Receives number of keys
-----	----------------	-------------------------

Returns

Pointer to key state array cast to `int64_t`

Definition at line 536 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_keyboard_state\(\)](#).

12.30.2.26 get_mouse_button()

```
int64_t get_mouse_button (
    void )
```

Get the mouse button from the last mouse event.

Definition at line 202 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_get_mouse_button\(\)](#).

12.30.2.27 get_mouse_buttons()

```
int64_t get_mouse_buttons (
    void )
```

Get the current mouse button bitmask (pointerless variant).

Definition at line 514 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_mouse_buttons\(\)](#).

12.30.2.28 get_mouse_state()

```
int64_t get_mouse_state (
    int64_t * x,
    int64_t * y)
```

Get the current mouse state with pointer coordinates.

Parameters

out	<i>x</i>	Receives mouse x position
out	<i>y</i>	Receives mouse y position

Returns

Button bitmask

Definition at line 693 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_mouse_state\(\)](#).

12.30.2.29 get_mouse_x()

```
int64_t get_mouse_x (
    void )
```

Get the mouse x coordinate from the last mouse event.

Definition at line 194 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_get_mouse_x\(\)](#).

12.30.2.30 get_mouse_y()

```
int64_t get_mouse_y (
    void )
```

Get the mouse y coordinate from the last mouse event.

Definition at line 198 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_get_mouse_y\(\)](#).

12.30.2.31 get_num_keys()

```
int64_t get_num_keys (
    void )
```

Get the total number of keyboard keys.

Definition at line 551 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_num_keys\(\)](#).

12.30.2.32 get_queued_audio_size()

```
int64_t get_queued_audio_size (
    void )
```

Get the number of bytes of queued audio.

Definition at line 506 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_queued_audio_size\(\)](#).

12.30.2.33 get_relative_mouse_buttons()

```
int64_t get_relative_mouse_buttons (
    void )
```

Get the relative mouse button bitmask (pointerless variant).

Definition at line 531 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_relative_mouse_buttons\(\)](#).

12.30.2.34 get_relative_mouse_state()

```
int64_t get_relative_mouse_state (
    int64_t * x,
    int64_t * y)
```

Get the relative mouse state.

Parameters

out	<i>x</i>	Receives relative x delta
out	<i>y</i>	Receives relative y delta

Returns

Button bitmask

Definition at line 703 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_relative_mouse_state\(\)](#).

12.30.2.35 get_relative_mouse_x()

```
int64_t get_relative_mouse_x (
    void )
```

Get the relative mouse x delta (pointerless variant).

Definition at line 519 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_relative_mouse_x\(\)](#).

12.30.2.36 get_relative_mouse_y()

```
int64_t get_relative_mouse_y (
    void )
```

Get the relative mouse y delta (pointerless variant).

Definition at line 525 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_relative_mouse_y\(\)](#).

12.30.2.37 get_renderer_output_size()

```
void get_renderer_output_size (
    int64_t renderer_id,
    int64_t * w,
    int64_t * h)
```

Get the renderer output size.

Parameters

out	<i>w</i>	Receives width
out	<i>h</i>	Receives height

Definition at line 596 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_get_renderer_output_size\(\)](#).

12.30.2.38 get_ticks()

```
int64_t get_ticks (
    void )
```

Get the number of milliseconds since SDL initialization.

Definition at line 450 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_ticks\(\)](#).

12.30.2.39 get_window_size()

```
void get_window_size (
    int64_t window_id,
    int64_t * w,
    int64_t * h)
```

Get the size of a window.

Parameters

out	<i>w</i>	Receives width
out	<i>h</i>	Receives height

Definition at line 581 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_get_window_size\(\)](#).

12.30.2.40 init()

```
int64_t init (
    void )
```

Initialize SDL2 subsystems.

Returns

0 on success, negative on failure

Definition at line 26 of file [sdl.c](#).

Referenced by [action_interpret\(\)](#), and [mxvm_sdl_init\(\)](#).

12.30.2.41 init_text()

```
int64_t init_text (
    void )
```

Initialize the SDL_ttf text rendering subsystem.

Definition at line 609 of file [sdl.c](#).

Referenced by [mxvm_sdl_init_text\(\)](#).

12.30.2.42 is_key_pressed()

```
int64_t is_key_pressed (
    int64_t scancode)
```

Check whether a specific key is currently pressed.

Parameters

<i>scancode</i>	SDL scancode
-----------------	--------------

Returns

Non-zero if pressed

Definition at line 543 of file [sdl.c](#).

Referenced by [mxvm_sdl_is_key_pressed\(\)](#).

12.30.2.43 load_font()

```
int64_t load_font (
    const char * file,
    int64_t psize)
```

Load a TrueType font.

Parameters

<i>file</i>	Font file path
<i>psize</i>	Font point size

Returns

Font ID handle

Definition at line 626 of file [sdl.c](#).

References [g_font_count](#), and [g_fonts](#).

Referenced by [mxvm_sdl_load_font\(\)](#).

12.30.2.44 load_texture()

```
int64_t load_texture (
    int64_t renderer_id,
    const char * file_path)
```

Load a texture from an image file.

Parameters

<i>renderer↔ _id</i>	Renderer handle
<i>file_path</i>	Path to the image file

Returns

Texture ID handle

Load a texture from an image file.

Loads the bitmap via [SDL_LoadBMP](#), sets pure black (0,0,0) as the transparent color key with [SDL_SetColorKey](#), then creates an accelerated texture from the surface. The surface is freed after texture creation.

Parameters

<i>renderer↔ _id</i>	Index into the renderer array.
<i>file_path</i>	Path to the .bmp file.

Returns

Texture index on success, -1 on bad renderer, -2 on load failure, -3 on texture creation failure.

Definition at line 306 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_load_texture\(\)](#).

12.30.2.45 load_texture_color_key()

```
int64_t load_texture_color_key (
    int64_t renderer_id,
    const char * file_path)
```

Load a BMP file as a texture with black color-key transparency.

Definition at line 330 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_load_texture_color_key\(\)](#).

12.30.2.46 load_texture_color_key_rgb()

```
int64_t load_texture_color_key_rgb (
    int64_t renderer_id,
    const char * file_path,
    int64_t r,
    int64_t g,
    int64_t b)
```

Load a BMP file as a texture with a custom RGB color-key transparency.

Load a BMP file as a texture with a custom RGB color-key transparency.

Loads the bitmap via [SDL_LoadBMP](#), sets the specified (r, g, b) color as the transparent color key with [SDL_SetColorKey](#), then creates an accelerated texture from the surface. The surface is freed after texture creation.

Parameters

<i>renderer_id</i>	Index into the renderer array.
<i>file_path</i>	Path to the .bmp file.
<i>r</i>	Red component of the color key (0–255).
<i>g</i>	Green component of the color key (0–255).
<i>b</i>	Blue component of the color key (0–255).

Returns

Texture index on success, -1 on bad renderer or allocation failure, -2 on load failure, -3 on texture creation failure.

Definition at line 371 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_load_texture_color_key_rgb\(\)](#).

12.30.2.47 load_wav()

```
int64_t load_wav (
    const char * file_path,
    int64_t * audio_buf,
    int64_t * audio_len,
    int64_t * audio_spec)
```

Load a WAV file.

Parameters

	<i>file_path</i>	Path to the WAV file
out	<i>audio_buf</i>	Receives the audio buffer pointer
out	<i>audio_len</i>	Receives the audio length in bytes
out	<i>audio_spec</i>	Receives the audio spec pointer

Returns

Non-zero on success

Definition at line 478 of file [sdl.c](#).

Referenced by [mxvm_sdl_load_wav\(\)](#).

12.30.2.48 lock_texture()

```
int64_t lock_texture (  
    int64_t texture_id,  
    void ** pixels,  
    int64_t * pitch)
```

Lock a texture for direct pixel access.

Parameters

out	<i>pixels</i>	Receives pointer to pixel data
out	<i>pitch</i>	Receives row pitch in bytes

Definition at line 434 of file [sdl.c](#).

References [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_lock_texture\(\)](#).

12.30.2.49 open_audio()

```
int64_t open_audio (  
    int64_t freq,  
    int64_t format,  
    int64_t channels,  
    int64_t samples)
```

Open the audio device.

Parameters

<i>freq</i>	Sample rate (e.g. 44100)
<i>format</i>	Audio format
<i>channels</i>	Number of channels
<i>samples</i>	Buffer size in samples

Returns

0 on success

Definition at line 458 of file [sdl.c](#).

Referenced by [mxvm_sdl_open_audio\(\)](#).

12.30.2.50 `pause_audio()`

```
void pause_audio (
    int64_t pause_on)
```

Pause or unpause audio playback.

Definition at line 474 of file [sdl.c](#).

Referenced by [mxvm_sdl_pause_audio\(\)](#).

12.30.2.51 `poll_event()`

```
int64_t poll_event (
    void )
```

Poll for a pending SDL event.

Returns

1 if an event was available, 0 otherwise

Definition at line 182 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_poll_event\(\)](#).

12.30.2.52 `present()`

```
void present (
    int64_t renderer_id)
```

Present the renderer's back buffer to the screen.

Definition at line 218 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_present\(\)](#).

12.30.2.53 `present_scaled()`

```
void present_scaled (
    int64_t renderer_id,
    int64_t target_id,
    int64_t scale_width,
    int64_t scale_height)
```

Present a render target scaled to the given dimensions.

Definition at line 224 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_present_scaled\(\)](#).

12.30.2.54 `present_stretched()`

```
void present_stretched (
    int64_t renderer_id,
    int64_t target_id,
    int64_t dst_width,
    int64_t dst_height)
```

Present a render target stretched to the given dimensions.

Definition at line 233 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_present_stretched\(\)](#).

12.30.2.55 `queue_audio()`

```
int64_t queue_audio (
    const void * data,
    int64_t len)
```

Queue audio data for playback.

Definition at line 502 of file [sdl.c](#).

Referenced by [mxvm_sdl_queue_audio\(\)](#).

12.30.2.56 quit()

```
void quit (  
    void )
```

Shut down SDL2 and release all resources.

Definition at line 34 of file [sdl.c](#).

References [g_font_count](#), [g_fonts](#), [g_render_target_count](#), [g_renderer_count](#), [g_renderers](#), [g_target_heights](#), [g_target_widths](#), [g_texture_count](#), [g_textures](#), [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_quit\(\)](#).

12.30.2.57 quit_text()

```
void quit_text (  
    void )
```

Shut down the SDL_ttf subsystem.

Definition at line 613 of file [sdl.c](#).

References [g_font_count](#), and [g_fonts](#).

Referenced by [mxvm_sdl_quit_text\(\)](#).

12.30.2.58 render_texture()

```
void render_texture (  
    int64_t  renderer_id,  
    int64_t  texture_id,  
    int64_t  src_x,  
    int64_t  src_y,  
    int64_t  src_w,  
    int64_t  src_h,  
    int64_t  dst_x,  
    int64_t  dst_y,  
    int64_t  dst_w,  
    int64_t  dst_h)
```

Render a portion of a texture to a destination rectangle.

Definition at line 397 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_render_texture\(\)](#).

12.30.2.59 set_clipboard_text()

```
void set_clipboard_text (  
    const char * text)
```

Set the system clipboard text.

Definition at line 557 of file [sdl.c](#).

Referenced by [mxvm_sdl_set_clipboard_text\(\)](#).

12.30.2.60 set_draw_color()

```
void set_draw_color (  
    int64_t  renderer_id,  
    int64_t  r,  
    int64_t  g,  
    int64_t  b,  
    int64_t  a)
```

Set the current draw color (RGBA).

Definition at line 206 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_set_draw_color\(\)](#).

12.30.2.61 **set_render_target()**

```
void set_render_target (  
    int64_t render_id,  
    int64_t target_id)
```

Set the current render target (0 to render to the window).

Definition at line 173 of file [sdl.c](#).

Referenced by [mxvm_sdl_set_render_target\(\)](#).

12.30.2.62 **set_window_fullscreen()**

```
void set_window_fullscreen (  
    int64_t window_id,  
    int64_t fullscreen)
```

Set or clear fullscreen mode for a window.

Definition at line 590 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_set_window_fullscreen\(\)](#).

12.30.2.63 **set_window_icon()**

```
void set_window_icon (  
    uint64_t window_id,  
    const char * path)
```

Set the window icon from an image file.

Definition at line 113 of file [sdl.c](#).

References [g_windows](#).

Referenced by [mxvm_sdl_set_window_icon\(\)](#).

12.30.2.64 **set_window_position()**

```
void set_window_position (  
    int64_t window_id,  
    int64_t x,  
    int64_t y)
```

Set the position of a window.

Definition at line 575 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_set_window_position\(\)](#).

12.30.2.65 **set_window_title()**

```
void set_window_title (  
    int64_t window_id,  
    const char * title)
```

Set the title of a window.

Definition at line 569 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_set_window_title\(\)](#).

12.30.2.66 **show_cursor()**

```
void show_cursor (  
    int64_t show)
```

Show or hide the mouse cursor.

Definition at line 605 of file [sdl.c](#).

Referenced by [mxvm_sdl_show_cursor\(\)](#).

12.30.2.67 unlock_texture()

```
void unlock_texture (
    int64_t texture_id)
```

Unlock a previously locked texture.

Definition at line 444 of file [sdl.c](#).

References [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_unlock_texture\(\)](#).

12.30.2.68 update_texture()

```
int64_t update_texture (
    int64_t texture_id,
    const void * pixels,
    int64_t pitch)
```

Update a texture with new pixel data.

Definition at line 427 of file [sdl.c](#).

References [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_update_texture\(\)](#).

12.31 mx_sdl.h

[Go to the documentation of this file.](#)

```
00001
00006 #ifndef __SDL2__H__
00007 #define __SDL2__H__
00008
00009 #include <stdint.h>
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00017
00021 int64_t init(void);
00022
00024 void quit(void);
00026
00029
00039 int64_t create_window(const char *title, int64_t x, int64_t y, int64_t w, int64_t h, int64_t flags);
00040
00042 void destroy_window(int64_t window_id);
00043
00045 void set_window_title(int64_t window_id, const char *title);
00046
00048 void set_window_position(int64_t window_id, int64_t x, int64_t y);
00049
00054 void get_window_size(int64_t window_id, int64_t *w, int64_t *h);
00055
00057 void set_window_fullscreen(int64_t window_id, int64_t fullscreen);
00058
00060 void set_window_icon(uint64_t window_id, const char *path);
00064
00071 int64_t create_renderer(int64_t window_id, int64_t index, int64_t flags);
00072
00074 void destroy_renderer(int64_t renderer_id);
00075
00077 void set_draw_color(int64_t renderer_id, int64_t r, int64_t g, int64_t b, int64_t a);
00078
00080 void clear(int64_t renderer_id);
00081
00083 void present(int64_t renderer_id);
00084
00089 void get_renderer_output_size(int64_t renderer_id, int64_t *w, int64_t *h);
00091
00094
00096 void draw_point(int64_t renderer_id, int64_t x, int64_t y);
00097
00099 void draw_line(int64_t renderer_id, int64_t x1, int64_t y1, int64_t x2, int64_t y2);
00100
00102 void draw_rect(int64_t renderer_id, int64_t x, int64_t y, int64_t w, int64_t h);
00103
00105 void fill_rect(int64_t renderer_id, int64_t x, int64_t y, int64_t w, int64_t h);
00107
00110
```

```
00114 int64_t poll_event(void);
00115
00117 int64_t get_event_type(void);
00118
00120 int64_t get_key_code(void);
00121
00123 int64_t get_mouse_x(void);
00124
00126 int64_t get_mouse_y(void);
00127
00129 int64_t get_mouse_button(void);
00131
00134
00138 int64_t create_rgb_surface(int64_t width, int64_t height, int64_t depth);
00139
00141 void free_surface(int64_t surf_ptr);
00142
00144 int64_t blit_surface(int64_t src_ptr, int64_t dst_ptr, int64_t x, int64_t y);
00146
00149
00155 int64_t get_mouse_state(int64_t *x, int64_t *y);
00156
00162 int64_t get_relative_mouse_state(int64_t *x, int64_t *y);
00163
00165 int64_t get_mouse_buttons(void);
00166
00168 int64_t get_relative_mouse_x(void);
00169
00171 int64_t get_relative_mouse_y(void);
00172
00174 int64_t get_relative_mouse_buttons(void);
00176
00179
00184 int64_t get_keyboard_state(int64_t *numkeys);
00185
00190 int64_t is_key_pressed(int64_t scancode);
00191
00193 int64_t get_num_keys(void);
00195
00198
00200 void set_clipboard_text(const char *text);
00201
00203 const char *get_clipboard_text(void);
00205
00208
00210 void show_cursor(int64_t show);
00212
00215
00224 int64_t create_texture(int64_t renderer_id, int64_t format, int64_t access, int64_t w, int64_t h);
00225
00227 void destroy_texture(int64_t texture_id);
00228
00234 int64_t load_texture(int64_t renderer_id, const char *file_path);
00235
00237 int64_t load_texture_color_key(int64_t renderer_id, const char *file_path);
00238
00240 int64_t load_texture_color_key_rgb(int64_t renderer_id, const char *file_path, int64_t r, int64_t g,
int64_t b);
00241
00243 void render_texture(int64_t renderer_id, int64_t texture_id, int64_t src_x, int64_t src_y, int64_t
src_w, int64_t src_h, int64_t dst_x, int64_t dst_y, int64_t dst_w, int64_t dst_h);
00245
00248
00250 int64_t get_ticks(void);
00251
00253 void delay(int64_t ms);
00255
00265 int64_t open_audio(int64_t freq, int64_t format, int64_t channels, int64_t samples);
00266
00268 void close_audio(void);
00269
00271 void pause_audio(int64_t pause_on);
00272
00280 int64_t load_wav(const char *file_path, int64_t *audio_buf, int64_t *audio_len, int64_t *audio_spec);
00281
00283 void free_wav(int64_t audio_buf);
00284
00286 int64_t queue_audio(const void *data, int64_t len);
00287
00289 int64_t get_queued_audio_size(void);
00290
00292 void clear_queued_audio(void);
00294
00297
00299 int64_t update_texture(int64_t texture_id, const void *pixels, int64_t pitch);
00300
00305 int64_t lock_texture(int64_t texture_id, void **pixels, int64_t *pitch);
```

```

00306
00308 void unlock_texture(int64_t texture_id);
00310
00313
00315 int64_t init_text(void);
00316
00318 void quit_text(void);
00319
00325 int64_t load_font(const char *file, int64_t ptsize);
00326
00328 void draw_text(int64_t renderer_id, int64_t font_id, const char *text, int64_t x, int64_t y, int64_t
r, int64_t g, int64_t b, int64_t a);
00330
00333
00335 int64_t create_render_target(int64_t renderer_id, int64_t width, int64_t height);
00336
00338 void set_render_target(int64_t renderer_id, int64_t target_id);
00339
00341 void destroy_render_target(int64_t target_id);
00342
00344 void present_scaled(int64_t renderer_id, int64_t target_id, int64_t scale_width, int64_t
scale_height);
00345
00347 void present_stretched(int64_t renderer_id, int64_t target_id, int64_t dst_width, int64_t dst_height);
00349
00350 #ifdef __cplusplus
00351 }
00352 #endif
00353
00354 #endif

```

12.32 modules/sdl/sdl.c File Reference

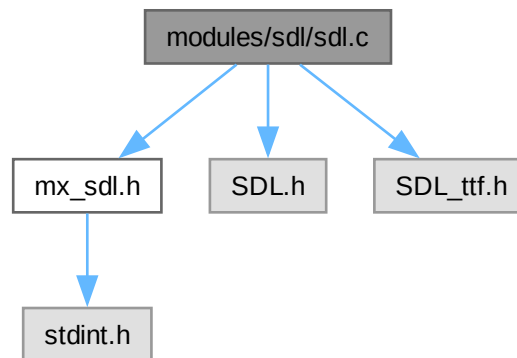
SDL2 module C implementation — window, renderer, texture, and event handling.

```
#include "mx_sdl.h"
```

```
#include "SDL.h"
```

```
#include "SDL_ttf.h"
```

Include dependency graph for sdl.c:



Functions

- int64_t [init](#) (void)
Initialize SDL2 subsystems.
- void [quit](#) (void)
Shut down SDL2 and release all resources.
- int64_t [create_window](#) (const char *title, int64_t x, int64_t y, int64_t w, int64_t h, int64_t flags)
Create an SDL window.

- void [destroy_window](#) (int64_t window_id)
Destroy a window by ID.
- void [set_window_icon](#) (uint64_t window_id, const char *path)
Set the window icon from an image file.
- int64_t [create_renderer](#) (int64_t window_id, int64_t index, int64_t flags)
Create a hardware-accelerated renderer for a window.
- void [destroy_renderer](#) (int64_t renderer_id)
Destroy a renderer by ID.
- int64_t [create_render_target](#) (int64_t renderer_id, int64_t width, int64_t height)
Create an off-screen render target texture.
- void [set_render_target](#) (int64_t renderer_id, int64_t target_id)
Set the current render target (0 to render to the window).
- void [destroy_render_target](#) (int64_t target_id)
Destroy a render target texture.
- int64_t [poll_event](#) (void)
Poll for a pending SDL event.
- int64_t [get_event_type](#) (void)
Get the type of the last polled event.
- int64_t [get_key_code](#) (void)
Get the key code from the last keyboard event.
- int64_t [get_mouse_x](#) (void)
Get the mouse x coordinate from the last mouse event.
- int64_t [get_mouse_y](#) (void)
Get the mouse y coordinate from the last mouse event.
- int64_t [get_mouse_button](#) (void)
Get the mouse button from the last mouse event.
- void [set_draw_color](#) (int64_t renderer_id, int64_t r, int64_t g, int64_t b, int64_t a)
Set the current draw color (RGBA).
- void [clear](#) (int64_t renderer_id)
Clear the renderer with the current draw color.
- void [present](#) (int64_t renderer_id)
Present the renderer's back buffer to the screen.
- void [present_scaled](#) (int64_t renderer_id, int64_t target_id, int64_t scale_width, int64_t scale_height)
Present a render target scaled to the given dimensions.
- void [present_stretched](#) (int64_t renderer_id, int64_t target_id, int64_t dst_width, int64_t dst_height)
Present a render target stretched to the given dimensions.
- void [draw_point](#) (int64_t renderer_id, int64_t x, int64_t y)
Draw a single pixel.
- void [draw_line](#) (int64_t renderer_id, int64_t x1, int64_t y1, int64_t x2, int64_t y2)
Draw a line between two points.
- void [draw_rect](#) (int64_t renderer_id, int64_t x, int64_t y, int64_t w, int64_t h)
Draw a rectangle outline.
- void [fill_rect](#) (int64_t renderer_id, int64_t x, int64_t y, int64_t w, int64_t h)
Draw a filled rectangle.
- int64_t [create_texture](#) (int64_t renderer_id, int64_t format, int64_t access, int64_t w, int64_t h)
Create a blank texture.
- void [destroy_texture](#) (int64_t texture_id)
Destroy a texture.
- int64_t [load_texture](#) (int64_t renderer_id, const char *file_path)
Load a BMP file as an SDL texture with black color-key transparency.
- int64_t [load_texture_color_key](#) (int64_t renderer_id, const char *file_path)

- Load a BMP file as a texture with black color-key transparency.*

 - `int64_t load_texture_color_key_rgb` (`int64_t` renderer_id, `const char *`file_path, `int64_t` r, `int64_t` g, `int64_t` b)
- Load a BMP file as an SDL texture with a custom RGB color-key transparency.*

 - `void render_texture` (`int64_t` renderer_id, `int64_t` texture_id, `int64_t` src_x, `int64_t` src_y, `int64_t` src_w, `int64_t` src_h, `int64_t` dst_x, `int64_t` dst_y, `int64_t` dst_w, `int64_t` dst_h)
- Render a portion of a texture to a destination rectangle.*

 - `int64_t update_texture` (`int64_t` texture_id, `const void *`pixels, `int64_t` pitch)
- Update a texture with new pixel data.*

 - `int64_t lock_texture` (`int64_t` texture_id, `void **`pixels, `int64_t *`pitch)
- Lock a texture for direct pixel access.*

 - `void unlock_texture` (`int64_t` texture_id)
- Unlock a previously locked texture.*

 - `int64_t get_ticks` (`void`)
- Get the number of milliseconds since SDL initialization.*

 - `void delay` (`int64_t` ms)
- Pause execution for the given number of milliseconds.*

 - `int64_t open_audio` (`int64_t` freq, `int64_t` format, `int64_t` channels, `int64_t` samples)
- Open the audio device.*

 - `void close_audio` (`void`)
- Close the audio device.*

 - `void pause_audio` (`int64_t` pause_on)
- Pause or unpause audio playback.*

 - `int64_t load_wav` (`const char *`file_path, `int64_t *`audio_buf, `int64_t *`audio_len, `int64_t *`audio_spec)
- Load a WAV file.*

 - `void free_wav` (`int64_t` audio_buf)
- Free a loaded WAV buffer.*

 - `int64_t queue_audio` (`const void *`data, `int64_t` len)
- Queue audio data for playback.*

 - `int64_t get_queued_audio_size` (`void`)
- Get the number of bytes of queued audio.*

 - `void clear_queued_audio` (`void`)
- Clear all queued audio data.*

 - `int64_t get_mouse_buttons` (`void`)
- Get the current mouse button bitmask (pointerless variant).*

 - `int64_t get_relative_mouse_x` (`void`)
- Get the relative mouse x delta (pointerless variant).*

 - `int64_t get_relative_mouse_y` (`void`)
- Get the relative mouse y delta (pointerless variant).*

 - `int64_t get_relative_mouse_buttons` (`void`)
- Get the relative mouse button bitmask (pointerless variant).*

 - `int64_t get_keyboard_state` (`int64_t *`numkeys)
- Get the keyboard state array.*

 - `int64_t is_key_pressed` (`int64_t` scancode)
- Check whether a specific key is currently pressed.*

 - `int64_t get_num_keys` (`void`)
- Get the total number of keyboard keys.*

 - `void set_clipboard_text` (`const char *`text)
- Set the system clipboard text.*

 - `const char *``get_clipboard_text` (`void`)
- Get the system clipboard text.*

 - `void set_window_title` (`int64_t` window_id, `const char *`title)

- Set the title of a window.*

 - void [set_window_position](#) (int64_t window_id, int64_t x, int64_t y)
- Set the position of a window.*

 - void [get_window_size](#) (int64_t window_id, int64_t *w, int64_t *h)
- Get the size of a window.*

 - void [set_window_fullscreen](#) (int64_t window_id, int64_t fullscreen)
- Set or clear fullscreen mode for a window.*

 - void [get_renderer_output_size](#) (int64_t renderer_id, int64_t *w, int64_t *h)
- Get the renderer output size.*

 - void [show_cursor](#) (int64_t show)
- Show or hide the mouse cursor.*

 - int64_t [init_text](#) (void)
- Initialize the SDL_ttf text rendering subsystem.*

 - void [quit_text](#) (void)
- Shut down the SDL_ttf subsystem.*

 - int64_t [load_font](#) (const char *file, int64_t ptsize)
- Load a TrueType font.*

 - void [draw_text](#) (int64_t renderer_id, int64_t font_id, const char *text, int64_t x, int64_t y, int64_t r, int64_t g, int64_t b, int64_t a)
- Draw text with a font at the given position and color.*

 - int64_t [create_rgb_surface](#) (int64_t width, int64_t height, int64_t depth)
- Create an RGB surface.*

 - void [free_surface](#) (int64_t surf_ptr)
- Free a surface.*

 - int64_t [blit_surface](#) (int64_t src_ptr, int64_t dst_ptr, int64_t x, int64_t y)
- Blit a source surface onto a destination surface at (x, y).*

 - int64_t [get_mouse_state](#) (int64_t *x, int64_t *y)
- Get the current mouse state with pointer coordinates.*

 - int64_t [get_relative_mouse_state](#) (int64_t *x, int64_t *y)
- Get the relative mouse state.*

Variables

- static SDL_Event [g_event](#)
- static SDL_Window ** [g_windows](#) = NULL
- static SDL_Renderer ** [g_renderers](#) = NULL
- static SDL_Texture ** [g_textures](#) = NULL
- static int64_t [g_window_count](#) = 0
- static int64_t [g_renderer_count](#) = 0
- static int64_t [g_texture_count](#) = 0
- static TTF_Font ** [g_fonts](#) = NULL
- static int64_t [g_font_count](#) = 0
- static SDL_Texture ** [g_render_targets](#) = NULL
- static int64_t * [g_target_widths](#) = NULL
- static int64_t * [g_target_heights](#) = NULL
- static int64_t [g_render_target_count](#) = 0

12.32.1 Detailed Description

SDL2 module C implementation — window, renderer, texture, and event handling.

Author

Jared Bruni

Definition in file [sdl.c](#).

12.32.2 Function Documentation

12.32.2.1 blit_surface()

```
int64_t blit_surface (
    int64_t src_ptr,
    int64_t dst_ptr,
    int64_t x,
    int64_t y)
```

Blit a source surface onto a destination surface at (x, y).

Definition at line 684 of file [sdl.c](#).

Referenced by [mxvm_sdl_blit_surface\(\)](#).

12.32.2.2 clear()

```
void clear (
    int64_t renderer_id)
```

Clear the renderer with the current draw color.

Definition at line 212 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_clear\(\)](#).

12.32.2.3 clear_queued_audio()

```
void clear_queued_audio (
    void )
```

Clear all queued audio data.

Definition at line 510 of file [sdl.c](#).

Referenced by [mxvm_sdl_clear_queued_audio\(\)](#).

12.32.2.4 close_audio()

```
void close_audio (
    void )
```

Close the audio device.

Definition at line 470 of file [sdl.c](#).

Referenced by [mxvm_sdl_close_audio\(\)](#).

12.32.2.5 create_render_target()

```
int64_t create_render_target (
    int64_t renderer_id,
    int64_t width,
    int64_t height)
```

Create an off-screen render target texture.

Definition at line 146 of file [sdl.c](#).

References [g_render_target_count](#), [g_renderer_count](#), [g_renderers](#), [g_target_heights](#), and [g_target_widths](#).

Referenced by [mxvm_sdl_create_render_target\(\)](#).

12.32.2.6 create_renderer()

```
int64_t create_renderer (
    int64_t window_id,
    int64_t index,
    int64_t flags)
```

Create a hardware-accelerated renderer for a window.

Parameters

<i>window</i> ↔ <i>_id</i>	Target window
<i>index</i>	Rendering driver index (-1 for first available)
<i>flags</i>	SDL renderer flags

Returns

Renderer ID handle

Definition at line 121 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_create_renderer\(\)](#).

12.32.2.7 create_rgb_surface()

```
int64_t create_rgb_surface (
    int64_t width,
    int64_t height,
    int64_t depth)
```

Create an RGB surface.

Returns

Surface pointer cast to int64_t

Definition at line 663 of file [sdl.c](#).

Referenced by [mxvm_sdl_create_rgb_surface\(\)](#).

12.32.2.8 create_texture()

```
int64_t create_texture (
    int64_t renderer_id,
    int64_t format,
    int64_t access,
    int64_t w,
    int64_t h)
```

Create a blank texture.

Parameters

<i>renderer</i> ↔ <i>_id</i>	Renderer handle
<i>format</i>	Pixel format
<i>access</i>	Texture access pattern (static, streaming, target)
<i>w</i>	Width
<i>h</i>	Height

Returns

Texture ID handle

Definition at line 268 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_create_texture\(\)](#).

12.32.2.9 create_window()

```
int64_t create_window (
    const char * title,
    int64_t x,
    int64_t y,
    int64_t w,
    int64_t h,
    int64_t flags)
```

Create an SDL window.

Parameters

<i>title</i>	Window title
<i>x</i>	Initial x position
<i>y</i>	Initial y position
<i>w</i>	Width in pixels
<i>h</i>	Height in pixels
<i>flags</i>	SDL window flags

Returns

Window ID handle

Definition at line 86 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_create_window\(\)](#).

12.32.2.10 delay()

```
void delay (
    int64_t ms)
```

Pause execution for the given number of milliseconds.

Definition at line 454 of file [sdl.c](#).

Referenced by [mxvm_sdl_delay\(\)](#).

12.32.2.11 destroy_render_target()

```
void destroy_render_target (
    int64_t target_id)
```

Destroy a render target texture.

Definition at line 178 of file [sdl.c](#).

Referenced by [mxvm_sdl_destroy_render_target\(\)](#).

12.32.2.12 destroy_renderer()

```
void destroy_renderer (
    int64_t renderer_id)
```

Destroy a renderer by ID.

Definition at line 139 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_destroy_renderer\(\)](#).

12.32.2.13 destroy_texture()

```
void destroy_texture (
    int64_t texture_id)
```

Destroy a texture.

Definition at line 286 of file [sdl.c](#).

References [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_destroy_texture\(\)](#).

12.32.2.14 [destroy_window\(\)](#)

```
void destroy_window (  
    int64_t window_id)
```

Destroy a window by ID.

Definition at line 106 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_destroy_window\(\)](#).

12.32.2.15 [draw_line\(\)](#)

```
void draw_line (  
    int64_t renderer_id,  
    int64_t x1,  
    int64_t y1,  
    int64_t x2,  
    int64_t y2)
```

Draw a line between two points.

Definition at line 248 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_draw_line\(\)](#).

12.32.2.16 [draw_point\(\)](#)

```
void draw_point (  
    int64_t renderer_id,  
    int64_t x,  
    int64_t y)
```

Draw a single pixel.

Definition at line 242 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_draw_point\(\)](#).

12.32.2.17 [draw_rect\(\)](#)

```
void draw_rect (  
    int64_t renderer_id,  
    int64_t x,  
    int64_t y,  
    int64_t w,  
    int64_t h)
```

Draw a rectangle outline.

Definition at line 254 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_draw_rect\(\)](#).

12.32.2.18 [draw_text\(\)](#)

```
void draw_text (  
    int64_t renderer_id,  
    int64_t font_id,  
    const char * text,  
    int64_t x,  
    int64_t y,
```

```
int64_t r,  
int64_t g,  
int64_t b,  
int64_t a)
```

Draw text with a font at the given position and color.

Definition at line 640 of file [sdl.c](#).

References [g_font_count](#), [g_fonts](#), [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_draw_text\(\)](#).

12.32.2.19 fill_rect()

```
void fill_rect (  
    int64_t renderer_id,  
    int64_t x,  
    int64_t y,  
    int64_t w,  
    int64_t h)
```

Draw a filled rectangle.

Definition at line 261 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_fill_rect\(\)](#).

12.32.2.20 free_surface()

```
void free_surface (  
    int64_t surf_ptr)
```

Free a surface.

Definition at line 679 of file [sdl.c](#).

Referenced by [mxvm_sdl_free_surface\(\)](#).

12.32.2.21 free_wav()

```
void free_wav (  
    int64_t audio_buf)
```

Free a loaded WAV buffer.

Definition at line 496 of file [sdl.c](#).

Referenced by [mxvm_sdl_free_wav\(\)](#).

12.32.2.22 get_clipboard_text()

```
const char * get_clipboard_text (  
    void )
```

Get the system clipboard text.

Definition at line 561 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_clipboard_text\(\)](#).

12.32.2.23 get_event_type()

```
int64_t get_event_type (  
    void )
```

Get the type of the last polled event.

Definition at line 186 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_get_event_type\(\)](#).

12.32.2.24 get_key_code()

```
int64_t get_key_code (  
    void )
```

Get the key code from the last keyboard event.
Definition at line 190 of file [sdl.c](#).
References [g_event](#).
Referenced by [mxvm_sdl_get_key_code\(\)](#).

12.32.2.25 get_keyboard_state()

```
int64_t get_keyboard_state (  
    int64_t * numkeys)
```

Get the keyboard state array.

Parameters

out	<i>numkeys</i>	Receives number of keys
-----	----------------	-------------------------

Returns

Pointer to key state array cast to `int64_t`

Definition at line 536 of file [sdl.c](#).
Referenced by [mxvm_sdl_get_keyboard_state\(\)](#).

12.32.2.26 get_mouse_button()

```
int64_t get_mouse_button (  
    void )
```

Get the mouse button from the last mouse event.
Definition at line 202 of file [sdl.c](#).
References [g_event](#).
Referenced by [mxvm_sdl_get_mouse_button\(\)](#).

12.32.2.27 get_mouse_buttons()

```
int64_t get_mouse_buttons (  
    void )
```

Get the current mouse button bitmask (pointerless variant).
Definition at line 514 of file [sdl.c](#).
Referenced by [mxvm_sdl_get_mouse_buttons\(\)](#).

12.32.2.28 get_mouse_state()

```
int64_t get_mouse_state (  
    int64_t * x,  
    int64_t * y)
```

Get the current mouse state with pointer coordinates.

Parameters

out	<i>x</i>	Receives mouse x position
out	<i>y</i>	Receives mouse y position

Returns

Button bitmask

Definition at line 693 of file [sdl.c](#).
Referenced by [mxvm_sdl_get_mouse_state\(\)](#).

12.32.2.29 get_mouse_x()

```
int64_t get_mouse_x (
    void )
```

Get the mouse x coordinate from the last mouse event.

Definition at line 194 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_get_mouse_x\(\)](#).

12.32.2.30 get_mouse_y()

```
int64_t get_mouse_y (
    void )
```

Get the mouse y coordinate from the last mouse event.

Definition at line 198 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_get_mouse_y\(\)](#).

12.32.2.31 get_num_keys()

```
int64_t get_num_keys (
    void )
```

Get the total number of keyboard keys.

Definition at line 551 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_num_keys\(\)](#).

12.32.2.32 get_queued_audio_size()

```
int64_t get_queued_audio_size (
    void )
```

Get the number of bytes of queued audio.

Definition at line 506 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_queued_audio_size\(\)](#).

12.32.2.33 get_relative_mouse_buttons()

```
int64_t get_relative_mouse_buttons (
    void )
```

Get the relative mouse button bitmask (pointerless variant).

Definition at line 531 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_relative_mouse_buttons\(\)](#).

12.32.2.34 get_relative_mouse_state()

```
int64_t get_relative_mouse_state (
    int64_t * x,
    int64_t * y)
```

Get the relative mouse state.

Parameters

out	x	Receives relative x delta
out	y	Receives relative y delta

Returns

Button bitmask

Definition at line 703 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_relative_mouse_state\(\)](#).

12.32.2.35 get_relative_mouse_x()

```
int64_t get_relative_mouse_x (
    void )
```

Get the relative mouse x delta (pointerless variant).

Definition at line 519 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_relative_mouse_x\(\)](#).

12.32.2.36 get_relative_mouse_y()

```
int64_t get_relative_mouse_y (
    void )
```

Get the relative mouse y delta (pointerless variant).

Definition at line 525 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_relative_mouse_y\(\)](#).

12.32.2.37 get_renderer_output_size()

```
void get_renderer_output_size (
    int64_t renderer_id,
    int64_t * w,
    int64_t * h)
```

Get the renderer output size.

Parameters

out	<i>w</i>	Receives width
out	<i>h</i>	Receives height

Definition at line 596 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_get_renderer_output_size\(\)](#).

12.32.2.38 get_ticks()

```
int64_t get_ticks (
    void )
```

Get the number of milliseconds since SDL initialization.

Definition at line 450 of file [sdl.c](#).

Referenced by [mxvm_sdl_get_ticks\(\)](#).

12.32.2.39 get_window_size()

```
void get_window_size (
    int64_t window_id,
    int64_t * w,
    int64_t * h)
```

Get the size of a window.

Parameters

out	<i>w</i>	Receives width
out	<i>h</i>	Receives height

Definition at line 581 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_get_window_size\(\)](#).

12.32.2.40 `init()`

```
int64_t init (  
    void )
```

Initialize SDL2 subsystems.

Returns

0 on success, negative on failure

Definition at line 26 of file [sdl.c](#).

Referenced by [action_interpret\(\)](#), and [mxvm_sdl_init\(\)](#).

12.32.2.41 `init_text()`

```
int64_t init_text (  
    void )
```

Initialize the SDL_ttf text rendering subsystem.

Definition at line 609 of file [sdl.c](#).

Referenced by [mxvm_sdl_init_text\(\)](#).

12.32.2.42 `is_key_pressed()`

```
int64_t is_key_pressed (  
    int64_t scancode)
```

Check whether a specific key is currently pressed.

Parameters

<i>scancode</i>	SDL scancode
-----------------	--------------

Returns

Non-zero if pressed

Definition at line 543 of file [sdl.c](#).

Referenced by [mxvm_sdl_is_key_pressed\(\)](#).

12.32.2.43 `load_font()`

```
int64_t load_font (  
    const char * file,  
    int64_t psize)
```

Load a TrueType font.

Parameters

<i>file</i>	Font file path
<i>psize</i>	Font point size

Returns

Font ID handle

Definition at line 626 of file [sdl.c](#).

References [g_font_count](#), and [g_fonts](#).

Referenced by [mxvm_sdl_load_font\(\)](#).

12.32.2.44 load_texture()

```
int64_t load_texture (
    int64_t renderer_id,
    const char * file_path)
```

Load a BMP file as an SDL texture with black color-key transparency.

Load a texture from an image file.

Loads the bitmap via `SDL_LoadBMP`, sets pure black (0,0,0) as the transparent color key with `SDL_SetColorKey`, then creates an accelerated texture from the surface. The surface is freed after texture creation.

Parameters

<i>renderer_id</i>	Index into the renderer array.
<i>file_path</i>	Path to the .bmp file.

Returns

Texture index on success, -1 on bad renderer, -2 on load failure, -3 on texture creation failure.

Definition at line 306 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_load_texture\(\)](#).

12.32.2.45 load_texture_color_key()

```
int64_t load_texture_color_key (
    int64_t renderer_id,
    const char * file_path)
```

Load a BMP file as a texture with black color-key transparency.

Definition at line 330 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_load_texture_color_key\(\)](#).

12.32.2.46 load_texture_color_key_rgb()

```
int64_t load_texture_color_key_rgb (
    int64_t renderer_id,
    const char * file_path,
    int64_t r,
    int64_t g,
    int64_t b)
```

Load a BMP file as an SDL texture with a custom RGB color-key transparency.

Load a BMP file as a texture with a custom RGB color-key transparency.

Loads the bitmap via `SDL_LoadBMP`, sets the specified (r, g, b) color as the transparent color key with `SDL_SetColorKey`, then creates an accelerated texture from the surface. The surface is freed after texture creation.

Parameters

<i>renderer_id</i>	Index into the renderer array.
<i>file_path</i>	Path to the .bmp file.
<i>r</i>	Red component of the color key (0–255).
<i>g</i>	Green component of the color key (0–255).
<i>b</i>	Blue component of the color key (0–255).

Returns

Texture index on success, -1 on bad renderer or allocation failure, -2 on load failure, -3 on texture creation failure.

Definition at line 371 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_load_texture_color_key_rgb\(\)](#).

12.32.2.47 load_wav()

```
int64_t load_wav (
    const char * file_path,
    int64_t * audio_buf,
    int64_t * audio_len,
    int64_t * audio_spec)
```

Load a WAV file.

Parameters

	<i>file_path</i>	Path to the WAV file
out	<i>audio_buf</i>	Receives the audio buffer pointer
out	<i>audio_len</i>	Receives the audio length in bytes
out	<i>audio_spec</i>	Receives the audio spec pointer

Returns

Non-zero on success

Definition at line 478 of file [sdl.c](#).

Referenced by [mxvm_sdl_load_wav\(\)](#).

12.32.2.48 lock_texture()

```
int64_t lock_texture (
    int64_t texture_id,
    void ** pixels,
    int64_t * pitch)
```

Lock a texture for direct pixel access.

Parameters

out	<i>pixels</i>	Receives pointer to pixel data
out	<i>pitch</i>	Receives row pitch in bytes

Definition at line 434 of file [sdl.c](#).

References [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_lock_texture\(\)](#).

12.32.2.49 open_audio()

```
int64_t open_audio (
    int64_t freq,
    int64_t format,
    int64_t channels,
    int64_t samples)
```

Open the audio device.

Parameters

<i>freq</i>	Sample rate (e.g. 44100)
<i>format</i>	Audio format
<i>channels</i>	Number of channels
<i>samples</i>	Buffer size in samples

Returns

0 on success

Definition at line 458 of file [sdl.c](#).

Referenced by [mxvm_sdl_open_audio\(\)](#).

12.32.2.50 pause_audio()

```
void pause_audio (
    int64_t pause_on)
```

Pause or unpause audio playback.

Definition at line 474 of file [sdl.c](#).

Referenced by [mxvm_sdl_pause_audio\(\)](#).

12.32.2.51 poll_event()

```
int64_t poll_event (
    void )
```

Poll for a pending SDL event.

Returns

1 if an event was available, 0 otherwise

Definition at line 182 of file [sdl.c](#).

References [g_event](#).

Referenced by [mxvm_sdl_poll_event\(\)](#).

12.32.2.52 present()

```
void present (
    int64_t renderer_id)
```

Present the renderer's back buffer to the screen.

Definition at line 218 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_present\(\)](#).

12.32.2.53 present_scaled()

```
void present_scaled (
    int64_t renderer_id,
    int64_t target_id,
    int64_t scale_width,
    int64_t scale_height)
```

Present a render target scaled to the given dimensions.

Definition at line 224 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_present_scaled\(\)](#).

12.32.2.54 present_stretched()

```
void present_stretched (
    int64_t renderer_id,
    int64_t target_id,
    int64_t dst_width,
    int64_t dst_height)
```

Present a render target stretched to the given dimensions.

Definition at line 233 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_present_stretched\(\)](#).

12.32.2.55 queue_audio()

```
int64_t queue_audio (
    const void * data,
    int64_t len)
```

Queue audio data for playback.

Definition at line 502 of file [sdl.c](#).

Referenced by [mxvm_sdl_queue_audio\(\)](#).

12.32.2.56 quit()

```
void quit (
    void )
```

Shut down SDL2 and release all resources.

Definition at line 34 of file [sdl.c](#).

References [g_font_count](#), [g_fonts](#), [g_render_target_count](#), [g_renderer_count](#), [g_renderers](#), [g_target_heights](#), [g_target_widths](#), [g_texture_count](#), [g_textures](#), [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_quit\(\)](#).

12.32.2.57 quit_text()

```
void quit_text (
    void )
```

Shut down the SDL_ttf subsystem.

Definition at line 613 of file [sdl.c](#).

References [g_font_count](#), and [g_fonts](#).

Referenced by [mxvm_sdl_quit_text\(\)](#).

12.32.2.58 render_texture()

```
void render_texture (
    int64_t renderer_id,
    int64_t texture_id,
    int64_t src_x,
    int64_t src_y,
    int64_t src_w,
    int64_t src_h,
    int64_t dst_x,
    int64_t dst_y,
    int64_t dst_w,
    int64_t dst_h)
```

Render a portion of a texture to a destination rectangle.

Definition at line 397 of file [sdl.c](#).

References [g_renderer_count](#), [g_renderers](#), [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_render_texture\(\)](#).

12.32.2.59 `set_clipboard_text()`

```
void set_clipboard_text (  
    const char * text)
```

Set the system clipboard text.

Definition at line 557 of file [sdl.c](#).

Referenced by [mxvm_sdl_set_clipboard_text\(\)](#).

12.32.2.60 `set_draw_color()`

```
void set_draw_color (  
    int64_t renderer_id,  
    int64_t r,  
    int64_t g,  
    int64_t b,  
    int64_t a)
```

Set the current draw color (RGBA).

Definition at line 206 of file [sdl.c](#).

References [g_renderer_count](#), and [g_renderers](#).

Referenced by [mxvm_sdl_set_draw_color\(\)](#).

12.32.2.61 `set_render_target()`

```
void set_render_target (  
    int64_t renderer_id,  
    int64_t target_id)
```

Set the current render target (0 to render to the window).

Definition at line 173 of file [sdl.c](#).

Referenced by [mxvm_sdl_set_render_target\(\)](#).

12.32.2.62 `set_window_fullscreen()`

```
void set_window_fullscreen (  
    int64_t window_id,  
    int64_t fullscreen)
```

Set or clear fullscreen mode for a window.

Definition at line 590 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_set_window_fullscreen\(\)](#).

12.32.2.63 `set_window_icon()`

```
void set_window_icon (  
    uint64_t window_id,  
    const char * path)
```

Set the window icon from an image file.

Definition at line 113 of file [sdl.c](#).

References [g_windows](#).

Referenced by [mxvm_sdl_set_window_icon\(\)](#).

12.32.2.64 `set_window_position()`

```
void set_window_position (  
    int64_t window_id,  
    int64_t x,  
    int64_t y)
```

Set the position of a window.

Definition at line 575 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_set_window_position\(\)](#).

12.32.2.65 `set_window_title()`

```
void set_window_title (  
    int64_t window_id,  
    const char * title)
```

Set the title of a window.

Definition at line 569 of file [sdl.c](#).

References [g_window_count](#), and [g_windows](#).

Referenced by [mxvm_sdl_set_window_title\(\)](#).

12.32.2.66 `show_cursor()`

```
void show_cursor (  
    int64_t show)
```

Show or hide the mouse cursor.

Definition at line 605 of file [sdl.c](#).

Referenced by [mxvm_sdl_show_cursor\(\)](#).

12.32.2.67 `unlock_texture()`

```
void unlock_texture (  
    int64_t texture_id)
```

Unlock a previously locked texture.

Definition at line 444 of file [sdl.c](#).

References [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_unlock_texture\(\)](#).

12.32.2.68 `update_texture()`

```
int64_t update_texture (  
    int64_t texture_id,  
    const void * pixels,  
    int64_t pitch)
```

Update a texture with new pixel data.

Definition at line 427 of file [sdl.c](#).

References [g_texture_count](#), and [g_textures](#).

Referenced by [mxvm_sdl_update_texture\(\)](#).

12.32.3 Variable Documentation

12.32.3.1 `g_event`

```
SDL_Event g_event [static]
```

Definition at line 11 of file [sdl.c](#).

Referenced by [get_event_type\(\)](#), [get_key_code\(\)](#), [get_mouse_button\(\)](#), [get_mouse_x\(\)](#), [get_mouse_y\(\)](#), and [poll_event\(\)](#).

12.32.3.2 `g_font_count`

```
int64_t g_font_count = 0 [static]
```

Definition at line 19 of file [sdl.c](#).

Referenced by [draw_text\(\)](#), [load_font\(\)](#), [quit\(\)](#), and [quit_text\(\)](#).

12.32.3.3 `g_fonts`

```
TTF_Font** g_fonts = NULL [static]
```

Definition at line 18 of file [sdl.c](#).

Referenced by [draw_text\(\)](#), [load_font\(\)](#), [quit\(\)](#), and [quit_text\(\)](#).

12.32.3.4 g_render_target_count

```
int64_t g_render_target_count = 0 [static]
```

Definition at line 24 of file [sdl.c](#).

Referenced by [create_render_target\(\)](#), and [quit\(\)](#).

12.32.3.5 g_render_targets

```
SDL_Texture** g_render_targets = NULL [static]
```

Definition at line 21 of file [sdl.c](#).

12.32.3.6 g_renderer_count

```
int64_t g_renderer_count = 0 [static]
```

Definition at line 16 of file [sdl.c](#).

Referenced by [clear\(\)](#), [create_render_target\(\)](#), [create_renderer\(\)](#), [create_texture\(\)](#), [destroy_renderer\(\)](#), [draw_line\(\)](#), [draw_point\(\)](#), [draw_rect\(\)](#), [draw_text\(\)](#), [fill_rect\(\)](#), [get_renderer_output_size\(\)](#), [load_texture\(\)](#), [load_texture_color_key\(\)](#), [load_texture_color_key_rgb\(\)](#), [present\(\)](#), [present_scaled\(\)](#), [present_stretched\(\)](#), [quit\(\)](#), [render_texture\(\)](#), and [set_draw_color\(\)](#).

12.32.3.7 g_renderers

```
SDL_Renderer** g_renderers = NULL [static]
```

Definition at line 13 of file [sdl.c](#).

Referenced by [clear\(\)](#), [create_render_target\(\)](#), [create_renderer\(\)](#), [create_texture\(\)](#), [destroy_renderer\(\)](#), [draw_line\(\)](#), [draw_point\(\)](#), [draw_rect\(\)](#), [draw_text\(\)](#), [fill_rect\(\)](#), [get_renderer_output_size\(\)](#), [load_texture\(\)](#), [load_texture_color_key\(\)](#), [load_texture_color_key_rgb\(\)](#), [present\(\)](#), [present_scaled\(\)](#), [present_stretched\(\)](#), [quit\(\)](#), [render_texture\(\)](#), and [set_draw_color\(\)](#).

12.32.3.8 g_target_heights

```
int64_t* g_target_heights = NULL [static]
```

Definition at line 23 of file [sdl.c](#).

Referenced by [create_render_target\(\)](#), and [quit\(\)](#).

12.32.3.9 g_target_widths

```
int64_t* g_target_widths = NULL [static]
```

Definition at line 22 of file [sdl.c](#).

Referenced by [create_render_target\(\)](#), and [quit\(\)](#).

12.32.3.10 g_texture_count

```
int64_t g_texture_count = 0 [static]
```

Definition at line 17 of file [sdl.c](#).

Referenced by [create_texture\(\)](#), [destroy_texture\(\)](#), [load_texture\(\)](#), [load_texture_color_key\(\)](#), [load_texture_color_key_rgb\(\)](#), [lock_texture\(\)](#), [quit\(\)](#), [render_texture\(\)](#), [unlock_texture\(\)](#), and [update_texture\(\)](#).

12.32.3.11 g_textures

```
SDL_Texture** g_textures = NULL [static]
```

Definition at line 14 of file [sdl.c](#).

Referenced by [create_texture\(\)](#), [destroy_texture\(\)](#), [load_texture\(\)](#), [load_texture_color_key\(\)](#), [load_texture_color_key_rgb\(\)](#), [lock_texture\(\)](#), [quit\(\)](#), [render_texture\(\)](#), [unlock_texture\(\)](#), and [update_texture\(\)](#).

12.32.3.12 g_window_count

```
int64_t g_window_count = 0 [static]
```

Definition at line 15 of file [sdl.c](#).

Referenced by [create_renderer\(\)](#), [create_window\(\)](#), [destroy_window\(\)](#), [get_window_size\(\)](#), [quit\(\)](#), [set_window_fullscreen\(\)](#), [set_window_position\(\)](#), and [set_window_title\(\)](#).

12.32.3.13 g_windows

SDL_Window** g_windows = NULL [static]

Definition at line 12 of file [sdl.c](#).

Referenced by [create_renderer\(\)](#), [create_window\(\)](#), [destroy_window\(\)](#), [get_window_size\(\)](#), [quit\(\)](#), [set_window_fullscreen\(\)](#), [set_window_icon\(\)](#), [set_window_position\(\)](#), and [set_window_title\(\)](#).

12.33 sdl.c

[Go to the documentation of this file.](#)

```

00001
00006 #include "mx_sdl.h"
00007 #include "SDL.h"
00008 #include "SDL_ttf.h"
00009
00010
00011 static SDL_Event g_event;
00012 static SDL_Window **g_windows = NULL;
00013 static SDL_Renderer **g_renderers = NULL;
00014 static SDL_Texture **g_textures = NULL;
00015 static int64_t g_window_count = 0;
00016 static int64_t g_renderer_count = 0;
00017 static int64_t g_texture_count = 0;
00018 static TTF_Font **g_fonts = NULL;
00019 static int64_t g_font_count = 0;
00020
00021 static SDL_Texture **g_render_targets = NULL;
00022 static int64_t *g_target_widths = NULL;
00023 static int64_t *g_target_heights = NULL;
00024 static int64_t g_render_target_count = 0;
00025
00026 int64_t init(void) {
00027     SDL_SetMainReady();
00028     if (SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO | SDL_INIT_JOYSTICK) == 0) {
00029         return 0;
00030     }
00031     return 1;
00032 }
00033
00034 void quit(void) {
00035     if (g_fonts) {
00036         for (int64_t i = 0; i < g_font_count; ++i) {
00037             if (g_fonts[i])
00038                 TTF_CloseFont(g_fonts[i]);
00039         }
00040         free(g_fonts);
00041         g_fonts = NULL;
00042         g_font_count = 0;
00043     }
00044
00045     if (g_target_widths) {
00046         free(g_target_widths);
00047         g_target_widths = NULL;
00048     }
00049     if (g_target_heights) {
00050         free(g_target_heights);
00051         g_target_heights = NULL;
00052     }
00053     g_render_target_count = 0;
00054
00055     if (g_textures) {
00056         for (int64_t i = 0; i < g_texture_count; ++i) {
00057             if (g_textures[i])
00058                 SDL_DestroyTexture(g_textures[i]);
00059             g_textures[i] = NULL;
00060         }
00061         free(g_textures);
00062         g_textures = NULL;
00063         g_texture_count = 0;
00064     }
00065     if (g_renderers) {
00066         for (int64_t i = 0; i < g_renderer_count; ++i) {
00067             if (g_renderers[i])
00068                 SDL_DestroyRenderer(g_renderers[i]);
00069         }
00070         free(g_renderers);
00071         g_renderers = NULL;

```

```

00072         g_renderer_count = 0;
00073     }
00074     if (g_windows) {
00075         for (int64_t i = 0; i < g_window_count; ++i) {
00076             if (g_windows[i])
00077                 SDL_DestroyWindow(g_windows[i]);
00078         }
00079         free(g_windows);
00080         g_windows = NULL;
00081         g_window_count = 0;
00082     }
00083     SDL_Quit();
00084 }
00085
00086 int64_t create_window(const char *title, int64_t x, int64_t y, int64_t w, int64_t h, int64_t flags) {
00087     SDL_Window *window = SDL_CreateWindow(title, (int)x, (int)y, (int)w, (int)h, (Uint32)flags);
00088     if (!window)
00089         return -1;
00090
00091     void *tmp = realloc(g_windows, sizeof(SDL_Window *) * (g_window_count + 1));
00092     if (!tmp) {
00093         SDL_DestroyWindow(window);
00094         return -1;
00095     }
00096     g_windows = tmp;
00097     g_windows[g_window_count] = window;
00098     SDL_Surface *surf = SDL_LoadBMP("icon.bmp");
00099     if (surf != NULL) {
00100         SDL_SetWindowIcon(window, surf);
00101         SDL_FreeSurface(surf);
00102     }
00103     return g_window_count++;
00104 }
00105
00106 void destroy_window(int64_t window_id) {
00107     if (window_id >= 0 && window_id < g_window_count && g_windows[window_id]) {
00108         SDL_DestroyWindow(g_windows[window_id]);
00109         g_windows[window_id] = NULL;
00110     }
00111 }
00112
00113 void set_window_icon(uint64_t window_id, const char *path) {
00114     SDL_Surface *surf = SDL_LoadBMP(path);
00115     if (!surf)
00116         return;
00117     SDL_SetWindowIcon(g_windows[window_id], surf);
00118     SDL_FreeSurface(surf);
00119 }
00120
00121 int64_t create_renderer(int64_t window_id, int64_t index, int64_t flags) {
00122     if (window_id < 0 || window_id >= g_window_count || !g_windows[window_id])
00123         return -1;
00124
00125     SDL_Renderer *renderer = SDL_CreateRenderer(g_windows[window_id], (int)index, (Uint32)flags);
00126     if (!renderer)
00127         return -1;
00128
00129     void *tmp = realloc(g_renderers, sizeof(SDL_Renderer *) * (g_renderer_count + 1));
00130     if (!tmp) {
00131         SDL_DestroyRenderer(renderer);
00132         return -1;
00133     }
00134     g_renderers = tmp;
00135     g_renderers[g_renderer_count] = renderer;
00136     return g_renderer_count++;
00137 }
00138
00139 void destroy_renderer(int64_t renderer_id) {
00140     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id]) {
00141         SDL_DestroyRenderer(g_renderers[renderer_id]);
00142         g_renderers[renderer_id] = NULL;
00143     }
00144 }
00145
00146 int64_t create_render_target(int64_t renderer_id, int64_t width, int64_t height) {
00147     if (renderer_id < 0 || renderer_id >= g_renderer_count || !g_renderers[renderer_id])
00148         return -1;
00149
00150     SDL_RenderSetLogicalSize(g_renderers[renderer_id], (int)width, (int)height);
00151
00152     void *tmp2 = realloc(g_target_widths, sizeof(int64_t) * (g_render_target_count + 1));
00153     void *tmp3 = realloc(g_target_heights, sizeof(int64_t) * (g_render_target_count + 1));
00154     if (!tmp2 || !tmp3) {
00155         if (tmp2)
00156             g_target_widths = tmp2;
00157         if (tmp3)
00158             g_target_heights = tmp3;

```

```

00159         return -1;
00160     }
00161     g_target_widths = tmp2;
00162     g_target_heights = tmp3;
00163
00164     g_target_widths[g_render_target_count] = width;
00165     g_target_heights[g_render_target_count] = height;
00166
00167     SDL_SetRenderDrawColor(g_renderers[renderer_id], 0, 0, 0, 255);
00168     SDL_RenderClear(g_renderers[renderer_id]);
00169
00170     return g_render_target_count++;
00171 }
00172
00173 void set_render_target(int64_t renderer_id, int64_t target_id) {
00174     (void)renderer_id;
00175     (void)target_id;
00176 }
00177
00178 void destroy_render_target(int64_t target_id) {
00179     (void)target_id;
00180 }
00181
00182 int64_t poll_event(void) {
00183     return SDL_PollEvent(&g_event);
00184 }
00185
00186 int64_t get_event_type(void) {
00187     return g_event.type;
00188 }
00189
00190 int64_t get_key_code(void) {
00191     return g_event.key.keysym.sym;
00192 }
00193
00194 int64_t get_mouse_x(void) {
00195     return g_event.motion.x;
00196 }
00197
00198 int64_t get_mouse_y(void) {
00199     return g_event.motion.y;
00200 }
00201
00202 int64_t get_mouse_button(void) {
00203     return g_event.button.button;
00204 }
00205
00206 void set_draw_color(int64_t renderer_id, int64_t r, int64_t g, int64_t b, int64_t a) {
00207     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id]) {
00208         SDL_SetRenderDrawColor(g_renderers[renderer_id], (Uint8)r, (Uint8)g, (Uint8)b, (Uint8)a);
00209     }
00210 }
00211
00212 void clear(int64_t renderer_id) {
00213     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id]) {
00214         SDL_RenderClear(g_renderers[renderer_id]);
00215     }
00216 }
00217
00218 void present(int64_t renderer_id) {
00219     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id]) {
00220         SDL_RenderPresent(g_renderers[renderer_id]);
00221     }
00222 }
00223
00224 void present_scaled(int64_t renderer_id, int64_t target_id, int64_t scale_width, int64_t scale_height)
00225 {
00226     (void)target_id;
00227     (void)scale_width;
00228     (void)scale_height;
00229     if (renderer_id < 0 || renderer_id >= g_renderer_count || !g_renderers[renderer_id])
00230         return;
00231     SDL_RenderPresent(g_renderers[renderer_id]);
00232 }
00233 void present_stretched(int64_t renderer_id, int64_t target_id, int64_t dst_width, int64_t dst_height)
00234 {
00235     (void)target_id;
00236     (void)dst_width;
00237     (void)dst_height;
00238     if (renderer_id < 0 || renderer_id >= g_renderer_count || !g_renderers[renderer_id])
00239         return;
00240     SDL_RenderPresent(g_renderers[renderer_id]);
00241 }
00242 void draw_point(int64_t renderer_id, int64_t x, int64_t y) {
00243     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id]) {

```

```

00244     SDL_RenderDrawPoint(g_renderers[renderer_id], (int)x, (int)y);
00245 }
00246 }
00247
00248 void draw_line(int64_t renderer_id, int64_t x1, int64_t y1, int64_t x2, int64_t y2) {
00249     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id]) {
00250         SDL_RenderDrawLine(g_renderers[renderer_id], (int)x1, (int)y1, (int)x2, (int)y2);
00251     }
00252 }
00253
00254 void draw_rect(int64_t renderer_id, int64_t x, int64_t y, int64_t w, int64_t h) {
00255     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id]) {
00256         SDL_Rect rect = {(int)x, (int)y, (int)w, (int)h};
00257         SDL_RenderDrawRect(g_renderers[renderer_id], &rect);
00258     }
00259 }
00260
00261 void fill_rect(int64_t renderer_id, int64_t x, int64_t y, int64_t w, int64_t h) {
00262     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id]) {
00263         SDL_Rect rect = {(int)x, (int)y, (int)w, (int)h};
00264         SDL_RenderFillRect(g_renderers[renderer_id], &rect);
00265     }
00266 }
00267
00268 int64_t create_texture(int64_t renderer_id, int64_t format, int64_t access, int64_t w, int64_t h) {
00269     if (renderer_id < 0 || renderer_id >= g_renderer_count || !g_renderers[renderer_id])
00270         return -1;
00271
00272     SDL_Texture *texture = SDL_CreateTexture(g_renderers[renderer_id], (Uint32)format, (int)access,
(int)w, (int)h);
00273     if (!texture)
00274         return -1;
00275
00276     void *tmp = realloc(g_textures, sizeof(SDL_Texture *) * (g_texture_count + 1));
00277     if (!tmp) {
00278         SDL_DestroyTexture(texture);
00279         return -1;
00280     }
00281     g_textures = tmp;
00282     g_textures[g_texture_count] = texture;
00283     return g_texture_count++;
00284 }
00285
00286 void destroy_texture(int64_t texture_id) {
00287     if (texture_id >= 0 && texture_id < g_texture_count && g_textures[texture_id]) {
00288         SDL_DestroyTexture(g_textures[texture_id]);
00289         g_textures[texture_id] = NULL;
00290     }
00291 }
00292
00306 int64_t load_texture(int64_t renderer_id, const char *file_path) {
00307     if (renderer_id < 0 || renderer_id >= g_renderer_count || !g_renderers[renderer_id])
00308         return -1;
00309
00310     SDL_Surface *surface = SDL_LoadBMP(file_path);
00311     if (!surface)
00312         return -2;
00313
00314     SDL_Texture *texture = SDL_CreateTextureFromSurface(g_renderers[renderer_id], surface);
00315     SDL_FreeSurface(surface);
00316     if (!texture)
00317         return -3;
00318
00319     void *tmp = realloc(g_textures, sizeof(SDL_Texture *) * (g_texture_count + 1));
00320     if (!tmp) {
00321         SDL_DestroyTexture(texture);
00322         return -1;
00323     }
00324     g_textures = tmp;
00325     g_textures[g_texture_count] = texture;
00326     return g_texture_count++;
00327 }
00328 }
00329
00330 int64_t load_texture_color_key(int64_t renderer_id, const char *file_path) {
00331     if (renderer_id < 0 || renderer_id >= g_renderer_count || !g_renderers[renderer_id])
00332         return -1;
00333
00334     SDL_Surface *surface = SDL_LoadBMP(file_path);
00335     if (!surface)
00336         return -2;
00337
00338     SDL_SetColorKey(surface, SDL_TRUE, SDL_MapRGB(surface->format, 0, 0, 0));
00339     SDL_Texture *texture = SDL_CreateTextureFromSurface(g_renderers[renderer_id], surface);
00340     SDL_FreeSurface(surface);
00341     if (!texture)
00342         return -3;

```

```

00343     SDL_SetTextureBlendMode(texture, SDL_BLENDMODE_BLEND);
00344
00345     void *tmp = realloc(g_textures, sizeof(SDL_Texture *) * (g_texture_count + 1));
00346     if (!tmp) {
00347         SDL_DestroyTexture(texture);
00348         return -1;
00349     }
00350     g_textures = tmp;
00351     g_textures[g_texture_count] = texture;
00352
00353     return g_texture_count++;
00354 }
00355
00371 int64_t load_texture_color_key_rgb(int64_t renderer_id, const char *file_path, int64_t r, int64_t g,
int64_t b) {
00372     if (renderer_id < 0 || renderer_id >= g_renderer_count || !g_renderers[renderer_id])
00373         return -1;
00374
00375     SDL_Surface *surface = SDL_LoadBMP(file_path);
00376     if (!surface)
00377         return -2;
00378
00379     SDL_SetColorKey(surface, SDL_TRUE, SDL_MapRGB(surface->format, (Uint8)r, (Uint8)g, (Uint8)b));
00380     SDL_Texture *texture = SDL_CreateTextureFromSurface(g_renderers[renderer_id], surface);
00381     SDL_FreeSurface(surface);
00382     if (!texture)
00383         return -3;
00384     SDL_SetTextureBlendMode(texture, SDL_BLENDMODE_BLEND);
00385
00386     void *tmp = realloc(g_textures, sizeof(SDL_Texture *) * (g_texture_count + 1));
00387     if (!tmp) {
00388         SDL_DestroyTexture(texture);
00389         return -1;
00390     }
00391     g_textures = tmp;
00392     g_textures[g_texture_count] = texture;
00393
00394     return g_texture_count++;
00395 }
00396
00397 void render_texture(int64_t renderer_id, int64_t texture_id, int64_t src_x, int64_t src_y, int64_t
src_w, int64_t src_h, int64_t dst_x, int64_t dst_y, int64_t dst_w, int64_t dst_h) {
00398     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id] &&
00399         texture_id >= 0 && texture_id < g_texture_count && g_textures[texture_id]) {
00400         SDL_Renderer *renderer = g_renderers[renderer_id];
00401         SDL_Texture *texture = g_textures[texture_id];
00402
00403         SDL_Rect *psrc = NULL;
00404         SDL_Rect src_rect;
00405         if (!(src_x == -1 || src_y == -1) && src_w == -1 && src_h == -1) {
00406             src_rect.x = (int)src_x;
00407             src_rect.y = (int)src_y;
00408             src_rect.w = (int)src_w;
00409             src_rect.h = (int)src_h;
00410             psrc = &src_rect;
00411         }
00412
00413         SDL_Rect *pdst = NULL;
00414         SDL_Rect dst_rect;
00415         if (!(dst_x == -1 && dst_y == -1 && dst_w == -1 && dst_h == -1)) {
00416             dst_rect.x = (int)dst_x;
00417             dst_rect.y = (int)dst_y;
00418             dst_rect.w = (int)dst_w;
00419             dst_rect.h = (int)dst_h;
00420             pdst = &dst_rect;
00421         }
00422
00423         SDL_RenderCopy(renderer, texture, psrc, pdst);
00424     }
00425 }
00426
00427 int64_t update_texture(int64_t texture_id, const void *pixels, int64_t pitch) {
00428     if (texture_id >= 0 && texture_id < g_texture_count && g_textures[texture_id]) {
00429         return SDL_UpdateTexture(g_textures[texture_id], NULL, pixels, (int)pitch) == 0 ? 1 : 0;
00430     }
00431     return 0;
00432 }
00433
00434 int64_t lock_texture(int64_t texture_id, void **pixels, int64_t *pitch) {
00435     if (texture_id >= 0 && texture_id < g_texture_count && g_textures[texture_id]) {
00436         int p;
00437         int result = SDL_LockTexture(g_textures[texture_id], NULL, pixels, &p);
00438         *pitch = p;
00439         return result == 0 ? 1 : 0;
00440     }
00441     return 0;
00442 }

```

```
00443
00444 void unlock_texture(int64_t texture_id) {
00445     if (texture_id >= 0 && texture_id < g_texture_count && g_textures[texture_id]) {
00446         SDL_UnlockTexture(g_textures[texture_id]);
00447     }
00448 }
00449
00450 int64_t get_ticks(void) {
00451     return SDL_GetTicks();
00452 }
00453
00454 void delay(int64_t ms) {
00455     SDL_Delay((Uint32)ms);
00456 }
00457
00458 int64_t open_audio(int64_t freq, int64_t format, int64_t channels, int64_t samples) {
00459     SDL_AudioSpec wanted_spec;
00460     wanted_spec.freq = (int)freq;
00461     wanted_spec.format = (SDL_AudioFormat)format;
00462     wanted_spec.channels = (Uint8)channels;
00463     wanted_spec.samples = (Uint16)samples;
00464     wanted_spec.callback = NULL;
00465     wanted_spec.userdata = NULL;
00466
00467     return SDL_OpenAudio(&wanted_spec, NULL) == 0 ? 1 : 0;
00468 }
00469
00470 void close_audio(void) {
00471     SDL_CloseAudio();
00472 }
00473
00474 void pause_audio(int64_t pause_on) {
00475     SDL_PauseAudio(pause_on ? 1 : 0);
00476 }
00477
00478 int64_t load_wav(const char *file_path, int64_t *audio_buf, int64_t *audio_len, int64_t *audio_spec) {
00479     SDL_AudioSpec *spec = (SDL_AudioSpec *)malloc(sizeof(SDL_AudioSpec));
00480     if (!spec)
00481         return 0;
00482     Uint8 *buf;
00483     Uint32 len;
00484
00485     if (SDL_LoadWAV(file_path, spec, &buf, &len) == NULL) {
00486         free(spec);
00487         return 0;
00488     }
00489
00490     *audio_buf = (int64_t)buf;
00491     *audio_len = len;
00492     *audio_spec = (int64_t)spec;
00493     return 1;
00494 }
00495
00496 void free_wav(int64_t audio_buf) {
00497     if (audio_buf) {
00498         SDL_FreeWAV((Uint8 *)audio_buf);
00499     }
00500 }
00501
00502 int64_t queue_audio(const void *data, int64_t len) {
00503     return SDL_QueueAudio(1, data, (Uint32)len) == 0 ? 1 : 0;
00504 }
00505
00506 int64_t get_queued_audio_size(void) {
00507     return SDL_GetQueuedAudioSize(1);
00508 }
00509
00510 void clear_queued_audio(void) {
00511     SDL_ClearQueuedAudio(1);
00512 }
00513
00514 int64_t get_mouse_buttons(void) {
00515     int x, y;
00516     return SDL_GetMouseState(&x, &y);
00517 }
00518
00519 int64_t get_relative_mouse_x(void) {
00520     int x, y;
00521     SDL_GetRelativeMouseState(&x, &y);
00522     return x;
00523 }
00524
00525 int64_t get_relative_mouse_y(void) {
00526     int x, y;
00527     SDL_GetRelativeMouseState(&x, &y);
00528     return y;
00529 }
```

```

00530
00531 int64_t get_relative_mouse_buttons(void) {
00532     int x, y;
00533     return SDL_GetRelativeMouseState(&x, &y);
00534 }
00535
00536 int64_t get_keyboard_state(int64_t *numkeys) {
00537     int nk;
00538     const Uint8 *state = SDL_GetKeyboardState(&nk);
00539     *numkeys = nk;
00540     return (int64_t)state;
00541 }
00542
00543 int64_t is_key_pressed(int64_t scancode) {
00544     int numkeys;
00545     const Uint8 *state = SDL_GetKeyboardState(&numkeys);
00546     if (scancode < 0 || scancode >= numkeys)
00547         return 0;
00548     return state[scancode] ? 1 : 0;
00549 }
00550
00551 int64_t get_num_keys(void) {
00552     int numkeys;
00553     SDL_GetKeyboardState(&numkeys);
00554     return numkeys;
00555 }
00556
00557 void set_clipboard_text(const char *text) {
00558     SDL_SetClipboardText(text);
00559 }
00560
00561 const char *get_clipboard_text(void) {
00562     static char *last_clipboard = NULL;
00563     if (last_clipboard)
00564         SDL_free(last_clipboard);
00565     last_clipboard = SDL_GetClipboardText();
00566     return last_clipboard;
00567 }
00568
00569 void set_window_title(int64_t window_id, const char *title) {
00570     if (window_id >= 0 && window_id < g_window_count && g_windows[window_id]) {
00571         SDL_SetWindowTitle(g_windows[window_id], title);
00572     }
00573 }
00574
00575 void set_window_position(int64_t window_id, int64_t x, int64_t y) {
00576     if (window_id >= 0 && window_id < g_window_count && g_windows[window_id]) {
00577         SDL_SetWindowPosition(g_windows[window_id], (int)x, (int)y);
00578     }
00579 }
00580
00581 void get_window_size(int64_t window_id, int64_t *w, int64_t *h) {
00582     if (window_id >= 0 && window_id < g_window_count && g_windows[window_id]) {
00583         int ww, hh;
00584         SDL_GetWindowSize(g_windows[window_id], &ww, &hh);
00585         *w = ww;
00586         *h = hh;
00587     }
00588 }
00589
00590 void set_window_fullscreen(int64_t window_id, int64_t fullscreen) {
00591     if (window_id >= 0 && window_id < g_window_count && g_windows[window_id]) {
00592         SDL_SetWindowFullscreen(g_windows[window_id], fullscreen ? SDL_WINDOW_FULLSCREEN : 0);
00593     }
00594 }
00595
00596 void get_renderer_output_size(int64_t renderer_id, int64_t *w, int64_t *h) {
00597     if (renderer_id >= 0 && renderer_id < g_renderer_count && g_renderers[renderer_id]) {
00598         int ww, hh;
00599         SDL_GetRendererOutputSize(g_renderers[renderer_id], &ww, &hh);
00600         *w = ww;
00601         *h = hh;
00602     }
00603 }
00604
00605 void show_cursor(int64_t show) {
00606     SDL_ShowCursor(show ? SDL_ENABLE : SDL_DISABLE);
00607 }
00608
00609 int64_t init_text(void) {
00610     return TTF_Init() == 0 ? 1 : 0;
00611 }
00612
00613 void quit_text(void) {
00614     if (g_fonts) {
00615         for (int64_t i = 0; i < g_font_count; ++i) {
00616             if (g_fonts[i])

```

```

00617         TTF_CloseFont(g_fonts[i]);
00618     }
00619     free(g_fonts);
00620     g_fonts = NULL;
00621     g_font_count = 0;
00622 }
00623 TTF_Quit();
00624 }
00625
00626 int64_t load_font(const char *file, int64_t ptsize) {
00627     TTF_Font *font = TTF_OpenFont(file, (int)ptsize);
00628     if (!font)
00629         return -1;
00630     void *tmp = realloc(g_fonts, sizeof(TTF_Font *) * (g_font_count + 1));
00631     if (!tmp) {
00632         TTF_CloseFont(font);
00633         return -1;
00634     }
00635     g_fonts = tmp;
00636     g_fonts[g_font_count] = font;
00637     return g_font_count++;
00638 }
00639
00640 void draw_text(int64_t renderer_id, int64_t font_id, const char *text, int64_t x, int64_t y, int64_t
r, int64_t g, int64_t b, int64_t a) {
00641     if (renderer_id < 0 || renderer_id >= g_renderer_count || !g_renderers[renderer_id])
00642         return;
00643     if (font_id < 0 || font_id >= g_font_count || !g_fonts[font_id])
00644         return;
00645
00646     SDL_Color color = {(Uint8)r, (Uint8)g, (Uint8)b, (Uint8)a};
00647     SDL_Surface *surface = TTF_RenderUTF8_Blended(g_fonts[font_id], text, color);
00648     if (!surface)
00649         return;
00650
00651     SDL_Texture *texture = SDL_CreateTextureFromSurface(g_renderers[renderer_id], surface);
00652     if (!texture) {
00653         SDL_FreeSurface(surface);
00654         return;
00655     }
00656
00657     SDL_Rect dst = {(int)x, (int)y, surface->w, surface->h};
00658     SDL_RenderCopy(g_renderers[renderer_id], texture, NULL, &dst);
00659     SDL_DestroyTexture(texture);
00660     SDL_FreeSurface(surface);
00661 }
00662
00663 int64_t create_rgb_surface(int64_t width, int64_t height, int64_t depth) {
00664     SDL_Surface *s = NULL;
00665     int w = (int)width, h = (int)height, d = (int)depth;
00666
00667     if (d == 32)
00668         s = SDL_CreateRGBSurfaceWithFormat(0, w, h, 32, SDL_PIXELFORMAT_RGBA32);
00669     else if (d == 24)
00670         s = SDL_CreateRGBSurfaceWithFormat(0, w, h, 24, SDL_PIXELFORMAT_RGB24);
00671     else if (d == 16)
00672         s = SDL_CreateRGBSurfaceWithFormat(0, w, h, 16, SDL_PIXELFORMAT_RGB565);
00673     else
00674         return 0;
00675
00676     return (int64_t)s;
00677 }
00678
00679 void free_surface(int64_t surf_ptr) {
00680     if (surf_ptr)
00681         SDL_FreeSurface((SDL_Surface *)surf_ptr);
00682 }
00683
00684 int64_t blit_surface(int64_t src_ptr, int64_t dst_ptr, int64_t x, int64_t y) {
00685     if (!src_ptr || !dst_ptr)
00686         return 0;
00687     SDL_Surface *src = (SDL_Surface *)src_ptr;
00688     SDL_Surface *dst = (SDL_Surface *)dst_ptr;
00689     SDL_Rect dst_rc = {(int)x, (int)y, 0, 0};
00690     return SDL_BlittedSurface(src, NULL, dst, &dst_rc) == 0 ? 1 : 0;
00691 }
00692
00693 int64_t get_mouse_state(int64_t *x, int64_t *y) {
00694     int ix, iy;
00695     Uint32 m = SDL_GetMouseState(&ix, &iy);
00696     if (x)
00697         *x = ix;
00698     if (y)
00699         *y = iy;
00700     return (int64_t)m;
00701 }
00702

```



```

00703 int64_t get_relative_mouse_state(int64_t *x, int64_t *y) {
00704     int ix, iy;
00705     Uint32 m = SDL_GetRelativeMouseState(&ix, &iy);
00706     if (x)
00707         *x = ix;
00708     if (y)
00709         *y = iy;
00710     return (int64_t)m;
00711 }

```

12.34 modules/sdl/sdl.cpp File Reference

SDL2 module C++ runtime bindings for the MXVM interpreter.

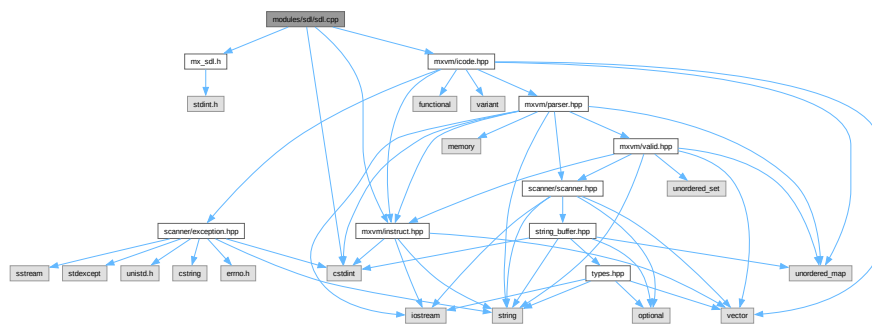
```
#include "mx_sdl.h"
```

```
#include <stdint>
```

```
#include <mxvm/icode.hpp>
```

```
#include <mxvm/instruct.hpp>
```

Include dependency graph for sdl.cpp:



Functions

- void `mxvm_sdl_init` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_quit` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_create_window` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_destroy_window` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_create_renderer` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_destroy_renderer` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_set_window_title` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_set_window_position` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_get_window_size` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_set_window_fullscreen` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_get_renderer_output_size` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_get_mouse_buttons` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_get_relative_mouse_x` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_get_relative_mouse_y` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_get_relative_mouse_buttons` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_get_mouse_state` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_get_relative_mouse_state` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)
- void `mxvm_sdl_get_keyboard_state` (`mxvm::Program` *program, `std::vector`< `mxvm::Operand` > &operand)

- void [mxvm_sdl_is_key_pressed](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_get_num_keys](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_set_clipboard_text](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_get_clipboard_text](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_show_cursor](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_create_rgb_surface](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_free_surface](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_blit_surface](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_free_wav](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_poll_event](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_get_event_type](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_get_key_code](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_get_mouse_x](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_get_mouse_y](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_get_mouse_button](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_set_draw_color](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_clear](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_present](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_draw_point](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_draw_line](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_draw_rect](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_fill_rect](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_create_texture](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_destroy_texture](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_load_texture](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_load_texture_color_key](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_load_texture_color_key_rgb](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)

MXVM binding for load_texture_color_key_rgb.

- void [mxvm_sdl_render_texture](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_get_ticks](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_delay](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_open_audio](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_close_audio](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_pause_audio](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_load_wav](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_queue_audio](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_get_queued_audio_size](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_clear_queued_audio](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_update_texture](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_lock_texture](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_unlock_texture](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_init_text](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_quit_text](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_load_font](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_draw_text](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_create_render_target](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_set_render_target](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_destroy_render_target](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_present_scaled](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_present_stretched](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_sdl_set_window_icon](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)

12.34.1 Detailed Description

SDL2 module C++ runtime bindings for the MXVM interpreter.

Author

Jared Bruni

Definition in file [sdl.cpp](#).

12.34.2 Function Documentation

12.34.2.1 mxvm_sdl_blit_surface()

```
void mxvm_sdl_blit_surface (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 386 of file [sdl.cpp](#).

References [blit_surface\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.2 mxvm_sdl_clear()

```
void mxvm_sdl_clear (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 477 of file [sdl.cpp](#).

References [clear\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.3 mxvm_sdl_clear_queued_audio()

```
void mxvm_sdl_clear_queued_audio (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 822 of file [sdl.cpp](#).

References [clear_queued_audio\(\)](#).

12.34.2.4 mxvm_sdl_close_audio()

```
void mxvm_sdl_close_audio (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 731 of file [sdl.cpp](#).

References [close_audio\(\)](#).

12.34.2.5 mxvm_sdl_create_render_target()

```
void mxvm_sdl_create_render_target (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 956 of file [sdl.cpp](#).

References [create_render_target\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.6 mxvm_sdl_create_renderer()

```
void mxvm_sdl_create_renderer (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 65 of file [sdl.cpp](#).

References [create_renderer\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.7 mxvm_sdl_create_rgb_surface()

```
void mxvm_sdl_create_rgb_surface (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 361 of file [sdl.cpp](#).

References [create_rgb_surface\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.8 mxvm_sdl_create_texture()

```
void mxvm_sdl_create_texture (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 557 of file [sdl.cpp](#).

References [create_texture\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.9 mxvm_sdl_create_window()

```
void mxvm_sdl_create_window (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 24 of file [sdl.cpp](#).

References [create_window\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.10 mxvm_sdl_delay()

```
void mxvm_sdl_delay (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 704 of file [sdl.cpp](#).

References [delay\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.11 mxvm_sdl_destroy_render_target()

```
void mxvm_sdl_destroy_render_target (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 982 of file [sdl.cpp](#).

References [destroy_render_target\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.12 mxvm_sdl_destroy_renderer()

```
void mxvm_sdl_destroy_renderer (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 81 of file [sdl.cpp](#).

References [destroy_renderer\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.13 mxvm_sdl_destroy_texture()

```
void mxvm_sdl_destroy_texture (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 575 of file [sdl.cpp](#).

References [destroy_texture\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.14 mxvm_sdl_destroy_window()

```
void mxvm_sdl_destroy_window (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 54 of file [sdl.cpp](#).

References [destroy_window\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.15 mxvm_sdl_draw_line()

```
void mxvm_sdl_draw_line (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 512 of file [sdl.cpp](#).

References [draw_line\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.16 mxvm_sdl_draw_point()

```
void mxvm_sdl_draw_point (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 499 of file [sdl.cpp](#).

References [draw_point\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.17 mxvm_sdl_draw_rect()

```
void mxvm_sdl_draw_rect (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 527 of file [sdl.cpp](#).

References [draw_rect\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.18 mxvm_sdl_draw_text()

```
void mxvm_sdl_draw_text (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 927 of file [sdl.cpp](#).

References [draw_text\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

12.34.2.19 mxvm_sdl_fill_rect()

```
void mxvm_sdl_fill_rect (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 542 of file [sdl.cpp](#).

References [fill_rect\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.20 mxvm_sdl_free_surface()

```
void mxvm_sdl_free_surface (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 376 of file [sdl.cpp](#).

References [free_surface\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.21 mxvm_sdl_free_wav()

```
void mxvm_sdl_free_wav (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 403 of file [sdl.cpp](#).

References [free_wav\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.22 mxvm_sdl_get_clipboard_text()

```
void mxvm_sdl_get_clipboard_text (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 341 of file [sdl.cpp](#).

References [get_clipboard_text\(\)](#), [mxvm::VAR_POINTER](#), and [mxvm::Base::vars](#).

12.34.2.23 mxvm_sdl_get_event_type()

```
void mxvm_sdl_get_event_type (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 422 of file [sdl.cpp](#).

References [get_event_type\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.24 mxvm_sdl_get_key_code()

```
void mxvm_sdl_get_key_code (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 430 of file [sdl.cpp](#).

References [get_key_code\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.25 mxvm_sdl_get_keyboard_state()

```
void mxvm_sdl_get_keyboard_state (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 279 of file [sdl.cpp](#).

References [get_keyboard_state\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.26 mxvm_sdl_get_mouse_button()

```
void mxvm_sdl_get_mouse_button (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 454 of file [sdl.cpp](#).

References [get_mouse_button\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.27 mxvm_sdl_get_mouse_buttons()

```
void mxvm_sdl_get_mouse_buttons (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 192 of file [sdl.cpp](#).

References [get_mouse_buttons\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.28 mxvm_sdl_get_mouse_state()

```
void mxvm_sdl_get_mouse_state (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 220 of file [sdl.cpp](#).

References [get_mouse_state\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.29 mxvm_sdl_get_mouse_x()

```
void mxvm_sdl_get_mouse_x (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 438 of file [sdl.cpp](#).

References [get_mouse_x\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.30 mxvm_sdl_get_mouse_y()

```
void mxvm_sdl_get_mouse_y (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 446 of file [sdl.cpp](#).

References [get_mouse_y\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.31 mxvm_sdl_get_num_keys()

```
void mxvm_sdl_get_num_keys (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 313 of file [sdl.cpp](#).

References [get_num_keys\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.32 mxvm_sdl_get_queued_audio_size()

```
void mxvm_sdl_get_queued_audio_size (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 815 of file [sdl.cpp](#).

References [get_queued_audio_size\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.33 mxvm_sdl_get_relative_mouse_buttons()

```
void mxvm_sdl_get_relative_mouse_buttons (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 213 of file [sdl.cpp](#).

References [get_relative_mouse_buttons\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.34 mxvm_sdl_get_relative_mouse_state()

```
void mxvm_sdl_get_relative_mouse_state (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 249 of file [sdl.cpp](#).

References [get_relative_mouse_state\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.35 mxvm_sdl_get_relative_mouse_x()

```
void mxvm_sdl_get_relative_mouse_x (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 199 of file [sdl.cpp](#).

References [get_relative_mouse_x\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.36 mxvm_sdl_get_relative_mouse_y()

```
void mxvm_sdl_get_relative_mouse_y (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 206 of file [sdl.cpp](#).

References [get_relative_mouse_y\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.37 mxvm_sdl_get_renderer_output_size()

```
void mxvm_sdl_get_renderer_output_size (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 164 of file [sdl.cpp](#).

References [get_renderer_output_size\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_INTEGER](#), and [mxvm::Variable::var_value](#).

12.34.2.38 mxvm_sdl_get_ticks()

```
void mxvm_sdl_get_ticks (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 697 of file [sdl.cpp](#).

References [get_ticks\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.39 mxvm_sdl_get_window_size()

```
void mxvm_sdl_get_window_size (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 126 of file [sdl.cpp](#).

References [get_window_size\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::Variable_Value::type](#), [mxvm::VAR_INTEGER](#), and [mxvm::Variable::var_value](#).

12.34.2.40 mxvm_sdl_init()

```
void mxvm_sdl_init (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 11 of file [sdl.cpp](#).

References [init\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.41 mxvm_sdl_init_text()

```
void mxvm_sdl_init_text (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 893 of file [sdl.cpp](#).

References [init_text\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.42 mxvm_sdl_is_key_pressed()

```
void mxvm_sdl_is_key_pressed (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 300 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [is_key_pressed\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.43 mxvm_sdl_load_font()

```
void mxvm_sdl_load_font (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 904 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [load_font\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.44 mxvm_sdl_load_texture()

```
void mxvm_sdl_load_texture (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 586 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [load_texture\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.45 mxvm_sdl_load_texture_color_key()

```
void mxvm_sdl_load_texture_color_key (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 612 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [load_texture_color_key\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.46 mxvm_sdl_load_texture_color_key_rgb()

```
void mxvm_sdl_load_texture_color_key_rgb (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

MXVM binding for `load_texture_color_key_rgb`.

Loads a BMP texture with a caller-specified RGB color key for transparency. Expects 5 operands: `renderer_id`, `file_path`, `r`, `g`, `b`. Returns the texture ID in `%rax`.

Parameters

<i>program</i>	Pointer to the running MXVM program.
<i>operand</i>	Vector of 5 operands.

Definition at line 647 of file `sdl.cpp`.

References `mxvm::Program::getVariable()`, `mxvm::Variable_Value::int_value`, `mxvm::Program::isVariable()`, `load_texture_color_key_rgb()`, `mxvm::Variable_Value::ptr_value`, `mxvm::Variable_Value::str_value`, `mxvm::Variable::type`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, `mxvm::Variable::var_value`, and `mxvm::Base::vars`.

12.34.2.47 `mxvm_sdl_load_wav()`

```
void mxvm_sdl_load_wav (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 745 of file `sdl.cpp`.

References `mxvm::Program::getVariable()`, `mxvm::Variable_Value::int_value`, `mxvm::Program::isVariable()`, `load_wav()`, `mxvm::Variable_Value::ptr_value`, `mxvm::Variable_Value::str_value`, `mxvm::Variable::type`, `mxvm::Variable_Value::type`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::VAR_STRING`, `mxvm::Variable::var_value`, and `mxvm::Base::vars`.

12.34.2.48 `mxvm_sdl_lock_texture()`

```
void mxvm_sdl_lock_texture (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 852 of file `sdl.cpp`.

References `mxvm::Program::getVariable()`, `mxvm::Variable_Value::int_value`, `mxvm::Program::isVariable()`, `lock_texture()`, `mxvm::Variable_Value::ptr_value`, `mxvm::Variable::type`, `mxvm::Variable_Value::type`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, `mxvm::Variable::var_value`, and `mxvm::Base::vars`.

12.34.2.49 `mxvm_sdl_open_audio()`

```
void mxvm_sdl_open_audio (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 715 of file `sdl.cpp`.

References `mxvm::Program::getVariable()`, `mxvm::Variable_Value::int_value`, `mxvm::Program::isVariable()`, `open_audio()`, `mxvm::VAR_INTEGER`, `mxvm::Variable::var_value`, and `mxvm::Base::vars`.

12.34.2.50 `mxvm_sdl_pause_audio()`

```
void mxvm_sdl_pause_audio (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 735 of file `sdl.cpp`.

References `mxvm::Program::getVariable()`, `mxvm::Variable_Value::int_value`, `mxvm::Program::isVariable()`, `pause_audio()`, and `mxvm::Variable::var_value`.

12.34.2.51 `mxvm_sdl_poll_event()`

```
void mxvm_sdl_poll_event (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 414 of file `sdl.cpp`.

References [poll_event\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.34.2.52 mxvm_sdl_present()

```
void mxvm_sdl_present (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 488 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [present\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.53 mxvm_sdl_present_scaled()

```
void mxvm_sdl_present_scaled (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 992 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [present_scaled\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.54 mxvm_sdl_present_stretched()

```
void mxvm_sdl_present_stretched (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 1005 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [present_stretched\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.55 mxvm_sdl_queue_audio()

```
void mxvm_sdl_queue_audio (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 792 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [queue_audio\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.34.2.56 mxvm_sdl_quit()

```
void mxvm_sdl_quit (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 19 of file [sdl.cpp](#).

References [quit\(\)](#).

12.34.2.57 mxvm_sdl_quit_text()

```
void mxvm_sdl_quit_text (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 900 of file [sdl.cpp](#).

References [quit_text\(\)](#).

12.34.2.58 mxvm_sdl_render_texture()

```
void mxvm_sdl_render_texture (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 677 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [render_texture\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.59 mxvm_sdl_set_clipboard_text()

```
void mxvm_sdl_set_clipboard_text (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 321 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [set_clipboard_text\(\)](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

12.34.2.60 mxvm_sdl_set_draw_color()

```
void mxvm_sdl_set_draw_color (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 462 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [set_draw_color\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.61 mxvm_sdl_set_render_target()

```
void mxvm_sdl_set_render_target (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 971 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [set_render_target\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.62 mxvm_sdl_set_window_fullscreen()

```
void mxvm_sdl_set_window_fullscreen (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 153 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [set_window_fullscreen\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.63 mxvm_sdl_set_window_icon()

```
void mxvm_sdl_set_window_icon (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 1018 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [set_window_icon\(\)](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

12.34.2.64 mxvm_sdl_set_window_position()

```
void mxvm_sdl_set_window_position (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 114 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [set_window_position\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.65 mxvm_sdl_set_window_title()

```
void mxvm_sdl_set_window_title (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 92 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [set_window_title\(\)](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

12.34.2.66 mxvm_sdl_show_cursor()

```
void mxvm_sdl_show_cursor (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 350 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [show_cursor\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.67 mxvm_sdl_unlock_texture()

```
void mxvm_sdl_unlock_texture (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 883 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [unlock_texture\(\)](#), and [mxvm::Variable::var_value](#).

12.34.2.68 mxvm_sdl_update_texture()

```
void mxvm_sdl_update_texture (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 827 of file [sdl.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [update_texture\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.35 sdl.cpp

[Go to the documentation of this file.](#)

```
00001
00006 #include "mx_sdl.h"
00007 #include <cstdint>
00008 #include <mxvm/icode.hpp>
00009 #include <mxvm/instruct.hpp>
00010
00011 extern "C" void mxvm_sdl_init(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00012     int64_t result = init();
00013     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00014     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00015     program->vars["%rax"].var_value.int_value = result;
00016 }
00017
00018 // SDL_Quit
00019 extern "C" void mxvm_sdl_quit(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00020     quit();
00021 }
00022
00023 // SDL_CreateWindow
00024 extern "C" void mxvm_sdl_create_window(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00025     if (operand.size() != 6) {
00026         throw mx::Exception("sdl_create_window requires 6 arguments (title, x, y, w, h, flags).");
00027     }
00028
00029     std::string title;
00030     if (program->isVariable(operand[0].op)) {
```

```

00031         mxvm::Variable &var = program->getVariable(operand[0].op);
00032         if (var.type == mxvm::VarType::VAR_STRING) {
00033             title = var.var_value.str_value;
00034         } else if (var.type == mxvm::VarType::VAR_POINTER) {
00035             title = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00036         }
00037     } else {
00038         title = operand[0].op;
00039     }
00040
00041     int64_t x = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00042     int64_t y = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00043     int64_t w = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00044     int64_t h = program->isVariable(operand[4].op) ?
program->getVariable(operand[4].op).var_value.int_value : operand[4].op_value;
00045     int64_t flags = program->isVariable(operand[5].op) ?
program->getVariable(operand[5].op).var_value.int_value : operand[5].op_value;
00046
00047     int64_t result = create_window(title.c_str(), x, y, w, h, flags);
00048     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00049     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00050     program->vars["%rax"].var_value.int_value = result;
00051 }
00052
00053 // SDL_DestroyWindow
00054 extern "C" void mxvm_sdl_destroy_window(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00055     if (operand.size() != 1) {
00056         throw mx::Exception("sdl_destroy_window requires 1 argument (window_id).");
00057     }
00058
00059     int64_t window_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00060
00061     destroy_window(window_id);
00062 }
00063
00064 // SDL_CreateRenderer
00065 extern "C" void mxvm_sdl_create_renderer(mxvm::Program *program, std::vector<mxvm::Operand> &operand)
{
00066     if (operand.size() != 3) {
00067         throw mx::Exception("sdl_create_renderer requires 3 arguments (window_id, index, flags).");
00068     }
00069
00070     int64_t window_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00071     int64_t index = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00072     int64_t flags = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00073
00074     int64_t result = create_renderer(window_id, index, flags);
00075     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00076     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00077     program->vars["%rax"].var_value.int_value = result;
00078 }
00079
00080 // SDL_DestroyRenderer
00081 extern "C" void mxvm_sdl_destroy_renderer(mxvm::Program *program, std::vector<mxvm::Operand> &operand)
{
00082     if (operand.size() != 1) {
00083         throw mx::Exception("sdl_destroy_renderer requires 1 argument (renderer_id).");
00084     }
00085
00086     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00087
00088     destroy_renderer(renderer_id);
00089 }
00090
00091 // Window management functions
00092 extern "C" void mxvm_sdl_set_window_title(mxvm::Program *program, std::vector<mxvm::Operand> &operand)
{
00093     if (operand.size() != 2) {
00094         throw mx::Exception("sdl_set_window_title requires 2 arguments (window_id, title).");
00095     }
00096
00097     int64_t window_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00098
00099     std::string title;
00100     if (program->isVariable(operand[1].op)) {
00101         mxvm::Variable &var = program->getVariable(operand[1].op);
00102         if (var.type == mxvm::VarType::VAR_STRING) {
00103             title = var.var_value.str_value;

```

```

00104         } else if (var.type == mxvm::VarType::VAR_POINTER) {
00105             title = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00106         }
00107     } else {
00108         title = operand[1].op;
00109     }
00110
00111     set_window_title(window_id, title.c_str());
00112 }
00113
00114 extern "C" void mxvm_sdl_set_window_position(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00115     if (operand.size() != 3) {
00116         throw mx::Exception("sdl_set_window_position requires 3 arguments (window_id, x, y).");
00117     }
00118
00119     int64_t window_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00120     int64_t x = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00121     int64_t y = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00122
00123     set_window_position(window_id, x, y);
00124 }
00125
00126 extern "C" void mxvm_sdl_get_window_size(mxvm::Program *program, std::vector<mxvm::Operand> &operand)
{
00127     if (operand.size() != 3) {
00128         throw mx::Exception("sdl_get_window_size requires 3 arguments (window_id, w_var, h_var).");
00129     }
00130
00131     int64_t window_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00132
00133     int64_t w, h;
00134     get_window_size(window_id, &w, &h);
00135
00136     // Store width in the second operand variable
00137     if (program->isVariable(operand[1].op)) {
00138         mxvm::Variable &w_var = program->getVariable(operand[1].op);
00139         w_var.type = mxvm::VarType::VAR_INTEGER;
00140         w_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00141         w_var.var_value.int_value = w;
00142     }
00143
00144     // Store height in the third operand variable
00145     if (program->isVariable(operand[2].op)) {
00146         mxvm::Variable &h_var = program->getVariable(operand[2].op);
00147         h_var.type = mxvm::VarType::VAR_INTEGER;
00148         h_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00149         h_var.var_value.int_value = h;
00150     }
00151 }
00152
00153 extern "C" void mxvm_sdl_set_window_fullscreen(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00154     if (operand.size() != 2) {
00155         throw mx::Exception("sdl_set_window_fullscreen requires 2 arguments (window_id,
fullscreen).");
00156     }
00157
00158     int64_t window_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00159     int64_t fullscreen = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00160
00161     set_window_fullscreen(window_id, fullscreen);
00162 }
00163
00164 extern "C" void mxvm_sdl_get_renderer_output_size(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00165     if (operand.size() != 3) {
00166         throw mx::Exception("sdl_get_renderer_output_size requires 3 arguments (renderer_id, w_var,
h_var).");
00167     }
00168
00169     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00170
00171     int64_t w, h;
00172     get_renderer_output_size(renderer_id, &w, &h);
00173
00174     // Store width in the second operand variable
00175     if (program->isVariable(operand[1].op)) {
00176         mxvm::Variable &w_var = program->getVariable(operand[1].op);
00177         w_var.type = mxvm::VarType::VAR_INTEGER;

```

```

00178         w_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00179         w_var.var_value.int_value = w;
00180     }
00181
00182     // Store height in the third operand variable
00183     if (program->isVariable(operand[2].op)) {
00184         mxvm::Variable &h_var = program->getVariable(operand[2].op);
00185         h_var.type = mxvm::VarType::VAR_INTEGER;
00186         h_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00187         h_var.var_value.int_value = h;
00188     }
00189 }
00190
00191 // Mouse functions
00192 extern "C" void mxvm_sdl_get_mouse_buttons(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00193     int64_t result = get_mouse_buttons();
00194     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00195     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00196     program->vars["%rax"].var_value.int_value = result;
00197 }
00198
00199 extern "C" void mxvm_sdl_get_relative_mouse_x(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00200     int64_t result = get_relative_mouse_x();
00201     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00202     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00203     program->vars["%rax"].var_value.int_value = result;
00204 }
00205
00206 extern "C" void mxvm_sdl_get_relative_mouse_y(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00207     int64_t result = get_relative_mouse_y();
00208     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00209     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00210     program->vars["%rax"].var_value.int_value = result;
00211 }
00212
00213 extern "C" void mxvm_sdl_get_relative_mouse_buttons(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00214     int64_t result = get_relative_mouse_buttons();
00215     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00216     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00217     program->vars["%rax"].var_value.int_value = result;
00218 }
00219
00220 extern "C" void mxvm_sdl_get_mouse_state(mxvm::Program *program, std::vector<mxvm::Operand> &operand)
{
00221     if (operand.size() != 2) {
00222         throw mx::Exception("sdl_get_mouse_state requires 2 arguments (x_var, y_var).");
00223     }
00224
00225     int64_t x, y;
00226     int64_t result = get_mouse_state(&x, &y);
00227
00228     // Store x in the first operand variable
00229     if (program->isVariable(operand[0].op)) {
00230         mxvm::Variable &x_var = program->getVariable(operand[0].op);
00231         x_var.type = mxvm::VarType::VAR_INTEGER;
00232         x_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00233         x_var.var_value.int_value = x;
00234     }
00235
00236     // Store y in the second operand variable
00237     if (program->isVariable(operand[1].op)) {
00238         mxvm::Variable &y_var = program->getVariable(operand[1].op);
00239         y_var.type = mxvm::VarType::VAR_INTEGER;
00240         y_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00241         y_var.var_value.int_value = y;
00242     }
00243
00244     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00245     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00246     program->vars["%rax"].var_value.int_value = result;
00247 }
00248
00249 extern "C" void mxvm_sdl_get_relative_mouse_state(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00250     if (operand.size() != 2) {
00251         throw mx::Exception("sdl_get_relative_mouse_state requires 2 arguments (x_var, y_var).");
00252     }
00253
00254     int64_t x, y;
00255     int64_t result = get_relative_mouse_state(&x, &y);
00256
00257     // Store x in the first operand variable
00258     if (program->isVariable(operand[0].op)) {

```



```

00259         mxvm::Variable &x_var = program->getVariable(operand[0].op);
00260         x_var.type = mxvm::VarType::VAR_INTEGER;
00261         x_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00262         x_var.var_value.int_value = x;
00263     }
00264
00265     // Store y in the second operand variable
00266     if (program->isVariable(operand[1].op)) {
00267         mxvm::Variable &y_var = program->getVariable(operand[1].op);
00268         y_var.type = mxvm::VarType::VAR_INTEGER;
00269         y_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00270         y_var.var_value.int_value = y;
00271     }
00272
00273     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00274     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00275     program->vars["%rax"].var_value.int_value = result;
00276 }
00277
00278 // Keyboard functions
00279 extern "C" void mxvm_sdl_get_keyboard_state(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00280     if (operand.size() != 1) {
00281         throw mx::Exception("sdl_get_keyboard_state requires 1 argument (numkeys_var).");
00282     }
00283
00284     int64_t numkeys;
00285     int64_t result = get_keyboard_state(&numkeys);
00286
00287     // Store numkeys in the operand variable
00288     if (program->isVariable(operand[0].op)) {
00289         mxvm::Variable &nk_var = program->getVariable(operand[0].op);
00290         nk_var.type = mxvm::VarType::VAR_INTEGER;
00291         nk_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00292         nk_var.var_value.int_value = numkeys;
00293     }
00294
00295     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00296     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_POINTER;
00297     program->vars["%rax"].var_value.ptr_value = (void *)result;
00298 }
00299
00300 extern "C" void mxvm_sdl_is_key_pressed(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00301     if (operand.size() != 1) {
00302         throw mx::Exception("sdl_is_key_pressed requires 1 argument (scancode).");
00303     }
00304
00305     int64_t scancode = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00306
00307     int64_t result = is_key_pressed(scancode);
00308     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00309     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00310     program->vars["%rax"].var_value.int_value = result;
00311 }
00312
00313 extern "C" void mxvm_sdl_get_num_keys(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00314     int64_t result = get_num_keys();
00315     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00316     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00317     program->vars["%rax"].var_value.int_value = result;
00318 }
00319
00320 // Clipboard functions
00321 extern "C" void mxvm_sdl_set_clipboard_text(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00322     if (operand.size() != 1) {
00323         throw mx::Exception("sdl_set_clipboard_text requires 1 argument (text).");
00324     }
00325
00326     std::string text;
00327     if (program->isVariable(operand[0].op)) {
00328         mxvm::Variable &var = program->getVariable(operand[0].op);
00329         if (var.type == mxvm::VarType::VAR_STRING) {
00330             text = var.var_value.str_value;
00331         } else if (var.type == mxvm::VarType::VAR_POINTER) {
00332             text = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00333         }
00334     } else {
00335         text = operand[0].op;
00336     }
00337
00338     set_clipboard_text(text.c_str());
00339 }
00340
00341 extern "C" void mxvm_sdl_get_clipboard_text(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {

```

```

00342     const char *result = get_clipboard_text();
00343     // Store result as a string pointer in %rax
00344     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00345     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_POINTER;
00346     program->vars["%rax"].var_value.ptr_value = (void *)result;
00347 }
00348
00349 // Cursor functions
00350 extern "C" void mxvm_sdl_show_cursor(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00351     if (operand.size() != 1) {
00352         throw mx::Exception("sdl_show_cursor requires 1 argument (show).");
00353     }
00354
00355     int64_t show = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00356
00357     show_cursor(show);
00358 }
00359
00360 // Surface functions
00361 extern "C" void mxvm_sdl_create_rgb_surface(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00362     if (operand.size() != 3) {
00363         throw mx::Exception("sdl_create_rgb_surface requires 3 arguments (width, height, depth).");
00364     }
00365
00366     int64_t width = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00367     int64_t height = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00368     int64_t depth = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00369
00370     int64_t result = create_rgb_surface(width, height, depth);
00371     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00372     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_POINTER;
00373     program->vars["%rax"].var_value.ptr_value = (void *)result;
00374 }
00375
00376 extern "C" void mxvm_sdl_free_surface(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00377     if (operand.size() != 1) {
00378         throw mx::Exception("sdl_free_surface requires 1 argument (surface_ptr).");
00379     }
00380
00381     int64_t surf_ptr = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00382
00383     free_surface(surf_ptr);
00384 }
00385
00386 extern "C" void mxvm_sdl_blit_surface(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00387     if (operand.size() != 4) {
00388         throw mx::Exception("sdl_blit_surface requires 4 arguments (src_ptr, dst_ptr, x, y).");
00389     }
00390
00391     int64_t src_ptr = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00392     int64_t dst_ptr = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00393     int64_t x = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00394     int64_t y = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00395
00396     int64_t result = blit_surface(src_ptr, dst_ptr, x, y);
00397     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00398     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00399     program->vars["%rax"].var_value.int_value = result;
00400 }
00401
00402 // Additional functions
00403 extern "C" void mxvm_sdl_free_wav(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00404     if (operand.size() != 1) {
00405         throw mx::Exception("sdl_free_wav requires 1 argument (audio_buf).");
00406     }
00407
00408     int64_t audio_buf = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00409
00410     free_wav(audio_buf);
00411 }
00412
00413 // SDL_PollEvent
00414 extern "C" void mxvm_sdl_poll_event(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00415     int64_t result = poll_event();
00416     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00417     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;

```

```

00418     program->vars["%rax"].var_value.int_value = result;
00419 }
00420
00421 // SDL_GetEventType
00422 extern "C" void mxvm_sdl_get_event_type(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00423     int64_t result = get_event_type();
00424     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00425     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00426     program->vars["%rax"].var_value.int_value = result;
00427 }
00428
00429 // SDL_GetKeyCode
00430 extern "C" void mxvm_sdl_get_key_code(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00431     int64_t result = get_key_code();
00432     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00433     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00434     program->vars["%rax"].var_value.int_value = result;
00435 }
00436
00437 // SDL_GetMouseX
00438 extern "C" void mxvm_sdl_get_mouse_x(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00439     int64_t result = get_mouse_x();
00440     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00441     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00442     program->vars["%rax"].var_value.int_value = result;
00443 }
00444
00445 // SDL_GetMouseY
00446 extern "C" void mxvm_sdl_get_mouse_y(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00447     int64_t result = get_mouse_y();
00448     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00449     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00450     program->vars["%rax"].var_value.int_value = result;
00451 }
00452
00453 // SDL_GetMouseButton
00454 extern "C" void mxvm_sdl_get_mouse_button(mxvm::Program *program, std::vector<mxvm::Operand> &operand)
00455 {
00456     int64_t result = get_mouse_button();
00457     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00458     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00459     program->vars["%rax"].var_value.int_value = result;
00460 }
00461 // SDL_SetDrawColor
00462 extern "C" void mxvm_sdl_set_draw_color(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00463     if (operand.size() != 5) {
00464         throw mx::Exception("sdl_set_draw_color requires 5 arguments (renderer_id, r, g, b, a).");
00465     }
00466
00467     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00468     int64_t r = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00469     int64_t g = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00470     int64_t b = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00471     int64_t a = program->isVariable(operand[4].op) ?
program->getVariable(operand[4].op).var_value.int_value : operand[4].op_value;
00472
00473     set_draw_color(renderer_id, r, g, b, a);
00474 }
00475
00476 // SDL_Clear
00477 extern "C" void mxvm_sdl_clear(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00478     if (operand.size() != 1) {
00479         throw mx::Exception("sdl_clear requires 1 argument (renderer_id).");
00480     }
00481
00482     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00483
00484     clear(renderer_id);
00485 }
00486
00487 // SDL_Present
00488 extern "C" void mxvm_sdl_present(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00489     if (operand.size() != 1) {
00490         throw mx::Exception("sdl_present requires 1 argument (renderer_id).");
00491     }
00492
00493     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00494
00495     present(renderer_id);
00496 }

```

```

00497
00498 // SDL_DrawPoint
00499 extern "C" void mxvm_sdl_draw_point(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00500     if (operand.size() != 3) {
00501         throw mx::Exception("sdl_draw_point requires 3 arguments (renderer_id, x, y).");
00502     }
00503
00504     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00505     int64_t x = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00506     int64_t y = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00507
00508     draw_point(renderer_id, x, y);
00509 }
00510
00511 // SDL_DrawLine
00512 extern "C" void mxvm_sdl_draw_line(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00513     if (operand.size() != 5) {
00514         throw mx::Exception("sdl_draw_line requires 5 arguments (renderer_id, x1, y1, x2, y2).");
00515     }
00516
00517     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00518     int64_t x1 = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00519     int64_t y1 = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00520     int64_t x2 = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00521     int64_t y2 = program->isVariable(operand[4].op) ?
program->getVariable(operand[4].op).var_value.int_value : operand[4].op_value;
00522
00523     draw_line(renderer_id, x1, y1, x2, y2);
00524 }
00525
00526 // SDL_DrawRect
00527 extern "C" void mxvm_sdl_draw_rect(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00528     if (operand.size() != 5) {
00529         throw mx::Exception("sdl_draw_rect requires 5 arguments (renderer_id, x, y, w, h).");
00530     }
00531
00532     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00533     int64_t x = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00534     int64_t y = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00535     int64_t w = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00536     int64_t h = program->isVariable(operand[4].op) ?
program->getVariable(operand[4].op).var_value.int_value : operand[4].op_value;
00537
00538     draw_rect(renderer_id, x, y, w, h);
00539 }
00540
00541 // SDL_FillRect
00542 extern "C" void mxvm_sdl_fill_rect(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00543     if (operand.size() != 5) {
00544         throw mx::Exception("sdl_fill_rect requires 5 arguments (renderer_id, x, y, w, h).");
00545     }
00546
00547     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00548     int64_t x = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00549     int64_t y = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00550     int64_t w = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00551     int64_t h = program->isVariable(operand[4].op) ?
program->getVariable(operand[4].op).var_value.int_value : operand[4].op_value;
00552
00553     fill_rect(renderer_id, x, y, w, h);
00554 }
00555
00556 // SDL_CreateTexture
00557 extern "C" void mxvm_sdl_create_texture(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00558     if (operand.size() != 5) {
00559         throw mx::Exception("sdl_create_texture requires 5 arguments (renderer_id, format, access, w,
h).");
00560     }
00561
00562     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00563     int64_t format = program->isVariable(operand[1].op) ?

```

```

    program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00564     int64_t access = program->isVariable(operand[2].op) ?
    program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00565     int64_t w = program->isVariable(operand[3].op) ?
    program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00566     int64_t h = program->isVariable(operand[4].op) ?
    program->getVariable(operand[4].op).var_value.int_value : operand[4].op_value;
00567
00568     int64_t result = create_texture(renderer_id, format, access, w, h);
00569     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00570     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00571     program->vars["%rax"].var_value.int_value = result;
00572 }
00573
00574 // SDL_DestroyTexture
00575 extern "C" void mxvm_sdl_destroy_texture(mxvm::Program *program, std::vector<mxvm::Operand> &operand)
{
00576     if (operand.size() != 1) {
00577         throw mx::Exception("sdl_destroy_texture requires 1 argument (texture_id).");
00578     }
00579
00580     int64_t texture_id = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00581
00582     destroy_texture(texture_id);
00583 }
00584
00585 // SDL_LoadTexture
00586 extern "C" void mxvm_sdl_load_texture(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00587     if (operand.size() != 2) {
00588         throw mx::Exception("sdl_load_texture requires 2 arguments (renderer_id, file_path).");
00589     }
00590
00591     int64_t renderer_id = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00592
00593     std::string file_path;
00594     if (program->isVariable(operand[1].op)) {
00595         mxvm::Variable &var = program->getVariable(operand[1].op);
00596         if (var.type == mxvm::VarType::VAR_STRING) {
00597             file_path = var.var_value.str_value;
00598         } else if (var.type == mxvm::VarType::VAR_POINTER) {
00599             file_path = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00600         }
00601     } else {
00602         file_path = operand[1].op;
00603     }
00604
00605     int64_t result = load_texture(renderer_id, file_path.c_str());
00606     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00607     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00608     program->vars["%rax"].var_value.int_value = result;
00609 }
00610
00611 // SDL_LoadTextureColorKey
00612 extern "C" void mxvm_sdl_load_texture_color_key(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00613     if (operand.size() != 2) {
00614         throw mx::Exception("sdl_load_texture_color_key requires 2 arguments (renderer_id,
file_path).");
00615     }
00616
00617     int64_t renderer_id = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00618
00619     std::string file_path;
00620     if (program->isVariable(operand[1].op)) {
00621         mxvm::Variable &var = program->getVariable(operand[1].op);
00622         if (var.type == mxvm::VarType::VAR_STRING) {
00623             file_path = var.var_value.str_value;
00624         } else if (var.type == mxvm::VarType::VAR_POINTER) {
00625             file_path = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00626         }
00627     } else {
00628         file_path = operand[1].op;
00629     }
00630
00631     int64_t result = load_texture_color_key(renderer_id, file_path.c_str());
00632     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00633     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00634     program->vars["%rax"].var_value.int_value = result;
00635 }
00636
00647 extern "C" void mxvm_sdl_load_texture_color_key_rgb(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00648     if (operand.size() != 5) {
00649         throw mx::Exception("sdl_load_texture_color_key_rgb requires 5 arguments (renderer_id,

```

```

        file_path, r, g, b).");
00650     }
00651
00652     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00653
00654     std::string file_path;
00655     if (program->isVariable(operand[1].op)) {
00656         mxvm::Variable &var = program->getVariable(operand[1].op);
00657         if (var.type == mxvm::VarType::VAR_STRING) {
00658             file_path = var.var_value.str_value;
00659         } else if (var.type == mxvm::VarType::VAR_POINTER) {
00660             file_path = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00661         }
00662     } else {
00663         file_path = operand[1].op;
00664     }
00665
00666     int64_t r = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00667     int64_t g = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00668     int64_t b = program->isVariable(operand[4].op) ?
program->getVariable(operand[4].op).var_value.int_value : operand[4].op_value;
00669
00670     int64_t result = load_texture_color_key_rgb(renderer_id, file_path.c_str(), r, g, b);
00671     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00672     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00673     program->vars["%rax"].var_value.int_value = result;
00674 }
00675
00676 // SDL_RenderTexture
00677 extern "C" void mxvm_sdl_render_texture(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00678     if (operand.size() != 10) {
00679         throw mx::Exception("sdl_render_texture requires 10 arguments (renderer_id, texture_id, src_x,
src_y, src_w, src_h, dst_x, dst_y, dst_w, dst_h).");
00680     }
00681
00682     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00683     int64_t texture_id = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00684     int64_t src_x = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00685     int64_t src_y = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00686     int64_t src_w = program->isVariable(operand[4].op) ?
program->getVariable(operand[4].op).var_value.int_value : operand[4].op_value;
00687     int64_t src_h = program->isVariable(operand[5].op) ?
program->getVariable(operand[5].op).var_value.int_value : operand[5].op_value;
00688     int64_t dst_x = program->isVariable(operand[6].op) ?
program->getVariable(operand[6].op).var_value.int_value : operand[6].op_value;
00689     int64_t dst_y = program->isVariable(operand[7].op) ?
program->getVariable(operand[7].op).var_value.int_value : operand[7].op_value;
00690     int64_t dst_w = program->isVariable(operand[8].op) ?
program->getVariable(operand[8].op).var_value.int_value : operand[8].op_value;
00691     int64_t dst_h = program->isVariable(operand[9].op) ?
program->getVariable(operand[9].op).var_value.int_value : operand[9].op_value;
00692
00693     render_texture(renderer_id, texture_id, src_x, src_y, src_w, src_h, dst_x, dst_y, dst_w, dst_h);
00694 }
00695
00696 // Timing functions
00697 extern "C" void mxvm_sdl_get_ticks(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00698     int64_t result = get_ticks();
00699     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00700     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00701     program->vars["%rax"].var_value.int_value = result;
00702 }
00703
00704 extern "C" void mxvm_sdl_delay(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00705     if (operand.size() != 1) {
00706         throw mx::Exception("sdl_delay requires 1 argument (ms).");
00707     }
00708
00709     int64_t ms = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00710
00711     delay(ms);
00712 }
00713
00714 // Audio functions
00715 extern "C" void mxvm_sdl_open_audio(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00716     if (operand.size() != 4) {
00717         throw mx::Exception("sdl_open_audio requires 4 arguments (freq, format, channels, samples).");
00718     }
00719

```

```

00720     int64_t frequency = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00721     int64_t format = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00722     int64_t channels = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00723     int64_t samples = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00724
00725     int64_t result = open_audio(frequency, format, channels, samples);
00726     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00727     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00728     program->vars["%rax"].var_value.int_value = result;
00729 }
00730
00731 extern "C" void mxvm_sdl_close_audio(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00732     close_audio();
00733 }
00734
00735 extern "C" void mxvm_sdl_pause_audio(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00736     if (operand.size() != 1) {
00737         throw mx::Exception("sdl_pause_audio requires 1 argument (pause_on).");
00738     }
00739
00740     int64_t pause_on = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00741
00742     pause_audio(pause_on);
00743 }
00744
00745 extern "C" void mxvm_sdl_load_wav(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00746     if (operand.size() != 4) {
00747         throw mx::Exception("sdl_load_wav requires 4 arguments (file_path, audio_buf_var,
audio_len_var, audio_spec_var).");
00748     }
00749
00750     std::string file_path;
00751     if (program->isVariable(operand[0].op)) {
00752         mxvm::Variable &var = program->getVariable(operand[0].op);
00753         if (var.type == mxvm::VarType::VAR_STRING) {
00754             file_path = var.var_value.str_value;
00755         } else if (var.type == mxvm::VarType::VAR_POINTER) {
00756             file_path = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00757         }
00758     } else {
00759         file_path = operand[0].op;
00760     }
00761
00762     int64_t audio_buf, audio_len, audio_spec;
00763     int64_t result = load_wav(file_path.c_str(), &audio_buf, &audio_len, &audio_spec);
00764
00765     // Store results in variables
00766     if (program->isVariable(operand[1].op)) {
00767         mxvm::Variable &buf_var = program->getVariable(operand[1].op);
00768         buf_var.type = mxvm::VarType::VAR_INTEGER;
00769         buf_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00770         buf_var.var_value.int_value = audio_buf;
00771     }
00772
00773     if (program->isVariable(operand[2].op)) {
00774         mxvm::Variable &len_var = program->getVariable(operand[2].op);
00775         len_var.type = mxvm::VarType::VAR_INTEGER;
00776         len_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00777         len_var.var_value.int_value = audio_len;
00778     }
00779
00780     if (program->isVariable(operand[3].op)) {
00781         mxvm::Variable &spec_var = program->getVariable(operand[3].op);
00782         spec_var.type = mxvm::VarType::VAR_INTEGER;
00783         spec_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00784         spec_var.var_value.int_value = audio_spec;
00785     }
00786
00787     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00788     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00789     program->vars["%rax"].var_value.int_value = result;
00790 }
00791
00792 extern "C" void mxvm_sdl_queue_audio(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00793     if (operand.size() != 2) {
00794         throw mx::Exception("sdl_queue_audio requires 2 arguments (data, len).");
00795     }
00796
00797     void *data = nullptr;
00798     if (program->isVariable(operand[0].op)) {
00799         mxvm::Variable &var = program->getVariable(operand[0].op);
00800         if (var.type == mxvm::VarType::VAR_POINTER) {

```



```

00801         data = var.var_value.ptr_value;
00802     }
00803 } else {
00804     data = reinterpret_cast<void *>(static_cast<uintptr_t>(operand[0].op_value));
00805 }
00806
00807     int64_t len = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00808
00809     int64_t result = queue_audio(data, len);
00810     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00811     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00812     program->vars["%rax"].var_value.int_value = result;
00813 }
00814
00815 extern "C" void mxvm_sdl_get_queued_audio_size(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00816     int64_t result = get_queued_audio_size();
00817     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00818     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00819     program->vars["%rax"].var_value.int_value = result;
00820 }
00821
00822 extern "C" void mxvm_sdl_clear_queued_audio(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00823     clear_queued_audio();
00824 }
00825
00826 // Texture functions
00827 extern "C" void mxvm_sdl_update_texture(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00828     if (operand.size() != 3) {
00829         throw mx::Exception("sdl_update_texture requires 3 arguments (texture_id, pixels, pitch).");
00830     }
00831
00832     int64_t texture_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00833
00834     void *pixels = nullptr;
00835     if (program->isVariable(operand[1].op)) {
00836         mxvm::Variable &var = program->getVariable(operand[1].op);
00837         if (var.type == mxvm::VarType::VAR_POINTER) {
00838             pixels = var.var_value.ptr_value;
00839         }
00840     } else {
00841         pixels = reinterpret_cast<void *>(static_cast<uintptr_t>(operand[1].op_value));
00842     }
00843
00844     int64_t pitch = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00845
00846     int64_t result = update_texture(texture_id, pixels, pitch);
00847     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00848     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00849     program->vars["%rax"].var_value.int_value = result;
00850 }
00851
00852 extern "C" void mxvm_sdl_lock_texture(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00853     if (operand.size() != 3) {
00854         throw mx::Exception("sdl_lock_texture requires 3 arguments (texture_id, pixels_var,
pitch_var).");
00855     }
00856
00857     int64_t texture_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00858
00859     void *pixels;
00860     int64_t pitch;
00861     int64_t result = lock_texture(texture_id, &pixels, &pitch);
00862
00863     // Store results in variables
00864     if (program->isVariable(operand[1].op)) {
00865         mxvm::Variable &pixels_var = program->getVariable(operand[1].op);
00866         pixels_var.type = mxvm::VarType::VAR_POINTER;
00867         pixels_var.var_value.type = mxvm::VarType::VAR_POINTER;
00868         pixels_var.var_value.ptr_value = pixels;
00869     }
00870
00871     if (program->isVariable(operand[2].op)) {
00872         mxvm::Variable &pitch_var = program->getVariable(operand[2].op);
00873         pitch_var.type = mxvm::VarType::VAR_INTEGER;
00874         pitch_var.var_value.type = mxvm::VarType::VAR_INTEGER;
00875         pitch_var.var_value.int_value = pitch;
00876     }
00877
00878     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00879     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00880     program->vars["%rax"].var_value.int_value = result;

```



```

00881 }
00882
00883 extern "C" void mxvm_sdl_unlock_texture(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00884     if (operand.size() != 1) {
00885         throw mx::Exception("sdl_unlock_texture requires 1 argument (texture_id).");
00886     }
00887
00888     int64_t texture_id = program->isVariable(operand[0].op) ?
00889         program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00889
00890     unlock_texture(texture_id);
00891 }
00892
00893 extern "C" void mxvm_sdl_init_text(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00894     int64_t result = init_text();
00895     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00896     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00897     program->vars["%rax"].var_value.int_value = result;
00898 }
00899
00900 extern "C" void mxvm_sdl_quit_text(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00901     quit_text();
00902 }
00903
00904 extern "C" void mxvm_sdl_load_font(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00905     if (operand.size() != 2) {
00906         throw mx::Exception("sdl_load_font requires 2 arguments (file, ptsize).");
00907     }
00908     std::string file;
00909     if (program->isVariable(operand[0].op)) {
00910         mxvm::Variable &var = program->getVariable(operand[0].op);
00911         if (var.type == mxvm::VarType::VAR_STRING) {
00912             file = var.var_value.str_value;
00913         } else if (var.type == mxvm::VarType::VAR_POINTER) {
00914             file = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00915         }
00916     } else {
00917         file = operand[0].op;
00918     }
00919     int64_t ptsize = program->isVariable(operand[1].op) ?
00920         program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00921
00922     int64_t result = load_font(file.c_str(), ptsize);
00923     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00924     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00925     program->vars["%rax"].var_value.int_value = result;
00926 }
00927 extern "C" void mxvm_sdl_draw_text(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00928     if (operand.size() != 9) {
00929         throw mx::Exception("sdl_draw_text requires 9 arguments (renderer_id, font_id, text, x, y, r,
00930 g, b, a).");
00931     }
00932     int64_t renderer_id = program->isVariable(operand[0].op) ?
00933         program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00934     int64_t font_id = program->isVariable(operand[1].op) ?
00935         program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00936
00937     std::string text;
00938     if (program->isVariable(operand[2].op)) {
00939         mxvm::Variable &var = program->getVariable(operand[2].op);
00940         if (var.type == mxvm::VarType::VAR_STRING) {
00941             text = var.var_value.str_value;
00942         } else if (var.type == mxvm::VarType::VAR_POINTER) {
00943             text = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00944         }
00945     } else {
00946         text = operand[2].op;
00947     }
00948
00949     int64_t x = program->isVariable(operand[3].op) ?
00950         program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
00951     int64_t y = program->isVariable(operand[4].op) ?
00952         program->getVariable(operand[4].op).var_value.int_value : operand[4].op_value;
00953     int64_t r = program->isVariable(operand[5].op) ?
00954         program->getVariable(operand[5].op).var_value.int_value : operand[5].op_value;
00955     int64_t g = program->isVariable(operand[6].op) ?
00956         program->getVariable(operand[6].op).var_value.int_value : operand[6].op_value;
00957     int64_t b = program->isVariable(operand[7].op) ?
00958         program->getVariable(operand[7].op).var_value.int_value : operand[7].op_value;
00959     int64_t a = program->isVariable(operand[8].op) ?
00960         program->getVariable(operand[8].op).var_value.int_value : operand[8].op_value;
00961
00962     draw_text(renderer_id, font_id, text.c_str(), x, y, r, g, b, a);
00963 }
00964
00965 extern "C" void mxvm_sdl_create_render_target(mxvm::Program *program, std::vector<mxvm::Operand>

```

```

        &operand) {
00957     if (operand.size() != 3) {
00958         throw mx::Exception("sdl_create_render_target requires 3 arguments (renderer_id, width,
height).");
00959     }
00960
00961     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00962     int64_t width = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00963     int64_t height = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
00964
00965     int64_t result = create_render_target(renderer_id, width, height);
00966     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00967     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00968     program->vars["%rax"].var_value.int_value = result;
00969 }
00970
00971 extern "C" void mxvm_sdl_set_render_target(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00972     if (operand.size() != 2) {
00973         throw mx::Exception("sdl_set_render_target requires 2 arguments (renderer_id, target_id).");
00974     }
00975
00976     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00977     int64_t target_id = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00978
00979     set_render_target(renderer_id, target_id);
00980 }
00981
00982 extern "C" void mxvm_sdl_destroy_render_target(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00983     if (operand.size() != 1) {
00984         throw mx::Exception("sdl_destroy_render_target requires 1 argument (target_id).");
00985     }
00986
00987     int64_t target_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00988
00989     destroy_render_target(target_id);
00990 }
00991
00992 extern "C" void mxvm_sdl_present_scaled(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00993     if (operand.size() != 4) {
00994         throw mx::Exception("sdl_present_scaled requires 4 arguments (renderer_id, target_id,
src_width, src_height).");
00995     }
00996
00997     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00998     int64_t target_id = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00999     int64_t src_w = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
10000     int64_t src_h = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
10001
10002     present_scaled(renderer_id, target_id, src_w, src_h);
10003 }
10004
10005 extern "C" void mxvm_sdl_present_stretched(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
10006     if (operand.size() != 4) {
10007         throw mx::Exception("sdl_present_stretched requires 4 arguments (renderer_id, target_id,
dst_width, dst_height).");
10008     }
10009
10010     int64_t renderer_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
10011     int64_t target_id = program->isVariable(operand[1].op) ?
program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
10012     int64_t dst_w = program->isVariable(operand[2].op) ?
program->getVariable(operand[2].op).var_value.int_value : operand[2].op_value;
10013     int64_t dst_h = program->isVariable(operand[3].op) ?
program->getVariable(operand[3].op).var_value.int_value : operand[3].op_value;
10014
10015     present_stretched(renderer_id, target_id, dst_w, dst_h);
10016 }
10017
10018 extern "C" void mxvm_sdl_set_window_icon(mxvm::Program *program, std::vector<mxvm::Operand> &operand)
{
10019
10020     int64_t window_id = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;

```

```

01021
01022     std::string path;
01023     if (program->isVariable(operand[1].op)) {
01024         mxvm::Variable &var = program->getVariable(operand[1].op);
01025         if (var.type == mxvm::VarType::VAR_STRING) {
01026             path = var.var_value.str_value;
01027         } else if (var.type == mxvm::VarType::VAR_POINTER) {
01028             path = std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
01029         }
01030     } else {
01031         path = operand[0].op;
01032     }
01033     set_window_icon(window_id, path.c_str());
01034 }
01035

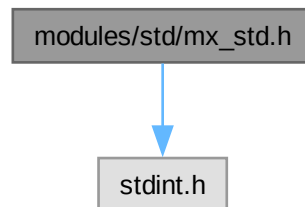
```

12.36 modules/std/mx_std.h File Reference

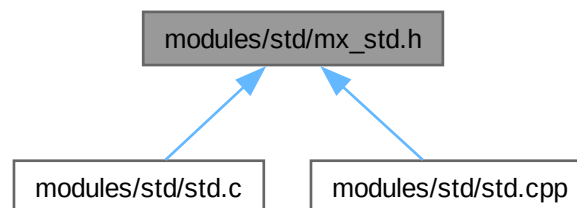
Standard library module C API — function declarations exported to MXVM programs.

```
#include <stdint.h>
```

Include dependency graph for mx_std.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [set_program_args](#) (int [argc](#), const char **[argv](#))
Store command-line arguments for later access via [argc\(\)](#)/[argv\(\)](#).
- int [argc](#) (void)
Return the number of stored command-line arguments.
- const char * [argv](#) (int idx)
Return the command-line argument at the given index.

- void [free_program_args](#) (void)

Free the stored command-line argument memory.

12.36.1 Detailed Description

Standard library module C API — function declarations exported to MXVM programs.

Author

Jared Bruni

Definition in file [mx_std.h](#).

12.36.2 Function Documentation

12.36.2.1 [argc\(\)](#)

```
int argc (
    void )
```

Return the number of stored command-line arguments.

Definition at line 72 of file [std.c](#).

References [g_argc](#).

Referenced by [mx::Argz< String >::Argz\(\)](#), [mx::Argz< String >::initArgs\(\)](#), [main\(\)](#), [mxvm_std_argc\(\)](#), [mxvm_std_set_program_args\(\)](#), [mx::proc_args\(\)](#), [proc_args\(\)](#), and [set_program_args\(\)](#).

12.36.2.2 [argv\(\)](#)

```
const char * argv (
    int idx)
```

Return the command-line argument at the given index.

Parameters

<i>idx</i>	Zero-based argument index
------------	---------------------------

Returns

Pointer to the argument string

Definition at line 76 of file [std.c](#).

References [g_argc](#), and [g_argv](#).

Referenced by [mx::Argz< String >::Argz\(\)](#), [action_interpret\(\)](#), [mx::Argz< String >::initArgs\(\)](#), [main\(\)](#), [mxvm_std_argv\(\)](#), [mxvm_std_set_program_args\(\)](#), [mx::proc_args\(\)](#), [proc_args\(\)](#), [set_program_args\(\)](#), and [mxvm::Program::setArgs\(\)](#).

12.36.2.3 [free_program_args\(\)](#)

```
void free_program_args (
    void )
```

Free the stored command-line argument memory.

Definition at line 23 of file [std.c](#).

References [g_argc](#), and [g_argv](#).

Referenced by [mxvm_std_free_program_args\(\)](#), and [set_program_args\(\)](#).

12.36.2.4 [set_program_args\(\)](#)

```
void set_program_args (
    int argc,
    const char ** argv)
```

Store command-line arguments for later access via [argc\(\)](#)/[argv\(\)](#).

Parameters

argc	Number of arguments
argv	Argument string array

Definition at line 35 of file [std.c](#).

References [argc\(\)](#), [argv\(\)](#), [free_program_args\(\)](#), [g_argc](#), [g_args_registered](#), [g_argv](#), and [mx_strdup_c\(\)](#).

Referenced by [mxvm_std_set_program_args\(\)](#).

12.37 mx_std.h

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef _MX_STD_H__
00007 #define _MX_STD_H__
00008
00009 #include <stdint.h>
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00019 void set_program_args(int argc, const char **argv);
00020
00022 int argc(void);
00023
00028 const char *argv(int idx);
00029
00031 void free_program_args(void);
00032
00033 #ifdef __cplusplus
00034 }
00035 #endif
00036
00037 #endif

```

12.38 modules/std/std.c File Reference

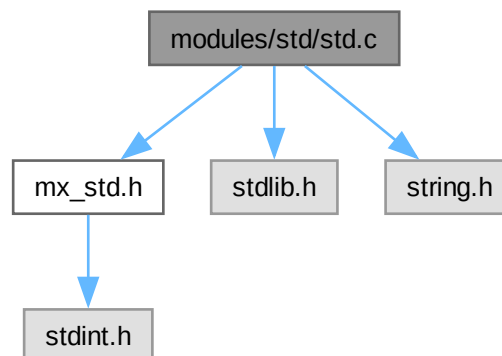
Standard library module C implementation — math, string conversion, and utility functions.

```
#include "mx_std.h"
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

Include dependency graph for std.c:



Functions

- static char * [mx_strdup_c](#) (const char *s)
- void [free_program_args](#) (void)
Free the stored command-line argument memory.
- void [set_program_args](#) (int [argc](#), const char **[argv](#))
Store command-line arguments for later access via [argc\(\)](#)/[argv\(\)](#).
- int [argc](#) (void)
Return the number of stored command-line arguments.
- const char * [argv](#) (int idx)
Return the command-line argument at the given index.
- int64_t [float_to_int](#) (double d)
- double [int_to_float](#) (int64_t i)
- void [release](#) (void *ptr)

Variables

- static int [g_argc](#) = 0
- static char ** [g_argv](#) = NULL
- static int [g_args_registered](#) = 0

12.38.1 Detailed Description

Standard library module C implementation — math, string conversion, and utility functions.

Author

Jared Bruni

Definition in file [std.c](#).

12.38.2 Function Documentation

12.38.2.1 [argc\(\)](#)

```
int argc (
    void )
```

Return the number of stored command-line arguments.

Definition at line 72 of file [std.c](#).

References [g_argc](#).

Referenced by [mx::Argz< String >::Argz\(\)](#), [mx::Argz< String >::initArgs\(\)](#), [main\(\)](#), [mxvm_std_argc\(\)](#), [mxvm_std_set_program_args\(\)](#), [mx::proc_args\(\)](#), [proc_args\(\)](#), and [set_program_args\(\)](#).

12.38.2.2 [argv\(\)](#)

```
const char * argv (
    int idx)
```

Return the command-line argument at the given index.

Parameters

<i>idx</i>	Zero-based argument index
------------	---------------------------

Returns

Pointer to the argument string

Definition at line 76 of file [std.c](#).

References [g_argc](#), and [g_argv](#).

Referenced by [mx::Argz< String >::Argz\(\)](#), [action_interpret\(\)](#), [mx::Argz< String >::initArgs\(\)](#), [main\(\)](#), [mxvm_std_argv\(\)](#), [mxvm_std_set_program_args\(\)](#), [mx::proc_args\(\)](#), [proc_args\(\)](#), [set_program_args\(\)](#), and [mxvm::Program::setArgs\(\)](#).

12.38.2.3 float_to_int()

```
int64_t float_to_int (
    double d)
```

Definition at line 82 of file [std.c](#).

12.38.2.4 free_program_args()

```
void free_program_args (
    void )
```

Free the stored command-line argument memory.

Definition at line 23 of file [std.c](#).

References [g_argc](#), and [g_argv](#).

Referenced by [mxvm_std_free_program_args\(\)](#), and [set_program_args\(\)](#).

12.38.2.5 int_to_float()

```
double int_to_float (
    int64_t i)
```

Definition at line 86 of file [std.c](#).

12.38.2.6 mx_strdup_c()

```
char * mx_strdup_c (
    const char * s) [static]
```

Definition at line 15 of file [std.c](#).

Referenced by [set_program_args\(\)](#).

12.38.2.7 release()

```
void release (
    void * ptr)
```

Definition at line 90 of file [std.c](#).

12.38.2.8 set_program_args()

```
void set_program_args (
    int argc,
    const char ** argv)
```

Store command-line arguments for later access via [argc\(\)](#)/[argv\(\)](#).

Parameters

argc	Number of arguments
argv	Argument string array

Definition at line 35 of file [std.c](#).

References [argc\(\)](#), [argv\(\)](#), [free_program_args\(\)](#), [g_argc](#), [g_args_registered](#), [g_argv](#), and [mx_strdup_c\(\)](#).

Referenced by [mxvm_std_set_program_args\(\)](#).

12.38.3 Variable Documentation**12.38.3.1 g_argc**

```
int g_argc = 0 [static]
```

Definition at line 11 of file [std.c](#).

Referenced by [argc\(\)](#), [argv\(\)](#), [free_program_args\(\)](#), and [set_program_args\(\)](#).

12.38.3.2 g_args_registered

int g_args_registered = 0 [static]

Definition at line 13 of file [std.c](#).

Referenced by [set_program_args\(\)](#).

12.38.3.3 g_argv

char** g_argv = NULL [static]

Definition at line 12 of file [std.c](#).

Referenced by [argv\(\)](#), [free_program_args\(\)](#), and [set_program_args\(\)](#).

12.39 std.c

[Go to the documentation of this file.](#)

```

00001
00006 // ...existing code...
00007 #include "mx_std.h"
00008 #include <stdlib.h>
00009 #include <string.h>
00010
00011 static int g_argc = 0;
00012 static char **g_argv = NULL;
00013 static int g_args_registered = 0;
00014
00015 static char *mx_strdup_c(const char *s) {
00016     size_t len = strlen(s) + 1;
00017     char *d = (char *)malloc(len);
00018     if (d)
00019         memcpy(d, s, len);
00020     return d;
00021 }
00022
00023 void free_program_args(void) {
00024     if (g_argv) {
00025         int i;
00026         for (i = 0; i < g_argc; ++i) {
00027             free(g_argv[i]);
00028         }
00029         free(g_argv);
00030         g_argv = NULL;
00031         g_argc = 0;
00032     }
00033 }
00034
00035 void set_program_args(int argc, const char **argv) {
00036     int i;
00037     if (g_argv) {
00038         free_program_args();
00039     }
00040
00041     if (argc <= 0 || argv == NULL) {
00042         return;
00043     }
00044
00045     g_argv = (char **)malloc(sizeof(char *) * (size_t)argc);
00046     if (!g_argv) {
00047         g_argc = 0;
00048         return;
00049     }
00050
00051     for (i = 0; i < argc; ++i) {
00052         const char *s = argv[i] ? argv[i] : "";
00053         g_argv[i] = mx_strdup_c(s);
00054         if (!g_argv[i]) {
00055             int j;
00056             for (j = 0; j < i; ++j)
00057                 free(g_argv[j]);
00058             free(g_argv);
00059             g_argv = NULL;
00060             g_argc = 0;
00061             return;
00062         }
00063     }
00064     g_argc = argc;
00065
00066     if (!g_args_registered) {
00067         // atexit(mxvm_free_program_args);
00068         g_args_registered = 1;

```



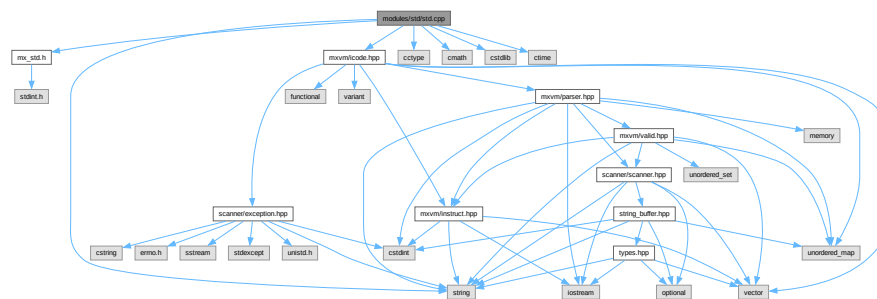
```
00069     }
00070 }
00071
00072 int argc(void) {
00073     return g_argc;
00074 }
00075
00076 const char *argv(int idx) {
00077     if (idx < 0 || idx >= g_argc)
00078         return NULL;
00079     return g_argv[idx];
00080 }
00081
00082 int64_t float_to_int(double d) {
00083     return (int64_t)d;
00084 }
00085
00086 double int_to_float(int64_t i) {
00087     return (double)i;
00088 }
00089
00090 void release(void *ptr) {
00091     if (ptr != NULL)
00092         free(ptr);
00093 }
```

12.40 modules/std/std.cpp File Reference

Standard library module C++ runtime bindings for the MXVM interpreter.

```
#include "mx_std.h"
#include <mxvm/icode.hpp>
#include <cctype>
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <string>
```

Include dependency graph for std.cpp:



Functions

- void `mxvm_std_abs` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_fabs` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_sqrt` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_pow` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_sin` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_cos` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_tan` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_floor` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_ceil` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_rand` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_srand` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)
- void `mxvm_std_malloc` (`mxvm::Program` *program, std::vector< `mxvm::Operand` > &operand)

- void [mxvm_std_calloc](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_release](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_toupper](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_tolower](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_isalpha](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_isdigit](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_isspace](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_atoi](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_atof](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_exit](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_system](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_memcpy](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_memcmp](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_memmove](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_memset](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_exp](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_exp2](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_log](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_log10](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_log2](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_fmod](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_atan2](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_asin](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_acos](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_atan](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_sinh](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_cosh](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_tanh](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_hypot](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_round](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_trunc](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_argc](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_argv](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_free_program_args](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_set_program_args](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_float_to_int](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)
- void [mxvm_std_int_to_float](#) (mxvm::Program *program, std::vector< [mxvm::Operand](#) > &operand)

12.40.1 Detailed Description

Standard library module C++ runtime bindings for the MXVM interpreter.

Author

Jared Bruni

Definition in file [std.cpp](#).

12.40.2 Function Documentation

12.40.2.1 mxvm_std_abs()

```
void mxvm_std_abs (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 14 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.2 mxvm_std_acos()

```
void mxvm_std_acos (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 670 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.3 mxvm_std_argc()

```
void mxvm_std_argc (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 760 of file [std.cpp](#).

References [argc\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.40.2.4 mxvm_std_argv()

```
void mxvm_std_argv (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 768 of file [std.cpp](#).

References [argv\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::OP_CONSTANT](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.5 mxvm_std_asin()

```
void mxvm_std_asin (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 659 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.6 mxvm_std_atan()

```
void mxvm_std_atan (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 681 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.7 mxvm_std_atan2()

```
void mxvm_std_atan2 (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 646 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.8 mxvm_std_atof()

```
void mxvm_std_atof (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 355 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.9 mxvm_std_atoi()

```
void mxvm_std_atoi (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 326 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.10 mxvm_std_calloc()

```
void mxvm_std_calloc (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 218 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.11 mxvm_std_ceil()

```
void mxvm_std_ceil (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 162 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.12 mxvm_std_cos()

```
void mxvm_std_cos (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 108 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.13 mxvm_std_cosh()

```
void mxvm_std_cosh (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 703 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.14 mxvm_std_exit()

```
void mxvm_std_exit (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 385 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.40.2.15 mxvm_std_exp()

```
void mxvm_std_exp (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 578 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.16 mxvm_std_exp2()

```
void mxvm_std_exp2 (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 589 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.17 mxvm_std_fabs()

```
void mxvm_std_fabs (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 27 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.18 mxvm_std_float_to_int()

```
void mxvm_std_float_to_int (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 826 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.19 mxvm_std_floor()

```
void mxvm_std_floor (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 144 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.20 mxvm_std_fmod()

```
void mxvm_std_fmod (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 633 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.21 mxvm_std_free_program_args()

```
void mxvm_std_free_program_args (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 792 of file [std.cpp](#).

References [free_program_args\(\)](#).

12.40.2.22 mxvm_std_hypot()

```
void mxvm_std_hypot (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 725 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.23 mxvm_std_int_to_float()

```
void mxvm_std_int_to_float (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 851 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.24 mxvm_std_isalpha()

```
void mxvm_std_isalpha (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 286 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.25 mxvm_std_isdigit()

```
void mxvm_std_isdigit (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 299 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.26 mxvm_std_isspace()

```
void mxvm_std_isspace (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 312 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.27 mxvm_std_log()

```
void mxvm_std_log (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 600 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.28 mxvm_std_log10()

```
void mxvm_std_log10 (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 611 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.29 mxvm_std_log2()

```
void mxvm_std_log2 (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 622 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.30 mxvm_std_malloc()

```
void mxvm_std_malloc (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 199 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.31 mxvm_std_memcmp()

```
void mxvm_std_memcmp (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 462 of file [std.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.32 mxvm_std_memcpy()

```
void mxvm_std_memcpy (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 425 of file [std.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.33 mxvm_std_memmove()

```
void mxvm_std_memmove (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 499 of file [std.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.34 mxvm_std_memset()

```
void mxvm_std_memset (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 536 of file [std.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.35 mxvm_std_pow()

```
void mxvm_std_pow (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 63 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.36 mxvm_std_rand()

```
void mxvm_std_rand (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 181 of file [std.cpp](#).

References [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.40.2.37 mxvm_std_release()

```
void mxvm_std_release (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 238 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::owns](#), [mxvm::Variable_Value::ptr_code](#), [mxvm::Variable_Value::ptr_size](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

12.40.2.38 mxvm_std_round()

```
void mxvm_std_round (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 738 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.39 mxvm_std_set_program_args()

```
void mxvm_std_set_program_args (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 796 of file [std.cpp](#).

References [argc\(\)](#), [argv\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [set_program_args\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

12.40.2.40 mxvm_std_sin()

```
void mxvm_std_sin (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 90 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.41 mxvm_std_sinh()

```
void mxvm_std_sinh (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 692 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.42 mxvm_std_sqrt()

```
void mxvm_std_sqrt (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 45 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.43 mxvm_std_srand()

```
void mxvm_std_srand (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 188 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), and [mxvm::Variable::var_value](#).

12.40.2.44 mxvm_std_system()

```
void mxvm_std_system (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 394 of file [std.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.45 mxvm_std_tan()

```
void mxvm_std_tan (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 126 of file [std.cpp](#).

References [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable::type](#), [mxvm::VAR_FLOAT](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.40.2.46 mxvm_std_tanh()

```
void mxvm_std_tanh (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 714 of file `std.cpp`.

References `mxvm::Variable_Value::float_value`, `mxvm::Program::getVariable()`, `mxvm::Variable_Value::int_value`, `mxvm::Program::isVariable()`, `mxvm::VAR_FLOAT`, `mxvm::Variable::var_value`, and `mxvm::Base::vars`.

12.40.2.47 mxvm_std_tolower()

```
void mxvm_std_tolower (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 273 of file `std.cpp`.

References `mxvm::Program::getVariable()`, `mxvm::Variable_Value::int_value`, `mxvm::Program::isVariable()`, `mxvm::VAR_INTEGER`, `mxvm::Variable::var_value`, and `mxvm::Base::vars`.

12.40.2.48 mxvm_std_toupper()

```
void mxvm_std_toupper (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 260 of file `std.cpp`.

References `mxvm::Program::getVariable()`, `mxvm::Variable_Value::int_value`, `mxvm::Program::isVariable()`, `mxvm::VAR_INTEGER`, `mxvm::Variable::var_value`, and `mxvm::Base::vars`.

12.40.2.49 mxvm_std_trunc()

```
void mxvm_std_trunc (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 749 of file `std.cpp`.

References `mxvm::Variable_Value::float_value`, `mxvm::Program::getVariable()`, `mxvm::Variable_Value::int_value`, `mxvm::Program::isVariable()`, `mxvm::VAR_FLOAT`, `mxvm::Variable::var_value`, and `mxvm::Base::vars`.

12.41 std.cpp

[Go to the documentation of this file.](#)

```
00001
00006 #include "mx_std.h"
00007 #include <mxvm/icode.hpp>
00008 #include <cctype>
00009 #include <cmath>
00010 #include <cstdlib>
00011 #include <ctime>
00012 #include <string>
00013
00014 extern "C" void mxvm_std_abs(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00015     if (operand.size() != 1) {
00016         throw mx::Exception("abs requires 1 argument (value).");
00017     }
00018
00019     int64_t value = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00020
00021     int64_t result = std::abs(value);
00022     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00023     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00024     program->vars["%rax"].var_value.int_value = result;
00025 }
00026
00027 extern "C" void mxvm_std_fabs(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00028     if (operand.size() != 1) {
00029         throw mx::Exception("fabs requires 1 argument (value).");
00030     }
00031
00032     if (!program->isVariable(operand[0].op)) {
```

```

00033         throw mx::Exception("fabs argument must be a variable.");
00034     }
00035
00036     mxvm::Variable &var = program->getVariable(operand[0].op);
00037     double value = (var.type == mxvm::VarType::VAR_FLOAT) ? var.var_value.float_value :
static_cast<double>(var.var_value.int_value);
00038
00039     double result = std::fabs(value);
00040     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00041     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00042     program->vars["%rax"].var_value.float_value = result;
00043 }
00044
00045 extern "C" void mxvm_std_sqrt(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00046     if (operand.size() != 1) {
00047         throw mx::Exception("sqrt requires 1 argument (value).");
00048     }
00049
00050     if (!program->isVariable(operand[0].op)) {
00051         throw mx::Exception("sqrt argument must be a variable.");
00052     }
00053
00054     mxvm::Variable &var = program->getVariable(operand[0].op);
00055     double value = (var.type == mxvm::VarType::VAR_FLOAT) ? var.var_value.float_value :
static_cast<double>(var.var_value.int_value);
00056
00057     double result = std::sqrt(value);
00058     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00059     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00060     program->vars["%rax"].var_value.float_value = result;
00061 }
00062
00063 extern "C" void mxvm_std_pow(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00064     if (operand.size() != 2) {
00065         throw mx::Exception("pow requires 2 arguments (base, exponent).");
00066     }
00067
00068     double base, exponent;
00069
00070     if (program->isVariable(operand[0].op)) {
00071         mxvm::Variable &var = program->getVariable(operand[0].op);
00072         base = (var.type == mxvm::VarType::VAR_FLOAT) ? var.var_value.float_value :
static_cast<double>(var.var_value.int_value);
00073     } else {
00074         base = static_cast<double>(operand[0].op_value);
00075     }
00076
00077     if (program->isVariable(operand[1].op)) {
00078         mxvm::Variable &var = program->getVariable(operand[1].op);
00079         exponent = (var.type == mxvm::VarType::VAR_FLOAT) ? var.var_value.float_value :
static_cast<double>(var.var_value.int_value);
00080     } else {
00081         exponent = static_cast<double>(operand[1].op_value);
00082     }
00083
00084     double result = std::pow(base, exponent);
00085     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00086     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00087     program->vars["%rax"].var_value.float_value = result;
00088 }
00089
00090 extern "C" void mxvm_std_sin(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00091     if (operand.size() != 1) {
00092         throw mx::Exception("sin requires 1 argument (angle).");
00093     }
00094
00095     if (!program->isVariable(operand[0].op)) {
00096         throw mx::Exception("sin argument must be a variable.");
00097     }
00098
00099     mxvm::Variable &var = program->getVariable(operand[0].op);
00100     double value = (var.type == mxvm::VarType::VAR_FLOAT) ? var.var_value.float_value :
static_cast<double>(var.var_value.int_value);
00101
00102     double result = std::sin(value);
00103     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00104     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00105     program->vars["%rax"].var_value.float_value = result;
00106 }
00107
00108 extern "C" void mxvm_std_cos(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00109     if (operand.size() != 1) {
00110         throw mx::Exception("cos requires 1 argument (angle).");
00111     }
00112
00113     if (!program->isVariable(operand[0].op)) {
00114         throw mx::Exception("cos argument must be a variable.");

```

```

00115     }
00116
00117     mxvm::Variable &var = program->getVariable(operand[0].op);
00118     double value = (var.type == mxvm::VarType::VAR_FLOAT) ? var.var_value.float_value :
static_cast<double>(var.var_value.int_value);
00119
00120     double result = std::cos(value);
00121     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00122     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00123     program->vars["%rax"].var_value.float_value = result;
00124 }
00125
00126 extern "C" void mxvm_std_tan(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00127     if (operand.size() != 1) {
00128         throw mx::Exception("tan requires 1 argument (angle).");
00129     }
00130
00131     if (!program->isVariable(operand[0].op)) {
00132         throw mx::Exception("tan argument must be a variable.");
00133     }
00134
00135     mxvm::Variable &var = program->getVariable(operand[0].op);
00136     double value = (var.type == mxvm::VarType::VAR_FLOAT) ? var.var_value.float_value :
static_cast<double>(var.var_value.int_value);
00137
00138     double result = std::tan(value);
00139     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00140     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00141     program->vars["%rax"].var_value.float_value = result;
00142 }
00143
00144 extern "C" void mxvm_std_floor(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00145     if (operand.size() != 1) {
00146         throw mx::Exception("floor requires 1 argument (value).");
00147     }
00148
00149     if (!program->isVariable(operand[0].op)) {
00150         throw mx::Exception("floor argument must be a variable.");
00151     }
00152
00153     mxvm::Variable &var = program->getVariable(operand[0].op);
00154     double value = (var.type == mxvm::VarType::VAR_FLOAT) ? var.var_value.float_value :
static_cast<double>(var.var_value.int_value);
00155
00156     double result = std::floor(value);
00157     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00158     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00159     program->vars["%rax"].var_value.float_value = result;
00160 }
00161
00162 extern "C" void mxvm_std_ceil(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00163     if (operand.size() != 1) {
00164         throw mx::Exception("ceil requires 1 argument (value).");
00165     }
00166
00167     if (!program->isVariable(operand[0].op)) {
00168         throw mx::Exception("ceil argument must be a variable.");
00169     }
00170
00171     mxvm::Variable &var = program->getVariable(operand[0].op);
00172     double value = (var.type == mxvm::VarType::VAR_FLOAT) ? var.var_value.float_value :
static_cast<double>(var.var_value.int_value);
00173
00174     double result = std::ceil(value);
00175     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00176     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00177     program->vars["%rax"].var_value.float_value = result;
00178 }
00179
00180 // Random number functions
00181 extern "C" void mxvm_std_rand(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00182     int64_t result = std::rand();
00183     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00184     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00185     program->vars["%rax"].var_value.int_value = result;
00186 }
00187
00188 extern "C" void mxvm_std_srand(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00189     if (operand.size() != 1) {
00190         throw mx::Exception("srand requires 1 argument (seed).");
00191     }
00192
00193     int64_t seed = program->isVariable(operand[0].op) ?
program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00194
00195     std::srand(static_cast<unsigned int>(seed));
00196 }

```

```

00197
00198 // Memory functions
00199 extern "C" void mxvm_std_malloc(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00200     if (operand.size() != 1) {
00201         throw mx::Exception("malloc requires 1 argument (size).");
00202     }
00203
00204     int64_t size = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00205
00206     void *result = std::malloc(static_cast<size_t>(size));
00207     if (result == nullptr) {
00208         throw mx::Exception("malloc failed to allocate " + std::to_string(size) + " bytes");
00209     }
00210     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00211     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_POINTER;
00212     program->vars["%rax"].var_value.ptr_value = result;
00213     program->vars["%rax"].var_value.owns = true;
00214     program->vars["%rax"].var_value.ptr_size = static_cast<uint64_t>(size);
00215     program->vars["%rax"].var_value.ptr_count = 1;
00216 }
00217
00218 extern "C" void mxvm_std_calloc(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00219     if (operand.size() != 2) {
00220         throw mx::Exception("calloc requires 2 arguments (count, size).");
00221     }
00222
00223     int64_t count = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00224     int64_t size = program->isVariable(operand[1].op) ?
    program->getVariable(operand[1].op).var_value.int_value : operand[1].op_value;
00225
00226     void *result = std::calloc(static_cast<size_t>(count), static_cast<size_t>(size));
00227     if (result == nullptr) {
00228         throw mx::Exception("calloc failed to allocate " + std::to_string(count) + " * " +
    std::to_string(size) + " bytes");
00229     }
00230     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00231     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_POINTER;
00232     program->vars["%rax"].var_value.ptr_value = result;
00233     program->vars["%rax"].var_value.owns = true;
00234     program->vars["%rax"].var_value.ptr_size = static_cast<uint64_t>(size);
00235     program->vars["%rax"].var_value.ptr_count = static_cast<uint64_t>(count);
00236 }
00237
00238 extern "C" void mxvm_std_release(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00239     if (operand.size() != 1) {
00240         throw mx::Exception("free requires 1 argument (pointer).");
00241     }
00242
00243     if (!program->isVariable(operand[0].op)) {
00244         throw mx::Exception("free argument must be a variable.");
00245     }
00246
00247     mxvm::Variable &var = program->getVariable(operand[0].op);
00248     if (var.type != mxvm::VarType::VAR_POINTER) {
00249         throw mx::Exception("free argument must be a pointer variable.");
00250     }
00251
00252     std::free(var.var_value.ptr_value);
00253     var.var_value.ptr_value = nullptr;
00254     var.var_value.owns = false;
00255     var.var_value.ptr_size = 0;
00256     var.var_value.ptr_count = 0;
00257 }
00258
00259 // Character functions
00260 extern "C" void mxvm_std_toupper(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00261     if (operand.size() != 1) {
00262         throw mx::Exception("toupper requires 1 argument (character).");
00263     }
00264
00265     int64_t ch = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00266
00267     int64_t result = std::toupper(static_cast<int>(ch));
00268     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00269     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00270     program->vars["%rax"].var_value.int_value = result;
00271 }
00272
00273 extern "C" void mxvm_std_tolower(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00274     if (operand.size() != 1) {
00275         throw mx::Exception("tolower requires 1 argument (character).");
00276     }
00277
00278     int64_t ch = program->isVariable(operand[0].op) ?

```

```

    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00279
00280     int64_t result = std::tolower(static_cast<int>(ch));
00281     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00282     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00283     program->vars["%rax"].var_value.int_value = result;
00284 }
00285
00286 extern "C" void mxvm_std_isalpha(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00287     if (operand.size() != 1) {
00288         throw mx::Exception("isalpha requires 1 argument (character).");
00289     }
00290
00291     int64_t ch = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00292
00293     int64_t result = std::isalpha(static_cast<int>(ch)) ? 1 : 0;
00294     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00295     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00296     program->vars["%rax"].var_value.int_value = result;
00297 }
00298
00299 extern "C" void mxvm_std_isdigit(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00300     if (operand.size() != 1) {
00301         throw mx::Exception("isdigit requires 1 argument (character).");
00302     }
00303
00304     int64_t ch = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00305
00306     int64_t result = std::isdigit(static_cast<int>(ch)) ? 1 : 0;
00307     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00308     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00309     program->vars["%rax"].var_value.int_value = result;
00310 }
00311
00312 extern "C" void mxvm_std_isspace(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00313     if (operand.size() != 1) {
00314         throw mx::Exception("isspace requires 1 argument (character).");
00315     }
00316
00317     int64_t ch = program->isVariable(operand[0].op) ?
    program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00318
00319     int64_t result = std::isspace(static_cast<int>(ch)) ? 1 : 0;
00320     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00321     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00322     program->vars["%rax"].var_value.int_value = result;
00323 }
00324
00325 // String conversion functions
00326 extern "C" void mxvm_std_atoi(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00327     if (operand.size() != 1) {
00328         throw mx::Exception("atoi requires 1 argument (string).");
00329     }
00330
00331     if (!program->isVariable(operand[0].op)) {
00332         throw mx::Exception("atoi argument must be a variable.");
00333     }
00334
00335     mxvm::Variable &var = program->getVariable(operand[0].op);
00336     const char *str = nullptr;
00337
00338     if (var.type == mxvm::VarType::VAR_STRING) {
00339         str = var.var_value.str_value.c_str();
00340     } else if (var.type == mxvm::VarType::VAR_POINTER) {
00341         if (var.var_value.ptr_value == nullptr) {
00342             throw mx::Exception("atoi: pointer argument is null");
00343         }
00344         str = reinterpret_cast<const char *>(var.var_value.ptr_value);
00345     } else {
00346         throw mx::Exception("atoi argument must be a string or pointer variable.");
00347     }
00348
00349     int64_t result = std::atoi(str);
00350     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00351     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00352     program->vars["%rax"].var_value.int_value = result;
00353 }
00354
00355 extern "C" void mxvm_std_atof(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00356     if (operand.size() != 1) {
00357         throw mx::Exception("atof requires 1 argument (string).");
00358     }
00359
00360     if (!program->isVariable(operand[0].op)) {
00361         throw mx::Exception("atof argument must be a variable.");

```

```

00362     }
00363
00364     mxvm::Variable &var = program->getVariable(operand[0].op);
00365     const char *str = nullptr;
00366
00367     if (var.type == mxvm::VarType::VAR_STRING) {
00368         str = var.var_value.str_value.c_str();
00369     } else if (var.type == mxvm::VarType::VAR_POINTER) {
00370         if (var.var_value.ptr_value == nullptr) {
00371             throw mx::Exception("atof: pointer argument is null");
00372         }
00373         str = reinterpret_cast<const char *>(var.var_value.ptr_value);
00374     } else {
00375         throw mx::Exception("atof argument must be a string or pointer variable.");
00376     }
00377
00378     double result = std::atof(str);
00379     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00380     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00381     program->vars["%rax"].var_value.float_value = result;
00382 }
00383
00384 // Utility functions
00385 extern "C" void mxvm_std_exit(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00386     int64_t exit_code = 0;
00387     if (!operand.empty()) {
00388         exit_code = program->isVariable(operand[0].op) ?
00389         program->getVariable(operand[0].op).var_value.int_value : operand[0].op_value;
00390     }
00391     std::exit(static_cast<int>(exit_code));
00392 }
00393
00394 extern "C" void mxvm_std_system(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00395     if (operand.size() != 1) {
00396         throw mx::Exception("system requires 1 argument (command).");
00397     }
00398
00399     if (!program->isVariable(operand[0].op)) {
00400         throw mx::Exception("system argument must be a variable.");
00401     }
00402
00403     mxvm::Variable &var = program->getVariable(operand[0].op);
00404     const char *command = nullptr;
00405
00406     if (var.type == mxvm::VarType::VAR_STRING) {
00407         command = var.var_value.str_value.c_str();
00408     } else if (var.type == mxvm::VarType::VAR_POINTER) {
00409         if (var.var_value.ptr_value == nullptr) {
00410             throw mx::Exception("system: pointer argument is null");
00411         }
00412         command = reinterpret_cast<const char *>(var.var_value.ptr_value);
00413     } else {
00414         throw mx::Exception("system argument must be a string or pointer variable.");
00415     }
00416
00417     int64_t result = std::system(command);
00418     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00419     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00420     program->vars["%rax"].var_value.int_value = result;
00421 }
00422
00423 // ...existing code...
00424
00425 extern "C" void mxvm_std_memcpy(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00426     if (operand.size() != 3) {
00427         throw mx::Exception("memcpy requires 3 arguments (dest, src, size).");
00428     }
00429
00430     if (!program->isVariable(operand[0].op) || !program->isVariable(operand[1].op)) {
00431         throw mx::Exception("memcpy dest and src must be pointer variables.");
00432     }
00433
00434     mxvm::Variable &dest = program->getVariable(operand[0].op);
00435     mxvm::Variable &src = program->getVariable(operand[1].op);
00436
00437     if (dest.type != mxvm::VarType::VAR_POINTER || src.type != mxvm::VarType::VAR_POINTER) {
00438         throw mx::Exception("memcpy dest and src must be pointer variables.");
00439     }
00440
00441     mxvm::except_assert("memcpy: dest pointer is null", dest.var_value.ptr_value != nullptr);
00442     mxvm::except_assert("memcpy: src pointer is null", src.var_value.ptr_value != nullptr);
00443
00444     size_t n = 0;
00445     if (program->isVariable(operand[2].op)) {
00446         mxvm::Variable &size_var = program->getVariable(operand[2].op);
00447         n = static_cast<size_t>(size_var.var_value.int_value);

```

```

00448     } else {
00449         n = static_cast<size_t>(operand[2].op_value);
00450     }
00451
00452     if (n > 0) {
00453         memcpy(dest.var_value.ptr_value, src.var_value.ptr_value, n);
00454     }
00455
00456     // Return dest pointer in %rax
00457     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00458     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_POINTER;
00459     program->vars["%rax"].var_value.ptr_value = dest.var_value.ptr_value;
00460 }
00461
00462 extern "C" void mxvm_std_memcmp(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00463     if (operand.size() != 3) {
00464         throw mx::Exception("memcmp requires 3 arguments (ptr1, ptr2, size).");
00465     }
00466
00467     if (!program->isVariable(operand[0].op) || !program->isVariable(operand[1].op)) {
00468         throw mx::Exception("memcmp ptr1 and ptr2 must be pointer variables.");
00469     }
00470
00471     mxvm::Variable &ptr1 = program->getVariable(operand[0].op);
00472     mxvm::Variable &ptr2 = program->getVariable(operand[1].op);
00473
00474     if (ptr1.type != mxvm::VarType::VAR_POINTER || ptr2.type != mxvm::VarType::VAR_POINTER) {
00475         throw mx::Exception("memcmp ptr1 and ptr2 must be pointer variables.");
00476     }
00477
00478     mxvm::except_assert("memcmp: ptr1 is null", ptr1.var_value.ptr_value != nullptr);
00479     mxvm::except_assert("memcmp: ptr2 is null", ptr2.var_value.ptr_value != nullptr);
00480
00481     size_t n = 0;
00482     if (program->isVariable(operand[2].op)) {
00483         mxvm::Variable &size_var = program->getVariable(operand[2].op);
00484         n = static_cast<size_t>(size_var.var_value.int_value);
00485     } else {
00486         n = static_cast<size_t>(operand[2].op_value);
00487     }
00488
00489     int result = 0;
00490     if (n > 0) {
00491         result = memcmp(ptr1.var_value.ptr_value, ptr2.var_value.ptr_value, n);
00492     }
00493
00494     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00495     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00496     program->vars["%rax"].var_value.int_value = static_cast<int64_t>(result);
00497 }
00498
00499 extern "C" void mxvm_std_memmove(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00500     if (operand.size() != 3) {
00501         throw mx::Exception("memmove requires 3 arguments (dest, src, size).");
00502     }
00503
00504     if (!program->isVariable(operand[0].op) || !program->isVariable(operand[1].op)) {
00505         throw mx::Exception("memmove dest and src must be pointer variables.");
00506     }
00507
00508     mxvm::Variable &dest = program->getVariable(operand[0].op);
00509     mxvm::Variable &src = program->getVariable(operand[1].op);
00510
00511     if (dest.type != mxvm::VarType::VAR_POINTER || src.type != mxvm::VarType::VAR_POINTER) {
00512         throw mx::Exception("memmove dest and src must be pointer variables.");
00513     }
00514
00515     mxvm::except_assert("memmove: dest pointer is null", dest.var_value.ptr_value != nullptr);
00516     mxvm::except_assert("memmove: src pointer is null", src.var_value.ptr_value != nullptr);
00517
00518     size_t n = 0;
00519     if (program->isVariable(operand[2].op)) {
00520         mxvm::Variable &size_var = program->getVariable(operand[2].op);
00521         n = static_cast<size_t>(size_var.var_value.int_value);
00522     } else {
00523         n = static_cast<size_t>(operand[2].op_value);
00524     }
00525
00526     if (n > 0) {
00527         memmove(dest.var_value.ptr_value, src.var_value.ptr_value, n);
00528     }
00529
00530     // Return dest pointer in %rax
00531     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00532     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_POINTER;
00533     program->vars["%rax"].var_value.ptr_value = dest.var_value.ptr_value;
00534 }

```



```

00535
00536 extern "C" void mxvm_std_memset(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00537     if (operand.size() != 3) {
00538         throw mx::Exception("memset requires 3 arguments (dest, value, size).");
00539     }
00540
00541     if (!program->isVariable(operand[0].op)) {
00542         throw mx::Exception("memset dest must be a pointer variable.");
00543     }
00544
00545     mxvm::Variable &dest = program->getVariable(operand[0].op);
00546
00547     if (dest.type != mxvm::VarType::VAR_POINTER) {
00548         throw mx::Exception("memset dest must be a pointer variable.");
00549     }
00550
00551     mxvm::except_assert("memset: dest pointer is null", dest.var_value.ptr_value != nullptr);
00552
00553     int value = 0;
00554     if (program->isVariable(operand[1].op)) {
00555         mxvm::Variable &val_var = program->getVariable(operand[1].op);
00556         value = static_cast<int>(val_var.var_value.int_value);
00557     } else {
00558         value = static_cast<int>(operand[1].op_value);
00559     }
00560
00561     size_t n = 0;
00562     if (program->isVariable(operand[2].op)) {
00563         mxvm::Variable &size_var = program->getVariable(operand[2].op);
00564         n = static_cast<size_t>(size_var.var_value.int_value);
00565     } else {
00566         n = static_cast<size_t>(operand[2].op_value);
00567     }
00568
00569     if (n > 0) {
00570         memset(dest.var_value.ptr_value, value, n);
00571     }
00572
00573     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00574     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_POINTER;
00575     program->vars["%rax"].var_value.ptr_value = dest.var_value.ptr_value;
00576 }
00577
00578 extern "C" void mxvm_std_exp(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00579     if (operand.size() != 1)
00580         throw mx::Exception("exp requires 1 argument.");
00581     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
: (double)operand[0].op_value;
00582
00583     double r = std::exp(v);
00584     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00585     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00586     program->vars["%rax"].var_value.float_value = r;
00587 }
00588
00589 extern "C" void mxvm_std_exp2(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00590     if (operand.size() != 1)
00591         throw mx::Exception("exp2 requires 1 argument.");
00592     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
: (double)operand[0].op_value;
00593
00594     double r = std::exp2(v);
00595     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00596     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00597     program->vars["%rax"].var_value.float_value = r;
00598 }
00599
00600 extern "C" void mxvm_std_log(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00601     if (operand.size() != 1)
00602         throw mx::Exception("log requires 1 argument.");
00603     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
: (double)operand[0].op_value;
00604
00605     double r = std::log(v);
00606     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00607     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00608     program->vars["%rax"].var_value.float_value = r;
00609 }
00610
00611 extern "C" void mxvm_std_log10(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00612     if (operand.size() != 1)
00613         throw mx::Exception("log10 requires 1 argument.");
00614     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :

```

```

        (double)program->getVariable(operand[0].op).var_value.int_value)
00615                                     : (double)operand[0].op_value;
00616         double r = std::log10(v);
00617         program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00618         program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00619         program->vars["%rax"].var_value.float_value = r;
00620     }
00621
00622 extern "C" void mxvm_std_log2(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00623     if (operand.size() != 1)
00624         throw mx::Exception("log2 requires 1 argument.");
00625     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
00626                                     : (double)operand[0].op_value;
00627     double r = std::log2(v);
00628     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00629     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00630     program->vars["%rax"].var_value.float_value = r;
00631 }
00632
00633 extern "C" void mxvm_std_fmod(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00634     if (operand.size() != 2)
00635         throw mx::Exception("fmod requires 2 arguments (x, y).");
00636     double x = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
00637                                     : (double)operand[0].op_value;
00638     double y = program->isVariable(operand[1].op) ? (program->getVariable(operand[1].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[1].op).var_value.float_value :
(double)program->getVariable(operand[1].op).var_value.int_value)
00639                                     : (double)operand[1].op_value;
00640     double r = std::fmod(x, y);
00641     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00642     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00643     program->vars["%rax"].var_value.float_value = r;
00644 }
00645
00646 extern "C" void mxvm_std_atan2(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00647     if (operand.size() != 2)
00648         throw mx::Exception("atan2 requires 2 arguments (y, x).");
00649     double y = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
00650                                     : (double)operand[0].op_value;
00651     double x = program->isVariable(operand[1].op) ? (program->getVariable(operand[1].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[1].op).var_value.float_value :
(double)program->getVariable(operand[1].op).var_value.int_value)
00652                                     : (double)operand[1].op_value;
00653     double r = std::atan2(y, x);
00654     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00655     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00656     program->vars["%rax"].var_value.float_value = r;
00657 }
00658
00659 extern "C" void mxvm_std_asin(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00660     if (operand.size() != 1)
00661         throw mx::Exception("asin requires 1 argument.");
00662     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
00663                                     : (double)operand[0].op_value;
00664     double r = std::asin(v);
00665     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00666     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00667     program->vars["%rax"].var_value.float_value = r;
00668 }
00669
00670 extern "C" void mxvm_std_acos(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00671     if (operand.size() != 1)
00672         throw mx::Exception("acos requires 1 argument.");
00673     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
00674                                     : (double)operand[0].op_value;
00675     double r = std::acos(v);
00676     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00677     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00678     program->vars["%rax"].var_value.float_value = r;
00679 }
00680
00681 extern "C" void mxvm_std_atan(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00682     if (operand.size() != 1)
00683         throw mx::Exception("atan requires 1 argument.");
00684     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)

```

```

00685                                     : (double)operand[0].op_value;
00686     double r = std::atan(v);
00687     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00688     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00689     program->vars["%rax"].var_value.float_value = r;
00690 }
00691
00692 extern "C" void mxvm_std_sinh(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00693     if (operand.size() != 1)
00694         throw mx::Exception("sinh requires 1 argument.");
00695     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
: (double)operand[0].op_value;
00696     double r = std::sinh(v);
00697     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00698     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00699     program->vars["%rax"].var_value.float_value = r;
00700 }
00701
00702
00703 extern "C" void mxvm_std_cosh(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00704     if (operand.size() != 1)
00705         throw mx::Exception("cosh requires 1 argument.");
00706     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
: (double)operand[0].op_value;
00707     double r = std::cosh(v);
00708     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00709     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00710     program->vars["%rax"].var_value.float_value = r;
00711 }
00712
00713
00714 extern "C" void mxvm_std_tanh(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00715     if (operand.size() != 1)
00716         throw mx::Exception("tanh requires 1 argument.");
00717     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
: (double)operand[0].op_value;
00718     double r = std::tanh(v);
00719     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00720     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00721     program->vars["%rax"].var_value.float_value = r;
00722 }
00723
00724
00725 extern "C" void mxvm_std_hypot(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00726     if (operand.size() != 2)
00727         throw mx::Exception("hypot requires 2 arguments (x, y).");
00728     double x = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
: (double)operand[0].op_value;
00729     double y = program->isVariable(operand[1].op) ? (program->getVariable(operand[1].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[1].op).var_value.float_value :
(double)program->getVariable(operand[1].op).var_value.int_value)
: (double)operand[1].op_value;
00730     double r = std::hypot(x, y);
00731     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00732     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00733     program->vars["%rax"].var_value.float_value = r;
00734 }
00735
00736
00737
00738 extern "C" void mxvm_std_round(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00739     if (operand.size() != 1)
00740         throw mx::Exception("round requires 1 argument.");
00741     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
: (double)operand[0].op_value;
00742     double r = std::round(v);
00743     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00744     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00745     program->vars["%rax"].var_value.float_value = r;
00746 }
00747
00748
00749 extern "C" void mxvm_std_trunc(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00750     if (operand.size() != 1)
00751         throw mx::Exception("trunc requires 1 argument.");
00752     double v = program->isVariable(operand[0].op) ? (program->getVariable(operand[0].op).type ==
mxvm::VarType::VAR_FLOAT ? program->getVariable(operand[0].op).var_value.float_value :
(double)program->getVariable(operand[0].op).var_value.int_value)
: (double)operand[0].op_value;
00753     double r = std::trunc(v);
00754     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00755     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00756     program->vars["%rax"].var_value.float_value = r;
00757 }

```

```

00758 }
00759
00760 extern "C" void mxvm_std_argc(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00761     (void)operand;
00762     int v = argc();
00763     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00764     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00765     program->vars["%rax"].var_value.int_value = (int64_t)v;
00766 }
00767
00768 extern "C" void mxvm_std_argv(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00769     if (operand.size() != 1) {
00770         throw mx::Exception("argv(index) requires 1 argument");
00771     }
00772
00773     int idx = 0;
00774     if (operand[0].type == mxvm::OperandType::OP_CONSTANT) {
00775         idx = (int)operand[0].op_value;
00776     } else if (program->isVariable(operand[0].op)) {
00777         mxvm::Variable &v = program->getVariable(operand[0].op);
00778         if (v.type != mxvm::VarType::VAR_INTEGER) {
00779             throw mx::Exception("argv index must be an integer");
00780         }
00781         idx = (int)v.var_value.int_value;
00782     } else {
00783         throw mx::Exception("argv argument must be an index constant or integer variable");
00784     }
00785
00786     const char *s = argv(idx);
00787     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00788     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_POINTER;
00789     program->vars["%rax"].var_value.ptr_value = (void *)s; /* may be NULL */
00790 }
00791
00792 extern "C" void mxvm_std_free_program_args(mxvm::Program *program, std::vector<mxvm::Operand>
&operand) {
00793     free_program_args();
00794 }
00795
00796 extern "C" void mxvm_std_set_program_args(mxvm::Program *program, std::vector<mxvm::Operand> &operand)
{
00797     if (operand.size() != 2) {
00798         throw mx::Exception("set_program_args requires 2 arguments (argc, argv)");
00799     }
00800
00801     if (!program->isVariable(operand[0].op) || !program->isVariable(operand[1].op)) {
00802         throw mx::Exception("set_program_args arguments must be variables");
00803     }
00804
00805     mxvm::Variable &argc_var = program->getVariable(operand[0].op);
00806     mxvm::Variable &argv_var = program->getVariable(operand[1].op);
00807
00808     if (argc_var.type != mxvm::VarType::VAR_INTEGER) {
00809         throw mx::Exception("set_program_args first argument (argc) must be an integer");
00810     }
00811
00812     if (argv_var.type != mxvm::VarType::VAR_POINTER) {
00813         throw mx::Exception("set_program_args second argument (argv) must be a pointer");
00814     }
00815
00816     int argc = (int)argc_var.var_value.int_value;
00817     const char **argv = (const char **)argv_var.var_value.ptr_value;
00818
00819     if (argv == nullptr && argc > 0) {
00820         throw mx::Exception("set_program_args: argv pointer is null but argc > 0");
00821     }
00822
00823     set_program_args(argc, argv);
00824 }
00825
00826 extern "C" void mxvm_std_float_to_int(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00827     if (operand.size() != 1) {
00828         throw mx::Exception("float_to_int requires 1 argument (float value).");
00829     }
00830
00831     double value = 0.0;
00832     if (program->isVariable(operand[0].op)) {
00833         mxvm::Variable &var = program->getVariable(operand[0].op);
00834         if (var.type == mxvm::VarType::VAR_FLOAT) {
00835             value = var.var_value.float_value;
00836         } else if (var.type == mxvm::VarType::VAR_INTEGER) {
00837             value = static_cast<double>(var.var_value.int_value);
00838         } else {
00839             throw mx::Exception("float_to_int argument must be a float or integer variable.");
00840         }
00841     } else {
00842         value = static_cast<double>(operand[0].op_value);
00843     }

```

```

00843     }
00844
00845     int64_t result = static_cast<int64_t>(value);
00846     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00847     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00848     program->vars["%rax"].var_value.int_value = result;
00849 }
00850
00851 extern "C" void mxvm_std_int_to_float(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00852     if (operand.size() != 1) {
00853         throw mx::Exception("int_to_float requires 1 argument (integer value).");
00854     }
00855
00856     int64_t value = 0;
00857     if (program->isVariable(operand[0].op)) {
00858         mxvm::Variable &var = program->getVariable(operand[0].op);
00859         if (var.type == mxvm::VarType::VAR_INTEGER) {
00860             value = var.var_value.int_value;
00861         } else if (var.type == mxvm::VarType::VAR_FLOAT) {
00862             value = static_cast<int64_t>(var.var_value.float_value);
00863         } else {
00864             throw mx::Exception("int_to_float argument must be an integer or float variable.");
00865         }
00866     } else {
00867         value = operand[0].op_value;
00868     }
00869
00870     double result = static_cast<double>(value);
00871     program->vars["%rax"].type = mxvm::VarType::VAR_FLOAT;
00872     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_FLOAT;
00873     program->vars["%rax"].var_value.float_value = result;
00874 }

```

12.42 modules/string/cstring.c File Reference

String module C implementation — string manipulation, comparison, and formatting.

```
#include <stddef.h>
```

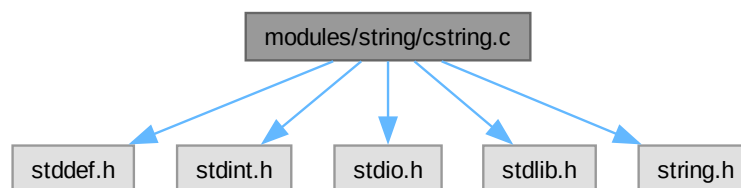
```
#include <stdint.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

Include dependency graph for cstring.c:



Functions

- `int64_t strfind` (const char *src, const char *search, long n)
- `int64_t substr` (char *dest, int64_t size, const char *src, int64_t pos, int64_t len)
- `int64_t strat` (const char *src, int64_t pos)
- `int64_t pos` (const char *substr, const char *s)
- `char * copy` (const char *s, int64_t index, int64_t count)
- `char * insert` (const char *source, const char *dest, int64_t index)
- `char * delete_str` (const char *s, int64_t index, int64_t count)
- `char * inttostr` (int64_t value)
- `int64_t strtoint` (const char *s)

12.42.1 Detailed Description

String module C implementation — string manipulation, comparison, and formatting.

Author

Jared Bruni

Definition in file [cstring.c](#).

12.42.2 Function Documentation

12.42.2.1 `copy()`

```
char * copy (
    const char * s,
    int64_t index,
    int64_t count)
```

Definition at line 60 of file [cstring.c](#).

12.42.2.2 `delete_str()`

```
char * delete_str (
    const char * s,
    int64_t index,
    int64_t count)
```

Definition at line 106 of file [cstring.c](#).

12.42.2.3 `insert()`

```
char * insert (
    const char * source,
    const char * dest,
    int64_t index)
```

Definition at line 83 of file [cstring.c](#).

12.42.2.4 `inttostr()`

```
char * inttostr (
    int64_t value)
```

Definition at line 136 of file [cstring.c](#).

12.42.2.5 `pos()`

```
int64_t pos (
    const char * substr,
    const char * s)
```

Definition at line 51 of file [cstring.c](#).

References [substr\(\)](#).

Referenced by [mxvm::ModuleParser::at\(\)](#), [mxvm::darwin_prefix_calls\(\)](#), [dirOf\(\)](#), [scan::buffer::StringBuffer< Ch, String >::eof\(\)](#), [scan::buffer::StringBuffer< char >::eof\(\)](#), [extractDeclName\(\)](#), [mxvm::Program::gen_jump\(\)](#), [mxvm::Program::getVariable\(\)](#), [scan::Scanner::grabDigits\(\)](#), [scan::Scanner::grabSingle\(\)](#), [scan::Scanner::grabString\(\)](#), [scan::Scanner::grabSymbols\(\)](#), [mxvm_string_strat\(\)](#), [mxvm_string_strfind\(\)](#), [mxvm_string_substr\(\)](#), [operator<<\(\)](#), [mxvm::ModuleParser::operator\[\]\(\)](#), [mxvm::Parser::operator\[\]\(\)](#), [mxx::TPValidator::parseUses\(\)](#), [mxx::Argz< String >::proc\(\)](#), [mxx::proc_args\(\)](#), [process_arguments\(\)](#), [scan::buffer::StringBuffer< Ch, String >::reset\(\)](#), [scan::buffer::StringBuffer< char >::reset\(\)](#), [scan::Scanner::scan\(\)](#), [strat\(\)](#), [substr\(\)](#), and [mxvm::Program::x64_gen_jump\(\)](#).

12.42.2.6 `strat()`

```
int64_t strat (
    const char * src,
    int64_t pos)
```

Definition at line 41 of file [cstring.c](#).

References [pos\(\)](#).

12.42.2.7 strfind()

```
int64_t strfind (
    const char * src,
    const char * search,
    long n)
```

Definition at line 12 of file [cstring.c](#).

12.42.2.8 strtoint()

```
int64_t strtoint (
    const char * s)
```

Definition at line 144 of file [cstring.c](#).

12.42.2.9 substr()

```
int64_t substr (
    char * dest,
    int64_t size,
    const char * src,
    int64_t pos,
    int64_t len)
```

Definition at line 25 of file [cstring.c](#).

References [pos\(\)](#).

Referenced by [pos\(\)](#).

12.43 cstring.c

[Go to the documentation of this file.](#)

```
00001
00006 #include <stddef.h>
00007 #include <stdint.h>
00008 #include <stdio.h>
00009 #include <stdlib.h>
00010 #include <string.h>
00011
00012 int64_t strfind(const char *src, const char *search, long n) {
00013     if (!src || !search || n < 0)
00014         return -1;
00015     size_t src_len = strlen(src);
00016     if ((size_t)n > src_len)
00017         return -1;
00018     const char *start = src + n;
00019     const char *found = strstr(start, search);
00020     if (!found)
00021         return -1;
00022     return (long)(found - src);
00023 }
00024
00025 int64_t substr(char *dest, int64_t size, const char *src, int64_t pos, int64_t len) {
00026     if (!dest || !src || size <= 0 || pos < 0 || len < 0)
00027         return 0;
00028     size_t src_len = strlen(src);
00029     if ((size_t)pos > src_len)
00030         return 0;
00031     size_t copy_len = (size_t)len;
00032     if (pos + copy_len > src_len)
00033         copy_len = src_len - pos;
00034     if (copy_len >= (size_t)size)
00035         copy_len = (size_t)size - 1;
00036     memcpy(dest, src + pos, copy_len);
00037     dest[copy_len] = '\0';
00038     return (long)copy_len;
00039 }
00040
00041 int64_t strat(const char *src, int64_t pos) {
00042     if (!src || pos < 0)
00043         return 0;
```

```

00044     size_t len = strlen(src);
00045     if ((size_t)pos >= len)
00046         return 0;
00047     char c = *(src + pos);
00048     return (int64_t)c;
00049 }
00050
00051 int64_t pos(const char *substr, const char *s) {
00052     if (!substr || !s)
00053         return 0;
00054     const char *found = strstr(s, substr);
00055     if (!found)
00056         return 0;
00057     return (int64_t)(found - s) + 1;
00058 }
00059
00060 char *copy(const char *s, int64_t index, int64_t count) {
00061     if (!s || index < 1 || count < 0)
00062         return NULL;
00063
00064     size_t s_len = strlen(s);
00065     size_t start_pos = (size_t)index - 1;
00066     if (start_pos > s_len)
00067         return NULL;
00068
00069     size_t copy_len = (size_t)count;
00070     if (start_pos + copy_len > s_len) {
00071         copy_len = s_len - start_pos;
00072     }
00073
00074     char *new_buf = (char *)malloc(copy_len + 1);
00075     if (!new_buf)
00076         return NULL;
00077
00078     memcpy(new_buf, s + start_pos, copy_len);
00079     new_buf[copy_len] = '\0';
00080     return new_buf;
00081 }
00082
00083 char *insert(const char *source, const char *dest, int64_t index) {
00084     if (!source || !dest || index < 1)
00085         return NULL;
00086
00087     size_t source_len = strlen(source);
00088     size_t dest_len = strlen(dest);
00089     size_t insert_pos = (size_t)index - 1;
00090     if (insert_pos > dest_len)
00091         insert_pos = dest_len;
00092
00093     size_t new_len = source_len + dest_len;
00094     char *new_buf = (char *)malloc(new_len + 1);
00095     if (!new_buf)
00096         return NULL;
00097
00098     memcpy(new_buf, dest, insert_pos);
00099     memcpy(new_buf + insert_pos, source, source_len);
00100     memcpy(new_buf + insert_pos + source_len, dest + insert_pos, dest_len - insert_pos);
00101
00102     new_buf[new_len] = '\0';
00103     return new_buf;
00104 }
00105
00106 char *delete_str(const char *s, int64_t index, int64_t count) {
00107     if (!s || index < 1 || count < 0)
00108         return NULL;
00109
00110     size_t s_len = strlen(s);
00111     size_t start_pos = (size_t)index - 1;
00112     if (start_pos >= s_len) {
00113         char *new_buf = (char *)malloc(s_len + 1);
00114         if (!new_buf)
00115             return NULL;
00116         strcpy(new_buf, s);
00117         return new_buf;
00118     }
00119
00120     size_t del_count = (size_t)count;
00121     if (start_pos + del_count > s_len) {
00122         del_count = s_len - start_pos;
00123     }
00124
00125     size_t new_len = s_len - del_count;
00126     char *new_buf = (char *)malloc(new_len + 1);
00127     if (!new_buf)
00128         return NULL;
00129
00130     memcpy(new_buf, s, start_pos);

```



```

00131     memcpy(new_buf + start_pos, s + start_pos + del_count, s_len - (start_pos + del_count));
00132     new_buf[new_len] = '\0';
00133     return new_buf;
00134 }
00135
00136 char *inttostr(int64_t value) {
00137     char *buf = (char *)malloc(21);
00138     if (!buf)
00139         return NULL;
00140     snprintf(buf, 21, "%lld", value);
00141     return buf;
00142 }
00143
00144 int64_t strtoint(const char *s) {
00145     if (!s)
00146         return 0;
00147     return atoll(s);
00148 }

```

12.44 modules/string/string.cpp File Reference

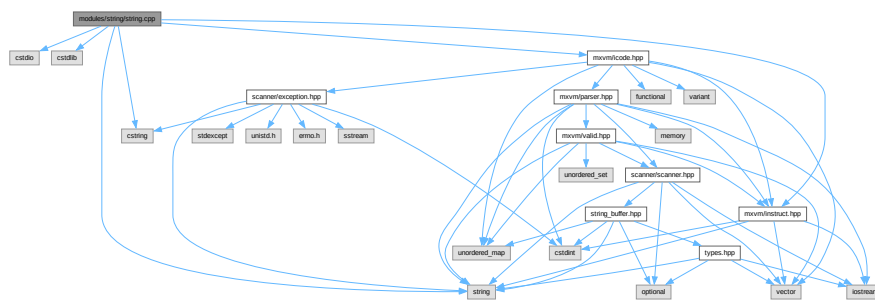
String module C++ runtime bindings for the MXVM interpreter.

```

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <mxvm/icode.hpp>
#include <mxvm/instruct.hpp>
#include <string>

```

Include dependency graph for string.cpp:



Functions

- static std::string [getStringFromVar](#) (mxvm::Program *program, const std::string &varName)
 - static bool [isStringLike](#) (mxvm::Variable &v)
 - static const char * [asReadPtr](#) (mxvm::Variable &v)
 - static char * [asWritePtr](#) (mxvm::Variable &v)
 - void [mxvm_string_strlen](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_strcmp](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_strncpy](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_strncat](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_snprintf](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_strfind](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_substr](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_strat](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_pos](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_copy](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_insert](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_delete](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
 - void [mxvm_string_inttostr](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)
- Convert an integer to its decimal string representation.*
- void [mxvm_string_strtoint](#) (mxvm::Program *program, std::vector< mxvm::Operand > &operand)

12.44.1 Detailed Description

String module C++ runtime bindings for the MXVM interpreter.

Author

Jared Bruni

Definition in file [string.cpp](#).

12.44.2 Function Documentation

12.44.2.1 asReadPtr()

```
const char * asReadPtr (
    mxvm::Variable & v) [inline], [static]
```

Definition at line 31 of file [string.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [mxvm_string_strfind\(\)](#), [mxvm_string_strncat\(\)](#), [mxvm_string_strncpy\(\)](#), and [mxvm_string_substr\(\)](#).

12.44.2.2 asWritePtr()

```
char * asWritePtr (
    mxvm::Variable & v) [inline], [static]
```

Definition at line 38 of file [string.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), and [mxvm::Variable::var_value](#).

Referenced by [mxvm_string_strncpy\(\)](#).

12.44.2.3 getStringFromVar()

```
std::string getStringFromVar (
    mxvm::Program * program,
    const std::string & varName) [static]
```

Definition at line 13 of file [string.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), and [mxvm::Variable::var_value](#).

Referenced by [mxvm_string_copy\(\)](#), [mxvm_string_delete\(\)](#), [mxvm_string_insert\(\)](#), [mxvm_string_pos\(\)](#), and [mxvm_string_strtoint\(\)](#).

12.44.2.4 isStringLike()

```
bool isStringLike (
    mxvm::Variable & v) [inline], [static]
```

Definition at line 28 of file [string.cpp](#).

References [mxvm::Variable::type](#), [mxvm::VAR_POINTER](#), and [mxvm::VAR_STRING](#).

Referenced by [mxvm_string_strfind\(\)](#), [mxvm_string_strncat\(\)](#), [mxvm_string_strncpy\(\)](#), and [mxvm_string_substr\(\)](#).

12.44.2.5 mxvm_string_copy()

```
void mxvm_string_copy (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 385 of file [string.cpp](#).

References [getStringFromVar\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.6 mxvm_string_delete()

```
void mxvm_string_delete (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 438 of file [string.cpp](#).

References [getStringFromVar\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.7 mxvm_string_insert()

```
void mxvm_string_insert (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 413 of file [string.cpp](#).

References [getStringFromVar\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.8 mxvm_string_inttostr()

```
void mxvm_string_inttostr (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Convert an integer to its decimal string representation.

Allocates a new heap buffer for the result string and returns it via `rax` as a pointer. Frees any previously-owned pointer in `rax` before overwriting to prevent memory leaks from repeated calls (e.g. `inttostr(score)` inside a render loop).

Parameters

<i>program</i>	VM program context.
<i>operand</i>	Single operand: the integer variable to convert.

Definition at line 477 of file [string.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.9 mxvm_string_pos()

```
void mxvm_string_pos (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 371 of file [string.cpp](#).

References [getStringFromVar\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.44.2.10 mxvm_string_snprintf()

```
void mxvm_string_snprintf (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 186 of file [string.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Variable_Value::float_value](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_BYTE](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_FLOAT](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.11 mxvm_string_strat()

```
void mxvm_string_strat (
```

```
mxvm::Program * program,
std::vector< mxvm::Operand > & operand)
```

Definition at line 351 of file [string.cpp](#).

References [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [mxvm::Program::isVariable\(\)](#), [mxvm::OP_CONSTANT](#), [pos\(\)](#), [mxvm::Variable_Value::str_value](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.12 mxvm_string_strcmp()

```
void mxvm_string_strcmp (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 76 of file [string.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_name](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.13 mxvm_string_strfind()

```
void mxvm_string_strfind (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 284 of file [string.cpp](#).

References [asReadPtr\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isStringLike\(\)](#), [mxvm::Program::isVariable\(\)](#), [pos\(\)](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.14 mxvm_string_strlen()

```
void mxvm_string_strlen (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 46 of file [string.cpp](#).

References [mxvm::except_assert\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_EXTERN](#), [mxvm::VAR_INTEGER](#), [mxvm::Variable::var_name](#), [mxvm::VAR_POINTER](#), [mxvm::VAR_STRING](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.15 mxvm_string_strncat()

```
void mxvm_string_strncat (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 151 of file [string.cpp](#).

References [asReadPtr\(\)](#), [mxvm::Variable_Value::buffer_size](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isStringLike\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.16 mxvm_string_strncpy()

```
void mxvm_string_strncpy (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)
```

Definition at line 117 of file [string.cpp](#).

References [asReadPtr\(\)](#), [asWritePtr\(\)](#), [mxvm::Variable_Value::buffer_size](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isStringLike\(\)](#), [mxvm::Program::isVariable\(\)](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.44.2.17 mxvm_string_strtoint()

```
void mxvm_string_strtoint (
```

```

    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)

```

Definition at line 502 of file [string.cpp](#).

References [getStringFromVar\(\)](#), [mxvm::VAR_INTEGER](#), and [mxvm::Base::vars](#).

12.44.2.18 mxvm_string_substr()

```

void mxvm_string_substr (
    mxvm::Program * program,
    std::vector< mxvm::Operand > & operand)

```

Definition at line 310 of file [string.cpp](#).

References [asReadPtr\(\)](#), [mxvm::Variable_Value::buffer_size](#), [mxvm::except_assert\(\)](#), [mxvm::Program::getVariable\(\)](#), [mxvm::Variable_Value::int_value](#), [isStringLike\(\)](#), [mxvm::Program::isVariable\(\)](#), [pos\(\)](#), [mxvm::Variable_Value::ptr_value](#), [mxvm::Variable_Value::str_value](#), [mxvm::Variable::type](#), [mxvm::VAR_INTEGER](#), [mxvm::VAR_POINTER](#), [mxvm::Variable::var_value](#), and [mxvm::Base::vars](#).

12.45 string.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include <cstdio>
00007 #include <cstdlib>
00008 #include <cstring>
00009 #include <mxvm/icode.hpp>
00010 #include <mxvm/instruct.hpp>
00011 #include <string>
00012
00013 static std::string getStringFromVar(mxvm::Program *program, const std::string &varName) {
00014     if (!program->isVariable(varName)) {
00015         throw mx::Exception("Argument must be a variable, got: " + varName);
00016     }
00017     mxvm::Variable &var = program->getVariable(varName);
00018     if (var.type == mxvm::VarType::VAR_STRING) {
00019         return var.var_value.str_value;
00020     }
00021     if (var.type == mxvm::VarType::VAR_POINTER) {
00022         mxvm::except_assert("Pointer argument '" + varName + "' is null", var.var_value.ptr_value !=
00023             nullptr);
00024         return std::string(reinterpret_cast<const char *>(var.var_value.ptr_value));
00025     }
00026     throw mx::Exception("Argument '" + varName + "' must be a string or pointer variable.");
00027 }
00028 static inline bool isStringLike(mxvm::Variable &v) {
00029     return v.type == mxvm::VarType::VAR_STRING || v.type == mxvm::VarType::VAR_POINTER;
00030 }
00031 static inline const char *asReadPtr(mxvm::Variable &v) {
00032     if (v.type == mxvm::VarType::VAR_POINTER) {
00033         mxvm::except_assert("null pointer", v.var_value.ptr_value != nullptr);
00034         return reinterpret_cast<const char *>(v.var_value.ptr_value);
00035     }
00036     return v.var_value.str_value.c_str();
00037 }
00038 static inline char *asWritePtr(mxvm::Variable &v) {
00039     if (v.type == mxvm::VarType::VAR_POINTER) {
00040         mxvm::except_assert("null pointer", v.var_value.ptr_value != nullptr);
00041         return reinterpret_cast<char *>(v.var_value.ptr_value);
00042     }
00043     return nullptr;
00044 }
00045
00046 extern "C" void mxvm_string_strlen(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00047     if (operand.size() != 1) {
00048         throw mx::Exception("strlen requires exactly 1 argument (string | pointer).");
00049     }
00050     std::string &name = operand[0].op;
00051     if (!program->isVariable(name)) {
00052         throw mx::Exception("strlen argument must be a declared variable (string | pointer): " +
00053             name);
00054     }
00055     mxvm::Variable &v = program->getVariable(name);
00056     const char *src = nullptr;
00057     switch (v.type) {
00058     case mxvm::VarType::VAR_STRING:
00059         src = v.var_value.str_value.c_str();
00060         break;

```

```

00061     case mxvm::VarType::VAR_POINTER:
00062     case mxvm::VarType::VAR_EXTERN:
00063         mxvm::except_assert("strlen: null pointer in variable '" + v.var_name + "'",
v.var_value.ptr_value != nullptr);
00064         src = reinterpret_cast<const char *>(v.var_value.ptr_value);
00065         break;
00066     default:
00067         throw mx::Exception("strlen argument '" + v.var_name + "' must be string or pointer variable
(got different type).");
00068     }
00069
00070     int64_t length = static_cast<int64_t>(std::strlen(src));
00071     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00072     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00073     program->vars["%rax"].var_value.int_value = length;
00074 }
00075
00076 extern "C" void mxvm_string_strcmp(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00077     if (operand.size() != 2) {
00078         throw mx::Exception("strcmp requires two pointer/string arguments.");
00079     }
00080     std::string &var1 = operand[0].op;
00081     std::string &var2 = operand[1].op;
00082
00083     if (!program->isVariable(var1) || !program->isVariable(var2)) {
00084         throw mx::Exception("strcmp arguments must be variables (pointer or string).");
00085     }
00086
00087     mxvm::Variable &v1 = program->getVariable(var1);
00088     mxvm::Variable &v2 = program->getVariable(var2);
00089
00090     const char *s1 = nullptr;
00091     const char *s2 = nullptr;
00092
00093     if (v1.type == mxvm::VarType::VAR_POINTER) {
00094         mxvm::except_assert("strcmp: pointer: " + v1.var_name + " is null", (v1.var_value.ptr_value !=
nullptr));
00095         s1 = reinterpret_cast<const char *>(v1.var_value.ptr_value);
00096     } else if (v1.type == mxvm::VarType::VAR_STRING) {
00097         s1 = v1.var_value.str_value.c_str();
00098     } else {
00099         throw mx::Exception("strcmp first argument must be a pointer or string variable.");
00100     }
00101
00102     if (v2.type == mxvm::VarType::VAR_POINTER) {
00103         mxvm::except_assert("strcmp: pointer: " + v2.var_name + " is null", (v2.var_value.ptr_value !=
nullptr));
00104         s2 = reinterpret_cast<const char *>(v2.var_value.ptr_value);
00105     } else if (v2.type == mxvm::VarType::VAR_STRING) {
00106         s2 = v2.var_value.str_value.c_str();
00107     } else {
00108         throw mx::Exception("strcmp second argument must be a pointer or string variable.");
00109     }
00110
00111     int64_t cmp_result = strcmp(s1, s2);
00112     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00113     program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00114     program->vars["%rax"].var_value.int_value = cmp_result;
00115 }
00116
00117 extern "C" void mxvm_string_strncpy(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00118     if (operand.size() != 3)
00119         throw mx::Exception("strncpy requires (dest, src, len)");
00120     if (!program->isVariable(operand[0].op) || !program->isVariable(operand[1].op))
00121         throw mx::Exception("strncpy arguments must be variables");
00122     mxvm::Variable &dest = program->getVariable(operand[0].op);
00123     mxvm::Variable &src = program->getVariable(operand[1].op);
00124
00125     int64_t n = operand[2].op_value;
00126     if (program->isVariable(operand[2].op)) {
00127         mxvm::Variable &iv = program->getVariable(operand[2].op);
00128         n = iv.var_value.int_value;
00129     }
00130
00131     if (!isStringLike(dest))
00132         throw mx::Exception("strncpy dest must be pointer or string");
00133     if (!isStringLike(src))
00134         throw mx::Exception("strncpy src must be pointer or string");
00135
00136     const char *sptr = asReadPtr(src);
00137     if (dest.type == mxvm::VarType::VAR_POINTER) {
00138         if (dest.var_value.buffer_size && (int64_t)dest.var_value.buffer_size < n + 1)
00139             throw mx::Exception("strncpy dest buffer too small");
00140         char *dptr = asWritePtr(dest);
00141         std::strncpy(dptr, sptr, n);
00142         dptr[n] = '\0';
00143     } else {

```

```

00144         dest.var_value.str_value.assign(sptr, (size_t)n);
00145     }
00146
00147     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00148     program->vars["%rax"].var_value.int_value = n;
00149 }
00150
00151 extern "C" void mxvm_string_strncat(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00152     if (operand.size() != 3)
00153         throw mx::Exception("strncat requires (dest, src, len)");
00154     if (!program->isVariable(operand[0].op) || !program->isVariable(operand[1].op))
00155         throw mx::Exception("strncat arguments must be variables");
00156     mxvm::Variable &dest = program->getVariable(operand[0].op);
00157     mxvm::Variable &src = program->getVariable(operand[1].op);
00158
00159     int64_t n = operand[2].op_value;
00160     if (program->isVariable(operand[2].op)) {
00161         mxvm::Variable &iv = program->getVariable(operand[2].op);
00162         n = iv.var_value.int_value;
00163     }
00164
00165     if (!isStringLike(dest))
00166         throw mx::Exception("strncat dest must be pointer or string");
00167     if (!isStringLike(src))
00168         throw mx::Exception("strncat src must be pointer or string");
00169
00170     const char *sptr = asReadPtr(src);
00171     if (dest.type == mxvm::VarType::VAR_POINTER) {
00172         if (dest.var_value.buffer_size) {
00173             size_t have = std::strlen(reinterpret_cast<const char *>(dest.var_value.ptr_value));
00174             if (have + (size_t)n + 1 > dest.var_value.buffer_size)
00175                 throw mx::Exception("strncat dest buffer too small");
00176         }
00177         std::strncat(reinterpret_cast<char *>(dest.var_value.ptr_value), sptr, n);
00178     } else {
00179         dest.var_value.str_value.append(sptr, (size_t)n);
00180     }
00181
00182     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00183     program->vars["%rax"].var_value.int_value = n;
00184 }
00185
00186 extern "C" void mxvm_string_snprintf(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00187     if (operand.size() < 4) {
00188         throw mx::Exception("sprintf requires at least destination, size, format, and one
argument.");
00189     }
00190     std::string &dest_var = operand[0].op;
00191     int64_t n = operand[1].op_value;
00192     std::string &fmt_var = operand[2].op;
00193
00194     if (!program->isVariable(dest_var) || !program->isVariable(fmt_var)) {
00195         throw mx::Exception("sprintf destination: " + dest_var + " and format: " + fmt_var + " must
be variables.");
00196     }
00197
00198     if (program->isVariable(operand[1].op)) {
00199         mxvm::Variable &iv = program->getVariable(operand[1].op);
00200         n = iv.var_value.int_value;
00201     }
00202
00203     mxvm::except_assert("sprintf size is zero", n != 0);
00204     mxvm::Variable &dest = program->getVariable(dest_var);
00205     mxvm::Variable &fmt = program->getVariable(fmt_var);
00206
00207     if (fmt.type != mxvm::VarType::VAR_STRING) {
00208         throw mx::Exception("sprintf format must be a string variable.");
00209     }
00210
00211     std::ostringstream oss;
00212     size_t argIndex = 3;
00213     const char *format = fmt.var_value.str_value.c_str();
00214     char buffer[4096];
00215
00216     for (size_t i = 0; i < fmt.var_value.str_value.length(); ++i) {
00217         if (format[i] == '%' && i + 1 < fmt.var_value.str_value.length()) {
00218             size_t start = i;
00219             size_t j = i + 1;
00220             while (j < fmt.var_value.str_value.length() &&
00221                 (format[j] == '-' || format[j] == '+' || format[j] == ' ' || format[j] == '#' ||
format[j] == '0' ||
00222                 (format[j] >= '0' && format[j] <= '9') || format[j] == '.' ||
00223                 format[j] == 'l' || format[j] == 'h' || format[j] == 'z' || format[j] == 'j' ||
format[j] == 't')) {
00224                 ++j;
00225             }
00226             if (j < fmt.var_value.str_value.length() && format[j] == '%') {

```

```

00227         oss << '%';
00228         i = j;
00229         continue;
00230     }
00231     if (j >= fmt.var_value.str_value.length())
00232         break;
00233     if (!std::isalpha(format[j])) {
00234         oss << format[i];
00235         continue;
00236     }
00237     std::string spec(format + start, format + j + 1);
00238     if (argIndex < operand.size()) {
00239         std::string &arg_var = operand[argIndex++].op;
00240         if (!program->isVariable(arg_var)) {
00241             throw mx::Exception("snprintf argument must be a variable.");
00242         }
00243         mxvm::Variable &arg = program->getVariable(arg_var);
00244         if (arg.type == mxvm::VarType::VAR_INTEGER) {
00245             std::snprintf(buffer, sizeof(buffer), spec.c_str(), arg.var_value.int_value);
00246         } else if (arg.type == mxvm::VarType::VAR_POINTER || arg.type ==
mxvm::VarType::VAR_EXTERN) {
00247             std::snprintf(buffer, sizeof(buffer), spec.c_str(), arg.var_value.ptr_value);
00248         } else if (arg.type == mxvm::VarType::VAR_FLOAT) {
00249             std::snprintf(buffer, sizeof(buffer), spec.c_str(), arg.var_value.float_value);
00250         } else if (arg.type == mxvm::VarType::VAR_STRING) {
00251             std::snprintf(buffer, sizeof(buffer), spec.c_str(),
arg.var_value.str_value.c_str());
00252         } else if (arg.type == mxvm::VarType::VAR_BYTE) {
00253             std::snprintf(buffer, sizeof(buffer), spec.c_str(), arg.var_value.int_value);
00254         } else {
00255             std::snprintf(buffer, sizeof(buffer), "%s", "(unsupported)");
00256         }
00257         oss << buffer;
00258     } else {
00259         oss << spec;
00260     }
00261     i = j;
00262 } else {
00263     oss << format[i];
00264 }
00265 }
00266
00267 if (dest.type == mxvm::VarType::VAR_POINTER) {
00268     mxvm::except_assert("snprintf dest pointer is null", dest.var_value.ptr_value != nullptr);
00269     std::strncpy(reinterpret_cast<char *>(dest.var_value.ptr_value), oss.str().c_str(), n);
00270     if (n > 0) {
00271         reinterpret_cast<char *>(dest.var_value.ptr_value)[n - 1] = '\0';
00272     }
00273 } else if (dest.type == mxvm::VarType::VAR_STRING) {
00274     dest.var_value.str_value = oss.str().substr(0, n);
00275 } else {
00276     throw mx::Exception("snprintf destination must be a pointer or string variable.");
00277 }
00278
00279 program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00280 program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00281 program->vars["%rax"].var_value.int_value = oss.str().length();
00282 }
00283
00284 extern "C" void mxvm_string_strfind(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00285     if (operand.size() != 3)
00286         throw mx::Exception("strfind requires (haystack, needle, start)");
00287     if (!program->isVariable(operand[0].op) || !program->isVariable(operand[1].op))
00288         throw mx::Exception("strfind arguments must be variables");
00289     mxvm::Variable &hay = program->getVariable(operand[0].op);
00290     mxvm::Variable &needle = program->getVariable(operand[1].op);
00291
00292     int64_t start = operand[2].op_value;
00293     if (program->isVariable(operand[2].op)) {
00294         mxvm::Variable &iv = program->getVariable(operand[2].op);
00295         start = iv.var_value.int_value;
00296     }
00297
00298     if (!isStringLike(hay) || !isStringLike(needle))
00299         throw mx::Exception("strfind args must be pointer or string");
00300
00301     std::string H = asReadPtr(hay);
00302     std::string N = asReadPtr(needle);
00303
00304     size_t pos = H.find(N, (size_t)start);
00305     int64_t result = (pos == std::string::npos) ? -1 : (int64_t)pos;
00306     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00307     program->vars["%rax"].var_value.int_value = result;
00308 }
00309
00310 extern "C" void mxvm_string_substr(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00311     if (operand.size() != 5)

```



```

00312         throw mx::Exception("substr requires 5 args");
00313     if (!program->isVariable(operand[0].op) || !program->isVariable(operand[2].op))
00314         throw mx::Exception("substr dest/src must be variables");
00315     mxvm::Variable &dest = program->getVariable(operand[0].op);
00316     mxvm::Variable &src = program->getVariable(operand[2].op);
00317
00318     if (!isStringLike(dest) || !isStringLike(src))
00319         throw mx::Exception("substr dest/src must be pointer or string");
00320
00321     int64_t size = operand[1].op_value;
00322     int64_t pos = operand[3].op_value;
00323     int64_t len = operand[4].op_value;
00324     if (program->isVariable(operand[1].op))
00325         size = program->getVariable(operand[1].op).var_value.int_value;
00326     if (program->isVariable(operand[3].op))
00327         pos = program->getVariable(operand[3].op).var_value.int_value;
00328     if (program->isVariable(operand[4].op))
00329         len = program->getVariable(operand[4].op).var_value.int_value;
00330
00331     std::string S = asReadPtr(src);
00332     if (pos < 0)
00333         pos = 0;
00334     std::string out = (pos > (int64_t)S.size()) ? "" : S.substr((size_t)pos, (size_t)len);
00335
00336     if (dest.type == mxvm::VarType::VAR_POINTER) {
00337         mxvm::exception_assert("substr dest ptr null", dest.var_value.ptr_value != nullptr);
00338         if (size <= 0 || (dest.var_value.buffer_size && (size_t)size > dest.var_value.buffer_size))
00339             size = (int64_t)dest.var_value.buffer_size;
00340         std::strncpy(reinterpret_cast<char*>(dest.var_value.ptr_value), out.c_str(), (size_t)size);
00341         if (size > 0)
00342             reinterpret_cast<char*>(dest.var_value.ptr_value)[size - 1] = '\0';
00343     } else {
00344         dest.var_value.str_value = out.substr(0, (size_t)size);
00345     }
00346
00347     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00348     program->vars["%rax"].var_value.int_value = (int64_t)out.size();
00349 }
00350
00351 extern "C" void mxvm_string_strat(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00352     if (operand.size() == 2 && program->isVariable(operand[0].op)) {
00353
00354         int64_t pos = 0;
00355         if (operand[1].type == mxvm::OperandType::OP_CONSTANT) {
00356             pos = operand[1].op_value;
00357         } else {
00358             mxvm::Variable &v = program->getVariable(operand[1].op);
00359             pos = v.var_value.int_value;
00360         }
00361         mxvm::Variable &var = program->getVariable(operand[0].op);
00362         if (pos < 0 || static_cast<size_t>(pos) >= var.var_value.str_value.size()) {
00363             throw mx::Exception("strat: index " + std::to_string(pos) + " out of bounds for string of
length " + std::to_string(var.var_value.str_value.size()));
00364         }
00365         program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00366         program->vars["%rax"].var_value.type = mxvm::VarType::VAR_INTEGER;
00367         program->vars["%rax"].var_value.int_value =
static_cast<int64_t>(var.var_value.str_value[static_cast<size_t>(pos)]);
00368     }
00369 }
00370
00371 extern "C" void mxvm_string_pos(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00372     if (operand.size() != 2)
00373         throw mx::Exception("pos requires 2 arguments: (substr, s)");
00374
00375     std::string sub = getStringFromVar(program, operand[0].op);
00376     std::string s = getStringFromVar(program, operand[1].op);
00377
00378     size_t position = s.find(sub);
00379     int64_t result = (position == std::string::npos) ? 0 : static_cast<int64_t>(position + 1);
00380
00381     program->vars["%rax"].var_value.int_value = result;
00382     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00383 }
00384
00385 extern "C" void mxvm_string_copy(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00386     if (operand.size() != 3)
00387         throw mx::Exception("copy requires 3 arguments: (s, index, count)");
00388
00389     std::string s = getStringFromVar(program, operand[0].op);
00390     int64_t index = program->getVariable(operand[1].op).var_value.int_value;
00391     int64_t count = program->getVariable(operand[2].op).var_value.int_value;
00392
00393     if (index < 1 || static_cast<size_t>(index - 1) > s.size()) {
00394         throw mx::Exception("copy: index " + std::to_string(index) + " out of bounds for string of
length " + std::to_string(s.size()));
00395     }

```

```

00396     if (count < 0) {
00397         throw mx::Exception("copy: count must be non-negative");
00398     }
00399     std::string result_str = s.substr(static_cast<size_t>(index - 1), static_cast<size_t>(count));
00400
00401     char *new_buf = static_cast<char *>(malloc(result_str.length() + 1));
00402     if (!new_buf)
00403         throw mx::Exception("malloc failed in copy()");
00404     strcpy(new_buf, result_str.c_str());
00405
00406     program->vars["%rax"].var_value.ptr_value = new_buf;
00407     program->vars["%rax"].var_value.ptr_size = 1;
00408     program->vars["%rax"].var_value.ptr_count = static_cast<int64_t>(result_str.length() + 1);
00409     program->vars["%rax"].var_value.owns = true;
00410     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00411 }
00412
00413 extern "C" void mxvm_string_insert(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00414     if (operand.size() != 3)
00415         throw mx::Exception("insert requires 3 arguments: (source, dest, index)");
00416
00417     std::string source = getStringFromVar(program, operand[0].op);
00418     std::string dest = getStringFromVar(program, operand[1].op);
00419     int64_t index = program->getVariable(operand[2].op).var_value.int_value;
00420
00421     if (index < 1 || static_cast<size_t>(index - 1) > dest.size()) {
00422         throw mx::Exception("insert: index " + std::to_string(index) + " out of bounds for string of
length " + std::to_string(dest.size()));
00423     }
00424     dest.insert(static_cast<size_t>(index - 1), source);
00425
00426     char *new_buf = static_cast<char *>(malloc(dest.length() + 1));
00427     if (!new_buf)
00428         throw mx::Exception("malloc failed in insert()");
00429     strcpy(new_buf, dest.c_str());
00430
00431     program->vars["%rax"].var_value.ptr_value = new_buf;
00432     program->vars["%rax"].var_value.ptr_size = 1;
00433     program->vars["%rax"].var_value.ptr_count = static_cast<int64_t>(dest.length() + 1);
00434     program->vars["%rax"].var_value.owns = true;
00435     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00436 }
00437
00438 extern "C" void mxvm_string_delete(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00439     if (operand.size() != 3)
00440         throw mx::Exception("delete requires 3 arguments: (s, index, count)");
00441
00442     std::string s = getStringFromVar(program, operand[0].op);
00443     int64_t index = program->getVariable(operand[1].op).var_value.int_value;
00444     int64_t count = program->getVariable(operand[2].op).var_value.int_value;
00445
00446     if (index < 1 || static_cast<size_t>(index - 1) > s.size()) {
00447         throw mx::Exception("delete: index " + std::to_string(index) + " out of bounds for string of
length " + std::to_string(s.size()));
00448     }
00449     if (count < 0) {
00450         throw mx::Exception("delete: count must be non-negative");
00451     }
00452     s.erase(static_cast<size_t>(index - 1), static_cast<size_t>(count));
00453
00454     char *new_buf = static_cast<char *>(malloc(s.length() + 1));
00455     if (!new_buf)
00456         throw mx::Exception("malloc failed in delete()");
00457     strcpy(new_buf, s.c_str());
00458
00459     program->vars["%rax"].var_value.ptr_value = new_buf;
00460     program->vars["%rax"].var_value.ptr_size = 1;
00461     program->vars["%rax"].var_value.ptr_count = static_cast<int64_t>(s.length() + 1);
00462     program->vars["%rax"].var_value.owns = true;
00463     program->vars["%rax"].type = mxvm::VarType::VAR_POINTER;
00464 }
00465
00477 extern "C" void mxvm_string_inttostr(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00478     if (operand.size() != 1)
00479         throw mx::Exception("inttostr requires 1 argument: (integer)");
00480
00481     int64_t val = program->getVariable(operand[0].op).var_value.int_value;
00482     std::string result_str = std::to_string(val);
00483
00484     char *new_buf = static_cast<char *>(malloc(result_str.length() + 1));
00485     if (!new_buf)
00486         throw mx::Exception("malloc failed in inttostr()");
00487     strcpy(new_buf, result_str.c_str());
00488
00489     // Free old owned pointer in %rax before overwriting
00490     auto &rax = program->vars["%rax"];
00491     if (rax.type == mxvm::VarType::VAR_POINTER && rax.var_value.ptr_value && rax.var_value.owns) {

```

```

00492         std::free(rax.var_value.ptr_value);
00493     }
00494
00495     rax.var_value.ptr_value = new_buf;
00496     rax.var_value.ptr_size = 1;
00497     rax.var_value.ptr_count = static_cast<int64_t>(result_str.length() + 1);
00498     rax.var_value.owns = true;
00499     rax.type = mxvm::VarType::VAR_POINTER;
00500 }
00501
00502 extern "C" void mxvm_string_strtoint(mxvm::Program *program, std::vector<mxvm::Operand> &operand) {
00503     if (operand.size() != 1)
00504         throw mx::Exception("strtoint requires 1 argument: (string)");
00505
00506     std::string s = getStringFromVar(program, operand[0].op);
00507     int64_t result = 0;
00508     try {
00509         result = std::stoll(s);
00510     } catch (const std::exception &e) {
00511         result = 0;
00512     }
00513
00514     program->vars["%rax"].var_value.int_value = result;
00515     program->vars["%rax"].type = mxvm::VarType::VAR_INTEGER;
00516 }

```

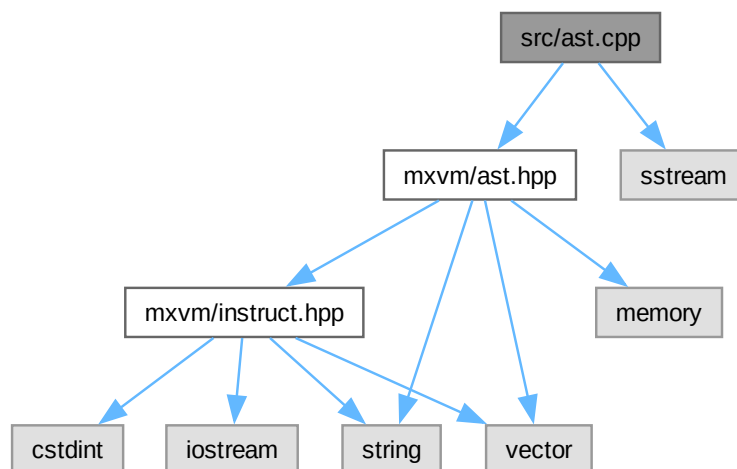
12.46 src/ast.cpp File Reference

AST node accept() and toString() implementations for the MXVM Visitor pattern.

```
#include "mxvm/ast.hpp"
```

```
#include <sstream>
```

Include dependency graph for ast.cpp:



Namespaces

- namespace `mxvm`

12.46.1 Detailed Description

AST node accept() and toString() implementations for the MXVM Visitor pattern.

Author

Jared Bruni

Definition in file [ast.cpp](#).

12.47 ast.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "mxvm/ast.hpp"
00008 #include <sstream>
00009
00010 namespace mxvm {
00011
00012     void ProgramNode::accept(ASTVisitor &visitor) {
00013         visitor.visit(*this);
00014     }
00015
00016     std::string ProgramNode::toString() const {
00017         std::ostringstream oss;
00018         std::string ptype = (object == true) ? "Object" : "Program";
00019         oss << ptype << " {\n";
00020         for (const auto &section : sections) {
00021             oss << "   " << section->toString() << "\n";
00022         }
00023         oss << "}";
00024         return oss.str();
00025     }
00026
00027     void SectionNode::accept(ASTVisitor &visitor) {
00028         visitor.visit(*this);
00029     }
00030
00031     std::string SectionNode::toString() const {
00032         std::ostringstream oss;
00033         oss << "Section " << (type == DATA ? "data" : "code") << " {\n";
00034         for (const auto &stmt : statements) {
00035             oss << "   " << stmt->toString() << "\n";
00036         }
00037         oss << "}";
00038         return oss.str();
00039     }
00040
00041     void InstructionNode::accept(ASTVisitor &visitor) {
00042         visitor.visit(*this);
00043     }
00044
00045     std::string InstructionNode::toString() const {
00046         std::ostringstream oss;
00047         oss << "Instruction: " << instruction;
00048         for (const auto &operand : operands) {
00049             oss << " " << operand.op;
00050         }
00051         return oss.str();
00052     }
00053
00054     void VariableNode::accept(ASTVisitor &visitor) {
00055         visitor.visit(*this);
00056     }
00057
00058     std::string VariableNode::toString() const {
00059         std::ostringstream oss;
00060         oss << "Variable: " << name << " (type: " << type << ")";
00061         if (hasInitializer) {
00062             oss << " = " << initialValue;
00063         }
00064         return oss.str();
00065     }
00066
00067     void ModuleNode::accept(ASTVisitor &visitor) {
00068         visitor.visit(*this);
00069     }
00070
00071     std::string ModuleNode::toString() const {
00072         std::ostringstream stream;
00073         stream << "Module: (" << name << ")";
00074         return stream.str();
00075     }
00076
00077     void ObjectNode::accept(ASTVisitor &visitor) {
00078         visitor.visit(*this);

```

```

00079     }
00080
00081     std::string ObjectNode::toString() const {
00082         std::ostringstream stream;
00083         stream << "Object: (" << name << ")";
00084         return stream.str();
00085     }
00086
00087     void LabelNode::accept(ASTVisitor &visitor) {
00088         visitor.visit(*this);
00089     }
00090
00091     std::string LabelNode::toString() const {
00092         return "Label: " + name;
00093     }
00094
00095     void ExpressionNode::accept(ASTVisitor &visitor) {
00096         visitor.visit(*this);
00097     }
00098
00099     std::string ExpressionNode::toString() const {
00100         return "Expression: " + value + (isNumeric ? " (numeric)" : " (symbolic)");
00101     }
00102
00103     void CommentNode::accept(ASTVisitor &visitor) {
00104         visitor.visit(*this);
00105     }
00106
00107     std::string CommentNode::toString() const {
00108         return "Comment: " + text;
00109     }
00110 } // namespace mxvm

```

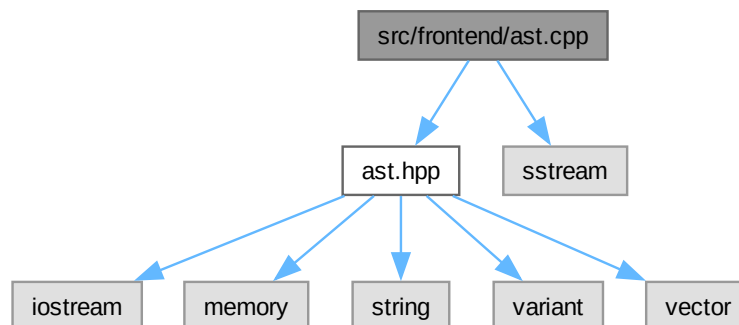
12.48 src/frontend/ast.cpp File Reference

Pascal AST node accept(), toString(), and print() implementations.

```
#include "ast.hpp"
```

```
#include <sstream>
```

Include dependency graph for ast.cpp:



Namespaces

- namespace `pascal`

12.48.1 Detailed Description

Pascal AST node accept(), toString(), and print() implementations.

Author

Jared Bruni

Definition in file [ast.cpp](#).

12.49 ast.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "ast.hpp"
00007 #include <sstream>
00008
00009 namespace pascal {
00010
00011     BlockNode::BlockNode(std::vector<std::unique_ptr<ASTNode> > decls, std::unique_ptr<CompoundStmtNode>
stmt)
00012         : declarations(std::move(decls)), compoundStatement(std::move(stmt)) {}
00013
00014     ProgramNode::ProgramNode(const std::string &p_name, std::unique_ptr<BlockNode> blk)
00015         : name(p_name), block(std::move(blk)) {}
00016
00017     void ASTNode::print(std::ostream &out, int indent) const {
00018         for (int i = 0; i < indent; ++i)
00019             out << " ";
00020         out << toString() << std::endl;
00021     }
00022
00023     void ProgramNode::accept(ASTVisitor &visitor) {
00024         visitor.visit(*this);
00025     }
00026
00027     std::string ProgramNode::toString() const {
00028         return "Program: " + name;
00029     }
00030
00031     void UnitNode::accept(ASTVisitor &visitor) {
00032         visitor.visit(*this);
00033     }
00034
00035     std::string UnitNode::toString() const {
00036         return "Unit: " + name;
00037     }
00038
00039     void BlockNode::accept(ASTVisitor &visitor) {
00040         visitor.visit(*this);
00041     }
00042
00043     std::string BlockNode::toString() const {
00044         return "Block with " + std::to_string(declarations.size()) + " declarations";
00045     }
00046
00047     void VarDeclNode::accept(ASTVisitor &visitor) {
00048         visitor.visit(*this);
00049     }
00050
00051     std::string VarDeclNode::toString() const {
00052         std::ostringstream oss;
00053         oss << "VarDecl: ";
00054         for (size_t i = 0; i < identifiers.size(); ++i) {
00055             if (i > 0)
00056                 oss << ", ";
00057             oss << identifiers[i];
00058         }
00059         oss << " : ";
00060         std::visit([&oss](const auto &t) {
00061             using T = std::decay_t<decltype(t)>;
00062             if constexpr (std::is_same_v<T, std::string>) {
00063                 oss << t;
00064             } else if constexpr (std::is_same_v<T, std::unique_ptr<ASTNode> >) {
00065                 if (t)
00066                     oss << t->toString();
00067                 else
00068                     oss << "<null>";
00069             }
00070         },
00071             type);
00072         return oss.str();
00073     }
00074
00075     void ProcDeclNode::accept(ASTVisitor &visitor) {
00076         visitor.visit(*this);
00077     }

```

```

00078
00079     std::string ProcDeclNode::toString() const {
00080         return "ProcDecl: " + name + " (" + std::to_string(parameters.size()) + " params)";
00081     }
00082
00083     void FuncDeclNode::accept(ASTVisitor &visitor) {
00084         visitor.visit(*this);
00085     }
00086
00087     std::string FuncDeclNode::toString() const {
00088         return "FuncDecl: " + name + " -> " + returnType;
00089     }
00090
00091     void ParameterNode::accept(ASTVisitor &visitor) {
00092         visitor.visit(*this);
00093     }
00094
00095     std::string ParameterNode::toString() const {
00096         std::ostringstream oss;
00097         oss << "Param: ";
00098         if (isVar)
00099             oss << "var ";
00100         for (size_t i = 0; i < identifiers.size(); ++i) {
00101             if (i > 0)
00102                 oss << ", ";
00103             oss << identifiers[i];
00104         }
00105         oss << " : " << type;
00106         return oss.str();
00107     }
00108
00109     void CompoundStmtNode::accept(ASTVisitor &visitor) {
00110         visitor.visit(*this);
00111     }
00112
00113     std::string CompoundStmtNode::toString() const {
00114         return "CompoundStmt: " + std::to_string(statements.size()) + " statements";
00115     }
00116
00117     void AssignmentNode::accept(ASTVisitor &visitor) {
00118         visitor.visit(*this);
00119     }
00120
00121     std::string AssignmentNode::toString() const {
00122         return "Assignment";
00123     }
00124
00125     void IfStmtNode::accept(ASTVisitor &visitor) {
00126         visitor.visit(*this);
00127     }
00128
00129     std::string IfStmtNode::toString() const {
00130         std::string s;
00131         if (elseStatement != nullptr) {
00132             s = "with else";
00133         }
00134         return "IfStmt " + s;
00135     }
00136
00137     void WhileStmtNode::accept(ASTVisitor &visitor) {
00138         visitor.visit(*this);
00139     }
00140
00141     std::string WhileStmtNode::toString() const {
00142         return "WhileStmt";
00143     }
00144
00145     void ForStmtNode::accept(ASTVisitor &visitor) {
00146         visitor.visit(*this);
00147     }
00148
00149     std::string ForStmtNode::toString() const {
00150         return "ForStmt: " + variable + (isDownto ? " downto" : " to");
00151     }
00152
00153     void RepeatStmtNode::accept(ASTVisitor &visitor) {
00154         visitor.visit(*this);
00155     }
00156
00157     void ProcCallNode::accept(ASTVisitor &visitor) {
00158         visitor.visit(*this);
00159     }
00160
00161     std::string ProcCallNode::toString() const {
00162         return "ProcCall: " + name + " (" + std::to_string(arguments.size()) + " args)";
00163     }
00164

```

```

00165 void BinaryOpNode::accept(ASTVisitor &visitor) {
00166     visitor.visit(*this);
00167 }
00168
00169 std::string BinaryOpNode::toString() const {
00170     return "BinaryOp: " + opToString(operator_);
00171 }
00172
00173 std::string BinaryOpNode::opToString(OpType op) {
00174     switch (op) {
00175         case PLUS:
00176             return "+";
00177         case MINUS:
00178             return "-";
00179         case MULTIPLY:
00180             return "*";
00181         case DIVIDE:
00182             return "/";
00183         case DIV:
00184             return "div";
00185         case MOD:
00186             return "mod";
00187         case EQUAL:
00188             return "=";
00189         case NOT_EQUAL:
00190             return "<>";
00191         case LESS:
00192             return "<";
00193         case LESS_EQUAL:
00194             return "<=";
00195         case GREATER:
00196             return ">";
00197         case GREATER_EQUAL:
00198             return ">=";
00199         case AND:
00200             return "and";
00201         case OR:
00202             return "or";
00203         case IN:
00204             return "in";
00205         default:
00206             return "unknown";
00207     }
00208 }
00209
00210 void UnaryOpNode::accept(ASTVisitor &visitor) {
00211     visitor.visit(*this);
00212 }
00213
00214 std::string UnaryOpNode::toString() const {
00215     return "UnaryOp: " + opToString(operator_);
00216 }
00217
00218 std::string UnaryOpNode::opToString(Operator op) {
00219     switch (op) {
00220         case PLUS:
00221             return "+";
00222         case MINUS:
00223             return "-";
00224         case NOT:
00225             return "not";
00226         default:
00227             return "unknown";
00228     }
00229 }
00230
00231 void FuncCallNode::accept(ASTVisitor &visitor) {
00232     visitor.visit(*this);
00233 }
00234
00235 std::string FuncCallNode::toString() const {
00236     return "FuncCall: " + name + " (" + std::to_string(arguments.size()) + " args)";
00237 }
00238
00239 void VariableNode::accept(ASTVisitor &visitor) {
00240     visitor.visit(*this);
00241 }
00242
00243 std::string VariableNode::toString() const {
00244     return "Variable: " + name;
00245 }
00246
00247 void NumberNode::accept(ASTVisitor &visitor) {
00248     visitor.visit(*this);
00249 }
00250
00251 std::string NumberNode::toString() const {

```



```

00252         return "Number: " + value + (isInteger ? " (int)" : " (real)");
00253     }
00254
00255     void StringNode::accept(ASTVisitor &visitor) {
00256         visitor.visit(*this);
00257     }
00258
00259     std::string StringNode::toString() const {
00260         return "String: \"" + value + "\"";
00261     }
00262
00263     void BooleanNode::accept(ASTVisitor &visitor) {
00264         visitor.visit(*this);
00265     }
00266
00267     std::string BooleanNode::toString() const {
00268         std::string s;
00269         if (value) {
00270             s = "true";
00271         } else {
00272             s = "false";
00273         }
00274         return "Boolean: " + s;
00275     }
00276
00277     void EmptyStmtNode::accept(ASTVisitor &visitor) {
00278         visitor.visit(*this);
00279     }
00280
00281     std::string EmptyStmtNode::toString() const {
00282         return "EmptyStmt";
00283     }
00284
00285     void ConstDeclNode::accept(ASTVisitor &visitor) {
00286         visitor.visit(*this);
00287     }
00288
00289     std::string ConstDeclNode::toString() const {
00290         return "ConstDecl";
00291     }
00292
00293     void CaseStmtNode::accept(ASTVisitor &visitor) {
00294         visitor.visit(*this);
00295     }
00296
00297     std::string CaseStmtNode::toString() const {
00298         return "CaseStmt: " + std::to_string(branches.size()) + " branches";
00299     }
00300
00301     void ArrayTypeNode::accept(ASTVisitor &visitor) {
00302         visitor.visit(*this);
00303     }
00304
00305     std::string ArrayTypeNode::toString() const {
00306         std::string elemTypeStr = "unknown";
00307         if (elementType) {
00308             elemTypeStr = elementType->toString();
00309         }
00310
00311         std::string lowerStr = "?";
00312         if (lowerBound) {
00313             lowerStr = lowerBound->toString();
00314         }
00315
00316         std::string upperStr = "?";
00317         if (upperBound) {
00318             upperStr = upperBound->toString();
00319         }
00320
00321         return "ArrayType: " + elemTypeStr + "[" + lowerStr + ".." + upperStr + "]";
00322     }
00323
00324     std::string ExitNode::toString() const { return "exit"; }
00325
00326     void ArrayDeclarationNode::accept(ASTVisitor &visitor) {
00327         visitor.visit(*this);
00328     }
00329
00330     std::string ArrayDeclarationNode::toString() const {
00331         return "ArrayDecl: " + name + " of " + arrayType->toString();
00332     }
00333
00334     void ArrayAccessNode::accept(ASTVisitor &visitor) {
00335         visitor.visit(*this);
00336     }
00337
00338     std::string ArrayAccessNode::toString() const {

```

```

00339         return "ArrayAccess: ";
00340     }
00341
00342     void ArrayAssignmentNode::accept(ASTVisitor &visitor) {
00343         visitor.visit(*this);
00344     }
00345
00346     std::string ArrayAssignmentNode::toString() const {
00347         return "ArrayAssignment: " + arrayName + "[" + index->toString() + "]" := " +
value->toString();
00348     }
00349
00350     RecordTypeNode::RecordTypeNode(std::vector<std::unique_ptr<ASTNode>> fields)
00351         : fields(std::move(fields)) {}
00352
00353     void RecordTypeNode::accept(ASTVisitor &visitor) { visitor.visit(*this); }
00354
00355     std::string RecordTypeNode::toString() const { return "RecordType"; }
00356
00357     RecordDeclarationNode::RecordDeclarationNode(const std::string &name,
std::unique_ptr<RecordTypeNode> recordType)
00358         : name(name), recordType(std::move(recordType)) {}
00359
00360     void RecordDeclarationNode::accept(ASTVisitor &visitor) { visitor.visit(*this); }
00361
00362     std::string RecordDeclarationNode::toString() const { return "RecordDeclaration: " + name; }
00363
00364     FieldAccessNode::FieldAccessNode(std::unique_ptr<ASTNode> recordExpr, const std::string
&fieldName)
00365         : recordExpr(std::move(recordExpr)), fieldName(fieldName) {}
00366
00367     void FieldAccessNode::accept(ASTVisitor &visitor) { visitor.visit(*this); }
00368
00369     std::string FieldAccessNode::toString() const { return "FieldAccess: " + fieldName; }
00370
00371     void TypeDeclNode::accept(ASTVisitor &visitor) {
00372         visitor.visit(*this);
00373     }
00374
00375     void TypeAliasNode::accept(ASTVisitor &visitor) {
00376         visitor.visit(*this);
00377     }
00378
00379     std::string TypeDeclNode::toString() const {
00380         std::string result = "TypeDeclNode(";
00381         for (const auto &decl : typeDeclarations) {
00382             result += decl->toString() + ", ";
00383         }
00384         if (!typeDeclarations.empty())
00385             result.pop_back(), result.pop_back();
00386         result += ")";
00387         return result;
00388     }
00389
00390     std::string TypeAliasNode::toString() const {
00391         return "TypeAliasNode(" + typeName + " = " + baseType + ")";
00392     }
00393
00394     void ArrayTypeDeclarationNode::accept(ASTVisitor &visitor) {
00395         visitor.visit(*this);
00396     }
00397
00398     std::string ArrayTypeDeclarationNode::toString() const {
00399         return name;
00400     }
00401
00402     void ExitNode::accept(ASTVisitor &visitor) {
00403         visitor.visit(*this);
00404     }
00405
00406     void SimpleTypeNode::accept(ASTVisitor &visitor) {
00407         visitor.visit(*this);
00408     }
00409
00410     void ContinueNode::accept(ASTVisitor &visitor) {
00411         visitor.visit(*this);
00412     }
00413
00414     std::string ContinueNode::toString() const {
00415         return "Continue Node";
00416     }
00417
00418     void BreakNode::accept(ASTVisitor &visitor) {
00419         visitor.visit(*this);
00420     }
00421
00422     std::string BreakNode::toString() const {

```

```

00423         return "Continue Node";
00424     }
00425
00426     void NilNode::accept(ASTVisitor &visitor) {
00427         visitor.visit(*this);
00428     }
00429
00430     void PointerTypeNode::accept(ASTVisitor &visitor) {
00431         visitor.visit(*this);
00432     }
00433
00434     void PointerDerefNode::accept(ASTVisitor &visitor) {
00435         visitor.visit(*this);
00436     }
00437
00438     void AddressOfNode::accept(ASTVisitor &visitor) {
00439         visitor.visit(*this);
00440     }
00441
00442     void WithStmtNode::accept(ASTVisitor &visitor) {
00443         visitor.visit(*this);
00444     }
00445
00446     void GotoStmtNode::accept(ASTVisitor &visitor) {
00447         visitor.visit(*this);
00448     }
00449
00450     void LabelStmtNode::accept(ASTVisitor &visitor) {
00451         visitor.visit(*this);
00452     }
00453
00454     void SetLiteralNode::accept(ASTVisitor &visitor) {
00455         visitor.visit(*this);
00456     }
00457
00458     void SetTypeNode::accept(ASTVisitor &visitor) {
00459         visitor.visit(*this);
00460     }
00461
00462     void EnumTypeDeclNode::accept(ASTVisitor &visitor) {
00463         visitor.visit(*this);
00464     }
00465
00466 } // namespace pascal

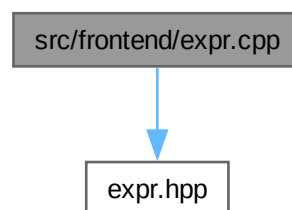
```

12.50 src/frontend/expr.cpp File Reference

Expression utility placeholder (reserved).

```
#include "expr.hpp"
```

Include dependency graph for expr.cpp:



Namespaces

- namespace `mxx`

12.50.1 Detailed Description

Expression utility placeholder (reserved).

Author

Jared Bruni

Definition in file [expr.cpp](#).

12.51 expr.cpp

[Go to the documentation of this file.](#)

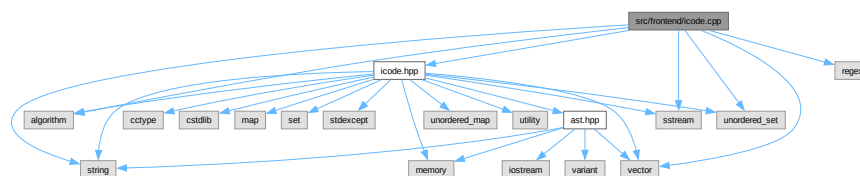
```
00001
00006 #include "expr.hpp"
00007
00008 namespace mxx {
00009
00010 }
```

12.52 src/frontend/icode.cpp File Reference

Pascal-to-MXVM code generator — CodeGenVisitor visit methods and assembly output.

```
#include "icode.hpp"
#include <algorithm>
#include <regex>
#include <sstream>
#include <string>
#include <unordered_set>
#include <vector>
```

Include dependency graph for icode.cpp:



Namespaces

- namespace [pascal](#)

Functions

- static std::string [pascal::trim_comment](#) (const std::string &line)
- static std::string [pascal::trim](#) (const std::string &s)
- static void [pascal::locate_sections](#) (const std::vector< std::string > &lines, size_t &dataStart, size_t &dataEnd, size_t &codeStart, size_t &codeEnd)
- static std::unordered_set< std::string > [pascal::collect_used](#) (const std::vector< std::string > &lines, size_t codeStart, size_t codeEnd)
- static void [pascal::sweep_unused_data](#) (std::vector< std::string > &lines)
- static std::vector< std::pair< size_t, std::string > > [pascal::nextNonEmpty](#) (const std::vector< std::string > &code, size_t start, size_t n)
- static std::string [pascal::invertJump](#) (const std::string &j)
- static bool [pascal::parse2](#) (const std::string &line, const std::string &op, std::string &a, std::string &b)
- static bool [pascal::parse1](#) (const std::string &line, const std::string &op, std::string &a)

- static bool `pascal::parseJump` (const std::string &line, std::string &jop, std::string &label)
 - static bool `pascal::isLabelDef` (const std::string &line, const std::string &label)
 - static void `pascal::foldCmpTest` (std::vector< std::string > &code)
 - static void `pascal::foldAndTest` (std::vector< std::string > &code)
 - static void `pascal::foldOrTest` (std::vector< std::string > &code)
 - static bool `pascal::isRegisterName` (const std::string &s)
- Forward declaration — checks whether a name is a hardware register.*
- static void `pascal::copyPropagation` (std::vector< std::string > &code)
- Copy-propagation peephole pass.*
- static void `pascal::foldMovArithMov` (std::vector< std::string > &code)
 - static void `pascal::peepholeOpt` (std::vector< std::string > &code)
 - std::string `pascal::mxvmOpt` (const std::string &text)

12.52.1 Detailed Description

Pascal-to-MXVM code generator — CodeGenVisitor visit methods and assembly output.

Author

Jared Bruni

Definition in file `icode.cpp`.

12.53 icode.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "icode.hpp"
00007 #include <algorithm>
00008 #include <regex>
00009 #include <sstream>
00010 #include <string>
00011 #include <unordered_set>
00012 #include <vector>
00013
00014 namespace pascal {
00015     static inline std::string rtrim_comment(const std::string &line) {
00016         auto h = line.find('#');
00017         auto s = line.find("//");
00018         size_t cut = std::min(h == std::string::npos ? line.size() : h,
00019                               s == std::string::npos ? line.size() : s);
00020         std::string out = line.substr(0, cut);
00021         while (!out.empty() && (unsigned char)out.back() <= ' ')
00022             out.pop_back();
00023         return out;
00024     }
00025     static inline std::string trim(const std::string &s) {
00026         size_t a = 0, b = s.size();
00027         while (a < b && (unsigned char)s[a] <= ' ')
00028             ++a;
00029         while (b > a && (unsigned char)s[b - 1] <= ' ')
00030             --b;
00031         return s.substr(a, b - a);
00032     }
00033     static void locate_sections(const std::vector<std::string> &lines,
00034                                size_t &dataStart, size_t &dataEnd,
00035                                size_t &codeStart, size_t &codeEnd) {
00036         dataStart = dataEnd = codeStart = codeEnd = 0;
00037         for (size_t i = 0; i < lines.size(); ++i) {
00038             if (!dataStart && lines[i].find("section data {") != std::string::npos) {
00039                 dataStart = i + 1;
00040                 for (size_t j = dataStart; j < lines.size(); ++j) {
00041                     if (lines[j].find('}') != std::string::npos) {
00042                         dataEnd = j - 1;
00043                         break;
00044                     }
00045                 }
00046             }
00047             if (!codeStart && lines[i].find("section code {") != std::string::npos) {
00048                 codeStart = i + 1;
00049                 for (size_t j = codeStart; j < lines.size(); ++j) {
00050                     if (lines[j].find('}') != std::string::npos) {
00051                         codeEnd = j - 1;

```

```

00052             break;
00053         }
00054     }
00055 }
00056 if (dataStart && dataEnd && codeStart && codeEnd)
00057     break;
00058 }
00059 }
00060 static std::unordered_set<std::string> collect_used(const std::vector<std::string> &lines,
00061                                                    size_t codeStart, size_t codeEnd) {
00062     std::string code;
00063     for (size_t i = codeStart; i <= codeEnd && i < lines.size(); ++i) {
00064         code += lines[i];
00065         code.push_back('\n');
00066     }
00067     std::unordered_set<std::string> used = {"rax", "fmt_int", "fmt_str", "fmt_chr", "fmt_float",
"newline"};
00068     std::regex varUsagePattern("\\b([a-zA-Z_][a-zA-Z0-9_]*)\\b");
00069     for (auto it = std::sregex_iterator(code.begin(), code.end(), varUsagePattern);
00070          it != std::sregex_iterator(); ++it)
00071         used.insert((*it).str(1));
00072     return used;
00073 }
00074 static void sweep_unused_data(std::vector<std::string> &lines) {
00075     size_t dS = 0, dE = 0, cS = 0, cE = 0;
00076     locate_sections(lines, dS, dE, cS, cE);
00077     if (!dS || !cS)
00078         return;
00079     auto used = collect_used(lines, cS, cE);
00080     std::regex
decl("^\\s*(export)?\\s*(int|string|ptr|float|double)\\s+([A-Za-z_][A-Za-z0-9_]*)");
00081     std::vector<size_t> erase;
00082     for (size_t i = dS; i <= dE && i < lines.size(); ++i) {
00083         std::smatch m;
00084         if (std::regex_search(lines[i], m, decl)) {
00085             bool isExport = m[1].length() > 0;
00086             std::string name = m[3];
00087             if (!isExport && !used.count(name))
00088                 erase.push_back(i);
00089         }
00090     }
00091     std::sort(erase.begin(), erase.end(), std::greater<size_t>());
00092     for (auto idx : erase)
00093         if (idx < lines.size())
00094             lines.erase(lines.begin() + idx);
00095 }
00096 }
00097 static std::vector<std::pair<size_t, std::string> nextNonEmpty(
00098     const std::vector<std::string> &code, size_t start, size_t n) {
00099     std::vector<std::pair<size_t, std::string> out;
00100     for (size_t i = start; i < code.size() && out.size() < n; ++i) {
00101         std::string t = trim(rtrim_comment(code[i]));
00102         if (!t.empty())
00103             out.push_back({i, t});
00104     }
00105     return out;
00106 }
00107 }
00108 static std::string invertJump(const std::string &j) {
00109     if (j == "jg")
00110         return "jle";
00111     else if (j == "jle")
00112         return "jg";
00113     else if (j == "jl")
00114         return "jge";
00115     else if (j == "jge")
00116         return "jl";
00117     else if (j == "je")
00118         return "jne";
00119     else if (j == "jne")
00120         return "je";
00121     else if (j == "ja")
00122         return "jbe";
00123     else if (j == "jbe")
00124         return "ja";
00125     else if (j == "jb")
00126         return "jae";
00127     else if (j == "jae")
00128         return "jb";
00129     return "";
00130 }
00131 }
00132 static bool parse2(const std::string &line, const std::string &op,
00133                  std::string &a, std::string &b) {
00134     if (line.size() <= op.size() || line.compare(0, op.size(), op) != 0 ||
00135         line[op.size()] != ' ')
00136         return false;

```

```

00137         std::string rest = line.substr(op.size() + 1);
00138         auto c = rest.find(',');
00139         if (c == std::string::npos)
00140             return false;
00141         a = trim(rest.substr(0, c));
00142         b = trim(rest.substr(c + 1));
00143         return !a.empty() && !b.empty();
00144     }
00145     static bool parse1(const std::string &line, const std::string &op, std::string &a) {
00146         if (line.size() <= op.size() || line.compare(0, op.size(), op) != 0 ||
00147             line[op.size()] != ' ')
00148             return false;
00149         a = trim(line.substr(op.size() + 1));
00150         return !a.empty();
00151     }
00152     static bool parseJump(const std::string &line, std::string &jop, std::string &label) {
00153         if (line.size() < 3 || line[0] != 'j')
00154             return false;
00155         auto sp = line.find(' ');
00156         if (sp == std::string::npos)
00157             return false;
00158         jop = line.substr(0, sp);
00159         label = trim(line.substr(sp + 1));
00160         return !label.empty() && jop != "jmp";
00161     }
00162     static bool isLabelDef(const std::string &line, const std::string &label) {
00163         return line == label + ":";
00164     }
00165 }
00166 static void foldCmpTest(std::vector<std::string> &code) {
00167     bool changed = true;
00168     while (changed) {
00169         changed = false;
00170         for (size_t i = 0; i < code.size(); ++i) {
00171             auto mat = nextNonEmpty(code, i, 7);
00172             if (mat.size() < 7)
00173                 break;
00174
00175             std::string cmpA, cmpB;
00176             bool isFcmp = parse2(mat[0].second, "fcmp", cmpA, cmpB);
00177             if (!isFcmp && !parse2(mat[0].second, "cmp", cmpA, cmpB))
00178                 continue;
00179             std::string cmpOp = isFcmp ? "fcmp" : "cmp";
00180
00181             std::string jCC, trueLabel;
00182             if (!parseJump(mat[1].second, jCC, trueLabel))
00183                 continue;
00184
00185             std::string reg, zero;
00186             if (!parse2(mat[2].second, "mov", reg, zero) || zero != "0")
00187                 continue;
00188
00189             std::string endLabel;
00190             if (!parse1(mat[3].second, "jmp", endLabel))
00191                 continue;
00192
00193             if (!isLabelDef(mat[4].second, trueLabel))
00194                 continue;
00195
00196             std::string r5, one;
00197             if (!parse2(mat[5].second, "mov", r5, one) || r5 != reg || one != "1")
00198                 continue;
00199
00200             if (!isLabelDef(mat[6].second, endLabel))
00201                 continue;
00202
00203             auto rest = nextNonEmpty(code, mat[6].first + 1, 30);
00204             bool found = false;
00205             for (size_t j = 0; j + 1 < rest.size(); ++j) {
00206                 if (rest[j].second.back() == ':')
00207                     break;
00208                 std::string wdst, wsrc;
00209                 if (parse2(rest[j].second, "mov", wdst, wsrc) && wdst == reg)
00210                     break;
00211
00212                 std::string rt, zt;
00213                 if (parse2(rest[j].second, "cmp", rt, zt) && rt == reg && zt == "0") {
00214                     std::string jTest, target;
00215                     if (j + 1 < rest.size() && parseJump(rest[j + 1].second, jTest, target) &&
00216                         (jTest == "je" || jTest == "jne")) {
00217                         std::string newJ = (jTest == "je") ? invertJump(jCC) : jCC;
00218                         if (newJ.empty())
00219                             break;
00220
00221                         bool labelUsedElsewhere = false;
00222                         for (size_t k = 0; k < code.size(); ++k) {
00223                             if (k == mat[1].first || k == rest[j + 1].first)

```

```

00224         continue;
00225         std::string t = trim(rtrim_comment(code[k]));
00226         if (!t.empty() && t.back() != ':' && t.find(trueLabel) !=
std::string::npos) {
00227             labelUsedElsewhere = true;
00228             break;
00229         }
00230     }
00231     if (labelUsedElsewhere)
00232         break;
00233
00234     code[mat[0].first] = "\t\t" + cmpOp + " " + cmpA + ", " + cmpB;
00235     code[mat[1].first] = "\t\t" + newJ + " " + target;
00236     for (int k = 2; k <= 6; ++k)
00237         code[mat[k].first] = "";
00238     code[rest[j].first] = "";
00239     code[rest[j + 1].first] = "";
00240
00241     found = true;
00242     changed = true;
00243 }
00244 break;
00245 }
00246 }
00247 if (found)
00248     break;
00249 }
00250 }
00251 }
00252
00253 static void foldAndTest(std::vector<std::string> &code) {
00254     bool changed = true;
00255     while (changed) {
00256         changed = false;
00257         for (size_t i = 0; i < code.size(); ++i) {
00258             auto seq = nextNonEmpty(code, i, 11);
00259             if (seq.size() < 11)
00260                 break;
00261
00262             std::string r1, z0;
00263             if (!parse2(seq[0].second, "cmp", r1, z0) || z0 != "0")
00264                 continue;
00265             std::string j1, zeroLabel;
00266             if (!parse1(seq[1].second, "je", zeroLabel))
00267                 continue;
00268             std::string r2, z2;
00269             if (!parse2(seq[2].second, "cmp", r2, z2) || z2 != "0")
00270                 continue;
00271             std::string j3, z13;
00272             if (!parse1(seq[3].second, "je", z13) || z13 != zeroLabel)
00273                 continue;
00274             std::string r3, one;
00275             if (!parse2(seq[4].second, "mov", r3, one) || one != "1")
00276                 continue;
00277             std::string endLabel;
00278             if (!parse1(seq[5].second, "jmp", endLabel))
00279                 continue;
00280             if (!isLabelDef(seq[6].second, zeroLabel))
00281                 continue;
00282             std::string r7, z7;
00283             if (!parse2(seq[7].second, "mov", r7, z7) || r7 != r3 || z7 != "0")
00284                 continue;
00285             if (!isLabelDef(seq[8].second, endLabel))
00286                 continue;
00287             std::string r9, z9;
00288             if (!parse2(seq[9].second, "cmp", r9, z9) || r9 != r3 || z9 != "0")
00289                 continue;
00290             std::string target;
00291             if (!parse1(seq[10].second, "je", target))
00292                 continue;
00293
00294             code[seq[0].first] = "\t\tcmp " + r1 + ", 0";
00295             code[seq[1].first] = "\t\tje " + target;
00296             code[seq[2].first] = "\t\tcmp " + r2 + ", 0";
00297             code[seq[3].first] = "\t\tje " + target;
00298             for (int k = 4; k <= 10; ++k)
00299                 code[seq[k].first] = "";
00300             changed = true;
00301             break;
00302         }
00303     }
00304 }
00305
00306 static void foldOrTest(std::vector<std::string> &code) {
00307     bool changed = true;
00308     while (changed) {
00309         changed = false;

```



```

00310         for (size_t i = 0; i < code.size(); ++i) {
00311             auto seq = nextNonEmpty(code, i, 11);
00312             if (seq.size() < 11)
00313                 break;
00314
00315             std::string r1, z0;
00316             if (!parse2(seq[0].second, "cmp", r1, z0) || z0 != "0")
00317                 continue;
00318             std::string oneLabel;
00319             if (!parse1(seq[1].second, "jne", oneLabel))
00320                 continue;
00321             std::string r2, z2;
00322             if (!parse2(seq[2].second, "cmp", r2, z2) || z2 != "0")
00323                 continue;
00324             std::string o13;
00325             if (!parse1(seq[3].second, "jne", o13) || o13 != oneLabel)
00326                 continue;
00327             std::string r3, z4;
00328             if (!parse2(seq[4].second, "mov", r3, z4) || z4 != "0")
00329                 continue;
00330             std::string endLabel;
00331             if (!parse1(seq[5].second, "jmp", endLabel))
00332                 continue;
00333             if (!isLabelDef(seq[6].second, oneLabel))
00334                 continue;
00335             std::string r7, o7;
00336             if (!parse2(seq[7].second, "mov", r7, o7) || r7 != r3 || o7 != "1")
00337                 continue;
00338             if (!isLabelDef(seq[8].second, endLabel))
00339                 continue;
00340             std::string r9, z9;
00341             if (!parse2(seq[9].second, "cmp", r9, z9) || r9 != r3 || z9 != "0")
00342                 continue;
00343             std::string target;
00344             if (!parse1(seq[10].second, "je", target))
00345                 continue;
00346
00347             code[seq[0].first] = "\t\tcmp " + r1 + ", 0";
00348             code[seq[1].first] = "\t\tjne " + oneLabel;
00349             code[seq[2].first] = "\t\tcmp " + r2 + ", 0";
00350             code[seq[3].first] = "\t\tje " + target;
00351             code[seq[4].first] = "\t" + oneLabel + ":";
00352             for (int k = 5; k <= 10; ++k)
00353                 code[seq[k].first] = "";
00354             changed = true;
00355             break;
00356         }
00357     }
00358 }
00359
00360 static bool isRegisterName(const std::string &s);
00361
00376 static void copyPropagation(std::vector<std::string> &code) {
00377     static const std::unordered_set<std::string> twoOpInstructions = {
00378         "add", "sub", "mul", "div", "mod", "cmp", "fcmp", "mov", "and", "or", "xor"};
00379     bool changed = true;
00380     while (changed) {
00381         changed = false;
00382         for (size_t i = 0; i < code.size(); ++i) {
00383             std::string t0 = trim(rtrim_comment(code[i]));
00384             if (t0.empty())
00385                 continue;
00386
00387             std::string movDst, movSrc;
00388             if (!parse2(t0, "mov", movDst, movSrc))
00389                 continue;
00390             if (movDst == movSrc) {
00391                 code[i] = "";
00392                 changed = true;
00393                 break;
00394             }
00395
00396             size_t j = i + 1;
00397             while (j < code.size() && trim(rtrim_comment(code[j])).empty())
00398                 ++j;
00399             if (j >= code.size())
00400                 continue;
00401
00402             std::string t1 = trim(rtrim_comment(code[j]));
00403             if (!t1.empty() && t1.back() == ':')
00404                 continue;
00405
00406             auto sp = t1.find(' ');
00407             if (sp == std::string::npos)
00408                 continue;
00409             std::string op = t1.substr(0, sp);
00410

```

```

00411         if (twoOpInstructions.find(op) == twoOpInstructions.end())
00412             continue;
00413
00414         std::string arg1, arg2;
00415         if (!parse2(tl, op, arg1, arg2))
00416             continue;
00417         if (arg1 != movDst)
00418             continue;
00419         if (arg2 == movDst)
00420             continue;
00421
00422         bool usedLater = false;
00423         std::string pattern = "\\b" + movDst + "\\b";
00424         std::regex wordPat(pattern);
00425         for (size_t k = j + 1; k < code.size(); ++k) {
00426             std::string tk = rtrim_comment(code[k]);
00427             if (tk.empty())
00428                 continue;
00429             if (std::regex_search(tk, wordPat)) {
00430                 usedLater = true;
00431                 break;
00432             }
00433             if (!tk.empty() && tk.find("function ") != std::string::npos)
00434                 break;
00435             // A ret/done ends the current procedure/program.
00436             // Non-register stores before a return are observable
00437             // by the caller, so treat them as live.
00438             std::string ttk = trim(tk);
00439             if ((ttk == "ret" || ttk == "done") && !isRegisterName(movDst)) {
00440                 usedLater = true;
00441                 break;
00442             }
00443         }
00444
00445         if (usedLater)
00446             continue;
00447
00448         if (op == "cmp" || op == "fcmp") {
00449             code[j] = "\t\t" + op + " " + movSrc + ", " + arg2;
00450             code[i] = "";
00451             changed = true;
00452             break;
00453         }
00454
00455         if (op == "mov") {
00456             code[i] = "";
00457             changed = true;
00458             break;
00459         }
00460
00461         // Arithmetic ops require a variable as operand 1 (it's the destination).
00462         // Don't propagate constants into that position.
00463         if (!movSrc.empty() && (std::isdigit((unsigned char)movSrc[0]) || movSrc[0] == '-' ||
movSrc[0] == '"' || movSrc[0] == '\\'))
00464             continue;
00465
00466         bool srcUsedLater = false;
00467         std::string srcPattern = "\\b" + movSrc + "\\b";
00468         std::regex srcPat(srcPattern);
00469         for (size_t k = j + 1; k < code.size(); ++k) {
00470             std::string tk = rtrim_comment(code[k]);
00471             if (tk.empty())
00472                 continue;
00473             if (std::regex_search(tk, srcPat)) {
00474                 srcUsedLater = true;
00475                 break;
00476             }
00477             if (!tk.empty() && tk.find("function ") != std::string::npos)
00478                 break;
00479         }
00480
00481         if (!srcUsedLater) {
00482             code[j] = "\t\t" + op + " " + movSrc + ", " + arg2;
00483             code[i] = "";
00484             changed = true;
00485             break;
00486         }
00487     }
00488 }
00489 }
00490
00491 static bool isRegisterName(const std::string &s) {
00492     static const std::unordered_set<std::string> regs = {
00493         "rax", "rbx", "rcx", "rdx", "rsi", "rdi", "r8", "r9", "r10", "r11", "r12", "r13", "r14",
"r15"};
00494     if (regs.count(s))
00495         return true;

```

```

00496         if (s.size() >= 4 && s.compare(0, 3, "xmm") == 0)
00497             return true;
00498         return false;
00499     }
00500
00501     static void foldMovArithMov(std::vector<std::string> &code) {
00502         static const std::unordered_set<std::string> arithOps = {
00503             "add", "sub", "mul", "div", "mod", "and", "or", "xor";
00504         bool changed = true;
00505         while (changed) {
00506             changed = false;
00507             for (size_t i = 0; i + 2 < code.size(); ++i) {
00508                 std::string t0 = trim(rtrim_comment(code[i]));
00509                 if (t0.empty())
00510                     continue;
00511                 std::string mDst, mSrc;
00512                 if (!parse2(t0, "mov", mDst, mSrc))
00513                     continue;
00514                 if (!isRegisterName(mDst))
00515                     continue;
00516
00517                 size_t j = i + 1;
00518                 while (j < code.size() && trim(rtrim_comment(code[j])).empty())
00519                     ++j;
00520                 if (j >= code.size())
00521                     continue;
00522                 std::string t1 = trim(rtrim_comment(code[j]));
00523                 if (t1.empty() || t1.back() == ':')
00524                     continue;
00525                 auto sp = t1.find(' ');
00526                 if (sp == std::string::npos)
00527                     continue;
00528                 std::string op = t1.substr(0, sp);
00529                 if (arithOps.find(op) == arithOps.end())
00530                     continue;
00531                 std::string a1, a2;
00532                 if (!parse2(t1, op, a1, a2))
00533                     continue;
00534                 if (a1 != mDst)
00535                     continue;
00536                 if (a2 == mDst)
00537                     continue;
00538
00539                 size_t k = j + 1;
00540                 while (k < code.size() && trim(rtrim_comment(code[k])).empty())
00541                     ++k;
00542                 if (k >= code.size())
00543                     continue;
00544                 std::string t2 = trim(rtrim_comment(code[k]));
00545                 std::string m2Dst, m2Src;
00546                 if (!parse2(t2, "mov", m2Dst, m2Src))
00547                     continue;
00548                 if (m2Src != mDst)
00549                     continue;
00550                 if (isRegisterName(m2Dst))
00551                     continue;
00552
00553                 bool usedAfter = false;
00554                 std::string pattern = "\\b" + mDst + "\\b";
00555                 std::regex wordPat(pattern);
00556                 for (size_t w = k + 1; w < code.size(); ++w) {
00557                     std::string tw = rtrim_comment(code[w]);
00558                     if (tw.empty())
00559                         continue;
00560                     if (std::regex_search(tw, wordPat)) {
00561                         usedAfter = true;
00562                         break;
00563                     }
00564                     if (tw.find("function ") != std::string::npos)
00565                         break;
00566                 }
00567                 if (usedAfter)
00568                     continue;
00569
00570                 code[i] = "\t\tmov " + m2Dst + ", " + mSrc;
00571                 code[j] = "\t\t" + op + " " + m2Dst + ", " + a2;
00572                 code[k] = "";
00573                 changed = true;
00574                 break;
00575             }
00576         }
00577     }
00578
00579     static void peepholeOpt(std::vector<std::string> &code) {
00580         copyPropagation(code);
00581         foldMovArithMov(code);
00582         foldAndTest(code);

```

```

00583         foldOrTest(code);
00584         foldCmpTest(code);
00585         foldCmpTest(code);
00586         copyPropagation(code);
00587     }
00588
00589     std::string mxvmOpt(const std::string &text) {
00590         std::vector<std::string> lines;
00591         {
00592             std::istringstream is(text);
00593             std::string ln;
00594             while (std::getline(is, ln))
00595                 lines.push_back(ln);
00596         }
00597
00598         size_t dataStart = 0, dataEnd = 0, codeStart = 0, codeEnd = 0;
00599         locate_sections(lines, dataStart, dataEnd, codeStart, codeEnd);
00600         if (!codeStart) {
00601             std::string out;
00602             for (auto &ln : lines)
00603                 out += ln + "\n";
00604             return out;
00605         }
00606
00607         sweep_unused_data(lines);
00608         locate_sections(lines, dataStart, dataEnd, codeStart, codeEnd);
00609         if (!codeStart) {
00610             std::string out;
00611             for (auto &ln : lines)
00612                 out += ln + "\n";
00613             return out;
00614         }
00615
00616         size_t cStart = codeStart, cEnd = std::min(codeEnd, lines.size() ? lines.size() - 1 : 0);
00617         std::vector<std::string> code;
00618         code.reserve(cEnd - cStart + 1);
00619         for (size_t i = cStart; i <= cEnd; ++i)
00620             code.push_back(lines[i]);
00621
00622         std::vector<std::string> pass1;
00623         pass1.reserve(code.size());
00624         std::regex movPat("^(\\s*(\\s*)mov\\s+([^\s]+)\\s*,\\s*(\\s*(\\s*#;)+)\\s*(?:[;#].*)?$",
std::regex::icase);
00625
00626         for (size_t i = 0; i < code.size(); i) {
00627             std::string raw0 = code[i];
00628             std::string noCom0 = rtrim_comment(raw0);
00629             std::smatch m0;
00630             if (std::regex_match(noCom0, m0, movPat)) {
00631                 std::string indent = m0[1].str();
00632                 std::string dst0 = trim(m0[2].str());
00633                 std::string src0 = trim(m0[3].str());
00634                 if (dst0 == src0) {
00635                     ++i;
00636                     continue;
00637                 }
00638                 size_t k = i + 1;
00639                 while (k < code.size() && rtrim_comment(code[k]).empty())
00640                     ++k;
00641                 if (k < code.size()) {
00642                     std::string raw1 = code[k];
00643                     std::string noCom1 = rtrim_comment(raw1);
00644                     std::smatch m1;
00645                     if (std::regex_match(noCom1, m1, movPat)) {
00646                         std::string dst1 = trim(m1[2].str());
00647                         std::string src1 = trim(m1[3].str());
00648                         if (dst1 == src0 && src1 == dst0) {
00649                             pass1.push_back("\t\tmov " + dst0 + ", " + src0);
00650                             i = k + 1;
00651                             continue;
00652                         }
00653                     }
00654                 }
00655                 pass1.push_back(raw0);
00656                 ++i;
00657                 continue;
00658             }
00659             pass1.push_back(raw0);
00660             ++i;
00661         }
00662
00663         std::vector<std::string> pass2;
00664         pass2.reserve(pass1.size());
00665         std::regex movDstPat("^(\\s*(\\s*)mov\\s+([A-Za-z_][A-Za-z0-9_]*)\\s*,", std::regex::icase);
00666
00667         for (size_t i = 0; i < pass1.size(); ++i) {
00668             std::string raw = pass1[i];

```

```

00669         std::string noCom = rtrim_comment(raw);
00670         std::smatch mm;
00671         if (std::regex_match(noCom, mm, movDstPat)) {
00672             std::string dst = mm[1].str();
00673             bool usedLater = false;
00674             std::regex word("\\b" + dst + "\\b");
00675             for (size_t j = i + 1; j < pass1.size(); ++j) {
00676                 std::string nxt = rtrim_comment(pass1[j]);
00677                 if (!nxt.empty() && std::regex_search(nxt, word)) {
00678                     usedLater = true;
00679                     break;
00680                 }
00681                 // A ret/done ends the current procedure/program.
00682                 // Non-register stores before a return are observable
00683                 // by the caller, so treat them as live.
00684                 if (!nxt.empty()) {
00685                     std::string tnxt = trim(nxt);
00686                     if ((tnxt == "ret" || tnxt == "done") && !isRegisterName(dst)) {
00687                         usedLater = true;
00688                         break;
00689                     }
00690                 }
00691             }
00692             if (!usedLater)
00693                 continue;
00694         }
00695         pass2.push_back(raw);
00696     }
00697
00698     peepholeOpt(pass2);
00699
00700     std::vector<std::string> pass3;
00701     pass3.reserve(pass2.size());
00702     for (auto &ln : pass2)
00703         if (!ln.empty())
00704             pass3.push_back(ln);
00705
00706     std::vector<std::string> finalLines;
00707     finalLines.insert(finalLines.end(), lines.begin(), lines.begin() + cStart);
00708     finalLines.insert(finalLines.end(), pass3.begin(), pass3.end());
00709     if (cEnd + 1 < lines.size())
00710         finalLines.insert(finalLines.end(), lines.begin() + cEnd + 1, lines.end());
00711
00712     sweep_unused_data(finalLines);
00713
00714     std::string result;
00715     for (auto &ln : finalLines)
00716         result += ln + "\n";
00717     return result;
00718 }
00719 } // namespace pascal

```

12.54 src/icode.cpp File Reference

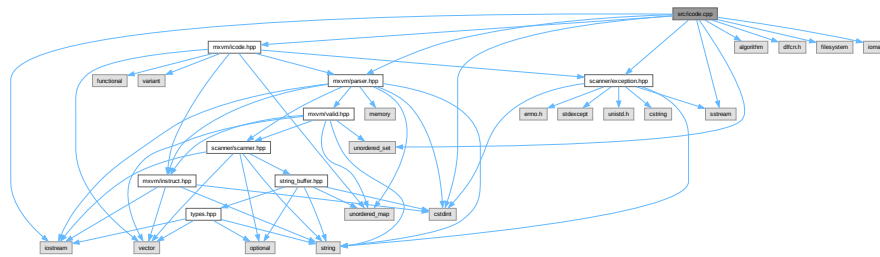
Base and Program class core methods — instruction storage, variable management, and standard library registration.

```

#include "mxvm/icode.hpp"
#include "mxvm/parser.hpp"
#include "scanner/exception.hpp"
#include <algorithm>
#include <cstdint>
#include <dlfcn.h>
#include <filesystem>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <unordered_set>

```

Include dependency graph for icode.cpp:



Namespaces

- namespace `mxvm`

Functions

- void `mxvm::except_assert` (std::string reason, bool value)
Assert a condition, throwing an Exception with the given reason on failure.

12.54.1 Detailed Description

Base and Program class core methods — instruction storage, variable management, and standard library registration.

Author

Jared Bruni

Definition in file `icode.cpp`.

12.55 icode.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "mxvm/icode.hpp"
00007 #include "mxvm/parser.hpp"
00008 #include "scanner/exception.hpp"
00009 #include <algorithm>
00010 #include <cstdint>
00011 #include <dlfcn.h>
00012 #include <filesystem>
00013 #include <iomanip>
00014 #include <iostream>
00015 #include <sstream>
00016 #include <unordered_set>
00017
00018 namespace mxvm {
00019
00020     std::unordered_map<std::string, Program *> Base::object_map;
00021
00022     std::string Program::getMangledName(const std::string &var) {
00023         auto dot_pos = var.find('.');
00024         if (dot_pos != std::string::npos) {
00025             std::string sym = var;
00026             std::replace(sym.begin(), sym.end(), '.', '_');
00027             if (platform == Platform::DARWIN && !sym.empty() && sym[0] != '_') {
00028                 if (var == "stdout" || var == "stdin" || var == "stderr") {
00029                     return var;
00030                 }
00031                 return sym;
00032             } else {
00033                 return sym;
00034             }
00035         }
00036
00037         if (!isVariable(var))
00038             return var;
00039     }
00040 }
  
```

```

00039     Variable &v = getVariable(var);
00040     if (!v.obj_name.empty()) {
00041         if (platform == Platform::DARWIN) {
00042             return "_" + v.obj_name + "_" + var;
00043         } else {
00044             return v.obj_name + "_" + var;
00045         }
00046     }
00047
00048     if (platform == Platform::DARWIN && !var.empty() && var[0] != '-') {
00049         return "_" + var;
00050     } else {
00051         return var;
00052     }
00053 }
00054
00055 std::string Program::getMangledName(const Operand &op) {
00056     if (op.op.find('.') == std::string::npos) {
00057         auto it = labels.find(op.op);
00058         if (it != labels.end()) {
00059             return name + "_" + op.op;
00060         }
00061     }
00062     if (!op.object.empty() && !op.label.empty()) {
00063         if (op.op == "stdout" || op.op == "stderr" || op.op == "stdin") {
00064             return op.op;
00065         }
00066         std::string base = op.object + "_" + op.label;
00067         return (platform == Platform::DARWIN) ? ("_" + base) : base;
00068     }
00069     return getMangledName(op.op);
00070 }
00071
00072 Base::Base(Base &&other) noexcept
00073 : name(std::move(other.name)), inc(std::move(other.inc)), vars(std::move(other.vars)),
  labels(std::move(other.labels)), external(std::move(other.external)),
  external_functions(std::move(other.external_functions)) {
00074 }
00075
00076 Base &Base::operator=(Base &&other) noexcept {
00077     if (this != &other) {
00078         name = std::move(other.name);
00079         inc = std::move(other.inc);
00080         vars = std::move(other.vars);
00081         labels = std::move(other.labels);
00082         external = std::move(other.external);
00083         external_functions = std::move(other.external_functions);
00084     }
00085     return *this;
00086 }
00087
00088 std::unordered_map<std::string, Variable> Base::allocated;
00089
00090 void Base::add_allocated(const std::string &name, Variable &v) {
00091     auto it = allocated.find(name);
00092     if (it == allocated.end())
00093         allocated[name] = v;
00094 }
00095
00096 Program::Program() : pc(0), running(false) {
00097
00098     add_extern("main", "strlen", true);
00099     add_extern("main", "printf", true);
00100     add_extern("main", "calloc", true);
00101     add_extern("main", "free", true);
00102     add_extern("main", "exit", true);
00103     add_extern("main", "atol", true);
00104     add_extern("main", "atof", true);
00105 }
00106
00107 void Program::add_standard() {
00108     Variable vstdout, vstdin, vstderr;
00109 #ifndef __APPLE__
00110     vstdout.setExtern("stdout", stdout);
00111     vstdin.setExtern("stdin", stdin);
00112     vstderr.setExtern("stderr", stderr);
00113     add_variable(name + "." + std::string("stdout"), vstdout);
00114     add_variable(name + "." + std::string("stdin"), vstdin);
00115     add_variable(name + "." + std::string("stderr"), vstderr);
00116 #else
00117
00118     vstdout.setExtern("stdout", stdout);
00119     vstdin.setExtern("stdin", stdin);
00120     vstderr.setExtern("stderr", stderr);
00121     add_variable(name + "." + std::string("stdout"), vstdout);
00122     add_variable(name + "." + std::string("stdin"), vstdin);
00123     add_variable(name + "." + std::string("stderr"), vstderr);

```

```

00124
00125 #endif
00126     }
00127
00128 void Program::setArgs(const std::vector<std::string> &argv) {
00129     std::copy(argv.begin(), argv.end(), std::back_inserter(args));
00130 }
00131
00132 void Program::setObject(bool obj) {
00133     object = obj;
00134 }
00135
00136 void Base::add_object(const std::string &name, Program *prog) {
00137     if (base != nullptr) {
00138         auto it = base->object_map.find(name);
00139         if (it == base->object_map.end())
00140             base->object_map[name] = prog;
00141     }
00142 }
00143
00144 Program::~Program() {
00145     std::unordered_set<void *> freed_ptrs;
00146     for (auto &i : vars) {
00147         if (i.second.var_value.ptr_value != nullptr && i.second.var_value.owns) {
00148             if (freed_ptrs.find(i.second.var_value.ptr_value) == freed_ptrs.end()) {
00149                 freed_ptrs.insert(i.second.var_value.ptr_value);
00150                 free(i.second.var_value.ptr_value);
00151                 if (debug_mode) {
00152                     std::cerr << Col("MXVM: Warning ", mx::Color::RED) << "Possible Memory Leak,
Pointer: " << name << "." << i.first << "\n";
00153                 }
00154             }
00155             i.second.var_value.ptr_value = nullptr;
00156             i.second.var_value.owns = false;
00157         }
00158     }
00159
00160     for (auto &h : RuntimeFunction::handles) {
00161         if (h.second) {
00162             dlclose(h.second);
00163             h.second = nullptr;
00164             char *errf = dlerror();
00165             if (errf != nullptr) {
00166                 std::cerr << "Error releasing module: " << errf << "\n";
00167             } else {
00168                 if (mxvm::debug_mode) {
00169                     std::cout << "Released module: " << h.first << "\n";
00170                 }
00171             }
00172         }
00173     }
00174 }
00175 bool Program::isFunctionValid(const std::string &label) {
00176     auto dot_pos = label.find('.');
00177     if (dot_pos != std::string::npos) {
00178         std::string obj_name = label.substr(0, dot_pos);
00179         std::string func_name = label.substr(dot_pos + 1);
00180
00181         auto it = object_map.find(obj_name);
00182         if (it != object_map.end() && it->second != nullptr) {
00183             Program *obj_prog = it->second;
00184             auto label_it = obj_prog->labels.find(func_name);
00185             return label_it != obj_prog->labels.end() && label_it->second.second; // second==true
means function
00186         }
00187         return false;
00188     }
00189
00190     auto it = labels.find(label);
00191     if (it != labels.end() && it->second.second) {
00192         return true;
00193     }
00194
00195     return false;
00196 }
00197
00198 std::unordered_map<std::string, void *> RuntimeFunction::handles;
00199 std::string Base::root_name;
00200
00201 RuntimeFunction::RuntimeFunction(const std::string &mod, const std::string &name) {
00202     fname = name;
00203     handle = nullptr;
00204     if (handles.find(mod) == handles.end()) {
00205         handle = dlopen(mod.c_str(), RTLD_LAZY);
00206         handles[mod] = handle;
00207     } else {
00208         handle = handles[mod];

```



```

00209     }
00210     if (handle == nullptr) {
00211         throw mx::Exception("Error could not open module: " + mod + " try using --path to point to
module path");
00212     }
00213     func = (void *)dlsym(handle, name.c_str());
00214     if (func == nullptr) {
00215         char *dl_e = dlerror();
00216         throw mx::Exception("Error could not find symbol: " + name + " in: " + mod + " Error: " +
((dl_e != nullptr) ? dl_e : ""));
00217     }
00218     char *dl_err = dlerror();
00219     if (dl_err != nullptr) {
00220         throw mx::Exception("Error: " + std::string(dl_err));
00221     }
00222 }
00223
00224 void RuntimeFunction::call(Program *program, std::vector<Operand> &operands) {
00225     if (!func || !handle) {
00226         throw mx::Exception("RuntimeFunction: function: " + this->fname + " pointer is null: " +
this->mod_name);
00227     }
00228     using FuncType = void (*)(Program *program, std::vector<Operand> &);
00229     FuncType f = reinterpret_cast<FuncType>(func);
00230     if (f != nullptr)
00231         f(program, operands);
00232 }
00233
00234 Base *Base::base = nullptr;
00235 std::vector<std::string> Base::filenames;
00236
00237 void Base::add_instruction(const Instruction &i) {
00238     inc.push_back(i);
00239 }
00240
00241 void Base::add_label(const std::string &name, uint64_t address, bool f) {
00242     if (labels.find(name) == labels.end())
00243         labels[name] = std::make_pair(address, f);
00244 }
00245
00246 void Base::add_variable(const std::string &name, const Variable &v) {
00247     if (vars.find(name) == vars.end()) {
00248         vars[name] = v;
00249     } else {
00250         throw mx::Exception("Duplicate variable name: " + name);
00251     }
00252 }
00253
00254 void Base::add_global(const std::string &objname, const std::string &name, const Variable &v) {
00255 }
00256
00257 void Base::add_filename(const std::string &fname) {
00258     if (base != nullptr) {
00259         auto it = std::find(base->filenames.begin(), base->filenames.end(), fname);
00260         if (it == base->filenames.end()) {
00261             base->filenames.push_back(fname);
00262         }
00263     }
00264 }
00265
00266 void Base::add_extern(const std::string &mod, const std::string &name, bool module) {
00267     ExternalFunction f = {name, mod, module};
00268     auto it = std::find(external.begin(), external.end(), f);
00269     if (it == external.end())
00270         external.push_back({name, mod, module});
00271 }
00272
00273 void Base::add_runtime_extern(const std::string &mod_name, const std::string &mod, const
std::string &func_name, const std::string &name) {
00274     if (mod_name.empty() || mod.empty() || func_name.empty() || name.empty()) {
00275         throw mx::Exception("External function missing information.");
00276     }
00277
00278     if (base != nullptr) {
00279         if (base->external_functions.find(name) == base->external_functions.end()) {
00280             base->external_functions[name] = RuntimeFunction(mod, func_name);
00281             base->external_functions[name].mod_name = mod_name;
00282         }
00283     }
00284 }
00285
00286 Program *Program::getObjectByName(const std::string &searchName) {
00287     if (this->name == searchName)
00288         return this;
00289     for (auto &obj : objects) {
00290         Program *found = obj->getObjectByName(searchName);
00291         if (found)
00292             return found;

```

```

00292     }
00293     return nullptr;
00294 }
00295
00296 std::string Program::escapeNewLines(const std::string &input) {
00297     std::string result;
00298     for (char c : input) {
00299         switch (c) {
00300             case '\\t':
00301                 result += "\\t";
00302                 break;
00303             case '\\n':
00304                 result += "\\n";
00305                 break;
00306             case '\\r':
00307                 result += "\\r";
00308                 break;
00309             default:
00310                 result += c;
00311                 break;
00312         }
00313     }
00314     return result;
00315 }
00316 void Program::memoryDump(std::ostream &out) {
00317     out << "Current working directory: ";
00318     try {
00319         out << std::filesystem::current_path().string() << "\n";
00320     } catch (const std::exception &e) {
00321         out << "(unable to retrieve: " << e.what() << ")\n";
00322     }
00323
00324     std::function<void(Program *, std::ostream &, int)> dumpProgram;
00325     dumpProgram = [&](Program *prog, std::ostream &out, int depth = 0) {
00326         std::string indent(depth * 2, ' ');
00327         out << indent << "=== MEMORY DUMP for " << prog->name << " ===\n";
00328         if (!prog->labels.empty()) {
00329             out << indent << "  Labels:\n";
00330             out << indent << "      " << std::left << std::setw(20) << "Name"
00331                 << std::setw(15) << "Address"
00332                 << std::setw(12) << "IsFunc"
00333                 << "\n";
00334             out << indent << "      " << std::string(42, '-') << "\n";
00335             for (const auto &label : prog->labels) {
00336                 out << indent << "      " << std::setw(20) << label.first
00337                     << std::setw(15) << label.second.first
00338                     << std::setw(10) << (label.second.second ? "yes" : "no")
00339                     << "\n";
00340             }
00341         }
00342         if (prog->vars.empty()) {
00343             out << indent << "  (no variables)\n";
00344         } else {
00345             out << indent << std::left << std::setw(25) << "Name"
00346                 << std::setw(18) << "Type"
00347                 << std::setw(30) << "Value"
00348                 << std::setw(18) << "PtrSize"
00349                 << std::setw(18) << "PtrCount"
00350                 << std::setw(12) << "Owns"
00351                 << "\n";
00352             out << indent << std::string(121, '-') << "\n";
00353             for (const auto &var : prog->vars) {
00354                 out << indent << std::setw(25) << var.first;
00355                 std::string typeStr;
00356                 switch (var.second.type) {
00357                     case VarType::VAR_INTEGER:
00358                         typeStr = "int";
00359                         break;
00360                     case VarType::VAR_FLOAT:
00361                         typeStr = "float";
00362                         break;
00363                     case VarType::VAR_STRING:
00364                         typeStr = "string";
00365                         break;
00366                     case VarType::VAR_POINTER:
00367                         typeStr = "ptr";
00368                         break;
00369                     case VarType::VAR_LABEL:
00370                         typeStr = "label";
00371                         break;
00372                     case VarType::VAR_EXTERN:
00373                         typeStr = "external";
00374                         break;
00375                     case VarType::VAR_ARRAY:
00376                         typeStr = "array";
00377                         break;
00378                     case VarType::VAR_BYTE:

```

```

00379         typeStr = "byte";
00380         break;
00381     default:
00382         typeStr = "unknown";
00383         break;
00384     }
00385     out << std::setw(18) << typeStr;
00386     switch (var.second.type) {
00387     case VarType::VAR_INTEGER:
00388     case VarType::VAR_BYTE:
00389         out << std::setw(30) << var.second.var_value.int_value;
00390         break;
00391     case VarType::VAR_FLOAT:
00392         out << std::setw(30) << std::fixed << std::setprecision(6)
00393             << var.second.var_value.float_value;
00394         break;
00395     case VarType::VAR_STRING:
00396         out << std::setw(30) << ("\" +
Program::escapeNewLines(var.second.var_value.str_value) + "\"");
00397         break;
00398     case VarType::VAR_POINTER:
00399     case VarType::VAR_EXTERN:
00400         if (var.second.var_value.ptr_value == nullptr)
00401             out << std::setw(30) << "null";
00402         else {
00403             std::ostringstream ptr_oss;
00404             ptr_oss << var.second.var_value.ptr_value;
00405             out << std::setw(30) << ptr_oss.str() << std::dec;
00406         }
00407         break;
00408     case VarType::VAR_LABEL:
00409         out << std::setw(30) << var.second.var_value.label_value;
00410         break;
00411     default:
00412         out << std::setw(30) << var.second.var_value.int_value;
00413         break;
00414     }
00415     if (var.second.type == VarType::VAR_POINTER) {
00416         out << std::setw(18) << var.second.var_value.ptr_size
00417             << std::setw(18) << var.second.var_value.ptr_count
00418             << std::setw(12) << (var.second.var_value.owns ? "yes" : "no");
00419     } else {
00420         out << std::setw(18) << "-"
00421             << std::setw(18) << "-"
00422             << std::setw(12) << "-";
00423     }
00424     out << "\n";
00425 }
00426 }
00427 out << "\n";
00428 for (const auto &obj : prog->objects) {
00429     if (obj)
00430         dumpProgram(obj.get(), out, depth + 1);
00431 }
00432 };
00433 dumpProgram(this, out, 0);
00434 }
00435
00436 void except_assert(std::string reason, bool value) {
00437     if (value == false) {
00438         throw mx::Exception(reason);
00439     }
00440 }
00441 } // namespace mxvm

```

12.56 src/frontend/icode_gen.cpp File Reference

12.57 icode_gen.cpp

[Go to the documentation of this file.](#)

00001

12.58 src/icode_gen.cpp File Reference

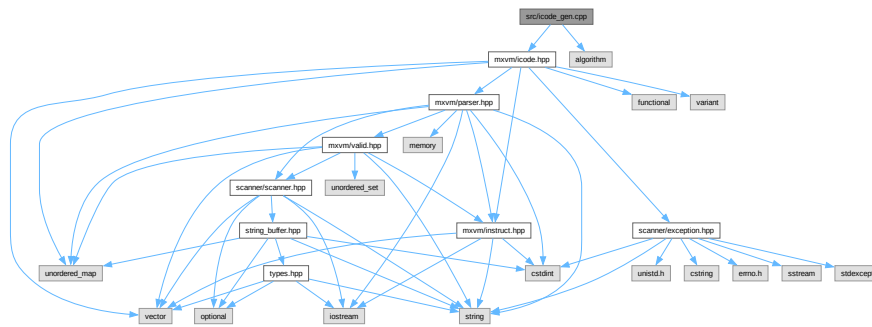
x86-64 System V ABI native code generator for Linux/macOS

```

#include "mxvm/icode.hpp"
#include <algorithm>

```

Include dependency graph for `icode_gen.cpp`:



Namespaces

- namespace `mxvm`

Variables

- static int `mxvm::error_label_count` = 0

12.58.1 Detailed Description

x86-64 System V ABI native code generator for Linux/macOS

Author

Jared Bruni

Definition in file [icode_gen.cpp](#).

12.59 icode_gen.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "mxvm/icode.hpp"
00007
00008 #include <algorithm>
00009
00010 namespace mxvm {
00011
00012     static int error_label_count = 0;
00013
00014     std::string Program::getPlatformSymbolName(const std::string &name) {
00015         if (platform == Platform::DARWIN) {
00016             if (!name.empty() && name[0] != '_' )
00017                 return "_" + name;
00018             else
00019                 return name;
00020         }
00021         return name;
00022     }
00023
00024     void Program::sysv_analyzeRegAlloc(bool uses_std_module) {
00025         sysv_reg_vars.clear();
00026         sysv_reg_save_order.clear();
00027
00028         // %rbx is reserved for stack alignment scratch
00029         // %rsi, %rdi are caller-saved on System V (not available)
00030         std::vector<std::string> available_regs = {"%r14", "%r15"};
00031         if (!uses_std_module) {
00032             available_regs.push_back("%r12");
00033             available_regs.push_back("%r13");
00034         }
00035
00036         std::unordered_map<std::string, int> usage_count;
00037         auto countOp = [&](const Operand &op) {

```

```

00038         if (!op.op.empty() && isVariable(op.op)) {
00039             Variable &v = getVariable(op.op);
00040             if (v.type == VarType::VAR_INTEGER && !v.is_global) {
00041                 usage_count[op.op]++;
00042             }
00043         }
00044     };
00045
00046     for (const auto &instr : inc) {
00047         countOp(instr.op1);
00048         countOp(instr.op2);
00049         countOp(instr.op3);
00050         for (const auto &vop : instr.vop)
00051             countOp(vop);
00052     }
00053
00054     std::vector<std::pair<std::string, int> sorted_vars(usage_count.begin(), usage_count.end());
00055     std::sort(sorted_vars.begin(), sorted_vars.end(),
00056             [](const auto &a, const auto &b) { return a.second > b.second; });
00057
00058     size_t n = std::min(available_regs.size(), sorted_vars.size());
00059     for (size_t i = 0; i < n; ++i) {
00060         if (sorted_vars[i].second >= 2) {
00061             sysv_reg_vars[sorted_vars[i].first] = available_regs[i];
00062             sysv_reg_save_order.push_back(available_regs[i]);
00063         }
00064     }
00065 }
00066
00067 void Program::sysv_emitFlushRegs(std::ostream &out) {
00068     for (const auto &[var, reg] : sysv_reg_vars) {
00069         out << "\tmovq " << reg << ", " << getMangledName(var) << "(%rip)\n";
00070     }
00071 }
00072
00073 void Program::sysv_emitReloadRegs(std::ostream &out) {
00074     for (const auto &[var, reg] : sysv_reg_vars) {
00075         out << "\tmovq " << getMangledName(var) << "(%rip), " << reg << "\n";
00076     }
00077 }
00078
00079 void Program::sysv_emitSaveRegs(std::ostream &out) {
00080     for (const auto &reg : sysv_reg_save_order) {
00081         out << "\tpushq " << reg << "\n";
00082     }
00083 }
00084
00085 void Program::sysv_emitRestoreRegs(std::ostream &out) {
00086     for (auto it = sysv_reg_save_order.rbegin(); it != sysv_reg_save_order.rend(); ++it) {
00087         out << "\tpopq " << *it << "\n";
00088     }
00089 }
00090
00091 void Program::sysv_emitStoreVar(std::ostream &out, const std::string &srcReg, const Operand &dest)
00092 {
00093     auto it = sysv_reg_vars.find(dest.op);
00094     if (it != sysv_reg_vars.end()) {
00095         if (srcReg != it->second)
00096             out << "\tmovq " << srcReg << ", " << it->second << "\n";
00097     } else {
00098         out << "\tmovq " << srcReg << ", " << getMangledName(dest) << "(%rip)\n";
00099     }
00100 }
00101
00102 void Program::sysv_emitStoreVarImm(std::ostream &out, const std::string &imm, const Operand &dest)
00103 {
00104     auto it = sysv_reg_vars.find(dest.op);
00105     if (it != sysv_reg_vars.end()) {
00106         out << "\tmovq $" << imm << ", " << it->second << "\n";
00107     } else {
00108         out << "\tmovq $" << imm << ", " << getMangledName(dest) << "(%rip)\n";
00109     }
00110 }
00111
00112 void Program::sysv_emitLoadVar(std::ostream &out, const std::string &dstReg, const Operand &src) {
00113     auto it = sysv_reg_vars.find(src.op);
00114     if (it != sysv_reg_vars.end()) {
00115         if (dstReg != it->second)
00116             out << "\tmovq " << it->second << ", " << dstReg << "\n";
00117     } else {
00118         out << "\tmovq " << getMangledName(src) << "(%rip), " << dstReg << "\n";
00119     }
00120 }
00121
00122 void Program::generateCode(const Platform &platform, bool obj, std::ostream &out) {
00123     if (platform == Platform::WINX64) {

```

```

00123         x64_generateCode(platform, obj, out);
00124         return;
00125     }
00126
00127     this->add_standard();
00128     std::unordered_map<int, std::string> labels_;
00129     for (auto &l : labels) {
00130         labels_[l.second.first] = l.first;
00131     }
00132
00133     bool uses_std_module = false;
00134     for (const auto &e : external) {
00135         if (e.mod == "std") {
00136             uses_std_module = true;
00137             break;
00138         }
00139     }
00140
00141     if (platform == Platform::DARWIN)
00142         out << ".section __DATA, __data\n";
00143     else
00144         out << ".section .data\n";
00145
00146     std::vector<std::string> var_names;
00147     for (auto &v : vars) {
00148         var_names.push_back(v.first);
00149     }
00150     std::sort(var_names.begin(), var_names.end());
00151
00152     for (auto &v : var_names) {
00153         const std::string var_out_name = getPlatformSymbolName(getMangledName(v));
00154         auto varx = getVariable(v);
00155         if (varx.type == VarType::VAR_INTEGER) {
00156             out << "\t" << var_out_name << ": .quad " << vars[v].var_value.int_value << "\n";
00157             continue;
00158         }
00159         if (varx.type == VarType::VAR_FLOAT) {
00160             out << "\t" << var_out_name << ": .double " << vars[v].var_value.float_value << "\n";
00161             continue;
00162         }
00163         if (varx.type == VarType::VAR_BYTE) {
00164             out << "\t" << var_out_name << ": .byte " << vars[v].var_value.int_value << "\n";
00165         }
00166     }
00167     for (auto &v : var_names) {
00168         auto varx = getVariable(v);
00169     }
00170
00171     if (platform == Platform::LINUX) {
00172         out << "\t.extern " << getPlatformSymbolName("stderr") << "\n";
00173         out << "\t.extern " << getPlatformSymbolName("stdin") << "\n";
00174         out << "\t.extern " << getPlatformSymbolName("stdout") << "\n";
00175     }
00176
00177     for (auto &v : var_names) {
00178         auto varx = getVariable(v);
00179         if (varx.type == VarType::VAR_STRING && varx.var_value.buffer_size == 0) {
00180             const std::string var_out_name = getPlatformSymbolName(getMangledName(v));
00181             out << "\t" << var_out_name << ": .asciz " << "\"" <<
escapeNewLines(vars[v].var_value.str_value) << "\"\n";
00182             continue;
00183         }
00184     }
00185
00186     if (base != nullptr) {
00187         for (auto &v : var_names) {
00188             auto varx = getVariable(v);
00189             auto t = varx.type;
00190             if (varx.is_global || object) {
00191                 switch (t) {
00192                     case VarType::VAR_BYTE:
00193                     case VarType::VAR_FLOAT:
00194                     case VarType::VAR_INTEGER:
00195                         out << "\t.global " << getMangledName(v) << "\n";
00196                         break;
00197                     default:
00198                         break;
00199                 }
00200             }
00201         }
00202     }
00203 }
00204
00205 if (platform == Platform::DARWIN)
00206     out << ".section __DATA, __bss\n";
00207 else
00208     out << ".section .bss\n";

```

```

00209
00210     for (auto &v : var_names) {
00211         const std::string var_out_name = getPlatformSymbolName(getMangledName(v));
00212         auto varx = getVariable(v);
00213         if (varx.type == VarType::VAR_POINTER) {
00214             out << "\t.comm " << var_out_name << ", 8\n";
00215         } else if (varx.type == VarType::VAR_STRING && varx.var_value.buffer_size > 0) {
00216             out << "\t.comm " << var_out_name << ", " << varx.var_value.buffer_size << "\n";
00217         }
00218     }
00219     if (object) {
00220         for (auto &v : var_names) {
00221             auto varx = getVariable(v);
00222             if (varx.type == VarType::VAR_STRING || varx.type == VarType::VAR_POINTER) {
00223                 out << "\t.global " << getMangledName(v) << "\n";
00224             }
00225         }
00226     }
00227     if (platform == Platform::DARWIN)
00228         out << ".section __TEXT, __text\n";
00229     else
00230         out << ".section .text\n";
00231
00232     for (auto &lbl : labels) {
00233         if (lbl.second.second == true) {
00234             out << "\t.global " << name + "_" + lbl.first << "\n";
00235         }
00236     }
00237     std::sort(external.begin(), external.end(), [](const ExternalFunction &a, const
ExternalFunction &b) {
00238         if (a.mod == b.mod)
00239             return a.name < b.name;
00240         return a.mod < b.mod;
00241     });
00242
00243     for (auto &e : external) {
00244         if (e.module == true)
00245             out << "\t.extern " << getPlatformSymbolName(e.name) << "\n";
00246         else
00247             out << "\t.extern " << getPlatformSymbolName(e.mod + "_" + e.name) << "\n";
00248     }
00249
00250     sysv_analyzeRegAlloc(uses_std_module);
00251
00252     if (!this->object) {
00253         out << "\t.p2align 4, 0x90\n";
00254         out << ".global " << "main" << "\n";
00255         out << "main" << ":\n";
00256         out << "\tpush %rbp\n";
00257         out << "\tmov %rsp, %rbp\n";
00258         out << "\tpush %rbx\n";
00259         if (uses_std_module) {
00260             out << "\tpush %r12\n";
00261             out << "\tpush %r13\n";
00262             out << "\tsub $8, %rsp\n";
00263             out << "\tmov %rdi, %r12\n";
00264             out << "\tmov %rsi, %r13\n";
00265             out << "\tmovq %rsp, %rbx\n";
00266             out << "\tandq $-16, %rsp\n";
00267             out << "\tcall " << getPlatformSymbolName("set_program_args") << "\n";
00268             out << "\tmovq %rbx, %rsp\n";
00269         } else {
00270             out << "\tsub $8, %rsp\n";
00271         }
00272         sysv_emitSaveRegs(out);
00273         sysv_emitReloadRegs(out);
00274     }
00275
00276     bool done_found = false;
00277
00278     for (size_t i = 0; i < inc.size(); ++i) {
00279         const Instruction &instr = inc[i];
00280         for (auto l : labels) {
00281             if (l.second.first == i && l.second.second) {
00282                 out << "\t.p2align 4, 0x90\n";
00283                 out << getPlatformSymbolName(name + "_" + l.first) << ":\n";
00284                 out << "\tpush %rbp\n";
00285                 out << "\tmov %rsp, %rbp\n";
00286                 out << "\tpush %rbx\n";
00287                 out << "\tsub $8, %rsp\n";
00288                 sysv_emitSaveRegs(out);
00289                 sysv_emitReloadRegs(out);
00290                 if (isVariable("rax") && getVariable("rax").type == VarType::VAR_INTEGER) {
00291                     Operand rax_op;
00292                     rax_op.op = "rax";
00293                     sysv_emitStoreVar(out, "%rax", rax_op);
00294                 }

```

```

00295         break;
00296     }
00297 }
00298 for (auto l : labels) {
00299     if (l.second.first == i && !l.second.second) {
00300         out << "." << l.first << ":\n";
00301     }
00302 }
00303 if (instr.instruction == DONE)
00304     done_found = true;
00305 generateInstruction(out, instr);
00306 }
00307
00308 #ifndef __EMSCRIPTEN__
00309 if (this->object == false && done_found == false)
00310     throw mx::Exception("Program missing done to signal completion.\n");
00311 #endif
00312 if (platform == Platform::LINUX)
00313     out << "\n\n.section .note.GNU-stack,\"\",@progbits\n\n";
00314
00315 std::string mainFunc = " Object";
00316 if (root_name == name)
00317     mainFunc = " Program";
00318 std::cout << Col("MXVM: Compiled: ", mx::Color::BRIGHT_BLUE) << name << ".s" << mainFunc << Col("
platform: ", mx::Color::BRIGHT_CYAN) << ((platform == Platform::LINUX) ? "Linux" : "macOS") << "\n";
00319 }
00320
00321 void Program::generateInstruction(std::ostream &out, const Instruction &i) {
00322     if (i.instruction < JMP || i.instruction > JNS) {
00323         last_cmp_type = CMP_NONE;
00324     }
00325     switch (i.instruction) {
00326     case ADD:
00327         gen_arth(out, "add", i);
00328         break;
00329     case SUB:
00330         gen_arth(out, "sub", i);
00331         break;
00332     case MUL:
00333         gen_arth(out, "mul", i);
00334         break;
00335     case PRINT:
00336         gen_print(out, i);
00337         break;
00338     case EXIT:
00339         gen_exit(out, i);
00340         break;
00341     case MOV:
00342         gen_mov(out, i);
00343         break;
00344     case JMP:
00345     case JE:
00346     case JNE:
00347     case JL:
00348     case JLE:
00349     case JG:
00350     case JGE:
00351     case JZ:
00352     case JNZ:
00353     case JA:
00354     case JB:
00355         gen_jump(out, i);
00356         break;
00357     case CMP:
00358         gen_cmp(out, i);
00359         break;
00360     case ALLOC:
00361         gen_alloc(out, i);
00362         break;
00363     case FREE:
00364         gen_free(out, i);
00365         break;
00366     case REALLOC:
00367         gen_realloc(out, i);
00368         break;
00369     case LOAD:
00370         gen_load(out, i);
00371         break;
00372     case STORE:
00373         gen_store(out, i);
00374         break;
00375     case RET:
00376         gen_ret(out, i);
00377         break;
00378     case CALL:
00379         gen_call(out, i);
00380         break;

```



```
00381         case DONE:
00382             gen_done(out, i);
00383             break;
00384         case AND:
00385             gen_bitop(out, "and", i);
00386             break;
00387         case OR:
00388             gen_bitop(out, "or", i);
00389             break;
00390         case XOR:
00391             gen_bitop(out, "xor", i);
00392             break;
00393         case NOT:
00394             gen_not(out, i);
00395             break;
00396         case DIV:
00397             gen_div(out, i);
00398             break;
00399         case MOD:
00400             gen_mod(out, i);
00401             break;
00402         case PUSH:
00403             gen_push(out, i);
00404             break;
00405         case POP:
00406             gen_pop(out, i);
00407             break;
00408         case STACK_LOAD:
00409             gen_stack_load(out, i);
00410             break;
00411         case STACK_STORE:
00412             gen_stack_store(out, i);
00413             break;
00414         case STACK_SUB:
00415             gen_stack_sub(out, i);
00416             break;
00417         case GETLINE:
00418             gen_getline(out, i);
00419             break;
00420         case TO_INT:
00421             gen_to_int(out, i);
00422             break;
00423         case TO_FLOAT:
00424             gen_to_float(out, i);
00425             break;
00426         case INVOKE:
00427             gen_invoke(out, i);
00428             break;
00429         case RETURN:
00430             gen_return(out, i);
00431             break;
00432         case NEG:
00433             gen_neg(out, i);
00434             break;
00435         case FCMP:
00436             gen_fcmp(out, i);
00437             break;
00438         case JAE:
00439             gen_jae(out, i);
00440             break;
00441         case JBE:
00442             gen_jbe(out, i);
00443             break;
00444         case JC:
00445             gen_jc(out, i);
00446             break;
00447         case JNC:
00448             gen_jnc(out, i);
00449             break;
00450         case JP:
00451             gen_jp(out, i);
00452             break;
00453         case JNP:
00454             gen_jnp(out, i);
00455             break;
00456         case JO:
00457             gen_jo(out, i);
00458             break;
00459         case JNO:
00460             gen_jno(out, i);
00461             break;
00462         case JS:
00463             gen_js(out, i);
00464             break;
00465         case JNS:
00466             gen_jns(out, i);
00467             break;
```

```

00468         case LEA:
00469             gen_lea(out, i);
00470             break;
00471         default:
00472             throw mx::Exception("Invalid or unsupported instruction: " +
std::to_string(static_cast<unsigned int>(i.instruction)));
00473     }
00474 }
00475 void Program::gen_done(std::ostream &out, const Instruction &i) {
00476     bool uses_std_module = false;
00477     for (const auto &e : external) {
00478         if (e.mod == "std") {
00479             uses_std_module = true;
00480             break;
00481         }
00482     }
00483
00484     if (uses_std_module && !this->object) {
00485         sysv_emitFlushRegs(out);
00486         out << "\tmovq %rsp, %rbx\n";
00487         out << "\tandq $-16, %rsp\n";
00488         out << "\tcall " << getPlatformSymbolName("free_program_args") << "\n";
00489         out << "\tmovq %rbx, %rsp\n";
00490     } else {
00491         sysv_emitFlushRegs(out);
00492     }
00493
00494     out << "\txor %eax, %eax\n";
00495     sysv_emitRestoreRegs(out);
00496     if (!this->object && uses_std_module) {
00497         out << "\tmovq -24(%rbp), %r13\n";
00498         out << "\tmovq -16(%rbp), %r12\n";
00499     }
00500     out << "\tmovq -8(%rbp), %rbx\n";
00501     out << "\tleave\n";
00502     out << "\tret\n";
00503 }
00504
00505 void Program::gen_ret(std::ostream &out, const Instruction &i) {
00506     sysv_emitFlushRegs(out);
00507     if (isVariable("rax") && getVariable("rax").type == VarType::VAR_INTEGER) {
00508         Operand rax_op;
00509         rax_op.op = "rax";
00510         sysv_emitLoadVar(out, "%rax", rax_op);
00511     }
00512     sysv_emitRestoreRegs(out);
00513     out << "\tmovq -8(%rbp), %rbx\n";
00514     out << "\tleave\n";
00515     out << "\tret\n";
00516 }
00517
00518 void Program::gen_return(std::ostream &out, const Instruction &i) {
00519     if (isVariable(i.op1.op)) {
00520         Variable &v = getVariable(i.op1.op);
00521         if (this->last_call_returns_owned_ptr) {
00522             v.var_value.owns = true;
00523         }
00524         switch (v.type) {
00525             case VarType::VAR_FLOAT:
00526                 out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
00527                 break;
00528             default:
00529                 sysv_emitStoreVar(out, "%rax", i.op1);
00530         }
00531     } else {
00532         throw mx::Exception("return: " + i.op1.op + " requires variable\n");
00533     }
00534 }
00535
00536
00537 void Program::gen_neg(std::ostream &out, const Instruction &i) {
00538     if (isVariable(i.op1.op)) {
00539         Variable &v = getVariable(i.op1.op);
00540         if (v.type == VarType::VAR_INTEGER) {
00541             generateLoadVar(out, v.type, "%rcx", i.op1);
00542             out << "\tnegq %rcx\n";
00543             sysv_emitStoreVar(out, "%rcx", i.op1);
00544         } else if (v.type == VarType::VAR_FLOAT) {
00545             generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
00546             out << "\txorpd %xmm1, %xmm1\n";
00547             out << "\tsubsd %xmm0, %xmm1\n";
00548             out << "\tmovsd %xmm1, " << getMangledName(i.op1) << "(%rip)\n";
00549         } else {
00550             throw mx::Exception("neg requires float or integer.");
00551         }
00552     } else {
00553         throw mx::Exception("neg requires variable");
00554     }
00555 }

```

```

00554     }
00555 }
00556
00557 void Program::gen_lea(std::ostream &out, const Instruction &i) {
00558     if (!isVariable(i.op1.op))
00559         throw mx::Exception("LEA destination must be a variable");
00560     if (!isVariable(i.op2.op))
00561         throw mx::Exception("LEA source must be a variable");
00562
00563     // Flush source register to memory before taking its address
00564     auto it = sysv_reg_vars.find(i.op2.op);
00565     if (it != sysv_reg_vars.end()) {
00566         out << "\tmovq " << it->second << ", " << getMangledName(i.op2) << "(%rip)\n";
00567     }
00568
00569     out << "\tleaq " << getMangledName(i.op2) << "(%rip), %rax\n";
00570     out << "\tmovq %rax, " << getMangledName(i.op1) << "(%rip)\n";
00571     getVariable(i.op1.op).var_value.owns = false;
00572 }
00573
00574 void Program::gen_invoke(std::ostream &out, const Instruction &i) {
00575     if (i.op1.op.empty()) {
00576         throw mx::Exception("invoke instruction requires operand of instruction name");
00577     }
00578     std::vector<Operand> op;
00579     op.push_back(i.op1);
00580     if (!i.op2.op.empty())
00581         op.push_back(i.op2);
00582     if (!i.op3.op.empty())
00583         op.push_back(i.op3);
00584     for (const auto &vx : i.vop) {
00585         if (!vx.op.empty())
00586             op.push_back(vx);
00587     }
00588     generateInvokeCall(out, op);
00589 }
00590
00591 void Program::generateInvokeCall(std::ostream &out, std::vector<Operand> &op) {
00592     if (op.empty() || op[0].op.empty()) {
00593         throw mx::Exception("invoke instruction requires operand of instruction name");
00594     }
00595     std::string name = op[0].op;
00596     std::vector<Operand> args;
00597     for (size_t i = 1; i < op.size(); ++i) {
00598         if (!op[i].op.empty())
00599             args.push_back(op[i]);
00600     }
00601
00602     if (isFunctionReturningOwnedPtr(name)) {
00603         this->last_call_returns_owned_ptr = true;
00604     } else {
00605         this->last_call_returns_owned_ptr = false;
00606     }
00607     generateFunctionCall(out, name, args);
00608 }
00609
00610 void Program::generateFunctionCall(std::ostream &out, const std::string &name,
std::vector<Operand> &op) {
00611     const std::vector<Operand> &args = op;
00612     xmm_offset = 0;
00613     size_t stack_args = (args.size() > 6) ? (args.size() - 6) : 0;
00614
00615     out << "\tmovq %rsp, %rbx\n";
00616     out << "\tandq $-16, %rsp\n";
00617
00618     if (stack_args) {
00619         size_t stack_bytes = stack_args * 8;
00620         size_t aligned = (stack_bytes + 15) & ~(size_t)15;
00621         out << "\tsubq $" << aligned << ", %rsp\n";
00622         for (size_t idx = 6, slot = 0; idx < args.size(); ++idx, ++slot) {
00623             generateLoadVar(out, VarType::VAR_INTEGER, "%rax", args[idx]);
00624             out << "\tmovq %rax, " << (slot * 8) << "(%rsp)\n";
00625         }
00626     }
00627
00628     int reg_count = 0;
00629     for (size_t z = 0; z < args.size() && reg_count < 6; ++z, ++reg_count) {
00630         generateLoadVar(out, reg_count, args[z]);
00631     }
00632
00633     out << "\txor %eax, %eax\n";
00634     out << "\tcall " << getPlatformSymbolName(name) << "\n";
00635     out << "\tmovq %rbx, %rsp\n";
00636 }
00637
00638 void Program::gen_call(std::ostream &out, const Instruction &i) {
00639     if (isFunctionValid(i.op1.op)) {

```

```

00640         sysv_emitFlushRegs(out);
00641     if (isVariable("rax") && getVariable("rax").type == VarType::VAR_INTEGER) {
00642         Operand rax_op;
00643         rax_op.op = "rax";
00644         sysv_emitLoadVar(out, "%rax", rax_op);
00645     }
00646     out << "\tmovq %rsp, %rbx\n";
00647     out << "\tandq $-16, %rsp\n";
00648     out << "\tcall " << getPlatformSymbolName(getMangledName(i.op1)) << "\n";
00649     out << "\tmovq %rbx, %rsp\n";
00650     if (isVariable("rax") && getVariable("rax").type == VarType::VAR_INTEGER) {
00651         Operand rax_op;
00652         rax_op.op = "rax";
00653         out << "\tmovq %rax, " << getMangledName(rax_op) << "(%rip)\n";
00654     }
00655     sysv_emitReloadRegs(out);
00656 } else {
00657     throw mx::Exception("Function " + i.op1.op + " not found!");
00658 }
00659 }
00660
00661 void Program::gen_alloc(std::ostream &out, const Instruction &i) {
00662     if (!isVariable(i.op1.op)) {
00663         throw mx::Exception("ALLOC destination must be a variable");
00664     }
00665     Variable &v = getVariable(i.op1.op);
00666     if (v.type != VarType::VAR_POINTER) {
00667         throw mx::Exception("ALLOC destination must be a pointer\n");
00668     }
00669
00670     if (!i.op2.op.empty()) {
00671         if (isVariable(i.op2.op)) {
00672             sysv_emitLoadVar(out, "%rsi", i.op2);
00673         } else {
00674             out << "\tmovq $" << i.op2.op << ", %rsi\n";
00675         }
00676     } else {
00677         out << "\tmovq $8, %rsi\n";
00678     }
00679
00680     if (!i.op3.op.empty()) {
00681         if (isVariable(i.op3.op)) {
00682             sysv_emitLoadVar(out, "%rdi", i.op3);
00683         } else if (i.op3.type == OperandType::OP_CONSTANT) {
00684             out << "\tmovq $" << i.op3.op << ", %rdi\n";
00685         } else {
00686             throw mx::Exception("ALLOC: invalid count operand " + i.op3.op);
00687         }
00688     } else {
00689         out << "\tmovq $1, %rdi\n";
00690     }
00691
00692     out << "\tmovq %rsp, %rbx\n";
00693     out << "\tandq $-16, %rsp\n";
00694     out << "\tcall " << getPlatformSymbolName("calloc") << "\n";
00695     out << "\tmovq %rbx, %rsp\n";
00696     out << "\ttest %rax, %rax\n";
00697     out << "\tjz .alloc_failed_" << error_label_count << "\n";
00698     out << "\tmovq %rax, " << getMangledName(i.op1) << "(%rip)\n";
00699     out << "\tjmp .alloc_done_" << error_label_count << "\n";
00700     out << ".alloc_failed_" << error_label_count << ":\n";
00701     out << "\tmovq $0, " << getMangledName(i.op1) << "(%rip)\n";
00702     out << ".alloc_done_" << error_label_count << ":\n";
00703     error_label_count++;
00704     v.var_value.owns = true;
00705 }
00706
00715 void Program::gen_realloc(std::ostream &out, const Instruction &i) {
00716     if (!isVariable(i.op1.op)) {
00717         throw mx::Exception("REALLOC destination must be a variable");
00718     }
00719     Variable &v = getVariable(i.op1.op);
00720     if (v.type != VarType::VAR_POINTER) {
00721         throw mx::Exception("REALLOC destination must be a pointer");
00722     }
00723
00724     sysv_emitFlushRegs(out);
00725
00726     // Compute new total size = count * elemSize into %rsi
00727     // %rdi = existing ptr (for realloc)
00728     out << "\tmovq " << getMangledName(i.op1) << "(%rip), %rdi\n";
00729
00730     if (!i.op3.op.empty()) {
00731         if (isVariable(i.op3.op)) {
00732             sysv_emitLoadVar(out, "%rax", i.op3);
00733         } else {
00734             out << "\tmovq $" << i.op3.op << ", %rax\n";

```

```

00735     }
00736   } else {
00737     out << "\tmovq $1, %rax\n";
00738   }
00739
00740   if (!i.op2.op.empty()) {
00741     if (isVariable(i.op2.op)) {
00742       sysv_emitLoadVar(out, "%rcx", i.op2);
00743     } else {
00744       out << "\tmovq $" << i.op2.op << ", %rcx\n";
00745     }
00746   } else {
00747     out << "\tmovq $8, %rcx\n";
00748   }
00749
00750   out << "\timulq %rcx, %rax\n";
00751   out << "\tmovq %rax, %rsi\n";
00752   out << "\tmovq %rsp, %rbx\n";
00753   out << "\tandq $-16, %rsp\n";
00754   out << "\tcall " << getPlatformSymbolName("realloc") << "\n";
00755   out << "\tmovq %rbx, %rsp\n";
00756   out << "\ttest %rax, %rax\n";
00757   out << "\tjz .realloc_failed_" << error_label_count << "\n";
00758   out << "\tmovq %rax, " << getMangledName(i.op1) << "(%rip)\n";
00759   out << "\tjmp .realloc_done_" << error_label_count << "\n";
00760   out << ".realloc_failed_" << error_label_count << ":\n";
00761   out << "\t# realloc failed, keep old pointer\n";
00762   out << ".realloc_done_" << error_label_count << ":\n";
00763   error_label_count++;
00764   v.var_value.owns = true;
00765   sysv_emitReloadRegs(out);
00766 }
00767
00768 void Program::gen_load(std::ostream &out, const Instruction &i) {
00769   if (!isVariable(i.op1.op)) {
00770     throw mx::Exception("LOAD destination: " + i.op1.op + " must be a variable");
00771   }
00772   Variable &dest = getVariable(i.op1.op);
00773
00774   if (!isVariable(i.op2.op)) {
00775     throw mx::Exception("LOAD source: " + i.op2.op + " must be a pointer variable");
00776   }
00777   Variable &ptrVar = getVariable(i.op2.op);
00778
00779   if (ptrVar.type == VarType::VAR_POINTER) {
00780     // Flush all cached registers so pointer reads see current values
00781     sysv_emitFlushRegs(out);
00782     out << "\tmovq " << getMangledName(i.op2) << "(%rip), %rax\n";
00783   } else if (ptrVar.type == VarType::VAR_STRING) {
00784     out << "\tleaq " << getMangledName(i.op2) << "(%rip), %rax\n";
00785   } else {
00786     throw mx::Exception("LOAD source must be a pointer or string buffer: " + i.op2.op);
00787   }
00788
00789   if (!i.op3.op.empty()) {
00790     if (isVariable(i.op3.op)) {
00791       sysv_emitLoadVar(out, "%rcx", i.op3);
00792     } else {
00793       out << "\tmovq $" << i.op3.op << ", %rcx\n";
00794     }
00795   } else {
00796     out << "\txorq %rcx, %rcx\n";
00797   }
00798
00799   if (!i.vop.empty() && !i.vop[0].op.empty()) {
00800     if (isVariable(i.vop[0].op)) {
00801       sysv_emitLoadVar(out, "%rdx", i.vop[0]);
00802       out << "\timulq %rdx, %rcx\n";
00803       out << "\taddq %rcx, %rax\n";
00804
00805       switch (dest.type) {
00806       case VarType::VAR_INTEGER:
00807       case VarType::VAR_POINTER:
00808       case VarType::VAR_EXTERN:
00809         out << "\tmovq (%rax), %rdx\n";
00810         sysv_emitStoreVar(out, "%rdx", i.op1);
00811         break;
00812       case VarType::VAR_FLOAT:
00813         out << "\tmovsd (%rax), %xmm0\n";
00814         out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
00815         break;
00816       case VarType::VAR_BYTE:
00817         out << "\tmovzbq (%rax), %rdx\n";
00818         out << "\tmovb %dl, " << getMangledName(i.op1) << "(%rip)\n";
00819         break;
00820       default:
00821         throw mx::Exception("LOAD: unsupported destination type");

```

```

00822     }
00823   } else {
00824     size_t stride = static_cast<size_t>(std::stoll(i.vop[0].op, nullptr, 0));
00825
00826     if (stride == 1 || stride == 2 || stride == 4 || stride == 8) {
00827       switch (dest.type) {
00828         case VarType::VAR_INTEGER:
00829         case VarType::VAR_POINTER:
00830         case VarType::VAR_EXTERN:
00831           out << "\tmovq (%rax,%rcx," << stride << "), %rdx\n";
00832           sysv_emitStoreVar(out, "%rdx", i.op1);
00833           break;
00834         case VarType::VAR_FLOAT:
00835           out << "\tmovsd (%rax,%rcx," << stride << "), %xmm0\n";
00836           out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "({%rip})\n";
00837           break;
00838         case VarType::VAR_BYTE:
00839           out << "\tmovzbq (%rax,%rcx," << stride << "), %rdx\n";
00840           out << "\tmovb %dl, " << getMangledName(i.op1) << "({%rip})\n";
00841           break;
00842         default:
00843           throw mx::Exception("LOAD: unsupported destination type");
00844       }
00845     } else {
00846       out << "\timulq $" << stride << ", %rcx\n";
00847       out << "\taddq %rcx, %rax\n";
00848
00849       switch (dest.type) {
00850         case VarType::VAR_INTEGER:
00851         case VarType::VAR_POINTER:
00852         case VarType::VAR_EXTERN:
00853           out << "\tmovq (%rax), %rdx\n";
00854           sysv_emitStoreVar(out, "%rdx", i.op1);
00855           break;
00856         case VarType::VAR_FLOAT:
00857           out << "\tmovsd (%rax), %xmm0\n";
00858           out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "({%rip})\n";
00859           break;
00860         case VarType::VAR_BYTE:
00861           out << "\tmovzbq (%rax), %rdx\n";
00862           out << "\tmovb %dl, " << getMangledName(i.op1) << "({%rip})\n";
00863           break;
00864         default:
00865           throw mx::Exception("LOAD: unsupported destination type");
00866       }
00867     }
00868   }
00869 } else {
00870   switch (dest.type) {
00871     case VarType::VAR_INTEGER:
00872     case VarType::VAR_POINTER:
00873     case VarType::VAR_EXTERN:
00874       out << "\tmovq (%rax,%rcx,8), %rdx\n";
00875       sysv_emitStoreVar(out, "%rdx", i.op1);
00876       break;
00877     case VarType::VAR_FLOAT:
00878       out << "\tmovsd (%rax,%rcx,8), %xmm0\n";
00879       out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "({%rip})\n";
00880       break;
00881     case VarType::VAR_BYTE:
00882       out << "\tmovzbq (%rax,%rcx,8), %rdx\n";
00883       out << "\tmovb %dl, " << getMangledName(i.op1) << "({%rip})\n";
00884       break;
00885     default:
00886       throw mx::Exception("LOAD: unsupported destination type");
00887   }
00888 }
00889 }
00890
00891 void Program::gen_store(std::ostream &out, const Instruction &i) {
00892   if (!isVariable(i.op2.op)) {
00893     throw mx::Exception("STORE destination must be a pointer variable: " + i.op2.op);
00894   }
00895   Variable &ptrVar = getVariable(i.op2.op);
00896
00897   if (ptrVar.type == VarType::VAR_POINTER) {
00898     // Flush all cached registers before pointer store
00899     sysv_emitFlushRegs(out);
00900     out << "\tmovq " << getMangledName(i.op2) << "({%rip}), %rax\n";
00901   } else if (ptrVar.type == VarType::VAR_STRING && ptrVar.var_value.buffer_size > 0) {
00902     out << "\tleaq " << getMangledName(i.op2) << "({%rip}), %rax\n";
00903   } else {
00904     throw mx::Exception("STORE must target pointer or string buffer");
00905   }
00906
00907   if (!i.op3.op.empty()) {
00908     if (isVariable(i.op3.op)) {

```

```

00909         sysv_emitLoadVar(out, "%rcx", i.op3);
00910     } else {
00911         out << "\tmovq $" << i.op3.op << ", %rcx\n";
00912     }
00913 } else {
00914     out << "\txorq %rcx, %rcx\n";
00915 }
00916
00917 if (isVariable(i.op1.op)) {
00918     Variable &src = getVariable(i.op1.op);
00919     switch (src.type) {
00920     case VarType::VAR_INTEGER:
00921     case VarType::VAR_POINTER:
00922     case VarType::VAR_EXTERN:
00923         sysv_emitLoadVar(out, "%rdx", i.op1);
00924         break;
00925     case VarType::VAR_STRING:
00926         out << "\tleaq " << getMangledName(i.op1) << "(%rip), %rdx\n";
00927         break;
00928     case VarType::VAR_BYTE:
00929         out << "\tmovzbq " << getMangledName(i.op1) << "(%rip), %rdx\n";
00930         break;
00931     case VarType::VAR_FLOAT:
00932         out << "\tmovsd " << getMangledName(i.op1) << "(%rip), %xmm0\n";
00933         break;
00934     default:
00935         throw mx::Exception("STORE: unsupported source type");
00936     }
00937 } else {
00938     out << "\tmovq $" << i.op1.op << ", %rdx\n";
00939 }
00940
00941 if (!i.vop.empty() && !i.vop[0].op.empty()) {
00942     if (isVariable(i.vop[0].op)) {
00943         sysv_emitLoadVar(out, "%r8", i.vop[0]);
00944         out << "\timulq %r8, %rcx\n";
00945         out << "\taddq %rcx, %rax\n";
00946
00947         if (isVariable(i.op1.op)) {
00948             Variable &src = getVariable(i.op1.op);
00949             if (src.type == VarType::VAR_FLOAT) {
00950                 out << "\tmovsd %xmm0, (%rax)\n";
00951             } else if (src.type == VarType::VAR_BYTE) {
00952                 out << "\tmovb %dl, (%rax)\n";
00953             } else {
00954                 out << "\tmovq %rdx, (%rax)\n";
00955             }
00956         } else {
00957             out << "\tmovq %rdx, (%rax)\n";
00958         }
00959     } else {
00960         size_t stride = static_cast<size_t>(std::stoll(i.vop[0].op, nullptr, 0));
00961
00962         if (stride == 1 || stride == 2 || stride == 4 || stride == 8) {
00963             if (isVariable(i.op1.op)) {
00964                 Variable &src = getVariable(i.op1.op);
00965                 if (src.type == VarType::VAR_FLOAT) {
00966                     out << "\tmovsd %xmm0, (%rax,%rcx," << stride << ")\n";
00967                 } else if (src.type == VarType::VAR_BYTE) {
00968                     out << "\tmovb %dl, (%rax,%rcx," << stride << ")\n";
00969                 } else {
00970                     out << "\tmovq %rdx, (%rax,%rcx," << stride << ")\n";
00971                 }
00972             } else {
00973                 out << "\tmovq %rdx, (%rax,%rcx," << stride << ")\n";
00974             }
00975         } else {
00976             out << "\timulq $" << stride << ", %rcx\n";
00977             out << "\taddq %rcx, %rax\n";
00978
00979             if (isVariable(i.op1.op)) {
00980                 Variable &src = getVariable(i.op1.op);
00981                 if (src.type == VarType::VAR_FLOAT) {
00982                     out << "\tmovsd %xmm0, (%rax)\n";
00983                 } else if (src.type == VarType::VAR_BYTE) {
00984                     out << "\tmovb %dl, (%rax)\n";
00985                 } else {
00986                     out << "\tmovq %rdx, (%rax)\n";
00987                 }
00988             } else {
00989                 out << "\tmovq %rdx, (%rax)\n";
00990             }
00991         }
00992     }
00993 } else {
00994     if (isVariable(i.op1.op)) {
00995         Variable &src = getVariable(i.op1.op);

```

```

00996         if (src.type == VarType::VAR_FLOAT) {
00997             out << "\tmovsd %xmm0, (%rax,%rcx,8)\n";
00998         } else if (src.type == VarType::VAR_BYTE) {
00999             out << "\tmovb %dl, (%rax,%rcx,8)\n";
01000         } else {
01001             out << "\tmovq %rdx, (%rax,%rcx,8)\n";
01002         }
01003     } else {
01004         out << "\tmovq %rdx, (%rax,%rcx,8)\n";
01005     }
01006 }
01007 // Reload cached registers after pointer store (memory may have changed)
01008 if (ptrVar.type == VarType::VAR_POINTER) {
01009     sysv_emitReloadRegs(out);
01010 }
01011 }
01012
01013 void Program::gen_free(std::ostream &out, const Instruction &i) {
01014     if (!isVariable(i.op1.op)) {
01015         throw mx::Exception("FREE argument must be a variable");
01016     }
01017     Variable &v = getVariable(i.op1.op);
01018     if (v.type != VarType::VAR_POINTER) {
01019         throw mx::Exception("FREE argument must be a pointer");
01020     }
01021
01022     out << "\tmovq " << getMangledName(i.op1) << "(%rip), %rdi\n";
01023     out << "\ttest %rdi, %rdi\n";
01024     out << "\tjz lf\n";
01025     out << "\tmovq %rsp, %rbx\n";
01026     out << "\tandq $-16, %rsp\n";
01027     out << "\tcall " << getPlatformSymbolName("free") << "\n";
01028     out << "\tmovq %rbx, %rsp\n";
01029     out << "\tmovq $0, " << getMangledName(i.op1) << "(%rip)\n";
01030     out << "l:\n";
01031     v.var_value.owns = false;
01032 }
01033
01034 void Program::gen_to_int(std::ostream &out, const Instruction &i) {
01035     if (!isVariable(i.op1.op)) {
01036         throw mx::Exception("to_int requires destination variable (dest int)");
01037     }
01038     if (i.op2.op.empty()) {
01039         throw mx::Exception("to_int requires source operand");
01040     }
01041
01042     Variable &dest = getVariable(i.op1.op);
01043     if (dest.type != VarType::VAR_INTEGER) {
01044         throw mx::Exception("to_int: first argument must be an integer variable");
01045     }
01046
01047     if (isVariable(i.op2.op)) {
01048         Variable &src = getVariable(i.op2.op);
01049         switch (src.type) {
01050             case VarType::VAR_INTEGER:
01051             case VarType::VAR_POINTER:
01052             case VarType::VAR_EXTERN: {
01053                 auto ra_src = sysv_reg_vars.find(i.op2.op);
01054                 if (ra_src != sysv_reg_vars.end()) {
01055                     out << "\tmovq " << ra_src->second << ", %rax\n";
01056                 } else {
01057                     out << "\tmovq " << getPlatformSymbolName(getMangledName(i.op2)) << "(%rip), %rax\n";
01058                 }
01059                 sysv_emitStoreVar(out, "%rax", i.op1);
01060                 break;
01061             }
01062
01063             case VarType::VAR_BYTE:
01064                 out << "\tmovzbq " << getPlatformSymbolName(getMangledName(i.op2)) << "(%rip), %rax\n";
01065                 sysv_emitStoreVar(out, "%rax", i.op1);
01066                 break;
01067
01068             case VarType::VAR_FLOAT:
01069                 out << "\tmovsd " << getPlatformSymbolName(getMangledName(i.op2)) << "(%rip), %xmm0\n";
01070                 out << "\tvcvttsd2si %xmm0, %rax\n";
01071                 sysv_emitStoreVar(out, "%rax", i.op1);
01072                 break;
01073
01074             case VarType::VAR_STRING:
01075                 out << "\tleaq " << getPlatformSymbolName(getMangledName(i.op2)) << "(%rip), %rdi\n";
01076                 out << "\tmovq %rsp, %rbx\n";
01077                 out << "\tandq $-16, %rsp\n";
01078                 out << "\tcall " << getPlatformSymbolName("atoi") << "\n";
01079                 out << "\tmovq %rbx, %rsp\n";
01080                 sysv_emitStoreVar(out, "%rax", i.op1);
01081                 break;
01082

```



```

01083         default:
01084             throw mx::Exception("to_int: unsupported source variable type");
01085         }
01086     } else if (i.op2.type == OperandType::OP_CONSTANT) {
01087         out << "\tmovq $" << i.op2.op << ", %rax\n";
01088         sysv_emitStoreVar(out, "%rax", i.op1);
01089     } else {
01090         throw mx::Exception("to_int: unsupported source operand");
01091     }
01092 }
01093 void Program::gen_to_float(std::ostream &out, const Instruction &i) {
01094     if (!isVariable(i.op1.op)) {
01095         throw mx::Exception("to_float requires destination variable (dest float)");
01096     }
01097     if (i.op2.op.empty()) {
01098         throw mx::Exception("to_float requires source operand");
01099     }
01100     Variable &dest = getVariable(i.op1.op);
01101     if (dest.type != VarType::VAR_FLOAT) {
01102         throw mx::Exception("to_float: first argument must be a float variable");
01103     }
01104 }
01105 if (isVariable(i.op2.op)) {
01106     Variable &src = getVariable(i.op2.op);
01107     switch (src.type) {
01108     case VarType::VAR_FLOAT:
01109         out << "\tmovsd " << getPlatformSymbolName(getMangledName(i.op2)) << "(%rip), %xmm0\n";
01110         out << "\tmovsd %xmm0, " << getPlatformSymbolName(getMangledName(i.op1)) << "(%rip)\n";
01111         break;
01112     case VarType::VAR_INTEGER:
01113     case VarType::VAR_POINTER:
01114     case VarType::VAR_EXTERN: {
01115         auto ra_src = sysv_reg_vars.find(i.op2.op);
01116         if (ra_src != sysv_reg_vars.end()) {
01117             out << "\tmovq " << ra_src->second << ", %rax\n";
01118         } else {
01119             out << "\tmovq " << getPlatformSymbolName(getMangledName(i.op2)) << "(%rip), %rax\n";
01120         }
01121         out << "\tcvtsi2sd %rax, %xmm0\n";
01122         out << "\tmovsd %xmm0, " << getPlatformSymbolName(getMangledName(i.op1)) << "(%rip)\n";
01123         break;
01124     }
01125     case VarType::VAR_BYTE:
01126         out << "\tmovzbq " << getPlatformSymbolName(getMangledName(i.op2)) << "(%rip), %rax\n";
01127         out << "\tcvtsi2sd %rax, %xmm0\n";
01128         out << "\tmovsd %xmm0, " << getPlatformSymbolName(getMangledName(i.op1)) << "(%rip)\n";
01129         break;
01130     case VarType::VAR_STRING:
01131         out << "\tleaq " << getPlatformSymbolName(getMangledName(i.op2)) << "(%rip), %rdi\n";
01132         out << "\tmovq %rsp, %rbx\n";
01133         out << "\tandq $-16, %rsp\n";
01134         out << "\tcall " << getPlatformSymbolName("atof") << "\n";
01135         out << "\tmovq %rbx, %rsp\n";
01136         out << "\tmovsd %xmm0, " << getPlatformSymbolName(getMangledName(i.op1)) << "(%rip)\n";
01137         break;
01138     default:
01139         throw mx::Exception("to_float: unsupported source variable type");
01140     }
01141 } else if (i.op2.type == OperandType::OP_CONSTANT) {
01142     out << "\tmovq $" << i.op2.op << ", %rax\n";
01143     out << "\tcvtsi2sd %rax, %xmm0\n";
01144     out << "\tmovsd %xmm0, " << getPlatformSymbolName(getMangledName(i.op1)) << "(%rip)\n";
01145 } else {
01146     throw mx::Exception("to_float: unsupported source operand");
01147 }
01148 }
01149 void Program::gen_div(std::ostream &out, const Instruction &i) {
01150     if (i.op3.op.empty()) {
01151         if (isVariable(i.op1.op)) {
01152             Variable &v = getVariable(i.op1.op);
01153             if (v.type == VarType::VAR_INTEGER) {
01154                 generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op1);
01155                 generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op2);
01156                 out << "\tcmpq $0, %rcx\n";
01157                 out << "\tje 1f\n";
01158                 out << "\tcdqo\n";
01159                 out << "\tidivq %rcx\n";
01160                 out << "\tjmp 2f\n";
01161                 out << "1:\n";
01162                 out << "\txor %eax, %eax\n";
01163                 out << "2:\n";
01164             }

```

```

01170         sysv_emitStoreVar(out, "%rax", i.op1);
01171     } else if (v.type == VarType::VAR_FLOAT) {
01172         generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
01173         generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op2);
01174         out << "\txorpd %xmm2, %xmm2\n";
01175         out << "\tucomisd %xmm2, %xmm1\n";
01176         out << "\tje 1f\n";
01177         out << "\tdivsd %xmm1, %xmm0\n";
01178         out << "\tjmp 2f\n";
01179         out << "1:\n";
01180         out << "\txorpd %xmm0, %xmm0\n";
01181         out << "2:\n";
01182         out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01183     } else {
01184         throw mx::Exception("DIV: unsupported variable type");
01185     }
01186 } else {
01187     throw mx::Exception("DIV: first argument must be a variable");
01188 }
01189 } else {
01190     if (isVariable(i.op1.op)) {
01191         Variable &v = getVariable(i.op1.op);
01192         if (v.type == VarType::VAR_INTEGER) {
01193             generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01194             generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op3);
01195             out << "\tcmpq $0, %rcx\n";
01196             out << "\tje 1f\n";
01197             out << "\tqto\n";
01198             out << "\tidivq %rcx\n";
01199             out << "\tjmp 2f\n";
01200             out << "1:\n";
01201             out << "\txor %eax, %eax\n";
01202             out << "2:\n";
01203             sysv_emitStoreVar(out, "%rax", i.op1);
01204         } else if (v.type == VarType::VAR_FLOAT) {
01205             generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op2);
01206             generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op3);
01207             out << "\txorpd %xmm2, %xmm2\n";
01208             out << "\tucomisd %xmm2, %xmm1\n";
01209             out << "\tje 1f\n";
01210             out << "\tdivsd %xmm1, %xmm0\n";
01211             out << "\tjmp 2f\n";
01212             out << "1:\n";
01213             out << "\txorpd %xmm0, %xmm0\n";
01214             out << "2:\n";
01215             out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01216         } else {
01217             throw mx::Exception("DIV: unsupported variable type");
01218         }
01219     } else {
01220         throw mx::Exception("DIV: first argument must be a variable");
01221     }
01222 }
01223 }
01224
01225 void Program::gen_mod(std::ostream &out, const Instruction &i) {
01226     if (i.op3.op.empty()) {
01227         if (isVariable(i.op1.op)) {
01228             Variable &v = getVariable(i.op1.op);
01229             if (v.type == VarType::VAR_INTEGER) {
01230                 generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op1);
01231                 generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op2);
01232                 out << "\tcmpq $0, %rcx\n";
01233                 out << "\tje 1f\n";
01234                 out << "\tqto\n";
01235                 out << "\tidivq %rcx\n";
01236                 out << "\tjmp 2f\n";
01237                 out << "1:\n";
01238                 out << "\txor %edx, %edx\n";
01239                 out << "2:\n";
01240                 sysv_emitStoreVar(out, "%rdx", i.op1);
01241             } else {
01242                 throw mx::Exception("MOD only supports integer variables");
01243             }
01244         } else {
01245             throw mx::Exception("MOD: first argument must be a variable");
01246         }
01247     } else {
01248         if (isVariable(i.op1.op)) {
01249             Variable &v = getVariable(i.op1.op);
01250             if (v.type == VarType::VAR_INTEGER) {
01251                 generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01252                 generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op3);
01253                 out << "\tcmpq $0, %rcx\n";
01254                 out << "\tje 1f\n";
01255                 out << "\tqto\n";
01256                 out << "\tidivq %rcx\n";

```

```

01257         out << "\tjmp 2f\n";
01258         out << "1:\n";
01259         out << "\txor %edx, %edx\n";
01260         out << "2:\n";
01261         sysv_emitStoreVar(out, "%rdx", i.op1);
01262     } else {
01263         throw mx::Exception("MOD only supports integer variables");
01264     }
01265 } else {
01266     throw mx::Exception("MOD: first argument must be a variable");
01267 }
01268 }
01269 }
01270
01271 void Program::gen_getline(std::ostream &out, const Instruction &i) {
01272     if (!isVariable(i.op1.op)) {
01273         throw mx::Exception("GETLINE destination must be a variable");
01274     }
01275     Variable &dest = getVariable(i.op1.op);
01276     if (dest.type != VarType::VAR_STRING || dest.var_value.buffer_size == 0) {
01277         throw mx::Exception("GETLINE destination must be a string buffer variable");
01278     }
01279     static int over_count = 0;
01280     out << "\tleaq " << getMangledName(i.op1) << " (%rip), %rdi\n";
01281     out << "\tmovq $" << dest.var_value.buffer_size << ", %rsi\n";
01282     out << "\tmovq " << getPlatformSymbolName("stdin") << " (%rip), %rdx\n";
01283     out << "\tmovq %rsp, %rbx\n";
01284     out << "\tandq $-16, %rsp\n";
01285     out << "\tcall " << getPlatformSymbolName("fgets") << "\n";
01286     out << "\tmovq %rbx, %rsp\n";
01287     out << "\tleaq " << getMangledName(i.op1) << " (%rip), %rdi\n";
01288     out << "\tmovq %rsp, %rbx\n";
01289     out << "\tandq $-16, %rsp\n";
01290     out << "\tcall " << getPlatformSymbolName("strlen") << "\n";
01291     out << "\tmovq %rbx, %rsp\n";
01292     out << "\tmovq %rax, %rcx\n";
01293     out << "\tcmp $0, %rax\n";
01294     out << "\tje .over" << over_count << "\n";
01295     out << "\tsub $1, %rcx\n";
01296     out << "\tleaq " << getMangledName(i.op1) << " (%rip), %rdi\n";
01297     out << "\tmovb $0, (%rdi, %rcx, 1)\n";
01298     out << ".over" << over_count << ": \n";
01299     over_count++;
01300 }
01301
01302 void Program::gen_bitop(std::ostream &out, const std::string &opc, const Instruction &i) {
01303     if (i.op3.op.empty()) {
01304         if (isVariable(i.op1.op)) {
01305             Variable &v = getVariable(i.op1.op);
01306             if (v.type == VarType::VAR_INTEGER) {
01307                 generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op1);
01308                 generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op2);
01309                 out << "\t" << opc << "q %rcx, %rax\n";
01310                 sysv_emitStoreVar(out, "%rax", i.op1);
01311             } else {
01312                 throw mx::Exception("Bitwise operations only supported for integer variables");
01313             }
01314         } else {
01315             throw mx::Exception("First argument of bitop instruction must be a variable.");
01316         }
01317     } else {
01318         if (isVariable(i.op1.op)) {
01319             Variable &v = getVariable(i.op1.op);
01320             if (v.type == VarType::VAR_INTEGER) {
01321                 generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01322                 generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op3);
01323                 out << "\t" << opc << "q %rcx, %rax\n";
01324                 sysv_emitStoreVar(out, "%rax", i.op1);
01325             } else {
01326                 throw mx::Exception("Bitwise operations only supported for integer variables");
01327             }
01328         } else {
01329             throw mx::Exception("First argument of bitop instruction must be a variable.");
01330         }
01331     }
01332 }
01333
01334 void Program::gen_not(std::ostream &out, const Instruction &i) {
01335     if (!i.op1.op.empty()) {
01336         if (isVariable(i.op1.op)) {
01337             Variable &v = getVariable(i.op1.op);
01338             if (v.type != VarType::VAR_INTEGER) {
01339                 throw mx::Exception("NOT instruction expect integer variable");
01340             }
01341             generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op1);
01342             out << "\ttstq %rax, %rax\n";
01343             out << "\tsete %al\n";

```

```

01344         out << "\tmovzbq %al, %rax\n";
01345         sysv_emitStoreVar(out, "%rax", i.op1);
01346     } else {
01347         throw mx::Exception("NOT instruction requires variable");
01348     }
01349 } else {
01350     throw mx::Exception("NOT instruction requires operand");
01351 }
01352 }
01353
01365 void Program::gen_push(std::ostream &out, const Instruction &i) {
01366     if (isVariable(i.op1.op)) {
01367         Variable &v = getVariable(i.op1.op);
01368         if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER || v.type ==
VarType::VAR_EXTERN) {
01369             sysv_emitLoadVar(out, "%rax", i.op1);
01370             out << "\tpushq %rax\n";
01371         } else if (v.type == VarType::VAR_STRING) {
01372             out << "\tleaq " << getMangledName(i.op1) << "(%rip), %rax\n";
01373             out << "\tpushq %rax\n";
01374         } else if (v.type == VarType::VAR_FLOAT) {
01375             out << "\tmovsd " << getMangledName(i.op1) << "(%rip), %xmm0\n";
01376             out << "\tsubq $8, %rsp\n";
01377             out << "\tmovsd %xmm0, (%rsp)\n";
01378         } else {
01379             throw mx::Exception("PUSH only supports integer, pointer, or float variables");
01380         }
01381     } else if (i.op1.type == OperandType::OP_CONSTANT) {
01382         out << "\tmovq $" << i.op1.op << ", %rax\n";
01383         out << "\tpushq %rax\n";
01384     } else {
01385         throw mx::Exception("PUSH argument must be a variable or integer constant");
01386     }
01387 }
01388
01400 void Program::gen_pop(std::ostream &out, const Instruction &i) {
01401     if (!isVariable(i.op1.op)) {
01402         throw mx::Exception("POP destination must be a variable");
01403     }
01404     Variable &v = getVariable(i.op1.op);
01405     if (v.type == VarType::VAR_INTEGER) {
01406         out << "\tpopq %rax\n";
01407         sysv_emitStoreVar(out, "%rax", i.op1);
01408     } else if (v.type == VarType::VAR_POINTER || v.type == VarType::VAR_EXTERN) {
01409         out << "\tpopq %rax\n";
01410         out << "\tmovq %rax, " << getMangledName(i.op1) << "(%rip)\n";
01411     } else if (v.type == VarType::VAR_FLOAT) {
01412         out << "\tmovsd (%rsp), %xmm0\n";
01413         out << "\taddq $8, %rsp\n";
01414         out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01415     } else {
01416         throw mx::Exception("POP only supports integer, pointer, or float variables");
01417     }
01418 }
01419
01420 void Program::gen_stack_load(std::ostream &out, const Instruction &i) {
01421     if (!isVariable(i.op1.op)) {
01422         throw mx::Exception("stack_load, requires first argument to be a variable.");
01423     }
01424     Variable &dest = getVariable(i.op1.op);
01425     if (i.op2.op.empty()) {
01426         throw mx::Exception("stack_load, requires second argument");
01427     }
01428     generateLoadVar(out, dest.type, "%rax", i.op2);
01429     out << "\tmovq (%rsp, %rax, 8), " << "%rcx\n";
01430     sysv_emitStoreVar(out, "%rcx", i.op1);
01431 }
01432
01433 void Program::gen_stack_store(std::ostream &out, const Instruction &i) {
01434     if (!isVariable(i.op1.op)) {
01435         throw mx::Exception("stack_store, requires first argument to be a variable.");
01436     }
01437     Variable &dest = getVariable(i.op1.op);
01438     if (i.op2.op.empty()) {
01439         throw mx::Exception("stack_store, requires second argument");
01440     }
01441     generateLoadVar(out, dest.type, "%rax", i.op2);
01442     sysv_emitLoadVar(out, "%rcx", i.op1);
01443     out << "\tmovq %rcx, (%rsp, %rax, 8)\n";
01444 }
01445
01446 void Program::gen_stack_sub(std::ostream &out, const Instruction &i) {
01447     if (!i.op1.op.empty()) {
01448         if (isVariable(i.op1.op)) {
01449             sysv_emitLoadVar(out, "%rcx", i.op1);
01450         } else {
01451             out << "\tmovq $" << i.op1.op << ", %rcx\n";

```

```

01452     }
01453     } else {
01454         out << "\tmovq $1, %rcx\n";
01455     }
01456     out << "\tshl $3, %rcx\n";
01457     out << "\taddq %rcx, %rsp\n";
01458 }
01459
01460 int Program::generateLoadVar(std::ostream &out, int r, const Operand &op) {
01461     int count = 0;
01462
01463     if (isVariable(op.op)) {
01464         if (op.type != OperandType::OP_VARIABLE) {
01465             throw mx::Exception("Operand expected variable instead I found: " + op.op);
01466         }
01467         Variable &v = getVariable(op.op);
01468
01469         std::string reg = getRegisterByIndex(r, v.type);
01470         switch (v.type) {
01471             case VarType::VAR_INTEGER:
01472             case VarType::VAR_POINTER:
01473             case VarType::VAR_EXTERN: {
01474                 auto ra = sysv_reg_vars.find(op.op);
01475                 if (ra != sysv_reg_vars.end()) {
01476                     if (ra->second != reg)
01477                         out << "\tmovq " << ra->second << ", " << reg << "\n";
01478                 } else {
01479                     out << "\tmovq " << getPlatformSymbolName(getMangledName(op)) << "(%rip), " << reg <<
01480 "\n";
01481                 }
01482                 count = 0;
01483                 break;
01484             }
01485             case VarType::VAR_FLOAT:
01486                 out << "\tmovsd " << getPlatformSymbolName(getMangledName(op)) << "(%rip), " << reg <<
01487 "\n";
01488                 count = 1;
01489                 break;
01490             case VarType::VAR_STRING:
01491                 count = 0;
01492                 out << "\tleaq " << getPlatformSymbolName(getMangledName(op)) << "(%rip), " << reg << "\n";
01493                 break;
01494             case VarType::VAR_BYTE:
01495                 count = 0;
01496                 out << "\tmovzbq " << getPlatformSymbolName(getMangledName(op)) << "(%rip), " << reg <<
01497 "\n";
01498                 break;
01499             default:
01500                 break;
01501         }
01502     } else {
01503         if (op.type == OperandType::OP_CONSTANT) {
01504             out << "\tmovq $" << op.op << ", " << getRegisterByIndex(r, VarType::VAR_INTEGER) << "\n";
01505         }
01506     }
01507     return count;
01508 }
01509
01510 int Program::generateLoadVar(std::ostream &out, VarType reqType, std::string reg, const Operand
01511 &op) {
01512     int count = 0;
01513     if (isVariable(op.op)) {
01514         if (op.type != OperandType::OP_VARIABLE) {
01515             throw mx::Exception("Operand expected variable instead I found: " + op.op);
01516         }
01517         Variable &v = getVariable(op.op);
01518         if (reqType == VarType::VAR_FLOAT) {
01519             if (v.type == VarType::VAR_FLOAT) {
01520                 out << "\tmovsd " << getPlatformSymbolName(getMangledName(op)) << "(%rip), " << reg <<
01521 "\n";
01522                 count = 1;
01523             } else if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER || v.type
01524 == VarType::VAR_EXTERN) {
01525                 auto ra = sysv_reg_vars.find(op.op);
01526                 if (ra != sysv_reg_vars.end()) {
01527                     out << "\tmovq " << ra->second << ", %rax\n";
01528                 } else {
01529                     out << "\tmovq " << getPlatformSymbolName(getMangledName(op)) << "(%rip),
01530 %rax\n";
01531                 }
01532                 out << "\tcvttsi2sd %rax, " << reg << "\n";
01533                 count = 1;
01534             } else if (v.type == VarType::VAR_BYTE) {
01535                 out << "\tmovzbq " << getPlatformSymbolName(getMangledName(op)) << "(%rip), %rax\n";
01536                 out << "\tcvttsi2sd %rax, " << reg << "\n";
01537                 count = 1;
01538             } else if (v.type == VarType::VAR_STRING) {

```

```

01532         out « "\tleaq " « getPlatformSymbolName(getMangledName(op)) « "(%rip), %rdi\n";
01533         out « "\tmovq %rsp, %rbx\n";
01534         out « "\tandq $-16, %rsp\n";
01535         out « "\tcall " « getPlatformSymbolName("atof") « "\n";
01536         out « "\tmovq %rbx, %rsp\n";
01537         out « "\tmovsd %xmm0, " « reg « "\n";
01538         count = 1;
01539     } else {
01540         throw mx::Exception("LoadVar: unsupported source type for float");
01541     }
01542     return count;
01543 }
01544
01545     if (reqType == VarType::VAR_INTEGER || reqType == VarType::VAR_POINTER || reqType ==
VarType::VAR_EXTERN) {
01546         if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER || v.type ==
VarType::VAR_EXTERN) {
01547             auto ra = sysv_reg_vars.find(op.op);
01548             if (ra != sysv_reg_vars.end()) {
01549                 if (ra->second != reg)
01550                     out « "\tmovq " « ra->second « ", " « reg « "\n";
01551             } else {
01552                 out « "\tmovq " « getPlatformSymbolName(getMangledName(op)) « "(%rip), " « reg
« "\n";
01553             }
01554         } else if (v.type == VarType::VAR_BYTE) {
01555             out « "\tmovzbq " « getPlatformSymbolName(getMangledName(op)) « "(%rip), " « reg «
"\n";
01556         } else if (v.type == VarType::VAR_FLOAT) {
01557             out « "\tmovsd " « getPlatformSymbolName(getMangledName(op)) « "(%rip), %xmm0\n";
01558             out « "\tcvtsd2si %xmm0, " « reg « "\n";
01559         } else if (v.type == VarType::VAR_STRING) {
01560             out « "\tleaq " « getPlatformSymbolName(getMangledName(op)) « "(%rip), %rdi\n";
01561             out « "\tmovq %rsp, %rbx\n";
01562             out « "\tandq $-16, %rsp\n";
01563             out « "\tcall " « getPlatformSymbolName("atoi") « "\n";
01564             out « "\tmovq %rbx, %rsp\n";
01565             out « "\tmovq %rax, " « reg « "\n";
01566         } else {
01567             throw mx::Exception("LoadVar: unsupported source type for integer");
01568         }
01569         return count;
01570     }
01571
01572     if (reqType == VarType::VAR_STRING) {
01573         if (v.type == VarType::VAR_STRING) {
01574             out « "\tleaq " « getPlatformSymbolName(getMangledName(op)) « "(%rip), " « reg «
"\n";
01575         } else if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER || v.type
== VarType::VAR_EXTERN) {
01576             auto ra = sysv_reg_vars.find(op.op);
01577             if (ra != sysv_reg_vars.end()) {
01578                 if (ra->second != reg)
01579                     out « "\tmovq " « ra->second « ", " « reg « "\n";
01580             } else {
01581                 out « "\tmovq " « getPlatformSymbolName(getMangledName(op)) « "(%rip), " « reg
« "\n";
01582             }
01583         } else {
01584             throw mx::Exception("LoadVar: unsupported source type for string");
01585         }
01586         return count;
01587     }
01588
01589     switch (v.type) {
01590     case VarType::VAR_INTEGER:
01591     case VarType::VAR_POINTER:
01592     case VarType::VAR_EXTERN: {
01593         auto ra = sysv_reg_vars.find(op.op);
01594         if (ra != sysv_reg_vars.end()) {
01595             if (ra->second != reg)
01596                 out « "\tmovq " « ra->second « ", " « reg « "\n";
01597         } else {
01598             out « "\tmovq " « getPlatformSymbolName(getMangledName(op)) « "(%rip), " « reg «
"\n";
01599         }
01600         count = 0;
01601         break;
01602     }
01603     case VarType::VAR_BYTE:
01604         out « "\tmovzbq " « getPlatformSymbolName(getMangledName(op)) « "(%rip), " « reg «
"\n";
01605         break;
01606     case VarType::VAR_FLOAT:
01607         out « "\tmovsd " « getPlatformSymbolName(getMangledName(op)) « "(%rip), " « reg «
"\n";
01608         count = 1;

```

```

01609         break;
01610     case VarType::VAR_STRING:
01611         out << "\tleaq " << getPlatformSymbolName(getMangledName(op)) << "({rip}), " << reg << "\n";
01612         break;
01613     default:
01614         break;
01615     }
01616 } else {
01617     if (op.type == OperandType::OP_CONSTANT) {
01618         if (reqType == VarType::VAR_INTEGER || reqType == VarType::VAR_POINTER || reqType ==
VarType::VAR_EXTERN) {
01619             out << "\tmovq $" << op.op << ", " << reg << "\n";
01620         } else if (reqType == VarType::VAR_FLOAT) {
01621             out << "\tmovq $" << op.op << ", %rax\n";
01622             out << "\tcdvtsi2sd %rax, " << reg << "\n";
01623             count = 1;
01624         } else if (reqType == VarType::VAR_STRING) {
01625             out << "\tmovq $" << op.op << ", " << reg << "\n";
01626         } else {
01627             out << "\tmovq $" << op.op << ", " << reg << "\n";
01628         }
01629     }
01630 }
01631 return count;
01632 }
01633
01634 std::string Program::getRegisterByIndex(int index, VarType type) {
01635     if (index < 6 && (type == VarType::VAR_INTEGER || type == VarType::VAR_STRING || type ==
VarType::VAR_POINTER || type == VarType::VAR_EXTERN || type == VarType::VAR_BYTE)) {
01636         static const char *reg[] = {"%rdi", "%rsi", "%rdx", "%rcx", "%r8", "%r9"};
01637         return reg[index];
01638     } else if (index < 9 && type == VarType::VAR_FLOAT) {
01639         return "%xmm" + std::to_string(xmm_offset++);
01640     }
01641     return "[stack]";
01642 }
01643
01644 void Program::gen_print(std::ostream &out, const Instruction &i) {
01645     xmm_offset = 0;
01646     out << "\tleaq " << getMangledName(i.op1) << "({rip}), %rdi\n";
01647     std::vector<Operand> args;
01648     if (!i.op2.op.empty())
01649         args.push_back(i.op2);
01650     if (!i.op3.op.empty())
01651         args.push_back(i.op3);
01652     for (const auto &v : i.vop) {
01653         if (!v.op.empty())
01654             args.push_back(v);
01655     }
01656
01657     int total = 0;
01658     const size_t stack_args = (args.size() > 5) ? (args.size() - 5) : 0;
01659
01660     out << "\tmovq %rsp, %rbx\n";
01661     out << "\tandq $-16, %rsp\n";
01662
01663     if (stack_args) {
01664         size_t stack_bytes = stack_args * 8;
01665         size_t aligned = (stack_bytes + 15) & ~(size_t)15;
01666         out << "\tsubq $" << aligned << ", %rsp\n";
01667         size_t slot = 0;
01668         for (size_t idx = 6; idx <= args.size(); ++idx, ++slot) {
01669             size_t aidx = idx - 1;
01670             if (isVariable(args[aidx].op)) {
01671                 Variable &v = getVariable(args[aidx].op);
01672                 if (v.type == VarType::VAR_FLOAT) {
01673                     generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", args[aidx]);
01674                     out << "\tmovsd %xmm0, " << (slot * 8) << "({rsp})\n";
01675                 } else {
01676                     generateLoadVar(out, v.type, "%rax", args[aidx]);
01677                     out << "\tmovq %rax, " << (slot * 8) << "({rsp})\n";
01678                 }
01679             } else {
01680                 out << "\tmovq $" << args[aidx].op << ", " << (slot * 8) << "({rsp})\n";
01681             }
01682         }
01683     }
01684
01685     int reg_count = 1;
01686     for (size_t z = 0; z < args.size() && reg_count < 6; ++z, ++reg_count) {
01687         total += generateLoadVar(out, reg_count, args[z]);
01688     }
01689     if (total == 0) {
01690         out << "\txor %eax, %eax\n";
01691     } else {
01692         out << "\tmovb $" << total << ", %al\n";
01693     }

```

```

01694         out << "\tcall " << getPlatformSymbolName("printf") << "\n";
01695         out << "\tmovq %rbx, %rsp\n";
01696     }
01697
01698 void Program::gen_jump(std::ostream &out, const Instruction &i) {
01699     if (!i.op1.op.empty()) {
01700         auto pos = labels.find(i.op1.op);
01701         if (pos == labels.end()) {
01702             throw mx::Exception("Jump instruction must have valid label: " + i.op1.op);
01703         }
01704         const char *mnem = nullptr;
01705         switch (i.instruction) {
01706             case JMP:
01707                 mnem = "jmp";
01708                 break;
01709             case JE:
01710                 mnem = "je";
01711                 break;
01712             case JNE:
01713                 mnem = "jne";
01714                 break;
01715             case JZ:
01716                 mnem = "jz";
01717                 break;
01718             case JNZ:
01719                 mnem = "jnz";
01720                 break;
01721
01722             case JL:
01723                 mnem = (last_cmp_type == CMP_FLOAT) ? "jb" : "jl";
01724                 break;
01725             case JLE:
01726                 mnem = (last_cmp_type == CMP_FLOAT) ? "jbe" : "jle";
01727                 break;
01728             case JG:
01729                 mnem = (last_cmp_type == CMP_FLOAT) ? "ja" : "jg";
01730                 break;
01731             case JGE:
01732                 mnem = (last_cmp_type == CMP_FLOAT) ? "jae" : "jge";
01733                 break;
01734             case JA:
01735                 mnem = "ja";
01736                 break;
01737             case JB:
01738                 mnem = "jb";
01739                 break;
01740             default:
01741                 throw mx::Exception("Invalid jump opcode for gen_jump");
01742         }
01743         out << "\t" << mnem << " " << i.op1.op << "\n";
01744     } else {
01745         throw mx::Exception("Jump instruction must have label: " + i.op1.op);
01746     }
01747 }
01748
01749 void Program::gen_cmp(std::ostream &out, const Instruction &i) {
01750     if (i.op2.op.empty()) {
01751         throw mx::Exception("CMP requires two operands");
01752     }
01753
01754     VarType type1 = VarType::VAR_INTEGER;
01755     VarType type2 = VarType::VAR_INTEGER;
01756     if (isVariable(i.op1.op)) {
01757         type1 = getVariable(i.op1.op).type;
01758     }
01759     if (isVariable(i.op2.op)) {
01760         type2 = getVariable(i.op2.op).type;
01761     }
01762
01763     if (type1 == VarType::VAR_FLOAT || type2 == VarType::VAR_FLOAT) {
01764         generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
01765         generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op2);
01766         out << "\tcomisd %xmm1, %xmm0\n";
01767         last_cmp_type = CMP_FLOAT;
01768     } else if (type1 == VarType::VAR_POINTER && (type2 == VarType::VAR_INTEGER || type2 ==
01769 VarType::VAR_BYTE)) {
01770         generateLoadVar(out, type1, "%rax", i.op1);
01771         generateLoadVar(out, type2, "%rcx", i.op2);
01772         out << "\tcmpq %rcx, %rax\n";
01773         last_cmp_type = CMP_INTEGER;
01774     } else if (!isVariable(i.op2.op) && i.op2.type == OperandType::OP_CONSTANT &&
01775 isVariable(i.op1.op)) {
01776         auto ra = sysv_reg_vars.find(i.op1.op);
01777         if (ra != sysv_reg_vars.end()) {
01778             out << "\tcmpq $" << i.op2.op << ", " << ra->second << "\n";
01779         } else if (type1 == VarType::VAR_BYTE) {

```



```

01779         out << "\tmovzbq " << getMangledName(i.op1) << "(%rip), %rax\n";
01780         out << "\tcmpq $" << i.op2.op << ", %rax\n";
01781     } else {
01782         out << "\tcmpq $" << i.op2.op << ", " << getMangledName(i.op1) << "(%rip)\n";
01783     }
01784     last_cmp_type = CMP_INTEGER;
01785 } else {
01786     generateLoadVar(out, type1, "%rax", i.op1);
01787     generateLoadVar(out, type2, "%rcx", i.op2);
01788     out << "\tcmpq %rcx, %rax\n";
01789     last_cmp_type = CMP_INTEGER;
01790 }
01791 }
01792
01793 void Program::gen_fcmp(std::ostream &out, const Instruction &i) {
01794     if (i.op2.op.empty()) {
01795         throw mx::Exception("FCMP requires two operands");
01796     }
01797
01798     generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
01799     generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op2);
01800     out << "\tcomisd %xmm1, %xmm0\n";
01801     last_cmp_type = CMP_FLOAT;
01802 }
01803
01804 void Program::gen_jae(std::ostream &out, const Instruction &i) {
01805     out << "\tjae ." << i.op1.op << "\n";
01806 }
01807
01808 void Program::gen_jbe(std::ostream &out, const Instruction &i) {
01809     out << "\tjbe ." << i.op1.op << "\n";
01810 }
01811
01812 void Program::gen_jc(std::ostream &out, const Instruction &i) {
01813     out << "\tjc ." << i.op1.op << "\n";
01814 }
01815
01816 void Program::gen_jnc(std::ostream &out, const Instruction &i) {
01817     out << "\tjnc ." << i.op1.op << "\n";
01818 }
01819
01820 void Program::gen_jp(std::ostream &out, const Instruction &i) {
01821     out << "\tjp ." << i.op1.op << "\n";
01822 }
01823
01824 void Program::gen_jnp(std::ostream &out, const Instruction &i) {
01825     out << "\tjnp ." << i.op1.op << "\n";
01826 }
01827
01828 void Program::gen_jo(std::ostream &out, const Instruction &i) {
01829     out << "\tjo ." << i.op1.op << "\n";
01830 }
01831
01832 void Program::gen_jno(std::ostream &out, const Instruction &i) {
01833     out << "\tjno ." << i.op1.op << "\n";
01834 }
01835
01836 void Program::gen_js(std::ostream &out, const Instruction &i) {
01837     out << "\tjs ." << i.op1.op << "\n";
01838 }
01839
01840 void Program::gen_jns(std::ostream &out, const Instruction &i) {
01841     out << "\tjns ." << i.op1.op << "\n";
01842 }
01843
01844 void Program::gen_mov(std::ostream &out, const Instruction &i) {
01845     if (!isVariable(i.op1.op)) {
01846         throw mx::Exception("MOV destination must be a variable: " + i.op1.op);
01847     }
01848     Variable &dest = getVariable(i.op1.op);
01849
01850     if (isVariable(i.op2.op)) {
01851         Variable &src = getVariable(i.op2.op);
01852         if (dest.type == VarType::VAR_FLOAT && src.type == VarType::VAR_FLOAT) {
01853             out << "\tmovsd " << getMangledName(i.op2) << "(%rip), %xmm0\n";
01854             out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01855         } else if (dest.type == VarType::VAR_FLOAT && src.type != VarType::VAR_FLOAT) {
01856             if (src.type == VarType::VAR_BYTE) {
01857                 out << "\tmovzbq " << getMangledName(i.op2) << "(%rip), %rax\n";
01858                 out << "\tcvtsi2sd %rax, %xmm0\n";
01859             } else if (src.type == VarType::VAR_STRING) {
01860                 out << "\tleaq " << getMangledName(i.op2) << "(%rip), %rdi\n";
01861                 out << "\tmovq %rsp, %rbx\n";
01862                 out << "\tandq $-16, %rsp\n";
01863                 out << "\tcall " << getPlatformSymbolName("atof") << "\n";
01864                 out << "\tmovq %rbx, %rsp\n";
01865             } else {

```

```

01866         sysv_emitLoadVar(out, "%rax", i.op2);
01867         out << "\tvcvtsi2sd %rax, %xmm0\n";
01868     }
01869     out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01870 } else if (dest.type != VarType::VAR_FLOAT && src.type == VarType::VAR_FLOAT) {
01871     out << "\tmovsd " << getMangledName(i.op2) << "(%rip), %xmm0\n";
01872     out << "\tvcvtsd2si %xmm0, %rax\n";
01873     if (dest.type == VarType::VAR_BYTE) {
01874         out << "\tmovb %al, " << getMangledName(i.op1) << "(%rip)\n";
01875     } else {
01876         sysv_emitStoreVar(out, "%rax", i.op1);
01877     }
01878 } else if (dest.type == VarType::VAR_POINTER && src.type == VarType::VAR_STRING) {
01879     out << "\tleaq " << getMangledName(i.op2) << "(%rip), %rax\n";
01880     out << "\tmovq %rax, " << getMangledName(i.op1) << "(%rip)\n";
01881 } else if (dest.type == VarType::VAR_INTEGER && src.type == VarType::VAR_STRING) {
01882     out << "\tleaq " << getMangledName(i.op2) << "(%rip), %rdi\n";
01883     out << "\tmovq %rsp, %rbx\n";
01884     out << "\tandq $-16, %rsp\n";
01885     out << "\tcall " << getPlatformSymbolName("atol") << "\n";
01886     out << "\tmovq %rbx, %rsp\n";
01887     sysv_emitStoreVar(out, "%rax", i.op1);
01888 } else if (dest.type == VarType::VAR_BYTE && src.type != VarType::VAR_BYTE) {
01889     if (src.type == VarType::VAR_FLOAT) {
01890         out << "\tmovsd " << getMangledName(i.op2) << "(%rip), %xmm0\n";
01891         out << "\tvcvtsd2si %xmm0, %rax\n";
01892         out << "\tmovb %al, " << getMangledName(i.op1) << "(%rip)\n";
01893     } else {
01894         sysv_emitLoadVar(out, "%rax", i.op2);
01895         out << "\tmovb %al, " << getMangledName(i.op1) << "(%rip)\n";
01896     }
01897 } else if (dest.type != VarType::VAR_BYTE && src.type == VarType::VAR_BYTE) {
01898     out << "\tmovzbq " << getMangledName(i.op2) << "(%rip), %rax\n";
01899     if (dest.type == VarType::VAR_FLOAT) {
01900         out << "\tvcvtsi2sd %rax, %xmm0\n";
01901         out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01902     } else {
01903         sysv_emitStoreVar(out, "%rax", i.op1);
01904     }
01905 } else {
01906     switch (dest.type) {
01907     case VarType::VAR_INTEGER:
01908     case VarType::VAR_POINTER:
01909     case VarType::VAR_EXTERN:
01910         sysv_emitLoadVar(out, "%rax", i.op2);
01911         sysv_emitStoreVar(out, "%rax", i.op1);
01912         break;
01913     case VarType::VAR_BYTE:
01914         out << "\tmovb " << getMangledName(i.op2) << "(%rip), %al\n";
01915         out << "\tmovb %al, " << getMangledName(i.op1) << "(%rip)\n";
01916         break;
01917     case VarType::VAR_STRING:
01918         out << "\tleaq " << getMangledName(i.op2) << "(%rip), %rax\n";
01919         out << "\tmovq %rax, " << getMangledName(i.op1) << "(%rip)\n";
01920         break;
01921     default:
01922         throw mx::Exception("MOV: unsupported destination type");
01923     }
01924 }
01925 } else {
01926     if (i.op2.type == OperandType::OP_CONSTANT) {
01927         if (dest.type == VarType::VAR_FLOAT) {
01928             double val = std::stod(i.op2.op);
01929             uint64_t bits;
01930             std::memcpy(&bits, &val, sizeof(bits));
01931             out << "\tmovq $" << bits << ", %rax\n";
01932             out << "\tmovq %rax, " << getMangledName(i.op1) << "(%rip)\n";
01933         } else if (dest.type == VarType::VAR_BYTE) {
01934             out << "\tmovb $" << i.op2.op << ", " << getMangledName(i.op1) << "(%rip)\n";
01935         } else {
01936             sysv_emitStoreVarImm(out, i.op2.op, i.op1);
01937         }
01938     } else {
01939         throw mx::Exception("MOV: unsupported source operand type");
01940     }
01941 }
01942 }
01943
01944 void Program::gen_arth(std::ostream &out, std::string arth, const Instruction &i) {
01945     auto emitIntegerOp = [&](const Operand &lhs, const Operand &rhs, const Operand &dest) {
01946         if (arth == "mul")
01947             arth = "imul";
01948         if (!isVariable(rhs.op) && rhs.type == OperandType::OP_CONSTANT && arth != "imul" &&
01949             lhs.op == dest.op && isVariable(lhs.op)) {
01949             auto ra = sysv_reg_vars.find(lhs.op);
01950             if (ra != sysv_reg_vars.end()) {
01951                 out << "\t" << arth << "q $" << rhs.op << ", " << ra->second << "\n";

```

```

01952         } else {
01953             out << "\t" << arth << "q $" << rhs.op << ", " << getMangledName(lhs) << "(%rip)\n";
01954         }
01955         return;
01956     }
01957     generateLoadVar(out, VarType::VAR_INTEGER, "%r10", lhs);
01958     if (!isVariable(rhs.op) && rhs.type == OperandType::OP_CONSTANT) {
01959         out << "\t" << arth << "q $" << rhs.op << ", %r10\n";
01960     } else {
01961         generateLoadVar(out, VarType::VAR_INTEGER, "%r11", rhs);
01962         out << "\t" << arth << "q %r11, %r10\n";
01963     }
01964     sysv_emitStoreVar(out, "%r10", dest);
01965 };
01966
01967 if (i.op3.op.empty()) {
01968     if (!isVariable(i.op1.op)) {
01969         throw mx::Exception("First argument of arithmetic must be a variable: " + i.op1.op);
01970     }
01971     Variable &v = getVariable(i.op1.op);
01972
01973     if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER) {
01974         emitIntegerOp(i.op1, i.op2, i.op1);
01975     } else if (v.type == VarType::VAR_FLOAT) {
01976         generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
01977         generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op2);
01978         out << "\t" << arth << "sd %xmm1, %xmm0\n";
01979         out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01980     } else {
01981         throw mx::Exception("Unsupported variable type for arithmetic");
01982     }
01983 } else {
01984     if (!isVariable(i.op1.op)) {
01985         throw mx::Exception("First argument of arithmetic must be a variable: " + i.op1.op);
01986     }
01987     Variable &v = getVariable(i.op1.op);
01988
01989     if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER) {
01990         emitIntegerOp(i.op2, i.op3, i.op1);
01991     } else if (v.type == VarType::VAR_FLOAT) {
01992         generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op2);
01993         generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op3);
01994         out << "\t" << arth << "sd %xmm1, %xmm0\n";
01995         out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01996     } else {
01997         throw mx::Exception("Unsupported variable type for arithmetic");
01998     }
01999 }
02000 }
02001
02002 void Program::gen_exit(std::ostream &out, const Instruction &i) {
02003     bool uses_std_module = false;
02004     for (const auto &e : external) {
02005         if (e.mod == "std") {
02006             uses_std_module = true;
02007             break;
02008         }
02009     }
02010     if (uses_std_module) {
02011         out << "\tmovq %rsp, %rbx\n";
02012         out << "\tandq $-16, %rsp\n";
02013         out << "\tcall " << getPlatformSymbolName("free_program_args") << "\n";
02014         out << "\tmovq %rbx, %rsp\n";
02015     }
02016
02017     if (!i.op1.op.empty()) {
02018         std::vector<Operand> opz;
02019         opz.push_back(i.op1);
02020         generateFunctionCall(out, "exit", opz);
02021     } else {
02022         throw mx::Exception("exit requires argument.\n");
02023     }
02024 }
02025 } // namespace mxvm

```

12.60 src/frontend/icode_imp.cpp File Reference

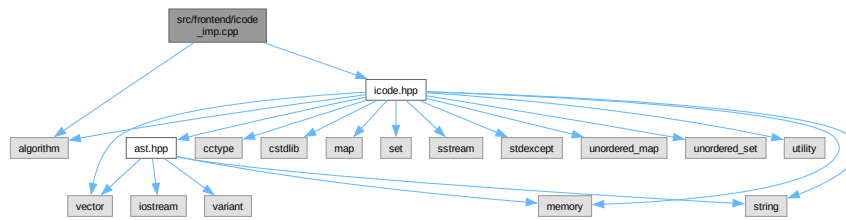
Pascal code generator helper methods and builtin function handler implementations.

```

#include "icode.hpp"
#include <algorithm>

```

Include dependency graph for `icode_imp.cpp`:



Namespaces

- namespace [pascal](#)

Functions

- static `std::string pascal::toLower` (const `std::string &s`)
Convert a string to lowercase for case-insensitive comparison.

Variables

- `std::unordered_set< std::string > pascal::sdlFunctions`
- `std::unordered_set< std::string > pascal::stdFunctions`
- static `std::unordered_set< std::string > pascal::fileFunctions`

12.60.1 Detailed Description

Pascal code generator helper methods and builtin function handler implementations.

Author

Jared Bruni

Definition in file [icode_imp.cpp](#).

12.61 icode_imp.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "icode.hpp"
00007 #include <algorithm>
00008
00009 namespace pascal {
00010
00016     static std::string toLower(const std::string &s) {
00017         std::string r = s;
00018         std::transform(r.begin(), r.end(), r.begin(),
00019             [](unsigned char c) { return std::tolower(c); });
00020         return r;
00021     }
00022
00023     std::unordered_set<std::string> sdlFunctions = {
00024         "sdl_init", "sdl_quit", "sdl_create_window", "sdl_destroy_window", "sdl_set_window_title",
00025         "sdl_set_window_position", "sdl_get_window_size", "sdl_set_window_fullscreen",
00026         "sdl_set_window_icon",
00027         "sdl_create_renderer", "sdl_destroy_renderer", "sdl_set_draw_color", "sdl_clear",
00028         "sdl_present",
00029         "sdl_get_renderer_output_size", "sdl_draw_point", "sdl_draw_line", "sdl_draw_rect",
00030         "sdl_fill_rect",
00031         "sdl_poll_event", "sdl_get_event_type", "sdl_get_key_code", "sdl_get_mouse_x",
00032         "sdl_get_mouse_y",
00033         "sdl_get_mouse_button", "sdl_get_mouse_buttons", "sdl_get_relative_mouse_x",
00034         "sdl_get_relative_mouse_y",

```

```

00030         "sdl_get_relative_mouse_buttons", "sdl_create_rgb_surface", "sdl_free_surface",
"sdl_blit_surface",
00031         "sdl_get_mouse_state", "sdl_get_relative_mouse_state", "sdl_get_keyboard_state",
"sdl_is_key_pressed",
00032         "sdl_get_num_keys", "sdl_set_clipboard_text", "sdl_get_clipboard_text", "sdl_show_cursor",
00033         "sdl_create_texture", "sdl_destroy_texture", "sdl_load_texture", "sdl_load_texture_color_key",
"sdl_load_texture_color_key_rgb", "sdl_render_texture",
00034         "sdl_update_texture", "sdl_lock_texture", "sdl_unlock_texture", "sdl_get_ticks", "sdl_delay",
00035         "sdl_open_audio", "sdl_close_audio", "sdl_pause_audio", "sdl_load_wav", "sdl_free_wav",
00036         "sdl_queue_audio", "sdl_get_queued_audio_size", "sdl_clear_queued_audio", "sdl_init_text",
00037         "sdl_quit_text", "sdl_load_font", "sdl_draw_text", "sdl_create_render_target",
"sdl_set_render_target",
00038         "sdl_destroy_render_target", "sdl_present_scaled", "sdl_present_stretched");
00039
00040     std::unordered_set<std::string> stdFunctions = {
00041         "argc", "argv", "set_program_args", "free_program_args", "abs", "fabs",
00042         "sqrt", "pow", "sin", "cos", "tan", "floor", "ceil", "rand", "srand",
00043         "malloc", "calloc", "free", "toupper", "tolower", "isalpha", "isdigit",
00044         "isspace", "atoi", "atof", "exit", "system", "memcpy", "memcmp",
00045         "memmove", "memset", "exp", "exp2", "log", "log10", "log2", "fmod",
00046         "atan2", "asin", "acos", "atan", "sinh", "cosh", "tanh", "hypot",
00047         "round", "trunc", "float_to_int", "int_to_float", "halt",
00048         "ord", "chr", "succ", "pred", "inc", "dec", "include", "exclude");
00049
00050     bool IOFunctionHandler::canHandle(const std::string &funcName) const {
00051         auto f = toLower(funcName);
00052         return f == "writeln" || f == "write" || f == "readln" ||
00053             f == "seed_random" || f == "rand_number";
00054     }
00055
00056     void IOFunctionHandler::generate(CoGenVisitor &visitor, const std::string &funcName_,
const std::vector<std::unique_ptr<ASTNode>> &arguments) {
00057         auto funcName = toLower(funcName_);
00058         if (funcName == "writeln" || funcName == "write") {
00059             // Check if first argument is a file variable
00060             bool fileOutput = false;
00061             std::string fileSlot;
00062             size_t startArg = 0;
00063             if (!arguments.empty()) {
00064                 if (auto *varNode = dynamic_cast<VariableNode *>(arguments[0].get())) {
00065                     std::string mangledName = visitor.findMangledName(varNode->name);
00066                     if (visitor.fileVars.count(toLower(mangledName)) ||
00067                         visitor.fileVars.count(toLower(varNode->name))) {
00068                         fileOutput = true;
00069                         fileSlot = visitor.eval(arguments[0].get());
00070                         startArg = 1;
00071                         visitor.usedModules.insert("io");
00072                     }
00073                 }
00074             }
00075         }
00076
00077         for (size_t i = startArg; i < arguments.size(); ++i) {
00078             const auto &arg = arguments[i];
00079             std::string val = visitor.eval(arg.get());
00080             VarType type = visitor.getExpressionType(arg.get());
00081
00082             if (fileOutput) {
00083                 // Use fprintf for file output
00084                 if (type == VarType::STRING || type == VarType::PTR) {
00085                     visitor.usedStrings.insert("fmt_str");
00086                     visitor.emit_invoke("fprintf", {fileSlot, "fmt_str", val});
00087                 } else if (type == VarType::DOUBLE) {
00088                     visitor.usedStrings.insert("fmt_float");
00089                     visitor.emit_invoke("fprintf", {fileSlot, "fmt_float", val});
00090                 } else if (type == VarType::CHAR) {
00091                     visitor.usedStrings.insert("fmt_chr");
00092                     visitor.emit_invoke("fprintf", {fileSlot, "fmt_chr", val});
00093                 } else {
00094                     visitor.usedStrings.insert("fmt_int");
00095                     visitor.emit_invoke("fprintf", {fileSlot, "fmt_int", val});
00096                 }
00097             } else {
00098                 if (type == VarType::STRING || type == VarType::PTR) {
00099                     visitor.usedStrings.insert("fmt_str");
00100                     visitor.emit2("print", "fmt_str", val);
00101                 } else if (type == VarType::DOUBLE) {
00102                     visitor.usedStrings.insert("fmt_float");
00103                     visitor.emit2("print", "fmt_float", val);
00104                 } else if (type == VarType::CHAR) {
00105                     visitor.usedStrings.insert("fmt_chr");
00106                     visitor.emit2("print", "fmt_chr", val);
00107                 } else {
00108                     visitor.usedStrings.insert("fmt_int");
00109                     visitor.emit2("print", "fmt_int", val);
00110                 }
00111             }
00112             if (visitor.isReg(val) && !visitor.isParmReg(val)) {

```

```

00113         visitor.freeReg(val);
00114     }
00115 }
00116 if (funcName == "writeln") {
00117     if (fileOutput) {
00118         std::string nlLabel = visitor.internString("\n");
00119         visitor.emit_invoke("fputs", {nlLabel, fileSlot});
00120     } else {
00121         visitor.usedStrings.insert("newline");
00122         visitor.emit1("print", "newline");
00123     }
00124 }
00125 if (fileOutput && visitor.isReg(fileSlot) && !visitor.isParmReg(fileSlot))
00126     visitor.freeReg(fileSlot);
00127 } else if (funcName == "readln") {
00128     if (arguments.empty()) {
00129         visitor.emit1("getline", "input_buffer");
00130         return;
00131     }
00132
00133     // Check if first argument is a file variable
00134     bool fileInput = false;
00135     std::string fileSlot;
00136     size_t startArg = 0;
00137     if (auto *firstVar = dynamic_cast<VariableNode*>(arguments[0].get())) {
00138         std::string mangledName = visitor.findMangledName(firstVar->name);
00139         if (visitor.fileVars.count(toLower(mangledName)) ||
00140             visitor.fileVars.count(toLower(firstVar->name))) {
00141             fileInput = true;
00142             fileSlot = visitor.eval(arguments[0].get());
00143             startArg = 1;
00144             visitor.usedModules.insert("io");
00145         }
00146     }
00147
00148     for (size_t i = startArg; i < arguments.size(); ++i) {
00149         auto &arg = arguments[i];
00150         if (auto varNode = dynamic_cast<VariableNode*>(arg.get())) {
00151             std::string varName = varNode->name;
00152             int slot = visitor.newSlotFor(varName);
00153             std::string memLoc = visitor.slotVar(slot);
00154
00155             auto varType = visitor.getVarType(varName);
00156
00157             if (fileInput) {
00158                 // Read line from file using mxvm_fgets
00159                 visitor.emit_invoke("mxvm_fgets", {fileSlot});
00160                 std::string retReg = visitor.allocReg();
00161                 visitor.emit("return " + retReg);
00162                 if (varType == VarType::DOUBLE) {
00163                     visitor.emit2("to_float", memLoc, retReg);
00164                 } else if (varType == VarType::INT) {
00165                     visitor.emit2("to_int", memLoc, retReg);
00166                 } else {
00167                     visitor.emit2("mov", memLoc, retReg);
00168                 }
00169                 visitor.freeReg(retReg);
00170             } else {
00171                 visitor.emit1("getline", "input_buffer");
00172                 if (varType == VarType::DOUBLE) {
00173                     visitor.emit2("to_float", memLoc, "input_buffer");
00174                 } else if (varType == VarType::INT) {
00175                     visitor.emit2("to_int", memLoc, "input_buffer");
00176                 }
00177             }
00178
00179             visitor.recordLocation(varName, {CodeGenVisitor::ValueLocation::MEMORY, memLoc});
00180         } else {
00181             int lineNum = arg->getLineNumber();
00182             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00183                                     ": readln argument must be a variable");
00184         }
00185     }
00186     if (fileInput && visitor.isReg(fileSlot) && !visitor.isParmReg(fileSlot))
00187         visitor.freeReg(fileSlot);
00188 } else if (funcName == "seed_random") {
00189     visitor.usedModules.insert("io");
00190     if (arguments.size() != 0)
00191         throw std::runtime_error("seed_random requires 0 arguments");
00192     visitor.emit_invoke("seed_random", {});
00193 }
00194 }
00195
00196 bool IOFunctionHandler::generateWithResult(CodeGenVisitor &visitor, const std::string &funcName_,
00197                                           const std::vector<std::unique_ptr<ASTNode>> &arguments)
00198 {
00199     auto funcName = toLower(funcName_);

```

```

00199         if (funcName == "seed_random") {
00200             visitor.usedModules.insert("io");
00201             if (arguments.size() != 0)
00202                 throw std::runtime_error("seed_random requires 0 arguments");
00203             std::string dst = visitor.allocReg();
00204             visitor.emit_invoke("seed_random", {});
00205             visitor.emit("return " + dst);
00206             visitor.pushValue(dst);
00207             return true;
00208         } else if (funcName == "rand_number") {
00209             visitor.usedModules.insert("io");
00210             if (arguments.size() != 1)
00211                 throw std::runtime_error("rand_number requires 1 argument (size)");
00212             std::string arg = visitor.eval(arguments[0].get());
00213             std::string dst = visitor.allocReg();
00214             visitor.emit_invoke("rand_number", {arg});
00215             visitor.emit("return " + dst);
00216             if (visitor.isReg(arg) && !visitor.isParmReg(arg))
00217                 visitor.freeReg(arg);
00218             visitor.pushValue(dst);
00219             return true;
00220         }
00221         return false;
00222     }
00223
00224     bool StdFunctionHandler::canHandle(const std::string &funcName) const {
00225         return stdFunctions.find(toLower(funcName)) != stdFunctions.end();
00226     }
00227
00228     void StdFunctionHandler::generate(CoGenVisitor &visitor, const std::string &funcName,
00229                                     const std::vector<std::unique_ptr<ASTNode> &arguments) {
00230         visitor.usedModules.insert("std");
00231
00232         int lineNum = arguments.empty() ? 1 : arguments[0]->getLineNumber();
00233
00234         std::vector<std::string> args;
00235         for (auto &arg : arguments) {
00236             args.push_back(visitor.eval(arg.get()));
00237         }
00238
00239         if (funcName == "srand") {
00240             if (arguments.size() != 1)
00241                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00242                                         ": srand requires 1 argument");
00243             visitor.emit_invoke("srand", args);
00244         } else if (funcName == "free") {
00245             if (arguments.size() != 1)
00246                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00247                                         ": free requires 1 argument");
00248             visitor.emit_invoke("release", args);
00249         } else if (funcName == "halt") {
00250             if (arguments.size() != 1)
00251                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00252                                         ": halt requires 1 argument");
00253             visitor.emit("exit " + args[0]);
00254         } else if (funcName == "inc") {
00255             if (arguments.size() < 1 || arguments.size() > 2)
00256                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00257                                         ": inc requires 1 or 2 arguments");
00258             std::string amount = (arguments.size() == 2) ? args[1] : "1";
00259             visitor.emit2("add", args[0], amount);
00260         } else if (funcName == "dec") {
00261             if (arguments.size() < 1 || arguments.size() > 2)
00262                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00263                                         ": dec requires 1 or 2 arguments");
00264             std::string amount = (arguments.size() == 2) ? args[1] : "1";
00265             visitor.emit2("sub", args[0], amount);
00266         } else if (funcName == "include") {
00267             if (arguments.size() != 2)
00268                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00269                                         ": include requires 2 arguments (set, element)");
00270             std::string setPtr = visitor.ensurePtrBase(args[0]);
00271             std::string oneReg = visitor.allocReg();
00272             visitor.emit2("mov", oneReg, "1");
00273             visitor.emit4("store", oneReg, setPtr, args[1], "8");
00274             visitor.freeReg(oneReg);
00275         } else if (funcName == "exclude") {
00276             if (arguments.size() != 2)
00277                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00278                                         ": exclude requires 2 arguments (set, element)");
00279             std::string setPtr = visitor.ensurePtrBase(args[0]);
00280             std::string zeroReg = visitor.allocReg();
00281             visitor.emit2("mov", zeroReg, "0");
00282             visitor.emit4("store", zeroReg, setPtr, args[1], "8");
00283             visitor.freeReg(zeroReg);
00284         }
00285     }

```

```

00286         for (const std::string &arg : args) {
00287             if (visitor.isReg(arg))
00288                 visitor.freeReg(arg);
00289         }
00290     }
00291
00292     bool StdFunctionHandler::generateWithResult(CoGenVisitor &visitor, const std::string &funcName,
00293                                                const std::vector<std::unique_ptr<ASTNode> &arguments)
00294     {
00295         visitor.usedModules.insert("std");
00296
00297         int lineNum = arguments.empty() ? 1 : arguments[0]->getLineNumber();
00298
00299         std::unordered_set<std::string> floatMathFuncs = {
00300             "sin", "cos", "tan", "asin", "acos", "atan", "sinh", "cosh", "tanh",
00301             "exp", "log", "log10", "sqrt", "pow", "fmod", "ceil", "floor", "fabs",
00302             "round", "trunc", "hypot"};
00303
00304         if (floatMathFuncs.find(funcName) != floatMathFuncs.end()) {
00305             int requiredArgs = (funcName == "pow" || funcName == "fmod" || funcName == "hypot") ? 2 :
1;
00306             if (static_cast<int>(arguments.size()) != requiredArgs)
00307                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
+ " argument(s)");
00308
00309             std::vector<std::string> args;
00310             args.reserve(arguments.size());
00311
00312             for (const auto &argNode : arguments) {
00313                 std::string argLocation = visitor.eval(argNode.get());
00314                 auto argType = visitor.getExpressionType(argNode.get());
00315
00316                 if (argType != VarType::DOUBLE) {
00317                     std::string floatReg = visitor.allocFloatReg();
00318                     visitor.emit2("to_float", floatReg, argLocation);
00319                     if (visitor.isReg(argLocation))
00320                         visitor.freeReg(argLocation);
00321                     args.push_back(floatReg);
00322                 } else {
00323                     if (!visitor.isFloatReg(argLocation)) {
00324                         std::string floatReg = visitor.allocFloatReg();
00325                         visitor.emit2("mov", floatReg, argLocation);
00326                         args.push_back(floatReg);
00327                     } else {
00328                         args.push_back(argLocation);
00329                     }
00330                 }
00331             }
00332
00333             visitor.emit_invoke(funcName, args);
00334             std::string resultReg = visitor.allocFloatReg();
00335             visitor.emit("return " + resultReg);
00336             visitor.pushValue(resultReg);
00337             for (const std::string &fr : args) {
00338                 if (visitor.isFloatReg(fr)) {
00339                     visitor.freeFloatReg(fr);
00340                 }
00341             }
00342             return true;
00343         } else if (funcName == "abs") {
00344             if (arguments.size() != 1)
00345                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
+ " abs requires 1 argument");
00346
00347             std::string a = visitor.eval(arguments[0].get());
00348
00349             std::string resultReg = visitor.allocReg();
00350             visitor.emit_invoke("abs", {a});
00351             visitor.emit("return " + resultReg);
00352             visitor.pushValue(resultReg);
00353             if (visitor.isReg(a))
00354                 visitor.freeReg(a);
00355             return true;
00356         } else if (funcName == "rand") {
00357             if (arguments.size() != 0)
00358                 throw std::runtime_error("Error on line " + std::to_string(lineNum) +
+ " rand requires 0 arguments");
00359
00360             visitor.emit_invoke("rand", {});
00361             std::string resultReg = visitor.allocReg();
00362             visitor.emit("return " + resultReg);
00363             visitor.pushValue(resultReg);
00364             return true;
00365         } else if (funcName == "float" || funcName == "real") {
00366             if (arguments.size() != 1) {
00367                 throw std::runtime_error("float/real expects one argument");
00368             }
00369             std::string argLocation = visitor.eval(arguments[0].get());

```



```

00370
00371     auto argType = visitor.getExpressionType(arguments[0].get());
00372     if (argType == VarType::DOUBLE) {
00373         visitor.pushValue(argLocation);
00374         return true;
00375     }
00376
00377     std::string floatReg = visitor.allocFloatReg();
00378     visitor.emit2("to_float", floatReg, argLocation);
00379     visitor.pushValue(floatReg);
00380     if (visitor.isReg(argLocation))
00381         visitor.freeReg(argLocation);
00382     return true;
00383 } else if (funcName == "ord") {
00384     if (arguments.size() != 1)
00385         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00386                                 ": ord requires 1 argument");
00387     std::string a = visitor.eval(arguments[0].get());
00388     // ord() is identity for integers/chars/enums -- value is already an ordinal
00389     visitor.pushValue(a);
00390     return true;
00391 } else if (funcName == "chr") {
00392     if (arguments.size() != 1)
00393         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00394                                 ": chr requires 1 argument");
00395     std::string a = visitor.eval(arguments[0].get());
00396     // chr() is identity -- just changes interpretation to char
00397     visitor.pushValue(a);
00398     return true;
00399 } else if (funcName == "succ") {
00400     if (arguments.size() != 1)
00401         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00402                                 ": succ requires 1 argument");
00403     std::string a = visitor.eval(arguments[0].get());
00404     std::string r = visitor.allocReg();
00405     visitor.emit2("mov", r, a);
00406     visitor.emit2("add", r, "1");
00407     if (visitor.isReg(a) && !visitor.isParmReg(a))
00408         visitor.freeReg(a);
00409     visitor.pushValue(r);
00410     return true;
00411 } else if (funcName == "pred") {
00412     if (arguments.size() != 1)
00413         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00414                                 ": pred requires 1 argument");
00415     std::string a = visitor.eval(arguments[0].get());
00416     std::string r = visitor.allocReg();
00417     visitor.emit2("mov", r, a);
00418     visitor.emit2("sub", r, "1");
00419     if (visitor.isReg(a) && !visitor.isParmReg(a))
00420         visitor.freeReg(a);
00421     visitor.pushValue(r);
00422     return true;
00423 }
00424
00425     return false;
00426 }
00427
00428 bool SDLFunctionHandler::canHandle(const std::string &funcName) const {
00429     return sdlFunctions.count(toLower(funcName)) != 0;
00430 }
00431
00432 void SDLFunctionHandler::generate(CoGenVisitor &visitor, const std::string &funcName, const
std::vector<std::unique_ptr<ASTNode> &arguments) {
00433     visitor.usedModules.insert("sdl");
00434
00435     int lineNum = arguments.empty() ? 1 : arguments[0]->getLineNumber();
00436
00437     std::vector<std::string> args;
00438     args.reserve(arguments.size());
00439     for (auto &arg : arguments)
00440         args.push_back(visitor.eval(arg.get()));
00441
00442     auto freeArgs = [&]() {
00443         for (const std::string &a : args)
00444             if (visitor.isReg(a))
00445                 visitor.freeReg(a);
00446     };
00447
00448     if (funcName == "sdl_init") {
00449         if (arguments.size() != 0)
00450             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00451                                     ": sdl_init requires 0 arguments");
00452         visitor.emit_invoke("init", {});
00453     } else if (funcName == "sdl_quit") {
00454         if (arguments.size() != 0)
00455             throw std::runtime_error("Error on line " + std::to_string(lineNum) +

```

```

00456         ": sdl_quit requires 0 arguments");
00457     visitor.emit_invoke("quit", {});
00458 } else if (funcName == "sdl_destroy_window") {
00459     if (arguments.size() != 1)
00460         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00461             ": sdl_destroy_window requires 1 argument (window_id)");
00462     visitor.emit_invoke("destroy_window", args);
00463 } else if (funcName == "sdl_destroy_renderer") {
00464     if (arguments.size() != 1)
00465         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00466             ": sdl_destroy_renderer requires 1 argument (renderer_id)");
00467     visitor.emit_invoke("destroy_renderer", args);
00468 } else if (funcName == "sdl_set_window_title") {
00469     if (arguments.size() != 2)
00470         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00471             ": sdl_set_window_title requires 2 arguments (window_id,
00472 title)");
00472     visitor.emit_invoke("set_window_title", args);
00473 } else if (funcName == "sdl_set_window_position") {
00474     if (arguments.size() != 3)
00475         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00476             ": sdl_set_window_position requires 3 arguments (window_id,
00477 x, y)");
00477     visitor.emit_invoke("set_window_position", args);
00478 } else if (funcName == "sdl_get_window_size") {
00479     if (arguments.size() != 3)
00480         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00481             ": sdl_get_window_size requires 3 arguments (window_id,
00482 w_var, h_var)");
00482     visitor.emit_invoke("get_window_size", args);
00483 } else if (funcName == "sdl_set_window_fullscreen") {
00484     if (arguments.size() != 2)
00485         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00486             ": sdl_set_window_fullscreen requires 2 arguments (window_id,
00487 fullscreen)");
00487     visitor.emit_invoke("set_window_fullscreen", args);
00488 } else if (funcName == "sdl_set_window_icon") {
00489     if (arguments.size() != 2)
00490         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00491             ": sdl_set_window_icon requires 2 arguments (window_id,
00492 path)");
00492     visitor.emit_invoke("set_window_icon", args);
00493 } else if (funcName == "sdl_set_draw_color") {
00494     if (arguments.size() != 5)
00495         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00496             ": sdl_set_draw_color requires 5 arguments (renderer_id, r,
00497 g, b, a)");
00497     visitor.emit_invoke("set_draw_color", args);
00498 } else if (funcName == "sdl_clear") {
00499     if (arguments.size() != 1)
00500         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00501             ": sdl_clear requires 1 argument (renderer_id)");
00502     visitor.emit_invoke("clear", args);
00503 } else if (funcName == "sdl_present") {
00504     if (arguments.size() != 1)
00505         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00506             ": sdl_present requires 1 argument (renderer_id)");
00507     visitor.emit_invoke("present", args);
00508 } else if (funcName == "sdl_draw_point") {
00509     if (arguments.size() != 3)
00510         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00511             ": sdl_draw_point requires 3 arguments (renderer_id, x, y)");
00512     visitor.emit_invoke("draw_point", args);
00513 } else if (funcName == "sdl_draw_line") {
00514     if (arguments.size() != 5)
00515         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00516             ": sdl_draw_line requires 5 arguments (renderer_id, x1, y1,
00517 x2, y2)");
00517     visitor.emit_invoke("draw_line", args);
00518 } else if (funcName == "sdl_draw_rect") {
00519     if (arguments.size() != 5)
00520         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00521             ": sdl_draw_rect requires 5 arguments (renderer_id, x, y, w,
00522 h)");
00522     visitor.emit_invoke("draw_rect", args);
00523 } else if (funcName == "sdl_fill_rect") {
00524     if (arguments.size() != 5)
00525         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00526             ": sdl_fill_rect requires 5 arguments (renderer_id, x, y, w,
00527 h)");
00527     visitor.emit_invoke("fill_rect", args);
00528 } else if (funcName == "sdl_get_mouse_state") {
00529     if (arguments.size() != 2)
00530         throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00531             ": sdl_get_mouse_state requires 2 arguments (x_var, y_var)");
00532     visitor.emit_invoke("get_mouse_state", args);
00533 } else if (funcName == "sdl_get_relative_mouse_state") {

```

```

00534         if (arguments.size() != 2)
00535             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00536                                     ": sdl_get_relative_mouse_state requires 2 arguments (x_var,
y_var)");
00537         visitor.emit_invoke("get_relative_mouse_state", args);
00538     } else if (funcName == "sdl_get_keyboard_state") {
00539         if (arguments.size() != 1)
00540             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00541                                     ": sdl_get_keyboard_state requires 1 argument
(numkeys_var)");
00542         visitor.emit_invoke("get_keyboard_state", args);
00543     } else if (funcName == "sdl_show_cursor") {
00544         if (arguments.size() != 1)
00545             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00546                                     ": sdl_show_cursor requires 1 argument (show)");
00547         visitor.emit_invoke("show_cursor", args);
00548     } else if (funcName == "sdl_free_surface") {
00549         if (arguments.size() != 1)
00550             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00551                                     ": sdl_free_surface requires 1 argument (surf_ptr)");
00552         visitor.emit_invoke("free_surface", args);
00553     } else if (funcName == "sdl_set_clipboard_text") {
00554         if (arguments.size() != 1)
00555             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00556                                     ": sdl_set_clipboard_text requires 1 argument (text)");
00557         visitor.emit_invoke("set_clipboard_text", args);
00558     } else if (funcName == "sdl_destroy_texture") {
00559         if (arguments.size() != 1)
00560             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00561                                     ": sdl_destroy_texture requires 1 argument (texture_id)");
00562         visitor.emit_invoke("destroy_texture", args);
00563     } else if (funcName == "sdl_render_texture") {
00564         if (arguments.size() != 10)
00565             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00566                                     ": sdl_render_texture requires 10 arguments (renderer_id,
texture_id, src_x, src_y, src_w, src_h, dst_x, dst_y, dst_w, dst_h)");
00567         visitor.emit_invoke("render_texture", args);
00568     } else if (funcName == "sdl_delay") {
00569         if (arguments.size() != 1)
00570             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00571                                     ": sdl_delay requires 1 argument (ms)");
00572         visitor.emit_invoke("delay", args);
00573     } else if (funcName == "sdl_close_audio") {
00574         if (arguments.size() != 0)
00575             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00576                                     ": sdl_close_audio requires 0 arguments");
00577         visitor.emit_invoke("close_audio", {});
00578     } else if (funcName == "sdl_pause_audio") {
00579         if (arguments.size() != 1)
00580             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00581                                     ": sdl_pause_audio requires 1 argument (pause_on)");
00582         visitor.emit_invoke("pause_audio", args);
00583     } else if (funcName == "sdl_free_wav") {
00584         if (arguments.size() != 1)
00585             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00586                                     ": sdl_free_wav requires 1 argument (audio_buf)");
00587         visitor.emit_invoke("free_wav", args);
00588     } else if (funcName == "sdl_queue_audio") {
00589         if (arguments.size() != 2)
00590             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00591                                     ": sdl_queue_audio requires 2 arguments (data, len)");
00592         visitor.emit_invoke("queue_audio", args);
00593     } else if (funcName == "sdl_clear_queued_audio") {
00594         if (arguments.size() != 0)
00595             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00596                                     ": sdl_clear_queued_audio requires 0 arguments");
00597         visitor.emit_invoke("clear_queued_audio", {});
00598     } else if (funcName == "sdl_update_texture") {
00599         if (arguments.size() != 3)
00600             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00601                                     ": sdl_update_texture requires 3 arguments (texture_id,
pixels, pitch)");
00602         visitor.emit_invoke("update_texture", args);
00603     } else if (funcName == "sdl_unlock_texture") {
00604         if (arguments.size() != 1)
00605             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00606                                     ": sdl_unlock_texture requires 1 argument (texture_id)");
00607         visitor.emit_invoke("unlock_texture", args);
00608     } else if (funcName == "sdl_init_text") {
00609         if (arguments.size() != 0)
00610             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00611                                     ": sdl_init_text requires 0 arguments");
00612         visitor.emit_invoke("init_text", {});
00613     } else if (funcName == "sdl_quit_text") {
00614         if (arguments.size() != 0)
00615             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00616                                     ": sdl_quit_text requires 0 arguments");

```

```

00617         visitor.emit_invoke("quit_text", {});
00618     } else if (funcName == "sdl_draw_text") {
00619         if (arguments.size() != 9)
00620             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00621                                     ": sdl_draw_text requires 9 arguments (renderer_id, font_id,
text, x, y, r, g, b, a)");
00622         visitor.emit_invoke("draw_text", args);
00623     } else if (funcName == "sdl_set_render_target") {
00624         if (arguments.size() != 2)
00625             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00626                                     ": sdl_set_render_target requires 2 arguments (renderer_id,
target_id)");
00627         visitor.emit_invoke("set_render_target", args);
00628     } else if (funcName == "sdl_destroy_render_target") {
00629         if (arguments.size() != 1)
00630             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00631                                     ": sdl_destroy_render_target requires 1 argument
(target_id)");
00632         visitor.emit_invoke("destroy_render_target", args);
00633     } else if (funcName == "sdl_present_scaled") {
00634         if (arguments.size() != 4)
00635             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00636                                     ": sdl_present_scaled requires 4 arguments (renderer_id,
target_id, src_width, src_height)");
00637         visitor.emit_invoke("present_scaled", args);
00638     } else if (funcName == "sdl_present_stretched") {
00639         if (arguments.size() != 4)
00640             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00641                                     ": sdl_present_stretched requires 4 arguments (renderer_id,
target_id, dst_width, dst_height)");
00642         visitor.emit_invoke("present_stretched", args);
00643     } else if (funcName == "sdl_lock_texture") {
00644         if (arguments.size() != 3)
00645             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00646                                     ": sdl_lock_texture requires 3 arguments (texture_id,
pixels_var, pitch_var)");
00647         visitor.emit_invoke("lock_texture", args);
00648     }
00649
00650     freeArgs();
00651 }
00652
00653 bool SDLFunctionHandler::generateWithResult(CoGenVisitor &visitor, const std::string &funcName,
const std::vector<std::unique_ptr<ASTNode> &arguments) {
00654     visitor.usedModules.insert("sdl");
00655
00656     int lineNum = arguments.empty() ? 1 : arguments[0]->getLineNumber();
00657
00658     std::vector<std::string> args;
00659     args.reserve(arguments.size());
00660     for (auto &arg : arguments)
00661         args.push_back(visitor.eval(arg.get()));
00662
00663     auto freeArgs = [&]() {
00664         for (const std::string &a : args)
00665             if (visitor.isReg(a))
00666                 visitor.freeReg(a);
00667     };
00668
00669     auto emitCallWithReturn = [&](const std::string &name) -> std::string {
00670         std::string dst = visitor.allocReg();
00671         visitor.emit_invoke(name, args);
00672         visitor.emit("return " + dst);
00673         return dst;
00674     };
00675
00676     if (funcName == "sdl_init") {
00677         if (arguments.size() != 0)
00678             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00679                                     ": sdl_init requires 0 arguments");
00680         std::string r = emitCallWithReturn("init");
00681         visitor.pushValue(r);
00682         return true;
00683     } else if (funcName == "sdl_create_window") {
00684         if (arguments.size() != 6)
00685             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00686                                     ": sdl_create_window requires 6 arguments (title, x, y, w, h,
flags)");
00687         std::string r = emitCallWithReturn("create_window");
00688         visitor.pushValue(r);
00689         freeArgs();
00690         return true;
00691     } else if (funcName == "sdl_create_renderer") {
00692         if (arguments.size() != 3)
00693             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00694                                     ": sdl_create_renderer requires 3 arguments (window_id,
index, flags)");

```

```

00695         std::string r = emitCallWithReturn("create_renderer");
00696         visitor.pushValue(r);
00697         freeArgs();
00698         return true;
00699     } else if (funcName == "sdl_poll_event") {
00700         if (arguments.size() != 0)
00701             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00702                                     " : sdl_poll_event requires 0 arguments");
00703         std::string r = emitCallWithReturn("poll_event");
00704         visitor.pushValue(r);
00705         return true;
00706     } else if (funcName == "sdl_get_event_type") {
00707         if (arguments.size() != 0)
00708             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00709                                     " : sdl_get_event_type requires 0 arguments");
00710         std::string r = emitCallWithReturn("get_event_type");
00711         visitor.pushValue(r);
00712         return true;
00713     } else if (funcName == "sdl_get_key_code") {
00714         if (arguments.size() != 0)
00715             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00716                                     " : sdl_get_key_code requires 0 arguments");
00717         std::string r = emitCallWithReturn("get_key_code");
00718         visitor.pushValue(r);
00719         return true;
00720     } else if (funcName == "sdl_get_mouse_x") {
00721         if (arguments.size() != 0)
00722             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00723                                     " : sdl_get_mouse_x requires 0 arguments");
00724         std::string r = emitCallWithReturn("get_mouse_x");
00725         visitor.pushValue(r);
00726         return true;
00727     } else if (funcName == "sdl_get_mouse_y") {
00728         if (arguments.size() != 0)
00729             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00730                                     " : sdl_get_mouse_y requires 0 arguments");
00731         std::string r = emitCallWithReturn("get_mouse_y");
00732         visitor.pushValue(r);
00733         return true;
00734     } else if (funcName == "sdl_get_mouse_button") {
00735         if (arguments.size() != 0)
00736             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00737                                     " : sdl_get_mouse_button requires 0 arguments");
00738         std::string r = emitCallWithReturn("get_mouse_button");
00739         visitor.pushValue(r);
00740         return true;
00741     } else if (funcName == "sdl_get_mouse_buttons") {
00742         if (arguments.size() != 0)
00743             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00744                                     " : sdl_get_mouse_buttons requires 0 arguments");
00745         std::string r = emitCallWithReturn("get_mouse_buttons");
00746         visitor.pushValue(r);
00747         return true;
00748     } else if (funcName == "sdl_get_relative_mouse_x") {
00749         if (arguments.size() != 0)
00750             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00751                                     " : sdl_get_relative_mouse_x requires 0 arguments");
00752         std::string r = emitCallWithReturn("get_relative_mouse_x");
00753         visitor.pushValue(r);
00754         return true;
00755     } else if (funcName == "sdl_get_relative_mouse_y") {
00756         if (arguments.size() != 0)
00757             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00758                                     " : sdl_get_relative_mouse_y requires 0 arguments");
00759         std::string r = emitCallWithReturn("get_relative_mouse_y");
00760         visitor.pushValue(r);
00761         return true;
00762     } else if (funcName == "sdl_get_relative_mouse_buttons") {
00763         if (arguments.size() != 0)
00764             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00765                                     " : sdl_get_relative_mouse_buttons requires 0 arguments");
00766         std::string r = emitCallWithReturn("get_relative_mouse_buttons");
00767         visitor.pushValue(r);
00768         return true;
00769     } else if (funcName == "sdl_is_key_pressed") {
00770         if (arguments.size() != 1)
00771             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00772                                     " : sdl_is_key_pressed requires 1 argument (scancode)");
00773         std::string r = emitCallWithReturn("is_key_pressed");
00774         visitor.pushValue(r);
00775         freeArgs();
00776         return true;
00777     } else if (funcName == "sdl_get_num_keys") {
00778         if (arguments.size() != 0)
00779             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00780                                     " : sdl_get_num_keys requires 0 arguments");
00781         std::string r = emitCallWithReturn("get_num_keys");

```

```

00782         visitor.pushValue(r);
00783         return true;
00784     } else if (funcName == "sdl_get_clipboard_text") {
00785         if (arguments.size() != 0)
00786             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00787                                     "": sdl_get_clipboard_text requires 0 arguments");
00788         std::string r = emitCallWithReturn("get_clipboard_text");
00789         visitor.pushValue(r);
00790         return true;
00791     } else if (funcName == "sdl_create_rgb_surface") {
00792         if (arguments.size() != 3)
00793             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00794                                     "": sdl_create_rgb_surface requires 3 arguments (width,
height, depth)");
00795         std::string r = emitCallWithReturn("create_rgb_surface");
00796         visitor.pushValue(r);
00797         freeArgs();
00798         return true;
00799     } else if (funcName == "sdl_blit_surface") {
00800         if (arguments.size() != 4)
00801             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00802                                     "": sdl_blit_surface requires 4 arguments (src_ptr, dst_ptr,
x, y)");
00803         std::string r = emitCallWithReturn("blit_surface");
00804         visitor.pushValue(r);
00805         freeArgs();
00806         return true;
00807     } else if (funcName == "sdl_create_texture") {
00808         if (arguments.size() != 5)
00809             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00810                                     "": sdl_create_texture requires 5 arguments (renderer_id,
format, access, w, h)");
00811         std::string r = emitCallWithReturn("create_texture");
00812         visitor.pushValue(r);
00813         freeArgs();
00814         return true;
00815     } else if (funcName == "sdl_load_texture") {
00816         if (arguments.size() != 2)
00817             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00818                                     "": sdl_load_texture requires 2 arguments (renderer_id,
file_path)");
00819         std::string r = emitCallWithReturn("load_texture");
00820         visitor.pushValue(r);
00821         freeArgs();
00822         return true;
00823     } else if (funcName == "sdl_load_texture_color_key") {
00824         if (arguments.size() != 2)
00825             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00826                                     "": sdl_load_texture_color_key requires 2 arguments
(renderer_id, file_path)");
00827         std::string r = emitCallWithReturn("load_texture_color_key");
00828         visitor.pushValue(r);
00829         freeArgs();
00830         return true;
00831     } else if (funcName == "sdl_load_texture_color_key_rgb") {
00832         if (arguments.size() != 5)
00833             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00834                                     "": sdl_load_texture_color_key_rgb requires 5 arguments
(renderer_id, file_path, r, g, b)");
00835         std::string r = emitCallWithReturn("load_texture_color_key_rgb");
00836         visitor.pushValue(r);
00837         freeArgs();
00838         return true;
00839     } else if (funcName == "sdl_get_ticks") {
00840         if (arguments.size() != 0)
00841             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00842                                     "": sdl_get_ticks requires 0 arguments");
00843         std::string r = emitCallWithReturn("get_ticks");
00844         visitor.pushValue(r);
00845         return true;
00846     } else if (funcName == "sdl_open_audio") {
00847         if (arguments.size() != 4)
00848             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00849                                     "": sdl_open_audio requires 4 arguments (freq, format,
channels, samples)");
00850         std::string r = emitCallWithReturn("open_audio");
00851         visitor.pushValue(r);
00852         freeArgs();
00853         return true;
00854     } else if (funcName == "sdl_load_wav") {
00855         if (arguments.size() != 4)
00856             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00857                                     "": sdl_load_wav requires 4 arguments (file_path,
audio_buf_var, audio_len_var, audio_spec_var)");
00858         std::string r = emitCallWithReturn("load_wav");
00859         visitor.pushValue(r);
00860         freeArgs();

```

```

00861         return true;
00862     } else if (funcName == "sdl_get_queued_audio_size") {
00863         if (arguments.size() != 0)
00864             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00865                                     " : sdl_get_queued_audio_size requires 0 arguments");
00866         std::string r = emitCallWithReturn("get_queued_audio_size");
00867         visitor.pushValue(r);
00868         return true;
00869     } else if (funcName == "sdl_init_text") {
00870         if (arguments.size() != 0)
00871             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00872                                     " : sdl_init_text requires 0 arguments");
00873         std::string r = emitCallWithReturn("init_text");
00874         visitor.pushValue(r);
00875         return true;
00876     } else if (funcName == "sdl_load_font") {
00877         if (arguments.size() != 2)
00878             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00879                                     " : sdl_load_font requires 2 arguments (file, ptsize)");
00880         std::string r = emitCallWithReturn("load_font");
00881         visitor.pushValue(r);
00882         freeArgs();
00883         return true;
00884     } else if (funcName == "sdl_create_render_target") {
00885         if (arguments.size() != 3)
00886             throw std::runtime_error("Error on line " + std::to_string(lineNum) +
00887                                     " : sdl_create_render_target requires 3 arguments
(render_id, width, height)");
00888         std::string r = emitCallWithReturn("create_render_target");
00889         visitor.pushValue(r);
00890         freeArgs();
00891         return true;
00892     }
00893
00894     return false;
00895 }
00896
00897 bool StringFunctionHandler::canHandle(const std::string &f) const {
00898     static const std::unordered_set<std::string> funcs = {
00899         "length", "pos", "copy", "insert", "delete", "inttostr", "strtoint"};
00900     return funcs.count(toLower(f)) != 0;
00901 }
00902
00903 VarType StringFunctionHandler::getReturnType(const std::string &f_) const {
00904     auto f = toLower(f_);
00905     if (f == "length" || f == "pos" || f == "strtoint")
00906         return VarType::INT;
00907     if (f == "copy" || f == "insert" || f == "delete" || f == "inttostr")
00908         return VarType::PTR;
00909     return VarType::UNKNOWN;
00910 }
00911
00912 void StringFunctionHandler::generate(CoGenVisitor &v,
00913                                     const std::string &f,
00914                                     const std::vector<std::unique_ptr<ASTNode>> &args) {
00915     generateWithResult(v, f, args);
00916 }
00917
00918 bool StringFunctionHandler::generateWithResult(CoGenVisitor &v,
00919                                                const std::string &f,
00920                                                const std::vector<std::unique_ptr<ASTNode>>
&arguments) {
00921     v.usedModules.insert("string");
00922     std::vector<std::string> a;
00923     a.reserve(arguments.size());
00924     for (auto &n : arguments)
00925         a.push_back(v.eval(n.get()));
00926
00927     auto freeArgs = [&]() {
00928         for (auto &x : a)
00929             if (v.isReg(x) && !v.isParmReg(x))
00930                 v.freeReg(x);
00931     };
00932
00933     auto emitIntRet = [&](const std::string &name) {
00934         v.emit_invoke(name, a);
00935         std::string r = v.allocReg();
00936         v.emit("return " + r);
00937         v.pushValue(r);
00938     };
00939
00940     auto emitPtrRet = [&](const std::string &name) {
00941         v.emit_invoke(name, a);
00942         std::string r = v.allocTempPtr();
00943         v.emit("return " + r);
00944         v.pushValue(r);
00945         v.markAllocatedPtr(r);

```



```

00946     };
00947
00948     if (f == "length") {
00949         if (a.size() != 1)
00950             throw std::runtime_error("length expects 1 arg");
00951         emitIntRet("strlen");
00952     } else if (f == "pos") {
00953         if (a.size() != 2)
00954             throw std::runtime_error("pos expects 2 args");
00955         emitIntRet("strfind");
00956     } else if (f == "strtoint") {
00957         if (a.size() != 1)
00958             throw std::runtime_error("strtoint expects 1 arg");
00959         emitIntRet("strtoint");
00960     } else if (f == "inttostr") {
00961         if (a.size() != 1)
00962             throw std::runtime_error("inttostr expects 1 arg");
00963         emitPtrRet("inttostr");
00964     } else if (f == "copy") {
00965         if (a.size() != 3)
00966             throw std::runtime_error("copy expects 3 args");
00967         emitPtrRet("copy");
00968     } else if (f == "insert") {
00969         if (a.size() != 3)
00970             throw std::runtime_error("insert expects 3 args");
00971         emitPtrRet("insert");
00972     } else if (f == "delete") {
00973         if (a.size() != 3)
00974             throw std::runtime_error("delete expects 3 args");
00975         emitPtrRet("delete");
00976     } else {
00977         freeArgs();
00978         return false;
00979     }
00980
00981     freeArgs();
00982     return true;
00983 }
00984
00985 static std::unordered_set<std::string> fileFunctions = {
00986     "assign", "reset", "rewrite", "append", "close", "eof", "eoln";
00987
00988 bool FileFunctionHandler::canHandle(const std::string &funcName) const {
00989     return fileFunctions.find(toLower(funcName)) != fileFunctions.end();
00990 }
00991
00992 void FileFunctionHandler::generate(CoGenVisitor &visitor, const std::string &funcName_,
00993     const std::vector<std::unique_ptr<ASTNode> &arguments) {
00994     auto funcName = toLower(funcName_);
00995     visitor.usedModules.insert("io");
00996
00997     if (arguments.empty())
00998         throw std::runtime_error(funcName + " requires at least 1 argument");
00999
01000     // Get the file variable name from the first argument
01001     auto *fileVarNode = dynamic_cast<VariableNode> *(arguments[0].get());
01002     if (!fileVarNode)
01003         throw std::runtime_error(funcName + " first argument must be a file variable");
01004
01005     std::string fileVarName = fileVarNode->name;
01006     std::string mangledFile = visitor.findMangledName(fileVarName);
01007     std::string fileSlot = visitor.eval(arguments[0].get());
01008
01009     if (funcName == "assign") {
01010         if (arguments.size() != 2)
01011             throw std::runtime_error("assign requires 2 arguments (file, filename)");
01012         std::string filenameVal = visitor.eval(arguments[1].get());
01013         // Store filename in companion variable
01014         auto fnIt = visitor.fileVarNames.find(mangledFile);
01015         if (fnIt == visitor.fileVarNames.end())
01016             fnIt = visitor.fileVarNames.find(fileVarName);
01017         if (fnIt != visitor.fileVarNames.end()) {
01018             auto slotIt = visitor.varSlot.find(fnIt->second);
01019             if (slotIt != visitor.varSlot.end())
01020                 visitor.emit2("mov", visitor.slotVar(slotIt->second), filenameVal);
01021             else
01022                 visitor.emit2("mov", fnIt->second, filenameVal);
01023         }
01024         if (visitor.isReg(filenameVal) && !visitor.isParmReg(filenameVal))
01025             visitor.freeReg(filenameVal);
01026     } else if (funcName == "reset" || funcName == "rewrite" || funcName == "append") {
01027         // Open file: reset='r', rewrite='w', append='a'
01028         std::string mode;
01029         if (funcName == "reset") mode = "r";
01030         else if (funcName == "rewrite") mode = "w";
01031         else mode = "a";
01032

```



```

01033         // Get filename from companion variable
01034         auto fnIt = visitor.fileVarNames.find(mangledFile);
01035         if (fnIt == visitor.fileVarNames.end())
01036             fnIt = visitor.fileVarNames.find(fileVarName);
01037         std::string filenameSlot;
01038         if (fnIt != visitor.fileVarNames.end()) {
01039             auto slotIt = visitor.varSlot.find(fnIt->second);
01040             if (slotIt != visitor.varSlot.end())
01041                 filenameSlot = visitor.slotVar(slotIt->second);
01042             else
01043                 filenameSlot = fnIt->second;
01044         } else {
01045             throw std::runtime_error(funcName + ": file variable not found: " + fileVarName);
01046         }
01047
01048         // Create mode string constant
01049         std::string modeLabel = visitor.internString(mode);
01050
01051         visitor.emit_invoke("fopen", {filenameSlot, modeLabel});
01052         std::string retReg = visitor.allocReg();
01053         visitor.emit("return " + retReg);
01054         // Store the file pointer back into the file variable
01055         auto fSlotIt = visitor.varSlot.find(mangledFile);
01056         if (fSlotIt != visitor.varSlot.end())
01057             visitor.emit2("mov", visitor.slotVar(fSlotIt->second), retReg);
01058         else
01059             visitor.emit2("mov", mangledFile, retReg);
01060         visitor.freeReg(retReg);
01061     } else if (funcName == "close") {
01062         visitor.emit_invoke("fclose", {fileSlot});
01063     }
01064
01065     if (visitor.isReg(fileSlot) && !visitor.isParmReg(fileSlot))
01066         visitor.freeReg(fileSlot);
01067 }
01068
01069 bool FileFunctionHandler::generateWithResult(CoGenVisitor &visitor, const std::string
&funcName_,
01070                                             const std::vector<std::unique_ptr<ASTNode>
&arguments) {
01071     auto funcName = toLower(funcName_);
01072     visitor.usedModules.insert("io");
01073
01074     if (funcName == "eof") {
01075         if (arguments.size() != 1)
01076             throw std::runtime_error("eof requires 1 argument (file variable)");
01077         std::string fileSlot = visitor.eval(arguments[0].get());
01078         visitor.emit_invoke("feof", {fileSlot});
01079         std::string dst = visitor.allocReg();
01080         visitor.emit("return " + dst);
01081         if (visitor.isReg(fileSlot) && !visitor.isParmReg(fileSlot))
01082             visitor.freeReg(fileSlot);
01083         visitor.pushValue(dst);
01084         return true;
01085     } else if (funcName == "eoln") {
01086         // eoln not fully supported, return false as a stub
01087         if (arguments.size() != 1)
01088             throw std::runtime_error("eoln requires 1 argument (file variable)");
01089         std::string dst = visitor.allocReg();
01090         visitor.emit2("mov", dst, "0");
01091         visitor.pushValue(dst);
01092         return true;
01093     }
01094     return false;
01095 }
01096
01097 VarType FileFunctionHandler::getReturnType(const std::string &funcName) const {
01098     auto f = toLower(funcName);
01099     if (f == "eof" || f == "eoln")
01100         return VarType::INT;
01101     return VarType::UNKNOWN;
01102 }
01103
01104 } // namespace pascal

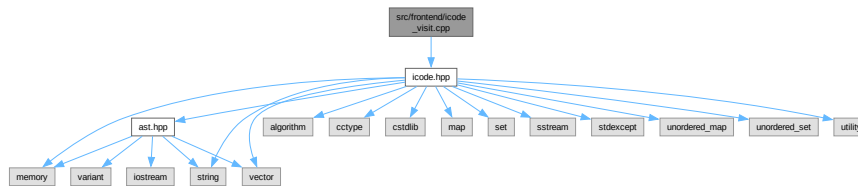
```

12.62 src/frontend/icode_visit.cpp File Reference

Pascal code generator AST visitor method implementations.

```
#include "icode.hpp"
```

Include dependency graph for icode_visit.cpp:



Namespaces

- namespace [pascal](#)

12.62.1 Detailed Description

Pascal code generator AST visitor method implementations.

Author

Jared Bruni

Definition in file [icode_visit.cpp](#).

12.63 icode_visit.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "icode.hpp"
00007
00008 namespace pascal {
00009
00010     CodeGenVisitor::CodeGenVisitor()
00011         : regInUse(registers.size(), false),
00012           ptrRegInUse(ptrRegisters.size(), false) {
00013         initializeFloatRegisters();
00014         initializeBuiltins();
00015     }
00016
00017     void CodeGenVisitor::visit(ProgramNode &node) {
00018         name = node.name;
00019         // Separate native modules from unit dependencies
00020         static const std::unordered_set<std::string> nativeModules = {"io", "std", "string", "sdl",
"strlib"};
00021         for (const auto &mod : node.uses) {
00022             if (mod == "strlib")
00023                 usedModules.insert("string");
00024             else if (nativeModules.count(mod))
00025                 usedModules.insert(mod);
00026             else
00027                 objectDeps.push_back(mod);
00028         }
00029         if (node.block) {
00030             node.block->accept(*this);
00031         }
00032     }
00033
00034     void CodeGenVisitor::visit(UnitNode &node) {
00035         name = node.name;
00036         isUnit = true;
00037         // Process uses clause
00038         static const std::unordered_set<std::string> nativeModules = {"io", "std", "string", "sdl",
"strlib"};
00039         for (const auto &mod : node.uses) {
00040             if (mod == "strlib")
00041                 usedModules.insert("string");
00042             else if (nativeModules.count(mod))
00043                 usedModules.insert(mod);
00044             else
00045                 objectDeps.push_back(mod);

```

```

00046     }
00047
00048     // Emit UNIT_INIT function label -- allocs from interface vars go here
00049     emitLabel("function PROC_UNIT_INIT");
00050
00051     // Register interface declarations' signatures WITHOUT deferring code
00052     // (interface declarations have null blocks -- they are forward declarations)
00053     for (auto &decl : node.interfaceDecls) {
00054         if (!decl)
00055             continue;
00056         if (auto *funcDecl = dynamic_cast<FuncDeclNode *>(decl.get())) {
00057             // Just register the signature info; don't call accept()
00058             FuncInfo funcInfo;
00059             funcInfo.returnType = getTypeFromString(funcDecl->returnType);
00060             for (auto &p : funcDecl->parameters) {
00061                 if (auto pn = dynamic_cast<ParameterNode *>(p.get())) {
00062                     for (size_t i = 0; i < pn->identifiers.size(); ++i) {
00063                         funcInfo.paramTypes.push_back(getTypeFromString(pn->type));
00064                     }
00065                 }
00066             }
00067             funcSignatures[funcDecl->name] = funcInfo;
00068             setVarType(funcDecl->name, funcInfo.returnType);
00069         } else if (dynamic_cast<ConstDeclNode *>(decl.get()) ||
00070                 dynamic_cast<VarDeclNode *>(decl.get()) ||
00071                 dynamic_cast<TypeDeclNode *>(decl.get())) {
00072             // Process interface const/var/type so they are available
00073             // to the unit's own implementation code
00074             decl->accept(*this);
00075         }
00076         // ProcDeclNode forward declarations don't need signature registration
00077         // (they have no return type to track)
00078     }
00079
00080     // Close UNIT_INIT function
00081     emit("ret");
00082
00083     // Process implementation declarations (actual code)
00084     for (auto &decl : node.implDecls) {
00085         if (decl)
00086             decl->accept(*this);
00087     }
00088 }
00089
00090 void CodeGenVisitor::visit(BlockNode &node) {
00091     for (auto &decl : node.declarations) {
00092         if (!decl)
00093             continue;
00094         if (dynamic_cast<TypeDeclNode *>(decl.get())) {
00095             decl->accept(*this);
00096         }
00097     }
00098
00099     for (auto &decl : node.declarations) {
00100         if (!decl)
00101             continue;
00102         if (dynamic_cast<TypeDeclNode *>(decl.get()))
00103             continue;
00104
00105         if (generatingDeferredCode) {
00106             if (dynamic_cast<ProcDeclNode *>(decl.get()) ||
00107                 dynamic_cast<FuncDeclNode *>(decl.get())) {
00108                 continue;
00109             }
00110         }
00111         decl->accept(*this);
00112     }
00113
00114     if (node.compoundStatement) {
00115         node.compoundStatement->accept(*this);
00116     }
00117 }
00118
00119 void CodeGenVisitor::visit(VarDeclNode &node) {
00120     for (const auto &varName : node.identifiers) {
00121         std::string mangledName = mangleVariableName(varName);
00122         std::string typeName;
00123
00124         if (std::holds_alternative<std::string>(node.type)) {
00125             typeName = std::get<std::string>(node.type);
00126         } else if (std::holds_alternative<std::unique_ptr<ASTNode>>(node.type)) {
00127             auto &typeNode = std::get<std::unique_ptr<ASTNode>>(node.type);
00128
00129             if (auto *arrayTypeNode = dynamic_cast<ArrayTypeNode *>(typeNode.get())) {
00130                 ArrayInfo info = buildArrayInfoFromNode(arrayTypeNode);
00131                 arrayInfo[mangledName] = std::move(info);
00132             }

```

```

00133         setVarType(varName, VarType::PTR);
00134         int slot = newSlotFor(mangledName);
00135         setSlotType(slot, VarType::PTR);
00136         varSlot[varName] = slot;
00137         varSlot[mangledName] = slot;
00138
00139         updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00140
00141         if (arrayInfo[mangledName].isDynamic) {
00142             // Dynamic array: no alloc at declaration, starts as null
00143             // Create companion length variable
00144             std::string lenName = mangledName + "_dynlen";
00145             int lenSlot = newSlotFor(lenName);
00146             setSlotType(lenSlot, VarType::INT);
00147             varSlot[lenName] = lenSlot;
00148             updateDataSectionInitialValue(slotVar(lenSlot), "int", "0");
00149             dynArrayLenSlot[mangledName] = lenSlot;
00150
00151             std::string currentScope = getCurrentScopeName();
00152             if (currentScope.empty())
00153                 globalArrays.push_back(slotVar(slot));
00154             else
00155                 functionScopedArrays[currentScope].push_back(slotVar(slot));
00156         } else {
00157             emit3("alloc",
00158                 slotVar(slot),
00159                 std::to_string(arrayInfo[mangledName].elementSize),
00160                 std::to_string(arrayInfo[mangledName].size));
00161
00162             std::string currentScope = getCurrentScopeName();
00163             if (currentScope.empty())
00164                 globalArrays.push_back(slotVar(slot));
00165             else
00166                 functionScopedArrays[currentScope].push_back(slotVar(slot));
00167         }
00168         continue;
00169     }
00170
00171     if (auto *ptrTypeNode = dynamic_cast<PointerTypeNode *>(typeNode.get())) {
00172         setVarType(varName, VarType::PTR);
00173         int slot = newSlotFor(mangledName);
00174         setSlotType(slot, VarType::PTR);
00175         varSlot[varName] = slot;
00176         varSlot[mangledName] = slot;
00177         pointerBaseType[mangledName] = ptrTypeNode->baseTypeName;
00178         pointerBaseType[varName] = ptrTypeNode->baseTypeName;
00179         updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00180         continue;
00181     }
00182
00183     if (dynamic_cast<SetTypeNode *>(typeNode.get())) {
00184         setVarType(varName, VarType::PTR);
00185         int slot = newSlotFor(mangledName);
00186         setSlotType(slot, VarType::PTR);
00187         varSlot[varName] = slot;
00188         varSlot[mangledName] = slot;
00189         setVars.insert(mangledName);
00190         setVars.insert(varName);
00191         updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00192         // Allocate 256 bytes (one byte per ordinal 0..255)
00193         emit3("alloc", slotVar(slot), "8", "256");
00194         // Zero-fill the set
00195         std::string loopIdx = allocReg();
00196         std::string loopStart = newLabel("SET_INIT");
00197         std::string loopEnd = newLabel("SET_INIT_END");
00198         emit2("mov", loopIdx, "0");
00199         emitLabel(loopStart);
00200         emit2("cmp", loopIdx, "256");
00201         emit1("jge", loopEnd);
00202         emit4("store", "0", slotVar(slot), loopIdx, "8");
00203         emit2("add", loopIdx, "1");
00204         emit1("jmp", loopStart);
00205         emitLabel(loopEnd);
00206         freeReg(loopIdx);
00207
00208         std::string currentScope = getCurrentScopeName();
00209         if (currentScope.empty())
00210             globalArrays.push_back(slotVar(slot));
00211         else
00212             functionScopedArrays[currentScope].push_back(slotVar(slot));
00213         continue;
00214     }
00215
00216     if (auto *simpleTypeNode = dynamic_cast<SimpleTypeNode *>(typeNode.get())) {
00217         typeName = simpleTypeNode->typeName;
00218     } else if (dynamic_cast<RecordTypeNode *>(typeNode.get())) {
00219         typeName = "record";

```

```

00220     }
00221   } else {
00222     typeName = "unknown";
00223   }
00224
00225   int slot = newSlotFor(mangledName);
00226   varSlot[varName] = slot;
00227   varSlot[mangledName] = slot;
00228
00229   std::string normalizedTypeName = resolveTypeName(1c(typeName));
00230
00231   // Handle set type aliases (e.g., type MySet = set of integer; var s: MySet;)
00232   if (normalizedTypeName.rfind("set of ", 0) == 0) {
00233     setVarType(varName, VarType::PTR);
00234     setSlotType(slot, VarType::PTR);
00235     setVars.insert(mangledName);
00236     setVars.insert(varName);
00237     updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00238     emit3("alloc", slotVar(slot), "8", "256");
00239     // Zero-fill the set
00240     std::string loopIdx = allocReg();
00241     std::string loopStart = newLabel("SET_INIT");
00242     std::string loopEnd = newLabel("SET_INIT_END");
00243     emit2("mov", loopIdx, "0");
00244     emitLabel(loopStart);
00245     emit2("cmp", loopIdx, "256");
00246     emit1("jge", loopEnd);
00247     emit4("store", "0", slotVar(slot), loopIdx, "8");
00248     emit2("add", loopIdx, "1");
00249     emit1("jmp", loopStart);
00250     emitLabel(loopEnd);
00251     freeReg(loopIdx);
00252     std::string currentScope = getCurrentScopeName();
00253     if (currentScope.empty())
00254       globalArrays.push_back(slotVar(slot));
00255     else
00256       functionScopedArrays[currentScope].push_back(slotVar(slot));
00257     continue;
00258   }
00259
00260   // Handle file/text type variables
00261   if (normalizedTypeName == "file" || normalizedTypeName == "text") {
00262     setVarType(varName, VarType::PTR);
00263     setSlotType(slot, VarType::PTR);
00264     fileVars.insert(mangledName);
00265     fileVars.insert(varName);
00266     updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00267     // Create companion filename variable
00268     std::string fileNameVar = mangledName + "_filename";
00269     int fnSlot = newSlotFor(fileNameVar);
00270     setSlotType(fnSlot, VarType::PTR);
00271     setVarType(fileNameVar, VarType::PTR);
00272     varSlot[fileNameVar] = fnSlot;
00273     fileVarNames[mangledName] = fileNameVar;
00274     fileVarNames[varName] = fileNameVar;
00275     continue;
00276   }
00277
00278   auto recordTypeIt = recordTypes.find(normalizedTypeName);
00279   if (recordTypeIt != recordTypes.end()) {
00280     varRecordType[mangledName] = normalizedTypeName;
00281     varRecordType[varName] = normalizedTypeName;
00282
00283     int recordSize = recordTypeIt->second.size;
00284
00285     setVarType(varName, VarType::RECORD);
00286     setSlotType(slot, VarType::PTR);
00287
00288     updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00289     emit3("alloc", slotVar(slot), std::to_string(recordSize), "1");
00290
00291     allocateRecordFieldArrays(mangledName, normalizedTypeName);
00292
00293     std::string currentScope = getCurrentScopeName();
00294     recordsToFreeInScope[currentScope].push_back(mangledName);
00295     continue;
00296   }
00297
00298   auto aliasIt = arrayInfo.find(normalizedTypeName);
00299   if (aliasIt != arrayInfo.end()) {
00300     arrayInfo[mangledName] = aliasIt->second;
00301
00302     setVarType(varName, VarType::PTR);
00303     setSlotType(slot, VarType::PTR);
00304
00305     updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00306     emit3("alloc",

```

```

00307         slotVar(slot),
00308         std::to_string(arrayInfo[mangledName].elementSize),
00309         std::to_string(arrayInfo[mangledName].size));
00310
00311         std::string currentScope = getCurrentScopeName();
00312         if (currentScope.empty())
00313             globalArrays.push_back(slotVar(slot));
00314         else
00315             functionScopedArrays[currentScope].push_back(slotVar(slot));
00316         continue;
00317     }
00318
00319     if (!normalizedTypeName.empty() && normalizedTypeName[0] == '^') {
00320         std::string baseType = normalizedTypeName.substr(1);
00321         setVarType(varName, VarType::PTR);
00322         setSlotType(slot, VarType::PTR);
00323         pointerBaseType[mangledName] = baseType;
00324         pointerBaseType[varName] = baseType;
00325         updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00326         continue;
00327     }
00328
00329     VarType vType = getTypeFromString(typeName);
00330     setVarType(varName, vType);
00331     setSlotType(slot, vType);
00332
00333     if (!currentFunctionName.empty())
00334         currentFuncLocalSlots.push_back(slotVar(slot));
00335
00336     if (vType == VarType::PTR || vType == VarType::RECORD) {
00337         updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
00338     } else if (vType == VarType::DOUBLE) {
00339         updateDataSectionInitialValue(slotVar(slot), "float", "0.0");
00340     } else {
00341         updateDataSectionInitialValue(slotVar(slot), "int", "0");
00342     }
00343 }
00344 }
00345
00346 void CodeGenVisitor::visit(ProcCallNode &node) {
00347     std::string procLower = node.name;
00348     std::transform(procLower.begin(), procLower.end(), procLower.begin(),
00349         [](unsigned char c) { return std::tolower(c); });
00350
00351     if (procLower == "new") {
00352         if (node.arguments.size() != 1)
00353             throw std::runtime_error("new() requires exactly one pointer argument");
00354
00355         if (auto *fieldNode = dynamic_cast<FieldAccessNode *>(node.arguments[0].get())) {
00356             std::string baseName;
00357             std::string recType;
00358             bool baseIsDirectPointer = false;
00359
00360             if (auto *v = dynamic_cast<VariableNode *>(fieldNode->recordExpr.get())) {
00361                 baseName = findMangledName(v->name);
00362                 recType = getVarRecordTypeName(v->name);
00363             } else {
00364                 fieldNode->recordExpr->accept(*this);
00365                 baseName = popValue();
00366                 baseIsDirectPointer = true;
00367                 recType = getVarRecordTypeNameFromExpr(fieldNode->recordExpr.get());
00368             }
00369
00370             recType = resolveTypeName(lc(recType));
00371             auto recTypeIt = recordTypes.find(recType);
00372             if (recTypeIt == recordTypes.end())
00373                 throw std::runtime_error("new(): unknown record type: " + recType);
00374
00375             const std::string fieldName = lc(fieldNode->fieldName);
00376             auto it = recTypeIt->second.nameToIndex.find(fieldName);
00377             if (it == recTypeIt->second.nameToIndex.end())
00378                 throw std::runtime_error("new(): field not found: " + fieldName);
00379
00380             const auto &fieldInfo = recTypeIt->second.fields[it->second];
00381             const int byteOffset = fieldInfo.offset;
00382
00383             std::string fieldType = fieldInfo.typeName;
00384             int elemSize = 8;
00385             if (!fieldType.empty() && fieldType[0] == '^') {
00386                 std::string baseType = resolveTypeName(lc(fieldType.substr(1)));
00387                 elemSize = getSizeByType(baseType);
00388                 if (isRecordTypeName(baseType)) {
00389                     auto rit = recordTypes.find(baseType);
00390                     if (rit != recordTypes.end())
00391                         elemSize = rit->second.size;
00392                 }
00393             }

```

```

00394
00395         std::string tmp = allocTempPtr();
00396         emit3("alloc", tmp, std::to_string(elemSize), "1");
00397
00398         std::string basePtr = baseIsDirectPointer ? baseName : ensurePtrBase(baseName);
00399         emit4("store", tmp, basePtr, std::to_string(byteOffset), "1");
00400
00401         if (isReg(tmp) && !isParmReg(tmp))
00402             freeReg(tmp);
00403         if (baseIsDirectPointer && isReg(basePtr) && !isParmReg(basePtr))
00404             freeReg(basePtr);
00405         return;
00406     }
00407
00408     if (auto *arrNode = dynamic_cast<ArrayAccessNode *>(node.arguments[0].get())) {
00409         ArrayInfo *info = getArrayInfoForArrayAccess(arrNode);
00410         if (!info)
00411             throw std::runtime_error("new(): unknown array: " +
getArrayNameFromBase(arrNode->base.get()));
00412
00413         int elemSize = 8;
00414         std::string et = info->elementType;
00415         if (!et.empty() && et[0] == '^') {
00416             std::string baseType = resolveTypeName(lc(et.substr(1)));
00417             elemSize = getTypeSizeByName(baseType);
00418             if (isRecordTypeName(baseType)) {
00419                 auto rit = recordTypes.find(baseType);
00420                 if (rit != recordTypes.end())
00421                     elemSize = rit->second.size;
00422             }
00423         }
00424
00425         std::string idx = eval(arrNode->index.get());
00426         if (getExpressionType(arrNode->index.get()) == VarType::DOUBLE) {
00427             std::string intIdx = allocReg();
00428             emit2("mov", intIdx, idx);
00429             if (isReg(intIdx) && !isParmReg(intIdx))
00430                 freeReg(intIdx);
00431             idx = intIdx;
00432         }
00433
00434         std::string elemIndex;
00435         if (isReg(idx) && !isParmReg(idx)) {
00436             elemIndex = idx;
00437         } else {
00438             elemIndex = allocReg();
00439             emit2("mov", elemIndex, idx);
00440         }
00441         if (info->lowerBound != 0)
00442             emit2("sub", elemIndex, std::to_string(info->lowerBound));
00443
00444         std::string base;
00445         if (auto var = dynamic_cast<VariableNode *>(arrNode->base.get())) {
00446             std::string mangled = findMangledArrayName(var->name);
00447             base = ensurePtrBase(storageSymbolFor(mangled));
00448         } else {
00449             arrNode->base->accept(*this);
00450             base = popValue();
00451         }
00452
00453         std::string tmp = allocTempPtr();
00454         emit3("alloc", tmp, std::to_string(elemSize), "1");
00455         emit4("store", tmp, base, elemIndex, std::to_string(info->elementSize));
00456
00457         if (isReg(tmp) && !isParmReg(tmp))
00458             freeReg(tmp);
00459         if (elemIndex != idx && isReg(idx) && !isParmReg(idx))
00460             freeReg(idx);
00461         freeReg(elemIndex);
00462         return;
00463     }
00464
00465     auto *varNode = dynamic_cast<VariableNode *>(node.arguments[0].get());
00466     if (!varNode)
00467         throw std::runtime_error("new() argument must be a variable, record field, or array
element");
00468
00469     std::string mangled = findMangledName(varNode->name);
00470     std::string sym = storageSymbolFor(mangled);
00471     int elemSize = getPointerElementSize(varNode->name);
00472
00473     std::string baseType = getPointerBaseTypeName(varNode->name);
00474     baseType = resolveTypeName(lc(baseType));
00475     int allocCount = 1;
00476     if (isRecordTypeName(baseType)) {
00477         auto it = recordTypes.find(baseType);
00478         if (it != recordTypes.end()) {

```

```

00479         elemSize = it->second.size;
00480     }
00481 }
00482
00483 emit3("alloc", sym, std::to_string(elemSize), std::to_string(allocCount));
00484 return;
00485 }
00486
00487 if (procLower == "dispose") {
00488     if (node.arguments.size() != 1)
00489         throw std::runtime_error("dispose() requires exactly one pointer argument");
00490
00491     if (auto *fieldNode = dynamic_cast<FieldAccessNode *>(node.arguments[0].get())) {
00492         std::string baseName;
00493         std::string recType;
00494         bool baseIsDirectPointer = false;
00495
00496         if (auto *v = dynamic_cast<VariableNode *>(fieldNode->recordExpr.get())) {
00497             baseName = findMangledName(v->name);
00498             recType = getVarRecordTypeName(v->name);
00499         } else {
00500             fieldNode->recordExpr->accept(*this);
00501             baseName = popValue();
00502             baseIsDirectPointer = true;
00503             recType = getVarRecordTypeNameFromExpr(fieldNode->recordExpr.get());
00504         }
00505
00506         recType = resolveTypeName(lc(recType));
00507         auto recTypeIt = recordTypes.find(recType);
00508         if (recTypeIt == recordTypes.end())
00509             throw std::runtime_error("dispose(): unknown record type: " + recType);
00510
00511         const std::string fieldName = lc(fieldNode->fieldName);
00512         auto it = recTypeIt->second.nameToIndex.find(fieldName);
00513         if (it == recTypeIt->second.nameToIndex.end())
00514             throw std::runtime_error("dispose(): field not found: " + fieldName);
00515
00516         const auto &fieldInfo = recTypeIt->second.fields[it->second];
00517         const int byteOffset = fieldInfo.offset;
00518
00519         std::string basePtr = baseIsDirectPointer ? baseName : ensurePtrBase(baseName);
00520         std::string tmp = allocTempPtr();
00521         emit4("load", tmp, basePtr, std::to_string(byteOffset), "1");
00522         emit1("free", tmp);
00523
00524         if (isReg(tmp) && !isParmReg(tmp))
00525             freeReg(tmp);
00526         if (baseIsDirectPointer && isReg(basePtr) && !isParmReg(basePtr))
00527             freeReg(basePtr);
00528         return;
00529     }
00530
00531     if (auto *arrNode = dynamic_cast<ArrayAccessNode *>(node.arguments[0].get())) {
00532         ArrayInfo *info = getArrayInfoForArrayAccess(arrNode);
00533         if (!info)
00534             throw std::runtime_error("dispose(): unknown array: " +
getArrayNameFromBase(arrNode->base.get()));
00535
00536         std::string idx = eval(arrNode->index.get());
00537         if (getExpressionType(arrNode->index.get()) == VarType::DOUBLE) {
00538             std::string intIdx = allocReg();
00539             emit2("mov", intIdx, idx);
00540             if (isReg(idx) && !isParmReg(idx))
00541                 freeReg(idx);
00542             idx = intIdx;
00543         }
00544
00545         std::string elemIndex;
00546         if (isReg(idx) && !isParmReg(idx)) {
00547             elemIndex = idx;
00548         } else {
00549             elemIndex = allocReg();
00550             emit2("mov", elemIndex, idx);
00551         }
00552         if (info->lowerBound != 0)
00553             emit2("sub", elemIndex, std::to_string(info->lowerBound));
00554
00555         std::string base;
00556         if (auto var = dynamic_cast<VariableNode *>(arrNode->base.get())) {
00557             std::string mangled = findMangledArrayName(var->name);
00558             base = ensurePtrBase(storageSymbolFor(mangled));
00559         } else {
00560             arrNode->base->accept(*this);
00561             base = popValue();
00562         }
00563
00564         std::string tmp = allocTempPtr();

```



```

00565         emit4("load", tmp, base, elemIndex, std::to_string(info->elementSize));
00566         emit1("free", tmp);
00567
00568         if (isReg(tmp) && !isParmReg(tmp))
00569             freeReg(tmp);
00570         if (elemIndex != idx && isReg(idx) && !isParmReg(idx))
00571             freeReg(idx);
00572         freeReg(elemIndex);
00573         return;
00574     }
00575
00576     auto *varNode = dynamic_cast<VariableNode *>(node.arguments[0].get());
00577     if (!varNode)
00578         throw std::runtime_error("dispose() argument must be a variable, record field, or
array element");
00579
00580     std::string mangled = findMangledName(varNode->name);
00581     std::string sym = storageSymbolFor(mangled);
00582     emit1("free", sym);
00583     return;
00584 }
00585
00586 // SetLength(arr, newSize) -- resize a dynamic array
00587 if (procLower == "setlength" && node.arguments.size() == 2) {
00588     auto *varNode = dynamic_cast<VariableNode *>(node.arguments[0].get());
00589     if (!varNode)
00590         throw std::runtime_error("SetLength: first argument must be a dynamic array
variable");
00591
00592     std::string mangled = findMangledArrayName(varNode->name);
00593     auto it = arrayInfo.find(mangled);
00594     if (it == arrayInfo.end())
00595         it = arrayInfo.find(varNode->name);
00596     if (it == arrayInfo.end() || !it->second.isDynamic)
00597         throw std::runtime_error("SetLength: " + varNode->name + " is not a dynamic
array");
00598
00599     std::string arrMangled = it->first;
00600     std::string sym = storageSymbolFor(arrMangled);
00601
00602     // Evaluate the new size
00603     std::string newSize = eval(node.arguments[1].get());
00604
00605     // Emit realloc: realloc arrPtr, elementSize, newCount
00606     emit3("realloc", sym, std::to_string(it->second.elementSize), newSize);
00607
00608     // Update companion length variable
00609     auto lenIt = dynArrayLenSlot.find(arrMangled);
00610     if (lenIt != dynArrayLenSlot.end()) {
00611         emit2("mov", slotVar(lenIt->second), newSize);
00612     }
00613
00614     if (isReg(newSize) && !isParmReg(newSize))
00615         freeReg(newSize);
00616     return;
00617 }
00618
00619 auto handler = builtinRegistry.findHandler(procLower);
00620 if (handler) {
00621     handler->generate(*this, procLower, node.arguments);
00622     return;
00623 }
00624
00625 std::vector<std::string> evaluated_args;
00626 std::vector<VarType> argTypes;
00627
00628 for (auto &arg : node.arguments) {
00629     std::string argVal = eval(arg.get());
00630     evaluated_args.push_back(argVal);
00631     argTypes.push_back(getExpressionType(arg.get()));
00632 }
00633
00634 // Build target register list
00635 std::vector<std::string> targetRegs(evaluated_args.size());
00636 size_t intRegIdx = 1;
00637 size_t ptrRegIdx = 0;
00638 size_t floatRegIdx = 0;
00639
00640 for (size_t i = 0; i < evaluated_args.size(); ++i) {
00641     if (argTypes[i] == VarType::STRING || argTypes[i] == VarType::PTR || argTypes[i] ==
VarType::RECORD) {
00642         if (ptrRegIdx < ptrRegisters.size())
00643             targetRegs[i] = ptrRegisters[ptrRegIdx++];
00644     } else if (argTypes[i] == VarType::DOUBLE) {
00645         if (floatRegIdx < floatRegisters.size())
00646             targetRegs[i] = floatRegisters[floatRegIdx++];
00647     } else {

```

```

00648         if (intRegIdx < registers.size())
00649             targetRegs[i] = registers[intRegIdx++];
00650     }
00651 }
00652
00653 // Spill any evaluated arg that sits in another arg's target register
00654 for (size_t i = 0; i < evaluated_args.size(); ++i) {
00655     if (evaluated_args[i].empty() || targetRegs[i].empty())
00656         continue;
00657     if (evaluated_args[i] == targetRegs[i])
00658         continue;
00659     for (size_t j = 0; j < evaluated_args.size(); ++j) {
00660         if (j == i)
00661             continue;
00662         if (evaluated_args[j] == targetRegs[i]) {
00663             // arg j's value is in a register that arg i needs as target
00664             std::string spill = allocReg();
00665             emit2("mov", spill, evaluated_args[j]);
00666             evaluated_args[j] = spill;
00667         }
00668     }
00669 }
00670
00671 // Save caller's in-use registers that could be clobbered by this call
00672 std::vector<std::string> savedRegs;
00673 for (size_t i = 1; i < regInUse.size() && i < registers.size(); ++i) {
00674     if (!regInUse[i]) continue;
00675     bool isEvalArg = false;
00676     for (const auto &ea : evaluated_args) {
00677         if (ea == registers[i]) { isEvalArg = true; break; }
00678     }
00679     if (isEvalArg) continue;
00680     savedRegs.push_back(registers[i]);
00681 }
00682 for (const auto &sr : savedRegs)
00683     emit1("push", sr);
00684
00685 // Now move into target registers
00686 {
00687     auto eit = externalFuncs.find(node.name);
00688     std::string objPrefix = (eit != externalFuncs.end()) ? eit->second + "." : "";
00689     for (size_t i = 0; i < evaluated_args.size(); ++i) {
00690         if (!targetRegs[i].empty())
00691             emit2("mov", objPrefix + targetRegs[i], evaluated_args[i]);
00692     }
00693 }
00694
00695 std::string mangledName = findMangledFuncName(node.name, true);
00696 {
00697     std::string prefix = "PROC_";
00698     if (funcSignatures.count(node.name))
00699         prefix = "FUNC_";
00700     std::string label = prefix + mangledName;
00701     auto eit = externalFuncs.find(node.name);
00702     if (eit != externalFuncs.end())
00703         label = eit->second + "." + label;
00704
00705     if (!currentFuncLocalSlots.empty()) {
00706         for (const auto &slot : currentFuncLocalSlots)
00707             emit1("push", slot);
00708     }
00709
00710     emit1("call", label);
00711
00712     if (!currentFuncLocalSlots.empty()) {
00713         for (auto it2 = currentFuncLocalSlots.rbegin(); it2 != currentFuncLocalSlots.rend();
00714 ++it2)
00715             emit1("pop", *it2);
00716     }
00717 }
00718
00719 // Restore caller's saved registers
00720 for (auto rit = savedRegs.rbegin(); rit != savedRegs.rend(); ++rit)
00721     emit1("pop", *rit);
00722
00723 for (const auto &arg : evaluated_args)
00724     if (isReg(arg) && !isParmReg(arg))
00725         freeReg(arg);
00726 }
00727
00728 void CodeGenVisitor::visit(FuncCallNode &node) {
00729     // Handle array-specific Length, High, Low before string Length builtin
00730     std::string fnLower = node.name;
00731     std::transform(fnLower.begin(), fnLower.end(), fnLower.begin(),
00732         [](unsigned char c) { return std::tolower(c); });
00733     if ((fnLower == "length" || fnLower == "high" || fnLower == "low") && node.arguments.size() ==
00734 1) {

```

```

00733         if (auto *varNode = dynamic_cast<VariableNode *>(node.arguments[0].get())) {
00734             std::string mangled = findMangledArrayName(varNode->name);
00735             auto it = arrayInfo.find(mangled);
00736             if (it == arrayInfo.end())
00737                 it = arrayInfo.find(varNode->name);
00738             if (it != arrayInfo.end()) {
00739                 if (fnLower == "low") {
00740                     // Dynamic arrays are always 0-based; static arrays use lowerBound
00741                     std::string r = allocReg();
00742                     emit2("mov", r, std::to_string(it->second.isDynamic ? 0 :
it->second.lowerBound));
00743                     pushValue(r);
00744                     return;
00745                 }
00746                 if (it->second.isDynamic) {
00747                     // Runtime: read from companion length variable
00748                     std::string arrMangled = (it == arrayInfo.find(mangled)) ? mangled :
varNode->name;
00749                     auto lenIt = dynArrayLenSlot.find(arrMangled);
00750                     if (lenIt == dynArrayLenSlot.end())
00751                         throw std::runtime_error("Length/High: dynamic array has no companion
length variable: " + varNode->name);
00752                     std::string lenSym = slotVar(lenIt->second);
00753                     std::string r = allocReg();
00754                     emit2("mov", r, lenSym);
00755                     if (fnLower == "high") {
00756                         emit2("sub", r, "1");
00757                     }
00758                     pushValue(r);
00759                     return;
00760                 } else {
00761                     // Static array: compile-time constants
00762                     std::string r = allocReg();
00763                     if (fnLower == "length") {
00764                         emit2("mov", r, std::to_string(it->second.size));
00765                     } else { // high
00766                         emit2("mov", r, std::to_string(it->second.upperBound));
00767                     }
00768                     pushValue(r);
00769                     return;
00770                 }
00771             }
00772         }
00773     }
00774
00775     auto handler = builtinRegistry.findHandler(fnLower);
00776     if (handler) {
00777         if (handler->generateWithResult(*this, fnLower, node.arguments))
00778             return;
00779     }
00780
00781     std::vector<std::string> evaluated_args;
00782     std::vector<VarType> argTypes;
00783
00784     for (auto &arg : node.arguments) {
00785         std::string argVal = eval(arg.get());
00786         evaluated_args.push_back(argVal);
00787         argTypes.push_back(getExpressionType(arg.get()));
00788     }
00789
00790     // Build target register list
00791     std::vector<std::string> targetRegs(evaluated_args.size());
00792     size_t intRegIdx = 1;
00793     size_t ptrRegIdx = 0;
00794     size_t floatRegIdx = 0;
00795
00796     for (size_t i = 0; i < evaluated_args.size(); ++i) {
00797         if (argTypes[i] == VarType::STRING || argTypes[i] == VarType::PTR || argTypes[i] ==
VarType::RECORD) {
00798             if (ptrRegIdx < ptrRegisters.size())
00799                 targetRegs[i] = ptrRegisters[ptrRegIdx++];
00800             } else if (argTypes[i] == VarType::DOUBLE) {
00801                 if (floatRegIdx < floatRegisters.size())
00802                     targetRegs[i] = floatRegisters[floatRegIdx++];
00803             } else {
00804                 if (intRegIdx < registers.size())
00805                     targetRegs[i] = registers[intRegIdx++];
00806             }
00807         }
00808
00809         // Spill any evaluated arg that sits in another arg's target register
00810         for (size_t i = 0; i < evaluated_args.size(); ++i) {
00811             if (evaluated_args[i].empty() || targetRegs[i].empty())
00812                 continue;
00813             if (evaluated_args[i] == targetRegs[i])
00814                 continue;
00815             for (size_t j = 0; j < evaluated_args.size(); ++j) {

```

```

00816         if (j == i)
00817             continue;
00818         if (evaluated_args[j] == targetRegs[i]) {
00819             std::string spill = allocReg();
00820             emit2("mov", spill, evaluated_args[j]);
00821             evaluated_args[j] = spill;
00822         }
00823     }
00824 }
00825
00826 // Save caller's in-use registers that could be clobbered by this call
00827 std::vector<std::string> savedRegs;
00828 for (size_t i = 1; i < regInUse.size() && i < registers.size(); ++i) {
00829     if (!regInUse[i]) continue;
00830     bool isEvalArg = false;
00831     for (const auto &ea : evaluated_args) {
00832         if (ea == registers[i]) { isEvalArg = true; break; }
00833     }
00834     if (isEvalArg) continue;
00835     savedRegs.push_back(registers[i]);
00836 }
00837 for (const auto &sr : savedRegs)
00838     emit1("push", sr);
00839
00840 // Now move into target registers
00841 {
00842     auto eit = externalFuncs.find(node.name);
00843     std::string objPrefix = (eit != externalFuncs.end()) ? eit->second + "." : "";
00844     for (size_t i = 0; i < evaluated_args.size(); ++i) {
00845         if (!targetRegs[i].empty())
00846             emit2("mov", objPrefix + targetRegs[i], evaluated_args[i]);
00847     }
00848 }
00849
00850 std::string mangledName = findMangledFuncName(node.name, false);
00851 {
00852     std::string label = "FUNC_" + mangledName;
00853     auto eit = externalFuncs.find(node.name);
00854     if (eit != externalFuncs.end())
00855         label = eit->second + "." + label;
00856
00857     if (!currentFuncLocalSlots.empty()) {
00858         for (const auto &slot : currentFuncLocalSlots)
00859             emit1("push", slot);
00860     }
00861
00862     emit1("call", label);
00863
00864     if (!currentFuncLocalSlots.empty()) {
00865         for (auto it2 = currentFuncLocalSlots.rbegin(); it2 != currentFuncLocalSlots.rend();
00866 ++it2)
00867             emit1("pop", *it2);
00868     }
00869
00870 // Restore caller's saved registers
00871 for (auto rit = savedRegs.rbegin(); rit != savedRegs.rend(); ++rit)
00872     emit1("pop", *rit);
00873
00874 auto it = funcSignatures.find(node.name);
00875 VarType returnType = (it != funcSignatures.end()) ? it->second.returnType : VarType::INT;
00876
00877 std::string resultLocation;
00878 {
00879     auto eit = externalFuncs.find(node.name);
00880     std::string objPrefix = (eit != externalFuncs.end()) ? eit->second + "." : "";
00881     if (returnType == VarType::DOUBLE) {
00882         resultLocation = allocFloatReg();
00883         emit2("mov", resultLocation, objPrefix + "xmm0");
00884     } else if (returnType == VarType::STRING || returnType == VarType::PTR || returnType ==
00885 VarType::RECORD) {
00886         resultLocation = allocTempPtr();
00887         emit2("mov", resultLocation, objPrefix + "arg0");
00888     } else {
00889         resultLocation = allocReg();
00890         emit2("mov", resultLocation, objPrefix + "rax");
00891     }
00892     pushValue(resultLocation);
00893
00894     for (const auto &arg : evaluated_args)
00895         if (isReg(arg) && !isParmReg(arg))
00896             freeReg(arg);
00897 }
00898
00899 void CodeGenVisitor::visit(FuncDeclNode &node) {
00900     std::string mangledName = mangleVariableName(node.name);

```

```

00901         if (declaredFuncs.count(mangledName)) {
00902             return;
00903         }
00904         declaredFuncs[mangledName] = true;
00905
00906         FuncInfo funcInfo;
00907         funcInfo.returnType = getTypeFromString(node.returnType);
00908         for (auto &p : node.parameters) {
00909             if (auto pn = dynamic_cast<ParameterNode *>(p.get())) {
00910                 for (size_t i = 0; i < pn->identifiers.size(); ++i) {
00911                     funcInfo.paramTypes.push_back(getTypeFromString(pn->type));
00912                 }
00913             }
00914         }
00915         funcSignatures[node.name] = funcInfo;
00916         setVarType(node.name, funcInfo.returnType);
00917
00918         // Only defer code generation if there's an actual body (not a forward declaration)
00919         if (node.block) {
00920             auto path = scopeHierarchy;
00921             path.push_back(node.name);
00922             deferredFuncs.push_back({&node, path});
00923
00924             scopeHierarchy.push_back(node.name);
00925             if (auto blockNode = dynamic_cast<BlockNode *>(node.block.get())) {
00926                 for (auto &decl : blockNode->declarations) {
00927                     if (dynamic_cast<ProcDeclNode *>(decl.get()) || dynamic_cast<FuncDeclNode
*>(decl.get())) {
00928                         decl->accept(*this);
00929                     }
00930                 }
00931             }
00932             scopeHierarchy.pop_back();
00933         }
00934     }
00935
00936     void CodeGenVisitor::visit(ProcDeclNode &node) {
00937         std::string mangledName = mangleVariableName(node.name);
00938         if (declaredProcs.count(mangledName)) {
00939             return;
00940         }
00941         declaredProcs[mangledName] = true;
00942
00943         // Only defer code generation if there's an actual body (not a forward declaration)
00944         if (node.block) {
00945             auto path = scopeHierarchy;
00946             path.push_back(node.name);
00947             deferredProcs.push_back({&node, path});
00948
00949             scopeHierarchy.push_back(node.name);
00950             if (auto blockNode = dynamic_cast<BlockNode *>(node.block.get())) {
00951                 for (auto &decl : blockNode->declarations) {
00952                     if (dynamic_cast<ProcDeclNode *>(decl.get()) || dynamic_cast<FuncDeclNode
*>(decl.get())) {
00953                         decl->accept(*this);
00954                     }
00955                 }
00956             }
00957             scopeHierarchy.pop_back();
00958         }
00959     }
00960
00961     void CodeGenVisitor::visit(TypeAliasNode &node) {
00962         auto lc = [](std::string s) { std::transform(s.begin(), s.end(), s.begin(),
00963             [](unsigned char c){ return std::tolower(c); }); return s; };
00964         typeAliases[lc(node.typeName)] = lc(node.baseType);
00965     }
00966
00967     void CodeGenVisitor::visit(ParameterNode &node) {
00968         auto lc = [](std::string s) { std::transform(s.begin(), s.end(), s.begin(),
00969             [](unsigned char c) { return std::tolower(c); }); };
00970         return s;
00971     };
00972
00973     for (auto &idRaw : node.identifiers) {
00974         std::string id = idRaw;
00975         int slot = newSlotFor(id);
00976         std::string t = lc(node.type);
00977         if (t == "string") {
00978             setVarType(id, VarType::STRING);
00979             setSlotType(slot, VarType::STRING);
00980         } else if (t == "integer" || t == "boolean") {
00981             setVarType(id, VarType::INT);
00982             setSlotType(slot, VarType::INT);
00983         } else if (t == "real") {
00984             setVarType(id, VarType::DOUBLE);
00985             setSlotType(slot, VarType::DOUBLE);

```

```

00986         } else if (!t.empty() && t[0] == '^') {
00987             setVarType(id, VarType::PTR);
00988             setSlotType(slot, VarType::PTR);
00989         } else {
00990             std::string rt = resolveTypeName(t);
00991             if (recordTypes.count(rt)) {
00992                 setVarType(id, VarType::RECORD);
00993                 setSlotType(slot, VarType::PTR);
00994                 varRecordType[id] = rt;
00995                 currentParamTypes[id] = rt;
00996             }
00997         }
00998     }
00999 }
01000
01010 void CodeGenVisitor::visit(CompoundStmtNode &node) {
01011     for (auto &stmt : node.statements) {
01012         if (!stmt)
01013             continue;
01014
01015         // snapshot allocated temp ptrs before statement
01016         auto ptrsBefore = allocatedPtrs;
01017         auto escapedBefore = escapedTempPtrs;
01018
01019         if (auto v = dynamic_cast<VariableNode *>(stmt.get())) {
01020             std::string mangled = findMangledFuncName(v->name, true);
01021             if (declaredProcs.count(mangled)) {
01022                 std::string prefix = "PROC_";
01023                 if (funcSignatures.count(v->name))
01024                     prefix = "FUNC_";
01025                 std::string label = prefix + mangled;
01026                 auto eit = externalFuncs.find(v->name);
01027                 if (eit != externalFuncs.end())
01028                     label = eit->second + "." + label;
01029                 emit1("call", label);
01030                 // free any temp ptrs allocated during this statement (not escaped)
01031                 for (auto &p : allocatedPtrs) {
01032                     if (!ptrsBefore.count(p) && !escapedTempPtrs.count(p))
01033                         emit("free " + p);
01034                 }
01035                 allocatedPtrs = ptrsBefore;
01036                 continue;
01037             }
01038         }
01039         stmt->accept(*this);
01040
01041         // free any temp ptrs allocated during this statement (not escaped)
01042         for (auto &p : allocatedPtrs) {
01043             if (!ptrsBefore.count(p) && !escapedTempPtrs.count(p))
01044                 emit("free " + p);
01045         }
01046         allocatedPtrs = ptrsBefore;
01047     }
01048 }
01049
01050 void CodeGenVisitor::visit(IfStmtNode &node) {
01051     std::string elseL = newLabel("ELSE");
01052     std::string endL = newLabel("ENDIF");
01053
01054     std::string condResult = eval(node.condition.get());
01055
01056     emit2("cmp", condResult, "0");
01057     emit1("je", elseL);
01058
01059     if (isReg(condResult) && !isParmReg(condResult)) {
01060         freeReg(condResult);
01061     }
01062     if (node.thenStatement) {
01063         node.thenStatement->accept(*this);
01064     }
01065     if (node.elseStatement) {
01066         emit1("jmp", endL);
01067         emitLabel(elseL);
01068         node.elseStatement->accept(*this);
01069         emitLabel(endL);
01070     } else {
01071         emitLabel(elseL);
01072     }
01073 }
01074
01075 void CodeGenVisitor::visit(WhileStmtNode &node) {
01076     std::string start = newLabel("WHILE");
01077     std::string end = newLabel("ENDWHILE");
01078     loopContinueLabels.push_back(start);
01079     loopEndLabels.push_back(end);
01080     emitLabel(start);
01081     std::string c = eval(node.condition.get());

```

```

01082         emit2("cmp", c, "0");
01083         emit1("je", end);
01084         if (isReg(c) && !isParmReg(c))
01085             freeReg(c);
01086         if (node.statement)
01087             node.statement->accept(*this);
01088         emit1("jmp", start);
01089         emitLabel(end);
01090         loopContinueLabels.pop_back();
01091         loopEndLabels.pop_back();
01092     }
01093
01094 void CodeGenVisitor::visit(ForStmtNode &node) {
01095     std::string startVal = eval(node.startValue.get());
01096     std::string endVal = eval(node.endValue.get());
01097
01098     startVal = coerceToIntImmediate(startVal);
01099     endVal = coerceToIntImmediate(endVal);
01100
01101     std::string mangledLoopVar = mangleVariableName(node.variable);
01102     int slot = newSlotFor(mangledLoopVar);
01103     emit2("mov", slotVar(slot), startVal);
01104     if (isReg(startVal) && !isParmReg(startVal))
01105         freeReg(startVal);
01106
01107     std::string loopStartLabel = newLabel("FOR");
01108     std::string loopEndLabel = newLabel("ENDFOR");
01109     std::string continueLabel = newLabel("FOR_CONTINUE");
01110
01111     loopContinueLabels.push_back(continueLabel);
01112     loopEndLabels.push_back(loopEndLabel);
01113
01114     std::string endCmp = endVal;
01115
01116     emitLabel(loopStartLabel);
01117
01118     emit2("cmp", slotVar(slot), endCmp);
01119     if (node.isDownto)
01120         emit1("jl", loopEndLabel);
01121     else
01122         emit1("jg", loopEndLabel);
01123
01124     if (node.statement)
01125         node.statement->accept(*this);
01126     emitLabel(continueLabel);
01127     if (node.isDownto)
01128         emit2("sub", slotVar(slot), "1");
01129     else
01130         emit2("add", slotVar(slot), "1");
01131     emit1("jmp", loopStartLabel);
01132     emitLabel(loopEndLabel);
01133
01134     loopContinueLabels.pop_back();
01135     loopEndLabels.pop_back();
01136
01137     if (isReg(endCmp) && !isParmReg(endCmp))
01138         freeReg(endCmp);
01139 }
01140
01141 void CodeGenVisitor::visit(BinaryOpNode &node) {
01142     auto isStrLike = [&](VarType v) { return v == VarType::STRING || v == VarType::PTR; };
01143     VarType lt = getExpressionType(node.left.get());
01144     VarType rt = getExpressionType(node.right.get());
01145
01146     // Handle 'in' operator: expr in [val1, val2, ...] or expr in setVar
01147     if (node.operator_ == BinaryOpNode::IN) {
01148         auto *setNode = dynamic_cast<SetLiteralNode *>(node.right.get());
01149         if (setNode) {
01150             // Inline set literal: compare against each element
01151             std::string val = eval(node.left.get());
01152             std::string foundLabel = newLabel("IN_FOUND");
01153             std::string endLabel = newLabel("IN_END");
01154             std::string result = allocReg();
01155
01156             for (auto &elem : setNode->elements) {
01157                 std::string elemVal = eval(elem.get());
01158                 emit2("cmp", val, elemVal);
01159                 emit1("je", foundLabel);
01160                 if (isReg(elemVal) && !isParmReg(elemVal))
01161                     freeReg(elemVal);
01162             }
01163             emit2("mov", result, "0");
01164             emit1("jmp", endLabel);
01165             emitLabel(foundLabel);
01166             emit2("mov", result, "1");
01167             emitLabel(endLabel);
01168         }

```

```

01169         if (isReg(val) && !isParmReg(val))
01170             freeReg(val);
01171         pushValue(result);
01172         return;
01173     }
01174     // Set variable: load byte at index
01175     std::string val = eval(node.left.get());
01176     std::string setVal = eval(node.right.get());
01177     std::string byte = allocReg();
01178     std::string result = allocReg();
01179     emit4("load", byte, setVal, val, "8");
01180     emit2("cmp", byte, "0");
01181     std::string notFoundLabel = newLabel("IN_NOT_FOUND");
01182     std::string endLabel = newLabel("IN_END");
01183     emit1("je", notFoundLabel);
01184     emit2("mov", result, "1");
01185     emit1("jmp", endLabel);
01186     emitLabel(notFoundLabel);
01187     emit2("mov", result, "0");
01188     emitLabel(endLabel);
01189     freeReg(byte);
01190     if (isReg(val) && !isParmReg(val))
01191         freeReg(val);
01192     if (isReg(setVal) && !isParmReg(setVal))
01193         freeReg(setVal);
01194     pushValue(result);
01195     return;
01196 }
01197
01198 // Set operations: + (union), - (difference), * (intersection)
01199 auto isSetExpr = [&](ASTNode *n) -> bool {
01200     if (dynamic_cast<SetLiteralNode *>(n))
01201         return true;
01202     if (auto *v = dynamic_cast<VariableNode *>(n)) {
01203         std::string mangled = findMangledName(v->name);
01204         return setVars.count(mangled) || setVars.count(v->name);
01205     }
01206     return false;
01207 };
01208 if ((node.operator_ == BinaryOpNode::PLUS ||
01209     node.operator_ == BinaryOpNode::MINUS ||
01210     node.operator_ == BinaryOpNode::MULTIPLY) &&
01211     (isSetExpr(node.left.get()) || isSetExpr(node.right.get()))) {
01212
01213     std::string leftSet = eval(node.left.get());
01214     std::string rightSet = eval(node.right.get());
01215
01216     // Allocate result set
01217     std::string resultSet = allocTempPtr();
01218     emit3("alloc", resultSet, "8", "256");
01219
01220     std::string idx = allocReg();
01221     std::string lb = allocReg();
01222     std::string rb = allocReg();
01223     std::string loopStart = newLabel("SET_OP");
01224     std::string loopEnd = newLabel("SET_OP_END");
01225     std::string storeBit = newLabel("SET_OP_STORE");
01226     std::string storeZero = newLabel("SET_OP_ZERO");
01227
01228     emit2("mov", idx, "0");
01229     emitLabel(loopStart);
01230     emit2("cmp", idx, "256");
01231     emit1("jge", loopEnd);
01232     emit4("load", lb, leftSet, idx, "8");
01233     emit4("load", rb, rightSet, idx, "8");
01234
01235     if (node.operator_ == BinaryOpNode::PLUS) {
01236         // Union: result = left OR right
01237         emit2("cmp", lb, "0");
01238         emit1("jne", storeBit);
01239         emit2("cmp", rb, "0");
01240         emit1("jne", storeBit);
01241         emit1("jmp", storeZero);
01242     } else if (node.operator_ == BinaryOpNode::MULTIPLY) {
01243         // Intersection: result = left AND right
01244         emit2("cmp", lb, "0");
01245         emit1("je", storeZero);
01246         emit2("cmp", rb, "0");
01247         emit1("je", storeZero);
01248         emit1("jmp", storeBit);
01249     } else {
01250         // Difference: result = left AND NOT right
01251         emit2("cmp", lb, "0");
01252         emit1("jne", storeZero);
01253         emit2("cmp", rb, "0");
01254         emit1("jne", storeZero);
01255         emit1("jmp", storeBit);
01256     }
01257 }

```



```

01256     }
01257     emitLabel(storeBit);
01258     emit4("store", "1", resultSet, idx, "8");
01259     std::string nextLabel = newLabel("SET_OP_NEXT");
01260     emit1("jmp", nextLabel);
01261     emitLabel(storeZero);
01262     emit4("store", "0", resultSet, idx, "8");
01263     emitLabel(nextLabel);
01264     emit2("add", idx, "1");
01265     emit1("jmp", loopStart);
01266     emitLabel(loopEnd);
01267
01268     freeReg(idx);
01269     freeReg(lb);
01270     freeReg(rb);
01271     if (isReg(leftSet) && !isParmReg(leftSet))
01272         freeReg(leftSet);
01273     if (isReg(rightSet) && !isParmReg(rightSet))
01274         freeReg(rightSet);
01275     pushValue(resultSet);
01276     return;
01277 }
01278
01279 if (node.operator_ == BinaryOpNode::PLUS && (isStrLike(lt) || isStrLike(rt))) {
01280     usedModules.insert("string");
01281     std::string left = eval(node.left.get());
01282     std::string right = eval(node.right.get());
01283     std::string len1 = allocReg(), len2 = allocReg(), totalLen = allocReg();
01284     emit_invoke("strlen", {left});
01285     emit("return " + len1);
01286     emit_invoke("strlen", {right});
01287     emit("return " + len2);
01288     emit2("mov", totalLen, len1);
01289     emit2("add", totalLen, len2);
01290     emit2("add", totalLen, "1");
01291
01292     std::string result_str = allocTempPtr();
01293     emit3("alloc", result_str, "1", totalLen);
01294     emit_invoke("strncpy", {result_str, left, len1});
01295     emit_invoke("strncat", {result_str, right, len2});
01296
01297     pushValue(result_str);
01298     freeReg(len1);
01299     freeReg(len2);
01300     freeReg(totalLen);
01301     if (isReg(left) && !isParmReg(left))
01302         freeReg(left);
01303     if (isReg(right) && !isParmReg(right))
01304         freeReg(right);
01305     return;
01306 }
01307 try {
01308     std::string folded = foldNumeric(&node);
01309     if (!folded.empty()) {
01310         if (node.operator_ == BinaryOpNode::DIVIDE && isIntegerLiteral(folded))
01311             folded += ".0";
01312         if (isFloatLiteral(folded))
01313             pushValue(ensureFloatConstSymbol(folded));
01314         else
01315             pushValue(folded);
01316         return;
01317     }
01318 } catch (...) {
01319 }
01320
01321 auto evalOperand = [&](ASTNode *n) -> std::string {
01322     if (auto var = dynamic_cast<VariableNode *>(n)) {
01323         std::string v;
01324         if (tryGetConstNumeric(var->name, v))
01325             return v;
01326     }
01327     return eval(n);
01328 };
01329
01330 std::string left = evalOperand(node.left.get());
01331 std::string right = evalOperand(node.right.get());
01332
01333 bool needsFloatOp = (lt == VarType::DOUBLE || rt == VarType::DOUBLE ||
01334                     isFloatLiteral(left) || isFloatLiteral(right) ||
01335                     node.operator_ == BinaryOpNode::DIVIDE);
01336
01337 if (node.operator_ == BinaryOpNode::DIVIDE) {
01338     needsFloatOp = true;
01339     if (isIntegerLiteral(left) && !isFloatLiteral(left))
01340         left += ".0";
01341     if (isIntegerLiteral(right) && !isFloatLiteral(right))
01342         right += ".0";

```

```

01343     }
01344
01345     if (node.operator_ == BinaryOpNode::DIV && (isFloatLiteral(left) || isFloatLiteral(right))) {
01346         if (isFloatLiteral(left))
01347             left = std::to_string((long long)std::stod(left));
01348         if (isFloatLiteral(right))
01349             right = std::to_string((long long)std::stod(right));
01350         needsFloatOp = false;
01351     }
01352
01353     if (needsFloatOp && (node.operator_ == BinaryOpNode::PLUS ||
01354         node.operator_ == BinaryOpNode::MINUS ||
01355         node.operator_ == BinaryOpNode::MULTIPLY ||
01356         node.operator_ == BinaryOpNode::DIVIDE)) {
01357         std::string dst;
01358         if (isFloatReg(left) && !isParmReg(left)) {
01359             dst = left;
01360         } else {
01361             dst = allocFloatReg();
01362             emit2("mov", dst, left);
01363         }
01364
01365         emit2(node.operator_ == BinaryOpNode::DIVIDE ? "div" : node.operator_ ==
BinaryOpNode::MULTIPLY ? "mul"
01366
01367         : node.operator_ == BinaryOpNode::MINUS
? "sub"
01368
01369         : "add",
dst, right);
01370
01371         if (isReg(right) && !isParmReg(right))
01372             freeReg(right);
01373         pushValue(dst);
01374         return;
01375     }
01376
01377     auto emitBinary = [&](const char *op) {
01378         std::string dst;
01379         bool leftIsUsableReg = isReg(left) && !isParmReg(left);
01380         if (leftIsUsableReg)
01381             dst = left;
01382         else {
01383             dst = allocReg();
01384             emit2("mov", dst, left);
01385         }
01386         emit2(op, dst, right);
01387         if (isReg(right) && !isParmReg(right))
01388             freeReg(right);
01389         pushValue(dst);
01390     };
01391
01392     switch (node.operator_) {
01393     case BinaryOpNode::PLUS:
01394         emitBinary("add");
01395         break;
01396     case BinaryOpNode::MINUS:
01397         emitBinary("sub");
01398         break;
01399     case BinaryOpNode::MULTIPLY:
01400         emitBinary("mul");
01401         break;
01402     case BinaryOpNode::DIVIDE:
01403         emitBinary("div");
01404         break;
01405     case BinaryOpNode::DIV:
01406         emitBinary("div");
01407         break;
01408     case BinaryOpNode::MOD:
01409         emitBinary("mod");
01410         break;
01411     case BinaryOpNode::AND:
01412         pushLogicalAnd(left, right);
01413         break;
01414     case BinaryOpNode::OR:
01415         pushLogicalOr(left, right);
01416         break;
01417     case BinaryOpNode::EQUAL:
01418         pushCmpResult(left, right, "je");
01419         break;
01420     case BinaryOpNode::NOT_EQUAL:
01421         pushCmpResult(left, right, "jne");
01422         break;
01423     case BinaryOpNode::LESS:
01424         pushCmpResult(left, right, "jl");
01425         break;
01426     case BinaryOpNode::LESS_EQUAL:
01427         pushCmpResult(left, right, "jle");

```

```

01427         break;
01428     case BinaryOpNode::GREATER:
01429         pushCmpResult(left, right, "jg");
01430         break;
01431     case BinaryOpNode::GREATER_EQUAL:
01432         pushCmpResult(left, right, "jge");
01433         break;
01434     default:
01435         throw std::runtime_error("Unsupported binary operator");
01436     }
01437 }
01438
01439 void CodeGenVisitor::visit(UnaryOpNode &node) {
01440     std::string v = eval(node.operand.get());
01441     switch (node.operator_) {
01442     case UnaryOpNode::MINUS: {
01443         std::string t = allocReg();
01444         emit2("mov", t, "0");
01445         emit2("sub", t, v);
01446         pushValue(t);
01447         break;
01448     }
01449     case UnaryOpNode::PLUS: {
01450         pushValue(v);
01451         break;
01452     }
01453     case UnaryOpNode::NOT: {
01454         emit("not " + v);
01455         pushValue(v);
01456         break;
01457     }
01458     }
01459 }
01460
01461 void CodeGenVisitor::visit(VariableNode &node) {
01462     // 'result' inside a function refers to the return value
01463     if (lc(node.name) == "result" && !currentFunctionName.empty()) {
01464         if (!currentFunctionReturnSlot.empty()) {
01465             pushValue(currentFunctionReturnSlot);
01466         } else {
01467             VarType rt = getVarType(currentFunctionName);
01468             int slot = newSlotFor("__funcret_" + currentFunctionName);
01469             if (rt == VarType::DOUBLE)
01470                 setSlotType(slot, VarType::DOUBLE);
01471             else if (rt == VarType::STRING || rt == VarType::PTR || rt == VarType::RECORD)
01472                 setSlotType(slot, VarType::PTR);
01473             else
01474                 setSlotType(slot, VarType::INT);
01475             currentFunctionReturnSlot = slotVar(slot);
01476             pushValue(currentFunctionReturnSlot);
01477         }
01478         return;
01479     }
01480
01481     // Check with-scopes: if this name is a field in an active 'with' block,
01482     // treat it as recordVar.fieldName
01483     for (auto it = withFieldScopes.rbegin(); it != withFieldScopes.rend(); ++it) {
01484         auto fit = it->find(lc(node.name));
01485         if (fit != it->end()) {
01486             auto recExpr = std::make_unique<VariableNode>(fit->second);
01487             FieldAccessNode syntheticField(std::move(recExpr), node.name);
01488             visit(syntheticField);
01489             return;
01490         }
01491     }
01492
01493     if (auto pit = currentParamLocations.find(node.name); pit != currentParamLocations.end()) {
01494         pushValue(pit->second);
01495         return;
01496     }
01497
01498     std::string mangled = findMangledName(node.name);
01499
01500     if (auto ct = compileTimeConstants.find(mangled); ct != compileTimeConstants.end()) {
01501         pushValue(ct->second);
01502         return;
01503     }
01504     if (auto ct2 = compileTimeConstants.find(node.name); ct2 != compileTimeConstants.end()) {
01505         pushValue(ct2->second);
01506         return;
01507     }
01508     if (auto ct3 = compileTimeConstants.find(lc(node.name)); ct3 != compileTimeConstants.end()) {
01509         pushValue(ct3->second);
01510         return;
01511     }
01512 }
01513

```

```

01514         if (auto sit = varSlot.find(mangled); sit != varSlot.end()) {
01515             pushValue(slotVar(sit->second));
01516             return;
01517         }
01518
01519         // Zero-arg function call without parentheses
01520         if (externalFuncs.count(node.name) || funcSignatures.count(node.name)) {
01521             FuncCallNode syntheticCall(node.name, {});
01522             visit(syntheticCall);
01523             return;
01524         }
01525
01526         pushValue(mangled);
01527     }
01528
01529     void CodeGenVisitor::visit(NumberNode &node) {
01530         if (node.isReal || isRealNumber(node.value)) {
01531             std::string reg = allocFloatReg();
01532             emit2("mov", reg, ensureFloatConstSymbol(node.value));
01533             pushValue(reg);
01534         } else {
01535             pushValue(node.value);
01536         }
01537     }
01538
01539     void CodeGenVisitor::visit(StringNode &node) {
01540         std::string sym = internString(node.value);
01541         pushValue(sym);
01542     }
01543
01544     void CodeGenVisitor::visit(BooleanNode &node) {
01545         pushValue(node.value ? "1" : "0");
01546     }
01547
01548     void CodeGenVisitor::visit(EmptyStmtNode &node) {
01549         // No operation
01550     }
01551
01552     void CodeGenVisitor::visit(ConstDeclNode &node) {
01553         for (const auto &assignment : node.assignments) {
01554             if (auto strNode = dynamic_cast<StringNode*>(assignment->value.get())) {
01555                 std::string sym = internString(strNode->value);
01556
01557                 std::string mangledName = mangleVariableName(assignment->identifier);
01558                 compileTimeConstants[mangledName] = mangledName;
01559                 compileTimeConstants[assignment->identifier] = mangledName;
01560
01561                 int slot = newSlotFor(mangledName);
01562                 setVarType(assignment->identifier, VarType::PTR);
01563                 setSlotType(slot, VarType::PTR);
01564                 updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
01565                 prolog.push_back("mov " + slotVar(slot) + ", " + sym);
01566                 continue;
01567             }
01568
01569             try {
01570                 std::string literalValue = evaluateConstantExpression(assignment->value.get());
01571                 bool isFloat = isRealNumber(literalValue);
01572                 if (isFloat)
01573                     literalValue = ensureFloatConstSymbol(literalValue);
01574
01575                 std::string varType = isFloat ? "float" : "int";
01576                 std::string mangledName = mangleVariableName(assignment->identifier);
01577
01578                 compileTimeConstants[mangledName] = (isFloat ? realConstants[literalValue] :
01579 literalValue);
01579                 compileTimeConstants[assignment->identifier] = (isFloat ? realConstants[literalValue]
01580 : literalValue);
01581
01582                 int slot = newSlotFor(mangledName);
01583                 VarType vType = isFloat ? VarType::DOUBLE : VarType::INT;
01584                 setVarType(assignment->identifier, vType);
01585                 setSlotType(slot, vType);
01586
01587                 updateDataSectionInitialValue(slotVar(slot), varType,
01588 isFloat ? realConstants[literalValue] : literalValue);
01588             } catch (const std::runtime_error &) {
01589                 std::string valueReg = eval(assignment->value.get());
01590                 int slot = newSlotFor(assignment->identifier);
01591                 VarType exprType = getExpressionType(assignment->value.get());
01592                 setVarType(assignment->identifier, exprType);
01593                 setSlotType(slot, exprType);
01594                 std::string varLocation = slotVar(slot);
01595                 emit2("mov", varLocation, valueReg);
01596                 if (isReg(valueReg) && !isParmReg(valueReg))
01597                     freeReg(valueReg);
01598             }
01599         }
01600     }

```

```

01599     }
01600 }
01601
01602 void CodeGenVisitor::visit(RepeatStmtNode &node) {
01603     std::string startLabel = newLabel("REPEAT");
01604     std::string endLabel = newLabel("UNTIL");
01605     std::string continueLabel = newLabel("REPEAT_CONTINUE");
01606
01607     loopContinueLabels.push_back(continueLabel);
01608     loopEndLabels.push_back(endLabel);
01609
01610     emitLabel(startLabel);
01611     for (auto &stmt : node.statements)
01612         if (stmt)
01613             stmt->accept(*this);
01614     emitLabel(continueLabel);
01615     std::string condResult = eval(node.condition.get());
01616     emit2("cmp", condResult, "0");
01617     emit1("je", startLabel);
01618     if (isReg(condResult) && !isParmReg(condResult))
01619         freeReg(condResult);
01620     emitLabel(endLabel);
01621     loopContinueLabels.pop_back();
01622     loopEndLabels.pop_back();
01623 }
01624
01625 void CodeGenVisitor::visit(CaseStmtNode &node) {
01626     VarType exprType = getExpressionType(node.expression.get());
01627     std::string switchExpr = eval(node.expression.get());
01628     std::string endLabel = newLabel("CASE_END");
01629     std::vector<std::string> branchLabels;
01630     for (size_t i = 0; i < node.branches.size(); i++)
01631         branchLabels.push_back(newLabel("CASE_" + std::to_string(i)));
01632     std::string elseLabel = newLabel("CASE_ELSE");
01633     for (size_t i = 0; i < node.branches.size(); i++) {
01634         auto &branch = node.branches[i];
01635         for (auto &value : branch->values) {
01636             std::string caseValue;
01637             if ((exprType == VarType::CHAR || exprType == VarType::INT) &&
01638                 dynamic_cast<StringNode*>(value.get())) {
01639                 auto *strNode = static_cast<StringNode*>(value.get());
01640                 if (strNode->value.size() == 1) {
01641                     caseValue = std::to_string((int)(unsigned char)strNode->value[0]);
01642                 } else {
01643                     caseValue = eval(value.get());
01644                 }
01645             } else {
01646                 caseValue = eval(value.get());
01647             }
01648             emit2("cmp", switchExpr, caseValue);
01649             emit1("je", branchLabels[i]);
01650             if (isReg(caseValue) && !isParmReg(caseValue))
01651                 freeReg(caseValue);
01652         }
01653     }
01654     if (node.elseStatement)
01655         emit1("jmp", elseLabel);
01656     else
01657         emit1("jmp", endLabel);
01658     for (size_t i = 0; i < node.branches.size(); i++) {
01659         emitLabel(branchLabels[i]);
01660         if (node.branches[i]->statement)
01661             node.branches[i]->statement->accept(*this);
01662         emit1("jmp", endLabel);
01663     }
01664     if (node.elseStatement) {
01665         emitLabel(elseLabel);
01666         node.elseStatement->accept(*this);
01667     }
01668     emitLabel(endLabel);
01669     if (isReg(switchExpr) && !isParmReg(switchExpr))
01670         freeReg(switchExpr);
01671 }
01672
01673 void CodeGenVisitor::visit(ArrayTypeNode &node) {
01674     // Typically handled within other nodes like VarDeclNode or TypeDeclNode
01675 }
01676
01677 void CodeGenVisitor::visit(ArrayDeclarationNode &node) {
01678     std::string mangledName = findMangledName(node.name);
01679
01680     std::string elementType;
01681     if (auto simpleType = dynamic_cast<SimpleTypeNode*>(node.arrayType->elementType.get())) {
01682         elementType = simpleType->typeName;
01683     } else if (dynamic_cast<ArrayTypeNode*>(node.arrayType->elementType.get())) {
01684         elementType = "array";
01685     } else if (dynamic_cast<RecordTypeNode*>(node.arrayType->elementType.get())) {

```

```

01686         elementType = "record";
01687     } else if (auto *pt = dynamic_cast<PointerTypeNode *>(node.arrayType->elementType.get())) {
01688         elementType = "^" + pt->baseTypeName;
01689     } else {
01690         elementType = "integer";
01691     }
01692
01693     int lowerBound = std::stoi(evaluateConstantExpression(node.arrayType->lowerBound.get()));
01694     int upperBound = std::stoi(evaluateConstantExpression(node.arrayType->upperBound.get()));
01695     int size = upperBound - lowerBound + 1;
01696
01697     int elementSize = 8;
01698
01699     if (isRecordTypeName(elementType)) {
01700         auto it = recordTypes.find(elementType);
01701         if (it != recordTypes.end()) {
01702             elementSize = it->second.size;
01703         }
01704     } else {
01705         elementSize = getArrayElementSize(elementType);
01706     }
01707
01708     ArrayInfo info;
01709     info.elementType = elementType;
01710     info.lowerBound = lowerBound;
01711     info.upperBound = upperBound;
01712     info.size = size;
01713     info.elementSize = elementSize;
01714     mangledName = findMangledName(node.name);
01715     arrayInfo[mangledName] = std::move(info);
01716     int slot = newSlotFor(mangledName);
01717     varSlot[node.name] = slot;
01718     varSlot[mangledName] = slot;
01719     setVarType(node.name, VarType::PTR);
01720     setSlotType(slot, VarType::PTR);
01721     updateDataSectionInitialValue(slotVar(slot), "ptr", "null");
01722     emit3("alloc",
01723         slotVar(slot),
01724         std::to_string(elementSize),
01725         std::to_string(size));
01726     std::string currentScope = getCurrentScopeName();
01727     if (currentScope.empty())
01728         globalArrays.push_back(slotVar(slot));
01729     else
01730         functionScopedArrays[currentScope].push_back(slotVar(slot));
01731 }
01732
01733 void CodeGenVisitor::visit(FieldAccessNode &node) {
01734     // Cross-unit variable reference: UnitName.varName
01735     if (auto *v = dynamic_cast<VariableNode *>(node.recordExpr.get())) {
01736         if (isImportedUnit(v->name)) {
01737             std::string qualifiedName = v->name + "." + node.fieldName;
01738
01739             // Check for imported constant first
01740             auto ct = compileTimeConstants.find(qualifiedName);
01741             if (ct != compileTimeConstants.end()) {
01742                 pushValue(ct->second);
01743                 return;
01744             }
01745
01746             // Check for zero-arg function call: UnitName.FuncName
01747             auto eit = externalFuncs.find(node.fieldName);
01748             if (eit != externalFuncs.end()) {
01749                 FuncCallNode syntheticCall(node.fieldName, {});
01750                 visit(syntheticCall);
01751                 return;
01752             }
01753
01754             VarType vt = getVarType(qualifiedName);
01755             std::string dst;
01756             if (vt == VarType::DOUBLE)
01757                 dst = allocFloatReg();
01758             else if (vt == VarType::PTR || vt == VarType::STRING)
01759                 dst = allocTempPtr();
01760             else
01761                 dst = allocReg();
01762             emit2("mov", dst, qualifiedName);
01763             pushValue(dst);
01764             return;
01765         }
01766     }
01767
01768     std::string baseName;
01769     std::string recType;
01770     bool baseIsDirectPointer = false;
01771
01772     if (auto *v = dynamic_cast<VariableNode *>(node.recordExpr.get())) {

```

```

01773         baseName = findMangledName(v->name);
01774         recType = getVarRecordTypeName(v->name);
01775     } else if (dynamic_cast<ArrayAccessNode *>(node.recordExpr.get())) {
01776         node.recordExpr->accept(*this);
01777         baseName = popValue();
01778         baseIsDirectPointer = true;
01779         recType = getVarRecordTypeNameFromExpr(node.recordExpr.get());
01780     } else {
01781         node.recordExpr->accept(*this);
01782         baseName = popValue();
01783         baseIsDirectPointer = true;
01784         recType = getVarRecordTypeNameFromExpr(node.recordExpr.get());
01785     }
01786
01787     recType = resolveTypeName(lc(recType));
01788     if (recType.empty()) {
01789         throw std::runtime_error("Unknown record type: (empty) for field " + node.fieldName);
01790     }
01791
01792     auto recTypeIt = recordTypes.find(recType);
01793     if (recTypeIt == recordTypes.end()) {
01794         throw std::runtime_error("Unknown record type: " + recType + " for field " +
node.fieldName);
01795     }
01796
01797     const std::string fieldName = lc(node.fieldName);
01798     auto &recInfo = recTypeIt->second;
01799     auto it = recInfo.nameToIndex.find(fieldName);
01800     if (it == recInfo.nameToIndex.end()) {
01801         throw std::runtime_error("Field not found in record: " + fieldName + " in type " +
recType);
01802     }
01803
01804     const auto &fieldInfo = recInfo.fields[it->second];
01805     const int byteOffset = fieldInfo.offset;
01806
01807     std::string basePtr = baseIsDirectPointer ? baseName : ensurePtrBase(baseName);
01808
01809     if (fieldInfo.isArray || isRecordTypeName(fieldInfo.typeName)) {
01810         std::string p = allocTempPtr();
01811         emit2("mov", p, basePtr);
01812         emit2("add", p, std::to_string(byteOffset));
01813         pushValue(p);
01814         return;
01815     }
01816
01817     VarType fieldType = getTypeFromString(fieldInfo.typeName);
01818     std::string dst;
01819     if (fieldType == VarType::DOUBLE) {
01820         dst = allocFloatReg();
01821     } else if (fieldType == VarType::PTR || fieldType == VarType::STRING) {
01822         dst = allocTempPtr();
01823     } else {
01824         dst = allocReg();
01825     }
01826
01827     emit4("load", dst, basePtr, std::to_string(byteOffset), "1");
01828     pushValue(dst);
01829 }
01830
01831 void CodeGenVisitor::visit(AssignmentNode &node) {
01832     // Intercept 'with' scope: if the LHS is a variable matching a record field, rewrite
01833     if (auto *varLHS = dynamic_cast<VariableNode *>(node.variable.get())) {
01834         for (auto it = withFieldScopes.rbegin(); it != withFieldScopes.rend(); ++it) {
01835             auto fit = it->find(lc(varLHS->name));
01836             if (fit != it->end()) {
01837                 auto recExpr = std::make_unique<VariableNode>(fit->second);
01838                 auto fieldAccess = std::make_unique<FieldAccessNode>(std::move(recExpr),
varLHS->name);
01839                 AssignmentNode syntheticAssign(std::move(fieldAccess),
std::move(node.expression));
01840                 visit(syntheticAssign);
01841                 // Move expression back so the original node is not left in a moved-from state
01842                 node.expression = std::move(syntheticAssign.expression);
01843                 return;
01844             }
01845         }
01846     }
01847
01848     if (auto deref = dynamic_cast<PointerDerefNode *>(node.variable.get())) {
01849         std::string rhs = eval(node.expression.get());
01850         std::string ptrVal = eval(deref->pointer.get());
01851         std::string base = ensurePtrBase(ptrVal);
01852
01853         emit4("store", rhs, base, "0", "1");
01854
01855         if (isReg(rhs) && !isParmReg(rhs))

```

```

01856         freeReg(rhs);
01857     if (isReg(ptrVal) && !isParmReg(ptrVal) && ptrVal != base)
01858         freeReg(ptrVal);
01859     if (isReg(base) && !isParmReg(base))
01860         freeReg(base);
01861     return;
01862 }
01863
01864 if (auto arr = dynamic_cast<ArrayAccessNode*>(node.variable.get())) {
01865     ArrayInfo *info = getArrayInfoForArrayAccess(arr);
01866     if (!info)
01867         throw std::runtime_error("Unknown array: " + getArrayNameFromBase(arr->base.get()));
01868
01869     std::string rhs = eval(node.expression.get());
01870     std::string idx = eval(arr->index.get());
01871
01872     if (getExpressionType(arr->index.get()) == VarType::DOUBLE) {
01873         std::string intIdx = allocReg();
01874         emit2("mov", intIdx, idx);
01875         if (isReg(intIdx) && !isParmReg(intIdx))
01876             freeReg(intIdx);
01877         idx = intIdx;
01878     }
01879
01880 #ifdef MXVM_BOUNDS_CHECK
01881     if (info->isDynamic) {
01882         std::string arrName = getArrayNameFromBase(arr->base.get());
01883         std::string mangled = findMangledArrayName(arrName);
01884         auto lenIt = dynArrayLenSlot.find(mangled);
01885         if (lenIt == dynArrayLenSlot.end())
01886             lenIt = dynArrayLenSlot.find(arrName);
01887         if (lenIt != dynArrayLenSlot.end())
01888             emitDynArrayBoundsCheck(idx, slotVar(lenIt->second));
01889     } else {
01890         emitArrayBoundsCheck(idx, info->lowerBound, info->upperBound);
01891     }
01892 #endif
01893
01894     std::string elemIndex;
01895     if (isReg(idx) && !isParmReg(idx)) {
01896         elemIndex = idx;
01897     } else {
01898         elemIndex = allocReg();
01899         emit2("mov", elemIndex, idx);
01900     }
01901     if (info->lowerBound != 0)
01902         emit2("sub", elemIndex, std::to_string(info->lowerBound));
01903
01904     VarType elemType = VarType::INT;
01905     if (info->elementType == "real")
01906         elemType = VarType::DOUBLE;
01907     else if (info->elementType == "string" || info->elementType == "ptr" ||
01908              info->elementType == "pointer" ||
01909              (!info->elementType.empty() && info->elementType[0] == '^'))
01910         elemType = VarType::PTR;
01911
01912     VarType rhsType = getExpressionType(node.expression.get());
01913     if (elemType == VarType::DOUBLE && rhsType != VarType::DOUBLE && !isFloatReg(rhs)) {
01914         std::string f = allocFloatReg();
01915         emit2("mov", f, rhs);
01916         if (isReg(rhs) && !isParmReg(rhs))
01917             freeReg(rhs);
01918         rhs = f;
01919     } else if (elemType != VarType::DOUBLE && rhsType == VarType::DOUBLE && isFloatReg(rhs)) {
01920         std::string ir = allocReg();
01921         emit2("mov", ir, rhs);
01922         if (isReg(rhs) && !isParmReg(rhs))
01923             freeReg(rhs);
01924         rhs = ir;
01925     }
01926
01927     std::string base;
01928     if (auto var = dynamic_cast<VariableNode*>(arr->base.get())) {
01929         std::string mangled = findMangledArrayName(var->name);
01930         base = ensurePtrBase(storageSymbolFor(mangled));
01931     } else if (auto field = dynamic_cast<FieldAccessNode*>(arr->base.get())) {
01932         // Cross-unit array assignment: UnitName.arrayName[index] := value
01933         if (auto *baseVar = dynamic_cast<VariableNode*>(field->recordExpr.get())) {
01934             baseVar && isImportedUnit(baseVar->name) {
01935                 std::string qualifiedName = baseVar->name + "." + field->fieldName;
01936                 base = ensurePtrBase(qualifiedName);
01937             } else {
01938                 field->recordExpr->accept(*this);
01939                 std::string recPtr = popValue();
01940                 std::string recType = getVarRecordTypeNameFromExpr(field->recordExpr.get());
01941                 auto ofs_sz = getRecordFieldOffsetAndSize(recType, field->fieldName);
01942                 int fieldOffset = ofs_sz.first;

```



```

01943
01944         std::string arrayPtr = allocTempPtr();
01945         emit2("mov", arrayPtr, recPtr);
01946         emit2("add", arrayPtr, std::to_string(fieldOffset));
01947         base = arrayPtr;
01948     }
01949     } else if (dynamic_cast<ArrayAccessNode *>(arr->base.get())) {
01950         arr->base->accept(*this);
01951         base = popValue();
01952         base = ensurePtrBase(base);
01953     } else {
01954         throw std::runtime_error("Unsupported array base in assignment");
01955     }
01956
01957     emit4("store", rhs, base, elemIndex, std::to_string(info->elementSize));
01958
01959     if (isReg(rhs) && !isParmReg(rhs))
01960         freeReg(rhs);
01961     if (elemIndex != idx && isReg(idx) && !isParmReg(idx))
01962         freeReg(idx);
01963     freeReg(elemIndex);
01964     return;
01965 }
01966
01967 if (auto field = dynamic_cast<FieldAccessNode *>(node.variable.get())) {
01968     // Cross-unit simple variable assignment: UnitName.varName := value
01969     if (auto *v = dynamic_cast<VariableNode *>(field->recordExpr.get())) {
01970         v && isImportedUnit(v->name)) {
01971         std::string qualifiedName = v->name + "." + field->fieldName;
01972         std::string rhs = eval(node.expression.get());
01973         emit2("mov", qualifiedName, rhs);
01974         if (isReg(rhs) && !isParmReg(rhs))
01975             freeReg(rhs);
01976         return;
01977     }
01978
01979     std::string rhs = eval(node.expression.get());
01980
01981     std::string baseName;
01982     std::string recType;
01983     bool baseIsDirectPointer = false;
01984
01985     if (auto *v = dynamic_cast<VariableNode *>(field->recordExpr.get())) {
01986         baseName = findMangledName(v->name);
01987         recType = getVarRecordTypeName(v->name);
01988     } else {
01989         field->recordExpr->accept(*this);
01990         baseName = popValue();
01991         baseIsDirectPointer = true;
01992         recType = getVarRecordTypeNameFromExpr(field->recordExpr.get());
01993     }
01994
01995     recType = resolveTypeName(lc(recType));
01996     if (recType.empty()) {
01997         throw std::runtime_error("Unknown record type for field assignment: " +
01998             field->fieldName);
01999     }
02000
02001     auto recTypeIt = recordTypes.find(recType);
02002     if (recTypeIt == recordTypes.end()) {
02003         throw std::runtime_error("Unknown record type: " + recType + " for field " +
02004             field->fieldName);
02005     }
02006
02007     const std::string fieldName = lc(field->fieldName);
02008     auto &recInfo = recTypeIt->second;
02009     auto it = recInfo.nameToIndex.find(fieldName);
02010     if (it == recInfo.nameToIndex.end()) {
02011         throw std::runtime_error("Field not found in record: " + fieldName + " in type " +
02012             recType);
02013     }
02014
02015     const auto &fieldInfo = recInfo.fields[it->second];
02016     const int byteOffset = fieldInfo.offset;
02017
02018     std::string basePtr = baseIsDirectPointer ? baseName : ensurePtrBase(baseName);
02019
02020     // Convert between float and int if field type doesn't match rhs type.
02021     // When assigning a float expression (e.g. trunc() result in xmm reg) to an
02022     // integer record field, emit a mov to convert from float register to int
02023     // register first. Without this, the raw IEEE 754 double bit pattern would
02024     // be stored into the integer field, producing garbage values.
02025     VarType fieldType = getTypeFromString(fieldInfo.typeName);
02026     if (fieldType == VarType::INT && isFloatReg(rhs)) {
02027         std::string intReg = allocReg();
02028         emit2("mov", intReg, rhs);
02029         if (isReg(rhs) && !isParmReg(rhs))

```

```

02027         freeReg(rhs);
02028         rhs = intReg;
02029     } else if (fieldType == VarType::DOUBLE && !isFloatReg(rhs) && isReg(rhs)) {
02030         std::string fltReg = allocFloatReg();
02031         emit2("mov", fltReg, rhs);
02032         if (!isParmReg(rhs))
02033             freeReg(rhs);
02034         rhs = fltReg;
02035     }
02036
02037     emit4("store", rhs, basePtr, std::to_string(byteOffset), "1");
02038
02039     if (isReg(rhs) && !isParmReg(rhs))
02040         freeReg(rhs);
02041     if (baseIsDirectPointer && isReg(basePtr) && !isParmReg(basePtr))
02042         freeReg(basePtr);
02043
02044     return;
02045 }
02046
02047 auto varPtr = dynamic_cast<VariableNode *>(node.variable.get());
02048 if (!varPtr)
02049     return;
02050
02051 std::string varName = varPtr->name;
02052
02053 // Set assignment: copy 256 bytes from RHS set to LHS set
02054 if (setVars.count(lc(varName)) || setVars.count(lc(findMangledName(varName)))) {
02055     std::string rhs = eval(node.expression.get());
02056     std::string mangled = findMangledName(varName);
02057     std::string destBase;
02058     auto sit = varSlot.find(mangled);
02059     if (sit != varSlot.end())
02060         destBase = slotVar(sit->second);
02061     else
02062         destBase = mangled;
02063
02064     std::string srcBase = ensurePtrBase(rhs);
02065     std::string dstBase = ensurePtrBase(destBase);
02066     std::string idx = allocReg();
02067     std::string byteVal = allocReg();
02068     emit2("mov", idx, "0");
02069     std::string loopLbl = newLabel("set_copy");
02070     std::string endLbl = newLabel("set_copy_end");
02071     emitLabel(loopLbl);
02072     emit2("cmp", idx, "256");
02073     emit1("jge", endLbl);
02074     emit4("load", byteVal, srcBase, idx, "8");
02075     emit4("store", byteVal, dstBase, idx, "8");
02076     emit2("add", idx, "1");
02077     emit1("jmp", loopLbl);
02078     emitLabel(endLbl);
02079     freeReg(idx);
02080     freeReg(byteVal);
02081     if (isReg(rhs) && !isParmReg(rhs))
02082         freeReg(rhs);
02083     return;
02084 }
02085
02086 std::string rhs;
02087 VarType varType = getVarType(varName);
02088 if ((varType == VarType::CHAR || varType == VarType::INT)) {
02089     if (auto *strNode = dynamic_cast<StringNode *>(node.expression.get())) {
02090         if (strNode->value.size() == 1) {
02091             rhs = std::to_string((int)(unsigned char)strNode->value[0]);
02092         } else {
02093             rhs = eval(node.expression.get());
02094         }
02095     } else {
02096         rhs = eval(node.expression.get());
02097     }
02098 } else {
02099     rhs = eval(node.expression.get());
02100 }
02101
02102 auto it = currentParamLocations.find(varName);
02103 if (it != currentParamLocations.end()) {
02104     emit2("mov", it->second, rhs);
02105     if (isReg(rhs) && !isParmReg(rhs))
02106         freeReg(rhs);
02107     return;
02108 }
02109
02110 if (!currentFunctionName.empty() && (varName == currentFunctionName || varName == "result")) {
02111     VarType rt = getVarType(currentFunctionName);
02112     if (rt == VarType::STRING || rt == VarType::PTR || rt == VarType::RECORD) {
02113         if (currentFunctionReturnsSlot.empty()) {

```

```

02114         int slot = newSlotFor("__funcrct_" + currentFunctionName);
02115         currentFunctionReturnSlot = slotVar(slot);
02116         setSlotType(slot, VarType::PTR);
02117     }
02118     emit2("mov", currentFunctionReturnSlot, rhs);
02119 } else if (rt == VarType::DOUBLE) {
02120     if (currentFunctionReturnSlot.empty()) {
02121         int slot = newSlotFor("__funcrct_" + currentFunctionName);
02122         currentFunctionReturnSlot = slotVar(slot);
02123         setSlotType(slot, VarType::DOUBLE);
02124     }
02125     emit2("mov", currentFunctionReturnSlot, rhs);
02126 } else {
02127     if (currentFunctionReturnSlot.empty()) {
02128         int slot = newSlotFor("__funcrct_" + currentFunctionName);
02129         currentFunctionReturnSlot = slotVar(slot);
02130         setSlotType(slot, VarType::INT);
02131     }
02132     emit2("mov", currentFunctionReturnSlot, rhs);
02133 }
02134 functionSetReturn = true;
02135 } else {
02136     std::string mangled = findMangledName(varName);
02137     auto it = varSlot.find(mangled);
02138     if (it != varSlot.end()) {
02139         emit2("mov", slotVar(it->second), rhs);
02140         recordLocation(varName, {ValueLocation::MEMORY, slotVar(it->second)});
02141     } else {
02142         emit2("mov", mangled, rhs);
02143         recordLocation(varName, {ValueLocation::MEMORY, mangled});
02144     }
02145 }
02146
02147 if (isTempPtr(rhs) && allocatedPtrs.count(rhs) && !isTempPtr(varName))
02148     escapedTempPtrs.insert(rhs);
02149
02150 if (isReg(rhs) && !isParmReg(rhs))
02151     freeReg(rhs);
02152 }
02153
02154 void CodeGenVisitor::visit(ArrayAssignmentNode &node) {
02155     auto it = arrayInfo.find(node.arrayName);
02156     if (it == arrayInfo.end())
02157         throw std::runtime_error("Unknown array: " + node.arrayName);
02158     ArrayInfo &info = it->second;
02159
02160     std::string value = eval(node.value.get());
02161     std::string index = eval(node.index.get());
02162
02163     if (getExpressionType(node.index.get()) == VarType::DOUBLE) {
02164         std::string intIndex = allocReg();
02165         emit2("mov", intIndex, index);
02166         if (isReg(index) && !isParmReg(index))
02167             freeReg(index);
02168         index = intIndex;
02169     }
02170
02171 #ifdef MXVM_BOUNDS_CHECK
02172     if (info.isDynamic) {
02173         auto lenIt = dynArrayLenSlot.find(node.arrayName);
02174         if (lenIt != dynArrayLenSlot.end())
02175             emitDynArrayBoundsCheck(index, slotVar(lenIt->second));
02176     } else {
02177         emitArrayBoundsCheck(index, info.lowerBound, info.upperBound);
02178     }
02179 #endif
02180
02181     std::string elementIndex;
02182     if (isReg(index) && !isParmReg(index)) {
02183         elementIndex = index;
02184     } else {
02185         elementIndex = allocReg();
02186         emit2("mov", elementIndex, index);
02187     }
02188     if (info.lowerBound != 0)
02189         emit2("sub", elementIndex, std::to_string(info.lowerBound));
02190
02191     std::string base = storageSymbolFor(node.arrayName);
02192     base = ensurePtrBase(base);
02193
02194     VarType elemType = VarType::INT;
02195     if (info.elementType == "real")
02196         elemType = VarType::DOUBLE;
02197     else if (info.elementType == "string" || info.elementType == "ptr" ||
02198             info.elementType == "pointer" ||
02199             (!info.elementType.empty() && info.elementType[0] == '^'))
02200         elemType = VarType::PTR;

```

```

02201     else if (isRecordTypeName(info.elementType))
02202         elemType = VarType::RECORD;
02203
02204     VarType rhsType = getExpressionType(node.value.get());
02205     if (elemType == VarType::DOUBLE && rhsType != VarType::DOUBLE && !isFloatReg(value)) {
02206         std::string f = allocFloatReg();
02207         emit2("mov", f, value);
02208         if (isReg(value) && !isParmReg(value))
02209             freeReg(value);
02210         value = f;
02211     } else if (elemType != VarType::DOUBLE && rhsType == VarType::DOUBLE && isFloatReg(value)) {
02212         std::string i = allocReg();
02213         emit2("mov", i, value);
02214         if (isReg(value) && !isParmReg(value))
02215             freeReg(value);
02216         value = i;
02217     }
02218
02219     emit4("store", value, base, elementIndex, std::to_string(info.elementSize));
02220
02221     if (isReg(value) && !isParmReg(value))
02222         freeReg(value);
02223     if (elementIndex != index && isReg(index) && !isParmReg(index))
02224         freeReg(index);
02225     freeReg(elementIndex);
02226 }
02227
02228 void CodeGenVisitor::visit(ArrayAccessNode &node) {
02229     ArrayInfo *info = getArrayInfoForArrayAccess(&node);
02230     if (!info) {
02231         std::string arrayName = getArrayNameFromBase(node.base.get());
02232         throw std::runtime_error("Unknown array: " + arrayName);
02233     }
02234
02235     std::string idx = eval(node.index.get());
02236
02237     if (getExpressionType(node.index.get()) == VarType::DOUBLE) {
02238         std::string intIdx = allocReg();
02239         emit2("mov", intIdx, idx);
02240         if (isReg(idx) && !isParmReg(idx))
02241             freeReg(idx);
02242         idx = intIdx;
02243     }
02244
02245     std::string elemIndex;
02246     if (isReg(idx) && !isParmReg(idx)) {
02247         elemIndex = idx;
02248     } else {
02249         elemIndex = allocReg();
02250         emit2("mov", elemIndex, idx);
02251     }
02252     if (info->lowerBound != 0)
02253         emit2("sub", elemIndex, std::to_string(info->lowerBound));
02254
02255     std::string base;
02256     if (auto var = dynamic_cast<VariableNode*>(node.base.get())) {
02257         std::string mangled = findMangledArrayName(var->name);
02258         base = ensurePtrBase(storageSymbolFor(mangled));
02259     } else if (auto field = dynamic_cast<FieldAccessNode*>(node.base.get())) {
02260         // Cross-unit array access: UnitName.arrayName[index]
02261         if (auto *baseVar = dynamic_cast<VariableNode*>(field->recordExpr.get())) {
02262             baseVar && isImportedUnit(baseVar->name) {
02263                 std::string qualifiedName = baseVar->name + "." + field->fieldName;
02264                 base = ensurePtrBase(qualifiedName);
02265             } else {
02266                 field->recordExpr->accept(*this);
02267                 std::string recPtr = popValue();
02268                 std::string recType = getVarRecordTypeNameFromExpr(field->recordExpr.get());
02269                 auto ofs_sz = getRecordFieldOffsetAndSize(recType, field->fieldName);
02270                 int fieldOffset = ofs_sz.first;
02271
02272                 std::string arrayPtr = allocTempPtr();
02273                 emit2("mov", arrayPtr, recPtr);
02274                 emit2("add", arrayPtr, std::to_string(fieldOffset));
02275                 base = arrayPtr;
02276             }
02277         } else if (dynamic_cast<ArrayAccessNode*>(node.base.get())) {
02278             node.base->accept(*this);
02279             base = popValue();
02280         } else {
02281             throw std::runtime_error("Unsupported array base in access");
02282         }
02283
02284         if (info->elementIsArray || isRecordTypeName(info->elementType)) {
02285             std::string offsetBytes = allocReg();
02286             emit2("mov", offsetBytes, elemIndex);
02287             emit2("mul", offsetBytes, std::to_string(info->elementSize));

```

```

02288
02289         std::string elemPtr = allocTempPtr();
02290         emit2("mov", elemPtr, base);
02291         emit2("add", elemPtr, offsetBytes);
02292         pushValue(elemPtr);
02293
02294         if (elemIndex != idx && isReg(idx) && !isParmReg(idx))
02295             freeReg(idx);
02296         freeReg(elemIndex);
02297         freeReg(offsetBytes);
02298         return;
02299     }
02300
02301     VarType elemType = getExpressionType(&node);
02302     std::string dst;
02303     if (elemType == VarType::DOUBLE)
02304         dst = allocFloatReg();
02305     else if (elemType == VarType::PTR ||
02306             elemType == VarType::STRING)
02307         dst = allocTempPtr();
02308     else
02309         dst = allocReg();
02310
02311     emit4("load", dst, base, elemIndex, std::to_string(info->elementSize));
02312     pushValue(dst);
02313
02314     if (elemIndex != idx && isReg(idx) && !isParmReg(idx))
02315         freeReg(idx);
02316     freeReg(elemIndex);
02317 }
02318
02319 void CodeGenVisitor::visit(RecordTypeNode &node) {
02320     // Typically handled within other nodes
02321 }
02322
02323 void CodeGenVisitor::visit(RecordDeclarationNode &node) {
02324     RecordTypeInfo info;
02325     int offset = 0;
02326
02327     auto lc = [](std::string s) {
02328         std::transform(s.begin(), s.end(), s.begin(), [](unsigned char c) { return
std::tolower(c); });
02329         return s;
02330     };
02331
02332     auto resolveTypeName = [](std::string t) {
02333         t = lc(t);
02334         std::unordered_set<std::string> seen;
02335         while (typeAliases.count(t) && !seen.count(t)) {
02336             seen.insert(t);
02337             t = lc(typeAliases.at(t));
02338         }
02339         return t;
02340     };
02341
02342     for (auto &f : node.recordType->fields) {
02343         auto &fieldDecl = static_cast<VarDeclNode &>(*f);
02344         bool fieldIsArray = false;
02345         ArrayInfo arrInfo{};
02346         std::string fieldTypeName = "unknown";
02347
02348         if (std::holds_alternative<std::unique_ptr<ASTNode>>(fieldDecl.type)) {
02349             auto &typeNode = std::get<std::unique_ptr<ASTNode>>(fieldDecl.type);
02350             if (auto *at = dynamic_cast<ArrayTypeNode *>(typeNode.get())) {
02351                 fieldIsArray = true;
02352                 arrInfo = buildArrayInfoFromNode(at);
02353                 fieldTypeName = "array";
02354             } else if (dynamic_cast<RecordTypeNode *>(typeNode.get())) {
02355                 fieldTypeName = "record";
02356             } else if (auto *pt = dynamic_cast<PointerTypeNode *>(typeNode.get())) {
02357                 fieldTypeName = "^" + lc(pt->baseTypeName);
02358             } else if (auto *st = dynamic_cast<SimpleTypeNode *>(typeNode.get())) {
02359                 fieldTypeName = resolveTypeName(st->typeName);
02360             }
02361         } else {
02362             fieldTypeName = resolveTypeName(std::get<std::string>(fieldDecl.type));
02363         }
02364
02365         for (const auto &rawName : fieldDecl.identifiers) {
02366             RecordField rf;
02367             rf.name = lc(rawName);
02368             rf.offset = offset;
02369             rf.isArray = fieldIsArray;
02370
02371             if (fieldIsArray) {
02372                 rf.arrayInfo = arrInfo;
02373                 rf.typeName = "array";
02374             }
02375         }
02376     }

```

```

02374         rf.size = rf.arrayInfo.size * rf.arrayInfo.elementSize;
02375     } else if (isRecordTypeName(fieldTypeName)) {
02376         rf.typeName = fieldTypeName;
02377         auto it = recordTypes.find(fieldTypeName);
02378         rf.size = (it != recordTypes.end()) ? it->second.size : 8;
02379     } else {
02380         rf.typeName = fieldTypeName;
02381         rf.size = getTypeSizeByName(rf.typeName);
02382     }
02383
02384     info.nameToIndex[rf.name] = (int)info.fields.size();
02385     info.fields.push_back(rf);
02386     offset += rf.size;
02387 }
02388 }
02389
02390 // Handle variant part (if present)
02391 if (!node.recordType->variantTagName.empty()) {
02392     // Add the tag field as a regular integer field
02393     RecordField tagField;
02394     tagField.name = lc(node.recordType->variantTagName);
02395     tagField.typeName = resolveTypeName(node.recordType->variantTagType);
02396     tagField.offset = offset;
02397     tagField.size = getTypeSizeByName(tagField.typeName);
02398     tagField.isArray = false;
02399     info.nameToIndex[tagField.name] = (int)info.fields.size();
02400     info.fields.push_back(tagField);
02401     offset += tagField.size;
02402
02403     // All variant arms share the same starting offset (union semantics)
02404     int variantStart = offset;
02405     int maxArmSize = 0;
02406
02407     for (auto &arm : node.recordType->variantArms) {
02408         int armOffset = variantStart;
02409         int armSize = 0;
02410
02411         for (auto &f : arm.fields) {
02412             auto &fieldDecl = static_cast<VarDeclNode &>(*f);
02413             bool fieldIsArray = false;
02414             ArrayInfo arrInfo{};
02415             std::string fieldTypeName = "unknown";
02416
02417             if (std::holds_alternative<std::unique_ptr<ASTNode>>(fieldDecl.type)) {
02418                 auto &typeNode = std::get<std::unique_ptr<ASTNode>>(fieldDecl.type);
02419                 if (auto *at = dynamic_cast<ArrayTypeNode *>(typeNode.get())) {
02420                     fieldIsArray = true;
02421                     arrInfo = buildArrayInfoFromNode(at);
02422                     fieldTypeName = "array";
02423                 } else if (dynamic_cast<RecordTypeNode *>(typeNode.get())) {
02424                     fieldTypeName = "record";
02425                 } else if (auto *pt = dynamic_cast<PointerTypeNode *>(typeNode.get())) {
02426                     fieldTypeName = "^" + lc(pt->baseTypeName);
02427                 } else if (auto *st = dynamic_cast<SimpleTypeNode *>(typeNode.get())) {
02428                     fieldTypeName = resolveTypeName(st->typeName);
02429                 }
02430             } else {
02431                 fieldTypeName = resolveTypeName(std::get<std::string>(fieldDecl.type));
02432             }
02433
02434             for (const auto &rawName : fieldDecl.identifiers) {
02435                 RecordField rf;
02436                 rf.name = lc(rawName);
02437                 rf.offset = armOffset;
02438                 rf.isArray = fieldIsArray;
02439
02440                 if (fieldIsArray) {
02441                     rf.arrayInfo = arrInfo;
02442                     rf.typeName = "array";
02443                     rf.size = rf.arrayInfo.size * rf.arrayInfo.elementSize;
02444                 } else if (isRecordTypeName(fieldTypeName)) {
02445                     rf.typeName = fieldTypeName;
02446                     auto it = recordTypes.find(fieldTypeName);
02447                     rf.size = (it != recordTypes.end()) ? it->second.size : 8;
02448                 } else {
02449                     rf.typeName = fieldTypeName;
02450                     rf.size = getTypeSizeByName(rf.typeName);
02451                 }
02452
02453                 info.nameToIndex[rf.name] = (int)info.fields.size();
02454                 info.fields.push_back(rf);
02455                 armOffset += rf.size;
02456                 armSize += rf.size;
02457             }
02458         }
02459     }
02460     if (armSize > maxArmSize)

```

```

02461         maxArmSize = armSize;
02462     }
02463
02464     offset = variantStart + maxArmSize;
02465 }
02466
02467     info.size = offset;
02468     std::string recordName = lc(node.name);
02469     recordTypes[recordName] = info;
02470 }
02471
02472 void CodeGenVisitor::visit(TypeDeclNode &node) {
02473     for (auto &typeDecl : node.typeDeclarations) {
02474         typeDecl->accept(*this);
02475     }
02476 }
02477
02478 void CodeGenVisitor::visit(SimpleTypeNode &node) {
02479     // Typically handled within other nodes
02480 }
02481
02482 void CodeGenVisitor::visit(ArrayTypeDeclarationNode &node) {
02483     std::string typeKey = resolveTypeName(lc(node.name));
02484     ArrayInfo info = buildArrayInfoFromNode(node.arrayType.get());
02485     arrayInfo[typeKey] = std::move(info);
02486 }
02487
02488 void CodeGenVisitor::visit(ExitNode &node) {
02489     if (node.expr) {
02490         std::string retVal = eval(node.expr.get());
02491         VarType rt = VarType::UNKNOWN;
02492         if (!currentFunctionName.empty()) {
02493             rt = getVarType(currentFunctionName);
02494         }
02495
02496         if (rt == VarType::STRING || rt == VarType::PTR || rt == VarType::RECORD) {
02497             emit2("mov", "arg0", retVal);
02498         } else if (rt == VarType::DOUBLE) {
02499             emit2("mov", "xmm0", retVal);
02500         } else {
02501             emit2("mov", "rax", retVal);
02502         }
02503
02504         functionSetReturn = true;
02505         if (isReg(retVal) && !isParmReg(retVal))
02506             freeReg(retVal);
02507     }
02508     std::string endLabel = getCurrentEndLabel();
02509     if (!endLabel.empty()) {
02510         emit1("jmp", endLabel);
02511     } else {
02512         emit("ret");
02513     }
02514 }
02515
02516 void CodeGenVisitor::visit(BreakNode &node) {
02517     if (!loopEndLabels.empty()) {
02518         emit1("jmp", loopEndLabels.back());
02519     }
02520 }
02521
02522 void CodeGenVisitor::visit(ContinueNode &node) {
02523     if (!loopContinueLabels.empty()) {
02524         emit1("jmp", loopContinueLabels.back());
02525     }
02526 }
02527
02528 void CodeGenVisitor::visit(NilNode &node) {
02529     pushValue("0");
02530 }
02531
02532 void CodeGenVisitor::visit(PointerTypeNode &node) {
02533     // Handled in VarDeclNode visitor
02534 }
02535
02536 void CodeGenVisitor::visit(PointerDerefNode &node) {
02537     // Evaluate the pointer expression
02538     std::string ptrVal = eval(node.pointer.get());
02539
02540     // Determine the pointed-to type
02541     VarType derefType = VarType::INT;
02542     if (auto varNode = dynamic_cast<VariableNode *>(node.pointer.get())) {
02543         derefType = getPointerDerefType(varNode->name);
02544     }
02545
02546     std::string base = ensurePtrBase(ptrVal);
02547 }

```

```

02548         if (derefType == VarType::DOUBLE) {
02549             std::string result = allocFloatReg();
02550             emit4("load", result, base, "0", "1");
02551             pushValue(result);
02552         } else if (derefType == VarType::PTR || derefType == VarType::STRING) {
02553             std::string result = allocTempPtr();
02554             emit4("load", result, base, "0", "1");
02555             pushValue(result);
02556         } else {
02557             std::string result = allocReg();
02558             emit4("load", result, base, "0", "1");
02559             pushValue(result);
02560         }
02561
02562         if (isReg(ptrVal) && !isParmReg(ptrVal) && ptrVal != base)
02563             freeReg(ptrVal);
02564     }
02565
02566     void CodeGenVisitor::visit(AddressOfNode &node) {
02567         if (auto varNode = dynamic_cast<VariableNode *>(node.operand.get())) {
02568             std::string mangled = findMangledName(varNode->name);
02569             std::string result = allocTempPtr();
02570             emit2("lea", result, mangled);
02571             pushValue(result);
02572         } else {
02573             throw std::runtime_error("@ operator requires a variable operand");
02574         }
02575     }
02576
02577     void CodeGenVisitor::visit(WithStmtNode &node) {
02578         std::string recType = getVarRecordTypeName(node.recordVar);
02579         recType = resolveTypeName(recType);
02580         if (recType.empty()) {
02581             throw std::runtime_error("'with' variable '" + node.recordVar + "' is not a record type");
02582         }
02583         auto it = recordTypes.find(recType);
02584         if (it == recordTypes.end()) {
02585             throw std::runtime_error("Unknown record type '" + recType + "' in with statement");
02586         }
02587         std::unordered_map<std::string, std::string> fieldMap;
02588         for (const auto &[fieldName, idx] : it->second.nameToIndex) {
02589             fieldMap[fieldName] = node.recordVar;
02590         }
02591         withFieldScopes.push_back(std::move(fieldMap));
02592         if (node.statement)
02593             node.statement->accept(*this);
02594         withFieldScopes.pop_back();
02595     }
02596
02597     void CodeGenVisitor::visit(GotoStmtNode &node) {
02598         std::string lbl;
02599         auto it = gotoLabels.find(node.label);
02600         if (it != gotoLabels.end()) {
02601             lbl = it->second;
02602         } else {
02603             lbl = newLabel("GOTO_" + node.label);
02604             gotoLabels[node.label] = lbl;
02605         }
02606         emit1("jmp", lbl);
02607     }
02608
02609     void CodeGenVisitor::visit(LabelStmtNode &node) {
02610         std::string lbl;
02611         auto it = gotoLabels.find(node.label);
02612         if (it != gotoLabels.end()) {
02613             lbl = it->second;
02614         } else {
02615             lbl = newLabel("GOTO_" + node.label);
02616             gotoLabels[node.label] = lbl;
02617         }
02618         emitLabel(lbl);
02619         if (node.statement)
02620             node.statement->accept(*this);
02621     }
02622
02623     void CodeGenVisitor::visit(SetLiteralNode &node) {
02624         // Build a runtime set value: allocate 256 bytes, zero-fill, then set each element
02625         std::string setPtr = allocTempPtr();
02626         emit3("alloc", setPtr, "8", "256");
02627
02628         // Zero-fill
02629         std::string loopIdx = allocReg();
02630         std::string loopStart = newLabel("SET_LIT_INIT");
02631         std::string loopEnd = newLabel("SET_LIT_INIT_END");
02632         emit2("mov", loopIdx, "0");
02633         emitLabel(loopStart);
02634         emit2("cmp", loopIdx, "256");

```



```

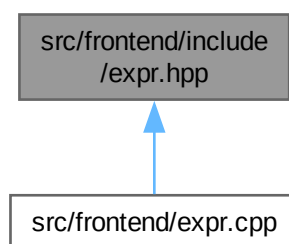
02635         emit1("jge", loopEnd);
02636         emit4("store", "0", setPtr, loopIdx, "8");
02637         emit2("add", loopIdx, "1");
02638         emit1("jmp", loopStart);
02639         emitLabel(loopEnd);
02640         freeReg(loopIdx);
02641
02642         // Set each element
02643         for (auto &elem : node.elements) {
02644             std::string elemVal = eval(elem.get());
02645             emit4("store", "1", setPtr, elemVal, "8");
02646             if (isReg(elemVal) && !isParmReg(elemVal))
02647                 freeReg(elemVal);
02648         }
02649         pushValue(setPtr);
02650     }
02651
02652     void CodeGenVisitor::visit(SetTypeNode &node) {
02653         // Set type declarations handled via type aliases; nothing to emit here
02654     }
02655
02656     void CodeGenVisitor::visit(EnumTypeDeclNode &node) {
02657         auto lc = [](std::string s) {
02658             std::transform(s.begin(), s.end(), s.begin(), [](unsigned char c) { return
std::tolower(c); });
02659             return s;
02660         };
02661         std::string typeName = lc(node.typeName);
02662         std::vector<std::string> valueNames;
02663         for (int i = 0; i < static_cast<int>(node.values.size()); ++i) {
02664             std::string valName = lc(node.values[i]);
02665             enumConstants[valName] = i;
02666             compileTimeConstants[valName] = std::to_string(i);
02667             valueNames.push_back(valName);
02668         }
02669         enumTypes[typeName] = std::move(valueNames);
02670         // Treat enum type as an alias for integer
02671         typeAliases[typeName] = "integer";
02672     }
02673
02674 } // namespace pascal

```

12.64 src/frontend/include/expr.hpp File Reference

Placeholder for Pascal expression utilities (reserved).

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [mxx](#)

12.64.1 Detailed Description

Placeholder for Pascal expression utilities (reserved).

Author

Jared Bruni

Definition in file [expr.hpp](#).

12.65 expr.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef __INCLUDE_EX_H_
00007 #define __INCLUDE_EX_H_
00008
00009 namespace mxx {
00010
00011 }
00012
00013 #endif

```

12.66 src/scanner/include/scanner/expr.hpp File Reference

Expression parser and evaluator for constant integer expressions.

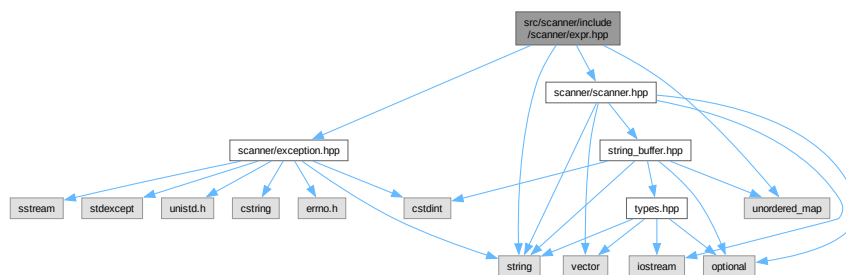
#include "scanner/exception.hpp"

#include "scanner/scanner.hpp"

#include <string>

#include <unordered_map>

Include dependency graph for expr.hpp:

**Classes**

- struct [expr_parser::ExprToken](#)
A single expression token holding its type and optional numeric value.
- class [expr_parser::ExprLexer](#)
Lexer for constant integer expressions.
- class [expr_parser::ExprParser](#)
Recursive-descent parser and evaluator for constant integer expressions.

Namespaces

- namespace [expr_parser](#)

Enumerations

- enum class [expr_parser::ExprTokenType](#) {
[expr_parser::END](#) , [expr_parser::NUMBER](#) , [expr_parser::PLUS](#) , [expr_parser::MINUS](#) ,
[expr_parser::MUL](#) , [expr_parser::DIV](#) , [expr_parser::MOD](#) , [expr_parser::LPAREN](#) ,
[expr_parser::RPAREN](#) , [expr_parser::PLUSEQ](#) , [expr_parser::MINUSEQ](#) , [expr_parser::XOR](#) ,

```

expr_parser::OR , expr_parser::AND , expr_parser::LOGICAL_AND , expr_parser::LOGICAL_OR ,
expr_parser::NOT , expr_parser::EQ , expr_parser::NEQ , expr_parser::GT ,
expr_parser::GTE , expr_parser::LT , expr_parser::LTE , expr_parser::VARIABLE }

```

Token types for the constant expression parser.

Variables

- `std::unordered_map< std::string, uint64_t > expr_parser::vars`

Global variable map for the expression evaluator.

12.66.1 Detailed Description

Expression parser and evaluator for constant integer expressions.

Author

Jared Bruni

Definition in file [expr.hpp](#).

12.67 expr.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef __EXPR_H_
00007 #define __EXPR_H_
00008 #include "scanner/exception.hpp"
00009 #include "scanner/scanner.hpp"
00010 #include <string>
00011 #include <unordered_map>
00012
00013 namespace expr_parser {
00014
00016     enum class ExprTokenType {
00017         END,
00018         NUMBER,
00019         PLUS,
00020         MINUS,
00021         MUL,
00022         DIV,
00023         MOD,
00024         LPAREN,
00025         RPAREN,
00026         PLUSEQ,
00027         MINUSEQ,
00028         XOR,
00029         OR,
00030         AND,
00031         LOGICAL_AND,
00032         LOGICAL_OR,
00033         NOT,
00034         EQ,
00035         NEQ,
00036         GT,
00037         GTE,
00038         LT,
00039         LTE,
00040         VARIABLE
00041     };
00042
00044     struct ExprToken {
00045         ExprTokenType type;
00046         uint64_t value;
00047     };
00048
00050     inline std::unordered_map<std::string, uint64_t> vars;
00051
00058     class ExprLexer {
00059     public:
00060         std::string variable;
00065         ExprLexer(const std::string &input) : scanner(input), pos(0) {
00066             scanner.scan();
00067             next();
00068         }
00069
00071         void next() {

```

```

00072         if (pos >= scanner.size()) {
00073             current = {ExprTokenType::END, 0};
00074             return;
00075         }
00076         auto t = scanner[pos++];
00077         auto val = t.getTokenValue();
00078         if (t.getTokenType() == types::TokenType::TT_NUM) {
00079             current = {ExprTokenType::NUMBER, std::stoull(val)};
00080         } else if (t.getTokenType() == types::TokenType::TT_HEX) {
00081             current = {ExprTokenType::NUMBER, std::stoull(val, nullptr, 16)};
00082         } else if (t.getTokenType() == types::TokenType::TT_ID) {
00083             current = {ExprTokenType::VARIABLE, 0};
00084             variable = val;
00085         } else if (val == "+") {
00086             current = {ExprTokenType::PLUS, 0};
00087         } else if (val == "-") {
00088             current = {ExprTokenType::MINUS, 0};
00089         } else if (val == "*") {
00090             current = {ExprTokenType::MUL, 0};
00091         } else if (val == "/") {
00092             current = {ExprTokenType::DIV, 0};
00093         } else if (val == "%") {
00094             current = {ExprTokenType::MOD, 0};
00095         } else if (val == "(") {
00096             current = {ExprTokenType::LPAREN, 0};
00097         } else if (val == ")") {
00098             current = {ExprTokenType::RPAREN, 0};
00099         } else if (val == "+=") {
00100             current = {ExprTokenType::PLUSEQ, 0};
00101         } else if (val == "-=") {
00102             current = {ExprTokenType::MINUSEQ, 0};
00103         } else if (val == "^") {
00104             current = {ExprTokenType::XOR, 0};
00105         } else if (val == "|") {
00106             current = {ExprTokenType::OR, 0};
00107         } else if (val == "&") {
00108             current = {ExprTokenType::AND, 0};
00109         } else if (val == "&&") {
00110             current = {ExprTokenType::LOGICAL_AND, 0};
00111         } else if (val == "||") {
00112             current = {ExprTokenType::LOGICAL_OR, 0};
00113         } else if (val == "!") {
00114             current = {ExprTokenType::NOT, 0};
00115         } else if (val == "==") {
00116             current = {ExprTokenType::EQ, 0};
00117         } else if (val == "!=") {
00118             current = {ExprTokenType::NEQ, 0};
00119         } else if (val == ">") {
00120             current = {ExprTokenType::GT, 0};
00121         } else if (val == ">=") {
00122             current = {ExprTokenType::GTE, 0};
00123         } else if (val == "<") {
00124             current = {ExprTokenType::LT, 0};
00125         } else if (val == "<=") {
00126             current = {ExprTokenType::LTE, 0};
00127         } else {
00128             current = {ExprTokenType::END, 0};
00129         }
00130     }
00131
00133     ExprToken peek() const { return current; }
00135     void consume() { next(); }
00136
00137 private:
00138     Scanner scanner;
00139     size_t pos;
00140     ExprToken current;
00141 };
00142
00150 class ExprParser {
00151 public:
00156     ExprParser(ExprLexer &lexer) : lexer(lexer) {}
00161     uint64_t parse() { return parseLogical(); }
00162
00163 private:
00164     ExprLexer &lexer;
00165
00167     uint64_t parseAdd() {
00168         uint64_t val = parseTerm();
00169         while (lexer.peek().type == ExprTokenType::PLUS ||
00170              lexer.peek().type == ExprTokenType::MINUS) {
00171             if (lexer.peek().type == ExprTokenType::PLUS) {
00172                 lexer.consume();
00173                 val += parseTerm();
00174             } else {
00175                 lexer.consume();
00176                 val -= parseTerm();
00177             }
00178         }
00179     }

```

```

00177         }
00178     }
00179     return val;
00180 }
00181
00183 uint64_t parseBitwise() {
00184     uint64_t val = parseAdd();
00185     while (lexer.peek().type == ExprTokenType::XOR ||
00186            lexer.peek().type == ExprTokenType::OR ||
00187            lexer.peek().type == ExprTokenType::AND) {
00188         if (lexer.peek().type == ExprTokenType::XOR) {
00189             lexer.consume();
00190             val ^= parseAdd();
00191         } else if (lexer.peek().type == ExprTokenType::OR) {
00192             lexer.consume();
00193             val |= parseAdd();
00194         } else {
00195             lexer.consume();
00196             val &= parseAdd();
00197         }
00198     }
00199     return val;
00200 }
00201
00203 uint64_t parseComparison() {
00204     uint64_t val = parseBitwise();
00205     while (lexer.peek().type == ExprTokenType::EQ ||
00206            lexer.peek().type == ExprTokenType::NEQ ||
00207            lexer.peek().type == ExprTokenType::GT ||
00208            lexer.peek().type == ExprTokenType::GTE ||
00209            lexer.peek().type == ExprTokenType::LT ||
00210            lexer.peek().type == ExprTokenType::LTE) {
00211         if (lexer.peek().type == ExprTokenType::EQ) {
00212             lexer.consume();
00213             val = (val == parseBitwise());
00214         } else if (lexer.peek().type == ExprTokenType::NEQ) {
00215             lexer.consume();
00216             val = (val != parseBitwise());
00217         } else if (lexer.peek().type == ExprTokenType::GT) {
00218             lexer.consume();
00219             val = (val > parseBitwise());
00220         } else if (lexer.peek().type == ExprTokenType::GTE) {
00221             lexer.consume();
00222             val = (val >= parseBitwise());
00223         } else if (lexer.peek().type == ExprTokenType::LT) {
00224             lexer.consume();
00225             val = (val < parseBitwise());
00226         } else {
00227             lexer.consume();
00228             val = (val <= parseBitwise());
00229         }
00230     }
00231     return val;
00232 }
00233
00235 uint64_t parseLogical() {
00236     uint64_t val = parseComparison();
00237     while (lexer.peek().type == ExprTokenType::LOGICAL_OR ||
00238            lexer.peek().type == ExprTokenType::LOGICAL_AND) {
00239         if (lexer.peek().type == ExprTokenType::LOGICAL_OR) {
00240             lexer.consume();
00241             val = val || parseComparison();
00242         } else {
00243             lexer.consume();
00244             val = val && parseComparison();
00245         }
00246     }
00247     return val;
00248 }
00249
00251 uint64_t parseTerm() {
00252     uint64_t val = parseFactor();
00253     while (lexer.peek().type == ExprTokenType::MUL ||
00254            lexer.peek().type == ExprTokenType::DIV ||
00255            lexer.peek().type == ExprTokenType::MOD) {
00256         if (lexer.peek().type == ExprTokenType::MUL) {
00257             lexer.consume();
00258             val *= parseFactor();
00259         } else if (lexer.peek().type == ExprTokenType::DIV) {
00260             lexer.consume();
00261             uint64_t d = parseFactor();
00262             if (d == 0)
00263                 throw std::runtime_error("Division by zero");
00264             val /= d;
00265         } else {
00266             lexer.consume();
00267             uint64_t d = parseFactor();

```

```

00268             if (d == 0)
00269                 throw std::runtime_error("Modulo by zero");
00270             val %= d;
00271         }
00272     }
00273     return val;
00274 }
00275
00277 uint64_t parseFactor() {
00278     if (lexer.peek().type == ExprTokenType::NOT) {
00279         lexer.consume();
00280         return !parseFactor();
00281     } else if (lexer.peek().type == ExprTokenType::MINUS) {
00282         lexer.consume();
00283         return -parseFactor();
00284     } else if (lexer.peek().type == ExprTokenType::NUMBER) {
00285         uint64_t v = lexer.peek().value;
00286         lexer.consume();
00287         return v;
00288     } else if (lexer.peek().type == ExprTokenType::VARIABLE) {
00289         std::string var_name = lexer.variable;
00290         lexer.consume();
00291         auto v = vars.find(var_name);
00292         if (v == vars.end()) {
00293             throw mx::Exception("Error variable name not found...\n");
00294         }
00295         return vars[var_name];
00296     } else if (lexer.peek().type == ExprTokenType::LPAREN) {
00297         lexer.consume();
00298         uint64_t v = parseLogical();
00299         if (lexer.peek().type != ExprTokenType::RPAREN)
00300             throw std::runtime_error("Expected ')'");
00301         lexer.consume();
00302         return v;
00303     }
00304     throw std::runtime_error("Unexpected token");
00305 }
00306 };
00307 } // namespace expr_parser
00308
00309 #endif

```

12.68 src/frontend/include/types.hpp File Reference

Pascal keyword enumeration and string table.

Namespaces

- namespace `mx`

Enumerations

- enum class `mx::Keywords` {
 - `mx::AND`, `mx::ARRAY`, `mx::BEGIN`, `mx::CASE`,
 - `mx::CONST`, `mx::DIV`, `mx::DO`, `mx::DOWNT`,
 - `mx::ELSE`, `mx::END`, `mx::FILE`, `mx::FOR`,
 - `mx::FUNCTION`, `mx::GOTO`, `mx::IF`, `mx::IN`,
 - `mx::LABEL`, `mx::MOD`, `mx::NIL`, `mx::NOT`,
 - `mx::OF`, `mx::OR`, `mx::PACKED`, `mx::PROCEDURE`,
 - `mx::PROGRAM`, `mx::RECORD`, `mx::REPEAT`, `mx::SET`,
 - `mx::THEN`, `mx::TO`, `mx::TYPE`, `mx::UNTIL`,
 - `mx::VAR`, `mx::WHILE`, `mx::WITH` }

Standard Pascal reserved keyword enumeration.

Variables

- static const char * `mx::key_str` []

String representations of `Keywords` values, indexed by enum ordinal.

12.68.1 Detailed Description

Pascal keyword enumeration and string table.

Author

Jared Bruni

Definition in file [types.hpp](#).

12.69 types.hpp

[Go to the documentation of this file.](#)

```
00001
00006 #ifndef __TYPES__H_HPP
00007 #define __TYPES__H_HPP
00008
00009 namespace mxx {
00010
00012     enum class Keywords {
00013         AND,
00014         ARRAY,
00015         BEGIN,
00016         CASE,
00017         CONST,
00018         DIV,
00019         DO,
00020         DOWNT,
00021         ELSE,
00022         END,
00023         FILE,
00024         FOR,
00025         FUNCTION,
00026         GOTO,
00027         IF,
00028         IN,
00029         LABEL,
00030         MOD,
00031         NIL,
00032         NOT,
00033         OF,
00034         OR,
00035         PACKED,
00036         PROCEDURE,
00037         PROGRAM,
00038         RECORD,
00039         REPEAT,
00040         SET,
00041         THEN,
00042         TO,
00043         TYPE,
00044         UNTIL,
00045         VAR,
00046         WHILE,
00047         WITH
00048     };
00049
00051     static const char *key_str[] = {
00052         "and",
00053         "array",
00054         "begin",
00055         "case",
00056         "const",
00057         "div",
00058         "do",
00059         "downto",
00060         "else",
00061         "end",
00062         "file",
00063         "for",
00064         "function",
00065         "goto",
00066         "if",
00067         "in",
00068         "label",
00069         "mod",
00070         "nil",
00071         "not",
00072         "of",
00073         "or",
00074         "packed",
00075         "procedure",
```


`types::TT_NUM` , `types::TT_HEX` , `types::TT_NULL` }

Classification of scanned tokens.

- enum class `types::CharType` {
`types::TT_CHAR` , `types::TT_DIGIT` , `types::TT_SYMBOL` , `types::TT_STRING` ,
`types::TT_SINGLE` , `types::TT_SPACE` , `types::TT_NULL` }

Classification of individual characters during lexing.

- enum class `types::OperatorType` {
`types::OP_INC` , `types::OP_DEC` , `types::OP_LSHIFT_ASSIGN` , `types::OP_RSHIFT_ASSIGN` ,
`types::OP_PLUS_ASSIGN` , `types::OP_MINUS_ASSIGN` , `types::OP_MUL_ASSIGN` , `types::OP_DIV_ASSIGN`
,
`types::OP_MOD_ASSIGN` , `types::OP_AND_ASSIGN` , `types::OP_OR_ASSIGN` , `types::OP_XOR_ASSIGN`
,
`types::OP_LSHIFT` , `types::OP_RSHIFT` , `types::OP_EQ` , `types::OP_NEQ` ,
`types::OP_LE` , `types::OP_GE` , `types::OP_AND_AND` , `types::OP_OR_OR` ,
`types::OP_ARROW` , `types::OP_PLUS` , `types::OP_MINUS` , `types::OP_MUL` ,
`types::OP_DIV` , `types::OP_MOD` , `types::OP_ASSIGN` , `types::OP_AND` ,
`types::OP_OR` , `types::OP_XOR` , `types::OP_NOT` , `types::OP_LT` ,
`types::OP_GT` , `types::OP_LPAREN` , `types::OP_RPAREN` , `types::OP_LBRACKET` ,
`types::OP_RBRACKET` , `types::OP_LBRACE` , `types::OP_RBRACE` , `types::OP_COMMA` ,
`types::OP_SEMICOLON` , `types::OP_COLON` , `types::OP_DOT` , `types::OP_QUESTION` ,
`types::OP_HASH` , `types::OP_SCOPE` , `types::OP_TILDE` , `types::OP_DOLLAR` ,
`types::OP_AT` }

Enumeration of all recognised operator symbols.

- enum class `types::KeywordType` {
`types::KW_LET` , `types::KW_PROC` , `types::KW_DEFINE` , `types::KW_IF` ,
`types::KW_ELSE` , `types::KW_SWITCH` , `types::KW_WHILE` , `types::KW_FOR` ,
`types::KW_RETURN` , `types::KW_BREAK` , `types::KW_CONTINUE` }

Keyword type enumeration.

Functions

- void `types::print_type_TokenType` (std::ostream &out, const `TokenType` &tt)
Print a `TokenType` name to a stream.
- void `types::print_type_CharType` (std::ostream &out, const `CharType` &c)
Print a `CharType` name to a stream.
- std::optional< `OperatorType` > `types::lookUp` (const std::string &op)
Look up an operator string and return its `OperatorType`.
- std::optional< `KeywordType` > `types::lookUp_Keyword` (const std::string &op)
Look up a keyword string and return its `KeywordType`.

Variables

- std::vector< std::string > `types::opName`
Human-readable display names indexed by `OperatorType`.
- std::vector< std::string > `types::opStrings`
Operator string representations indexed by `OperatorType`.
- std::vector< std::string > `types::kwStr` {"let", "proc", "define", "if", "else", "switch", "while", "for", "return", "break", "continue"}
Keyword string representations indexed by `KeywordType`.
- std::vector< std::string > `types::strTokenType` {"Identifier", "Argument", "Symbols", "String", "Numeric", "Hex", "NULL"}
string names of `TokenType` values
- std::vector< std::string > `types::sstrCharType`
string names of `CharType` values

12.70.1 Detailed Description

Token, character, and operator type enumerations for the scanner.

Author

Jared Bruni

Definition in file [types.hpp](#).

12.71 types.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef _TYPES_X_
00007 #define _TYPES_X_
00008
00009 #include <iostream>
00010 #include <optional>
00011 #include <string>
00012 #include <vector>
00013
00014 namespace types {
00015     enum class TokenType {
00016         TT_ID,
00017         TT_ARG,
00018         TT_SYM,
00019         TT_STR,
00020         TT_NUM,
00021         TT_HEX,
00022         TT_NULL
00023     };
00024     enum class CharType {
00025         TT_CHAR,
00026         TT_DIGIT,
00027         TT_SYMBOL,
00028         TT_STRING,
00029         TT_SINGLE,
00030         TT_SPACE,
00031         TT_NULL
00032     };
00033     enum class OperatorType {
00034         OP_INC, // '++'
00035         OP_DEC, // '--'
00036         OP_LSHIFT_ASSIGN, // '<<='
00037         OP_RSHIFT_ASSIGN, // '>>='
00038         OP_PLUS_ASSIGN, // '+='
00039         OP_MINUS_ASSIGN, // '-='
00040         OP_MUL_ASSIGN, // '*='
00041         OP_DIV_ASSIGN, // '/='
00042         OP_MOD_ASSIGN, // '%='
00043         OP_AND_ASSIGN, // '&='
00044         OP_OR_ASSIGN, // '|='
00045         OP_XOR_ASSIGN, // '^='
00046         OP_LSHIFT, // '<<'
00047         OP_RSHIFT, // '>>'
00048         OP_EQ, // '=='
00049         OP_NEQ, // '!='
00050         OP_LE, // '<='
00051         OP_GE, // '>='
00052         OP_AND_AND, // '&&'
00053         OP_OR_OR, // '||'
00054         OP_ARROW, // '->'
00055         OP_PLUS, // '+'
00056         OP_MINUS, // '-'
00057         OP_MUL, // '*'
00058         OP_DIV, // '/'
00059         OP_MOD, // '%'
00060         OP_ASSIGN, // '='
00061         OP_AND, // '&'
00062         OP_OR, // '|'
00063         OP_XOR, // '^'
00064         OP_NOT, // '!'
00065         OP_LT, // '<'
00066         OP_GT, // '>'
00067         OP_LPAREN, // '('
00068         OP_RPAREN, // ')'
00069         OP_LBRACKET, // '['
00070         OP_RBRACKET, // ']'
00071         OP_LBRACE, // '{'
00072         OP_RBRACE, // '}'
00073         OP_COMMA, // ','

```

```

00077         OP_SEMICOLON,      // ';'
00078         OP_COLON,          // ':'
00079         OP_DOT,            // '.'
00080         OP_QUESTION,       // '?'
00081         OP_HASH,           // '#'
00082         OP_SCOPE,          // '::'
00083         OP_TILDE,          // '~'
00084         OP_DOLLAR,         // '$'
00085         OP_AT,             // '@'
00086     };
00087
00089     inline std::vector<std::string> opName = {
00090         "Increment",        // OP_INC          -> '++'
00091         "Decrement",        // OP_DEC          -> '--'
00092         "Left Shift Assign", // OP_LSHIFT_ASSIGN -> '<='
00093         "Right Shift Assign", // OP_RSHIFT_ASSIGN -> '>='
00094         "Plus Assign",       // OP_PLUS_ASSIGN  -> '+'
00095         "Minus Assign",      // OP_MINUS_ASSIGN -> '-'
00096         "Multiply Assign",   // OP_MUL_ASSIGN   -> '*'
00097         "Divide Assign",     // OP_DIV_ASSIGN   -> '/'
00098         "Modulo Assign",     // OP_MOD_ASSIGN   -> '%'
00099         "Bitwise And Assign", // OP_AND_ASSIGN   -> '&'
00100         "Bitwise Or Assign", // OP_OR_ASSIGN    -> '|'
00101         "Bitwise Xor Assign", // OP_XOR_ASSIGN   -> '^'
00102         "Left Shift",       // OP_LSHIFT       -> '<<'
00103         "Right Shift",      // OP_RSHIFT       -> '>>'
00104         "Equal",            // OP_EQ           -> '=='
00105         "Not Equal",        // OP_NEQ         -> '!='
00106         "Less Than Or Equal", // OP_LE          -> '<='
00107         "Greater Than Or Equal", // OP_GE         -> '>='
00108         "Logical And",      // OP_AND_AND      -> '&&'
00109         "Logical Or",       // OP_OR_OR        -> '||'
00110         "Arrow",            // OP_ARROW        -> '->'
00111         "Plus",             // OP_PLUS         -> '+'
00112         "Minus",            // OP_MINUS        -> '-'
00113         "Multiply",         // OP_MUL          -> '*'
00114         "Divide",           // OP_DIV          -> '/'
00115         "Modulo",           // OP_MOD          -> '%'
00116         "Assign",           // OP_ASSIGN       -> '='
00117         "Bitwise And",      // OP_AND          -> '&'
00118         "Bitwise Or",       // OP_OR           -> '|'
00119         "Bitwise Xor",      // OP_XOR          -> '^'
00120         "Not",              // OP_NOT          -> '!'
00121         "Less Than",        // OP_LT           -> '<'
00122         "Greater Than",     // OP_GT           -> '>'
00123         "Left Parenthesis", // OP_LPAREN       -> '('
00124         "Right Parenthesis", // OP_RPAREN       -> ')'
00125         "Left Bracket",     // OP_LBRACKET     -> '['
00126         "Right Bracket",    // OP_RBRACKET     -> ']'
00127         "Left Brace",       // OP_LBRACE       -> '{'
00128         "Right Brace",      // OP_RBRACE       -> '}'
00129         "Comma",            // OP_COMMA        -> ','
00130         "Semicolon",        // OP_SEMICOLON    -> ';'
00131         "Colon",            // OP_COLON        -> ':'
00132         "Dot",              // OP_DOT          -> '.'
00133         "Question Mark",    // OP_QUESTION     -> '?'
00134         "Hash",             // OP_HASH         -> '#'
00135         "Scope",            // OP_SCOPE        -> '::'
00136         "Tilde",            // OP_TILDE        -> '~'
00137         "Dollar",           // OP_DOLLAR       -> '$'
00138         "AT",               // OP_AT           -> '@'
00139     };
00140
00142     inline std::vector<std::string> opStrings = {
00143         "+", // OP_INC
00144         "--", // OP_DEC
00145         "<=", // OP_LSHIFT_ASSIGN
00146         ">=", // OP_RSHIFT_ASSIGN
00147         "+=", // OP_PLUS_ASSIGN
00148         "-=", // OP_MINUS_ASSIGN
00149         "*=", // OP_MUL_ASSIGN
00150         "/=", // OP_DIV_ASSIGN
00151         "%=", // OP_MOD_ASSIGN
00152         "&=", // OP_AND_ASSIGN
00153         "|=", // OP_OR_ASSIGN
00154         "^=", // OP_XOR_ASSIGN
00155         "<<", // OP_LSHIFT
00156         ">>", // OP_RSHIFT
00157         "==", // OP_EQ
00158         "!=", // OP_NEQ
00159         "<=", // OP_LE
00160         ">=", // OP_GE
00161         "&&", // OP_AND_AND
00162         "||", // OP_OR_OR
00163         "->", // OP_ARROW
00164         "+", // OP_PLUS
00165         "-", // OP_MINUS

```

```

00166      "*", // OP_MUL
00167      "/", // OP_DIV
00168      "%", // OP_MOD
00169      "=", // OP_ASSIGN
00170      "&", // OP_AND
00171      "|", // OP_OR
00172      "^", // OP_XOR
00173      "!", // OP_NOT
00174      "<", // OP_LT
00175      ">", // OP_GT
00176      "(", // OP_LPAREN
00177      ")", // OP_RPAREN
00178      "[", // OP_LBRACKET
00179      "]", // OP_RBRACKET
00180      "{", // OP_LBRACE
00181      "}", // OP_RBRACE
00182      ",", // OP_COMMA
00183      ";", // OP_SEMICOLON
00184      ":", // OP_COLON
00185      ".", // OP_DOT
00186      "?", // OP_QUESTION
00187      "#", // OP_HASH
00188      ":", // OP_COLON
00189      "~", // OP_TILDE
00190      "$",
00191      "@",
00192
00193 };
00194
00196 enum class KeywordType {
00197     KW_LET,
00198     KW_PROC,
00199     KW_DEFINE,
00200     KW_IF,
00201     KW_ELSE,
00202     KW_SWITCH,
00203     KW_WHILE,
00204     KW_FOR,
00205     KW_RETURN,
00206     KW_BREAK,
00207     KW_CONTINUE
00208 };
00210 inline std::vector<std::string> kwStr{"let", "proc", "define", "if", "else", "switch", "while",
    "for", "return", "break", "continue"};
00211
00212 extern std::vector<std::string> strTokenType;
00213 extern std::vector<std::string> sstrCharType;
00219 void print_type_TokenType(std::ostream &out, const TokenType &tt);
00225 void print_type_CharType(std::ostream &out, const CharType &c);
00231 std::optional<OperatorType> lookUp(const std::string &op);
00237 std::optional<KeywordType> lookUp_Keyword(const std::string &op);
00238 } // namespace types
00239
00240 #endif

```

12.72 src/frontend/include/validator.hpp File Reference

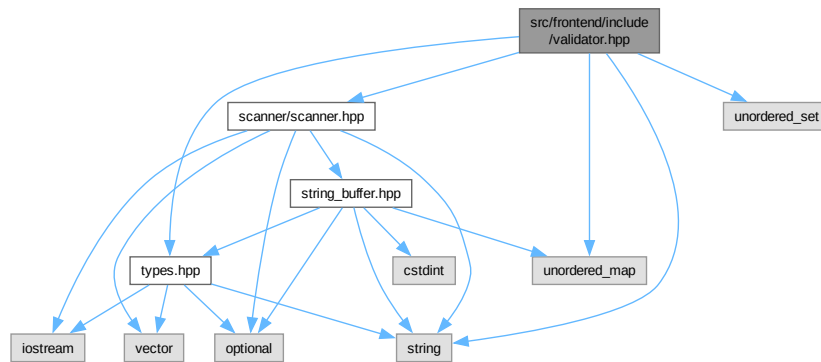
Semantic validator for Pascal programs — scope, type, and declaration checking.

```

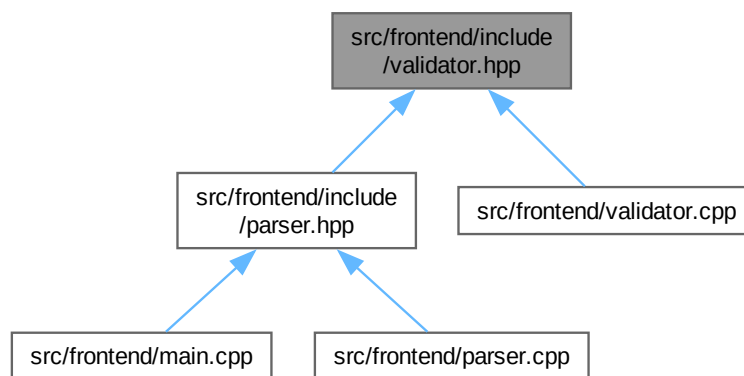
#include "scanner/scanner.hpp"
#include "scanner/types.hpp"
#include <string>
#include <unordered_map>
#include <unordered_set>

```

Include dependency graph for validator.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [mxx::Scope](#)
A single scope level, tracking declared identifiers by category.
- class [mxx::TPValidator](#)
Semantic validator for Pascal programs.

Namespaces

- namespace [mxx](#)

12.72.1 Detailed Description

Semantic validator for Pascal programs — scope, type, and declaration checking.

Author

Jared Bruni

Definition in file [validator.hpp](#).

12.73 validator.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef __VALIDATOR_H_
00007 #define __VALIDATOR_H_
00008
00009 #include "scanner/scanner.hpp"
00010 #include "scanner/types.hpp"
00011 #include <string>
00012 #include <unordered_map>
00013 #include <unordered_set>
00014
00015 namespace mxx {
00016
00017     struct Scope {
00018         std::unordered_set<std::string> vars;
00019         std::unordered_set<std::string> consts;
00020         std::unordered_set<std::string> types;
00021         std::unordered_set<std::string> funcs;
00022         std::unordered_set<std::string> procs;
00023         std::unordered_set<std::string> params;
00024     };
00025
00026
00034     class TPValidator {
00035     public:
00040         explicit TPValidator(const std::string &source_);
00041
00047         bool validate(const std::string &name);
00048
00049     private:
00050         std::unordered_map<std::string, std::unordered_set<std::string> > recordFieldScopesByType;
00051         std::string currentRecordTypeName;
00052         std::vector<Scope> scopeStack;
00053         scan::Scanner scanner;
00054         const scan::TToken *token = nullptr;
00055         std::string source;
00056         std::string filename;
00057         size_t index = 0;
00058         int inFunctionDepth = 0;
00059
00061         bool isPascalKeyword(const std::string &s) const;
00062
00063         std::unordered_set<std::string> declaredVars;
00064         std::unordered_set<std::string> declaredConsts;
00065         std::unordered_set<std::string> declaredFuncs;
00066         std::unordered_set<std::string> declaredProcs;
00067         std::unordered_set<std::string> importedUnits;
00069         void pushScope();
00071         void popScope();
00073         Scope *currentScope();
00075         const Scope *currentScope() const;
00077         bool isVariableInCurrentScope(const std::string &name) const;
00079         std::string currentScopeName() const;
00080
00082         bool isVarDeclaredHere(const std::string &name) const;
00084         bool isTypeDeclaredHere(const std::string &name) const;
00086         bool isFuncDeclaredHere(const std::string &name) const;
00088         bool isProcDeclaredHere(const std::string &name) const;
00090         bool isParamDeclaredHere(const std::string &name) const;
00092         void declareVar(const std::string &name, const scan::TToken *at);
00094         void declareConst(const std::string &name, const scan::TToken *at);
00096         void declareFunc(const std::string &name, const scan::TToken *at);
00098         void declareProc(const std::string &name, const scan::TToken *at);
00100         void declareParam(const std::string &name, const scan::TToken *at);
00101
00103         void checkVar(const std::string &name, const scan::TToken *at);
00105         void checkVarOrConst(const std::string &name, const scan::TToken *at);
00107         void checkConstOnly(const std::string &name, const scan::TToken *at);
00108
00114         void parseConstExpr(const std::unordered_set<std::string> &stops,
00115                             bool constOnly);
00116
00117         std::unordered_set<std::string> declaredTypes;
00118
00120         bool isBuiltinType() const;
00122         void declareType(const std::string &name, const scan::TToken *at);
00124         void checkType(const std::string &name, const scan::TToken *at);
00126         bool isBuiltinConst(const std::string &name) const;
00128         bool next();
00130         bool match(const std::string &s) const;
00132         bool match(types::TokenType t) const;
00134         bool peekIs(const std::string &s);
00135
00137         void require(const std::string &s);
00139         void require(types::TokenType t);

```

```

00141     void requireKW(const std::string &k);
00142
00144     void fail(const std::string &msg);
00146     void failHere(const std::string &msg);
00148     void failAt(const scan::TToken *at, const std::string &msg);
00149
00151     static std::string tokenTypeToString(types::TokenType t);
00153     static std::string lower(std::string s);
00155     std::string found() const;
00157     bool isKW(const std::string &k) const;
00158
00160     void parseProgram();
00162     void parseUnit();
00164     void parseInterfaceSection();
00166     void parseUses();
00168     void parseBlock();
00170     void parseLabelSection();
00172     void parseConstSection();
00174     void parseTypeSection();
00176     void parseVarSection();
00178     void parseSubprogram();
00180     void parseFormalParams();
00182     void parseIdentList();
00184     void parseType(const std::string &);
00186     void parseTypeName();
00188     void parseSubrange();
00190     void parseConstant();
00192     void parseConstSimple();
00194     void parseCompoundStatement();
00196     void parseStatement();
00198     void parseIf();
00200     void parseWhile();
00202     void parseRepeat();
00204     void parseFor();
00206     void parseCase();
00208     void parseCaseLabelList();
00210     void parseCaseLabel();
00212     void parseWith();
00214     void parseGoto();
00216     void parseSimpleOrCallOrAssign();
00218     void parseDesignator();
00220     void parseActualParams();
00222     void parseExprStop(const std::unordered_set<std::string> &stops);
00223
00225     bool isRelOp() const;
00227     bool isAddOp() const;
00229     bool isMulOp() const;
00231     bool isSetOp() const;
00232
00233 private:
00235     static std::string toLower(const std::string &s);
00237     void predeclareType(const std::string &name);
00239     bool isBuiltinTypeName(const std::string &key) const;
00241     void pushRecordFieldScope(const std::string &recordTypeName);
00243     void popRecordFieldScope();
00245     bool inRecordFieldScope() const;
00246
00248     void declareRecordField(const std::string &name, const scan::TToken *at);
00250     void parseFieldIdentList();
00251
00252     int withDepth = 0;
00253 };
00254
00255 } // namespace mxs
00256 #endif

```

12.74 src/frontend/main.cpp File Reference

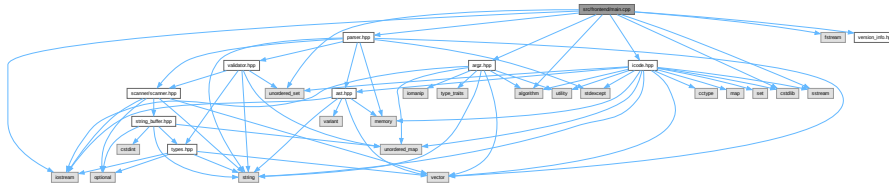
Pascal-to-MXVM compiler command-line entry point.

```

#include "icode.hpp"
#include "parser.hpp"
#include <algorithm>
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <sstream>
#include <unordered_set>
#include "argz.hpp"
#include "version_info.hpp"

```

Include dependency graph for main.cpp:



Functions

- `std::string removeComments` (`const std::string &text`)
- `static bool compileFile` (`const std::string &inputPath`, `const std::string &outputPath`)
- `static std::string extractPasName` (`const std::string &filePath`)
- `int main` (`int argc`, `char **argv`)

12.74.1 Detailed Description

Pascal-to-MXVM compiler command-line entry point.

Author

Jared Bruni

Definition in file [main.cpp](#).

12.74.2 Function Documentation

12.74.2.1 compileFile()

```
bool compileFile (
    const std::string & inputPath,
    const std::string & outputPath) [static]
```

Definition at line 86 of file [main.cpp](#).

References [pascal::CodeGenVisitor::generate\(\)](#), [pascal::PascalParser::isUnitSource\(\)](#), [pascal::mxvmOpt\(\)](#), [pascal::PascalParser::parseProgram\(\)](#), [pascal::PascalParser::parseUnit\(\)](#), [pascal::CodeGenVisitor::registerExternalFunc\(\)](#), [pascal::CodeGenVisitor::registerImportedConst\(\)](#), [pascal::CodeGenVisitor::registerImportedVar\(\)](#), [removeComments\(\)](#), [mxx::TPValidator::validate\(\)](#), [pascal::PascalParser::validator](#), and [pascal::CodeGenVisitor::writeTo\(\)](#).

Referenced by [main\(\)](#).

12.74.2.2 extractPasName()

```
std::string extractPasName (
    const std::string & filePath) [static]
```

Definition at line 245 of file [main.cpp](#).

References [removeComments\(\)](#).

Referenced by [main\(\)](#).

12.74.2.3 main()

```
int main (
    int argc,
    char ** argv)
```

Definition at line 274 of file [main.cpp](#).

References [mxx::Argz<String>::addOptionSingle\(\)](#), [mxx::Argument<String>::arg_value](#), [argc\(\)](#), [argv\(\)](#), [compileFile\(\)](#), [extractPasName\(\)](#), [mxx::Argz<String>::proc\(\)](#), [mxx::ArgException<String>::text\(\)](#), and [VERSION_INFO](#).

12.74.2.4 removeComments()

```
std::string removeComments (
    const std::string & text)
```

Definition at line 19 of file [main.cpp](#).

Referenced by [compileFile\(\)](#), and [extractPasName\(\)](#).

12.75 main.cpp

[Go to the documentation of this file.](#)

```
00001
00006 #include "icode.hpp"
00007 #include "parser.hpp"
00008 #include <algorithm>
00009 #include <cstdlib>
00010 #include <fstream>
00011 #include <iostream>
00012 #include <sstream>
00013 #include <unordered_set>
00014 #include "argz.hpp"
00015 #include "version_info.hpp"
00016
00017
00018
00019 std::string removeComments(const std::string &text) {
00020     std::ostringstream stream;
00021     enum State {
00022         CODE,
00023         BRACE_COMMENT,
00024         PAREN_COMMENT,
00025         STRING
00026     };
00027     State state = CODE;
00028
00029     for (size_t i = 0; i < text.length(); ++i) {
00030         char c = text[i];
00031         char next = (i + 1 < text.length()) ? text[i + 1] : '\0';
00032
00033         switch (state) {
00034             case CODE:
00035                 if (c == '{') {
00036                     state = BRACE_COMMENT;
00037                 } else if (c == '(' && next == '*') {
00038                     state = PAREN_COMMENT;
00039                     i++;
00040                 } else if (c == '"') {
00041                     stream << c;
00042                     state = STRING;
00043                 } else if (c == '/' && next == '/') {
00044
00045                     while (i < text.length() && text[i] != '\n') {
00046                         i++;
00047                     }
00048                     if (i < text.length()) {
00049                         stream << '\n';
00050                     }
00051                 } else {
00052                     stream << c;
00053                 }
00054                 break;
00055             case BRACE_COMMENT:
00056                 if (c == '}') {
00057                     state = CODE;
00058                 }
00059                 break;
00060             case PAREN_COMMENT:
00061                 if (c == '*' && next == ')') {
00062                     state = CODE;
00063                     i++;
00064                 }
00065                 break;
00066             case STRING:
00067                 stream << c;
00068                 if (c == '"') {
00069                     if (next == '"') {
00070                         stream << next;
00071                         i++;
00072                     } else {
00073                         state = CODE;
00074                     }
00075                 }
00076                 break;
00077         }
00078     }
00079     return stream.str();
00080 }
```

```

00077         }
00078     }
00079     break;
00080 }
00081 }
00082
00083     return stream.str();
00084 }
00085
00086 static bool compileFile(const std::string &inputPath, const std::string &outputPath) {
00087     try {
00088         std::ifstream file(inputPath);
00089         if (!file.is_open()) {
00090             std::cerr << "mxx: cannot open file: " << inputPath << "\n";
00091             return false;
00092         }
00093         std::ostringstream buffer;
00094         buffer << file.rdbuf();
00095
00096         if (buffer.str().empty()) {
00097             std::cerr << "mxx: file is empty: " << inputPath << "\n";
00098             return false;
00099         }
00100
00101         std::string source = buffer.str();
00102         pascal::PascalParser parser(removeComments(source));
00103         if (!parser.validator.validate(inputPath)) {
00104             std::cerr << "mxx: validation failed: " << inputPath << "\n";
00105             return false;
00106         }
00107
00108         pascal::CodeGenVisitor emitter;
00109
00110         if (parser.isUnitSource()) {
00111             auto ast = parser.parseUnit();
00112             if (ast) {
00113                 // For units with uses clause, parse each dependency's interface
00114                 // to import their declarations (vars, arrays, consts, types, funcs)
00115                 std::string inputDir = inputPath.substr(0, inputPath.find_last_of("/") + 1);
00116                 if (inputDir.empty()) inputDir = ".";
00117
00118                 static const std::unordered_set<std::string> nativeModules = {"io", "std", "string",
00119 "sdl", "strlib"};
00120                 for (const auto &dep : ast->uses) {
00121                     if (nativeModules.count(dep)) continue;
00122
00123                     std::string unitFile = inputDir + dep + ".pas";
00124                     std::ifstream uf(unitFile);
00125                     if (!uf.is_open()) {
00126                         std::string lowerDep = dep;
00127                         std::transform(lowerDep.begin(), lowerDep.end(), lowerDep.begin(), ::tolower);
00128                         unitFile = inputDir + lowerDep + ".pas";
00129                         uf.open(unitFile);
00130                     }
00131                     if (uf.is_open()) {
00132                         std::ostringstream ubuf;
00133                         ubuf << uf.rdbuf();
00134                         uf.close();
00135                         try {
00136                             pascal::PascalParser unitParser(removeComments(ubuf.str()));
00137                             auto unitAst = unitParser.parseUnit();
00138                             if (unitAst) {
00139                                 for (auto &decl : unitAst->interfaceDecls) {
00140                                     if (auto *fd = dynamic_cast<pascal::FuncDeclNode *>(decl.get()))
00141                                         emitter.registerExternalFunc(fd->name, dep);
00142                                     else if (auto *pd = dynamic_cast<pascal::ProcDeclNode
00143 *>(decl.get()))
00144                                         emitter.registerExternalFunc(pd->name, dep);
00145                                     else if (auto *vd = dynamic_cast<pascal::VarDeclNode
00146 *>(decl.get())) {
00147                                         emitter.registerImportedVar(dep, *vd);
00148                                         continue;
00149                                     } else if (auto *cd = dynamic_cast<pascal::ConstDeclNode
00150 *>(decl.get())) {
00151                                         emitter.registerImportedConst(dep, *cd);
00152                                         continue;
00153                                     }
00154                                 }
00155                                 ast->interfaceDecls.push_back(std::move(decl));
00156                             }
00157                         } catch (...) {
00158                             //
00159                         }
00160                     }
00161                 }
00162             }
00163         }
00164         emitter.generate(ast.get());
00165     }
00166 }

```

```

00160     } else {
00161         auto ast = parser.parseProgram();
00162         if (ast) {
00163             // For programs with unit imports, parse each unit's interface
00164             // to register function/procedure signatures
00165             std::string inputDir = inputPath.substr(0, inputPath.find_last_of("/") + 1);
00166             if (inputDir.empty()) inputDir = ".";
00167
00168             for (const auto &dep : ast->uses) {
00169                 // Skip native modules
00170                 static const std::unordered_set<std::string> nativeModules = {"io", "std",
"string", "sdl", "strlib"};
00171                 if (nativeModules.count(dep)) continue;
00172
00173                 // Try to find and parse the unit's .pas file for interface declarations
00174                 std::string unitFile = inputDir + dep + ".pas";
00175                 std::ifstream uf(unitFile);
00176                 if (!uf.is_open()) {
00177                     // Try lowercase
00178                     std::string lowerDep = dep;
00179                     std::transform(lowerDep.begin(), lowerDep.end(), lowerDep.begin(), ::tolower);
00180                     unitFile = inputDir + lowerDep + ".pas";
00181                     uf.open(unitFile);
00182                 }
00183                 if (uf.is_open()) {
00184                     std::ostringstream ubuf;
00185                     ubuf << uf.rdbuf();
00186                     uf.close();
00187                     try {
00188                         pascal::PascalParser unitParser(removeComments(ubuf.str()));
00189                         auto unitAst = unitParser.parseUnit();
00190                         if (unitAst) {
00191                             // Register interface declarations as forward declarations in the main
AST
00192                             // and track which functions/procedures are external
00193                             for (auto &decl : unitAst->interfaceDecls) {
00194                                 if (auto *fd = dynamic_cast<pascal::FuncDeclNode *>(decl.get()))
00195                                     emitter.registerExternalFunc(fd->name, dep);
00196                                 else if (auto *pd = dynamic_cast<pascal::ProcDeclNode
*>(decl.get()))
00197                                     emitter.registerExternalFunc(pd->name, dep);
00198                                 else if (auto *vd = dynamic_cast<pascal::VarDeclNode
*>(decl.get())) {
00199                                     emitter.registerImportedVar(dep, *vd);
00200                                     continue;
00201                                 } else if (auto *cd = dynamic_cast<pascal::ConstDeclNode
*>(decl.get())) {
00202                                     emitter.registerImportedConst(dep, *cd);
00203                                     continue;
00204                                 }
00205                                 ast->block->declarations.push_back(std::move(decl));
00206                             }
00207                         }
00208                     } catch (...) {
00209                         // Unit source not available; user must ensure correct call signatures
00210                     }
00211                 }
00212             }
00213             emitter.generate(ast.get());
00214         }
00215     }
00216 }
00217
00218 {
00219     std::fstream outFile;
00220     outFile.open(outputPath, std::ios::out);
00221     if (!outFile.is_open()) {
00222         std::cerr << "mxx: cannot open output file: " << outputPath << "\n";
00223         return false;
00224     }
00225     std::ostringstream output;
00226     emitter.writeTo(output);
00227     std::string opt = pascal::mxvmOpt(output.str());
00228     outFile << opt << "\n";
00229 }
00230 return true;
00231 } catch (const pascal::ParseException &e) {
00232     std::cerr << "Parse Error: " << e.what() << "\n";
00233     return false;
00234 } catch (const std::exception &e) {
00235     std::cerr << "Error: " << e.what() << "\n";
00236     return false;
00237 } catch (const scan::ScanExcept &e) {
00238     std::cerr << "Syntax Error: " << e.why() << "\n";
00239     return false;
00240 }
00241 }

```

```

00242
00243 // Extract the declared program or unit name from a Pascal source file.
00244 // Returns empty string if not found.
00245 static std::string extractPasName(const std::string &filePath) {
00246     std::ifstream f(filePath);
00247     if (!f.is_open()) return {};
00248     std::ostringstream buf;
00249     buf << f.rdbuf();
00250     std::string src = removeComments(buf.str());
00251     // Scan tokens: skip whitespace, look for "program"|"unit" then the identifier before ';'
00252     size_t i = 0;
00253     auto skipWS = [&]() {
00254         while (i < src.size() && (src[i] == ' ' || src[i] == '\t' || src[i] == '\r' || src[i] ==
'\n'))
00255             ++i;
00256     };
00257     auto readIdent = [&]() -> std::string {
00258         skipWS();
00259         std::string id;
00260         while (i < src.size() && (std::isalnum(static_cast<unsigned char>(src[i])) || src[i] == '_'))
00261             id += src[i++];
00262         return id;
00263     };
00264     std::string kw = readIdent();
00265     std::string lkw = kw;
00266     std::transform(lkw.begin(), lkw.end(), lkw.begin(), ::tolower);
00267     if (lkw == "program" || lkw == "unit") {
00268         std::string name = readIdent();
00269         if (!name.empty()) return name;
00270     }
00271     return {};
00272 }
00273
00274 int main(int argc, char **argv) {
00275     mx::Argz<std::string> args(argc, argv);
00276     args.addOptionSingle('c', "Compile one or more Pascal source files to .mxvm bytecode")
00277         .addOptionSingleValue('i', "Input Pascal source file")
00278         .addOptionSingleValue('o', "Output .mxvm file");
00279
00280     bool compile_mode = false;
00281     std::string input_file;
00282     std::string output_file;
00283     std::vector<std::string> files;
00284
00285     try {
00286         mx::Argument<std::string> arg;
00287         int r;
00288         while ((r = args.proc(arg)) != -1) {
00289             switch (r) {
00290                 case 'c':
00291                     compile_mode = true;
00292                     break;
00293                 case 'i':
00294                     input_file = arg.arg_value;
00295                     break;
00296                 case 'o':
00297                     output_file = arg.arg_value;
00298                     break;
00299                 case '-':
00300                     files.push_back(arg.arg_value);
00301                     break;
00302             }
00303         }
00304     } catch (const mx::ArgException<std::string> &e) {
00305         std::cerr << "mxx: " << e.text() << "\n";
00306         return EXIT_FAILURE;
00307     }
00308
00309     if (!input_file.empty()) {
00310         // -i / -o mode
00311         if (output_file.empty()) {
00312             std::string name = extractPasName(input_file);
00313             if (!name.empty()) {
00314                 output_file = name + ".mxvm";
00315             } else {
00316                 output_file = input_file;
00317                 if (output_file.size() > 4 && output_file.substr(output_file.size() - 4) == ".pas")
00318                     output_file = output_file.substr(0, output_file.size() - 4) + ".mxvm";
00319                 else
00320                     output_file += ".mxvm";
00321             }
00322         }
00323         return compileFile(input_file, output_file) ? 0 : EXIT_FAILURE;
00324     } else if (compile_mode) {
00325         if (files.empty()) {
00326             std::cerr << "mxx: -c requires at least one source file\n";
00327             return EXIT_FAILURE;

```

12.76 src/vm/main.cpp File Reference

```
#include "version_info.hpp"
#include "argz.hpp"
#include <csignal>
#include <dlfcn.h>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <memory>
#include <mxvm/mxvm.hpp>
#include <set>
#include <sstream>
#include <string>
#include <string_view>
#include <vector>
```

- struct **Args**

Enumerations

- enum class `vm_action` { `null_action` = 0 , `translate` , `interpret` , `compile` }
- enum class `vm_target` { `x86_64_linux` , `x86_64_macos` , `x86_64_windows` }

Functions

- `std::ostream & operator<<` (`std::ostream &out`, `const vm_action &ae`)
- `std::ostream & operator<<` (`std::ostream &out`, `const vm_target &t`)
- `template<typename T>`
`void print_help` (`T &type`)
- `int process_arguments` (`Args *args`)
- `int action_translate` (`const mxvm::Platform &platform`, `std::unique_ptr< mxvm::Program > &program`, `Args *args`)
- `int action_interpret` (`bool only_test`, `std::string_view include_path`, `std::string_view object_path`, `const std::vector< std::string > &argv`, `std::string_view input`, `std::string_view mod_path`)
- `int translate_x64` (`const mxvm::Platform &platform`, `std::unique_ptr< mxvm::Program > &program`, `Args *args`)
- `void collectAndRegisterAllExterns` (`std::unique_ptr< mxvm::Program > &program`)
- `void createMakefile` (`Args *args`)
- `Args proc_args` (`int argc`, `char **argv`)
- `int main` (`int argc`, `char **argv`)
- `void signal_action` (`int signum`)

Variables

- `mxvm::Program * signal_program` = `nullptr`

12.76.1 Detailed Description

MXVM command-line entry point — argument parsing, interpretation, and compilation dispatch.

Author

Jared Bruni

Definition in file `main.cpp`.

12.76.2 Enumeration Type Documentation

12.76.2.1 vm_action

```
enum class vm_action [strong]
```

Enumerator

<code>null_action</code>	
<code>translate</code>	
<code>interpret</code>	
<code>compile</code>	

Definition at line 24 of file `main.cpp`.

12.76.2.2 vm_target

```
enum class vm_target [strong]
```

Enumerator

x86_64_linux	
x86_64_macos	
x86_64_windows	

Definition at line 30 of file [main.cpp](#).

12.76.3 Function Documentation

12.76.3.1 action_interpret()

```
int action_interpret (
    bool only_test,
    std::string_view include_path,
    std::string_view object_path,
    const std::vector< std::string > & argv,
    std::string_view input,
    std::string_view mod_path)
```

Definition at line 584 of file [main.cpp](#).

References [argv\(\)](#), [mxvm::Base::base](#), [mx::Color::BRIGHT_GREEN](#), [Col\(\)](#), [mxvm::debug_mode](#), [mxvm::Parser::generateDebugHTML](#), [mxvm::Parser::generateProgramCode\(\)](#), [mxvm::html_mode](#), [mxvm::Parser::include_path](#), [init\(\)](#), [mxvm::MODE_INTERPRET](#), [mxvm::Parser::module_path](#), [mxvm::Parser::object_path](#), [mx::Color::RED](#), [mxvm::Parser::scan\(\)](#), [signal_action\(\)](#), and [signal_program](#).

Referenced by [process_arguments\(\)](#).

12.76.3.2 action_translate()

```
int action_translate (
    const mxvm::Platform & platform,
    std::unique_ptr< mxvm::Program > & program,
    Args * args)
```

Definition at line 455 of file [main.cpp](#).

References [translate_x64\(\)](#).

Referenced by [process_arguments\(\)](#).

12.76.3.3 collectAndRegisterAllExterns()

```
void collectAndRegisterAllExterns (
    std::unique_ptr< mxvm::Program > & program)
```

Definition at line 749 of file [main.cpp](#).

References [mxvm::Base::base](#).

Referenced by [translate_x64\(\)](#).

12.76.3.4 createMakefile()

```
void createMakefile (
    Args * args)
```

Definition at line 761 of file [main.cpp](#).

References [mx::Color::BRIGHT_GREEN](#), [mx::Color::BRIGHT_RED](#), [Col\(\)](#), [Args::include_path](#), [Args::module_path](#), [Args::object_path](#), [Args::source_file](#), [Args::target](#), and [Args::toolchain](#).

Referenced by [process_arguments\(\)](#).

12.76.3.5 main()

```
int main (
```

```
int argc,
char ** argv)
```

Definition at line 278 of file [main.cpp](#).

References [argc\(\)](#), [argv\(\)](#), [proc_args\(\)](#), and [process_arguments\(\)](#).

12.76.3.6 operator<<() [1/2]

```
std::ostream & operator<< (
    std::ostream & out,
    const vm_action & ae)
```

Definition at line 36 of file [main.cpp](#).

References [compile](#), [interpret](#), [null_action](#), and [translate](#).

12.76.3.7 operator<<() [2/2]

```
std::ostream & operator<< (
    std::ostream & out,
    const vm_target & t)
```

Definition at line 57 of file [main.cpp](#).

References [x86_64_linux](#), [x86_64_macos](#), and [x86_64_windows](#).

12.76.3.8 print_help()

```
template<typename T>
void print_help (
    T & type)
```

Definition at line 90 of file [main.cpp](#).

References [mx::Color::BRIGHT_BLUE](#), [mx::Color::BRIGHT_CYAN](#), [Col\(\)](#), and [VERSION_INFO](#).

Referenced by [proc_args\(\)](#).

12.76.3.9 proc_args()

```
Args proc_args (
    int argc,
    char ** argv)
```

Definition at line 104 of file [main.cpp](#).

References [Args::action](#), [mx::Argz<String>::addOptionSingleValue\(\)](#), [mx::Argument<String>::arg_value](#), [argc\(\)](#), [Args::argv](#), [argv\(\)](#), [Col\(\)](#), [compile](#), [mxvm::DARWIN](#), [mxvm::debug_mode](#), [mxvm::html_mode](#), [Args::include_path](#), [mxvm::instruct_mode](#), [interpret](#), [mxvm::LINUX](#), [Args::Makefile](#), [Args::module_path](#), [Args::object_path](#), [Args::only_test](#), [Args::output_file](#), [Args::platform](#), [Args::platform_argc](#), [Args::platform_argv](#), [print_help\(\)](#), [mx::Argz<String>::proc\(\)](#), [mx::Color::RED](#), [Args::source_file](#), [Args::target](#), [mx::ArgException<String>::text\(\)](#), [Args::toolchain](#), [translate](#), [mxvm::WINX64](#), [x86_64_linux](#), [x86_64_macos](#), and [x86_64_windows](#).

Referenced by [main\(\)](#).

12.76.3.10 process_arguments()

```
int process_arguments (
    Args * args)
```

Definition at line 283 of file [main.cpp](#).

References [Args::action](#), [action_interpret\(\)](#), [action_translate\(\)](#), [Args::argv](#), [mxvm::Base::base](#), [Col\(\)](#), [compile](#), [createMakefile\(\)](#), [mxvm::DARWIN](#), [Args::include_path](#), [interpret](#), [mxvm::LINUX](#), [Args::Makefile](#), [Args::module_path](#), [null_action](#), [Args::object_path](#), [Args::only_test](#), [Args::platform](#), [pos\(\)](#), [mx::Color::RED](#), [Args::source_file](#), [Args::toolchain](#), [translate](#), and [mxvm::WINX64](#).

Referenced by [main\(\)](#).

12.76.3.11 signal_action()

```
void signal_action (
    int signum)
```


Definition at line 552 of file [main.cpp](#).

References [mxvm::debug_mode](#), and [signal_program](#).

Referenced by [action_interpret\(\)](#).

12.76.3.12 translate_x64()

```
int translate_x64 (
    const mxvm::Platform & platform,
    std::unique_ptr< mxvm::Program > & program,
    Args * args)
```

Definition at line 459 of file [main.cpp](#).

References [mxvm::Base::base](#), [mx::Color::BRIGHT_GREEN](#), [Col\(\)](#), [collectAndRegisterAllExterns\(\)](#), [mxvm::debug_mode](#), [mxvm::Parser::generateDebugHTML\(\)](#), [mxvm::Parser::generateProgramCode\(\)](#), [mxvm::html_mode](#), [Args::include_path](#), [mxvm::Parser::include_path](#), [mxvm::MODE_COMPILE](#), [Args::module_path](#), [mxvm::Parser::module_path](#), [Args::object_path](#), [mxvm::Parser::object_path](#), [Args::output_file](#), [mxvm::Parser::platform](#), [mx::Color::RED](#), [mxvm::Parser::scan\(\)](#), and [Args::source_file](#).

Referenced by [action_translate\(\)](#).

12.76.4 Variable Documentation

12.76.4.1 signal_program

```
mxvm::Program* signal_program = nullptr
```

Definition at line 549 of file [main.cpp](#).

Referenced by [action_interpret\(\)](#), and [signal_action\(\)](#).

12.77 main.cpp

[Go to the documentation of this file.](#)

```
00001
00006 #include "version_info.hpp"
00007 #include "argz.hpp"
00008 #include <csignal>
00009 #include <dlfcn.h>
00010 #include <filesystem>
00011 #include <fstream>
00012 #include <iostream>
00013 #include <memory>
00014 #include <mxvm/mxvm.hpp>
00015 #include <set>
00016 #include <sstream>
00017 #include <string>
00018 #include <string_view>
00019 #include <vector>
00020 #ifdef _WIN32
00021 #include <windows.h>
00022 #endif
00023
00024 enum class vm_action {
00025     null_action = 0,
00026     translate,
00027     interpret,
00028     compile
00029 };
00030 enum class vm_target {
00031     x86_64_linux,
00032     x86_64_macos,
00033     x86_64_windows
00034 };
00035
00036 std::ostream &operator<<(std::ostream &out, const vm_action &ae) {
00037     switch (ae) {
00038     case vm_action::null_action:
00039         out << "null action";
00040         break;
00041     case vm_action::translate:
00042         out << "translate";
00043         break;
00044     case vm_action::interpret:
00045         out << "interpret";
00046         break;
00047     case vm_action::compile:
```

```

00048         out << "compille";
00049         break;
00050     default:
00051         std::cerr << "Error: ";
00052         break;
00053     }
00054     return out;
00055 }
00056
00057 std::ostream &operator<<(std::ostream &out, const vm_target &t) {
00058     switch (t) {
00059     case vm_target::x86_64_linux:
00060         out << "linux";
00061         break;
00062     case vm_target::x86_64_macos:
00063         out << "macos";
00064         break;
00065     case vm_target::x86_64_windows:
00066         out << "windows";
00067         break;
00068     }
00069     return out;
00070 }
00071
00072 struct Args {
00073     std::string source_file;
00074     std::string output_file;
00075     std::string module_path = "/usr/local/lib";
00076     std::string object_path = ".";
00077     std::string include_path = "/usr/local/include/mxvm/modules";
00078     vm_action action = vm_action::null_action;
00079     vm_target target = vm_target::x86_64_linux;
00080     std::vector<std::string> argv;
00081     mxvm::Platform platform = mxvm::Platform::LINUX;
00082     int platform_argc = 0;
00083     char **platform_argv = nullptr;
00084     bool Makefile = false;
00085     bool only_test = false;
00086     std::string toolchain;
00087 };
00088
00089 template <typename T>
00090 void print_help(T &type) {
00091     std::cout << Col("MXVM: Compiler/Interpreter ", mx::Color::BRIGHT_CYAN) << "v" << VERSION_INFO <<
00092         "\n";
00093     std::cout << "(C) 2026 " << Col("LostSideDead Software", mx::Color::BRIGHT_BLUE) << "\n";
00094     std::cout << "https://lostsidedead.biz\n";
00095     type.help(std::cout);
00096 }
00097
00098 int process_arguments(Args *args);
00099 int action_translate(const mxvm::Platform &platform, std::unique_ptr<mxvm::Program> &program, Args
00100     *args);
00101 int action_interpret(bool only_test, std::string_view include_path, std::string_view object_path,
00102     const std::vector<std::string> &argv, std::string_view input, std::string_view mod_path);
00103 int translate_x64(const mxvm::Platform &platform, std::unique_ptr<mxvm::Program> &program, Args
00104     *args);
00105 void collectAndRegisterAllExterns(std::unique_ptr<mxvm::Program> &program);
00106 void createMakefile(Args *args);
00107
00108 Args proc_args(int argc, char **argv) {
00109     Args args;
00110     std::string p = "linux";
00111     #ifdef _WIN32
00112         p = "windows";
00113     #endif
00114     #ifdef __APPLE__
00115         p = "macos";
00116     #endif
00117     #ifdef __linux__
00118         p = "linux";
00119     #endif
00120     if (p == "macos") {
00121         args.platform = mxvm::Platform::DARWIN;
00122         args.target = vm_target::x86_64_macos;
00123     } else if (p == "linux") {
00124         args.platform = mxvm::Platform::LINUX;
00125         args.target = vm_target::x86_64_linux;
00126     } else if (p == "windows") {
00127         args.platform = mxvm::Platform::WINX64;
00128         args.target = vm_target::x86_64_windows;
00129     }
00130
00131     std::vector<char *> v;
00132     int stop = 0;
00133     for (int i = 0; i < argc; ++i) {
00134         if (std::string(argv[i]) != "--args") {

```

```

00131         v.push_back(argv[i]);
00132     } else {
00133         stop = i;
00134         break;
00135     }
00136 }
00137 std::vector<std::string> arg_v;
00138 for (int i = stop + 1; i < argc; ++i) {
00139     args.argv.push_back(argv[i]);
00140 }
00141 mx::Argz<std::string> argz(static_cast<int>(v.size()), v.data());
00142 args.platform_argc = argc;
00143 args.platform_argv = argv;
00144 argz.addOptionSingleValue('o', "output file")
00145     .addOptionSingleValue('a', "action")
00146     .addOptionDoubleValue(128, "action", "action to take [translate, interpret]")
00147     .addOptionSingleValue('t', "target")
00148     .addOptionDoubleValue(129, "target", "output target: [linux, macos]")
00149     .addOptionSingle('d', "debug mode")
00150     .addOptionDouble(130, "debug", "debug mode")
00151     .addOptionSingle('i', "instruction trace mode")
00152     .addOptionDouble(131, "instruction", "instruction trace on")
00153     .addOptionSingle('h', "Help")
00154     .addOptionDouble(132, "help", "Help output")
00155     .addOptionDouble(133, "html", "debug html")
00156     .addOptionSingle('D', "print html")
00157     .addOptionSingleValue('x', "object path")
00158     .addOptionDoubleValue(136, "object-path", "Object path")
00159     .addOptionSingleValue('p', "path")
00160     .addOptionDoubleValue(134, "path", "module path")
00161     .addOptionSingleValue('I', "include path")
00162     .addOptionDoubleValue(137, "include", "include path")
00163     .addOptionSingle('m', "generate Makefile")
00164     .addOptionDouble(140, "makefile", "generate Makefile")
00165     .addOptionDouble(141, "dry-run", "check for correctness do not execute")
00166     .addOptionSingleValue('T', "toolchain prefix")
00167     .addOptionDoubleValue(142, "toolchain", "cross-compilation toolchain prefix (e.g.
x86_64-w64-mingw32)");
00168
00169 if (argc == 1) {
00170     print_help(argz);
00171 }
00172 mx::Argument<std::string> arg;
00173 int value = 0;
00174 try {
00175     while ((value = argz.proc(arg)) != -1) {
00176         switch (value) {
00177             case 141:
00178                 args.only_test = true;
00179                 break;
00180             case 'T':
00181             case 142:
00182                 args.toolchain = arg.arg_value;
00183                 break;
00184             case 'm':
00185             case 140:
00186                 args.Makefile = true;
00187                 break;
00188             case 'I':
00189             case 137:
00190                 args.include_path = arg.arg_value;
00191                 break;
00192             case 'x':
00193             case 136:
00194                 args.object_path = arg.arg_value;
00195                 break;
00196             case 134:
00197             case 'p':
00198                 args.module_path = arg.arg_value;
00199                 break;
00200             case 133:
00201             case 'D':
00202                 mxvm::html_mode = true;
00203                 break;
00204             case 'h':
00205             case 132:
00206                 print_help(argz);
00207                 break;
00208             case 'i':
00209             case 131:
00210                 mxvm::instruct_mode = true;
00211                 break;
00212             case 'd':
00213             case 130:
00214                 mxvm::debug_mode = true;
00215                 break;
00216             case 'o':

```

```

00217         args.output_file = arg.arg_value;
00218         break;
00219     case 'a':
00220     case 128:
00221         if (arg.arg_value == "translate") {
00222             args.action = vm_action::translate;
00223         } else if (arg.arg_value == "interpret") {
00224             args.action = vm_action::interpret;
00225         } else if (arg.arg_value == "compile") {
00226             args.action = vm_action::compile;
00227         } else {
00228             throw mx::ArgException<std::string>("Error invalid action value");
00229         }
00230         break;
00231     case 't':
00232     case 129:
00233         if (arg.arg_value == "macos") {
00234             args.platform = mxvm::Platform::DARWIN;
00235             args.target = vm_target::x86_64_macos;
00236         } else if (arg.arg_value == "linux") {
00237             args.platform = mxvm::Platform::LINUX;
00238             args.target = vm_target::x86_64_linux;
00239         } else if (arg.arg_value == "windows") {
00240             args.platform = mxvm::Platform::WINX64;
00241             args.target = vm_target::x86_64_windows;
00242         } else {
00243             throw mx::Exception("Error platform unsupported.\n");
00244         }
00245         break;
00246     default:
00247     case '-':
00248         args.source_file = arg.arg_value;
00249         break;
00250     }
00251 }
00252 } catch (mx::ArgException<std::string> &exec) {
00253     std::cerr << Col("MXVM: ", mx::Color::RED) << "Command Line Argument Parsing Sytnax Error: " <<
exec.text() << "\n";
00254     exit(EXIT_FAILURE);
00255 } catch (const mx::Exception &e) {
00256     std::cerr << Col("MXVM: ", mx::Color::RED) << " Exception: " << e.what() << "\n";
00257 }
00258
00259 if (!args.source_file.empty()) {
00260     constexpr std::string_view ext = ".mxvm";
00261     if (args.source_file.size() < ext.size() ||
00262         std::string_view(args.source_file).substr(args.source_file.size() - ext.size()) != ext) {
00263         args.source_file += ext;
00264     }
00265 }
00266 if (args.source_file.empty()) {
00267     std::cerr << Col("MXVM: Error ", mx::Color::RED) << "source file required.\n";
00268     exit(EXIT_FAILURE);
00269 }
00270 if (args.module_path.empty()) {
00271     std::cerr << Col("MXVM: Error: ", mx::Color::RED) << "please set module path.\n";
00272     exit(EXIT_FAILURE);
00273 }
00274
00275 return args;
00276 }
00277
00278 int main(int argc, char **argv) {
00279     Args args = proc_args(argc, argv);
00280     return process_arguments(&args);
00281 }
00282
00283 int process_arguments(Args *args) {
00284     if (args->Makefile) {
00285         createMakefile(args);
00286         return EXIT_SUCCESS;
00287     }
00288     int exitCode = 0;
00289     if (!std::filesystem::is_regular_file(args->source_file) ||
!std::filesystem::exists(args->source_file)) {
00290         std::cerr << Col("MXVM: Error: ", mx::Color::RED) << "input file: " << args->source_file << " does
not exist or is not regular file.\n";
00291         return EXIT_FAILURE;
00292     }
00293
00294     std::unique_ptr<mxvm::Program> program(new mxvm::Program());
00295     program->setMainBase(program.get());
00296     program->platform = args->platform;
00297     if (args->action == vm_action::compile) {
00298         exitCode = action_translate(args->platform, program, args);
00299         if (exitCode == 0) {
00300             if (mxvm::Program::base != nullptr && !mxvm::Program::base->root_name.empty()) {

```

```

00301         if (args->platform == mxvm::Platform::LINUX || args->platform ==
mxvm::Platform::WINX64) {
00302             std::vector<std::string> object_files;
00303             std::string assembler = args->toolchain.empty() ? "as" : args->toolchain + "-as";
00304             const char *as_env = getenv("AS");
00305             if (as_env != nullptr) {
00306                 assembler = as_env;
00307             }
00308             std::string asflags;
00309             const char *asf = getenv("ASFLAGS");
00310             if (asf != nullptr) {
00311                 asflags = asf;
00312             }
00313
00314             for (auto &f : mxvm::Program::base->filenames) {
00315                 std::string obj_file = f;
00316                 size_t pos = obj_file.rfind(".s");
00317                 if (pos != std::string::npos) {
00318                     obj_file.replace(pos, 2, ".o");
00319                 } else {
00320                     obj_file += ".o";
00321                 }
00322                 std::ostringstream as_cmd;
00323                 as_cmd << assembler << " " << asflags << " " << f << " -o " << obj_file;
00324                 std::cout << as_cmd.str() << "\n";
00325
00326                 int as_result = system(as_cmd.str().c_str());
00327                 if (as_result != 0) {
00328                     std::cerr << Col("MXVM: ", mx::Color::RED) << "Assembly failed\n";
00329                     return EXIT_FAILURE;
00330                 }
00331                 object_files.push_back(obj_file);
00332             }
00333
00334             std::string linker = args->toolchain.empty() ? "cc" : args->toolchain + "-gcc";
00335             const char *cc_env = getenv("CC");
00336             if (cc_env != nullptr) {
00337                 linker = cc_env;
00338             }
00339             std::string ldflags;
00340             const char *ldf = getenv("LDFLAGS");
00341             if (ldf != nullptr) {
00342                 ldflags = ldf;
00343             }
00344             ldflags += " -lm ";
00345
00346             std::ostringstream modules_archives;
00347             std::set<std::string> arch;
00348             for (auto &m : mxvm::Program::base->external) {
00349                 if (m.module == true && m.mod != "main" && m.name != "strlen") {
00350                     arch.insert(m.mod);
00351                 }
00352             }
00353
00354             for (auto &m : arch) {
00355                 modules_archives << args->module_path << "/modules/" << m << "/libmxvm_" << m <<
"_static.a ";
00356             }
00357
00358             std::ostringstream cc_cmd;
00359             cc_cmd << linker << " ";
00360
00361             for (auto &obj : object_files) {
00362                 cc_cmd << obj << " ";
00363             }
00364
00365             cc_cmd << modules_archives.str() << " "
00366                 << ldflags << " "
00367                 << "-o " << mxvm::Program::base->root_name;
00368
00369             std::cout << cc_cmd.str() << "\n";
00370             int cc_result = system(cc_cmd.str().c_str());
00371             if (cc_result != 0) {
00372                 std::cerr << Col("MXVM: ", mx::Color::RED) << "Linking failed\n";
00373                 return EXIT_FAILURE;
00374             }
00375         } else if (args->platform == mxvm::Platform::DARWIN) {
00376             std::vector<std::string> object_files;
00377             std::string clang = args->toolchain.empty() ? "clang" : args->toolchain +
"-clang";
00378             const char *clang_env = getenv("CLANG");
00379             if (clang_env != nullptr) {
00380                 clang = clang_env;
00381             }
00382             std::string cflags;
00383             const char *cflags_env = getenv("CFLAGS");
00384             if (cflags_env != nullptr) {

```

```

00385         cflags = cflags_env;
00386     }
00387
00388     for (auto &f : mxvm::Program::base->filenames) {
00389         std::string obj_file = f;
00390         size_t pos = obj_file.rfind(".s");
00391         if (pos != std::string::npos) {
00392             obj_file.replace(pos, 2, ".o");
00393         } else {
00394             obj_file += ".o";
00395         }
00396         std::ostringstream clang_cmd;
00397         clang_cmd << clang << " -c " << cflags << " " << f << " -o " << obj_file;
00398         std::cout << clang_cmd.str() << "\n";
00399         int clang_result = system(clang_cmd.str().c_str());
00400         if (clang_result != 0) {
00401             std::cerr << Col("MXVM: ", mx::Color::RED) << "Assembly failed\n";
00402             return EXIT_FAILURE;
00403         }
00404         object_files.push_back(obj_file);
00405     }
00406
00407     std::string ldflags;
00408     const char *ldf = getenv("LDFLAGS");
00409     if (ldf != nullptr) {
00410         ldflags = ldf;
00411     }
00412
00413     ldflags += " -lm ";
00414
00415     std::ostringstream modules_archives;
00416     std::set<std::string> arch;
00417     for (auto &m : mxvm::Program::base->external) {
00418         if (m.module == true && m.mod != "main" && m.name != "strlen") {
00419             arch.insert(m.mod);
00420         }
00421     }
00422     for (auto &m : arch) {
00423         modules_archives << args->module_path << "/modules/" << m << "/libmxvm_" << m <<
00424         "_static.a ";
00425     }
00426     std::ostringstream clang_link_cmd;
00427     clang_link_cmd << clang << " ";
00428     for (auto &obj : object_files) {
00429         clang_link_cmd << obj << " ";
00430     }
00431     clang_link_cmd << modules_archives.str() << " "
00432         << ldflags << " "
00433         << "-o " << mxvm::Program::base->root_name;
00434     std::cout << clang_link_cmd.str() << "\n";
00435     int clang_link_result = system(clang_link_cmd.str().c_str());
00436     if (clang_link_result != 0) {
00437         std::cerr << Col("MXVM: ", mx::Color::RED) << "Linking failed\n";
00438         return EXIT_FAILURE;
00439     }
00440 }
00441 }
00442 } else if (args->action == vm_action::translate) {
00443     exitCode = action_translate(args->platform, program, args);
00444 } else if (args->action == vm_action::interpret && !args->source_file.empty()) {
00445     exitCode = action_interpret(args->only_test, args->include_path, args->object_path,
00446     args->argv, args->source_file, args->module_path);
00447 } else if (args->action == vm_action::null_action && !args->source_file.empty()) {
00448     exitCode = action_interpret(args->only_test, args->include_path, args->object_path,
00449     args->argv, args->source_file, args->module_path);
00450 } else {
00451     std::cerr << Col("MXVM: Error ", mx::Color::RED) << "invalid action/command\n";
00452     return EXIT_FAILURE;
00453 }
00454 return exitCode;
00455 }
00456 int action_translate(const mxvm::Platform &platform, std::unique_ptr<mxvm::Program> &program, Args
00457 *args) {
00458     return translate_x64(platform, program, args);
00459 }
00460 int translate_x64(const mxvm::Platform &platform, std::unique_ptr<mxvm::Program> &program, Args *args)
00461 {
00462     std::string_view include_path = args->include_path;
00463     std::string_view object_path = args->object_path;
00464     std::string_view input = args->source_file;
00465     std::string_view mod_path = args->module_path;
00466     std::string_view output = args->output_file;
00467     try {

```

```

00467         std::string input_file(input);
00468         std::fstream file;
00469         file.open(input_file, std::ios::in);
00470         if (!file.is_open()) {
00471             throw mx::Exception("Error could not open file: " + input_file);
00472         }
00473         std::ostringstream stream;
00474         stream < file.rdbuf();
00475         file.close();
00476         mxvm::Parser parser(stream.str());
00477         parser.platform = platform;
00478         parser.scan();
00479
00480         program->filename = input_file;
00481         parser.module_path = std::string(mod_path);
00482         parser.object_path = std::string(object_path);
00483         parser.include_path = include_path;
00484         if (parser.generateProgramCode(mxvm::Mode::MODE_COMPILE, program)) {
00485             collectAndRegisterAllExterns(program);
00486             std::string output_file(output);
00487             std::string program_name = output_file.empty() ? program->name + ".s" : output_file;
00488             std::fstream file;
00489             if (program->root_name == program->name)
00490                 program->object = false;
00491             else
00492                 program->object = true;
00493             file.open(program_name, std::ios::out);
00494             if (file.is_open()) {
00495                 std::ostringstream code_v;
00496                 program->generateCode(platform, program->object, code_v);
00497                 program->assembly_code = code_v.str();
00498                 std::string opt_code = program->gen_optimize(program->assembly_code, platform);
00499                 file < opt_code;
00500                 if (mxvm::html_mode) {
00501                     std::ofstream htmlFile(program->name + ".html");
00502                     if (htmlFile.is_open()) {
00503                         parser.generateDebugHTML(htmlFile, program);
00504                         std::cout < Col("MXVM: Generated Debug HTML for: ", mx::Color::BRIGHT_GREEN) <
program->name < "\n";
00505                     }
00506                     htmlFile.close();
00507                 }
00508                 file.close();
00509                 if (mxvm::Program::base != nullptr) {
00510                     mxvm::Program::base->add_filename(program->name + ".s");
00511                 }
00512             }
00513             else {
00514                 std::cerr < Col("MXVM: Exception: ", mx::Color::RED) < "Failed to generate intermedaite
code" < "\n";
00515                 exit(EXIT_FAILURE);
00516             }
00517             } catch (const mx::Exception &e) {
00518                 std::cerr < Col("MXVM: Exception: ", mx::Color::RED) < e.what() < "\n";
00519                 if (mxvm::debug_mode) {
00520                     program->print(std::cerr);
00521                     program->memoryDump(std::cerr);
00522                 }
00523                 return EXIT_FAILURE;
00524             } catch (const std::runtime_error &e) {
00525                 std::cerr < Col("MXVM: Runtime Error: ", mx::Color::RED) < e.what() < "\n";
00526                 if (mxvm::debug_mode) {
00527                     program->print(std::cerr);
00528                     program->memoryDump(std::cerr);
00529                 }
00530                 return EXIT_FAILURE;
00531             } catch (const std::exception &e) {
00532                 std::cerr < Col("MXVM: Exception: ", mx::Color::RED) < e.what() < "\n";
00533                 if (mxvm::debug_mode) {
00534                     program->print(std::cerr);
00535                     program->memoryDump(std::cerr);
00536                 }
00537                 return EXIT_FAILURE;
00538             } catch (...) {
00539                 std::cerr < Col("MXVM: ", mx::Color::RED) < "Unknown Exception.\n";
00540                 if (mxvm::debug_mode) {
00541                     program->print(std::cerr);
00542                     program->memoryDump(std::cerr);
00543                 }
00544                 return EXIT_FAILURE;
00545             }
00546             return EXIT_SUCCESS;
00547         }
00548
00549         mxvm::Program *signal_program = nullptr;
00550
00551         #ifndef _WIN32

```

```

00552 void signal_action(int signum) {
00553     if (signum == SIGINT) {
00554         std::cout << "MXVM: Signal SIGINT Received Exiting...\n";
00555         if (mxvm::debug_mode) {
00556             std::cout << "MXVM: Debug Mode Dumping Memory.\n";
00557             if (signal_program != nullptr)
00558                 signal_program->memoryDump(std::cout);
00559         }
00560         exit(EXIT_FAILURE);
00561     }
00562 }
00563 #else
00564 BOOL WINAPI CtrlHandler(DWORD ctrlType) {
00565     switch (ctrlType) {
00566     case CTRL_C_EVENT: {
00567         std::cout << "MXVM: Signal SIGINT Received Exiting...\n";
00568         if (mxvm::debug_mode) {
00569             std::cout << "MXVM: Debug Mode Dumping Memory.\n";
00570             if (signal_program != nullptr)
00571                 signal_program->memoryDump(std::cout);
00572         }
00573         exit(EXIT_FAILURE);
00574     }
00575     return TRUE;
00576     case CTRL_CLOSE_EVENT:
00577         return TRUE;
00578     default:
00579         return FALSE;
00580     }
00581 }
00582 #endif
00583
00584 int action_interpret(bool only_test, std::string_view include_path, std::string_view object_path,
00585     const std::vector<std::string> &argv, std::string_view input, std::string_view mod_path) {
00586     int exitCode = 0;
00587     std::unique_ptr<mxvm::Program> program(new mxvm::Program());
00588     program->setArgs(argv);
00589     program->setMainBase(program.get());
00590     std::fstream debug_output;
00591     if (mxvm::debug_mode) {
00592         debug_output.open("debug_info.txt", std::ios::out);
00593         if (!debug_output.is_open()) {
00594             std::cerr << Col("MXVM: Error ", mx::Color::RED) << "could not open debug_info.txt\n";
00595         }
00596     }
00597     signal_program = program.get();
00598 #ifndef _WIN32
00599     struct sigaction sa;
00600     sa.sa_handler = signal_action;
00601     sigemptyset(&sa.sa_mask);
00602     sa.sa_flags = 0;
00603     sigaction(SIGINT, &sa, nullptr);
00604 #else
00605     SetConsoleCtrlHandler(CtrlHandler, TRUE);
00606 #endif
00607
00608     try {
00609         std::string input_file(input);
00610         std::fstream file;
00611         file.open(input_file, std::ios::in);
00612         if (!file.is_open()) {
00613             throw mx::Exception("Error could not open file: " + input_file);
00614         }
00615         program->filename = input_file;
00616         std::ostream stream;
00617         stream << file.rdbuf();
00618         file.close();
00619         mxvm::Parser parser(stream.str());
00620         parser.scan();
00621         bool uses_std_module = false;
00622         parser.module_path = std::string(mod_path);
00623         parser.object_path = std::string(object_path);
00624         parser.include_path = std::string(include_path);
00625
00626         class ArgsRaii {
00627         public:
00628             ArgsRaii() = default;
00629             void init(const std::vector<std::string> &argv, void *handle) {
00630                 std::vector<const char *> c_argv;
00631                 c_argv.push_back("mxvmc");
00632                 for (const auto &arg : argv) {
00633                     c_argv.push_back(arg.c_str());
00634                 }
00635
00636                 void *set_args = (void *)dlsym(handle, "set_program_args");
00637                 using set_program_args_t = void (*)(int, const char **);
00638                 if (set_args) {

```



```

00638         auto func = reinterpret_cast<set_program_args_t>(set_args);
00639         func(static_cast<int>(c_argv.size()), c_argv.data());
00640         set = true;
00641         handle_ = handle;
00642     }
00643 }
00644 ~ArgsRaii() {
00645     void *free_args = (void *)dlsym(handle_, "free_program_args");
00646     using set_program_args_t = void (*)();
00647     if (free_args) {
00648         auto func = reinterpret_cast<set_program_args_t>(free_args);
00649         func();
00650         set = false;
00651     }
00652 }
00653
00654 bool set = false;
00655 void *handle_ = nullptr;
00656 };
00657
00658 if (parser.generateProgramCode(mxvm::Mode::MODE_INTERPRET, program)) {
00659     if (program->object) {
00660         throw mx::Exception("Requires one program object to execute");
00661     }
00662     ArgsRaii args_raii;
00663
00664     if (mxvm::Program::base != nullptr) {
00665         void *handle = nullptr;
00666         for (const auto &ext : mxvm::Program::base->external_functions) {
00667             if (ext.second.mod_name == "std") {
00668                 uses_std_module = true;
00669                 handle = ext.second.handle;
00670                 break;
00671             }
00672         }
00673         if (uses_std_module) {
00674             args_raii.init(argv, handle);
00675         }
00676     }
00677     program->flatten(program.get());
00678     if (only_test == false)
00679         exitCode = program->exec();
00680
00681     if (mxvm::debug_mode) {
00682         if (debug_output.is_open()) {
00683             program->print(debug_output);
00684             program->post(debug_output);
00685             program->memoryDump(debug_output);
00686         }
00687     }
00688
00689     if (mxvm::html_mode) {
00690         std::cout << Col("MXVM: Generated ", mx::Color::BRIGHT_GREEN) << program->name <<
00691         ".html\n";
00692         std::ofstream fout(program->name + ".html");
00693         parser.generateDebugHTML(fout, program);
00694         fout.close();
00695     } else {
00696         std::cerr << Col("MXVM: Error: ", mx::Color::RED) << "Failed to generate intermediate
00697         code.\n";
00698         return EXIT_FAILURE;
00699     }
00700 } catch (const mx::Exception &e) {
00701     std::cerr << Col("MXVM: Exception: ", mx::Color::RED) << e.what() << "\n";
00702     if (mxvm::debug_mode) {
00703         if (debug_output.is_open()) {
00704             program->print(debug_output);
00705             program->memoryDump(debug_output);
00706         }
00707     }
00708     return EXIT_FAILURE;
00709 } catch (const std::runtime_error &e) {
00710     std::cerr << Col("MXVM: Runtime Error: ", mx::Color::RED) << e.what() << "\n";
00711     if (mxvm::debug_mode) {
00712         if (debug_output.is_open()) {
00713             program->print(debug_output);
00714             program->memoryDump(debug_output);
00715         }
00716     }
00717     return EXIT_FAILURE;
00718 } catch (const std::exception &e) {
00719     std::cerr << Col("MXVM: Exception: ", mx::Color::RED) << e.what() << "\n";
00720     if (mxvm::debug_mode) {
00721         if (debug_output.is_open()) {
00722             program->print(debug_output);
00723             program->memoryDump(debug_output);
00724         }
00725     }
00726     return EXIT_FAILURE;
00727 }

```

```

00723     }
00724     }
00725     return EXIT_FAILURE;
00726 } catch (...) {
00727     std::cerr << Col("MXVM: ", mx::Color::RED) << "Unknown Exception.\n";
00728     if (mxvm::debug_mode) {
00729         if (debug_output.is_open()) {
00730             program->print(debug_output);
00731             program->memoryDump(debug_output);
00732         }
00733     }
00734     return EXIT_FAILURE;
00735 }
00736
00737 if (mxvm::debug_mode) {
00738     if (debug_output.is_open()) {
00739         debug_output.close();
00740         std::cout << Col("MXVM: Generated: ", mx::Color::BRIGHT_GREEN) << "debug information:
debug_info.txt\n";
00741     }
00742 }
00743 if (mxvm::debug_mode) {
00744     std::cout << Col("MXVM: ", mx::Color::RED) << "Program exited with: " << exitCode << "\n";
00745 }
00746 return exitCode;
00747 }
00748
00749 void collectAndRegisterAllExterns(std::unique_ptr<mxvm::Program> &program) {
00750     for (const auto &obj : program->objects) {
00751         for (auto &lbl : obj->labels) {
00752             if (lbl.second.second)
00753                 mxvm::Program::base->add_extern(obj->name, lbl.first, false);
00754         }
00755         for (auto &v : obj->vars) {
00756             mxvm::Program::base->add_extern(obj->name, v.first, false);
00757         }
00758     }
00759 }
00760
00761 void createMakefile(Args *args) {
00762     if (args == nullptr) {
00763         std::cerr << "Error arguments null.\n";
00764         return;
00765     }
00766
00767     std::ofstream file("Makefile");
00768     if (!file.is_open()) {
00769         std::cerr << Col("MXVM Error: ", mx::Color::BRIGHT_RED) << " could not open file\n";
00770         exit(EXIT_FAILURE);
00771     }
00772     char *ldflags = getenv("LDFLAGS");
00773     std::string ld_flags;
00774     if (ldflags != nullptr)
00775         ld_flags = ldflags;
00776
00777     // Derive program name from source file (strip path and .mxvm extension)
00778     std::string prog_name = args->source_file;
00779     {
00780         size_t slash = prog_name.rfind('/');
00781         if (slash != std::string::npos)
00782             prog_name = prog_name.substr(slash + 1);
00783         size_t dot = prog_name.rfind(".mxvm");
00784         if (dot != std::string::npos)
00785             prog_name = prog_name.substr(0, dot);
00786     }
00787
00788     // Build the mxvmc compile command with all relevant flags
00789     std::ostringstream compile_cmd;
00790     compile_cmd << "mxvmc \" " << args->source_file << "\" "
00791         << " --path \" " << args->module_path << "\" "
00792         << " --object-path \" " << args->object_path << "\" "
00793         << " --action compile"
00794         << " --target \" " << args->target;
00795     if (!args->toolchain.empty())
00796         compile_cmd << " --toolchain \" " << args->toolchain;
00797     if (!args->include_path.empty())
00798         compile_cmd << " --include \" " << args->include_path << "\"";
00799
00800     file << "### Makefile generated by MXVM\n";
00801     file << "### Source: \" " << args->source_file << "\"\n";
00802     file << "### Target: \" " << args->target << "\"\n";
00803     if (!args->toolchain.empty())
00804         file << "### Toolchain: \" " << args->toolchain << "\"\n";
00805     file << "\n";
00806     file << "PROGRAM = \" " << prog_name << "\"\n";
00807     file << "LDFLAGS = \" " << ld_flags << "\"\n";

```

```

00809     file << "\n";
00810     file << ".PHONY: all run clean\n\n";
00811     file << "all:\n";
00812     file << "\tLDFLAGS=\${LDFLAGS}\n" << compile_cmd.str() << "\n\n";
00813     file << "run:\n";
00814     file << "\txvmc \"" << args->source_file << "\"\n"
00815         << "  --path \"" << args->module_path << "\"\n"
00816         << "  --object-path \"" << args->object_path << "\"\n"
00817         << "  --target \"" << args->target;
00818     if (!args->toolchain.empty())
00819         file << "  --toolchain \"" << args->toolchain;
00820     file << "\n\n";
00821     file << "clean:\n";
00822     file << "\trm -f $(PROGRAM) $(PROGRAM).exe *.o *.s\n";
00823     file.close();
00824     std::cout << Col("MXVM: ", mx::Color::BRIGHT_GREEN) << "Created Makefile for " << prog_name << "\n";
00825 }

```

12.78 src/frontend/parser.cpp File Reference

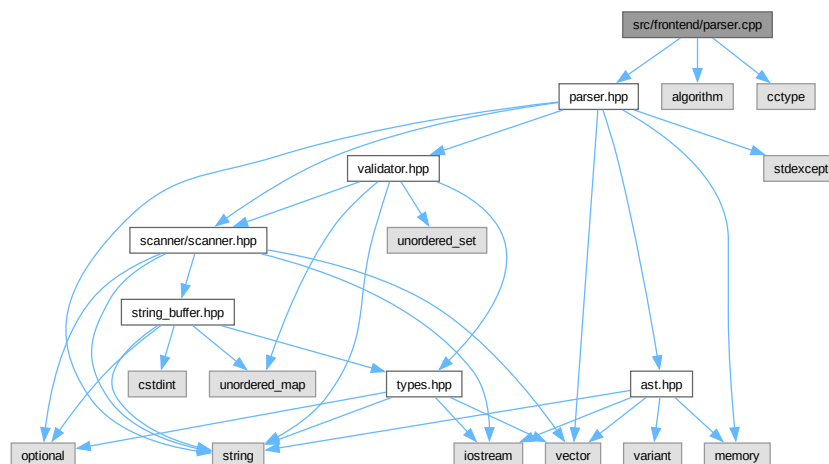
Pascal recursive-descent parser implementation.

```
#include "parser.hpp"
```

```
#include <algorithm>
```

```
#include <cctype>
```

Include dependency graph for parser.cpp:



Namespaces

- namespace [pascal](#)

12.78.1 Detailed Description

Pascal recursive-descent parser implementation.

Author

Jared Bruni

Definition in file [parser.cpp](#).

12.79 parser.cpp

[Go to the documentation of this file.](#)

00001

```

00006 #include "parser.hpp"
00007 #include <algorithm>
00008 #include <cctype>
00009
00010 namespace pascal {
00011
00012     void PascalParser::removeBraceComments() {
00013         auto &toks = scanner.getTokens();
00014         for (size_t i = 0; i < toks.size(); i++) {
00015             if (toks[i].getTokenValue() == "{") {
00016                 size_t start = i;
00017                 ++i;
00018                 while (i < toks.size() && toks[i].getTokenValue() != ";") {
00019                     ++i;
00020                 }
00021                 if (i < toks.size()) {
00022                     ++i; // skip closing }
00023                 }
00024                 toks.erase(toks.begin() + static_cast<int64_t>(start),
00025                             toks.begin() + static_cast<int64_t>(i));
00026                 i = start;
00027             } else {
00028                 ++i;
00029             }
00030         }
00031     }
00032
00033     void PascalParser::error(const std::string &message) {
00034         throw ParseException("Parse error: " + message + (token ? " at '" + token->getTokenValue() +
00035         "' : " + " at end of input"));
00036     }
00037
00038     void PascalParser::expectToken(const std::string &expected) {
00039         if (!peekIs(expected))
00040             error("Expected '" + expected + "' + " on Line: " + std::to_string(token->getLine()));
00041     }
00042
00043     void PascalParser::expectToken(types::TokenType expected) {
00044         if (!peekIs(expected))
00045             error("Expected " + tokenTypeToString(expected));
00046     }
00047
00048     std::unique_ptr<ProgramNode> PascalParser::parseProgram() {
00049         expectToken("program");
00050         int lineNum = token->getLine();
00051         next();
00052         expectToken(types::TokenType::TT_ID);
00053         std::string programName = token->getTokenValue();
00054         next();
00055         expectToken(";");
00056         next();
00057         std::vector<std::string> usesList;
00058         if (peekIs("uses")) {
00059             next();
00060             expectToken(types::TokenType::TT_ID);
00061             usesList.push_back(token->getTokenValue());
00062             next();
00063             while (peekIs(",")) {
00064                 next();
00065                 expectToken(types::TokenType::TT_ID);
00066                 usesList.push_back(token->getTokenValue());
00067                 next();
00068             }
00069             expectToken(";");
00070             next();
00071         }
00072         auto block = parseBlock();
00073         expectToken(".");
00074         auto programNode = std::make_unique<ProgramNode>(programName, std::move(block));
00075         programNode->uses = std::move(usesList);
00076         programNode->setLineNumber(lineNum);
00077         return programNode;
00078     }
00079
00080     bool PascalParser::match(const std::string &s) {
00081         if (peekIs(s)) {
00082             next();
00083             return true;
00084         }
00085         return false;
00086     }
00087
00088     bool PascalParser::isUnitSource() const {
00089         if (!token)
00090             return false;
00091         std::string val = token->getTokenValue();
00092         std::transform(val.begin(), val.end(), val.begin(), ::tolower);

```

```

00092         return val == "unit";
00093     }
00094
00095     std::unique_ptr<UnitNode> PascalParser::parseUnit() {
00096         expectToken("unit");
00097         int lineNum = token->getLine();
00098         next();
00099         expectToken(types::TokenType::TT_ID);
00100         std::string unitName = token->getTokenValue();
00101         next();
00102         expectToken(";");
00103         next();
00104
00105         auto unitNode = std::make_unique<UnitNode>(unitName);
00106         unitNode->setLineNumber(lineNum);
00107
00108         // interface section
00109         expectToken("interface");
00110         next();
00111
00112         // optional uses clause in interface
00113         std::vector<std::string> usesList;
00114         if (peekIs("uses")) {
00115             next();
00116             expectToken(types::TokenType::TT_ID);
00117             usesList.push_back(token->getTokenValue());
00118             next();
00119             while (peekIs(",")) {
00120                 next();
00121                 expectToken(types::TokenType::TT_ID);
00122                 usesList.push_back(token->getTokenValue());
00123                 next();
00124             }
00125             expectToken(";");
00126             next();
00127         }
00128         unitNode->uses = std::move(usesList);
00129
00130         // parse interface declarations (forward proc/func signatures)
00131         unitNode->interfaceDecls = parseInterfaceDeclarations();
00132
00133         // implementation section
00134         expectToken("implementation");
00135         next();
00136
00137         // optional uses clause in implementation (additional modules)
00138         if (peekIs("uses")) {
00139             next();
00140             expectToken(types::TokenType::TT_ID);
00141             unitNode->uses.push_back(token->getTokenValue());
00142             next();
00143             while (peekIs(",")) {
00144                 next();
00145                 expectToken(types::TokenType::TT_ID);
00146                 unitNode->uses.push_back(token->getTokenValue());
00147                 next();
00148             }
00149             expectToken(";");
00150             next();
00151         }
00152
00153         // parse implementation declarations (full proc/func with bodies, vars, consts, types)
00154         unitNode->implDecls = parseDeclarations();
00155
00156         // end.
00157         expectToken("end");
00158         next();
00159         expectToken(".");
00160         return unitNode;
00161     }
00162
00163     std::vector<std::unique_ptr<ASTNode>> PascalParser::parseInterfaceDeclarations() {
00164         std::vector<std::unique_ptr<ASTNode>> decls;
00165         while (peekIs("procedure") || peekIs("function") || peekIs("type") || peekIs("const") ||
00166             peekIs("var")) {
00167             if (peekIs("procedure")) {
00168                 decls.push_back(parseProcedureForwardDecl());
00169             } else if (peekIs("function")) {
00170                 decls.push_back(parseFunctionForwardDecl());
00171             } else if (peekIs("type")) {
00172                 decls.push_back(parseTypeDeclaration());
00173             } else if (peekIs("const")) {
00174                 decls.push_back(parseConstDeclaration());
00175             } else if (peekIs("var")) {
00176                 if (match("var")) {
00177                     while (peekIs(types::TokenType::TT_ID) && !isKeyword(token->getTokenValue())) {
00178                         auto decl = parseVarDeclaration();

```

```

00178         decls.push_back(std::move(decl));
00179     }
00180     }
00181 }
00182 }
00183 return decls;
00184 }
00185
00186 std::unique_ptr<ASTNode> PascalParser::parseProcedureForwardDecl() {
00187     expectToken("procedure");
00188     int lineNum = token->getLine();
00189     next();
00190     expectToken(types::TokenType::TT_ID);
00191     std::string procName = token->getTokenValue();
00192     next();
00193     std::vector<std::unique_ptr<ASTNode>> parameters;
00194     if (peekIs("(")) {
00195         next();
00196         parameters = parseParameterList();
00197         expectToken(")");
00198         next();
00199     }
00200     expectToken(";");
00201     next();
00202     auto procDeclNode = std::make_unique<ProcDeclNode>(procName, std::move(parameters), nullptr);
00203     procDeclNode->setLineNumber(lineNum);
00204     return procDeclNode;
00205 }
00206
00207 std::unique_ptr<ASTNode> PascalParser::parseFunctionForwardDecl() {
00208     expectToken("function");
00209     int lineNum = token->getLine();
00210     next();
00211     expectToken(types::TokenType::TT_ID);
00212     std::string funcName = token->getTokenValue();
00213     next();
00214     std::vector<std::unique_ptr<ASTNode>> parameters;
00215     if (peekIs("(")) {
00216         next();
00217         parameters = parseParameterList();
00218         expectToken(")");
00219         next();
00220     }
00221     expectToken(":");
00222     next();
00223     std::string returnType;
00224     if (peekIs("^")) {
00225         next();
00226         expectToken(types::TokenType::TT_ID);
00227         returnType = "^" + token->getTokenValue();
00228         next();
00229     } else {
00230         expectToken(types::TokenType::TT_ID);
00231         returnType = token->getTokenValue();
00232         next();
00233     }
00234     expectToken(";");
00235     next();
00236     auto funcDeclNode = std::make_unique<FuncDeclNode>(funcName, std::move(parameters),
00237     returnType, nullptr);
00238     funcDeclNode->setLineNumber(lineNum);
00239     return funcDeclNode;
00240 }
00241
00242 bool PascalParser::match(types::TokenType t) {
00243     if (peekIs(t)) {
00244         next();
00245         return true;
00246     }
00247     return false;
00248 }
00249
00250 std::unique_ptr<BlockNode> PascalParser::parseBlock() {
00251     int lineNum = token ? token->getLine() : 1;
00252     if (peekIs("label")) {
00253         parseLabelDeclaration();
00254     }
00255     auto declarations = parseDeclarations();
00256     auto compoundStatement = parseCompoundStatement();
00257     auto blockNode = std::make_unique<BlockNode>(std::move(declarations),
00258     std::move(compoundStatement));
00259     blockNode->setLineNumber(lineNum);
00260     return blockNode;
00261 }
00262
00263 bool PascalParser::isKeyword(const std::string &tok) {
00264     std::string lower = tok;

```

```

00263         std::transform(lower.begin(), lower.end(), lower.begin(), ::tolower);
00264     return lower == "program" || lower == "var" || lower == "begin" || lower == "end" ||
00265         lower == "if" || lower == "then" || lower == "else" || lower == "while" ||
00266         lower == "do" || lower == "for" || lower == "to" || lower == "downto" ||
00267         lower == "procedure" || lower == "function" || lower == "integer" ||
00268         lower == "real" || lower == "boolean" || lower == "string" || lower == "char" ||
00269         lower == "true" || lower == "false" || lower == "div" || lower == "mod" ||
00270         lower == "and" || lower == "or" || lower == "not" || lower == "in" ||
00271         lower == "case" || lower == "of" || lower == "repeat" || lower == "until" ||
00272         lower == "array" || lower == "type" || lower == "record" || lower == "exit" || lower ==
00273         "break" || lower == "continue" ||
00274         lower == "nil" || lower == "new" || lower == "dispose" || lower == "pointer" || lower
00275         == "uses" ||
00276         lower == "unit" || lower == "interface" || lower == "implementation" ||
00277         lower == "with" || lower == "goto" || lower == "label" ||
00278         lower == "packed" || lower == "set" || lower == "file";
00279     }
00280     std::unique_ptr<ASTNode> PascalParser::parseProcedureDeclaration() {
00281         expectToken("procedure");
00282         int lineNum = token->getLine();
00283         next();
00284         expectToken(types::TokenType::TT_ID);
00285         std::string procName = token->getTokenValue();
00286         next();
00287         std::vector<std::unique_ptr<ASTNode>> parameters;
00288         if (peekIs("(")) {
00289             next();
00290             parameters = parseParameterList();
00291             expectToken(")");
00292             next();
00293         }
00294         expectToken(";");
00295         next();
00296         auto block = parseBlock();
00297         expectToken(";");
00298         next();
00299         auto procDeclNode = std::make_unique<ProcDeclNode>(procName, std::move(parameters),
00300             std::move(block));
00301         procDeclNode->setLineNumber(lineNum);
00302         return procDeclNode;
00303     }
00304     std::unique_ptr<ASTNode> PascalParser::parseFunctionDeclaration() {
00305         expectToken("function");
00306         int lineNum = token->getLine();
00307         next();
00308         expectToken(types::TokenType::TT_ID);
00309         std::string funcName = token->getTokenValue();
00310         next();
00311         std::vector<std::unique_ptr<ASTNode>> parameters;
00312         if (peekIs("(")) {
00313             next();
00314             parameters = parseParameterList();
00315             expectToken(")");
00316             next();
00317         }
00318         expectToken(":");
00319         next();
00320         std::string returnType;
00321         if (peekIs("^")) {
00322             next();
00323             expectToken(types::TokenType::TT_ID);
00324             returnType = "^" + token->getTokenValue();
00325             next();
00326         } else {
00327             expectToken(types::TokenType::TT_ID);
00328             returnType = token->getTokenValue();
00329             next();
00330         }
00331         expectToken(";");
00332         next();
00333         auto block = parseBlock();
00334         expectToken(";");
00335         next();
00336         auto funcDeclNode = std::make_unique<FuncDeclNode>(
00337             funcName, std::move(parameters), returnType, std::move(block));
00338         funcDeclNode->setLineNumber(lineNum);
00339         return funcDeclNode;
00340     }
00341 }

```

```

00347
00348     std::vector<std::unique_ptr<ASTNode> PascalParser::parseParameterList() {
00349         std::vector<std::unique_ptr<ASTNode> parameters;
00350         if (!peekIs("(")) {
00351             parameters.push_back(parseParameter());
00352             while (peekIs(";")) {
00353                 next();
00354                 parameters.push_back(parseParameter());
00355             }
00356         }
00357         return parameters;
00358     }
00359
00360     std::unique_ptr<ASTNode> PascalParser::parseParameter() {
00361         int lineNum = token ? token->getLine() : 1;
00362         bool isVar = false;
00363         if (peekIs("var")) {
00364             isVar = true;
00365             next();
00366         }
00367
00368         std::vector<std::string> identifiers;
00369         expectToken(types::TokenType::TT_ID);
00370         identifiers.push_back(token->getTokenValue());
00371         next();
00372         while (peekIs(",")) {
00373             next();
00374             expectToken(types::TokenType::TT_ID);
00375             identifiers.push_back(token->getTokenValue());
00376             next();
00377         }
00378
00379         expectToken(":");
00380         next();
00381
00382         std::string typeName;
00383         if (peekIs("^")) {
00384             next();
00385             expectToken(types::TokenType::TT_ID);
00386             typeName = "^" + token->getTokenValue();
00387             next();
00388         } else {
00389             expectToken(types::TokenType::TT_ID);
00390             typeName = token->getTokenValue();
00391             next();
00392         }
00393
00394         auto parameterNode = std::make_unique<ParameterNode>(
00395             std::move(identifiers), typeName, isVar);
00396         parameterNode->setLineNumber(lineNum);
00397         return parameterNode;
00398     }
00399
00400     std::unique_ptr<CompoundStmtNode> PascalParser::parseCompoundStatement() {
00401         int lineNum = token ? token->getLine() : 1;
00402         expectToken("begin");
00403         next();
00404         auto statements = parseStatementList();
00405         expectToken("end");
00406         next();
00407         auto compoundNode = std::make_unique<CompoundStmtNode>(std::move(statements));
00408         compoundNode->setLineNumber(lineNum);
00409         return compoundNode;
00410     }
00411
00412     std::unique_ptr<ASTNode> PascalParser::parseStatement() {
00413         if (!token) {
00414             auto emptyNode = std::make_unique<EmptyStmtNode>();
00415             emptyNode->setLineNumber(1);
00416             return emptyNode;
00417         }
00418         if (peekIs("exit")) {
00419             int lineNum = token->getLine();
00420             next();
00421             std::unique_ptr<ASTNode> expr;
00422             if (peekIs("(")) {
00423                 next();
00424                 expr = parseExpression();
00425                 expectToken(")");
00426                 next();
00427             } else if (!peekIs(";")) {
00428                 expr = parseExpression();
00429             }
00430             expectToken(";");
00431             auto exitNode = std::make_unique<ExitNode>(std::move(expr));
00432             exitNode->setLineNumber(lineNum);
00433             return exitNode;

```



```

00434     } else if (peekIs("break")) {
00435         int lineNum = token->getLine();
00436         next();
00437         expectToken(";");
00438         auto breakNode = std::make_unique<BreakNode>();
00439         breakNode->setLineNumber(lineNum);
00440         return breakNode;
00441     } else if (peekIs("continue")) {
00442         int lineNum = token->getLine();
00443         next();
00444         expectToken(";");
00445         auto continueNode = std::make_unique<ContinueNode>();
00446         continueNode->setLineNumber(lineNum);
00447         return continueNode;
00448     } else if (peekIs("begin")) {
00449         return parseCompoundStatement();
00450     } else if (peekIs("if")) {
00451         return parseIfStatement();
00452     } else if (peekIs("while")) {
00453         return parseWhileStatement();
00454     } else if (peekIs("for")) {
00455         return parseForStatement();
00456     } else if (peekIs("repeat")) {
00457         return parseRepeatStatement();
00458     } else if (peekIs("case")) {
00459         return parseCaseStatement();
00460     } else if (peekIs("with")) {
00461         return parseWithStatement();
00462     } else if (peekIs("goto")) {
00463         return parseGotoStatement();
00464     } else if (peekIs(types::TokenType::TT_NUM)) {
00465         // Label definition: 100: statement
00466         std::string lbl = token->getTokenValue();
00467         int lineNum = token->getLine();
00468         next();
00469         if (peekIs(":")) {
00470             next();
00471             auto stmt = parseStatement();
00472             auto labelNode = std::make_unique<LabelStmtNode>(lbl, std::move(stmt));
00473             labelNode->setLineNumber(lineNum);
00474             return labelNode;
00475         }
00476         error("Expected ':' after label number");
00477         return nullptr;
00478     } else if (peekIs(types::TokenType::TT_ID)) {
00479         auto lhs = parseLValue();
00480         int lineNum = token ? token->getLine() : 1;
00481         if (peekIs(":=")) {
00482             next();
00483             auto rhs = parseExpression();
00484             auto assignmentNode = std::make_unique<AssignmentNode>(std::move(lhs),
std::move(rhs));
00485             assignmentNode->setLineNumber(lineNum);
00486             return assignmentNode;
00487         }
00488         if (auto varNode = dynamic_cast<VariableNode*>(lhs.get())) {
00489             if (peekIs("(")) {
00490                 return parseProcedureCall(varNode->name);
00491             }
00492             // Bare identifier as statement = parameterless procedure call
00493             auto procCallNode = std::make_unique<ProcCallNode>(varNode->name,
std::vector<std::unique_ptr<ASTNode>>{});
00494             procCallNode->setLineNumber(lineNum);
00495             return procCallNode;
00496         }
00497         if (auto fieldNode = dynamic_cast<FieldAccessNode*>(lhs.get())) {
00498             if (dynamic_cast<VariableNode*>(fieldNode->recordExpr.get())) {
00499                 if (peekIs("(")) {
00500                     return parseProcedureCall(fieldNode->fieldName);
00501                 }
00502                 // Bare qualified name as statement = parameterless procedure call
00503                 auto procCallNode = std::make_unique<ProcCallNode>(fieldNode->fieldName,
std::vector<std::unique_ptr<ASTNode>>{});
00504                 procCallNode->setLineNumber(lineNum);
00505                 return procCallNode;
00506             }
00507         }
00508         error("Invalid statement: expected ':= ' for assignment or '(' for procedure call");
00509         return nullptr;
00510     } else {
00511         auto emptyNode = std::make_unique<EmptyStmtNode>();
00512         emptyNode->setLineNumber(token ? token->getLine() : 1);
00513         return emptyNode;
00514     }
00515 }
00516
00517 std::unique_ptr<ASTNode> PascalParser::parseAssignmentOrProcCall() {

```

```

00518     expectToken(types::TokenType::TT_ID);
00519     std::string name = token->getTokenValue();
00520     int lineNum = token->getLine();
00521     next();
00522     if (peekIs(":=")) {
00523         next();
00524         auto variable = std::make_unique<VariableNode>(name);
00525         variable->setLineNumber(lineNum);
00526         auto expression = parseExpression();
00527         auto assignmentNode = std::make_unique<AssignmentNode>(std::move(variable),
std::move(expression));
00528         assignmentNode->setLineNumber(lineNum);
00529         return assignmentNode;
00530     } else {
00531         return parseProcedureCall(name);
00532     }
00533 }
00534
00535 std::unique_ptr<ASTNode> PascalParser::parseIfStatement() {
00536     expectToken("if");
00537     int lineNum = token->getLine();
00538     next();
00539     auto condition = parseExpression();
00540     expectToken("then");
00541     next();
00542     auto thenStatement = parseStatement();
00543     std::unique_ptr<ASTNode> elseStatement = nullptr;
00544     if (peekIs("else")) {
00545         next();
00546         elseStatement = parseStatement();
00547     }
00548     auto ifStmtNode = std::make_unique<IfStmtNode>(std::move(condition), std::move(thenStatement),
std::move(elseStatement));
00549     ifStmtNode->setLineNumber(lineNum);
00550     return ifStmtNode;
00551 }
00552
00553 std::unique_ptr<ASTNode> PascalParser::parseWhileStatement() {
00554     expectToken("while");
00555     int lineNum = token->getLine();
00556     next();
00557     auto condition = parseExpression();
00558     expectToken("do");
00559     next();
00560     auto statement = parseStatement();
00561     auto whileStmtNode = std::make_unique<WhileStmtNode>(std::move(condition),
std::move(statement));
00562     whileStmtNode->setLineNumber(lineNum);
00563     return whileStmtNode;
00564 }
00565
00566 std::unique_ptr<ASTNode> PascalParser::parseForStatement() {
00567     expectToken("for");
00568     int lineNum = token->getLine();
00569     next();
00570     expectToken(types::TokenType::TT_ID);
00571     std::string variable = token->getTokenValue();
00572     next();
00573     expectToken(":=");
00574     next();
00575     auto startValue = parseExpression();
00576     bool isDownto = false;
00577     if (peekIs("to")) {
00578         next();
00579     } else if (peekIs("downto")) {
00580         isDownto = true;
00581         next();
00582     } else {
00583         error("Expected 'to' or 'downto' in for loop");
00584     }
00585     auto endValue = parseExpression();
00586     expectToken("do");
00587     next();
00588     auto statement = parseStatement();
00589     auto forStmtNode = std::make_unique<ForStmtNode>(variable, std::move(startValue),
std::move(endValue), isDownto, std::move(statement));
00590     forStmtNode->setLineNumber(lineNum);
00591     return forStmtNode;
00592 }
00593
00594 std::unique_ptr<ASTNode> PascalParser::parseRepeatStatement() {
00595     expectToken("repeat");
00596     int lineNum = token->getLine();
00597     next();
00598     std::vector<std::unique_ptr<ASTNode>> statements;
00599     while (!peekIs("until")) {
00600         statements.push_back(parseStatement());

```

```

00601         if (peekIs(";")) {
00602             next();
00603         } else if (!peekIs("until")) {
00604             error("Expected ';' or 'until'");
00605         }
00606     }
00607     expectToken("until");
00608     next();
00609     auto condition = parseExpression();
00610     auto repeatStmtNode = std::make_unique<RepeatStmtNode>(std::move(statements),
std::move(condition));
00611     repeatStmtNode->setLineNumber(lineNum);
00612     return repeatStmtNode;
00613 }
00614
00615 std::unique_ptr<ASTNode> PascalParser::parseExpression() {
00616     int lineNum = token ? token->getLine() : 1;
00617     auto left = parseSimpleExpression();
00618     if (isRelationalOperator()) {
00619         if (peekIs("in")) {
00620             next();
00621             // Allow both set literal [1,2,3] and set variable
00622             auto right = parseSimpleExpression();
00623             left = std::make_unique<BinaryOpNode>(std::move(left), BinaryOpNode::IN,
std::move(right));
00624             left->setLineNumber(lineNum);
00625         } else {
00626             std::string op = getRelationalOp();
00627             next();
00628             auto right = parseSimpleExpression();
00629             BinaryOpNode::OpType enumOp = BinaryOpNode::EQUAL;
00630             if (op == "=")
00631                 enumOp = BinaryOpNode::EQUAL;
00632             else if (op == "<>")
00633                 enumOp = BinaryOpNode::NOT_EQUAL;
00634             else if (op == "<")
00635                 enumOp = BinaryOpNode::LESS;
00636             else if (op == "<=")
00637                 enumOp = BinaryOpNode::LESS_EQUAL;
00638             else if (op == ">")
00639                 enumOp = BinaryOpNode::GREATER;
00640             else if (op == ">=")
00641                 enumOp = BinaryOpNode::GREATER_EQUAL;
00642             left = std::make_unique<BinaryOpNode>(std::move(left), enumOp, std::move(right));
00643             left->setLineNumber(lineNum);
00644         }
00645     }
00646     while (peekIs("and") || peekIs("or")) {
00647         BinaryOpNode::OpType op;
00648         if (peekIs("and")) {
00649             op = BinaryOpNode::AND;
00650             next();
00651         } else {
00652             op = BinaryOpNode::OR;
00653             next();
00654         }
00655         auto right = parseSimpleExpression();
00656         left = std::make_unique<BinaryOpNode>(std::move(left), op, std::move(right));
00657         left->setLineNumber(lineNum);
00658     }
00659     return left;
00660 }
00661
00662 std::unique_ptr<ASTNode> PascalParser::parseSimpleExpression() {
00663     std::unique_ptr<ASTNode> result;
00664     int lineNum = token ? token->getLine() : 1;
00665     if ((peekIs("+") || peekIs("-")) && peekIs(types::TokenType::TT_SYM)) || peekIs("not")) {
00666         UnaryOpNode::Operator op = UnaryOpNode::PLUS;
00667         if (peekIs("+"))
00668             op = UnaryOpNode::PLUS;
00669         else if (peekIs("-"))
00670             op = UnaryOpNode::MINUS;
00671         else if (peekIs("not"))
00672             op = UnaryOpNode::NOT;
00673         next();
00674         auto operand = parseTerm();
00675         result = std::make_unique<UnaryOpNode>(op, std::move(operand));
00676         result->setLineNumber(lineNum);
00677     } else {
00678         result = parseTerm();
00679     }
00680     while ((peekIs("+") || peekIs("-")) && peekIs(types::TokenType::TT_SYM)) {
00681         BinaryOpNode::OpType op;
00682         if (peekIs("+"))
00683             op = BinaryOpNode::PLUS;
00684         else
00685             op = BinaryOpNode::MINUS;

```

```

00686         next();
00687         auto right = parseTerm();
00688         result = std::make_unique<BinaryOpNode>(std::move(result), op, std::move(right));
00689         result->setLineNumber(lineNum);
00690     }
00691     return result;
00692 }
00693
00694 std::unique_ptr<ASTNode> PascalParser::parseTerm() {
00695     int lineNum = token ? token->getLine() : 1;
00696     auto result = parseFactor();
00697     while (isMulOperator()) {
00698         BinaryOpNode::OpType op = BinaryOpNode::MULTIPLY;
00699         if (peekIs("*"))
00700             op = BinaryOpNode::MULTIPLY;
00701         else if (peekIs("/"))
00702             op = BinaryOpNode::DIVIDE;
00703         else if (peekIs("div"))
00704             op = BinaryOpNode::DIV;
00705         else if (peekIs("mod"))
00706             op = BinaryOpNode::MOD;
00707         next();
00708         auto right = parseFactor();
00709         result = std::make_unique<BinaryOpNode>(std::move(result), op, std::move(right));
00710         result->setLineNumber(lineNum);
00711     }
00712     return result;
00713 }
00714
00715 std::unique_ptr<ASTNode> PascalParser::parseFactor() {
00716     int lineNum = token ? token->getLine() : 1;
00717     if (peekIs(types::TokenType::TT_NUM)) {
00718         std::string value = token->getTokenValue();
00719         next();
00720         bool isReal = value.find('.') != std::string::npos || value.find('e') != std::string::npos
00721 || value.find('E') != std::string::npos;
00722         auto numberNode = std::make_unique<NumberNode>(value, !isReal, isReal);
00723         numberNode->setLineNumber(lineNum);
00724         return numberNode;
00725     } else if (peekIs(types::TokenType::TT_STR)) {
00726         std::string value = token->getTokenValue();
00727         next();
00728         auto stringNode = std::make_unique<StringNode>(value);
00729         stringNode->setLineNumber(lineNum);
00730         return stringNode;
00731     } else if (peekIs("true") || peekIs("false")) {
00732         bool value = (token->getTokenValue() == "true");
00733         next();
00734         auto booleanNode = std::make_unique<BooleanNode>(value);
00735         booleanNode->setLineNumber(lineNum);
00736         return booleanNode;
00737     } else if (peekIs("nil")) {
00738         next();
00739         auto nilNode = std::make_unique<NilNode>();
00740         nilNode->setLineNumber(lineNum);
00741         return nilNode;
00742     } else if (peekIs("@")) {
00743         next();
00744         auto operand = parseFactor();
00745         auto addrNode = std::make_unique<AddressOfNode>(std::move(operand));
00746         addrNode->setLineNumber(lineNum);
00747         return addrNode;
00748     } else if (peekIs "[")) {
00749         // Set constructor: [elem1, elem2, ...]
00750         next(); // consume '['
00751         std::vector<std::unique_ptr<ASTNode>> elements;
00752         if (!peekIs("]")) {
00753             elements.push_back(parseExpression());
00754             while (peekIs(",")) {
00755                 next();
00756                 elements.push_back(parseExpression());
00757             }
00758         }
00759         expectToken("]");
00760         next();
00761         auto setNode = std::make_unique<SetLiteralNode>(std::move(elements));
00762         setNode->setLineNumber(lineNum);
00763         return setNode;
00764     } else if (peekIs("(")) {
00765         next();
00766         auto expr = parseExpression();
00767         expectToken(")");
00768         next();
00769         return expr;
00770     } else if (peekIs(types::TokenType::TT_ID)) {
00771         std::string name = token->getTokenValue();
00772         next();

```

```

00772         std::unique_ptr<ASTNode> left = std::make_unique<VariableNode>(name);
00773         left->setLineNumber(lineNum);
00774         while (true) {
00775             if (peekIs("(")) {
00776                 next();
00777                 auto index = parseExpression();
00778                 expectToken("]");
00779                 next();
00780                 left = std::make_unique<ArrayAccessNode>(std::move(left), std::move(index));
00781                 left->setLineNumber(lineNum);
00782             } else if (peekIs(".")) {
00783                 next();
00784                 expectToken(types::TokenType::TT_ID);
00785                 std::string fieldName = token->getTokenValue();
00786                 next();
00787                 left = std::make_unique<FieldAccessNode>(std::move(left), fieldName);
00788                 left->setLineNumber(lineNum);
00789             } else if (peekIs("^")) {
00790                 next();
00791                 left = std::make_unique<PointerDerefNode>(std::move(left));
00792                 left->setLineNumber(lineNum);
00793             } else if (peekIs("(")) {
00794                 if (auto varNode = dynamic_cast<VariableNode*>(left.get())) {
00795                     return parseFunctionCall(varNode->name);
00796                 } else if (auto fieldNode = dynamic_cast<FieldAccessNode*>(left.get())) {
00797                     if (dynamic_cast<VariableNode*>(fieldNode->recordExpr.get())) {
00798                         return parseFunctionCall(fieldNode->fieldName);
00799                     }
00800                     error("Function call must be on a simple or qualified identifier");
00801                 } else {
00802                     error("Function call must be on a simple identifier");
00803                 }
00804             } else {
00805                 break;
00806             }
00807         }
00808         return left;
00809     } else {
00810         error("Expected factor");
00811         return nullptr;
00812     }
00813 }
00814
00815 std::unique_ptr<ASTNode> PascalParser::parseProcedureCall(const std::string &name) {
00816     int lineNum = token ? token->getLine() : 1;
00817     std::vector<std::unique_ptr<ASTNode>> arguments;
00818     if (peekIs("(")) {
00819         next();
00820         arguments = parseArgumentList();
00821         expectToken(")");
00822         next();
00823     }
00824     auto procCallNode = std::make_unique<ProcCallNode>(name, std::move(arguments));
00825     procCallNode->setLineNumber(lineNum);
00826     return procCallNode;
00827 }
00828
00829 std::unique_ptr<ASTNode> PascalParser::parseFunctionCall(const std::string &name) {
00830     int lineNum = token ? token->getLine() : 1;
00831     std::vector<std::unique_ptr<ASTNode>> arguments;
00832     expectToken("(");
00833     next();
00834     arguments = parseArgumentList();
00835     expectToken(")");
00836     next();
00837     auto funcCallNode = std::make_unique<FuncCallNode>(name, std::move(arguments));
00838     funcCallNode->setLineNumber(lineNum);
00839     return funcCallNode;
00840 }
00841
00842 std::unique_ptr<ASTNode> PascalParser::parseCaseStatement() {
00843     expectToken("case");
00844     int lineNum = token->getLine();
00845     next();
00846     auto expression = parseExpression();
00847     expectToken("of");
00848     next();
00849     std::vector<std::unique_ptr<CaseStmtNode::CaseBranch>> branches;
00850     while (!peekIs("end") && !peekIs("else")) {
00851         std::vector<std::unique_ptr<ASTNode>> values;
00852         values.push_back(parseExpression());
00853         while (peekIs(",")) {
00854             next();
00855             values.push_back(parseExpression());
00856         }
00857         expectToken(":");
00858         next();

```

```

00859         auto statement = parseStatement();
00860         branches.push_back(std::make_unique<CaseStmtNode::CaseBranch>(std::move(values),
std::move(statement)));
00861         if (peekIs(";"))
00862             next();
00863     }
00864     std::unique_ptr<ASTNode> elseStatement = nullptr;
00865     if (peekIs("else")) {
00866         next();
00867         elseStatement = parseStatement();
00868         if (peekIs(";"))
00869             next();
00870     }
00871     expectToken("end");
00872     next();
00873     auto caseStmtNode = std::make_unique<CaseStmtNode>(std::move(expression), std::move(branches),
std::move(elseStatement));
00874     caseStmtNode->setLineNumber(lineNum);
00875     return caseStmtNode;
00876 }
00877
00878 std::unique_ptr<ASTNode> PascalParser::parseConstDeclaration() {
00879     expectToken("const");
00880     int lineNum = token->getLine();
00881     next();
00882     std::vector<std::unique_ptr<ConstDeclNode::ConstAssignment>> assignments;
00883     do {
00884         expectToken(types::TokenType::TT_ID);
00885         std::string identifier = token->getTokenValue();
00886         next();
00887         expectToken("=");
00888         next();
00889         auto value = parseExpression();
00890         assignments.push_back(std::make_unique<ConstDeclNode::ConstAssignment>(identifier,
std::move(value)));
00891         expectToken(";");
00892         next();
00893     } while (peekIs(types::TokenType::TT_ID) && !isKeyword(token->getTokenValue()));
00894     auto constDeclNode = std::make_unique<ConstDeclNode>(std::move(assignments));
00895     constDeclNode->setLineNumber(lineNum);
00896     return constDeclNode;
00897 }
00898
00899 std::vector<std::unique_ptr<ASTNode>> PascalParser::parseDeclarations() {
00900     std::vector<std::unique_ptr<ASTNode>> declarations;
00901     while (peekIs("type") || peekIs("const") || peekIs("var") || peekIs("procedure") ||
peekIs("function") || peekIs("label")) {
00902         if (peekIs("label")) {
00903             parseLabelDeclaration();
00904         } else if (peekIs("type")) {
00905             declarations.push_back(parseTypeDeclaration());
00906         } else if (peekIs("const")) {
00907             declarations.push_back(parseConstDeclaration());
00908         } else if (peekIs("var")) {
00909             if (match("var")) {
00910                 while (peekIs(types::TokenType::TT_ID) && !isKeyword(token->getTokenValue())) {
00911                     auto decl = parseVarDeclaration();
00912                     declarations.push_back(std::move(decl));
00913                 }
00914             }
00915         } else if (peekIs("procedure")) {
00916             declarations.push_back(parseProcedureDeclaration());
00917         } else if (peekIs("function")) {
00918             declarations.push_back(parseFunctionDeclaration());
00919         }
00920     }
00921     return declarations;
00922 }
00923
00924 std::vector<std::unique_ptr<ASTNode>> PascalParser::parseStatementList() {
00925     std::vector<std::unique_ptr<ASTNode>> statements;
00926     while (!peekIs("end") && !peekIs("else") && !peekIs("until")) {
00927         statements.push_back(parseStatement());
00928         if (peekIs(";")) {
00929             next();
00930             if (peekIs("end") || peekIs("else") || peekIs("until"))
00931                 break;
00932         } else {
00933             break;
00934         }
00935     }
00936     return statements;
00937 }
00938
00939 std::vector<std::unique_ptr<ASTNode>> PascalParser::parseArgumentList() {
00940     std::vector<std::unique_ptr<ASTNode>> arguments;
00941     if (!peekIs(",")) {

```

```

00942         arguments.push_back(parseExpression());
00943         while (peekIs(",", ")) {
00944             next();
00945             arguments.push_back(parseExpression());
00946         }
00947     }
00948     return arguments;
00949 }
00950
00951 bool PascalParser::isMulOperator() {
00952     if (!peekIs(types::TokenType::TT_SYM) && !peekIs(types::TokenType::TT_ID))
00953         return false;
00954     return peekIs("*") || peekIs("/") || peekIs("div") || peekIs("mod");
00955 }
00956
00957 bool PascalParser::isRelationalOperator() {
00958     if (peekIs("in"))
00959         return true;
00960     if (!peekIs(types::TokenType::TT_SYM))
00961         return false;
00962     return peekIs("=") || peekIs("<>") || peekIs("<") || peekIs("<=") || peekIs(">") ||
peekIs(">=");
00963 }
00964
00965 std::string PascalParser::getRelationalOp() {
00966     if (peekIs("in"))
00967         return "in";
00968     if (peekIs("="))
00969         return "=";
00970     if (peekIs("<>"))
00971         return "<>";
00972     if (peekIs("<="))
00973         return "<=";
00974     if (peekIs(">="))
00975         return ">=";
00976     if (peekIs("<"))
00977         return "<";
00978     if (peekIs(">"))
00979         return ">";
00980     error("Invalid relational operator");
00981     return "";
00982 }
00983
00984 std::unique_ptr<ASTNode> PascalParser::parseTypeDeclaration() {
00985     expectToken("type");
00986     int lineNum = token->getLine();
00987     next();
00988
00989     std::vector<std::unique_ptr<ASTNode> typeDeclarations;
00990
00991     do {
00992         expectToken(types::TokenType::TT_ID);
00993         std::string typeName = token->getTokenValue();
00994         next();
00995         expectToken("=");
00996         next();
00997
00998         std::unique_ptr<ASTNode> typeDefinition;
00999         //bool isPacked = false;
01000         if (peekIs("packed")) {
01001             // isPacked = true;
01002             next(); // consume 'packed' modifier (accepted but has no effect)
01003         }
01004         if (peekIs("record")) {
01005             auto recordType = parseRecordType();
01006             typeDefinition = std::make_unique<RecordDeclarationNode>(
01007                 typeName,
01008                 std::unique_ptr<RecordTypeNode>(static_cast<RecordTypeNode
*>(recordType.release())));
01009         } else if (peekIs("(")) {
01010             // Enumerated type: Color = (Red, Green, Blue)
01011             next(); // consume '('
01012             std::vector<std::string> enumValues;
01013             expectToken(types::TokenType::TT_ID);
01014             enumValues.push_back(token->getTokenValue());
01015             next();
01016             while (peekIs(",")) {
01017                 next();
01018                 expectToken(types::TokenType::TT_ID);
01019                 enumValues.push_back(token->getTokenValue());
01020                 next();
01021             }
01022             expectToken(")");
01023             next();
01024             typeDefinition = std::make_unique<EnumTypeDeclNode>(typeName, std::move(enumValues));
01025         } else if (peekIs("array")) {
01026             auto arrayType = parseArrayType();

```

```

01027         typeDefinition = std::make_unique<ArrayTypeDeclarationNode>(
01028             typeName,
01029             std::unique_ptr<ArrayTypeNode>(static_cast<ArrayTypeNode>
*>(arrayType.release())));
01030     } else if (peekIs("set")) {
01031         next(); // consume 'set'
01032         expectToken("of");
01033         next();
01034         expectToken(types::TokenType::TT_ID);
01035         std::string baseType = token->getTokenValue();
01036         next();
01037         typeDefinition = std::make_unique<TypeAliasNode>(typeName, "set of " + baseType);
01038     } else if (peekIs("file")) {
01039         next(); // consume 'file'
01040         if (peekIs("of")) {
01041             next(); // consume 'of'
01042             expectToken(types::TokenType::TT_ID);
01043             next(); // consume element type (ignored -- all files are byte-stream)
01044         }
01045         typeDefinition = std::make_unique<TypeAliasNode>(typeName, "file");
01046     } else if (peekIs("^")) {
01047         next();
01048         expectToken(types::TokenType::TT_ID);
01049         std::string baseType = token->getTokenValue();
01050         next();
01051         typeDefinition = std::make_unique<TypeAliasNode>(typeName, "^" + baseType);
01052     } else {
01053         expectToken(types::TokenType::TT_ID);
01054         std::string baseType = token->getTokenValue();
01055         next();
01056         typeDefinition = std::make_unique<TypeAliasNode>(typeName, baseType);
01057     }
01058
01059     expectToken(";");
01060     next();
01061
01062     typeDeclarations.push_back(std::move(typeDefinition));
01063 } while (peekIs(types::TokenType::TT_ID) && !isKeyword(token->getTokenValue()));
01064
01065 auto typeDeclNode = std::make_unique<TypeDeclNode>(std::move(typeDeclarations));
01066 typeDeclNode->setLineNumber(lineNum);
01067 return typeDeclNode;
01068 }
01069
01070 std::unique_ptr<ASTNode> PascalParser::parseRecordType() {
01071     expectToken("record");
01072     next();
01073     std::vector<std::unique_ptr<ASTNode>> fields;
01074     while (!peekIs("end") && !peekIs("case")) {
01075         std::vector<std::string> ids;
01076         expectToken(types::TokenType::TT_ID);
01077         ids.push_back(token->getTokenValue());
01078         next();
01079         while (peekIs(",")) {
01080             next();
01081             expectToken(types::TokenType::TT_ID);
01082             ids.push_back(token->getTokenValue());
01083             next();
01084         }
01085         expectToken(":");
01086         next();
01087         auto fieldType = parseTypeSpec();
01088         fields.push_back(std::make_unique<VarDeclNode>(
01089             std::move(ids),
01090             std::move(fieldType),
01091             std::vector<std::unique_ptr<ASTNode>>{}));
01092         expectToken(";");
01093         next();
01094     }
01095
01096     auto recordNode = std::make_unique<RecordTypeNode>(std::move(fields));
01097
01098     // Parse variant part: case tag: type of ...
01099     if (peekIs("case")) {
01100         next(); // consume 'case'
01101         expectToken(types::TokenType::TT_ID);
01102         recordNode->variantTagName = token->getTokenValue();
01103         next();
01104         expectToken(":");
01105         next();
01106         expectToken(types::TokenType::TT_ID);
01107         recordNode->variantTagType = token->getTokenValue();
01108         next();
01109         expectToken("of");
01110         next();
01111
01112         while (!peekIs("end")) {

```



```

01113         VariantArm arm;
01114         // Parse case label(s): e.g. 1, 2:
01115         arm.caseLabels.push_back(parseExpression());
01116         while (peekIs(",")) {
01117             next();
01118             arm.caseLabels.push_back(parseExpression());
01119         }
01120         expectToken(":");
01121         next();
01122         expectToken("(");
01123         next();
01124         // Parse fields inside parentheses
01125         while (!peekIs(")")) {
01126             std::vector<std::string> ids;
01127             expectToken(types::TokenType::TT_ID);
01128             ids.push_back(token->getTokenValue());
01129             next();
01130             while (peekIs(",")) {
01131                 next();
01132                 expectToken(types::TokenType::TT_ID);
01133                 ids.push_back(token->getTokenValue());
01134                 next();
01135             }
01136             expectToken(":");
01137             next();
01138             auto fieldType = parseTypeSpec();
01139             arm.fields.push_back(std::make_unique<VarDeclNode>(
01140                 std::move(ids),
01141                 std::move(fieldType),
01142                 std::vector<std::unique_ptr<ASTNode>>{}));
01143             if (peekIs(";"))
01144                 next();
01145         }
01146         expectToken(")");
01147         next();
01148         if (peekIs(";"))
01149             next();
01150         recordNode->variantArms.push_back(std::move(arm));
01151     }
01152 }
01153
01154 expectToken("end");
01155 next();
01156 return recordNode;
01157 }
01158
01159 std::unique_ptr<ASTNode> PascalParser::parseVarDeclaration() {
01160     std::vector<std::string> identifiers;
01161     std::vector<std::unique_ptr<ASTNode>> initializers;
01162
01163     do {
01164         expectToken(types::TokenType::TT_ID);
01165         identifiers.push_back(token->getTokenValue());
01166         next();
01167         if (peekIs(","))
01168             next();
01169         else
01170             break;
01171     } while (true);
01172
01173     expectToken(":");
01174     next();
01175
01176     auto typeAst = parseTypeSpec();
01177
01178     if (peekIs(":=")) {
01179         next();
01180         initializers.push_back(parseExpression());
01181     }
01182
01183     expectToken(";");
01184     next();
01185
01186     return std::make_unique<VarDeclNode>(
01187         std::move(identifiers),
01188         std::move(typeAst),
01189         std::move(initializers));
01190 }
01191
01192 std::unique_ptr<ASTNode> PascalParser::parseTypeSpec() {
01193     if (peekIs("packed"))
01194         next(); // consume 'packed' modifier (accepted but has no effect)
01195     if (peekIs("set")) {
01196         next(); // consume 'set'
01197         expectToken("of");
01198         next();
01199         expectToken(types::TokenType::TT_ID);

```

```

01200         std::string baseType = token->getTokenValue();
01201         next();
01202         return std::make_unique<SetTypeNode>(baseType);
01203     }
01204     if (peekIs("file")) {
01205         next(); // consume 'file'
01206         if (peekIs("of")) {
01207             next(); // consume 'of' (element type accepted but treated as untyped)
01208             expectToken(types::TokenType::TT_ID);
01209             next(); // consume element type (ignored -- all files are byte-stream)
01210         }
01211         return std::make_unique<SimpleTypeNode>("file");
01212     }
01213     if (peekIs("array"))
01214         return parseArrayType();
01215     if (peekIs("record"))
01216         return parseRecordType();
01217     if (peekIs("^")) {
01218         next();
01219         expectToken(types::TokenType::TT_ID);
01220         std::string baseType = token->getTokenValue();
01221         next();
01222         return std::make_unique<PointerTypeNode>(baseType);
01223     }
01224     expectToken(types::TokenType::TT_ID);
01225     auto t = std::make_unique<SimpleTypeNode>(token->getTokenValue());
01226     next();
01227     return t;
01228 }
01229
01230 std::unique_ptr<ASTNode> PascalParser::parseArrayType() {
01231     expectToken("array");
01232     next();
01233     // Dynamic array: array of <type> (no bounds)
01234     if (peekIs("of")) {
01235         next();
01236         auto elementType = parseTypeSpec();
01237         return std::make_unique<ArrayTypeNode>(
01238             std::move(elementType),
01239             nullptr,
01240             nullptr);
01241     }
01242     // Static array: array[lo..hi] of <type>
01243     expectToken("[");
01244     next();
01245     auto lowerBound = parseExpression();
01246     expectToken("..");
01247     next();
01248     auto upperBound = parseExpression();
01249     expectToken("]");
01250     next();
01251     expectToken("of");
01252     next();
01253     auto elementType = parseTypeSpec();
01254     return std::make_unique<ArrayTypeNode>(
01255         std::move(elementType),
01256         std::move(lowerBound),
01257         std::move(upperBound));
01258 }
01259
01260 std::string PascalParser::tokenTypeToString(types::TokenType type) {
01261     switch (type) {
01262     case types::TokenType::TT_ID:
01263         return "identifier";
01264     case types::TokenType::TT_NUM:
01265         return "number";
01266     case types::TokenType::TT_STR:
01267         return "string";
01268     case types::TokenType::TT_SYM:
01269         return "symbol";
01270     default:
01271         return "unknown token";
01272     }
01273 }
01274
01275 std::unique_ptr<ASTNode> PascalParser::parseLValue() {
01276     int lineNum = token ? token->getLine() : 1;
01277     expectToken(types::TokenType::TT_ID);
01278     std::string name = token->getTokenValue();
01279     next();
01280
01281     std::unique_ptr<ASTNode> left = std::make_unique<VariableNode>(name);
01282     left->setLineNumber(lineNum);
01283
01284     while (true) {
01285         if (peekIs("(")) {
01286             next();

```

```

01287         auto index = parseExpression();
01288         expectToken("]");
01289         next();
01290         left = std::make_unique<ArrayAccessNode>(std::move(left), std::move(index));
01291         left->setLineNumber(lineNum);
01292     } else if (peekIs(".")) {
01293         next();
01294         expectToken(types::TokenType::TT_ID);
01295         std::string fieldName = token->getTokenValue();
01296         next();
01297         left = std::make_unique<FieldAccessNode>(std::move(left), fieldName);
01298         left->setLineNumber(lineNum);
01299     } else if (peekIs("^")) {
01300         next();
01301         left = std::make_unique<PointerDerefNode>(std::move(left));
01302         left->setLineNumber(lineNum);
01303     } else {
01304         break;
01305     }
01306 }
01307 return left;
01308 }
01309
01310 std::unique_ptr<ASTNode> PascalParser::parseWithStatement() {
01311     expectToken("with");
01312     int lineNum = token->getLine();
01313     next();
01314     expectToken(types::TokenType::TT_ID);
01315     std::string recordVar = token->getTokenValue();
01316     next();
01317     expectToken("do");
01318     next();
01319     auto stmt = parseStatement();
01320     auto withNode = std::make_unique<WithStmtNode>(recordVar, std::move(stmt));
01321     withNode->setLineNumber(lineNum);
01322     return withNode;
01323 }
01324
01325 std::unique_ptr<ASTNode> PascalParser::parseGotoStatement() {
01326     expectToken("goto");
01327     int lineNum = token->getLine();
01328     next();
01329     expectToken(types::TokenType::TT_NUM);
01330     std::string label = token->getTokenValue();
01331     next();
01332     auto gotoNode = std::make_unique<GotoStmtNode>(label);
01333     gotoNode->setLineNumber(lineNum);
01334     return gotoNode;
01335 }
01336
01337 void PascalParser::parseLabelDeclaration() {
01338     expectToken("label");
01339     next();
01340     do {
01341         expectToken(types::TokenType::TT_NUM);
01342         declaredLabels.insert(token->getTokenValue());
01343         next();
01344         if (peekIs(",")) {
01345             next();
01346         } else {
01347             break;
01348         }
01349     } while (true);
01350     expectToken(";");
01351     next();
01352 }
01353
01354 } // namespace pascal

```

12.80 src/parser.cpp File Reference

MXVM parser — tokenization, AST construction, and intermediate code generation.

```

#include "mxvm/parser.hpp"
#include "mxvm/ast.hpp"
#include "mxvm/icode.hpp"
#include "scanner/exception.hpp"
#include "scanner/scanner.hpp"
#include <algorithm>
#include <filesystem>
#include <fstream>

```



```

00040         default:
00041             out.push_back(c);
00042             break;
00043     }
00044 }
00045 return out;
00046 }
00047 std::string js_escape(const std::string &s) {
00048     std::string out;
00049     out.reserve(s.size() * 12 / 10 + 8);
00050     for (char c : s) {
00051         switch (c) {
00052             case '\\':
00053                 out += "\\\\";
00054                 break;
00055             case '\"':
00056                 out += "\\\"";
00057                 break;
00058             case '\'' :
00059                 out += "\\\"";
00060                 break;
00061             case '\n':
00062                 out += "\\n";
00063                 break;
00064             case '\r':
00065                 out += "\\r";
00066                 break;
00067             case '\0':
00068                 out += "\\0";
00069                 break;
00070             case '`':
00071                 out += "\\`";
00072                 break;
00073             case '$':
00074                 out += "\\$";
00075                 break;
00076             case '/':
00077                 out += "\\/";
00078                 break;
00079             default:
00080                 if (static_cast<unsigned char>(c) < 0x20) {
00081                     char buf[8];
00082                     std::snprintf(buf, sizeof(buf), "\\x%02x", static_cast<unsigned char>(c));
00083                     out += buf;
00084                 } else {
00085                     out.push_back(c);
00086                 }
00087                 break;
00088         }
00089     }
00090     return html_escape(out);
00091 }
00092 } // namespace
00093
00094 namespace mxvm {
00095
00096     bool debug_mode = false;
00097     bool instruct_mode = false;
00098     bool html_mode = false;
00099
00100     ModuleParser::ModuleParser(const Mode &mode, const std::string &m, const std::string &source) :
00101         mod_name(m), scanner(source), parser_mode(mode) {}
00102
00103     uint64_t ModuleParser::scan() {
00104         scanner.scan();
00105         return scanner.size();
00106     }
00107
00108     bool ModuleParser::parse() {
00109         next();
00110         require("module");
00111         next();
00112         require(types::TokenType::TT_ID);
00113         std::string mod_name = token->getTokenValue();
00114         next();
00115         require("{}");
00116         while (!match("{}")) {
00117             if (match(types::TokenType::TT_ID) && token->getTokenValue() == "extern") {
00118                 next();
00119                 require(types::TokenType::TT_ID);
00120                 std::string func_name = token->getTokenValue();
00121                 functions.push_back({func_name, mod_name});
00122                 next();
00123                 continue;
00124             }
00125             next();
00126         }
00127     }

```

```

00126         return true;
00127     }
00128
00129     scan::TToken ModuleParser::operator[](size_t pos) {
00130         return at(pos);
00131     }
00132
00133     scan::TToken ModuleParser::at(size_t pos) {
00134         if (pos < scanner.size())
00135             return scanner[pos];
00136
00137         throw scan::ScanExcept("Out of bounds");
00138         return scanner[0];
00139     }
00140
00141     bool ModuleParser::generateProgramCode(const Mode &m, const std::string &mod_id, const std::string
&mod_name1, std::unique_ptr<Program> &program) {
00142         for (auto &f : functions) {
00143             if (m == Mode::MODE_INTERPRET)
00144                 program->add_runtime_extern(mod_name, mod_name1, "mxvm_" + mod_id + "_" + f.name,
f.name);
00145             else
00146                 program->add_extern(mod_name, f.name, true);
00147         }
00148         return true;
00149     }
00150
00151     bool ModuleParser::next() {
00152         while (index < scanner.size() && scanner[index].getTokenType() != types::TokenType::TT_STR &&
scanner[index].getTokenValue() == "\n") {
00153             index++;
00154         }
00155         if (index < scanner.size()) {
00156             token = &scanner[index++];
00157             return true;
00158         }
00159         return false;
00160     }
00161
00162     bool ModuleParser::match(const std::string &s) {
00163         return (token->getTokenValue() == s);
00164     }
00165
00166     bool ModuleParser::match(const types::TokenType &t) {
00167         return (token->getTokenType() == t);
00168     }
00169
00170     void ModuleParser::require(const std::string &s) {
00171         if (!match(s)) {
00172             throw mx::Exception("Syntax Error: Looking for: " + s + " on line " +
std::to_string(token->getLine()));
00173         }
00174     }
00175     void ModuleParser::require(const types::TokenType &t) {
00176         if (!match(t)) {
00177             throw mx::Exception("Syntax Error: Looking for: " + std::to_string(static_cast<unsigned
int>(t)) + " on line " + std::to_string(token->getLine()));
00178         }
00179     }
00180
00181     Parser::Parser(const std::string &source) : source_file(source), scanner(source),
validator(source) {
00182     }
00183
00184     uint64_t Parser::scan() {
00185         try {
00186             scanner.scan();
00187         } catch (scan::ScanExcept &e) {
00188             std::cerr << "Scanner error: " << e.why() << "\n";
00189         }
00190         return scanner.size();
00191     }
00192
00193     auto Parser::operator[](size_t pos) {
00194         if (pos < scanner.size()) {
00195             return scanner[pos];
00196         }
00197         throw mx::Exception("Error: scanner index out of bounds.\n");
00198         return scanner[0];
00199     }
00200
00201     std::unique_ptr<ProgramNode> Parser::parseAST() {
00202         std::unique_ptr<ProgramNode> mainProgram = nullptr;
00203         std::vector<std::unique_ptr<ProgramNode>> allDeclarations;
00204         std::string name;
00205         for (uint64_t i = 0; i < scanner.size(); ++i) {
00206             auto token = this->operator[](i);

```

```

00207         std::string tokenValue = token.getTokenValue();
00208
00209         if (i + 1 < scanner.size())
00210             name = this->operator[](i + 1).getTokenValue();
00211
00212         if (tokenValue == "program" || tokenValue == "object") {
00213             auto declaration = parseProgramOrObject(i);
00214             if (declaration) {
00215                 if (tokenValue == "program") {
00216                     declaration->object = false;
00217                     declaration->name = name;
00218                     mainProgram = std::move(declaration);
00219                     object_name = name;
00220                 } else {
00221                     declaration->object = true;
00222                     declaration->name = name;
00223                     object_name = name;
00224                     allDeclarations.push_back(std::move(declaration));
00225                 }
00226             }
00227         }
00228     }
00229 }
00230
00231 if (!mainProgram) {
00232     mainProgram = std::make_unique<ProgramNode>();
00233     mainProgram->name = name;
00234     mainProgram->object = false;
00235     object_name = name;
00236 }
00237
00238 for (auto &obj : allDeclarations) {
00239     mainProgram->addInlineObject(std::move(obj));
00240 }
00241
00242 return mainProgram;
00243 }
00244
00245 std::unique_ptr<ObjectNode> Parser::parseObject(uint64_t &index) {
00246     if (index >= scanner.size())
00247         return nullptr;
00248     std::string name = this->operator[](index).getTokenValue();
00249     index++;
00250     if (index < scanner.size()) {
00251         if (this->operator[](index).getTokenValue() == ",")
00252             index++;
00253     }
00254     return std::make_unique<ObjectNode>(name);
00255 }
00256
00257 void Parser::processObjectSection(SectionNode *sectionNode, std::unique_ptr<Program> &program) {
00258     for (const auto &statement : sectionNode->statements) {
00259         auto objNode = dynamic_cast<ObjectNode*>(statement.get());
00260         if (objNode) {
00261             std::string &name = objNode->name;
00262             processObjectFile(name, program);
00263         }
00264     }
00265 }
00266
00267 void Parser::processObjectFile(const std::string &src, std::unique_ptr<Program> &program) {
00268     static std::set<std::string> processing_files;
00269     if (processing_files.count(src)) {
00270         return;
00271     }
00272     processing_files.insert(src);
00273     std::fstream file;
00274     std::string path;
00275     if (object_path.ends_with("/"))
00276         path = object_path;
00277     else
00278         path = object_path + "/";
00279
00280     file.open(path + src + ".mxvm", std::ios::in);
00281     if (!file.is_open()) {
00282         return;
00283     }
00284     std::ostringstream stream;
00285     stream << file.rdbuf();
00286     file.close();
00287
00288     std::string full_path = path + src + ".mxvm";
00289
00290     if (!std::filesystem::exists(full_path)) {
00291         throw mx::Exception("Object file not found: " + full_path);
00292     }
00293 }

```

```

00294         mxvm::Validator file_validator(stream.str());
00295     try {
00296         if (!file_validator.validate(full_path)) {
00297             throw mx::Exception("Validation failed for: " + full_path);
00298         }
00299     } catch (mx::Exception &e) {
00300         throw;
00301     }
00302
00303     std::unique_ptr<Parser> parser(new Parser(stream.str()));
00304     parser->module_path = module_path;
00305     parser->object_path = object_path;
00306     parser->object_mode = true;
00307     parser->parser_mode = parser_mode;
00308     parser->platform = platform;
00309     parser->scan();
00310
00311     auto ast = parser->parseAST();
00312     if (!ast)
00313         return;
00314
00315     for (const auto &inlineObj : ast->inlineObjects) {
00316         auto objProgram = std::make_unique<Program>();
00317         objProgram->name = inlineObj->name;
00318         objProgram->object = true;
00319         objProgram->object_external = true;
00320         objProgram->filename = path + src + ".mxvm";
00321         objProgram->platform = platform;
00322         for (const auto &section : inlineObj->sections) {
00323             auto sectionNode = dynamic_cast<SectionNode *>(section.get());
00324             if (!sectionNode)
00325                 continue;
00326
00327             if (sectionNode->type == SectionNode::DATA) {
00328                 parser->processDataSection(sectionNode, objProgram);
00329             } else if (sectionNode->type == SectionNode::CODE) {
00330                 parser->processCodeSection(sectionNode, objProgram);
00331             } else if (sectionNode->type == SectionNode::MODULE) {
00332                 parser->processModuleSection(sectionNode, objProgram);
00333             } else if (sectionNode->type == SectionNode::OBJECT) {
00334                 parser->processObjectSection(sectionNode, objProgram);
00335             }
00336         }
00337
00338         if (Program::base != nullptr && objProgram->object)
00339             Program::base->add_object(objProgram->name, objProgram.get());
00340
00341         registerObjectExterns(program, objProgram);
00342
00343         program->objects.push_back(std::move(objProgram));
00344     }
00345     processing_files.erase(src);
00346 }
00347 std::unique_ptr<SectionNode> Parser::parseSection(uint64_t &index) {
00348     index++;
00349
00350     if (index >= scanner.size())
00351         return nullptr;
00352
00353     auto sectionNameToken = this->operator[](index);
00354     std::string sectionName = sectionNameToken.getTokenValue();
00355
00356     SectionNode::SectionType sectionType;
00357     if (sectionName == "data") {
00358         sectionType = SectionNode::DATA;
00359     } else if (sectionName == "code") {
00360         sectionType = SectionNode::CODE;
00361     } else if (sectionName == "module") {
00362         sectionType = SectionNode::MODULE;
00363     } else if (sectionName == "object") {
00364         sectionType = SectionNode::OBJECT;
00365     } else {
00366         return nullptr;
00367     }
00368
00369     auto section = std::make_unique<SectionNode>(sectionType);
00370     index++;
00371
00372     if (index < scanner.size() && this->operator[](index).getTokenValue() == "{") {
00373         index++;
00374
00375         while (index < scanner.size()) {
00376             auto token = this->operator[](index);
00377             std::string value = token.getTokenValue();
00378
00379             if (value == ";") {
00380                 index++;

```



```

00381         break;
00382     }
00383
00384     if (value == "\n" || value == " " || value == "\t") {
00385         index++;
00386         continue;
00387     }
00388
00389     if (value == "/" || value.starts_with("/") || value == "#" ||
value.starts_with("#")) {
00390         auto comment = parseComment(index);
00391         if (comment) {
00392             section->addStatement(std::move(comment));
00393         }
00394         continue;
00395     }
00396
00397     if (sectionType == SectionNode::DATA) {
00398         auto variable = parseDataVariable(index);
00399         if (variable) {
00400             section->addStatement(std::move(variable));
00401         } else {
00402             index++;
00403         }
00404     } else if (sectionType == SectionNode::MODULE) {
00405         auto module = parseModule(index);
00406         if (module) {
00407             section->addStatement(std::move(module));
00408         } else {
00409             index++;
00410         }
00411     } else if (sectionType == SectionNode::OBJECT) {
00412         auto obj = parseObject(index);
00413         if (obj) {
00414             section->addStatement(std::move(obj));
00415         } else {
00416             index++;
00417         }
00418     } else if (sectionType == SectionNode::CODE) {
00419         if (token.getTokenType() == types::TokenType::TT_ID &&
index + 1 < scanner.size() &&
this->operator[](index + 1).getTokenValue() == ":") {
00420             auto label = parseLabel(index);
00421             if (label) {
00422                 section->addStatement(std::move(label));
00423             }
00424         } else if (token.getTokenType() == types::TokenType::TT_ID &&
token.getTokenValue() == "function" && index + 2 < scanner.size() && this->operator[](index +
1).getTokenType() == types::TokenType::TT_ID && this->operator[](index + 2).getTokenValue() == ":") {
00425             index++;
00426             auto label = parseLabel(index);
00427             if (label) {
00428                 label->function = true;
00429                 section->addStatement(std::move(label));
00430                 continue;
00431             }
00432         } else {
00433             auto instruction = parseCodeInstruction(index);
00434             if (instruction) {
00435                 section->addStatement(std::move(instruction));
00436             } else {
00437                 index++;
00438             }
00439         }
00440     }
00441 }
00442 }
00443 }
00444 }
00445
00446 return section;
00447 }
00448
00449 std::unique_ptr<ModuleNode> Parser::parseModule(uint64_t &index) {
00450     if (index >= scanner.size())
00451         return nullptr;
00452     std::string name = this->operator[](index).getTokenValue();
00453     index++;
00454     if (index < scanner.size()) {
00455         if (this->operator[](index).getTokenValue() == ",")
00456             index++;
00457     }
00458     return std::make_unique<ModuleNode>(name);
00459 }
00460
00461 std::unique_ptr<VariableNode> Parser::parseDataVariable(uint64_t &index) {
00462     if (index >= scanner.size())
00463         return nullptr;
00464

```

```

00465     bool is_global = false;
00466     auto token = this->operator[](index);
00467     if (token.getTokenValue() == "export") {
00468         is_global = true;
00469         index++;
00470         if (index >= scanner.size())
00471             return nullptr;
00472         token = this->operator[](index);
00473     }
00474
00475     std::string typeStr = token.getTokenValue();
00476     VarType varType;
00477     if (typeStr == "int")
00478         varType = VarType::VAR_INTEGER;
00479     else if (typeStr == "float")
00480         varType = VarType::VAR_FLOAT;
00481     else if (typeStr == "string")
00482         varType = VarType::VAR_STRING;
00483     else if (typeStr == "ptr")
00484         varType = VarType::VAR_POINTER;
00485     else if (typeStr == "array")
00486         varType = VarType::VAR_ARRAY;
00487     else if (typeStr == "byte")
00488         varType = VarType::VAR_BYTE;
00489     else
00490         return nullptr;
00491
00492     index++;
00493     if (index >= scanner.size())
00494         return nullptr;
00495     auto nameToken = this->operator[](index);
00496
00497     if (nameToken.getTokenType() != types::TokenType::TT_ID) {
00498         return nullptr;
00499     }
00500     std::string varName = nameToken.getTokenValue();
00501     index++;
00502
00503     if (index < scanner.size() && this->operator[](index).getTokenValue() == "=") {
00504         index++;
00505
00506         std::string add_op;
00507         if (index < scanner.size() && this->operator[](index).getTokenValue() == "-"
00508             && this->operator[](index).getTokenType() == types::TokenType::TT_SYM) {
00509             add_op = "-";
00510             index++;
00511         }
00512         if (index < scanner.size()) {
00513             auto valueToken = this->operator[](index);
00514             std::string value = valueToken.getTokenValue();
00515             types::TokenType tokenType = valueToken.getTokenType();
00516             switch (tokenType) {
00517                 case types::TokenType::TT_STR:
00518                     if (value.starts_with("\"") && value.ends_with("\"")) {
00519                         value = value.substr(1, value.length() - 2);
00520                     }
00521                     index++;
00522                     {
00523                         auto node = std::make_unique<VariableNode>(varType, varName, value);
00524                         node->is_global = is_global;
00525                         return node;
00526                     }
00527
00528                 case types::TokenType::TT_NUM:
00529                     index++;
00530                     {
00531                         auto node = std::make_unique<VariableNode>(varType, varName, add_op + value);
00532                         node->is_global = is_global;
00533                         return node;
00534                     }
00535
00536                 case types::TokenType::TT_HEX:
00537                     index++;
00538                     {
00539                         auto node = std::make_unique<VariableNode>(varType, varName, value);
00540                         node->is_global = is_global;
00541                         return node;
00542                     }
00543
00544                 case types::TokenType::TT_ID:
00545                     index++;
00546                     {
00547                         auto node = std::make_unique<VariableNode>(varType, varName, value);
00548                         node->is_global = is_global;
00549                         return node;
00550                     }
00551                 default:

```

```

00552         index++;
00553     {
00554         auto node = std::make_unique<VariableNode>(varType, varName, value);
00555         node->is_global = is_global;
00556         return node;
00557     }
00558 }
00559 }
00560 } else if (index < scanner.size() && this->operator[](index).getTokenValue() == "," && varType
== VarType::VAR_STRING) {
00561     index++;
00562     size_t buf_size = 0;
00563     if (index < scanner.size()) {
00564         buf_size = std::stoll(this->operator[](index).getTokenValue(), nullptr, 0);
00565     }
00566     if (buf_size == 0) {
00567         throw mx::Exception("string buffer: " + varName + " requires valid size");
00568     }
00569     auto node = std::make_unique<VariableNode>(varType, varName, buf_size);
00570     node->is_global = is_global;
00571     return node;
00572 }
00573
00574 auto node = std::make_unique<VariableNode>(varType, varName);
00575 node->is_global = is_global;
00576 return node;
00577 }
00578
00579 std::unique_ptr<InstructionNode> Parser::parseCodeInstruction(uint64_t &index) {
00580     static std::unordered_map<std::string, Inc> instructionMap = {
00581         {"mov", MOV}, {"load", LOAD}, {"store", STORE}, {"add", ADD}, {"sub", SUB}, {"mul", MUL},
{"div", DIV}, {"or", OR}, {"and", AND}, {"xor", XOR}, {"not", NOT}, {"mod", MOD}, {"cmp", CMP},
{"fcmp", FCMP}, {"jmp", JMP}, {"je", JE}, {"jne", JNE}, {"jl", JL}, {"jle", JLE}, {"jg", JG}, {"jge",
JGE}, {"jz", JZ}, {"jnz", JNZ}, {"ja", JA}, {"jb", JB}, {"jae", JAE}, {"jbe", JBE}, {"jc", JC},
{"jnc", JNC}, {"jp", JP}, {"jnp", JNP}, {"jo", JO}, {"jno", JNO}, {"js", JS}, {"jns", JNS}, {"print",
PRINT}, {"exit", EXIT}, {"alloc", ALLOC}, {"free", FREE}, {"getline", GETLINE}, {"push", PUSH},
{"pop", POP}, {"stack_load", STACK_LOAD}, {"stack_store", STACK_STORE}, {"stack_sub", STACK_SUB},
{"call", CALL}, {"ret", RET}, {"string_print", STRING_PRINT}, {"done", DONE}, {"to_int", TO_INT},
{"to_float", TO_FLOAT}, {"invoke", INVOKE}, {"return", RETURN}, {"neg", NEG}, {"lea", LEA},
{"realloc", REALLOC}};
00582
00583     if (index >= scanner.size())
00584         return nullptr;
00585
00586     auto token = this->operator[](index);
00587     std::string tokenValue = token.getTokenValue();
00588
00589     if (token.getTokenType() == types::TokenType::TT_ID) {
00590         if (index + 1 < scanner.size() && this->operator[](index + 1).getTokenValue() == ":") {
00591             return nullptr;
00592         }
00593     }
00594
00595     auto instIt = instructionMap.find(tokenValue);
00596     if (instIt == instructionMap.end()) {
00597         throw mx::Exception("Error invalid instruction: " + tokenValue);
00598         return nullptr;
00599     }
00600
00601     Inc instruction = instIt->second;
00602     index++;
00603
00604     std::vector<Operand> operands;
00605
00606     while (index < scanner.size()) {
00607         std::string add_value;
00608         if (index < scanner.size() && this->operator[](index).getTokenValue() == "-") {
00609             add_value = "-";
00610             index++;
00611         }
00612         auto operandToken = this->operator[](index);
00613         std::string value = operandToken.getTokenValue();
00614         types::TokenType tokenType = operandToken.getTokenType();
00615
00616         if (value == "\n" || value == "/" || value.starts_with("/") ||
value == "#" || value.starts_with("#") || value == "{") {
00617             break;
00618         }
00619
00620         if (value == "," || value == " " || value == "\t") {
00621             index++;
00622             continue;
00623         }
00624
00625         Operand operand;
00626
00627
00628

```

```

00629         switch (tokenType) {
00630         case types::TokenType::TT_NUM:
00631             operand.op = add_value + value;
00632             operand.op_value = std::stoll(add_value + value);
00633             operand.type = OperandType::OP_CONSTANT;
00634             break;
00635
00636         case types::TokenType::TT_HEX:
00637             operand.op = value;
00638             if (value.starts_with("0x") || value.starts_with("0X")) {
00639                 operand.op_value = std::stoll(value, nullptr, 16);
00640             } else {
00641                 operand.op_value = std::stoll(value, nullptr, 16);
00642             }
00643             operand.type = OperandType::OP_CONSTANT;
00644             break;
00645
00646         case types::TokenType::TT_ID:
00647             if (index + 1 < scanner.size() && this->operator[](index + 1).getTokenValue() == ".")
00648             {
00649                 operand.object = value;
00650                 index += 2;
00651                 if (index < scanner.size())
00652                     value = this->operator[](index).getTokenValue();
00653             }
00654
00655             if (operand.object.empty())
00656                 operand.op = value;
00657             else
00658                 operand.op = operand.object + "." + value;
00659             operand.label = value;
00660             operand.type = OperandType::OP_VARIABLE;
00661             break;
00662
00663         case types::TokenType::TT_STR:
00664             operand.op = value;
00665             operand.type = OperandType::OP_CONSTANT;
00666             break;
00667
00668         case types::TokenType::TT_SYM:
00669             if (value != ",", ")") {
00670                 operand.op = value;
00671             }
00672             break;
00673         default:
00674             operand.op = value;
00675             break;
00676     }
00677
00678     if (!operand.op.empty()) {
00679         operands.push_back(operand);
00680     }
00681
00682     index++;
00683 }
00684
00685 return std::make_unique<InstructionNode>(instruction, operands);
00686 }
00687
00688 std::unique_ptr<CommentNode> Parser::parseComment(uint64_t &index) {
00689     std::string commentText;
00690
00691     auto token = this->operator[](index);
00692     if (token.getTokenValue() == "/*" || token.getTokenValue() == "#" ||
00693         token.getTokenValue().starts_with("//") || token.getTokenValue().starts_with("#")) {
00694         index++;
00695     }
00696
00697     while (index < scanner.size()) {
00698         auto token = this->operator[](index);
00699         if (token.getTokenValue() == "\n") {
00700             break;
00701         }
00702         commentText += token.getTokenValue() + " ";
00703         index++;
00704     }
00705
00706     return std::make_unique<CommentNode>(commentText);
00707 }
00708
00709 std::unique_ptr<LabelNode> Parser::parseLabel(uint64_t &index) {
00710     if (index >= scanner.size())
00711         return nullptr;
00712
00713     auto token = this->operator[](index);
00714

```

```

00715         if (token.getTokenType() != types::TokenType::TT_ID) {
00716             return nullptr;
00717         }
00718
00719         std::string labelName = token.getTokenValue();
00720         index++;
00721
00722         if (index < scanner.size() && this->operator[](index).getTokenValue() == ":") {
00723             index++;
00724             return std::make_unique<LabelNode>(labelName);
00725         }
00726
00727         return nullptr;
00728     }
00729
00730     void Parser::parse() {
00731         auto ast = parseAST();
00732         if (ast) {
00733             std::cout << ast->toString() << std::endl;
00734         }
00735     }
00736
00737     bool Parser::generateProgramCode(const Mode &mode, std::unique_ptr<Program> &program) {
00738         parser_mode = mode;
00739         try {
00740             if (!validator.validate(program->filename)) {
00741                 return false;
00742             }
00743         } catch (mx::Exception &e) {
00744             std::cerr << e.what() << "\n";
00745             return false;
00746         }
00747
00748         auto ast = parseAST();
00749         if (ast) {
00750             program->name = ast->name;
00751             if (!ast->root_name.empty()) {
00752                 program->root_name = ast->root_name;
00753                 program->name = ast->root_name;
00754                 program->object = false;
00755             } else {
00756                 program->object = true;
00757             }
00758
00759             for (const auto &section : ast->sections) {
00760                 auto sectionNode = dynamic_cast<SectionNode *>(section.get());
00761                 if (!sectionNode)
00762                     continue;
00763
00764                 if (sectionNode->type == SectionNode::DATA) {
00765                     processDataSection(sectionNode, program);
00766                 } else if (sectionNode->type == SectionNode::CODE) {
00767                     processCodeSection(sectionNode, program);
00768                 } else if (sectionNode->type == SectionNode::MODULE) {
00769                     processModuleSection(sectionNode, program);
00770                 } else if (sectionNode->type == SectionNode::OBJECT) {
00771                     processObjectSection(sectionNode, program);
00772                 }
00773             }
00774
00775             for (const auto &inlineObj : ast->inlineObjects) {
00776                 auto objProgram = std::make_unique<Program>();
00777                 objProgram->name = inlineObj->name;
00778                 objProgram->object = true;
00779                 objProgram->object_external = false;
00780                 objProgram->filename = program->filename;
00781                 objProgram->platform = platform;
00782                 objProgram->object_name = inlineObj->name;
00783
00784                 for (const auto &section : inlineObj->sections) {
00785                     auto sectionNode = dynamic_cast<SectionNode *>(section.get());
00786                     if (!sectionNode)
00787                         continue;
00788
00789                     if (sectionNode->type == SectionNode::DATA) {
00790                         processDataSection(sectionNode, objProgram);
00791                     } else if (sectionNode->type == SectionNode::CODE) {
00792                         processCodeSection(sectionNode, objProgram);
00793                     } else if (sectionNode->type == SectionNode::MODULE) {
00794                         processModuleSection(sectionNode, objProgram);
00795                     } else if (sectionNode->type == SectionNode::OBJECT) {
00796                         processObjectSection(sectionNode, objProgram);
00797                     }
00798                 }
00799
00800                 registerObjectExterns(program, objProgram);
00801

```

```

00802         if (Program::base != nullptr && objProgram->object)
00803             Program::base->add_object(objProgram->name, objProgram.get());
00804
00805         program->objects.push_back(std::move(objProgram));
00806     }
00807
00808     if (mode == Mode::MODE_COMPILE) {
00809         std::set<std::string> generated_objects;
00810
00811         std::function<void(std::unique_ptr<Program> &)> compileAllObjects;
00812         compileAllObjects =
00813             [&](std::unique_ptr<Program> &p) {
00814                 if (!p)
00815                     return;
00816                 for (auto &obj : p->objects) {
00817                     if (obj && generated_objects.find(obj->name) == generated_objects.end()) {
00818                         generateObjectAssemblyFile(obj);
00819                         generated_objects.insert(obj->name);
00820                         if (html_mode) {
00821                             std::ofstream htmlFile(obj->name + ".html");
00822                             if (htmlFile.is_open()) {
00823                                 generateDebugHTML(htmlFile, obj);
00824                                 std::cout << Col{"MXVM: Generated ", mx::Color::BRIGHT_GREEN} <<
"Debug HTML for: " << obj->name << "\n";
00825                             }
00826                         }
00827                     }
00828                     compileAllObjects(obj);
00829                 }
00830             };
00831         compileAllObjects(program);
00832     }
00833
00834     if (program->object == false) {
00835         if (!program->validateNames(validator)) {
00836             throw mx::Exception("Could not validate variables/functions\n");
00837         }
00838     }
00839 }
00840 return true;
00841 }
00842 void Parser::collectObjectNames(std::vector<std::pair<std::string, std::string> &names, const
std::unique_ptr<Program> &program) {
00843     for (const auto &objPtr : program->objects) {
00844         if (!objPtr)
00845             continue;
00846         for (const auto &ext : objPtr->external) {
00847             if (std::find(names.begin(), names.end(), std::make_pair(ext.mod, ext.name)) ==
names.end())
00848                 names.push_back(std::make_pair(ext.mod, ext.name));
00849         }
00850         collectObjectNames(names, objPtr);
00851     }
00852 }
00853
00854 void Parser::printObjectHTML(std::ostream &out, const std::unique_ptr<Program> &objPtr) {
00855     out << R"(<div class="object-section">
00856         <div class="object-title">Object: <span style="color: lime;"><strong>) "
00857         << html_escape(objPtr->name) << R"(</span></strong></div>) ";
00858     out << R" (
00859         <div class="stats" style="margin-bottom:30px;">
00860             <div class="stat-card">
00861                 <span class="number">) "
00862         << objPtr->vars.size() << R"(</span>
00863             <span class="label">Variables</span>
00864             </div>
00865             <div class="stat-card">
00866                 <span class="number">) "
00867         << objPtr->inc.size() << R"(</span>
00868             <span class="label">Instructions</span>
00869             </div>
00870             <div class="stat-card">
00871                 <span class="number">) "
00872         << objPtr->labels.size() << R"(</span>
00873             <span class="label">Labels</span>
00874             </div>
00875             <div class="stat-card">
00876                 <span class="number">) "
00877         << objPtr->objects.size() << R"(</span>
00878             <span class="label">Objects</span>
00879             </div>
00880             </div>
00881         ) ";
00882     out << R" (
00883         <div class="section">
00884             <div class="section-header"><span class="icon">&#x1F9EE;</span> Variables</div>
00885             <div class="section-content">) ";

```

```

00886         if (objPtr->vars.empty()) {
00887             out << R"(<div class="no-data">No variables defined</div>)" ;
00888         } else {
00889             out << R"(<div class="variables-grid">)" ;
00890             for (const auto &var : objPtr->vars) {
00891                 out << R"(<div class="variable-card">
00892                     <div class="variable-name">)"
00893                     << html_escape(var.first) << R"(</div>
00894                     <div class="variable-type">)" ;
00895                 switch (var.second.type) {
00896                     case VarType::VAR_INTEGER:
00897                         out << "Integer";
00898                         break;
00899                     case VarType::VAR_FLOAT:
00900                         out << "Float";
00901                         break;
00902                     case VarType::VAR_STRING:
00903                         out << "String";
00904                         break;
00905                     case VarType::VAR_POINTER:
00906                         out << "Pointer";
00907                         break;
00908                     case VarType::VAR_LABEL:
00909                         out << "Label";
00910                         break;
00911                     case VarType::VAR_EXTERN:
00912                         out << "External";
00913                         break;
00914                     case VarType::VAR_ARRAY:
00915                         out << "Array";
00916                         break;
00917                     case VarType::VAR_BYTE:
00918                         out << "Byte";
00919                         break;
00920                     default:
00921                         out << "Unknown";
00922                         break;
00923                 }
00924                 out << R"(</div>
00925                     <div class="variable-value">)" ;
00926                 switch (var.second.type) {
00927                     case VarType::VAR_INTEGER:
00928                     case VarType::VAR_BYTE:
00929                     out << var.second.var_value.int_value;
00930                     break;
00931                     case VarType::VAR_FLOAT:
00932                     out << std::fixed << std::setprecision(6) << var.second.var_value.float_value;
00933                     break;
00934                     case VarType::VAR_STRING:
00935                     out << "\"" << html_escape(Program::escapeNewLines(var.second.var_value.str_value))
00936                     << "\"";
00937                     break;
00938                     case VarType::VAR_POINTER:
00939                     case VarType::VAR_EXTERN:
00940                     if (var.second.var_value.ptr_value == nullptr)
00941                         out << "null";
00942                     else
00943                         out << var.second.var_value.ptr_value;
00944                     break;
00945                     case VarType::VAR_LABEL:
00946                     out << var.second.var_value.label_value;
00947                     break;
00948                     default:
00949                     out << var.second.var_value.int_value;
00950                     break;
00951                 }
00952                 out << R"(</div>
00953                     </div>)" ;
00954             }
00955             out << R"(</div>)" ;
00956         }
00957         out << R"(</div>
00958             </div>)" ;
00959         out << R"(<div class="section">
00960             <div class="section-header"><span class="icon">&#x1F516;</span> Labels</div>
00961             <div class="section-content">)" ;
00962         if (objPtr->labels.empty()) {
00963             out << R"(<div class="no-data">No labels defined</div>)" ;
00964         } else {
00965             out << R"(<table class="instructions-table">
00966                 <thead>
00967                 <tr>
00968                     <th>Name</th>
00969                     <th>Address</th>
00970                     <th>Function</th>
00971                 </tr>
                 </thead>

```

```

00972         <tbody>");
00973     for (const auto &label : objPtr->labels) {
00974         out « R"(<tr>
00975             <td>)"
00976             « html_escape(label.first) « R"(</td>
00977             <td>)"
00978             « "0x" « std::hex « label.second.first « std::dec « R"(</td>
00979             <td>)"
00980             « (label.second.second ? "true" : "false") « R"(</td>
00981             </tr>)" ;
00982     }
00983     out « R"(</tbody>
00984         </table>)" ;
00985 }
00986 out « R"(</div>
00987 </div>)" ;
00988 out « R" (
00989     <div class="section">
00990         <div class="section-header"><span class="icon">&#9881;</span> Instructions</div>
00991         <div class="section-content">)" ;
00992 if (objPtr->inc.empty()) {
00993     out « R"(<div class="no-data">No instructions defined</div>)" ;
00994 } else {
00995     out « R"(<table class="instructions-table">
00996         <thead>
00997             <tr>
00998                 <th>#</th>
00999                 <th>Opcode</th>
01000                 <th>Instruction</th>
01001                 <th>Operand 1</th>
01002                 <th>Operand 2</th>
01003                 <th>Operand 3</th>
01004                 <th>Extra Operands</th>
01005             </tr>
01006         </thead>
01007         <tbody>)" ;
01008     for (size_t i = 0; i < objPtr->inc.size(); ++i) {
01009         const auto &instr = objPtr->inc[i];
01010         out « " <tr>"
01011             « " <td>0x" « std::hex « std::uppercase « i « std::dec « " </td>"
01012             « " <td class=\"opcode\">0x" « std::hex « std::uppercase «
static_cast<int>(instr.instruction) « std::dec « " </td>"
01013             « " <td class=\"opcode-name\">" «
html_escape(IncType[static_cast<int>(instr.instruction)]) « " </td>"
01014             « " <td class=\"operand\">" « html_escape(instr.op1.op) « " </td>"
01015             « " <td class=\"operand\">" « html_escape(instr.op2.op) « " </td>"
01016             « " <td class=\"operand\">" « html_escape(instr.op3.op) « " </td>"
01017             « " <td class=\"operand\">";
01018         for (size_t j = 0; j < instr.vop.size(); ++j) {
01019             if (j > 0)
01020                 out « ", ";
01021             out « html_escape(instr.vop[j].op);
01022         }
01023         out « " </td></tr>";
01024     }
01025     out « R"(</tbody>
01026         </table>)" ;
01027 }
01028 out « R"(</div>
01029 </div>)" ;
01030 if (!objPtr->objects.empty()) {
01031     out « R"(<div class="section">
01032         <div class="section-header"><span class="icon">&#x1F9E9;</span> Nested
Objects</div>
01033         <div class="section-content">)" ;
01034     for (const auto &nestedObj : objPtr->objects) {
01035         if (nestedObj) {
01036             printObjectHTML(out, nestedObj);
01037         }
01038     }
01039     out « R"(</div>
01040 </div>)" ;
01041 }
01042 out « R"(<div class="section">
01043 <div class="section-header"><span class="icon">&#x1F4DC;</span> Compiled Assembly</div>
01044 <div class="section-content">
01045     <textarea id="asm-"
01046         « html_escape(objPtr->name) « R"(" readonly
style="width:100%;height:300px;background:#222;color:#e0e0e0;border-radius:8px;padding:16px;font-family:'Fira
Mono', 'Consolas', 'Courier New', monospace;font-size:1rem;">)" ;
01047     out « html_escape(objPtr->assembly_code);
01048     out « R"(</textarea>
01049     <div style="margin-top:10px;">)" ;
01050     out « "<button class=\"btn\" onclick=\"copyAsm('asm-" « js_escape(objPtr->name) «
'">Copy</button>";
01051     out « "<button class=\"btn\" onclick=\"downloadAsm('asm-" « js_escape(objPtr->name) « "' , ' " «
js_escape(objPtr->name) « ".s')\">Download</button>";

```



```

01052         out << R"(</div>
01053         </div>
01054 </div>
01055 )";
01056     }
01057
01058     bool Parser::generateDebugHTML(std::ostream &out, std::unique_ptr<Program> &program) {
01059         out << R"(<!DOCTYPE html>
01060         <html lang="en">
01061         <head>
01062             <meta charset="UTF-8">
01063             <meta name="viewport" content="width=device-width, initial-scale=1.0">
01064             <title>MXVM Debug Report - )"
01065         << html_escape(program->name) << R"(</title>
01066         </head>
01067         <body>
01068             <style>
01069                 * {
01070                     margin: 0;
01071                     padding: 0;
01072                     box-sizing: border-box;
01073                 }
01074                 body {
01075                     font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
01076                     background: #000;
01077                     color: #e0e0e0;
01078                     line-height: 1.6;
01079                     min-height: 100vh;
01080                 }
01081                 .container {
01082                     width: 100%;
01083                     max-width: 100vw;
01084                     margin: 0 auto;
01085                     padding: 20px;
01086                     background: transparent;
01087                 }
01088                 .header {
01089                     background: #222;
01090                     border-radius: 15px;
01091                     padding: 30px;
01092                     margin-bottom: 30px;
01093                     box-shadow: 0 8px 32px rgba(183, 28, 28, 0.2);
01094                     border: 1px solid #b71c1c;
01095                 }
01096                 .header h1 {
01097                     color: #ff5252;
01098                     font-size: 2.5rem;
01099                     margin-bottom: 10px;
01100                     font-weight: 700;
01101                     text-shadow: 1px 1px 2px #000;
01102                 }
01103                 .header .subtitle {
01104                     color: #e57373;
01105                     font-size: 1.2rem;
01106                     margin-bottom: 20px;
01107                 }
01108                 .stats {
01109                     display: flex;
01110                     gap: 20px;
01111                     flex-wrap: wrap;
01112                 }
01113                 .stat-card {
01114                     background: #b71c1c;
01115                     color: #fff;
01116                     padding: 15px 25px;
01117                     border-radius: 10px;
01118                     text-align: center;
01119                     min-width: 150px;
01120                     box-shadow: 0 4px 15px rgba(183, 28, 28, 0.3);
01121                 }
01122                 .stat-card .number {
01123                     font-size: 2rem;
01124                     font-weight: bold;
01125                     display: block;
01126                 }
01127                 .stat-card .label {
01128                     font-size: 0.9rem;
01129                     opacity: 0.9;
01130                 }
01131                 .section {
01132                     background: #181818;
01133                     border-radius: 15px;
01134                     margin-bottom: 30px;
01135                     box-shadow: 0 8px 32px rgba(183, 28, 28, 0.1);
01136                     border: 1px solid #b71c1c;
01137                     overflow: hidden;
01138

```

```

01139     .section-header {
01140         background: #b71c1c;
01141         color: #fff;
01142         padding: 20px 30px;
01143         font-size: 1.4rem;
01144         font-weight: 600;
01145         display: flex;
01146         align-items: center;
01147         gap: 10px;
01148     }
01149     .section-content {
01150         padding: 30px;
01151     }
01152     .variables-grid {
01153         display: grid;
01154         gap: 15px;
01155         grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
01156     }
01157     .variable-card {
01158         background: #222;
01159         border: 1px solid #b71c1c;
01160         border-radius: 10px;
01161         padding: 20px;
01162         color: #fff;
01163         transition: transform 0.2s, box-shadow 0.2s;
01164     }
01165     .variable-card:hover {
01166         transform: translateY(-2px);
01167         box-shadow: 0 4px 20px rgba(183, 28, 28, 0.2);
01168     }
01169     .variable-name {
01170         font-weight: 600;
01171         font-size: 1.1rem;
01172         color: #ff5252;
01173         margin-bottom: 8px;
01174     }
01175     .variable-type {
01176         display: inline-block;
01177         background: #b71c1c;
01178         color: #fff;
01179         padding: 4px 12px;
01180         border-radius: 20px;
01181         font-size: 0.8rem;
01182         font-weight: 500;
01183         margin-bottom: 10px;
01184     }
01185     .variable-value {
01186         font-family: 'Courier New', monospace;
01187         background: #181818;
01188         border: 1px solid #b71c1c;
01189         border-radius: 5px;
01190         padding: 10px;
01191         color: #e0e0e0;
01192         word-break: break-all;
01193     }
01194     .instructions-table {
01195         width: 100%;
01196         border-collapse: collapse;
01197         background: #222;
01198         border-radius: 10px;
01199         overflow: hidden;
01200         box-shadow: 0 2px 10px rgba(183, 28, 28, 0.1);
01201     }
01202     .instructions-table th {
01203         background: #b71c1c;
01204         color: #fff;
01205         padding: 15px;
01206         text-align: left;
01207         font-weight: 600;
01208     }
01209     .instructions-table td {
01210         padding: 12px 15px;
01211         border-bottom: 1px solid #b71c1c;
01212         color: #e0e0e0;
01213     }
01214     .instructions-table tr:nth-child(even) {
01215         background: #181818;
01216     }
01217     .instructions-table tr:hover {
01218         background: #2d2d2d;
01219     }
01220     .opcode {
01221         font-weight: 600;
01222         color: #ff5252;
01223     }
01224     .operand {
01225         font-family: 'Courier New', monospace;

```

```

01226         color: #e0e0e0;
01227     }
01228     .object-section {
01229         margin-left: 20px;
01230         border-left: 3px solid #b71c1c;
01231         padding-left: 20px;
01232     }
01233     .object-title {
01234         font-size: 1.2rem;
01235         font-weight: 600;
01236         color: #ff5252;
01237         margin-bottom: 15px;
01238         padding: 10px 15px;
01239         background: #222;
01240         border-radius: 8px;
01241         border-left: 4px solid #b71c1c;
01242     }
01243     .btn {
01244         background: #b71c1c;
01245         color: white;
01246         border: none;
01247         padding: 8px 16px;
01248         border-radius: 5px;
01249         cursor: pointer;
01250         margin-right: 10px;
01251         font-weight: 600;
01252         transition: background 0.2s;
01253     }
01254     .btn:hover {
01255         background: #ff5252;
01256     }
01257     .icon {
01258         width: 20px;
01259         height: 20px;
01260         display: inline-block;
01261     }
01262     .no-data {
01263         text-align: center;
01264         color: #e57373;
01265         font-style: italic;
01266         padding: 40px;
01267     }
01268     @media (max-width: 768px) {
01269         .container {
01270             padding: 10px;
01271         }
01272         .header h1 {
01273             font-size: 2rem;
01274         }
01275         .stats {
01276             justify-content: center;
01277         }
01278         .variables-grid {
01279             grid-template-columns: 1fr;
01280         }
01281         .instructions-table {
01282             font-size: 0.9rem;
01283         }
01284     }
01285     </style>
01286     ">
01287
01288     bool mainHasContent = !program->vars.empty() || !program->inc.empty() ||
!program->labels.empty();
01289     bool hasObjects = !program->objects.empty();
01290
01291     out << R"(
01292         <div class="container">
01293             <div class="header">
01294                 <h1>MXVM Debug Report</h1>
01295                 <div class="subtitle">Program: <span style="color: blue;"><strong>"
01296                 << html_escape(program->name) << R"(</span></strong></div>
01297                 <div class="stats">
01298                     <div class="stat-card">
01299                         <span class="number">"
01300                     << program->vars.size() << R"(</span>
01301                         <span class="label">Variables</span>
01302                     </div>
01303                     <div class="stat-card">
01304                         <span class="number">"
01305                     << program->inc.size() << R"(</span>
01306                         <span class="label">Instructions</span>
01307                     </div>
01308                     <div class="stat-card">
01309                         <span class="number">"
01310                     << program->objects.size() << R"(</span>
01311                         <span class="label">Objects</span>

```

```

01312             </div>
01313             <div class="stat-card">
01314                 <span class="number">> "
01315         « program->labels.size() « R"(</span>
01316             <span class="label">Labels</span>
01317             </div>
01318         </div>
01319     </div>";
01320
01321     if (mainHasContent) {
01322         out « R"(<div class="section">
01323             <div class="section-header">
01324                 <span class="icon">&#x1F9EE;</span>
01325                 Variables
01326             </div>
01327             <div class="section-content">);";
01328
01329         if (program->vars.empty()) {
01330             out « R"(<div class="no-data">No variables defined</div>);";
01331         } else {
01332             out « R"(<div class="variables-grid">";
01333             for (const auto &var : program->vars) {
01334                 out « R"(<div class="variable-card">
01335                     <div class="variable-name">)"
01336                     « html_escape(var.first) « R"(</div>
01337                     <div class="variable-type">);";
01338
01339                 switch (var.second.type) {
01340                     case VarType::VAR_INTEGER:
01341                         out « "Integer";
01342                         break;
01343                     case VarType::VAR_FLOAT:
01344                         out « "Float";
01345                         break;
01346                     case VarType::VAR_STRING:
01347                         out « "String";
01348                         break;
01349                     case VarType::VAR_POINTER:
01350                         out « "Pointer";
01351                         break;
01352                     case VarType::VAR_LABEL:
01353                         out « "Label";
01354                         break;
01355                     case VarType::VAR_EXTERN:
01356                         out « "External";
01357                         break;
01358                     case VarType::VAR_ARRAY:
01359                         out « "Array";
01360                         break;
01361                     case VarType::VAR_BYTE:
01362                         out « "Byte";
01363                         break;
01364                     default:
01365                         out « "Unknown";
01366                         break;
01367                 }
01368
01369                 out « R"(</div>
01370                     <div class="variable-value">);";
01371
01372                 switch (var.second.type) {
01373                     case VarType::VAR_INTEGER:
01374                     case VarType::VAR_BYTE:
01375                         out « var.second.var_value.int_value;
01376                         break;
01377                     case VarType::VAR_FLOAT:
01378                         out « std::fixed « std::setprecision(6) « var.second.var_value.float_value;
01379                         break;
01380                     case VarType::VAR_STRING:
01381                         out « "\"" «
html_escape(Program::escapeNewLines(var.second.var_value.str_value)) « "\"";
01382                         break;
01383                     case VarType::VAR_POINTER:
01384                     case VarType::VAR_EXTERN:
01385                         if (var.second.var_value.ptr_value == nullptr)
01386                             out « "null";
01387                         else
01388                             out « var.second.var_value.ptr_value;
01389                         break;
01390                     case VarType::VAR_LABEL:
01391                         out « var.second.var_value.label_value;
01392                         break;
01393                     default:
01394                         out « var.second.var_value.int_value;
01395                         break;
01396                 }
01397             }

```

```

01398         out << R("</div>
01399             </div>");
01400     }
01401     out << R("</div>");
01402 }
01403 out << R("</div>
01404     </div>");
01405
01406 out << R("<div class=\"section\">
01407     <div class=\"section-header\"><span class=\"icon\">&#x1F516;</span> Labels</div>
01408     <div class=\"section-content\">");
01409 if (program->labels.empty()) {
01410     out << R("<div class=\"no-data\">No labels defined</div>");
01411 } else {
01412     out << R("<table class=\"instructions-table\">
01413         <thead>
01414             <tr>
01415                 <th>Name</th>
01416                 <th>Function</th>
01417             </tr>
01418         </thead>
01419         <tbody>");
01420     for (const auto &label : program->labels) {
01421         out << " <tr><td>" << html_escape(label.first) << " </td><td>0x" << std::hex <<
label.second.first << std::dec << R("</td>
01422             <td>")
01423             << (label.second.second ? "true" : "false") << R("</td>
01424             </tr>");
01425     }
01426     out << R("</tbody>
01427         </table>");
01428 }
01429 out << R("</div>
01430     </div>");
01431
01432 out << R("<div class=\"section\">
01433     <div class=\"section-header\">
01434         <span class=\"icon\">&#9881;</span>
01435         Instructions
01436     </div>
01437     <div class=\"section-content\">");
01438
01439 if (program->inc.empty()) {
01440     out << R("<div class=\"no-data\">No instructions defined</div>");
01441 } else {
01442     out << R("<table class=\"instructions-table\">
01443         <thead>
01444             <tr>
01445                 <th>#</th>
01446                 <th>Opcode</th>
01447                 <th>Instruction</th>
01448                 <th>Operand 1</th>
01449                 <th>Operand 2</th>
01450                 <th>Operand 3</th>
01451                 <th>Extra Operands</th>
01452             </tr>
01453         </thead>
01454         <tbody>");
01455
01456     for (size_t i = 0; i < program->inc.size(); ++i) {
01457         const auto &instr = program->inc[i];
01458         out << " <tr>"
01459             << " <td>0x" << std::hex << std::uppercase << i << std::dec << " </td>"
01460             << " <td class=\"opcode\">0x" << std::hex << std::uppercase <<
static_cast<int>(instr.instruction) << std::dec << " </td>"
01461             << " <td class=\"operand-name\">" <<
html_escape(IncType[static_cast<int>(instr.instruction)]) << " </td>"
01462             << " <td class=\"operand\">" << html_escape(instr.op1.op) << " </td>"
01463             << " <td class=\"operand\">" << html_escape(instr.op2.op) << " </td>"
01464             << " <td class=\"operand\">" << html_escape(instr.op3.op) << " </td>"
01465             << " <td class=\"operand\">";
01466     for (size_t j = 0; j < instr.vop.size(); ++j) {
01467         if (j > 0)
01468             out << ", ";
01469         out << html_escape(instr.vop[j].op);
01470     }
01471     out << " </td></tr>";
01472 }
01473     out << R("</tbody>
01474         </table>");
01475 }
01476 out << R("<div class=\"section\">
01477     <div class=\"section-header\"><span class=\"icon\">&#x1F4DC;</span> Compiled
Assembly</div>
01478     <div class=\"section-content\">
01479         <textarea id=\"asm-main\" readonly
style=\"width:100%;height:300px;background:#222;color:#e0e0e0;border-radius:8px;padding:16px;font-family:'Fira

```

```

Mono', 'Consolas', 'Courier New', monospace;font-size:1rem;">");
01480     out « html_escape(program->assembly_code);
01481     out « R"(</textarea>
01482             <div style="margin-top:10px;">");
01483     out « "<button class=\"btn\" onclick=\"copyAsm('asm-main')\">Copy</button>";
01484     out « "<button class=\"btn\" onclick=\"downloadAsm('asm-main', ' " «
js_escape(program->name) « ".s')\">Download</button>";
01485     out « R"(</div>
01486             </div>
01487             </div>
01488             );";
01489 }
01490
01491 if (hasObjects) {
01492     out « R"(<div class="section">
01493             <div class="section-header">
01494                 <span class="icon">&#x1F4E6;</span>
01495                 Objects
01496             </div>
01497             <div class="section-content">");
01498
01499     for (const auto &obj : program->objects) {
01500         if (obj) {
01501             printObjectHTML(out, obj);
01502         }
01503     }
01504
01505     out « R"(</div>
01506             </div>");
01507 }
01508
01509 out « R"(<
01510             </div>");
01511 out « R"(<script>
01512     function copyAsm(id) {
01513         var textarea = document.getElementById(id);
01514         textarea.select();
01515         navigator.clipboard.writeText(textarea.value);
01516     }
01517     function downloadAsm(id, filename) {
01518         var textarea = document.getElementById(id);
01519         var blob = new Blob([textarea.value], {type: 'text/plain'});
01520         var link = document.createElement('a');
01521         link.href = window.URL.createObjectURL(blob);
01522         link.download = filename;
01523         document.body.appendChild(link);
01524         link.click();
01525         document.body.removeChild(link);
01526     }
01527 </script>
01528 </body>
01529 </html>
01530 </script>
01531 </script>
01532 </script>
01533 return true;
01534 }
01535
01536 void Parser::processDataSection(SectionNode *sectionNode, std::unique_ptr<Program> &program) {
01537     for (const auto &statement : sectionNode->statements) {
01538         auto variableNode = dynamic_cast<VariableNode *>(statement.get());
01539         if (variableNode) {
01540             Variable var;
01541             var.type = variableNode->type;
01542             var.var_name = program->name + "." + variableNode->name;
01543             if (variableNode->hasInitializer) {
01544                 setVariableValue(var, variableNode->type, variableNode->initialValue);
01545             } else {
01546                 setDefaultVariableValue(var, variableNode->type);
01547             }
01548             var.var_value.buffer_size = variableNode->buffer_size;
01549             var.is_global = variableNode->is_global;
01550             var.obj_name = program->name;
01551             program->add_variable(var.var_name, var);
01552         }
01553     }
01554 }
01555
01556 void Parser::processModuleSection(SectionNode *sectionNode, std::unique_ptr<Program> &program) {
01557     for (const auto &statement : sectionNode->statements) {
01558         auto moduleNode = dynamic_cast<ModuleNode *>(statement.get());
01559         if (moduleNode) {
01560             std::string &name = moduleNode->name;
01561             processModuleFile(name, program);
01562         }
01563     }
01564 }

```

```

01565     void Parser::processModuleFile(const std::string &src, std::unique_ptr<Program> &program) {
01566         std::string module_name = "libmxvm_" + src;
01567         if (!module_path.ends_with("/"))
01568             module_path += "/";
01569         if (!include_path.ends_with("/"))
01570             include_path += "/";
01571         std::string shared_ext = ".so";
01572 #ifdef _WIN32
01573         shared_ext = ".dll";
01574 #elif defined(__APPLE__)
01575         shared_ext = ".dylib";
01576 #endif
01577
01578         std::string module_path_so = module_path + "modules/" + src + "/" + module_name + shared_ext;
01579 #ifdef _WIN32
01580         // On Windows, DLLs may not have the "lib" prefix (e.g. mxvm_sdl.dll vs libmxvm_sdl.dll)
01581         if (!std::filesystem::exists(module_path_so)) {
01582             std::string alt = module_path + "modules/" + src + "/mxvm_" + src + shared_ext;
01583             if (std::filesystem::exists(alt))
01584                 module_path_so = alt;
01585         }
01586 #endif
01587         std::string msys2prefix;
01588 #ifdef _WIN32
01589         const char *msys2env = std::getenv("MSYSTEM_PREFIX");
01590         if (msys2env != nullptr) {
01591             msys2prefix = msys2env;
01592             size_t pos = msys2prefix.find("/msys64/");
01593             if (pos != std::string::npos) {
01594                 msys2prefix = msys2prefix.substr(0, pos + 8);
01595             } else {
01596                 msys2prefix = "";
01597             }
01598         }
01599 #endif
01600         std::string module_src = msys2prefix + include_path + src + "/" + src + ".mxvm";
01601         std::fstream file;
01602         file.open(module_src, std::ios::in);
01603         if (!file.is_open()) {
01604             // Fall back to looking in the module directory
01605             module_src = module_path + "modules/" + src + "/" + src + ".mxvm";
01606             file.open(module_src, std::ios::in);
01607         }
01608         if (!file.is_open()) {
01609             throw mx::Exception("Error could not find module source file: " + module_src);
01610         }
01611         std::ostringstream data;
01612         data << file.rdbuf();
01613         ModuleParser mod_parser(this->parser_mode, src, data.str());
01614         if (mod_parser.scan() > 0) {
01615             if (mod_parser.parse() && mod_parser.generateProgramCode(this->parser_mode, src,
01616 module_path_so, program)) {
01617                 return;
01618             } else {
01619                 throw mx::Exception("Error parsing module file.\n");
01620             }
01621         }
01622
01623     void Parser::processCodeSection(SectionNode *sectionNode, std::unique_ptr<Program> &program) {
01624         std::unordered_map<std::string, size_t> labelMap;
01625
01626         size_t instructionIndex = 0;
01627         for (const auto &statement : sectionNode->statements) {
01628             if (auto labelNode = dynamic_cast<LabelNode *>(statement.get())) {
01629                 labelMap[labelNode->name] = instructionIndex;
01630                 program->add_label(labelNode->name, instructionIndex, labelNode->function);
01631             } else if (dynamic_cast<InstructionNode *>(statement.get())) {
01632                 instructionIndex++;
01633             }
01634         }
01635         for (const auto &statement : sectionNode->statements) {
01636             auto instructionNode = dynamic_cast<InstructionNode *>(statement.get());
01637             if (instructionNode) {
01638                 Instruction instr;
01639                 instr.instruction = instructionNode->instruction;
01640                 if (instructionNode->operands.size() > 0) {
01641                     instr.op1 = instructionNode->operands[0];
01642                     resolveLabelReference(instr.op1, labelMap);
01643                 }
01644                 if (instructionNode->operands.size() > 1) {
01645                     instr.op2 = instructionNode->operands[1];
01646                     resolveLabelReference(instr.op2, labelMap);
01647                 }
01648                 if (instructionNode->operands.size() > 2) {
01649                     instr.op3 = instructionNode->operands[2];
01650                     resolveLabelReference(instr.op3, labelMap);

```

```

01651         }
01652         if (instructionNode->operands.size() > 3) {
01653             for (size_t i = 3; i < instructionNode->operands.size(); ++i) {
01654                 Operand extraOp = instructionNode->operands[i];
01655                 resolveLabelReference(extraOp, labelMap);
01656                 instr.vop.push_back(extraOp);
01657             }
01658         }
01659         program->add_instruction(instr);
01660     }
01661 }
01662 }
01663
01664 void Parser::setVariableValue(Variable &var, VarType type, const std::string &value, size_t
buf_size) {
01665     switch (type) {
01666     case VarType::VAR_BYTE:
01667     case VarType::VAR_INTEGER:
01668         if (value.starts_with("0x") || value.starts_with("0X")) {
01669             var.var_value.int_value = std::stoull(value, nullptr, 16);
01670         } else {
01671             var.var_value.int_value = std::stoull(value);
01672         }
01673         var.var_value.type = type;
01674         break;
01675
01676     case VarType::VAR_FLOAT:
01677         var.var_value.float_value = std::stod(value);
01678         var.var_value.type = VarType::VAR_FLOAT;
01679         break;
01680
01681     case VarType::VAR_STRING:
01682         var.var_value.str_value = value;
01683         var.var_value.type = VarType::VAR_STRING;
01684         var.var_value.buffer_size = buf_size;
01685         break;
01686
01687     case VarType::VAR_POINTER:
01688         var.var_value.ptr_value = nullptr;
01689         if (value == "null" || value == "0") {
01690             var.var_value.ptr_value = nullptr;
01691             var.var_value.str_value = "null";
01692         }
01693         var.var_value.type = VarType::VAR_POINTER;
01694         break;
01695
01696     case VarType::VAR_LABEL:
01697         var.var_value.label_value = value;
01698         var.var_value.type = VarType::VAR_LABEL;
01699         break;
01700
01701     default:
01702         setDefaultVariableValue(var, type);
01703         break;
01704     }
01705 }
01706
01707 void Parser::setDefaultVariableValue(Variable &var, VarType type) {
01708     switch (type) {
01709     case VarType::VAR_INTEGER:
01710         var.var_value.int_value = 0;
01711         var.var_value.type = VarType::VAR_INTEGER;
01712         break;
01713
01714     case VarType::VAR_FLOAT:
01715         var.var_value.float_value = 0.0;
01716         var.var_value.type = VarType::VAR_FLOAT;
01717         break;
01718
01719     case VarType::VAR_STRING:
01720         var.var_value.str_value = "";
01721         var.var_value.type = VarType::VAR_STRING;
01722         break;
01723
01724     case VarType::VAR_POINTER:
01725         var.var_value.ptr_value = nullptr;
01726         var.var_value.type = VarType::VAR_POINTER;
01727         break;
01728
01729     case VarType::VAR_LABEL:
01730         var.var_value.label_value = "";
01731         var.var_value.type = VarType::VAR_LABEL;
01732         break;
01733
01734     default:
01735         var.var_value.type = VarType::VAR_NULL;
01736         break;
01737     }
01738 }

```



```

01737     }
01738
01739     void Parser::resolveLabelReference(Operand &operand, const std::unordered_map<std::string, size_t>
&labelMap) {
01740         if (!operand.label.empty()) {
01741             auto it = labelMap.find(operand.label);
01742             if (it != labelMap.end()) {
01743                 operand.op_value = static_cast<int>(it->second);
01744             }
01745         }
01746     }
01747
01748     std::unique_ptr<ProgramNode> Parser::parseProgramOrObject(uint64_t &i) {
01749         auto program = std::make_unique<ProgramNode>();
01750
01751         std::string tokenValue = this->operator[](i).getTokenValue();
01752         program->object = (tokenValue == "object");
01753         i++;
01754
01755         if (i < scanner.size()) {
01756             auto nameToken = this->operator[](i);
01757             program->name = nameToken.getTokenValue();
01758             if (program->object == false)
01759                 program->root_name = nameToken.getTokenValue();
01760             i++;
01761         }
01762
01763         if (i < scanner.size() && this->operator[](i).getTokenValue() == "{") {
01764             i++;
01765
01766             while (i < scanner.size()) {
01767                 auto innerToken = this->operator[](i);
01768                 std::string innerValue = innerToken.getTokenValue();
01769
01770                 if (innerValue == "}") {
01771                     i++;
01772                     break;
01773                 }
01774
01775                 if (innerValue == "section") {
01776                     auto section = parseSection(i);
01777                     if (section) {
01778                         program->addSection(std::move(section));
01779                     }
01780                     continue;
01781                 }
01782                 i++;
01783             }
01784         }
01785
01786         return program;
01787     }
01788
01789     void Parser::generateObjectAssemblyFile(std::unique_ptr<Program> &objProgram) {
01790         std::string objectFileName = objProgram->name + ".s";
01791         std::ofstream objectFile(objectFileName);
01792         if (!objectFile.is_open()) {
01793             throw mx::Exception("Could not create object assembly file: " + objectFileName);
01794         }
01795
01796         std::ostringstream code_v;
01797         objProgram->generateCode(platform, objProgram->object, code_v);
01798         objProgram->assembly_code = code_v.str();
01799         std::string opt_code = objProgram->gen_optimize(objProgram->assembly_code, platform);
01800         objProgram->assembly_code = opt_code;
01801         objectFile << opt_code;
01802         objectFile.close();
01803         if (debug_mode) {
01804             std::cout << "Generated object assembly file: " << objectFileName << std::endl;
01805         }
01806         if (Program::base != nullptr)
01807             Program::base->add_filename(objectFileName);
01808     }
01809
01810     void Parser::registerObjectExterns(std::unique_ptr<Program> &mainProgram,
const std::unique_ptr<Program> &objProgram) {
01811         for (const auto &[labelName, labelInfo] : objProgram->labels) {
01812             if (labelInfo.second && Program::base != nullptr)
01813                 Program::base->add_extern(objProgram->name, labelName, false);
01814         }
01815
01816         for (const auto &ext : objProgram->external) {
01817             if (Program::base != nullptr)
01818                 Program::base->add_extern(ext.mod, ext.name, true);
01819         }
01820
01821         for (const auto &child : objProgram->objects) {

```

```

01823         if (child)
01824             registerObjectExterns(mainProgram, child);
01825     }
01826 }
01827 } // namespace mxvm

```

12.82 src/frontend/validator.cpp File Reference

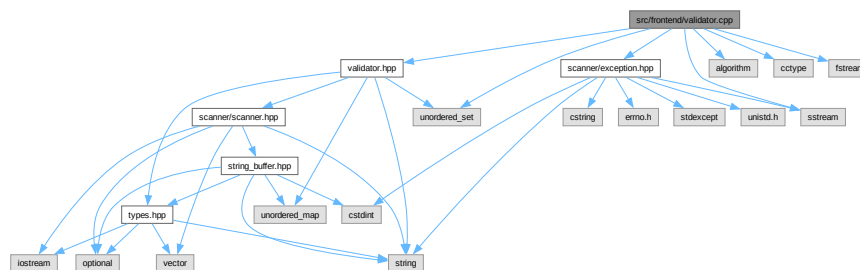
Pascal semantic validator — scope tracking, type checking, and declaration verification.

```

#include "validator.hpp"
#include "scanner/exception.hpp"
#include <algorithm>
#include <cctype>
#include <fstream>
#include <sstream>
#include <unordered_set>

```

Include dependency graph for validator.cpp:



Namespaces

- namespace [mxv](#)

Functions

- static bool [mxv::ciEquals](#) (const std::string &a, const std::string &b)
Case-insensitive string equality comparison.

Variables

- static const std::unordered_set< std::string > [mxv::pascal_keywords](#)

12.82.1 Detailed Description

Pascal semantic validator — scope tracking, type checking, and declaration verification.

Author

Jared Bruni

Definition in file [validator.cpp](#).

12.83 validator.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "validator.hpp"
00007 #include "scanner/exception.hpp"
00008 #include <algorithm>

```

```

00009 #include <cctype>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <unordered_set>
00013
00014 namespace mxx {
00015
00016     TPValidator::TPValidator(const std::string &source_) : scanner(source_), source(source_) {}
00017
00018     bool TPValidator::validate(const std::string &name) {
00019         filename = name;
00020         scanner.scan();
00021
00022         { // strip { } comments
00023             auto &toks = scanner.getTokens();
00024             for (size_t i = 0; i < toks.size(); i++) {
00025                 if (toks[i].getTokenValue() == "{") {
00026                     size_t start = i;
00027                     ++i;
00028                     while (i < toks.size() && toks[i].getTokenValue() != "}") {
00029                         ++i;
00030                     }
00031                     if (i < toks.size()) {
00032                         ++i;
00033                     }
00034                     toks.erase(toks.begin() + static_cast<int64_t>(start),
00035                               toks.begin() + static_cast<int64_t>(i));
00036                     i = start;
00037                 } else {
00038                     ++i;
00039                 }
00040             }
00041         }
00042
00043         index = 0;
00044         token = nullptr;
00045         scopeStack.clear();
00046         declaredProcs.clear();
00047         declaredFuncs.clear();
00048         importedUnits.clear();
00049         pushScope();
00050         next();
00051         if (isKW("unit"))
00052             parseUnit();
00053         else
00054             parseProgram();
00055         if (token)
00056             failHere("Unexpected tokens after end of program");
00057         popScope();
00058         return true;
00059     }
00060
00061     static bool ciEquals(const std::string &a, const std::string &b) {
00062         if (a.size() != b.size()) return false;
00063         for (size_t i = 0; i < a.size(); ++i)
00064             if (std::tolower(static_cast<unsigned char>(a[i])) !=
00065                 std::tolower(static_cast<unsigned char>(b[i])))
00066                 return false;
00067         return true;
00068     }
00069
00070     bool TPValidator::match(const std::string &s) const {
00071         return token && ciEquals(token->getTokenValue(), s);
00072     }
00073
00074     bool TPValidator::match(types::TokenType t) const {
00075         return token && token->getTokenType() == t;
00076     }
00077
00078     bool TPValidator::next() {
00079         while (index < scanner.size() &&
00080               scanner[index].getTokenType() == types::TokenType::TT_SYM &&
00081               scanner[index].getTokenValue() == "\n") {
00082             ++index;
00083         }
00084         if (index < scanner.size()) {
00085             token = &scanner[index++];
00086             return true;
00087         }
00088         token = nullptr;
00089         return false;
00090     }
00091
00092     bool TPValidator::peekIs(const std::string &s) {
00093         return index < scanner.size() && ciEquals(scanner[index].getTokenValue(), s);
00094     }
00095
00096     }
00097
00098     bool TPValidator::peekIs(const std::string &s) {
00099         return index < scanner.size() && ciEquals(scanner[index].getTokenValue(), s);
00100     }
00101

```

```

00102     void TPValidator::require(const std::string &s) {
00103         if (!match(s)) {
00104             if (token)
00105                 failAt(token, "Required: " + s + " Found: " + found());
00106             fail("Required: " + s + " Found: EOF");
00107         }
00108     }
00109
00110     void TPValidator::require(types::TokenType t) {
00111         if (!match(t)) {
00112             if (token)
00113                 failAt(token, "Required: " + tokenTypeToString(t) + " Found: " + found());
00114             fail("Required: " + tokenTypeToString(t) + " Found: EOF");
00115         }
00116     }
00117
00118     void TPValidator::requireKW(const std::string &k) {
00119         if (!isKW(k)) {
00120             if (token)
00121                 failAt(token, "Required: " + k + " Found: " + found());
00122             fail("Required: " + k + " Found: EOF");
00123         }
00124     }
00125
00126     void TPValidator::fail(const std::string &msg) {
00127         throw mx::Exception("Syntax Error in '" + filename + "': " + msg);
00128     }
00129
00130     void TPValidator::failHere(const std::string &msg) {
00131         if (token)
00132             throw mx::Exception("Syntax Error in '" + filename + "': " + msg + " at line " +
std::to_string(token->getLine()));
00133         throw mx::Exception("Syntax Error in '" + filename + "': " + msg);
00134     }
00135
00136     void TPValidator::failAt(const scan::TToken *at, const std::string &msg) {
00137         if (at)
00138             throw mx::Exception("Syntax Error in '" + filename + "': " + msg + " at line " +
std::to_string(at->getLine()));
00139         throw mx::Exception("Syntax Error in '" + filename + "': " + msg);
00140     }
00141
00142     std::string TPValidator::tokenTypeToString(types::TokenType t) {
00143         switch (t) {
00144             case types::TokenType::TT_ID:
00145                 return "Identifier";
00146             case types::TokenType::TT_NUM:
00147                 return "Number";
00148             case types::TokenType::TT_HEX:
00149                 return "Hex";
00150             case types::TokenType::TT_STR:
00151                 return "String";
00152             case types::TokenType::TT_SYM:
00153                 return "Symbol";
00154             default:
00155                 return "Unknown";
00156         }
00157     }
00158
00159     std::string TPValidator::lower(std::string s) {
00160         for (auto &c : s)
00161             c = (char)std::tolower((unsigned char)c);
00162         return s;
00163     }
00164
00165     std::string TPValidator::found() const {
00166         if (!token)
00167             return "EOF";
00168         return token->getTokenValue() + ":" + tokenTypeToString(token->getTokenType());
00169     }
00170
00171     bool TPValidator::isKW(const std::string &k) const {
00172         if (!token || !isPascalKeyword(k) || token->getTokenType() != types::TokenType::TT_ID)
00173             return false;
00174         return lower(token->getTokenValue()) == k;
00175     }
00176
00177     void TPValidator::pushScope() {
00178         scopeStack.emplace_back();
00179     }
00180
00181     void TPValidator::popScope() {
00182         if (!scopeStack.empty()) {
00183             scopeStack.pop_back();
00184         }
00185     }
00186

```

```

00187 void TPValidator::declareConst(const std::string &name, const scan::TToken *at) {
00188     auto key = lower(name);
00189     if (scopeStack.back().consts.count(key)) {
00190         failAt(at, "Redeclaration of constant '" + name + "' in the same scope");
00191     }
00192     scopeStack.back().consts.insert(key);
00193 }
00194
00195 bool TPValidator::isBuiltinConst(const std::string &name) const {
00196     std::string key = lower(name);
00197     return key == "true" || key == "false" || key == "nil";
00198 }
00199
00200 void TPValidator::checkVar(const std::string &name, const scan::TToken *at) {
00201     auto key = lower(name);
00202
00203     if (key == "result" && inFunctionDepth > 0)
00204         return;
00205
00206     for (auto it = scopeStack.rbegin(); it != scopeStack.rend(); ++it) {
00207         if (it->vars.count(key))
00208             return;
00209     }
00210
00211     if (withDepth > 0)
00212         return;
00213
00214     if (!declaredFuncs.count(key) && !declaredProcs.count(key) && !isBuiltinConst(name) &&
00215         !isPascalKeyword(key) && !importedUnits.count(key)) {
00216         failAt(at, "Use of undeclared identifier '" + name + "'");
00217     }
00218 }
00219
00220 void TPValidator::checkVarOrConst(const std::string &name, const scan::TToken *at) {
00221     auto key = lower(name);
00222
00223     if (key == "result" && inFunctionDepth > 0)
00224         return;
00225
00226     for (auto it = scopeStack.rbegin(); it != scopeStack.rend(); ++it) {
00227         if (it->vars.count(key) || it->consts.count(key))
00228             return;
00229     }
00230
00231     if (withDepth > 0)
00232         return;
00233
00234     if (!declaredFuncs.count(key) && !declaredProcs.count(key) && !isBuiltinConst(name) &&
00235         !isPascalKeyword(key) && !importedUnits.count(key)) {
00236         failAt(at, "Use of undeclared identifier '" + name + "'");
00237     }
00238 }
00239
00240 void TPValidator::checkConstOnly(const std::string &name, const scan::TToken *at) {
00241     auto key = lower(name);
00242     for (auto it = scopeStack.rbegin(); it != scopeStack.rend(); ++it) {
00243         if (it->consts.count(key))
00244             return;
00245     }
00246     if (!isBuiltinConst(name)) {
00247         failAt(at, "Constant expression requires constant '" + name + "'");
00248     }
00249 }
00250
00251 void TPValidator::parseProgram() {
00252     requireKW("program");
00253     next();
00254     require(types::TokenType::TT_ID);
00255     next();
00256     require(";");
00257     next();
00258     if (isKW("uses"))
00259         parseUses();
00260     parseBlock();
00261     require(".");
00262     next();
00263 }
00264
00265 void TPValidator::parseUnit() {
00266     requireKW("unit");
00267     next();
00268     require(types::TokenType::TT_ID);
00269     next();
00270     require(";");
00271     next();
00272     // interface section

```

```

00272         requireKW("interface");
00273         next();
00274         if (isKW("uses"))
00275             parseUses();
00276         parseInterfaceSection();
00277
00278         // implementation section
00279         requireKW("implementation");
00280         next();
00281         if (isKW("uses"))
00282             parseUses();
00283
00284         // parse implementation declarations (const, type, var, procedure, function)
00285         while (isKW("const") || isKW("type") || isKW("var") || isKW("procedure") || isKW("function"))
00286         {
00287             if (isKW("const"))
00288                 parseConstSection();
00289             else if (isKW("type"))
00290                 parseTypeSection();
00291             else if (isKW("var"))
00292                 parseVarSection();
00293             else if (isKW("procedure") || isKW("function"))
00294                 parseSubprogram();
00295         }
00296         requireKW("end");
00297         next();
00298         require(".");
00299         next();
00300     }
00301
00302 void TPValidator::parseInterfaceSection() {
00303     while (isKW("procedure") || isKW("function") || isKW("type") || isKW("const") || isKW("var"))
00304     {
00305         if (isKW("procedure")) {
00306             next();
00307             require(types::TokenType::TT_ID);
00308             declaredProcs.insert(lower(token->getTokenValue()));
00309             next();
00310             pushScope();
00311             if (match("("))
00312                 parseFormalParams();
00313             require(";");
00314             next();
00315             popScope();
00316         } else if (isKW("function")) {
00317             next();
00318             require(types::TokenType::TT_ID);
00319             declaredFuncs.insert(lower(token->getTokenValue()));
00320             next();
00321             pushScope();
00322             if (match("("))
00323                 parseFormalParams();
00324             require(":");
00325             next();
00326             parseType("");
00327             require(";");
00328             next();
00329             popScope();
00330         } else if (isKW("type")) {
00331             parseTypeSection();
00332         } else if (isKW("const")) {
00333             parseConstSection();
00334         } else if (isKW("var")) {
00335             parseVarSection();
00336         }
00337     }
00338 }
00339
00339 void TPValidator::parseUses() {
00340     requireKW("uses");
00341     next();
00342     require(types::TokenType::TT_ID);
00343     std::vector<std::string> usedUnits;
00344     usedUnits.push_back(token->getTokenValue());
00345     next();
00346     while (match(", ")) {
00347         next();
00348         require(types::TokenType::TT_ID);
00349         usedUnits.push_back(token->getTokenValue());
00350         next();
00351     }
00352     require(";");
00353     next();
00354
00355     // Import interface declarations from used units
00356     static const std::unordered_set<std::string> nativeModules = {"io", "std", "string", "sdl",

```

```

"strlib");
00357     std::string inputDir;
00358     {
00359         auto pos = filename.find_last_of("/\\");
00360         inputDir = (pos != std::string::npos) ? filename.substr(0, pos + 1) : "./";
00361     }
00362     for (const auto &unit : usedUnits) {
00363         importedUnits.insert(lower(unit));
00364         if (nativeModules.count(unit))
00365             continue;
00366         // Try to find the unit's .pas file
00367         std::string unitFile = inputDir + unit + ".pas";
00368         std::ifstream uf(unitFile);
00369         if (!uf.is_open()) {
00370             std::string lowerUnit = unit;
00371             std::transform(lowerUnit.begin(), lowerUnit.end(), lowerUnit.begin(), ::tolower);
00372             unitFile = inputDir + lowerUnit + ".pas";
00373             uf.open(unitFile);
00374         }
00375         if (!uf.is_open())
00376             continue;
00377         std::ostringstream buf;
00378         buf << uf.rdbuf();
00379         uf.close();
00380         try {
00381             // Use a scanner to extract interface declarations
00382             scan::Scanner unitScanner(buf.str());
00383             unitScanner.scan();
00384             auto &toks = unitScanner.getTokens();
00385             // Strip brace comments
00386             for (size_t i = 0; i < toks.size(); i++) {
00387                 if (toks[i].getTokenValue() == "{") {
00388                     size_t start = i;
00389                     ++i;
00390                     while (i < toks.size() && toks[i].getTokenValue() != "}") ++i;
00391                     if (i < toks.size()) ++i;
00392                     toks.erase(toks.begin() + static_cast<int64_t>(start),
00393                               toks.begin() + static_cast<int64_t>(i));
00394                     i = start;
00395                 } else {
00396                     ++i;
00397                 }
00398             }
00399             // Find 'interface' keyword, then collect procedure/function names until
'implementation'
00400             size_t ti = 0;
00401             auto tlower = [] (const std::string &s) {
00402                 std::string r = s;
00403                 std::transform(r.begin(), r.end(), r.begin(), ::tolower);
00404                 return r;
00405             };
00406             // Helper to skip newline tokens in the raw token stream
00407             auto skipNL = [&]() {
00408                 while (ti < toks.size() && toks[ti].getTokenValue() == "\n") ++ti;
00409             };
00410             // Skip to 'interface'
00411             while (ti < toks.size() && tlower(toks[ti].getTokenValue()) != "interface") ++ti;
00412             if (ti < toks.size()) ++ti; // move past 'interface'
00413             skipNL();
00414             // Skip optional uses in interface
00415             if (ti < toks.size() && tlower(toks[ti].getTokenValue()) == "uses") {
00416                 while (ti < toks.size() && toks[ti].getTokenValue() != ";") ++ti;
00417                 if (ti < toks.size()) ++ti;
00418                 skipNL();
00419             }
00420             // Collect procedure/function/const/var/type names until 'implementation'
00421             while (ti < toks.size() && tlower(toks[ti].getTokenValue()) != "implementation") {
00422                 skipNL();
00423                 if (ti >= toks.size() || tlower(toks[ti].getTokenValue()) == "implementation")
00424                     break;
00425                 std::string kw = tlower(toks[ti].getTokenValue());
00426                 if (kw == "procedure" && ti + 1 < toks.size()) {
00427                     ++ti; skipNL();
00428                     declaredProcs.insert(lower(toks[ti].getTokenValue()));
00429                     while (ti < toks.size() && toks[ti].getTokenValue() != ";") ++ti;
00430                     if (ti < toks.size()) ++ti;
00431                     skipNL();
00432                 } else if (kw == "function" && ti + 1 < toks.size()) {
00433                     ++ti; skipNL();
00434                     declaredFuncs.insert(lower(toks[ti].getTokenValue()));
00435                     while (ti < toks.size() && toks[ti].getTokenValue() != ";") ++ti;
00436                     if (ti < toks.size()) ++ti;
00437                     skipNL();
00438                 } else if (kw == "const") {
00439                     ++ti; skipNL(); // skip 'const'
00440                     // Collect constant names: name = expr ;

```

```

00441         while (ti < toks.size() && tlower(toks[ti].getTokenValue()) !=
"implementation"
00442             && toks[ti].getTokenType() == types::TokenType::TT_ID
00443             && tlower(toks[ti].getTokenValue()) != "var"
00444             && tlower(toks[ti].getTokenValue()) != "type"
00445             && tlower(toks[ti].getTokenValue()) != "procedure"
00446             && tlower(toks[ti].getTokenValue()) != "function") {
00447             scopeStack.back().consts.insert(lower(toks[ti].getTokenValue()));
00448             // Skip past = expr ;
00449             while (ti < toks.size() && toks[ti].getTokenValue() != ";") ++ti;
00450             if (ti < toks.size()) ++ti;
00451             skipNL();
00452         }
00453     } else if (kw == "var") {
00454         ++ti; skipNL(); // skip 'var'
00455         // Collect variable names: name [, name]* : type ;
00456         while (ti < toks.size() && tlower(toks[ti].getTokenValue()) !=
"implementation"
00457             && toks[ti].getTokenType() == types::TokenType::TT_ID
00458             && tlower(toks[ti].getTokenValue()) != "const"
00459             && tlower(toks[ti].getTokenValue()) != "type"
00460             && tlower(toks[ti].getTokenValue()) != "procedure"
00461             && tlower(toks[ti].getTokenValue()) != "function") {
00462             scopeStack.back().vars.insert(lower(toks[ti].getTokenValue()));
00463             ++ti;
00464             while (ti < toks.size() && toks[ti].getTokenValue() == ",") {
00465                 ++ti; // skip ','
00466                 if (ti < toks.size()) {
00467                     scopeStack.back().vars.insert(lower(toks[ti].getTokenValue()));
00468                     ++ti;
00469                 }
00470             }
00471             // Skip past : type ;
00472             while (ti < toks.size() && toks[ti].getTokenValue() != ";") ++ti;
00473             if (ti < toks.size()) ++ti;
00474             skipNL();
00475         }
00476     } else if (kw == "type") {
00477         ++ti; skipNL(); // skip 'type'
00478         // Collect type names: name = typedecl ;
00479         while (ti < toks.size() && tlower(toks[ti].getTokenValue()) !=
"implementation"
00480             && toks[ti].getTokenType() == types::TokenType::TT_ID
00481             && tlower(toks[ti].getTokenValue()) != "const"
00482             && tlower(toks[ti].getTokenValue()) != "var"
00483             && tlower(toks[ti].getTokenValue()) != "procedure"
00484             && tlower(toks[ti].getTokenValue()) != "function") {
00485             scopeStack.back().types.insert(lower(toks[ti].getTokenValue()));
00486             // Skip past = typedecl ;
00487             while (ti < toks.size() && toks[ti].getTokenValue() != ";") ++ti;
00488             if (ti < toks.size()) ++ti;
00489             skipNL();
00490         }
00491     } else {
00492         ++ti; skipNL();
00493     }
00494 }
00495 } catch (...) {
00496     // Unit parsing failed; skip importing
00497 }
00498 }
00499 }
00500
00501 void TPValidator::parseBlock() {
00502     if (isKW("label"))
00503         parseLabelSection();
00504     while (isKW("const") || isKW("type") || isKW("var") || isKW("procedure") || isKW("function")
|| isKW("label")) {
00505         if (isKW("label"))
00506             parseLabelSection();
00507         else if (isKW("const"))
00508             parseConstSection();
00509         else if (isKW("type"))
00510             parseTypeSection();
00511         else if (isKW("var"))
00512             parseVarSection();
00513         else if (isKW("procedure") || isKW("function"))
00514             parseSubprogram();
00515     }
00516     parseCompoundStatement();
00517 }
00518
00519 void TPValidator::parseLabelSection() {
00520     requireKW("label");
00521     next();
00522     require(types::TokenType::TT_NUM);
00523     next();

```



```

00524         while (match(",")) {
00525             next();
00526             require(types::TokenType::TT_NUM);
00527             next();
00528         }
00529         require(";");
00530         next();
00531     }
00532
00533 void TPValidator::parseConstSection() {
00534     requireKW("const");
00535     next();
00536     while (match(types::TokenType::TT_ID)) {
00537         if (isKW("type") || isKW("var") || isKW("label") ||
00538             isKW("begin") || isKW("procedure") || isKW("function") ||
00539             isKW("implementation") || isKW("end") || isKW("interface")) {
00540             break;
00541         }
00542         const auto *nameTok = token;
00543         std::string name = token->getTokenValue();
00544         next();
00545         require("=");
00546         next();
00547         parseConstExpr({";", true);
00548         declareConst(name, nameTok);
00549         require(";");
00550         next();
00551     }
00552 }
00553
00554 void TPValidator::parseTypeSection() {
00555     requireKW("type");
00556     next();
00557
00558     while (match(types::TokenType::TT_ID)) {
00559         if (isKW("procedure") || isKW("function") || isKW("var") || isKW("begin") || isKW("const")
00560 ||
00561             isKW("implementation") || isKW("end") || isKW("interface")) {
00562             break;
00563         }
00564         const auto *nameTok = token;
00565         std::string name = token->getTokenValue();
00566         next();
00567         require("=");
00568         next();
00569
00570         parseType(name);
00571
00572         declareType(name, nameTok);
00573         require(";");
00574         next();
00575     }
00576 }
00577
00578 void TPValidator::parseVarSection() {
00579     requireKW("var");
00580     next();
00581     while (match(types::TokenType::TT_ID)) {
00582         if (isKW("procedure") || isKW("function") || isKW("begin") ||
00583             isKW("type") || isKW("const") || isKW("label") ||
00584             isKW("implementation") || isKW("end") || isKW("interface")) {
00585             break;
00586         }
00587         declareVar(token->getTokenValue(), token);
00588         next();
00589         while (match(",")) {
00590             next();
00591             require(types::TokenType::TT_ID);
00592             declareVar(token->getTokenValue(), token);
00593             next();
00594         }
00595         require(":");
00596         next();
00597         parseType("");
00598         if (match("=")) {
00599             next();
00600             parseConstExpr({";", false);
00601         }
00602         require(";");
00603         next();
00604     }
00605 }
00606 void TPValidator::parseSubprogram() {
00607     bool isProc = isKW("procedure");
00608     if (isProc) {
00609         requireKW("procedure");

```

```

00610         next();
00611         require(types::TokenType::TT_ID);
00612         declaredProcs.insert(lower(token->getTokenValue()));
00613         next();
00614     } else {
00615         requireKW("function");
00616         next();
00617         require(types::TokenType::TT_ID);
00618         declaredFuncs.insert(lower(token->getTokenValue()));
00619         next();
00620     }
00621
00622     pushScope();
00623
00624     if (match("("))
00625         parseFormalParams();
00626
00627     if (isProc) {
00628         require(";");
00629         next();
00630     } else {
00631         require(":");
00632         next();
00633         parseType("");
00634         require(";");
00635         next();
00636     }
00637
00638     if (isKW("forward")) {
00639         next();
00640         require(";");
00641         next();
00642         popScope();
00643         return;
00644     }
00645
00646     if (!isProc)
00647         ++inFunctionDepth;
00648     parseBlock();
00649     if (!isProc)
00650         --inFunctionDepth;
00651     require(";");
00652     next();
00653     popScope();
00654 }
00655
00656 void TPValidator::parseFormalParams() {
00657     require("(");
00658     next();
00659     if (match("(")) {
00660         next();
00661         return;
00662     }
00663     while (true) {
00664         bool byRef = isKW("var");
00665         if (byRef)
00666             next();
00667         parseIdentList();
00668         require(":");
00669         next();
00670         parseType("");
00671         if (match(";")) {
00672             next();
00673             continue;
00674         }
00675         require(")");
00676         next();
00677         break;
00678     }
00679 }
00680
00681 void TPValidator::parseIdentList() {
00682     require(types::TokenType::TT_ID);
00683     declareVar(token->getTokenValue(), token);
00684     next();
00685     while (match(",")) {
00686         next();
00687         require(types::TokenType::TT_ID);
00688         declareVar(token->getTokenValue(), token);
00689         next();
00690     }
00691 }
00692
00693 void TPValidator::parseType(const std::string &typeName) {
00694     if (isKW("packed")) {
00695         next(); // consume 'packed' modifier (accepted but has no effect)
00696     }

```

```

00697         if (isKW("array")) {
00698             next();
00699             // Dynamic array: array of <type> (no bounds)
00700             if (isKW("of")) {
00701                 next();
00702                 parseType("");
00703                 return;
00704             }
00705             // Static array: array[lo..hi] of <type>
00706             require("[");
00707             next();
00708             parseSubrange();
00709             while (match(", ")) {
00710                 next();
00711                 parseSubrange();
00712             }
00713             require("]");
00714             next();
00715             requireKW("of");
00716             next();
00717             parseType("");
00718             return;
00719         }
00720         if (isKW("record")) {
00721             next();
00722             pushRecordFieldScope(typeName);
00723             while (!isKW("end") && !isKW("case")) {
00724                 parseFieldIdentList();
00725                 require(":");
00726                 next();
00727                 parseType("");
00728                 require(";");
00729                 next();
00730             }
00731             if (isKW("case")) {
00732                 next();
00733                 require(types::TokenType::TT_ID);
00734                 next();
00735                 require(":");
00736                 next();
00737                 parseType("");
00738                 requireKW("of");
00739                 next();
00740                 while (!isKW("end")) {
00741                     parseConstExpr({"":", ", "}, false);
00742                     while (match(", ")) {
00743                         next();
00744                         parseConstExpr({"":", ", "}, false);
00745                     }
00746                     require(":");
00747                     next();
00748                     require("(");
00749                     next();
00750                     while (!match(")")) {
00751                         require(types::TokenType::TT_ID);
00752                         next();
00753                         while (match(", ")) {
00754                             next();
00755                             require(types::TokenType::TT_ID);
00756                             next();
00757                         }
00758                         require(":");
00759                         next();
00760                         parseType("");
00761                         if (match(";"))
00762                             next();
00763                     }
00764                     require(")");
00765                     next();
00766                     if (match(";"))
00767                         next();
00768                 }
00769             }
00770             requireKW("end");
00771             next();
00772             popRecordFieldScope();
00773             return;
00774         }
00775         if (isKW("set")) {
00776             next();
00777             requireKW("of");
00778             next();
00779             parseTypeName();
00780             return;
00781         }
00782         if (isKW("file")) {
00783             next();

```

```

00784         if (isKW("of")) {
00785             next();
00786             parseTypeName();
00787         }
00788         return;
00789     }
00790     if (match("(")) {
00791         next();
00792         while (match(types::TokenType::TT_ID)) {
00793             declareConst(token->getTokenValue(), token);
00794             next();
00795             if (match(","))
00796                 next();
00797         }
00798         require(")");
00799         next();
00800         return;
00801     }
00802     if (match("^")) {
00803         next();
00804         parseTypeName();
00805         return;
00806     }
00807     if (isKW("string")) {
00808         next();
00809         if (match("[") {
00810             next();
00811             parseConstExpr({"["}, true);
00812             require("]");
00813             next();
00814         }
00815         return;
00816     }
00817     if (isBuiltinType()) {
00818         next();
00819         return;
00820     }
00821     parseTypeName();
00822 }
00823
00824 void TPValidator::parseTypeName() {
00825     require(types::TokenType::TT_ID);
00826     std::string name = token->getTokenValue();
00827     checkType(name, token);
00828     next();
00829 }
00830
00831 void TPValidator::parseSubrange() {
00832     parseConstExpr({"..", "]", ",", true);
00833     if (match("..")) {
00834         next();
00835         parseConstExpr({"]", ",", true);
00836     }
00837 }
00838
00839 void TPValidator::parseConstant() {
00840     if (match("-")) {
00841         next();
00842         parseConstSimple();
00843         return;
00844     }
00845     parseConstSimple();
00846 }
00847
00848 void TPValidator::parseConstSimple() {
00849     if (match(types::TokenType::TT_NUM) || match(types::TokenType::TT_HEX) ||
match(types::TokenType::TT_STR)) {
00850         next();
00851         return;
00852     }
00853     if (isKW("true") || isKW("false") || isKW("nil")) {
00854         next();
00855         return;
00856     }
00857     require(types::TokenType::TT_ID);
00858     checkConstOnly(token->getTokenValue(), token);
00859     next();
00860 }
00861
00862 void TPValidator::parseCompoundStatement() {
00863     requireKW("begin");
00864     next();
00865
00866     if (isKW("end")) {
00867         requireKW("end");
00868         next();
00869         return;

```

```

00870     }
00871
00872     while (!isKW("end")) {
00873         if (token == nullptr) {
00874             fail("Unexpected end of file inside compound statement");
00875         }
00876
00877         parseStatement();
00878
00879         if (match(";")) {
00880             next();
00881
00882             if (isKW("end")) {
00883                 break;
00884             }
00885         } else if (isKW("end")) {
00886             break;
00887         } else {
00888             failHere("Expected ';' or 'end'");
00889         }
00890     }
00891
00892     requireKW("end");
00893     next();
00894 }
00895
00896 void TPValidator::parseStatement() {
00897     if (!token)
00898         failHere("Unexpected EOF in statement");
00899     if (isKW("begin")) {
00900         parseCompoundStatement();
00901         return;
00902     }
00903     if (isKW("if")) {
00904         parseIf();
00905         return;
00906     }
00907     if (isKW("while")) {
00908         parseWhile();
00909         return;
00910     }
00911     if (isKW("repeat")) {
00912         parseRepeat();
00913         return;
00914     }
00915     if (isKW("for")) {
00916         parseFor();
00917         return;
00918     }
00919     if (isKW("case")) {
00920         parseCase();
00921         return;
00922     }
00923     if (isKW("with")) {
00924         parseWith();
00925         return;
00926     }
00927     if (isKW("goto")) {
00928         parseGoto();
00929         return;
00930     }
00931     if (match(types::TokenType::TT_NUM)) {
00932         // Label prefix: 100: statement
00933         next();
00934         if (match(":")) {
00935             next();
00936             parseStatement();
00937             return;
00938         }
00939         failHere("Statement cannot start with number (expected ':' for label)");
00940     }
00941     if (match(types::TokenType::TT_ID)) {
00942         parseSimpleOrCallOrAssign();
00943         return;
00944     }
00945     if (match(";") || match("")) {
00946         return;
00947     }
00948     if (isKW("end")) {
00949         return;
00950     }
00951     failHere("Invalid statement start");
00952 }
00953
00954 void TPValidator::parseIf() {
00955     requireKW("if");
00956     next();

```

```

00957         parseExprStop({"then"});
00958         requireKW("then");
00959         next();
00960         parseStatement();
00961         if (isKW("else")) {
00962             next();
00963             parseStatement();
00964         }
00965     }
00966
00967     void TPValidator::parseWhile() {
00968         requireKW("while");
00969         next();
00970         parseExprStop({"do"});
00971         requireKW("do");
00972         next();
00973         parseStatement();
00974     }
00975
00976     void TPValidator::parseRepeat() {
00977         requireKW("repeat");
00978         next();
00979         while (true) {
00980             if (isKW("until")) {
00981                 break;
00982             }
00983             parseStatement();
00984             if (match(";")) {
00985                 next();
00986                 if (isKW("until")) {
00987                     break;
00988                 }
00989             } else if (isKW("until")) {
00990                 break;
00991             } else {
00992                 failHere("Expected ';' or 'until'");
00993             }
00994         }
00995         requireKW("until");
00996         next();
00997         parseExprStop({";", "end"});
00998     }
00999
01000     void TPValidator::parseFor() {
01001         requireKW("for");
01002         next();
01003         require(types::TokenType::TT_ID);
01004         checkVar(token->getTokenValue(), token);
01005         next();
01006         require(":=");
01007         next();
01008         parseExprStop({"to", "downto"});
01009         if (isKW("to"))
01010             next();
01011         else {
01012             requireKW("downto");
01013             next();
01014         }
01015         parseExprStop({"do"});
01016         requireKW("do");
01017         next();
01018         parseStatement();
01019     }
01020
01021     void TPValidator::parseCase() {
01022         requireKW("case");
01023         next();
01024         parseExprStop({"of"});
01025         requireKW("of");
01026         next();
01027         while (!isKW("end")) {
01028             parseCaseLabelList();
01029             require(":");
01030             next();
01031             parseStatement();
01032             if (match(";"))
01033                 next();
01034             if (isKW("else")) {
01035                 next();
01036                 parseStatement();
01037                 if (match(";"))
01038                     next();
01039                 break;
01040             }
01041             if (isKW("end"))
01042                 break;
01043         }

```

```

01044         requireKW("end");
01045         next();
01046     }
01047
01048     void TPValidator::parseCaseLabelList() {
01049         parseCaseLabel();
01050         while (match(",")) {
01051             next();
01052             parseCaseLabel();
01053         }
01054     }
01055
01056     void TPValidator::parseCaseLabel() {
01057         if (match("-"))
01058             next();
01059         if (match(types::TokenType::TT_NUM) || match(types::TokenType::TT_STR) ||
match(types::TokenType::TT_ID)) {
01060             const auto *at = token;
01061             next();
01062             if (match("..")) {
01063                 next();
01064                 if (match("-"))
01065                     next();
01066                 if (match(types::TokenType::TT_NUM) || match(types::TokenType::TT_STR) ||
match(types::TokenType::TT_ID)) {
01067                     next();
01068                     return;
01069                 }
01070                 failAt(at, "Invalid case label range");
01071             }
01072             return;
01073         }
01074         failHere("Invalid case label");
01075     }
01076
01077     void TPValidator::parseWith() {
01078         requireKW("with");
01079         next();
01080         parseDesignator();
01081         while (match(",")) {
01082             next();
01083             parseDesignator();
01084         }
01085         requireKW("do");
01086         next();
01087         ++withDepth;
01088         parseStatement();
01089         --withDepth;
01090     }
01091
01092     void TPValidator::parseGoto() {
01093         requireKW("goto");
01094         next();
01095         require(types::TokenType::TT_NUM);
01096         next();
01097     }
01098
01099     void TPValidator::parseSimpleOrCallOrAssign() {
01100         if (match(types::TokenType::TT_ID) && peekIs("(")) {
01101             next();
01102             parseActualParams();
01103             return;
01104         }
01105         parseDesignator();
01106         if (match(":=")) {
01107             next();
01108             parseExprStop({";", "end", "else", ")", "]", "until", "of", "do", "then"});
01109             return;
01110         }
01111         if (match("(")) {
01112             parseActualParams();
01113             return;
01114         }
01115     }
01116
01117     void TPValidator::parseDesignator() {
01118         require(types::TokenType::TT_ID);
01119         checkVar(token->getTokenValue(), token);
01120         next();
01121         while (true) {
01122             if (match(".")) {
01123                 next();
01124                 require(types::TokenType::TT_ID);
01125                 next();
01126                 continue;
01127             }
01128             if (match("[") {

```

Generated by Doxygen


```

match(types::TokenType::TT_STR)) {
01216     next();
01217     expectOperand = false;
01218     continue;
01219 }
01220 if (match(types::TokenType::TT_ID)) {
01221     const auto *at = token;
01222     std::string name = token->getTokenValue();
01223     next();
01224     if (match("(")) {
01225         parseActualParams();
01226         expectOperand = false;
01227     } else {
01228         checkVarOrConst(name, at);
01229         expectOperand = false;
01230     }
01231 }
01232 while (true) {
01233     if (match("^")) {
01234         next();
01235         continue;
01236     }
01237     if (match(".")) {
01238         next();
01239         require(types::TokenType::TT_ID);
01240         next();
01241         continue;
01242     }
01243     if (match "[")) {
01244         next();
01245         parseExprStop("{}");
01246         while (match(",")) {
01247             next();
01248             parseExprStop("{}");
01249         }
01250         require("]");
01251         next();
01252         continue;
01253     }
01254     if (match("(")) {
01255         parseActualParams();
01256         continue;
01257     }
01258     break;
01259 }
01260 continue;
01261 }
01262 failHere("Invalid expression");
01263 } else {
01264     if (bracket > 0 && match(",")) {
01265         next();
01266         expectOperand = true;
01267         continue;
01268     }
01269     if (isRelOp() || isAddOp() || isMulOp() || isSetOp()) {
01270         next();
01271         expectOperand = true;
01272         continue;
01273     }
01274 }
01275 return;
01276 }
01277 }
01278 }
01279
01280 void TPValidator::parseConstExpr(const std::unordered_set<std::string> &stops, bool constOnly) {
01281     int paren = 0;
01282     bool expectOperand = true;
01283     while (token) {
01284         if (paren == 0) {
01285             if (token->getTokenType() == types::TokenType::TT_ID &&
stops.count(lower(token->getTokenValue())))
01286                 return;
01287             if (token->getTokenType() == types::TokenType::TT_SYM &&
stops.count(token->getTokenValue()))
01288                 return;
01289         }
01290         if (match("(")) {
01291             ++paren;
01292             next();
01293             expectOperand = true;
01294             continue;
01295         }
01296         if (match(")")) {
01297             if (paren <= 0)
01298                 break;
01299             --paren;

```

```

01300         next();
01301         expectOperand = false;
01302         continue;
01303     }
01304     if (expectOperand) {
01305         if (match("+") || match("-") || isKW("not")) {
01306             next();
01307             continue;
01308         }
01309         if (match(types::TokenType::TT_NUM) || match(types::TokenType::TT_HEX) ||
match(types::TokenType::TT_STR)) {
01310             next();
01311             expectOperand = false;
01312             continue;
01313         }
01314         if (match(types::TokenType::TT_ID)) {
01315             const auto *at = token;
01316             if (constOnly)
01317                 checkConstOnly(token->getTokenValue(), token);
01318             next();
01319             if (match("("))
01320                 failAt(at, "Function call not allowed in constant expression");
01321             expectOperand = false;
01322             continue;
01323         }
01324         failHere("Invalid constant expression");
01325     } else {
01326         if (match("+") || match("-") || match("*") || match("/") ||
isKW("div") || isKW("mod") || isKW("and") || isKW("or") || isKW("xor") ||
01327         match("<") || match("<=") || match(">") || match(">=") || match("=") ||
match(">")) {
01328             next();
01329             expectOperand = true;
01330             continue;
01331         }
01332     }
01333     return;
01334 }
01335 }
01336 }
01337
01338 bool TPValidator::isRelOp() const { return isKW("in") || match("=") || match("<") || match("<=")
|| match(">") || match(">=") || match(">="); }
01339 bool TPValidator::isAddOp() const { return match("+") || match("-") || isKW("or") || isKW("xor"); }
01340 }
01341 bool TPValidator::isMulOp() const { return match("*") || match("/") || isKW("div") || isKW("mod")
|| isKW("and") || isKW("shl") || isKW("shr"); }
01342 bool TPValidator::isSetOp() const { return isKW("union") || isKW("exclude") || isKW("symdiff"); }
01343
01344 bool TPValidator::isBuiltinType() const {
01345     return isKW("integer") || isKW("real") || isKW("boolean") || isKW("char") ||
isKW("byte") || isKW("word") || isKW("longint") || isKW("shortint") ||
01346     isKW("smallint") || isKW("cardinal") || isKW("string") || isKW("text") ||
isKW("pointer");
01347 }
01348
01349 void TPValidator::checkType(const std::string &name, const scan::TToken *at) {
01350     auto key = lower(name);
01351     for (auto it = scopeStack.rbegin(); it != scopeStack.rend(); ++it) {
01352         if (it->types.count(key))
01353             return;
01354     }
01355     if (!isBuiltinType()) {
01356         failAt(at, "Unknown type identifier '" + name + "'");
01357     }
01358 }
01359
01360 mx::Scope *TPValidator::currentScope() {
01361     if (scopeStack.empty())
01362         return nullptr;
01363     return &scopeStack.back();
01364 }
01365
01366 const mx::Scope *TPValidator::currentScope() const {
01367     if (scopeStack.empty())
01368         return nullptr;
01369     return &scopeStack.back();
01370 }
01371
01372 static const std::unordered_set<std::string> pascal_keywords = {
01373     "program", "unit", "interface", "implementation",
01374     "uses", "var", "const", "type", "procedure", "function", "begin", "end",
01375     "if", "then", "else", "while", "do", "for", "to", "downto", "repeat", "until",
01376     "case", "of", "with", "goto", "label", "exit", "break", "continue",
01377     "nil", "new", "dispose", "setlength", "high", "low",
01378     "writeln", "write", "readln", "read", "seed_random", "rand_number",
01379     "div", "mod", "and", "or", "not", "in",

```

```

01382     "integer", "real", "boolean", "char", "byte", "word", "longint", "shortint",
01383     "smallint", "cardinal", "string", "text", "double", "single", "extended",
01384     "comp", "currency", "ptr", "pointer", "array", "record", "set",
01385     "packed", "file", "text",
01386     "assign", "reset", "rewrite", "append", "close", "eof", "eoln",
01387     "include", "exclude");
01388
01389 bool TPValidator::isPascalKeyword(const std::string &s) const {
01390     return pascal_keywords.count(lower(s));
01391 }
01392
01393 bool TPValidator::isVarDeclaredHere(const std::string &name) const {
01394     return currentScope() && currentScope()->vars.count(lower(name));
01395 }
01396 bool TPValidator::isTypeDeclaredHere(const std::string &name) const {
01397     return currentScope() && currentScope()->types.count(lower(name));
01398 }
01399 bool TPValidator::isFuncDeclaredHere(const std::string &name) const {
01400     return currentScope() && currentScope()->funcs.count(lower(name));
01401 }
01402 bool TPValidator::isProcDeclaredHere(const std::string &name) const {
01403     return currentScope() && currentScope()->procs.count(lower(name));
01404 }
01405 bool TPValidator::iParamDeclaredHere(const std::string &name) const {
01406     return currentScope() && currentScope()->params.count(lower(name));
01407 }
01408
01409 void TPValidator::declareVar(const std::string &name, const scan::TToken *at) {
01410     std::string key = lower(name);
01411     if (isVarDeclaredHere(key) || iParamDeclaredHere(key) ||
01412         isTypeDeclaredHere(key) || isFuncDeclaredHere(key) || isProcDeclaredHere(key)) {
01413         failAt(at, "Redeclaration of variable '" + name + "' in this scope");
01414     }
01415     currentScope()->vars.insert(key);
01416 }
01417
01418 void TPValidator::declareFunc(const std::string &name, const scan::TToken *at) {
01419     std::string key = lower(name);
01420     if (isFuncDeclaredHere(key) || isVarDeclaredHere(key) ||
01421         isTypeDeclaredHere(key) || isProcDeclaredHere(key) || iParamDeclaredHere(key)) {
01422         failAt(at, "Redeclaration of function '" + name + "' in this scope");
01423     }
01424     currentScope()->funcs.insert(key);
01425 }
01426
01427 void TPValidator::declareProc(const std::string &name, const scan::TToken *at) {
01428     std::string key = lower(name);
01429     if (isProcDeclaredHere(key) || isVarDeclaredHere(key) ||
01430         isTypeDeclaredHere(key) || isFuncDeclaredHere(key) || iParamDeclaredHere(key)) {
01431         failAt(at, "Redeclaration of procedure '" + name + "' in this scope");
01432     }
01433     currentScope()->procs.insert(key);
01434 }
01435
01436 void TPValidator::declareParam(const std::string &name, const scan::TToken *at) {
01437     std::string key = lower(name);
01438     if (iParamDeclaredHere(key) || isVarDeclaredHere(key) ||
01439         isTypeDeclaredHere(key) || isFuncDeclaredHere(key) || isProcDeclaredHere(key)) {
01440         failAt(at, "Redeclaration of parameter '" + name + "' in this scope");
01441     }
01442     currentScope()->params.insert(key);
01443 }
01444
01445 void TPValidator::declareType(const std::string &name, const scan::TToken *at) {
01446     auto key = lower(name);
01447     if (isTypeDeclaredHere(key) || isVarDeclaredHere(key) ||
01448         isFuncDeclaredHere(key) || isProcDeclaredHere(key) || iParamDeclaredHere(key)) {
01449         failAt(at, "Redeclaration of type '" + name + "' in this scope");
01450     }
01451     currentScope()->types.insert(key);
01452 }
01453
01454 void TPValidator::pushRecordFieldScope(const std::string &recordTypeName) {
01455     currentRecordTypeName = recordTypeName;
01456
01457     if (recordFieldScopesByType.find(recordTypeName) == recordFieldScopesByType.end()) {
01458         recordFieldScopesByType[recordTypeName] = std::unordered_set<std::string>();
01459     }
01460 }
01461
01462 void TPValidator::popRecordFieldScope() {
01463     currentRecordTypeName.clear();
01464 }
01465
01466 bool TPValidator::inRecordFieldScope() const {
01467     return !currentRecordTypeName.empty();
01468 }

```

```

01469
01470 void TPValidator::declareRecordField(const std::string &name, const scan::TToken *at) {
01471     if (!inRecordFieldScope()) {
01472         declareVar(name, at);
01473         return;
01474     }
01475
01476     std::string key = lower(name);
01477     auto &fields = recordFieldScopesByType[currentRecordTypeName];
01478
01479     if (fields.count(key)) {
01480         failAt(at, "Redeclaration of field '" + name + "' in this record");
01481     }
01482
01483     fields.insert(key);
01484 }
01485
01486 void TPValidator::parseFieldIdentList() {
01487     require(types::TokenType::TT_ID);
01488     declareRecordField(token->getTokenValue(), token);
01489     next();
01490     while (match(",")) {
01491         next();
01492         require(types::TokenType::TT_ID);
01493         declareRecordField(token->getTokenValue(), token);
01494         next();
01495     }
01496 }
01497
01498 } // namespace mxvm

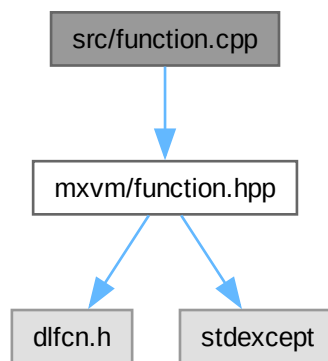
```

12.84 src/function.cpp File Reference

RuntimeFunction implementation — dlopen/dlsym dynamic function loading.

#include "mxvm/function.hpp"

Include dependency graph for function.cpp:



Namespaces

- namespace `mxvm`

12.84.1 Detailed Description

RuntimeFunction implementation — dlopen/dlsym dynamic function loading.

Author

Jared Bruni

Definition in file [function.cpp](#).

12.85 function.cpp

[Go to the documentation of this file.](#)

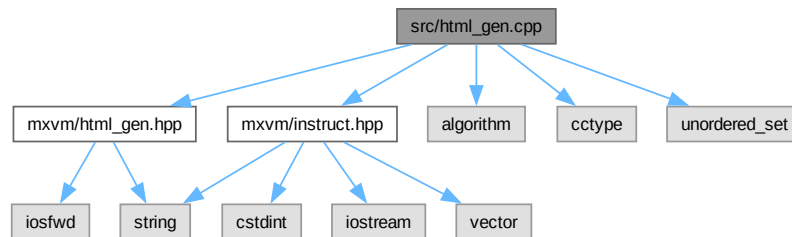
```
00001
00006 #include "mxvm/function.hpp"
00007
00008 namespace mxvm {
00009
00010 }
```

12.86 src/html_gen.cpp File Reference

Syntax-highlighted HTML generation from MXVM source code.

```
#include "mxvm/html_gen.hpp"
#include "mxvm/instruct.hpp"
#include <algorithm>
#include <cctype>
#include <unordered_set>
```

Include dependency graph for html_gen.cpp:



Namespaces

- namespace `mxvm`

Functions

- static `std::unordered_set< std::string > mxvm::buildInstructionSet()`
- static `std::unordered_set< std::string > mxvm::buildKeywordSet()`

Variables

- static const `std::unordered_set< std::string > mxvm::kInstructions = buildInstructionSet()`
- static const `std::unordered_set< std::string > mxvm::kKeywords = buildKeywordSet()`
- static const `std::unordered_set< std::string > mxvm::kTypes`

12.86.1 Detailed Description

Syntax-highlighted HTML generation from MXVM source code.

Author

Jared Bruni

Definition in file `html_gen.cpp`.

12.87 html_gen.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "mxvm/html_gen.hpp"
00007 #include "mxvm/instruct.hpp"
00008 #include <algorithm>
00009 #include <cctype>
00010 #include <unordered_set>
00011
00012 namespace {
00013     inline bool isIdentStart(char c) { return std::isalpha(static_cast<unsigned char>(c)) || c == '_';
00014 }
00015     inline bool isIdentChar(char c) { return std::isalnum(static_cast<unsigned char>(c)) || c == '_';
00016 }
00017     std::string lower(std::string s) {
00018         std::transform(s.begin(), s.end(), s.begin(), [](unsigned char c) { return std::tolower(c);
00019 });
00020         return s;
00021     }
00022     std::string html_escape(const std::string &s) {
00023         std::string out;
00024         out.reserve(s.size() * 12 / 10 + 8);
00025         for (char c : s) {
00026             switch (c) {
00027                 case '&':
00028                     out += "&amp;";
00029                     break;
00030                 case '<':
00031                     out += "&lt;";
00032                     break;
00033                 case '>':
00034                     out += "&gt;";
00035                     break;
00036                 case '"':
00037                     out += "&quot;";
00038                     break;
00039                 case '\\':
00040                     out += "&#39;";
00041                     break;
00042                 case '\n':
00043                     out += "\\n";
00044                     break;
00045                 case '\t':
00046                     out += "\\t";
00047                     break;
00048                 default:
00049                     out.push_back(c);
00050                     break;
00051             }
00052         }
00053         return out;
00054     }
00055 } // namespace
00056 namespace mxvm {
00057     static std::unordered_set<std::string> buildInstructionSet() {
00058         std::unordered_set<std::string> s;
00059         for (const auto &name : IncType) {
00060             if (name == "NULL")
00061                 continue;
00062             s.insert(lower(name));
00063         }
00064         return s;
00065     }
00066     static std::unordered_set<std::string> buildKeywordSet() {
00067         std::unordered_set<std::string> s;
00068         for (const auto &k : keywords)
00069             s.insert(lower(k));
00070         const char *extra[] = {
00071             "function", "export", "return", "start", "done", "true", "false", "null",
00072             "string", "int", "byte", "float", "ptr", "object", "program", "module", "data", "code",
00073             "section"};
00074         for (auto *p : extra)
00075             s.insert(p);
00076         return s;
00077     }
00078     static const std::unordered_set<std::string> kInstructions = buildInstructionSet();
00079     static const std::unordered_set<std::string> kKeywords = buildKeywordSet();
00080     static const std::unordered_set<std::string> kTypes = [] {
00081         return std::unordered_set<std::string>{"string", "int", "byte", "float", "ptr"};
00082     }();
00083 }

```

```

00084     HTMLGen::HTMLGen(const std::string &text) : source(text) {}
00085
00086     void HTMLGen::output(std::ostream &out, std::string filename) {
00087         out << "<!doctype html><html><head><meta charset=\"utf-8\"><meta name=\"viewport\"
content=\"width=device-width,initial-scale=1\">";
00088         out << "<title>" << filename << " - [ MXVM Source ]</title>";
00089         out << "<style>";
00090         out << "body{background:#FFFFFF;color:#000000;margin:0;font:13px/1.6 \"Courier
New\", Courier, monospace}";
00091         out << ".wrap{max-width:1100px;margin:0 auto;padding:24px}";
00092         out <<


```
pre.code{white-space:pre-wrap;word-wrap:break-word;background:#FFFFFF;padding:16px;border:1px solid
#CCCCC;border-radius:2px}";
```


00093         out << ".kw{color:#000080;font-weight:bold}";
00094         out << ".op{color:#000080;font-weight:bold}";
00095         out << ".id{color:#000000}";
00096         out << ".num{color:#800000}";
00097         out << ".str{color:#008080}";
00098         out << ".com{color:#808080;font-style:italic}";
00099         out << ".lbl{color:#008000;font-weight:bold}";
00100         out << ".typ{color:#000080;font-weight:bold}";
00101         out << "</style></head><body><div class=\"wrap\"><pre class=\"code\"><code>";
00102         const std::string &s = source;
00103         size_t i = 0, n = s.size();
00104
00105         // State for suppressing "string" highlighting inside section module { ... }
00106         enum class ModState { None, SawSection, SawModule, Inside };
00107         ModState modState = ModState::None;
00108
00109         while (i < n) {
00110             char c = s[i];
00111             if (c == '\r') {
00112                 ++i;
00113                 continue;
00114             }
00115             if (c == '\n') {
00116                 out << "\n";
00117                 ++i;
00118                 continue;
00119             }
00120             if (c == '\t') {
00121                 out << "  ";
00122                 ++i;
00123                 continue;
00124             }
00125             if (c == ' ') {
00126                 out << " ";
00127                 ++i;
00128                 continue;
00129             }
00130
00131             if (c == '#') {
00132                 size_t j = i;
00133                 while (j < n && s[j] != '\n')
00134                     ++j;
00135                 std::string t = s.substr(i, j - i);
00136                 out << "<span class=\"com\">" << html_escape(t) << "</span>";
00137                 i = j;
00138                 continue;
00139             }
00140
00141             if (c == '"') {
00142                 size_t j = i + 1;
00143                 bool esc = false;
00144                 while (j < n) {
00145                     if (!esc && s[j] == '"') {
00146                         ++j;
00147                         break;
00148                     }
00149                     esc = (!esc && s[j] == '\\');
00150                     ++j;
00151                 }
00152                 std::string t = s.substr(i, j - i);
00153                 out << "<span class=\"str\">" << html_escape(t) << "</span>";
00154                 i = j;
00155                 continue;
00156             }
00157
00158             if (std::isdigit(static_cast<unsigned char>(c)) || (c == '-' && i + 1 < n &&
std::isdigit(static_cast<unsigned char>(s[i + 1])))) {
00159                 size_t j = i;
00160                 if (s[j] == '-')
00161                     ++j;
00162                 if (j + 1 < n && s[j] == '0' && (s[j + 1] == 'x' || s[j + 1] == 'X')) {
00163                     j += 2;
00164                     while (j < n && std::isxdigit(static_cast<unsigned char>(s[j])))
00165                         ++j;

```

```

00166         } else {
00167             while (j < n && std::isdigit(static_cast<unsigned char>(s[j])))
00168                 ++j;
00169             if (j < n && s[j] == '.') {
00170                 ++j;
00171                 while (j < n && std::isdigit(static_cast<unsigned char>(s[j])))
00172                     ++j;
00173             }
00174         }
00175         std::string t = s.substr(i, j - i);
00176         out << "<span class=\"num\">" < html_escape(t) < "</span>";
00177         i = j;
00178         continue;
00179     }
00180
00181     if (isIdentStart(c)) {
00182         size_t j = i + 1;
00183         while (j < n && isIdentChar(s[j]))
00184             ++j;
00185         bool is_label = (j < n && s[j] == ':');
00186         std::string ident = s.substr(i, j - i);
00187         std::string lid = lower(ident);
00188         if (is_label) {
00189             out << "<span class=\"lbl\">" < html_escape(ident) < "</span>:";
00190             i = j + 1;
00191             modState = ModState::None;
00192             continue;
00193         }
00194
00195         // Advance the section module { ... } state machine
00196         if (lid == "section" && modState == ModState::None) {
00197             modState = ModState::SawSection;
00198         } else if (lid == "module" && modState == ModState::SawSection) {
00199             modState = ModState::SawModule;
00200         } else if (modState != ModState::Inside) {
00201             modState = ModState::None;
00202         }
00203
00204         // Inside a module section, suppress type/keyword highlighting
00205         // so that module names like "string" render as plain identifiers.
00206         bool suppress = (modState == ModState::Inside);
00207
00208         if (kInstructions.count(lid)) {
00209             out << "<span class=\"op\">" < html_escape(ident) < "</span>";
00210         } else if (!suppress && (kTypes.count(lid) || kKeywords.count(lid))) {
00211             if (kTypes.count(lid))
00212                 out << "<span class=\"typ\">" < html_escape(ident) < "</span>";
00213             else
00214                 out << "<span class=\"kw\">" < html_escape(ident) < "</span>";
00215         } else {
00216             out << "<span class=\"id\">" < html_escape(ident) < "</span>";
00217         }
00218         i = j;
00219         continue;
00220     }
00221
00222     if (c == '{' || c == '}' || c == ',' || c == '(' || c == ')' || c == '[' || c == ']') {
00223         if (c == '{' && modState == ModState::SawModule) {
00224             modState = ModState::Inside;
00225         } else if (c == '}' && modState == ModState::Inside) {
00226             modState = ModState::None;
00227         }
00228         out << html_escape(std::string(1, c));
00229         ++i;
00230         continue;
00231     }
00232
00233     out << html_escape(std::string(1, c));
00234     ++i;
00235 }
00236 out << "</code></pre></div></body></html>";
00237 }
00238
00239 } // namespace mxvm
00240
00241 // =====
00242 // Pascal highlighter
00243 // =====
00244
00245 namespace {
00246
00247     std::string pas_html_escape(const std::string &s) {
00248         std::string out;
00249         out.reserve(s.size() + 16);
00250         for (char c : s) {
00251             switch (c) {
00252                 case '&': out += "&amp;"; break;

```



```

00254         case '<': out += "<"; break;
00255         case '>': out += ">"; break;
00256         case '"': out += """; break;
00257         case '\\': out += "'"; break;
00258         default: out.push_back(c); break;
00259     }
00260 }
00261 return out;
00262 }
00263
00264 // ---- Pascal keyword sets -----
00265
00266 const std::unordered_set<std::string> &kPasKeywords() {
00267     static const std::unordered_set<std::string> s = {
00268         "program", "unit", "interface", "implementation", "uses",
00269         "var", "const", "type", "procedure", "function", "forward",
00270         "begin", "end",
00271         "if", "then", "else", "while", "do",
00272         "for", "to", "downto", "repeat", "until",
00273         "case", "of", "with", "goto", "label",
00274         "in", "and", "or", "not", "div", "mod", "xor", "shl", "shr",
00275         "packed", "set", "file", "array", "record",
00276     };
00277 }
00278 return s;
00279 }
00280
00281 const std::unordered_set<std::string> &kPasTypes() {
00282     static const std::unordered_set<std::string> s = {
00283         "integer", "real", "boolean", "char",
00284         "byte", "word", "longint", "shortint", "smallint", "cardinal",
00285         "string", "text", "double", "single", "extended",
00286         "comp", "currency", "ptr", "pointer",
00287     };
00288 }
00289 return s;
00290 }
00291
00292 const std::unordered_set<std::string> &kPasBuiltins() {
00293     static const std::unordered_set<std::string> s = {
00294         // I/O
00295         "writeln", "write", "readln", "read",
00296         "assign", "reset", "rewrite", "append", "close", "eof", "eoln",
00297         // memory / dynamic
00298         "new", "dispose", "setlength", "high", "low",
00299         // control
00300         "exit", "break", "continue", "halt",
00301         // predefined consts
00302         "nil", "true", "false",
00303         // special function return var
00304         "result",
00305         // math / conversion
00306         "ord", "chr", "succ", "pred",
00307         "inc", "dec",
00308         "abs", "sqr", "sqrt", "trunc", "round",
00309         "odd", "upcase",
00310         "random", "randomize", "seed_random", "rand_number",
00311         // string
00312         "length", "copy", "pos", "concat", "delete", "insert", "str", "val",
00313         // set
00314         "include", "exclude",
00315     };
00316 }
00317 return s;
00318 }
00319
00320 } // anonymous namespace
00321
00322 namespace mxvm {
00323
00324 PasHTMLGen::PasHTMLGen(const std::string &src) : source(src) {}
00325
00326 std::string PasHTMLGen::defaultCss() {
00327     return
00328         "/* mxvm-highlight.css -- syntax highlight stylesheet for mxvm-html\n"
00329         "   Inspired by the Borland Delphi 7 editor default colour scheme.\n"
00330         "   Edit this file to change colours; it takes precedence over the\n"
00331         "   inline fallback styles that are embedded in every HTML file. */\n"
00332         "\n"
00333         "body {\n"
00334         "    background: #FFFFFF;\n"
00335         "    color: #000000;\n"
00336         "    margin: 0;\n"
00337         "    font: 13px/1.6 \"Courier New\", Courier, monospace;\n"
00338         "}\n"
00339         ".wrap {\n"
00340         "    max-width: 1100px;\n"
00341         "    margin: 0 auto;\n"
00342         "    padding: 24px;\n"
00343         "}\n"

```

```

00344     "pre.code {\n"
00345     "     white-space: pre-wrap;\n"
00346     "     word-wrap: break-word;\n"
00347     "     background: #FFFFFF;\n"
00348     "     padding: 16px;\n"
00349     "     border: 1px solid #CCCCC;\n"
00350     "     border-radius: 2px;\n"
00351     "}\n"
00352     "\n"
00353     "/* ---- token spans ---- */\n"
00354     "\n"
00355     "/* reserved keywords -- bold navy (classic Delphi blue) */\n"
00356     ".kw { color: #000080; font-weight: bold; }\n"
00357     "/* built-in types -- same as keywords */\n"
00358     ".typ { color: #000080; font-weight: bold; }\n"
00359     "/* built-in routines and predefined constants */\n"
00360     ".bi { color: #000080; }\n"
00361     "/* string / char literals -- teal */\n"
00362     ".str { color: #008080; }\n"
00363     "/* numeric literals -- maroon */\n"
00364     ".num { color: #800000; }\n"
00365     "/* comments -- gray italic */\n"
00366     ".com { color: #808080; font-style: italic; }\n"
00367     "/* plain identifiers -- black */\n"
00368     ".id { color: #000000; }\n"
00369     "\n"
00370     "/* ---- mxvm-specific (also used by .mxvm output) ---- */\n"
00371     "\n"
00372     "/* VM instructions */\n"
00373     ".op { color: #000080; font-weight: bold; }\n"
00374     "/* VM labels */\n"
00375     ".lbl { color: #008000; font-weight: bold; }\n";
00376 }
00377
00378 void PasHTMLGen::output(std::ostream &out, const std::string &filename,
00379                        const std::string &cssHref) {
00380     out << "<!doctype html><html><head>"
00381     << "<meta charset='utf-8'>"
00382     << "<meta name='viewport' content='width=device-width,initial-scale=1'>"
00383     << "<title>" << pas_html_escape(filename) << " - [ Pascal Source ]</title>"
00384     // Inline fallback: guarantees highlighting even if the CSS file is absent
00385     << "<style>\n" << defaultCss() << "</style>"
00386     // External file: user edits this to customise; it overrides the inline block
00387     << "<link rel='stylesheet' href='\" << pas_html_escape(cssHref) << "\">"
00388     << "</head><body><div class='wrap'><pre class='\"code\"><code>";
00389
00390     const std::string &s = source;
00391     size_t i = 0, n = s.size();
00392
00393     while (i < n) {
00394         char c = s[i];
00395
00396         // -- whitespace -----
00397         if (c == '\r') { ++i; continue; }
00398         if (c == '\n') { out << '\n'; ++i; continue; }
00399         if (c == '\t') { out << '\t'; ++i; continue; }
00400         if (c == ' ') { out << ' '; ++i; continue; }
00401
00402         // -- line comment // ... -----
00403         if (c == '/' && i + 1 < n && s[i + 1] == '/') {
00404             size_t j = i;
00405             while (j < n && s[j] != '\n') ++j;
00406             out << "<span class='com'>" << pas_html_escape(s.substr(i, j - i)) << "</span>";
00407             i = j;
00408             continue;
00409         }
00410
00411         // -- block comment { ... } -----
00412         if (c == '{') {
00413             size_t j = i + 1;
00414             while (j < n && s[j] != '}') ++j;
00415             if (j < n) ++j; // include '}'
00416             out << "<span class='com'>" << pas_html_escape(s.substr(i, j - i)) << "</span>";
00417             i = j;
00418             continue;
00419         }
00420
00421         // -- block comment (* ... *) -----
00422         if (c == '(' && i + 1 < n && s[i + 1] == '*') {
00423             size_t j = i + 2;
00424             while (j + 1 < n && !(s[j] == '*' && s[j + 1] == ' ')) ++j;
00425             if (j + 1 < n) j += 2; // include '*'
00426             out << "<span class='com'>" << pas_html_escape(s.substr(i, j - i)) << "</span>";
00427             i = j;
00428             continue;
00429         }
00430     }

```

```

00431         // -- Pascal char literal #65 or #541 -----
00432         if (c == '#') {
00433             size_t j = i + 1;
00434             if (j < n && s[j] == '$') {
00435                 ++j;
00436                 while (j < n && std::isxdigit(static_cast<unsigned char>(s[j]))) ++j;
00437             } else {
00438                 while (j < n && std::isdigit(static_cast<unsigned char>(s[j]))) ++j;
00439             }
00440             out << "<span class=\"num\">" << pas_html_escape(s.substr(i, j - i)) << "</span>";
00441             i = j;
00442             continue;
00443         }
00444
00445         // -- string literal 'hello' with " escape -----
00446         if (c == '"') {
00447             size_t j = i + 1;
00448             while (j < n) {
00449                 if (s[j] == '"') {
00450                     ++j;
00451                     if (j < n && s[j] == '\\') { ++j; continue; } // " + single quote
00452                     break;
00453                 }
00454                 ++j;
00455             }
00456             out << "<span class=\"str\">" << pas_html_escape(s.substr(i, j - i)) << "</span>";
00457             i = j;
00458             continue;
00459         }
00460
00461         // -- hex literal $FF -----
00462         if (c == '$') {
00463             size_t j = i + 1;
00464             while (j < n && std::isxdigit(static_cast<unsigned char>(s[j]))) ++j;
00465             out << "<span class=\"num\">" << pas_html_escape(s.substr(i, j - i)) << "</span>";
00466             i = j;
00467             continue;
00468         }
00469
00470         // -- decimal / float (don't consume '.' as a decimal point) ----
00471         if (std::isdigit(static_cast<unsigned char>(c))) {
00472             size_t j = i;
00473             while (j < n && std::isdigit(static_cast<unsigned char>(s[j]))) ++j;
00474             if (j < n && s[j] == '.' && (j + 1 <= n || s[j + 1] != '.')) {
00475                 ++j;
00476                 while (j < n && std::isdigit(static_cast<unsigned char>(s[j]))) ++j;
00477             }
00478             if (j < n && (s[j] == 'e' || s[j] == 'E')) {
00479                 ++j;
00480                 if (j < n && (s[j] == '+' || s[j] == '-')) ++j;
00481                 while (j < n && std::isdigit(static_cast<unsigned char>(s[j]))) ++j;
00482             }
00483             out << "<span class=\"num\">" << pas_html_escape(s.substr(i, j - i)) << "</span>";
00484             i = j;
00485             continue;
00486         }
00487
00488         // -- identifier / keyword -----
00489         if (isIdentStart(c)) {
00490             size_t j = i + 1;
00491             while (j < n && isIdentChar(s[j])) ++j;
00492             std::string ident = s.substr(i, j - i);
00493             std::string lid = lower(ident);
00494             i = j;
00495
00496             if (kPasKeywords().count(lid))
00497                 out << "<span class=\"kw\">" << pas_html_escape(ident) << "</span>";
00498             else if (kPasTypes().count(lid))
00499                 out << "<span class=\"typ\">" << pas_html_escape(ident) << "</span>";
00500             else if (kPasBuiltins().count(lid))
00501                 out << "<span class=\"bi\">" << pas_html_escape(ident) << "</span>";
00502             else
00503                 out << "<span class=\"id\">" << pas_html_escape(ident) << "</span>";
00504             continue;
00505         }
00506
00507         // -- everything else (operators, punctuation) -----
00508         out << pas_html_escape(std::string(1, c));
00509         ++i;
00510     }
00511
00512     out << "</code></pre></div></body></html>";
00513 }
00514
00515 } // namespace mxvm

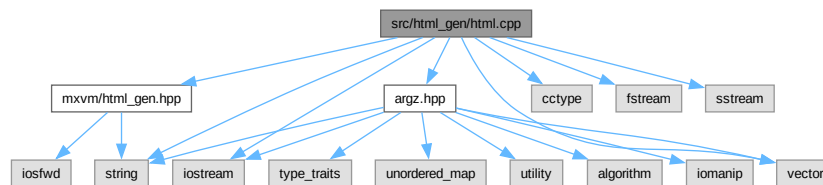
```

12.88 src/html_gen/html.cpp File Reference

mxvm-html command-line tool — converts MXVM source to syntax-highlighted HTML

```
#include "mxvm/html_gen.hpp"
#include "argz.hpp"
#include <cctype>
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
```

Include dependency graph for html.cpp:



Functions

- static std::string [readFile](#) (const std::string &path)
- static bool [endsWith](#) (const std::string &s, const std::string &suffix)
- static std::string [dirOf](#) (const std::string &path)
- static bool [fileExists](#) (const std::string &path)
- static void [ensureCss](#) (const std::string &outPath)
- static void [writeHtml](#) (const std::string &src, const std::string &inPath, const std::string &outPath, const std::string &cssHref="")
- static std::string [extractDeclName](#) (const std::string &src)
- int [main](#) (int argc, char **argv)

12.88.1 Detailed Description

mxvm-html command-line tool — converts MXVM source to syntax-highlighted HTML

Author

Jared Bruni

Usage: mxvm-html -i input.mxvm [-o output.html] [-s style.css] mxvm-html -i input.pas [-o output.html] [-s style.css]
 mxvm-html -c file1.mxvm file2.pas ... [-s style.css] mxvm-html -h

For .pas input the output is Pascal-syntax-highlighted using PasHTMLGen. An external stylesheet is linked from the HTML so you can change the look without recompiling. By default mxvm-highlight.css is written to the same directory as the first output file (only when the file does not exist yet). Use -s to point to an existing stylesheet instead.

Output filenames are auto-derived from the program/unit/object declaration in the source unless -o is supplied.

Definition in file [html.cpp](#).

12.88.2 Function Documentation

12.88.2.1 dirOf()

```
std::string dirOf (
    const std::string & path) [static]
```

Return the directory portion of a path, including the trailing slash.

Definition at line 47 of file [html.cpp](#).

References [pos\(\)](#).

Referenced by [ensureCss\(\)](#).

12.88.2.2 endsWith()

```
bool endsWith (
    const std::string & s,
    const std::string & suffix) [static]
```

Definition at line 41 of file [html.cpp](#).

Referenced by [writeHtml\(\)](#).

12.88.2.3 ensureCss()

```
void ensureCss (
    const std::string & outPath) [static]
```

Write mxvm-highlight.css next to outPath if it does not already exist.

Definition at line 58 of file [html.cpp](#).

References [mxvm::PasHTMLGen::defaultCss\(\)](#), [dirOf\(\)](#), and [fileExists\(\)](#).

Referenced by [writeHtml\(\)](#).

12.88.2.4 extractDeclName()

```
std::string extractDeclName (
    const std::string & src) [static]
```

Scan source for the first "program|unit|object <Identifier>" and return the identifier. Skips //-line comments, MXVM #-comments, Pascal {}-comments, and Pascal (*)-comments.

Definition at line 98 of file [html.cpp](#).

References [pos\(\)](#).

Referenced by [main\(\)](#).

12.88.2.5 fileExists()

```
bool fileExists (
    const std::string & path) [static]
```

Definition at line 52 of file [html.cpp](#).

Referenced by [ensureCss\(\)](#).

12.88.2.6 main()

```
int main (
    int argc,
    char ** argv)
```

Definition at line 161 of file [html.cpp](#).

References [mx::Argz<String>::addOptionSingleValue\(\)](#), [mx::Argument<String>::arg_value](#), [argc\(\)](#), [argv\(\)](#), [extractDeclName\(\)](#), [mx::Argz<String>::help\(\)](#), [mx::Argz<String>::proc\(\)](#), [readFile\(\)](#), [mx::ArgException<String>::text\(\)](#), and [writeHtml\(\)](#).

12.88.2.7 readFile()

```
std::string readFile (
    const std::string & path) [static]
```

Definition at line 30 of file [html.cpp](#).

Referenced by [main\(\)](#).

12.88.2.8 writeHtml()

```
void writeHtml (
    const std::string & src,
```

```

    const std::string & inPath,
    const std::string & outPath,
    const std::string & cssHref = "") [static]

```

Render src to outPath. For .pas input PasHTMLGen is used and the HTML links to cssHref; if cssHref is empty the default stylesheet is written alongside the output.

Definition at line 74 of file [html.cpp](#).

References [endsWith\(\)](#), [ensureCss\(\)](#), [mxvm::HTMLGen::output\(\)](#), and [mxvm::PasHTMLGen::output\(\)](#).

Referenced by [main\(\)](#).

12.89 html.cpp

[Go to the documentation of this file.](#)

```

00001
00021 #include "mxvm/html_gen.hpp"
00022 #include "argz.hpp"
00023 #include <cctype>
00024 #include <fstream>
00025 #include <iostream>
00026 #include <sstream>
00027 #include <string>
00028 #include <vector>
00029
00030 static std::string readFile(const std::string &path) {
00031     std::fstream f(path, std::ios::in);
00032     if (!f.is_open()) {
00033         std::cerr << "mxvm-html: could not open file: " << path << "\n";
00034         exit(EXIT_FAILURE);
00035     }
00036     std::ostringstream ss;
00037     ss << f.rdbuf();
00038     return ss.str();
00039 }
00040
00041 static bool endsWith(const std::string &s, const std::string &suffix) {
00042     return s.size() >= suffix.size() &&
00043         s.compare(s.size() - suffix.size(), suffix.size(), suffix) == 0;
00044 }
00045
00046 static std::string dirOf(const std::string &path) {
00047     auto pos = path.find_last_of("/\\");
00048     return pos == std::string::npos ? "./" : path.substr(0, pos + 1);
00049 }
00050
00051 static bool fileExists(const std::string &path) {
00052     std::ifstream f(path);
00053     return f.good();
00054 }
00055
00056 static void ensureCss(const std::string &outPath) {
00057     std::string cssPath = dirOf(outPath) + "mxvm-highlight.css";
00058     if (!fileExists(cssPath)) {
00059         std::ofstream f(cssPath);
00060         if (f.is_open()) {
00061             f << mxvm::PasHTMLGen::defaultCss();
00062             std::cout << "mxvm-html: wrote " << cssPath << "\n";
00063         }
00064     }
00065 }
00066
00067 static void writeHtml(const std::string &src, const std::string &inPath,
00068                     const std::string &outPath, const std::string &cssHref = "") {
00069     std::fstream ofile(outPath, std::ios::out);
00070     if (!ofile.is_open()) {
00071         std::cerr << "mxvm-html: could not open output file: " << outPath << "\n";
00072         exit(EXIT_FAILURE);
00073     }
00074
00075     if (endsWith(inPath, ".pas")) {
00076         std::string href = cssHref.empty() ? "mxvm-highlight.css" : cssHref;
00077         if (cssHref.empty()) ensureCss(outPath);
00078         mxvm::PasHTMLGen gen(src);
00079         gen.output(ofile, inPath, href);
00080     } else {
00081         mxvm::HTMLGen gen(src);
00082         gen.output(ofile, inPath);
00083     }
00084
00085     ofile.close();
00086     std::cout << "mxvm-html: wrote " << outPath << "\n";
00087 }
00088
00089

```

```

00095
00098 static std::string extractDeclName(const std::string &src) {
00099     const std::string decls[] = {"program", "unit", "object"};
00100     std::size_t pos = 0, n = src.size();
00101     while (pos < n) {
00102         char c = src[pos];
00103         // whitespace
00104         if (std::isspace(static_cast<unsigned char>(c))) { ++pos; continue; }
00105         // // line comment
00106         if (c == '/' && pos + 1 < n && src[pos + 1] == '/') {
00107             while (pos < n && src[pos] != '\n') ++pos;
00108             continue;
00109         }
00110         // MXVM # comment
00111         if (c == '#') {
00112             while (pos < n && src[pos] != '\n') ++pos;
00113             continue;
00114         }
00115         // Pascal { } comment
00116         if (c == '{') {
00117             while (pos < n && src[pos] != '}') ++pos;
00118             if (pos < n) ++pos;
00119             continue;
00120         }
00121         // Pascal (* *) comment
00122         if (c == '(' && pos + 1 < n && src[pos + 1] == '*') {
00123             pos += 2;
00124             while (pos + 1 < n && !(src[pos] == '*' && src[pos + 1] == '))') ++pos;
00125             if (pos + 1 < n) pos += 2;
00126             continue;
00127         }
00128         // MXVM /* */ comment
00129         if (c == '/' && pos + 1 < n && src[pos + 1] == '*') {
00130             pos += 2;
00131             while (pos + 1 < n && !(src[pos] == '*' && src[pos + 1] == '/')) ++pos;
00132             if (pos + 1 < n) pos += 2;
00133             continue;
00134         }
00135         if (std::isalpha(static_cast<unsigned char>(c)) || c == '_') {
00136             std::size_t start = pos;
00137             while (pos < n && (std::isalnum(static_cast<unsigned char>(src[pos])) || src[pos] == '_'))
00138                 ++pos;
00139             std::string tok = src.substr(start, pos - start);
00140             // case-insensitive match for Pascal (unit/program)
00141             std::string ltok = tok;
00142             for (auto &ch : ltok) ch = static_cast<char>(std::tolower(static_cast<unsigned
char>(ch)));
00143             for (const auto &kw : decls) {
00144                 if (ltok == kw) {
00145                     while (pos < n && std::isspace(static_cast<unsigned char>(src[pos]))) ++pos;
00146                     if (pos < n && (std::isalpha(static_cast<unsigned char>(src[pos])) || src[pos] ==
'_')) {
00147                         std::size_t ns = pos;
00148                         while (pos < n && (std::isalnum(static_cast<unsigned char>(src[pos])) ||
src[pos] == '_'))
00149                             ++pos;
00150                         return src.substr(ns, pos - ns);
00151                     }
00152                 }
00153             }
00154             continue;
00155         }
00156         ++pos;
00157     }
00158     return {};
00159 }
00160
00161 int main(int argc, char **argv) {
00162     mx::Argv<std::string> args(argc, argv);
00163     args.addOptionSingleValue('i', "input source file (.mxvm or .pas)")
00164         .addOptionSingleValue('o', "output .html file (single-file mode only)")
00165         .addOptionSingleValue('s', "link to this CSS file instead of writing mxvm-highlight.css")
00166         .addOptionSingle('c', "batch: convert each listed file, auto-name output from
program/unit/object")
00167         .addOptionSingle('h', "show this help");
00168
00169     std::string inputFile, outputFile, cssHref;
00170     bool batchMode = false;
00171     std::vector<std::string> positional;
00172
00173     mx::Argument<std::string> arg;
00174     int opt;
00175     try {
00176         while ((opt = args.proc(arg)) != -1) {
00177             switch (opt) {
00178                 case 'i': inputFile = arg.arg_value; break;
00179                 case 'o': outputFile = arg.arg_value; break;

```

```

00180         case 's': cssHref      = arg.arg_value; break;
00181         case 'c': batchMode    = true; break;
00182         case 'h':
00183             std::cout << "mxvm-html: MXVM / Pascal source to HTML converter\n\n";
00184             args.help(std::cout);
00185             return EXIT_SUCCESS;
00186         case '-':
00187             positional.push_back(arg.arg_value);
00188             break;
00189         default: break;
00190     }
00191 }
00192 } catch (const mx::ArgException<std::string> &e) {
00193     std::cerr << "mxvm-html: " << e.text() << "\n";
00194     return EXIT_FAILURE;
00195 }
00196
00197 if (batchMode) {
00198     if (positional.empty()) {
00199         std::cerr << "mxvm-html: -c requires at least one input file\n";
00200         return EXIT_FAILURE;
00201     }
00202     for (const auto &path : positional) {
00203         std::string src = readFile(path);
00204         std::string name = extractDeclName(src);
00205         if (name.empty()) {
00206             std::cerr << "mxvm-html: no program/unit/object declaration found in: " << path << "\n";
00207             continue;
00208         }
00209         writeHtml(src, path, name + ".html", cssHref);
00210     }
00211     return EXIT_SUCCESS;
00212 }
00213
00214 if (!inputFile.empty()) {
00215     std::string src = readFile(inputFile);
00216     if (outputFile.empty()) {
00217         std::string name = extractDeclName(src);
00218         if (name.empty()) {
00219             std::cerr << "mxvm-html: no program/unit/object declaration found in: " << inputFile
00220                 << " -- use -o to specify output filename\n";
00221             return EXIT_FAILURE;
00222         }
00223         outputFile = name + ".html";
00224     }
00225     writeHtml(src, inputFile, outputFile, cssHref);
00226     return EXIT_SUCCESS;
00227 }
00228
00229 std::cerr << "mxvm-html: no input specified. Use -i <file> [-o <output>] [-s <css>] or -c <file1>
[file2...]\n";
00230 std::cerr << "Run with -h for help.\n";
00231 return EXIT_FAILURE;
00232 }

```

12.90 src/icode_exec.cpp File Reference

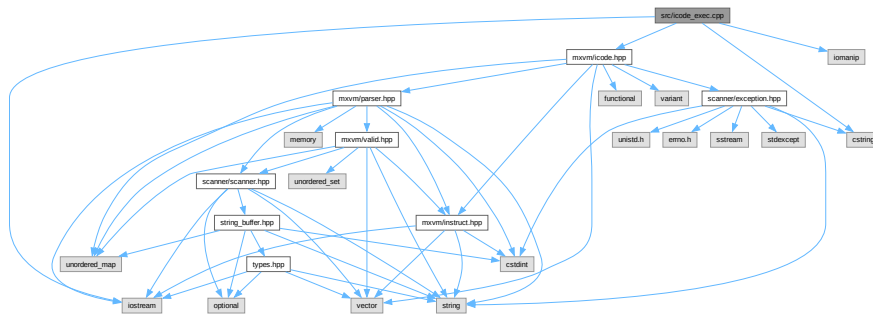
MXVM interpreter — executes intermediate code instructions at runtime.

```

#include "mxvm/icode.hpp"
#include <cstring>
#include <iomanip>
#include <iostream>

```


Include dependency graph for icode_exec.cpp:



Namespaces

- namespace `mxvm`

Functions

- static void `mxvm::releaseOwnedPointer` (`Variable &v`)
- `std::string` `mxvm::toStringFromVarType` (`const VarType &v`)

12.90.1 Detailed Description

MXVM interpreter — executes intermediate code instructions at runtime.

Author

Jared Bruni

Definition in file `icode_exec.cpp`.

12.91 icode_exec.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "mxvm/icode.h"
00007 #include <cstring>
00008 #include <iomanip>
00009 #include <iostream>
00010
00011 namespace mxvm {
00012
00013     static inline void releaseOwnedPointer(Variable &v) {
00014         if (v.type == VarType::VAR_POINTER && v.var_value.ptr_value && v.var_value.owns) {
00015             std::free(v.var_value.ptr_value);
00016             v.var_value.ptr_value = nullptr;
00017             v.var_value.owns = false;
00018             v.var_value.ptr_size = 0;
00019             v.var_value.ptr_count = 0;
00020         }
00021     }
00022
00023     void Program::stop() {
00024         running = false;
00025     }
00026
00027     void Program::flatten_inc(Program *root, Instruction &i) {
00028         if (root != this) {
00029             auto qualifyVar = [&](Operand &op) {
00030                 if (op.op.empty())
00031                     return;
00032                 if (op.type != OperandType::OP_VARIABLE)
00033                     return;
00034                 if (!op.object.empty())
00035                     return;
00036                 if (op.op.find('.') != std::string::npos)

```

```

00037         return;
00038
00039         std::string qualified = this->name + "." + op.op;
00040         if (vars.find(qualified) != vars.end()) {
00041             op.object = this->name;
00042             op.label = op.op;
00043             op.op = qualified;
00044         }
00045     };
00046     auto qualifyLabel = [&](Operand &op) {
00047         if (op.op.empty())
00048             return;
00049         if (op.op.find('.') != std::string::npos)
00050             return;
00051         if (labels.find(op.op) != labels.end()) {
00052             op.object = this->name;
00053             op.label = op.op;
00054             op.op = this->name + "." + op.op;
00055         }
00056     };
00057     Instruction ci = i;
00058     qualifyVar(ci.op1);
00059     qualifyVar(ci.op2);
00060     qualifyVar(ci.op3);
00061     for (auto &vo : ci.vop)
00062         qualifyVar(vo);
00063     switch (ci.instruction) {
00064     case CALL:
00065     case JMP:
00066     case JE:
00067     case JNE:
00068     case JL:
00069     case JLE:
00070     case JG:
00071     case JGE:
00072     case JZ:
00073     case JNZ:
00074     case JA:
00075     case JB:
00076         qualifyLabel(ci.op1);
00077         break;
00078     default:
00079         break;
00080     }
00081     root->add_instruction(ci);
00082     return;
00083 }
00084 root->add_instruction(i);
00085 }
00086
00087 void Program::flatten_label(Program *root, int64_t offset, const std::string &label, bool func) {
00088
00089     if (root != this) {
00090         root->add_label(this->name + "." + label, offset, func);
00091         return;
00092     }
00093     root->add_label(label, offset, func);
00094 }
00095
00096 void Program::flatten_external(Program *root, const std::string &e, RuntimeFunction &r) {
00097     if (root->external_functions.find(e) == root->external_functions.end()) {
00098         root->external_functions[e] = r;
00099     }
00100 }
00101
00102 void Program::flatten(Program *program) {
00103     if (program != this) {
00104         int64_t size = program->inc.size();
00105         for (auto &i : inc) {
00106             flatten_inc(program, i);
00107         }
00108         for (auto &lbl : labels) {
00109             flatten_label(program, lbl.second.first + size, lbl.first, lbl.second.second);
00110         }
00111         for (auto &e : external_functions) {
00112             flatten_external(program, e.first, e.second);
00113         }
00114     }
00115
00116     for (auto &obj : objects) {
00117         if (obj.get() != program)
00118             obj->flatten(program);
00119     }
00120 }
00121
00122 int Program::exec() {
00123

```

```

00124         this->add_standard();
00125
00126         if (inc.empty()) {
00127             std::cerr << "No instructions to execute\n";
00128             return EXIT_FAILURE;
00129         }
00130         pc = 0;
00131         running = true;
00132
00133         while (running && pc < inc.size()) {
00134             const Instruction &instr = inc.at(pc);
00135             if (mxvm::instruct_mode)
00136                 std::cout << instr << "\n";
00137
00138             switch (instr.instruction) {
00139             case MOV:
00140                 exec_mov(instr);
00141                 break;
00142             case ADD:
00143                 exec_add(instr);
00144                 break;
00145             case SUB:
00146                 exec_sub(instr);
00147                 break;
00148             case MUL:
00149                 exec_mul(instr);
00150                 break;
00151             case DIV:
00152                 exec_div(instr);
00153                 break;
00154             case CMP:
00155                 exec_cmp(instr);
00156                 break;
00157             case FCMP:
00158                 exec_fcmp(instr);
00159                 break;
00160             case JMP:
00161                 exec_jmp(instr);
00162                 continue;
00163             case JE:
00164                 exec_je(instr);
00165                 continue;
00166             case JNE:
00167                 exec_jne(instr);
00168                 continue;
00169             case JL:
00170                 exec_jl(instr);
00171                 continue;
00172             case JLE:
00173                 exec_jle(instr);
00174                 continue;
00175             case JG:
00176                 exec_jg(instr);
00177                 continue;
00178             case JGE:
00179                 exec_jge(instr);
00180                 continue;
00181             case JZ:
00182                 exec_jz(instr);
00183                 continue;
00184             case JNZ:
00185                 exec_jnz(instr);
00186                 continue;
00187             case JA:
00188                 exec_ja(instr);
00189                 continue;
00190             case JB:
00191                 exec_jb(instr);
00192                 continue;
00193             case JAE:
00194                 exec_jae(instr);
00195                 continue;
00196             case JBE:
00197                 exec_jbe(instr);
00198                 continue;
00199             case JC:
00200                 exec_jc(instr);
00201                 continue;
00202             case JNC:
00203                 exec_jnc(instr);
00204                 continue;
00205             case JP:
00206                 exec_jp(instr);
00207                 continue;
00208             case JNP:
00209                 exec_jnp(instr);
00210                 continue;

```

```
00211         case JO:
00212             exec_jo(instr);
00213             continue;
00214         case JNO:
00215             exec_jno(instr);
00216             continue;
00217         case JS:
00218             exec_js(instr);
00219             continue;
00220         case JNS:
00221             exec_jns(instr);
00222             continue;
00223         case LOAD:
00224             exec_load(instr);
00225             break;
00226         case STORE:
00227             exec_store(instr);
00228             break;
00229         case OR:
00230             exec_or(instr);
00231             break;
00232         case AND:
00233             exec_and(instr);
00234             break;
00235         case XOR:
00236             exec_xor(instr);
00237             break;
00238         case NOT:
00239             exec_not(instr);
00240             break;
00241         case MOD:
00242             exec_mod(instr);
00243             break;
00244         case PRINT:
00245             exec_print(instr);
00246             break;
00247         case ALLOC:
00248             exec_alloc(instr);
00249             break;
00250         case FREE:
00251             exec_free(instr);
00252             break;
00253         case EXIT:
00254             exec_exit(instr);
00255             return getExitCode();
00256         case GETLINE:
00257             exec_getline(instr);
00258             break;
00259         case PUSH:
00260             exec_push(instr);
00261             break;
00262         case POP:
00263             exec_pop(instr);
00264             break;
00265         case STACK_LOAD:
00266             exec_stack_load(instr);
00267             break;
00268         case STACK_STORE:
00269             exec_stack_store(instr);
00270             break;
00271         case STACK_SUB:
00272             exec_stack_sub(instr);
00273             break;
00274         case CALL:
00275             exec_call(instr);
00276             continue;
00277         case RET:
00278             exec_ret(instr);
00279             break;
00280         case STRING_PRINT:
00281             exec_string_print(instr);
00282             break;
00283         case DONE:
00284             exec_done(instr);
00285             return EXIT_SUCCESS;
00286         case TO_INT:
00287             exec_to_int(instr);
00288             break;
00289         case TO_FLOAT:
00290             exec_to_float(instr);
00291             break;
00292         case INVOKE:
00293             exec_invoke(instr);
00294             break;
00295         case RETURN:
00296             exec_return(instr);
00297             break;
```

```

00298         case NEG:
00299             exec_neg(instr);
00300             break;
00301         case LEA:
00302             exec_lea(instr);
00303             break;
00304         case REALLOC:
00305             exec_realloc(instr);
00306             break;
00307         default:
00308             throw mx::Exception("Unknown instruction: " + std::to_string(instr.instruction));
00309             break;
00310     }
00311     pc++;
00312 }
00313 return getExitCode();
00314 }
00315
00316 void Program::exec_mov(const Instruction &instr) {
00317     if (!isVariable(instr.op1.op)) {
00318         std::cerr << "Error: MOV destination: " + instr.op1.op + " must be a variable, not a
constant\n";
00319         return;
00320     }
00321     Variable &dest = getVariable(instr.op1.op);
00322     // Before freeing dest's old pointer, transfer ownership to any alias
00323     if (dest.type == VarType::VAR_POINTER && dest.var_value.ptr_value && dest.var_value.owns) {
00324         void *old_ptr = dest.var_value.ptr_value;
00325         bool transferred = false;
00326         for (auto &kv : vars) {
00327             if (&kv.second != &dest &&
00328                 kv.second.type == VarType::VAR_POINTER &&
00329                 kv.second.var_value.ptr_value == old_ptr) {
00330                 kv.second.var_value.owns = true;
00331                 transferred = true;
00332                 break;
00333             }
00334         }
00335         if (!transferred) {
00336             std::free(old_ptr);
00337         }
00338         dest.var_value.ptr_value = nullptr;
00339         dest.var_value.owns = false;
00340         dest.var_value.ptr_size = 0;
00341         dest.var_value.ptr_count = 0;
00342     } else {
00343         releaseOwnedPointer(dest);
00344     }
00345
00346     if (isVariable(instr.op2.op)) {
00347         Variable &src = getVariable(instr.op2.op);
00348
00349         if (dest.type == VarType::VAR_INTEGER && src.type == VarType::VAR_FLOAT) {
00350             dest.var_value.int_value = static_cast<int64_t>(src.var_value.float_value);
00351             dest.var_value.type = VarType::VAR_INTEGER;
00352             return;
00353         } else if (dest.type == VarType::VAR_FLOAT && src.type == VarType::VAR_INTEGER) {
00354             dest.var_value.float_value = static_cast<double>(src.var_value.int_value);
00355             dest.var_value.type = VarType::VAR_FLOAT;
00356             return;
00357         }
00358
00359         dest.var_value = src.var_value;
00360         dest.type = src.type;
00361         if (dest.type == VarType::VAR_POINTER || dest.type == VarType::VAR_EXTERN) {
00362             dest.var_value.owns = false;
00363         }
00364     } else {
00365         if (dest.type == VarType::VAR_POINTER) {
00366             std::string v = instr.op2.op;
00367             if (v == "null" || v == "NULL" || v == "0") {
00368                 dest.var_value.ptr_value = nullptr;
00369             } else {
00370                 uintptr_t addr = std::stoull(v, nullptr, 0);
00371                 dest.var_value.ptr_value = reinterpret_cast<void *>(addr);
00372             }
00373             dest.var_value.type = VarType::VAR_POINTER;
00374             dest.var_value.owns = false;
00375         } else {
00376             setVariableFromConstant(dest, instr.op2.op);
00377         }
00378     }
00379 }
00380 }
00381
00382
00383 void Program::exec_add(const Instruction &instr) {

```

```

00384         if (!isVariable(instr.op1.op)) {
00385             std::cerr << "Error: ADD destination: " + instr.op1.op + " must be a variable, not a
constant\n";
00386             return;
00387         }
00388         Variable &dest = getVariable(instr.op1.op);
00389         Variable *src1 = nullptr, *src2 = nullptr;
00390         Variable temp1, temp2;
00391
00392         {
00393             VarType constType = (dest.type == VarType::VAR_POINTER) ? VarType::VAR_INTEGER :
dest.type;
00394             if (instr.op3.op.empty()) {
00395                 src1 = &dest;
00396                 if (isVariable(instr.op2.op)) {
00397                     src2 = &getVariable(instr.op2.op);
00398                 } else {
00399                     temp2 = createTempVariable(constType, instr.op2.op);
00400                     src2 = &temp2;
00401                 }
00402             } else {
00403                 if (isVariable(instr.op2.op)) {
00404                     src1 = &getVariable(instr.op2.op);
00405                 } else {
00406                     temp1 = createTempVariable(constType, instr.op2.op);
00407                     src1 = &temp1;
00408                 }
00409                 if (isVariable(instr.op3.op)) {
00410                     src2 = &getVariable(instr.op3.op);
00411                 } else {
00412                     temp2 = createTempVariable(constType, instr.op3.op);
00413                     src2 = &temp2;
00414                 }
00415             }
00416         }
00417         addVariables(dest, *src1, *src2);
00418     }
00419 }
00420
00421 void Program::exec_sub(const Instruction &instr) {
00422     if (!isVariable(instr.op1.op)) {
00423         std::cerr << "Error: SUB destination must be a variable, not a constant\n";
00424         return;
00425     }
00426     Variable &dest = getVariable(instr.op1.op);
00427
00428     Variable *src1 = nullptr, *src2 = nullptr;
00429     Variable temp1, temp2;
00430
00431     {
00432         VarType constType = (dest.type == VarType::VAR_POINTER) ? VarType::VAR_INTEGER :
dest.type;
00433         if (instr.op3.op.empty()) {
00434             src1 = &dest;
00435             if (isVariable(instr.op2.op)) {
00436                 src2 = &getVariable(instr.op2.op);
00437             } else {
00438                 temp2 = createTempVariable(constType, instr.op2.op);
00439                 src2 = &temp2;
00440             }
00441         } else {
00442             if (isVariable(instr.op2.op)) {
00443                 src1 = &getVariable(instr.op2.op);
00444             } else {
00445                 temp1 = createTempVariable(constType, instr.op2.op);
00446                 src1 = &temp1;
00447             }
00448             if (isVariable(instr.op3.op)) {
00449                 src2 = &getVariable(instr.op3.op);
00450             } else {
00451                 temp2 = createTempVariable(constType, instr.op3.op);
00452                 src2 = &temp2;
00453             }
00454         }
00455     }
00456     subVariables(dest, *src1, *src2);
00457 }
00458
00459 void Program::exec_mul(const Instruction &instr) {
00460     if (!isVariable(instr.op1.op)) {
00461         std::cerr << "Error: MUL destination must be a variable, not a constant\n";
00462         return;
00463     }
00464     Variable &dest = getVariable(instr.op1.op);
00465
00466     Variable *src1 = nullptr, *src2 = nullptr;

```

```

00468     Variable temp1, temp2;
00469
00470     if (instr.op3.op.empty()) {
00471         src1 = &dest;
00472         if (isVariable(instr.op2.op)) {
00473             src2 = &getVariable(instr.op2.op);
00474         } else {
00475             temp2 = createTempVariable(dest.type, instr.op2.op);
00476             src2 = &temp2;
00477         }
00478     } else {
00479         if (isVariable(instr.op2.op)) {
00480             src1 = &getVariable(instr.op2.op);
00481         } else {
00482             temp1 = createTempVariable(dest.type, instr.op2.op);
00483             src1 = &temp1;
00484         }
00485         if (isVariable(instr.op3.op)) {
00486             src2 = &getVariable(instr.op3.op);
00487         } else {
00488             temp2 = createTempVariable(dest.type, instr.op3.op);
00489             src2 = &temp2;
00490         }
00491     }
00492
00493     mulVariables(dest, *src1, *src2);
00494 }
00495
00496 void Program::exec_div(const Instruction &instr) {
00497     if (!isVariable(instr.op1.op)) {
00498         std::cerr << "Error: DIV destination must be a variable, not a constant\n";
00499         return;
00500     }
00501     Variable &dest = getVariable(instr.op1.op);
00502
00503     Variable *src1 = nullptr, *src2 = nullptr;
00504     Variable temp1, temp2;
00505
00506     if (instr.op3.op.empty()) {
00507         src1 = &dest;
00508         if (isVariable(instr.op2.op)) {
00509             src2 = &getVariable(instr.op2.op);
00510         } else {
00511             temp2 = createTempVariable(dest.type, instr.op2.op);
00512             src2 = &temp2;
00513         }
00514     } else {
00515         if (isVariable(instr.op2.op)) {
00516             src1 = &getVariable(instr.op2.op);
00517         } else {
00518             temp1 = createTempVariable(dest.type, instr.op2.op);
00519             src1 = &temp1;
00520         }
00521         if (isVariable(instr.op3.op)) {
00522             src2 = &getVariable(instr.op3.op);
00523         } else {
00524             temp2 = createTempVariable(dest.type, instr.op3.op);
00525             src2 = &temp2;
00526         }
00527     }
00528
00529     divVariables(dest, *src1, *src2);
00530 }
00531
00532 void Program::exec_cmp(const Instruction &instr) {
00533     Variable *var1 = nullptr;
00534     Variable *var2 = nullptr;
00535     Variable temp1, temp2;
00536
00537     if (isVariable(instr.op1.op)) {
00538         var1 = &getVariable(instr.op1.op);
00539     } else {
00540         temp1 = createTempVariable(VarType::VAR_INTEGER, instr.op1.op);
00541         var1 = &temp1;
00542     }
00543     if (isVariable(instr.op2.op)) {
00544         var2 = &getVariable(instr.op2.op);
00545     } else {
00546         temp2 = createTempVariable(VarType::VAR_INTEGER, instr.op2.op);
00547         var2 = &temp2;
00548     }
00549     zero_flag = false;
00550     less_flag = false;
00551     greater_flag = false;
00552     if ((var1->type == VarType::VAR_INTEGER || var1->type == VarType::VAR_BYTE) &&
00553         (var2->type == VarType::VAR_INTEGER || var2->type == VarType::VAR_BYTE)) {
00554         int64_t val1 = var1->var_value.int_value;

```

```

00555         int64_t val2 = var2->var_value.int_value;
00556         if (val1 == val2)
00557             zero_flag = true;
00558         else if (val1 < val2)
00559             less_flag = true;
00560         else
00561             greater_flag = true;
00562     } else if (var1->type == VarType::VAR_FLOAT && var2->type == VarType::VAR_FLOAT) {
00563         double val1 = var1->var_value.float_value;
00564         double val2 = var2->var_value.float_value;
00565         if (val1 == val2)
00566             zero_flag = true;
00567         else if (val1 < val2)
00568             less_flag = true;
00569         else
00570             greater_flag = true;
00571     } else if (var1->type == VarType::VAR_FLOAT && (var2->type == VarType::VAR_INTEGER ||
var2->type == VarType::VAR_BYTE)) {
00572         double val1 = var1->var_value.float_value;
00573         double val2 = static_cast<double>(var2->var_value.int_value);
00574         if (val1 == val2)
00575             zero_flag = true;
00576         else if (val1 < val2)
00577             less_flag = true;
00578         else
00579             greater_flag = true;
00580     } else if ((var1->type == VarType::VAR_INTEGER || var1->type == VarType::VAR_BYTE) &&
var2->type == VarType::VAR_FLOAT) {
00581         double val1 = static_cast<double>(var1->var_value.int_value);
00582         double val2 = var2->var_value.float_value;
00583         if (val1 == val2)
00584             zero_flag = true;
00585         else if (val1 < val2)
00586             less_flag = true;
00587         else
00588             greater_flag = true;
00589     } else if (var1->type == VarType::VAR_POINTER && var2->type == VarType::VAR_POINTER) {
00590         uintptr_t val1 = reinterpret_cast<uintptr_t>(var1->var_value.ptr_value);
00591         uintptr_t val2 = reinterpret_cast<uintptr_t>(var2->var_value.ptr_value);
00592         if (val1 == val2)
00593             zero_flag = true;
00594         else if (val1 < val2)
00595             less_flag = true;
00596         else
00597             greater_flag = true;
00598     } else if (var1->type == VarType::VAR_POINTER && (var2->type == VarType::VAR_INTEGER ||
var2->type == VarType::VAR_BYTE)) {
00599         uintptr_t val1 = reinterpret_cast<uintptr_t>(var1->var_value.ptr_value);
00600         uint64_t val2 = var2->var_value.int_value;
00601         if (val1 == val2)
00602             zero_flag = true;
00603         else if (val1 < val2)
00604             less_flag = true;
00605         else
00606             greater_flag = true;
00607     } else if ((var1->type == VarType::VAR_INTEGER || var1->type == VarType::VAR_BYTE) &&
var2->type == VarType::VAR_POINTER) {
00608         uint64_t val1 = var1->var_value.int_value;
00609         uintptr_t val2 = reinterpret_cast<uintptr_t>(var2->var_value.ptr_value);
00610         if (val1 == val2)
00611             zero_flag = true;
00612         else if (val1 < val2)
00613             less_flag = true;
00614         else
00615             greater_flag = true;
00616     } else {
00617         throw mx::Exception("cmp: unsupported type combination: " +
                                var1->toString() + " vs " + var2->toString());
00618     }
00619 }
00620 }
00621
00622 void Program::exec_jmp(const Instruction &instr) {
00623     auto it = labels.find(instr.op1.op);
00624     if (it != labels.end()) {
00625         pc = it->second.first;
00626     } else {
00627         throw mx::Exception("Label not found: " + instr.op1.op);
00628     }
00629 }
00630
00631 void Program::exec_print(const Instruction &instr) {
00632     std::string format;
00633     std::vector<Variable> tempArgs;
00634     std::vector<Variable*> args;
00635     if (isVariable(instr.op1.op)) {
00636         Variable &fmt = getVariable(instr.op1.op);
00637         if (fmt.type != VarType::VAR_STRING)

```



```

00638         throw mx::Exception("PRINT format must be a string variable");
00639         format = fmt.var_value.str_value;
00640     } else {
00641         format = instr.op1.op;
00642     }
00643     auto getArgVariable = [&](const Operand &op, VarType type = VarType::VAR_INTEGER) -> Variable
* {
00644         if (isVariable(op.op)) {
00645             return &getVariable(op.op);
00646         } else {
00647             if (op.type == OperandType::OP_VARIABLE) {
00648                 throw mx::Exception("Instruction variable not defined: " + op.op);
00649             }
00650             tempArgs.push_back(createTempVariable(type, op.op));
00651             return &tempArgs.back();
00652         }
00653     };
00654     if (!instr.op2.op.empty()) {
00655         args.push_back(getArgVariable(instr.op2));
00656     }
00657     if (!instr.op3.op.empty()) {
00658         args.push_back(getArgVariable(instr.op3));
00659     }
00660     for (const auto &vop : instr.vop) {
00661         if (!vop.op.empty()) {
00662             args.push_back(getArgVariable(vop));
00663         }
00664     }
00665     printFormatted(format, args);
00666 }
00667
00668 void Program::exec_exit(const Instruction &instr) {
00669     int exit_code = 0;
00670     if (!instr.op1.op.empty()) {
00671         if (isVariable(instr.op1.op)) {
00672             exit_code = getVariable(instr.op1.op).var_value.int_value;
00673         } else {
00674             exit_code = std::stoll(instr.op1.op, nullptr, 0);
00675         }
00676     }
00677     exitCode = exit_code;
00678     stop();
00679 }
00680
00681 Variable &Program::getVariable(const std::string &n) {
00682     auto rax_pos = n.find("%");
00683     if (rax_pos != std::string::npos) {
00684         auto dot = n.find(".");
00685         if (dot != std::string::npos) {
00686             std::string n_ = n.substr(dot + 1);
00687             auto e = vars.find(n_);
00688             if (e != vars.end())
00689                 return e->second;
00690         } else {
00691             auto e = vars.find(n);
00692             if (e != vars.end())
00693                 return e->second;
00694         }
00695     }
00696     auto pos = n.find(".");
00697     if (pos == std::string::npos) {
00698         auto it = vars.find(name + "." + n);
00699         if (it != vars.end()) {
00700             return it->second;
00701         }
00702     } else {
00703         auto it = vars.find(n);
00704         if (it != vars.end())
00705             return it->second;
00706     }
00707
00708     if (Program::base != nullptr) {
00709         for (auto &obj : Base::base->object_map) {
00710             if (pos == std::string::npos) {
00711                 auto it = obj.second->vars.find(name + "." + n);
00712                 if (it != obj.second->vars.end())
00713                     return it->second;
00714             } else {
00715                 auto it = obj.second->vars.find(n);
00716                 if (it != obj.second->vars.end())
00717                     return it->second;
00718             }
00719         }
00720     }
00721     throw mx::Exception("Variable not found: " + name + "." + n);
00722 }
00723

```

```

00724     bool Program::isVariable(const std::string &n) {
00725
00726         auto rax_pos = n.find("%");
00727
00728         if (rax_pos != std::string::npos) {
00729             return true;
00730         }
00731
00732         if (Program::base != nullptr) {
00733             for (auto &obj : Base::base->object_map) {
00734                 auto it = obj.second->vars.find(n);
00735                 if (it != obj.second->vars.end()) {
00736                     return true;
00737                 }
00738
00739                 it = obj.second->vars.find(name + "." + n);
00740                 if (it != obj.second->vars.end())
00741                     return true;
00742             }
00743         }
00744
00745         if (vars.find(name + "." + n) != vars.end())
00746             return true;
00747
00748         if (vars.find(n) != vars.end())
00749             return true;
00750
00751         return false;
00752     }
00753
00754     bool Program::validateNames(Validator &v) {
00755         for (auto &variable : v.var_names) {
00756             auto it = vars.find(variable.second->getTokenValue());
00757             if (it == vars.end()) {
00758                 for (auto &o : objects) {
00759                     if (o->name == variable.first && !o->isVariable(variable.second->getTokenValue()))
00760                     {
00761                         throw mx::Exception("Syntax Error: Argument variable not defined: Object: " +
00762                             variable.first + " variable: " +
00763                             variable.second->getTokenValue() +
00764                             " at line " + std::to_string(variable.second->getLine()));
00765                     }
00766                 }
00767             }
00768             return true;
00769         }
00770
00771     void Program::setVariableFromString(Variable &var, const std::string &value) {
00772         if (var.type == VarType::VAR_INTEGER) {
00773             if (value.starts_with("0x") || value.starts_with("0X")) {
00774                 var.var_value.int_value = std::stoll(value, nullptr, 16);
00775             } else {
00776                 var.var_value.int_value = std::stoll(value);
00777             }
00778             var.var_value.type = VarType::VAR_INTEGER;
00779         } else if (var.type == VarType::VAR_FLOAT) {
00780             var.var_value.float_value = std::stod(value);
00781             var.var_value.type = VarType::VAR_FLOAT;
00782         } else if (var.type == VarType::VAR_STRING) {
00783             var.var_value.str_value = value;
00784             var.var_value.type = VarType::VAR_STRING;
00785         }
00786     }
00787
00788     void Program::addVariables(Variable &dest, Variable &src1, Variable &src2) {
00789         if (dest.type == VarType::VAR_POINTER) {
00790             char *base = static_cast<char *>(src1.var_value.ptr_value);
00791             int64_t offset = (src2.type == VarType::VAR_FLOAT) ?
00792                 static_cast<int64_t>(src2.var_value.float_value) : src2.var_value.int_value;
00793             dest.var_value.ptr_value = base + offset;
00794             dest.var_value.type = VarType::VAR_POINTER;
00795             dest.var_value.owns = false;
00796         } else if (dest.type == VarType::VAR_INTEGER) {
00797             int64_t v1 = (src1.type == VarType::VAR_FLOAT) ?
00798                 static_cast<int64_t>(src1.var_value.float_value) : src1.var_value.int_value;
00799             int64_t v2 = (src2.type == VarType::VAR_FLOAT) ?
00800                 static_cast<int64_t>(src2.var_value.float_value) : src2.var_value.int_value;
00801             dest.var_value.int_value = v1 + v2;
00802             dest.var_value.type = VarType::VAR_INTEGER;
00803         } else if (dest.type == VarType::VAR_FLOAT) {
00804             double v1 = (src1.type == VarType::VAR_INTEGER) ?
00805                 static_cast<double>(src1.var_value.int_value) : src1.var_value.float_value;
00806             double v2 = (src2.type == VarType::VAR_INTEGER) ?
00807                 static_cast<double>(src2.var_value.int_value) : src2.var_value.float_value;
00808             dest.var_value.float_value = v1 + v2;
00809             dest.var_value.type = VarType::VAR_FLOAT;
00810         }
00811     }

```

```

00805     }
00806
00807     void Program::subVariables(Variable &dest, Variable &src1, Variable &src2) {
00808         if (dest.type == VarType::VAR_POINTER) {
00809             char *base = static_cast<char *>(src1.var_value.ptr_value);
00810             int64_t offset = (src2.type == VarType::VAR_FLOAT) ?
static_cast<int64_t>(src2.var_value.float_value) : src2.var_value.int_value;
00811             dest.var_value.ptr_value = base - offset;
00812             dest.var_value.type = VarType::VAR_POINTER;
00813             dest.var_value.owns = false;
00814         } else if (dest.type == VarType::VAR_INTEGER) {
00815             int64_t v1 = (src1.type == VarType::VAR_FLOAT) ?
static_cast<int64_t>(src1.var_value.float_value) : src1.var_value.int_value;
00816             int64_t v2 = (src2.type == VarType::VAR_FLOAT) ?
static_cast<int64_t>(src2.var_value.float_value) : src2.var_value.int_value;
00817             dest.var_value.int_value = v1 - v2;
00818             dest.var_value.type = VarType::VAR_INTEGER;
00819         } else if (dest.type == VarType::VAR_FLOAT) {
00820             double v1 = (src1.type == VarType::VAR_INTEGER) ?
static_cast<double>(src1.var_value.int_value) : src1.var_value.float_value;
00821             double v2 = (src2.type == VarType::VAR_INTEGER) ?
static_cast<double>(src2.var_value.int_value) : src2.var_value.float_value;
00822             dest.var_value.float_value = v1 - v2;
00823             dest.var_value.type = VarType::VAR_FLOAT;
00824         }
00825     }
00826
00827     void Program::mulVariables(Variable &dest, Variable &src1, Variable &src2) {
00828         if (dest.type == VarType::VAR_INTEGER) {
00829             int64_t v1 = (src1.type == VarType::VAR_FLOAT) ?
static_cast<int64_t>(src1.var_value.float_value) : src1.var_value.int_value;
00830             int64_t v2 = (src2.type == VarType::VAR_FLOAT) ?
static_cast<int64_t>(src2.var_value.float_value) : src2.var_value.int_value;
00831             dest.var_value.int_value = v1 * v2;
00832             dest.var_value.type = VarType::VAR_INTEGER;
00833         } else if (dest.type == VarType::VAR_FLOAT) {
00834             double v1 = (src1.type == VarType::VAR_INTEGER) ?
static_cast<double>(src1.var_value.int_value) : src1.var_value.float_value;
00835             double v2 = (src2.type == VarType::VAR_INTEGER) ?
static_cast<double>(src2.var_value.int_value) : src2.var_value.float_value;
00836             dest.var_value.float_value = v1 * v2;
00837             dest.var_value.type = VarType::VAR_FLOAT;
00838         }
00839     }
00840
00841     void Program::divVariables(Variable &dest, Variable &src1, Variable &src2) {
00842         if (dest.type == VarType::VAR_INTEGER) {
00843             int64_t v1 = (src1.type == VarType::VAR_FLOAT) ?
static_cast<int64_t>(src1.var_value.float_value) : src1.var_value.int_value;
00844             int64_t v2 = (src2.type == VarType::VAR_FLOAT) ?
static_cast<int64_t>(src2.var_value.float_value) : src2.var_value.int_value;
00845             if (v2 == 0) {
00846                 throw mx::Exception("DIV: integer division by zero");
00847             }
00848             if (v1 == INT64_MIN && v2 == -1) {
00849                 throw mx::Exception("DIV: signed integer overflow (INT64_MIN / -1)");
00850             }
00851             dest.var_value.int_value = v1 / v2;
00852             dest.var_value.type = VarType::VAR_INTEGER;
00853         } else if (dest.type == VarType::VAR_FLOAT) {
00854             double v1 = (src1.type == VarType::VAR_INTEGER) ?
static_cast<double>(src1.var_value.int_value) : src1.var_value.float_value;
00855             double v2 = (src2.type == VarType::VAR_INTEGER) ?
static_cast<double>(src2.var_value.int_value) : src2.var_value.float_value;
00856             if (v2 == 0.0) {
00857                 throw mx::Exception("DIV: floating-point division by zero");
00858             }
00859             dest.var_value.float_value = v1 / v2;
00860             dest.var_value.type = VarType::VAR_FLOAT;
00861         }
00862     }
00863
00864     void Program::exec_load(const Instruction &instr) {
00865         if (!isVariable(instr.op1.op)) {
00866             throw mx::Exception("LOAD destination must be a variable");
00867         }
00868         Variable &dest = getVariable(instr.op1.op);
00869         void *ptr = nullptr;
00870         size_t allocated_size = 0;
00871
00872         bool is_string_source = false;
00873
00874         if (isVariable(instr.op2.op)) {
00875             Variable &ptrVar = getVariable(instr.op2.op);
00876             if (ptrVar.type != VarType::VAR_POINTER && ptrVar.type != VarType::VAR_STRING) {
00877                 throw mx::Exception("LOAD source must be a valid pointer/string buffer");
00878             }

```

```

00879         if (ptrVar.type == VarType::VAR_POINTER) {
00880             ptr = ptrVar.var_value.ptr_value;
00881             allocated_size = ptrVar.var_value.ptr_size * ptrVar.var_value.ptr_count;
00882         } else {
00883
00884             ptr = (void *)ptrVar.var_value.str_value.data();
00885             allocated_size = ptrVar.var_value.str_value.size() + 1;
00886             is_string_source = true;
00887         }
00888     } else {
00889         throw mx::Exception("LOAD source must be a pointer variable");
00890     }
00891
00892     if (ptr == nullptr) {
00893
00894         switch (dest.type) {
00895             case VarType::VAR_INTEGER:
00896             case VarType::VAR_BYTE:
00897                 dest.var_value.int_value = 0;
00898                 break;
00899             case VarType::VAR_FLOAT:
00900                 dest.var_value.float_value = 0.0;
00901                 break;
00902             case VarType::VAR_POINTER:
00903                 dest.var_value.ptr_value = nullptr;
00904                 break;
00905             case VarType::VAR_STRING:
00906                 dest.var_value.str_value = "";
00907                 break;
00908             default:
00909
00910                 break;
00911         }
00912         dest.var_value.type = dest.type;
00913         return;
00914     }
00915
00916     size_t index = 0;
00917     if (!instr.op3.op.empty()) {
00918         if (isVariable(instr.op3.op)) {
00919             index = static_cast<size_t>(getVariable(instr.op3.op).var_value.int_value);
00920         } else {
00921             index = static_cast<size_t>(std::stoll(instr.op3.op, nullptr, 0));
00922         }
00923     }
00924
00925     size_t stride = 8;
00926     if (!instr.vop.empty() && !instr.vop[0].op.empty()) {
00927         const auto &sizeOp = instr.vop[0];
00928         if (isVariable(sizeOp.op)) {
00929             stride = static_cast<size_t>(getVariable(sizeOp.op).var_value.int_value);
00930         } else {
00931             stride = static_cast<size_t>(std::stoll(sizeOp.op, nullptr, 0));
00932         }
00933     }
00934
00935     if (stride == 0) {
00936         throw mx::Exception("LOAD: stride must be non-zero");
00937     }
00938     if (index > SIZE_MAX / stride) {
00939         throw mx::Exception("LOAD: index * stride overflow");
00940     }
00941     size_t offset = index * stride;
00942     if (allocated_size > 0 && (offset + stride > allocated_size)) {
00943         throw mx::Exception("LOAD: index out of bounds, offset " + std::to_string(offset) +
00944             " + stride " + std::to_string(stride) +
00945             " exceeds allocated size " + std::to_string(allocated_size));
00946     }
00947
00948     try {
00949         char *base = static_cast<char *>(ptr) + offset;
00950
00951         switch (dest.type) {
00952             case VarType::VAR_INTEGER:
00953                 std::memcpy(&dest.var_value.int_value, base, sizeof(int64_t));
00954                 dest.var_value.type = VarType::VAR_INTEGER;
00955                 break;
00956             case VarType::VAR_BYTE:
00957                 dest.var_value.int_value = 0;
00958                 std::memcpy(&dest.var_value.int_value, base, 1);
00959                 dest.var_value.type = VarType::VAR_BYTE;
00960                 break;
00961             case VarType::VAR_FLOAT:
00962                 std::memcpy(&dest.var_value.float_value, base, sizeof(double));
00963                 dest.var_value.type = VarType::VAR_FLOAT;
00964                 break;
00965             case VarType::VAR_POINTER:

```

```

00966         std::memcpy(&dest.var_value.ptr_value, base, sizeof(void *));
00967         dest.var_value.type = VarType::VAR_POINTER;
00968         dest.var_value.owns = false;
00969         break;
00970     case VarType::VAR_STRING:
00971         if (is_string_source && index < allocated_size) {
00972             dest.var_value.str_value = std::string(1, *(base));
00973         } else {
00974             const char *str_ptr = nullptr;
00975             std::memcpy(&str_ptr, base, sizeof(const char *));
00976             if (str_ptr != nullptr) {
00977                 dest.var_value.str_value = str_ptr;
00978             } else {
00979                 dest.var_value.str_value = "";
00980             }
00981             dest.var_value.type = VarType::VAR_STRING;
00982             break;
00983         }
00984     default:
00985         throw mx::Exception("LOAD: unsupported destination type");
00986     }
00987     catch (const std::exception &e) {
00988         throw mx::Exception(std::string("LOAD: memory access error: ") + e.what());
00989     }
00990     catch (...) {
00991         throw mx::Exception("LOAD: unknown memory access error");
00992     }
00993 }
00994
00995 void Program::exec_store(const Instruction &instr) {
00996     void *ptr = nullptr;
00997     size_t allocated_size = 0;
00998     bool is_owned = false;
00999
01000     if (!isVariable(instr.op2.op)) {
01001         throw mx::Exception("STORE destination must be a variable or register");
01002     }
01003
01004     Variable &ptrVar = getVariable(instr.op2.op);
01005
01006     if (ptrVar.type == VarType::VAR_POINTER) {
01007         if (ptrVar.var_value.ptr_value == nullptr) {
01008             throw mx::Exception("STORE destination pointer is null: " + ptrVar.var_name);
01009         }
01010         ptr = ptrVar.var_value.ptr_value;
01011         allocated_size = ptrVar.var_value.ptr_size * ptrVar.var_value.ptr_count;
01012         is_owned = ptrVar.var_value.owns;
01013     } else if (ptrVar.type == VarType::VAR_INTEGER) {
01014         if (ptrVar.var_value.int_value == 0) {
01015             throw mx::Exception("STORE destination contains null pointer: " + ptrVar.var_name);
01016         }
01017         ptr = reinterpret_cast<void *>(static_cast<uintptr_t>(ptrVar.var_value.int_value));
01018     } else {
01019         throw mx::Exception("STORE destination must be a pointer or integer containing a pointer
01020 address");
01021     }
01022
01023     size_t index = 0;
01024     if (!instr.op3.op.empty()) {
01025         if (isVariable(instr.op3.op)) {
01026             index = static_cast<size_t>(getVariable(instr.op3.op).var_value.int_value);
01027         } else {
01028             index = static_cast<size_t>(std::stoll(instr.op3.op, nullptr, 0));
01029         }
01030     }
01031
01032     size_t stride = 8;
01033     if (!instr.vop.empty() && !instr.vop[0].op.empty()) {
01034         const auto &sizeOp = instr.vop[0];
01035         if (isVariable(sizeOp.op)) {
01036             stride = static_cast<size_t>(getVariable(sizeOp.op).var_value.int_value);
01037         } else {
01038             stride = static_cast<size_t>(std::stoll(sizeOp.op, nullptr, 0));
01039         }
01040     }
01041
01042     if (stride == 0) {
01043         throw mx::Exception("STORE: stride must be non-zero");
01044     }
01045     if (index > SIZE_MAX / stride) {
01046         throw mx::Exception("STORE: index * stride overflow");
01047     }
01048     size_t offset = index * stride;
01049     if (is_owned && allocated_size > 0 && (offset + stride > allocated_size)) {

```

```

01063         throw mx::Exception("STORE: index out of bounds, offset " + std::to_string(offset) +
01064                               " + stride " + std::to_string(stride) +
01065                               " exceeds allocated size " + std::to_string(allocated_size));
01066     }
01067
01068     char *base = static_cast<char *>(ptr) + offset;
01069
01070     if (instr.op1.type == OperandType::OP_CONSTANT) {
01071         int64_t cval = std::stoll(instr.op1.op, nullptr, 0);
01072         std::memcpy(base, &cval, sizeof(int64_t));
01073     } else {
01074         Variable &src = getVariable(instr.op1.op);
01075         switch (src.type) {
01076             case VarType::VAR_INTEGER:
01077                 std::memcpy(base, &src.var_value.int_value, sizeof(int64_t));
01078                 break;
01079             case VarType::VAR_BYTE: {
01080                 uint8_t byte_val = static_cast<uint8_t>(src.var_value.int_value & 0xFF);
01081                 std::memcpy(base, &byte_val, 1);
01082                 break;
01083             }
01084             case VarType::VAR_FLOAT:
01085                 std::memcpy(base, &src.var_value.float_value, sizeof(double));
01086                 break;
01087             case VarType::VAR_POINTER:
01088                 std::memcpy(base, &src.var_value.ptr_value, sizeof(void *));
01089                 break;
01090             case VarType::VAR_STRING: {
01091
01092                 if (is_owned && stride >= sizeof(char *)) {
01093                     char *old_str = nullptr;
01094                     std::memcpy(&old_str, base, sizeof(char *));
01095                     size_t str_len = src.var_value.str_value.size() + 1;
01096
01097                     if (old_str != nullptr) {
01098                         std::free(old_str);
01099                     }
01100
01101                     char *new_str = static_cast<char *>(std::malloc(str_len));
01102                     if (new_str == nullptr) {
01103                         throw mx::Exception("STORE: failed to allocate memory for string copy");
01104                     }
01105                     std::strncpy(new_str, src.var_value.str_value.c_str(), str_len);
01106                     new_str[str_len - 1] = '\0';
01107                     std::memcpy(base, &new_str, sizeof(char *));
01108                 } else {
01109                     const char *cstr = src.var_value.str_value.c_str();
01110                     std::memcpy(base, &cstr, sizeof(const char *));
01111                 }
01112                 break;
01113             }
01114             default:
01115                 throw mx::Exception("STORE: unsupported source type");
01116         }
01117     }
01118 }
01119
01119 void Program::exec_and(const Instruction &instr) {
01120     if (!isVariable(instr.op1.op)) {
01121         throw mx::Exception("AND destination must be a variable");
01122     }
01123     Variable &dest = getVariable(instr.op1.op);
01124     int64_t v1, v2;
01125     if (instr.op3.op.empty()) {
01126         v1 = dest.var_value.int_value;
01127         v2 = isVariable(instr.op2.op) ? getVariable(instr.op2.op).var_value.int_value :
std::stoll(instr.op2.op, nullptr, 0);
01128     } else {
01129         v1 = isVariable(instr.op2.op) ? getVariable(instr.op2.op).var_value.int_value :
std::stoll(instr.op2.op, nullptr, 0);
01130         v2 = isVariable(instr.op3.op) ? getVariable(instr.op3.op).var_value.int_value :
std::stoll(instr.op3.op, nullptr, 0);
01131     }
01132     dest.var_value.int_value = v1 & v2;
01133     dest.var_value.type = VarType::VAR_INTEGER;
01134 }
01135
01136 void Program::exec_or(const Instruction &instr) {
01137     if (!isVariable(instr.op1.op)) {
01138         throw mx::Exception("OR destination must be a variable");
01139     }
01140     Variable &dest = getVariable(instr.op1.op);
01141     int64_t v1, v2;
01142     if (instr.op3.op.empty()) {
01143         v1 = dest.var_value.int_value;
01144         v2 = isVariable(instr.op2.op) ? getVariable(instr.op2.op).var_value.int_value :
std::stoll(instr.op2.op, nullptr, 0);
01145     } else {

```

```

01146         v1 = isVariable(instr.op2.op) ? getVariable(instr.op2.op).var_value.int_value :
std::stoll(instr.op2.op, nullptr, 0);
01147         v2 = isVariable(instr.op3.op) ? getVariable(instr.op3.op).var_value.int_value :
std::stoll(instr.op3.op, nullptr, 0);
01148     }
01149     dest.var_value.int_value = v1 | v2;
01150     dest.var_value.type = VarType::VAR_INTEGER;
01151 }
01152
01153 void Program::exec_xor(const Instruction &instr) {
01154     if (!isVariable(instr.op1.op)) {
01155         throw mx::Exception("XOR destination must be a variable");
01156     }
01157     Variable &dest = getVariable(instr.op1.op);
01158     int64_t v1, v2;
01159     if (instr.op3.op.empty()) {
01160         v1 = dest.var_value.int_value;
01161         v2 = isVariable(instr.op2.op) ? getVariable(instr.op2.op).var_value.int_value :
std::stoll(instr.op2.op, nullptr, 0);
01162     } else {
01163         v1 = isVariable(instr.op2.op) ? getVariable(instr.op2.op).var_value.int_value :
std::stoll(instr.op2.op, nullptr, 0);
01164         v2 = isVariable(instr.op3.op) ? getVariable(instr.op3.op).var_value.int_value :
std::stoll(instr.op3.op, nullptr, 0);
01165     }
01166     dest.var_value.int_value = v1 ^ v2;
01167     dest.var_value.type = VarType::VAR_INTEGER;
01168 }
01169
01170 void Program::exec_alloc(const Instruction &instr) {
01171     if (!isVariable(instr.op1.op)) {
01172         throw mx::Exception("ALLOC destination must be a variable");
01173     }
01174     Variable &dest = getVariable(instr.op1.op);
01175     int64_t size = 0;
01176     int64_t count = 1;
01177
01178     if (!instr.op2.op.empty()) {
01179         if (isVariable(instr.op2.op)) {
01180             size = getVariable(instr.op2.op).var_value.int_value;
01181         } else {
01182             size = std::stoll(instr.op2.op, nullptr, 0);
01183         }
01184     }
01185
01186     if (!instr.op3.op.empty()) {
01187         if (isVariable(instr.op3.op)) {
01188             count = getVariable(instr.op3.op).var_value.int_value;
01189         } else {
01190             count = std::stoll(instr.op3.op, nullptr, 0);
01191         }
01192     }
01193     if (size <= 0 || count <= 0) {
01194         throw mx::Exception("ALLOC: size and count must be positive");
01195     }
01196     releaseOwnedPointer(dest);
01197     dest.type = VarType::VAR_POINTER;
01198     dest.var_value.type = VarType::VAR_POINTER;
01199     dest.var_value.ptr_value = calloc(static_cast<size_t>(count), static_cast<size_t>(size));
01200     if (dest.var_value.ptr_value == nullptr) {
01201         throw mx::Exception("ALLOC failed: calloc returned nullptr");
01202     }
01203     dest.var_value.ptr_size = size;
01204     dest.var_value.ptr_count = count;
01205     dest.var_value.owns = true;
01206 }
01207 void Program::exec_free(const Instruction &instr) {
01208     if (!isVariable(instr.op1.op)) {
01209         throw mx::Exception("FREE argument must be a variable");
01210     }
01211     Variable &var = getVariable(instr.op1.op);
01212     if (var.type == VarType::VAR_POINTER && var.var_value.ptr_value != nullptr) {
01213         void *ptr_to_free = var.var_value.ptr_value;
01214         std::free(ptr_to_free);
01215         // Clear this variable
01216         var.var_value.ptr_value = nullptr;
01217         var.var_value.owns = false;
01218         var.var_value.ptr_size = 0;
01219         var.var_value.ptr_count = 0;
01220         // Clear any other variables pointing to the same address to prevent double-free
01221         for (auto &kv : vars) {
01222             if (&kv.second != &var &&
01223                 kv.second.type == VarType::VAR_POINTER &&
01224                 kv.second.var_value.ptr_value == ptr_to_free) {
01225                 kv.second.var_value.ptr_value = nullptr;
01226                 kv.second.var_value.owns = false;
01227                 kv.second.var_value.ptr_size = 0;

```

```

01228         kv.second.var_value.ptr_count = 0;
01229     }
01230 }
01231 }
01232 }
01233
01234 void Program::exec_lea(const Instruction &instr) {
01235     if (!isVariable(instr.op1.op))
01236         throw mx::Exception("LEA destination must be a variable");
01237     if (!isVariable(instr.op2.op))
01238         throw mx::Exception("LEA source must be a variable");
01239
01240     Variable &dest = getVariable(instr.op1.op);
01241     Variable &src = getVariable(instr.op2.op);
01242
01243     dest.type = VarType::VAR_POINTER;
01244     dest.var_value.type = VarType::VAR_POINTER;
01245
01246     switch (src.type) {
01247     case VarType::VAR_INTEGER:
01248     case VarType::VAR_BYTE:
01249         dest.var_value.ptr_value = &src.var_value.int_value;
01250         dest.var_value.ptr_size = sizeof(int64_t);
01251         break;
01252     case VarType::VAR_FLOAT:
01253         dest.var_value.ptr_value = &src.var_value.float_value;
01254         dest.var_value.ptr_size = sizeof(double);
01255         break;
01256     case VarType::VAR_POINTER:
01257         dest.var_value.ptr_value = &src.var_value.ptr_value;
01258         dest.var_value.ptr_size = sizeof(void *);
01259         break;
01260     default:
01261         dest.var_value.ptr_value = &src.var_value.int_value;
01262         dest.var_value.ptr_size = sizeof(int64_t);
01263         break;
01264     }
01265     dest.var_value.ptr_count = 1;
01266     dest.var_value.owns = false;
01267 }
01268
01269 void Program::exec_not(const Instruction &instr) {
01270     if (!isVariable(instr.op1.op)) {
01271         throw mx::Exception("NOT destination must be a variable");
01272     }
01273     Variable &dest = getVariable(instr.op1.op);
01274     if (dest.var_value.type != VarType::VAR_INTEGER) {
01275         throw mx::Exception("Error NOT bitwise operation must be on integer value");
01276     }
01277     dest.var_value.int_value = (dest.var_value.int_value == 0) ? 1 : 0;
01278     dest.var_value.type = VarType::VAR_INTEGER;
01279 }
01280
01281 void Program::exec_mod(const Instruction &instr) {
01282     if (!isVariable(instr.op1.op)) {
01283         std::cerr << "Error: MOD destination must be a variable, not a constant\n";
01284         return;
01285     }
01286     Variable &dest = getVariable(instr.op1.op);
01287
01288     Variable *src1 = nullptr, *src2 = nullptr;
01289     Variable temp1, temp2;
01290
01291     if (instr.op3.op.empty()) {
01292         src1 = &dest;
01293         if (isVariable(instr.op2.op)) {
01294             src2 = &getVariable(instr.op2.op);
01295         } else {
01296             temp2 = createTempVariable(dest.type, instr.op2.op);
01297             src2 = &temp2;
01298         }
01299     } else {
01300         if (isVariable(instr.op2.op)) {
01301             src1 = &getVariable(instr.op2.op);
01302         } else {
01303             temp1 = createTempVariable(dest.type, instr.op2.op);
01304             src1 = &temp1;
01305         }
01306         if (isVariable(instr.op3.op)) {
01307             src2 = &getVariable(instr.op3.op);
01308         } else {
01309             temp2 = createTempVariable(dest.type, instr.op3.op);
01310             src2 = &temp2;
01311         }
01312     }
01313
01314     if (dest.type == VarType::VAR_INTEGER || dest.type == VarType::VAR_BYTE) {

```



```

01315         int64_t v1 = (src1->type == VarType::VAR_FLOAT) ?
static_cast<int64_t>(src1->var_value.float_value) : src1->var_value.int_value;
01316         int64_t v2 = (src2->type == VarType::VAR_FLOAT) ?
static_cast<int64_t>(src2->var_value.float_value) : src2->var_value.int_value;
01317         if (v2 == 0) {
01318             throw mx::Exception("MOD: integer modulo by zero");
01319         } else {
01320             dest.var_value.int_value = v1 % v2;
01321         }
01322         dest.var_value.type = VarType::VAR_INTEGER;
01323     } else {
01324         std::cerr << "MOD only supports integer types\n";
01325     }
01326 }
01327
01328 void Program::exec_je(const Instruction &instr) {
01329     if (zero_flag)
01330         exec_jump(instr);
01331     else
01332         pc++;
01333 }
01334
01335 void Program::exec_jne(const Instruction &instr) {
01336     if (!zero_flag)
01337         exec_jump(instr);
01338     else
01339         pc++;
01340 }
01341
01342 void Program::exec_jl(const Instruction &instr) {
01343     if (less_flag)
01344         exec_jump(instr);
01345     else
01346         pc++;
01347 }
01348
01349 void Program::exec_jle(const Instruction &instr) {
01350     if (less_flag || zero_flag)
01351         exec_jump(instr);
01352     else
01353         pc++;
01354 }
01355
01356 void Program::exec_jg(const Instruction &instr) {
01357     if (greater_flag)
01358         exec_jump(instr);
01359     else
01360         pc++;
01361 }
01362
01363 void Program::exec_jge(const Instruction &instr) {
01364     if (greater_flag || zero_flag)
01365         exec_jump(instr);
01366     else
01367         pc++;
01368 }
01369
01370 void Program::exec_jz(const Instruction &instr) {
01371     if (zero_flag)
01372         exec_jump(instr);
01373     else
01374         pc++;
01375 }
01376
01377 void Program::exec_jnz(const Instruction &instr) {
01378     if (!zero_flag)
01379         exec_jump(instr);
01380     else
01381         pc++;
01382 }
01383
01384 void Program::exec_ja(const Instruction &instr) {
01385     if (greater_flag)
01386         exec_jump(instr);
01387     else
01388         pc++;
01389 }
01390
01391 void Program::exec_jb(const Instruction &instr) {
01392     if (less_flag)
01393         exec_jump(instr);
01394     else
01395         pc++;
01396 }
01397
01398 void Program::exec_string_print(const Instruction &instr) {
01399     if (!isVariable(instr.opl.op)) {

```

```

01400         throw mx::Exception("STRING_PRINT destination must be a variable");
01401     }
01402     Variable &dest = getVariable(instr.op1.op);
01403
01404     if (dest.type != VarType::VAR_STRING) {
01405         throw mx::Exception("STRING_PRINT destination must be a string variable");
01406     }
01407
01408     std::string format;
01409     std::vector<Variable> tempArgs;
01410     std::vector<Variable *> args;
01411     if (isVariable(instr.op2.op)) {
01412         Variable &fmtVar = getVariable(instr.op2.op);
01413         if (fmtVar.type != VarType::VAR_STRING)
01414             throw mx::Exception("STRING_PRINT format must be a string variable");
01415         format = fmtVar.var_value.str_value;
01416     } else {
01417         format = instr.op2.op;
01418     }
01419
01420     auto getArgVariable = [&](const Operand &op, VarType type = VarType::VAR_INTEGER) -> Variable
    * {
01421         if (isVariable(op.op)) {
01422             return &getVariable(op.op);
01423         } else {
01424             if (op.type == OperandType::OP_VARIABLE) {
01425                 throw mx::Exception("string_print instruction variable not defined: " + op.op);
01426             }
01427             tempArgs.push_back(createTempVariable(type, op.op));
01428             return &tempArgs.back();
01429         }
01430     };
01431
01432     if (!instr.op3.op.empty()) {
01433         args.push_back(getArgVariable(instr.op3));
01434     }
01435     for (const auto &vop : instr.vop) {
01436         if (!vop.op.empty()) {
01437             args.push_back(getArgVariable(vop));
01438         }
01439     }
01440
01441     std::ostringstream oss;
01442     size_t argIndex = 0;
01443     const char *fmt = format.c_str();
01444     char buffer[4096];
01445
01446     for (size_t i = 0; i < format.length(); ++i) {
01447         if (fmt[i] == '%' && i + 1 < format.length()) {
01448             size_t start = i;
01449             size_t j = i + 1;
01450             while (j < format.length() &&
01451                 (fmt[j] == '-' || fmt[j] == '+' || fmt[j] == ' ' || fmt[j] == '#' || fmt[j] ==
01452                  '0' ||
01453                  (fmt[j] >= '0' && fmt[j] <= '9') || fmt[j] == '.' ||
01454                  fmt[j] == 'l' || fmt[j] == 'h' || fmt[j] == 'z' || fmt[j] == 'j' || fmt[j] ==
01455                  't')) {
01456                 ++j;
01457             }
01458             if (j < format.length() && fmt[j] == '%') {
01459                 oss << '%';
01460                 i = j;
01461                 continue;
01462             }
01463             if (j >= format.length())
01464                 break;
01465             if (!std::isalpha(fmt[j])) {
01466                 oss << fmt[i];
01467                 continue;
01468             }
01469             std::string spec(fmt + start, fmt + j + 1);
01470             if (argIndex < args.size()) {
01471                 Variable *arg = args[argIndex++];
01472                 if (arg->type == VarType::VAR_INTEGER) {
01473                     std::snprintf(buffer, sizeof(buffer), spec.c_str(), arg->var_value.int_value);
01474                 } else if (arg->type == VarType::VAR_POINTER || arg->var_value.type ==
01475                     VarType::VAR_EXTERN) {
01476                     if (spec.find("%s") != std::string::npos) {
01477                         std::snprintf(buffer, sizeof(buffer), spec.c_str(), static_cast<char
01478                             *>(arg->var_value.ptr_value));
01479                     } else {
01480                         std::snprintf(buffer, sizeof(buffer), spec.c_str(),
01481                             arg->var_value.ptr_value);
01482                     }
01483                 } else if (arg->type == VarType::VAR_FLOAT) {
01484                     std::snprintf(buffer, sizeof(buffer), spec.c_str(),
01485                         arg->var_value.float_value);
01486                 }
01487             }
01488             oss << buffer;
01489         }
01490     }

```

```

01480         } else if (arg->type == VarType::VAR_STRING) {
01481             std::snprintf(buffer, sizeof(buffer), spec.c_str(),
arg->var_value.str_value.c_str());
01482         } else if (arg->type == VarType::VAR_BYTE) {
01483             std::snprintf(buffer, sizeof(buffer), spec.c_str(), arg->var_value.int_value);
01484         } else {
01485             std::snprintf(buffer, sizeof(buffer), "%s", "(unsupported)");
01486         }
01487         oss << buffer;
01488     } else {
01489         oss << spec;
01490     }
01491     i = j;
01492 } else {
01493     oss << fmt[i];
01494 }
01495 }
01496
01497 dest.var_value.str_value = oss.str();
01498 dest.var_value.type = VarType::VAR_STRING;
01499 }
01500
01501 std::string Program::printFormatted(const std::string &format, const std::vector<Variable *>
&args, bool output) {
01502     std::ostringstream oss;
01503     size_t argIndex = 0;
01504     const char *fmt = format.c_str();
01505     char buffer[4096];
01506     for (size_t i = 0; i < format.length(); ++i) {
01507         if (fmt[i] == '%' && i + 1 < format.length()) {
01508             size_t start = i;
01509             size_t j = i + 1;
01510             while (j < format.length() &&
01511                 (fmt[j] == '-' || fmt[j] == '+' || fmt[j] == ' ' || fmt[j] == '#' || fmt[j] ==
01512                 '0' ||
01513                 (fmt[j] >= '0' && fmt[j] <= '9') || fmt[j] == '.' ||
01514                 fmt[j] == 'l' || fmt[j] == 'h' || fmt[j] == 'z' || fmt[j] == 'j' || fmt[j] ==
01515                 't')) {
01516                 ++j;
01517             }
01518             if (j < format.length() && fmt[j] == '%') {
01519                 oss << '%';
01520                 i = j;
01521                 continue;
01522             }
01523             if (j >= format.length())
01524                 break;
01525             if (!std::isalpha(fmt[j])) {
01526                 oss << fmt[i];
01527                 continue;
01528             }
01529             std::string spec(fmt + start, fmt + j + 1);
01530             if (argIndex < args.size()) {
01531                 Variable *arg = args[argIndex++];
01532                 if (arg->type == VarType::VAR_INTEGER) {
01533                     std::snprintf(buffer, sizeof(buffer), spec.c_str(), arg->var_value.int_value);
01534                 } else if (arg->type == VarType::VAR_POINTER || arg->var_value.type ==
VarType::VAR_EXTERN) {
01535                     if (spec.find("%s") != std::string::npos) {
01536                         std::snprintf(buffer, sizeof(buffer), spec.c_str(), static_cast<char
*>(arg->var_value.ptr_value));
01537                     } else {
01538                         std::snprintf(buffer, sizeof(buffer), spec.c_str(),
arg->var_value.ptr_value);
01539                     }
01540                 } else if (arg->type == VarType::VAR_FLOAT) {
01541                     std::snprintf(buffer, sizeof(buffer), spec.c_str(),
arg->var_value.float_value);
01542                 } else if (arg->type == VarType::VAR_STRING) {
01543                     std::snprintf(buffer, sizeof(buffer), spec.c_str(),
arg->var_value.str_value.c_str());
01544                 } else if (arg->type == VarType::VAR_BYTE) {
01545                     std::snprintf(buffer, sizeof(buffer), spec.c_str(), arg->var_value.int_value);
01546                 } else {
01547                     std::snprintf(buffer, sizeof(buffer), "%s", "(unsupported)");
01548                 }
01549                 oss << buffer;
01550             } else {
01551                 oss << spec;
01552             }
01553             i = j;
01554         } else {
01555             oss << fmt[i];
01556         }
01557     }
    if (output)

```

```

01558         std::cout << oss.str();
01559         return oss.str();
01560     }
01561     void Program::exec_getline(const Instruction &instr) {
01562         if (!isVariable(instr.opl.op)) {
01563             throw mx::Exception("GETLINE destination must be a variable");
01564         }
01565         Variable &dest = getVariable(instr.opl.op);
01566         std::string input;
01567         std::getline(std::cin, input);
01568
01569         switch (dest.type) {
01570             case VarType::VAR_STRING:
01571                 dest.var_value.str_value = input;
01572                 dest.var_value.type = VarType::VAR_STRING;
01573                 break;
01574             default:
01575                 throw mx::Exception("GETLINE: unsupported variable type");
01576         }
01577     }
01578
01588     void Program::exec_push(const Instruction &instr) {
01589         if (!isVariable(instr.opl.op)) {
01590             try {
01591                 int64_t int_val = std::stoll(instr.opl.op, nullptr, 0);
01592                 stack.push(int_val);
01593                 return;
01594             } catch (...) {
01595                 throw mx::Exception("PUSH argument must be a variable or integer constant");
01596             }
01597         }
01598         Variable &var = getVariable(instr.opl.op);
01599         if (var.var_value.type == VarType::VAR_INTEGER || var.var_value.type == VarType::VAR_BYTE) {
01600             stack.push(var.var_value.int_value);
01601         } else if (var.var_value.type == VarType::VAR_POINTER || var.var_value.type ==
01602 VarType::VAR_EXTERN) {
01603             stack.push(var.var_value.ptr_value);
01604         } else if (var.var_value.type == VarType::VAR_FLOAT) {
01605             stack.push(var.var_value.float_value);
01606         } else if (var.var_value.type == VarType::VAR_STRING) {
01607             stack.push(var.var_value.str_value);
01608         } else {
01609             throw mx::Exception("PUSH: unsupported variable type");
01610         }
01611     }
01621     void Program::exec_pop(const Instruction &instr) {
01622         if (!isVariable(instr.opl.op)) {
01623             throw mx::Exception("POP destination must be a variable");
01624         }
01625         Variable &var = getVariable(instr.opl.op);
01626
01627         if (stack.empty()) {
01628             throw mx::Exception("POP from empty stack");
01629         }
01630
01631         StackValue value = stack.pop();
01632         if (var.type == VarType::VAR_INTEGER || var.type == VarType::VAR_BYTE) {
01633             if (!std::holds_alternative<int64_t>(value)) {
01634                 throw mx::Exception("POP type mismatch: expected integer");
01635             }
01636             var.var_value.int_value = std::get<int64_t>(value);
01637             var.var_value.type = var.type;
01638         } else if (var.type == VarType::VAR_POINTER || var.type == VarType::VAR_EXTERN) {
01639             if (!std::holds_alternative<void *>(value)) {
01640                 throw mx::Exception("POP type mismatch: expected pointer");
01641             }
01642             var.var_value.ptr_value = std::get<void *>(value);
01643             var.var_value.type = var.type;
01644         } else if (var.type == VarType::VAR_FLOAT) {
01645             if (!std::holds_alternative<double>(value)) {
01646                 throw mx::Exception("POP type mismatch: expected float");
01647             }
01648             var.var_value.float_value = std::get<double>(value);
01649             var.var_value.type = VarType::VAR_FLOAT;
01650         } else if (var.type == VarType::VAR_STRING) {
01651             if (!std::holds_alternative<std::string>(value)) {
01652                 throw mx::Exception("POP type mismatch: expected string");
01653             }
01654             var.var_value.str_value = std::get<std::string>(value);
01655             var.var_value.type = VarType::VAR_STRING;
01656         } else {
01657             throw mx::Exception("POP: unsupported variable type");
01658         }
01659     }
01660
01661     void Program::exec_stack_load(const Instruction &instr) {

```

```

01662         if (!isVariable(instr.op1.op)) {
01663             throw mx::Exception("STACK_LOAD destination must be a variable");
01664         }
01665         Variable &dest = getVariable(instr.op1.op);
01666
01667         size_t index = 0;
01668         if (!instr.op2.op.empty()) {
01669             if (isVariable(instr.op2.op)) {
01670                 index = static_cast<size_t>(getVariable(instr.op2.op).var_value.int_value);
01671             } else {
01672                 index = static_cast<size_t>(std::stoll(instr.op2.op, nullptr, 0));
01673             }
01674         }
01675
01676         if (index >= stack.size()) {
01677             throw mx::Exception("STACK_LOAD: index out of bounds");
01678         }
01679
01680         StackValue &value = stack[index];
01681         if (dest.type == VarType::VAR_INTEGER || dest.type == VarType::VAR_BYTE) {
01682             if (!std::holds_alternative<int64_t>(value)) {
01683                 throw mx::Exception("STACK_LOAD type mismatch: expected integer");
01684             }
01685             dest.var_value.int_value = std::get<int64_t>(value);
01686             dest.var_value.type = dest.type;
01687         } else if (dest.type == VarType::VAR_POINTER || dest.type == VarType::VAR_EXTERN) {
01688             if (!std::holds_alternative<void *>(value)) {
01689                 throw mx::Exception("STACK_LOAD type mismatch: expected pointer");
01690             }
01691             dest.var_value.ptr_value = std::get<void *>(value);
01692             dest.var_value.type = dest.type;
01693         } else if (dest.type == VarType::VAR_FLOAT) {
01694             if (!std::holds_alternative<double>(value)) {
01695                 throw mx::Exception("STACK_LOAD type mismatch: expected float");
01696             }
01697             dest.var_value.float_value = std::get<double>(value);
01698             dest.var_value.type = VarType::VAR_FLOAT;
01699         } else if (dest.type == VarType::VAR_STRING) {
01700             if (!std::holds_alternative<std::string>(value)) {
01701                 throw mx::Exception("STACK_LOAD type mismatch: expected string");
01702             }
01703             dest.var_value.str_value = std::get<std::string>(value);
01704             dest.var_value.type = VarType::VAR_STRING;
01705         } else {
01706             throw mx::Exception("STACK_LOAD: unsupported variable type");
01707         }
01708     }
01709
01710 void Program::exec_stack_store(const Instruction &instr) {
01711     if (!isVariable(instr.op1.op)) {
01712         throw mx::Exception("STACK_STORE source must be a variable");
01713     }
01714     Variable &src = getVariable(instr.op1.op);
01715
01716     size_t index = 0;
01717     if (!instr.op2.op.empty()) {
01718         if (isVariable(instr.op2.op)) {
01719             index = static_cast<size_t>(getVariable(instr.op2.op).var_value.int_value);
01720         } else {
01721             index = static_cast<size_t>(std::stoll(instr.op2.op, nullptr, 0));
01722         }
01723     }
01724
01725     if (index >= stack.size()) {
01726         throw mx::Exception("STACK_STORE: index out of bounds");
01727     }
01728     StackValue &value = stack[index];
01729     if (src.type == VarType::VAR_INTEGER || src.type == VarType::VAR_BYTE) {
01730         value = src.var_value.int_value;
01731     } else if (src.type == VarType::VAR_POINTER || src.type == VarType::VAR_EXTERN) {
01732         value = src.var_value.ptr_value;
01733     } else if (src.type == VarType::VAR_FLOAT) {
01734         value = src.var_value.float_value;
01735     } else if (src.type == VarType::VAR_STRING) {
01736         value = src.var_value.str_value;
01737     } else {
01738         throw mx::Exception("STACK_STORE: unsupported variable type");
01739     }
01740 }
01741
01742 void Program::exec_stack_sub(const Instruction &instr) {
01743     size_t count = 1;
01744     if (!instr.op1.op.empty()) {
01745         if (isVariable(instr.op1.op)) {
01746             count = static_cast<size_t>(getVariable(instr.op1.op).var_value.int_value);
01747         } else {
01748             count = static_cast<size_t>(std::stoll(instr.op1.op, nullptr, 0));

```

```

01749         }
01750     }
01751     if (count > stack.size()) {
01752         throw mx::Exception("STACK_SUB: not enough values on stack to pop " +
std::to_string(count));
01753     }
01754     for (size_t i = 0; i < count; ++i) {
01755         (void)stack.pop();
01756     }
01757 }
01758
01759 void Program::exec_call(const Instruction &instr) {
01760     if (instr.op1.op.empty()) {
01761         throw mx::Exception("CALL requires a label operand");
01762     }
01763     auto it = labels.find(instr.op1.op);
01764     if (it != labels.end()) {
01765         stack.push(static_cast<int64_t>(pc + 1));
01766         pc = it->second.first;
01767     } else {
01768         throw mx::Exception("CALL Label not found: " + instr.op1.op);
01769     }
01770 }
01771
01772 void Program::exec_ret(const Instruction &instr) {
01773     if (stack.empty()) {
01774         throw mx::Exception("RET: stack is empty, no return address");
01775     }
01776     StackValue value = stack.pop();
01777     if (!std::holds_alternative<int64_t>(value)) {
01778         throw mx::Exception("RET: return address on stack is not an integer");
01779     }
01780     pc = static_cast<size_t>(std::get<int64_t>(value)) - 1;
01781 }
01782
01783 void Program::exec_done(const Instruction &i) {
01784     exitCode = 0;
01785     stop();
01786 }
01787
01788 void Program::exec_to_int(const Instruction &instr) {
01789     if (!instr.op1.op.empty() && isVariable(instr.op1.op)) {
01790         Variable &v = getVariable(instr.op1.op);
01791         if (v.type == VarType::VAR_INTEGER) {
01792             if (isVariable(instr.op2.op)) {
01793                 Variable &s = getVariable(instr.op2.op);
01794                 if (s.type == VarType::VAR_STRING) {
01795                     try {
01796                         v.var_value.int_value = std::stoll(s.var_value.str_value, nullptr, 0);
01797                     } catch (...) {
01798                         v.var_value.int_value = 0;
01799                     }
01800                 } else if (s.type == VarType::VAR_FLOAT) {
01801                     v.var_value.int_value = static_cast<int64_t>(s.var_value.float_value);
01802                 }
01803             } else {
01804                 throw mx::Exception("to_int second argument must be a variable");
01805             }
01806         } else {
01807             throw mx::Exception("to_int first argument must be an integer variable");
01808         }
01809     } else {
01810         throw mx::Exception("to_int first argument must be a variable");
01811     }
01812 }
01813
01814 void Program::exec_to_float(const Instruction &instr) {
01815     if (!instr.op1.op.empty() && isVariable(instr.op1.op)) {
01816         Variable &v = getVariable(instr.op1.op);
01817         if (v.type == VarType::VAR_FLOAT) {
01818             if (isVariable(instr.op2.op)) {
01819                 Variable &s = getVariable(instr.op2.op);
01820                 if (s.type == VarType::VAR_STRING) {
01821                     try {
01822                         v.var_value.float_value = std::stod(s.var_value.str_value);
01823                         v.var_value.type = VarType::VAR_FLOAT;
01824                     } catch (...) {
01825                         v.var_value.float_value = 0.0;
01826                         v.var_value.type = VarType::VAR_FLOAT;
01827                     }
01828                 } else if (s.type == VarType::VAR_INTEGER || s.type == VarType::VAR_BYTE) {
01829                     v.var_value.float_value = static_cast<double>(s.var_value.int_value);
01830                     v.var_value.type = VarType::VAR_FLOAT;
01831                 } else if (s.type == VarType::VAR_FLOAT) {
01832                     v.var_value.float_value = s.var_value.float_value;
01833                     v.var_value.type = VarType::VAR_FLOAT;
01834                 } else if (s.type == VarType::VAR_POINTER || s.type == VarType::VAR_EXTERN) {

```

```

01835         v.var_value.float_value =
static_cast<double>(reinterpret_cast<uintptr_t>(s.var_value.ptr_value));
01836         v.var_value.type = VarType::VAR_FLOAT;
01837     } else {
01838         throw mx::Exception("to_float second argument must be a variable");
01839     }
01840     } else {
01841         v.var_value.float_value = static_cast<double>(std::stoll(instr.op2.op, nullptr,
01842 0));
01843         v.var_value.type = VarType::VAR_FLOAT;
01844     } else {
01845         throw mx::Exception("to_float first argument must be a float variable");
01846     }
01847 } else {
01848     throw mx::Exception("to_float first argument must be a variable");
01849 }
01850 }
01851
01852 std::string toStringFromVarType(const VarType &v) {
01853     switch (v) {
01854     case VarType::VAR_BYTE:
01855         return "Byte";
01856     case VarType::VAR_INTEGER:
01857         return "Integer";
01858     case VarType::VAR_FLOAT:
01859         return "Float";
01860     case VarType::VAR_STRING:
01861         return "String";
01862     case VarType::VAR_POINTER:
01863         return "Pointer";
01864     case VarType::VAR_EXTERN:
01865         return "Extern";
01866     default:
01867         return "Unknown";
01868     }
01869     return "Unknown";
01870 }
01871
01872 void Program::exec_return(const Instruction &instr) {
01873     if (!instr.op1.op.empty() && isVariable(instr.op1.op)) {
01874         Variable &v = getVariable(instr.op1.op);
01875         std::string name = v.var_name;
01876         Variable &r = getVariable(result.op);
01877         releaseOwnedPointer(v);
01878         v = r;
01879         v.var_name = name;
01880         if (r.type == VarType::VAR_POINTER) {
01881             r.var_value.owns = false;
01882         }
01883     }
01884 }
01885
01886 void Program::exec_neg(const Instruction &instr) {
01887     if (!instr.op1.op.empty() && isVariable(instr.op1.op)) {
01888         Variable &v = getVariable(instr.op1.op);
01889         switch (v.type) {
01890         case VarType::VAR_INTEGER:
01891             v.var_value.int_value = -v.var_value.int_value;
01892             break;
01893         case VarType::VAR_FLOAT:
01894             v.var_value.float_value = -v.var_value.float_value;
01895             break;
01896         default:
01897             throw mx::Exception("NEG: unsupported variable type");
01898         }
01899     }
01900 }
01901
01902 void Program::exec_realloc(const Instruction &instr) {
01903     if (!isVariable(instr.op1.op)) {
01904         throw mx::Exception("REALLOC destination must be a variable");
01905     }
01906     Variable &dest = getVariable(instr.op1.op);
01907     int64_t size = 0;
01908     int64_t count = 1;
01909
01910     if (!instr.op2.op.empty()) {
01911         if (isVariable(instr.op2.op)) {
01912             size = getVariable(instr.op2.op).var_value.int_value;
01913         } else {
01914             size = std::stoll(instr.op2.op, nullptr, 0);
01915         }
01916     }
01917
01918     if (!instr.op3.op.empty()) {
01919         if (isVariable(instr.op3.op)) {

```

```

01928         count = getVariable(instr.op3.op).var_value.int_value;
01929     } else {
01930         count = std::stoll(instr.op3.op, nullptr, 0);
01931     }
01932 }
01933 if (size <= 0 || count < 0) {
01934     throw mx::Exception("REALLOC: size must be positive, count must be non-negative");
01935 }
01936
01937 size_t newBytes = static_cast<size_t>(count) * static_cast<size_t>(size);
01938 size_t oldBytes = static_cast<size_t>(dest.var_value.ptr_count) *
static_cast<size_t>(dest.var_value.ptr_size);
01939
01940 if (count == 0) {
01941     // SetLength(arr, 0) -- free the memory
01942     if (dest.var_value.ptr_value != nullptr && dest.var_value.owns) {
01943         std::free(dest.var_value.ptr_value);
01944     }
01945     dest.var_value.ptr_value = nullptr;
01946     dest.var_value.ptr_size = size;
01947     dest.var_value.ptr_count = 0;
01948     dest.var_value.owns = true;
01949     dest.type = VarType::VAR_POINTER;
01950     dest.var_value.type = VarType::VAR_POINTER;
01951     return;
01952 }
01953
01954 void *newPtr = std::realloc(dest.var_value.ptr_value, newBytes);
01955 if (newPtr == nullptr) {
01956     throw mx::Exception("REALLOC failed: realloc returned nullptr");
01957 }
01958
01959 // Zero-initialize newly allocated portion
01960 if (newBytes > oldBytes) {
01961     std::memset(static_cast<char *>(newPtr) + oldBytes, 0, newBytes - oldBytes);
01962 }
01963
01964 dest.type = VarType::VAR_POINTER;
01965 dest.var_value.type = VarType::VAR_POINTER;
01966 dest.var_value.ptr_value = newPtr;
01967 dest.var_value.ptr_size = size;
01968 dest.var_value.ptr_count = count;
01969 dest.var_value.owns = true;
01970 }
01971
01972 Variable Program::createTempVariable(VarType type, const std::string &value) {
01973     Variable temp;
01974     temp.type = type;
01975     temp.var_name = "";
01976     if (type == VarType::VAR_INTEGER) {
01977         if (value.empty())
01978             throw mx::Exception("empty integer constant: " + value);
01979         temp.var_value.int_value = std::stoll(value, nullptr, 0);
01980         temp.var_value.type = VarType::VAR_INTEGER;
01981     } else if (type == VarType::VAR_FLOAT) {
01982         temp.var_value.float_value = std::stod(value);
01983         temp.var_value.type = VarType::VAR_FLOAT;
01984     } else if (type == VarType::VAR_STRING) {
01985         temp.var_value.str_value = value;
01986         temp.var_value.type = VarType::VAR_STRING;
01987     }
01988     return temp;
01989 }
01990
01991 void Program::setVariableFromConstant(Variable &var, const std::string &value) {
01992     if (var.type == VarType::VAR_INTEGER) {
01993         var.var_value.int_value = std::stoll(value, nullptr, 0);
01994         var.var_value.type = VarType::VAR_INTEGER;
01995     } else if (var.type == VarType::VAR_FLOAT) {
01996         var.var_value.float_value = std::stod(value);
01997         var.var_value.type = VarType::VAR_FLOAT;
01998     } else if (var.type == VarType::VAR_STRING) {
01999         var.var_value.str_value = value;
02000         var.var_value.type = VarType::VAR_STRING;
02001     } else if (var.type == VarType::VAR_POINTER) {
02002         if (value == "null" || value == "NULL" || value == "0") {
02003             var.var_value.ptr_value = nullptr;
02004         } else {
02005             uintptr_t addr = std::stoull(value, nullptr, 0);
02006             var.var_value.ptr_value = reinterpret_cast<void *>(addr);
02007         }
02008         var.var_value.type = VarType::VAR_POINTER;
02009         var.var_value.owns = false;
02010     }
02011 }
02012
02013 void Program::print(std::ostream &out) {

```



```

02014         out << "Debug Information..\n";
02015         out << "=== INSTRUCTIONS ===\n";
02016         if (inc.empty()) {
02017             out << " (no instructions)\n";
02018         } else {
02019             out << std::setw(10) << "Addr"
02020                 << std::setw(20) << "Instruction"
02021                 << std::setw(25) << "Operand1"
02022                 << std::setw(25) << "Operand2"
02023                 << std::setw(25) << "Operand3"
02024                 << std::setw(30) << "Extra Operands" << "\n";
02025             out << std::string(130, '-') << "\n";
02026
02027             for (size_t i = 0; i < inc.size(); ++i) {
02028                 const auto &instr = inc[i];
02029                 out << std::setw(10) << i;
02030                 if (instr.instruction >= 0 && instr.instruction < static_cast<int>(IncType.size())) {
02031                     out << std::setw(12) << IncType[instr.instruction];
02032                 } else {
02033                     out << std::setw(12) << "UNKNOWN";
02034                 }
02035                 out << std::setw(25) << (instr.op1.op.empty() ? "-" : instr.op1.op);
02036                 out << std::setw(25) << (instr.op2.op.empty() ? "-" : instr.op2.op);
02037                 out << std::setw(25) << (instr.op3.op.empty() ? "-" : instr.op3.op);
02038                 if (!instr.vop.empty()) {
02039                     std::string extraOps;
02040                     for (size_t j = 0; j < instr.vop.size(); ++j) {
02041                         if (j > 0)
02042                             extraOps += ", ";
02043                         extraOps += instr.vop[j].op;
02044                     }
02045                     out << std::setw(30) << extraOps;
02046                 } else {
02047                     out << std::setw(30) << "-";
02048                 }
02049
02050                 out << "\n";
02051             }
02052         }
02053         out << "\n";
02054     }
02055
02056 void Program::exec_invoke(const Instruction &instr) {
02057     auto it = external_functions.find(instr.op1.op);
02058     if (it == external_functions.end()) {
02059         throw mx::Exception("INVOKE: external function not found: " + instr.op1.op);
02060     }
02061
02062     std::vector<Operand> args;
02063
02064     auto process_operand = [&](Operand op) {
02065         if (op.op.empty())
02066             return;
02067
02068         if (isVariable(op.op)) {
02069             Variable &v = getVariable(op.op);
02070             if ((v.type == VarType::VAR_POINTER || v.type == VarType::VAR_EXTERN)) {
02071                 if (v.var_value.ptr_value == nullptr) {
02072                     if (instr.op1.op == "strlen" || instr.op1.op == "strcpy" ||
02073                         instr.op1.op == "strncat" || instr.op1.op == "draw_text") {
02074                         op.op = "";
02075                         op.type = OperandType::OP_CONSTANT;
02076                     }
02077                 }
02078             }
02079         }
02080         args.push_back(op);
02081     };
02082
02083     process_operand(instr.op2);
02084     process_operand(instr.op3);
02085     for (const auto &vop : instr.vop) {
02086         process_operand(vop);
02087     }
02088
02089     it->second.call(this, args);
02090     result.op = "%rax";
02091 }
02092
02093 void Program::post(std::ostream &out) {
02094     if (stack.empty()) {
02095         out << "Stack aligned.\n";
02096     } else {
02097         out << "Stack unaligned. {\n";
02098         for (size_t i = 0; i < stack.size(); ++i) {
02099             out << "\tAddress: [" << i << "] = ";
02100             const StackValue &value = stack[i];

```

```

02101         if (std::holds_alternative<int64_t>(value)) {
02102             out << std::get<int64_t>(value);
02103         } else if (std::holds_alternative<void *>(value)) {
02104             void *ptr = std::get<void *>(value);
02105             if (ptr == nullptr) {
02106                 out << "null";
02107             } else {
02108                 out << "0x" << std::hex << reinterpret_cast<uintptr_t>(ptr) << std::dec;
02109             }
02110         } else if (std::holds_alternative<double>(value)) {
02111             out << std::get<double>(value);
02112         } else if (std::holds_alternative<std::string>(value)) {
02113             out << "\"" << std::get<std::string>(value) << "\"";
02114         } else {
02115             out << "(unknown)";
02116         }
02117         out << "\n";
02118     }
02119     out << "]\n";
02120 }
02121 }
02122
02123 Variable Program::variableFromOperand(const Operand &op) {
02124     if (isVariable(op.op)) {
02125         return getVariable(op.op);
02126     } else if (op.type == OperandType::OP_CONSTANT) {
02127         return createTempVariable(VarType::VAR_INTEGER, op.op);
02128     }
02129     throw mx::Exception("Could not create variable from operand: " + op.op);
02130 }
02131
02132 void Program::exec_fcmp(const Instruction &instr) {
02133     Variable *var1 = nullptr;
02134     Variable *var2 = nullptr;
02135     Variable temp1, temp2;
02136
02137     if (isVariable(instr.op1.op)) {
02138         var1 = &getVariable(instr.op1.op);
02139     } else {
02140         temp1 = createTempVariable(VarType::VAR_FLOAT, instr.op1.op);
02141         var1 = &temp1;
02142     }
02143     if (isVariable(instr.op2.op)) {
02144         var2 = &getVariable(instr.op2.op);
02145     } else {
02146         temp2 = createTempVariable(VarType::VAR_FLOAT, instr.op2.op);
02147         var2 = &temp2;
02148     }
02149
02150     zero_flag = false;
02151     less_flag = false;
02152     greater_flag = false;
02153
02154     double val1, val2;
02155     if (var1->type == VarType::VAR_FLOAT) {
02156         val1 = var1->var_value.float_value;
02157     } else if (var1->type == VarType::VAR_INTEGER || var1->type == VarType::VAR_BYTE) {
02158         val1 = static_cast<double>(var1->var_value.int_value);
02159     } else {
02160         throw mx::Exception("FCMP: unsupported operand type for var1");
02161     }
02162
02163     if (var2->type == VarType::VAR_FLOAT) {
02164         val2 = var2->var_value.float_value;
02165     } else if (var2->type == VarType::VAR_INTEGER || var2->type == VarType::VAR_BYTE) {
02166         val2 = static_cast<double>(var2->var_value.int_value);
02167     } else {
02168         throw mx::Exception("FCMP: unsupported operand type for var2");
02169     }
02170
02171     if (val1 == val2) {
02172         zero_flag = true;
02173     } else if (val1 < val2) {
02174         less_flag = true;
02175     } else {
02176         greater_flag = true;
02177     }
02178 }
02179
02180 void Program::exec_jae(const Instruction &instr) {
02181     if (greater_flag || zero_flag)
02182         exec_jump(instr);
02183     else
02184         pc++;
02185 }
02186
02187 void Program::exec_jbe(const Instruction &instr) {

```

```

02188         if (less_flag || zero_flag)
02189             exec_jump(instr);
02190         else
02191             pc++;
02192     }
02193
02194     void Program::exec_jc(const Instruction &instr) {
02195         if (carry_flag)
02196             exec_jump(instr);
02197         else
02198             pc++;
02199     }
02200
02201     void Program::exec_jnc(const Instruction &instr) {
02202         if (!carry_flag)
02203             exec_jump(instr);
02204         else
02205             pc++;
02206     }
02207
02208     void Program::exec_jp(const Instruction &instr) {
02209         pc++;
02210     }
02211
02212     void Program::exec_jnp(const Instruction &instr) {
02213         exec_jump(instr);
02214     }
02215
02216     void Program::exec_jo(const Instruction &instr) {
02217         pc++;
02218     }
02219
02220     void Program::exec_jno(const Instruction &instr) {
02221         exec_jump(instr);
02222     }
02223
02224     void Program::exec_js(const Instruction &instr) {
02225         pc++;
02226     }
02227
02228     void Program::exec_jns(const Instruction &instr) {
02229         exec_jump(instr);
02230     }
02231
02232 } // namespace mxvm

```

12.92 src/icode_gen_x64.cpp File Reference

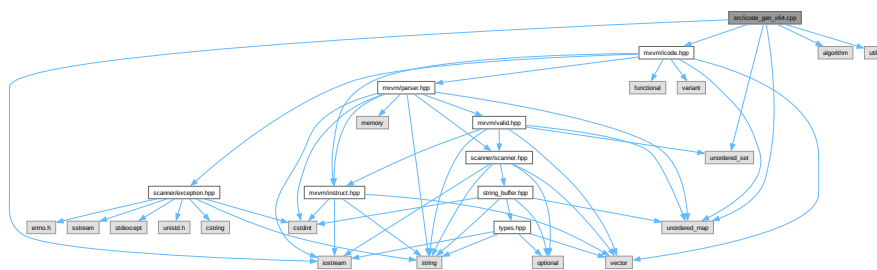
x86-64 Win64 ABI native code generator for Windows

```

#include "mxvm/icode.hpp"
#include <algorithm>
#include <iostream>
#include <unordered_map>
#include <unordered_set>
#include <utility>

```

Include dependency graph for icode_gen_x64.cpp:



Namespaces

- namespace `mxvm`

Functions

- static bool `mxvm::is_stdio_name` (const std::string &s)
- static int `mxvm::stdio_index` (const std::string &s)
- bool `mxvm::isFunctionReturningOwnedPtr` (const std::string &funcName)

Check whether a named function returns an owned (heap-allocated) pointer.

Variables

- static unsigned `mxvm::x64_sp_mod16` = 0
- size_t `mxvm::xmm_offset`
- static int `mxvm::error_label_count` = 0

12.92.1 Detailed Description

x86-64 Win64 ABI native code generator for Windows

Author

Jared Bruni

Definition in file `icode_gen_x64.cpp`.

12.93 icode_gen_x64.cpp

[Go to the documentation of this file.](#)

```
00001
00006 #include "mxvm/icode.hpp"
00007
00008 #include <algorithm>
00009 #include <iostream>
00010 #include <unordered_map>
00011 #include <unordered_set>
00012 #include <utility>
00013
00014 namespace mxvm {
00015
00016     static unsigned x64_sp_mod16 = 0;
00017     extern size_t xmm_offset;
00018     static int error_label_count = 0;
00019
00020     std::string Program::x64_getRegisterByIndex(int index, VarType type) {
00021         if (type == VarType::VAR_FLOAT) {
00022             if (index < 4)
00023                 return "%xmm" + std::to_string(index);
00024             return "[stack]";
00025         }
00026         static const char *ireg[] = {"%rcx", "%rdx", "%r8", "%r9"};
00027         if (index < 4)
00028             return ireg[index];
00029         return "[stack]";
00030     }
00031
00032     static inline bool is_stdio_name(const std::string &s) {
00033         return s == "stdin" || s == "stdout" || s == "stderr";
00034     }
00035     static inline int stdio_index(const std::string &s) {
00036         return s == "stdin" ? 0 : (s == "stdout" ? 1 : 2);
00037     }
00038
00039     void Program::x64_analyzeRegAlloc(bool uses_std_module) {
00040         x64_reg_vars.clear();
00041         x64_reg_save_order.clear();
00042
00043         std::vector<std::string> available_regs = {"%rbx", "%r14", "%r15", "%rsi", "%rdi"};
00044         if (!uses_std_module) {
00045             available_regs.push_back("%r12");
00046             available_regs.push_back("%r13");
00047         }
00048
00049         std::unordered_map<std::string, int> usage_count;
00050         auto countOp = [&](const Operand &op) {
00051             if (!op.op.empty() && isVariable(op.op)) {
00052                 Variable &v = getVariable(op.op);
00053                 if (v.type == VarType::VAR_INTEGER && !v.is_global && !is_stdio_name(op.op)) {
```

```

00054         usage_count[op.op]++;
00055     }
00056 }
00057 };
00058
00059 for (const auto &instr : inc) {
00060     countOp(instr.op1);
00061     countOp(instr.op2);
00062     countOp(instr.op3);
00063     for (const auto &vop : instr.vop)
00064         countOp(vop);
00065 }
00066
00067 std::vector<std::pair<std::string, int> sorted_vars(usage_count.begin(), usage_count.end());
00068 std::sort(sorted_vars.begin(), sorted_vars.end(),
00069     [](const auto &a, const auto &b) { return a.second > b.second; });
00070
00071 size_t n = std::min(available_regs.size(), sorted_vars.size());
00072 for (size_t i = 0; i < n; ++i) {
00073     if (sorted_vars[i].second >= 2) {
00074         x64_reg_vars[sorted_vars[i].first] = available_regs[i];
00075         x64_reg_save_order.push_back(available_regs[i]);
00076     }
00077 }
00078 }
00079
00080 void Program::x64_emitFlushRegs(std::ostream &out) {
00081     for (const auto &[var, reg] : x64_reg_vars) {
00082         out << "\tmovq " << reg << ", " << getMangledName(var) << "(%rip)\n";
00083     }
00084 }
00085
00086 void Program::x64_emitReloadRegs(std::ostream &out) {
00087     for (const auto &[var, reg] : x64_reg_vars) {
00088         out << "\tmovq " << getMangledName(var) << "(%rip), " << reg << "\n";
00089     }
00090 }
00091
00092 void Program::x64_emitSaveRegs(std::ostream &out) {
00093     for (const auto &reg : x64_reg_save_order) {
00094         out << "\tpushq " << reg << "\n";
00095         x64_sp_mod16 ^= 8;
00096     }
00097 }
00098
00099 void Program::x64_emitRestoreRegs(std::ostream &out) {
00100     for (auto it = x64_reg_save_order.rbegin(); it != x64_reg_save_order.rend(); ++it) {
00101         out << "\tpopq " << *it << "\n";
00102         x64_sp_mod16 ^= 8;
00103     }
00104 }
00105
00106 void Program::x64_emitStoreVar(std::ostream &out, const std::string &srcReg, const Operand &dest)
00107 {
00108     auto it = x64_reg_vars.find(dest.op);
00109     if (it != x64_reg_vars.end()) {
00110         if (srcReg != it->second)
00111             out << "\tmovq " << srcReg << ", " << it->second << "\n";
00112         else {
00113             out << "\tmovq " << srcReg << ", " << getMangledName(dest) << "(%rip)\n";
00114         }
00115     }
00116 }
00117
00118 void Program::x64_emitStoreVarImm(std::ostream &out, const std::string &imm, const Operand &dest)
00119 {
00120     auto it = x64_reg_vars.find(dest.op);
00121     if (it != x64_reg_vars.end()) {
00122         out << "\tmovq $" << imm << ", " << it->second << "\n";
00123     } else {
00124         out << "\tmovq $" << imm << ", " << getMangledName(dest) << "(%rip)\n";
00125     }
00126 }
00127
00128 void Program::x64_emitLoadVar(std::ostream &out, const std::string &dstReg, const Operand &src) {
00129     auto it = x64_reg_vars.find(src.op);
00130     if (it != x64_reg_vars.end()) {
00131         if (dstReg != it->second)
00132             out << "\tmovq " << it->second << ", " << dstReg << "\n";
00133         else {
00134             out << "\tmovq " << getMangledName(src) << "(%rip), " << dstReg << "\n";
00135         }
00136     }
00137 }
00138
00139 size_t Program::x64_reserve_call_area(std::ostream &out, size_t spill_bytes) {
00140     const size_t need = 32 + spill_bytes;
00141     size_t aligned_need = (need + 15) & ~15;
00142     // Ensure RSP will be 16-byte aligned before the call instruction.

```

```

00139         // If the stack is currently misaligned (x64_sp_mod16 == 8), add 8 more bytes.
00140         if ((x64_sp_mod16 + aligned_need) % 16 != 0)
00141             aligned_need += 8;
00142         out << "\tsub $" << aligned_need << ", %rsp\n";
00143         x64_sp_mod16 = (x64_sp_mod16 + aligned_need) % 16;
00144         return aligned_need;
00145     }
00146
00147 void Program::x64_release_call_area(std::ostream &out, size_t total) {
00148     out << "\tadd $" << total << ", %rsp\n";
00149     x64_sp_mod16 = (x64_sp_mod16 + (16 - (total % 16))) % 16;
00150 }
00151 void Program::x64_emit_iob_func(std::ostream &out, int index, const std::string &dstReg) {
00152     out << "\tmov $" << index << ", %ecx\n";
00153     size_t total = x64_reserve_call_area(out, 0);
00154     out << "\tcall __acrt_iob_func\n";
00155     x64_release_call_area(out, total);
00156     if (dstReg != "%rax")
00157         out << "\tmov %rax, " << dstReg << "\n";
00158 }
00159
00160 int Program::x64_generateLoadVar(std::ostream &out, VarType type, std::string reg, const Operand
&op) {
00161     int count = 0;
00162     if (isVariable(op.op)) {
00163         Variable &v = getVariable(op.op);
00164
00165         if ((v.type == VarType::VAR_EXTERN || v.type == VarType::VAR_POINTER) &&
is_stdio_name(op.op)) {
00166             x64_emit_iob_func(out, stdio_index(op.op), reg);
00167             return 0;
00168         }
00169
00170         switch (v.type) {
00171             case VarType::VAR_INTEGER:
00172             case VarType::VAR_POINTER:
00173             case VarType::VAR_EXTERN: {
00174                 auto ra = x64_reg_vars.find(op.op);
00175                 if (ra != x64_reg_vars.end()) {
00176                     if (ra->second != reg)
00177                         out << "\tmovq " << ra->second << ", " << reg << "\n";
00178                     } else {
00179                         out << "\tmovq " << getMangledName(op) << "(%rip), " << reg << "\n";
00180                     }
00181                     break;
00182                 }
00183             case VarType::VAR_BYTE:
00184                 out << "\tmovzbq " << getMangledName(op) << "(%rip), " << reg << "\n";
00185                 break;
00186             case VarType::VAR_FLOAT:
00187                 out << "\tmovsd " << getMangledName(op) << "(%rip), " << reg << "\n";
00188                 count = 1;
00189                 break;
00190             case VarType::VAR_STRING:
00191                 out << "\tleaq " << getMangledName(op) << "(%rip), " << reg << "\n";
00192                 break;
00193             default:
00194                 break;
00195         }
00196     } else {
00197         if (op.type == OperandType::OP_CONSTANT && type == VarType::VAR_INTEGER)
00198             out << "\tmovq $" << op.op << ", " << reg << "\n";
00199         else if (op.type == OperandType::OP_CONSTANT && type == VarType::VAR_FLOAT)
00200             throw mx::Exception("Constant to float not supported");
00201     }
00202     return count;
00203 }
00204
00205 int Program::x64_generateLoadVar(std::ostream &out, int r, const Operand &op) {
00206     int count = 0;
00207     if (isVariable(op.op)) {
00208         Variable &v = getVariable(op.op);
00209         std::string reg = x64_getRegisterByIndex(r, v.type);
00210
00211         if ((v.type == VarType::VAR_EXTERN || v.type == VarType::VAR_POINTER) &&
is_stdio_name(op.op)) {
00212             x64_emit_iob_func(out, stdio_index(op.op), reg);
00213             return 0;
00214         }
00215
00216         switch (v.type) {
00217             case VarType::VAR_INTEGER:
00218             case VarType::VAR_POINTER:
00219             case VarType::VAR_EXTERN: {
00220                 auto ra = x64_reg_vars.find(op.op);
00221                 if (ra != x64_reg_vars.end()) {
00222                     if (ra->second != reg)

```

```

00223         out << "\tmovq " << ra->second << ", " << reg << "\n";
00224     } else {
00225         out << "\tmovq " << getMangledName(op) << "(%rip), " << reg << "\n";
00226     }
00227     break;
00228 }
00229 case VarType::VAR_FLOAT:
00230     out << "\tmovsd " << getMangledName(op) << "(%rip), " << reg << "\n";
00231     count = 1;
00232     break;
00233 case VarType::VAR_STRING:
00234     out << "\tleaq " << getMangledName(op) << "(%rip), " << reg << "\n";
00235     break;
00236 case VarType::VAR_BYTE:
00237     out << "\tmovzbq " << getMangledName(op) << "(%rip), " << reg << "\n";
00238     break;
00239 default:
00240     break;
00241 }
00242 } else {
00243     if (op.type == OperandType::OP_CONSTANT)
00244         out << "\tmovq $" << op.op << ", " << x64_getRegisterByIndex(r, VarType::VAR_INTEGER) <<
"\n";
00245     }
00246     return count;
00247 }
00248
00249 bool isFunctionReturningOwnedPtr(const std::string &funcName) {
00250     static const std::unordered_set<std::string> funcs = {
00251         "inttostr",
00252         "floattostr",
00253     };
00254     return funcs.count(funcName) > 0;
00255 }
00256
00257 void Program::x64_generateFunctionCall(std::ostream &out,
00258                                         const std::string &name,
00259                                         std::vector<Operand> &args) {
00260     xmm_offset = 0;
00261     const size_t stack_args = (args.size() > 4) ? (args.size() - 4) : 0;
00262     const size_t spill_bytes = stack_args * 8;
00263     size_t frame = x64_reserve_call_area(out, spill_bytes);
00264
00265     static const char *GPR[4] = {"%rcx", "%rdx", "%r8", "%r9"};
00266
00267     for (size_t i = 0; i < args.size(); ++i) {
00268         VarType t = VarType::VAR_INTEGER;
00269         if (isVariable(args[i].op))
00270             t = getVariable(args[i].op).type;
00271
00272         if (i < 4) {
00273             if (t == VarType::VAR_FLOAT) {
00274                 std::string xr = "%xmm" + std::to_string(i);
00275                 x64_generateLoadVar(out, VarType::VAR_FLOAT, xr, args[i]);
00276             } else {
00277                 x64_generateLoadVar(out, VarType::VAR_INTEGER, GPR[i], args[i]);
00278             }
00279         } else {
00280             const size_t off = 32 + 8 * (i - 4);
00281             if (t == VarType::VAR_FLOAT) {
00282                 x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm7", args[i]);
00283                 out << "\tmovsd %xmm7, " << off << "(%rsp)\n";
00284             } else {
00285                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", args[i]);
00286                 out << "\tmovq %rax, " << off << "(%rsp)\n";
00287             }
00288         }
00289     }
00290
00291     out << "\txor %eax, %eax\n";
00292     out << "\tcall " << name << "\n";
00293     x64_release_call_area(out, frame);
00294 }
00295
00296 void Program::x64_generateInvokeCall(std::ostream &out, std::vector<Operand> &op) {
00297     if (op.empty() || op[0].op.empty())
00298         throw mx::Exception("invoke requires instruction name");
00299     std::string n = op[0].op;
00300     std::vector<Operand> a;
00301     for (size_t i = 1; i < op.size(); ++i)
00302         if (!op[i].op.empty())
00303             a.push_back(op[i]);
00304
00305     if (isFunctionReturningOwnedPtr(n)) {
00306         this->last_call_returns_owned_ptr = true;
00307     } else {
00308         this->last_call_returns_owned_ptr = false;
00309     }

```

```

00309     }
00310
00311     x64_generateFunctionCall(out, n, a);
00312 }
00313
00314 void Program::x64_generateCode(const Platform &, bool, std::ostream &out) {
00315     this->add_standard();
00316     std::unordered_map<int, std::string> labels_;
00317     for (auto &l : labels)
00318         labels_[l.second.first] = l.first;
00319
00320     bool uses_std_module = false;
00321     for (const auto &e : external) {
00322         if (e.mod == "std") {
00323             uses_std_module = true;
00324             break;
00325         }
00326     }
00327
00328     x64_analyzeRegAlloc(uses_std_module);
00329
00330     out << ".section .data\n";
00331
00332     std::vector<std::string> var_names;
00333     for (auto &v : vars)
00334         var_names.push_back(v.first);
00335     std::sort(var_names.begin(), var_names.end());
00336
00337     for (auto &v : var_names) {
00338         auto varx = getVariable(v);
00339         const std::string nm = getMangledName(v);
00340         if (varx.type == VarType::VAR_INTEGER) {
00341             out << "\t" << nm << ": .quad " << vars[v].var_value.int_value << "\n";
00342             continue;
00343         }
00344         if (varx.type == VarType::VAR_FLOAT) {
00345             out << "\t" << nm << ": .double " << vars[v].var_value.float_value << "\n";
00346             continue;
00347         }
00348         if (varx.type == VarType::VAR_BYTE) {
00349             out << "\t" << nm << ": .byte " << vars[v].var_value.int_value << "\n";
00350         }
00351     }
00352
00353     for (auto &v : var_names) {
00354         auto varx = getVariable(v);
00355         if (varx.type == VarType::VAR_STRING && varx.var_value.buffer_size == 0) {
00356             const std::string nm = getMangledName(v);
00357             out << "\t" << nm << ": .asciz \"" << escapeNewLines(vars[v].var_value.str_value) <<
00358                 "\"\n";
00359         }
00360
00361         if (base != nullptr) {
00362             for (auto &v : var_names) {
00363                 auto varx = getVariable(v);
00364                 if (varx.is_global || object) {
00365                     switch (varx.type) {
00366                         case VarType::VAR_BYTE:
00367                         case VarType::VAR_FLOAT:
00368                         case VarType::VAR_INTEGER:
00369                             out << "\t.globl " << getMangledName(v) << "\n";
00370                             break;
00371                         default:
00372                             break;
00373                     }
00374                 }
00375             }
00376         }
00377
00378         out << ".section .bss\n";
00379         for (auto &v : var_names) {
00380             const std::string nm = getMangledName(v);
00381             auto varx = getVariable(v);
00382             if (varx.type == VarType::VAR_POINTER)
00383                 out << "\t.comm " << nm << ", 8\n";
00384             else if (varx.type == VarType::VAR_STRING && varx.var_value.buffer_size > 0)
00385                 out << "\t.comm " << nm << ", " << varx.var_value.buffer_size << "\n";
00386         }
00387         if (object) {
00388             for (auto &v : var_names) {
00389                 auto varx = getVariable(v);
00390                 if (varx.type == VarType::VAR_STRING || varx.type == VarType::VAR_POINTER)
00391                     out << "\t.globl " << getMangledName(v) << "\n";
00392             }
00393         }
00394     }

```



```

00395         out << ".section .text\n";
00396
00397         out << "\t.extern __acrt_iob_func\n";
00398
00399         for (auto &lbl : labels)
00400             if (lbl.second.second == true)
00401                 out << "\t.globl " << name + "_" + lbl.first << "\n";
00402
00403         std::sort(external.begin(), external.end(), [](const ExternalFunction &a, const
ExternalFunction &b) {
00404             if (a.mod == b.mod)
00405                 return a.name < b.name;
00406             return a.mod < b.mod;
00407         });
00408         for (auto &e : external)
00409             out << "\t.extern " << (e.module ? e.name : e.mod + "_" + e.name) << "\n";
00410
00411         if (!this->object) {
00412             out << "\t.p2align 4, 0x90\n";
00413             out << ".globl main\n";
00414             out << "main:\n";
00415             out << "\tpush %rbp\n";
00416             out << "\tmov %rsp, %rbp\n";
00417             x64_sp_mod16 = 0;
00418             if (uses_std_module) {
00419                 out << "\tpush %r12\n";
00420                 x64_sp_mod16 ^= 8;
00421                 out << "\tpush %r13\n";
00422                 x64_sp_mod16 ^= 8;
00423                 out << "\tmov %rcx, %r12\n";
00424                 out << "\tmov %rdx, %r13\n";
00425                 size_t total = x64_reserve_call_area(out, 0);
00426                 out << "\tcall set_program_args\n";
00427                 x64_release_call_area(out, total);
00428             }
00429             x64_emitSaveRegs(out);
00430             x64_emitReloadRegs(out);
00431         }
00432
00433         bool done_found = false;
00434
00435         for (size_t i = 0; i < inc.size(); ++i) {
00436             const Instruction &instr = inc[i];
00437             for (auto l : labels) {
00438                 if (l.second.first == i && l.second.second) {
00439                     out << "\t.p2align 4, 0x90\n";
00440                     out << name + "_" + l.first << ":\n";
00441                     out << "\tpush %rbp\n";
00442                     out << "\tmov %rsp, %rbp\n";
00443                     x64_sp_mod16 = 0;
00444                     x64_emitSaveRegs(out);
00445                     x64_emitReloadRegs(out);
00446                     if (isVariable("rax") && getVariable("rax").type == VarType::VAR_INTEGER) {
00447                         Operand rax_op;
00448                         rax_op.op = "rax";
00449                         x64_emitStoreVar(out, "%rax", rax_op);
00450                     }
00451                     break;
00452                 }
00453             }
00454             for (auto l : labels) {
00455                 if (l.second.first == i && !l.second.second)
00456                     out << "." << l.first << ":\n";
00457             }
00458             if (instr.instruction == DONE)
00459                 done_found = true;
00460             x64_generateInstruction(out, instr);
00461         }
00462
00463 #ifndef __EMSCRIPTEN__
00464         if (this->object == false && done_found == false)
00465             throw mx::Exception("Program missing done to signal completion.\n");
00466 #endif
00467
00468         out << "\n\n";
00469
00470         std::string mainFunc = " Object";
00471         if (root_name == name)
00472             mainFunc = " Program";
00473         std::cout << Col("MXVM: Compiled: ", mx::Color::BRIGHT_BLUE) << name << ".s" << mainFunc << Col("
platform: ", mx::Color::BRIGHT_CYAN) << "Windows" << "\n";
00474     }
00475
00476     void Program::x64_gen_done(std::ostream &out, const Instruction &) {
00477         bool uses_std_module = false;
00478         for (const auto &e : external) {
00479             if (e.mod == "std") {

```

```

00480         uses_std_module = true;
00481         break;
00482     }
00483 }
00484
00485 if (uses_std_module && !this->object) {
00486     x64_emitFlushRegs(out);
00487     size_t total = x64_reserve_call_area(out, 0);
00488     out << "\tcall free_program_args\n";
00489     x64_release_call_area(out, total);
00490 }
00491
00492 out << "\txor %eax, %eax\n";
00493 x64_emitRestoreRegs(out);
00494 if (!this->object && uses_std_module) {
00495     out << "\tmovq -8(%rbp), %r12\n";
00496     out << "\tmovq -16(%rbp), %r13\n";
00497 }
00498 out << "\tleave\n";
00499 out << "\tret\n";
00500 }
00501
00502 void Program::x64_gen_ret(std::ostream &out, const Instruction &i) {
00503     x64_emitFlushRegs(out);
00504     if (isVariable("rax") && getVariable("rax").type == VarType::VAR_INTEGER) {
00505         Operand rax_op;
00506         rax_op.op = "rax";
00507         x64_emitLoadVar(out, "%rax", rax_op);
00508     }
00509     x64_emitRestoreRegs(out);
00510     out << "\tleave\n";
00511     out << "\tret\n";
00512 }
00513
00514 void Program::x64_gen_return(std::ostream &out, const Instruction &i) {
00515     if (isVariable(i.op1.op)) {
00516         if (this->last_call_returns_owned_ptr) {
00517             Variable &v = getVariable(i.op1.op);
00518             v.var_value.owns = true;
00519             this->last_call_returns_owned_ptr = false;
00520         }
00521         x64_emitStoreVar(out, "%rax", i.op1);
00522     } else
00523         throw mx::Exception("return requires variable");
00524 }
00525
00526 void Program::x64_gen_neg(std::ostream &out, const Instruction &i) {
00527     if (isVariable(i.op1.op)) {
00528         Variable &v = getVariable(i.op1.op);
00529         if (v.type == VarType::VAR_INTEGER) {
00530             x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op1);
00531             out << "\tnegq %rcx\n";
00532             x64_emitStoreVar(out, "%rcx", i.op1);
00533         } else if (v.type == VarType::VAR_FLOAT) {
00534             x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
00535             out << "\txorpd %xmm1, %xmm1\n";
00536             out << "\tsubsd %xmm0, %xmm1\n";
00537             out << "\tmovsd %xmm1, " << getMangledName(i.op1) << "(%rip)\n";
00538         } else
00539             throw mx::Exception("neg requires float or integer");
00540     } else
00541         throw mx::Exception("neg requires variable");
00542 }
00543
00544 void Program::x64_gen_lea(std::ostream &out, const Instruction &i) {
00545     if (!isVariable(i.op1.op))
00546         throw mx::Exception("LEA destination must be a variable");
00547     if (!isVariable(i.op2.op))
00548         throw mx::Exception("LEA source must be a variable");
00549
00550     // Flush source register to memory before taking its address
00551     auto it = x64_reg_vars.find(i.op2.op);
00552     if (it != x64_reg_vars.end()) {
00553         out << "\tmovq " << it->second << ", " << getMangledName(i.op2) << "(%rip)\n";
00554     }
00555
00556     out << "\tleaq " << getMangledName(i.op2) << "(%rip), %rax\n";
00557     out << "\tmovq %rax, " << getMangledName(i.op1) << "(%rip)\n";
00558     getVariable(i.op1.op).var_value.owns = false;
00559 }
00560
00561 void Program::x64_generateInstruction(std::ostream &out, const Instruction &i) {
00562
00563     if (i.instruction < JMP || i.instruction > JNS) {
00564         last_cmp_type = CMP_NONE;
00565     }
00566     switch (i.instruction) {

```

```
00567         case ADD:
00568             x64_gen_arth(out, "add", i);
00569             break;
00570         case SUB:
00571             x64_gen_arth(out, "sub", i);
00572             break;
00573         case MUL:
00574             x64_gen_arth(out, "mul", i);
00575             break;
00576         case PRINT:
00577             x64_gen_print(out, i);
00578             break;
00579         case EXIT:
00580             x64_gen_exit(out, i);
00581             break;
00582         case MOV:
00583             x64_gen_mov(out, i);
00584             break;
00585         case JMP:
00586         case JE:
00587         case JNE:
00588         case JL:
00589         case JLE:
00590         case JG:
00591         case JGE:
00592         case JZ:
00593         case JNZ:
00594         case JA:
00595         case JB:
00596             x64_gen_jump(out, i);
00597             break;
00598         case CMP:
00599             x64_gen_cmp(out, i);
00600             break;
00601         case ALLOC:
00602             x64_gen_alloc(out, i);
00603             break;
00604         case FREE:
00605             x64_gen_free(out, i);
00606             break;
00607         case REALLOC:
00608             x64_gen_realloc(out, i);
00609             break;
00610         case LOAD:
00611             x64_gen_load(out, i);
00612             break;
00613         case STORE:
00614             x64_gen_store(out, i);
00615             break;
00616         case RET:
00617             x64_gen_ret(out, i);
00618             break;
00619         case CALL:
00620             x64_gen_call(out, i);
00621             break;
00622         case DONE:
00623             x64_gen_done(out, i);
00624             break;
00625         case AND:
00626             x64_gen_bitop(out, "and", i);
00627             break;
00628         case OR:
00629             x64_gen_bitop(out, "or", i);
00630             break;
00631         case XOR:
00632             x64_gen_bitop(out, "xor", i);
00633             break;
00634         case NOT:
00635             x64_gen_not(out, i);
00636             break;
00637         case DIV:
00638             x64_gen_div(out, i);
00639             break;
00640         case MOD:
00641             x64_gen_mod(out, i);
00642             break;
00643         case PUSH:
00644             x64_gen_push(out, i);
00645             break;
00646         case POP:
00647             x64_gen_pop(out, i);
00648             break;
00649         case STACK_LOAD:
00650             x64_gen_stack_load(out, i);
00651             break;
00652         case STACK_STORE:
00653             x64_gen_stack_store(out, i);
```

```

00654         break;
00655     case STACK_SUB:
00656         x64_gen_stack_sub(out, i);
00657         break;
00658     case GETLINE:
00659         x64_gen_getline(out, i);
00660         break;
00661     case TO_INT:
00662         x64_gen_to_int(out, i);
00663         break;
00664     case TO_FLOAT:
00665         x64_gen_to_float(out, i);
00666         break;
00667     case INVOKE:
00668         x64_gen_invoke(out, i);
00669         break;
00670     case RETURN:
00671         x64_gen_return(out, i);
00672         break;
00673     case NEG:
00674         x64_gen_neg(out, i);
00675         break;
00676     case FCMP:
00677         x64_gen_fcmp(out, i);
00678         break;
00679     case JAE:
00680         x64_gen_jae(out, i);
00681         break;
00682     case JBE:
00683         x64_gen_jbe(out, i);
00684         break;
00685     case JC:
00686         x64_gen_jc(out, i);
00687         break;
00688     case JNC:
00689         x64_gen_jnc(out, i);
00690         break;
00691     case JP:
00692         x64_gen_jp(out, i);
00693         break;
00694     case JNP:
00695         x64_gen_jnp(out, i);
00696         break;
00697     case JO:
00698         x64_gen_jo(out, i);
00699         break;
00700     case JNO:
00701         x64_gen_jno(out, i);
00702         break;
00703     case JS:
00704         x64_gen_js(out, i);
00705         break;
00706     case JNS:
00707         x64_gen_jns(out, i);
00708         break;
00709     case LEA:
00710         x64_gen_lea(out, i);
00711         break;
00712     default:
00713         throw mx::Exception("Invalid or unsupported instruction");
00714     }
00715 }
00716
00717 void Program::x64_gen_invoke(std::ostream &out, const Instruction &i) {
00718     if (i.op1.op.empty())
00719         throw mx::Exception("invoke requires instruction name");
00720     std::vector<Operand> op;
00721     op.push_back(i.op1);
00722     if (!i.op2.op.empty())
00723         op.push_back(i.op2);
00724     if (!i.op3.op.empty())
00725         op.push_back(i.op3);
00726     for (const auto &vx : i.vop)
00727         if (!vx.op.empty())
00728             op.push_back(vx);
00729     x64_generateInvokeCall(out, op);
00730 }
00731
00732 void Program::x64_gen_call(std::ostream &out, const Instruction &i) {
00733     if (!isFunctionValid(i.op1.op))
00734         throw mx::Exception("Function not found");
00735     x64_emitFlushRegs(out);
00736     if (isVariable("rax") && getVariable("rax").type == VarType::VAR_INTEGER) {
00737         Operand rax_op;
00738         rax_op.op = "rax";
00739         x64_emitLoadVar(out, "%rax", rax_op);
00740     }

```

```

00741     size_t total = x64_reserve_call_area(out, 0);
00742     out << "\tcall " << getMangledName(i.op1) << "\n";
00743     x64_release_call_area(out, total);
00744     if (isVariable("rax") && getVariable("rax").type == VarType::VAR_INTEGER) {
00745         Operand rax_op;
00746         rax_op.op = "rax";
00747         out << "\tmovq %rax, " << getMangledName(rax_op) << "(%rip)\n";
00748     }
00749     x64_emitReloadRegs(out);
00750 }
00751 void Program::x64_gen_alloc(std::ostream &out, const Instruction &i) {
00752     if (!isVariable(i.op1.op))
00753         throw mx::Exception("ALLOC destination must be a variable");
00754     Variable &v = getVariable(i.op1.op);
00755     if (v.type != VarType::VAR_POINTER)
00756         throw mx::Exception("ALLOC destination must be a pointer");
00757
00758     if (!i.op3.op.empty()) {
00759         if (isVariable(i.op3.op)) {
00760             x64_emitLoadVar(out, "%rcx", i.op3);
00761         } else {
00762             out << "\tmovq $" << i.op3.op << ", %rcx\n";
00763         }
00764     } else {
00765         out << "\tmovq $1, %rcx\n";
00766     }
00767
00768     if (!i.op2.op.empty()) {
00769         if (isVariable(i.op2.op)) {
00770             x64_emitLoadVar(out, "%rdx", i.op2);
00771         } else {
00772             out << "\tmovq $" << i.op2.op << ", %rdx\n";
00773         }
00774     } else {
00775         out << "\tmovq $8, %rdx\n";
00776     }
00777
00778     size_t total = x64_reserve_call_area(out, 0);
00779     out << "\tcall calloc\n";
00780     x64_release_call_area(out, total);
00781
00782     out << "\ttest %rax, %rax\n";
00783     out << "\tjz .alloc_failed_" << error_label_count << "\n";
00784
00785     out << "\tmovq %rax, " << getMangledName(i.op1) << "(%rip)\n";
00786     getVariable(i.op1.op).var_value.owns = true;
00787
00788     out << "\tjmp .alloc_done_" << error_label_count << "\n";
00789     out << ".alloc_failed_" << error_label_count << ":\n";
00790     out << "\t# Handle allocation failure\n";
00791     out << "\tmovq $0, " << getMangledName(i.op1) << "(%rip)\n";
00792     out << ".alloc_done_" << error_label_count << ":\n";
00793
00794     error_label_count++;
00795 }
00796
00804 void Program::x64_gen_realloc(std::ostream &out, const Instruction &i) {
00805     if (!isVariable(i.op1.op))
00806         throw mx::Exception("REALLOC destination must be a variable");
00807     Variable &v = getVariable(i.op1.op);
00808     if (v.type != VarType::VAR_POINTER)
00809         throw mx::Exception("REALLOC destination must be a pointer");
00810
00811     x64_emitFlushRegs(out);
00812
00813     // %rcx = existing ptr (first arg to realloc on Win x64)
00814     out << "\tmovq " << getMangledName(i.op1) << "(%rip), %rcx\n";
00815
00816     // Compute new total size = count * elemSize
00817     if (!i.op3.op.empty()) {
00818         if (isVariable(i.op3.op)) {
00819             x64_emitLoadVar(out, "%rax", i.op3);
00820         } else {
00821             out << "\tmovq $" << i.op3.op << ", %rax\n";
00822         }
00823     } else {
00824         out << "\tmovq $1, %rax\n";
00825     }
00826
00827     if (!i.op2.op.empty()) {
00828         if (isVariable(i.op2.op)) {
00829             x64_emitLoadVar(out, "%r8", i.op2);
00830         } else {
00831             out << "\tmovq $" << i.op2.op << ", %r8\n";
00832         }
00833     } else {
00834         out << "\tmovq $8, %r8\n";

```

```

00835     }
00836
00837     out << "\timulq %r8, %rax\n";
00838     out << "\tmovq %rax, %rdx\n"; // %rdx = new size (second arg)
00839
00840     size_t total = x64_reserve_call_area(out, 0);
00841     out << "\tcall realloc\n";
00842     x64_release_call_area(out, total);
00843
00844     out << "\ttest %rax, %rax\n";
00845     out << "\tjz .realloc_failed_" << error_label_count << "\n";
00846     out << "\tmovq %rax, " << getMangledName(i.op1) << " (%rip)\n";
00847     out << "\tjmp .realloc_done_" << error_label_count << "\n";
00848     out << ".realloc_failed_" << error_label_count << ":\n";
00849     out << "\t# realloc failed, keep old pointer\n";
00850     out << ".realloc_done_" << error_label_count << ":\n";
00851     error_label_count++;
00852     v.var_value.owns = true;
00853     x64_emitReloadRegs(out);
00854 }
00855
00856 void Program::x64_gen_free(std::ostream &out, const Instruction &i) {
00857     if (!isVariable(i.op1.op))
00858         throw mx::Exception("FREE argument must be a variable");
00859     Variable &v = getVariable(i.op1.op);
00860     if (v.type != VarType::VAR_POINTER)
00861         throw mx::Exception("FREE argument must be a pointer");
00862
00863     out << "\tmovq " << getMangledName(i.op1) << " (%rip), %rcx\n";
00864     out << "\ttest %rcx, %rcx\n";
00865     out << "\tjz 1f\n";
00866     size_t total = x64_reserve_call_area(out, 0);
00867     out << "\tcall free\n";
00868     x64_release_call_area(out, total);
00869     out << "\tmovq $0, " << getMangledName(i.op1) << " (%rip)\n";
00870     out << "1:\n";
00871     v.var_value.owns = false;
00872 }
00873
00874 void Program::x64_gen_mov(std::ostream &out, const Instruction &i) {
00875     if (!isVariable(i.op1.op)) {
00876         throw mx::Exception("MOV first operand must be a variable");
00877     }
00878     Variable &dest = getVariable(i.op1.op);
00879
00880     if (isVariable(i.op2.op)) {
00881         Variable &src = getVariable(i.op2.op);
00882         if (dest.type == VarType::VAR_INTEGER && src.type == VarType::VAR_FLOAT) {
00883             out << "\tmovsd " << getMangledName(i.op2) << " (%rip), %xmm0\n";
00884             out << "\tvcvttsd2si %xmm0, %rax\n";
00885             x64_emitStoreVar(out, "%rax", i.op1);
00886         } else if (dest.type == VarType::VAR_FLOAT && src.type == VarType::VAR_INTEGER) {
00887             x64_emitLoadVar(out, "%rax", i.op2);
00888             out << "\tvcvtsi2sd %rax, %xmm0\n";
00889             out << "\tmovsd %xmm0, " << getMangledName(i.op1) << " (%rip)\n";
00890         } else if (dest.type == src.type) {
00891
00892             if (dest.type == VarType::VAR_FLOAT) {
00893                 x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op2);
00894                 out << "\tmovsd %xmm0, " << getMangledName(i.op1) << " (%rip)\n";
00895             } else if (dest.type == VarType::VAR_BYTE) {
00896                 x64_generateLoadVar(out, VarType::VAR_BYTE, "%rax", i.op2);
00897                 out << "\tmovb %al, " << getMangledName(i.op1) << " (%rip)\n";
00898             } else {
00899                 x64_generateLoadVar(out, dest.type, "%rax", i.op2);
00900                 x64_emitStoreVar(out, "%rax", i.op1);
00901             }
00902         } else {
00903
00904             if (dest.type == VarType::VAR_FLOAT) {
00905                 if (src.type == VarType::VAR_INTEGER || src.type == VarType::VAR_BYTE) {
00906                     x64_generateLoadVar(out, src.type, "%rax", i.op2);
00907                     out << "\tvcvtsi2sdq %rax, %xmm0\n";
00908                     out << "\tmovsd %xmm0, " << getMangledName(i.op1) << " (%rip)\n";
00909                 } else {
00910                     throw mx::Exception("MOV: unsupported conversion to float");
00911                 }
00912             } else if (src.type == VarType::VAR_FLOAT) {
00913                 if (dest.type == VarType::VAR_INTEGER || dest.type == VarType::VAR_BYTE) {
00914                     x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op2);
00915                     out << "\tvcvttsd2siq %xmm0, %rax\n";
00916                     if (dest.type == VarType::VAR_BYTE) {
00917                         out << "\tmovb %al, " << getMangledName(i.op1) << " (%rip)\n";
00918                     } else {
00919                         x64_emitStoreVar(out, "%rax", i.op1);
00920                     }
00921                 } else {

```

```

00922         throw mx::Exception("MOV: unsupported conversion from float");
00923     }
00924     } else if (dest.type == VarType::VAR_POINTER && src.type == VarType::VAR_STRING) {
00925         out << "\tleaq " << getMangledName(i.op2.op) << "({rip}), %rax\n";
00926         out << "\tmovq %rax, " << getMangledName(i.op1.op) << "({rip})\n";
00927         getVariable(i.op1.op).var_value.owns = false;
00928     } else {
00929         x64_generateLoadVar(out, src.type, "%rax", i.op2);
00930         if (dest.type == VarType::VAR_BYTE) {
00931             out << "\tmovb %al, " << getMangledName(i.op1) << "({rip})\n";
00932         } else {
00933             x64_emitStoreVar(out, "%rax", i.op1);
00934         }
00935     }
00936 }
00937 } else {
00938     if (dest.type == VarType::VAR_FLOAT) {
00939         double val = std::stod(i.op2.op);
00940         uint64_t bits;
00941         std::memcpy(&bits, &val, sizeof(bits));
00942         out << "\tmovq $" << bits << ", %rax\n";
00943         out << "\tmovq %rax, " << getMangledName(i.op1) << "({rip})\n";
00944     } else if (dest.type == VarType::VAR_BYTE) {
00945         out << "\tmovb $" << i.op2.op << ", " << getMangledName(i.op1) << "({rip})\n";
00946     } else {
00947         x64_emitStoreVarImm(out, i.op2.op, i.op1);
00948     }
00949 }
00950 }
00951
00952 void Program::x64_gen_load(std::ostream &out, const Instruction &i) {
00953     if (!isVariable(i.op1.op))
00954         throw mx::Exception("LOAD dest must be variable");
00955     Variable &dest = getVariable(i.op1.op);
00956     if (!isVariable(i.op2.op))
00957         throw mx::Exception("LOAD source must be pointer variable");
00958     Variable &ptrVar = getVariable(i.op2.op);
00959
00960     if (ptrVar.type != VarType::VAR_POINTER &&
00961         !(ptrVar.type == VarType::VAR_STRING && ptrVar.var_value.buffer_size > 0)) {
00962         throw mx::Exception("LOAD: invalid base type");
00963     }
00964
00965     if (ptrVar.type == VarType::VAR_POINTER) {
00966         // Flush all cached registers so pointer reads see current values
00967         x64_emitFlushRegs(out);
00968         out << "\tmovq " << getMangledName(i.op2) << "({rip}), %rax\n";
00969     } else {
00970         out << "\tleaq " << getMangledName(i.op2) << "({rip}), %rax\n";
00971     }
00972
00973     out << "\ttest %rax, %rax\n";
00974     out << "\tjz .null_ptr_error_" << error_label_count << "\n";
00975
00976     size_t base_offset = 0;
00977     if (!i.op3.op.empty()) {
00978         if (isVariable(i.op3.op)) {
00979             x64_emitLoadVar(out, "%rcx", i.op3);
00980         } else {
00981             base_offset = std::stoll(i.op3.op);
00982             out << "\tmovq $" << base_offset << ", %rcx\n";
00983         }
00984     } else {
00985         out << "\txorq %rcx, %rcx\n";
00986     }
00987
00988     if (!i.vop.empty() && !i.vop[0].op.empty()) {
00989         if (isVariable(i.vop[0].op)) {
00990             x64_emitLoadVar(out, "%r8", i.vop[0]);
00991             out << "\timulq %r8, %rcx\n";
00992             out << "\taddq %rcx, %rax\n";
00993
00994             switch (dest.type) {
00995                 case VarType::VAR_INTEGER:
00996                 case VarType::VAR_POINTER:
00997                 case VarType::VAR_EXTERN:
00998                     out << "\tmovq (%rax), %rdx\n";
00999                     x64_emitStoreVar(out, "%rdx", i.op1);
01000                     break;
01001                 case VarType::VAR_FLOAT:
01002                     out << "\tmovsd (%rax), %xmm0\n";
01003                     out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "({rip})\n";
01004                     break;
01005                 case VarType::VAR_BYTE:
01006                     out << "\tmovzbq (%rax), %rdx\n";
01007                     out << "\tmovb %dl, " << getMangledName(i.op1) << "({rip})\n";
01008                     break;

```

```

01009         default:
01010             throw mx::Exception("LOAD: unsupported destination type");
01011     }
01012 } else {
01013     size_t stride = static_cast<size_t>(std::stoll(i.vop[0].op, nullptr, 0));
01014
01015     if (stride == 1 || stride == 2 || stride == 4 || stride == 8) {
01016         switch (dest.type) {
01017             case VarType::VAR_INTEGER:
01018             case VarType::VAR_POINTER:
01019             case VarType::VAR_EXTERN:
01020                 out << "\tmovq (%rax,%rcx," << stride << "), %rdx\n";
01021                 x64_emitStoreVar(out, "%rdx", i.op1);
01022                 break;
01023             case VarType::VAR_FLOAT:
01024                 out << "\tmovsd (%rax,%rcx," << stride << "), %xmm0\n";
01025                 out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01026                 break;
01027             case VarType::VAR_BYTE:
01028                 out << "\tmovzbq (%rax,%rcx," << stride << "), %rdx\n";
01029                 out << "\tmovb %dl, " << getMangledName(i.op1) << "(%rip)\n";
01030                 break;
01031             default:
01032                 throw mx::Exception("LOAD: unsupported destination type");
01033         }
01034     } else {
01035         out << "\timulq $" << stride << ", %rcx\n";
01036         out << "\taddq %rcx, %rax\n";
01037
01038         switch (dest.type) {
01039             case VarType::VAR_INTEGER:
01040             case VarType::VAR_POINTER:
01041             case VarType::VAR_EXTERN:
01042                 out << "\tmovq (%rax), %rdx\n";
01043                 x64_emitStoreVar(out, "%rdx", i.op1);
01044                 break;
01045             case VarType::VAR_FLOAT:
01046                 out << "\tmovsd (%rax), %xmm0\n";
01047                 out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01048                 break;
01049             case VarType::VAR_BYTE:
01050                 out << "\tmovzbq (%rax), %rdx\n";
01051                 out << "\tmovb %dl, " << getMangledName(i.op1) << "(%rip)\n";
01052                 break;
01053             default:
01054                 throw mx::Exception("LOAD: unsupported destination type");
01055         }
01056     }
01057 }
01058 } else {
01059     switch (dest.type) {
01060     case VarType::VAR_INTEGER:
01061     case VarType::VAR_POINTER:
01062     case VarType::VAR_EXTERN:
01063         out << "\tmovq (%rax,%rcx,8), %rdx\n";
01064         x64_emitStoreVar(out, "%rdx", i.op1);
01065         break;
01066     case VarType::VAR_FLOAT:
01067         out << "\tmovsd (%rax,%rcx,8), %xmm0\n";
01068         out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01069         break;
01070     case VarType::VAR_BYTE:
01071         out << "\tmovzbq (%rax,%rcx,8), %rdx\n";
01072         out << "\tmovb %dl, " << getMangledName(i.op1) << "(%rip)\n";
01073         break;
01074     default:
01075         throw mx::Exception("LOAD: unsupported destination type");
01076     }
01077 }
01078
01079 out << "\tjmp .load_done_" << error_label_count << "\n";
01080 out << ".null_ptr_error_" << error_label_count << ":\n";
01081 out << "\t# Handle null pointer error\n";
01082 out << ".load_done_" << error_label_count << ":\n";
01083
01084 error_label_count++;
01085 }
01086
01087 void Program::x64_gen_store(std::ostream &out, const Instruction &i) {
01088     if (!isVariable(i.op2.op)) {
01089         throw mx::Exception("STORE destination must be a variable");
01090     }
01091
01092     Variable &ptrVar = getVariable(i.op2.op);
01093     if (ptrVar.type != VarType::VAR_POINTER &&
01094         !(ptrVar.type == VarType::VAR_STRING && ptrVar.var_value.buffer_size > 0)) {
01095         throw mx::Exception("STORE destination must be a pointer or string buffer");
01096     }

```



```

01096     }
01097
01098     if (ptrVar.type == VarType::VAR_POINTER) {
01099         // Flush all cached registers before pointer store
01100         x64_emitFlushRegs(out);
01101         out << "\tmovq " << getMangledName(i.op2) << "(%rip), %rcx\n";
01102     } else {
01103         out << "\tleaq " << getMangledName(i.op2) << "(%rip), %rcx\n";
01104     }
01105
01106     out << "\ttest %rcx, %rcx\n";
01107     out << "\tjz .null_ptr_error_" << error_label_count << "\n";
01108
01109     size_t base_offset = 0;
01110     if (!i.op3.op.empty()) {
01111         if (isVariable(i.op3.op)) {
01112             x64_emitLoadVar(out, "%rdx", i.op3);
01113         } else {
01114             base_offset = std::stoll(i.op3.op);
01115             out << "\tmovq $" << base_offset << ", %rdx\n";
01116         }
01117     } else {
01118         out << "\txorb %rdx, %rdx\n";
01119     }
01120
01121     // Load source value into %rax / %xmm0 / %dl before computing final address
01122     // (so we don't clobber %rcx/%rdx during source load)
01123     bool src_is_var = isVariable(i.op1.op);
01124     VarType src_type = VarType::VAR_INTEGER;
01125     if (src_is_var) {
01126         Variable &src = getVariable(i.op1.op);
01127         src_type = src.type;
01128         switch (src.type) {
01129             case VarType::VAR_INTEGER:
01130             case VarType::VAR_POINTER:
01131             case VarType::VAR_EXTERN:
01132                 x64_emitLoadVar(out, "%rax", i.op1);
01133                 break;
01134             case VarType::VAR_STRING:
01135                 out << "\tleaq " << getMangledName(i.op1) << "(%rip), %rax\n";
01136                 break;
01137             case VarType::VAR_BYTE:
01138                 out << "\tmovzbq " << getMangledName(i.op1) << "(%rip), %rax\n";
01139                 break;
01140             case VarType::VAR_FLOAT:
01141                 out << "\tmovsd " << getMangledName(i.op1) << "(%rip), %xmm0\n";
01142                 break;
01143             default:
01144                 throw mx::Exception("STORE: unsupported source type");
01145         }
01146     } else {
01147         if (i.op1.op.find('.') != std::string::npos ||
01148             i.op1.op.find('e') != std::string::npos ||
01149             i.op1.op.find('E') != std::string::npos) {
01150             double val = std::stod(i.op1.op);
01151             uint64_t bits;
01152             memcpy(&bits, &val, sizeof(bits));
01153             out << "\tmovq $" << bits << ", %rax\n";
01154         } else {
01155             out << "\tmovq $" << i.op1.op << ", %rax\n";
01156         }
01157     }
01158
01159     if (!i.vop.empty() && !i.vop[0].op.empty()) {
01160         if (isVariable(i.vop[0].op)) {
01161             x64_emitLoadVar(out, "%r8", i.vop[0]);
01162             out << "\timulq %r8, %rdx\n";
01163             out << "\taddq %rdx, %rcx\n";
01164
01165             if (src_is_var && src_type == VarType::VAR_FLOAT)
01166                 out << "\tmovsd %xmm0, (%rcx)\n";
01167             else if (src_is_var && src_type == VarType::VAR_BYTE)
01168                 out << "\tmovb %al, (%rcx)\n";
01169             else
01170                 out << "\tmovq %rax, (%rcx)\n";
01171         } else {
01172             size_t stride = static_cast<size_t>(std::stoll(i.vop[0].op, nullptr, 0));
01173
01174             if (stride == 1 || stride == 2 || stride == 4 || stride == 8) {
01175                 if (src_is_var && src_type == VarType::VAR_FLOAT)
01176                     out << "\tmovsd %xmm0, (%rcx,%rdx," << stride << ")\n";
01177                 else if (src_is_var && src_type == VarType::VAR_BYTE)
01178                     out << "\tmovb %al, (%rcx,%rdx," << stride << ")\n";
01179                 else
01180                     out << "\tmovq %rax, (%rcx,%rdx," << stride << ")\n";
01181             } else {
01182                 out << "\timulq $" << stride << ", %rdx\n";

```

```

01183         out << "\taddq %rdx, %rcx\n";
01184
01185         if (src_is_var && src_type == VarType::VAR_FLOAT)
01186             out << "\tmovsd %xmm0, (%rcx)\n";
01187         else if (src_is_var && src_type == VarType::VAR_BYTE)
01188             out << "\tmovb %al, (%rcx)\n";
01189         else
01190             out << "\tmovq %rax, (%rcx)\n";
01191     }
01192 }
01193 } else {
01194     out << "\tshlq $3, %rdx\n";
01195     out << "\taddq %rdx, %rcx\n";
01196
01197     if (src_is_var && src_type == VarType::VAR_FLOAT)
01198         out << "\tmovsd %xmm0, (%rcx)\n";
01199     else if (src_is_var && src_type == VarType::VAR_BYTE)
01200         out << "\tmovb %al, (%rcx)\n";
01201     else
01202         out << "\tmovq %rax, (%rcx)\n";
01203 }
01204
01205 out << "\tjmp .store_done_" << error_label_count << "\n";
01206 out << ".null_ptr_error_" << error_label_count << ":\n";
01207 out << "\t# Handle null pointer error\n";
01208 out << ".store_done_" << error_label_count << ":\n";
01209
01210 // Reload cached registers after pointer store (memory may have changed)
01211 if (ptrVar.type == VarType::VAR_POINTER) {
01212     x64_emitReloadRegs(out);
01213 }
01214
01215 error_label_count++;
01216 }
01217
01218 void Program::x64_gen_to_int(std::ostream &out, const Instruction &i) {
01219     out << "\tleaq " << getMangledName(i.op2) << "(%rip), %rcx\n";
01220     size_t total = x64_reserve_call_area(out, 0);
01221     out << "\tcall atol\n";
01222     x64_release_call_area(out, total);
01223     x64_emitStoreVar(out, "%rax", i.op1);
01224 }
01225
01226 void Program::x64_gen_to_float(std::ostream &out, const Instruction &i) {
01227     if (!isVariable(i.op1.op) || !isVariable(i.op2.op)) {
01228         throw mx::Exception("TO_FLOAT requires variable operands");
01229     }
01230     x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01231     out << "\tcvtsi2sdq %rax, %xmm0\n";
01232     out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01233 }
01234
01235 void Program::x64_gen_div(std::ostream &out, const Instruction &i) {
01236     if (i.op3.op.empty()) {
01237         if (isVariable(i.op1.op)) {
01238             Variable &v = getVariable(i.op1.op);
01239             if (v.type == VarType::VAR_INTEGER) {
01240                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op2);
01241                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op1);
01242                 out << "\tcmpq $0, %rcx\n";
01243                 out << "\tje 1f\n";
01244                 out << "\tqto\n";
01245                 out << "\tidivq %rcx\n";
01246                 out << "\tjmp 2f\n";
01247                 out << "1:\n";
01248                 out << "\txor %eax, %eax\n";
01249                 out << "2:\n";
01250                 x64_emitStoreVar(out, "%rax", i.op1);
01251             } else if (v.type == VarType::VAR_FLOAT) {
01252                 x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
01253                 x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op2);
01254                 out << "\txorpd %xmm2, %xmm2\n";
01255                 out << "\tucomisd %xmm2, %xmm1\n";
01256                 out << "\tje 1f\n";
01257                 out << "\tdivsd %xmm1, %xmm0\n";
01258                 out << "\tjmp 2f\n";
01259                 out << "1:\n";
01260                 out << "\txorpd %xmm0, %xmm0\n";
01261                 out << "2:\n";
01262                 out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01263             } else
01264                 throw mx::Exception("DIV unsupported type");
01265         } else
01266             throw mx::Exception("DIV: first must be variable");
01267     } else {
01268         if (isVariable(i.op1.op)) {
01269             Variable &v = getVariable(i.op1.op);

```

```

01270         if (v.type == VarType::VAR_INTEGER) {
01271             x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op3);
01272             x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01273             out << "\tcmpq $0, %rcx\n";
01274             out << "\tje 1f\n";
01275             out << "\tcqto\n";
01276             out << "\tidivq %rcx\n";
01277             out << "\tjmp 2f\n";
01278             out << "1:\n";
01279             out << "\txor %eax, %eax\n";
01280             out << "2:\n";
01281             x64_emitStoreVar(out, "%rax", i.op1);
01282         } else if (v.type == VarType::VAR_FLOAT) {
01283             x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op2);
01284             x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op3);
01285             out << "\txorpd %xmm2, %xmm2\n";
01286             out << "\tucomisd %xmm2, %xmm1\n";
01287             out << "\tje 1f\n";
01288             out << "\tdivsd %xmm1, %xmm0\n";
01289             out << "\tjmp 2f\n";
01290             out << "1:\n";
01291             out << "\txorpd %xmm0, %xmm0\n";
01292             out << "2:\n";
01293             out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01294         } else
01295             throw mx::Exception("DIV unsupported type");
01296     } else
01297         throw mx::Exception("DIV: first must be variable");
01298 }
01299
01300 void Program::x64_gen_mod(std::ostream &out, const Instruction &i) {
01301     if (i.op3.op.empty()) {
01302         if (isVariable(i.op1.op)) {
01303             Variable &v = getVariable(i.op1.op);
01304             if (v.type == VarType::VAR_INTEGER) {
01305                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op2);
01306                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op1);
01307                 out << "\tcmpq $0, %rcx\n";
01308                 out << "\tje 1f\n";
01309                 out << "\tcqto\n";
01310                 out << "\tidivq %rcx\n";
01311                 out << "\tjmp 2f\n";
01312                 out << "1:\n";
01313                 out << "\txor %edx, %edx\n";
01314                 out << "2:\n";
01315                 x64_emitStoreVar(out, "%rdx", i.op1);
01316             } else
01317                 throw mx::Exception("MOD int only");
01318         } else
01319             throw mx::Exception("MOD: first must be variable");
01320     } else {
01321         if (isVariable(i.op1.op)) {
01322             Variable &v = getVariable(i.op1.op);
01323             if (v.type == VarType::VAR_INTEGER) {
01324                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op3);
01325                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01326                 out << "\tcmpq $0, %rcx\n";
01327                 out << "\tje 1f\n";
01328                 out << "\tcqto\n";
01329                 out << "\tidivq %rcx\n";
01330                 out << "\tjmp 2f\n";
01331                 out << "1:\n";
01332                 out << "\txor %edx, %edx\n";
01333                 out << "2:\n";
01334                 x64_emitStoreVar(out, "%rdx", i.op1);
01335             } else
01336                 throw mx::Exception("MOD int only");
01337         } else
01338             throw mx::Exception("MOD: first must be variable");
01339     }
01340 }
01341
01342 void Program::x64_gen_getline(std::ostream &out, const Instruction &i) {
01343     if (!isVariable(i.op1.op))
01344         throw mx::Exception("GETLINE: dest must be var");
01345     Variable &dest = getVariable(i.op1.op);
01346     if (dest.type != VarType::VAR_STRING || dest.var_value.buffer_size == 0)
01347         throw mx::Exception("GETLINE: needs string buffer");
01348
01349     out << "\txor %ecx, %ecx\n";
01350     size_t t0 = x64_reserve_call_area(out, 0);
01351     out << "\tcall __acrt_iob_func\n";
01352     x64_release_call_area(out, t0);
01353     out << "\tmov %rax, %r8\n";
01354
01355     out << "\tleaq " << getMangledName(i.op1) << "(%rip), %rcx\n";

```

```

01357         out << "\tmovq $" << dest.var_value.buffer_size << ", %rdx\n";
01358         size_t total = x64_reserve_call_area(out, 0);
01359         out << "\tcall fgets\n";
01360         x64_release_call_area(out, total);
01361
01362         static size_t over_count = 0;
01363         out << "\ttest %rax, %rax\n";
01364         out << "\tje .over" << over_count << "\n";
01365
01366         out << "\tleaq " << getMangledName(i.op1) << "(%rip), %rcx\n";
01367         size_t tlen = x64_reserve_call_area(out, 0);
01368         out << "\tcall strlen\n";
01369         x64_release_call_area(out, tlen);
01370
01371         out << "\tmov %rax, %rcx\n";
01372         out << "\tcmp $0, %rax\n";
01373         out << "\tje .over" << over_count << "\n";
01374         out << "\tsub $1, %rcx\n";
01375         out << "\tleaq " << getMangledName(i.op1) << "(%rip), %rax\n";
01376         out << "\tmovb $0, (%rax, %rcx, 1)\n";
01377         out << ".over" << over_count++ << "\n";
01378     }
01379
01380 void Program::x64_gen_bitop(std::ostream &out, const std::string &opc, const Instruction &i) {
01381     if (i.op3.op.empty()) {
01382         if (isVariable(i.op1.op)) {
01383             Variable &v = getVariable(i.op1.op);
01384             if (v.type == VarType::VAR_INTEGER) {
01385                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op1);
01386                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op2);
01387                 out << "\t" << opc << "q %rcx, %rax\n";
01388                 x64_emitStoreVar(out, "%rax", i.op1);
01389             } else
01390                 throw mx::Exception("bitop int only");
01391         } else
01392             throw mx::Exception("bitop first must be variable");
01393     } else {
01394         if (isVariable(i.op1.op)) {
01395             Variable &v = getVariable(i.op1.op);
01396             if (v.type == VarType::VAR_INTEGER) {
01397                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01398                 x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", i.op3);
01399                 out << "\t" << opc << "q %rcx, %rax\n";
01400                 x64_emitStoreVar(out, "%rax", i.op1);
01401             } else
01402                 throw mx::Exception("bitop int only");
01403         } else
01404             throw mx::Exception("bitop first must be variable");
01405     }
01406 }
01407
01408 void Program::x64_gen_not(std::ostream &out, const Instruction &i) {
01409     if (!i.op1.op.empty()) {
01410         if (isVariable(i.op1.op)) {
01411             Variable &v = getVariable(i.op1.op);
01412             if (v.type != VarType::VAR_INTEGER)
01413                 throw mx::Exception("NOT int only");
01414             x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op1);
01415             out << "\ttestq %rax, %rax\n";
01416             out << "\tsete %al\n";
01417             out << "\tmovzbq %al, %rax\n";
01418             x64_emitStoreVar(out, "%rax", i.op1);
01419         } else
01420             throw mx::Exception("NOT requires variable");
01421     } else
01422         throw mx::Exception("NOT requires operand");
01423 }
01424
01425 void Program::x64_gen_push(std::ostream &out, const Instruction &i) {
01426     if (isVariable(i.op1.op)) {
01427         Variable &v = getVariable(i.op1.op);
01428         if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER || v.type ==
VarType::VAR_EXTERN) {
01429             x64_emitLoadVar(out, "%rax", i.op1);
01430             out << "\tpushq %rax\n";
01431             x64_sp_mod16 ^= 8;
01432         } else if (v.type == VarType::VAR_STRING) {
01433             out << "\tleaq " << getMangledName(i.op1) << "(%rip), %rax\n";
01434             out << "\tpushq %rax\n";
01435             x64_sp_mod16 ^= 8;
01436         } else {
01437             throw mx::Exception("PUSH supports int/pointer/string");
01438         }
01439     } else if (i.op1.type == OperandType::OP_CONSTANT) {
01440         out << "\tmovq $" << i.op1.op << ", %rax\n";
01441         out << "\tpushq %rax\n";
01442         x64_sp_mod16 ^= 8;

```

```

01443         } else {
01444             throw mx::Exception("PUSH requires var or const");
01445         }
01446     }
01447
01448     void Program::x64_gen_pop(std::ostream &out, const Instruction &i) {
01449         if (!isVariable(i.op1.op))
01450             throw mx::Exception("POP dest must be a variable");
01451         Variable &v = getVariable(i.op1.op);
01452         if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER || v.type ==
VarType::VAR_EXTERN) {
01453             out << "\tpopq %rax\n";
01454             x64_sp_mod16 ^= 8;
01455             x64_emitStoreVar(out, "%rax", i.op1);
01456         } else {
01457             throw mx::Exception("POP supports int/pointer");
01458         }
01459     }
01460
01461     void Program::x64_gen_stack_sub(std::ostream &out, const Instruction &i) {
01462         if (!i.op1.op.empty()) {
01463             if (isVariable(i.op1.op))
01464                 x64_emitLoadVar(out, "%rcx", i.op1);
01465             else /* constant */
01466                 out << "\tmovq $" << i.op1.op << ", %rcx\n";
01467         } else {
01468             out << "\tmovq $1, %rcx\n";
01469         }
01470         out << "\tshl $3, %rcx\n";
01471         out << "\taddq %rcx, %rsp\n";
01472
01473         if (!i.op1.op.empty() && !isVariable(i.op1.op)) {
01474             unsigned long long n = std::stoull(i.op1.op, nullptr, 0);
01475             if (n & 1ull)
01476                 x64_sp_mod16 ^= 8;
01477         }
01478     }
01479
01480     void Program::x64_gen_stack_load(std::ostream &out, const Instruction &i) {
01481         if (!isVariable(i.op1.op))
01482             throw mx::Exception("stack_load first arg variable");
01483         if (i.op2.op.empty())
01484             throw mx::Exception("stack_load requires index");
01485         x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01486         out << "\tmovq (%rsp, %rax, 8), %rcx\n";
01487         x64_emitStoreVar(out, "%rcx", i.op1);
01488     }
01489
01490     void Program::x64_gen_stack_store(std::ostream &out, const Instruction &i) {
01491         if (!isVariable(i.op1.op))
01492             throw mx::Exception("stack_store first arg variable");
01493         if (i.op2.op.empty())
01494             throw mx::Exception("stack_store requires index");
01495         x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01496         x64_emitLoadVar(out, "%rcx", i.op1);
01497         out << "\tmovq %rcx, (%rsp, %rax, 8)\n";
01498     }
01499
01500     void Program::x64_gen_print(std::ostream &out, const Instruction &i) {
01501         xmm_offset = 0;
01502         std::vector<Operand> args;
01503         args.push_back(i.op1);
01504         if (!i.op2.op.empty())
01505             args.push_back(i.op2);
01506         if (!i.op3.op.empty())
01507             args.push_back(i.op3);
01508         for (const auto &v : i.vop)
01509             if (!v.op.empty())
01510                 args.push_back(v);
01511         x64_generateFunctionCall(out, "printf", args);
01512     }
01513
01514     void Program::x64_gen_jmp(std::ostream &out, const Instruction &i) {
01515         if (!i.op1.op.empty()) {
01516             auto pos = labels.find(i.op1.op);
01517             if (pos == labels.end())
01518                 throw mx::Exception("Jump must have valid label: " + i.op1.op);
01519             const char *m = nullptr;
01520             switch (i.instruction) {
01521                 case JMP:
01522                     m = "jmp";
01523                     break;
01524                 case JE:
01525                     m = "je";
01526                     break;
01527                 case JNE:
01528                     m = "jne";

```

```

01529         break;
01530     case JL:
01531         m = (last_cmp_type == CMP_FLOAT) ? "jb" : "jl";
01532         break;
01533     case JLE:
01534         m = (last_cmp_type == CMP_FLOAT) ? "jbe" : "jle";
01535         break;
01536     case JG:
01537         m = (last_cmp_type == CMP_FLOAT) ? "ja" : "jg";
01538         break;
01539     case JGE:
01540         m = (last_cmp_type == CMP_FLOAT) ? "jae" : "jge";
01541         break;
01542     case JZ:
01543         m = "jz";
01544         break;
01545     case JNZ:
01546         m = "jnz";
01547         break;
01548     case JA:
01549         m = "ja";
01550         break;
01551     case JB:
01552         m = "jb";
01553         break;
01554     default:
01555         break;
01556     }
01557     out << "\t" << m << " ." << i.op1.op << "\n";
01558 } else
01559     throw mx::Exception("Jump requires label");
01560 }
01561
01562 void Program::x64_gen_cmp(std::ostream &out, const Instruction &i) {
01563     if (i.op2.op.empty())
01564         throw mx::Exception("CMP requires two operands");
01565     VarType t1 = VarType::VAR_INTEGER, t2 = VarType::VAR_INTEGER;
01566     if (isVariable(i.op1.op))
01567         t1 = getVariable(i.op1.op).type;
01568     if (isVariable(i.op2.op))
01569         t2 = getVariable(i.op2.op).type;
01570
01571     if (t1 == VarType::VAR_FLOAT || t2 == VarType::VAR_FLOAT) {
01572         if (t1 == VarType::VAR_FLOAT) {
01573             x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
01574         } else {
01575             x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op1);
01576             out << "\tcvtsi2sdq %rax, %xmm0\n";
01577         }
01578
01579         if (t2 == VarType::VAR_FLOAT) {
01580             x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op2);
01581         } else {
01582             x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", i.op2);
01583             out << "\tcvtsi2sdq %rax, %xmm1\n";
01584         }
01585
01586         out << "\tcomisd %xmm1, %xmm0\n";
01587         last_cmp_type = CMP_FLOAT;
01588     } else if (t1 == VarType::VAR_POINTER && (t2 == VarType::VAR_INTEGER || t2 ==
VarType::VAR_BYTE)) {
01589         x64_generateLoadVar(out, t1, "%rax", i.op1);
01590         x64_generateLoadVar(out, t2, "%rcx", i.op2);
01591         out << "\tcmpq %rcx, %rax\n";
01592         last_cmp_type = CMP_INTEGER;
01593     } else if (!isVariable(i.op2.op) && i.op2.type == OperandType::OP_CONSTANT &&
isVariable(i.op1.op)) {
01594         auto ra = x64_reg_vars.find(i.op1.op);
01595         if (ra != x64_reg_vars.end()) {
01596             out << "\tcmpq $" << i.op2.op << ", " << ra->second << "\n";
01597         } else if (t1 == VarType::VAR_BYTE) {
01598             out << "\tmovzbq " << getMangledName(i.op1) << "(%rip), %rax\n";
01599             out << "\tcmpq $" << i.op2.op << ", %rax\n";
01600         } else {
01601             out << "\tcmpq $" << i.op2.op << ", " << getMangledName(i.op1) << "(%rip)\n";
01602         }
01603         last_cmp_type = CMP_INTEGER;
01604     } else {
01605         x64_generateLoadVar(out, t1, "%rax", i.op1);
01606         x64_generateLoadVar(out, t2, "%rcx", i.op2);
01607         out << "\tcmpq %rcx, %rax\n";
01608         last_cmp_type = CMP_INTEGER;
01609     }
01610 }
01611 void Program::x64_gen_arth(std::ostream &out, std::string arth, const Instruction &i) {
01612     auto emitIntegerOp = [&](const Operand &lhs, const Operand &rhs, const Operand &dest) {
01613         if (arth == "mul")

```

```

01614         arth = "imul";
01615         if (!isVariable(rhs.op) && rhs.type == OperandType::OP_CONSTANT && arth != "imul" &&
lhs.op == dest.op && isVariable(lhs.op)) {
01616             auto ra = x64_reg_vars.find(lhs.op);
01617             if (ra != x64_reg_vars.end()) {
01618                 out << "\t" << arth << "q $" << rhs.op << ", " << ra->second << "\n";
01619             } else {
01620                 out << "\t" << arth << "q $" << rhs.op << ", " << getMangledName(lhs) << "(%rip)\n";
01621             }
01622             return;
01623         }
01624         x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rax", lhs);
01625         if (!isVariable(rhs.op) && rhs.type == OperandType::OP_CONSTANT) {
01626             out << "\t" << arth << "q $" << rhs.op << ", %rax\n";
01627         } else {
01628             x64_generateLoadVar(out, VarType::VAR_INTEGER, "%rcx", rhs);
01629             out << "\t" << arth << "q %rcx, %rax\n";
01630         }
01631         x64_emitStoreVar(out, "%rax", dest);
01632     };
01633
01634     if (i.op3.op.empty()) {
01635         if (!isVariable(i.op1.op))
01636             throw mx::Exception("arth first must be variable");
01637         Variable &v = getVariable(i.op1.op);
01638         if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER) {
01639             emitIntegerOp(i.op1, i.op2, i.op1);
01640         } else if (v.type == VarType::VAR_FLOAT) {
01641             x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
01642             x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op2);
01643             out << "\t" << arth << "sd %xmm1, %xmm0\n";
01644             out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01645         } else
01646             throw mx::Exception("arth unsupported type");
01647     } else {
01648         if (!isVariable(i.op1.op))
01649             throw mx::Exception("arth first must be variable");
01650         Variable &v = getVariable(i.op1.op);
01651         if (v.type == VarType::VAR_INTEGER || v.type == VarType::VAR_POINTER) {
01652             emitIntegerOp(i.op2, i.op3, i.op1);
01653         } else if (v.type == VarType::VAR_FLOAT) {
01654             x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op2);
01655             x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op3);
01656             out << "\t" << arth << "sd %xmm1, %xmm0\n";
01657             out << "\tmovsd %xmm0, " << getMangledName(i.op1) << "(%rip)\n";
01658         } else
01659             throw mx::Exception("arth unsupported type");
01660     }
01661 }
01662
01663 void Program::x64_gen_exit(std::ostream &out, const Instruction &i) {
01664
01665     bool uses_std_module = false;
01666     for (const auto &e : external) {
01667         if (e.mod == "std") {
01668             uses_std_module = true;
01669             break;
01670         }
01671     }
01672
01673     if (uses_std_module) {
01674         out << "\t# Clean up program arguments before exit\n";
01675         size_t total = x64_reserve_call_area(out, 0);
01676         out << "\tcall free_program_args\n";
01677         x64_release_call_area(out, total);
01678     }
01679
01680     if (!i.op1.op.empty()) {
01681         std::vector<Operand> opz;
01682         opz.push_back(i.op1);
01683         x64_generateFunctionCall(out, "exit", opz);
01684     } else
01685         throw mx::Exception("exit requires argument");
01686 }
01687
01688 void Program::x64_gen_fcmp(std::ostream &out, const Instruction &i) {
01689     if (i.op2.op.empty()) {
01690         throw mx::Exception("FCMP requires two operands");
01691     }
01692
01693     x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm0", i.op1);
01694     x64_generateLoadVar(out, VarType::VAR_FLOAT, "%xmm1", i.op2);
01695     out << "\tcomisd %xmm1, %xmm0\n";
01696     last_cmp_type = CMP_FLOAT;
01697 }
01698
01699 void Program::x64_gen_jae(std::ostream &out, const Instruction &i) {

```

```

01700         out << "\tjae ." << i.op1.op << "\n";
01701     }
01702
01703     void Program::x64_gen_jbe(std::ostream &out, const Instruction &i) {
01704         out << "\tjbe ." << i.op1.op << "\n";
01705     }
01706
01707     void Program::x64_gen_jc(std::ostream &out, const Instruction &i) {
01708         out << "\tjc ." << i.op1.op << "\n";
01709     }
01710
01711     void Program::x64_gen_jnc(std::ostream &out, const Instruction &i) {
01712         out << "\tjnc ." << i.op1.op << "\n";
01713     }
01714
01715     void Program::x64_gen_jp(std::ostream &out, const Instruction &i) {
01716         out << "\tjp ." << i.op1.op << "\n";
01717     }
01718
01719     void Program::x64_gen_jnp(std::ostream &out, const Instruction &i) {
01720         out << "\tjnp ." << i.op1.op << "\n";
01721     }
01722
01723     void Program::x64_gen_jo(std::ostream &out, const Instruction &i) {
01724         out << "\tjo ." << i.op1.op << "\n";
01725     }
01726
01727     void Program::x64_gen_jno(std::ostream &out, const Instruction &i) {
01728         out << "\tjno ." << i.op1.op << "\n";
01729     }
01730
01731     void Program::x64_gen_js(std::ostream &out, const Instruction &i) {
01732         out << "\tjs ." << i.op1.op << "\n";
01733     }
01734
01735     void Program::x64_gen_jns(std::ostream &out, const Instruction &i) {
01736         out << "\tjns ." << i.op1.op << "\n";
01737     }
01738 } // namespace mxvm

```

12.94 src/icode_opt.cpp File Reference

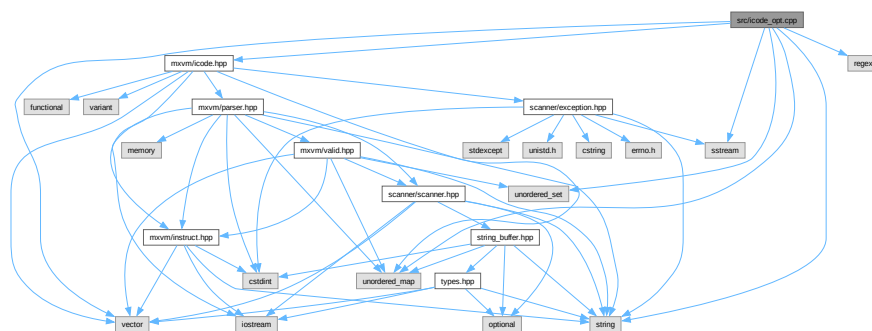
Peephole optimizer for generated x86-64 assembly (SysV and Win64).

```

#include "mxvm/icode.hpp"
#include <regex>
#include <sstream>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>

```

Include dependency graph for icode_opt.cpp:



Namespaces

- namespace `mxvm`

Functions

- static std::string [mxvm::join_lines](#) (const std::vector< std::string > &v)
- static bool [mxvm::is_label](#) (const std::string &line)
- static std::vector< std::string > [mxvm::opt_core_lines](#) (const std::vector< std::string > &lines)
- static std::vector< std::string > [mxvm::x64_opt_core_lines](#) (const std::vector< std::string > &lines)
- static std::string [mxvm::darwin_prefix_calls](#) (const std::string &line, const std::unordered_set< std::string > &macos_functions)
- static std::vector< std::string > [mxvm::opt_darwin_lines](#) (const std::vector< std::string > &lines)
- static std::vector< std::string > [mxvm::opt_x64_windows_lines](#) (const std::vector< std::string > &lines)
- static std::vector< std::string > [mxvm::opt_linux_lines](#) (const std::vector< std::string > &lines)

12.94.1 Detailed Description

Peephole optimizer for generated x86-64 assembly (SysV and Win64).

Author

Jared Bruni

Definition in file [icode_opt.cpp](#).

12.95 icode_opt.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "mxvm/icode.hpp"
00007 #include <regex>
00008 #include <sstream>
00009 #include <string>
00010 #include <unordered_map>
00011 #include <unordered_set>
00012 #include <vector>
00013
00014 namespace mxvm {
00015
00016     static std::string join_lines(const std::vector<std::string> &v) {
00017         std::ostringstream os;
00018         for (size_t i = 0; i < v.size(); ++i) {
00019             os << v[i];
00020             if (i + 1 < v.size())
00021                 os << '\n';
00022         }
00023         return os.str();
00024     }
00025
00026     /*static inline std::string trim(std::string s) {
00027         auto isspace = [](int c){ return std::isspace(c); };
00028         while (!s.empty() && isspace(s.front())) s.erase(s.begin());
00029         while (!s.empty() && isspace(s.back())) s.pop_back();
00030         return s;
00031     }*/
00032
00033     static bool is_label(const std::string &line) {
00034         static const std::regex re_label(R"(\s*(?:[A-Za-z_.$] [\w.$]*|\d+)\s*:)" );
00035         return std::regex_search(line, re_label);
00036     }
00037
00038     static std::vector<std::string> opt_core_lines(const std::vector<std::string> &lines) {
00039         std::regex re_mov_reg(R"(\s*mov\bwlq?\s+(\%[a-z0-9]+\s*,\s*(\%[a-z0-9]+\s*(?:\[[#;].*\)?\s$)|
std::regex::icase);
00040         std::regex
re_mem_to_reg(R"(\s*mov\bwlq?\s+(\%[A-Za-z0-9_]+(?:\(%rip\))?)\s*,\s*(\%[a-z0-9]+\s*(?:\[[#;].*\)?\s$)|
std::regex::icase);
00041         std::regex
re_reg_to_mem(R"(\s*mov\bwlq?\s+(\%[a-z0-9]+\s*,\s*(\%[A-Za-z0-9_]+(?:\(%rip\))?)\s*(?:\[[#;].*\)?\s$)|
std::regex::icase);
00042         std::regex
re_modifies_reg(R"(\s*(?:add|sub|mul|imul|div|idiv|and|or|xor|shl|shr|sal|sar|not|neg) [a-z]*\s*,\s*(\%[a-z0-9]+\s*(?:\[[#;].*\)?\s$)|
std::regex::icase);
00043         std::regex
re_call_or_jump(R"(\s*(?:call|jmp|je|jne|jz|jnz|jg|jge|jl|jle|ja|jae|jb|jbe) [a-z]*\s+)" ,
std::regex::icase);
00044         std::regex re_xor_eax(R"(\s*xorq?\s+%eax\s*,\s*(\%eax\s*(?:\[[#;].*\)?\s$)|
std::regex::icase);

```

```

00045     std::regex re_call_func(R"(^\\s*call\\s+([_A-Za-z][_A-Za-z0-9]*)\\s*(?:[#;].*)?$)",
std::regex::icase);
00046     static const std::unordered_set<std::string> variadic_funcs = {
00047         "printf", "fprintf", "sprintf", "snprintf", "scanf", "fscanf", "sscanf",
00048         "vprintf", "vfprintf", "vsprintf", "vsnprintf", "vscanf", "vfscanf", "vsscanf"};
00049     std::vector<std::string> out;
00050     out.reserve(lines.size());
00051     struct ValueInfo {
00052         std::string location;
00053         bool valid = true;
00054     };
00055     std::unordered_map<std::string, ValueInfo> reg_contents;
00056     std::unordered_map<std::string, ValueInfo> mem_contents;
00057     auto invalidate_tracking = [&]() {
00058         reg_contents.clear();
00059         mem_contents.clear();
00060     };
00061     auto clobber_reg = [&](const std::string &reg) {
00062         reg_contents.erase(reg);
00063         for (auto it = mem_contents.begin(); it != mem_contents.end(); ) {
00064             if (it->second.location == reg)
00065                 it = mem_contents.erase(it);
00066             else
00067                 ++it;
00068         }
00069     };
00070     for (size_t i = 0; i < lines.size(); ++i) {
00071         const std::string &line = lines[i];
00072
00073         if (std::regex_match(line, re_xor_eax) && (i + 1 < lines.size())) {
00074             std::smatch m;
00075             if (std::regex_match(lines[i + 1], m, re_call_func)) {
00076                 std::string func_name = m[1].str();
00077                 if (variadic_funcs.find(func_name) == variadic_funcs.end()) {
00078                     continue;
00079                 }
00080             }
00081         }
00082         if (is_label(line) || std::regex_search(line, re_call_or_jump)) {
00083             out.push_back(line);
00084             invalidate_tracking();
00085             continue;
00086         }
00087     }
00088     {
00089         std::smatch m;
00090         if (std::regex_match(line, m, re_modifies_reg)) {
00091             clobber_reg(m[1].str());
00092             out.push_back(line);
00093             continue;
00094         }
00095     }
00096     {
00097         std::smatch m;
00098         if (std::regex_match(line, m, re_mem_to_reg)) {
00099             const std::string mem = m[1].str();
00100             const std::string reg = m[2].str();
00101
00102             clobber_reg(reg);
00103
00104             auto it = mem_contents.find(mem);
00105             if (it != mem_contents.end() && it->second.valid) {
00106                 const std::string &src_reg = it->second.location;
00107                 if (src_reg != reg)
00108                     out.push_back("\tmovq " + src_reg + ", " + reg);
00109                 reg_contents[reg] = {mem, true};
00110                 continue;
00111             }
00112         }
00113         out.push_back(line);
00114         reg_contents[reg] = {mem, true};
00115         continue;
00116     }
00117     }
00118 }
00119
00120 {
00121     std::smatch m;
00122     if (std::regex_match(line, m, re_reg_to_mem)) {
00123         const std::string reg = m[1].str();
00124         const std::string mem = m[2].str();
00125         mem_contents[mem] = {reg, true};
00126         out.push_back(line);
00127         continue;
00128     }
00129 }
00130

```

```

00131         {
00132             std::smatch m;
00133             if (std::regex_match(line, m, re_mov_reg)) {
00134                 const std::string src = m[1].str();
00135                 const std::string dst = m[2].str();
00136                 if (src == dst)
00137                     continue;
00138
00139                 clobber_reg(dst);
00140
00141                 auto it_src = reg_contents.find(src);
00142                 if (it_src != reg_contents.end())
00143                     reg_contents[dst] = it_src->second;
00144                 else
00145                     reg_contents[dst] = {src, true};
00146
00147                 out.push_back(line);
00148                 continue;
00149             }
00150         }
00151
00152         invalidate_tracking();
00153         out.push_back(line);
00154     }
00155
00156     return out;
00157 }
00158
00159 static std::vector<std::string> x64_opt_core_lines(const std::vector<std::string> &lines) {
00160     std::regex re_mov_reg(R"(\s*movq?\s+(\%[a-z0-9]+\s*,\s*(\%[a-z0-9]+\s*(?:[#;].*)?))\s*",
std::regex::icase);
00161     std::regex
re_mem_to_reg(R"(\s*movq?\s+(\%[a-z0-9_]+(?:\(%rip\)\s*,\s*(\%[a-z0-9]+\s*(?:[#;].*)?))\s*",
std::regex::icase);
00162     std::regex
re_reg_to_mem(R"(\s*movq?\s+(\%[a-z0-9]+\s*,\s*(\%[a-z0-9_]+(?:\(%rip\)\s*(?:[#;].*)?))\s*",
std::regex::icase);
00163     std::regex
re_modifies_reg(R"(\s*(?:add|sub|mul|imul|div|idiv|and|or|xor|shl|shr|sal|sar|not|neg)[a-z]*\s*,\s*(\%[a-z0-9]+\s*(?:
std::regex::icase);
00164     std::regex
re_call_or_jump(R"(\s*(?:call|jmp|jle|jne|jz|jnz|jg|jge|jl|jle|ja|jae|jb|jbe)[a-z]*\s+)\s*",
std::regex::icase);
00165
00166     std::regex re_xor_eax(R"(\s*xorq?\s+%eax\s*,\s+%eax\s*(?:[#;].*)?)", std::regex::icase);
00167     std::regex re_call_func(R"(\s*call\s+(\[_A-Za-z\]_A-Za-Z0-9*)\s*(?:[#;].*)?)",
std::regex::icase);
00168     static const std::unordered_set<std::string> variadic_funcs = {
00169         "printf", "fprintf", "sprintf", "snprintf", "scanf", "fscanf", "sscanf",
00170         "vprintf", "vfprintf", "vsprintf", "vsnprintf", "vscanf", "vfscanf", "vsscanf"};
00171     auto local_is_label = [](const std::string &s) {
00172         static const std::regex re(R"(\s*(?:[_A-Za-z_.$][\w.$]*|\d+)\s*:)",
00173         return std::regex_search(s, re);
00174     };
00175     auto has_call_or_ret = [](const std::string &s) {
00176         return s.find(" call ") != std::string::npos ||
00177             s.find("\tcall ") != std::string::npos ||
00178             s.find(" ret") != std::string::npos ||
00179             s.find("\tret") != std::string::npos;
00180     };
00181     auto touches_rsp = [](const std::string &s) {
00182         return s.find("%rsp") != std::string::npos || s.find("(%rsp)") != std::string::npos;
00183     };
00184
00185     std::vector<std::string> out;
00186     out.reserve(lines.size());
00187     std::vector<int> frame_bytes;
00188
00189     std::regex re_sub_rsp(R"(\s*sub\s+\$([0-9]+\s*),\s*%rsp\s*(?:[#;].*)?)", std::regex::icase);
00190     std::regex re_add_rsp(R"(\s*add\s+\$([0-9]+\s*),\s*%rsp\s*(?:[#;].*)?)", std::regex::icase);
00191
00192     struct ValueInfo {
00193         std::string location;
00194         bool valid = true;
00195     };
00196     std::unordered_map<std::string, ValueInfo> reg_contents;
00197     std::unordered_map<std::string, ValueInfo> mem_contents;
00198
00199     auto invalidate_tracking = [&]() {
00200         reg_contents.clear();
00201         mem_contents.clear();
00202     };
00203
00204     auto clobber_reg = [&](const std::string &reg) {
00205         reg_contents.erase(reg);
00206         for (auto it = mem_contents.begin(); it != mem_contents.end(); ) {
00207             if (it->second.location == reg)

```

```

00208         it = mem_contents.erase(it);
00209     else
00210         ++it;
00211     }
00212 };
00213
00214 for (size_t i = 0; i < lines.size(); ++i) {
00215     const std::string &line = lines[i];
00216
00217     if (std::regex_match(line, re_xor_eax) && (i + 1 < lines.size())) {
00218         std::smatch m;
00219         if (std::regex_match(lines[i + 1], m, re_call_func)) {
00220             std::string func_name = m[1].str();
00221             if (!func_name.empty() && func_name[0] == '_') {
00222                 func_name = func_name.substr(1);
00223             }
00224             if (variadic_funcs.find(func_name) == variadic_funcs.end()) {
00225                 continue;
00226             }
00227         }
00228     }
00229     if (local_is_label(line)) {
00230         out.push_back(line);
00231         invalidate_tracking();
00232         continue;
00233     }
00234     {
00235         std::smatch m;
00236         if (std::regex_match(line, m, re_sub_rsp)) {
00237             frame_bytes.push_back(std::stoi(m[1].str()));
00238             out.push_back(line);
00239             continue;
00240         }
00241     }
00242     if (!frame_bytes.empty()) {
00243         out.push_back(line);
00244         std::smatch m;
00245         if (std::regex_match(line, m, re_add_rsp)) {
00246             int n = std::stoi(m[1].str());
00247             if (!frame_bytes.empty() && frame_bytes.back() == n)
00248                 frame_bytes.pop_back();
00249         }
00250         continue;
00251     }
00252     if (has_call_or_ret(line) || touches_rsp(line) || std::regex_search(line, re_call_or_jump))
00253     {
00254         out.push_back(line);
00255         invalidate_tracking();
00256         continue;
00257     }
00258     {
00259         std::smatch m;
00260         if (std::regex_match(line, m, re_modifies_reg)) {
00261             clobber_reg(m[1].str());
00262             out.push_back(line);
00263             continue;
00264         }
00265     }
00266     {
00267         std::smatch m;
00268         if (std::regex_match(line, m, re_reg_to_mem)) {
00269             const std::string reg = m[1].str();
00270             const std::string mem = m[2].str();
00271             mem_contents[mem] = {reg, true};
00272             out.push_back(line);
00273             continue;
00274         }
00275     }
00276     {
00277         std::smatch m;
00278         if (std::regex_match(line, m, re_mov_reg)) {
00279             const std::string src = m[1].str();
00280             const std::string dst = m[2].str();
00281             if (src == dst)
00282                 continue;
00283             clobber_reg(dst);
00284             auto it_src = reg_contents.find(src);
00285             if (it_src != reg_contents.end())
00286                 reg_contents[dst] = it_src->second;
00287             else
00288                 reg_contents[dst] = {src, true};
00289         }
00290     }
00291 }
00292
00293

```

```

00294         out.push_back(line);
00295         continue;
00296     }
00297 }
00298
00299 {
00300     std::smatch m;
00301     if (std::regex_match(line, m, re_mem_to_reg)) {
00302         const std::string mem = m[1].str();
00303         const std::string reg = m[2].str();
00304
00305         clobber_reg(reg);
00306
00307         auto it = mem_contents.find(mem);
00308         if (it != mem_contents.end() && it->second.valid) {
00309             const std::string &src_reg = it->second.location;
00310             if (src_reg != reg)
00311                 out.push_back("\tmovq " + src_reg + ", " + reg);
00312             reg_contents[reg] = {mem, true};
00313             continue;
00314         }
00315
00316         out.push_back(line);
00317         reg_contents[reg] = {mem, true};
00318         continue;
00319     }
00320 }
00321
00322 invalidate_tracking();
00323 out.push_back(line);
00324 }
00325
00326 return out;
00327 }
00328
00329 static std::string darwin_prefix_calls(const std::string &line,
00330                                       const std::unordered_set<std::string> &macos_functions) {
00331     static const std::regex re_call(R"(\b(call|jmp)\s+([_A-Za-z][_A-Za-z0-9]*)\b)");
00332     std::string result;
00333     result.reserve(line.size() + 16);
00334
00335     size_t last = 0;
00336     for (std::sregex_iterator it(line.begin(), line.end(), re_call), end; it != end; ++it) {
00337         const auto &m = *it;
00338         const auto pos = static_cast<size_t>(m.position());
00339         const auto len = static_cast<size_t>(m.length());
00340         const std::string fn = m[2].str();
00341
00342         result.append(line, last, pos - last);
00343
00344         std::string piece = m.str();
00345         if (fn == "main" || macos_functions.count(fn)) {
00346             const auto fnpos = piece.rfind(fn);
00347             if (fnpos != std::string::npos)
00348                 piece.replace(fnpos, fn.size(), "_" + fn);
00349         }
00350         result += piece;
00351
00352         last = pos + len;
00353     }
00354     result.append(line, last, std::string::npos);
00355     return result;
00356 }
00357
00358 static std::vector<std::string> opt_darwin_lines(const std::vector<std::string> &lines) {
00359     std::vector<std::string> out;
00360     out.reserve(lines.size());
00361
00362     std::regex re_stdin_access(R"(movq?\s+stdin\(%rip\)\s*,\s*(%w+))", std::regex::icase);
00363     std::regex re_stdout_access(R"(movq?\s+stdout\(%rip\)\s*,\s*(%w+))", std::regex::icase);
00364     std::regex re_stderr_access(R"(movq?\s+stderr\(%rip\)\s*,\s*(%w+))", std::regex::icase);
00365     std::regex re_global_main(R"(\s*\.(?:global|globl)\s+main\s*$)");
00366     std::regex re_main_label(R"(\s*main\s*:\s*$)");
00367     std::regex re_global_function(R"(\s*\.(?:global|globl)\s+([_a-zA-Z][_a-zA-Z0-9]*)\s*(?:\s|$))");
00368     std::regex re_function_label(R"(\s*([_a-zA-Z][_a-zA-Z0-9]*)\s*:\s*$)");
00369
00370     std::unordered_set<std::string> macos_functions;
00371
00372     for (const auto &raw : lines) {
00373         std::string line = raw;
00374
00375         if (std::regex_search(line, re_global_main)) {
00376             out.push_back("\t.globl _main");
00377             continue;
00378         }
00379         if (std::regex_search(line, re_main_label)) {
00380             out.push_back("_main:");

```

```

00381         continue;
00382     }
00383
00384     std::smatch m_func;
00385     if (std::regex_search(line, m_func, re_global_function)) {
00386         std::string fn = m_func[1].str();
00387         if (fn != "main") {
00388             macos_functions.insert(fn);
00389             out.push_back("\t.globl _" + fn);
00390         }
00391         continue;
00392     }
00393     if (std::regex_search(line, m_func, re_function_label)) {
00394         std::string fn = m_func[1].str();
00395         if (fn != "main" && macos_functions.count(fn)) {
00396             out.push_back("_" + fn + ":" );
00397         } else {
00398             out.push_back(line);
00399         }
00400         continue;
00401     }
00402
00403     std::smatch m;
00404     if (std::regex_search(line, m, re_stdin_access)) {
00405         std::string r = m[1].str();
00406         out.push_back("\tmovq __stdin@GOTPCREL(%rip), %rax");
00407         out.push_back("\tmovq (%rax), " + r);
00408         continue;
00409     }
00410     if (std::regex_search(line, m, re_stdout_access)) {
00411         std::string r = m[1].str();
00412         out.push_back("\tmovq __stdout@GOTPCREL(%rip), %rax");
00413         out.push_back("\tmovq (%rax), " + r);
00414         continue;
00415     }
00416     if (std::regex_search(line, m, re_stderr_access)) {
00417         std::string r = m[1].str();
00418         out.push_back("\tmovq __stderr@GOTPCREL(%rip), %rax");
00419         out.push_back("\tmovq (%rax), " + r);
00420         continue;
00421     }
00422
00423     std::regex re_single_stdout(R"(\s*movq?\s+stdout\(%rip\) \s*, \s*(%w+) \s*(?:[#;].*)? $)",
std::regex::icase);
00424     if (std::regex_search(line, m, re_single_stdout)) {
00425         std::string r = m[1].str();
00426         out.push_back("\tmovq __stdout@GOTPCREL(%rip), %rax");
00427         out.push_back("\tmovq (%rax), " + r);
00428         continue;
00429     }
00430     std::regex re_single_stderr(R"(\s*movq?\s+stderr\(%rip\) \s*, \s*(%w+) \s*(?:[#;].*)? $)",
std::regex::icase);
00431     if (std::regex_search(line, m, re_single_stderr)) {
00432         std::string r = m[1].str();
00433         out.push_back("\tmovq __stderr@GOTPCREL(%rip), %rax");
00434         out.push_back("\tmovq (%rax), " + r);
00435         continue;
00436     }
00437     std::regex re_single_stdin(R"(\s*movq?\s+stdin\(%rip\) \s*, \s*(%w+) \s*(?:[#;].*)? $)",
std::regex::icase);
00438     if (std::regex_search(line, m, re_single_stdin)) {
00439         std::string r = m[1].str();
00440         out.push_back("\tmovq __stdin@GOTPCREL(%rip), %rax");
00441         out.push_back("\tmovq (%rax), " + r);
00442         continue;
00443     }
00444
00445     line = darwin_prefix_calls(line, macos_functions);
00446     out.push_back(line);
00447 }
00448
00449 return out;
00450 }
00451
00452 static std::vector<std::string> opt_x64_windows_lines(const std::vector<std::string> &lines) {
00453     static const std::regex
add_rx(R"(\s*addq?\s+\$([0-9]+|0x[0-9a-fA-F]+) \s*, \s*%rsp\s*(?:[#;].*)? $)");
00454     static const std::regex
sub_rx(R"(\s*subq?\s+\$([0-9]+|0x[0-9a-fA-F]+) \s*, \s*%rsp\s*(?:[#;].*)? $)");
00455
00456     auto parse_imm = [](const std::string &s) -> unsigned long long {
00457         return std::stoull(s, nullptr, 0);
00458     };
00459
00460     std::vector<std::string> out;
00461     out.reserve(lines.size());
00462

```

```

00463         for (size_t i = 0; i < lines.size(); i++) {
00464             const std::string &a = lines[i];
00465
00466             std::smatch ma, mb;
00467             bool a_is_add = std::regex_match(a, ma, add_rx);
00468             bool a_is_sub = !a_is_add && std::regex_match(a, ma, sub_rx);
00469
00470             if ((a_is_add || a_is_sub) && (i + 1) < lines.size()) {
00471                 const std::string &b = lines[i + 1];
00472                 bool b_is_add = std::regex_match(b, mb, add_rx);
00473                 bool b_is_sub = !b_is_add && std::regex_match(b, mb, sub_rx);
00474
00475                 if ((a_is_add && b_is_sub) || (a_is_sub && b_is_add)) {
00476                     unsigned long long ia = parse_imm(ma[1].str());
00477                     unsigned long long ib = parse_imm(mb[1].str());
00478                     if (ia == ib) {
00479                         i += 2;
00480                         continue;
00481                     }
00482                 }
00483             }
00484
00485             out.push_back(a);
00486             ++i;
00487         }
00488
00489         return out;
00490     }
00491
00492     static std::vector<std::string> opt_linux_lines(const std::vector<std::string> &lines) {
00493         return lines;
00494     }
00495
00496     std::string Program::gen_optimize(const std::string &code, const Platform &platform_name) {
00497         std::vector<std::string> lines;
00498         {
00499             std::istringstream stream(code);
00500             std::string s;
00501             while (std::getline(stream, s))
00502                 lines.push_back(s);
00503         }
00504
00505         if (platform_name == Platform::WINX64) {
00506             auto core = x64_opt_core_lines(lines);
00507             auto win = opt_x64_windows_lines(core);
00508             return join_lines(win);
00509         } else {
00510             auto core = opt_core_lines(lines);
00511             auto os = (platform_name == Platform::DARWIN) ? opt_darwin_lines(core)
00512                                                            : opt_linux_lines(core);
00513             return join_lines(os);
00514         }
00515     }
00516 } // namespace mxvm

```

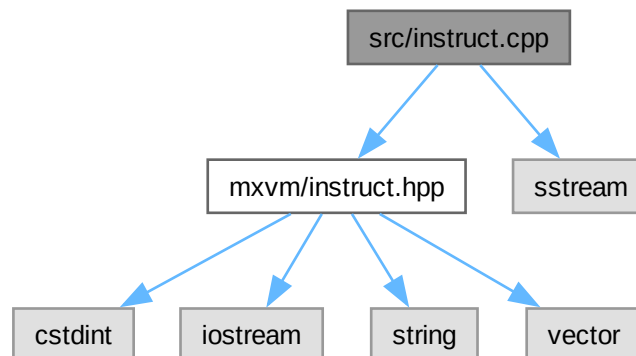
12.96 src/instruct.cpp File Reference

Instruction toString(), keyword table, and opcode stream operator.

```
#include "mxvm/instruct.hpp"
```

```
#include <sstream>
```

Include dependency graph for `instruct.cpp`:



Namespaces

- namespace `mxvm`

Functions

- `std::ostream & operator<< (std::ostream &out, const mxvm::Keywords &key)`
Stream insertion operator for `Keywords`.
- `std::ostream & operator<< (std::ostream &out, const enum Inc &i)`
Stream insertion operator for `Inc` opcodes.
- `std::ostream & operator<< (std::ostream &out, const mxvm::Instruction &inc)`
Stream insertion operator for `Instruction`.
- `std::ostream & operator<< (std::ostream &out, const mxvm::VarType &type)`
Stream insertion operator for `VarType`.

12.96.1 Detailed Description

Instruction `toString()`, keyword table, and opcode stream operator.

Author

Jared Bruni

Definition in file `instruct.cpp`.

12.96.2 Function Documentation

12.96.2.1 `operator<<()` [1/4]

```
std::ostream & operator<< (
    std::ostream & out,
    const enum Inc & i)
```

Stream insertion operator for `Inc` opcodes.

Parameters

<i>out</i>	Output stream
<i>i</i>	Opcode to print

Returns

Reference to the output stream

Definition at line 23 of file [instruct.cpp](#).

References [IncType](#), and [pos\(\)](#).

12.96.2.2 operator<<() [2/4]

```
std::ostream & operator<< (  
    std::ostream & out,  
    const mxvm::Instruction & inc)
```

Stream insertion operator for Instruction.

Parameters

<i>out</i>	Output stream
<i>inc</i>	Instruction to format and print

Returns

Reference to the output stream

Definition at line 32 of file [instruct.cpp](#).

References [mxvm::Instruction::instruction](#), [mxvm::Operand::op](#), [mxvm::Instruction::op1](#), [mxvm::Instruction::op2](#), [mxvm::Instruction::op3](#), and [mxvm::Instruction::vop](#).

12.96.2.3 operator<<() [3/4]

```
std::ostream & operator<< (  
    std::ostream & out,  
    const mxvm::Keywords & key)
```

Stream insertion operator for Keywords.

Parameters

<i>out</i>	Output stream
<i>key</i>	Keyword to print

Returns

Reference to the output stream

Definition at line 14 of file [instruct.cpp](#).

References [mxvm::keywords](#).

12.96.2.4 operator<<() [4/4]

```
std::ostream & operator<< (  
    std::ostream & out,  
    const mxvm::VarType & type)
```

Stream insertion operator for VarType.

Parameters

<i>out</i>	Output stream
<i>type</i>	Variable type to print

Returns

Reference to the output stream

Definition at line 54 of file `instruct.cpp`.

References `mxvm::VAR_BYTE`, `mxvm::VAR_EXTERN`, `mxvm::VAR_FLOAT`, `mxvm::VAR_INTEGER`, `mxvm::VAR_POINTER`, and `mxvm::VAR_STRING`.

12.97 instruct.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "mxvm/instruct.hpp"
00007 #include <sstream>
00008
00009 namespace mxvm {
00010     std::vector<std::string> keywords{"program", "object", "module", "data", "code", "section"};
00011 }
00012
00013
00014 std::ostream &operator<<(std::ostream &out, const mxvm::Keywords &key) {
00015     size_t i = static_cast<size_t>(key);
00016     if (i < mxvm::keywords.size())
00017         out << mxvm::keywords[i];
00018     else
00019         out << "Error";
00020     return out;
00021 }
00022
00023 std::ostream &operator<<(std::ostream &out, const enum Inc &i) {
00024     size_t pos = static_cast<size_t>(i);
00025     if (pos < IncType.size())
00026         out << IncType[pos];
00027     else
00028         out << "[undefined]";
00029     return out;
00030 }
00031
00032 std::ostream &operator<<(std::ostream &out, const mxvm::Instruction &inc) {
00033     out << "[" << inc.instruction;
00034     std::ostringstream opc;
00035     if (!inc.op1.op.empty())
00036         opc << " " << inc.op1.op;
00037     if (!inc.op2.op.empty())
00038         opc << ", " << inc.op2.op;
00039     if (!inc.op3.op.empty())
00040         opc << ", " << inc.op3.op;
00041     for (const auto &arg : inc.vop) {
00042         if (!arg.op.empty())
00043             opc << ", " << arg.op;
00044     }
00045     if (opc.str().length() > 0) {
00046         out << opc.str() << "]";
00047     } else {
00048         out << "]";
00049     }
00050
00051     return out;
00052 }
00053
00054 std::ostream &operator<<(std::ostream &out, const mxvm::VarType &type) {
00055     switch (type) {
00056     case mxvm::VarType::VAR_INTEGER:
00057         out << "int";
00058         break;
00059
00060     case mxvm::VarType::VAR_STRING:
00061         out << "string";
00062         break;
00063     case mxvm::VarType::VAR_EXTERN:
00064         out << "extern";
00065         break;
00066     case mxvm::VarType::VAR_POINTER:

```

```

00067         out << "ptr";
00068         break;
00069     case mxvm::VarType::VAR_FLOAT:
00070         out << "float";
00071         break;
00072     case mxvm::VarType::VAR_BYTE:
00073         out << "byte";
00074         break;
00075
00076     default:
00077         out << "unknown";
00078     }
00079     return out;
00080 }
00081
00082 namespace mxvm {
00083
00084     std::string Instruction::toString() const {
00085         std::ostringstream stream;
00086         stream << *this;
00087         return stream.str();
00088     }
00089
00090     std::string Variable::toString() const {
00091         std::ostringstream stream;
00092         stream << type;
00093         return stream.str();
00094     }
00095 } // namespace mxvm

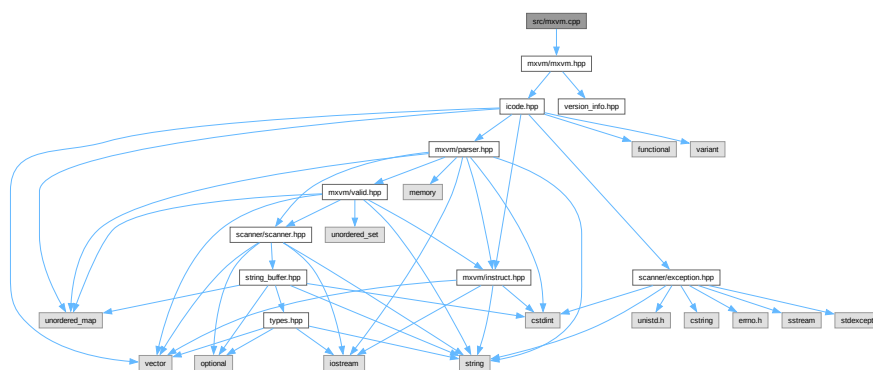
```

12.98 src/mxvm.cpp File Reference

MXVM namespace placeholder (reserved for future top-level utilities).

```
#include "mxvm/mxvm.hpp"
```

Include dependency graph for mxvm.cpp:



Namespaces

- namespace [mxvm](#)

12.98.1 Detailed Description

MXVM namespace placeholder (reserved for future top-level utilities).

Author

Jared Bruni

Definition in file [mxvm.cpp](#).

12.99 mxvm.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "mxvm/mxvm.hpp"
00007
00008 namespace mxvm {
00009
00010 }

```

12.100 src/scanner/exception.cpp File Reference

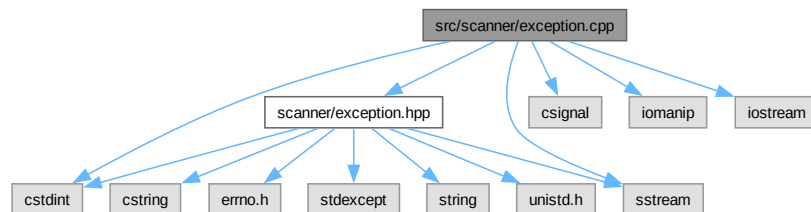
Hex formatting utilities and exception helper implementations.

```

#include "scanner/exception.hpp"
#include <csignal>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <sstream>

```

Include dependency graph for exception.cpp:



Namespaces

- namespace [mx](#)

Functions

- `std::string mx::format\_hex64 (uint64_t value)`
Format a 64-bit value as a hex string with 0x prefix.
- `std::string mx::format\_hex32 (uint32_t value)`
Format a 32-bit value as a hex string with 0x prefix.
- `std::string mx::format\_hex16 (uint16_t value)`
Format a 16-bit value as a hex string with 0x prefix.
- `std::string mx::format\_hex8 (uint8_t value)`
Format an 8-bit value as a hex string with 0x prefix.
- `std::string mx::format\_hex\_no\_prefix (uint64_t value)`
Format a 64-bit value as a hex string without 0x prefix.
- `std::string mx::format\_signal (uint32_t sig)`
Format a signal number as a descriptive string.

12.100.1 Detailed Description

Hex formatting utilities and exception helper implementations.

Author

Jared Bruni

Definition in file [exception.cpp](#).

12.101 exception.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "scanner/exception.hpp"
00007 #include <csignal>
00008 #include <cstdint>
00009 #include <iomanip>
00010 #include <iostream>
00011 #include <sstream>
00012
00013 namespace mx {
00014     std::string format_hex64(uint64_t value) {
00015         std::ostringstream oss;
00016         oss << "0x" << std::setfill('0') << std::setw(16) << std::hex << std::uppercase << value;
00017         return oss.str();
00018     }
00019
00020     std::string format_hex32(uint32_t value) {
00021         std::ostringstream oss;
00022         oss << "0x" << std::setfill('0') << std::setw(8) << std::hex << std::uppercase << value;
00023         return oss.str();
00024     }
00025
00026     std::string format_hex16(uint16_t value) {
00027         std::ostringstream oss;
00028         oss << "0x" << std::setfill('0') << std::setw(4) << std::hex << std::uppercase << value;
00029         return oss.str();
00030     }
00031
00032     std::string format_hex8(uint8_t value) {
00033         std::ostringstream oss;
00034         oss << "0x" << std::setfill('0') << std::setw(2) << std::hex << std::uppercase <<
static_cast<unsigned>(value);
00035         return oss.str();
00036     }
00037
00038     std::string format_hex_no_prefix(uint64_t value) {
00039         std::ostringstream stream;
00040         stream << std::hex << std::uppercase << value;
00041         return stream.str();
00042     }
00043
00044     std::string format_signal(uint32_t sig) {
00045         switch (sig) {
00046             case SIGHUP:
00047                 return "SIGHUP (Hangup)";
00048             case SIGINT:
00049                 return "SIGINT (Interrupt)";
00050             case SIGQUIT:
00051                 return "SIGQUIT (Quit)";
00052             case SIGILL:
00053                 return "SIGILL (Illegal instruction)";
00054             case SIGTRAP:
00055                 return "SIGTRAP (Trace/breakpoint trap)";
00056             case SIGABRT:
00057                 return "SIGABRT (Aborted)";
00058             case SIGBUS:
00059                 return "SIGBUS (Bus error)";
00060             case SIGFPE:
00061                 return "SIGFPE (Floating point exception)";
00062             case SIGKILL:
00063                 return "SIGKILL (Killed)";
00064             case SIGUSR1:
00065                 return "SIGUSR1 (User defined signal 1)";
00066             case SIGSEGV:
00067                 return "SIGSEGV (Segmentation fault)";
00068             case SIGUSR2:
00069                 return "SIGUSR2 (User defined signal 2)";
00070             case SIGPIPE:
00071                 return "SIGPIPE (Broken pipe)";
00072             case SIGALRM:
00073                 return "SIGALRM (Alarm clock)";
00074             case SIGTERM:
00075                 return "SIGTERM (Terminated)";
00076             case SIGCHLD:
00077                 return "SIGCHLD (Child exited)";
00078             case SIGCONT:
00079                 return "SIGCONT (Continued)";
00080             case SIGSTOP:
00081                 return "SIGSTOP (Stopped)";
00082             case SIGTSTP:
00083                 return "SIGTSTP (Stopped (tty input))";
00084             case SIGTTIN:
00085                 return "SIGTTIN (Stopped (tty input))";
00086             case SIGTTOU:

```


General-purpose exception with errno-aware factory method.

Namespaces

- namespace `mx`
- namespace `mx::Color`

ANSI escape code constants for terminal coloring.

Functions

- `std::string mx::format_hex64` (uint64_t value)
Format a 64-bit value as a hex string with 0x prefix.
- `std::string mx::format_hex32` (uint32_t value)
Format a 32-bit value as a hex string with 0x prefix.
- `std::string mx::format_hex16` (uint16_t value)
Format a 16-bit value as a hex string with 0x prefix.
- `std::string mx::format_hex8` (uint8_t value)
Format an 8-bit value as a hex string with 0x prefix.
- `std::string mx::format_signal` (uint32_t sig)
Format a signal number as a descriptive string.
- `std::string mx::format_hex_no_prefix` (uint64_t value)
Format a 64-bit value as a hex string without 0x prefix.
- `bool mx::terminal_supports_color` ()
Check whether stdout is a TTY that supports ANSI color.
- `std::string Col` (const std::string &col, std::string color)
Wrap a string with ANSI color codes if terminal supports color.

Variables

- `const std::string mx::Color::RESET` = "\033[0m"
- `const std::string mx::Color::BOLD` = "\033[1m"
- `const std::string mx::Color::DIM` = "\033[2m"
- `const std::string mx::Color::BLACK` = "\033[30m"
- `const std::string mx::Color::RED` = "\033[31m"
- `const std::string mx::Color::GREEN` = "\033[32m"
- `const std::string mx::Color::YELLOW` = "\033[33m"
- `const std::string mx::Color::BLUE` = "\033[34m"
- `const std::string mx::Color::MAGENTA` = "\033[35m"
- `const std::string mx::Color::CYAN` = "\033[36m"
- `const std::string mx::Color::WHITE` = "\033[37m"
- `const std::string mx::Color::BRIGHT_RED` = "\033[91m"
- `const std::string mx::Color::BRIGHT_GREEN` = "\033[92m"
- `const std::string mx::Color::BRIGHT_YELLOW` = "\033[93m"
- `const std::string mx::Color::BRIGHT_BLUE` = "\033[94m"
- `const std::string mx::Color::BRIGHT_MAGENTA` = "\033[95m"
- `const std::string mx::Color::BRIGHT_CYAN` = "\033[96m"
- `const std::string mx::Color::BG_RED` = "\033[41m"
- `const std::string mx::Color::BG_GREEN` = "\033[42m"
- `const std::string mx::Color::BG_YELLOW` = "\033[43m"
- `bool mx::color_ = terminal_supports_color`()
cached color support flag

12.102.1 Detailed Description

Exception class, hex formatting utilities, and terminal color definitions.

Author

Jared Bruni

Definition in file [exception.hpp](#).

12.102.2 Function Documentation

12.102.2.1 Col()

```
std::string Col (
    const std::string & col,
    std::string color) [inline]
```

Wrap a string with ANSI color codes if terminal supports color.

Parameters

<i>col</i>	The text to colorize
<i>color</i>	The ANSI escape sequence to apply

Returns

Colorized string, or plain text if color is unsupported

Definition at line 100 of file [exception.hpp](#).

References [mx::color_](#), and [mx::Color::RESET](#).

Referenced by [mxvm::Program::~~Program\(\)](#), [action_interpret\(\)](#), [createMakefile\(\)](#), [mxvm::Program::generateCode\(\)](#), [mxvm::Parser::generateProgramCode\(\)](#), [print_help\(\)](#), [proc_args\(\)](#), [process_arguments\(\)](#), [translate_x64\(\)](#), and [mxvm::Program::x64_generateCode\(\)](#).

12.103 exception.hpp

[Go to the documentation of this file.](#)

```
00001
00006 #ifndef _EXCEPTION_H_1
00007 #define _EXCEPTION_H_1
00008
00009 #include <cstdint>
00010 #include <cstring>
00011 #include <errno.h>
00012 #include <sstream>
00013 #include <stdexcept>
00014 #include <string>
00015 #include <unistd.h>
00016
00017 namespace mx {
00018
00020     std::string format_hex64(uint64_t value);
00022     std::string format_hex32(uint32_t value);
00024     std::string format_hex16(uint16_t value);
00026     std::string format_hex8(uint8_t value);
00028     std::string format_signal(uint32_t sig);
00030     std::string format_hex_no_prefix(uint64_t value);
00031
00038     class Exception : public std::runtime_error {
00039     public:
00044         explicit Exception(const std::string &message)
00045             : std::runtime_error(message) {}
00046
00051         explicit Exception(const char *message)
00052             : std::runtime_error(message) {}
00053
00059         static Exception error(const std::string &context) {
00060             return Exception(context + ": " + std::strerror(errno));
00061         }
00061
```



```

00062     };
00063
00065     namespace Color {
00066         const std::string RESET = "\033[0m";
00067         const std::string BOLD = "\033[1m";
00068         const std::string DIM = "\033[2m";
00069         const std::string BLACK = "\033[30m";
00070         const std::string RED = "\033[31m";
00071         const std::string GREEN = "\033[32m";
00072         const std::string YELLOW = "\033[33m";
00073         const std::string BLUE = "\033[34m";
00074         const std::string MAGENTA = "\033[35m";
00075         const std::string CYAN = "\033[36m";
00076         const std::string WHITE = "\033[37m";
00077         const std::string BRIGHT_RED = "\033[91m";
00078         const std::string BRIGHT_GREEN = "\033[92m";
00079         const std::string BRIGHT_YELLOW = "\033[93m";
00080         const std::string BRIGHT_BLUE = "\033[94m";
00081         const std::string BRIGHT_MAGENTA = "\033[95m";
00082         const std::string BRIGHT_CYAN = "\033[96m";
00083         const std::string BG_RED = "\033[41m";
00084         const std::string BG_GREEN = "\033[42m";
00085         const std::string BG_YELLOW = "\033[43m";
00086     } // namespace Color
00088     inline bool terminal_supports_color() {
00089         return isatty(STDOUT_FILENO) && getenv("TERM") != nullptr;
00090     }
00091     inline bool color_ = terminal_supports_color();
00092 } // namespace mx
00093
00100 inline std::string Col(const std::string &col, std::string color) {
00101     std::ostringstream o;
00102     if (mx::color_)
00103         o << color;
00104     o << col;
00105     if (mx::color_)
00106         o << mx::Color::RESET;
00107     return o.str();
00108 }
00109
00110 #endif

```

12.104 src/scanner/include/scanner/scanner.hpp File Reference

Lexical scanner that tokenizes source text into typed tokens.

```

#include "string_buffer.hpp"
#include <iostream>
#include <optional>
#include <string>
#include <vector>

```


Functions

- `std::ostream & operator<< (std::ostream &out, const types::CharType &c)`
Stream output operator for CharType.
- `std::ostream & operator<< (std::ostream &out, const types::TokenType &t)`
Stream output operator for TokenType.

12.104.1 Detailed Description

Lexical scanner that tokenizes source text into typed tokens.

Author

Jared Bruni

Definition in file [scanner.hpp](#).

12.104.2 Function Documentation

12.104.2.1 `operator<<()` [1/2]

```
std::ostream & operator<< (
    std::ostream & out,
    const types::CharType & c)
```

Stream output operator for CharType.

12.104.2.2 `operator<<()` [2/2]

```
std::ostream & operator<< (
    std::ostream & out,
    const types::TokenType & t)
```

Stream output operator for TokenType.

12.105 scanner.hpp

[Go to the documentation of this file.](#)

```
00001
00006 #ifndef _SCANNER_H_
00007 #define _SCANNER_H_
00008
00009 #include "string_buffer.hpp"
00010 #include <iostream>
00011 #include <optional>
00012 #include <string>
00013 #include <vector>
00014
00015 namespace scan {
00016
00017     using TString = buffer::StringBuffer<char>;
00018     using StringType = buffer::StringBuffer<char>::string_type;
00019     using TToken = token::Token<char>;
00020     using TMap = token::TokenMap<char>;
00021
00022     class Scanner {
00023     public:
00024         Scanner(const TString &b);
00025         uint64_t scan();
00026         std::optional<TToken> grabId();
00027         std::optional<TToken> grabDigits();
00028         std::optional<TToken> grabSymbols();
00029         std::optional<TToken> grabString();
00030         std::optional<TToken> grabSingle();
00031
00032         TToken &operator[](size_t index);
00033         size_t size() const;
00034         void removeEOL();
00035         std::vector<TToken> &getTokens() { return tokens; }
00036
00037     private:
00038         TString string_buffer;
```


Character-by-character string buffer with line/column tracking.

- class `scan::token::TokenMap< Ch, MAX_SIZE >`
Maps characters to their lexical category (letter, digit, symbol, etc.).
- class `scan::token::Token< Ch, String >`
A scanned token with type, value, and source position.

Namespaces

- namespace `scan`
- namespace `scan::buffer`
- namespace `scan::token`

Enumerations

- enum class `scan::token::CharType`
Classification of individual characters during lexing.

12.106.1 Detailed Description

Character-by-character string buffer with line/column tracking and Token/TokenMap types.

Author

Jared Bruni

Definition in file `string_buffer.hpp`.

12.107 string_buffer.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef _STR_BUF_X
00007 #define _STR_BUF_X
00008
00009 #include "types.hpp"
00010 #include <stdint>
00011 #include <optional>
00012 #include <string>
00013 #include <unordered_map>
00014
00015 namespace scan {
00016     namespace buffer {
00026         template <typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>
00027         class StringBuffer {
00028         public:
00029             using ch_type = Ch;
00030             using string_type = String;
00036             StringBuffer(const string_type &buf, const std::string &filename = "unknown");
00038             std::optional<ch_type> getch();
00040             std::optional<ch_type> curch();
00045             std::optional<ch_type> peekch(int num = 1);
00047             std::optional<ch_type> prevch();
00052             std::optional<ch_type> forward_step(int num = 1);
00057             std::optional<ch_type> backward_step(int num = 1);
00059             std::pair<uint64_t, uint64_t> cur_line();
00061             std::string cur_file() const;
00067             void process_line_directive(uint64_t newLine, const std::string &newFile);
00072             bool eof(uint64_t pos = 0);
00077             void reset(uint64_t pos = 0);
00078
00079         private:
00080             string_type buffer_;
00081             uint64_t index;
00082             uint64_t currentLine;
00083             uint64_t currentColumn;
00084             std::string currentFile;
00085         };
00086
00087         template <typename Ch, typename String>
00088         StringBuffer<Ch, String>::StringBuffer(const string_type &buf,
00089                                                 const std::string &filename)
00090             : buffer_{buf + " \n"}, index{0},

```

```

00091         currentLine{1}, currentColumn{1},
00092         currentFile{filename} {}
00093
00094 template <typename Ch, typename String>
00095 std::optional<Ch> StringBuffer<Ch, String>::getch() {
00096     if (index < buffer_.length()) {
00097         Ch ch = buffer_[index++];
00098         if (ch == '\n') {
00099             currentLine++;
00100             currentColumn = 1;
00101         } else {
00102             currentColumn++;
00103         }
00104         return ch;
00105     }
00106     return std::nullopt;
00107 }
00108
00109 template <typename Ch, typename String>
00110 std::optional<Ch> StringBuffer<Ch, String>::curch() {
00111     if (index < buffer_.length())
00112         return buffer_[index];
00113     return std::nullopt;
00114 }
00115
00116 template <typename Ch, typename String>
00117 std::optional<Ch> StringBuffer<Ch, String>::peekch(int num) {
00118     if (index + num < buffer_.length())
00119         return buffer_[index + num];
00120     return std::nullopt;
00121 }
00122
00123 template <typename Ch, typename String>
00124 std::optional<Ch> StringBuffer<Ch, String>::prevch() {
00125     if (index > 0)
00126         return buffer_[index - 1];
00127     return std::nullopt;
00128 }
00129
00130 template <typename Ch, typename String>
00131 std::optional<Ch> StringBuffer<Ch, String>::forward_step(int num) {
00132     if (index + num < buffer_.length()) {
00133         index += num;
00134         return buffer_[index];
00135     }
00136     return std::nullopt;
00137 }
00138
00139 template <typename Ch, typename String>
00140 std::optional<Ch> StringBuffer<Ch, String>::backward_step(int num) {
00141     if (static_cast<int64_t>(index) >= num) {
00142         index -= num;
00143         if (buffer_[index] == '\n') {
00144             currentLine--;
00145             currentColumn = 1;
00146         }
00147         return buffer_[index];
00148     }
00149     return std::nullopt;
00150 }
00151
00152 template <typename Ch, typename String>
00153 std::pair<uint64_t, uint64_t> StringBuffer<Ch, String>::cur_line() {
00154     uint64_t line = (currentColumn == 1 && index > 0) ? currentLine - 1 : currentLine;
00155     uint64_t col = (currentColumn > 1 ? currentColumn - 1 : 1);
00156     return std::make_pair(line, col);
00157 }
00158
00159 template <typename Ch, typename String>
00160 std::string StringBuffer<Ch, String>::cur_file() const {
00161     return currentFile;
00162 }
00163
00164 template <typename Ch, typename String>
00165 void StringBuffer<Ch, String>::process_line_directive(uint64_t newLine,
00166                                                         const std::string &newFile) {
00167     currentLine = newLine;
00168     currentColumn = 1;
00169     currentFile = newFile;
00170 }
00171
00172 template <typename Ch, typename String>
00173 bool StringBuffer<Ch, String>::eof(uint64_t pos) {
00174     return (pos + index >= buffer_.size());
00175 }
00176
00177 template <typename Ch, typename String>

```

```

00178     void StringBuffer<Ch, String>::reset(uint64_t pos) {
00179         index = pos;
00180         currentLine = 1;
00181         currentColumn = 1;
00182     }
00183 } // namespace buffer
00184
00185 namespace token {
00186
00187     using types::CharType;
00188
00194     template <typename Ch, int MAX_SIZE = 256>
00195     class TokenMap {
00196     public:
00197         TokenMap();
00204         std::optional<types::CharType> lookup_int8(int8_t c);
00205
00206     private:
00207         std::unordered_map<Ch, types::CharType> token_map;
00208     };
00209
00210     template <typename Ch, int MAX_CHARS>
00211     TokenMap<Ch, MAX_CHARS>::TokenMap() {
00212         std::size_t i = 0;
00213         for (i = 0; i < MAX_CHARS; ++i) {
00214             token_map[i] = CharType::TT_NULL;
00215         }
00216         for (i = 'a'; i <= 'z'; ++i) {
00217             token_map[i] = CharType::TT_CHAR;
00218         }
00219         for (i = 'A'; i <= 'Z'; ++i) {
00220             token_map[i] = CharType::TT_CHAR;
00221         }
00222         for (i = '0'; i <= '9'; ++i) {
00223             token_map[i] = CharType::TT_DIGIT;
00224         }
00225         token_map[' '] = CharType::TT_SPACE;
00226         token_map['+'] = CharType::TT_SYMBOL;
00227         token_map['-'] = CharType::TT_SYMBOL;
00228         token_map['*'] = CharType::TT_SYMBOL;
00229         token_map['/'] = CharType::TT_SYMBOL;
00230         token_map['%'] = CharType::TT_SYMBOL;
00231         token_map['&'] = CharType::TT_SYMBOL;
00232         token_map['|'] = CharType::TT_SYMBOL;
00233         token_map['^'] = CharType::TT_SYMBOL;
00234         token_map['~'] = CharType::TT_SYMBOL;
00235         token_map['!'] = CharType::TT_SYMBOL;
00236         token_map['='] = CharType::TT_SYMBOL;
00237         token_map['<'] = CharType::TT_SYMBOL;
00238         token_map['>'] = CharType::TT_SYMBOL;
00239         token_map['('] = CharType::TT_SYMBOL;
00240         token_map[')'] = CharType::TT_SYMBOL;
00241         token_map['['] = CharType::TT_SYMBOL;
00242         token_map[']'] = CharType::TT_SYMBOL;
00243         token_map['{'] = CharType::TT_SYMBOL;
00244         token_map['}'] = CharType::TT_SYMBOL;
00245         token_map[';'] = CharType::TT_SYMBOL;
00246         token_map[':'] = CharType::TT_SYMBOL;
00247         token_map['.,'] = CharType::TT_SYMBOL;
00248         token_map['.'] = CharType::TT_SYMBOL;
00249         token_map['?'] = CharType::TT_SYMBOL;
00250         token_map['#'] = CharType::TT_SYMBOL;
00251         token_map['_'] = CharType::TT_CHAR;
00252         token_map['\\'] = CharType::TT_SYMBOL;
00253         token_map['\"'] = CharType::TT_SINGLE;
00254         token_map['\'] = CharType::TT_STRING;
00255         token_map['"'] = CharType::TT_STRING;
00256         token_map['@'] = CharType::TT_SYMBOL;
00257         token_map['$'] = CharType::TT_SYMBOL;
00258     }
00259     template <typename Ch, int MAX_CHARS>
00260     std::optional<types::CharType> TokenMap<Ch, MAX_CHARS>::lookup_int8(int8_t c) {
00261         if (c >= 0 && c < MAX_CHARS) {
00262             auto f = token_map.find(c);
00263             if (f != token_map.end())
00264                 return f->second;
00265         }
00266         return std::nullopt;
00267     }
00268
00274     template <typename Ch = char, typename String = std::basic_string<Ch, std::char_traits<Ch>>
00275     class Token {
00276     public:
00277         using ch_type = Ch;
00278         using string_type = String;
00280         Token() : line{1}, col{1} {}
00282         Token(const types::TokenType &t) : type{t}, line{1}, col{1} {}

```

```

00284         Token(const Token<Ch, String> &t) : type{t.type}, value{t.value}, line{t.line},
col{t.col}, filename{t.filename} {}
00286         Token(Token<Ch, String> &&t) : type{t.type}, value{std::move(t.value)}, line{t.line},
col{t.col}, filename{t.filename} {}
00288         Token<Ch, String> &operator=(const types::TokenType &type);
00290         Token<Ch, String> &operator=(const Token<Ch, String> &t);
00292         Token<Ch, String> &operator=(Token<Ch, String> &&t);
00298         void setToken(const types::TokenType &type, const String &value);
00300         string_type getTokenValue() const { return value; }
00302         types::TokenType getTokenType() const { return type; }
00304         void print(std::ostream &out);
00306         const std::pair<uint64_t, uint64_t> get_pos() const { return std::make_pair(line, col); }
00308         void set_pos(const std::pair<uint64_t, uint64_t> &p);
00310         void set_filename(const std::string &filename) { this->filename = filename; }
00312         std::string get_filename() const { return this->filename; }
00314         uint64_t getLine() const { return line; }
00316         uint64_t getCol() const { return col; }
00317
00318     private:
00319         types::TokenType type;
00320         string_type value;
00321         uint64_t line = 1, col = 1;
00322         std::string filename;
00323 };
00324
00325     template <typename Ch, typename String>
00326     void Token<Ch, String>::setToken(const types::TokenType &type, const String &value) {
00327         this->type = type;
00328         this->value = value;
00329     }
00330
00331     template <typename Ch, typename String>
00332     Token<Ch, String> &Token<Ch, String>::operator=(const types::TokenType &type) {
00333         setToken(type, value);
00334         return *this;
00335     }
00336     template <typename Ch, typename String>
00337     Token<Ch, String> &Token<Ch, String>::operator=(const Token<Ch, String> &t) {
00338         setToken(t.type, t.value);
00339         filename = t.filename;
00340         return *this;
00341     }
00342     template <typename Ch, typename String>
00343     Token<Ch, String> &Token<Ch, String>::operator=(Token<Ch, String> &&t) {
00344         setToken(t.type, t.value);
00345         return *this;
00346     }
00347
00348     template <typename Ch, typename String>
00349     void Token<Ch, String>::print(std::ostream &out) {
00350         out << "value « " -> "\t";
00351         types::print_type_TokenType(out, this->type);
00352         out << "\n";
00353     }
00354     template <typename Ch, typename String>
00355     void Token<Ch, String>::set_pos(const std::pair<uint64_t, uint64_t> &p) {
00356         line = p.first;
00357         col = p.second;
00358     }
00359 } // namespace token
00360
00361 } // namespace scan
00362
00363 #endif

```

12.108 src/scanner/scanner.cpp File Reference

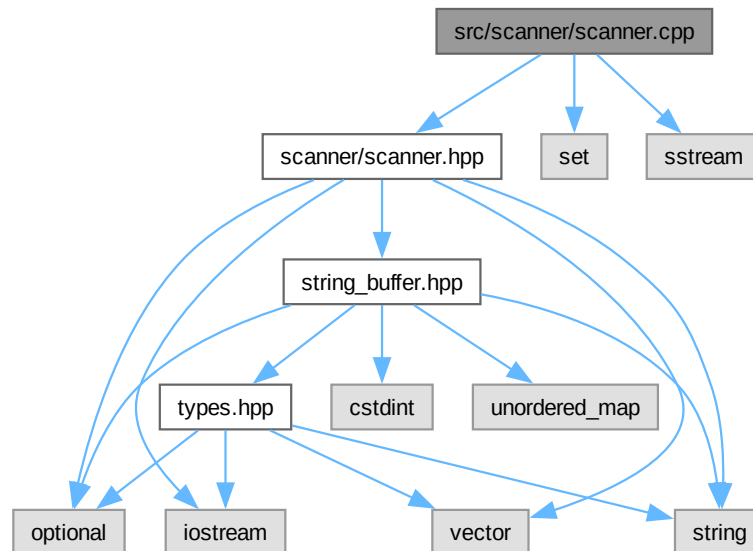
Lexical scanner implementation — tokenizes source text into identifiers, numbers, symbols, and strings.

```

#include "scanner/scanner.hpp"
#include <set>
#include <sstream>

```


Include dependency graph for scanner.cpp:



Namespaces

- namespace [scan](#)

12.108.1 Detailed Description

Lexical scanner implementation — tokenizes source text into identifiers, numbers, symbols, and strings.

Author

Jared Bruni

Definition in file [scanner.cpp](#).

12.109 scanner.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "scanner/scanner.hpp"
00007 #include <set>
00008 #include <sstream>
00009
00010 namespace scan {
00011
00012     Scanner::Scanner(const TString &b) : string_buffer{b} {}
00013
00014     void Scanner::removeEOL() {
00015         for (int64_t i = tokens.size() - 1; i >= 0; --i) {
00016             if (tokens[i].getTokenValue() == "\n" && tokens[i].getTokenType() ==
types::TokenType::TT_SYM) {
00017                 tokens.erase(tokens.begin() + i);
00018             }
00019         }
00020     }
00021
00022     uint64_t Scanner::parseLineNumber() {
00023         std::string lineNumberStr;
00024         while (true) {
00025             auto ch = string_buffer.getch();

```

```

00026         if (!ch.has_value()) {
00027             return 1;
00028         }
00029         if (isdigit(*ch)) {
00030             lineNumberStr += *ch;
00031             break;
00032         }
00033     }
00034     while (true) {
00035         auto ch = string_buffer.getch();
00036         if (!ch.has_value() || !isdigit(*ch)) {
00037             break;
00038         }
00039         lineNumberStr += *ch;
00040     }
00041     return !lineNumberStr.empty() ? std::stoull(lineNumberStr) : 1;
00042 }
00043
00044 std::string Scanner::parseFileName() {
00045     std::string fileName;
00046     while (true) {
00047         auto ch = string_buffer.getch();
00048         if (!ch.has_value() || !isspace(*ch)) {
00049             if (ch.has_value() && *ch == '\\')
00050                 break;
00051         }
00052     }
00053     while (true) {
00054         auto ch = string_buffer.getch();
00055         if (!ch.has_value() || *ch == '\\')
00056             break;
00057         fileName += *ch;
00058     }
00059     return fileName.empty() ? "unknown" : fileName;
00060 }
00061
00062 uint64_t Scanner::scan() {
00063     tokens.clear();
00064     while (!string_buffer.eof(0)) {
00065         auto ch = string_buffer.getch();
00066         if (!ch.has_value())
00067             break;
00068
00069         if (*ch == '\\n') {
00070             TToken token;
00071             auto pos = string_buffer.cur_line();
00072             auto filename = string_buffer.cur_file();
00073
00074             token.set_pos(pos);
00075             token.set_filename(filename);
00076             token.setToken(types::TokenType::TT_SYM, "\\n");
00077
00078             tokens.push_back(token);
00079             continue;
00080         }
00081
00082         if (*ch == '\\r') {
00083             auto next = string_buffer.getch();
00084             if (next.has_value() && *next == '\\n') {
00085                 continue;
00086             }
00087         }
00088
00089         if (*ch == '\"') {
00090             auto token = grabString();
00091             if (token.has_value()) {
00092                 tokens.push_back(*token);
00093             }
00094             continue;
00095         }
00096
00097         if (ch.has_value() && *ch == '#') {
00098             do {
00099                 ch = string_buffer.getch();
00100             } while (ch.has_value() && *ch != '\\n');
00101             string_buffer.backward_step(1);
00102             continue;
00103         } else if (ch.has_value() && *ch == '/') {
00104             auto ch2 = string_buffer.curch();
00105             if (ch2.has_value() && *ch2 == '/') {
00106                 auto ch_ln = string_buffer.curch();
00107                 do {
00108                     ch_ln = string_buffer.getch();
00109                 } while (ch_ln.has_value() && *ch_ln != '\\n');
00110                 continue;
00111             } else if (ch2.has_value() && *ch2 == '*') {
00112                 string_buffer.getch();

```

```

00113         while (true) {
00114             auto ch_in = string_buffer.getch();
00115             if (!ch_in.has_value())
00116                 break;
00117             if (*ch_in == '*') {
00118                 auto ch_ex = string_buffer.curch();
00119                 if (ch_ex.has_value() && *ch_ex == '/') {
00120                     string_buffer.getch();
00121                     break;
00122                 }
00123             }
00124         }
00125         continue;
00126     }
00127 }
00128 if (ch.has_value()) {
00129     auto t_ch = token_map.lookup_int8(*ch);
00130     if (t_ch.has_value()) {
00131         switch (*t_ch) {
00132             case types::CharType::TT_CHAR: {
00133                 auto tok = grabId();
00134                 if (tok.has_value()) {
00135                     tokens.push_back(*tok);
00136                 }
00137                 break;
00138             case types::CharType::TT_DIGIT: {
00139                 auto tok = grabDigits();
00140                 if (tok.has_value()) {
00141                     tokens.push_back(*tok);
00142                 }
00143                 break;
00144             case types::CharType::TT_SYMBOL: {
00145                 auto tok = grabSymbols();
00146                 if (tok.has_value()) {
00147                     tokens.push_back(*tok);
00148                 }
00149                 break;
00150             case types::CharType::TT_STRING: {
00151                 auto tok = grabString();
00152                 if (tok.has_value()) {
00153                     tokens.push_back(*tok);
00154                 }
00155                 break;
00156             case types::CharType::TT_SINGLE: {
00157                 auto tok = grabSingle();
00158                 if (tok.has_value()) {
00159                     tokens.push_back(*tok);
00160                 }
00161                 break;
00162             case types::CharType::TT_SPACE:
00163                 while (true) {
00164                     auto next = string_buffer.peekch(0);
00165                     if (next.has_value() && (*next == ' ' || *next == '\t' || *next == '\r'))
00166 {
00167                         string_buffer.getch();
00168                         continue;
00169                     }
00170                     if (!next.has_value() || token_map.lookup_int8(*next) !=
types::CharType::TT_SPACE) {
00171                         break;
00172                     }
00173                     string_buffer.getch();
00174                 }
00175                 continue;
00176             case types::CharType::TT_NULL:
00177                 continue;
00178             }
00179         } else
00180             break;
00181     }
00182     return tokens.size();
00183 }
00184 }
00185 std::optional<TToken> Scanner::grabId() {
00186     auto startPos = string_buffer.cur_line();
00187     auto filename = string_buffer.cur_file();
00188     string_buffer.backward_step(1);
00189     std::string val;
00190     while (true) {
00191         auto c = string_buffer.curch();
00192         if (!c.has_value())
00193             break;
00194         if (!std::isalnum(*c) && *c != '_'')

```

```

00198         break;
00199         val += *string_buffer.getch();
00200     }
00201
00202     if (val.empty())
00203         return std::nullopt;
00204
00205     TToken tok;
00206     tok.set_pos(startPos);
00207     tok.set_filename(filename);
00208     tok.setToken(types::TokenType::TT_ID, val);
00209     return tok;
00210 }
00211 std::optional<TToken> Scanner::grabDigits() {
00212     auto pos = string_buffer.cur_line();
00213     auto filename = string_buffer.cur_file();
00214     auto is_op = [](char ch) {
00215         return ch == '+' || ch == '-' || ch == '*' || ch == '/' ||
00216                ch == '%' || ch == '^' || ch == '|' || ch == '&' ||
00217                ch == '(' || ch == ')';
00218     };
00219     auto is_hex_digit = [](char ch) {
00220         return std::isdigit(ch) ||
00221                (ch >= 'a' && ch <= 'f') ||
00222                (ch >= 'A' && ch <= 'F');
00223     };
00224
00225     string_buffer.backward_step(1);
00226     TToken token;
00227     std::string tok_value;
00228
00229     auto first_opt = string_buffer.getch();
00230     if (!first_opt.has_value())
00231         return std::nullopt;
00232     char first = *first_opt;
00233     tok_value += first;
00234
00235     if (first == '0') {
00236         auto nx = string_buffer.peekch(0);
00237         if (nx.has_value() && (*nx == 'x' || *nx == 'X')) {
00238             tok_value += *string_buffer.getch();
00239             while (true) {
00240                 auto p = string_buffer.peekch(0);
00241                 if (!p.has_value() || !is_hex_digit(*p))
00242                     break;
00243                 tok_value += *string_buffer.getch();
00244             }
00245             token.set_pos(pos);
00246             token.set_filename(filename);
00247             token.setToken(types::TokenType::TT_HEX, tok_value);
00248             return token;
00249         }
00250     }
00251
00252     int decimal_dots = 0;
00253     while (true) {
00254         auto p = string_buffer.peekch(0);
00255         if (!p.has_value())
00256             break;
00257         char c = *p;
00258         if (std::isspace(c) || is_op(c))
00259             break;
00260         if (c == '.' && string_buffer.peekch(1).has_value() && *string_buffer.peekch(1) == '.')
00261             break;
00262         if (!std::isdigit(c) && c != '.')
00263             break;
00264
00265         string_buffer.getch();
00266         if (c == '.') {
00267             if (++decimal_dots > 1) {
00268                 std::ostringstream err;
00269                 auto lp = string_buffer.cur_line();
00270                 err << "File: " << filename << " Line: " << lp.first << " Col: " << lp.second << " -> Too
many decimal points in number.";
00271                 throw ScanExcept(err.str());
00272             }
00273         }
00274         tok_value += c;
00275     }
00276
00277     token.set_pos(pos);
00278     token.set_filename(filename);
00279     token.setToken(types::TokenType::TT_NUM, tok_value);
00280     return token;
00281 }
00282
00283 bool Scanner::is_c_sym(const StringType &str) {

```

```

00284     static const std::set<std::string> c_op = {
00285         "+", "--", ">=", "<=", "+=", "-=", "*=", "/=", "%=", "&=", "|=", "^=",
00286         "=", "!=", ">", "<", ">>", "<<", "&&", "||", "->", ".", "&", "*", "+",
00287         "-", "~", "!", "/", "%", ">", "<", "=", "^", "|", "?", ":", ";", ",", "(",
00288         "[", "]", "{", "}", "-", ":", "$", ":", "<>", "..";
00289     return c_op.find(str) != c_op.end();
00290 }
00291
00292 std::optional<Token> Scanner::grabSymbols() {
00293     Token token;
00294     std::string tok_value;
00295     string_buffer.backward_step(1);
00296     auto filename = string_buffer.cur_file();
00297     auto pos = string_buffer.cur_line();
00298
00299     auto ch = string_buffer.getch();
00300     if (!ch.has_value())
00301         return std::nullopt;
00302
00303     if (*ch == '-') {
00304         auto next = string_buffer.peekch(0);
00305         if (next.has_value() && *next == '-') {
00306             string_buffer.getch();
00307             std::string arg_value = "--";
00308             while (true) {
00309                 auto peeked = string_buffer.peekch(0);
00310                 if (peeked.has_value() && (isalnum(*peeked) || *peeked == '_')) {
00311                     arg_value += string_buffer.getch().value();
00312                 } else {
00313                     break;
00314                 }
00315             }
00316             token.set_pos(pos);
00317             token.set_filename(filename);
00318             token.setToken(types::TokenType::TT_ARG, arg_value);
00319             return token;
00320         } else {
00321             token.set_pos(pos);
00322             token.set_filename(filename);
00323             token.setToken(types::TokenType::TT_SYM, "-");
00324             return token;
00325         }
00326     }
00327
00328     if (*ch == '.' && string_buffer.peekch(0).has_value() && *string_buffer.peekch(0) == '.') {
00329         string_buffer.getch();
00330         token.set_pos(pos);
00331         token.set_filename(filename);
00332         token.setToken(types::TokenType::TT_SYM, "..");
00333         return token;
00334     }
00335
00336     tok_value = *ch;
00337     std::string temp_value = tok_value;
00338     int look = 0;
00339     bool found = false;
00340
00341     while (true) {
00342         auto temp = string_buffer.peekch(look);
00343         if (temp.has_value() && token_map.lookup_int8(*temp) == types::CharType::TT_SYMBOL) {
00344             temp_value += *temp;
00345             if (is_c_sym(temp_value)) {
00346                 tok_value = temp_value;
00347                 look++;
00348                 found = true;
00349             } else {
00350                 break;
00351             }
00352         } else {
00353             break;
00354         }
00355     }
00356
00357     if (found && look != 0) {
00358         string_buffer.forward_step(look);
00359     }
00360
00361     token.set_pos(pos);
00362     token.set_filename(filename);
00363     token.setToken(types::TokenType::TT_SYM, tok_value);
00364     return token;
00365 }
00366
00367 std::optional<Token> Scanner::grabString() {
00368     Token token;
00369     std::string tok_value;
00370     auto pos = string_buffer.cur_line();

```

```

00371         auto filename = string_buffer.cur_file();
00372         while (true) {
00373             auto ch = string_buffer.getch();
00374             if (!ch.has_value()) {
00375                 std::ostream stream;
00376                 auto cpos = string_buffer.cur_line();
00377                 stream << "File: " << string_buffer.cur_file() << " Line: " << cpos.first << " Col:" <<
cpos.second << " Double quote not terminated.";
00378                 throw ScanExcept(stream.str());
00379             }
00380             if (*ch == '\\') {
00381                 auto next_ch = string_buffer.getch();
00382                 if (!next_ch.has_value())
00383                     break;
00384                 switch (*next_ch) {
00385                     case 'n':
00386                         tok_value += '\\n';
00387                         break;
00388                     case 't':
00389                         tok_value += '\\t';
00390                         break;
00391                     case 'r':
00392                         tok_value += '\\r';
00393                         break;
00394                     case '\\':
00395                         tok_value += '\\\\';
00396                         break;
00397                     case '\"':
00398                         tok_value += "\\\"";
00399                         break;
00400                     case '\\':
00401                         tok_value += '\\';
00402                         break;
00403                     default:
00404                         tok_value += '\\';
00405                         tok_value += *next_ch;
00406                         break;
00407                 }
00408             } else if (*ch == '\"') {
00409                 break;
00410             } else {
00411                 tok_value += *ch;
00412             }
00413         }
00414         token.set_pos(pos);
00415         token.set_filename(filename);
00416         token.setToken(types::TokenType::TT_STR, tok_value);
00417         return token;
00418     }
00419
00420     std::optional<TToken> Scanner::grabSingle() {
00421         TToken token;
00422         std::string tok_value;
00423         auto filename = string_buffer.cur_file();
00424         auto pos = string_buffer.cur_line();
00425         while (true) {
00426             auto ch = string_buffer.getch();
00427             if (!ch.has_value()) {
00428                 std::ostream stream;
00429                 auto cpos = string_buffer.cur_line();
00430                 stream << "File: " << string_buffer.cur_file() << " Line: " << cpos.first << " Col:" <<
cpos.second << " String quote not terminated.";
00431                 throw ScanExcept(stream.str());
00432             }
00433             if (*ch == '\\') {
00434                 auto next_ch = string_buffer.getch();
00435                 if (!next_ch.has_value())
00436                     break;
00437                 switch (*next_ch) {
00438                     case 'n':
00439                         tok_value += '\\n';
00440                         break;
00441                     case 't':
00442                         tok_value += '\\t';
00443                         break;
00444                     case 'r':
00445                         tok_value += '\\r';
00446                         break;
00447                     case '\\':
00448                         tok_value += '\\\\';
00449                         break;
00450                     case '\"':
00451                         tok_value += "\\\"";
00452                         break;
00453                     case '\\':
00454                         tok_value += '\\';
00455                         break;

```

```

00456         default:
00457             tok_value += '\\';
00458             tok_value += *next_ch;
00459             break;
00460     }
00461 } else if (*ch == '\\') {
00462     // Pascal doubled-quote convention: " becomes a literal '
00463     if (string_buffer.peekch(0).has_value() && *string_buffer.peekch(0) == '\\') {
00464         string_buffer.getch();
00465         tok_value += '\\';
00466     } else {
00467         break;
00468     }
00469 } else {
00470     tok_value += *ch;
00471 }
00472 }
00473
00474 token.set_pos(pos);
00475 token.set_filename(filename);
00476 token.setToken(types::TokenType::TT_STR, tok_value);
00477 return token;
00478 }
00479
00480 TToken &Scanner::operator[](size_t index) {
00481     if (index < size())
00482         return tokens[index];
00483     std::ostringstream stream;
00484     stream << "Token index: index out of bounds, Scanner Exception: " << index << "\n";
00485     throw ScanExcept(stream.str());
00486 }
00487 size_t Scanner::size() const { return tokens.size(); }
00488 } // namespace scan

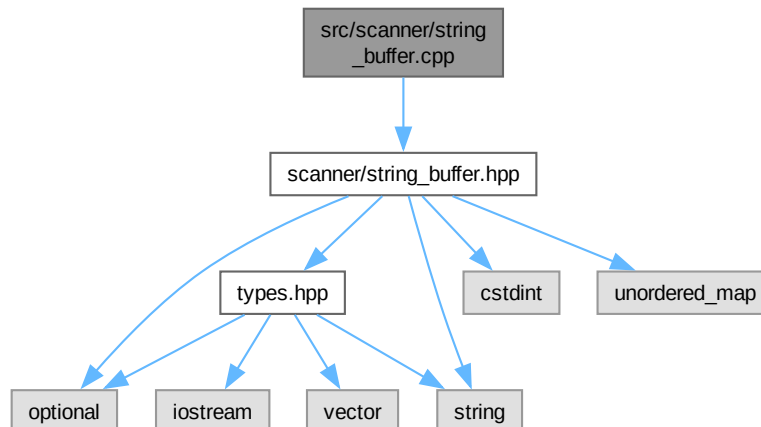
```

12.110 src/scanner/string_buffer.cpp File Reference

StringBuffer line/column tracking and line-directive processing.

#include "scanner/string_buffer.hpp"

Include dependency graph for string_buffer.cpp:



Namespaces

- namespace [scan](#)

12.110.1 Detailed Description

StringBuffer line/column tracking and line-directive processing.

Author

Jared Bruni

Definition in file [string_buffer.cpp](#).

12.111 string_buffer.cpp

[Go to the documentation of this file.](#)

```

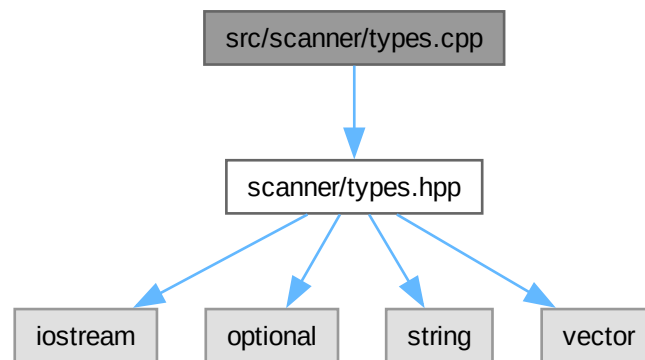
00001
00006 #include "scanner/string_buffer.hpp"
00007
00008 namespace scan {
00009
00010 }
```

12.112 src/scanner/types.cpp File Reference

Stream operators for TokenType and CharType enumerations.

#include "scanner/types.hpp"

Include dependency graph for types.cpp:



Namespaces

- namespace [types](#)

Functions

- void [types::print_type\(TokenType\)](#) (std::ostream &out, const [TokenType](#) &tt)
Print a [TokenType](#) name to a stream.
- void [types::print_type\(CharType\)](#) (std::ostream &out, const [CharType](#) &c)
Print a [CharType](#) name to a stream.
- std::optional< [OperatorType](#) > [types::lookUp](#) (const std::string &op)
Look up an operator string and return its [OperatorType](#).
- std::optional< [KeywordType](#) > [types::lookUp_Keyword](#) (const std::string &op)
Look up a keyword string and return its [KeywordType](#).

Variables

- `std::vector< std::string > types::strCharType` {"Characters", "Digits", "Symbols", "String", "Quote", "Space", "NULL"}

12.112.1 Detailed Description

Stream operators for TokenType and CharType enumerations.

Author

Jared Bruni

Definition in file [types.cpp](#).

12.113 types.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "scanner/types.hpp"
00007
00008 namespace types {
00009
00010     std::vector<std::string> strTokenType{"Identifier", "Argument", "Symbols", "String", "Numeric",
00011     "Hex", "NULL"};
00012     std::vector<std::string> strCharType{"Characters", "Digits", "Symbols", "String", "Quote",
00013     "Space", "NULL"};
00014
00015     void print_type(TokenType(std::ostream &out, const TokenType &tt) {
00016         unsigned int t_type = static_cast<unsigned int>(tt);
00017         if (t_type < strTokenType.size())
00018             out << strTokenType[t_type];
00019     }
00020
00021     void print_type(CharType(std::ostream &out, const CharType &c) {
00022         unsigned int t_type = static_cast<unsigned int>(c);
00023         if (t_type < strCharType.size())
00024             out << strCharType[t_type];
00025     }
00026
00027     std::optional<OperatorType> lookUp(const std::string &op) {
00028         for (size_t i = 0; i < opStrings.size(); ++i) {
00029             if (opStrings[i] == op)
00030                 return static_cast<OperatorType>(i);
00031         }
00032         return std::nullopt;
00033     }
00034
00035     std::optional<KeywordType> lookUp_Keyword(const std::string &op) {
00036         for (size_t i = 0; i < kwStr.size(); ++i) {
00037             if (kwStr[i] == op)
00038                 return static_cast<KeywordType>(i);
00039         }
00040         return std::nullopt;
00041     }
00042 } // namespace types

```

12.114 src/valid.cpp File Reference

MXVM instruction operand validator and variable/label checker.

```

#include "mxvm/valid.hpp"
#include "mxvm/instruct.hpp"
#include "scanner/exception.hpp"
#include <algorithm>
#include <sstream>
#include <unordered_map>
#include <unordered_set>

```



```

00031         continue;
00032     }
00033     if (c == ';' )
00034         return true;
00035 } else {
00036     if (escaped) {
00037         escaped = false;
00038         continue;
00039     }
00040     if (c == '\\') {
00041         escaped = true;
00042         continue;
00043     }
00044     if (c == '"') {
00045         in_str = false;
00046         continue;
00047     }
00048 }
00049 }
00050 return false;
00051 }
00052
00053 void Validator::collect_objects(std::unordered_set<std::string> &objects, size_t start_index,
size_t end_index) {
00054     bool in_object_section = false;
00055     int brace_depth = 0;
00056
00057     for (size_t i = start_index; i <= end_index && i < scanner.size(); ++i) {
00058         const auto &tok = scanner[i];
00059
00060         if (!in_object_section) {
00061             // Look for "section object {"
00062             if (tok.getTokenType() == types::TokenType::TT_ID &&
00063                 tok.getTokenValue() == "section" &&
00064                 i + 2 < scanner.size() &&
00065                 scanner[i + 1].getTokenValue() == "object" &&
00066                 scanner[i + 2].getTokenValue() == "{") {
00067                 in_object_section = true;
00068                 brace_depth = 1;
00069                 i += 2; // Skip "object" and "{"
00070                 continue;
00071             }
00072         } else {
00073             // Inside object section
00074             if (tok.getTokenValue() == "{") {
00075                 ++brace_depth;
00076             } else if (tok.getTokenValue() == "}") {
00077                 if (brace_depth <= 0) {
00078                     throw mx::Exception("Syntax Error in '" + filename + "': unexpected '}' in
object section");
00079                 }
00080                 --brace_depth;
00081                 if (brace_depth == 0) {
00082                     in_object_section = false;
00083                 }
00084             } else if (tok.getTokenType() == types::TokenType::TT_ID && brace_depth == 1) {
00085                 // This is an object name at the top level of the object section
00086                 objects.insert(tok.getTokenValue());
00087             }
00088         }
00089     }
00090 }
00091
00092 static const std::unordered_map<std::string, OpSpec> kOpSpecs = {
00093     {"mov", {"mov", {OpKind::Id, OpKind::Any}}},
00094     {"load", {"load", {OpKind::Id, OpKind::Id, OpKind::Any, VArity::None, 3, 4}}},
00095     {"store", {"store", {OpKind::Any, OpKind::Id, OpKind::Any, VArity::None, 3, 4}}},
00096     {"add", {"add", {OpKind::Id, OpKind::Any, VArity::AnyTail, 2, 3}}},
00097     {"sub", {"sub", {OpKind::Id, OpKind::Any, VArity::AnyTail, 2, 3}}},
00098     {"mul", {"mul", {OpKind::Id, OpKind::Any, VArity::AnyTail, 2, 3}}},
00099     {"div", {"div", {OpKind::Id, OpKind::Any, VArity::AnyTail, 2, 3}}},
00100     {"or", {"or", {OpKind::Id, OpKind::Any, VArity::AnyTail, 2, 3}}},
00101     {"and", {"and", {OpKind::Id, OpKind::Any, VArity::AnyTail, 2, 3}}},
00102     {"xor", {"xor", {OpKind::Id, OpKind::Any, VArity::AnyTail, 2, 3}}},
00103     {"not", {"not", {OpKind::Id}}},
00104     {"neg", {"neg", {OpKind::Id}}},
00105     {"mod", {"mod", {OpKind::Id, OpKind::Any, VArity::AnyTail, 2, 3}}},
00106     {"cmp", {"cmp", {OpKind::Any, OpKind::Any}}},
00107     {"fcmp", {"fcmp", {OpKind::Any, OpKind::Any}}},
00108     {"jmp", {"jmp", {OpKind::Label}}},
00109     {"je", {"je", {OpKind::Label}}},
00110     {"jne", {"jne", {OpKind::Label}}},
00111     {"jle", {"jle", {OpKind::Label}}},
00112     {"jge", {"jge", {OpKind::Label}}},
00113     {"jlt", {"jlt", {OpKind::Label}}},
00114     {"jgt", {"jgt", {OpKind::Label}}},
00115     {"jz", {"jz", {OpKind::Label}}},

```

```

00116     {"jnz", {"jnz", {OpKind::Label}}},
00117     {"ja", {"ja", {OpKind::Label}}},
00118     {"jb", {"jb", {OpKind::Label}}},
00119     {"print", {"print", {OpKind::Any}, VArity::AnyTail, 1, -1}},
00120     {"string_print", {"string_print", {OpKind::Any}}},
00121     {"exit", {"exit", {}, VArity::AnyTail, 0, 1}},
00122     {"alloc", {"alloc", {OpKind::Id, OpKind::Any, OpKind::Any}}},
00123     {"realloc", {"realloc", {OpKind::Id, OpKind::Any, OpKind::Any}}},
00124     {"free", {"free", {OpKind::Id}}},
00125     {"lea", {"lea", {OpKind::Id, OpKind::Id}}},
00126     {"getline", {"getline", {OpKind::Id}}},
00127     {"push", {"push", {OpKind::Any}}},
00128     {"pop", {"pop", {OpKind::Id}}},
00129     {"stack_load", {"stack_load", {OpKind::Id, OpKind::Any}}},
00130     {"stack_store", {"stack_store", {OpKind::Any, OpKind::Any}}},
00131     {"stack_sub", {"stack_sub", {OpKind::Any}}},
00132     {"call", {"call", {OpKind::Label}}},
00133     {"ret", {"ret", {}},
00134     {"done", {"done", {}},
00135     {"to_int", {"to_int", {OpKind::Id, OpKind::Any}}},
00136     {"to_float", {"to_float", {OpKind::Id, OpKind::Any}}},
00137     {"invoke", {"invoke", {OpKind::Id, VArity::ArgsTail}},
00138     {"return", {"return", {OpKind::Id}}};
00139
00140 static bool isIdLike(OpKind k) { return k == OpKind::Id || k == OpKind::Member; }
00141
00142 ParsedOp Validator::parseOperand() {
00143     if (!token)
00144         throw mx::Exception("Syntax Error in '" + filename + "': Unexpected EOF parsing operand");
00145     ParsedOp n;
00146     if (match(types::TokenType::TT_ID)) {
00147         std::string a = token->getTokenValue();
00148         const scan::TToken *at = token;
00149         next();
00150         if (match(".")) {
00151             next();
00152             require(types::TokenType::TT_ID);
00153             a += "." + token->getTokenValue();
00154             n.kind = OpKind::Member;
00155             n.text = a;
00156             n.at = at;
00157             next();
00158             return n;
00159         }
00160         n.kind = OpKind::Id;
00161         n.text = a;
00162         n.at = at;
00163         return n;
00164     }
00165     if (match("-") && peekIs(types::TokenType::TT_NUM)) {
00166         next();
00167     }
00168     if (match(types::TokenType::TT_NUM)) {
00169         std::string num_str = token->getTokenValue();
00170         if (num_str.find('.') != std::string::npos) {
00171             throw mx::Exception("Syntax Error in '" + filename + "': Floating point constants must
be declared as variables, not used directly in instructions at line " +
std::to_string(token->getLine()));
00172         }
00173         n.kind = OpKind::Num;
00174         n.text = token->getTokenValue();
00175         n.at = token;
00176         next();
00177         return n;
00178     }
00179     if (match(types::TokenType::TT_HEX)) {
00180         n.kind = OpKind::Hex;
00181         n.text = token->getTokenValue();
00182         n.at = token;
00183         next();
00184         return n;
00185     }
00186     if (match(types::TokenType::TT_STR)) {
00187         n.kind = OpKind::Str;
00188         n.text = token->getTokenValue();
00189         n.at = token;
00190         next();
00191         return n;
00192     }
00193     throw mx::Exception("Syntax Error in '" + filename + "': invalid operand '" +
token->getTokenValue() + "' at line " + std::to_string(token->getLine()));
00194 }
00195
00196 std::vector<ParsedOp> Validator::parseOperandList() {
00197     std::vector<ParsedOp> out;
00198     while (token && !match("}")) {
00199         size_t operand_line = token->getLine();

```

```

00200         out.push_back(parseOperand());
00201         if (match(",", "")) {
00202             next();
00203             if (token && token->getLine() != operand_line) {
00204                 throw mx::Exception("Syntax Error in file '" + filename + "': Operands must be on
the same line");
00205             }
00206             continue;
00207         }
00208
00209         if (token && token->getLine() == operand_line &&
00210             !token->getTokenValue().empty()) {
00211             throw mx::Exception("Syntax Error in file '" + filename + "': Multiple items or
missing comma between operands on line " +
00212                 std::to_string(operand_line));
00213         }
00214         break;
00215     }
00216     return out;
00217 }
00218
00219 void Validator::collect_labels(std::unordered_map<std::string, std::string> &labels) {
00220     for (size_t i = 0; i < scanner.size(); ++i) {
00221         const auto &tok = scanner[i];
00222         if (tok.getTokenType() == types::TokenType::TT_ID) {
00223             if (i + 1 < scanner.size() && scanner[i + 1].getTokenValue() == ":") {
00224                 labels[tok.getTokenValue()] = tok.getTokenValue();
00225             }
00226         }
00227     }
00228 }
00229
00230 void Validator::validateAgainstSpec(
00231     const std::string &op,
00232     const std::vector<ParsedOp> &ops,
00233     const std::unordered_map<std::string, Variable> &vars,
00234     const std::unordered_map<std::string, std::string> &labels,
00235     const std::unordered_set<std::string> &objects,
00236     std::vector<UseVar> &usedVarsRef,
00237     std::vector<UseLabel> &usedLabelsRef) {
00238     auto it = kOpSpecs.find(op);
00239     if (it == kOpSpecs.end()) {
00240         throw mx::Exception("Syntax Error in '" + filename + "': Unknown instruction '" + op +
00241             "'");
00242     }
00243     const OpSpec &spec = it->second;
00244
00245     int minArgs = (spec.minArgs >= 0) ? spec.minArgs : (int)spec.fixed.size();
00246     int maxArgs = (spec.maxArgs >= 0) ? spec.maxArgs : (spec.varPolicy == VArity::None ?
(int)spec.fixed.size() : INT32_MAX);
00247
00248     if ((int)ops.size() < minArgs || (int)ops.size() > maxArgs) {
00249         throw mx::Exception("Syntax Error in '" + filename + "': '" + op + "' expects " +
std::to_string(minArgs) + " (maxArgs == INT32_MAX) ? '..inf' : '.." + std::to_string(maxArgs) + "'
operands; found " + std::to_string(ops.size()));
00250     }
00251
00252     auto kindOk = [&](OpKind want, OpKind got) {
00253         if (want == OpKind::Any)
00254             return true;
00255         if (want == OpKind::Label)
00256             return got == OpKind::Id || got == OpKind::Member || got == OpKind::Label;
00257         if (want == OpKind::Id)
00258             return got == OpKind::Id || got == OpKind::Member;
00259         return want == got;
00260     };
00261
00262     for (size_t i = 0; i < spec.fixed.size() && i < ops.size(); ++i) {
00263         if (!kindOk(spec.fixed[i], ops[i].kind)) {
00264             throw mx::Exception("Syntax Error in '" + filename + "': '" + op + "' operand " +
std::to_string((int)i + 1) + " has wrong kind");
00265         }
00266     }
00267
00268     if (spec.varPolicy == VArity::ArgsTail) {
00269         for (size_t i = spec.fixed.size(); i < ops.size(); ++i) {
00270             if (!(isImmediate(ops[i].kind) || isIdLike(ops[i].kind))) {
00271                 throw mx::Exception("Syntax Error in '" + filename + "': '" + op + "' extra args
must be Id/Imm");
00272             }
00273         }
00274     }
00275
00276     auto pushVar = [&](const ParsedOp &p) {
00277         if (isIdLike(p.kind)) {
00278             if (p.kind == OpKind::Id && p.text.find('.') == std::string::npos) {
00279                 if (!vars.count(p.text)) {

```

```

00279         std::string msg = "Syntax Error in '" + filename + "': Undefined variable '" +
p.text + "'";
00280         if (p.at) {
00281             msg += " at line " + std::to_string(p.at->getLine());
00282         }
00283         throw mx::Exception(msg);
00284     }
00285 }
00286 // NEW: Validate object references
00287 else if (p.kind == OpKind::Member && p.text.find('.') != std::string::npos) {
00288     std::string objectName = p.text.substr(0, p.text.find('.'));
00289     if (!objects.count(objectName)) {
00290         std::string msg = "Syntax Error in '" + filename + "': Undefined object '" +
objectName + "'";
00291         if (p.at) {
00292             msg += " at line " + std::to_string(p.at->getLine());
00293         }
00294         throw mx::Exception(msg);
00295     }
00296 }
00297 usedVarsRef.push_back({p.text, p.at});
00298 }
00299 };
00300 auto pushLabel = [&](const ParsedOp &p) {
00301     if (p.text.find('.') == std::string::npos) {
00302         usedLabelsRef.push_back({p.text, p.at});
00303     }
00304 };
00305
00306 if (op == "jmp" || op == "je" || op == "jne" || op == "jl" || op == "jle" || op == "jg" || op
== "jge" || op == "jz" || op == "jnz" || op == "ja" || op == "jb") {
00307     if (!ops.empty())
00308         pushLabel(ops[0]);
00309 }
00310 if (op == "call") {
00311     if (!ops.empty())
00312         pushLabel(ops[0]);
00313 }
00314
00315 if (op == "mov" || op == "pop" || op == "stack_load" || op == "alloc" || op == "getline" ||
op == "return" || op == "not" || op == "neg" || op == "to_int" || op == "to_float") {
00316     if (!ops.empty() && isIdLike(ops[0].kind))
00317         pushVar(ops[0]);
00318 }
00319
00320 if (op == "add" || op == "sub" || op == "mul" || op == "div" || op == "or" || op == "and" ||
op == "xor" || op == "mod" || op == "cmp") {
00321     if (ops.size() >= 1 && isIdLike(ops[0].kind))
00322         pushVar(ops[0]);
00323     if (ops.size() >= 2 && isIdLike(ops[1].kind))
00324         pushVar(ops[1]);
00325     if (ops.size() >= 3 && isIdLike(ops[2].kind))
00326         pushVar(ops[2]);
00327 }
00328
00329 if (op == "load") {
00330     if (ops.size() >= 1 && isIdLike(ops[0].kind))
00331         pushVar(ops[0]);
00332     if (ops.size() >= 2 && isIdLike(ops[1].kind))
00333         pushVar(ops[1]);
00334     if (ops.size() >= 3 && isIdLike(ops[2].kind))
00335         pushVar(ops[2]);
00336 }
00337
00338 if (op == "store") {
00339     if (ops.size() >= 1 && isIdLike(ops[0].kind))
00340         pushVar(ops[0]);
00341     if (ops.size() >= 2 && isIdLike(ops[1].kind))
00342         pushVar(ops[1]);
00343     if (ops.size() >= 3 && isIdLike(ops[2].kind))
00344         pushVar(ops[2]);
00345 }
00346
00347 if (op == "free") {
00348     if (!ops.empty() && isIdLike(ops[0].kind))
00349         pushVar(ops[0]);
00350 }
00351
00352 if (op == "invoke") {
00353     for (size_t i = 1; i < ops.size(); ++i) {
00354         if (isIdLike(ops[i].kind))
00355             pushVar(ops[i]);
00356     }
00357 }
00358
00359 if (op == "print") {
00360     for (size_t i = 0; i < ops.size(); ++i) {
00361         if (isIdLike(ops[i].kind))
00362             pushVar(ops[i]);
00363     }
00364 }

```

```

00363         pushVar(ops[i]);
00364     }
00365 }
00366 }
00367
00368 bool Validator::validate(const std::string &name) {
00369     filename = name;
00370     scanner.scan();
00371     next();
00372
00373     while (token) {
00374         std::unordered_map<std::string, std::string> labels;
00375         std::unordered_set<std::string> objects;
00376         std::vector<UseVar> usedVars;
00377         std::vector<UseLabel> usedLabels;
00378
00379         size_t block_start = index - 1;
00380         size_t block_end = block_start;
00381
00382         int brace_count = 0;
00383         bool found_opening = false;
00384         for (size_t i = block_start; i < scanner.size(); ++i) {
00385             if (scanner[i].getTokenValue() == "{") {
00386                 found_opening = true;
00387                 brace_count++;
00388             } else if (scanner[i].getTokenValue() == "}") {
00389                 brace_count--;
00390                 if (found_opening && brace_count == 0) {
00391                     block_end = i;
00392                     break;
00393                 }
00394             }
00395         }
00396
00397         collect_labels(labels);
00398         collect_objects(objects, block_start, block_end);
00399
00400         std::vector<std::string> lines;
00401         {
00402             std::istringstream code_info(source);
00403             std::string l;
00404             while (std::getline(code_info, l)) {
00405                 lines.push_back(l);
00406             }
00407         }
00408
00409         std::vector<std::pair<int, int> > code_ranges;
00410         {
00411             bool seen_section = false;
00412             bool section_is_code = false;
00413             bool in_code = false;
00414             int brace_depth = 0;
00415             int range_start = -1;
00416             for (size_t i = 0; i < scanner.size(); ++i) {
00417                 const auto &t = scanner[i];
00418                 if (!in_code) {
00419                     if (t.getTokenType() == types::TokenType::TT_ID && t.getTokenValue() ==
"section") {
00420                         seen_section = true;
00421                         section_is_code = false;
00422                         continue;
00423                     }
00424                     if (seen_section && t.getTokenType() == types::TokenType::TT_ID) {
00425                         section_is_code = (t.getTokenValue() == "code");
00426                         continue;
00427                     }
00428                     if (section_is_code && t.getTokenValue() == "{") {
00429                         in_code = true;
00430                         brace_depth = 1;
00431                         range_start = t.getLine();
00432                         seen_section = false;
00433                         section_is_code = false;
00434                         continue;
00435                     }
00436                     } else {
00437                         if (t.getTokenValue() == "{") {
00438                             ++brace_depth;
00439                         } else if (t.getTokenValue() == "}") {
00440                             --brace_depth;
00441                             if (brace_depth == 0) {
00442                                 int range_end = t.getLine();
00443                                 code_ranges.emplace_back(range_start, range_end);
00444                                 in_code = false;
00445                             }
00446                         }
00447                     }
00448                 }

```

```

00449     }
00450
00451     for (const auto &pr : code_ranges) {
00452         const int start = std::max(1, pr.first);
00453         const int end = pr.second;
00454         for (int line = start; line <= end; ++line) {
00455             const int idx = line - 1;
00456             if (idx < 0 || idx >= static_cast<int>(lines.size()))
00457                 continue;
00458             const std::string &text = lines[idx];
00459             if (has_semicolon(text)) {
00460                 throw mx::Exception(
00461                     "Syntax Error in file '" + filename +
00462                     "': Semicolons are not allowed in code section at line " +
00463                     std::to_string(line) + ": '" + text + "'");
00464             }
00465         }
00466     }
00467
00468     std::unordered_map<int, int> line_instruction_count;
00469     auto is_in_code = [&](int line) -> bool {
00470         for (const auto &pr : code_ranges) {
00471             if (line >= pr.first && line <= pr.second)
00472                 return true;
00473         }
00474         return false;
00475     };
00476     for (size_t i = 0; i < scanner.size(); ++i) {
00477         const auto &tok = scanner[i];
00478         const int line = tok.getLine();
00479         if (!is_in_code(line))
00480             continue;
00481         if (tok.getTokenType() != types::TokenType::TT_ID)
00482             continue;
00483
00484         if (i + 1 < scanner.size() && scanner[i + 1].getTokenValue() == ":") {
00485             continue;
00486         }
00487         if (tok.getTokenValue() == "function") {
00488             continue;
00489         }
00490         if (std::find(IncType.begin(), IncType.end(), tok.getTokenValue()) != IncType.end()) {
00491             line_instruction_count[line]++;
00492         }
00493     }
00494     for (const auto &kv : line_instruction_count) {
00495         const int line_num = kv.first;
00496         const int count = kv.second;
00497         if (count > 1) {
00498             const int idx = line_num - 1;
00499             const std::string line_text =
00500                 (idx >= 0 && idx < static_cast<int>(lines.size())) ? lines[idx] :
std::string("<unknown>");
00501             throw mx::Exception("Syntax Error in file '" + filename +
00502                 "': Multiple instructions on same line at line " +
00503                 std::to_string(line_num) + ": '" + line_text + "'");
00504         }
00505     }
00506
00507     auto skipSeparators = [&]() {};
00508
00509     skipSeparators();
00510     if (!token)
00511         break;
00512
00513     if (match("program")) {
00514         next();
00515     } else {
00516         require("object");
00517         next();
00518     }
00519
00520     skipSeparators();
00521     require(types::TokenType::TT_ID);
00522     next();
00523     skipSeparators();
00524     require("{");
00525     next();
00526
00527     std::unordered_map<std::string, Variable> vars;
00528     for (auto &n : {"stdout", "stdin", "stderr"}) {
00529         vars[n] = Variable();
00530         vars[n].var_name = n;
00531     }
00532
00533     skipSeparators();
00534     while (token && !match("}")) {

```



```

00535         skipSeparators();
00536         require("section");
00537         next();
00538         skipSeparators();
00539         require(types::TokenType::TT_ID);
00540         std::string sectionName = token->getTokenValue();
00541         next();
00542         skipSeparators();
00543         require("{");
00544         next();
00545
00546         if (sectionName == "module" || sectionName == "object") {
00547             skipSeparators();
00548             while (token && !match("{}")) {
00549                 skipSeparators();
00550                 if (match(types::TokenType::TT_ID)) {
00551                     next();
00552                     skipSeparators();
00553                     if (match(",")) {
00554                         next();
00555                         continue;
00556                     }
00557                     continue;
00558                 }
00559                 break;
00560             }
00561             skipSeparators();
00562             require(")");
00563             next();
00564         } else if (sectionName == "data") {
00565             skipSeparators();
00566             while (token && !match("{}")) {
00567                 skipSeparators();
00568                 if (match(types::TokenType::TT_ID) &&
00569                     (token->getTokenValue() == "int" ||
00570                      token->getTokenValue() == "string" ||
00571                      token->getTokenValue() == "float" ||
00572                      token->getTokenValue() == "ptr" ||
00573                      token->getTokenValue() == "byte" ||
00574                      token->getTokenValue() == "export")) {
00575                     if (token->getTokenValue() == "export")
00576                         next();
00577
00578                     std::string vtype = token->getTokenValue();
00579                     next();
00580                     skipSeparators();
00581
00582                     require(types::TokenType::TT_ID);
00583                     std::string vname = token->getTokenValue();
00584                     vars[vname].var_name = vname;
00585                     next();
00586                     skipSeparators();
00587
00588                     if (match(",") && vtype == "string") {
00589                         next();
00590                         skipSeparators();
00591                         if (match(types::TokenType::TT_NUM) ||
00592                             match(types::TokenType::TT_HEX)) {
00593                             next();
00594                             skipSeparators();
00595                             continue;
00596                         } else {
00597                             throw mx::Exception("Syntax Error in file '" + filename + "':
00598                                 string buffer requires number on line " + std::to_string(token->getLine()));
00599                         }
00600                     }
00601                     require("=");
00602                     next();
00603                     skipSeparators();
00604                     if (match("-") && match(types::TokenType::TT_SYM)) {
00605                         next();
00606                         skipSeparators();
00607                     }
00608                     if (vtype == "byte") {
00609                         if (!(match(types::TokenType::TT_NUM) ||
00610                             match(types::TokenType::TT_HEX))) {
00611                             throw mx::Exception("Syntax Error in file '" + filename + "': byte
00612                                 must be a valid byte value integer 0-255 on line " + std::to_string(token->getLine()));
00613                         }
00614                         int64_t value = std::stoll(token->getTokenValue(), nullptr, 0);
00615                         if (value < 0 || value > 0xFF) {
00616                             throw mx::Exception("Syntax Error in file '" + filename + "': byte
00617                                 out of range 0-255 on line: " + std::to_string(token->getLine()));
00618                         }
00619                         next();
00620                     }

```

```

00617         skipSeparators();
00618     } else if (vtype == "string") {
00619         require(types::TokenType::TT_STR);
00620         next();
00621         skipSeparators();
00622     } else if (token->getTokenValue() == "null" ||
00623         match(types::TokenType::TT_NUM) ||
00624         match(types::TokenType::TT_HEX) ||
00625         match(types::TokenType::TT_STR)) {
00626         next();
00627         skipSeparators();
00628     } else {
00629         throw mx::Exception("Syntax Error in file '" + filename + "': Expected
value for variable, found: " + token->getTokenValue() + " at line " +
std::to_string(token->getLine()));
00630     }
00631     } else {
00632         throw mx::Exception("Syntax Error in file '" + filename + "': Expected
variable declaration, found: " + token->getTokenValue() + " at line " +
std::to_string(token->getLine()));
00633     }
00634 }
00635 skipSeparators();
00636 require("{}");
00637 next();
00638 } else if (sectionName == "code") {
00639     skipSeparators();
00640
00641     while (token && !match("{}")) {
00642         size_t old_index = index;
00643         skipSeparators();
00644
00645         if (token && token->getTokenValue() == ";") {
00646             throw mx::Exception("Syntax Error in file '" + filename +
                                "': Semicolons not allowed in code section at line " +
                                std::to_string(token->getLine()));
00647         }
00648     }
00649
00650     if (match(types::TokenType::TT_ID) && token->getTokenValue() == "function") {
00651         next();
00652         skipSeparators();
00653         require(types::TokenType::TT_ID);
00654         next();
00655         skipSeparators();
00656         require(":");
00657         next();
00658         continue;
00659     }
00660
00661     if (match(types::TokenType::TT_ID) && peekIs(":")) {
00662         next();
00663         next();
00664         continue;
00665     }
00666
00667     if (match(types::TokenType::TT_ID)) {
00668         std::string op = token->getTokenValue();
00669         if (std::find(IncType.begin(), IncType.end(), op) == IncType.end()) {
00670             throw mx::Exception("Syntax Error in file '" + filename + "': Unknown
instruction '" + op + "' at line " + std::to_string(token->getLine()));
00671         }
00672         next();
00673         skipSeparators();
00674
00675         if (op == "ret" || op == "done") {
00676             std::vector<ParsedOp> emptyOps;
00677             validateAgainstSpec(op, emptyOps, vars, labels, objects, usedVars,
usedLabels); // Add objects
00678             continue;
00679         }
00680
00681         auto ops = parseOperandList();
00682         validateAgainstSpec(op, ops, vars, labels, objects, usedVars, usedLabels);
00683         // Add objects
00684         continue;
00685     } else {
00686         throw mx::Exception("Syntax Error in file '" + filename + "': Unexpected
token '" + token->getTokenValue() + "' in code section at line " + std::to_string(token->getLine()));
00687     }
00688
00689     if (old_index == index) {
00690         if (!next())
00691             break;
00692     }
00693 }
00694
00695 skipSeparators();

```

```

00696         require("");
00697         next();
00698     } else {
00699         throw mx::Exception("Syntax Error in file '" + filename + "': Unknown section: " +
sectionName + " at line " + std::to_string(token->getLine()));
00700     }
00701     skipSeparators();
00702 }
00703
00704     skipSeparators();
00705     require("");
00706     next(); // Consume the closing brace of the object/program
00707
00708     for (auto &u : usedLabels) {
00709         if (!labels.count(u.name)) {
00710             throw mx::Exception("Syntax Error in '" + filename + "': Undefined label '" +
u.name + "' at line " + std::to_string(u.at->getLine()));
00711         }
00712     }
00713     skipSeparators(); // Prepare for the next object/program or EOF
00714 }
00715     return true;
00716 }
00717
00718 bool Validator::match(const std::string &m) {
00719     if (!token)
00720         return false;
00721     if (token->getTokenValue() != m)
00722         return false;
00723     return true;
00724 }
00725
00726 void Validator::require(const std::string &r) {
00727     if (!token) {
00728         throw mx::Exception("Syntax Error in '" + filename + "': Required: " + r + " but reached
end of file");
00729     }
00730     if (r != token->getTokenValue())
00731         throw mx::Exception(
00732             "Syntax Error in '" + filename + "': Required: " + r +
00733             " Found: " + token->getTokenValue() +
00734             " at line " + std::to_string(token->getLine()));
00735 }
00736
00737 bool Validator::match(const types::TokenType &t) {
00738     if (!token || index >= scanner.size())
00739         return false;
00740     if (t != token->getTokenType())
00741         return false;
00742     return true;
00743 }
00744
00745 void Validator::require(const types::TokenType &t) {
00746     if (!token || index >= scanner.size()) {
00747         throw mx::Exception("Syntax Error in '" + filename + "': unexpected EOF");
00748     }
00749     if (t != token->getTokenType())
00750         throw mx::Exception(
00751             "Syntax Error in '" + filename + "': Required: " + tokenTypeToString(t) +
00752             " instead found: " + token->getTokenValue() +
00753             " : " + tokenTypeToString(token->getTokenType()) +
00754             " at line " + std::to_string(token->getLine()));
00755 }
00756
00757 bool Validator::next() {
00758     while (index < scanner.size() &&
00759           scanner[index].getTokenValue() == "\n" &&
00760           scanner[index].getTokenType() == types::TokenType::TT_SYM) {
00761         index++;
00762     }
00763     if (index < scanner.size()) {
00764         token = &scanner[index++];
00765         return true;
00766     }
00767     token = nullptr;
00768     return false;
00769 }
00770
00771 bool Validator::peekIs(const std::string &s) {
00772     return index < scanner.size() && scanner[index].getTokenValue() == s;
00773 }
00774
00775 bool Validator::peekIs(const types::TokenType &t) {
00776     return index < scanner.size() && scanner[index].getTokenType() == t;
00777 }
00778
00779 std::string Validator::tokenTypeToString(types::TokenType t) {

```

```

00780         switch (t) {
00781             case types::TokenType::TT_ID:
00782                 return "Identifier";
00783             case types::TokenType::TT_NUM:
00784                 return "Number";
00785             case types::TokenType::TT_HEX:
00786                 return "Hex";
00787             case types::TokenType::TT_STR:
00788                 return "String";
00789             case types::TokenType::TT_SYM:
00790                 return "Symbol";
00791             default:
00792                 return "";
00793         }
00794     }
00795
00796     Validator::Validator(const std::string &source_) : scanner(source_), source(source_) {
00797     }
00798 } // namespace mxvm

```

12.116 src/vm/include/argz.hpp File Reference

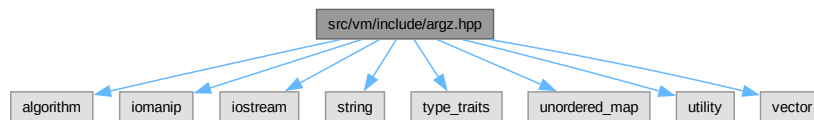
Generic command-line argument parser with concept-constrained string types.

```

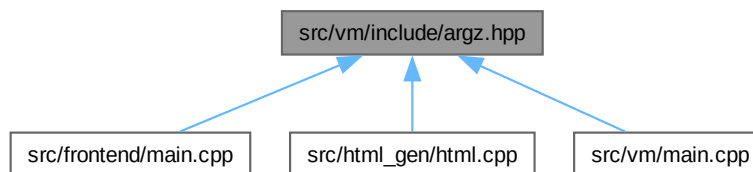
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <string>
#include <type_traits>
#include <unordered_map>
#include <utility>
#include <vector>

```

Include dependency graph for argz.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct `mx::Argument< String >`
A parsed or registered command-line argument definition.
- struct `mx::ArgumentData< String >`
Raw argument data collected from argc/argv.

- class [mx::ArgException< String >](#)
Exception type thrown for argument parsing errors.
- class [mx::Argz< String >](#)
Generic command-line argument parser supporting single/double-dash options.
- struct [mx::Arguments](#)
Parsed SDL program arguments (resolution, path, fullscreen).

Namespaces

- namespace [mx](#)

Concepts

- concept [mx::StringType](#)
Concept constraining String template parameters to class types with length(), operator[], operator+=", and operator= (e.g. std::string, std::wstring).

Enumerations

- enum class [mx::ArgType](#) {
 [mx::ARG_SINGLE](#) , [mx::ARG_SINGLE_VALUE](#) , [mx::ARG_DOUBLE](#) , [mx::ARG_DOUBLE_VALUE](#) ,
 [mx::ARG_NONE](#) }
Discriminates argument switch types (single-dash, double-dash, with/without value).

Functions

- [Arguments mx::proc_args](#) (int &argc, char **argv)
Parse SDL-specific command-line arguments.

12.116.1 Detailed Description

Generic command-line argument parser with concept-constrained string types.

Author

Jared Bruni

Definition in file [argz.hpp](#).

12.117 argz.hpp

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef _ARGZ_HPP_X
00007 #define _ARGZ_HPP_X
00008
00009 #include <algorithm>
00010 #include <iomanip>
00011 #include <iostream>
00012 #include <string>
00013 #include <type_traits>
00014 #include <unordered_map>
00015 #include <utility>
00016 #include <vector>
00017
00018 namespace mx {
00024     template <typename T>
00025     concept StringType = std::is_class_v<T> && requires(T type) {
00026         type.length();
00027         type[0];
00028         type += type;
00029         type = type;
00030         typename T::value_type;
00031         typename T::size_type;
00032         { type.length() } -> std::same_as<typename T::size_type>;

```

```

00033     { type[0] } -> std::same_as<typename T::value_type &>;
00034     { type += T{} } -> std::same_as<T &>;
00035     { type = T{} } -> std::same_as<T &>;
00036 };
00037
00039 enum class ArgType {
00040     ARG_SINGLE,
00041     ARG_SINGLE_VALUE,
00042     ARG_DOUBLE,
00043     ARG_DOUBLE_VALUE,
00044     ARG_NONE
00045 };
00046
00051 template <StringType String>
00052 struct Argument {
00053     String arg_name;
00054     int arg_letter;
00055     String arg_value;
00056     ArgType arg_type;
00057     String desc;
00058     Argument() : arg_name{}, arg_letter{}, arg_value{}, arg_type{}, desc{} {}
00059     Argument(const Argument &a) : arg_name{a.arg_name}, arg_letter{a.arg_letter},
arg_value{a.arg_value}, arg_type{a.arg_type}, desc{a.desc} {}
00060     Argument &operator=(const Argument<String> &a) {
00061         arg_name = a.arg_name;
00062         arg_letter = a.arg_letter;
00063         arg_value = a.arg_value;
00064         arg_type = a.arg_type;
00065         desc = a.desc;
00066         return *this;
00067     }
00068     auto operator<=>(const Argument<String> &a) const { return (arg_letter <=> a.arg_letter); }
00069 };
00070
00075 template <StringType String>
00076 struct ArgumentData {
00077     std::vector<String> args;
00078     int argc;
00079     ArgumentData() = default;
00080     ArgumentData(const ArgumentData<String> &a) : args{a.args}, argc{a.argc} {}
00081     ArgumentData &operator=(const ArgumentData<String> &a) {
00082         if (!args.empty()) {
00083             args.erase(args.begin(), args.end());
00084         }
00085         std::copy(a.args.begin(), a.args.end(), std::back_inserter(args));
00086         argc = a.argc;
00087         return *this;
00088     }
00089     ArgumentData(ArgumentData<String> &&a) : args{std::move(a.args)}, argc{a.argc} {}
00090     ArgumentData<String> &operator=(ArgumentData<String> &&a) {
00091         args = std::move(a.args);
00092         argc = a.argc;
00093         return *this;
00094     }
00095 };
00096
00101 template <StringType String>
00102 class ArgException {
00103 public:
00104     ArgException() = default;
00105
00107     ArgException(const String &s) : value{s} {}
00108
00110     String text() const { return value; }
00111
00112 private:
00113     String value;
00114 };
00115
00125 template <StringType String>
00126 class Argz {
00127 public:
00128     Argz() = default;
00129
00131     Argz(int argc, char **argv) { initArgs(argc, argv); }
00132
00134     Argz(const Argz<String> &a) : arg_data{a.arg_data}, arg_info{a.arg_info}, index{a.index},
cindex{a.cindex} {}
00135
00136     Argz<String> &operator=(const Argz<String> &a) {
00137         arg_data = a.arg_data;
00138         if (!arg_info.empty()) {
00139             arg_info.erase(arg_info.begin(), arg_info.end());
00140         }
00141         for (const auto &i : a.arg_info) {
00142             arg_info[i.first] = i.second;
00143         }

```

```

00144         index = a.index;
00145         cindex = a.cindex;
00146         return *this;
00147     }
00148
00150     Argz<String> &&a) : arg_data{std::move(a.arg_data)}, arg_info{std::move(a.arg_info)},
index{a.index}, cindex{a.cindex} {}
00151
00153     Argz<String> &operator=(Argz<String> &&a) {
00154         arg_data = std::move(a.arg_data);
00155         arg_info = std::move(a.arg_info);
00156         index = a.index;
00157         cindex = a.cindex;
00158         return *this;
00159     }
00160
00166     Argz<String> &initArgs(int argc, char **argv) {
00167         arg_data.argc = argc;
00168         if constexpr (std::is_same<typename String::value_type, char>::value) {
00169             for (int i = 1; i < argc; ++i) {
00170                 const char *a = argv[i];
00171                 arg_data.args.push_back(a);
00172             }
00173             reset();
00174             return *this;
00175         }
00176         if constexpr (std::is_same<typename String::value_type, wchar_t>::value) {
00177             for (int i = 1; i < argc; ++i) {
00178                 const char *a = argv[i];
00179                 String data;
00180                 for (size_t z = 0; a[z] != 0; ++z) {
00181                     data += static_cast<typename String::value_type>(a[z]);
00182                 }
00183                 arg_data.args.push_back(data);
00184             }
00185             reset();
00186             return *this;
00187         }
00188         reset();
00189         return *this;
00190     }
00191
00193     void reset() {
00194         index = 0;
00195         cindex = 1;
00196     }
00197
00203     Argz<String> &addOptionSingle(const int &c, const String &description) {
00204         Argument<String> a{};
00205         a.arg_letter = c;
00206         a.arg_type = ArgType::ARG_SINGLE;
00207         a.desc = description;
00208         arg_info[c] = a;
00209         return *this;
00210     }
00211
00217     Argz<String> &addOptionSingleValue(const int &c, const String &description) {
00218         Argument<String> a{};
00219         a.arg_letter = c;
00220         a.arg_type = ArgType::ARG_SINGLE_VALUE;
00221         a.desc = description;
00222         arg_info[c] = a;
00223         return *this;
00224     }
00225
00232     Argz<String> &addOptionDouble(const int &code, const String &value, const String &description)
{
00233         Argument<String> a{};
00234         a.arg_letter = code;
00235         a.arg_type = ArgType::ARG_DOUBLE;
00236         a.desc = description;
00237         a.arg_name = value;
00238         arg_info[code] = a;
00239         return *this;
00240     }
00241
00248     Argz<String> &addOptionDoubleValue(const int &code, const String &value, const String
&description) {
00249         Argument<String> a{};
00250         a.arg_letter = code;
00251         a.arg_type = ArgType::ARG_DOUBLE_VALUE;
00252         a.desc = description;
00253         a.arg_name = value;
00254         arg_info[code] = a;
00255         return *this;
00256     }
00257

```

```

00262     int lookUpCode(const String &value) {
00263         for (const auto &i : arg_info) {
00264             if (i.second.arg_name == value) {
00265                 return i.second.arg_letter;
00266             }
00267         }
00268         return -1;
00269     }
00270
00276     int proc(Argument<String> &a) {
00277         if (index < static_cast<int>(arg_data.args.size())) {
00278             const String &type{arg_data.args[index]};
00279             if (type.length() > 3 && type[0] == '-' && type[1] == '-') {
00280                 String name{};
00281                 for (size_t z = 2; z < type.length(); ++z)
00282                     name += type[z];
00283                 int code = lookUpCode(name);
00284                 if (code != -1) {
00285                     auto pos = arg_info.find(code);
00286                     if (pos != arg_info.end()) {
00287                         if (pos->second.arg_type == ArgType::ARG_DOUBLE) {
00288                             a = pos->second;
00289                             a.arg_name = name;
00290                             index++;
00291                             return code;
00292                         } else {
00293                             a = pos->second;
00294                             a.arg_name = name;
00295                             if (++index < static_cast<int>(arg_data.args.size())) /*&&
00296                                 arg_data.args[index][0] != '-'*/) {
00297                                 a.arg_name = name;
00298                                 a.arg_value = arg_data.args[index];
00299                                 index++;
00300                                 return code;
00301                             } else {
00302                                 if constexpr (std::is_same<typename String::value_type,
00303                                     char>::value) {
00304                                     throw ArgException<String>("Expected Value");
00305                                 }
00306                             }
00307                         } else {
00308                             if constexpr (std::is_same<typename String::value_type, char>::value) {
00309                                 String value = "Error argument: ";
00310                                 value += name;
00311                                 value += " switch not found";
00312                                 throw ArgException<String>(value);
00313                             }
00314                             if constexpr (std::is_same<typename String::value_type, wchar_t>::value) {
00315                                 String value = L"Error argument: ";
00316                                 value += name;
00317                                 value += L" switch not found";
00318                                 throw ArgException<String>(value);
00319                             }
00320                         }
00321                     } else if (type.length() == 1 && type[0] == '-') {
00322                         if constexpr (std::is_same<typename String::value_type, char>::value) {
00323                             throw ArgException<String>("Expected Value found -");
00324                         }
00325                         if constexpr (std::is_same<typename String::value_type, wchar_t>::value) {
00326                             throw ArgException<String>(L"Expected Value found -");
00327                         }
00328                     } else if (type.length() > 1 && (type[0] == '-')) {
00329                         const int c{type[cindex]};
00330                         const auto pos{arg_info.find(c)};
00331                         cindex++;
00332                         if (cindex >= static_cast<int>(type.length())) {
00333                             cindex = 1;
00334                             index++;
00335                         }
00336                         String name_val{};
00337                         name_val += static_cast<typename String::value_type>(c);
00338                         if (pos != arg_info.end()) {
00339                             if (pos->second.arg_type == ArgType::ARG_SINGLE) {
00340                                 a = pos->second;
00341                                 a.arg_name = name_val;
00342                                 return c;
00343                             } else if (pos->second.arg_type == ArgType::ARG_SINGLE_VALUE) {
00344                                 if (index < static_cast<int>(arg_data.args.size())) {
00345                                     const String &s{arg_data.args[index]};
00346                                     bool is_known_option = false;
00347                                     if (s.length() > 1 && s[0] == '-') {
00348                                         if (s[1] == '-') {
00349                                             String option_name = s.substr(2);
00350                                             is_known_option = (lookUpCode(option_name) != -1);
00351                                         } else {

```



```

00352         int option_char = static_cast<int>(s[1]);
00353         is_known_option = (arg_info.find(option_char) !=
arg_info.end());
00354     }
00355     } else if (s.length() == 1 && s[0] == '-') {
00356         if constexpr (std::is_same<typename String::value_type,
char>::value) {
00357             throw ArgException<String>("Expected Value found -");
00358         }
00359         if constexpr (std::is_same<typename String::value_type,
wchar_t>::value) {
00360             throw ArgException<String>(L"Expected Value found -");
00361         }
00362     }
00363     if (!is_known_option && s.length() > 0) {
00364         a = pos->second;
00365         a.arg_value = s;
00366         a.arg_name = name_val;
00367         index++;
00368         return c;
00369     } else {
00370         if constexpr (std::is_same<typename String::value_type,
char>::value) {
00371             throw ArgException<String>("Expected Value");
00372         }
00373         if constexpr (std::is_same<typename String::value_type,
wchar_t>::value) {
00374             throw ArgException<String>(L"Expected Value");
00375         }
00376     }
00377     } else {
00378         if constexpr (std::is_same<typename String::value_type, char>::value)
{
00379             throw ArgException<String>("Expected Value");
00380         }
00381         if constexpr (std::is_same<typename String::value_type,
wchar_t>::value) {
00382             throw ArgException<String>(L"Expected Value");
00383         }
00384     }
00385     } else {
00386         if constexpr (std::is_same<typename String::value_type, char>::value) {
00387             throw ArgException<String>("Invalid switch not found!");
00388         }
00389         if constexpr (std::is_same<typename String::value_type, wchar_t>::value) {
00390             throw ArgException<String>(L"Invalid switch not found!");
00391         }
00392     }
00393     } else {
00394         if constexpr (std::is_same<typename String::value_type, char>::value) {
00395             String value;
00396             value = "Error argument ";
00397             value += static_cast<typename String::value_type>(c);
00398             value += " switch not found.";
00399             throw ArgException<String>(value);
00400         }
00401         if constexpr (std::is_same<typename String::value_type, wchar_t>::value) {
00402             String value;
00403             value = L"Error argument ";
00404             value += static_cast<typename String::value_type>(c);
00405             value += L" switch not found.";
00406             throw ArgException<String>(value);
00407         }
00408     }
00409     }
00410     } else {
00411         a = Argument<String>();
00412         a.arg_name = String{};
00413         a.arg_type = ArgType::ARG_NONE;
00414         a.arg_name = a.arg_value = arg_data.args.at(index);
00415         index++;
00416         return '-';
00417     }
00418     }
00419     return -1;
00420 }
00421
00422 template <typename T>
00423 void help(T &cout) {
00424     using char_type = typename
std::decay<decltype(*std::declval<T>().rdbuf())>::type::char_type;
00425     std::vector<Argument<String>> v;
00426     std::vector<Argument<String>> v2;
00427     for (const auto &i : arg_info) {
00428         if (i.second.arg_type == ArgType::ARG_SINGLE || i.second.arg_type ==
ArgType::ARG_SINGLE_VALUE)
00429             v.push_back(i.second);

```

```

00434         else if (i.second.arg_type == ArgType::ARG_DOUBLE || i.second.arg_type ==
ArgType::ARG_DOUBLE_VALUE)
00435             v2.push_back(i.second);
00436     }
00437     std::sort(v.begin(), v.end());
00438     std::sort(v2.begin(), v2.end());
00439     std::vector<Argument<String>> farg;
00440     farg.reserve(v.size() + v2.size());
00441     std::copy(v.begin(), v.end(), std::back_inserter(farg));
00442     std::copy(v2.begin(), v2.end(), std::back_inserter(farg));
00443     for (auto a = farg.begin(); a != farg.end(); ++a) {
00444         if (a->arg_type == ArgType::ARG_SINGLE || a->arg_type == ArgType::ARG_SINGLE_VALUE) {
00445             if constexpr (std::is_same<char_type, char>::value) {
00446                 String item;
00447                 item += static_cast<char_type>(a->arg_letter);
00448                 cout << "-" << std::setfill(' ') << std::setw(9) << std::left << item << "\t";
00449                 cout << std::setfill(' ') << std::left << std::setw(10) << a->desc;
00450                 cout << '\n';
00451             } else if constexpr (std::is_same<char_type, wchar_t>::value) {
00452                 String item;
00453                 item += static_cast<char_type>(a->arg_letter);
00454                 cout << L "-" << std::setfill(L' ') << std::setw(9) << std::left << item << L"\t";
00455                 cout << std::setfill(L' ') << std::setw(10) << a->desc;
00456                 cout << L'\n';
00457             }
00458         } else {
00459             if constexpr (std::is_same<char_type, char>::value) {
00460                 cout << "--";
00461                 cout << std::setfill(' ') << std::left << std::setw(10) << a->arg_name;
00462                 cout << "\t";
00463                 cout << std::setw(10) << a->desc;
00464                 cout << '\n';
00465             } else if constexpr (std::is_same<char_type, wchar_t>::value) {
00466                 cout << L "--";
00467                 cout << std::setfill(L' ') << std::left << std::setw(10) << a->arg_name;
00468                 cout << L "\t";
00469                 cout << std::setw(10) << std::left << a->desc;
00470                 cout << L'\n';
00471             }
00472         }
00473     }
00474 }
00475
00476 protected:
00477     ArgumentData<String> arg_data;
00478     std::unordered_map<int, Argument<String>> arg_info;
00479
00480 private:
00481     int index = 0, cindex = 1;
00482 };
00483
00484 struct Arguments {
00485     int width, height;
00486     std::string path;
00487     bool fullscreen;
00488 };
00489
00490 inline Arguments proc_args(int &argc, char **argv) {
00491     Arguments args;
00492     Argz<std::string> parser(argc, argv);
00493     parser.addOptionSingle('h', "Display help message")
00494         .addOptionSingleValue('p', "assets path")
00495         .addOptionDoubleValue('P', "path", "assets path")
00496         .addOptionSingleValue('r', "Resolution WidthxHeight")
00497         .addOptionDoubleValue('R', "resolution", "Resolution WidthxHeight")
00498         .addOptionSingle('f', "fullscreen")
00499         .addOptionDouble('F', "fullscreen", "fullscreen");
00500     Argument<std::string> arg;
00501     std::string path;
00502     int value = 0;
00503     int tw = 1280, th = 720;
00504     bool fullscreen = false;
00505     try {
00506         while ((value = parser.proc(arg)) != -1) {
00507             switch (value) {
00508                 case 'h':
00509                     parser.help(std::cout);
00510                     exit(EXIT_SUCCESS);
00511                 case 'v':
00512                     break;
00513                 case 'p':
00514                 case 'P':
00515                     path = arg.arg_value;
00516                     break;
00517                 case 'r':
00518                 case 'R': {
00519                     auto pos = arg.arg_value.find("x");

```

```
00526         if (pos == std::string::npos) {
00527             std::cerr << "Error invalid resolution use WidthxHeight\n";
00528             std::cerr.flush();
00529             exit(EXIT_FAILURE);
00530         }
00531         std::string left, right;
00532         left = arg.arg_value.substr(0, pos);
00533         right = arg.arg_value.substr(pos + 1);
00534         tw = atoi(left.c_str());
00535         th = atoi(right.c_str());
00536     } break;
00537     case 'f':
00538     case 'F':
00539         fullscreen = true;
00540         break;
00541     }
00542 }
00543 } catch (const ArgException<std::string> &e) {
00544     std::cerr << "mx: Argument Exception" << e.text() << std::endl;
00545     // defaults
00546     args.width = 1280;
00547     args.height = 720;
00548     args.path = ".";
00549     args.fullscreen = false;
00550     return args;
00551 }
00552 if (path.empty()) {
00553     std::cerr << "mx: No path provided trying default current directory.\n";
00554     path = ".";
00555 }
00556 args.width = tw;
00557 args.height = th;
00558 args.path = path;
00559 args.fullscreen = fullscreen;
00560 return args;
00561 }
00562 } // namespace mx
00563 #endif
```


Index

- ~ASTNode
 - mxvm::ASTNode, [140](#)
 - pascal::ASTNode, [143](#)
- ~ASTVisitor
 - mxvm::ASTVisitor, [145](#)
 - pascal::ASTVisitor, [147](#)
- ~Base
 - mxvm::Base, [154](#)
- ~BuiltinFunctionHandler
 - pascal::BuiltinFunctionHandler, [169](#)
- ~CodeGenVisitor
 - pascal::CodeGenVisitor, [179](#)
- ~Function
 - mxvm::Function< R(Args...)>, [246](#)
- ~Program
 - mxvm::Program, [316](#)
- ~RuntimeFunction
 - mxvm::RuntimeFunction, [380](#)
- accept
 - mxvm::ASTNode, [140](#)
 - mxvm::CommentNode, [214](#)
 - mxvm::ExpressionNode, [227](#)
 - mxvm::InstructionNode, [256](#)
 - mxvm::LabelNode, [259](#)
 - mxvm::ModuleNode, [262](#)
 - mxvm::ObjectNode, [271](#)
 - mxvm::ProgramNode, [368](#)
 - mxvm::SectionNode, [391](#)
 - mxvm::VariableNode, [461](#)
 - pascal::AddressOfNode, [112](#)
 - pascal::ArrayAccessNode, [127](#)
 - pascal::ArrayAssignmentNode, [129](#)
 - pascal::ArrayDeclarationNode, [131](#)
 - pascal::ArrayTypeDeclarationNode, [135](#)
 - pascal::ArrayTypeNode, [137](#)
 - pascal::AssignmentNode, [139](#)
 - pascal::ASTNode, [143](#)
 - pascal::BinaryOpNode, [162](#)
 - pascal::BlockNode, [164](#)
 - pascal::BooleanNode, [166](#)
 - pascal::BreakNode, [167](#)
 - pascal::CaseStmtNode, [172](#)
 - pascal::CompoundStmtNode, [216](#)
 - pascal::ConstDeclNode, [218](#)
 - pascal::ContinueNode, [220](#)
 - pascal::EmptyStmtNode, [221](#)
 - pascal::EnumTypeDeclNode, [223](#)
 - pascal::ExitNode, [226](#)
 - pascal::FieldAccessNode, [235](#)
 - pascal::ForStmtNode, [239](#)
 - pascal::FuncCallNode, [241](#)
 - pascal::FuncDeclNode, [243](#)
 - pascal::GotoStmtNode, [249](#)
 - pascal::IfStmtNode, [252](#)
 - pascal::LabelStmtNode, [261](#)
 - pascal::NilNode, [268](#)
 - pascal::NumberNode, [270](#)
 - pascal::ParameterNode, [276](#)
 - pascal::PointerDerefNode, [302](#)
 - pascal::PointerTypeNode, [303](#)
 - pascal::ProcCallNode, [305](#)
 - pascal::ProcDeclNode, [307](#)
 - pascal::ProgramNode, [370](#)
 - pascal::RecordDeclarationNode, [372](#)
 - pascal::RecordTypeNode, [377](#)
 - pascal::RepeatStmtNode, [379](#)
 - pascal::SetLiteralNode, [393](#)
 - pascal::SetTypeNode, [395](#)
 - pascal::SimpleTypeNode, [396](#)
 - pascal::StringNode, [408](#)
 - pascal::TypeAliasNode, [435](#)
 - pascal::TypeDeclNode, [437](#)
 - pascal::UnaryOpNode, [439](#)
 - pascal::UnitNode, [441](#)
 - pascal::VarDeclNode, [450](#)
 - pascal::VariableNode, [463](#)
 - pascal::WhileStmtNode, [466](#)
 - pascal::WithStmtNode, [468](#)
- action
 - Args, [114](#)
- action_interpret
 - main.cpp, [785](#)
- action_translate
 - main.cpp, [785](#)
- ADD
 - instruct.hpp, [528](#)
- add_allocated
 - mxvm::Base, [154](#)
- add_extern
 - mxvm::Base, [155](#)
- add_filename
 - mxvm::Base, [155](#)
- add_global
 - mxvm::Base, [155](#)
- add_instruction
 - mxvm::Base, [155](#)
- add_label
 - mxvm::Base, [156](#)

- add_object
 - mxvm::Base, [156](#)
- add_runtime_extern
 - mxvm::Base, [156](#)
- add_standard
 - mxvm::Program, [317](#)
- add_variable
 - mxvm::Base, [157](#)
- addInlineObject
 - mxvm::ProgramNode, [368](#)
- addOperand
 - mxvm::InstructionNode, [256](#)
- addOptionDouble
 - mx::Argz< String >, [122](#)
- addOptionDoubleValue
 - mx::Argz< String >, [122](#)
- addOptionSingle
 - mx::Argz< String >, [123](#)
- addOptionSingleValue
 - mx::Argz< String >, [123](#)
- AddressOfNode
 - pascal::AddressOfNode, [112](#)
- addSection
 - mxvm::ProgramNode, [368](#)
- addStatement
 - mxvm::SectionNode, [391](#)
- addTempPtr
 - pascal::CodeGenVisitor, [179](#)
- addVariables
 - mxvm::Program, [317](#)
- ALLOC
 - instruct.hpp, [528](#)
- allocated
 - mxvm::Base, [157](#)
- allocatedPtrs
 - pascal::CodeGenVisitor, [205](#)
- allocateRecordFieldArrays
 - pascal::CodeGenVisitor, [179](#)
- allocFloatReg
 - pascal::CodeGenVisitor, [180](#)
- allocPtrReg
 - pascal::CodeGenVisitor, [180](#)
- allocReg
 - pascal::CodeGenVisitor, [180](#)
- allocTempPtr
 - pascal::CodeGenVisitor, [180](#)
- allTempPtrs
 - pascal::CodeGenVisitor, [205](#)
- AND
 - expr_parser, [76](#)
 - instruct.hpp, [528](#)
 - mxx, [91](#)
 - pascal::BinaryOpNode, [161](#)
- Any
 - mxvm, [85](#)
- AnyTail
 - mxvm, [86](#)
- arg_data
 - mx::Argz< String >, [125](#)
- ARG_DOUBLE
 - mx, [78](#)
- ARG_DOUBLE_VALUE
 - mx, [78](#)
- arg_info
 - mx::Argz< String >, [126](#)
- arg_letter
 - mx::Argument< String >, [117](#)
- arg_name
 - mx::Argument< String >, [117](#)
- ARG_NONE
 - mx, [78](#)
- ARG_SINGLE
 - mx, [77](#)
- ARG_SINGLE_VALUE
 - mx, [77](#)
- arg_type
 - mx::Argument< String >, [117](#)
- arg_value
 - mx::Argument< String >, [117](#)
- argc
 - mx::ArgumentData< String >, [119](#)
 - mx_std.h, [630](#)
 - std.c, [632](#)
- ArgException
 - mx::ArgException< String >, [113](#)
- Args, [113](#)
 - action, [114](#)
 - argv, [114](#)
 - include_path, [114](#)
 - Makefile, [114](#)
 - module_path, [114](#)
 - object_path, [114](#)
 - only_test, [114](#)
 - output_file, [115](#)
 - platform, [115](#)
 - platform_argc, [115](#)
 - platform_argv, [115](#)
 - source_file, [115](#)
 - target, [115](#)
 - toolchain, [115](#)
- args
 - mx::ArgumentData< String >, [119](#)
 - mxvm::Program, [364](#)
- ArgsTail
 - mxvm, [86](#)
- ArgType
 - mx, [77](#)
- Argument
 - mx::Argument< String >, [116](#)
- ArgumentData
 - mx::ArgumentData< String >, [118](#)
- arguments
 - pascal::FuncCallNode, [241](#)
 - pascal::ProcCallNode, [305](#)
- argv
 - Args, [114](#)

- mx_std.h, 630
- std.c, 632
- Argz
 - mx::Argz< String >, 121, 122
- ARRAY
 - mxx, 91
- ARRAY_DOUBLE
 - pascal, 97
- ARRAY_INT
 - pascal, 97
- ARRAY_STRING
 - pascal, 97
- ArrayAccessNode
 - pascal::ArrayAccessNode, 127
- ArrayAssignmentNode
 - pascal::ArrayAssignmentNode, 129
- ArrayDeclarationNode
 - pascal::ArrayDeclarationNode, 131
- ArrayInfo
 - pascal::ArrayInfo, 132
- arrayInfo
 - pascal::CodeGenVisitor, 205
 - pascal::CodeGenVisitor::RecordField, 374
- arrayName
 - pascal::ArrayAssignmentNode, 129
- arrayType
 - pascal::ArrayDeclarationNode, 131
 - pascal::ArrayTypeDeclarationNode, 135
- ArrayTypeDeclarationNode
 - pascal::ArrayTypeDeclarationNode, 135
- ArrayTypeNode
 - pascal::ArrayTypeNode, 137
- asReadPtr
 - string.cpp, 660
- assembly_code
 - mxvm::Base, 157
- AssignmentNode
 - pascal::AssignmentNode, 139
- assignments
 - pascal::ConstDeclNode, 218
- asWritePtr
 - string.cpp, 660
- at
 - mxvm::ModuleParser, 264
 - mxvm::ParsedOp, 277
 - mxvm::UseLabel, 442
 - mxvm::UseVar, 443
- backward_step
 - scan::buffer::StringBuffer< Ch, String >, 402
- Base
 - mxvm::Base, 154
- base
 - mxvm::Base, 158
 - pascal::ArrayAccessNode, 127
- baseType
 - pascal::TypeAliasNode, 435
- baseTypeName
 - pascal::PointerTypeNode, 303
- pascal::SetTypeNode, 395
- BEGIN
 - mxx, 91
- BG_GREEN
 - mx::Color, 80
- BG_RED
 - mx::Color, 80
- BG_YELLOW
 - mx::Color, 80
- BinaryOpNode
 - pascal::BinaryOpNode, 161
- BLACK
 - mx::Color, 80
- blit_surface
 - mx_sdl.h, 558
 - sdl.c, 579
- block
 - pascal::FuncDeclNode, 243
 - pascal::ProcDeclNode, 307
 - pascal::ProgramNode, 370
- BlockNode
 - pascal::BlockNode, 163
- BLUE
 - mx::Color, 80
- BOLD
 - mx::Color, 80
- BOOL
 - pascal, 97
- BooleanNode
 - pascal::BooleanNode, 165
- branches
 - pascal::CaseStmtNode, 173
- BreakNode
 - pascal::BreakNode, 167
- BRIGHT_BLUE
 - mx::Color, 80
- BRIGHT_CYAN
 - mx::Color, 80
- BRIGHT_GREEN
 - mx::Color, 80
- BRIGHT_MAGENTA
 - mx::Color, 80
- BRIGHT_RED
 - mx::Color, 80
- BRIGHT_YELLOW
 - mx::Color, 81
- buffer_
 - scan::buffer::StringBuffer< Ch, String >, 404
- buffer_size
 - mxvm::Variable_Value, 457
 - mxvm::VariableNode, 461
- bufferStringVars
 - pascal::CodeGenVisitor, 205
- buildArrayInfoFromNode
 - pascal::CodeGenVisitor, 180
- buildInstructionSet
 - mxvm, 86
- buildKeywordSet

- mxvm, 86
- builtinRegistry
 - pascal::CodeGenVisitor, 205
- CALL
 - instruct.hpp, 529
- call
 - mxvm::RuntimeFunction, 381
- canHandle
 - pascal::BuiltinFunctionHandler, 169
 - pascal::FileFunctionHandler, 237
 - pascal::IOFunctionHandler, 257
 - pascal::SDLFunctionHandler, 389
 - pascal::StdFunctionHandler, 399
 - pascal::StringFunctionHandler, 406
- carry_flag
 - mxvm::Program, 364
- CASE
 - mxx, 91
- CaseBranch
 - pascal::CaseStmtNode::CaseBranch, 171
- caseLabels
 - pascal::VariantArm, 464
- CaseStmtNode
 - pascal::CaseStmtNode, 172
- ch_type
 - scan::buffer::StringBuffer< Ch, String >, 401
 - scan::token::Token< Ch, String >, 409
- CHAR
 - pascal, 97
- CharType
 - scan::token, 102
 - types, 103
- checkConstOnly
 - mxx::TPValidator, 419
- checkType
 - mxx::TPValidator, 419
- checkVar
 - mxx::TPValidator, 419
- checkVarOrConst
 - mxx::TPValidator, 419
- ciEquals
 - mxx, 92
- cindex
 - mx::Argz< String >, 126
- clear
 - mx_sdl.h, 558
 - sdl.c, 579
- clear_queued_audio
 - mx_sdl.h, 558
 - sdl.c, 579
- close_audio
 - mx_sdl.h, 558
 - sdl.c, 579
- CMP
 - instruct.hpp, 528
- CMP_FLOAT
 - mxvm::Program, 316
- CMP_INTEGER
 - mxvm::Program, 316
- CMP_NONE
 - mxvm::Program, 316
- CODE
 - mxvm::SectionNode, 391
- CodeGenVisitor
 - pascal::CodeGenVisitor, 179
- coerceToIntImmediate
 - pascal::CodeGenVisitor, 180
- Col
 - exception.hpp, 930
- col
 - scan::token::Token< Ch, String >, 412
- collect_labels
 - mxvm::Validator, 445
- collect_objects
 - mxvm::Validator, 445
- collect_used
 - pascal, 97
- collectAndRegisterAllExterns
 - main.cpp, 785
- collectObjectNames
 - mxvm::Parser, 280
- color_
 - mx, 79
- CommentNode
 - mxvm::CommentNode, 214
- compile
 - main.cpp, 784
- compileFile
 - main.cpp, 778
- compileTimeConstants
 - pascal::CodeGenVisitor, 205
- compoundStatement
 - pascal::BlockNode, 164
- CompoundStmtNode
 - pascal::CompoundStmtNode, 215
- condition
 - pascal::IfStmtNode, 252
 - pascal::RepeatStmtNode, 379
 - pascal::WhileStmtNode, 466
- CONST
 - mxx, 91
- ConstAssignment
 - pascal::ConstDeclNode::ConstAssignment, 216
- ConstDeclNode
 - pascal::ConstDeclNode, 218
- constInitialValues
 - pascal::CodeGenVisitor, 205
- consts
 - mxx::Scope, 387
- consume
 - expr_parser::ExprLexer, 229
- ContinueNode
 - pascal::ContinueNode, 219
- convertCharLiteral
 - pascal::CodeGenVisitor, 181
- copy

- cstring.c, [656](#)
- copyPropagation
 - pascal, [97](#)
- create_render_target
 - mx_sdl.h, [559](#)
 - sdl.c, [579](#)
- create_renderer
 - mx_sdl.h, [559](#)
 - sdl.c, [579](#)
- create_rgb_surface
 - mx_sdl.h, [559](#)
 - sdl.c, [580](#)
- create_texture
 - mx_sdl.h, [559](#)
 - sdl.c, [580](#)
- create_window
 - mx_sdl.h, [560](#)
 - sdl.c, [580](#)
- createMakefile
 - main.cpp, [785](#)
- createTempVariable
 - mxvm::Program, [317](#)
- cstring.c
 - copy, [656](#)
 - delete_str, [656](#)
 - insert, [656](#)
 - inttostr, [656](#)
 - pos, [656](#)
 - strat, [656](#)
 - strfind, [657](#)
 - strtoint, [657](#)
 - substr, [657](#)
- cur_file
 - scan::buffer::StringBuffer< Ch, String >, [402](#)
- cur_line
 - scan::buffer::StringBuffer< Ch, String >, [402](#)
- curch
 - scan::buffer::StringBuffer< Ch, String >, [402](#)
- current
 - expr_parser::ExprLexer, [229](#)
- current_function
 - mxvm::Validator, [447](#)
- currentColumn
 - scan::buffer::StringBuffer< Ch, String >, [404](#)
- currentFile
 - scan::buffer::StringBuffer< Ch, String >, [404](#)
- currentFuncLocalSlots
 - pascal::CodeGenVisitor, [205](#)
- currentFunctionName
 - pascal::CodeGenVisitor, [205](#)
- currentFunctionReturnSlot
 - pascal::CodeGenVisitor, [206](#)
- currentLine
 - scan::buffer::StringBuffer< Ch, String >, [404](#)
- currentParamLocations
 - pascal::CodeGenVisitor, [206](#)
- currentParamTypes
 - pascal::CodeGenVisitor, [206](#)
- currentRecordTypeName
 - mx::TPValidator, [432](#)
- currentScope
 - mx::TPValidator, [420](#)
- currentScopeName
 - mx::TPValidator, [420](#)
- CYAN
 - mx::Color, [81](#)
- DARWIN
 - mxvm, [86](#)
- darwin_prefix_calls
 - mxvm, [86](#)
- DATA
 - mxvm::SectionNode, [391](#)
- data
 - mxvm::Stack, [398](#)
- debug_mode
 - mxvm, [89](#)
- declarations
 - pascal::BlockNode, [164](#)
- declareConst
 - mx::TPValidator, [420](#)
- declaredConsts
 - mx::TPValidator, [432](#)
- declaredFuncs
 - mx::TPValidator, [432](#)
 - pascal::CodeGenVisitor, [206](#)
- declaredLabels
 - pascal::PascalParser, [299](#)
- declaredProcs
 - mx::TPValidator, [432](#)
 - pascal::CodeGenVisitor, [206](#)
- declaredTypes
 - mx::TPValidator, [432](#)
- declaredVars
 - mx::TPValidator, [432](#)
- declareFunc
 - mx::TPValidator, [420](#)
- declareParam
 - mx::TPValidator, [420](#)
- declareProc
 - mx::TPValidator, [420](#)
- declareRecordField
 - mx::TPValidator, [421](#)
- declareType
 - mx::TPValidator, [421](#)
- declareVar
 - mx::TPValidator, [421](#)
- defaultCss
 - mxvm::PasHTMLGen, [300](#)
- deferredFuncs
 - pascal::CodeGenVisitor, [206](#)
- deferredProcs
 - pascal::CodeGenVisitor, [206](#)
- delay
 - mx_sdl.h, [560](#)
 - sdl.c, [581](#)
- delete_str

- cstring.c, [656](#)
- desc
 - mx::Argument< String >, [117](#)
- destroy_render_target
 - mx_sdl.h, [561](#)
 - sdl.c, [581](#)
- destroy_renderer
 - mx_sdl.h, [561](#)
 - sdl.c, [581](#)
- destroy_texture
 - mx_sdl.h, [561](#)
 - sdl.c, [581](#)
- destroy_window
 - mx_sdl.h, [561](#)
 - sdl.c, [582](#)
- DIM
 - mx::Color, [81](#)
- dirOf
 - html.cpp, [862](#)
- DIV
 - expr_parser, [76](#)
 - instruct.hpp, [528](#)
 - mxx, [91](#)
 - pascal::BinaryOpNode, [161](#)
- DIVIDE
 - pascal::BinaryOpNode, [161](#)
- divVariables
 - mxvm::Program, [317](#)
- DO
 - mxx, [91](#)
- DONE
 - instruct.hpp, [529](#)
- DOUBLE
 - pascal, [96](#)
- DOWNT0
 - mxx, [91](#)
- draw_line
 - mx_sdl.h, [561](#)
 - sdl.c, [582](#)
- draw_point
 - mx_sdl.h, [561](#)
 - sdl.c, [582](#)
- draw_rect
 - mx_sdl.h, [562](#)
 - sdl.c, [582](#)
- draw_text
 - mx_sdl.h, [562](#)
 - sdl.c, [582](#)
- dynArrayLenSlot
 - pascal::CodeGenVisitor, [206](#)
- elementArray
 - pascal::ArrayInfo, [133](#)
- elementIsArray
 - pascal::ArrayInfo, [133](#)
- elements
 - pascal::SetLiteralNode, [393](#)
- elementSize
 - pascal::ArrayInfo, [133](#)
- elementType
 - pascal::ArrayInfo, [133](#)
 - pascal::ArrayTypeNode, [137](#)
- ELSE
 - mxx, [91](#)
- elseStatement
 - pascal::CaseStmtNode, [173](#)
 - pascal::IfStmtNode, [252](#)
- emit
 - pascal::CodeGenVisitor, [181](#)
- emit1
 - pascal::CodeGenVisitor, [181](#)
- emit2
 - pascal::CodeGenVisitor, [181](#)
- emit3
 - pascal::CodeGenVisitor, [181](#)
- emit4
 - pascal::CodeGenVisitor, [181](#)
- emit_invoke
 - pascal::CodeGenVisitor, [182](#)
- emitArrayBoundsCheck
 - pascal::CodeGenVisitor, [182](#)
- emitDynArrayBoundsCheck
 - pascal::CodeGenVisitor, [182](#)
- emitFree
 - pascal::CodeGenVisitor, [183](#)
- emitLabel
 - pascal::CodeGenVisitor, [183](#)
- empty
 - mxvm::Stack, [397](#)
- EmptyStmtNode
 - pascal::EmptyStmtNode, [221](#)
- emptyString
 - pascal::CodeGenVisitor, [183](#)
- END
 - expr_parser, [75](#)
 - mxx, [91](#)
- endsWith
 - html.cpp, [863](#)
- endsWithColon
 - pascal::CodeGenVisitor, [183](#)
- endValue
 - pascal::ForStmtNode, [239](#)
- ensureCss
 - html.cpp, [863](#)
- ensureFloatConstSymbol
 - pascal::CodeGenVisitor, [183](#)
- ensurePtrBase
 - pascal::CodeGenVisitor, [183](#)
- enumConstants
 - pascal::CodeGenVisitor, [206](#)
- EnumTypeDeclNode
 - pascal::EnumTypeDeclNode, [222](#)
- enumTypes
 - pascal::CodeGenVisitor, [207](#)
- eof
 - scan::buffer::StringBuffer< Ch, String >, [402](#)
- EQ

- expr_parser, 76
- EQUAL
 - pascal::BinaryOpNode, 161
- error
 - mx::Exception, 224
 - pascal::PascalParser, 290
- error_label_count
 - mxvm, 89
- escapedTempPtrs
 - pascal::CodeGenVisitor, 207
- escapeNewLines
 - mxvm::Program, 318
- escapeStringForMxvm
 - pascal::CodeGenVisitor, 184
- eval
 - pascal::CodeGenVisitor, 184
- evalStack
 - pascal::CodeGenVisitor, 207
- evaluateConstantExpression
 - pascal::CodeGenVisitor, 184
- except_assert
 - mxvm, 87
- Exception
 - mx::Exception, 224
- exception.hpp
 - Col, 930
- exec
 - mxvm::Program, 318
- exec_add
 - mxvm::Program, 318
- exec_alloc
 - mxvm::Program, 318
- exec_and
 - mxvm::Program, 319
- exec_call
 - mxvm::Program, 319
- exec_cmp
 - mxvm::Program, 319
- exec_div
 - mxvm::Program, 319
- exec_done
 - mxvm::Program, 319
- exec_exit
 - mxvm::Program, 319
- exec_fcmp
 - mxvm::Program, 320
- exec_free
 - mxvm::Program, 320
- exec_getline
 - mxvm::Program, 320
- exec_invoke
 - mxvm::Program, 320
- exec_ja
 - mxvm::Program, 320
- exec_jae
 - mxvm::Program, 320
- exec_jb
 - mxvm::Program, 321
- exec_jbe
 - mxvm::Program, 321
- exec_jc
 - mxvm::Program, 321
- exec_je
 - mxvm::Program, 321
- exec_jg
 - mxvm::Program, 321
- exec_jge
 - mxvm::Program, 321
- exec_jl
 - mxvm::Program, 321
- exec_jle
 - mxvm::Program, 322
- exec_jmp
 - mxvm::Program, 322
- exec_jnc
 - mxvm::Program, 322
- exec_jne
 - mxvm::Program, 322
- exec_jno
 - mxvm::Program, 322
- exec_jnp
 - mxvm::Program, 322
- exec_jns
 - mxvm::Program, 322
- exec_jnz
 - mxvm::Program, 322
- exec_jo
 - mxvm::Program, 323
- exec_jp
 - mxvm::Program, 323
- exec_js
 - mxvm::Program, 323
- exec_jz
 - mxvm::Program, 323
- exec_lea
 - mxvm::Program, 323
- exec_load
 - mxvm::Program, 323
- exec_mod
 - mxvm::Program, 324
- exec_mov
 - mxvm::Program, 324
- exec_mul
 - mxvm::Program, 324
- exec_neg
 - mxvm::Program, 324
- exec_not
 - mxvm::Program, 324
- exec_or
 - mxvm::Program, 324
- exec_pop
 - mxvm::Program, 325
- exec_print
 - mxvm::Program, 325
- exec_push
 - mxvm::Program, 325

- exec_realloc
 - mxvm::Program, [325](#)
- exec_ret
 - mxvm::Program, [326](#)
- exec_return
 - mxvm::Program, [326](#)
- exec_stack_load
 - mxvm::Program, [326](#)
- exec_stack_store
 - mxvm::Program, [326](#)
- exec_stack_sub
 - mxvm::Program, [326](#)
- exec_store
 - mxvm::Program, [327](#)
- exec_string_print
 - mxvm::Program, [327](#)
- exec_sub
 - mxvm::Program, [327](#)
- exec_to_float
 - mxvm::Program, [327](#)
- exec_to_int
 - mxvm::Program, [327](#)
- exec_xor
 - mxvm::Program, [328](#)
- EXIT
 - instruct.hpp, [528](#)
- exitCode
 - mxvm::Program, [364](#)
- ExitNode
 - pascal::ExitNode, [226](#)
- expandFloatRegisters
 - pascal::CodeGenVisitor, [184](#)
- expectToken
 - pascal::PascalParser, [290](#), [291](#)
- expr
 - pascal::BreakNode, [168](#)
 - pascal::ContinueNode, [220](#)
 - pascal::ExitNode, [226](#)
- expr_parser, [75](#)
 - AND, [76](#)
 - DIV, [76](#)
 - END, [75](#)
 - EQ, [76](#)
 - ExprTokenType, [75](#)
 - GT, [76](#)
 - GTE, [76](#)
 - LOGICAL_AND, [76](#)
 - LOGICAL_OR, [76](#)
 - LPAREN, [76](#)
 - LT, [76](#)
 - LTE, [76](#)
 - MINUS, [75](#)
 - MINUSEQ, [76](#)
 - MOD, [76](#)
 - MUL, [76](#)
 - NEQ, [76](#)
 - NOT, [76](#)
 - NUMBER, [75](#)
 - OR, [76](#)
 - PLUS, [75](#)
 - PLUSEQ, [76](#)
 - RPAREN, [76](#)
 - VARIABLE, [76](#)
 - vars, [76](#)
 - XOR, [76](#)
- expr_parser::ExprLexer, [228](#)
 - consume, [229](#)
 - current, [229](#)
 - ExprLexer, [229](#)
 - next, [229](#)
 - peek, [229](#)
 - pos, [229](#)
 - scanner, [230](#)
 - variable, [230](#)
- expr_parser::ExprParser, [230](#)
 - ExprParser, [231](#)
 - lexer, [232](#)
 - parse, [231](#)
 - parseAdd, [231](#)
 - parseBitwise, [231](#)
 - parseComparison, [231](#)
 - parseFactor, [231](#)
 - parseLogical, [232](#)
 - parseTerm, [232](#)
- expr_parser::ExprToken, [232](#)
 - type, [233](#)
 - value, [233](#)
- expression
 - pascal::AssignmentNode, [139](#)
 - pascal::CaseStmtNode, [173](#)
- ExpressionNode
 - mxvm::ExpressionNode, [227](#)
- ExprLexer
 - expr_parser::ExprLexer, [229](#)
- ExprParser
 - expr_parser::ExprParser, [231](#)
- ExprTokenType
 - expr_parser, [75](#)
- external
 - mxvm::Base, [158](#)
- external_functions
 - mxvm::Base, [158](#)
- externalFuncs
 - pascal::CodeGenVisitor, [207](#)
- extractDeclName
 - html.cpp, [863](#)
- extractPasName
 - main.cpp, [778](#)
- fail
 - mxx::TPValidator, [421](#)
- failAt
 - mxx::TPValidator, [421](#)
- failHere
 - mxx::TPValidator, [422](#)
- FCMP
 - instruct.hpp, [529](#)

- FieldAccessNode
 - pascal::FieldAccessNode, [235](#)
- fieldName
 - pascal::FieldAccessNode, [236](#)
- fields
 - pascal::CodeGenVisitor::RecordTypeInfo, [375](#)
 - pascal::RecordTypeNode, [377](#)
 - pascal::VariantArm, [464](#)
- FILE
 - mxx, [91](#)
- fileExists
 - html.cpp, [863](#)
- FileFunctionHandler
 - pascal::CodeGenVisitor, [204](#)
- fileFunctions
 - pascal, [100](#)
- filename
 - mxvm::Program, [364](#)
 - mxvm::Validator, [447](#)
 - mxx::TPValidator, [433](#)
 - mxx::XParser, [471](#)
 - scan::token::Token< Ch, String >, [412](#)
- filenames
 - mxvm::Base, [158](#)
- fileVarNames
 - pascal::CodeGenVisitor, [207](#)
- fileVars
 - pascal::CodeGenVisitor, [207](#)
- fill_rect
 - mx_sdl.h, [562](#)
 - sdl.c, [583](#)
- findHandler
 - pascal::BuiltinFunctionRegistry, [170](#)
- findMangledArrayName
 - pascal::CodeGenVisitor, [184](#)
- findMangledFuncName
 - pascal::CodeGenVisitor, [184](#)
- findMangledName
 - pascal::CodeGenVisitor, [184](#)
- first
 - mxvm::Validator::VarNamePair, [464](#)
- fixed
 - mxvm::OpSpec, [274](#)
- flatten
 - mxvm::Program, [328](#)
- flatten_external
 - mxvm::Program, [328](#)
- flatten_inc
 - mxvm::Program, [328](#)
- flatten_label
 - mxvm::Program, [329](#)
- float_to_int
 - std.c, [632](#)
- float_value
 - mxvm::Variable_Value, [457](#)
- floatRegInUse
 - pascal::CodeGenVisitor, [207](#)
- floatRegisters
 - pascal::CodeGenVisitor, [207](#)
- Fn
 - mxvm::Function< R(Args...)>, [245](#)
- fn
 - mxvm::Function< R(Args...)>, [247](#)
- fname
 - mxvm::RuntimeFunction, [381](#)
- foldAndTest
 - pascal, [97](#)
- foldCmpTest
 - pascal, [97](#)
- foldMovArithMov
 - pascal, [97](#)
- foldNumeric
 - pascal::CodeGenVisitor, [185](#)
- foldOrTest
 - pascal, [98](#)
- FOR
 - mxx, [91](#)
- format_hex16
 - mx, [78](#)
- format_hex32
 - mx, [78](#)
- format_hex64
 - mx, [78](#)
- format_hex8
 - mx, [78](#)
- format_hex_no_prefix
 - mx, [78](#)
- format_signal
 - mx, [78](#)
- ForStmtNode
 - pascal::ForStmtNode, [239](#)
- forward_step
 - scan::buffer::StringBuffer< Ch, String >, [403](#)
- found
 - mxx::TPValidator, [422](#)
- FREE
 - instruct.hpp, [529](#)
- free_program_args
 - mx_std.h, [630](#)
 - std.c, [633](#)
- free_surface
 - mx_sdl.h, [562](#)
 - sdl.c, [583](#)
- free_wav
 - mx_sdl.h, [562](#)
 - sdl.c, [583](#)
- freeFloatReg
 - pascal::CodeGenVisitor, [185](#)
- freePtrReg
 - pascal::CodeGenVisitor, [185](#)
- freeReg
 - pascal::CodeGenVisitor, [185](#)
- freeTempPtr
 - pascal::CodeGenVisitor, [185](#)
- fsize
 - io.c, [546](#)

- fullscreen
 - mx::Arguments, [120](#)
- func
 - mxvm::RuntimeFunction, [381](#)
- FuncCallNode
 - pascal::FuncCallNode, [241](#)
- FuncDeclNode
 - pascal::FuncDeclNode, [243](#)
- funcLabel
 - pascal::CodeGenVisitor, [185](#)
- funcs
 - mx::Scope, [387](#)
- funcSignatures
 - pascal::CodeGenVisitor, [207](#)
- FUNCTION
 - mx, [92](#)
- Function
 - mxvm::Function< R(Args...)>, [245](#), [246](#)
- function
 - mxvm::LabelNode, [259](#)
- function_vars
 - mxvm::Validator, [447](#)
- functions
 - mxvm::ModuleParser, [266](#)
- functionScopedArrays
 - pascal::CodeGenVisitor, [208](#)
- functionSetReturn
 - pascal::CodeGenVisitor, [208](#)
- g_argc
 - std.c, [633](#)
- g_args_registered
 - std.c, [633](#)
- g_argv
 - std.c, [634](#)
- g_event
 - sdl.c, [593](#)
- g_font_count
 - sdl.c, [593](#)
- g_fonts
 - sdl.c, [593](#)
- g_render_target_count
 - sdl.c, [593](#)
- g_render_targets
 - sdl.c, [594](#)
- g_renderer_count
 - sdl.c, [594](#)
- g_renderers
 - sdl.c, [594](#)
- g_target_heights
 - sdl.c, [594](#)
- g_target_widths
 - sdl.c, [594](#)
- g_texture_count
 - sdl.c, [594](#)
- g_textures
 - sdl.c, [594](#)
- g_window_count
 - sdl.c, [594](#)
- g_windows
 - sdl.c, [595](#)
- gen_add
 - mxvm::Program, [329](#)
- gen_alloc
 - mxvm::Program, [329](#)
- gen_arth
 - mxvm::Program, [329](#)
- gen_bitop
 - mxvm::Program, [330](#)
- gen_call
 - mxvm::Program, [330](#)
- gen_cmp
 - mxvm::Program, [330](#)
- gen_div
 - mxvm::Program, [330](#)
- gen_done
 - mxvm::Program, [330](#)
- gen_exit
 - mxvm::Program, [330](#)
- gen_fcmp
 - mxvm::Program, [331](#)
- gen_free
 - mxvm::Program, [331](#)
- gen_getline
 - mxvm::Program, [331](#)
- gen_invoke
 - mxvm::Program, [331](#)
- gen_jae
 - mxvm::Program, [331](#)
- gen_jbe
 - mxvm::Program, [331](#)
- gen_jc
 - mxvm::Program, [332](#)
- gen_jmp
 - mxvm::Program, [332](#)
- gen_jnc
 - mxvm::Program, [332](#)
- gen_jno
 - mxvm::Program, [332](#)
- gen_jnp
 - mxvm::Program, [332](#)
- gen_jns
 - mxvm::Program, [332](#)
- gen_jo
 - mxvm::Program, [333](#)
- gen_jp
 - mxvm::Program, [333](#)
- gen_js
 - mxvm::Program, [333](#)
- gen_lea
 - mxvm::Program, [333](#)
- gen_load
 - mxvm::Program, [333](#)
- gen_mod
 - mxvm::Program, [333](#)
- gen_mov
 - mxvm::Program, [334](#)

- gen_mul
 - mxvm::Program, [334](#)
- gen_neg
 - mxvm::Program, [334](#)
- gen_not
 - mxvm::Program, [334](#)
- gen_optimize
 - mxvm::Program, [334](#)
- gen_pop
 - mxvm::Program, [335](#)
- gen_print
 - mxvm::Program, [335](#)
- gen_push
 - mxvm::Program, [335](#)
- gen_realloc
 - mxvm::Program, [336](#)
- gen_ret
 - mxvm::Program, [336](#)
- gen_return
 - mxvm::Program, [336](#)
- gen_stack_load
 - mxvm::Program, [336](#)
- gen_stack_store
 - mxvm::Program, [336](#)
- gen_stack_sub
 - mxvm::Program, [336](#)
- gen_store
 - mxvm::Program, [337](#)
- gen_sub
 - mxvm::Program, [337](#)
- gen_to_float
 - mxvm::Program, [337](#)
- gen_to_int
 - mxvm::Program, [337](#)
- generate
 - pascal::BuiltinFunctionHandler, [169](#)
 - pascal::CodeGenVisitor, [185](#)
 - pascal::FileFunctionHandler, [237](#)
 - pascal::IOFunctionHandler, [257](#)
 - pascal::SDLFunctionHandler, [389](#)
 - pascal::StdFunctionHandler, [399](#)
 - pascal::StringFunctionHandler, [406](#)
- generateCode
 - mxvm::Program, [337](#)
- generateDebugHTML
 - mxvm::Parser, [280](#)
- generateFunctionCall
 - mxvm::Program, [338](#)
- generateInstruction
 - mxvm::Program, [338](#)
- generateInvokeCall
 - mxvm::Program, [338](#)
- generateLoadVar
 - mxvm::Program, [339](#)
- generateObjectAssemblyFile
 - mxvm::Parser, [280](#)
- generateProgramCode
 - mxvm::ModuleParser, [265](#)
- mxvm::Parser, [280](#)
- generateRealConstantName
 - pascal::CodeGenVisitor, [186](#)
- generateWithResult
 - pascal::BuiltinFunctionHandler, [169](#)
 - pascal::FileFunctionHandler, [237](#)
 - pascal::IOFunctionHandler, [258](#)
 - pascal::SDLFunctionHandler, [390](#)
 - pascal::StdFunctionHandler, [400](#)
 - pascal::StringFunctionHandler, [406](#)
- generatingDeferredCode
 - pascal::CodeGenVisitor, [208](#)
- get_clipboard_text
 - mx_sdl.h, [563](#)
 - sdl.c, [583](#)
- get_event_type
 - mx_sdl.h, [563](#)
 - sdl.c, [583](#)
- get_filename
 - scan::token::Token< Ch, String >, [410](#)
- get_key_code
 - mx_sdl.h, [563](#)
 - sdl.c, [583](#)
- get_keyboard_state
 - mx_sdl.h, [563](#)
 - sdl.c, [584](#)
- get_mouse_button
 - mx_sdl.h, [563](#)
 - sdl.c, [584](#)
- get_mouse_buttons
 - mx_sdl.h, [563](#)
 - sdl.c, [584](#)
- get_mouse_state
 - mx_sdl.h, [564](#)
 - sdl.c, [584](#)
- get_mouse_x
 - mx_sdl.h, [564](#)
 - sdl.c, [584](#)
- get_mouse_y
 - mx_sdl.h, [564](#)
 - sdl.c, [585](#)
- get_num_keys
 - mx_sdl.h, [564](#)
 - sdl.c, [585](#)
- get_pos
 - scan::token::Token< Ch, String >, [410](#)
- get_queued_audio_size
 - mx_sdl.h, [564](#)
 - sdl.c, [585](#)
- get_relative_mouse_buttons
 - mx_sdl.h, [564](#)
 - sdl.c, [585](#)
- get_relative_mouse_state
 - mx_sdl.h, [565](#)
 - sdl.c, [585](#)
- get_relative_mouse_x
 - mx_sdl.h, [565](#)
 - sdl.c, [585](#)

- get_relative_mouse_y
 - mx_sdl.h, 565
 - sdl.c, 586
- get_renderer_output_size
 - mx_sdl.h, 565
 - sdl.c, 586
- get_ticks
 - mx_sdl.h, 565
 - sdl.c, 586
- get_window_size
 - mx_sdl.h, 566
 - sdl.c, 586
- getAddOp
 - pascal::PascalParser, 291
- getArithmeticOperator
 - pascal::PascalParser, 291
- getArrayElementSize
 - pascal::CodeGenVisitor, 186
- getArrayInfoForArrayAccess
 - pascal::CodeGenVisitor, 186
- getArrayNameFromBase
 - pascal::CodeGenVisitor, 186
- getch
 - scan::buffer::StringBuffer< Ch, String >, 403
- getCol
 - scan::token::Token< Ch, String >, 410
- getComparisonOperator
 - pascal::PascalParser, 291
- getCurrentEndLabel
 - pascal::CodeGenVisitor, 186
- getCurrentScopeName
 - pascal::CodeGenVisitor, 187
- getExitCode
 - mxvm::Program, 340
- getExpressionType
 - pascal::CodeGenVisitor, 187
- GETLINE
 - instruct.hpp, 529
- getLine
 - scan::token::Token< Ch, String >, 411
- getLineNumber
 - pascal::ASTNode, 143
- getLogicalOperator
 - pascal::PascalParser, 291
- getMangledName
 - mxvm::Program, 340
- getMulOp
 - pascal::PascalParser, 291
- getObjectByName
 - mxvm::Program, 341
- getPlatformSymbolName
 - mxvm::Program, 341
- getPointerBaseTypeName
 - pascal::CodeGenVisitor, 187
- getPointerDerefType
 - pascal::CodeGenVisitor, 187
- getPointerElementSize
 - pascal::CodeGenVisitor, 187
- getRecordFieldOffsetAndSize
 - pascal::CodeGenVisitor, 187
- getRecordTypeSize
 - pascal::CodeGenVisitor, 187
- getRegisterByIndex
 - mxvm::Program, 341
- getRelationalOp
 - pascal::PascalParser, 291
- getReturnType
 - pascal::BuiltinFunctionHandler, 169
 - pascal::FileFunctionHandler, 237
 - pascal::StringFunctionHandler, 406
- getStringFromVar
 - string.cpp, 660
- getTokens
 - scan::Scanner, 384
- getTokenType
 - scan::token::Token< Ch, String >, 411
- getTokenValue
 - scan::token::Token< Ch, String >, 411
- getTypeFromString
 - pascal::CodeGenVisitor, 188
- getTypeSizeByName
 - pascal::CodeGenVisitor, 188
- getTypeString
 - pascal::CodeGenVisitor, 188
- getUnaryOp
 - pascal::PascalParser, 291
- getVariable
 - mxvm::Program, 342
- getVarRecordTypeName
 - pascal::CodeGenVisitor, 188
- getVarRecordTypeNameFromExpr
 - pascal::CodeGenVisitor, 188
- getVarType
 - pascal::CodeGenVisitor, 188
- globalArrays
 - pascal::CodeGenVisitor, 208
- GOTO
 - mxx, 92
- gotoLabels
 - pascal::CodeGenVisitor, 208
- GotoStmtNode
 - pascal::GotoStmtNode, 248
- grabDigits
 - scan::Scanner, 384
- grabId
 - scan::Scanner, 384
- grabSingle
 - scan::Scanner, 384
- grabString
 - scan::Scanner, 385
- grabSymbols
 - scan::Scanner, 385
- GREATER
 - pascal::BinaryOpNode, 161
- GREATER_EQUAL
 - pascal::BinaryOpNode, 161

- greater_flag
 - mxvm::Program, [364](#)
- GREEN
 - mx::Color, [81](#)
- GT
 - expr_parser, [76](#)
- GTE
 - expr_parser, [76](#)
- handle
 - mxvm::Function< R(Args...)>, [247](#)
 - mxvm::RuntimeFunction, [381](#)
- handlers
 - pascal::BuiltinFunctionRegistry, [170](#)
- handles
 - mxvm::RuntimeFunction, [381](#)
- has_semicolon
 - mxvm, [87](#)
- hasInitializer
 - mxvm::VariableNode, [461](#)
- height
 - mx::Arguments, [120](#)
- help
 - mx::Argz< String >, [123](#)
- Hex
 - mxvm, [85](#)
- html.cpp
 - dirOf, [862](#)
 - endsWith, [863](#)
 - ensureCss, [863](#)
 - extractDeclName, [863](#)
 - fileExists, [863](#)
 - main, [863](#)
 - readFile, [863](#)
 - writeHtml, [863](#)
- html_mode
 - mxvm, [89](#)
- HTMLGen
 - mxvm::HTMLGen, [249](#)
- icode.hpp
 - MXVM_BOUNDS_CHECK, [500](#)
- Id
 - mxvm, [85](#)
- identifier
 - pascal::ConstDeclNode::ConstAssignment, [217](#)
- identifiers
 - pascal::ParameterNode, [276](#)
 - pascal::VarDeclNode, [451](#)
- IF
 - mxx, [92](#)
- IfStmtNode
 - pascal::IfStmtNode, [251](#)
- IMMEDIATE
 - pascal::CodeGenVisitor::ValueLocation, [449](#)
- implDecls
 - pascal::UnitNode, [441](#)
- importedArrayInfo
 - pascal::CodeGenVisitor, [208](#)
- importedUnitNames
 - pascal::CodeGenVisitor, [208](#)
- importedUnits
 - mxx::TPValidator, [433](#)
- importedVarTypes
 - pascal::CodeGenVisitor, [208](#)
- IN
 - mxx, [92](#)
 - pascal::BinaryOpNode, [161](#)
- Inc
 - instruct.hpp, [528](#)
- inc
 - mxvm::Base, [158](#)
- include Directory Reference, [65](#)
- include/mxvm Directory Reference, [68](#)
- include/mxvm/ast.hpp, [473](#), [474](#)
- include/mxvm/function.hpp, [488](#), [489](#)
- include/mxvm/html_gen.hpp, [490](#), [491](#)
- include/mxvm/icode.hpp, [491](#), [493](#)
- include/mxvm/instruct.hpp, [526](#), [531](#)
- include/mxvm/mxvm.hpp, [534](#), [535](#)
- include/mxvm/parser.hpp, [535](#), [537](#)
- include/mxvm/valid.hpp, [542](#), [544](#)
- include/mxvm/version_info.hpp, [545](#), [546](#)
- include_path
 - Args, [114](#)
 - mxvm::Parser, [285](#)
- IncType
 - instruct.hpp, [531](#)
- index
 - mx::Argz< String >, [126](#)
 - mxvm::ModuleParser, [266](#)
 - mxvm::Validator, [447](#)
 - mxx::TPValidator, [433](#)
 - mxx::XParser, [471](#)
 - pascal::ArrayAccessNode, [127](#)
 - pascal::ArrayAssignmentNode, [129](#)
 - scan::buffer::StringBuffer< Ch, String >, [405](#)
- inFunctionDepth
 - mxx::TPValidator, [433](#)
- init
 - mx_sdl.h, [566](#)
 - sdl.c, [586](#)
- init_text
 - mx_sdl.h, [566](#)
 - sdl.c, [587](#)
- initArgs
 - mx::Argz< String >, [124](#)
- initializeBuiltins
 - pascal::CodeGenVisitor, [189](#)
- initializeFloatRegisters
 - pascal::CodeGenVisitor, [189](#)
- initializers
 - pascal::ArrayDeclarationNode, [131](#)
 - pascal::VarDeclNode, [451](#)
- initialValue
 - mxvm::VariableNode, [461](#)
- inlineObjects

- mxvm::ProgramNode, [368](#)
- inRecordFieldScope
 - mxv::TPValidator, [422](#)
- insert
 - cstring.c, [656](#)
- instruct.cpp
 - operator<<, [922](#), [923](#)
- instruct.hpp
 - ADD, [528](#)
 - ALLOC, [528](#)
 - AND, [528](#)
 - CALL, [529](#)
 - CMP, [528](#)
 - DIV, [528](#)
 - DONE, [529](#)
 - EXIT, [528](#)
 - FCMP, [529](#)
 - FREE, [529](#)
 - GETLINE, [529](#)
 - Inc, [528](#)
 - IncType, [531](#)
 - INVOKE, [529](#)
 - JA, [528](#)
 - JAE, [529](#)
 - JB, [528](#)
 - JBE, [529](#)
 - JC, [529](#)
 - JE, [528](#)
 - JG, [528](#)
 - JGE, [528](#)
 - JL, [528](#)
 - JLE, [528](#)
 - JMP, [528](#)
 - JNC, [529](#)
 - JNE, [528](#)
 - JNO, [529](#)
 - JNP, [529](#)
 - JNS, [529](#)
 - JNZ, [528](#)
 - JO, [529](#)
 - JP, [529](#)
 - JS, [529](#)
 - JZ, [528](#)
 - LEA, [529](#)
 - LOAD, [528](#)
 - MOD, [528](#)
 - MOV, [528](#)
 - MUL, [528](#)
 - NEG, [529](#)
 - NOT, [528](#)
 - NULL_INC, [528](#)
 - operator<<, [529](#), [530](#)
 - OR, [528](#)
 - POP, [529](#)
 - PRINT, [528](#)
 - PUSH, [529](#)
 - REALLOC, [529](#)
 - RET, [529](#)
 - RETURN, [529](#)
 - STACK_LOAD, [529](#)
 - STACK_STORE, [529](#)
 - STACK_SUB, [529](#)
 - STORE, [528](#)
 - STRING_PRINT, [529](#)
 - SUB, [528](#)
 - TO_FLOAT, [529](#)
 - TO_INT, [529](#)
 - XOR, [528](#)
- instruct_mode
 - mxvm, [89](#)
- instruction
 - mxvm::Instruction, [253](#)
 - mxvm::InstructionNode, [256](#)
- InstructionNode
 - mxvm::InstructionNode, [256](#)
- instructions
 - pascal::CodeGenVisitor, [208](#)
- INT
 - pascal, [96](#)
- int_to_float
 - std.c, [633](#)
- int_value
 - mxvm::Variable_Value, [457](#)
- interfaceDecls
 - pascal::UnitNode, [441](#)
- internString
 - pascal::CodeGenVisitor, [189](#)
- interpret
 - main.cpp, [784](#)
- inttostr
 - cstring.c, [656](#)
- invertJump
 - pascal, [98](#)
- INVOKE
 - instruct.hpp, [529](#)
- io.c
 - fsize, [546](#)
 - mxvm_fgets, [546](#)
 - rand_number, [547](#)
 - seed_random, [547](#)
- io.cpp
 - mxvm_io_fclose, [548](#)
 - mxvm_io_feof, [548](#)
 - mxvm_io_fopen, [548](#)
 - mxvm_io_fprintf, [549](#)
 - mxvm_io_fputs, [549](#)
 - mxvm_io_fread, [549](#)
 - mxvm_io_fseek, [549](#)
 - mxvm_io_fsize, [549](#)
 - mxvm_io_fwrite, [549](#)
 - mxvm_io_mxvm_fgets, [550](#)
 - mxvm_io_rand_number, [550](#)
 - mxvm_io_seed_random, [550](#)
- IOFunctionHandler
 - pascal::CodeGenVisitor, [204](#)
- is_c_sym

- scan::Scanner, 385
- is_global
 - mxvm::Variable, 453
 - mxvm::VariableNode, 461
- is_key_pressed
 - mx_sdl.h, 566
 - sdl.c, 587
- is_label
 - mxvm, 87
- is_stdio_name
 - mxvm, 87
- isAddOp
 - mxx::TPValidator, 422
- isAddOperator
 - pascal::PascalParser, 291
- isArray
 - pascal::CodeGenVisitor::RecordField, 374
- isArrayAccess
 - pascal::PascalParser, 291
- isArrayTypeName
 - pascal::CodeGenVisitor, 189
- isBuiltinConst
 - mxx::TPValidator, 422
- isBuiltinFunction
 - pascal::PascalParser, 292
- isBuiltinProcedure
 - pascal::PascalParser, 292
- isBuiltinType
 - mxx::TPValidator, 422
- isBuiltinTypeName
 - mxx::TPValidator, 422
- isConstant
 - mxvm::Program, 343
- isDownto
 - pascal::ForStmtNode, 239
- isDynamic
 - pascal::ArrayInfo, 133
- isFloatLiteral
 - pascal::CodeGenVisitor, 189
- isFloatReg
 - pascal::CodeGenVisitor, 189
- isFuncDeclaredHere
 - mxx::TPValidator, 422
- isFunctionReturningOwnedPtr
 - mxvm, 87
- isFunctionValid
 - mxvm::Program, 343
- isIdLike
 - mxvm, 88
- isImmediate
 - mxvm, 88
- isImportedUnit
 - pascal::CodeGenVisitor, 189
- isInteger
 - pascal::NumberNode, 270
- isIntegerLiteral
 - pascal::CodeGenVisitor, 190
- isIntLiteral
 - pascal::CodeGenVisitor, 190
- isKeyword
 - pascal::PascalParser, 292
- isKW
 - mxx::TPValidator, 423
- isLabelDef
 - pascal, 98
- isMulOp
 - mxx::TPValidator, 423
- isMulOperator
 - pascal::PascalParser, 292
- isNumeric
 - mxvm::ExpressionNode, 228
- isParamDeclaredHere
 - mxx::TPValidator, 423
- isParmReg
 - pascal::CodeGenVisitor, 190
- isPascalKeyword
 - mxx::TPValidator, 423
- isProcDeclaredHere
 - mxx::TPValidator, 423
- isPtrLike
 - pascal::CodeGenVisitor, 190
- isPtrReg
 - pascal::CodeGenVisitor, 190
- isReal
 - pascal::NumberNode, 270
- isRealConstSymbol
 - pascal::CodeGenVisitor, 190
- isRealNumber
 - pascal::CodeGenVisitor, 190
- isRecordTypeName
 - pascal::CodeGenVisitor, 190
- isReg
 - pascal::CodeGenVisitor, 191
- isRegisterName
 - pascal, 98
- isRegisterSlot
 - pascal::CodeGenVisitor, 191
- isRelationalOperator
 - pascal::PascalParser, 292
- isRelOp
 - mxx::TPValidator, 423
- isSetOp
 - mxx::TPValidator, 424
- isStringLike
 - string.cpp, 660
- isStringVar
 - pascal::CodeGenVisitor, 191
- isTempPtr
 - pascal::CodeGenVisitor, 191
- isTempVar
 - pascal::CodeGenVisitor, 191
- isType
 - pascal::PascalParser, 292
- isTypeDeclaredHere
 - mxx::TPValidator, 424
- isUnit

- pascal::CodeGenVisitor, [209](#)
- isUnitSource
 - pascal::PascalParser, [292](#)
- isVar
 - pascal::ParameterNode, [276](#)
- isVarDeclaredHere
 - mxx::TPValidator, [424](#)
- isVariable
 - mxvm::Program, [343](#)
- isVariableInCurrentScope
 - mxx::TPValidator, [424](#)
- JA
 - instruct.hpp, [528](#)
- JAE
 - instruct.hpp, [529](#)
- JB
 - instruct.hpp, [528](#)
- JBE
 - instruct.hpp, [529](#)
- JC
 - instruct.hpp, [529](#)
- JE
 - instruct.hpp, [528](#)
- JG
 - instruct.hpp, [528](#)
- JGE
 - instruct.hpp, [528](#)
- JL
 - instruct.hpp, [528](#)
- JLE
 - instruct.hpp, [528](#)
- JMP
 - instruct.hpp, [528](#)
- JNC
 - instruct.hpp, [529](#)
- JNE
 - instruct.hpp, [528](#)
- JNO
 - instruct.hpp, [529](#)
- JNP
 - instruct.hpp, [529](#)
- JNS
 - instruct.hpp, [529](#)
- JNZ
 - instruct.hpp, [528](#)
- JO
 - instruct.hpp, [529](#)
- join_lines
 - mxvm, [88](#)
- JP
 - instruct.hpp, [529](#)
- JS
 - instruct.hpp, [529](#)
- JZ
 - instruct.hpp, [528](#)
- KEY_CODE
 - mxvm, [85](#)
- KEY_DATA
 - mxvm, [85](#)
- KEY_MODULE
 - mxvm, [85](#)
- KEY_OBJECT
 - mxvm, [85](#)
- KEY_PROGRAM
 - mxvm, [84](#)
- KEY_SECTION
 - mxvm, [85](#)
- key_str
 - mxx, [93](#)
- Keywords
 - mxvm, [84](#)
 - mxx, [91](#)
- keywords
 - mxvm, [90](#)
- KeywordType
 - types, [104](#)
- kind
 - mxvm::ParsedOp, [277](#)
- kInstructions
 - mxvm, [90](#)
- kKeywords
 - mxvm, [90](#)
- kOpSpecs
 - mxvm, [90](#)
- kTypes
 - mxvm, [90](#)
- KW_BREAK
 - types, [104](#)
- KW_CONTINUE
 - types, [104](#)
- KW_DEFINE
 - types, [104](#)
- KW_ELSE
 - types, [104](#)
- KW_FOR
 - types, [104](#)
- KW_IF
 - types, [104](#)
- KW_LET
 - types, [104](#)
- KW_PROC
 - types, [104](#)
- KW_RETURN
 - types, [104](#)
- KW_SWITCH
 - types, [104](#)
- KW_WHILE
 - types, [104](#)
- kwStr
 - types, [107](#)
- LABEL
 - mxx, [92](#)
- Label
 - mxvm, [85](#)
- label

- mxvm::Instruction, [253](#)
 - mxvm::Operand, [272](#)
 - pascal::GotoStmtNode, [249](#)
 - pascal::LabelStmtNode, [261](#)
- label_value
 - mxvm::Variable_Value, [458](#)
- labelCounter
 - pascal::CodeGenVisitor, [209](#)
- LabelNode
 - mxvm::LabelNode, [259](#)
- labels
 - mxvm::Base, [158](#)
- LabelStmtNode
 - pascal::LabelStmtNode, [260](#)
- last_call_ret_type
 - mxvm::Program, [364](#)
- last_call_returns_owned
 - mxvm::Program, [364](#)
- last_call_returns_owned_ptr
 - mxvm::Program, [364](#)
- last_cmp_type
 - mxvm::Program, [364](#)
- LastCmpType
 - mxvm::Program, [316](#)
- lc
 - pascal::CodeGenVisitor, [191](#)
- LEA
 - instruct.hpp, [529](#)
- left
 - pascal::BinaryOpNode, [162](#)
- LESS
 - pascal::BinaryOpNode, [161](#)
- LESS_EQUAL
 - pascal::BinaryOpNode, [161](#)
- less_flag
 - mxvm::Program, [365](#)
- lexer
 - expr_parser::ExprParser, [232](#)
- line
 - scan::token::Token< Ch, String >, [412](#)
- line_number
 - pascal::ASTNode, [144](#)
- LINUX
 - mxvm, [86](#)
- LOAD
 - instruct.hpp, [528](#)
- load_font
 - mx_sdl.h, [567](#)
 - sdl.c, [587](#)
- load_texture
 - mx_sdl.h, [567](#)
 - sdl.c, [587](#)
- load_texture_color_key
 - mx_sdl.h, [568](#)
 - sdl.c, [588](#)
- load_texture_color_key_rgb
 - mx_sdl.h, [568](#)
 - sdl.c, [588](#)
- load_wav
 - mx_sdl.h, [568](#)
 - sdl.c, [589](#)
- locate_sections
 - pascal, [98](#)
- location
 - pascal::CodeGenVisitor::ValueLocation, [449](#)
- lock_texture
 - mx_sdl.h, [569](#)
 - sdl.c, [589](#)
- LOGICAL_AND
 - expr_parser, [76](#)
- LOGICAL_OR
 - expr_parser, [76](#)
- lookUp
 - types, [106](#)
- lookup_int8
 - scan::token::TokenMap< Ch, MAX_SIZE >, [414](#)
- lookUp_Keyword
 - types, [106](#)
- lookUpCode
 - mx::Argz< String >, [124](#)
- loopContinueLabels
 - pascal::CodeGenVisitor, [209](#)
- loopEndLabels
 - pascal::CodeGenVisitor, [209](#)
- lower
 - mxx::TPValidator, [424](#)
- lowerBound
 - pascal::ArrayInfo, [133](#)
 - pascal::ArrayTypeNode, [137](#)
- LPAREN
 - expr_parser, [76](#)
- LT
 - expr_parser, [76](#)
- LTE
 - expr_parser, [76](#)
- MAGENTA
 - mx::Color, [81](#)
- main
 - html.cpp, [863](#)
 - main.cpp, [778](#), [785](#)
- main.cpp
 - action_interpret, [785](#)
 - action_translate, [785](#)
 - collectAndRegisterAllExterns, [785](#)
 - compile, [784](#)
 - compileFile, [778](#)
 - createMakefile, [785](#)
 - extractPasName, [778](#)
 - interpret, [784](#)
 - main, [778](#), [785](#)
 - null_action, [784](#)
 - operator<<, [786](#)
 - print_help, [786](#)
 - proc_args, [786](#)
 - process_arguments, [786](#)
 - removeComments, [778](#)

- signal_action, 786
- signal_program, 787
- translate, 784
- translate_x64, 787
- vm_action, 784
- vm_target, 784
- x86_64_linux, 785
- x86_64_macos, 785
- x86_64_windows, 785
- main_function
 - mxvm::Program, 365
- mainpage.md, 546
- Makefile
 - Args, 114
- mangleVariableName
 - pascal::CodeGenVisitor, 191
- mangleWithScope
 - pascal::CodeGenVisitor, 192
- markAllocatedPtr
 - pascal::CodeGenVisitor, 192
- match
 - mxvm::ModuleParser, 265
 - mxvm::Validator, 445
 - mxx::TPValidator, 424, 425
 - pascal::PascalParser, 292, 293
- MathFunctionHandler
 - pascal::CodeGenVisitor, 204
- maxArgs
 - mxvm::OpSpec, 274
- Member
 - mxvm, 85
- MEMORY
 - pascal::CodeGenVisitor::ValueLocation, 449
- memoryDump
 - mxvm::Program, 344
- minArgs
 - mxvm::OpSpec, 274
- MINUS
 - expr_parser, 75
 - pascal::BinaryOpNode, 161
 - pascal::UnaryOpNode, 439
- MINUSEQ
 - expr_parser, 76
- MOD
 - expr_parser, 76
 - instruct.hpp, 528
 - mxx, 92
 - pascal::BinaryOpNode, 161
- mod
 - mxvm::ExternalFunction, 234
- mod_name
 - mxvm::ModuleParser, 266
 - mxvm::RuntimeFunction, 381
- Mode
 - mxvm, 85
- MODE_COMPILE
 - mxvm, 85
- MODE_INTERPRET
 - mxvm, 85
- MODULE
 - mxvm::SectionNode, 391
- module
 - mxvm::ExternalFunction, 234
- module_path
 - Args, 114
 - mxvm::Parser, 285
- ModuleNode
 - mxvm::ModuleNode, 262
- ModuleParser
 - mxvm::ModuleParser, 264
 - mxvm::Parser, 285
- modules Directory Reference, 67
- modules/io Directory Reference, 67
- modules/io/io.c, 546, 547
- modules/io/io.cpp, 547, 550
- modules/sdl Directory Reference, 70
- modules/sdl/mx_sdl.h, 554, 573
- modules/sdl/sdl.c, 575, 595
- modules/sdl/sdl.cpp, 603, 615
- modules/std Directory Reference, 72
- modules/std/mx_std.h, 629, 631
- modules/std/std.c, 631, 634
- modules/std/std.cpp, 635, 644
- modules/string Directory Reference, 72
- modules/string/cstring.c, 655, 657
- modules/string/string.cpp, 659, 663
- MOV
 - instruct.hpp, 528
- msg
 - scan::ScanExcept, 383
- MUL
 - expr_parser, 76
 - instruct.hpp, 528
- MULTIPLY
 - pascal::BinaryOpNode, 161
- mulVariables
 - mxvm::Program, 345
- mx, 76
 - ARG_DOUBLE, 78
 - ARG_DOUBLE_VALUE, 78
 - ARG_NONE, 78
 - ARG_SINGLE, 77
 - ARG_SINGLE_VALUE, 77
 - ArgType, 77
 - color_, 79
 - format_hex16, 78
 - format_hex32, 78
 - format_hex64, 78
 - format_hex8, 78
 - format_hex_no_prefix, 78
 - format_signal, 78
 - proc_args, 78
 - terminal_supports_color, 79
- mx::ArgException< String >, 112
 - ArgException, 113
 - text, 113

- value, [113](#)
- mx::Argument< String >, [115](#)
 - arg_letter, [117](#)
 - arg_name, [117](#)
 - arg_type, [117](#)
 - arg_value, [117](#)
 - Argument, [116](#)
 - desc, [117](#)
 - operator<=>, [116](#)
 - operator=, [117](#)
- mx::ArgumentData< String >, [118](#)
 - argc, [119](#)
 - args, [119](#)
 - ArgumentData, [118](#)
 - operator=, [119](#)
- mx::Arguments, [119](#)
 - fullscreen, [120](#)
 - height, [120](#)
 - path, [120](#)
 - width, [120](#)
- mx::Argz< String >, [120](#)
 - addOptionDouble, [122](#)
 - addOptionDoubleValue, [122](#)
 - addOptionSingle, [123](#)
 - addOptionSingleValue, [123](#)
 - arg_data, [125](#)
 - arg_info, [126](#)
 - Argz, [121](#), [122](#)
 - cindex, [126](#)
 - help, [123](#)
 - index, [126](#)
 - initArgs, [124](#)
 - lookUpCode, [124](#)
 - operator=, [124](#), [125](#)
 - proc, [125](#)
 - reset, [125](#)
- mx::Color, [79](#)
 - BG_GREEN, [80](#)
 - BG_RED, [80](#)
 - BG_YELLOW, [80](#)
 - BLACK, [80](#)
 - BLUE, [80](#)
 - BOLD, [80](#)
 - BRIGHT_BLUE, [80](#)
 - BRIGHT_CYAN, [80](#)
 - BRIGHT_GREEN, [80](#)
 - BRIGHT_MAGENTA, [80](#)
 - BRIGHT_RED, [80](#)
 - BRIGHT_YELLOW, [81](#)
 - CYAN, [81](#)
 - DIM, [81](#)
 - GREEN, [81](#)
 - MAGENTA, [81](#)
 - RED, [81](#)
 - RESET, [81](#)
 - WHITE, [81](#)
 - YELLOW, [81](#)
- mx::Exception, [223](#)
 - error, [224](#)
 - Exception, [224](#)
- mx::StringType, [109](#)
- mx_sdl.h
 - blit_surface, [558](#)
 - clear, [558](#)
 - clear_queued_audio, [558](#)
 - close_audio, [558](#)
 - create_render_target, [559](#)
 - create_renderer, [559](#)
 - create_rgb_surface, [559](#)
 - create_texture, [559](#)
 - create_window, [560](#)
 - delay, [560](#)
 - destroy_render_target, [561](#)
 - destroy_renderer, [561](#)
 - destroy_texture, [561](#)
 - destroy_window, [561](#)
 - draw_line, [561](#)
 - draw_point, [561](#)
 - draw_rect, [562](#)
 - draw_text, [562](#)
 - fill_rect, [562](#)
 - free_surface, [562](#)
 - free_wav, [562](#)
 - get_clipboard_text, [563](#)
 - get_event_type, [563](#)
 - get_key_code, [563](#)
 - get_keyboard_state, [563](#)
 - get_mouse_button, [563](#)
 - get_mouse_buttons, [563](#)
 - get_mouse_state, [564](#)
 - get_mouse_x, [564](#)
 - get_mouse_y, [564](#)
 - get_num_keys, [564](#)
 - get_queued_audio_size, [564](#)
 - get_relative_mouse_buttons, [564](#)
 - get_relative_mouse_state, [565](#)
 - get_relative_mouse_x, [565](#)
 - get_relative_mouse_y, [565](#)
 - get_renderer_output_size, [565](#)
 - get_ticks, [565](#)
 - get_window_size, [566](#)
 - init, [566](#)
 - init_text, [566](#)
 - is_key_pressed, [566](#)
 - load_font, [567](#)
 - load_texture, [567](#)
 - load_texture_color_key, [568](#)
 - load_texture_color_key_rgb, [568](#)
 - load_wav, [568](#)
 - lock_texture, [569](#)
 - open_audio, [569](#)
 - pause_audio, [569](#)
 - poll_event, [570](#)
 - present, [570](#)
 - present_scaled, [570](#)
 - present_stretched, [570](#)

- queue_audio, 570
- quit, 571
- quit_text, 571
- render_texture, 571
- set_clipboard_text, 571
- set_draw_color, 571
- set_render_target, 572
- set_window_fullscreen, 572
- set_window_icon, 572
- set_window_position, 572
- set_window_title, 572
- show_cursor, 572
- unlock_texture, 572
- update_texture, 573
- mx_std.h
 - argc, 630
 - argv, 630
 - free_program_args, 630
 - set_program_args, 630
- mx_strdup_c
 - std.c, 633
- mxvm, 81
 - Any, 85
 - AnyTail, 86
 - ArgsTail, 86
 - buildInstructionSet, 86
 - buildKeywordSet, 86
 - DARWIN, 86
 - darwin_prefix_calls, 86
 - debug_mode, 89
 - error_label_count, 89
 - except_assert, 87
 - has_semicolon, 87
 - Hex, 85
 - html_mode, 89
 - ld, 85
 - instruct_mode, 89
 - is_label, 87
 - is_stdio_name, 87
 - isFunctionReturningOwnedPtr, 87
 - isIdLike, 88
 - isImmediate, 88
 - join_lines, 88
 - KEY_CODE, 85
 - KEY_DATA, 85
 - KEY_MODULE, 85
 - KEY_OBJECT, 85
 - KEY_PROGRAM, 84
 - KEY_SECTION, 85
 - Keywords, 84
 - keywords, 90
 - kInstructions, 90
 - kKeywords, 90
 - kOpSpecs, 90
 - kTypes, 90
 - Label, 85
 - LINUX, 86
 - Member, 85
 - Mode, 85
 - MODE_COMPILE, 85
 - MODE_INTERPRET, 85
 - None, 86
 - Num, 85
 - OP_CONSTANT, 85
 - OP_VARIABLE, 85
 - OperandType, 85
 - OpKind, 85
 - opt_core_lines, 88
 - opt_darwin_lines, 88
 - opt_linux_lines, 88
 - opt_x64_windows_lines, 88
 - Platform, 85
 - releaseOwnedPointer, 88
 - runtime_call, 84
 - StackValue, 84
 - stdio_index, 89
 - Str, 85
 - toStringFromVarType, 89
 - VAR_ARRAY, 86
 - VAR_BYTE, 86
 - VAR_EXTERN, 86
 - VAR_FLOAT, 86
 - VAR_INTEGER, 86
 - VAR_LABEL, 86
 - VAR_NULL, 86
 - VAR_POINTER, 86
 - VAR_STRING, 86
 - VAriety, 86
 - VarType, 86
 - WINX64, 86
 - x64_opt_core_lines, 89
 - x64_sp_mod16, 90
 - xmm_offset, 90
- MXVM – Virtual Machine, Compiler & Code Generator, 1
- mxvm::ASTNode, 139
 - ~ASTNode, 140
 - accept, 140
 - toString, 141
- mxvm::ASTVisitor, 144
 - ~ASTVisitor, 145
 - visit, 145
- mxvm::Base, 152
 - ~Base, 154
 - add_allocated, 154
 - add_extern, 155
 - add_filename, 155
 - add_global, 155
 - add_instruction, 155
 - add_label, 156
 - add_object, 156
 - add_runtime_extern, 156
 - add_variable, 157
 - allocated, 157
 - assembly_code, 157
 - Base, 154

- base, 158
- external, 158
- external_functions, 158
- filenames, 158
- inc, 158
- labels, 158
- name, 158
- object_map, 159
- operator=, 157
- root_name, 159
- setMainBase, 157
- vars, 159
- mxvm::CommentNode, 213
 - accept, 214
 - CommentNode, 214
 - text, 214
 - toString, 214
- mxvm::ExpressionNode, 226
 - accept, 227
 - ExpressionNode, 227
 - isNumeric, 228
 - toString, 227
 - value, 228
- mxvm::ExternalFunction, 233
 - mod, 234
 - module, 234
 - name, 234
 - operator==, 233
 - ret_owned, 234
 - ret_type, 234
- mxvm::Function< R(Args...)>, 244
 - ~Function, 246
 - Fn, 245
 - fn, 247
 - Function, 245, 246
 - handle, 247
 - operator(), 247
 - operator=, 247
- mxvm::Function< Sig >, 244
- mxvm::HTMLGen, 249
 - HTMLGen, 249
 - output, 250
 - source, 250
- mxvm::Instruction, 252
 - instruction, 253
 - label, 253
 - op1, 253
 - op2, 254
 - op3, 254
 - toString, 253
 - vop, 254
- mxvm::InstructionNode, 255
 - accept, 256
 - addOperand, 256
 - instruction, 256
 - InstructionNode, 256
 - operands, 256
 - toString, 256
- mxvm::LabelNode, 258
 - accept, 259
 - function, 259
 - LabelNode, 259
 - name, 259
 - toString, 259
- mxvm::ModuleNode, 261
 - accept, 262
 - ModuleNode, 262
 - name, 263
 - toString, 262
- mxvm::ModuleParser, 263
 - at, 264
 - functions, 266
 - generateProgramCode, 265
 - index, 266
 - match, 265
 - mod_name, 266
 - ModuleParser, 264
 - next, 265
 - operator=, 265
 - operator[], 266
 - parse, 266
 - parser_mode, 267
 - require, 266
 - scan, 266
 - scanner, 267
 - token, 267
- mxvm::ObjectNode, 270
 - accept, 271
 - name, 272
 - ObjectNode, 271
 - toString, 271
- mxvm::Operand, 272
 - label, 272
 - object, 272
 - op, 272
 - op_value, 273
 - type, 273
- mxvm::OpSpec, 274
 - fixed, 274
 - maxArgs, 274
 - minArgs, 274
 - name, 274
 - varPolicy, 274
- mxvm::ParsedOp, 276
 - at, 277
 - kind, 277
 - text, 277
- mxvm::Parser, 278
 - collectObjectNames, 280
 - generateDebugHTML, 280
 - generateObjectAssemblyFile, 280
 - generateProgramCode, 280
 - include_path, 285
 - module_path, 285
 - ModuleParser, 285
 - object_mode, 285

- object_name, [285](#)
- object_path, [285](#)
- operator=, [281](#)
- operator[], [281](#)
- parse, [281](#)
- parseAST, [281](#)
- parseCodeInstruction, [281](#)
- parseComment, [282](#)
- parseDataVariable, [282](#)
- parseLabel, [282](#)
- parseModule, [282](#)
- parseObject, [282](#)
- parseProgramOrObject, [282](#)
- Parser, [280](#)
- parser_mode, [285](#)
- parseSection, [283](#)
- platform, [285](#)
- printObjectHTML, [283](#)
- processCodeSection, [283](#)
- processDataSection, [283](#)
- processModuleFile, [283](#)
- processModuleSection, [283](#)
- processObjectFile, [283](#)
- processObjectSection, [284](#)
- registerObjectExterns, [284](#)
- resolveLabelReference, [284](#)
- scan, [284](#)
- scanner, [285](#)
- setDefaultVariableValue, [284](#)
- setVariableValue, [284](#)
- source_file, [286](#)
- validator, [286](#)
- mxvm::PasHTMLGen, [299](#)
 - defaultCss, [300](#)
 - output, [300](#)
 - PasHTMLGen, [300](#)
 - source, [300](#)
- mxvm::Program, [307](#)
 - ~Program, [316](#)
 - add_standard, [317](#)
 - addVariables, [317](#)
 - args, [364](#)
 - carry_flag, [364](#)
 - CMP_FLOAT, [316](#)
 - CMP_INTEGER, [316](#)
 - CMP_NONE, [316](#)
 - createTempVariable, [317](#)
 - divVariables, [317](#)
 - escapeNewLines, [318](#)
 - exec, [318](#)
 - exec_add, [318](#)
 - exec_alloc, [318](#)
 - exec_and, [319](#)
 - exec_call, [319](#)
 - exec_cmp, [319](#)
 - exec_div, [319](#)
 - exec_done, [319](#)
 - exec_exit, [319](#)
 - exec_fcmp, [320](#)
 - exec_free, [320](#)
 - exec_getline, [320](#)
 - exec_invoke, [320](#)
 - exec_ja, [320](#)
 - exec_jae, [320](#)
 - exec_jb, [321](#)
 - exec_jbe, [321](#)
 - exec_jc, [321](#)
 - exec_je, [321](#)
 - exec_jg, [321](#)
 - exec_jge, [321](#)
 - exec_jl, [321](#)
 - exec_jle, [322](#)
 - exec_jmp, [322](#)
 - exec_jnc, [322](#)
 - exec_jne, [322](#)
 - exec_jno, [322](#)
 - exec_jnp, [322](#)
 - exec_jns, [322](#)
 - exec_jnz, [322](#)
 - exec_jo, [323](#)
 - exec_jp, [323](#)
 - exec_js, [323](#)
 - exec_jz, [323](#)
 - exec_lea, [323](#)
 - exec_load, [323](#)
 - exec_mod, [324](#)
 - exec_mov, [324](#)
 - exec_mul, [324](#)
 - exec_neg, [324](#)
 - exec_not, [324](#)
 - exec_or, [324](#)
 - exec_pop, [325](#)
 - exec_print, [325](#)
 - exec_push, [325](#)
 - exec_realloc, [325](#)
 - exec_ret, [326](#)
 - exec_return, [326](#)
 - exec_stack_load, [326](#)
 - exec_stack_store, [326](#)
 - exec_stack_sub, [326](#)
 - exec_store, [327](#)
 - exec_string_print, [327](#)
 - exec_sub, [327](#)
 - exec_to_float, [327](#)
 - exec_to_int, [327](#)
 - exec_xor, [328](#)
 - exitCode, [364](#)
 - filename, [364](#)
 - flatten, [328](#)
 - flatten_external, [328](#)
 - flatten_inc, [328](#)
 - flatten_label, [329](#)
 - gen_add, [329](#)
 - gen_alloc, [329](#)
 - gen_arth, [329](#)
 - gen_bitop, [330](#)

gen_call, 330
gen_cmp, 330
gen_div, 330
gen_done, 330
gen_exit, 330
gen_fcmp, 331
gen_free, 331
gen_getline, 331
gen_invoke, 331
gen_jae, 331
gen_jbe, 331
gen_jc, 332
gen_jmp, 332
gen_jnc, 332
gen_jno, 332
gen_jnp, 332
gen_jns, 332
gen_jo, 333
gen_jp, 333
gen_js, 333
gen_lea, 333
gen_load, 333
gen_mod, 333
gen_mov, 334
gen_mul, 334
gen_neg, 334
gen_not, 334
gen_optimize, 334
gen_pop, 335
gen_print, 335
gen_push, 335
gen_realloc, 336
gen_ret, 336
gen_return, 336
gen_stack_load, 336
gen_stack_store, 336
gen_stack_sub, 336
gen_store, 337
gen_sub, 337
gen_to_float, 337
gen_to_int, 337
generateCode, 337
generateFunctionCall, 338
generateInstruction, 338
generateInvokeCall, 338
generateLoadVar, 339
getExitCode, 340
getMangledName, 340
getObjectByName, 341
getPlatformSymbolName, 341
getRegisterByIndex, 341
getVariable, 342
greater_flag, 364
isConstant, 343
isFunctionValid, 343
isVariable, 343
last_call_ret_type, 364
last_call_returns_owned, 364
last_call_returns_owned_ptr, 364
last_cmp_type, 364
LastCmpType, 316
less_flag, 365
main_function, 365
memoryDump, 344
mulVariables, 345
name, 365
object, 365
object_external, 365
objects, 365
operator=, 345
optimize_core, 345
optimize_darwin, 345
parent, 365
Parser, 363
pc, 365
platform, 366
post, 345
print, 346
printfFormatted, 346
Program, 316
result, 366
running, 366
setArgs, 346
setObject, 347
setVariableFromConstant, 347
setVariableFromString, 347
stack, 366
stop, 347
subVariables, 348
sysv_analyzeRegAlloc, 348
sysv_emitFlushRegs, 348
sysv_emitLoadVar, 348
sysv_emitReloadRegs, 349
sysv_emitRestoreRegs, 349
sysv_emitSaveRegs, 349
sysv_emitStoreVar, 349
sysv_emitStoreVarImm, 349
sysv_reg_save_order, 366
sysv_reg_vars, 366
validateNames, 350
variableFromOperand, 350
x64_analyzeRegAlloc, 350
x64_direct_stack_adjust, 351
x64_emit_job_func, 351
x64_emitFlushRegs, 351
x64_emitLoadVar, 351
x64_emitReloadRegs, 352
x64_emitRestoreRegs, 352
x64_emitSaveRegs, 352
x64_emitStoreVar, 352
x64_emitStoreVarImm, 352
x64_gen_alloc, 353
x64_gen_arth, 353
x64_gen_bitop, 353
x64_gen_call, 353
x64_gen_cmp, 354

- x64_gen_div, [354](#)
- x64_gen_done, [354](#)
- x64_gen_exit, [354](#)
- x64_gen_fcmp, [354](#)
- x64_gen_free, [354](#)
- x64_gen_getline, [355](#)
- x64_gen_invoke, [355](#)
- x64_gen_jae, [355](#)
- x64_gen_jbe, [355](#)
- x64_gen_jc, [355](#)
- x64_gen_jump, [355](#)
- x64_gen_jnc, [356](#)
- x64_gen_jno, [356](#)
- x64_gen_jnp, [356](#)
- x64_gen_jns, [356](#)
- x64_gen_jo, [356](#)
- x64_gen_jp, [356](#)
- x64_gen_js, [357](#)
- x64_gen_lea, [357](#)
- x64_gen_load, [357](#)
- x64_gen_mod, [357](#)
- x64_gen_mov, [357](#)
- x64_gen_neg, [357](#)
- x64_gen_not, [358](#)
- x64_gen_pop, [358](#)
- x64_gen_print, [358](#)
- x64_gen_push, [358](#)
- x64_gen_realloc, [358](#)
- x64_gen_ret, [359](#)
- x64_gen_return, [359](#)
- x64_gen_stack_load, [359](#)
- x64_gen_stack_store, [359](#)
- x64_gen_stack_sub, [359](#)
- x64_gen_store, [359](#)
- x64_gen_to_float, [360](#)
- x64_gen_to_int, [360](#)
- x64_generateCode, [360](#)
- x64_generateFunctionCall, [360](#)
- x64_generateInstruction, [361](#)
- x64_generateInvokeCall, [361](#)
- x64_generateLoadVar, [361](#), [362](#)
- x64_getRegisterByIndex, [362](#)
- x64_reg_save_order, [366](#)
- x64_reg_vars, [366](#)
- x64_release_call_area, [363](#)
- x64_reserve_call_area, [363](#)
- xmm_offset, [367](#)
- zero_flag, [367](#)
- mxvm::ProgramNode, [367](#)
 - accept, [368](#)
 - addInlineObject, [368](#)
 - addSection, [368](#)
 - inlineObjects, [368](#)
 - name, [368](#)
 - object, [368](#)
 - root_name, [369](#)
 - sections, [369](#)
 - toString, [368](#)
- mxvm::RuntimeFunction, [379](#)
 - ~RuntimeFunction, [380](#)
 - call, [381](#)
 - fname, [381](#)
 - func, [381](#)
 - handle, [381](#)
 - handles, [381](#)
 - mod_name, [381](#)
 - operator=, [381](#)
 - RuntimeFunction, [380](#)
- mxvm::SectionNode, [390](#)
 - accept, [391](#)
 - addStatement, [391](#)
 - CODE, [391](#)
 - DATA, [391](#)
 - MODULE, [391](#)
 - OBJECT, [391](#)
 - SectionNode, [391](#)
 - SectionType, [391](#)
 - statements, [392](#)
 - toString, [391](#)
 - type, [392](#)
- mxvm::Stack, [397](#)
 - data, [398](#)
 - empty, [397](#)
 - operator[], [397](#)
 - pop, [398](#)
 - push, [398](#)
 - size, [398](#)
 - Stack, [397](#)
- mxvm::UseLabel, [442](#)
 - at, [442](#)
 - name, [442](#)
- mxvm::UseVar, [442](#)
 - at, [443](#)
 - name, [443](#)
- mxvm::Validator, [443](#)
 - collect_labels, [445](#)
 - collect_objects, [445](#)
 - current_function, [447](#)
 - filename, [447](#)
 - function_vars, [447](#)
 - index, [447](#)
 - match, [445](#)
 - next, [445](#)
 - parseOperand, [445](#)
 - parseOperandList, [445](#)
 - peekIs, [446](#)
 - require, [446](#)
 - scanner, [448](#)
 - source, [448](#)
 - token, [448](#)
 - tokenTypeToString, [446](#)
 - validate, [446](#)
 - validateAgainstSpec, [447](#)
 - Validator, [444](#)
 - var_names, [448](#)
- mxvm::Validator::VarNamePair, [464](#)

- first, [464](#)
- second, [464](#)
- mxvm::Variable, [451](#)
 - is_global, [453](#)
 - obj_name, [453](#)
 - operator=, [452](#)
 - setExtern, [453](#)
 - toString, [453](#)
 - type, [453](#)
 - var_name, [454](#)
 - var_value, [454](#)
 - Variable, [452](#)
- mxvm::Variable_Value, [455](#)
 - buffer_size, [457](#)
 - float_value, [457](#)
 - int_value, [457](#)
 - label_value, [458](#)
 - operator=, [456](#)
 - owns, [458](#)
 - ptr_count, [458](#)
 - ptr_size, [458](#)
 - ptr_value, [458](#)
 - released, [459](#)
 - str_value, [459](#)
 - type, [459](#)
 - Variable_Value, [456](#)
- mxvm::VariableNode, [459](#)
 - accept, [461](#)
 - buffer_size, [461](#)
 - hasInitializer, [461](#)
 - initialValue, [461](#)
 - is_global, [461](#)
 - name, [461](#)
 - object, [462](#)
 - toString, [461](#)
 - type, [462](#)
 - VariableNode, [460](#), [461](#)
- MXVM_BOUNDS_CHECK
 - icode.hpp, [500](#)
- mxvm_fgets
 - io.c, [546](#)
- mxvm_io_fclose
 - io.cpp, [548](#)
- mxvm_io_feof
 - io.cpp, [548](#)
- mxvm_io_fopen
 - io.cpp, [548](#)
- mxvm_io_fprintf
 - io.cpp, [549](#)
- mxvm_io_fputs
 - io.cpp, [549](#)
- mxvm_io_fread
 - io.cpp, [549](#)
- mxvm_io_fseek
 - io.cpp, [549](#)
- mxvm_io_fsize
 - io.cpp, [549](#)
- mxvm_io_fwrite
 - io.cpp, [549](#)
- mxvm_io_mxvm_fgets
 - io.cpp, [550](#)
- mxvm_io_rand_number
 - io.cpp, [550](#)
- mxvm_io_seed_random
 - io.cpp, [550](#)
- mxvm_sdl_blit_surface
 - sdl.cpp, [605](#)
- mxvm_sdl_clear
 - sdl.cpp, [605](#)
- mxvm_sdl_clear_queued_audio
 - sdl.cpp, [605](#)
- mxvm_sdl_close_audio
 - sdl.cpp, [605](#)
- mxvm_sdl_create_render_target
 - sdl.cpp, [605](#)
- mxvm_sdl_create_renderer
 - sdl.cpp, [605](#)
- mxvm_sdl_create_rgb_surface
 - sdl.cpp, [606](#)
- mxvm_sdl_create_texture
 - sdl.cpp, [606](#)
- mxvm_sdl_create_window
 - sdl.cpp, [606](#)
- mxvm_sdl_delay
 - sdl.cpp, [606](#)
- mxvm_sdl_destroy_render_target
 - sdl.cpp, [606](#)
- mxvm_sdl_destroy_renderer
 - sdl.cpp, [606](#)
- mxvm_sdl_destroy_texture
 - sdl.cpp, [606](#)
- mxvm_sdl_destroy_window
 - sdl.cpp, [607](#)
- mxvm_sdl_draw_line
 - sdl.cpp, [607](#)
- mxvm_sdl_draw_point
 - sdl.cpp, [607](#)
- mxvm_sdl_draw_rect
 - sdl.cpp, [607](#)
- mxvm_sdl_draw_text
 - sdl.cpp, [607](#)
- mxvm_sdl_fill_rect
 - sdl.cpp, [607](#)
- mxvm_sdl_free_surface
 - sdl.cpp, [608](#)
- mxvm_sdl_free_wav
 - sdl.cpp, [608](#)
- mxvm_sdl_get_clipboard_text
 - sdl.cpp, [608](#)
- mxvm_sdl_get_event_type
 - sdl.cpp, [608](#)
- mxvm_sdl_get_key_code
 - sdl.cpp, [608](#)
- mxvm_sdl_get_keyboard_state
 - sdl.cpp, [608](#)
- mxvm_sdl_get_mouse_button

`sdl.cpp`, 608
`mxvm_sdl_get_mouse_buttons`
 `sdl.cpp`, 609
`mxvm_sdl_get_mouse_state`
 `sdl.cpp`, 609
`mxvm_sdl_get_mouse_x`
 `sdl.cpp`, 609
`mxvm_sdl_get_mouse_y`
 `sdl.cpp`, 609
`mxvm_sdl_get_num_keys`
 `sdl.cpp`, 609
`mxvm_sdl_get_queued_audio_size`
 `sdl.cpp`, 609
`mxvm_sdl_get_relative_mouse_buttons`
 `sdl.cpp`, 609
`mxvm_sdl_get_relative_mouse_state`
 `sdl.cpp`, 610
`mxvm_sdl_get_relative_mouse_x`
 `sdl.cpp`, 610
`mxvm_sdl_get_relative_mouse_y`
 `sdl.cpp`, 610
`mxvm_sdl_get_renderer_output_size`
 `sdl.cpp`, 610
`mxvm_sdl_get_ticks`
 `sdl.cpp`, 610
`mxvm_sdl_get_window_size`
 `sdl.cpp`, 610
`mxvm_sdl_init`
 `sdl.cpp`, 610
`mxvm_sdl_init_text`
 `sdl.cpp`, 611
`mxvm_sdl_is_key_pressed`
 `sdl.cpp`, 611
`mxvm_sdl_load_font`
 `sdl.cpp`, 611
`mxvm_sdl_load_texture`
 `sdl.cpp`, 611
`mxvm_sdl_load_texture_color_key`
 `sdl.cpp`, 611
`mxvm_sdl_load_texture_color_key_rgb`
 `sdl.cpp`, 611
`mxvm_sdl_load_wav`
 `sdl.cpp`, 612
`mxvm_sdl_lock_texture`
 `sdl.cpp`, 612
`mxvm_sdl_open_audio`
 `sdl.cpp`, 612
`mxvm_sdl_pause_audio`
 `sdl.cpp`, 612
`mxvm_sdl_poll_event`
 `sdl.cpp`, 612
`mxvm_sdl_present`
 `sdl.cpp`, 613
`mxvm_sdl_present_scaled`
 `sdl.cpp`, 613
`mxvm_sdl_present_stretched`
 `sdl.cpp`, 613
`mxvm_sdl_queue_audio`
 `sdl.cpp`, 613
`mxvm_sdl_quit`
 `sdl.cpp`, 613
`mxvm_sdl_quit_text`
 `sdl.cpp`, 613
`mxvm_sdl_render_texture`
 `sdl.cpp`, 613
`mxvm_sdl_set_clipboard_text`
 `sdl.cpp`, 614
`mxvm_sdl_set_draw_color`
 `sdl.cpp`, 614
`mxvm_sdl_set_render_target`
 `sdl.cpp`, 614
`mxvm_sdl_set_window_fullscreen`
 `sdl.cpp`, 614
`mxvm_sdl_set_window_icon`
 `sdl.cpp`, 614
`mxvm_sdl_set_window_position`
 `sdl.cpp`, 614
`mxvm_sdl_set_window_title`
 `sdl.cpp`, 614
`mxvm_sdl_show_cursor`
 `sdl.cpp`, 615
`mxvm_sdl_unlock_texture`
 `sdl.cpp`, 615
`mxvm_sdl_update_texture`
 `sdl.cpp`, 615
`mxvm_std_abs`
 `std.cpp`, 636
`mxvm_std_acos`
 `std.cpp`, 636
`mxvm_std_argc`
 `std.cpp`, 637
`mxvm_std_argv`
 `std.cpp`, 637
`mxvm_std_asin`
 `std.cpp`, 637
`mxvm_std_atan`
 `std.cpp`, 637
`mxvm_std_atan2`
 `std.cpp`, 637
`mxvm_std_atof`
 `std.cpp`, 637
`mxvm_std_atoi`
 `std.cpp`, 638
`mxvm_std_calloc`
 `std.cpp`, 638
`mxvm_std_ceil`
 `std.cpp`, 638
`mxvm_std_cos`
 `std.cpp`, 638
`mxvm_std_cosh`
 `std.cpp`, 638
`mxvm_std_exit`
 `std.cpp`, 638
`mxvm_std_exp`
 `std.cpp`, 638
`mxvm_std_exp2`

- std.cpp, [639](#)
- mxvm_std_fabs
 - std.cpp, [639](#)
- mxvm_std_float_to_int
 - std.cpp, [639](#)
- mxvm_std_floor
 - std.cpp, [639](#)
- mxvm_std_fmod
 - std.cpp, [639](#)
- mxvm_std_free_program_args
 - std.cpp, [639](#)
- mxvm_std_hypot
 - std.cpp, [640](#)
- mxvm_std_int_to_float
 - std.cpp, [640](#)
- mxvm_std_isalpha
 - std.cpp, [640](#)
- mxvm_std_isdigit
 - std.cpp, [640](#)
- mxvm_std_isspace
 - std.cpp, [640](#)
- mxvm_std_log
 - std.cpp, [640](#)
- mxvm_std_log10
 - std.cpp, [641](#)
- mxvm_std_log2
 - std.cpp, [641](#)
- mxvm_std_malloc
 - std.cpp, [641](#)
- mxvm_std_memcmp
 - std.cpp, [641](#)
- mxvm_std_memcpy
 - std.cpp, [641](#)
- mxvm_std_memmove
 - std.cpp, [641](#)
- mxvm_std_memset
 - std.cpp, [641](#)
- mxvm_std_pow
 - std.cpp, [642](#)
- mxvm_std_rand
 - std.cpp, [642](#)
- mxvm_std_release
 - std.cpp, [642](#)
- mxvm_std_round
 - std.cpp, [642](#)
- mxvm_std_set_program_args
 - std.cpp, [642](#)
- mxvm_std_sin
 - std.cpp, [642](#)
- mxvm_std_sinh
 - std.cpp, [643](#)
- mxvm_std_sqrt
 - std.cpp, [643](#)
- mxvm_std_srand
 - std.cpp, [643](#)
- mxvm_std_system
 - std.cpp, [643](#)
- mxvm_std_tan
 - std.cpp, [643](#)
- mxvm_std_tanh
 - std.cpp, [643](#)
- mxvm_std_tolower
 - std.cpp, [644](#)
- mxvm_std_toupper
 - std.cpp, [644](#)
- mxvm_std_trunc
 - std.cpp, [644](#)
- mxvm_string_copy
 - string.cpp, [660](#)
- mxvm_string_delete
 - string.cpp, [660](#)
- mxvm_string_insert
 - string.cpp, [661](#)
- mxvm_string_inttostr
 - string.cpp, [661](#)
- mxvm_string_pos
 - string.cpp, [661](#)
- mxvm_string_sprintf
 - string.cpp, [661](#)
- mxvm_string_strat
 - string.cpp, [661](#)
- mxvm_string_strcmp
 - string.cpp, [662](#)
- mxvm_string_strfind
 - string.cpp, [662](#)
- mxvm_string_strlen
 - string.cpp, [662](#)
- mxvm_string_strncat
 - string.cpp, [662](#)
- mxvm_string_strncpy
 - string.cpp, [662](#)
- mxvm_string_strtoint
 - string.cpp, [662](#)
- mxvm_string_substr
 - string.cpp, [663](#)
- mxvmOpt
 - pascal, [98](#)
- mxx, [90](#)
 - AND, [91](#)
 - ARRAY, [91](#)
 - BEGIN, [91](#)
 - CASE, [91](#)
 - ciEquals, [92](#)
 - CONST, [91](#)
 - DIV, [91](#)
 - DO, [91](#)
 - DOWNTO, [91](#)
 - ELSE, [91](#)
 - END, [91](#)
 - FILE, [91](#)
 - FOR, [91](#)
 - FUNCTION, [92](#)
 - GOTO, [92](#)
 - IF, [92](#)
 - IN, [92](#)
 - key_str, [93](#)

- Keywords, 91
- LABEL, 92
- MOD, 92
- NIL, 92
- NOT, 92
- OF, 92
- OR, 92
- PACKED, 92
- pascal_keywords, 93
- PROCEDURE, 92
- PROGRAM, 92
- RECORD, 92
- REPEAT, 92
- SET, 92
- THEN, 92
- TO, 92
- TYPE, 92
- UNTIL, 92
- VAR, 92
- WHILE, 92
- WITH, 92
- mx::Scope, 387
 - consts, 387
 - funcs, 387
 - params, 387
 - procs, 387
 - types, 387
 - vars, 388
- mx::TPValidator, 414
 - checkConstOnly, 419
 - checkType, 419
 - checkVar, 419
 - checkVarOrConst, 419
 - currentRecordTypeName, 432
 - currentScope, 420
 - currentScopeName, 420
 - declareConst, 420
 - declaredConsts, 432
 - declaredFuncs, 432
 - declaredProcs, 432
 - declaredTypes, 432
 - declaredVars, 432
 - declareFunc, 420
 - declareParam, 420
 - declareProc, 420
 - declareRecordField, 421
 - declareType, 421
 - declareVar, 421
 - fail, 421
 - failAt, 421
 - failHere, 422
 - filename, 433
 - found, 422
 - importedUnits, 433
 - index, 433
 - inFunctionDepth, 433
 - inRecordFieldScope, 422
 - isAddOp, 422
 - isBuiltinConst, 422
 - isBuiltinType, 422
 - isBuiltinTypeName, 422
 - isFuncDeclaredHere, 422
 - isKW, 423
 - isMulOp, 423
 - isParamDeclaredHere, 423
 - isPascalKeyword, 423
 - isProcDeclaredHere, 423
 - isRelOp, 423
 - isSetOp, 424
 - isTypeDeclaredHere, 424
 - isVarDeclaredHere, 424
 - isVariableInCurrentScope, 424
 - lower, 424
 - match, 424, 425
 - next, 425
 - parseActualParams, 425
 - parseBlock, 425
 - parseCase, 425
 - parseCaseLabel, 425
 - parseCaseLabelList, 425
 - parseCompoundStatement, 426
 - parseConstant, 426
 - parseConstExpr, 426
 - parseConstSection, 426
 - parseConstSimple, 426
 - parseDesignator, 426
 - parseExprStop, 427
 - parseFieldIdentList, 427
 - parseFor, 427
 - parseFormalParams, 427
 - parseGoto, 427
 - parseIdentList, 427
 - parseIf, 427
 - parseInterfaceSection, 428
 - parseLabelSection, 428
 - parseProgram, 428
 - parseRepeat, 428
 - parseSimpleOrCallOrAssign, 428
 - parseStatement, 428
 - parseSubprogram, 428
 - parseSubrange, 429
 - parseType, 429
 - parseTypeName, 429
 - parseTypeSection, 429
 - parseUnit, 429
 - parseUses, 429
 - parseVarSection, 430
 - parseWhile, 430
 - parseWith, 430
 - peekIs, 430
 - popRecordFieldScope, 430
 - popScope, 430
 - predeclareType, 430
 - pushRecordFieldScope, 430
 - pushScope, 431
 - recordFieldScopesByType, 433

- require, [431](#)
- requireKW, [431](#)
- scanner, [433](#)
- scopeStack, [433](#)
- source, [433](#)
- token, [434](#)
- tokenTypeToString, [431](#)
- toLower, [431](#)
- TPValidator, [419](#)
- validate, [432](#)
- withDepth, [434](#)
- mxx::XParser, [468](#)
 - filename, [471](#)
 - index, [471](#)
 - next, [470](#)
 - peekIs, [470](#)
 - scanner, [471](#)
 - token, [471](#)
 - tokenTypeToString, [470](#)
 - XParser, [469](#)
- name
 - mxvm::Base, [158](#)
 - mxvm::ExternalFunction, [234](#)
 - mxvm::LabelNode, [259](#)
 - mxvm::ModuleNode, [263](#)
 - mxvm::ObjectNode, [272](#)
 - mxvm::OpSpec, [274](#)
 - mxvm::Program, [365](#)
 - mxvm::ProgramNode, [368](#)
 - mxvm::UseLabel, [442](#)
 - mxvm::UseVar, [443](#)
 - mxvm::VariableNode, [461](#)
 - pascal::ArrayDeclarationNode, [131](#)
 - pascal::ArrayTypeDeclarationNode, [135](#)
 - pascal::CodeGenVisitor, [209](#)
 - pascal::CodeGenVisitor::RecordField, [374](#)
 - pascal::CodeGenVisitor::Scope, [388](#)
 - pascal::FuncCallNode, [241](#)
 - pascal::FuncDeclNode, [243](#)
 - pascal::ProcCallNode, [305](#)
 - pascal::ProcDeclNode, [307](#)
 - pascal::ProgramNode, [370](#)
 - pascal::RecordDeclarationNode, [372](#)
 - pascal::UnitNode, [442](#)
 - pascal::VariableNode, [463](#)
- nameToIndex
 - pascal::CodeGenVisitor::RecordTypeInfo, [375](#)
- needsEmptyString
 - pascal::CodeGenVisitor, [209](#)
- NEG
 - instruct.hpp, [529](#)
- NEQ
 - expr_parser, [76](#)
- nestedFuncs
 - pascal::CodeGenVisitor::Scope, [388](#)
- nestedProcs
 - pascal::CodeGenVisitor::Scope, [388](#)
- newLabel
 - pascal::CodeGenVisitor, [192](#)
- newSlotFor
 - pascal::CodeGenVisitor, [192](#)
- newTemp
 - pascal::CodeGenVisitor, [192](#)
- next
 - expr_parser::ExprLexer, [229](#)
 - mxvm::ModuleParser, [265](#)
 - mxvm::Validator, [445](#)
 - mxx::TPValidator, [425](#)
 - mxx::XParser, [470](#)
- nextFloatRegIndex
 - pascal::CodeGenVisitor, [209](#)
- nextNonEmpty
 - pascal, [98](#)
- nextSlot
 - pascal::CodeGenVisitor, [209](#)
- nextSpillReg
 - pascal::CodeGenVisitor, [209](#)
- nextTemp
 - pascal::CodeGenVisitor, [210](#)
- NIL
 - mxx, [92](#)
- NilNode
 - pascal::NilNode, [268](#)
- None
 - mxvm, [86](#)
- NOT
 - expr_parser, [76](#)
 - instruct.hpp, [528](#)
 - mxx, [92](#)
 - pascal::UnaryOpNode, [439](#)
- NOT_EQUAL
 - pascal::BinaryOpNode, [161](#)
- null_action
 - main.cpp, [784](#)
- NULL_INC
 - instruct.hpp, [528](#)
- Num
 - mxvm, [85](#)
- NUMBER
 - expr_parser, [75](#)
- NumberNode
 - pascal::NumberNode, [269](#)
- obj_name
 - mxvm::Variable, [453](#)
- OBJECT
 - mxvm::SectionNode, [391](#)
- object
 - mxvm::Operand, [272](#)
 - mxvm::Program, [365](#)
 - mxvm::ProgramNode, [368](#)
 - mxvm::VariableNode, [462](#)
- object_external
 - mxvm::Program, [365](#)
- object_map
 - mxvm::Base, [159](#)
- object_mode

- mxvm::Parser, 285
- object_name
 - mxvm::Parser, 285
- object_path
 - Args, 114
 - mxvm::Parser, 285
- objectDeps
 - pascal::CodeGenVisitor, 210
- ObjectNode
 - mxvm::ObjectNode, 271
- objects
 - mxvm::Program, 365
- OF
 - mxx, 92
- offset
 - pascal::CodeGenVisitor::RecordField, 374
- only_test
 - Args, 114
- op
 - mxvm::Operand, 272
- op1
 - mxvm::Instruction, 253
- op2
 - mxvm::Instruction, 254
- op3
 - mxvm::Instruction, 254
- OP_AND
 - types, 105
- OP_AND_AND
 - types, 105
- OP_AND_ASSIGN
 - types, 104
- OP_ARROW
 - types, 105
- OP_ASSIGN
 - types, 105
- OP_AT
 - types, 105
- OP_COLON
 - types, 105
- OP_COMMA
 - types, 105
- OP_CONSTANT
 - mxvm, 85
- OP_DEC
 - types, 104
- OP_DIV
 - types, 105
- OP_DIV_ASSIGN
 - types, 104
- OP_DOLLAR
 - types, 105
- OP_DOT
 - types, 105
- OP_EQ
 - types, 105
- OP_GE
 - types, 105
- OP_GT
 - types, 105
- OP_HASH
 - types, 105
- OP_INC
 - types, 104
- OP_LBRACE
 - types, 105
- OP_LBRACKET
 - types, 105
- OP_LE
 - types, 105
- OP_LPAREN
 - types, 105
- OP_LSHIFT
 - types, 104
- OP_LSHIFT_ASSIGN
 - types, 104
- OP_LT
 - types, 105
- OP_MINUS
 - types, 105
- OP_MINUS_ASSIGN
 - types, 104
- OP_MOD
 - types, 105
- OP_MOD_ASSIGN
 - types, 104
- OP_MUL
 - types, 105
- OP_MUL_ASSIGN
 - types, 104
- OP_NEQ
 - types, 105
- OP_NOT
 - types, 105
- OP_OR
 - types, 105
- OP_OR_ASSIGN
 - types, 104
- OP_OR_OR
 - types, 105
- OP_PLUS
 - types, 105
- OP_PLUS_ASSIGN
 - types, 104
- OP_QUESTION
 - types, 105
- OP_RBRACE
 - types, 105
- OP_RBRACKET
 - types, 105
- OP_LPAREN
 - types, 105
- OP_RSHIFT
 - types, 104
- OP_RSHIFT_ASSIGN
 - types, 104

- OP_SCOPE
 - types, [105](#)
- OP_SEMICOLON
 - types, [105](#)
- OP_TILDE
 - types, [105](#)
- op_value
 - mxvm::Operand, [273](#)
- OP_VARIABLE
 - mxvm, [85](#)
- OP_XOR
 - types, [105](#)
- OP_XOR_ASSIGN
 - types, [104](#)
- open_audio
 - mx_sdl.h, [569](#)
 - sdl.c, [589](#)
- operand
 - pascal::AddressOfNode, [112](#)
 - pascal::UnaryOpNode, [439](#)
- operands
 - mxvm::InstructionNode, [256](#)
- OperandType
 - mxvm, [85](#)
- Operator
 - pascal::UnaryOpNode, [439](#)
- operator<<
 - instruct.cpp, [922](#), [923](#)
 - instruct.hpp, [529](#), [530](#)
 - main.cpp, [786](#)
 - scanner.hpp, [933](#)
- operator<=>
 - mx::Argument< String >, [116](#)
- operator()
 - mxvm::Function< R(Args...)>, [247](#)
- operator=
 - mx::Argument< String >, [117](#)
 - mx::ArgumentData< String >, [119](#)
 - mx::Argz< String >, [124](#), [125](#)
 - mxvm::Base, [157](#)
 - mxvm::Function< R(Args...)>, [247](#)
 - mxvm::ModuleParser, [265](#)
 - mxvm::Parser, [281](#)
 - mxvm::Program, [345](#)
 - mxvm::RuntimeFunction, [381](#)
 - mxvm::Variable, [452](#)
 - mxvm::Variable_Value, [456](#)
 - pascal::ArrayInfo, [133](#)
 - pascal::CodeGenVisitor::RecordField, [373](#)
 - scan::token::Token< Ch, String >, [411](#)
- operator==
 - mxvm::ExternalFunction, [233](#)
- operator[]
 - mxvm::ModuleParser, [266](#)
 - mxvm::Parser, [281](#)
 - mxvm::Stack, [397](#)
 - scan::Scanner, [385](#)
- operator_
 - pascal::BinaryOpNode, [162](#)
 - pascal::UnaryOpNode, [440](#)
- OperatorType
 - types, [104](#)
- OpKind
 - mxvm, [85](#)
- opName
 - types, [107](#)
- opStrings
 - types, [107](#)
- opt_core_lines
 - mxvm, [88](#)
- opt_darwin_lines
 - mxvm, [88](#)
- opt_linux_lines
 - mxvm, [88](#)
- opt_x64_windows_lines
 - mxvm, [88](#)
- optimize_core
 - mxvm::Program, [345](#)
- optimize_darwin
 - mxvm::Program, [345](#)
- opToString
 - pascal::BinaryOpNode, [162](#)
 - pascal::UnaryOpNode, [439](#)
- OpType
 - pascal::BinaryOpNode, [161](#)
- OR
 - expr_parser, [76](#)
 - instruct.hpp, [528](#)
 - mxx, [92](#)
 - pascal::BinaryOpNode, [161](#)
- output
 - mxvm::HTMLGen, [250](#)
 - mxvm::PasHTMLGen, [300](#)
- output_file
 - Args, [115](#)
- owns
 - mxvm::Variable_Value, [458](#)
- PACKED
 - mxx, [92](#)
- ParameterNode
 - pascal::ParameterNode, [276](#)
- parameters
 - pascal::FuncDeclNode, [243](#)
 - pascal::ProcDeclNode, [307](#)
- params
 - mxx::Scope, [387](#)
- paramTypes
 - pascal::CodeGenVisitor::FuncInfo, [244](#)
- parent
 - mxvm::Program, [365](#)
- parse
 - expr_parser::ExprParser, [231](#)
 - mxvm::ModuleParser, [266](#)
 - mxvm::Parser, [281](#)
- parse1
 - pascal, [99](#)

- parse2
 - pascal, [99](#)
- parseActualParams
 - mxx::TPValidator, [425](#)
- parseAdd
 - expr_parser::ExprParser, [231](#)
- parseArgumentList
 - pascal::PascalParser, [293](#)
- parseArrayDeclaration
 - pascal::PascalParser, [293](#)
- parseArrayType
 - pascal::PascalParser, [293](#)
- parseAssignmentOrProcCall
 - pascal::PascalParser, [293](#)
- parseAST
 - mxvm::Parser, [281](#)
- parseBitwise
 - expr_parser::ExprParser, [231](#)
- parseBlock
 - mxx::TPValidator, [425](#)
 - pascal::PascalParser, [293](#)
- parseCase
 - mxx::TPValidator, [425](#)
- parseCaseLabel
 - mxx::TPValidator, [425](#)
- parseCaseLabelList
 - mxx::TPValidator, [425](#)
- parseCaseStatement
 - pascal::PascalParser, [293](#)
- parseCodeInstruction
 - mxvm::Parser, [281](#)
- parseComment
 - mxvm::Parser, [282](#)
- parseComparison
 - expr_parser::ExprParser, [231](#)
- parseCompoundStatement
 - mxx::TPValidator, [426](#)
 - pascal::PascalParser, [293](#)
- parseConstant
 - mxx::TPValidator, [426](#)
- parseConstDeclaration
 - pascal::PascalParser, [294](#)
- parseConstExpr
 - mxx::TPValidator, [426](#)
- parseConstSection
 - mxx::TPValidator, [426](#)
- parseConstSimple
 - mxx::TPValidator, [426](#)
- parseDataVariable
 - mxvm::Parser, [282](#)
- parseDeclarations
 - pascal::PascalParser, [294](#)
- parseDesignator
 - mxx::TPValidator, [426](#)
- ParseException
 - pascal::ParseException, [278](#)
- parseExpression
 - pascal::PascalParser, [294](#)
- parseExprStop
 - mxx::TPValidator, [427](#)
- parseFactor
 - expr_parser::ExprParser, [231](#)
 - pascal::PascalParser, [294](#)
- parseFieldIdentList
 - mxx::TPValidator, [427](#)
- parseFileName
 - scan::Scanner, [385](#)
- parseFor
 - mxx::TPValidator, [427](#)
- parseFormalParams
 - mxx::TPValidator, [427](#)
- parseForStatement
 - pascal::PascalParser, [294](#)
- parseFunctionCall
 - pascal::PascalParser, [294](#)
- parseFunctionDeclaration
 - pascal::PascalParser, [295](#)
- parseFunctionForwardDecl
 - pascal::PascalParser, [295](#)
- parseGoto
 - mxx::TPValidator, [427](#)
- parseGotoStatement
 - pascal::PascalParser, [295](#)
- parseIdentList
 - mxx::TPValidator, [427](#)
- parseIf
 - mxx::TPValidator, [427](#)
- parseIfStatement
 - pascal::PascalParser, [295](#)
- parseInterfaceDeclarations
 - pascal::PascalParser, [295](#)
- parseInterfaceSection
 - mxx::TPValidator, [428](#)
- parseJump
 - pascal, [99](#)
- parseLabel
 - mxvm::Parser, [282](#)
- parseLabelDeclaration
 - pascal::PascalParser, [295](#)
- parseLabelSection
 - mxx::TPValidator, [428](#)
- parseLineNumber
 - scan::Scanner, [385](#)
- parseLogical
 - expr_parser::ExprParser, [232](#)
- parseLValue
 - pascal::PascalParser, [296](#)
- parseModule
 - mxvm::Parser, [282](#)
- parseObject
 - mxvm::Parser, [282](#)
- parseOperand
 - mxvm::Validator, [445](#)
- parseOperandList
 - mxvm::Validator, [445](#)
- parseParameter

- pascal::PascalParser, 296
- parseParameterList
 - pascal::PascalParser, 296
- parseProcedureCall
 - pascal::PascalParser, 296
- parseProcedureDeclaration
 - pascal::PascalParser, 296
- parseProcedureForwardDecl
 - pascal::PascalParser, 296
- parseProgram
 - mxx::TPValidator, 428
 - pascal::PascalParser, 296
- parseProgramOrObject
 - mxvm::Parser, 282
- Parser
 - mxvm::Parser, 280
 - mxvm::Program, 363
- parser_mode
 - mxvm::ModuleParser, 267
 - mxvm::Parser, 285
- parseRecordType
 - pascal::PascalParser, 297
- parseRepeat
 - mxx::TPValidator, 428
- parseRepeatStatement
 - pascal::PascalParser, 297
- parseSection
 - mxvm::Parser, 283
- parseSimpleExpression
 - pascal::PascalParser, 297
- parseSimpleOrCallOrAssign
 - mxx::TPValidator, 428
- parseStatement
 - mxx::TPValidator, 428
 - pascal::PascalParser, 297
- parseStatementList
 - pascal::PascalParser, 297
- parseSubprogram
 - mxx::TPValidator, 428
- parseSubrange
 - mxx::TPValidator, 429
- parseTerm
 - expr_parser::ExprParser, 232
 - pascal::PascalParser, 297
- parseType
 - mxx::TPValidator, 429
- parseTypeDeclaration
 - pascal::PascalParser, 298
- parseTypeName
 - mxx::TPValidator, 429
- parseTypeSection
 - mxx::TPValidator, 429
- parseTypeSpec
 - pascal::PascalParser, 298
- parseUnit
 - mxx::TPValidator, 429
 - pascal::PascalParser, 298
- parseUses
 - mxx::TPValidator, 429
- parseVarDeclaration
 - pascal::PascalParser, 298
- parseVarSection
 - mxx::TPValidator, 430
- parseWhile
 - mxx::TPValidator, 430
- parseWhileStatement
 - pascal::PascalParser, 298
- parseWith
 - mxx::TPValidator, 430
- parseWithStatement
 - pascal::PascalParser, 298
- pascal, 93
 - ARRAY_DOUBLE, 97
 - ARRAY_INT, 97
 - ARRAY_STRING, 97
 - BOOL, 97
 - CHAR, 97
 - collect_used, 97
 - copyPropagation, 97
 - DOUBLE, 96
 - fileFunctions, 100
 - foldAndTest, 97
 - foldCmpTest, 97
 - foldMovArithMov, 97
 - foldOrTest, 98
 - INT, 96
 - invertJump, 98
 - isLabelDef, 98
 - isRegisterName, 98
 - locate_sections, 98
 - mxvmOpt, 98
 - nextNonEmpty, 98
 - parse1, 99
 - parse2, 99
 - parseJump, 99
 - peepholeOpt, 99
 - PTR, 97
 - RECORD, 97
 - rtrim_comment, 99
 - sdlFunctions, 100
 - stdFunctions, 100
 - STRING, 96
 - sweep_unused_data, 99
 - toLower, 99
 - trim, 100
 - UNKNOWN, 97
 - VarType, 96
- pascal::AddressOfNode, 111
 - accept, 112
 - AddressOfNode, 112
 - operand, 112
 - toString, 112
- pascal::ArrayAccessNode, 126
 - accept, 127
 - ArrayAccessNode, 127
 - base, 127

- index, 127
- toString, 127
- pascal::ArrayAssignmentNode, 128
 - accept, 129
 - ArrayAssignmentNode, 129
 - arrayName, 129
 - index, 129
 - toString, 129
 - value, 129
- pascal::ArrayDeclarationNode, 130
 - accept, 131
 - ArrayDeclarationNode, 131
 - arrayType, 131
 - initializers, 131
 - name, 131
 - toString, 131
- pascal::ArrayInfo, 132
 - ArrayInfo, 132
 - elementArray, 133
 - elementIsArray, 133
 - elementSize, 133
 - elementType, 133
 - isDynamic, 133
 - lowerBound, 133
 - operator=, 133
 - size, 134
 - upperBound, 134
- pascal::ArrayTypeDeclarationNode, 134
 - accept, 135
 - arrayType, 135
 - ArrayTypeDeclarationNode, 135
 - name, 135
 - toString, 135
- pascal::ArrayTypeNode, 136
 - accept, 137
 - ArrayTypeNode, 137
 - elementType, 137
 - lowerBound, 137
 - toString, 137
 - upperBound, 137
- pascal::AssignmentNode, 138
 - accept, 139
 - AssignmentNode, 139
 - expression, 139
 - toString, 139
 - variable, 139
- pascal::ASTNode, 141
 - ~ASTNode, 143
 - accept, 143
 - getLineNumber, 143
 - line_number, 144
 - print, 143
 - setLineNumber, 143
 - toString, 144
- pascal::ASTVisitor, 146
 - ~ASTVisitor, 147
 - visit, 147–152
- pascal::BinaryOpNode, 159
 - accept, 162
 - AND, 161
 - BinaryOpNode, 161
 - DIV, 161
 - DIVIDE, 161
 - EQUAL, 161
 - GREATER, 161
 - GREATER_EQUAL, 161
 - IN, 161
 - left, 162
 - LESS, 161
 - LESS_EQUAL, 161
 - MINUS, 161
 - MOD, 161
 - MULTIPLY, 161
 - NOT_EQUAL, 161
 - operator_, 162
 - opToString, 162
 - OpType, 161
 - OR, 161
 - PLUS, 161
 - right, 162
 - toString, 162
- pascal::BlockNode, 162
 - accept, 164
 - BlockNode, 163
 - compoundStatement, 164
 - declarations, 164
 - toString, 164
- pascal::BooleanNode, 164
 - accept, 166
 - BooleanNode, 165
 - toString, 166
 - value, 166
- pascal::BreakNode, 166
 - accept, 167
 - BreakNode, 167
 - expr, 168
 - toString, 167
- pascal::BuiltinFunctionHandler, 168
 - ~BuiltinFunctionHandler, 169
 - canHandle, 169
 - generate, 169
 - generateWithResult, 169
 - getReturnType, 169
- pascal::BuiltinFunctionRegistry, 169
 - findHandler, 170
 - handlers, 170
 - registerHandler, 170
- pascal::CaseStmtNode, 171
 - accept, 172
 - branches, 173
 - CaseStmtNode, 172
 - elseStatement, 173
 - expression, 173
 - toString, 172
- pascal::CaseStmtNode::CaseBranch, 170
 - CaseBranch, 171

- statement, [171](#)
- values, [171](#)
- pascal::CodeGenVisitor, [173](#)
 - ~CodeGenVisitor, [179](#)
 - addTempPtr, [179](#)
 - allocatedPtrs, [205](#)
 - allocateRecordFieldArrays, [179](#)
 - allocFloatReg, [180](#)
 - allocPtrReg, [180](#)
 - allocReg, [180](#)
 - allocTempPtr, [180](#)
 - allTempPtrs, [205](#)
 - arrayInfo, [205](#)
 - bufferStringVars, [205](#)
 - buildArrayInfoFromNode, [180](#)
 - builtinRegistry, [205](#)
 - CodeGenVisitor, [179](#)
 - coerceToIntImmediate, [180](#)
 - compileTimeConstants, [205](#)
 - constInitialValues, [205](#)
 - convertCharLiteral, [181](#)
 - currentFuncLocalSlots, [205](#)
 - currentFunctionName, [205](#)
 - currentFunctionReturnSlot, [206](#)
 - currentParamLocations, [206](#)
 - currentParamTypes, [206](#)
 - declaredFuncs, [206](#)
 - declaredProcs, [206](#)
 - deferredFuncs, [206](#)
 - deferredProcs, [206](#)
 - dynArrayLenSlot, [206](#)
 - emit, [181](#)
 - emit1, [181](#)
 - emit2, [181](#)
 - emit3, [181](#)
 - emit4, [181](#)
 - emit_invoke, [182](#)
 - emitArrayBoundsCheck, [182](#)
 - emitDynArrayBoundsCheck, [182](#)
 - emitFree, [183](#)
 - emitLabel, [183](#)
 - emptyString, [183](#)
 - endsWithColon, [183](#)
 - ensureFloatConstSymbol, [183](#)
 - ensurePtrBase, [183](#)
 - enumConstants, [206](#)
 - enumTypes, [207](#)
 - escapedTempPtrs, [207](#)
 - escapeStringForMxvm, [184](#)
 - eval, [184](#)
 - evalStack, [207](#)
 - evaluateConstantExpression, [184](#)
 - expandFloatRegisters, [184](#)
 - externalFuncs, [207](#)
 - FileFunctionHandler, [204](#)
 - fileVarNames, [207](#)
 - fileVars, [207](#)
 - findMangledArrayName, [184](#)
 - findMangledFuncName, [184](#)
 - findMangledName, [184](#)
 - floatRegInUse, [207](#)
 - floatRegisters, [207](#)
 - foldNumeric, [185](#)
 - freeFloatReg, [185](#)
 - freePtrReg, [185](#)
 - freeReg, [185](#)
 - freeTempPtr, [185](#)
 - funcLabel, [185](#)
 - funcSignatures, [207](#)
 - functionScopedArrays, [208](#)
 - functionSetReturn, [208](#)
 - generate, [185](#)
 - generateRealConstantName, [186](#)
 - generatingDeferredCode, [208](#)
 - getArrayElementSize, [186](#)
 - getArrayInfoForArrayAccess, [186](#)
 - getArrayNameFromBase, [186](#)
 - getCurrentEndLabel, [186](#)
 - getCurrentScopeName, [187](#)
 - getExpressionType, [187](#)
 - getPointerBaseTypeName, [187](#)
 - getPointerDerefType, [187](#)
 - getPointerElementSize, [187](#)
 - getRecordFieldOffsetAndSize, [187](#)
 - getRecordTypeSize, [187](#)
 - getTypeFromString, [188](#)
 - getTypeSizeByName, [188](#)
 - getTypeString, [188](#)
 - getVarRecordTypeName, [188](#)
 - getVarRecordTypeNameFromExpr, [188](#)
 - getVarType, [188](#)
 - globalArrays, [208](#)
 - gotoLabels, [208](#)
 - importedArrayInfo, [208](#)
 - importedUnitNames, [208](#)
 - importedVarTypes, [208](#)
 - initializeBuiltins, [189](#)
 - initializeFloatRegisters, [189](#)
 - instructions, [208](#)
 - internString, [189](#)
 - IOFunctionHandler, [204](#)
 - isArrayTypeName, [189](#)
 - isFloatLiteral, [189](#)
 - isFloatReg, [189](#)
 - isImportedUnit, [189](#)
 - isIntegerLiteral, [190](#)
 - isIntLiteral, [190](#)
 - isParmReg, [190](#)
 - isPtrLike, [190](#)
 - isPtrReg, [190](#)
 - isRealConstSymbol, [190](#)
 - isRealNumber, [190](#)
 - isRecordTypeName, [190](#)
 - isReg, [191](#)
 - isRegisterSlot, [191](#)
 - isStringVar, [191](#)

- isTempPtr, [191](#)
- isTempVar, [191](#)
- isUnit, [209](#)
- labelCounter, [209](#)
- lc, [191](#)
- loopContinueLabels, [209](#)
- loopEndLabels, [209](#)
- mangleVariableName, [191](#)
- mangleWithScope, [192](#)
- markAllocatedPtr, [192](#)
- MathFunctionHandler, [204](#)
- name, [209](#)
- needsEmptyString, [209](#)
- newLabel, [192](#)
- newSlotFor, [192](#)
- newTemp, [192](#)
- nextFloatRegIndex, [209](#)
- nextSlot, [209](#)
- nextSpillReg, [209](#)
- nextTemp, [210](#)
- objectDeps, [210](#)
- pointerBaseType, [210](#)
- popValue, [192](#)
- prolog, [210](#)
- ptrRegInUse, [210](#)
- ptrRegisters, [210](#)
- pushCmpResult, [192](#)
- pushFloatCmpResult, [193](#)
- pushLogicalAnd, [193](#)
- pushLogicalOr, [193](#)
- pushTri, [193](#)
- pushValue, [193](#)
- realConstantCounter, [210](#)
- realConstants, [210](#)
- recordLocation, [193](#)
- recordsToFreeInScope, [210](#)
- recordTypes, [211](#)
- regInUse, [211](#)
- registerExternalFunc, [194](#)
- registerImportedConst, [194](#)
- registerImportedVar, [194](#)
- registers, [211](#)
- releaseTempPtr, [194](#)
- resolveTypeName, [194](#)
- scopeHierarchy, [211](#)
- scopeStack, [211](#)
- SDLFunctionHandler, [204](#)
- setSlotType, [195](#)
- setVars, [211](#)
- setVarType, [195](#)
- slotToName, [211](#)
- slotToType, [211](#)
- slotVar, [195](#)
- StdFunctionHandler, [204](#)
- storageSymbolFor, [195](#)
- StringFunctionHandler, [204](#)
- stringLiterals, [211](#)
- tempPtrByScope, [212](#)
- tempPtrInUse, [212](#)
- tryGetConstNumeric, [195](#)
- typeAliases, [212](#)
- updateDataSectionInitialValue, [195](#)
- usedModules, [212](#)
- usedRealConstants, [212](#)
- usedStrings, [212](#)
- valueLocations, [212](#)
- varRecordType, [212](#)
- varSlot, [212](#)
- varTypes, [213](#)
- visit, [195–203](#)
- withFieldScopes, [213](#)
- writeTo, [203](#)
- pascal::CodeGenVisitor::FuncInfo, [244](#)
 - paramTypes, [244](#)
 - returnType, [244](#)
- pascal::CodeGenVisitor::RecordField, [373](#)
 - arrayInfo, [374](#)
 - isArray, [374](#)
 - name, [374](#)
 - offset, [374](#)
 - operator=, [373](#)
 - RecordField, [373](#)
 - size, [374](#)
 - typeName, [374](#)
- pascal::CodeGenVisitor::RecordTypeInfo, [374](#)
 - fields, [375](#)
 - nameToIndex, [375](#)
 - size, [375](#)
- pascal::CodeGenVisitor::Scope, [388](#)
 - name, [388](#)
 - nestedFuncs, [388](#)
 - nestedProcs, [388](#)
- pascal::CodeGenVisitor::ValueLocation, [448](#)
 - IMMEDIATE, [449](#)
 - location, [449](#)
 - MEMORY, [449](#)
 - REGISTER, [449](#)
 - Type, [449](#)
 - type, [449](#)
- pascal::CompoundStmtNode, [214](#)
 - accept, [216](#)
 - CompoundStmtNode, [215](#)
 - statements, [216](#)
 - toString, [216](#)
- pascal::ConstDeclNode, [217](#)
 - accept, [218](#)
 - assignments, [218](#)
 - ConstDeclNode, [218](#)
 - toString, [218](#)
- pascal::ConstDeclNode::ConstAssignment, [216](#)
 - ConstAssignment, [216](#)
 - identifier, [217](#)
 - value, [217](#)
- pascal::ContinueNode, [219](#)
 - accept, [220](#)
 - ContinueNode, [219](#)

- expr, 220
- toString, 220
- pascal::EmptyStmtNode, 220
 - accept, 221
 - EmptyStmtNode, 221
 - toString, 221
- pascal::EnumTypeDeclNode, 221
 - accept, 223
 - EnumTypeDeclNode, 222
 - toString, 223
 - typeName, 223
 - values, 223
- pascal::ExitNode, 225
 - accept, 226
 - ExitNode, 226
 - expr, 226
 - toString, 226
- pascal::FieldAccessNode, 234
 - accept, 235
 - FieldAccessNode, 235
 - fieldName, 236
 - recordExpr, 236
 - toString, 235
- pascal::FileFunctionHandler, 236
 - canHandle, 237
 - generate, 237
 - generateWithResult, 237
 - getReturnType, 237
- pascal::ForStmtNode, 238
 - accept, 239
 - endValue, 239
 - ForStmtNode, 239
 - isDownto, 239
 - startValue, 239
 - statement, 239
 - toString, 239
 - variable, 239
- pascal::FuncCallNode, 240
 - accept, 241
 - arguments, 241
 - FuncCallNode, 241
 - name, 241
 - toString, 241
- pascal::FuncDeclNode, 241
 - accept, 243
 - block, 243
 - FuncDeclNode, 243
 - name, 243
 - parameters, 243
 - returnType, 243
 - toString, 243
- pascal::GotoStmtNode, 247
 - accept, 249
 - GotoStmtNode, 248
 - label, 249
 - toString, 249
- pascal::IfStmtNode, 250
 - accept, 252
 - condition, 252
 - elseStatement, 252
 - IfStmtNode, 251
 - thenStatement, 252
 - toString, 252
- pascal::IOFunctionHandler, 257
 - canHandle, 257
 - generate, 257
 - generateWithResult, 258
- pascal::LabelStmtNode, 260
 - accept, 261
 - label, 261
 - LabelStmtNode, 260
 - statement, 261
 - toString, 261
- pascal::NilNode, 267
 - accept, 268
 - NilNode, 268
 - toString, 268
- pascal::NumberNode, 268
 - accept, 270
 - isInteger, 270
 - isReal, 270
 - NumberNode, 269
 - toString, 270
 - value, 270
- pascal::ParameterNode, 275
 - accept, 276
 - identifiers, 276
 - isVar, 276
 - ParameterNode, 276
 - toString, 276
 - type, 276
- pascal::ParseException, 277
 - ParseException, 278
- pascal::PascalParser, 286
 - declaredLabels, 299
 - error, 290
 - expectToken, 290, 291
 - getAddOp, 291
 - getArithmeticOperator, 291
 - getComparisonOperator, 291
 - getLogicalOperator, 291
 - getMulOp, 291
 - getRelationalOp, 291
 - getUnaryOp, 291
 - isAddOperator, 291
 - isArrayAccess, 291
 - isBuiltinFunction, 292
 - isBuiltinProcedure, 292
 - isKeyword, 292
 - isMulOperator, 292
 - isRelationalOperator, 292
 - isType, 292
 - isUnitSource, 292
 - match, 292, 293
 - parseArgumentList, 293
 - parseArrayDeclaration, 293

- parseArrayType, 293
- parseAssignmentOrProcCall, 293
- parseBlock, 293
- parseCaseStatement, 293
- parseCompoundStatement, 293
- parseConstDeclaration, 294
- parseDeclarations, 294
- parseExpression, 294
- parseFactor, 294
- parseForStatement, 294
- parseFunctionCall, 294
- parseFunctionDeclaration, 295
- parseFunctionForwardDecl, 295
- parseGotoStatement, 295
- parseIfStatement, 295
- parseInterfaceDeclarations, 295
- parseLabelDeclaration, 295
- parseLValue, 296
- parseParameter, 296
- parseParameterList, 296
- parseProcedureCall, 296
- parseProcedureDeclaration, 296
- parseProcedureForwardDecl, 296
- parseProgram, 296
- parseRecordType, 297
- parseRepeatStatement, 297
- parseSimpleExpression, 297
- parseStatement, 297
- parseStatementList, 297
- parseTerm, 297
- parseTypeDeclaration, 298
- parseTypeSpec, 298
- parseUnit, 298
- parseVarDeclaration, 298
- parseWhileStatement, 298
- parseWithStatement, 298
- PascalParser, 290
- removeBraceComments, 298
- tokenTypeToString, 299
- validator, 299
- pascal::PointerDerefNode, 301
 - accept, 302
 - pointer, 302
 - PointerDerefNode, 301
 - toString, 302
- pascal::PointerTypeNode, 302
 - accept, 303
 - baseTypeName, 303
 - PointerTypeNode, 303
 - toString, 303
- pascal::ProcCallNode, 304
 - accept, 305
 - arguments, 305
 - name, 305
 - ProcCallNode, 304
 - toString, 305
- pascal::ProcDeclNode, 305
 - accept, 307
 - block, 307
 - name, 307
 - parameters, 307
 - ProcDeclNode, 306
 - toString, 307
- pascal::ProgramNode, 369
 - accept, 370
 - block, 370
 - name, 370
 - ProgramNode, 370
 - toString, 370
 - uses, 371
- pascal::RecordDeclarationNode, 371
 - accept, 372
 - name, 372
 - RecordDeclarationNode, 372
 - recordType, 372
 - toString, 372
- pascal::RecordTypeNode, 375
 - accept, 377
 - fields, 377
 - RecordTypeNode, 377
 - toString, 377
 - variantArms, 377
 - variantTagName, 377
 - variantTagType, 377
- pascal::RepeatStmtNode, 378
 - accept, 379
 - condition, 379
 - RepeatStmtNode, 378
 - statements, 379
 - toString, 379
- pascal::SDLFunctionHandler, 388
 - canHandle, 389
 - generate, 389
 - generateWithResult, 390
- pascal::SetLiteralNode, 392
 - accept, 393
 - elements, 393
 - SetLiteralNode, 393
 - toString, 393
- pascal::SetTypeNode, 393
 - accept, 395
 - baseTypeName, 395
 - SetTypeNode, 394
 - toString, 395
- pascal::SimpleTypeNode, 395
 - accept, 396
 - SimpleTypeNode, 396
 - toString, 396
 - typeName, 396
- pascal::StdFunctionHandler, 399
 - canHandle, 399
 - generate, 399
 - generateWithResult, 400
- pascal::StringFunctionHandler, 405
 - canHandle, 406
 - generate, 406

- generateWithResult, 406
- getReturnType, 406
- pascal::StringNode, 406
 - accept, 408
 - StringNode, 407
 - toString, 408
 - value, 408
- pascal::TypeAliasNode, 434
 - accept, 435
 - baseType, 435
 - toString, 435
 - TypeAliasNode, 435
 - typeName, 435
- pascal::TypeDeclNode, 436
 - accept, 437
 - toString, 437
 - typeDeclarations, 437
 - TypeDeclNode, 437
- pascal::UnaryOpNode, 437
 - accept, 439
 - MINUS, 439
 - NOT, 439
 - operand, 439
 - Operator, 439
 - operator_, 440
 - opToString, 439
 - PLUS, 439
 - toString, 439
 - UnaryOpNode, 439
- pascal::UnitNode, 440
 - accept, 441
 - implDecls, 441
 - interfaceDecls, 441
 - name, 442
 - toString, 441
 - UnitNode, 441
 - uses, 442
- pascal::VarDeclNode, 449
 - accept, 450
 - identifiers, 451
 - initializers, 451
 - toString, 450
 - type, 451
 - VarDeclNode, 450
- pascal::VariableNode, 462
 - accept, 463
 - name, 463
 - toString, 463
 - VariableNode, 463
- pascal::VariantArm, 463
 - caseLabels, 464
 - fields, 464
- pascal::WhileStmtNode, 465
 - accept, 466
 - condition, 466
 - statement, 466
 - toString, 466
 - WhileStmtNode, 466
- pascal::WithStmtNode, 466
 - accept, 468
 - recordVar, 468
 - statement, 468
 - toString, 468
 - WithStmtNode, 467
- pascal_keywords
 - mxx, 93
- PascalParser
 - pascal::PascalParser, 290
- PasHTMLGen
 - mxvm::PasHTMLGen, 300
- path
 - mx::Arguments, 120
- pause_audio
 - mx_sdl.h, 569
 - sdl.c, 590
- pc
 - mxvm::Program, 365
- peek
 - expr_parser::ExprLexer, 229
- peekch
 - scan::buffer::StringBuffer< Ch, String >, 403
- peekls
 - mxvm::Validator, 446
 - mxx::TPValidator, 430
 - mxx::XParser, 470
- peepholeOpt
 - pascal, 99
- Platform
 - mxvm, 85
- platform
 - Args, 115
 - mxvm::Parser, 285
 - mxvm::Program, 366
- platform_argc
 - Args, 115
- platform_argv
 - Args, 115
- PLUS
 - expr_parser, 75
 - pascal::BinaryOpNode, 161
 - pascal::UnaryOpNode, 439
- PLUSEQ
 - expr_parser, 76
- pointer
 - pascal::PointerDerefNode, 302
- pointerBaseType
 - pascal::CodeGenVisitor, 210
- PointerDerefNode
 - pascal::PointerDerefNode, 301
- PointerTypeNode
 - pascal::PointerTypeNode, 303
- poll_event
 - mx_sdl.h, 570
 - sdl.c, 590
- POP
 - instruct.hpp, 529

- pop
 - mxvm::Stack, [398](#)
- popRecordFieldScope
 - mxx::TPValidator, [430](#)
- popScope
 - mxx::TPValidator, [430](#)
- popValue
 - pascal::CodeGenVisitor, [192](#)
- pos
 - cstring.c, [656](#)
 - expr_parser::ExprLexer, [229](#)
- post
 - mxvm::Program, [345](#)
- predeclareType
 - mxx::TPValidator, [430](#)
- present
 - mx_sdl.h, [570](#)
 - sdl.c, [590](#)
- present_scaled
 - mx_sdl.h, [570](#)
 - sdl.c, [590](#)
- present_stretched
 - mx_sdl.h, [570](#)
 - sdl.c, [590](#)
- prevch
 - scan::buffer::StringBuffer< Ch, String >, [403](#)
- PRINT
 - instruct.hpp, [528](#)
- print
 - mxvm::Program, [346](#)
 - pascal::ASTNode, [143](#)
 - scan::token::Token< Ch, String >, [411](#)
- print_help
 - main.cpp, [786](#)
- print_type_CharType
 - types, [106](#)
- print_type_TokenType
 - types, [107](#)
- printFormatted
 - mxvm::Program, [346](#)
- printObjectHTML
 - mxvm::Parser, [283](#)
- proc
 - mx::Argz< String >, [125](#)
- proc_args
 - main.cpp, [786](#)
 - mx, [78](#)
- ProcCallNode
 - pascal::ProcCallNode, [304](#)
- ProcDeclNode
 - pascal::ProcDeclNode, [306](#)
- PROCEDURE
 - mxx, [92](#)
- process_arguments
 - main.cpp, [786](#)
- process_line_directive
 - scan::buffer::StringBuffer< Ch, String >, [403](#)
- processCodeSection
 - mxvm::Parser, [283](#)
- processDataSection
 - mxvm::Parser, [283](#)
- processModuleFile
 - mxvm::Parser, [283](#)
- processModuleSection
 - mxvm::Parser, [283](#)
- processObjectFile
 - mxvm::Parser, [283](#)
- processObjectSection
 - mxvm::Parser, [284](#)
- procs
 - mxx::Scope, [387](#)
- PROGRAM
 - mxx, [92](#)
- Program
 - mxvm::Program, [316](#)
- ProgramNode
 - pascal::ProgramNode, [370](#)
- prolog
 - pascal::CodeGenVisitor, [210](#)
- PTR
 - pascal, [97](#)
- ptr_count
 - mxvm::Variable_Value, [458](#)
- ptr_size
 - mxvm::Variable_Value, [458](#)
- ptr_value
 - mxvm::Variable_Value, [458](#)
- ptrRegInUse
 - pascal::CodeGenVisitor, [210](#)
- ptrRegisters
 - pascal::CodeGenVisitor, [210](#)
- PUSH
 - instruct.hpp, [529](#)
- push
 - mxvm::Stack, [398](#)
- pushCmpResult
 - pascal::CodeGenVisitor, [192](#)
- pushFloatCmpResult
 - pascal::CodeGenVisitor, [193](#)
- pushLogicalAnd
 - pascal::CodeGenVisitor, [193](#)
- pushLogicalOr
 - pascal::CodeGenVisitor, [193](#)
- pushRecordFieldScope
 - mxx::TPValidator, [430](#)
- pushScope
 - mxx::TPValidator, [431](#)
- pushTri
 - pascal::CodeGenVisitor, [193](#)
- pushValue
 - pascal::CodeGenVisitor, [193](#)
- queue_audio
 - mx_sdl.h, [570](#)
 - sdl.c, [591](#)
- quit
 - mx_sdl.h, [571](#)

- sdl.c, [591](#)
- quit_text
 - mx_sdl.h, [571](#)
 - sdl.c, [591](#)
- rand_number
 - io.c, [547](#)
- readFile
 - html.cpp, [863](#)
- realConstantCounter
 - pascal::CodeGenVisitor, [210](#)
- realConstants
 - pascal::CodeGenVisitor, [210](#)
- REALLOC
 - instruct.hpp, [529](#)
- RECORD
 - mxx, [92](#)
 - pascal, [97](#)
- RecordDeclarationNode
 - pascal::RecordDeclarationNode, [372](#)
- recordExpr
 - pascal::FieldAccessNode, [236](#)
- RecordField
 - pascal::CodeGenVisitor::RecordField, [373](#)
- recordFieldScopesByType
 - mxx::TPValidator, [433](#)
- recordLocation
 - pascal::CodeGenVisitor, [193](#)
- recordsToFreeInScope
 - pascal::CodeGenVisitor, [210](#)
- recordType
 - pascal::RecordDeclarationNode, [372](#)
- RecordTypeNode
 - pascal::RecordTypeNode, [377](#)
- recordTypes
 - pascal::CodeGenVisitor, [211](#)
- recordVar
 - pascal::WithStmtNode, [468](#)
- RED
 - mxx::Color, [81](#)
- regInUse
 - pascal::CodeGenVisitor, [211](#)
- REGISTER
 - pascal::CodeGenVisitor::ValueLocation, [449](#)
- registerExternalFunc
 - pascal::CodeGenVisitor, [194](#)
- registerHandler
 - pascal::BuiltinFunctionRegistry, [170](#)
- registerImportedConst
 - pascal::CodeGenVisitor, [194](#)
- registerImportedVar
 - pascal::CodeGenVisitor, [194](#)
- registerObjectExterns
 - mxvm::Parser, [284](#)
- registers
 - pascal::CodeGenVisitor, [211](#)
- release
 - std.c, [633](#)
- released
 - mxvm::Variable_Value, [459](#)
- releaseOwnedPointer
 - mxvm, [88](#)
- releaseTempPtr
 - pascal::CodeGenVisitor, [194](#)
- removeBraceComments
 - pascal::PascalParser, [298](#)
- removeComments
 - main.cpp, [778](#)
- removeEOL
 - scan::Scanner, [386](#)
- render_texture
 - mx_sdl.h, [571](#)
 - sdl.c, [591](#)
- REPEAT
 - mxx, [92](#)
- RepeatStmtNode
 - pascal::RepeatStmtNode, [378](#)
- require
 - mxvm::ModuleParser, [266](#)
 - mxvm::Validator, [446](#)
 - mxx::TPValidator, [431](#)
- requireKW
 - mxx::TPValidator, [431](#)
- RESET
 - mxx::Color, [81](#)
- reset
 - mxx::Argz< String >, [125](#)
 - scan::buffer::StringBuffer< Ch, String >, [404](#)
- resolveLabelReference
 - mxvm::Parser, [284](#)
- resolveTypeName
 - pascal::CodeGenVisitor, [194](#)
- result
 - mxvm::Program, [366](#)
- RET
 - instruct.hpp, [529](#)
- ret_owned
 - mxvm::ExternalFunction, [234](#)
- ret_type
 - mxvm::ExternalFunction, [234](#)
- RETURN
 - instruct.hpp, [529](#)
- returnType
 - pascal::CodeGenVisitor::FuncInfo, [244](#)
 - pascal::FuncDeclNode, [243](#)
- right
 - pascal::BinaryOpNode, [162](#)
- root_name
 - mxvm::Base, [159](#)
 - mxvm::ProgramNode, [369](#)
- RPAREN
 - expr_parser, [76](#)
- rtrim_comment
 - pascal, [99](#)
- running
 - mxvm::Program, [366](#)
- runtime_call

- mxvm, 84
- RuntimeFunction
 - mxvm::RuntimeFunction, 380
- scan, 101
 - mxvm::ModuleParser, 266
 - mxvm::Parser, 284
 - scan::Scanner, 386
 - StringType, 101
 - TMap, 101
 - TString, 101
 - TToken, 101
- scan::buffer, 102
- scan::buffer::StringBuffer< Ch, String >, 400
 - backward_step, 402
 - buffer_, 404
 - ch_type, 401
 - cur_file, 402
 - cur_line, 402
 - curch, 402
 - currentColumn, 404
 - currentFile, 404
 - currentLine, 404
 - eof, 402
 - forward_step, 403
 - getch, 403
 - index, 405
 - peekch, 403
 - prevch, 403
 - process_line_directive, 403
 - reset, 404
 - string_type, 401
 - StringBuffer, 402
- scan::ScanExcept, 382
 - msg, 383
 - ScanExcept, 382
 - why, 382
- scan::Scanner, 383
 - getTokens, 384
 - grabDigits, 384
 - grabId, 384
 - grabSingle, 384
 - grabString, 385
 - grabSymbols, 385
 - is_c_sym, 385
 - operator[], 385
 - parseFileName, 385
 - parseLineNumber, 385
 - removeEOL, 386
 - scan, 386
 - Scanner, 384
 - size, 386
 - string_buffer, 386
 - token_map, 386
 - tokens, 386
- scan::token, 102
 - CharType, 102
- scan::token::Token< Ch, String >, 408
 - ch_type, 409
 - col, 412
 - filename, 412
 - get_filename, 410
 - get_pos, 410
 - getCol, 410
 - getLine, 411
 - getTokenType, 411
 - getTokenValue, 411
 - line, 412
 - operator=, 411
 - print, 411
 - set_filename, 412
 - set_pos, 412
 - setToken, 412
 - string_type, 409
 - Token, 410
 - type, 413
 - value, 413
- scan::token::TokenMap< Ch, MAX_SIZE >, 413
 - lookup_int8, 414
 - token_map, 414
 - TokenMap, 414
- ScanExcept
 - scan::ScanExcept, 382
- Scanner
 - scan::Scanner, 384
- scanner
 - expr_parser::ExprLexer, 230
 - mxvm::ModuleParser, 267
 - mxvm::Parser, 285
 - mxvm::Validator, 448
 - mxx::TPValidator, 433
 - mxx::XParser, 471
- scanner.hpp
 - operator<<, 933
- scopeHierarchy
 - pascal::CodeGenVisitor, 211
- scopeStack
 - mxx::TPValidator, 433
 - pascal::CodeGenVisitor, 211
- sdl.c
 - blit_surface, 579
 - clear, 579
 - clear_queued_audio, 579
 - close_audio, 579
 - create_render_target, 579
 - create_renderer, 579
 - create_rgb_surface, 580
 - create_texture, 580
 - create_window, 580
 - delay, 581
 - destroy_render_target, 581
 - destroy_renderer, 581
 - destroy_texture, 581
 - destroy_window, 582
 - draw_line, 582
 - draw_point, 582
 - draw_rect, 582

draw_text, 582
fill_rect, 583
free_surface, 583
free_wav, 583
g_event, 593
g_font_count, 593
g_fonts, 593
g_render_target_count, 593
g_render_targets, 594
g_renderer_count, 594
g_renderers, 594
g_target_heights, 594
g_target_widths, 594
g_texture_count, 594
g_textures, 594
g_window_count, 594
g_windows, 595
get_clipboard_text, 583
get_event_type, 583
get_key_code, 583
get_keyboard_state, 584
get_mouse_button, 584
get_mouse_buttons, 584
get_mouse_state, 584
get_mouse_x, 584
get_mouse_y, 585
get_num_keys, 585
get_queued_audio_size, 585
get_relative_mouse_buttons, 585
get_relative_mouse_state, 585
get_relative_mouse_x, 585
get_relative_mouse_y, 586
get_renderer_output_size, 586
get_ticks, 586
get_window_size, 586
init, 586
init_text, 587
is_key_pressed, 587
load_font, 587
load_texture, 587
load_texture_color_key, 588
load_texture_color_key_rgb, 588
load_wav, 589
lock_texture, 589
open_audio, 589
pause_audio, 590
poll_event, 590
present, 590
present_scaled, 590
present_stretched, 590
queue_audio, 591
quit, 591
quit_text, 591
render_texture, 591
set_clipboard_text, 591
set_draw_color, 592
set_render_target, 592
set_window_fullscreen, 592
set_window_icon, 592
set_window_position, 592
set_window_title, 592
show_cursor, 593
unlock_texture, 593
update_texture, 593
sdl.cpp
mxvm_sdl_blit_surface, 605
mxvm_sdl_clear, 605
mxvm_sdl_clear_queued_audio, 605
mxvm_sdl_close_audio, 605
mxvm_sdl_create_render_target, 605
mxvm_sdl_create_renderer, 605
mxvm_sdl_create_rgb_surface, 606
mxvm_sdl_create_texture, 606
mxvm_sdl_create_window, 606
mxvm_sdl_delay, 606
mxvm_sdl_destroy_render_target, 606
mxvm_sdl_destroy_renderer, 606
mxvm_sdl_destroy_texture, 606
mxvm_sdl_destroy_window, 607
mxvm_sdl_draw_line, 607
mxvm_sdl_draw_point, 607
mxvm_sdl_draw_rect, 607
mxvm_sdl_draw_text, 607
mxvm_sdl_fill_rect, 607
mxvm_sdl_free_surface, 608
mxvm_sdl_free_wav, 608
mxvm_sdl_get_clipboard_text, 608
mxvm_sdl_get_event_type, 608
mxvm_sdl_get_key_code, 608
mxvm_sdl_get_keyboard_state, 608
mxvm_sdl_get_mouse_button, 608
mxvm_sdl_get_mouse_buttons, 609
mxvm_sdl_get_mouse_state, 609
mxvm_sdl_get_mouse_x, 609
mxvm_sdl_get_mouse_y, 609
mxvm_sdl_get_num_keys, 609
mxvm_sdl_get_queued_audio_size, 609
mxvm_sdl_get_relative_mouse_buttons, 609
mxvm_sdl_get_relative_mouse_state, 610
mxvm_sdl_get_relative_mouse_x, 610
mxvm_sdl_get_relative_mouse_y, 610
mxvm_sdl_get_renderer_output_size, 610
mxvm_sdl_get_ticks, 610
mxvm_sdl_get_window_size, 610
mxvm_sdl_init, 610
mxvm_sdl_init_text, 611
mxvm_sdl_is_key_pressed, 611
mxvm_sdl_load_font, 611
mxvm_sdl_load_texture, 611
mxvm_sdl_load_texture_color_key, 611
mxvm_sdl_load_texture_color_key_rgb, 611
mxvm_sdl_load_wav, 612
mxvm_sdl_lock_texture, 612
mxvm_sdl_open_audio, 612
mxvm_sdl_pause_audio, 612
mxvm_sdl_poll_event, 612

- mxvm_sdl_present, 613
- mxvm_sdl_present_scaled, 613
- mxvm_sdl_present_stretched, 613
- mxvm_sdl_queue_audio, 613
- mxvm_sdl_quit, 613
- mxvm_sdl_quit_text, 613
- mxvm_sdl_render_texture, 613
- mxvm_sdl_set_clipboard_text, 614
- mxvm_sdl_set_draw_color, 614
- mxvm_sdl_set_render_target, 614
- mxvm_sdl_set_window_fullscreen, 614
- mxvm_sdl_set_window_icon, 614
- mxvm_sdl_set_window_position, 614
- mxvm_sdl_set_window_title, 614
- mxvm_sdl_show_cursor, 615
- mxvm_sdl_unlock_texture, 615
- mxvm_sdl_update_texture, 615
- SDLFunctionHandler
 - pascal::CodeGenVisitor, 204
- sdlFunctions
 - pascal, 100
- second
 - mxvm::Validator::VarNamePair, 464
- SectionNode
 - mxvm::SectionNode, 391
- sections
 - mxvm::ProgramNode, 369
- SectionType
 - mxvm::SectionNode, 391
- seed_random
 - io.c, 547
- SET
 - mxx, 92
- set_clipboard_text
 - mx_sdl.h, 571
 - sdl.c, 591
- set_draw_color
 - mx_sdl.h, 571
 - sdl.c, 592
- set_filename
 - scan::token::Token< Ch, String >, 412
- set_pos
 - scan::token::Token< Ch, String >, 412
- set_program_args
 - mx_std.h, 630
 - std.c, 633
- set_render_target
 - mx_sdl.h, 572
 - sdl.c, 592
- set_window_fullscreen
 - mx_sdl.h, 572
 - sdl.c, 592
- set_window_icon
 - mx_sdl.h, 572
 - sdl.c, 592
- set_window_position
 - mx_sdl.h, 572
 - sdl.c, 592
- set_window_title
 - mx_sdl.h, 572
 - sdl.c, 592
- setArgs
 - mxvm::Program, 346
- setDefaultVariableValue
 - mxvm::Parser, 284
- setExtern
 - mxvm::Variable, 453
- setLineNumber
 - pascal::ASTNode, 143
- SetLiteralNode
 - pascal::SetLiteralNode, 393
- setMainBase
 - mxvm::Base, 157
- setObject
 - mxvm::Program, 347
- setSlotType
 - pascal::CodeGenVisitor, 195
- setToken
 - scan::token::Token< Ch, String >, 412
- SetTypeNode
 - pascal::SetTypeNode, 394
- setVariableFromConstant
 - mxvm::Program, 347
- setVariableFromString
 - mxvm::Program, 347
- setVariableValue
 - mxvm::Parser, 284
- setVars
 - pascal::CodeGenVisitor, 211
- setVarType
 - pascal::CodeGenVisitor, 195
- show_cursor
 - mx_sdl.h, 572
 - sdl.c, 593
- signal_action
 - main.cpp, 786
- signal_program
 - main.cpp, 787
- SimpleTypeNode
 - pascal::SimpleTypeNode, 396
- size
 - mxvm::Stack, 398
 - pascal::ArrayInfo, 134
 - pascal::CodeGenVisitor::RecordField, 374
 - pascal::CodeGenVisitor::RecordTypeInfo, 375
 - scan::Scanner, 386
- slotToName
 - pascal::CodeGenVisitor, 211
- slotToType
 - pascal::CodeGenVisitor, 211
- slotVar
 - pascal::CodeGenVisitor, 195
- source
 - mxvm::HTMLGen, 250
 - mxvm::PasHTMLGen, 300
 - mxvm::Validator, 448

- mxv::TPValidator, [433](#)
- source_file
 - Args, [115](#)
 - mxvm::Parser, [286](#)
- src Directory Reference, [71](#)
- src/ast.cpp, [669](#), [670](#)
- src/frontend Directory Reference, [63](#)
- src/frontend/ast.cpp, [671](#), [672](#)
- src/frontend/expr.cpp, [677](#), [678](#)
- src/frontend/icode.cpp, [678](#), [679](#)
- src/frontend/icode_gen.cpp, [693](#)
- src/frontend/icode_imp.cpp, [717](#), [718](#)
- src/frontend/icode_visit.cpp, [731](#), [732](#)
- src/frontend/include Directory Reference, [65](#)
- src/frontend/include/ast.hpp, [476](#), [480](#)
- src/frontend/include/expr.hpp, [763](#), [764](#)
- src/frontend/include/icode.hpp, [499](#), [500](#)
- src/frontend/include/parser.hpp, [539](#), [540](#)
- src/frontend/include/types.hpp, [768](#), [769](#)
- src/frontend/include/validator.hpp, [774](#), [776](#)
- src/frontend/main.cpp, [777](#), [779](#)
- src/frontend/parser.cpp, [797](#)
- src/frontend/validator.cpp, [836](#)
- src/function.cpp, [854](#), [855](#)
- src/html_gen Directory Reference, [64](#)
- src/html_gen.cpp, [855](#), [856](#)
- src/html_gen/html.cpp, [862](#), [864](#)
- src/icode.cpp, [687](#), [688](#)
- src/icode_exec.cpp, [866](#), [867](#)
- src/icode_gen.cpp, [693](#), [694](#)
- src/icode_gen_x64.cpp, [893](#), [894](#)
- src/icode_opt.cpp, [914](#), [915](#)
- src/instruct.cpp, [921](#), [924](#)
- src/mxvm.cpp, [925](#)
- src/parser.cpp, [813](#), [814](#)
- src/scanner Directory Reference, [69](#)
- src/scanner/exception.cpp, [926](#), [927](#)
- src/scanner/include Directory Reference, [66](#)
- src/scanner/include/scanner Directory Reference, [69](#)
- src/scanner/include/scanner/exception.hpp, [928](#), [930](#)
- src/scanner/include/scanner/expr.hpp, [764](#), [765](#)
- src/scanner/include/scanner/scanner.hpp, [931](#), [933](#)
- src/scanner/include/scanner/string_buffer.hpp, [934](#), [935](#)
- src/scanner/include/scanner/types.hpp, [770](#), [772](#)
- src/scanner/scanner.cpp, [938](#), [939](#)
- src/scanner/string_buffer.cpp, [945](#), [946](#)
- src/scanner/types.cpp, [946](#), [947](#)
- src/valid.cpp, [947](#), [948](#)
- src/vm Directory Reference, [73](#)
- src/vm/include Directory Reference, [66](#)
- src/vm/include/argz.hpp, [958](#), [959](#)
- src/vm/main.cpp, [783](#), [787](#)
- sstrCharType
 - types, [107](#)
- Stack
 - mxvm::Stack, [397](#)
- stack
 - mxvm::Program, [366](#)
- STACK_LOAD
 - instruct.hpp, [529](#)
- STACK_STORE
 - instruct.hpp, [529](#)
- STACK_SUB
 - instruct.hpp, [529](#)
- StackValue
 - mxvm, [84](#)
- startValue
 - pascal::ForStmtNode, [239](#)
- statement
 - pascal::CaseStmtNode::CaseBranch, [171](#)
 - pascal::ForStmtNode, [239](#)
 - pascal::LabelStmtNode, [261](#)
 - pascal::WhileStmtNode, [466](#)
 - pascal::WithStmtNode, [468](#)
- statements
 - mxvm::SectionNode, [392](#)
 - pascal::CompoundStmtNode, [216](#)
 - pascal::RepeatStmtNode, [379](#)
- std.c
 - argc, [632](#)
 - argv, [632](#)
 - float_to_int, [632](#)
 - free_program_args, [633](#)
 - g_argc, [633](#)
 - g_args_registered, [633](#)
 - g_argv, [634](#)
 - int_to_float, [633](#)
 - mx_strdup_c, [633](#)
 - release, [633](#)
 - set_program_args, [633](#)
- std.cpp
 - mxvm_std_abs, [636](#)
 - mxvm_std_acos, [636](#)
 - mxvm_std_argc, [637](#)
 - mxvm_std_argv, [637](#)
 - mxvm_std_asin, [637](#)
 - mxvm_std_atan, [637](#)
 - mxvm_std_atan2, [637](#)
 - mxvm_std_atof, [637](#)
 - mxvm_std_atoi, [638](#)
 - mxvm_std_calloc, [638](#)
 - mxvm_std_ceil, [638](#)
 - mxvm_std_cos, [638](#)
 - mxvm_std_cosh, [638](#)
 - mxvm_std_exit, [638](#)
 - mxvm_std_exp, [638](#)
 - mxvm_std_exp2, [639](#)
 - mxvm_std_fabs, [639](#)
 - mxvm_std_float_to_int, [639](#)
 - mxvm_std_floor, [639](#)
 - mxvm_std_fmod, [639](#)
 - mxvm_std_free_program_args, [639](#)
 - mxvm_std_hypot, [640](#)
 - mxvm_std_int_to_float, [640](#)
 - mxvm_std_isalpha, [640](#)
 - mxvm_std_isdigit, [640](#)

- mxvm_std_isspace, [640](#)
- mxvm_std_log, [640](#)
- mxvm_std_log10, [641](#)
- mxvm_std_log2, [641](#)
- mxvm_std_malloc, [641](#)
- mxvm_std_memcmp, [641](#)
- mxvm_std_memcpy, [641](#)
- mxvm_std_memmove, [641](#)
- mxvm_std_memset, [641](#)
- mxvm_std_pow, [642](#)
- mxvm_std_rand, [642](#)
- mxvm_std_release, [642](#)
- mxvm_std_round, [642](#)
- mxvm_std_set_program_args, [642](#)
- mxvm_std_sin, [642](#)
- mxvm_std_sinh, [643](#)
- mxvm_std_sqrt, [643](#)
- mxvm_std_srand, [643](#)
- mxvm_std_system, [643](#)
- mxvm_std_tan, [643](#)
- mxvm_std_tanh, [643](#)
- mxvm_std_tolower, [644](#)
- mxvm_std_toupper, [644](#)
- mxvm_std_trunc, [644](#)
- StdFunctionHandler
 - pascal::CodeGenVisitor, [204](#)
- stdFunctions
 - pascal, [100](#)
- stdio_index
 - mxvm, [89](#)
- stop
 - mxvm::Program, [347](#)
- storageSymbolFor
 - pascal::CodeGenVisitor, [195](#)
- STORE
 - instruct.hpp, [528](#)
- Str
 - mxvm, [85](#)
- str_value
 - mxvm::Variable_Value, [459](#)
- strat
 - cstring.c, [656](#)
- strCharType
 - types, [107](#)
- strfind
 - cstring.c, [657](#)
- STRING
 - pascal, [96](#)
- string.cpp
 - asReadPtr, [660](#)
 - asWritePtr, [660](#)
 - getStringFromVar, [660](#)
 - isStringLike, [660](#)
 - mxvm_string_copy, [660](#)
 - mxvm_string_delete, [660](#)
 - mxvm_string_insert, [661](#)
 - mxvm_string_inttostr, [661](#)
 - mxvm_string_pos, [661](#)
 - mxvm_string_sprintf, [661](#)
 - mxvm_string_strat, [661](#)
 - mxvm_string_strcmp, [662](#)
 - mxvm_string_strfind, [662](#)
 - mxvm_string_strlen, [662](#)
 - mxvm_string_strncat, [662](#)
 - mxvm_string_strncpy, [662](#)
 - mxvm_string_strtoint, [662](#)
 - mxvm_string_substr, [663](#)
- string_buffer
 - scan::Scanner, [386](#)
- STRING_PRINT
 - instruct.hpp, [529](#)
- string_type
 - scan::buffer::StringBuffer< Ch, String >, [401](#)
 - scan::token::Token< Ch, String >, [409](#)
- StringBuffer
 - scan::buffer::StringBuffer< Ch, String >, [402](#)
- StringFunctionHandler
 - pascal::CodeGenVisitor, [204](#)
- stringLiterals
 - pascal::CodeGenVisitor, [211](#)
- StringNode
 - pascal::StringNode, [407](#)
- StringType
 - scan, [101](#)
- strtoint
 - cstring.c, [657](#)
- strTokenType
 - types, [107](#)
- SUB
 - instruct.hpp, [528](#)
- substr
 - cstring.c, [657](#)
- subVariables
 - mxvm::Program, [348](#)
- sweep_unused_data
 - pascal, [99](#)
- sysv_analyzeRegAlloc
 - mxvm::Program, [348](#)
- sysv_emitFlushRegs
 - mxvm::Program, [348](#)
- sysv_emitLoadVar
 - mxvm::Program, [348](#)
- sysv_emitReloadRegs
 - mxvm::Program, [349](#)
- sysv_emitRestoreRegs
 - mxvm::Program, [349](#)
- sysv_emitSaveRegs
 - mxvm::Program, [349](#)
- sysv_emitStoreVar
 - mxvm::Program, [349](#)
- sysv_emitStoreVarImm
 - mxvm::Program, [349](#)
- sysv_reg_save_order
 - mxvm::Program, [366](#)
- sysv_reg_vars
 - mxvm::Program, [366](#)

- target
 - Args, 115
- tempPtrByScope
 - pascal::CodeGenVisitor, 212
- tempPtrInUse
 - pascal::CodeGenVisitor, 212
- terminal_supports_color
 - mx, 79
- text
 - mx::ArgException< String >, 113
 - mxvm::CommentNode, 214
 - mxvm::ParsedOp, 277
- THEN
 - mxx, 92
- thenStatement
 - pascal::IfStmtNode, 252
- TMap
 - scan, 101
- TO
 - mxx, 92
- TO_FLOAT
 - instruct.hpp, 529
- TO_INT
 - instruct.hpp, 529
- Token
 - scan::token::Token< Ch, String >, 410
- token
 - mxvm::ModuleParser, 267
 - mxvm::Validator, 448
 - mxx::TPValidator, 434
 - mxx::XParser, 471
- token_map
 - scan::Scanner, 386
 - scan::token::TokenMap< Ch, MAX_SIZE >, 414
- TokenMap
 - scan::token::TokenMap< Ch, MAX_SIZE >, 414
- tokens
 - scan::Scanner, 386
- TokenType
 - types, 105
- tokenTypeToString
 - mxvm::Validator, 446
 - mxx::TPValidator, 431
 - mxx::XParser, 470
 - pascal::PascalParser, 299
- toLower
 - mxx::TPValidator, 431
 - pascal, 99
- toolchain
 - Args, 115
- toString
 - mxvm::ASTNode, 141
 - mxvm::CommentNode, 214
 - mxvm::ExpressionNode, 227
 - mxvm::Instruction, 253
 - mxvm::InstructionNode, 256
 - mxvm::LabelNode, 259
 - mxvm::ModuleNode, 262
 - mxvm::ObjectNode, 271
 - mxvm::ProgramNode, 368
 - mxvm::SectionNode, 391
 - mxvm::Variable, 453
 - mxvm::VariableNode, 461
 - pascal::AddressOfNode, 112
 - pascal::ArrayAccessNode, 127
 - pascal::ArrayAssignmentNode, 129
 - pascal::ArrayDeclarationNode, 131
 - pascal::ArrayTypeDeclarationNode, 135
 - pascal::ArrayTypeNode, 137
 - pascal::AssignmentNode, 139
 - pascal::ASTNode, 144
 - pascal::BinaryOpNode, 162
 - pascal::BlockNode, 164
 - pascal::BooleanNode, 166
 - pascal::BreakNode, 167
 - pascal::CaseStmtNode, 172
 - pascal::CompoundStmtNode, 216
 - pascal::ConstDeclNode, 218
 - pascal::ContinueNode, 220
 - pascal::EmptyStmtNode, 221
 - pascal::EnumTypeDeclNode, 223
 - pascal::ExitNode, 226
 - pascal::FieldAccessNode, 235
 - pascal::ForStmtNode, 239
 - pascal::FuncCallNode, 241
 - pascal::FuncDeclNode, 243
 - pascal::GotoStmtNode, 249
 - pascal::IfStmtNode, 252
 - pascal::LabelStmtNode, 261
 - pascal::NilNode, 268
 - pascal::NumberNode, 270
 - pascal::ParameterNode, 276
 - pascal::PointerDerefNode, 302
 - pascal::PointerTypeNode, 303
 - pascal::ProcCallNode, 305
 - pascal::ProcDeclNode, 307
 - pascal::ProgramNode, 370
 - pascal::RecordDeclarationNode, 372
 - pascal::RecordTypeNode, 377
 - pascal::RepeatStmtNode, 379
 - pascal::SetLiteralNode, 393
 - pascal::SetTypeNode, 395
 - pascal::SimpleTypeNode, 396
 - pascal::StringNode, 408
 - pascal::TypeAliasNode, 435
 - pascal::TypeDeclNode, 437
 - pascal::UnaryOpNode, 439
 - pascal::UnitNode, 441
 - pascal::VarDeclNode, 450
 - pascal::VariableNode, 463
 - pascal::WhileStmtNode, 466
 - pascal::WithStmtNode, 468
- toStringFromVarType
 - mxvm, 89
- TPValidator
 - mxx::TPValidator, 419

- translate
 - main.cpp, [784](#)
- translate_x64
 - main.cpp, [787](#)
- trim
 - pascal, [100](#)
- tryGetConstNumeric
 - pascal::CodeGenVisitor, [195](#)
- TString
 - scan, [101](#)
- TT_ARG
 - types, [106](#)
- TT_CHAR
 - types, [103](#)
- TT_DIGIT
 - types, [103](#)
- TT_HEX
 - types, [106](#)
- TT_ID
 - types, [106](#)
- TT_NULL
 - types, [104](#), [106](#)
- TT_NUM
 - types, [106](#)
- TT_SINGLE
 - types, [103](#)
- TT_SPACE
 - types, [104](#)
- TT_STR
 - types, [106](#)
- TT_STRING
 - types, [103](#)
- TT_SYM
 - types, [106](#)
- TT_SYMBOL
 - types, [103](#)
- TTToken
 - scan, [101](#)
- TYPE
 - mxx, [92](#)
- Type
 - pascal::CodeGenVisitor::ValueLocation, [449](#)
- type
 - expr_parser::ExprToken, [233](#)
 - mxvm::Operand, [273](#)
 - mxvm::SectionNode, [392](#)
 - mxvm::Variable, [453](#)
 - mxvm::Variable_Value, [459](#)
 - mxvm::VariableNode, [462](#)
 - pascal::CodeGenVisitor::ValueLocation, [449](#)
 - pascal::ParameterNode, [276](#)
 - pascal::VarDeclNode, [451](#)
 - scan::token::Token< Ch, String >, [413](#)
- typeAliases
 - pascal::CodeGenVisitor, [212](#)
- TypeAliasNode
 - pascal::TypeAliasNode, [435](#)
- typeDeclarations
 - pascal::TypeDeclNode, [437](#)
- TypeDeclNode
 - pascal::TypeDeclNode, [437](#)
- typeName
 - pascal::CodeGenVisitor::RecordField, [374](#)
 - pascal::EnumTypeDeclNode, [223](#)
 - pascal::SimpleTypeNode, [396](#)
 - pascal::TypeAliasNode, [435](#)
- types, [102](#)
 - CharType, [103](#)
 - KeywordType, [104](#)
 - KW_BREAK, [104](#)
 - KW_CONTINUE, [104](#)
 - KW_DEFINE, [104](#)
 - KW_ELSE, [104](#)
 - KW_FOR, [104](#)
 - KW_IF, [104](#)
 - KW_LET, [104](#)
 - KW_PROC, [104](#)
 - KW_RETURN, [104](#)
 - KW_SWITCH, [104](#)
 - KW_WHILE, [104](#)
 - kwStr, [107](#)
 - lookUp, [106](#)
 - lookUp_Keyword, [106](#)
 - mxx::Scope, [387](#)
 - OP_AND, [105](#)
 - OP_AND_AND, [105](#)
 - OP_AND_ASSIGN, [104](#)
 - OP_ARROW, [105](#)
 - OP_ASSIGN, [105](#)
 - OP_AT, [105](#)
 - OP_COLON, [105](#)
 - OP_COMMA, [105](#)
 - OP_DEC, [104](#)
 - OP_DIV, [105](#)
 - OP_DIV_ASSIGN, [104](#)
 - OP_DOLLAR, [105](#)
 - OP_DOT, [105](#)
 - OP_EQ, [105](#)
 - OP_GE, [105](#)
 - OP_GT, [105](#)
 - OP_HASH, [105](#)
 - OP_INC, [104](#)
 - OP_LBRACE, [105](#)
 - OP_LBRACKET, [105](#)
 - OP_LE, [105](#)
 - OP_LPAREN, [105](#)
 - OP_LSHIFT, [104](#)
 - OP_LSHIFT_ASSIGN, [104](#)
 - OP_LT, [105](#)
 - OP_MINUS, [105](#)
 - OP_MINUS_ASSIGN, [104](#)
 - OP_MOD, [105](#)
 - OP_MOD_ASSIGN, [104](#)
 - OP_MUL, [105](#)
 - OP_MUL_ASSIGN, [104](#)
 - OP_NEQ, [105](#)

- OP_NOT, [105](#)
- OP_OR, [105](#)
- OP_OR_ASSIGN, [104](#)
- OP_OR_OR, [105](#)
- OP_PLUS, [105](#)
- OP_PLUS_ASSIGN, [104](#)
- OP_QUESTION, [105](#)
- OP_RBRACE, [105](#)
- OP_RBACKET, [105](#)
- OP_LPAREN, [105](#)
- OP_RSHIFT, [104](#)
- OP_RSHIFT_ASSIGN, [104](#)
- OP_SCOPE, [105](#)
- OP_SEMICOLON, [105](#)
- OP_TILDE, [105](#)
- OP_XOR, [105](#)
- OP_XOR_ASSIGN, [104](#)
- OperatorType, [104](#)
- opName, [107](#)
- opStrings, [107](#)
- print_type_CharType, [106](#)
- print_type_TokenType, [107](#)
- sstrCharType, [107](#)
- strCharType, [107](#)
- strTokenType, [107](#)
- TokenType, [105](#)
- TT_ARG, [106](#)
- TT_CHAR, [103](#)
- TT_DIGIT, [103](#)
- TT_HEX, [106](#)
- TT_ID, [106](#)
- TT_NULL, [104](#), [106](#)
- TT_NUM, [106](#)
- TT_SINGLE, [103](#)
- TT_SPACE, [104](#)
- TT_STR, [106](#)
- TT_STRING, [103](#)
- TT_SYM, [106](#)
- TT_SYMBOL, [103](#)
- UnaryOpNode
 - pascal::UnaryOpNode, [439](#)
- UnitNode
 - pascal::UnitNode, [441](#)
- UNKNOWN
 - pascal, [97](#)
- unlock_texture
 - mx_sdl.h, [572](#)
 - sdl.c, [593](#)
- UNTIL
 - mxx, [92](#)
- update_texture
 - mx_sdl.h, [573](#)
 - sdl.c, [593](#)
- updateDataSectionInitialValue
 - pascal::CodeGenVisitor, [195](#)
- upperBound
 - pascal::ArrayInfo, [134](#)
 - pascal::ArrayTypeNode, [137](#)
- usedModules
 - pascal::CodeGenVisitor, [212](#)
- usedRealConstants
 - pascal::CodeGenVisitor, [212](#)
- usedStrings
 - pascal::CodeGenVisitor, [212](#)
- uses
 - pascal::ProgramNode, [371](#)
 - pascal::UnitNode, [442](#)
- validate
 - mxvm::Validator, [446](#)
 - mxx::TPValidator, [432](#)
- validateAgainstSpec
 - mxvm::Validator, [447](#)
- validateNames
 - mxvm::Program, [350](#)
- Validator
 - mxvm::Validator, [444](#)
- validator
 - mxvm::Parser, [286](#)
 - pascal::PascalParser, [299](#)
- value
 - expr_parser::ExprToken, [233](#)
 - mx::ArgException< String >, [113](#)
 - mxvm::ExpressionNode, [228](#)
 - pascal::ArrayAssignmentNode, [129](#)
 - pascal::BooleanNode, [166](#)
 - pascal::ConstDeclNode::ConstAssignment, [217](#)
 - pascal::NumberNode, [270](#)
 - pascal::StringNode, [408](#)
 - scan::token::Token< Ch, String >, [413](#)
- valueLocations
 - pascal::CodeGenVisitor, [212](#)
- values
 - pascal::CaseStmtNode::CaseBranch, [171](#)
 - pascal::EnumTypeDeclNode, [223](#)
- VAR
 - mxx, [92](#)
- VAR_ARRAY
 - mxvm, [86](#)
- VAR_BYTE
 - mxvm, [86](#)
- VAR_EXTERN
 - mxvm, [86](#)
- VAR_FLOAT
 - mxvm, [86](#)
- VAR_INTEGER
 - mxvm, [86](#)
- VAR_LABEL
 - mxvm, [86](#)
- var_name
 - mxvm::Variable, [454](#)
- var_names
 - mxvm::Validator, [448](#)
- VAR_NULL
 - mxvm, [86](#)
- VAR_POINTER
 - mxvm, [86](#)

- VAR_STRING
 - mxvm, [86](#)
- var_value
 - mxvm::Variable, [454](#)
- VarDeclNode
 - pascal::VarDeclNode, [450](#)
- VARIABLE
 - expr_parser, [76](#)
- Variable
 - mxvm::Variable, [452](#)
- variable
 - expr_parser::ExprLexer, [230](#)
 - pascal::AssignmentNode, [139](#)
 - pascal::ForStmtNode, [239](#)
- Variable_Value
 - mxvm::Variable_Value, [456](#)
- variableFromOperand
 - mxvm::Program, [350](#)
- VariableNode
 - mxvm::VariableNode, [460](#), [461](#)
 - pascal::VariableNode, [463](#)
- variantArms
 - pascal::RecordTypeNode, [377](#)
- variantTagName
 - pascal::RecordTypeNode, [377](#)
- variantTagType
 - pascal::RecordTypeNode, [377](#)
- VArity
 - mxvm, [86](#)
- varPolicy
 - mxvm::OpSpec, [274](#)
- varRecordType
 - pascal::CodeGenVisitor, [212](#)
- vars
 - expr_parser, [76](#)
 - mxvm::Base, [159](#)
 - mxv::Scope, [388](#)
- varSlot
 - pascal::CodeGenVisitor, [212](#)
- VarType
 - mxvm, [86](#)
 - pascal, [96](#)
- varTypes
 - pascal::CodeGenVisitor, [213](#)
- VERSION_INFO
 - version_info.hpp, [545](#)
- version_info.hpp
 - VERSION_INFO, [545](#)
- visit
 - mxvm::ASTVisitor, [145](#)
 - pascal::ASTVisitor, [147–152](#)
 - pascal::CodeGenVisitor, [195–203](#)
- vm_action
 - main.cpp, [784](#)
- vm_target
 - main.cpp, [784](#)
- vop
 - mxvm::Instruction, [254](#)
- WHILE
 - mxv, [92](#)
- WhileStmtNode
 - pascal::WhileStmtNode, [466](#)
- WHITE
 - mx::Color, [81](#)
- why
 - scan::ScanExcept, [382](#)
- width
 - mx::Arguments, [120](#)
- WINX64
 - mxvm, [86](#)
- WITH
 - mxv, [92](#)
- withDepth
 - mxv::TPValidator, [434](#)
- withFieldScopes
 - pascal::CodeGenVisitor, [213](#)
- WithStmtNode
 - pascal::WithStmtNode, [467](#)
- writeHtml
 - html.cpp, [863](#)
- writeTo
 - pascal::CodeGenVisitor, [203](#)
- x64_analyzeRegAlloc
 - mxvm::Program, [350](#)
- x64_direct_stack_adjust
 - mxvm::Program, [351](#)
- x64_emit_job_func
 - mxvm::Program, [351](#)
- x64_emitFlushRegs
 - mxvm::Program, [351](#)
- x64_emitLoadVar
 - mxvm::Program, [351](#)
- x64_emitReloadRegs
 - mxvm::Program, [352](#)
- x64_emitRestoreRegs
 - mxvm::Program, [352](#)
- x64_emitSaveRegs
 - mxvm::Program, [352](#)
- x64_emitStoreVar
 - mxvm::Program, [352](#)
- x64_emitStoreVarImm
 - mxvm::Program, [352](#)
- x64_gen_alloc
 - mxvm::Program, [353](#)
- x64_gen_arth
 - mxvm::Program, [353](#)
- x64_gen_bitop
 - mxvm::Program, [353](#)
- x64_gen_call
 - mxvm::Program, [353](#)
- x64_gen_cmp
 - mxvm::Program, [354](#)
- x64_gen_div
 - mxvm::Program, [354](#)
- x64_gen_done
 - mxvm::Program, [354](#)

x64_gen_exit
 mxvm::Program, [354](#)

x64_gen_fcmp
 mxvm::Program, [354](#)

x64_gen_free
 mxvm::Program, [354](#)

x64_gen_getline
 mxvm::Program, [355](#)

x64_gen_invoke
 mxvm::Program, [355](#)

x64_gen_jae
 mxvm::Program, [355](#)

x64_gen_jbe
 mxvm::Program, [355](#)

x64_gen_jc
 mxvm::Program, [355](#)

x64_gen_jmp
 mxvm::Program, [355](#)

x64_gen_jnc
 mxvm::Program, [356](#)

x64_gen_jno
 mxvm::Program, [356](#)

x64_gen_jnp
 mxvm::Program, [356](#)

x64_gen_jns
 mxvm::Program, [356](#)

x64_gen_jo
 mxvm::Program, [356](#)

x64_gen_jp
 mxvm::Program, [356](#)

x64_gen_js
 mxvm::Program, [357](#)

x64_gen_lea
 mxvm::Program, [357](#)

x64_gen_load
 mxvm::Program, [357](#)

x64_gen_mod
 mxvm::Program, [357](#)

x64_gen_mov
 mxvm::Program, [357](#)

x64_gen_neg
 mxvm::Program, [357](#)

x64_gen_not
 mxvm::Program, [358](#)

x64_gen_pop
 mxvm::Program, [358](#)

x64_gen_print
 mxvm::Program, [358](#)

x64_gen_push
 mxvm::Program, [358](#)

x64_gen_realloc
 mxvm::Program, [358](#)

x64_gen_ret
 mxvm::Program, [359](#)

x64_gen_return
 mxvm::Program, [359](#)

x64_gen_stack_load
 mxvm::Program, [359](#)

x64_gen_stack_store
 mxvm::Program, [359](#)

x64_gen_stack_sub
 mxvm::Program, [359](#)

x64_gen_store
 mxvm::Program, [359](#)

x64_gen_to_float
 mxvm::Program, [360](#)

x64_gen_to_int
 mxvm::Program, [360](#)

x64_generateCode
 mxvm::Program, [360](#)

x64_generateFunctionCall
 mxvm::Program, [360](#)

x64_generateInstruction
 mxvm::Program, [361](#)

x64_generateInvokeCall
 mxvm::Program, [361](#)

x64_generateLoadVar
 mxvm::Program, [361](#), [362](#)

x64_getRegisterByIndex
 mxvm::Program, [362](#)

x64_opt_core_lines
 mxvm, [89](#)

x64_reg_save_order
 mxvm::Program, [366](#)

x64_reg_vars
 mxvm::Program, [366](#)

x64_release_call_area
 mxvm::Program, [363](#)

x64_reserve_call_area
 mxvm::Program, [363](#)

x64_sp_mod16
 mxvm, [90](#)

x86_64_linux
 main.cpp, [785](#)

x86_64_macos
 main.cpp, [785](#)

x86_64_windows
 main.cpp, [785](#)

xmm_offset
 mxvm, [90](#)
 mxvm::Program, [367](#)

XOR
 expr_parser, [76](#)
 instruct.hpp, [528](#)

XParser
 mxx::XParser, [469](#)

YELLOW
 mx::Color, [81](#)

zero_flag
 mxvm::Program, [367](#)